

Jun 2026

Contents

Contents	i
1 Changes	1
1.1 .codespellignore	1
1.2 .github/workflows/PRMessage.yml	1
1.3 .github/workflows/alphaBuild.yml	2
1.4 AGENTS.md	4
1.5 AI-POLICY.md	6
1.6 CITATION.cff	7
1.7 CONTRIBUTING.md	7
1.8 Physlib.lean	7
1.9 Physlib/ClassicalMechanics/DampedHarmonicOscillator/API-map.yml	8
1.10 Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean	11
1.11 Physlib/ClassicalMechanics/DampedHarmonicOscillator/Solution.lean	11
1.12 Physlib/ClassicalMechanics/EulerLagrange.lean	13
1.13 Physlib/ClassicalMechanics/FreeParticle/Basic.lean	13
1.14 Physlib/ClassicalMechanics/HamiltonsEquations.lean	15
1.15 Physlib/ClassicalMechanics/HarmonicOscillator/Basic.lean	15
1.16 Physlib/ClassicalMechanics/HarmonicOscillator/Geometric/Basic.lean	15
1.17 Physlib/ClassicalMechanics/HarmonicOscillator/Geometric/Trajectory.lean	20
1.18 Physlib/ClassicalMechanics/HarmonicOscillator/Solution.lean	24
1.19 Physlib/ClassicalMechanics/Lagrangian/TotalDerivativeEquivalence.lean	41
1.20 Physlib/ClassicalMechanics/Mass/MassUnit.lean	48
1.21 Physlib/ClassicalMechanics/OrbitalMechanics/VisViva.lean	48
1.22 Physlib/ClassicalMechanics/RigidBody/API-map.yml	48
1.23 Physlib/ClassicalMechanics/RigidBody/SolidSphere.lean	50
1.24 Physlib/ClassicalMechanics/WaveEquation/Basic.lean	50
1.25 Physlib/CondensedMatter/TightBindingChain/Basic.lean	51
1.26 Physlib/Cosmology/FLRW/Basic.lean	51
1.27 Physlib/Cosmology/FLRW/ConformalTime.lean	52
1.28 Physlib/Cosmology/FLRW/DensityParameters.lean	53
1.29 Physlib/Cosmology/FLRW/Distances.lean	54
1.30 Physlib/Cosmology/FLRW/Dynamics.lean	55

1.31	Physlib/Cosmology/FLRW/MatterContent.lean	56
1.32	Physlib/Cosmology/FLRW/Solutions.lean	57
1.33	Physlib/Electromagnetism/Charge/ChargeUnit.lean	58
1.34	Physlib/Electromagnetism/Current/InfiniteWire.lean	58
1.35	Physlib/Electromagnetism/Distributional/Basic.lean	59
1.36	Physlib/Electromagnetism/Distributional/Dynamics/IsExtrema.lean	60
1.37	Physlib/Electromagnetism/Distributional/Dynamics/KineticTerm.lean	61
1.38	Physlib/Electromagnetism/Distributional/Dynamics/Lagrangian.lean	62
1.39	Physlib/Electromagnetism/Distributional/ElectricField.lean	62
1.40	Physlib/Electromagnetism/Distributional/FieldStrength.lean	62
1.41	Physlib/Electromagnetism/Distributional/MagneticField.lean	63
1.42	Physlib/Electromagnetism/Distributional/ScalarPotential.lean	63
1.43	Physlib/Electromagnetism/Distributional/VectorPotential.lean	63
1.44	Physlib/Electromagnetism/Dynamics/Basic.lean . .	63
1.45	Physlib/Electromagnetism/Dynamics/Hamiltonian.lean	63
1.46	Physlib/Electromagnetism/Dynamics/IsExtrema.lean	64
1.47	Physlib/Electromagnetism/Dynamics/KineticTerm.lean	65
1.48	Physlib/Electromagnetism/Dynamics/Lagrangian.lean	65
1.49	Physlib/Electromagnetism/Kinematics/EMPotential.lean	66
1.50	Physlib/Electromagnetism/Kinematics/ElectricField.lean	68
1.51	Physlib/Electromagnetism/Kinematics/FieldStrength.lean	68
1.52	Physlib/Electromagnetism/Kinematics/GaugeTransformation.lean	70
1.53	Physlib/Electromagnetism/Kinematics/MagneticField.lean	79
1.54	Physlib/Electromagnetism/Kinematics/VectorPotential.lean	81
1.55	Physlib/Electromagnetism/PointParticle/OneDimension.lean	81
1.56	Physlib/Electromagnetism/PointParticle/ThreeDimension.lean	81
1.57	Physlib/Electromagnetism/Vacuum/Constant.lean .	82
1.58	Physlib/Electromagnetism/Vacuum/HarmonicWave.lean	82
1.59	Physlib/Electromagnetism/Vacuum/IsPlaneWave.lean	83
1.60	Physlib/FluidDynamics/NavierStokes/Momentum.lean	84
1.61	Physlib/Mathematics/Calculus/AdjFDeriv.lean . .	84
1.62	Physlib/Mathematics/Calculus/ParametricIntegration.lean	85
1.63	Physlib/Mathematics/Calculus/Wirtinger/Basic.lean	85
1.64	Physlib/Mathematics/Calculus/Wirtinger/Coordinate.lean	104
1.65	Physlib/Mathematics/ConjModule.lean	120
1.66	Physlib/Mathematics/CrossProductMatrix.lean . .	123
1.67	Physlib/Mathematics/DataStructures/FourTree/Basic.lean	124
1.68	Physlib/Mathematics/DataStructures/FourTree/UniqueMap.lean	125
1.69	Physlib/Mathematics/DataStructures/Matrix/LieTrace.lean	125
1.70	Physlib/Mathematics/Distribution/Basic.lean . .	126
1.71	Physlib/Mathematics/Distribution/PowMul.lean . .	126
1.72	Physlib/Mathematics/FDerivCurry.lean	127
1.73	Physlib/Mathematics/Fin.lean	127
1.74	Physlib/Mathematics/Fin/Involutions.lean	128
1.75	Physlib/Mathematics/Geometry/Metric/PseudoRiemannian/Defs.lean	129
1.76	Physlib/Mathematics/Geometry/Metric/Riemannian/Defs.lean	129

1.77 Physlib/Mathematics/InnerProductSpace/Basic.lean	129
1.78 Physlib/Mathematics/KroneckerDelta.lean	131
1.79 Physlib/Mathematics/LinearMaps.lean	132
1.80 Physlib/Mathematics/LinearPMap.lean	132
1.81 Physlib/Mathematics/List.lean	141
1.82 Physlib/Mathematics/List/InsertionSort.lean	141
1.83 Physlib/Mathematics/SchurTriangulation.lean	142
1.84 Physlib/Mathematics/SpecialFunctions/PhysHermite.lean	142
1.85 Physlib/Mathematics/VariationalCalculus/Basic.lean	143
1.86 Physlib/Mathematics/VariationalCalculus/HasVarAdjDeriv.lean	144
1.87 Physlib/Mathematics/VariationalCalculus/HasVarAdjoint.lean	145
1.88 Physlib/Mathematics/VariationalCalculus/IsLocalizedFunctionTransform.lean	147
1.89 Physlib/Mathematics/VariationalCalculus/IsTestFunction.lean	148
1.90 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Basic.lean	148
1.91 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/FamilyMaps.lean	148
1.92 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/NoGrav/Basic.lean	148
1.93 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Ordinary/Basic.lean	148
1.94 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Ordinary/DimS.lean	148
1.95 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/BMinus.lean	148
1.96 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/Basic.lean	148
1.97 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/HyperC.lean	148
1.98 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/QuadSc.lean	148
1.99 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/QuadSc.lean	148
1.100 Physlib/Particles/BeyondTheStandardModel/TwoHDM/API-map.yaml	152
1.101 Physlib/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix.lean	155
1.102 Physlib/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean	156
1.103 Physlib/Particles/FlavorPhysics/CKMMatrix/PhaseFreedom.lean	157
1.104 Physlib/Particles/FlavorPhysics/CKMMatrix/Relations.lean	158
1.105 Physlib/Particles/FlavorPhysics/CKMMatrix/Rows.lean	158
1.106 Physlib/Particles/FlavorPhysics/CKMMatrix/StandardParameterization/Basic.lean	158
1.107 Physlib/Particles/FlavorPhysics/CKMMatrix/StandardParameterization/StandardPar.lean	158
1.108 Physlib/Particles/StandardModel/AnomalyCancellation/Basic.lean	161
1.109 Physlib/Particles/StandardModel/AnomalyCancellation/FamilyMaps.lean	162
1.110 Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/Basic.lean	162
1.111 Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/Lemmas.lean	162
1.112 Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/LinearParameter.lean	162
1.113 Physlib/Particles/StandardModel/AnomalyCancellation/Permutations.lean	164
1.114 Physlib/Particles/StandardModel/Basic.lean	164
1.115 Physlib/Particles/StandardModel/HiggsBoson/Basic.lean	176
1.116 Physlib/Particles/StandardModel/HiggsBoson/EffectivePotential.lean	178
1.117 Physlib/Particles/StandardModel/HiggsBoson/Potential.lean	183
1.118 Physlib/Particles/StandardModel/Representations.lean	185
1.119 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/B3.lean	185
1.120 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Basic.lean	185
1.121 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/LineY3B3.lean	186

1.122	Physlib/Particles/SuperSymmetry/MSSMnu/AnomalyCancellation/OrthogY3B3	
1.123	Physlib/Particles/SuperSymmetry/MSSMnu/AnomalyCancellation/Y3.lean	187
1.124	Physlib/Particles/SuperSymmetry/N1/Basic.lean	187
1.125	Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/AllowsTerm.lean	198
1.126	Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Basic.lean	198
1.127	Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Completions.lean	200
1.128	Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Map.lean	200
1.129	Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimalSuperSet.lean	
1.130	Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimallyAllowsTerm	
1.131	Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimallyAllowsTerm	
1.132	Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/OfPotentialTerm.lean	
1.133	Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/PhenoClosed.lean	200
1.134	Physlib/QFT/AnomalyCancellation/Basic.lean	202
1.135	Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/Basic.lean	202
1.136	Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/Grading.lean	203
1.137	Physlib/QFT/PerturbationTheory/FieldSpecification/CrAnSection.lean	203
1.138	Physlib/QFT/PerturbationTheory/FieldSpecification/NormalOrder.lean	203
1.139	Physlib/QFT/PerturbationTheory/FieldSpecification/TimeOrder.lean	203
1.140	Physlib/QFT/PerturbationTheory/WickAlgebra/Basic.lean	203
1.141	Physlib/QFT/PerturbationTheory/WickAlgebra/Grading.lean	204
1.142	Physlib/QFT/PerturbationTheory/WickAlgebra/SuperCommute.lean	204
1.143	Physlib/QFT/PerturbationTheory/WickContraction/ExtractEquiv.lean	204
1.144	Physlib/QFT/PerturbationTheory/WickContraction/InsertAndContractNat.lean	
1.145	Physlib/QFT/PerturbationTheory/WickContraction/Join.lean	204
1.146	Physlib/QFT/QED/AnomalyCancellation/Basic.lean	204
1.147	Physlib/QFT/QED/AnomalyCancellation/BasisLinear.lean	205
1.148	Physlib/QFT/QED/AnomalyCancellation/ConstAbs.lean	205
1.149	Physlib/QFT/QED/AnomalyCancellation/Even/BasisLinear.lean	205
1.150	Physlib/QFT/QED/AnomalyCancellation/LineInPlaneCond.lean	206
1.151	Physlib/QFT/QED/AnomalyCancellation/LowDim/One.lean	207
1.152	Physlib/QFT/QED/AnomalyCancellation/Odd/BasisLinear.lean	207
1.153	Physlib/QFT/QED/AnomalyCancellation/Permutations.lean	207
1.154	Physlib/QFT/QED/AnomalyCancellation/Sorts.lean	207
1.155	Physlib/QuantumMechanics/DDimensions/Basic.lean	207
1.156	Physlib/QuantumMechanics/DDimensions/Hydrogen/LaplaceRungeLenzVector.lean	
1.157	Physlib/QuantumMechanics/DDimensions/Operators/AngularMomentum.lean	208
1.158	Physlib/QuantumMechanics/DDimensions/Operators/Commutation.lean	208
1.159	Physlib/QuantumMechanics/DDimensions/Operators/Momentum.lean	209
1.160	Physlib/QuantumMechanics/DDimensions/Operators/Multiplication.lean	209
1.161	Physlib/QuantumMechanics/DDimensions/Operators/Position.lean	210
1.162	Physlib/QuantumMechanics/DDimensions/Operators/SpectralTheory/Basic.lean	
1.163	Physlib/QuantumMechanics/DDimensions/Operators/SpectralTheory/SelfAdj	
1.164	Physlib/QuantumMechanics/DDimensions/Operators/SpectralTheory/Spectra	
1.165	Physlib/QuantumMechanics/DDimensions/Operators/SpectralTheory/Symmetr	
1.166	Physlib/QuantumMechanics/DDimensions/Operators/Unbounded.lean	242
1.167	Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Basic.lean	247

1.168	Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/PolyBddSchwartzSubmodule.lean	
1.169	Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/SchwartzSubmodule.lean	
1.170	Physlib/QuantumMechanics/FiniteTarget/HilbertSpace.lean	251
1.171	Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/Completeness.lean	254
1.172	Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/Eigenfunction.lean	255
1.173	Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/TISE.lean	255
1.174	Physlib/QuantumMechanics/OneDimension/HilbertSpace/Basic.lean	256
1.175	Physlib/QuantumMechanics/OneDimension/Operators/Momentum.lean	256
1.176	Physlib/QuantumMechanics/OneDimension/Operators/Parity.lean	256
1.177	Physlib/Relativity/Bispinors/Basic.lean	257
1.178	Physlib/Relativity/CliffordAlgebra.lean	257
1.179	Physlib/Relativity/LorentzAlgebra/Basic.lean	257
1.180	Physlib/Relativity/LorentzAlgebra/ExponentialMap.lean	257
1.181	Physlib/Relativity/LorentzGroup/API-map.yaml	257
1.182	Physlib/Relativity/LorentzGroup/Basic.lean	259
1.183	Physlib/Relativity/LorentzGroup/Boosts/Apply.lean	261
1.184	Physlib/Relativity/LorentzGroup/Boosts/Basic.lean	261
1.185	Physlib/Relativity/LorentzGroup/Boosts/Generalized.lean	262
1.186	Physlib/Relativity/LorentzGroup/Orthochronous/Basic.lean	263
1.187	Physlib/Relativity/LorentzGroup/Rotations.lean	264
1.188	Physlib/Relativity/MinkowskiMatrix.lean	264
1.189	Physlib/Relativity/PauliMatrices/AsTensor.lean	264
1.190	Physlib/Relativity/PauliMatrices/CliffordAlgebra.lean	265
1.191	Physlib/Relativity/PauliMatrices/Relations.lean	265
1.192	Physlib/Relativity/PauliMatrices/SelfAdjoint.lean	265
1.193	Physlib/Relativity/PauliMatrices/ToTensor.lean	265
1.194	Physlib/Relativity/SL2C/Basic.lean	266
1.195	Physlib/Relativity/SL2C/SelfAdjoint.lean	266
1.196	Physlib/Relativity/Special/TwinParadox/Basic.lean	266
1.197	Physlib/Relativity/Tensors/Basic.lean	267
1.198	Physlib/Relativity/Tensors/ComplexTensor/Basic.lean	271
1.199	Physlib/Relativity/Tensors/ComplexTensor/Lemmas.lean	275
1.200	Physlib/Relativity/Tensors/ComplexTensor/Matrix/Pre.lean	275
1.201	Physlib/Relativity/Tensors/ComplexTensor/Metrics/Basic.lean	276
1.202	Physlib/Relativity/Tensors/ComplexTensor/Metrics/Lemmas.lean	277
1.203	Physlib/Relativity/Tensors/ComplexTensor/Metrics/Pre.lean	278
1.204	Physlib/Relativity/Tensors/ComplexTensor/OfRat.lean	279
1.205	Physlib/Relativity/Tensors/ComplexTensor/Units/Basic.lean	279
1.206	Physlib/Relativity/Tensors/ComplexTensor/Units/Pre.lean	281
1.207	Physlib/Relativity/Tensors/ComplexTensor/Units/Symm.lean	283
1.208	Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Basic.lean	283
1.209	Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Contraction.lean	285
1.210	Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Modules.lean	285
1.211	Physlib/Relativity/Tensors/ComplexTensor/Weyl/Basic.lean	286
1.212	Physlib/Relativity/Tensors/ComplexTensor/Weyl/Contraction.lean	288
1.213	Physlib/Relativity/Tensors/ComplexTensor/Weyl/Metric.lean	290

1.214	Physlib/Relativity/Tensors/ComplexTensor/Weyl/Modules.lean	294
1.215	Physlib/Relativity/Tensors/ComplexTensor/Weyl/Two.lean	296
1.216	Physlib/Relativity/Tensors/ComplexTensor/Weyl/Unit.lean	303
1.217	Physlib/Relativity/Tensors/ComponentIdx/Basic.lean	308
1.218	Physlib/Relativity/Tensors/ComponentIdx/Contraction.lean	309
1.219	Physlib/Relativity/Tensors/ComponentIdx/Product.lean	314
1.220	Physlib/Relativity/Tensors/ComponentIdx/Single.lean	316
1.221	Physlib/Relativity/Tensors/Conjugation/Basic.lean	317
1.222	Physlib/Relativity/Tensors/Constructors.lean	325
1.223	Physlib/Relativity/Tensors/Contraction/Basic.lean	329
1.224	Physlib/Relativity/Tensors/Contraction/Basis.lean	331
1.225	Physlib/Relativity/Tensors/Contraction/Products.lean	332
1.226	Physlib/Relativity/Tensors/Contraction/Pure.lean	333
1.227	Physlib/Relativity/Tensors/Contraction/SuccSuccAbove.lean	335
1.228	Physlib/Relativity/Tensors/Dual.lean	335
1.229	Physlib/Relativity/Tensors/Elab.lean	335
1.230	Physlib/Relativity/Tensors/Evaluation.lean	339
1.231	Physlib/Relativity/Tensors/LeviCivita/Basic.lean	342
1.232	Physlib/Relativity/Tensors/MetricTensor.lean	345
1.233	Physlib/Relativity/Tensors/OfInt.lean	345
1.234	Physlib/Relativity/Tensors/Product.lean	345
1.235	Physlib/Relativity/Tensors/RealTensor/Basic.lean	352
1.236	Physlib/Relativity/Tensors/RealTensor/CoVector/Basic.lean	353
1.237	Physlib/Relativity/Tensors/RealTensor/CoVector/Representation.lean	354
1.238	Physlib/Relativity/Tensors/RealTensor/CoVector/Tensorial.lean	356
1.239	Physlib/Relativity/Tensors/RealTensor/Matrix/Pre.lean	361
1.240	Physlib/Relativity/Tensors/RealTensor/Metrics/Basic.lean	362
1.241	Physlib/Relativity/Tensors/RealTensor/Metrics/Pre.lean	362
1.242	Physlib/Relativity/Tensors/RealTensor/ToComplex.lean	365
1.243	Physlib/Relativity/Tensors/RealTensor/Units/Pre.lean	371
1.244	Physlib/Relativity/Tensors/RealTensor/Vector/Basic.lean	372
1.245	Physlib/Relativity/Tensors/RealTensor/Vector/Causality/LightLike.lean	
1.246	Physlib/Relativity/Tensors/RealTensor/Vector/MinkowskiProduct.lean	373
1.247	Physlib/Relativity/Tensors/RealTensor/Vector/Pre/Basic.lean	374
1.248	Physlib/Relativity/Tensors/RealTensor/Vector/Pre/Contraction.lean	375
1.249	Physlib/Relativity/Tensors/RealTensor/Vector/Representation.lean	377
1.250	Physlib/Relativity/Tensors/RealTensor/Vector/Tensorial.lean	379
1.251	Physlib/Relativity/Tensors/RealTensor/VelocityEngine.lean	384
1.252	Physlib/Relativity/Tensors/Reindexing.lean	384
1.253	Physlib/Relativity/Tensors/TensorSpecies/Basic.lean	395
1.254	Physlib/Relativity/Tensors/Tensorial.lean	396
1.255	Physlib/Relativity/Tensors/UnitTensor.lean	398
1.256	Physlib/SpaceAndTime/GalileanGroup/Basic.lean	399
1.257	Physlib/SpaceAndTime/Space/API-map.yaml	406
1.258	Physlib/SpaceAndTime/Space/Basic.lean	408
1.259	Physlib/SpaceAndTime/Space/ConstantSliceDist.lean	409

1.260	Physlib/SpaceAndTime/Space/CrossProduct.lean	410
1.261	Physlib/SpaceAndTime/Space/Derivatives/Basic.lean	411
1.262	Physlib/SpaceAndTime/Space/Derivatives/Curl.lean	412
1.263	Physlib/SpaceAndTime/Space/Derivatives/Div.lean	414
1.264	Physlib/SpaceAndTime/Space/Derivatives/Grad.lean	414
1.265	Physlib/SpaceAndTime/Space/Derivatives/Iterated.lean	416
1.266	Physlib/SpaceAndTime/Space/Derivatives/Laplacian.lean	416
1.267	Physlib/SpaceAndTime/Space/DistOfFunction.lean	417
1.268	Physlib/SpaceAndTime/Space/EuclideanGroup/Action.lean	418
1.269	Physlib/SpaceAndTime/Space/EuclideanGroup/AffineGroup.lean	421
1.270	Physlib/SpaceAndTime/Space/EuclideanGroup/Basic.lean	426
1.271	Physlib/SpaceAndTime/Space/Integrals/Basic.lean	435
1.272	Physlib/SpaceAndTime/Space/Integrals/NormPow.lean	437
1.273	Physlib/SpaceAndTime/Space/Integrals/RadialAngularMeasure.lean	439
1.274	Physlib/SpaceAndTime/Space/IsDistBounded.lean	440
1.275	Physlib/SpaceAndTime/Space/LengthUnit.lean	441
1.276	Physlib/SpaceAndTime/Space/Module.lean	441
1.277	Physlib/SpaceAndTime/Space/Norm/Basic.lean	443
1.278	Physlib/SpaceAndTime/Space/Norm/IteratedLaplacian.lean	456
1.279	Physlib/SpaceAndTime/Space/Norm/Regularized.lean	459
1.280	Physlib/SpaceAndTime/Space/Origin.lean	461
1.281	Physlib/SpaceAndTime/Space/Slice.lean	463
1.282	Physlib/SpaceAndTime/Space/Translations.lean	463
1.283	Physlib/SpaceAndTime/SpaceTime/Basic.lean	463
1.284	Physlib/SpaceAndTime/SpaceTime/Boosts.lean	463
1.285	Physlib/SpaceAndTime/SpaceTime/Derivatives.lean	464
1.286	Physlib/SpaceAndTime/SpaceTime/LorentzAction.lean	467
1.287	Physlib/SpaceAndTime/SpaceTime/TimeSlice.lean	467
1.288	Physlib/SpaceAndTime/Time/Derivatives.lean	467
1.289	Physlib/SpaceAndTime/Time/TimeMan.lean	469
1.290	Physlib/SpaceAndTime/Time/TimeTransMan.lean	469
1.291	Physlib/SpaceAndTime/Time/TimeUnit.lean	469
1.292	Physlib/SpaceAndTime/TimeAndSpace/Basic.lean	470
1.293	Physlib/SpaceAndTime/TimeAndSpace/ConstantTimeDist.lean	472
1.294	Physlib/SpaceAndTime/TimeAndSpace/EuclideanGroup/Action.lean	474
1.295	Physlib/SpaceAndTime/TimeAndSpace/EuclideanGroup/SchwartzAction.lean	477
1.296	Physlib/StatisticalMechanics/CanonicalEnsemble/Basic.lean	480
1.297	Physlib/StatisticalMechanics/CanonicalEnsemble/Finite.lean	481
1.298	Physlib/StatisticalMechanics/CanonicalEnsemble/Lemmas.lean	482
1.299	Physlib/StatisticalMechanics/MicroCanonicalEnsemble/Basic.lean	482
1.300	Physlib/StatisticalMechanics/MicroCanonicalEnsemble/IdealGas.lean	483
1.301	Physlib/StatisticalMechanics/MicroCanonicalEnsemble/ThermoQuantities.lean	483
1.302	Physlib/StringTheory/FTheory/SU5/Fluxes/Basic.lean	484
1.303	Physlib/StringTheory/FTheory/SU5/Fluxes/NoExotics/ChiralIndices.lean	484
1.304	Physlib/StringTheory/FTheory/SU5/Quanta/FiveQuanta.lean	487
1.305	Physlib/StringTheory/FTheory/SU5/Quanta/TenQuanta.lean	489

1.306	Physlib/Thermodynamics/IdealGas/Basic.lean	491
1.307	Physlib/Thermodynamics/Temperature/Basic.lean	491
1.308	Physlib/Thermodynamics/Temperature/TemperatureUnits.lean	493
1.309	Physlib/Units/Basic.lean	493
1.310	Physlib/Units/Dimension.lean	493
1.311	Physlib/Units/Examples.lean	493
1.312	Physlib/Units/UnitDependent.lean	494
1.313	Physlib/Units/WithDim/Area.lean	494
1.314	Physlib/Units/WithDim/Basic.lean	495
1.315	Physlib/Units/WithDim/Speed.lean	497
1.316	PhyslibAlpha.lean	497
1.317	PhyslibAlpha/Basic.lean	498
1.318	PhyslibAlpha/ClassicalFieldTheory/Local/Action.lean	499
1.319	PhyslibAlpha/ClassicalFieldTheory/Local/EulerLagrange.lean	504
1.320	PhyslibAlpha/ClassicalFieldTheory/Local/FirstOrder.lean	506
1.321	PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation.lean	510
1.322	PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Basic.lean	512
1.323	PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Criterion.lean	
1.324	PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Density.lean	52
1.325	PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/IntegrationByP	
1.326	PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Regularity.lean	
1.327	PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Support.lean	53
1.328	PhyslibAlpha/ClassicalFieldTheory/Local/JetPoint.lean	542
1.329	PhyslibAlpha/ClassicalFieldTheory/Local/JetPointFiber.lean	546
1.330	PhyslibAlpha/ClassicalFieldTheory/Local/JetPointRegularity.lean	552
1.331	PhyslibAlpha/ClassicalFieldTheory/Local/Lagrangian.lean	554
1.332	PhyslibAlpha/ClassicalFieldTheory/Local/TotalDerivative.lean	558
1.333	PhyslibAlpha/ClassicalFieldTheory/Local/TotalDivergence.lean	561
1.334	PhyslibAlpha/ClassicalFieldTheory/Local/TotalDivergenceEquivalence.lean	
1.335	PhyslibAlpha/ClassicalFieldTheory/Local/Variation.lean	569
1.336	PhyslibAlpha/ClassicalMechanics/CoupledSpringPotential.lean	570
1.337	PhyslibAlpha/Mathematics/PartialDerivativeTest.lean	573
1.338	PhyslibAlpha/QuantumMechanics/QuantumHarmonicOscillator.lean	585
1.339	PhyslibAlpha/QuantumMechanics/StinespringDilation.lean	594
1.340	PhyslibAlpha/SpaceAndTime/Space/Surfaces/HalfPlane.lean	614
1.341	PhyslibAlpha/SpaceAndTime/Space/Surfaces/Line.lean	618
1.342	PhyslibAlpha/SpaceAndTime/Space/Surfaces/Ring.lean	621
1.343	PhyslibAlpha/SpaceAndTime/Space/Surfaces/SolidCylinder.lean	624
1.344	PhyslibAlpha/SpaceAndTime/Space/Surfaces/SolidSphere.lean	627
1.345	PhyslibAlpha/SpaceAndTime/Space/Surfaces/SphericalCylinder.lean	630
1.346	PhyslibAlpha/SpaceAndTime/Space/Surfaces/SphericalShell.lean	633
1.347	QuantumInfo/Channels/Bundled.lean	635
1.348	QuantumInfo/Channels/CPTP.lean	636
1.349	QuantumInfo/Channels/Dual.lean	636
1.350	QuantumInfo/Channels/Pinching.lean	636
1.351	QuantumInfo/Channels/Unbundled.lean	637

1.352QuantumInfo/ClassicalInfo/Distribution.lean	637
1.353QuantumInfo/ClassicalInfo/Prob.lean	637
1.354QuantumInfo/Entropy/DPI.lean	637
1.355QuantumInfo/Entropy/Relative.lean	638
1.356QuantumInfo/Entropy/SSA.lean	640
1.357QuantumInfo/Entropy/VonNeumann.lean	641
1.358QuantumInfo/ForMathlib/ComplexLaplaceTransform.lean	641
1.359QuantumInfo/ForMathlib/ContinuousLinearMap.lean	641
1.360QuantumInfo/ForMathlib/ContinuousSup.lean	642
1.361QuantumInfo/ForMathlib/HayataGroup/TraceInequality/BlockDiagonal.lean	642
1.362QuantumInfo/ForMathlib/HayataGroup/TraceInequality/GeneralizedPerspectiveFunc	
1.363QuantumInfo/ForMathlib/HayataGroup/TraceInequality/HilbertSchmidtOperatorSpace	
1.364QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperatorInequality.lean	
1.365QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperatorInequalityIIm	
1.366QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperatorInequalityIVt	
1.367QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LiebAndoTrace.lean	644
1.368QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LownerHeinzCore.lean	644
1.369QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LownerHeinzTheorem.lean	644
1.370QuantumInfo/ForMathlib/HermitianMat/Basic.lean	645
1.371QuantumInfo/ForMathlib/HermitianMat/CFC.lean	645
1.372QuantumInfo/ForMathlib/HermitianMat/Inner.lean	646
1.373QuantumInfo/ForMathlib/HermitianMat/LiebConcavity.lean	648
1.374QuantumInfo/ForMathlib/HermitianMat/LogExp.lean	648
1.375QuantumInfo/ForMathlib/HermitianMat/Order.lean	649
1.376QuantumInfo/ForMathlib/HermitianMat/Peierls.lean	649
1.377QuantumInfo/ForMathlib/HermitianMat/Proj.lean	649
1.378QuantumInfo/ForMathlib/HermitianMat/Reindex.lean	650
1.379QuantumInfo/ForMathlib/HermitianMat/Rpow.lean	650
1.380QuantumInfo/ForMathlib/HermitianMat/Schatten.lean	650
1.381QuantumInfo/ForMathlib/HermitianMat/Trace.lean	650
1.382QuantumInfo/ForMathlib/IsMaximalSelfAdjoint.lean	651
1.383QuantumInfo/ForMathlib/Isometry.lean	651
1.384QuantumInfo/ForMathlib/LimSupInf.lean	652
1.385QuantumInfo/ForMathlib/LinearEquiv.lean	652
1.386QuantumInfo/ForMathlib/Majorization.lean	653
1.387QuantumInfo/ForMathlib/Matrix.lean	653
1.388QuantumInfo/ForMathlib/MatrixNorm/TraceNorm.lean	654
1.389QuantumInfo/ForMathlib/Minimax.lean	654
1.390QuantumInfo/ForMathlib/Misc.lean	654
1.391QuantumInfo/ForMathlib/SionMinimax.lean	654
1.392QuantumInfo/ForMathlib/ULift.lean	655
1.393QuantumInfo/Measurements/POVM.lean	655
1.394QuantumInfo/Operators/Unitary.lean	655
1.395QuantumInfo/ResourceTheory/FreeState.lean	655
1.396QuantumInfo/ResourceTheory/HypothesisTesting.lean	656
1.397QuantumInfo/ResourceTheory/SteinsLemma.lean	656

1.398	QuantumInfo/States/Ensemble.lean	657
1.399	QuantumInfo/States/Entanglement.lean	658
1.400	QuantumInfo/States/Mixed/MState.lean	658
1.401	QuantumInfo/States/Mixed/TraceDistance.lean	659
1.402	QuantumInfo/States/Pure/Braket.lean	659
1.403	QuantumInfo/States/Pure/Qubit.lean	659
1.404	README.md	660
1.405	docs/references.bib	660
1.406	lake-manifest.json	661
1.407	lakefile.toml	661
1.408	lean-toolchain	664
1.409	scripts/LinterExemption.txt	664
1.410	scripts/MetaPrograms/TODO_to_yaml.lean	666
1.411	scripts/MetaPrograms/check_dup_tags.lean	667
1.412	scripts/MetaPrograms/informal.lean	667
1.413	scripts/MetaPrograms/local_stats.lean	667
1.414	scripts/MetaPrograms/module_doc_lint.lean	667
1.415	scripts/MetaPrograms/notes.lean	667
1.416	scripts/MetaPrograms/redundant_imports.lean	668
1.417	scripts/MetaPrograms/runPhyslibLinters.lean	668
1.418	scripts/MetaPrograms/sorry_lint.lean	669
1.419	scripts/MetaPrograms/spelling.lean	669
1.420	scripts/MetaPrograms/spellingWords.txt	669
1.421	scripts/MetaPrograms/style_lint.lean	669
1.422	scripts/PhyslibAlpha/README.md	670
1.423	scripts/PhyslibAlpha/alphaFileImports.py	671
1.424	scripts/PhyslibAlpha/alphaPythonLinters.sh	672
1.425	scripts/PhyslibAlpha/noAlphaImports.py	673
1.426	scripts/PhyslibAlpha/runPhyslibAlphaLinters.lean	674
1.427	scripts/README.md	676
1.428	scripts/check_file_imports.lean	676
1.429	scripts/dag_traversal.py	676
1.430	scripts/lint-style.sh	697
1.431	scripts/openAI_doc_check.lean	698
1.432	scripts/rm_set_option.py	698
1.433	scripts/set_option_utils.py	710
1.434	scripts/stats.lean	712

1 Changes

1.1 .codespellignore

ofSet

1.2 .github/workflows/PRMessage.yml

name: PR Comment

on:

 workflow_dispatch:
 pull_request_target:
 types: [opened]

jobs:

 build:

 name: Comment a pull_request
 runs-on: ubuntu-latest
 permissions:
 pull-requests: write

 steps:

- name: Checkout
 uses: actions/checkout@v2
- name: Comment a pull_request
 uses: mb2dev/github-action-comment-pull-request@1.0.0
 with:

 message: |

 Thank you for this PR, which will now be reviewed. If submitting to
community/physlib/blob/master/docs/ReviewGuidelines.md) if you are not familiar with
community/physlib/blob/master/scripts/README.md), including how to run them locally.

 If you are submitting to ./PhyslibAlpha there will be a lighter re

If you want to bring attention to this PR, please write a comment on the PR (e.g. `Physlib/topic/PR.20reviews/with/596447366`) of the Lean Zulip.

****Important:**** If a reviewer adds an ``awaiting-author`` label to your PR, once you have addressed the review comments, please add the `awaiting-author`` label. This helps us keep track of reviews.

```
GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

1.3 .github/workflows/alphaBuild.yml

```
on:
  push:
    branches:
      - master
  pull_request:
    branches:
      - master

name: Style linters

jobs:
  alpha_build:
    name: Lean based style linters
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install elan
        run: |
          set -o pipefail
          curl https://raw.githubusercontent.com/leanprover/elan/master/elan
init.sh -sSf | sh -s -- --default-toolchain none -y
          ~/.elan/bin/lean --version
          echo "$HOME/.elan/bin" >> $GITHUB_PATH

      - name: check versions
        run: |
          lean --version
          lake --version

      - name: build cache
        run: |
```

```

    lake exe cache get

  - name: build PhyslibAlpha
    id: build
    uses: liskin/gh-problem-matcher-wrap@v3
    with:
      linters: gcc
      run: |
        bash -o pipefail -c "env LEAN_ABORT_ON_PANIC=1 lake build -
KCI PhyslibAlpha | tee stdout.log"

  - name: runLinter on PhyslibAlpha
    if: ${{ always() && steps.build.outcome == 'success' || steps.build.outcome
id: lint
    uses: liskin/gh-problem-matcher-wrap@v3
    timeout-minutes: 4
    with:
      linters: gcc
      run: env LEAN_ABORT_ON_PANIC=1 lake exe runPhyslibAlphaLinters

style_lint:
  name: Python based linters
  runs-on: ubuntu-latest
  steps:
    - name: cleanup
      run: |
        find . -name . -o -prune -exec rm -rf -- {} +

    - uses: actions/checkout@v4

    # Run the case checker action
    - name: Check Case Sensitivity
      uses: credfeto/action-case-checker@v1.3.0

    - name: Look for ignored files
      uses: credfeto/action-no-ignored-files@v1.1.0

    - name: install Python
      if: ${{ 'ubuntu-latest' == 'ubuntu-latest' }}
      uses: actions/setup-python@v5
      with:
        python-version: 3.8

    - name: Check PhyslibAlpha imports
      run: |
        chmod u+x scripts/PhyslibAlpha/alphaFileImports.py

```

```
./scripts/PhyslibAlpha/alphaFileImports.py

- name: Check no PhyslibAlpha in Physlib and QuantumInfo
  run: |
    chmod u+x scripts/PhyslibAlpha/noAlphaImports.py
    ./scripts/PhyslibAlpha/noAlphaImports.py

- name: Python linters for PhyslibAlpha
  run: |
    chmod u+x scripts/PhyslibAlpha/alphaPythonLinters.sh
    ./scripts/PhyslibAlpha/alphaPythonLinters.sh
```

1.4 AGENTS.md

AGENTS file

Introduction

- The human-facing policy and disclosure obligations are in AI-POLICY.md; read it too.
- The human author must certify the code you write; the fact that it builds
- Obey also docs/ReviewGuidelines.md.

Content

- Use ``lemma``, not ``theorem``, unless the result is well known in the physics
- Never use the ``axiom`` declaration.
- Never use ``sorry``.
- Never use structure fields of type ``True``, or theorems returning ``True``.
- Never use an existential statement of the form ``∃ x, ..., True`` (any number of `x`'s).
- Never use a scope-level variable hypothesis that assumes the conclusion,
- Never include trailing white space.
- Make sure that hypotheses are distributed compactly and neatly over new lines.
- Do not add lemmas that are trivial rewrites of existing Mathlib or Physlib
- Place results in the appropriate existing file; do not create new files w
- Include sections which are numbered by ``# A. ...``, ``## A.1. ...``. See [Ph
- Every definition must have a docstring.
- Important lemmas should have a docstring.

Proof structure

In general proofs of lemmas and theorems should be short (under 50 LOC).

Long proofs should be split into smaller lemmas. Where possible this should

****Extract by meaning**** – the fragment says something independently true:

- The proof contains further properties about existing definitions. These properties
- The proof contains ``calc`` or ``have`` statements which contain physics context, and
- The proof contains ``have`` statements with no physics content but general mathematical goal). These belong as general lemmas.
- The proof first establishes a general statement and then instantiates it. Extract
- Part of the proof uses only the weaker form of a hypothesis (e.g. only continuity,
- A ``let`/`set`` constructs an object and proves properties about it inline. Promote

****Extract by structure**** – the proof breaks into independent pieces:

- Substantial ``rcases`/`match`` branches. Each branch becomes its own lemma, and the
- The base case and inductive step of an induction, when long.
- A long ``calc`` block. A contiguous run of steps establishing a named equality can b
- Symmetry or duality, where half the proof mirrors the other (``≤`` then ``≥``, swappin

Surface a recommendation when a condition above is met.

When a long proof cannot be split, make sure it contains comments.

Before you finish

- Make sure all new files are imported into ``Physlib.lean`` or ``PhyslibAlpha.lean`` (k
- Anything depending on ``sorry`` or ``Lean.ofReduceBool`` must be tagged ``@[sorryful]``
- New physics terms that trip the spell-checker go in ``scripts/MetaPrograms/spelling``
- Check that ``lake build`` works (run ``lake exe cache get`` first).
- Check that ``lake exe lint_all`` passes.
- Check ``./scripts/lint-style.sh``, but ****commit your changes first****; this linter re
- If edited a ``PhyslibAlpha`` file, check the following:
 - ``lake exe runPhyslibAlphaLinters``
 - ``./scripts/PhyslibAlpha/alphaFileImports.py``
 - ``./scripts/PhyslibAlpha/noAlphaImports.py``
 - ``./scripts/PhyslibAlpha/alphaPythonLinters.sh``

PR scope

A PR should add a ****single coherent concept****.

- Every definition and lemma should either ***be*** that concept or supply the minimal A

Commits

- Split a PR into atomic commits where it makes sense; keep commits focused.
- Commit titles must describe the lemmas or definitions added or changed.

```
- Include a [sign-off](https://git-scm.com/docs/SubmittingPatches.html#sign-off)-style
`Co-authored-by: Claude Opus 4.8 <no-reply+claude-opus-4-8@anthropic.com>` trailer on commits you
produced.
```

PR descriptions

If you write the PR description:

- The PR description must list all lemmas and definitions added or removed,
- The PR description should include a reviewer map: a brief guide indicating
- The PR description must be concise.

1.5 AI-POLICY.md

AI policy

This file is the human-facing contract for AI-assisted contributions to Physlib

1. Introduction

1.1. Physlib generally welcomes AI-assisted contributions.

1.2. If you use an AI tool to help author a pull request to Physlib, you must

1.3. Failure to follow these guidelines may result in your PR being closed

1.4. Throughout this document, "must" denotes a hard requirement and "should"

1.5. You must also follow any explicit instructions in [docs/ReviewGuidelines]

1.6. These guidelines apply to any contribution where an AI tool produced m

1.7. The human author(s) are fully responsible for every line of the PR, ho

2. Author obligations

2.1. You must verify any bibliographic references for correctness yourself:

2.2. You must confirm your contribution satisfies the content and structure

3. Review process

3.1. All communication with human reviewers must be conducted by humans, not by an AI agent.

3.2. If an AI agent is used to implement reviewer feedback, the author must independently review. This must not be left to the reviewer to check.

1.6 CITATION.cff

```
cff-version: 1.2.0
message: "If you use this library, please cite it as below."
authors:
- name: "The Physlib Community"
title: "Physlib: The Lean Physics Library"
date-released: 2024-04-16
url: "https://github.com/leanprover-community/physlib"
```

1.7 CONTRIBUTING.md

- <https://physlib.io/getting-started> to learn how to get started
- <https://physlib.io/get-involved> to see how to get involved.

1.8 Physlib.lean

```
public import Physlib.ClassicalMechanics.HarmonicOscillator.Geometric.Basic
public import Physlib.ClassicalMechanics.HarmonicOscillator.Geometric.Trajectory
public import Physlib.Cosmology.FLRW.ConformalTime
public import Physlib.Cosmology.FLRW.DensityParameters
public import Physlib.Cosmology.FLRW.Distances
public import Physlib.Cosmology.FLRW.Dynamics
public import Physlib.Cosmology.FLRW.MatterContent
public import Physlib.Cosmology.FLRW.Solutions
public import Physlib.Electromagnetism.Kinematics.GaugeTransformation
public import Physlib.Mathematics.Calculus.Wirtinger.Basic
public import Physlib.Mathematics.Calculus.Wirtinger.Coordinate
public import Physlib.Mathematics.ConjModule
public import Physlib.Mathematics.CrossProductMatrix
public import Physlib.Mathematics.LinearPMap
public import Physlib.Particles.StandardModel.HiggsBoson.EffectivePotential
public import Physlib.Particles.SuperSymmetry.N1.Basic
public import Physlib.QuantumMechanics.DDimensions.Operators.SpectralTheory.Basic
public import Physlib.QuantumMechanics.DDimensions.Operators.SpectralTheory.SelfAdjoint
public import Physlib.QuantumMechanics.DDimensions.Operators.SpectralTheory.Spectral
public import Physlib.QuantumMechanics.DDimensions.Operators.SpectralTheory.Symmetries
```

```

public import Physlib.Relativity.Tensors.ComponentIdx.Basic
public import Physlib.Relativity.Tensors.ComponentIdx.Contraction
public import Physlib.Relativity.Tensors.ComponentIdx.Product
public import Physlib.Relativity.Tensors.ComponentIdx.Single
public import Physlib.Relativity.Tensors.Conjugation.Basic
public import Physlib.Relativity.Tensors.LeviCivita.Basic
public import Physlib.Relativity.Tensors.RealTensor.CoVector.Representation
public import Physlib.Relativity.Tensors.RealTensor.CoVector.Tensorial
public import Physlib.Relativity.Tensors.RealTensor.Vector.Representation
public import Physlib.Relativity.Tensors.RealTensor.Vector.Tensorial
public import Physlib.Relativity.Tensors.Reindexing
public import Physlib.SpaceAndTime.GalileanGroup.Basic
public import Physlib.SpaceAndTime.Space.EuclideanGroup.Action
public import Physlib.SpaceAndTime.Space.EuclideanGroup.AffineGroup
public import Physlib.SpaceAndTime.Space.EuclideanGroup.Basic
public import Physlib.SpaceAndTime.Space.Norm.Basic
public import Physlib.SpaceAndTime.Space.Norm.IteratedLaplacian
public import Physlib.SpaceAndTime.Space.Norm.Regularized
public import Physlib.SpaceAndTime.Space.Origin
public import Physlib.SpaceAndTime.TimeAndSpace.EuclideanGroup.Action
public import Physlib.SpaceAndTime.TimeAndSpace.EuclideanGroup.SchwartzActi

```

1.9 Physlib/ClassicalMechanics/DampedHarmonicOscillator/API-map.yaml

version: v0.1

Title: Damped harmonic oscillator

Overview: |

The damped harmonic oscillator is a classical mechanical system consisting of a mass m under a restoring force $-kx$ and a damping force $-\gamma \dot{x}$, with equation of motion $m\ddot{x} + \gamma\dot{x} + kx = 0$. The key data structure `DampedHarmonicOscillator` extends `HarmonicOscillator` (inheriting the mass m , spring constant k , and the undamped natural frequency ω_0) and adds a nonnegative damping coefficient γ . Position and velocity are represented as `EuclideanSpace ℝ (Fin 1)` and trajectories as maps `Time → EuclideanSpace`.

The `Basic` module defines the equation of motion, the force $-kx - \gamma\dot{x}$, its equivalence with Newton's second law, the rate of mechanical energy dissipation along solutions, the three damping regimes (underdamped, critically damped, overdamped) selected by the sign of the discriminant $\gamma^2 - 4mk$ together with the decay rate and regime-selected angular frequency, and the reduction of the system to a single degree of freedom.

1.9

PHYSLIB/CLASSICALMECHANICS/DAMPEDHARMONICOSCILLATOR/API-

MAP.YAML

undamped harmonic oscillator when $\gamma = 0$. The `Solution` module defines the initial conditions, the explicit closed-form `trajectory` selected from the damping regime (trigonometric, polynomial, or hyperbolic, each times an exponential decay factor), and proves that the selected trajectory satisfies the equation of motion.

ParentAPIs:

- Harmonic oscillator (Physlib/ClassicalMechanics/HarmonicOscillator)
- Time (Physlib/SpaceAndTime/Time)

References:

- Landau & Lifshitz, Mechanics, page 76, section 25.
- Goldstein, Classical Mechanics, Chapter 2.

Requirements:

- description: >
The key input data structure of the damped harmonic oscillator (mass, spring constant, and damping coefficient) is defined.
done: true
location: Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean (Damped)
- description: The API contains a definition of the equation of motion $m \ddot{x} + \gamma \dot{x}$
done: true
location: Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean (Equati
- description: >
The API defines the force $-kx - \gamma \dot{x}$ and proves the equation of motion is equivalent to Newton's second law.
done: true
location: Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean (force,
- description: >
The API proves that along a solution the mechanical energy dissipates at rate $-\gamma \|\dot{x}\|^2$ and is strictly decreasing when the velocity is nonzero and $\gamma > 0$.
done: true
location: Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean (energy
- description: >
The API classifies the underdamped, critically damped, and overdamped regimes via the discriminant $\gamma^2 - 4mk$, and defines the decay rate and the regime-selected angular frequency.
done: true
location: Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean (discri
- description: >

- The API reduces the damped oscillator to the undamped harmonic oscillator $\gamma = 0$, showing the equations of motion agree.
done: true
location: Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean
- description: The API contains a definition of the initial conditions of
done: true
location: Physlib/ClassicalMechanics/DampedHarmonicOscillator/Solution.
 - description: >
The API contains a definition of the trajectory of the damped harmonic oscillator, selecting the closed form appropriate to the damping regime.
done: true
location: Physlib/ClassicalMechanics/DampedHarmonicOscillator/Solution.
 - description: The API proves that the selected trajectory satisfies the
done: true
location: Physlib/ClassicalMechanics/DampedHarmonicOscillator/Solution.
 - description: The API shall contain the definition of the kinetic energy
done: false
location: N/A
 - description: The API shall contain the definition of the potential energy
done: false
location: N/A
 - description: The API shall contain the definition of the Lagrangian.
done: false
location: N/A
 - description: >
The API shall contain a simplification of the variational gradient of the Lagrangian to the Euler-Lagrange equations.
done: false
location: N/A
 - description: >
The API shall prove that the selected trajectory is the unique solution to the equation of motion with the given initial conditions.
done: false
location: N/A

1.10.

PHYSLIB/CLASSICALMECHANICS/DAMPEDHARMONICOSCILLATOR/BASIC.LEAN

1.10 Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean

```
- **Underdamped** ( $\gamma^2 < 4 * m * k$ ) : oscillatory motion with exponentially decayi
- **Critically damped** ( $\gamma^2 = 4 * m * k$ ) : fastest return to equilibrium without
- **Overdamped** ( $4 * m * k < \gamma^2$ ) : slow return to equilibrium without oscillatio
  position as a configuration space and velocity as its tangent space, see the
  HarmonicOscillator file."
lemma equationOfMotion_iff_newtons_2nd_law (xt : Time → EuclideanSpace ℝ (Fin 1)) :
```

1.11 Physlib/ClassicalMechanics/DampedHarmonicOscillator/Solution.lean

```
rw [IsUnderdamped, IsCriticallyDamped] at *
rw [IsOverdamped] at hS
rw [IsUnderdamped]
rw [IsCriticallyDamped]
simp only [add_apply, ContinuousLinearMap.smulRight_apply,
  _root_.neg_apply, FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
simp only [add_apply, _root_.sub_apply,
  Pi.zero_apply, Time.fderiv_val, _root_.neg_apply,
  FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
rw [Time.deriv_eq, fderiv_fun_add (by fun_prop) (by fun_prop),
  fderiv_fun_const, fderiv_smul_const (by fun_prop)]
funext t
have fderiv_comp_val_eq_deriv : ∀ g : ℝ → ℝ, DifferentiableAt ℝ g t.val →
  (fderiv ℝ (fun s : Time => g s.val) t) 1 = _root_.deriv g t.val := by
  intro g hg
  rw [fderiv_fun_comp t hg (by fun_prop), ContinuousLinearMap.comp_apply, Time.fde
  simp
rw [Time.deriv_eq, fderiv_fun_add (by fun_prop) (by fun_prop),
  fderiv_smul_const (by fun_prop), fderiv_smul_const (by fun_prop)]
simp only [add_apply, ContinuousLinearMap.smulRight_apply]
rw [fderiv_comp_val_eq_deriv (fun s => cos (S.angularFrequency * s)) (by fun_prop)
  fderiv_comp_val_eq_deriv (fun s => sin (S.angularFrequency * s) / S.angularFreque
  (by fun_prop)]
simp [mul_comm, hΩ]
rw [S.underdampedBase_velocity IC hS]
funext t
have fderiv_comp_val_eq_deriv : ∀ g : ℝ → ℝ, DifferentiableAt ℝ g t.val →
  (fderiv ℝ (fun s : Time => g s.val) t) 1 = _root_.deriv g t.val := by
  intro g hg
  rw [fderiv_fun_comp t hg (by fun_prop), ContinuousLinearMap.comp_apply, Time.fde
  simp
rw [Time.deriv_eq, fderiv_fun_add (by fun_prop) (by fun_prop),
  fderiv_smul_const (by fun_prop), fderiv_smul_const (by fun_prop)]
```

```

simp only [add_apply, ContinuousLinearMap.smulRight_apply]
rw [fderiv_comp_val_eq_deriv (fun s => -S.angularFrequency * sin (S.angular
  (by fun_prop),
  fderiv_comp_val_eq_deriv (fun s => cos (S.angularFrequency * s)) (by fun
simp [mul_comm, smul_smul, smul_add]
have hΩ : S.angularFrequency ≠ 0 := S.angularFrequency_ne_zero_of_overdamp
have fderiv_comp_val_eq_deriv : ∀ g : ℝ → ℝ, DifferentiableAt ℝ g t.val →
  (fderiv ℝ (fun s : Time => g s.val) t) 1 = _root_.deriv g t.val := by
  intro g hg
  rw [fderiv_fun_comp t hg (by fun_prop), ContinuousLinearMap.comp_apply,
  simp
  simp
  simp [Time.deriv_eq, fderiv_fun_add (by fun_prop) (by fun_prop),
  fderiv_smul_const (by fun_prop), fderiv_smul_const (by fun_prop)]
simp only [add_apply, ContinuousLinearMap.smulRight_apply]
rw [fderiv_comp_val_eq_deriv (fun s => cosh (S.angularFrequency * s)) (by
  fderiv_comp_val_eq_deriv (fun s => sinh (S.angularFrequency * s) / S.an
  (by fun_prop)]
simp [mul_comm, hΩ]
have hΩ : S.angularFrequency ≠ 0 := S.angularFrequency_ne_zero_of_overdamp
funext t
have fderiv_comp_val_eq_deriv : ∀ g : ℝ → ℝ, DifferentiableAt ℝ g t.val →
  (fderiv ℝ (fun s : Time => g s.val) t) 1 = _root_.deriv g t.val := by
  intro g hg
  rw [fderiv_fun_comp t hg (by fun_prop), ContinuousLinearMap.comp_apply,
  simp
  simp
  simp [Time.deriv_eq, fderiv_fun_add (by fun_prop) (by fun_prop),
  fderiv_smul_const (by fun_prop), fderiv_smul_const (by fun_prop)]
simp only [add_apply, ContinuousLinearMap.smulRight_apply]
rw [fderiv_comp_val_eq_deriv (fun s => S.angularFrequency * sinh (S.angular
  (by fun_prop),
  fderiv_comp_val_eq_deriv (fun s => cosh (S.angularFrequency * s)) (by f
simp [mul_comm, smul_smul, smul_add]
field_simp [hΩ]
refine S.exp_decay_smul_equationOfMotion S.decayRate 0 (S.criticallyDamped
  unfold criticallyDampedBase
  fun_prop) ?_ hy hk
simp using S.criticallyDampedBase_acceleration IC
refine S.exp_decay_smul_equationOfMotion S.decayRate
  unfold underdampedBase
  fun_prop) ?_
  (S.underdampedBase_acceleration IC hS) hy hk
rw [show ∂t (S.underdampedBase IC) = _ from S.underdampedBase_velocity IC
fun_prop]
refine S.exp_decay_smul_equationOfMotion S.decayRate (S.angularFrequency^
  unfold overdampedBase
  fun_prop) ?_

```



```

(S.overdampedBase_acceleration IC hS) hy hk
rw [show ∂t (S.overdampedBase IC) = _ from S.overdampedBase_velocity IC hS]
fun_prop
  rw [IsOverdamped, IsUnderdamped] at *
  exact lt_of_le_of_ne (not_lt.mp hUnder) (Ne.symm hCritical)

```

1.12 Physlib/ClassicalMechanics/EulerLagrange.lean

```

simp only [eulerLagrangeOp_eq, Time.deriv_eq]
  apply ContDiff.differentiable (n := ∞) (by fun_prop) (by simp)
  · apply HasVarAdjDerivAt.fderiv (hu := hq)
simp (disch := fun_prop) [sub_eq_add_neg]
all_goals
  try funext t
  rw [gradient_eq_adjFderiv, adjFderiv_uncurry] <=>
    apply ContDiff.differentiable (n := ∞) (by fun_prop) (by simp)

```

1.13 Physlib/ClassicalMechanics/FreeParticle/Basic.lean

```

public import Mathlib.Analysis.Calculus.MeanValue
- `linearMomentum_conserved` shows that linear momentum stays constant over time.
- `kineticEnergy_conserved` shows that kinetic energy stays constant over time.
Newton's law → zero acceleration → constant velocity → constant momentum and energy.
- D. Momentum
  - D.1. Linear momentum
  - D.2. Momentum conservation
- E. Energy
  - E.1. Kinetic energy
  - E.2. Energy conservation
/--
The linear momentum of the free particle along a trajectory.

This is given by the classical one-dimensional expression  $p = m v$ ,
where  $m$  is the particle mass and  $v$  is the velocity.
-/
noncomputable
def linearMomentum (s : FreeParticle) (q : Trajectory) (t : Time) : ℝ :=
  s.mass * s.velocity q t

```

The second-order differentiability assumption is included to apply standard results from real analysis relating vanishing derivatives to constant functions.

```

(hcont : ContDiff ℝ 2 q) : ∃ v₀, ∀ t, deriv q t = v₀ := by

```

```

refine {deriv q 0, fun t => ?_}
refine is_const_of_fderiv_eq_zero (□ := ℝ) (f := deriv q) ?_ ?_ t 0
· change Differentiable ℝ ((fun L : Time → L[ℝ] ℝ => L 1) ∘ fun t => fderiv
  fun_prop
· intro t
  ext p
  simp only [zero_apply]
  have hp : p = p.val • (1 : Time) := by
    ext
    simp
  rw [hp]
  simp only [map_smul, smul_eq_mul, mul_eq_zero]
  right
  simp [Time.deriv_eq] using h t

/--
If a free-particle trajectory has constant velocity, then its linear momentum is constant.
-/
lemma linearMomentum_conserved_of_velocity_const (s : FreeParticle) (q : Trajectory)
  (h : ∃ v₀, ∀ t, s.velocity q t = v₀) :
  ∃ p, ∀ t, s.linearMomentum q t = p := by
  rcases h with ⟨v₀, hv⟩
  refine {s.mass * v₀, fun t => ?_}
  unfold linearMomentum
  rw [hv t]

/--
A free particle satisfying the equation of motion conserves linear momentum.

Newton's second law implies that the acceleration vanishes, so the velocity is constant.
Since the particle mass is fixed, the linear momentum is constant in time.
-/
theorem linearMomentum_conserved (s : FreeParticle) (q : Trajectory)
  (h : ∀ t, s.NewtonsSecondLaw q t) (hcont : ContDiff ℝ 2 q) :
  ∃ p, ∀ t, s.linearMomentum q t = p := by
  have h_acc : ∀ t, deriv (deriv q) t = 0 :=
    accel_zero s q h
  rcases velocity_const_of_zero_acc q h_acc hcont with ⟨v₀, hv⟩
  exact linearMomentum_conserved_of_velocity_const s q ⟨v₀, hv⟩

  (h : ∀ t, s.NewtonsSecondLaw q t) (hcont : ContDiff ℝ 2 q) :
  accel_zero s q h

```

1.14 Physlib/ClassicalMechanics/HamiltonsEquations.lean

```

simp [hamiltonEqOp_eq, funext_iff, Prod.mk_eq_zero, forall_and, add_eq_zero_iff_ne
  · exact HasVarAdjDerivAt.fmap _ _ (by fun_prop) (by fun_prop)
    fun x _ => (by fun_prop : DifferentiableAt ℝ _).hasAdjFDerivAt
  · exact HasVarAdjDerivAt.fst (HasVarAdjDerivAt.id _ (by fun_prop))
    exact HasVarAdjDerivAt.fmap _ _ (by fun_prop) (by fun_prop)
    fun x _ => (by fun_prop : DifferentiableAt ℝ _).hasAdjFDerivAt
  exact HasVarAdjDerivAt.fmap (fun t => 1(H t)) _ (by fun_prop) (by fun_prop)
    fun x _ => (((by fun_prop : ContDiff ℝ ∞ _).differentiable
      (by simp)).differentiableAt).hasAdjFDerivAt
  rw [adjFDeriv_uncurry
    ((by fun_prop : ContDiff ℝ ∞ _).differentiable (by simp)).differentiableAt]
  all_goals exact ((by fun_prop : ContDiff ℝ ∞ _).differentiable (by simp)).differ

```

1.15 Physlib/ClassicalMechanics/HarmonicOscillator/Basic.lean

```

public import Mathlib.Algebra.Order.Archimedean.Real.Hom
Here we both define and prove properties related to the angular frequency.
simp only [ContinuousLinearMap.smul_comp, FunLike.coe_smul,
  ContinuousLinearMap.coe_comp, Pi.smul_apply, Function.comp_apply, smul_eq_mul]
simp only [fderiv_fun_id, ContinuousLinearMap.coe_id', id_eq]
simp only [ContinuousLinearMap.smul_comp, FunLike.coe_smul,
  ContinuousLinearMap.coe_comp, Pi.smul_apply, Function.comp_apply, smul_eq_mul]
simp only [fderiv_fun_id, ContinuousLinearMap.coe_id', id_eq]
simp only [_root_.add_apply]
simp only [fderiv_fun_id, ContinuousLinearMap.coe_id', id_eq, real_inner_comm, inn
  exact congrArg (fun L => L 1) (fderiv_const_smul (c := S.m) (f := ∂t xt) h
simp only [_root_.zero_apply]
  exact congrArg (fun L => L 1) (fderiv_const_smul (c := S.m) (f := ∂t xt) h

```

1.16 Physlib/ClassicalMechanics/HarmonicOscillator/Geometric/Basic.lean

```

/-
Copyright (c) 2026 Nicola Bernini. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Nicola Bernini, Nathaneal Sajan
-/
module

public import Physlib.SpaceAndTime.Space.Basic
public import Mathlib.Geometry.Manifold.Diffeomorph
/-!

```

Configuration space of the harmonic oscillator

i. Overview

The configuration space ``Q`` of the one-dimensional harmonic oscillator is the set of possible positions of the oscillator, formalised here as a one-dimensional smooth manifold.

``Q`` carries a single chosen global coordinate, modeled by ``EuclideanSpace ℝ``. The `coordinate` supplies the topology and the smooth-manifold structure through the `chart`.

ii. Key results

- ``ConfigurationSpace`` : the configuration manifold ``Q`` of the harmonic oscillator, modeled by the chosen ``EuclideanSpace ℝ (Fin 1)`` coordinate.
- ``ConfigurationSpace.valEquiv`` : the coordinate equivalence identifying ``Q`` with the ``EuclideanSpace ℝ (Fin 1)`` model.
- ``ConfigurationSpace.valHomeomorphism`` : the global coordinate homeomorphism from ``Q`` to the manifold chart.
- ``ConfigurationSpace.valDiffeomorphism`` : the global coordinate chart as an analytic diffeomorphism, upgrading ``valHomeomorphism`` to a smooth identification of ``Q`` with the ``ChartedSpace`` and ``IsManifold`` instances, exhibiting ``Q`` as a one-dimensional analytic manifold modeled on ``EuclideanSpace ℝ (Fin 1)``.
- ``ConfigurationSpace.toSpace`` : the point of physical ``Space 1`` determined by the configuration.

iii. Table of contents

- A. The configuration space type
- B. Topology and coordinate homeomorphism
- C. Smooth manifold structure
- D. The coordinate diffeomorphism
- E. Map to physical space

iv. References

- Ivo Terek, Introductory Variational Calculus on Manifolds, page 1 (Section 1.1: definitions and examples).
- /

@[expose] public section

namespace ClassicalMechanics

1.16.

PHYSLIB/CLASSICALMECHANICS/HARMONICOSCILLATOR/GEOMETRIC/BASIC.LEAN

```
namespace HarmonicOscillator
```

```
TODO "The API around the configuration space
      should be improved to allow further development of a proper
      geometric model of the Harmonic Oscillator."
```

```
/-!
```

```
## A. The configuration space type
```

```
`ConfigurationSpace` wraps a single chosen global coordinate valued in
`EuclideanSpace  $\mathbb{R}$  (Fin 1)`. We record extensionality in this coordinate together with
function-like coordinate access mirroring that of `EuclideanSpace  $\mathbb{R}$  (Fin 1)`.
-/
```

```
/-- The one-dimensional configuration space `Q` of the harmonic oscillator: the space of
possible positions of the oscillator, equipped with a single chosen global coordinate
modeled by `EuclideanSpace  $\mathbb{R}$  (Fin 1)`. -/
```

```
structure ConfigurationSpace where
```

```
  /-- The chosen global coordinate of the configuration, valued in
  `EuclideanSpace  $\mathbb{R}$  (Fin 1)`. -/
  val : EuclideanSpace  $\mathbb{R}$  (Fin 1)
```

```
namespace ConfigurationSpace
```

```
@[ext]
```

```
lemma ext {x y : ConfigurationSpace} (h : x.val = y.val) : x = y := by
  cases x
  cases y
  subst h
  rfl
```

```
/-- A configuration may be applied like a function ` $\text{Fin } 1 \rightarrow \mathbb{R}$ `, evaluating its
underlying coordinate. This mirrors the function-like use of `EuclideanSpace  $\mathbb{R}$  (Fin 1)`.
/
```

```
instance : CoeFun ConfigurationSpace (fun _ =>  $\text{Fin } 1 \rightarrow \mathbb{R}$ ) where
  coe x := fun i => x.val i
```

```
lemma coe_apply (x : ConfigurationSpace) (i :  $\text{Fin } 1$ ) : x i = x.val i := rfl
```

```
/-!
```

```
## B. Topology and coordinate homeomorphism
```

```
`ConfigurationSpace` carries the topology induced by its chosen coordinate
`ConfigurationSpace.val`: a set of configurations is open exactly when it is the preimage of
an open set under `val`. The wrapper/unwrapper pair is the coordinate equivalence `val` and `coe`
which is a homeomorphism `valHomeomorphism` for this induced topology – its continuity is
guaranteed by the lemma `valHomeomorphism`.
```

both directions is just the universal property of the induced topology. We and second countability across it from the model space, so Q is a well-behaved topological manifold.

```

-/

/-- The topology on configuration space, induced by the chosen coordinate
`ConfigurationSpace.val` into `EuclideanSpace ℝ (Fin 1)`. -
/
instance : TopologicalSpace ConfigurationSpace :=
  TopologicalSpace.induced ConfigurationSpace.val inferInstance

/-- The coordinate equivalence between configuration space and its `Euclidean
model, given by `ConfigurationSpace.val` with its wrapper inverse. -
/
def valEquiv : ConfigurationSpace ≈ EuclideanSpace ℝ (Fin 1) where
  toFun := ConfigurationSpace.val
  invFun v := ⟨v⟩
  left_inv x := by cases x; rfl
  right_inv v := rfl

/-- The global coordinate homeomorphism between configuration space and its
`EuclideanSpace ℝ (Fin 1)` model. Continuity in both directions is exactly
property of the induced topology, so no norm or isometry structure is invol
homeomorphism underlies the single global chart used for the smooth-
manifold structure. -/
def valHomeomorphism : ConfigurationSpace ≈t EuclideanSpace ℝ (Fin 1) where
  toEquiv := valEquiv
  continuous_toFun := continuous_induced_dom
  continuous_invFun := by
    apply continuous_induced_rng.mpr
    exact continuous_id

/-- Configuration space is Hausdorff, transported from `EuclideanSpace ℝ (F
coordinate homeomorphism. -/
instance : T2Space ConfigurationSpace := valHomeomorphism.symm.t2Space

/-- Configuration space is second countable, transported from `EuclideanSpa
across the coordinate homeomorphism. -/
instance : SecondCountableTopology ConfigurationSpace :=
  valHomeomorphism.secondCountableTopology

/-!
## C. Smooth manifold structure

`ConfigurationSpace` is an analytic manifold modeled on `EuclideanSpace ℝ (

```

1.16.

PHYSLIB/CLASSICALMECHANICS/HARMONICOSCILLATOR/GEOMETRIC1BASIC.LEAN

single global chart `valHomeomorphism`. With one chart the only coordinate change is chart's self-transition, which is analytic, so chart compatibility is immediate.

-/

/-- The structure of a charted space on `ConfigurationSpace`, modeled on its
`EuclideanSpace  $\mathbb{R}$  (Fin 1)` coordinate via the single global chart `valHomeomorphism`
/

```
instance : ChartedSpace (EuclideanSpace  $\mathbb{R}$  (Fin 1)) ConfigurationSpace where
  atlas := { valHomeomorphism.toOpenPartialHomeomorph }
  chartAt _ := valHomeomorphism.toOpenPartialHomeomorph
  mem_chart_source := by
    simp
  chart_mem_atlas := by
    intro x
    simp
```

open Manifold ContDiff

/-- The structure of a smooth (indeed analytic) manifold on `ConfigurationSpace`. With
single global chart, the only coordinate change is the chart's self-
transition, which is
analytic. -/

```
instance : IsManifold  $\square(\mathbb{R}$ , EuclideanSpace  $\mathbb{R}$  (Fin 1))  $\omega$  ConfigurationSpace where
  compatible := by
    intro e1 e2 h1 h2
    simp [atlas, ChartedSpace.atlas] at h1 h2
    subst h1 h2
    exact symm_trans_mem_contDiffGroupoid valHomeomorphism.toOpenPartialHomeomorph
```

/-!

D. The coordinate diffeomorphism

The single global chart is an analytic diffeomorphism, not merely a homeomorphism, s
identified with its `EuclideanSpace  $\mathbb{R}$  (Fin 1)` model as a smooth manifold. With one
is immediate: the only transition is the analytic self-transition already recorded i
manifold structure. This smooth identification is what lets smoothness of maps to or
be tested in the chosen global coordinate, the device used for trajectories and thei
-/

/-- The global coordinate chart as an analytic diffeomorphism between `Configuration
its `EuclideanSpace  $\mathbb{R}$  (Fin 1)` model, upgrading `valHomeomorphism` to a smooth ident
/

```
def valDiffeomorph :
  ConfigurationSpace  $\approx_m^{\omega \square}(\mathbb{R}$ , EuclideanSpace  $\mathbb{R}$  (Fin 1)),  $\square(\mathbb{R}$ , EuclideanSpace  $\mathbb{R}$  (F
  EuclideanSpace  $\mathbb{R}$  (Fin 1) where
  toEquiv := valEquiv
```

```

contMDiff_toFun := by
  have h := contMDiffOn_chart (I :=  $\square(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$ ) (n :=
    (x := ( $\{0\}$ ) : ConfigurationSpace))
  rw [show (chartAt (EuclideanSpace  $\mathbb{R}$  (Fin 1)) ( $\{0\}$ ) : ConfigurationSpace)
    from rfl, contMDiffOn_univ] at h
  exact h
contMDiff_invFun := by
  have h := contMDiffOn_chart_symm (I :=  $\square(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$ )
    (x := ( $\{0\}$ ) : ConfigurationSpace))
  rw [show (chartAt (EuclideanSpace  $\mathbb{R}$  (Fin 1)) ( $\{0\}$ ) : ConfigurationSpace)
    from rfl, contMDiffOn_univ] at h
  exact h

/-!
## E. Map to physical space

The point of one-dimensional physical `Space 1` determined by a configuration
reading off the underlying coordinate. This links the abstract configuration
concrete coordinate model.
-/

/-- The position in one-dimensional space associated to the configuration.
/
def toSpace (q : ConfigurationSpace) : Space 1 :=  $\langle \text{fun } i \Rightarrow q.\text{val } i \rangle$ 

lemma toSpace_apply (q : ConfigurationSpace) (i : Fin 1) : q.toSpace i = q.
end ConfigurationSpace
end HarmonicOscillator
end ClassicalMechanics

```

1.17 Physlib/ClassicalMechanics/HarmonicOscillator/Geometric/Tr

```

/-
Copyright (c) 2026 Nathaneal Sajan. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Nathaneal Sajan
-/
module

public import Physlib.ClassicalMechanics.HarmonicOscillator.Geometric.Basic
public import Physlib.SpaceAndTime.Time.Derivatives

```


1.17.

PHYSLIB/CLASSICALMECHANICS/HARMONICOSCILLATOR/GEOMETRICTRAJECTORY.LEAN

```
public import Mathlib.Geometry.Manifold.ContMDiff.NormedSpace
```

```
public import Mathlib.Geometry.Manifold.MFDeriv.Basic
```

```
/-!
```

```
# Geometric trajectories of the harmonic oscillator
```

```
## i. Overview
```

A trajectory of the harmonic oscillator is a time-parametrized curve in the configuration manifold Q . Since this model of Q has a single global coordinate valued in $\text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)}$, every geometric trajectory has an associated coordinate

The coordinate diffeomorphism from Q to its model space lets smoothness of a geometric trajectory be tested as ordinary smoothness of its coordinate curve.

```
## ii. Key results
```

- `Trajectory` : a curve from `Time` into the configuration manifold.
- `Trajectory.coord` : the global Euclidean coordinate curve of a trajectory.
- `Trajectory.toSpace` : the physical-space position along a trajectory.
- `Trajectory.contMDiff_iff_contDiff_coord` : geometric smoothness of a trajectory is equivalent to ordinary smoothness of its coordinate curve.
- `Trajectory.velocity` : the geometric velocity of a trajectory as a tangent vector.
- `Trajectory.velocity_eq_deriv_coord` : in the global coordinate, geometric velocity is represented by the time derivative of the coordinate curve.

```
## iii. Table of contents
```

- A. The trajectory type and coordinate projection
- B. Smoothness of trajectories
- C. Velocity in the tangent bundle

```
## iv. References
```

- Ivo Terek, Introductory Variational Calculus on Manifolds, pages 1-2 (Section 1, Basic definitions and examples).
- ```
-/
```

```
@[expose] public section
```

```
namespace ClassicalMechanics
```

```
namespace HarmonicOscillator
```

```
open scoped Manifold
```

```
open Time
```

```

/-!
A. The trajectory type and coordinate projection

A trajectory is a curve in the configuration manifold `Q`, parametrized by
coordinate projection reads the same curve in the chosen global coordinate,
forgets the manifold structure and returns the corresponding physical-
space position.
-/

/-- A trajectory of the oscillator: a curve in the configuration manifold `Q`
/
abbrev Trajectory := Time → ConfigurationSpace

namespace Trajectory

/-- The coordinate curve of a trajectory: its reading in the global Euclidean space
/
def coord (γ : Trajectory) : Time → EuclideanSpace ℝ (Fin 1) :=
 fun t => chartAt (EuclideanSpace ℝ (Fin 1)) (γ t) (γ t)

/-- The physical position of the oscillator along a trajectory, over time.
/
def toSpace (γ : Trajectory) : Time → Space 1 := fun t => (γ t).toSpace

lemma coord_apply (γ : Trajectory) (t : Time) : coord γ t = (γ t).val := rfl

/-!
B. Smoothness of trajectories

Because the global coordinate is a diffeomorphism, composing a trajectory with
it and reflecting manifold smoothness. Since both `Time` and the coordinate model
spaces, this manifold-smoothness statement then becomes ordinary `ContDiff`
-/

/-- A trajectory is smooth as a curve in `Q` iff its coordinate curve is smooth
/
lemma contMDiff_iff_contDiff_coord {n : WithTop ℕ} (γ : Trajectory) :
 ContMDiff (n, Time) (n, EuclideanSpace ℝ (Fin 1)) n γ ↔
 ContDiff ℝ n (coord γ) := by
 rw [show coord γ = ConfigurationSpace.valDiffeomorph ∘ γ by rfl]
 rw [← contMDiff_iff_contDiff]
 exact (ConfigurationSpace.valDiffeomorph.contMDiff_diffeomorph_comp_iff
 (m := n) (f := γ) le_top).symm

/-!

```

1.17.

PHYSLIB/CLASSICALMECHANICS/HARMONICOSCILLATOR/GEOMETRICTRAJECTORY.LEAN

## C. Velocity in the tangent bundle

The velocity of a trajectory at time `t` is the tangent vector obtained by differentiating the curve in the direction of the unit time vector. In this one-chart model, the tangent space at each configuration is represented by the same Euclidean model space, so the geometric velocity can be compared with the derivative of the coordinate curve.  
-/

/-- The geometric velocity of a trajectory at time `t`: a tangent vector at `γ t`. -/  
/

```
noncomputable def velocity (γ : Trajectory) (t : Time) :
 TangentSpace (ℝ, EuclideanSpace ℝ (Fin 1)) (γ t) :=
 mfderiv (ℝ, Time) (ℝ, EuclideanSpace ℝ (Fin 1)) γ t
 ((1 : Time) : TangentSpace (ℝ, Time) t)
```

/-- A trajectory is `MDifferentiableAt` a time iff its coordinate curve is ordinarily differentiable there, since the two differ by the coordinate diffeomorphism. -/  
/

```
lemma mdifferentiableAt_iff_differentiableAt_coord (γ : Trajectory) (t : Time) :
 MDifferentiableAt (ℝ, Time) (ℝ, EuclideanSpace ℝ (Fin 1)) γ t ↔
 DifferentiableAt ℝ (coord γ) t := by
 rw [← mdifferentiableAt_iff_differentiableAt]
 constructor
 · intro hy
 rw [show coord γ = ConfigurationSpace.valDiffeomorph ∘ γ by rfl]
 exact MDifferentiableAt.comp t
 ((ConfigurationSpace.valDiffeomorph.mdifferentiable WithTop.top_ne_zero).mdifferentiableAt) hy
 · intro hcoord
 have hy : MDifferentiableAt (ℝ, Time) (ℝ, EuclideanSpace ℝ (Fin 1))
 (ConfigurationSpace.valDiffeomorph.symm ∘ coord γ) t :=
 MDifferentiableAt.comp t
 ((ConfigurationSpace.valDiffeomorph.symm.mdifferentiable
 WithTop.top_ne_zero).mdifferentiableAt) hcoord
 convert hy using 1
 funext s
 exact (ConfigurationSpace.valDiffeomorph.symm_apply_apply (γ s)).symm
```

/-- A trajectory and its coordinate curve have the same manifold derivative, since they differ only by the coordinate diffeomorphism. This is the one lemma that unfolds the proof; every later result uses only the stable `mfderiv`/`fderiv` API. -/  
/

```
private lemma mfderiv_eq_mfderiv_coord (γ : Trajectory) (t : Time) :
 mfderiv (ℝ, Time) (ℝ, EuclideanSpace ℝ (Fin 1)) γ t =
 mfderiv (ℝ, Time) (ℝ, EuclideanSpace ℝ (Fin 1)) (coord γ) t := by
```

```

by_cases hy : MDifferentiableAt $\square(\mathbb{R}, \text{Time}) \square(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} (\text{Fin } 1))$
· have hcoord : MDifferentiableAt $\square(\mathbb{R}, \text{Time}) \square(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} (\text{Fin } 1))$
 (coord γ) t :=
 ((mdifferentiableAt_iff_differentiableAt_coord γ t).mp hy).mdifferentiableAt
simp [mfderiv, coord, writtenInExtChartAt, hy, hcoord,
 ConfigurationSpace.valHomeomorphism, ConfigurationSpace.valEquiv, Function.injective,
 rfl]
· rw [mfderiv_zero_of_not_mdifferentiableAt hy,
 mfderiv_zero_of_not_mdifferentiableAt
 (fun h => hy ((mdifferentiableAt_iff_differentiableAt_coord γ t).mp h.differentiableAt))]
 rfl

/-- In the global coordinate, geometric velocity is represented by the time
coordinate curve. -/
lemma velocity_eq_deriv_coord (γ : Trajectory) (t : Time) :
 velocity γ t = ∂_t (coord γ) t := by
 rw [velocity, mfderiv_eq_mfderiv_coord]
 exact (Time.deriv_eq_mfderiv (coord γ) t).symm

end Trajectory

end HarmonicOscillator

end ClassicalMechanics

```

## 1.18 Physlib/ClassicalMechanics/HarmonicOscillator/Solution.lean

Authors: Nathaneal Sajan, Joseph Tooby-Smith, Lode Vermeulen

```

public import Mathlib.Analysis.SpecialFunctions.Complex.Arg
- A.2.1. Initial conditions at arbitrary time
- A.2.2. Initial conditions from two positions at different times
- A.2.3. Initial conditions from two velocities at different times
- D.1. Correctness of InitialConditionsAtTime conversion
- D.2. Correctness of InitialConditionsFromTwoPositions conversion
- D.3. Correctness of InitialConditionsFromTwoVelocities conversion
- E. Amplitude-phase parametrization
 - E.1. The amplitude-phase initial conditions
 - E.2. Conversion to standard initial conditions
 - E.3. The trajectory in normal form
 - E.4. Recovering the amplitude and phase
- F. Special conditions of the trajectory
 - F.1. Normal form for standard initial conditions
 - F.2. Times at which the velocity is zero

```

1.18.

PHYSLIB/CLASSICALMECHANICS/HARMONICOSCILLATOR/SOLUTION.1LEAN

- F.3. The position when the velocity is zero
- F.4. Times at which the trajectory passes through zero
- G. Periodicity and recurrence
  - G.1. The period
  - G.2. Periodicity of the trajectory
  - G.3. Return to the initial state

TODO "Split this file into smaller modules, keeping `Solution.lean` as an umbrella i  
The intended organization is:

- `Solution.Basic` for trajectory construction and equation-of-motion facts;
- `Solution.Energy` for energy-related lemmas;
- `Solution.InitialData` for alternative initial-condition parametrizations;
- `Solution.AmplitudePhase` for the amplitude-phase normal form;
- `Solution.SpecialTimes` for velocity-zero times, turning points, and zero crossing
- `Solution.Periodicity` for period and recurrence facts."

- **\*\*Initial conditions from two positions at different times\*\***: Specify the position distinct times `t<sub>1</sub>` and `t<sub>2</sub>` that satisfy the non-degeneracy condition.
- **\*\*Initial conditions from two velocities at different times\*\***: Specify the velocity distinct times `t<sub>1</sub>` and `t<sub>2</sub>` that satisfy the non-degeneracy condition.
- **\*\*Amplitude-phase parametrization\*\***: Specify the solution as a single shifted cosine  $x(t) = A \cos(\omega t - \phi)$  with amplitude `A` and phase `φ`.

The conversion to the standard `InitialConditions` works by "running the trajectory time" from `t<sub>0</sub>` to `0`. Given that we know `x(t<sub>0</sub>)` and `v(t<sub>0</sub>)`, we use the harmonic solution formula with time-reversal to determine what `x(0)` and `v(0)` must have been.

Mathematically, if  $x(t) = \cos(\omega t) \cdot x_0 + (\sin(\omega t)/\omega) \cdot v_0$ , then setting `t = t<sub>0</sub>`:

$$\begin{aligned} x(t_0) &= \cos(\omega t_0) \cdot x_0 + (\sin(\omega t_0)/\omega) \cdot v_0 \\ v(t_0) &= -\omega \cdot \sin(\omega t_0) \cdot x_0 + \cos(\omega t_0) \cdot v_0 \end{aligned}$$

Solving this linear system for `x<sub>0</sub>` and `v<sub>0</sub>` gives the formulas in `toInitialConditions`.

/-!

#### A.2.2. Initial conditions from two positions at different times

We define a type for initial conditions specified by two measured positions `x\_t1` and `x\_t2` at two distinct times `t1` and `t2`.

The conversion to the standard `InitialConditions` is obtained by solving for `x<sub>0</sub>` and `v<sub>0</sub>` from two equations given by evaluating the trajectory at `t1` and `t2`:

$$\begin{aligned} x_{t1} &= \cos(\omega t_1) \cdot x_0 + (\sin(\omega t_1)/\omega) \cdot v_0 \\ x_{t2} &= \cos(\omega t_2) \cdot x_0 + (\sin(\omega t_2)/\omega) \cdot v_0 \end{aligned}$$

This linear system has determinant  $(\cos(\omega t_1) \cdot \sin(\omega t_2) - \cos(\omega t_2) \cdot \sin(\omega t_1))/\omega = \sin(\omega(t_2 - t_1))$ .

$t_1)/\omega`.$

Writing  $\Delta = \sin(\omega(t_2 - t_1))`$ , solving the system gives the formulas used below

$$\begin{aligned} x_0 &= (\sin(\omega t_2) \cdot x_{t_1} - \sin(\omega t_1) \cdot x_{t_2}) / \Delta \\ v_0 &= \omega \cdot (\cos(\omega t_1) \cdot x_{t_2} - \cos(\omega t_2) \cdot x_{t_1}) / \Delta \end{aligned}$$

The conversion is defined as a total function, but it recovers the initial  $\Delta = \sin(\omega(t_2 - t_1)) \neq 0`$ , i.e. when  $t_2 - t_1`$  is not an integer multiple of correctness proofs, under this nondegeneracy condition, are given later in

-/

/-- Initial conditions for the harmonic oscillator specified by two positions  $x_{t_1}$  and  $x_{t_2}$  measured at two times  $t_1$  and  $t_2$  respectively.

The conditions can be converted to the standard `InitialConditions` form using the `toInitialConditions` function. -/

@[ext] structure InitialConditionsFromTwoPositions where

/-- The first measurement time. -/

$t_1$  : Time

/-- The position at time  $t_1$ . -/

$x_{t_1}$  : EuclideanSpace  $\mathbb{R}$  (Fin 1)

/-- The second measurement time. -/

$t_2$  : Time

/-- The position at time  $t_2$ . -/

$x_{t_2}$  : EuclideanSpace  $\mathbb{R}$  (Fin 1)

namespace InitialConditionsFromTwoPositions

/-- Convert two-position initial conditions to standard initial conditions

Obtained by solving the  $2 \times 2$  linear system from the trajectory formula at See `toInitialConditions_trajectory_at_t1` and `toInitialConditions_trajectory_at_t2` section D.2 for the correctness proofs (valid under  $\sin(S.\omega * (t_2 - t_1)) \neq 0$ )

/

noncomputable def toInitialConditions (S : HarmonicOscillator)

(IC : InitialConditionsFromTwoPositions) : InitialConditions where

$$\begin{aligned} x_0 &:= (\sin(S.\omega * IC.t_2) / \sin(S.\omega * (IC.t_2 - IC.t_1))) \cdot IC.x_{t_1} \\ &\quad - (\sin(S.\omega * IC.t_1) / \sin(S.\omega * (IC.t_2 - IC.t_1))) \cdot IC.x_{t_2} \end{aligned}$$

$$\begin{aligned} v_0 &:= (S.\omega * \cos(S.\omega * IC.t_1) / \sin(S.\omega * (IC.t_2 - IC.t_1))) \cdot IC.x_{t_2} \\ &\quad - (S.\omega * \cos(S.\omega * IC.t_2) / \sin(S.\omega * (IC.t_2 - IC.t_1))) \cdot IC.x_{t_1} \end{aligned}$$

end InitialConditionsFromTwoPositions

/-!

1.18.

PHYSLIB/CLASSICALMECHANICS/HARMONICOSCILLATOR/SOLUTION.DLEAN

#### A.2.3. Initial conditions from two velocities at different times

We define a type for initial conditions specified by two measured velocities  $v_{t_1}$  at two distinct times  $t_1$  and  $t_2$ .

The conversion to the standard `InitialConditions` is obtained by solving for  $x_0$  and  $v_0$  two equations given by evaluating the velocity of the trajectory at  $t_1$  and  $t_2$ :

$$\begin{aligned} v_{t_1} &= -\omega \cdot \sin(\omega t_1) \cdot x_0 + \cos(\omega t_1) \cdot v_0 \\ v_{t_2} &= -\omega \cdot \sin(\omega t_2) \cdot x_0 + \cos(\omega t_2) \cdot v_0 \end{aligned}$$

This linear system has determinant  $\omega \cdot (\cos(\omega t_1) \cdot \sin(\omega t_2) - \cos(\omega t_2) \cdot \sin(\omega t_1)) = \omega \cdot \sin(\omega(t_2 - t_1))$ .

Writing  $\Delta = \sin(\omega(t_2 - t_1))$ , solving the system gives the formulas used below:

$$\begin{aligned} x_0 &= (\cos(\omega t_2) \cdot v_{t_1} - \cos(\omega t_1) \cdot v_{t_2}) / (\omega \cdot \Delta) \\ v_0 &= (\sin(\omega t_2) \cdot v_{t_1} - \sin(\omega t_1) \cdot v_{t_2}) / \Delta \end{aligned}$$

The conversion is defined as a total function, but it recovers the initial condition  $\Delta = \sin(\omega(t_2 - t_1)) \neq 0$ , i.e. when  $t_2 - t_1$  is not an integer multiple of half a period. Correctness proofs, under this nondegeneracy condition, are given later in section D.3.

-/

`-- Initial conditions for the harmonic oscillator specified by two velocities  $v_{t_1}$  and  $v_{t_2}$  measured at two times  $t_1$  and  $t_2$  respectively.`

The conditions can be converted to the standard `InitialConditions` format using the `toInitialConditions` function. -/

`@[ext] structure InitialConditionsFromTwoVelocities where`

`-- The first measurement time. -/`

`t1 : Time`

`-- The velocity at time  $t_1$ . -/`

`v_t1 : EuclideanSpace ℝ (Fin 1)`

`-- The second measurement time. -/`

`t2 : Time`

`-- The velocity at time  $t_2$ . -/`

`v_t2 : EuclideanSpace ℝ (Fin 1)`

`namespace InitialConditionsFromTwoVelocities`

`-- Convert two-velocity initial conditions to standard initial conditions at  $t = 0$`

Obtained by solving the  $2 \times 2$  linear system from the velocity formula at  $t_1$  and  $t_2$ . See `toInitialConditions_velocity_at_t1` and `toInitialConditions_velocity_at_t2` section D.3 for the correctness proofs (valid under  $\sin(S \cdot \omega * (t_2 - t_1)) \neq 0$ ).

/

`noncomputable def toInitialConditions (S : HarmonicOscillator)`

```

(IC : InitialConditionsFromTwoVelocities) : InitialConditions where
x₀ := (cos (S.ω * IC.t₂) / (S.ω * sin (S.ω * (IC.t₂ - IC.t₁)))) • IC.v_t₁
 - (cos (S.ω * IC.t₁) / (S.ω * sin (S.ω * (IC.t₂ - IC.t₁)))) • IC.v_t₂
v₀ := (sin (S.ω * IC.t₂) / sin (S.ω * (IC.t₂ - IC.t₁))) • IC.v_t₁
 - (sin (S.ω * IC.t₁) / sin (S.ω * (IC.t₂ - IC.t₁))) • IC.v_t₂

end InitialConditionsFromTwoVelocities

 root.neg_apply, FunLike.coe_smul, Pi.smul_apply, fderiv_val,
 simp only [neg_smul, add_apply, ContinuousLinearMap.smulRight_apply]
 simp only [fderiv_fun_const, Pi.ofNat_apply, smul_zero, add_zero, _root_.
 FunLike.coe_smul, Pi.smul_apply, ContinuousLinearMap.smulRight_apply, f
for the given initial conditions.
 Given any smooth `x` satisfying the equation of motion with the same init
 position and velocity, the difference `y = x - IC.trajectory S` also solv
 equation of motion with zero initial conditions; energy conservation then
 its energy, and hence `y`, to vanish identically, so `x = IC.trajectory S
/
rintro {hEOM, hx0, hv0}
have hTraj : ContDiff ℝ ∞ (IC.trajectory S) := by fun_prop
-- Time-derivative of a difference of differentiable functions, used below
have dsub : ∀ f g : Time → EuclideanSpace ℝ (Fin 1),
 Differentiable ℝ f → Differentiable ℝ g →
 ∂ₜ (fun t => f t - g t) = fun t => ∂ₜ f t - ∂ₜ g t := by
 intro f g hf hg
 simp only [Time.deriv_eq, fderiv_fun_sub (hf t) (hg t), sub_apply]
-- The difference `y := x - traj` is smooth, again solves the equation of mo
-- linear), and has vanishing initial data; energy conservation then forc
set y : Time → EuclideanSpace ℝ (Fin 1) := fun t => x t - IC.trajectory S
have hyContDiff : ContDiff ℝ ∞ y := hx.sub hTraj
have hy_deriv : ∂ₜ y = fun t => ∂ₜ x t - ∂ₜ (IC.trajectory S) t :=
 dsub x _ (hx.differentiable (by simp)) (hTraj.differentiable (by simp))
have hy_deriv2 : ∂ₜ (∂ₜ y) = fun t => ∂ₜ (∂ₜ x) t - ∂ₜ (∂ₜ (IC.trajectory
 rw [hy_deriv]
 exact dsub _ _ (deriv_differentiable_of_contDiff _ hx)
 (deriv_differentiable_of_contDiff _ hTraj)
have hNewt_x := (S.equationOfMotion_iff_newtons_2nd_law x hx).1 hEOM
have hNewt_traj := (S.equationOfMotion_iff_newtons_2nd_law (IC.trajectory
 (trajectory_equationOfMotion S IC)
 (S.equationOfMotion_iff_newtons_2nd_law y hyContDiff).2 fun t => by
 rw [hy_deriv2]
 simp [smul_sub, hNewt_x, hNewt_traj, hydef, force_eq_linear]
have hE : ∀ t, S.energy y t = 0 := fun t =>
 (S.energy_conservation_of_equationOfMotion' y hyContDiff hEOM_y t).tran
 have hy0 : y 0 = 0 := by simp [hydef, hx0]
 have hyv0 : ∂ₜ y 0 = 0 := by

```



1.18.

PHYSLIB/CLASSICALMECHANICS/HARMONICOSCILLATOR/SOLUTION.19.EAN

```

 rw [congrFun hy_deriv 0, hv0, trajectory_velocity_at_zero S IC]; simp
 simp [HarmonicOscillator.energy, HarmonicOscillator.kineticEnergy,
 HarmonicOscillator.potentialEnergy, hy0, hv0, one_div, smul_eq_mul]
-- Both energies are nonnegative, so a vanishing total energy forces `y t = 0`.
have hk : 0 ≤ S.kineticEnergy y t := by
 simp only [HarmonicOscillator.kineticEnergy]
 exact mul_nonneg (mul_nonneg (by norm_num) S.m_pos.le) real_inner_self_nonneg
have hp : 0 ≤ S.potentialEnergy (y t) := by
 simp only [HarmonicOscillator.potentialEnergy, smul_eq_mul]
 exact mul_nonneg (by norm_num) (mul_nonneg S.k_pos.le real_inner_self_nonneg)
have hpe : S.potentialEnergy (y t) = 0 := ((add_eq_zero_iff_of_nonneg hk hp).mp (h
simp only [HarmonicOscillator.potentialEnergy, smul_eq_mul] at hpe
rcases mul_eq_zero.mp hpe with h | h
· norm_num at h
· have hyt : x t - IC.trajectory S t = 0 :=
 inner_self_eq_zero.mp ((mul_eq_zero.mp h).resolve_left S.k_ne_zero)
 exact sub_eq_zero.mp hyt
D.1. Correctness of InitialConditionsAtTime conversion
/-!

```

### D.2. Correctness of InitialConditionsFromTwoPositions conversion

The conversion recovers the initial conditions only when  $\sin(S.\omega * (t_2 - t_1)) \neq 0$ .  
 condition fails exactly when  $\omega \cdot (t_2 - t_1) = n \cdot \pi$  for some integer  $n$ , i.e. when  $t_2$   
 integer multiple of half a period; in that case  $x(t_2) = (-1)^n \cdot x(t_1)$  for every trajectory,  
 independent of  $v_0$ , so the two positions do not determine the initial conditions.

Under this nondegeneracy condition, we prove that the resulting trajectory passes through  
 at  $t_1$  and  $x_{t_2}$  at  $t_2$ .

-/

namespace InitialConditionsFromTwoPositions

```

/-- The trajectory from `toInitialConditions` passes through `x_t1` at time `t1`,
 provided $\sin(S.\omega * (t_2 - t_1)) \neq 0$. -/
lemma toInitialConditions_trajectory_at_t1 (S : HarmonicOscillator)
 (IC : InitialConditionsFromTwoPositions)
 (hΔ : $\sin(S.\omega * (IC.t_2 - IC.t_1)) \neq 0$) :
 (IC.toInitialConditions S).trajectory S IC.t1 = IC.x_t1 := by
 rw [InitialConditions.trajectory_eq, toInitialConditions]
 ext i
 simp only [PiLp.add_apply, PiLp.smul_apply, PiLp.sub_apply, smul_eq_mul]
 field_simp [S.ω_ne_zero]
 grind [mul_sub, Real.sin_sub]

```

```

/-- The trajectory from `toInitialConditions` passes through `x_t2` at time `t2`
provided `sin (S.ω * (t2 - t1)) ≠ 0`. -/
lemma toInitialConditions_trajectory_at_t2 (S : HarmonicOscillator)
 (IC : InitialConditionsFromTwoPositions)
 (hΔ : sin (S.ω * (IC.t2 - IC.t1)) ≠ 0) :
 (IC.toInitialConditions S).trajectory S IC.t2 = IC.x_t2 := by
 rw [InitialConditions.trajectory_eq, toInitialConditions]
 ext i
 simp only [PiLp.add_apply, PiLp.smul_apply, PiLp.sub_apply, smul_eq_mul]
 field_simp [S.ω_ne_zero]
 grind [mul_sub, Real.sin_sub]

end InitialConditionsFromTwoPositions
D.3. Correctness of InitialConditionsFromTwoVelocities conversion
The conversion recovers the initial conditions only when `sin (S.ω * (t2 - t1)) ≠ 0`
nondegeneracy condition, we prove that the resulting trajectory has velocity `v_t2`
at `t2`.
namespace InitialConditionsFromTwoVelocities

/-- The trajectory from `toInitialConditions` has velocity `v_t1` at time `t1`
provided `sin (S.ω * (t2 - t1)) ≠ 0`. -/
lemma toInitialConditions_velocity_at_t1 (S : HarmonicOscillator)
 (IC : InitialConditionsFromTwoVelocities)
 (hΔ : sin (S.ω * (IC.t2 - IC.t1)) ≠ 0) :
 ∂t ((IC.toInitialConditions S).trajectory S) IC.t1 = IC.v_t1 := by
 rw [InitialConditions.trajectory_velocity, toInitialConditions]
 ext i
 simp only [neg_smul, PiLp.add_apply, PiLp.neg_apply, PiLp.smul_apply, PiLp.smul_eq_mul]
 field_simp [S.ω_ne_zero]
 grind [mul_sub, Real.sin_sub]

/-- The trajectory from `toInitialConditions` has velocity `v_t2` at time `t2`
provided `sin (S.ω * (t2 - t1)) ≠ 0`. -/
lemma toInitialConditions_velocity_at_t2 (S : HarmonicOscillator)
 (IC : InitialConditionsFromTwoVelocities)
 (hΔ : sin (S.ω * (IC.t2 - IC.t1)) ≠ 0) :
 ∂t ((IC.toInitialConditions S).trajectory S) IC.t2 = IC.v_t2 := by
 rw [InitialConditions.trajectory_velocity, toInitialConditions]
 ext i
 simp only [neg_smul, PiLp.add_apply, PiLp.neg_apply, PiLp.smul_apply, PiLp.smul_eq_mul]
 field_simp [S.ω_ne_zero]
 grind [mul_sub, Real.sin_sub]

```

1.18.

PHYSLIB/CLASSICALMECHANICS/HARMONICOSCILLATOR/SOLUTION.BEAN

end InitialConditionsFromTwoVelocities

## E. Amplitude-phase parametrization

The state of the harmonic oscillator at  $t = 0$  is captured by `InitialConditions` a  $x_0$  and a velocity  $v_0$ . An equivalent and often more physical description writes  $x(t)$  as a single shifted cosine of amplitude  $A$  and phase  $\varphi$ :

$$x(t) = A \cos(\omega t - \varphi).$$

Expanding with the angle-subtraction identity,

$$A \cos(\omega t - \varphi) = (A \cos \varphi) \cos(\omega t) + (A \sin \varphi) \sin(\omega t),$$

and matching coefficients against the standard solution

$$x(t) = \cos(\omega t) x_0 + (\sin(\omega t) / \omega) v_0$$

gives the change of coordinates

$$x_0 = A \cos \varphi, \quad v_0 = A \omega \sin \varphi.$$

We implement the forward map  $(A, \varphi) \mapsto (x_0, v_0)$  as `toInitialConditions`, prove the trajectory is the cosine normal form above (with velocity  $-$

$A \omega \sin(\omega t - \varphi)$ ), and implement the

inverse map  $(x_0, v_0) \mapsto (A, \varphi)$  as `fromInitialConditions`, recovering  $A$  and  $\varphi$  as coordinates of the phase vector  $(x_0, v_0 / \omega)$ .

### E.1. The amplitude-phase initial conditions

We define a type for initial conditions specified by an amplitude  $A$  and a phase  $\varphi$ . An amplitude and an angle, these are stored as scalars, rather than as vectors as for initial-condition types.

`-- Initial conditions for the harmonic oscillator specified by an amplitude  $A$  and a phase offset  $\varphi$ , describing the solution  $x(t) = A \cos(\omega t - \varphi)$ .`

The conditions can be converted to the standard `InitialConditions` format using `toInitialConditions` function. `--`

`@[ext] structure AmplitudePhase where`

`-- The amplitude of the oscillation. --`

`A : ℝ`

`-- The phase offset of the oscillation. --`

`φ : ℝ`

`namespace AmplitudePhase`

`--!`

### E.2. Conversion to standard initial conditions

Using  $x_0 = A \cos \varphi$  and  $v_0 = A \omega \sin \varphi$ , we convert amplitude-phase data to the standard position and velocity at  $t = 0$ .

`--`

```

/-- Convert amplitude-phase initial conditions to standard initial conditions
`x_0 = A cos φ` and `v_0 = A ω sin φ`.

See `toInitialConditions_trajectory_eq_cos` and `toInitialConditions_velocity_eq_sin`
section E.3 for the correctness proofs. -/
noncomputable def toInitialConditions (S : HarmonicOscillator) (IC : AmplitudePhaseInitialConditions) where
 x_0 := EuclideanSpace.single 0 (IC.A * cos IC.φ)
 v_0 := EuclideanSpace.single 0 (IC.A * S.ω * sin IC.φ)

/-!

E.3. The trajectory in normal form

The trajectory built from amplitude-phase data is exactly the single cosine
`x(t) = A cos (ω t - φ)`, with velocity `v(t) = -A ω sin (ω t - φ)`. In the
factor `1 / ω` of the standard solution cancels the `ω` in `v_0 = A ω sin φ`.

-/

/-- The trajectory of amplitude-phase initial conditions is the cosine normal form
`x(t) = A cos (ω t - φ)`. -/
lemma toInitialConditions_trajectory_eq_cos (S : HarmonicOscillator) (IC : AmplitudePhaseInitialConditions) (t : Time) :
 (IC.toInitialConditions S).trajectory S t
 = EuclideanSpace.single 0 (IC.A * cos (S.ω * t - IC.φ)) := by
 rw [InitialConditions.trajectory_eq, toInitialConditions]
 fin_cases i
 simp [Real.cos_sub]
 field_simp [S.ω_ne_zero]

/-- The velocity of the amplitude-phase trajectory is `v(t) = -A ω sin (ω t - φ)`. -/
lemma toInitialConditions_velocity_eq_sin (S : HarmonicOscillator) (IC : AmplitudePhaseInitialConditions) (t : Time) :
 ∂_t ((IC.toInitialConditions S).trajectory S) t
 = EuclideanSpace.single 0 (-(IC.A * S.ω * sin (S.ω * t.val - IC.φ)))
 rw [InitialConditions.trajectory_velocity, toInitialConditions]
 ext i
 fin_cases i
 simp [Real.sin_sub]
 ring

/-!

E.4. Recovering the amplitude and phase

```

1.18.

PHYSLIB/CLASSICALMECHANICS/HARMONICOSCILLATOR/SOLUTION.1.18.AN

The inverse map  $(x_0, v_0) \mapsto (A, \varphi)$  must solve  $x_0 = A \cos \varphi$  and  $v_0 / \omega = A \sin \varphi$ , the angle with the real `arctan` covers only  $(-\pi/2, \pi/2)$  and forces a case split and instead embed the phase vector as the complex number  $z = x_0 + (v_0 / \omega) i$  and read  $\varphi = \text{Complex.arg } z$ , with `arg` in the canonical range  $(-\pi, \pi]$ . The degenerate state  $x_0 = v_0 = 0$  is covered by the convention  $\arg 0 = 0$ , so no case split is needed.

We prove that converting initial conditions to amplitude-phase form and back returns initial conditions.

-/

*-- Recover amplitude-phase data from standard initial conditions, as the polar coordinate phase vector  $(x_0, v_0 / \omega)$  embedded as  $z = x_0 + (v_0 / \omega) i$ : the amplitude is  $\|z\|$  and the phase is  $\text{Complex.arg } z$ .*

See `toInitialConditions_fromInitialConditions` for the right-inverse identity. -/

noncomputable def fromInitialConditions (S : HarmonicOscillator) (IC : InitialConditions) : AmplitudePhase where

A :=  $\|(\langle \text{IC.x}_0 \ 0, \text{IC.v}_0 \ 0 / \text{S.}\omega \rangle : \mathbb{C})\|$

$\varphi := \text{Complex.arg } (\langle \text{IC.x}_0 \ 0, \text{IC.v}_0 \ 0 / \text{S.}\omega \rangle : \mathbb{C})$

*-- `fromInitialConditions` is a right inverse of `toInitialConditions`: converting initial conditions to amplitude-phase form and back recovers them exactly. -*

/

lemma toInitialConditions\_fromInitialConditions (S : HarmonicOscillator) (IC : InitialConditions) :

(fromInitialConditions S IC).toInitialConditions S = IC := by

have hw :  $\text{S.}\omega \neq 0 := \text{S.}\omega\_ne\_zero$

set z :  $\mathbb{C} := (\langle \text{IC.x}_0 \ 0, \text{IC.v}_0 \ 0 / \text{S.}\omega \rangle : \mathbb{C})$

-- polar identities

have hcos :  $\|z\| * \cos (\text{Complex.arg } z) = z.\text{re} := \text{by}$

rcases eq\_or\_ne z 0 with h | h

· simp [h]

· rw [Complex.cos\_arg h]; field\_simp

have hsin :  $\|z\| * \sin (\text{Complex.arg } z) = z.\text{im} := \text{by}$

rcases eq\_or\_ne z 0 with h | h

· simp [h]

· rw [Complex.sin\_arg]; field\_simp

-- By construction the parts of  $z$  are exactly the original data.

have hre :  $z.\text{re} = \text{IC.x}_0 \ 0 := \text{rfl}$

have him :  $z.\text{im} = \text{IC.v}_0 \ 0 / \text{S.}\omega := \text{rfl}$

apply InitialConditions.ext

· -- Position:  $\|z\| \cos (\arg z) = \text{Re } z = \text{IC.x}_0 \ 0$ , and  $\|z\| \sin (\arg z) = \text{IC.v}_0 \ 0 / \text{S.}\omega$

```

show EuclideanSpace.single 0 (||z|| * cos (Complex.arg z)) = IC.x₀
rw [hcos, hre]
ext i; fin_cases i; simp
· -- Velocity: `||z|| ω sin (arg z) = ω · Im z = ω · (v₀ / ω) = IC.v₀ θ`, t
show EuclideanSpace.single 0 (||z|| * S.ω * sin (Complex.arg z)) = IC.v₀
have hv : ||z|| * S.ω * sin (Complex.arg z) = IC.v₀ θ := by
 rw [mul_right_comm, hsin, him]; field_simp
 rw [hv]
ext i; fin_cases i; simp

end AmplitudePhase

namespace InitialConditions

/-!

F. Special conditions of the trajectory

We use the amplitude-phase parametrization from section E to describe the s
trajectory. After converting arbitrary initial conditions to amplitude and
has the form `x(t) = A cos (ω t - φ)` and its velocity has the form `v(t) =
Aω sin (ω t - φ)`.

Thus the turning points of the motion are controlled by the zeros of `sin (
times at which the trajectory passes through the origin are controlled by t
`cos (ω t - φ)`.

-/

/-!

F.1. Normal form for standard initial conditions

The amplitude-phase normal form was first proved for data already expressed
`AmplitudePhase`. We now transport those identities back to ordinary `Initi
`AmplitudePhase.fromInitialConditions`.

-/

/-- Every trajectory of the harmonic oscillator is a single shifted cosine
initial conditions to amplitude-phase form. -/
lemma trajectory_eq_cos (IC : InitialConditions) (t : Time) :
 IC.trajectory S t =
 EuclideanSpace.single 0 ((AmplitudePhase.fromInitialConditions S IC).
 cos (S.ω * t - (AmplitudePhase.fromInitialConditions S IC).φ)) := b

```

1.18.

PHYSLIB/CLASSICALMECHANICS/HARMONICOSCILLATOR/SOLUTION.18.EAN

```

conv_lhs =>
 rw [← AmplitudePhase.toInitialConditions_fromInitialConditions S IC]
exact AmplitudePhase.toInitialConditions_trajectory_eq_cos S
 (AmplitudePhase.fromInitialConditions S IC) t

/-- The velocity of every trajectory is the corresponding shifted sine in amplitude-
phase form. -/
lemma trajectory_velocity_eq_sin (IC : InitialConditions) (t : Time) :
 ∂t (IC.trajectory S) t =
 EuclideanSpace.single 0 (-((AmplitudePhase.fromInitialConditions S IC).A * S.ω *
 sin (S.ω * t.val - (AmplitudePhase.fromInitialConditions S IC).φ))) := by
conv_lhs =>
 rw [← AmplitudePhase.toInitialConditions_fromInitialConditions S IC]
exact AmplitudePhase.toInitialConditions_velocity_eq_sin S
 (AmplitudePhase.fromInitialConditions S IC) t

/-!

F.2. Times at which the velocity is zero

In amplitude-phase form the velocity is $v(t) = -A\omega \sin(\omega t - \phi)$. For nonzero amplitude, the velocity vanishes exactly when $\sin(\omega t - \phi) = 0$, equivalently when $\omega t - \phi$ is an integer multiple of π .

-/

/-- For nonzero amplitude, the velocity vanishes exactly when the sine factor in
amplitude-phase form vanishes. -/
lemma trajectory_velocity_eq_zero_iff_sin_eq_zero (IC : InitialConditions)
 (hA : (AmplitudePhase.fromInitialConditions S IC).A ≠ 0) (t : Time) :
 ∂t (IC.trajectory S) t = 0 ↔
 sin (S.ω * t.val - (AmplitudePhase.fromInitialConditions S IC).φ) = 0 := by
rw [trajectory_velocity_eq_sin]
constructor
· intro h
 have hscalar :
 -((AmplitudePhase.fromInitialConditions S IC).A * S.ω *
 sin (S.ω * t.val - (AmplitudePhase.fromInitialConditions S IC).φ)) = 0 :=
 simpa using congrArg (fun x : EuclideanSpace ℝ (Fin 1) => x 0) h
 have hprod :
 (AmplitudePhase.fromInitialConditions S IC).A * S.ω *
 sin (S.ω * t.val - (AmplitudePhase.fromInitialConditions S IC).φ) = 0 := by
 simpa only [neg_eq_zero] using hscalar
 rcases mul_eq_zero.mp hprod with hAω | hsin
 · rcases mul_eq_zero.mp hAω with hA0 | hω
 · exact (hA hA0).elim

```

```

 · exact (S.ω_ne_zero hw).elim
 · exact hsin
 · intro h
 simp [h]

/-- For nonzero amplitude, the velocity is zero exactly at phase times `φ +
/
lemma trajectory_velocity_eq_zero_iff_exists_int (IC : InitialConditions)
 (hA : (AmplitudePhase.fromInitialConditions S IC).A ≠ 0) (t : Time) :
 ∂t (IC.trajectory S) t = 0 ↔
 ∃ n : ℤ,
 (t : ℝ) =
 ((AmplitudePhase.fromInitialConditions S IC).φ + n * π) / S.ω :=
rw [trajectory_velocity_eq_zero_iff_sin_eq_zero S IC hA t]
constructor
· intro h
 obtain ⟨n, hn⟩ := Real.sin_eq_zero_iff.mp h
 use n
 have hw : S.ω ≠ 0 := S.ω_ne_zero
 have ht :
 S.ω * t.val = (AmplitudePhase.fromInitialConditions S IC).φ + n * π
 linarith
calc
 (t : ℝ) = (S.ω * t) / S.ω := by field_simp [hw]
 _ = ((AmplitudePhase.fromInitialConditions S IC).φ + n * π) / S.ω :=
 rw [ht]
· intro h
 obtain ⟨n, hn⟩ := h
 rw [Real.sin_eq_zero_iff]
 use n
 have hw : S.ω ≠ 0 := S.ω_ne_zero
 rw [hn]
 field_simp [hw]
 ring
F.3. The position when the velocity is zero

```

The zeros of the velocity are the turning points of the oscillator. In amplitude form, these are the times when  $\sin(\omega t - \varphi) = 0$ ; equivalently,  $\cos(\omega t - \varphi) = \pm 1$ .

At exactly those times the trajectory has maximal norm, equal to the amplitude. The statement also covers the degenerate case  $A = 0$ : then the trajectory is identically zero, so both sides of the equivalence hold at every time.

```

/-- The velocity vanishes exactly when the trajectory has norm equal to the
/
lemma trajectory_velocity_eq_zero_iff_norm_eq_amplitude (IC : InitialConditions)

```



1.18.

PHYSLIB/CLASSICALMECHANICS/HARMONICOSCILLATOR/SOLUTION.BEAN

```

 (t : Time) :
 ||IC.trajectory S t|| = (AmplitudePhase.fromInitialConditions S IC).A := by
by_cases hA : (AmplitudePhase.fromInitialConditions S IC).A = 0
· constructor
· intro _
 rw [trajectory_eq_cos]
 simp [hA]
· intro _
 rw [trajectory_velocity_eq_sin]
 ext i
 fin_cases i
 simp [hA]
rw [trajectory_velocity_eq_zero_iff_sin_eq_zero S IC hA t]
rw [trajectory_eq_cos]
set A := (AmplitudePhase.fromInitialConditions S IC).A
set θ := S. ω * t.val - (AmplitudePhase.fromInitialConditions S IC). ϕ
show $\sin \theta = 0 \Leftrightarrow ||\text{EuclideanSpace.single } \theta (A * \cos \theta)|| = A$
have hA' : $A \neq 0$:= by simpa [A] using hA
have hA_nonneg : $0 \leq A$:= by
 show $0 \leq ||(\text{IC.x } 0, \text{IC.v } 0 / S.\omega) : \mathbb{C}||$
 exact norm_nonneg _
have hA_pos : $0 < A$:= lt_of_le_of_ne hA_nonneg (Ne.symm hA')
· intro hsin
 rcases Real.sin_eq_zero_iff_cos_eq.mp hsin with hcos | hcos
 · simp [hcos, abs_of_pos hA_pos]
 · simp [hcos, abs_of_pos hA_pos]
· intro hnorm
 have hnorm' : $|A * \cos \theta| = A$:= by
 simpa using hnorm
 have hcos_abs : $|\cos \theta| = 1$:= by
 $|\cos \theta| = |A * \cos \theta| / A$:= by
 rw [abs_mul, abs_of_pos hA_pos]
 field_simp [hA']
 _ = A / A := by rw [hnorm']
 _ = 1 := by field_simp [hA']
 obtain (n, hn) := Real.abs_cos_eq_one_iff.mp hcos_abs
 exact Real.sin_eq_zero_iff.mpr (n, hn)

/-!

```

### F.4. Times at which the trajectory passes through zero

In amplitude-phase form the trajectory is  $x(t) = A \cos(\omega t - \phi)$ . For nonzero amplitude,  $x(t)$  vanishes exactly when  $\cos(\omega t - \phi) = 0$ , equivalently when the phase is an odd multiple of  $\pi/2$ .

```

-/

/-- For nonzero amplitude, the trajectory passes through zero exactly when
 amplitude-phase form vanishes. -/
lemma trajectory_eq_zero_iff_cos_eq_zero (IC : InitialConditions)
 (hA : (AmplitudePhase.fromInitialConditions S IC).A ≠ 0) (t : Time) :
 IC.trajectory S t = 0 ↔
 cos (S.ω * t.val - (AmplitudePhase.fromInitialConditions S IC).φ) = 0
rw [trajectory_eq_cos]
constructor
· intro h
 have hscalar :
 (AmplitudePhase.fromInitialConditions S IC).A *
 cos (S.ω * t.val - (AmplitudePhase.fromInitialConditions S IC).φ)
 simp using congrArg (fun x : EuclideanSpace ℝ (Fin 1) => x 0) h
 rcases mul_eq_zero.mp hscalar with hA0 | hcos
 · exact (hA hA0).elim
 · exact hcos
· intro h
 ext i
 fin_cases i
 simp [h]

/-- For nonzero amplitude, the trajectory passes through zero exactly at ph
 `φ + (2n + 1)π / 2`. -/
lemma trajectory_eq_zero_iff_exists_int (IC : InitialConditions)
 (hA : (AmplitudePhase.fromInitialConditions S IC).A ≠ 0) (t : Time) :
 IC.trajectory S t = 0 ↔
 ∃ n : ℤ,
 (t : ℝ) =
 ((AmplitudePhase.fromInitialConditions S IC).φ + (2 * n + 1) * π
rw [trajectory_eq_zero_iff_cos_eq_zero S IC hA t]
constructor
· intro h
 obtain ⟨n, hn⟩ := Real.cos_eq_zero_iff.mp h
 use n
 have hw : S.ω ≠ 0 := S.ω_ne_zero
 have ht :
 S.ω * t.val =
 (AmplitudePhase.fromInitialConditions S IC).φ + (2 * n + 1) * π /
 linarith
calc
 (t : ℝ) = (S.ω * t) / S.ω := by field_simp [hw]
 _ = ((AmplitudePhase.fromInitialConditions S IC).φ + (2 * n + 1) * π
 rw [ht]
· intro h

```

1.18.

PHYSLIB/CLASSICALMECHANICS/HARMONICOSCILLATOR/SOLUTION.18.EAN

```

obtain (n, hn) := h
rw [Real.cos_eq_zero_iff]
use n
have hw : S.ω ≠ 0 := S.ω_ne_zero
rw [hn]
field_simp [hw]
ring

```

/-!

## G. Periodicity and recurrence

Every trajectory is a shifted cosine of angular frequency  $\omega$ , so it repeats after a  $T = 2\pi / \omega$ . We record the period, show the trajectory is periodic, and prove that non-trivial initial data – the trajectory returns to its initial position and velocity at integer multiples of the period.

-/

/-!

### G.1. The period

The period  $T = 2\pi / \omega$  is the time for one complete oscillation; it is positive since  $\omega \neq 0$ .

-/

/-!

### G.2. Periodicity of the trajectory

The trajectory satisfies  $x(t + T) = x(t)$ : advancing time by one period shifts the phase by  $2\pi$ , leaving  $\cos$  and  $\sin$  unchanged.

-/

### G.3. Return to the initial state

For non-trivial initial data, the trajectory returns to its initial position and velocity at integer multiples of the period.

/--

Assuming that the initial coordinate and velocity are not simultaneously zero, the time stamps when the harmonic oscillator returns to its initial coordinate and velocity are integer multiples of its period

-/

```

lemma return_time (IC : InitialConditions) (non_trivial : IC.x₀ ≠ 0 ∨ IC.v₀ ≠ 0)
 (t : Time) (ht : IC.trajectory S t = IC.x₀ ∧ ∂ₜ (IC.trajectory S) t = IC.v₀) :

```

```

 ∃ n : ℤ, (n : ℝ) * (T S) = t := by
have htx := ht.left
have htv := ht.right
rw [InitialConditions.trajectory_eq] at htx
rw [InitialConditions.trajectory_velocity] at htv
simp at htx
simp at htv
set c := cos (S.ω * t)
set s := sin (S.ω * t)
set xx := inner ℝ IC.x₀ IC.x₀
set vv := inner ℝ IC.v₀ IC.v₀
set xv := inner ℝ IC.x₀ IC.v₀
set det := vv + xx * S.ω^2
have zero_lt_det : 0 < det := by
 cases non_trivial with
 | inl hx =>
 have xx_gt_zero : 0 < xx := by
 apply real_inner_self_pos.mpr
 exact hx
 calc
 0 < xx * S.ω^2 := by bound
 _ ≤ ||IC.v₀||^2 + xx * S.ω^2 := by bound
 _ = vv + xx * S.ω^2 := by rw [← real_inner_self_eq_norm_sq IC.v₀]
 _ = det := by rfl
 | inr hv =>
 have vv_gt_zero : 0 < vv := by
 apply real_inner_self_pos.mpr
 exact hv
 calc
 0 < vv := vv_gt_zero
 _ ≤ vv + ||IC.x₀||^2 * S.ω^2 := by bound
 _ = vv + xx * S.ω^2 := by rw [← real_inner_self_eq_norm_sq IC.x₀]
 _ = det := by rfl
have det_ne_zero : det ≠ 0 := by bound
have hxx : c * xx + (s / S.ω) * xv = xx := by
 calc
 c * xx + (s / S.ω) * xv = (inner ℝ (c • IC.x₀) IC.x₀) + (s / S.ω) * xv
 rw [real_inner_smul_left]
 (inner ℝ (c • IC.x₀) IC.x₀) + (s / S.ω) * xv =
 (inner ℝ (c • IC.x₀) IC.x₀) + (s / S.ω) * inner ℝ IC.v₀ IC.x₀ := by
 rw [real_inner_comm IC.x₀ IC.v₀]
 _ = (inner ℝ (c • IC.x₀) IC.x₀) + inner ℝ ((s / S.ω) • IC.v₀) IC.x₀
 rw [real_inner_smul_left IC.v₀]
 _ = (inner ℝ (c • IC.x₀ + (s / S.ω) • IC.v₀) IC.x₀) := by rw [inner_a]
 _ = xx := by rw [htx]
have hvv : - S.ω * s * xv + c * vv = vv := by

```

1.19.

PHYSLIB/CLASSICALMECHANICS/LAGRANGIAN/TOTALDERIVATIVEEQUIVALENCE.LEAN

```

calc
 - S.ω * s * xv + c * vv = - S.ω * (s * xv) + c * vv := by ring_nf
 _ = - S.ω * inner ℝ (s • IC.x₀) IC.v₀ + c * vv := by rw[real_inner_smul_left]
 _ = inner ℝ (- S.ω • s • IC.x₀) IC.v₀ + c * vv := by rw [← real_inner_smul_le
 _ = inner ℝ (- S.ω • s • IC.x₀) IC.v₀ + inner ℝ (c • IC.v₀) IC.v₀ := by
 rw [← real_inner_smul_left]
 _ = inner ℝ (- S.ω • s • IC.x₀ + c • IC.v₀) IC.v₀ := by rw [inner_add_left]
 _ = inner ℝ (-(S.ω • s • IC.x₀) + c • IC.v₀) IC.v₀ := by rw [neg_smul]
 _ = vv := by rw [htv]
have hcos : 1 = cos (S.ω * t) := by
 calc
 1 = det / det := by simp only [ne_eq, det_ne_zero, not_false_eq_true, div_self]
 _ = (vv + xx * S.ω^2) / det := by rfl
 _ = c * ((vv + xx * S.ω^2) / det) + s * xv * S.ω * (S.ω / S.ω -
1) / det := by
 nth_rewrite 1 [← hvv, ← hxx]
 ring_nf
 _ = c * ((vv + xx * S.ω^2) / det) := by
 simp only [ne_eq, S.ω_ne_zero, not_false_eq_true,
 div_self, sub_self, mul_zero, zero_div, add_zero]
 _ = c * (det / det) := by rfl
 _ = c := by simp only [ne_eq, det_ne_zero, not_false_eq_true, div_self, mul_one]
 _ = _ := by rfl
let ⟨n, hn⟩ := (Real.cos_eq_one_iff (S.ω * t)).mp (Eq.symm hcos)
use n
calc
 (n : ℝ) * (T S) = (n : ℝ) * (2 * π / S.ω) := by rfl
 _ = ((n : ℝ) * (2 * π)) / S.ω := by ring_nf
 _ = (S.ω * t) / S.ω := by rw [hn]
 _ = t * (S.ω / S.ω) := by ring_nf
 _ = t := by simp only [ne_eq, S.ω_ne_zero, not_false_eq_true, div_self, mul_one]

```

## 1.19 Physlib/ClassicalMechanics/Lagrangian/TotalDerivativeEquivalence.

```

public import Physlib.SpaceAndTime.Time.Derivatives
public import Mathlib.Analysis.Calculus.ContDiff.CPolynomial
public import Physlib.Mathematics.VariationalCalculus.HasVarGradient

```

A general function  $\delta L(t, q, d_t q)$  is a total time derivative if there exists a function  $F(t, q)$  (independent of velocity) such that:

$$\delta L(t, q, d_t q) = d/dt F(t, q) = fderiv ℝ F (t q) (v, 1)$$

$$\delta L(t, q, d_t q) = \partial F / \partial t + \langle \nabla_r F, d_t q \rangle$$

For the special case where  $\delta L$  depends only on velocity  $d_t q$  (not position or time),

$$\delta L(d_t q) = \langle g, d_t q \rangle \text{ for some constant vector } g$$

1.  $d/dt F(t, q) = \partial F / \partial t + \langle \nabla F, d_t q \rangle$

2. For  $\delta L$  to be  $q$ -independent,  $\nabla F$  must be  $q$ -independent

4. The result is  $\delta L = \langle g, d_t q \rangle$  for constant  $g$

- `IsTotalTimeDerivative`: General case for  $\delta L(t, q, d_t q)$

- `IsTotalTimeDerivativeVelocity`: Velocity-only case, equivalent to  $\delta L(d_t \text{ variable } \{X\})$  [NormedAddCommGroup  $X$ ] [InnerProductSpace  $\mathbb{R} X$ ]

```
open InnerProductSpace ContDiff Time ContinuousMultilinearMap
```

```
-- A function $\delta L(t, q, d_t q)$ is a total time derivative if it can be written
for some function F that depends on position and time but not velocity,
```

```
 $\delta L(t, q, d_t q) = (d/dt) F(t, q)$
```

```
def IsTotalTimeDerivative
```

```
 ($\delta L : \text{Time} \rightarrow X \rightarrow X \rightarrow \mathbb{R}$) : Prop :=
```

```
 $\exists (F : \text{Time} \rightarrow X \rightarrow \mathbb{R}) (_ : \text{ContDiff } \mathbb{R} \infty 1F),$
```

```
 $\forall t (q : \text{Time} \rightarrow X), (\text{ContDiff } \mathbb{R} \infty q) \rightarrow \delta L \ t \ (q \ t) \ (\partial_t \ q \ t) = \partial_t \ (fun \ t' \Rightarrow$
```

```
--
```

```
 Explicit reformulation (by the chain rule):
```

```
 $\delta L(t, q, d_t q) = \partial F / \partial t(t, q) + \langle \nabla_r F(t, q), d_t q \rangle$
```

```
or
```

```
 $\delta L(t, q, d_t q) = fderiv \ \mathbb{R} \ F \ (t, q) \ (1, d_t q)$
```

```
-/
```

```
lemma isTotalTimeDerivative_explicit { $\delta L : \text{Time} \rightarrow X \rightarrow X \rightarrow \mathbb{R}$ } :
```

```
 IsTotalTimeDerivative $\delta L \leftrightarrow (\exists (F : \text{Time} \rightarrow X \rightarrow \mathbb{R}) (_ : \text{ContDiff } \mathbb{R} \infty 1F)$
```

```
 $\forall t \ q \ v, \delta L \ t \ q \ v = fderiv \ \mathbb{R} \ 1F \ (t, q) \ ((1 : \text{Time}), v)) := by$
```

```
-- Preliminary construction: properties of the function $t \Rightarrow (t, q \ t)$
```

```
let tq := fun (q : Time → X) t => (t, q t)
```

```
have h_tq_contDiff : $\forall (q : \text{Time} \rightarrow X), \text{ContDiff } \mathbb{R} \infty q \rightarrow \text{ContDiff } \mathbb{R} \infty (tq \ q)$
```

```
 fun_prop
```

```
have h_tq_der : $\forall (q : \text{Time} \rightarrow X) \ t, \text{ContDiff } \mathbb{R} \infty q \rightarrow \partial_t \ (tq \ q) \ t = (1,$
```

```
 intro q t h_ContDiff_q
```

```
 ext
```

```
 change $(\partial_t \ (tq \ q) \ t).1.val = (1 : \text{Time}).val$
```

```
 congr
```

```
 apply Eq.symm
```

```
 calc
```

```
 $(1 : \text{Time}) = fderiv \ \mathbb{R} \ (fun \ (t' : \text{Time}) \Rightarrow t') \ t \ 1 := by \text{simp only [fderiv, ContinuousLinearMap.coe_id', id_eq]}$
```

```
 $= fderiv \ \mathbb{R} \ (fun \ (t' : \text{Time}) \Rightarrow (tq \ q \ t')).1 \ t \ 1 := by \text{rfl}$
```

```
 $= (\partial_t \ (tq \ q) \ t).1 := by$
```

```
 rw [fderiv.fst]
```

```
 · simp
```

```
 rfl
```

```
 · apply ContDiffAt.differentiableAt
```

```
 · apply ContDiff.contDiffAt
```

1.19.

PHYSLIB/CLASSICALMECHANICS/LAGRANGIAN/TOTALDERIVATIVEEQUIVALENCE.LEAN

```

 exact h_tq_contDiff q h_ContDiff_q
 · by_contra
 rcases this
 apply Eq.symm
 calc
 (1, $\partial_t q t$).2 = fderiv $\mathbb{R}$ (fun t' => (tq q t').2) t 1 := by rfl
 _ = (∂_t (tq q) t).2 := by
 rw [fderiv_snd]
 · simp only [ContinuousLinearMap.comp_apply, ContinuousLinearMap.coe_snd']
 rfl
 · apply ContDiffAt.differentiableAt
 · apply ContDiff.contDiffAt
 exact h_tq_contDiff q h_ContDiff_q
 · by_contra
 rcases this
 have h_F_tq_der : $\forall (q : \text{Time} \rightarrow X) (F : \text{Time} \rightarrow X \rightarrow \mathbb{R}) t, (\text{ContDiff } \mathbb{R} \infty \text{ } \text{!}F) \rightarrow (\text{ContDiff } \mathbb{R} \infty \text{ } \text{!}F (t', q t')) t = \text{fderiv } \mathbb{R} \text{ } \text{!}F (t, q t) ((1 : \text{Time}), \partial_t q t) :=$
 intro q F t hF hq
 change fderiv $\mathbb{R} ((\text{!}F) \circ (tq q)) t 1 = \text{fderiv } \mathbb{R} \text{ } \text{!}F (t, q t) ((1 : \text{Time}), \partial_t q t)$
 rw [fderiv_comp]
 · simp only [ContinuousLinearMap.comp_apply]
 rw [← Time.deriv_eq, h_tq_der]
 exact hq
 · apply ContDiffAt.differentiableAt
 · apply ContDiff.contDiffAt
 exact hF
 · by_contra
 rcases this
 · apply ContDiffAt.differentiableAt
 · apply ContDiff.contDiffAt
 exact h_tq_contDiff q hq
 · by_contra
 rcases this
-- beginning of the proof
constructor
-- From total the total derivative to the explicit form
· intro h
 rcases h with ⟨F, hF⟩
 rcases hF with ⟨hFdif, hFder⟩
 use F
 use hFdif
 intro t q₀ v
 let qv := fun (t' : Time) => (q₀ - t.val • v) + t'.val • v
 have h_qv_contDiff : ContDiff $\mathbb{R} \infty$ qv := by
 change ContDiff $\mathbb{R} \infty$ (((fun (tR : $\mathbb{R}$) => (q₀ - t.val • v) + tR • v)) • Time.toRe
 fun_prop

```

```

have h_qv_t : qv t = q_0 := by
 calc
 qv t = (q_0 - t.val • v) + t.val • v := by rfl
 _ = q_0 := by module
have h_qv_der : ∂t qv t = v := by
 calc
 ∂t qv t = fderiv ℝ (fun t' => (q_0 - t.val • v) + t'.val • v) t 1 :=
 _ = v := by
 rw [fderiv_const_add, fderiv_smul_const]
 · simp only [ContinuousLinearMap.smulRight_apply, fderiv_val, one]
 · fun_prop
rw [← h_qv_t, ← h_qv_der, hFder, ← h_F_tq_der]
· rfl
· exact hFdif
· exact h_qv_contDiff
· exact h_qv_contDiff
-- From the explicit form to the total derivative
· intro h
 rcases h with ⟨F, hF⟩
 rcases hF with ⟨hFdif, hFder⟩
 use F
 use hFdif
 intro t q hq_contDiff
 rw [hFder, ← h_F_tq_der]
 · rfl
 · exact hFdif
 · exact hq_contDiff

/--
Elementary fact: if δL is a time derivative, then so is -
δL.
-/
lemma isTotalTimeDerivative_neg {δL : Time → X → X → ℝ} (h : IsTotalTimeDerivative δL) :
 IsTotalTimeDerivative (- δL) := by
 rcases h with ⟨F, h_contDiff, hF⟩
 set F_neg := (fun t q => - F t q)
 use F_neg
 have h_neg_F_contDiff : ContDiff ℝ ∞ 1 F_neg := by
 fun_prop
 use h_neg_F_contDiff
 intro t q hq
 simp only [Pi.neg_apply]
 rw [hF t q hq]
 unfold F_neg
 unfold Time.deriv
 simp only [fderiv_fun_neg, _root_.neg_apply]

```



1.19.

PHYSLIB/CLASSICALMECHANICS/LAGRANGIAN/TOTALDERIVATIVEEQUIVALENCE.LEAN

```

/-!
B. Total time derivative do not affect the physical content
The total time derivative does not affect the Euler-Lagrange equations, because its
derivative is zero:
 $\int d/dt F(t, q) = F(t_1, q_1) - F(t_0, q_0)$
is fixed by the boundary conditions.
-/

```

```

/--
Total time derivative has a variational derivative, which is zero
-/
lemma totalTimeDerivative_hasZeroVarGradient [CompleteSpace X] {δL : Time → X → X → ℝ}
 (h : IsTotalTimeDerivative δL) (q : Time → X) (hq : ContDiff ℝ ∞ q):
 HasVarGradientAt (fun q' t => δL t (q' t) (∂ₜ q' t)) (fun _ => 0) q := by
 rcases h with ⟨F, hF_contDiff, hF⟩
 let traj_deriv := fun (G : Time → ℝ) t => fderiv ℝ G t 1
 let F_traj := fun (q : Time → X) t => F t (q t)
 apply HasVarGradientAt.intro _
 · apply HasVarAdjDerivAt.congr (F := fun q' => traj_deriv (F_traj q'))
 · apply HasVarAdjDerivAt.comp (F := traj_deriv) (G := F_traj)
 · apply HasVarAdjDerivAt.fderiv
 fun_prop
 · apply HasVarAdjDerivAt.fmap (f := fun t => F t)
 · exact hq
 · fun_prop
 · intro t x
 apply DifferentiableAt.hasAdjFDerivAt
 apply Differentiable.differentiableAt
 apply ContDiff.differentiable
 fun_prop
 decide
 · intro q' hq'
 funext t'
 rw [hF t' q' hq']
 rfl
 funext t
 unfold adjFDeriv
 simp [adjoint_eq_clm_adjoint]

/--
If two lagrangians, L and L', differ by a total time derivative, and L has a variational
grad, then so does L'.
-/
lemma totalTimeDerivative_hasVarGradientAt_equivalence [CompleteSpace X] (L δL : Time → X → X → ℝ)

```

```

(hδL : IsTotalTimeDerivative δL)
(q : Time → X) (hq : ContDiff ℝ ∞ q) (grad : Time → X)
(hgrad : HasVarGradientAt (fun q' t => L t (q' t) (fderiv ℝ q' t 1))
 HasVarGradientAt (fun q' t => (L + δL) t (q' t) (fderiv ℝ q' t 1)) grad)
have h_add_zero : grad = grad + (fun _ => 0) := by
 funext t
 simp
rw [h_add_zero]
apply HasVarGradientAt.add
· exact hgrad
· exact totalTimeDerivative_hasZeroVarGradient hδL q hq

/-
Reformulation of the previous result:
If two lagrangians, L and L', differ by a total time derivative, their vari
coincide (or neither of them has a variational derivative).
-/
lemma totalTimeDerivative_varGradient_equivalence [CompleteSpace X] (L L'
 (htot : IsTotalTimeDerivative (L' - L))
 (q : Time → X) (hq : ContDiff ℝ ∞ q):
 (δ (q':=q), ∫ t, L' t (q' t) (fderiv ℝ q' t 1)) =
 (δ (q':=q), ∫ t, L t (q' t) (fderiv ℝ q' t 1)) := by
let δL := (fun t q v => L' t q v - L t q v)
by_cases hL : ∃ grad, HasVarGradientAt (fun q' t => L t (q' t) (fderiv ℝ
· apply HasVarGradientAt.varGradient
 have h_triv : L' = L + (L' - L) := by module
 rw [h_triv]
 apply totalTimeDerivative_hasVarGradientAt_equivalence
 · exact htot
 · exact hq
 · rcases hL with ⟨grad, hgrad⟩
 rw [HasVarGradientAt.varGradient (fun q' t => L t (q' t) (fderiv ℝ
 exact hgrad
· by_cases hL' : ∃ grad, HasVarGradientAt (fun q' t => L' t (q' t) (fderi
· apply Eq.symm
 apply HasVarGradientAt.varGradient
 have h_triv : L = L' + (-(L' - L)) := by module
 rw [h_triv]
 apply totalTimeDerivative_hasVarGradientAt_equivalence
 · apply isTotalTimeDerivative_neg
 exact htot
 · exact hq
 · rcases hL' with ⟨grad, hgrad⟩
 rw [HasVarGradientAt.varGradient (fun q' t => L' t (q' t) (fderiv ℝ
 exact hgrad

```

1.19.

PHYSLIB/CLASSICALMECHANICS/LAGRANGIAN/TOTALDERIVATIVEEQUIVALENCE.LEAN

```

 · unfold varGradient
 simp only [hL, hL', ↓reduceDite]
C. Velocity-Only Total Time Derivative
 If δL depends only on velocity and equals $d/dt F(t, q)$ for some F ,
 then $\delta L(d_t q) = \langle g, d_t q \rangle$ for some constant vector g .
 - $d/dt F(t, q) = \partial F/\partial t + \langle \nabla F, d_t q \rangle = \partial F/\partial t + \langle \nabla F, d_t q \rangle$
 - For the result to be independent of q and t , we need $\nabla F = g$ (constant) and $\partial F/\partial t = 0$
 - Thus $\delta L(d_t q) = \langle g, d_t q \rangle$
lemma isTotalTimeDerivativeVelocity [CompleteSpace X]
 (δL : X → ℝ)
 (h : IsTotalTimeDerivative (fun _ _ v => δL v)) :
 ∃ g : X, ∀ v, δL v = ⟨g, v⟩_ℝ := by
 rcases (isTotalTimeDerivative_explicit.mp h) with ⟨F, hFdiff, hEq⟩
 let dF : (Time × X) → L[ℝ] ℝ :=
 fderiv ℝ 1F ((0 : Time), (0 : X))
 have h_time : dF ((1 : Time), (0 : X)) = 0 := by
 δL (0 : X) =
 fderiv ℝ 1F ((0 : Time), (0 : X))
 ((1 : Time), (0 : X)) := by
 simp using (hEq (0 : Time) (0 : X) (0 : X))
 have : dF ((1 : Time), (0 : X)) =
 δL (0 : X) := by
 -- Induced continuous linear functional on velocity: $v \mapsto dF(0, v)$.
 let φ : X → L[ℝ] ℝ :=
 dF.comp (ContinuousLinearMap.inr ℝ Time X)
 have hφ : ∀ v : X, δL v = φ v := by
 fderiv ℝ 1F ((0 : Time), (0 : X))
 ((1 : Time), v) := by
 simp using (hEq (0 : Time) (0 : X) v)
 have hv' : δL v = dF ((1 : Time), v) := by
 δL v = dF ((1 : Time), v) := hv'
 _ = dF (((0 : Time), v) + ((1 : Time), (0 : X))) := by simp only [Prod.mk_add,
 add_zero]
 _ = dF ((0 : Time), v) + dF ((1 : Time), (0 : X)) := by
 (dF.map_add ((0 : Time), v) ((1 : Time), (0 : X)))
 _ = dF ((0 : Time), v) := by
 refine ⟨(InnerProductSpace.toDual ℝ (X)).symm φ, ?_⟩
 ⟨(InnerProductSpace.toDual ℝ (X)).symm φ, v⟩_ℝ = φ v := by
 (E := X) (x := v) (y := φ)
 _ = ⟨(InnerProductSpace.toDual ℝ (X)).symm φ, v⟩_ℝ := by

```

## 1.20 Physlib/ClassicalMechanics/Mass/MassUnit.lean

```
simp [toReal]
· exact zero_le
rfl
simp only [one_div, NNReal.coe_inv]
rw [toReal, inv_div]
 (x / y : ℝ) * (y / z : ℝ) = x / z := by
simp [div_eq_val, toReal]
rw [toReal]
simp [pounds, ounces]; rw [toReal]; norm_num
simp [shortTons, pounds]; rw [toReal]; norm_num
simp [longTons, pounds]; rw [toReal]; norm_num
```

## 1.21 Physlib/ClassicalMechanics/OrbitalMechanics/VisViva.lean

```
public import Mathlib.Analysis.Real.Sqrt
```

## 1.22 Physlib/ClassicalMechanics/RigidBody/API-map.yaml

version: v0.1

Title: Rigid body

Overview: |

The key data structure is `RigidBody d``, a rigid body in `d`` dimensions with its mass distribution (a linear map from smooth functions on `Space d`` provides the body's total mass, centre of mass and inertia tensor, and solid sphere as a concrete example.

ParentAPIs:

- Space (Physlib/SpaceAndTime/Space)

References:

- Landau and Lifshitz, Mechanics, Section 32 (page 100)

Requirements:

- description: >

The key data structure `RigidBody``, a rigid body specified by its mass distribution, is defined.

done: true

location: Physlib/ClassicalMechanics/RigidBody/Basic.lean (RigidBody)

- description: The API allows the calculation of the total mass of the rigid body.  
done: true  
location: Physlib/ClassicalMechanics/RigidBody/Basic.lean (RigidBody.mass)
- description: The API allows the calculation of the centre of mass of the rigid body.  
done: true  
location: Physlib/ClassicalMechanics/RigidBody/Basic.lean (RigidBody.centerOfMass)
- description: >  
The API allows the calculation of the moment of inertia (inertia tensor) of the rigid body.  
done: true  
location: Physlib/ClassicalMechanics/RigidBody/Basic.lean (RigidBody.inertiaTensor)
- description: The inertia tensor of the rigid body is symmetric.  
done: true  
location: Physlib/ClassicalMechanics/RigidBody/Basic.lean (RigidBody.inertiaTensor)
- description: >  
The API provides the solid sphere as a worked example, with its total mass and centre of mass computed.  
done: true  
location: >  
Physlib/ClassicalMechanics/RigidBody/SolidSphere.lean  
(RigidBody.solidSphere, RigidBody.solidSphere\_mass, RigidBody.solidSphere\_centerOfMass)
- description: >  
The inertia tensor of the solid sphere equals  $\frac{2}{5} m R^2$  times the identity.  
'solidSphere\_inertiaTensor' but currently proved with 'sorry'.  
done: false  
location: N/A
- description: The API contains the definition of the angular velocity of rotation.  
done: false  
location: N/A
- description: The API contains the definition of the velocity of the centre of mass.  
done: false  
location: N/A
- description: The API contains the definition of the kinetic energy of the rigid body.  
done: false  
location: N/A
- description: The API contains the definition of Euler angles for a rigid body.

```

done: false
location: N/A

- description: >
 Redefine `RigidBody` using continuous linear maps instead of plain li
 the space of smooth functions to $\mathbb{R}$ (existing TODO).
done: false
location: N/A

```

### 1.23 Physlib/ClassicalMechanics/RigidBody/SolidSphere.lean

```

lemma solidSphere_mass {d : ℕ} (m R : ℝ≥0) (hr : R ≠ 0) : (solidSphere d m
 simp only [LinearMap.coe_mk, AddHom.coe_mk, ContMDiffMap.coeFn_mk, integr
 MeasurableSet.univ, measureReal_restrict_apply, Set.univ_inter, smul_eq
 have h1 : (@volume (Space d) measureSpaceOfInnerProductSpace).real
 · apply measure_closedBall_lt_top.ne
 · apply (Metric.measure_closedBall_pos volume _).ne'
lemma solidSphere_centerOfMass {d : ℕ} (m R : ℝ≥0) : (solidSphere d m R).ce
 simp only [centerOfMass, solidSphere, one_div, LinearMap.coe_mk, AddHom.c
 ContMDiffMap.coeFn_mk, smul_eq_mul, Space.zero_apply, mul_eq_zero, inv_
 coe_eq_zero]
 suffices ∫ x in Metric.closedBall (0 : Space d) R, x i ∂MeasureSpace.volu
 = -∫ x in Metric.closedBall (0 : Space d) R, x i ∂MeasureSpace.volume b

```

### 1.24 Physlib/ClassicalMechanics/WaveEquation/Basic.lean

```

c^2 • Δ_v (f t) x - ∂_t (fun t => (∂_t (fun t => f t x) t)) t = 0
apply Differentiable.comp <=> fun_prop
 RingHom.id_apply, _root_.neg_apply,
 FunLike.coe_smul, Pi.smul_apply, PiLp.neg_apply, PiLp.smul_apply, smul_
simp only [fderiv_eq_smul_deriv, one_smul, neg_smul, _root_.neg_apply,
 FunLike.coe_smul, Pi.smul_apply, PiLp.neg_apply, PiLp.smul_apply, smul_
 ∂[i] (planeWave f_0 c s t) =
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply, fderiv_eq_s
simp only [fderiv_fun_const, Pi.zero_apply, _root_.zero_apply, inner_zero
 fderiv_fun_id, ContinuousLinearMap.coe_id', id_eq, basis_inner, zero_ad
 ∂[i] (fun x => planeWave f_0 c s t x j) =
simp only [ContinuousLinearMap.fderiv, ContinuousLinearMap.coe_comp, Func
 ∂[i] (∂[i] (planeWave f_0 c s t)) =
simp only [fderiv_eq_smul_deriv, one_smul, FunLike.coe_smul, Pi.smul_appl
 ∂[i] (∂[i] (fun x => planeWave f_0 c s t x j)) =
 (s.unit i) ^ 2 • fun x => planeWave (iteratedDeriv 2 f_0 ·) c s t x j :=
simp only [fderiv_eq_smul_deriv, one_smul, FunLike.coe_smul, Pi.smul_appl

```

### 1.25. PHYSLIB/CONDENSEDMATTER/TIGHTBINDINGCHAIN/BASIC.LEAN

```

 Δv (planeWave f0 c s t) = fun x => planeWave (iteratedDeriv 2 f0 ·) c s t x := b
rw [laplacianVec]
enter [1, 2, i]
rw [laplacian_eq_sum_snd_deriv]
enter [0, x, 2, i]
simp only [fderiv_fun_const, Pi.ofNat_apply, sub_zero, ContinuousLinearMap.coe_c
simp only [fderiv_fun_id, ContinuousLinearMap.coe_id', id_eq]
simp only [fderiv_fun_const, Pi.ofNat_apply, _root_.zero_apply, inner_zero_right
root.zero_apply, inner_zero_right, PiLp.smul_apply, smul_eq_mul,
simp only [ContinuousLinearMap.fderiv, ContinuousLinearMap.coe_comp, Function.comp
simp only [fderiv_fun_const, Pi.zero_apply, sub_zero, ContinuousLinearMap.coe_comp
root.zero_apply, inner_zero_right, fderiv_fun_id, ContinuousLinearMap.coe_id',

```

### 1.25 Physlib/CondensedMatter/TightBindingChain/Basic.lean

```

abbrev HilbertSpace := QuantumMechanics.FiniteHilbertSpace (Fin T.N)
QuantumMechanics.FiniteHilbertSpace.basisFun (Fin T.N)
rintro _ (n, rfl)
have hT := T.a_pos
have hNpos : 0 < T.N := lt_of_le_of_lt (Nat.zero_le _) n.isLt
simp only [BrillouinZone, Set.mem_Ico]
generalize T.N = x at *
generalize T.a = a at *
have hx : (0 : ℝ) < x := by exact_mod_cast hNpos
have hn : (n : ℝ) + 1 ≤ x := by exact_mod_cast n.isLt
have hn0 : (0 : ℝ) ≤ n := by positivity
have hx2 : 2 * ((x / 2 : ℕ) : ℝ) ≤ x := by exact_mod_cast (by omega : 2 * (x / 2)
have hx2' : (x : ℝ) ≤ 2 * ((x / 2 : ℕ) : ℝ) + 1 := by
 exact_mod_cast (by omega : x ≤ 2 * (x / 2) + 1)
refine {?, ?_}
· apply le_of_eq_of_le (by ring : _ = Real.pi / a * (-1 : ℝ))
 apply le_of_le_of_eq (b := Real.pi / a * (2 * ((n : ℝ) - (x / 2 : ℕ)) / x))
 · rw [le_div_iff₀ hx]; linarith [hx2, hn0]
· apply lt_of_lt_of_eq (b := Real.pi / a * (1 : ℝ))
 apply lt_of_eq_of_lt (b := Real.pi / a * (2 * ((n : ℝ) - (x / 2 : ℕ)) / x))
 · rw [div_lt_one hx]; linarith [hn, hx2']

```

### 1.26 Physlib/Cosmology/FLRW/Basic.lean

```

public import Physlib.Meta.TODO.Basic
public import Mathlib.Analysis.SpecialFunctions.Trigonometric.Deriv
TODO "Replace the placeholder `FLRW` type with a concrete structure bundling a posit
scale factor `a : Time → ℝ` (smooth, or at least twice differentiable) together wi

```

element of ``SpatialGeometry``."

TODO "Define the Hubble constant  $H_0 = H(t_0)$ , the present-day value of the Hubble parameter

``hubbleConstant``, and the present Hubble radius  $R_H = c / H_0$ . Keep ``par`` time-dependent  $H(t)$  distinct from ``constant`` (the number  $H_0$ )."

TODO "Express the Hubble parameter as a function of the scale factor,  $H(a)$  redshift,  $H(z)$ , using  $a = 1 / (1 + z)$  with the normalization  $a_0 = 1$  reduced Hubble function  $E(z) = H(z) / H_0$ ."

TODO "Prove the change-of-variable relations underlying the age and distance  $\partial_t a = a * \text{hubbleConstant } a$ ,  $dt = -dz / ((1 + z) * H)$  and  $d\chi = c *$

## 1.27 Physlib/Cosmology/FLRW/ConformalTime.lean

/-

Copyright (c) 2026 Jinzheng Li. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Jinzheng Li

-/

module

public import Physlib.Meta.TODO.Basic

/-!

# Conformal time in FLRW cosmology

Placeholder file collecting TODO items for the conformal time and the conformal Hubble factor of a Friedmann-Lemaître-Robertson-Walker model. This file contains only TODO items; no definitions or lemmas are formalized yet.

-/

@[expose] public section

TODO "Define the conformal time  $\eta$  of an FLRW model by  $a d\eta = dt$ , that is  $\eta(t) = \int dt' / a(t')$ ."

TODO "Define the conformal Hubble factor  $\dot{a} = a' / a$  (the prime denotes the derivative with respect to conformal time) and prove  $\dot{a} = a * H$ , relating to the Hubble parameter  $H = \partial_t a / a$ ."



TODO "State the FLRW metric in conformal time as  $a(\eta)^2$  times a static metric, making its conformal flatness manifest."

TODO "State the Friedmann and acceleration equations in conformal time:  
 $\dot{a}^2 = (8\pi G/3) \rho a^2 + (\Lambda c^2/3) a^2 - K c^2$  and  
 $\ddot{a}/a = (4\pi G/3)(\rho - 3P/c^2) a^2 + (2\Lambda c^2/3) a^2 - K c^2$ ."

TODO "Prove the cosmic-to-conformal change of variables  $f' = a * \partial_t f$  (from  $dt =$  and use it to derive the conformal-time Friedmann and acceleration equations from cosmic-time forms."

## 1.28 Physlib/Cosmology/FLRW/DensityParameters.lean

```

/-
Copyright (c) 2026 Jinzheng Li. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Jinzheng Li
-/
module

public import Physlib.Meta.TODO.Basic
/-!

Density parameters and the standard cosmological model

Placeholder file collecting TODO items for the critical density, the density
parameters, the closure relation, and the Friedmann equation of the standard
cosmological model written in terms of the reduced Hubble function. This file
contains only TODO items; no definitions or lemmas are formalized yet.

-/

@[expose] public section

TODO "Define the critical density $\rho_{cr} = 3 H^2 / (8 \pi G)$."

TODO "Define the density parameter $\Omega = \rho / \rho_{cr}$ and the curvature density
parameter $\Omega_K = -K c^2 / (H^2 a^2)$."

TODO "Prove the closure relation $\Omega + \Omega_K = 1$ as a rearrangement of the
Friedmann equation."

TODO "Define the reduced Hubble function $E(a) = H / H_0$ and the normalized
Friedmann equation $E(a)^2 = \sum_x \Omega_{\{x\}} f_x(a) + \Omega_{\{K\}} / a^2$."

```

TODO "Define the Friedmann equation of the standard cosmological model  
 $E(a)^2 = \Omega_\Lambda + \Omega_{m0} a^{-3} + \Omega_{r0} a^{-4} + \Omega_{K0} a^{-2}$ ."

TODO "Define the age of the universe  $t_0 = H_0^{-1} \int_0^1 da / (a E(a))$  and prove  
 $t_0 \propto 1 / H_0$ ."

TODO "Define the radiation-matter and matter- $\Lambda$  equality scale factors  
 $a_{eq} = \Omega_{r0} / \Omega_{m0}$  and  $a_\Lambda = (\Omega_{m0} / \Omega_\Lambda)^{1/3}$ ."

## 1.29 Physlib/Cosmology/FLRW/Distances.lean

/-

Copyright (c) 2026 Jinzheng Li. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Jinzheng Li

-/

module

public import Physlib.Meta.TODO.Basic

/-!

# Distances and redshift in FLRW cosmology

Placeholder file collecting TODO items for the notions of distance used in cosmology (comoving, proper, and the observational luminosity and angular-diameter distances), together with the redshift, the horizons, and lookback time. This file contains only TODO items; no definitions or lemmas formalized yet.

-/

@[expose] public section

TODO "Define the line-of-sight comoving distance  $\chi = c \int dt/a$ , equivalent  
 $\chi(z) = \int_0^z c dz' / H(z')$ ."

TODO "Define the proper distance  $D = a * \chi$  and prove the Hubble-Lemaître law

$\partial_t D = H * D$  at fixed comoving distance  $\chi$ ."

TODO "Define the Hubble radius  $R_H = c / H$ , the distance at which the recession velocity equals the speed of light."

```

TODO "Relate the transverse comoving distance $r(\chi)$ to the existing
 $\text{Cosmology.SpatialGeometry.S}$, with curvature radius $k = 1/\sqrt{|K|}$."

TODO "Define the redshift by $1 + z = a_0 / a$ and prove the cosmological redshift
 law $E \propto 1/a$ for a photon from the null geodesic equation."

TODO "Define the particle (comoving) horizon $\chi_p = c \int_0^t dt'/a$ and the event
 horizon $\chi_e = c \int_t^\infty dt'/a$."

TODO "Define the lookback time $t_0 - t$ and prove its model-
 independent expansion
 $t_0 - t = H_0^{-1} [z - \frac{1}{2}(2 + q_0) z^2 + \dots]$ in powers of the redshift."

TODO "Define the luminosity distance $d_L = (1 + z) r(\chi)$ and the angular-
 diameter
 distance $d_A = r(\chi) / (1 + z)$."

TODO "Prove Etherington's distance-duality relation $d_L = (1 + z)^2 d_A$."

TODO "Prove the low-redshift expansions of χ , d_L and d_A in powers of z
 in terms of the deceleration parameter q_0 ."

```

### 1.30 Physlib/Cosmology/FLRW/Dynamics.lean

```

/-
Copyright (c) 2026 Jinzheng Li. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Jinzheng Li
-/
module

public import Physlib.Meta.TODO.Basic
/-!

Dynamical criteria for FLRW cosmology

Placeholder file collecting TODO items for the qualitative consequences of the
Friedmann and acceleration equations: the energy conditions of the cosmic fluid,
the criterion for accelerated expansion, the deceleration parameter in terms of the
density parameters, the condition for eternal expansion, and the existence of a
Big-Bang singularity. This file contains only TODO items; no definitions or lemmas
are formalized yet.

-/

```

@[expose] public section

TODO "Define the energy conditions for the cosmic perfect fluid: the null weak  $\rho \geq 0 \wedge \rho + P/c^2 \geq 0$ , strong  $\rho + P/c^2 \geq 0 \wedge \rho + 3 P/c^2 \geq 0$ , and  $\rho \geq 0 \wedge |P| \leq \rho c^2$  energy conditions."

TODO "Prove that the expansion accelerates iff the cosmic fluid (with the constant folded in as a  $w = -1$  component) violates the strong energy condition  $\partial_t \partial_t a > 0 \leftrightarrow \rho_{\text{tot}} + 3 P_{\text{tot}} / c^2 < 0$ ."

TODO "Prove the deceleration parameter in terms of the density parameters at a state, for a spatially flat universe:  $q = \frac{1}{2} \sum_x \Omega_x (1 + 3 w_x)$ ; in particular  $q_0 = \frac{1}{2} \sum_x \Omega_{\{x0\}} (1 + 3 w_x)$ , giving  $q_0 = \frac{1}{2} (\Omega_{\{m0\}} + 2 \Omega_{\{r0\}} - 2 \Omega_{\{\Lambda0\}})$ ."

TODO "Prove that a universe with  $K \leq 0$ , nonnegative density and  $\Lambda \geq 0$  ensures the Friedmann right-hand side stays positive, so  $\partial_t a$  never vanishes, and  $\partial_t a > 0$  at one instant implies  $\partial_t a > 0$  at all later times."

TODO "Prove the existence of a Big-Bang singularity: a currently expanding, matter-dominated universe ( $\rho > 0$ ,  $P \geq 0$ ,  $\Lambda = 0$ ) reaches  $a = 0$  at a finite cosmic time  $t_0$  giving a finite age  $t_0 < 1 / H_0$ ; contrast the de Sitter case, which has no singularity."

### 1.31 Physlib/Cosmology/FLRW/MatterContent.lean

/-

Copyright (c) 2026 Jinzheng Li. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Jinzheng Li

-/

module

public import Physlib.Meta.TODO.Basic

/-!

# Matter content of FLRW cosmology

Placeholder file collecting TODO items for the matter content of a Friedmann-Lemaître-Robertson-Walker universe: the perfect-fluid stress-energy

tensor, the equation of state, the continuity equation, and the resulting density scaling for the standard components (dust, radiation, vacuum energy). This file contains only TODO items; no definitions or lemmas are formalized.

```

-/

@[expose] public section

TODO "Define the perfect-fluid stress-energy tensor
 $T_{\mu\nu} = (\rho + P/c^2) u_\mu u_\nu + P g_{\mu\nu}$ for the FLRW metric."

TODO "Define the continuity equation $\partial_t \rho + 3 H (\rho + P/c^2) = 0$ and derive it from
 the first- and second-order Friedmann equations."

TODO "Prove that the Friedmann equation, the acceleration equation and the
 continuity equation are not independent (a consequence of the Bianchi identity
 $\nabla_\nu G^{\nu}{}_\nu = 0$)."P = w \rho c^2 and prove the
 density scaling law $\rho = \rho_0 a^{-3(1+w)}$ for constant w ."

TODO "Specialize the density scaling law to dust ($w = 0$, $\rho \propto a^{-3}$), radiation
 ($w = 1/3$, $\rho \propto a^{-4}$) and vacuum energy ($w = -1$, ρ constant)."w = -1 perfect fluid with
 $\rho_\Lambda = \Lambda c^2 / (8 \pi G)$ and $P_\Lambda = -\rho_\Lambda c^2$."

```

### 1.32 Physlib/Cosmology/FLRW/Solutions.lean

```

/-
Copyright (c) 2026 Jinzheng Li. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Jinzheng Li
-/
module

public import Physlib.Meta.TODO.Basic
/-!

Exact solutions of the Friedmann equations

Placeholder file collecting TODO items for the standard closed-
form solutions of
the Friedmann equations: the de Sitter, radiation-dominated, Einstein-
de Sitter
(dust) and Milne models, together with the Einstein static universe. This file
contains only TODO items; no definitions or lemmas are formalized yet.

```

-/

@[expose] public section

TODO "Prove that the de Sitter solution  $a(t) = a_0 \exp(\pm \sqrt{\Lambda/3} \, c \, t)$  (with  $K = 0$ ,  $\Lambda > 0$ ) solves the Friedmann equations, and that its deceleration parameter is  $q = -1$ ."

TODO "Prove that the spatially flat radiation-dominated solution  $a = (t/t_0)$  solves the Friedmann equations, with  $q_0 = 1$  and  $t_0 = 1 / (2 H_0)$ ."

TODO "Prove that the Einstein-de Sitter (spatially flat, dust) solution  $a = (t/t_0)^{2/3}$  solves the Friedmann equations, with  $q_0 = 1/2$  and  $t_0 = 2 / (3 H_0)$ ."

TODO "Prove that the Milne solution  $a = c \, t$  (empty universe,  $K < 0$ ) has vanishing scalar curvature, i.e. it is Minkowski space in expanding coordinates."

TODO "Define the Einstein static universe ( $\partial_t a = 0$ , forcing  $K = 0$  and  $\rho_m = 2 \rho_\Lambda$ ) and prove that it is an unstable equilibrium."

### 1.33 Physlib/Electromagnetism/Charge/ChargeUnit.lean

```
simp [toReal]
· exact zero_le
rfl
simp only [one_div, NNReal.coe_inv]
rw [toReal, inv_div]
(x / y : ℝ) * (y / z : ℝ) = x / z := by
simp [div_eq_val, toReal]
rw [toReal]
```

### 1.34 Physlib/Electromagnetism/Current/InfiniteWire.lean

```
public import Physlib.SpaceAndTime.Space.Norm.Basic
((IsDistBounded.log_norm (by norm_num)).smul_const _)
 (by apply (IsDistBounded.log_norm (by norm_num)).smul_const))) := by
distOfFunction_eculid_eval, eq_comm]
simp [infiniteWire_vectorPotential _ I, constantTime_distSpaceDeriv,
 distDeriv_constantSliceDist_same]
root.zero_apply, one_div, wireCurrentDensity_chargeDensity, mul_zero,
rw [← _root_.add_apply, ← map_add constantTime]
```

```

 congr 1 <=>
 simp [distDeriv_apply, fderivD_apply, distGrad_apply]
 simp using distDiv_inv_pow_eq_dim (d := 2)
 all_goals
 simp only [Fin.mk_one, Fin.reduceFinMk, Fin.isValue, neg_sub, Finset.sum_sub_dis
 Fin.sum_univ_three, infiniteWire_vectorPotential_distSpaceDeriv_0, map_zero,
 root.zero_apply, PiLp.zero_apply, zero_add, add_sub_add_right_eq_sub,
 wireCurrentDensity_currentDensity_snd, wireCurrentDensity_currentDensity_thrd,

```

### 1.35 Physlib/Electromagnetism/Distributional/Basic.lean

```

public import Physlib.SpaceAndTime.Space.DistOfFunction
 FunLike.coe_smul, ContinuousLinearMap.coe_comp, Pi.smul_apply,
/-- The creation of an electromagnetic potential from a static scalar-
potential function.
If the function is distribution-bounded, it is first promoted to a distribution using
`Space.distOfFunction`. Otherwise the constructor returns zero. -
/
noncomputable def ofStaticScalarPotentialFunction {d} (c : SpeedOfLight)
 (φ : Space d → ℝ) : DistElectromagneticPotential d := by
 classical
 exact if hφ : Space.IsDistBounded φ then
 ofStaticScalarPotential c (Space.distOfFunction φ hφ) else 0

@[simp]
lemma ofStaticScalarPotentialFunction_of_isDistBounded {d} (c : SpeedOfLight)
 (φ : Space d → ℝ) (hφ : Space.IsDistBounded φ) :
 ofStaticScalarPotentialFunction c φ =
 ofStaticScalarPotential c (Space.distOfFunction φ hφ) := by
 classical
 simp [ofStaticScalarPotentialFunction, hφ]

@[simp]
lemma ofStaticScalarPotentialFunction_eq_zero_of_not_isDistBounded {d}
 (c : SpeedOfLight) (φ : Space d → ℝ) (hφ : ¬ Space.IsDistBounded φ) :
 ofStaticScalarPotentialFunction c φ = 0 := by
 classical
 simp [ofStaticScalarPotentialFunction, hφ]

/-- The creation of an electromagnetic potential from a vector potential. -
/
noncomputable def ofVectorPotential {d} (c : SpeedOfLight) :
 ((Time × Space d) →d ℝ EuclideanSpace ℝ (Fin d)) →1 ℝ
 DistElectromagneticPotential d where

```

```

toFun A := (distTimeSlice c).symm (Lorentz.Vector.ofSpatialComponent ∘ L A)
map_add' A1 A2 := by
 ext ε
 simp
map_smul' r A := by
 ext ε
 simp

/-- The creation of an electromagnetic potential from a static vector poten
/
noncomputable def ofStaticVectorPotential {d} (c : SpeedOfLight) :
 ((Space d) → d[ℝ] EuclideanSpace ℝ (Fin d)) →1 [ℝ] DistElectromagneticPot
ofVectorPotential c ∘1 Space.constantTime
lemma distTensorDeriv_eq_sum_sum {d} (A : DistElectromagneticPotential d)
 distTensorDeriv A ε = ∑ μ, ∑ ν, (SpaceTime.distDeriv μ A ε ν) •
 simp [distTensorDeriv_apply]
lemma distTensorDeriv_basis_repr_apply {d} {μν : (Fin 1 ⊕ Fin d) × (Fin 1 ⊕
 (Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis).repr (distT
 rw [distTensorDeriv_eq_sum_sum]
 simp only [map_sum, map_smul, Finsupp.coe_finsetSum, Finsupp.coe_smul, Fi
lemma toTensor_distTensorDeriv_basis_repr_apply {d} (A : DistElectromagneti
 (Tensor.basis _).repr (Tensorial.toTensor (distTensorDeriv A ε)) b =
 rw [Module.Basis.repr_reindex_apply, distTensorDeriv_basis_repr_apply]

```

### 1.36 Physlib/Electromagnetism/Distributional/Dynamics/IsExtrema

```

rw [isExtrema_iff_gradLagrangian]
refine (fun h => by simp [h], fun h => ?_)
ext1 ε
funext i
match i with
| Sum.inl 0 => exact h.1 ε
| Sum.inr j => exact h.2 ε j
have hsurj : Function.Surjective (SchwartzMap.compCLM0fContinuousLinearEq
 (SpaceTime.toTimeAndSpace □.c (d := d)).symm) := by
 intro f
 refine (SchwartzMap.compCLM0fContinuousLinearEquiv ℝ
 (SpaceTime.toTimeAndSpace □.c (d := d)).symm.symm f, ?_)
 ext x
 simp
have hgauss : ∀ a b : ℝ,
 1 / (□.μ0 * □.c.val) * a - □.c.val * b = 0 ↔ a = 1 / □.ε0 * b := by
 intro a b
 have c_sq_mul_eq : □.c.val * b * (□.μ0 * □.c.val) = 1 / □.ε0 * b := by

```



1.37.

PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/DYNAMICS/KINETIC6TERM.LEAN

```

 have hcb : $\square.c.val * b * (\square.\mu_0 * \square.c.val) = \square.\mu_0 * \square.c.val^2 * b :=$ by ring
 rw [hcb, $\square.c_sq$]
 rw [sub_eq_zero, div_mul_eq_mul_div, one_mul,
 div_eq_iff (mul_ne_zero $\square.\mu_0_ne_zero \square.c.val_ne_zero$), c_sq_mul_eq]
 have hampere : $\forall T S C : \mathbb{R}, \square.\mu_0^{-1} * (1 / \square.c.val^2 * T - S) + C = 0 \leftrightarrow$
 $\square.\mu_0 * \square.\epsilon_0 * T - S + \square.\mu_0 * C = 0 :=$ by
 intro T S C
 have mu0_factor_eq : $\square.\mu_0 * \square.\epsilon_0 * T - S + \square.\mu_0 * C$
 $= \square.\mu_0 * (\square.\mu_0^{-1} * (1 / \square.c.val^2 * T - S) + C) :=$ by
 rw [$\square.c_sq$]
 rw [mu0_factor_eq, mul_eq_zero, or_iff_right $\square.\mu_0_ne_zero$]
 refine and_congr ?_ ?_
 · simp only [gradLagrangian_sum_inl_0, SpaceTime.distTimeSlice_symm_apply]
 rw [hsurj.forall]
 exact forall_congr' fun $\epsilon \Rightarrow$ hgauss _ _
 · simp only [gradLagrangian_sum_inr_i, SpaceTime.distTimeSlice_symm_apply]
 rw [hsurj.forall]
 exact forall_congr' fun $\epsilon \Rightarrow$ forall_congr' fun i $\Rightarrow$ hampere _ _ _
 (permT id (IsReindexing.auto) $\{((1 / \square.\mu_0 : \mathbb{R}) \cdot$
 distTensorDeriv A.fieldStrength x | $\kappa \kappa v') + - (J x \mid v')\}^T) = 0 :=$ by
 ext1 x
 one_apply_eq_self, smul_add, actionT_smul, smul_neg] using h (schwartzAction A

```

### 1.37 Physlib/Electromagnetism/Distributional/Dynamics/KineticTerm.lean

```

 simp only [$\leftarrow$ Finset.sum_add_distrib]
 refine Finset.sum_congr rfl fun v _ $\Rightarrow$ Finset.sum_congr rfl fun μ _ $\Rightarrow$?_
 refine Finset.sum_congr rfl fun v _ $\Rightarrow$ Finset.sum_congr rfl fun μ _ $\Rightarrow$?_
 ext1 ϵ
 simp only [one_div, map_add, add_apply, Lorentz.Vector.apply_add,
 ContinuousLinearMap.coe_mk', LinearMap.coe_mk, AddHom.coe_mk, $\leftarrow$ Finset.sum_add]
 refine Finset.sum_congr rfl fun v _ $\Rightarrow$ Finset.sum_congr rfl fun μ _ $\Rightarrow$?_
 ext1 ϵ
 simp only [one_div, map_smul, _root_.smul_apply, Lorentz.Vector.apply_smul,
 refine Finset.sum_congr rfl fun v _ $\Rightarrow$ Finset.sum_congr rfl fun μ _ $\Rightarrow$?_
 (permT id (IsReindexing.auto) $\{(1 / \square.\mu_0 : \mathbb{R}) \cdot$
 (permT id (IsReindexing.auto) $\{(1 / \square.\mu_0 : \mathbb{R}) \cdot$ distTensorDeriv A.fieldStrength ϵ
 refine Finset.sum_congr rfl fun μ _ $\Rightarrow$?_
 rw [Tensorial.basis_toTensor_apply, Tensorial.basis_map_prod]
 ext {i, j}
 rw [hb, Module.Basis.repr_reindex_apply]
 fin_cases x <>>
 simp [Function.comp_apply, ComponentIdx.prod, ComponentIdx.DropPairSection.ofFin
 ComponentIdx.DropPairSection.ofFin]

```

- intro \_ h
- exact absurd (by decide) h
- split\_ifs with h1 h2
  - exact absurd h1 (by decide)
  - exact absurd h2 (by decide)

### 1.38 Physlib/Electromagnetism/Distributional/Dynamics/Lagrangia

```

ext1 ε
ext1 ε
FunLike.coe_smul, Pi.smul_apply, distTimeSlice_symm_apply,
ContinuousLinearMap.coe_comp, LinearMap.coe_toContinuousLinearMap', Fun
ContinuousLinearMap.coe_comp, Function.comp_apply]
(permT id (IsReindexing.auto) {J ε | v'}T)) v:= by
(permT id (IsReindexing.auto) {J ε | v'}T)) v
(permT id (IsReindexing.auto) {(1/ □.μ₀ : ℝ) • (distTensorDeriv A.fiel
simp only [FunLike.coe_sub, Pi.sub_apply, apply_sub, one_div,

```

### 1.39 Physlib/Electromagnetism/Distributional/ElectricField.lean

```

simp only [map_add, neg_add_rev, FunLike.coe_sub, Pi.sub_apply,
add_apply, _root_.neg_apply, PiLp.sub_apply, PiLp.add_apply,
simp only [map_smul, FunLike.coe_sub, FunLike.coe_smul, Pi.sub_apply,
root.neg_apply, Pi.smul_apply, PiLp.sub_apply, PiLp.neg_apply, PiLp
vectorPotential, Vector.spatialCLM, Space.distTimeDeriv_apply_CLM, FunL
ContinuousLinearMap.coe_comp, ContinuousLinearMap.coe_mk', Pi.sub_apply
root.neg_apply, FunLike.coe_smul, Pi.smul_apply,

```

### 1.40 Physlib/Electromagnetism/Distributional/FieldStrength.lean

```

public import Mathlib.Algebra.Order.Archimedean.Real.Hom
(permT id (IsReindexing.auto) {(η d | μ μ' ⊗ distTensorDeriv A ε | μ'
(η d | v v' ⊗ distTensorDeriv A ε | v' μ)}T)
Tensorial.toTensor.symm
(permT id (IsReindexing.auto) {(η d | μ μ' ⊗ distTensorDeriv A ε | μ'
- Tensorial.toTensor.symm (permT ![1, 0] (IsReindexing.auto)
{(η d | μ μ' ⊗ distTensorDeriv A ε | μ' v)}T) := by
(permT id (IsReindexing.auto) {(η d | μ μ' ⊗ distTensorDeriv A ε | μ' v
- (permT ![1, 0] (IsReindexing.auto)
{(η d | μ μ' ⊗ distTensorDeriv A ε | μ' v)}T) := by
simp only [map_sub, Finsupp.coe_sub, Pi.sub_apply]
rw [toTensor_distTensorDeriv_basis_repr_apply]

```

1.41.

*PHYSLIB/ELECTROMAGNETISM/DISTRIBUTIONAL/MAGNETICFIELD.lean*

```
rw [toTensor_distTensorDeriv_basis_repr_apply]
simp [map_sum, map_smul, Finsupp.coe_finsetSum, Finsupp.coe_smul, Finset.sum_apply,
 fieldStrengthAux_basis_repr_apply_eq_single, map_add, add_apply,
 fieldStrengthAux_basis_repr_apply_eq_single, map_smul, FunLike.coe_smul,
rw [fieldStrengthAux_eq_add, distTensorDeriv_equivariant, lorentzGroup_smul_dist_a
```

#### 1.41 Physlib/Electromagnetism/Distributional/MagneticField.lean

```
simp only [magneticFieldMatrix, LinearMap.coe_mk, AddHom.coe_mk, ContinuousLinearMap
map_smul, Finsupp.coe_finsetSum, Finsupp.coe_smul, Finset.sum_apply, Pi.smul_app
Space.distSpaceDeriv_apply_CLM, ContinuousLinearMap.coe_comp, ContinuousLinearMa
```

#### 1.42 Physlib/Electromagnetism/Distributional/ScalarPotential.lean

```
public import Mathlib.Algebra.Order.Archimedean.Real.Hom
 LinearMap.coe_mk, AddHom.coe_mk, ContinuousLinearMap.coe_comp, FunLike.coe_smu
```

#### 1.43 Physlib/Electromagnetism/Distributional/VectorPotential.lean

```
public import Mathlib.Algebra.Order.Archimedean.Real.Hom
 LinearMap.coe_mk, AddHom.coe_mk, FunLike.coe_smul, ContinuousLinearMap.coe_com
```

#### 1.44 Physlib/Electromagnetism/Dynamics/Basic.lean

```
public import Mathlib.Analysis.Real.Sqrt
```

#### 1.45 Physlib/Electromagnetism/Dynamics/Hamiltonian.lean

```
simp only [kineticTerm_add_time_mul_const _ (hA.differentiable (by simp))]
simp only [Fin.isValue, freeCurrentPotential, map_add, map_smul, _root_.add_appl
 FunLike.coe_smul, Pi.smul_apply, smul_eq_mul, fderiv_const_add,
 FunLike.coe_sub, Pi.sub_apply]
simp only [Fin.isValue, FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
simp (disch := fun_prop) only [Fin.isValue, Finset.sum_sub_distrib, one_div,
 fderiv_const_add, fderiv_fun_add, fderiv_const_mul, fderiv_fun_sub, fderiv_fun_s
 fderiv_mul_const, fderiv_fun_pow]
rw [← Finset.sum_sub_distrib, Finset.mul_sum]
simp only [Fin.isValue, RCLike.inner_apply, conj_trivial, equivEuclid_apply]
rw [electricField_eq_fieldStrengthMatrix (hA := hA.differentiable (by simp))]
```

- exact (hA.differentiable (by simp)).comp (by fun\_prop)
- exact vectorPotential\_differentiable\_time A (hA.differentiable (by simp))

## 1.46 Physlib/Electromagnetism/Dynamics/IsExtrema.lean

```

(permt id (IsReindexing.auto) {((1/ μ_0 : $\mathbb{R}$) • tensorDeriv A.toFie
have hcd : $\forall$ ij, ContDiff $\mathbb{R}^2$ (fun y => A.magneticFieldMatrix $\square$.c t y ij)
 fun ij => magneticFieldMatrix_space_contDiff _ (hA.of_le (right_eq_inf.
have hsd : $\forall$ ij k, Differentiable $\mathbb{R}$ ($\partial[k]$ (fun y => A.magneticFieldMatrix
 fun ij k => Space.deriv_differentiable (hcd ij) k
have hJd : $\forall$ i, Differentiable $\mathbb{R}$ (fun x => J.currentDensity $\square$.c t x i) :=
 fun i => LorentzCurrentDensity.currentDensity_apply_differentiable_spac
 (hJ.differentiable (by simp)) t i
rw [fderiv_fun_sub ((Differentiable.fun_sum fun i _ => hsd _ i).const_mul
 ((hJd _).const_mul _).differentiableAt,
 fderiv_const_mul (Differentiable.fun_sum fun i _ => hsd _ i).differenti
 fderiv_const_mul (hJd _).differentiableAt,
 fderiv_fun_sum fun i _ => (hsd _ i).differentiableAt]
rw [fderiv_fun_sub ((Differentiable.fun_sum fun i _ => hsd _ i).const_m
 ((hJd _).const_mul _).differentiableAt,
 fderiv_const_mul (Differentiable.fun_sum fun i _ => hsd _ i).differenti
 fderiv_const_mul (hJd _).differentiableAt,
 fderiv_fun_sum fun i _ => (hsd _ i).differentiableAt]
rw [fderiv_fun_sub (hsd _ _).differentiableAt (hsd _ _).differentiableA
rw [Space.deriv_commute _ (hcd _)]
rw [Space.deriv_commute _ (hcd _)]
have hEs : $\forall$ j, ContDiff $\mathbb{R}^2$ (fun y => A.electricField $\square$.c t y j) :=
 fun j => electricField_apply_contDiff_space (i := j) (hA.of_le (right_e
have hEd : $\forall$ j k, Differentiable $\mathbb{R}$ ($\partial[k]$ (fun y => A.electricField $\square$.c t
 fun j k => Space.deriv_differentiable (hEs j) k
have hBt : $\forall$ j, Differentiable $\mathbb{R}$
 (fun s => $\partial[j]$ (fun y => A.magneticFieldMatrix $\square$.c s y (j, i)) x) :=
 fun j => Space.space_deriv_differentiable_time (i := j)
 (magneticFieldMatrix_contDiff _ (hA.of_le (right_eq_inf.mp rfl)) (j,
have hJt : Differentiable $\mathbb{R}$ (fun s => J.currentDensity $\square$.c s x i) :=
 LorentzCurrentDensity.currentDensity_apply_differentiable_time (hJ.diff
 simp only [one_div, mul_inv_rev, FunLike.coe_sub, Pi.sub_apply]
 rw [fderiv_const_mul (Differentiable.fun_sum fun j _ => hBt j).differ
 rw [fderiv_const_mul hJt.differentiableAt]
 · exact ((Differentiable.fun_sum fun j _ => hBt j).const_mul _).diffe
 · exact hJt.differentiableAt.const_mul _
 rw [fderiv_fun_sum fun i _ => (hBt i).differentiableAt]
 simp only [FunLike.coe_sum, Finset.sum_apply]
 rw [fderiv_fun_sub (hEd _ _).differentiableAt (hEd _ _).differentia

```

```

rw [Space.deriv_commute _ (hEs _), Space.deriv_eq_fderiv_basis]
intro j _
exact (hEd j j).differentiableAt

```

### 1.47 Physlib/Electromagnetism/Dynamics/KineticTerm.lean

```

rw [← actionT_coMetric Λ]
simp only [Tensorial.self_toTensor_apply]
rw [coMetric_repr_apply_eq_minkowskiMatrix]
simp only [Tensorial.self_toTensor_apply]
rw [coMetric_repr_apply_eq_minkowskiMatrix]
simp only [Fin.isValue, _root_.add_apply, Lorentz.Vector.apply_add, Pi.add_apply]
simp only [Fin.isValue, _root_.add_apply, Lorentz.Vector.apply_add]
 simp only [hdiff]
 simp only [FunLike.coe_sub, Pi.sub_apply]
(permt id (IsReindexing.auto) {(1/ μ_0 : $\mathbb{R}$) •
 tensorDeriv A.toFieldStrength x | $\kappa \kappa v'$ }T)) v := by
(permt id (IsReindexing.auto) {(1/ μ_0 : $\mathbb{R}$) • tensorDeriv A.toFieldStrength x |

```

### 1.48 Physlib/Electromagnetism/Dynamics/Lagrangian.lean

```

simp [freeCurrentPotential]
exact (freeCurrentPotential_hasVarGradientAt A hA J hJ).varGradient
congr <=>
 simp [LorentzCurrentDensity.chargeDensity, LorentzCurrentDensity.currentDensity]
 (permt id (IsReindexing.auto) {J x | v' }T)) v := by
 (permt id (IsReindexing.auto) {J x | v' }T)) v
rw [lagrangian,
 kineticTerm_eq_electricMatrix_magneticFieldMatrix _ _ (hA.differentiable (by simp
 freeCurrentPotential_eq_sum_scalarPotential_vectorPotential $\square$ A J x)]
exact (lagrangian_hasVarGradientAt_eq_add_gradKineticTerm A hA J hJ).varGradient
module
rw [gradLagrangian_eq_kineticTerm_sub A hA J hJ, Pi.sub_apply,
 gradKineticTerm_eq_electric_magnetic _ _ hA,
 gradFreeCurrentPotential_eq_chargeDensity_currentDensity $\square$ A hA J hJ x,
 add_sub_add_comm, ← Finset.sum_sub_distrib]
congr 1
· module
· refine Finset.sum_congr rfl fun i _ => ?_
 rw [$\square$.c_sq, one_div_one_div]
 module
 (permt id (IsReindexing.auto) {(1/ μ_0 : $\mathbb{R}$) • tensorDeriv A.toFieldStrength x
 map_neg, map_add, permt_permT, CompTriple.comp_eq, apply_add, apply_smul,

```

Lorentz.Vector.neg\_apply]

### 1.49 Physlib/Electromagnetism/Kinematics/EMPotential.lean

```
instance {d} : Zero (ElectromagneticPotential d) where
 zero := ⟨fun _ => 0⟩

@[simp]
lemma zero_val {d} : (0 : ElectromagneticPotential d).val = 0 := rfl

@[simp]
lemma zero_apply {d} (x : SpaceTime d) : (0 : ElectromagneticPotential d) x = 0 := rfl

@[simp]
instance {d} : Neg (ElectromagneticPotential d) where
 neg A := ⟨fun x => - A x⟩

@[simp]
lemma neg_val {d} (A : ElectromagneticPotential d) :
 (- A).val = - A.val := rfl

@[simp]
lemma neg_apply {d} (A : ElectromagneticPotential d) (x : SpaceTime d) :
 (- A) x = - A x := rfl

instance {d} : Sub (ElectromagneticPotential d) where
 sub A B := ⟨fun x => A x - B x⟩

@[simp]
lemma sub_val {d} (A B : ElectromagneticPotential d) :
 (A - B).val = A.val - B.val := rfl

@[simp]
lemma sub_apply {d} (A B : ElectromagneticPotential d) (x : SpaceTime d) :
 (A - B) x = A x - B x := rfl

instance {d} : AddCommGroup (ElectromagneticPotential d) where
 add_assoc A B C := by
 ext x μ
 simp [add_assoc]
 zero_add A := by
 ext x μ
 simp
 add_zero A := by
```

```

 ext x μ
 simp
neg_add_cancel A := by
 ext x μ
 simp
add_comm A B := by
 ext x μ
 simp [add_comm]
nsmul := nsmulRec
zsmul := zsmulRec

@[simp]
ext1
ext1 x
@[simp]
lemma action_apply {d} (Λ : LorentzGroup d) (A : ElectromagneticPotential d)
 (x : SpaceTime d) :
 (Λ • A) x = Λ • A (Λ-1 • x) := rfl

 simp [mul_smul]
 simp [one_smul]
noncomputable instance {d} :
 DistribMulAction (LorentzGroup d) (ElectromagneticPotential d) where
 smul_zero Λ := by
 ext x μ
 simp
 smul_add Λ A B := by
 ext x μ
 simp [Lorentz.Vector.smul_add]
 rw [action_val, SpaceTime.deriv_equivariant A.val Λ x hA μ, Lorentz.Vector.apply_s
 Finset.sum_comm]
 refine Finset.sum_congr rfl (fun p _ => ?_)
 rw [Lorentz.Vector.apply_smul, Lorentz.Vector.smul_eq_sum, Finset.mul_sum]
 refine Finset.sum_congr rfl (fun κ _ => ?_)
 simp [f, f', inner_smul_left, Lorentz.Vector.basis_inner]
 trans (Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis).repr (deriv A x)
 change _ * (Lorentz.contrBasis d).repr (Lorentz.contrBasis d (b 1)) v = _
 have h1 : Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis =
 by_cases hμ : b 0 = μ <=> by_cases hv : b 1 = v <=>
 simp_all [Equiv.eq_symm_apply, show e b = (b 0, b 1) from rfl]
 rcases μv with ⟨μ, v⟩
 ext ⟨i, j⟩

```

### 1.50 Physlib/Electromagnetism/Kinematics/ElectricField.lean

```

rw [sub_eq_iff_eq_add', ← h, add_comm]
 convert! h
 · fun_prop
 · exact differentiable_component A hA _
 fun_prop
 · fun_prop
rw [fieldStrengthMatrix_antisymm A x (Sum.inr i) (Sum.inl 0),
 fieldStrengthMatrix_inl_inr_eq_electricField A x i hA]
ring
exact (fieldStrengthMatrix_contDiff hA).comp
 (ContinuousLinearEquiv.contDiff (toTimeAndSpace c).symm)
 (hA : ContDiff ℝ (n + 1) A) : ContDiff ℝ n (λ (fun t x => A.electricField c t x i)) :=
 (ContinuousLinearMap.contDiff (λ := ℝ) (EuclideanSpace.proj i)).comp (electricField c t x i) :=
 (ContDiff ℝ n (fun x => A.electricField c t x i) :=
 (electricField_apply_contDiff hA).comp (f := fun x => (t, x)) (by fun_prop)
 ContDiff ℝ n (fun t => A.electricField c t x i) :=
 (electricField_apply_contDiff hA).comp (f := fun t => (t, x)) (by fun_prop)
 (hA : ContDiff ℝ 2 A) : Differentiable ℝ (λ (A.electricField c t x i)) :=
 (electricField_contDiff (n := 1) hA).differentiable one_ne_zero
 (hA : ContDiff ℝ 2 A) (x : Space d) : Differentiable ℝ (A.electricField c t x i) :=
 (electricField_differentiable hA).comp (f := fun t => (t, x)) (by fun_prop)
 (hA : ContDiff ℝ 2 A) (t : Time) : Differentiable ℝ (A.electricField c t x i) :=
 (electricField_differentiable hA).comp (f := fun x => (t, x)) (by fun_prop)
 Differentiable ℝ (fun (tx : Time × Space d) => A.electricField c tx.1 tx.2) :=
 (ContinuousLinearMap.differentiable (λ := ℝ) (EuclideanSpace.proj i)).comp
 (electricField_differentiable hA)
 Differentiable ℝ (fun x => A.electricField c t x i) :=
 (electricField_apply_differentiable hA).comp (f := fun x => (t, x)) (by fun_prop)
 Differentiable ℝ (fun t => A.electricField c t x i) :=
 (electricField_apply_differentiable hA).comp (f := fun t => (t, x)) (by fun_prop)
rw [Time.deriv_eq, fderiv_const_mul]
 · exact (fieldStrengthMatrix_differentiable_time hA x).differentiableAt x
 exact (fieldStrengthMatrix_differentiable_space hA t).neg.differentiableAt x

```

### 1.51 Physlib/Electromagnetism/Kinematics/FieldStrength.lean

```

public import Mathlib.Algebra.Order.Archimedean.Real.Hom
 - A.5.1. Index evaluation
(permt id (IsReindexing.auto)
 {(η d | μ μ' ⊗ A.deriv x | μ' v) + - (η d | v v' ⊗ A.deriv x | v' μ)}T)
Tensorial.toTensor.symm (permt id IsReindexing.auto {(η d | μ μ' ⊗ A.deriv x | μ' v) + - (η d | v v' ⊗ A.deriv x | v' μ)}T)
Tensorial.toTensor.symm (permt id IsReindexing.auto {(η d | μ μ' ⊗ tens

```



1.51.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/FIELDSTRENGTH.LEAN 69

```

Tensorial.toTensor.symm (permT id (IsReindexing.auto) {(η d | μ μ' ⊗ A.deriv x |
- Tensorial.toTensor.symm (permT ![1, 0] (IsReindexing.auto)
Tensorial.toTensor.symm (permT id IsReindexing.auto {η d | μ μ' ⊗ tensorDeriv A
- Tensorial.toTensor.symm (permT ![1, 0] IsReindexing.auto
(permT id (IsReindexing.auto) {(η d | μ μ' ⊗ A.deriv x | μ' v)}T)
- (permT ![1, 0] (IsReindexing.auto) {(η d | μ μ' ⊗ A.deriv x | μ' v)}T) := by
exact prod_eq_sum_eval Vector.basis_eq_map_tensor_basis
Vector.basis_eq_map_tensor_basis (A.toFieldStrength x)
A.toFieldStrength x = ∑ μ, ∑ v, (∑ κ, (η μ κ * ∂_κ A x v - η v κ * ∂_κ A x μ))
simp only [map_sum, map_smul, Finsupp.coe_finsetSum, Finsupp.coe_smul,
toFieldStrength_eq_sub_tensorDeriv hA, ← deriv_eq_tensorDeriv _ hA, map_sub, Bas
Basis.map_repr, LinearEquiv.symm_symm, LinearEquiv.trans_apply, LinearEquiv.appl
Finsupp.coe_sub, Pi.sub_apply, Finsupp.mapDomain_equiv_apply, permT_basis_repr_s
Function.comp_apply, contrT_basis_repr_apply_eq_fin, prodT_basis_repr_apply,
contrMetric_repr_apply_eq_minkowskiMatrix,
simp only [map_sub, Finsupp.coe_sub, Pi.sub_apply]
A.5.1. Index evaluation

```

These lemmas express the components of the field strength tensor using index evaluation

-/

/-- Evaluating both tensor indices of the field strength gives the coefficient in the standard tensor-product basis. -/

```

lemma toFieldStrength_eval_eq_basis_repr {d} (A : ElectromagneticPotential d)
(x : SpaceTime d) (μ v : Fin 1 ⊕ Fin d) :
toField {A.toFieldStrength x | [μ] [v]}T =
(Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis).repr
(A.toFieldStrength x) (μ, v) := by
trans (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr
(A.toFieldStrength x) (μ, v)
· conv_rhs =>
rw [prod_eq_sum_eval Vector.basis_eq_map_tensor_basis
Vector.basis_eq_map_tensor_basis (A.toFieldStrength x)]
simp [Basis.tensorProduct_repr_tmul_apply, Finsupp.single_apply]
· rfl

```

/-- The evaluated components of the field strength tensor in terms of derivatives of electromagnetic potential. -/

```

lemma toFieldStrength_eval_apply {d} (A : ElectromagneticPotential d)
(x : SpaceTime d) (μ v : Fin 1 ⊕ Fin d) :
toField {A.toFieldStrength x | [μ] [v]}T =
∑ κ, (η μ κ * ∂_κ A x v - η v κ * ∂_κ A x μ) := by
rw [toFieldStrength_eval_eq_basis_repr]
exact toFieldStrength_basis_repr_apply (μv := (μ, v)) A x

```

```

/-- The evaluated components of the field strength tensor after using diagonal
Minkowski metric. -/
lemma toFieldStrength_eval_apply_eq_single {d} (A : ElectromagneticPotential d)
 (x : SpaceTime d) (μ ν : Fin 1 ⊕ Fin d) :
 toField {A.toFieldStrength x | [μ] [ν]}T =
 η μ μ * ∂_μ A x ν - η ν ν * ∂_ν A x μ := by
 rw [toFieldStrength_eval_eq_basis_repr]
 exact toFieldStrength_basis_repr_apply_eq_single (μν := (μ, ν)) A x

/-!

/-- Index evaluation of the field strength tensor agrees with the corresponding
the field strength matrix. -/
lemma toFieldStrength_eval_eq_fieldStrengthMatrix {d} (A : ElectromagneticPotential d)
 (x : SpaceTime d) (μ ν : Fin 1 ⊕ Fin d) :
 toField {A.toFieldStrength x | [μ] [ν]}T = A.fieldStrengthMatrix x (μ, ν)
 rw [toFieldStrength_eval_eq_basis_repr, fieldStrengthMatrix_eq]

simp only [_root_.add_apply, Lorentz.Vector.apply_add]
simp only [FunLike.coe_smul, Pi.smul_apply, Lorentz.Vector.apply_smul]

```

## 1.52 Physlib/Electromagnetism/Kinematics/GaugeTransformation.lean

```

/-
Copyright (c) 2026 Justin Findlay. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Justin Findlay
-/
module

public import Physlib.Electromagnetism.Kinematics.FieldStrength
/-!

Gauge Transformations of the Electromagnetic Potential

i. Overview

In this module we define gauge transformations of the electromagnetic potential
 $A^\mu \mapsto A^\mu + \partial^\mu \chi$ where $\chi : \text{SpaceTime } d \rightarrow \mathbb{R}$ is a smooth gauge function,
that the field strength tensor is invariant under such transformations.

The raised-index gradient $\partial^\mu \chi := \eta^{\mu\nu} \partial_\nu \chi$ is necessary because the
 $\partial_\mu \chi$ does not make $F^{\mu\nu}$ invariant. The formal witness is
fieldStrengthMatrix_bareGradient_inl_inr (§B.5), which computes a specific

```

1.52.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/GAUGETRANSFORMATION7LEAN

the field strength of a bare-gradient potential. The invariance theorem  
`toFieldStrength\_gaugeTransform` doubles as a correctness test of `ofGradient`.

## ## ii. Key results

- `ofGradient` : The pure-gauge potential  $A^\mu = \eta^{\{\mu\nu\}} \partial_\nu \chi$  built from a gauge function  $\chi$ .
- `gaugeTransform` : The gauge transformation  $A^\mu \mapsto A^\mu + \partial^\mu \chi$ .
- `toFieldStrength\_ofGradient` : A pure-gauge potential has vanishing field strength.
- `toFieldStrength\_gaugeTransform` : The field strength tensor is invariant under gauge transformations.
- `fieldStrengthMatrix\_gaugeTransform` : The field strength matrix is invariant under gauge transformations.
- `gaugeTransform\_gaugeTransform` : Composing two gauge shifts equals shifting by the sum; upgrades one-step F-invariance to invariance along any finite chain.
- `ofGradient\_equivariant` : `ofGradient` intertwines the Lorentz action with function transformations.
- `gaugeTransform\_equivariant` : Gauge transformations commute with Lorentz transformations.
- `fieldStrengthMatrix\_bareGradient\_inl\_inr` : The  $(inl\ 0, inr\ i)$  field-strength component of the bare-gradient potential  $\chi(x) = x^0 \cdot x^i$  equals  $\delta^i_0$ ; in particular the bare gradient does not give a gauge-invariant field strength (necessity of the metric contraction in the definition).

## ## iii. Table of contents

- A. The pure-gauge potential
  - A.1. Definition and basic lemmas
  - A.2. Differentiability of the pure-gauge potential
  - A.3. Vanishing field strength of the pure-gauge potential
  - A.4. Lorentz equivariance of the pure-gauge potential
- B. Gauge transformations
  - B.1. Definition and basic lemmas
  - B.2. Invariance of the field strength
  - B.3. Group structure of gauge shifts
  - B.4. Equivariance under Lorentz transformations
  - B.5. Necessity: bare gradient does not give gauge invariance

## ## iv. References

- [https://en.wikipedia.org/wiki/Mathematical\\_descriptions\\_of\\_the\\_electromagnetic\\_field](https://en.wikipedia.org/wiki/Mathematical_descriptions_of_the_electromagnetic_field)
- /

```
@[expose] public section
namespace Electromagnetism
open Module realLorentzTensor
open TensorSpecies
open Tensor
```

```

namespace ElectromagneticPotential

open TensorSpecies
open Tensor
open SpaceTime
open TensorProduct
open minkowskiMatrix Tensorial
open Lorentz

attribute [-simp] Fintype.sum_sum_type
attribute [-simp] Nat.succ_eq_add_one

/-!

A. The pure-gauge potential

-/

/-!

A.1. Definition and basic lemmas

-/

/-- The pure-gauge electromagnetic potential $A^\mu = \partial^\mu \chi = \eta^{\{\mu\nu\}} \partial_\nu \chi$ b
gauge function χ . The metric-contracted form $\sum \kappa, \eta \mu \kappa * \partial_\kappa \chi$ is t
the diagonal form (which equals $\eta \mu \mu * \partial_\mu \chi$ since η is diagonal) i
`ofGradient_apply`. -/
noncomputable def ofGradient {d} ($\chi : \text{SpaceTime } d \rightarrow \mathbb{R}$) : ElectromagneticPot
 val $x \mu := \sum \kappa, \eta \mu \kappa * \partial_\kappa \chi x$

/-- Unfolding of the summed definition of `ofGradient`. -
/
lemma ofGradient_apply_sum {d} ($\chi : \text{SpaceTime } d \rightarrow \mathbb{R}$) ($x : \text{SpaceTime } d$) ($\mu :$
 ofGradient $\chi x \mu = \sum \kappa, \eta \mu \kappa * \partial_\kappa \chi x := \text{rfl}$

/-- Evaluation of `ofGradient` in the diagonal form; the off-
diagonal entries of η vanish so
only the $\kappa = \mu$ term survives. -/
lemma ofGradient_apply {d} ($\chi : \text{SpaceTime } d \rightarrow \mathbb{R}$) ($x : \text{SpaceTime } d$) ($\mu : \text{Fin}$
 ofGradient $\chi x \mu = \eta \mu \mu * \partial_\mu \chi x := \text{by}$
 rw [ofGradient_apply_sum]
 rw [Finset.sum_eq_single μ]
 · intro $\kappa _ hk$
 rw [minkowskiMatrix.off_diag_zero (Ne.symm hk)]

```

1.52.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/GAUGETRANSFORMATION71LEAN

```

 simp
 · simp

/-- The pure-gauge potential built from the zero gauge function has all components zero
/
lemma ofGradient_zero {d} (x : SpaceTime d) (μ : Fin 1 ⊕ Fin d) :
 ofGradient (0 : SpaceTime d → ℝ) x μ = 0 := by
 rw [ofGradient_apply]
 simp [SpaceTime.deriv_eq]

/-- `ofGradient` is additive in the gauge function (when both summands are differentiable)
/
lemma ofGradient_add {d} {χ₁ χ₂ : SpaceTime d → ℝ}
 (hχ₁ : Differentiable ℝ χ₁) (hχ₂ : Differentiable ℝ χ₂) :
 ofGradient (χ₁ + χ₂) = ofGradient χ₁ + ofGradient χ₂ := by
 apply eq_of_val_eq; funext x μ
 show ofGradient (χ₁ + χ₂) x μ = ofGradient χ₁ x μ + ofGradient χ₂ x μ
 simp only [ofGradient_apply, SpaceTime.deriv_eq,
 fderiv_add hχ₁.differentiableAt hχ₂.differentiableAt, _root_.add_apply,
 mul_add]

/-!

A.2. Differentiability of the pure-gauge potential

-/

/-- The pure-gauge potential is differentiable when `χ` is `C^2`. -
/
lemma differentiable_ofGradient {d} {χ : SpaceTime d → ℝ} (hχ : ContDiff ℝ 2 χ) :
 Differentiable ℝ (ofGradient χ) := by
 show Differentiable ℝ (ofGradient χ).val
 rw [← SpaceTime.differentiable_vector]
 intro μ
 simp_rw [ofGradient_apply]
 exact (SpaceTime.differentiable_deriv μ χ hχ).const_mul _

/-- The pure-gauge potential is `C^n` when `χ` is `C^{n+1}`. -
/
lemma contDiff_ofGradient {n} {d} {χ : SpaceTime d → ℝ} (hχ : ContDiff ℝ (n + 1) χ) :
 ContDiff ℝ n (ofGradient χ) := by
 show ContDiff ℝ n (ofGradient χ).val
 rw [← SpaceTime.contDiff_vector]
 intro μ
 simp_rw [ofGradient_apply]
 have h := SpaceTime.contDiff_deriv μ χ hχ

```

```

 fun_prop

/-!

A.3. Vanishing field strength of the pure-gauge potential

-/

/-- A pure-gauge potential has vanishing field strength. -
/
lemma toFieldStrength_ofGradient {d} {χ : SpaceTime d → ℝ} (hχ : ContDiff ℝ
 (x : SpaceTime d) : (ofGradient χ).toFieldStrength x = 0 := by
 apply (Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis).repr.in
 apply Finsupp.ext
 intro μν
 simp only [toFieldStrength_basis_repr_apply_eq_single]
 rw [SpaceTime.deriv_apply_eq μν.1 μν.2 (ofGradient χ) (differentiable_ofG
 SpaceTime.deriv_apply_eq μν.2 μν.1 (ofGradient χ) (differentiable_ofGra
 simp only [ofGradient_apply]
 rw [fderiv_const_mul (SpaceTime.differentiable_deriv μν.1 χ hχ).different
 fderiv_const_mul (SpaceTime.differentiable_deriv μν.2 χ hχ).differentia
 simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
 -- simplify repr 0 to 0
 conv_rhs => rw [show (Lorentz.CoVector.basis.tensorProduct Lorentz.Vector
 (0 : Lorentz.Vector d ⊗[ℝ] Lorentz.Vector d) = 0 from map_zero _]
 simp only [Finsupp.zero_apply]
 -- use Clairaut: ∂_μ (∂_ν χ) x = ∂_ν (∂_μ χ) x, so the two terms cancel
 have heq : fderiv ℝ (∂_μν.2 χ) x (Lorentz.Vector.basis μν.1) =
 fderiv ℝ (∂_μν.1 χ) x (Lorentz.Vector.basis μν.2) := by
 change ∂_μν.1 (∂_μν.2 χ) x = ∂_μν.2 (∂_μν.1 χ) x
 rw [← SpaceTime.deriv_commute μν.2 μν.1 χ hχ]
 rw [heq]
 ring

/-!

A.4. Lorentz equivariance of the pure-gauge potential

`ofGradient` intertwines the Lorentz action on potentials with composition
function: $\Lambda \cdot \text{ofGradient } \chi = \text{ofGradient } (\chi \circ (\Lambda^{-1} \cdot \cdot))$. The proof reduce
metric-commutativity identity $\Lambda * \eta = \eta * (\Lambda^{-1})^\top$, which is the defining p
Lorentz group (LorentzGroup.comm_minkowskiMatrix).

-/

/-- `ofGradient` intertwines the Lorentz action on potentials with composition

```

1.52.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/GAUGETRANSFORMATION7LEAN

```

 gauge function: `Λ • ofGradient χ = ofGradient (χ • (Λ-1 • .))`. -
/
lemma ofGradient_equivariant {d} (χ : SpaceTime d → ℝ) (hχ : Differentiable ℝ χ)
 (Λ : LorentzGroup d) :
 Λ • ofGradient χ = ofGradient (χ • (Λ-1 • .)) := by
 -- The metric-commutativity row identity, extracted from `comm_minkowskiMatrix`:
 -- $\sum v, \Lambda.1 \mu v * \eta v \kappa = \sum v, \eta \mu v * (\Lambda^{-1}).1 \kappa v$
 have hmetric : ∀ (μ κ : Fin 1 ⊕ Fin d),
 $\sum v, \Lambda.1 \mu v * \eta v \kappa = \sum v, \eta \mu v * (\Lambda^{-1}).1 \kappa v$:= by
 intro μ κ
 -- comm_minkowskiMatrix: $\Lambda.1 * \eta = \eta * (\Lambda^{-1})^T$, applied at (μ, κ)
 have h := congr_fun2 (LorentzGroup.comm_minkowskiMatrix (Λ := Λ)) μ κ
 simp only [Matrix.mul_apply, LorentzGroup.transpose_val, Matrix.transpose_apply]
 exact h
 apply eq_of_val_eq; funext x μ
 -- Let a κ = ∂_κ χ (Λ-1 • x). Both sides equal $\sum \kappa, (\sum v, \Lambda.1 \mu v * \eta v \kappa) * a \kappa$.
 set a : Fin 1 ⊕ Fin d → ℝ := fun κ => ∂_κ χ (Λ-1 • x)
 -- Expand LHS: action_val → smul_eq_sum → ofGradient_apply_sum → reassociate + fact
 have hlhs : (Λ • ofGradient χ).val x μ = $\sum \kappa, (\sum v, \Lambda.1 \mu v * \eta v \kappa) * a \kappa$:= by
 simp only [ElectromagneticPotential.action_val, Lorentz.Vector.smul_eq_sum, a,
 ofGradient_apply_sum, Finset.mul_sum]
 rw [Finset.sum_comm]
 apply Finset.sum_congr rfl; intro κ _
 simp_rw [← mul_assoc]
 rw [← Finset.sum_mul]
 -- Expand RHS: ofGradient_apply_sum → deriv_comp_lorentz_action → sum_comm + fact
 have hrhs : (ofGradient (χ • (Λ-1 • .))).val x μ = $\sum \kappa, (\sum v, \Lambda.1 \mu v * \eta v \kappa) * a \kappa$
 simp only [ofGradient_apply_sum, a]
 conv_lhs =>
 enter [2, v]
 rw [show ∂_v (χ • (Λ-1 • .)) x = ∂_v (fun y => χ (Λ-1 • y)) x from rfl,
 SpaceTime.deriv_comp_lorentz_action v χ hχ Λ-1 x]
 simp only [smul_eq_mul, Finset.mul_sum]
 rw [Finset.sum_comm]
 apply Finset.sum_congr rfl; intro κ _
 simp_rw [← mul_assoc]
 rw [← Finset.sum_mul, ← hmetric]
 rw [hlhs, hrhs]

/-!

B. Gauge transformations

-/

/-!
```

### ### B.1. Definition and basic lemmas

-/

*-- The gauge transformation  $A^\mu \mapsto A^\mu + \partial^\mu \chi$  of an electromagnetic potential*  
*/*

noncomputable def gaugeTransform {d} ( $\chi$  : SpaceTime d  $\rightarrow \mathbb{R}$ ) (A : ElectromagneticPotential d) : ElectromagneticPotential d := A + ofGradient  $\chi$

*-- Evaluation of `gaugeTransform`. -/*

lemma gaugeTransform\_apply {d} ( $\chi$  : SpaceTime d  $\rightarrow \mathbb{R}$ ) (A : ElectromagneticPotential d) (x : SpaceTime d) : gaugeTransform  $\chi$  A x = A x + ofGradient  $\chi$  x := by  
 simp [gaugeTransform, add\_apply]

*/-!*

### ### B.2. Invariance of the field strength

The key ingredient – that a pure-gauge potential has vanishing field strength (`toFieldStrength_ofGradient`).

-/

*-- The field strength tensor is invariant under gauge transformations. -/*

lemma toFieldStrength\_gaugeTransform {d} (A : ElectromagneticPotential d) ( $\chi$  : SpaceTime d  $\rightarrow \mathbb{R}$ ) (hA : Differentiable  $\mathbb{R}$  A) (h $\chi$  : ContDiff  $\mathbb{R}$  2  $\chi$ ) (hA' : Differentiable  $\mathbb{R}$  (gaugeTransform  $\chi$  A)) :  
 (gaugeTransform  $\chi$  A).toFieldStrength x = A.toFieldStrength x := by  
 rw [gaugeTransform, toFieldStrength\_add A (ofGradient  $\chi$ ) x hA (differentiable\_of\_gaugeTransform hA'), toFieldStrength\_ofGradient h $\chi$ , add\_zero]

*-- The field strength matrix is invariant under gauge transformations. -/*

lemma fieldStrengthMatrix\_gaugeTransform {d} (A : ElectromagneticPotential d) ( $\chi$  : SpaceTime d  $\rightarrow \mathbb{R}$ ) (hA : Differentiable  $\mathbb{R}$  A) (h $\chi$  : ContDiff  $\mathbb{R}$  2  $\chi$ ) (hA' : Differentiable  $\mathbb{R}$  (gaugeTransform  $\chi$  A)) :  
 (gaugeTransform  $\chi$  A).fieldStrengthMatrix x = A.fieldStrengthMatrix x :=  
 rw [fieldStrengthMatrix, toFieldStrength\_gaugeTransform A  $\chi$  hA h $\chi$ ]

*/-!*

### ### B.3. Group structure of gauge shifts

Composing two gauge shifts by  $\chi_1$  and  $\chi_2$  is the same as shifting by  $\chi_1 + \chi_2$ . `gaugeTransform_zero` this shows that the map  $\chi \mapsto (A \mapsto \text{gaugeTransform } \chi A)$  is a homomorphism of the additive group of smooth functions. (We do not build the formal `Mul` instance for this map.)



1.52.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/GAUGETRANSFORMATION71LEAN

the composition lemma is the agreeable core, and the rest is straightforward from it

The `ofGradient` lemmas used here – `ofGradient\_zero` and `ofGradient\_add` – are pro

-/

/-- Shifting by the zero gauge function is the identity. -

/

```
lemma gaugeTransform_zero {d} (A : ElectromagneticPotential d) :
 gaugeTransform (0 : SpaceTime d → ℝ) A = A := by
 apply eq_of_val_eq; funext x μ
 show gaugeTransform (0 : SpaceTime d → ℝ) A x μ = A x μ
 simp [gaugeTransform, add_apply, ofGradient_zero]
```

/-- Two successive gauge shifts compose: shifting by  $\chi_2$  then  $\chi_1$  equals shifting by  $\chi_1 + \chi_2$ .  
This upgrades one-step F-invariance to invariance along any finite chain of gauge

/

```
lemma gaugeTransform_gaugeTransform {d} (A : ElectromagneticPotential d)
 (χ₁ χ₂ : SpaceTime d → ℝ) (hχ₁ : Differentiable ℝ χ₁) (hχ₂ : Differentiable ℝ χ₂) :
 gaugeTransform χ₁ (gaugeTransform χ₂ A) = gaugeTransform (χ₁ + χ₂) A := by
 apply eq_of_val_eq; funext x μ
 simp only [gaugeTransform, add_apply]
 rw [ofGradient_add hχ₁ hχ₂]
 simp [add_apply, add_comm, add_left_comm]
```

/-!

### ### B.4. Equivariance under Lorentz transformations

The gauge-transformation map commutes with the Lorentz group action: applying  $\Lambda$  to  $A$  and then gauge-transforming by  $\chi$  is the same as gauge-transforming by  $\chi \circ (\Lambda^{-1} \cdot \cdot)$  and then applying  $\Lambda$ . The proof delegates to `ofGradient\_equivariant` (§A.4).

-/

/-- Gauge transformations commute with Lorentz transformations: applying  $\Lambda$  and then a gauge transformation by  $\chi$  equals performing a gauge transformation by  $\chi \circ (\Lambda^{-1} \cdot \cdot)$  and then applying  $\Lambda$ . -/

```
lemma gaugeTransform_equivariant {d} (A : ElectromagneticPotential d)
 (χ : SpaceTime d → ℝ) (hχ : Differentiable ℝ χ) (Λ : LorentzGroup d) :
 Λ • gaugeTransform χ A = gaugeTransform (χ ∘ (Λ⁻¹ • ·)) (Λ • A) := by
 -- Unfold both sides to Λ • A + ofGradient (χ ∘ (Λ⁻¹ • ·))
 simp only [gaugeTransform]
 rw [← ofGradient_equivariant χ hχ Λ]
 -- Goal: Λ • (A + ofGradient χ) = Λ • A + Λ • ofGradient χ
 apply eq_of_val_eq; funext x μ
```

```

 simp only [ElectromagneticPotential.action_val, add_val, Pi.add_apply,
 Lorentz.Vector.smul_add]

/-!

B.5. Necessity: bare gradient does not give gauge invariance

We exhibit a concrete gauge function $\chi(x) = x^0 \cdot x^i$ whose bare covariant
 $B^\mu := \partial_\mu \chi$ has a nonzero field-strength component (equal to 2), certi-
contraction in ofGradient is required for gauge invariance.

-/

/-- The $(\text{inl } 0, \text{inr } i)$ component of the field strength matrix of the bare
gradient potential
 $B^\mu := \partial_\mu \chi$ for $\chi(x) = x^0 \cdot x^i$ equals 2 . This witnesses that the bare
does not produce a gauge-invariant field strength, so the raised-
index contraction
 $\eta^{\{\mu\nu\}} \partial_\nu \chi$ in ofGradient is necessary (see the module overview). -
/
lemma fieldStrengthMatrix_bareGradient_inl_inr {d : ℕ} (i : Fin d)
 (x : SpaceTime d) :
 let χ : SpaceTime d $\rightarrow$ $\mathbb{R}$:= fun y => y (Sum.inl 0) * y (Sum.inr i)
 let B : ElectromagneticPotential d := {fun y μ => $\partial_\mu \chi$ y}
 B.fieldStrengthMatrix x (Sum.inl 0, Sum.inr i) = 2 := by
 intro χ B
 have h χ : ContDiff $\mathbb{R}$ 2 χ := by
 show ContDiff $\mathbb{R}$ 2 (fun y : SpaceTime d => y (Sum.inl 0) * y (Sum.inr i))
 fun_prop
 have hB : Differentiable $\mathbb{R}$ B := by
 rw [← SpaceTime.differentiable_vector]; intro μ
 exact SpaceTime.differentiable_deriv μ χ h χ
 -- fieldStrengthMatrix (μ , ν) = $\eta_{\mu\mu} * \partial_\mu B \times \nu - \eta_{\nu\nu} * \partial_\nu B \times \mu$
 rw [toFieldStrength_basis_repr_apply_eq_single]
 -- Expand $\partial_\mu B \times \nu$ as $\partial_\mu (\text{fun } y => \partial_\nu \chi \ y) \ x = \partial_\mu (\partial_\nu \chi) \ x$
 rw [SpaceTime.deriv_apply_eq (Sum.inl 0) (Sum.inr i) B hB,
 SpaceTime.deriv_apply_eq (Sum.inr i) (Sum.inl 0) B hB]
 -- Now compute the mixed partials of $\chi = t \cdot x^i$ using fderiv_fun_mul + deri
 -- $\partial_t \chi = x^i$, $\partial_{\{x^i\}} \chi = t$; so $\partial_{\{x^i\}}(\partial_t \chi) = 1$ and $\partial_t(\partial_{\{x^i\}} \chi) = 1$
 have hfderiv : $\forall (y : \text{SpaceTime } d),$
 fderiv $\mathbb{R}$ χ y = (y (Sum.inr i)) • Lorentz.Vector.coordCLM (Sum.inl 0)
 + (y (Sum.inl 0)) • Lorentz.Vector.coordCLM (Sum.inr i)
 have h : fderiv $\mathbb{R}$ (fun z : SpaceTime d => z (Sum.inl 0) * z (Sum.inr i))
 (y (Sum.inr i)) • Lorentz.Vector.coordCLM (Sum.inl 0) +
 (y (Sum.inl 0)) • Lorentz.Vector.coordCLM (Sum.inr i) := by
 have hmul := fderiv_fun_mul (□ := $\mathbb{R}$)

```

1.53.

PHYSLIB/ELECTROMAGNETISM/KINEMATICS/MAGNETICFIELD.LEAN 79

```

 (hc := (Lorentz.Vector.coordCLM (Sum.inl 0)).differentiableAt)
 (hd := (Lorentz.Vector.coordCLM (Sum.inr i)).differentiableAt) (x := y)
 simp only [ContinuousLinearMap.fderiv, Lorentz.Vector.coordCLM_apply] at hmul
 -- hmul: fderiv ... y = y (Sum.inl 0) • coordCLM (Sum.inr i)
 -- + y (Sum.inr i) • coordCLM (Sum.inl 0)
 -- goal: ... = y (Sum.inr i) • coordCLM (Sum.inl 0)
 -- + y (Sum.inl 0) • coordCLM (Sum.inr i)
 rw [hmul, add_comm]
 exact h
simp only [SpaceTime.deriv_eq, B]
simp_rw [hfderiv]
simp only [_root_.add_apply, FunLike.coe_smul,
 Pi.smul_apply, Lorentz.Vector.coordCLM_apply, smul_eq_mul, Lorentz.Vector.basis_
simp only [mul_ite, mul_one, mul_zero, ite_add, zero_add, if_true]
simp only [minkowskiMatrix.inl_0_inl_0, minkowskiMatrix.inr_i_inr_i]
simp only [reduceCtorEq, ↓reduceIte, add_zero]
simp only [Lorentz.Vector.fderiv_coord, Lorentz.Vector.coordCLM_apply,
 Lorentz.Vector.basis_apply, ↓reduceIte]
norm_num

/-- The field strength of the bare-gradient potential $B^\mu := \partial_\mu \chi$ for
 $\chi(x) = x^0 \cdot x^i$ is nonzero (follows from `fieldStrengthMatrix_bareGradient_inl_inr`
/
lemma toFieldStrength_bareGradient_ne_zero {d : ℕ} (i : Fin d)
 (x : SpaceTime d) :
 let χ : SpaceTime d → ℝ := fun y => y (Sum.inl 0) * y (Sum.inr i)
 let B : ElectromagneticPotential d := {fun y μ => $\partial_\mu \chi$ y}
 B.toFieldStrength x ≠ 0 := by
intro χ B h
have h2 := fieldStrengthMatrix_bareGradient_inl_inr i x
dsimp only at h2
rw [fieldStrengthMatrix_eq, h] at h2
have h3 : ((Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis).repr
 (0 : Lorentz.CoVector d ⊗ [ℝ] Lorentz.Vector d)) = 0 := map_zero _
erw [h3, Finsupp.zero_apply] at h2
norm_num at h2

end ElectromagneticPotential

end Electromagnetism

```

### 1.53 Physlib/Electromagnetism/Kinematics/MagneticField.lean

```

lemma magneticField_coord_eq_fieldStrengthMatrix {i : Fin 3} {c : SpeedOfLight}

```

```

 A.magneticField c t x i =
 - A.fieldStrengthMatrix ((toTimeAndSpace c).symm (t, x)) (Sum.inr (i+1))
 · fun_prop
simp only [magneticField_eq]
change Differentiable ℝ fun x =>
let C : (Space) × ℝ → EuclideanSpace ℝ (Fin 3) := fun p =>
 convert! h using 1
 rw [fieldStrengthMatrix_antisymm]
 fin_cases i <;> fin_cases j <;>
 simp [magneticField_coord_eq_fieldStrengthMatrix A t x hA]
 repeat rw [fieldStrengthMatrix_antisymm]
simp [magneticFieldMatrix_eq]
 A.magneticFieldMatrix c t x (i, j) = - A.magneticFieldMatrix c t x (j,
 fieldStrengthMatrix_antisymm A ((toTimeAndSpace c).symm (t, x)) (Sum.inr
 A.magneticFieldMatrix c t x (i, i) = 0 :=
fieldStrengthMatrix_diag_eq_zero A ((toTimeAndSpace c).symm (t, x)) (Sum.
 - A.magneticFieldMatrix c t x ((i+1), (i+2))) := by
ext t x
simp [magneticFieldMatrix_eq, magneticField_coord_eq_fieldStrengthMatrix
simp only [curl, Fin.isValue, deriv_eq_fderiv_basis, fderiv_fun_neg,
 root.neg_apply, sub_neg_eq_add, Fin.sum_univ_three]
fin_cases i <;>
· simp only [Fin.reduceFinMk, Fin.isValue, Fin.reduceAdd, zero_add,
 magneticFieldMatrix_diag_eq_zero, fderiv_fun_const, Pi.ofNat_apply,
 root.zero_apply, add_zero]
 simp [add_comm]
simp only [magneticFieldMatrix_eq]
 fun_prop
exact (fieldStrengthMatrix_contDiff hA).comp (toTimeAndSpace c).symm.contDiff
exact (magneticFieldMatrix_contDiff A hA ij).comp (f := fun x => (t, x))
exact (magneticFieldMatrix_contDiff A hA ij).comp (f := fun t => (t, x))
exact (fieldStrengthMatrix_differentiable hA).comp (toTimeAndSpace c).symm
exact (magneticFieldMatrix_differentiable A hA ij).comp (f := fun x => (t,
exact (magneticFieldMatrix_differentiable A hA ij).comp (f := fun t => (t,
rw [fun_deriv_sub, fun_deriv_sub, fun_deriv_sub]
rw [Space.deriv_commute _ (vectorPotential_apply_contDiff_space _ hA _ i)
 Space.deriv_commute _ (vectorPotential_apply_contDiff_space _ hA _ j)
 Space.deriv_commute _ (vectorPotential_apply_contDiff_space _ hA _ k)
· apply Space.deriv_differentiable
 rw [Space.time_deriv_comm_space_deriv _, Space.time_deriv_comm_space_
 have hφ := scalarPotential_contDiff_space c A hA t
 have hd1 : ∀ k : Fin d, DifferentiableAt ℝ (fun x => -
(A.electricField c t x).ofLp k) x :=
 fun k => (electricField_apply_differentiable_space hA t k).neg.diff
 have hd2 : ∀ k : Fin d, DifferentiableAt ℝ (Space.deriv k (scalarPote
 fun k => (Space.deriv_differentiable hφ k).differentiableAt

```

1.54.

*PHYSLIB/ELECTROMAGNETISM/KINEMATICS/VECTORPOTENTIAL.lean*

```
rw [Space.deriv_eq_fderiv_basis, fderiv_fun_sub (hd1 i) (hd2 i), fderiv_fun_ne
 rw [Space.deriv_eq_fderiv_basis, fderiv_fun_sub (hd1 j) (hd2 j), fderiv_fun_ne
 simp only [FunLike.coe_sub, Pi.sub_apply, _root_.neg_apply, ← Space.deriv_eq_f
 rw [Space.deriv_commute _ hφ]
rw [Space.time_deriv_comm_space_deriv _, Space.time_deriv_comm_space_deriv _]
all_goals first
| (funext x
 | rw [Time.deriv_euclid]
 | apply electricField_differentiable_time (hA.of_le (right_eq_inf.mp rfl)))
| apply electricField_apply_contDiff hA
| (apply Differentiable.differentiableAt
 | apply Space.space_deriv_differentiable_time
 | apply electricField_apply_contDiff hA)
```

### 1.54 Physlib/Electromagnetism/Kinematics/VectorPotential.lean

```
public import Mathlib.Algebra.Order.Archimedean.Real.Hom
(B : Time → Space 3 → EuclideanSpace ℝ (Fin 3)) (i : Fin 3) (hB : Continuous 1B)
simp only [fderiv_fun_const, Pi.zero_apply, _root_.zero_apply, inner_zero_right,
```

### 1.55 Physlib/Electromagnetism/PointParticle/OneDimension.lean

```
public import Physlib.SpaceAndTime.Space.Norm.Basic
ContinuousLinearEquiv.apply_symm_apply, FunLike.coe_smul,
ContinuousLinearMap.coe_comp, LinearMap.coe_toContinuousLinearMap', Pi.smul_apply
AddHom.coe_mk, ContinuousLinearEquiv.apply_symm_apply, FunLike.coe_smul,
ContinuousLinearMap.coe_comp, LinearMap.coe_toContinuousLinearMap', Pi.smul_apply
simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul,
 root.neg_apply]
have h1 := Space.distGrad_distOfFunction_norm_zpow (d := 1) 1 (by grind)
 FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
simp only [map_smul, FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
```

### 1.56 Physlib/Electromagnetism/PointParticle/ThreeDimension.lean

```
public import Physlib.SpaceAndTime.Space.Norm.Basic
ContinuousLinearEquiv.apply_symm_apply, FunLike.coe_smul,
ContinuousLinearMap.coe_comp, LinearMap.coe_toContinuousLinearMap', Pi.smul_apply
(((IsDistBounded.inv (by norm_num)).const_mul_fun _).smul_const _)) := by
AddHom.coe_mk, ContinuousLinearEquiv.apply_symm_apply, FunLike.coe_smul,
ContinuousLinearMap.coe_comp, LinearMap.coe_toContinuousLinearMap', Pi.smul_apply
simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
```

```

have := Space.distGrad_distOfFunction_norm_zpow (d := 3) (-
1) (by grind)
 FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
simp only [one_div, map_smul, FunLike.coe_smul, Pi.smul_apply, smul_eq_

```

### 1.57 Physlib/Electromagnetism/Vacuum/Constant.lean

```

simp only [one_div, FunLike.coe_smul, FunLike.coe_sum, Pi.smul_apply,

```

### 1.58 Physlib/Electromagnetism/Vacuum/HarmonicWave.lean

```

have transverse_deriv_eq_zero : ∀ (g : ℝ → ℝ), Differentiable ℝ g →
 Space.deriv j.succ (fun y => g (y 0)) x = 0 := by
 intro g hg
 rw [Space.deriv_eq, show (fun y : Space d.succ => g (y 0)) = g ∘ (fun
 fderiv_comp _ hg.differentiableAt (by fun_prop))]
 simp [← Space.deriv_eq, Space.deriv_component, Fin.succ_ne_zero]
simp only [harmonicWaveX_vectorPotential_succ', mul_one]
exact transverse_deriv_eq_zero
 (fun u => -E₀ {i, by grind} / (□.c.val * k) *
 sin (k * (t.val * □.c.val - u) + φ {i, by grind})) (by fun_prop)
simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
simp only [fderiv_add_const, FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
simp only [smul_neg, _root_.neg_apply, FunLike.coe_smul, Pi.smul_apply,
exact vectorPotential_differentiable_time _ (harmonicWaveX_differentiable
simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
simp only [FunLike.coe_smul, Pi.smul_apply, Time.fderiv_val, smul_eq_mul,
· exact vectorPotential_differentiable_time _ (harmonicWaveX_differentiab
 have transverse_deriv_cos_eq_zero : ∀ (C a k b : ℝ) (l : Fin d) (y : Sp
 Space.deriv l.succ (fun x => C * Real.cos (a - k * x 0 + b)) y = 0
 intro C a k b l y
 rw [Space.deriv_eq, show (fun x : Space d.succ => C * Real.cos (a - k
 = (fun u => C * Real.cos (a - k * u + b)) ∘ (fun x => x 0) from r
 fderiv_comp _ (by fun_prop) (by fun_prop))]
 simp [← Space.deriv_eq, Space.deriv_component, Fin.succ_ne_zero]
 apply transverse_deriv_cos_eq_zero
simp only [Nat.succ_eq_add_one, FunLike.coe_smul, Pi.smul_apply, smul_eq_
simp only [fderiv_add_const, neg_smul, _root_.neg_apply,
 FunLike.coe_smul, Pi.smul_apply, smul_eq_mul, mul_neg, neg_inj]
simp only [FunLike.coe_smul, Pi.smul_apply, Time.fderiv_val, smul_eq_mul,
 have transverse_deriv_cos_eq_zero : ∀ (C a k b : ℝ) (l : Fin d) (y : Sp
 Space.deriv l.succ (fun x => C * Real.cos (a - k * x 0 + b)) y = 0

```

```

intro C a k b l y
rw [Space.deriv_eq, show (fun x : Space d.succ => C * Real.cos (a - k * x 0 +
 = (fun u => C * Real.cos (a - k * u + b)) ◦ (fun x => x 0) from rfl,
 fderiv_comp _ (by fun_prop) (by fun_prop))]
simp [← Space.deriv_eq, Space.deriv_component, Fin.succ_ne_zero]
apply transverse_deriv_cos_eq_zero
have transverse_deriv_cos_eq_zero : ∀ (C a k b : ℝ) (l : Fin d) (y : Space d.succ
 Space.deriv l.succ (fun x => C * Real.cos (a - k * x 0 + b)) y = 0 := by
intro C a k b l y
rw [Space.deriv_eq, show (fun x : Space d.succ => C * Real.cos (a - k * x 0 +
 = (fun u => C * Real.cos (a - k * u + b)) ◦ (fun x => x 0) from rfl,
 fderiv_comp _ (by fun_prop) (by fun_prop))]
simp [← Space.deriv_eq, Space.deriv_component, Fin.succ_ne_zero]
apply transverse_deriv_cos_eq_zero
simp only [Nat.succ_eq_add_one, FunLike.coe_smul, Pi.smul_apply, smul_eq_mul,
simp only [fderiv_add_const, neg_smul, _root_.neg_apply,
 FunLike.coe_smul, Pi.smul_apply, smul_eq_mul, mul_neg]
simp only [_root_.neg_apply, mul_neg, neg_neg]
simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
 exact electricField_differentiable_time (harmonicWaveX_contDiff 2 □ k E₀ φ) x
tauto
 exact electricField_differentiable_time (harmonicWaveX_contDiff 2 □ k E₀ φ) x
simp only [cos_add, sin_sub, cos_sub]
nlinarith [sin_sq_add_cos_sq τ, sin_sq_add_cos_sq (φ i), sin_sq_add_cos_sq (φ j)]

```

## 1.59 Physlib/Electromagnetism/Vacuum/IsPlaneWave.lean

```

apply (electricField_differentiable hA).comp
refine Differentiable.prodMk ?_ ?_
· change Differentiable ℝ (Time.toRealCLE.symm ◦ fun u => -
u / □.c.val)
 apply Differentiable.comp
 · fun_prop
 · fun_prop
apply (magneticFieldMatrix_differentiable A hA ij).comp
refine Differentiable.prodMk ?_ ?_
· change Differentiable ℝ (Time.toRealCLE.symm ◦ fun u => -
u / □.c.val)
 apply Differentiable.comp
 · fun_prop
 · fun_prop
rw [fderiv_fun_comp]
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply, fderiv_eq_smul_deriv
simp only [fderiv_fun_const, Pi.zero_apply, zero_sub, _root_.neg_apply, neg_smul,

```

```

simp only [FunLike.coe_smul, Pi.smul_apply, Time.fderiv_val, smul_eq_mul]
· exact (P.electricFunction_differentiable hA).differentiableAt
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply, fderiv_eq_s
simp only [fderiv_fun_const, Pi.zero_apply, zero_sub, _root_.neg_apply, n
simp only [FunLike.coe_smul, Pi.smul_apply, Time.fderiv_val, smul_eq_mul]
· exact (magneticFunction_differentiable P hA (i, j)).differentiableAt
rw [fderiv_fun_comp]
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply, fderiv_eq_s
· exact (P.electricFunction_differentiable hA).differentiableAt
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply, fderiv_eq_s
· exact (magneticFunction_differentiable P hA (i, j)).differentiableAt
have hBd : $\forall k$, Differentiable $\mathbb{R}$ (fun t => A.magneticFieldMatrix $\square$.c t x
 fun k => magneticFieldMatrix_differentiable_time A (hA.of_le ENat.LEInf
 simp only [FunLike.coe_sum, Finset.sum_apply]
 rw [fderiv_mul_const (hBd _).differentiableAt]
 simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
 exact (hBd k).differentiableAt
 exact (hBd k).differentiableAt
 simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]

```

## 1.60 Physlib/FluidDynamics/NavierStokes/Momentum.lean

```

rw [← Space.deriv_smul (u := j) (x := x)
 ((differentiable_euclidean.mp hMomentumDensity j).differentiableAt)]
rw [conservativeMomentumLHS, convectiveMomentumLHS, continuityResidual,
 timeDeriv_momentumDensity d fluid t x hRhoTime hVelocityTime,
 matrixDiv_momentumFlux d fluid t x hMomentumDensity hVelocitySpace]
have conservative_eq_convective_lhs : $\forall t x$, conservativeMomentumLHS d da
 convectiveMomentumLHS d data.toFluidState t x := by
 intro t x
 hContinuity t x (by simp using hRhoTime t x) (hMomentumDensity t).
 rw [conservativeMomentumLHS_eq_convectiveMomentumLHS_add_continuityResi
 d data.toFluidState t x (hRhoTime t x) (hVelocityTime t x)
 (hMomentumDensity t) (hVelocitySpace t), hResidual, zero_smul, add_
 exact {fun h t x => (conservative_eq_convective_lhs t x).symm.trans (h t
 fun h t x => (conservative_eq_convective_lhs t x).trans (h t x))}

```

## 1.61 Physlib/Mathematics/Calculus/AdjFDeriv.lean

```

simp (disch:=fun_prop) [fderiv_fun_comp]

```



1.62.

PHYSLIB/MATHEMATICS/CALCULUS/PARAMETRICINTEGRATION.LEAN85

## 1.62 Physlib/Mathematics/Calculus/ParametricIntegration.lean

```
variable {M N : Type}
 [NormedAddCommGroup M] [NormedSpace ℝ M] [ProperSpace M]
 [NormedAddCommGroup N] [NormedSpace ℝ N]
 ((isCompact_closedBall x₀ 1).prod isCompact_Icc)
 (continuous_norm.comp hF').continuousOn
have hx := hf.differentiable (by simp)
 (F' := F') (bound := fun t => ||F' a.1 a.2||)
· exact Metric.closedBall_mem_nhds x₀ one_pos
· apply Continuous.aestronglyMeasurable
· filter_upwards with t h x hx
 exact ha.2 (Set.mk_mem_prod hx (Set.Ioc_subset_Icc_self (by simp using h)))
· exact intervalIntegrable_const
· filter_upwards with t h x hx
 exact DifferentiableAt.hasFDerivAt (by fun_prop)
rw [(hasFDerivAt_parametric_intervalIntegral_of_contDiff hf x₀).fderiv]
 fderiv ℝ (fun (x : M) => ∫ (t : ℝ) in 0..1, F x t ∂(volume)) =
exact (hasFDerivAt_parametric_intervalIntegral_of_contDiff hf x).fderiv
refine {_, ?, hasFDerivAt_parametric_intervalIntegral_of_contDiff hf}
fun_prop
· exact contDiff_one_parametric_intervalIntegral_of_contDiff hf
 refine {ContDiff.differentiable
 (contDiff_one_parametric_intervalIntegral_of_contDiff (hf.of_le (by simp))) (b
 WithTop.one_ne_top, or_self, IsEmpty.forall_iff, true_and, contDiff_clm_apply_
 fderiv_apply_parameteric_intervalIntegral (hf.of_le (by simp)))]
 exact fun y => ih (by fun_prop)
· exact contDiff_zero.mpr (by fun_prop)
```

## 1.63 Physlib/Mathematics/Calculus/Wirtinger/Basic.lean

```
/-
Copyright (c) 2026 Andrea Pari. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Andrea Pari
-/
module

public import Mathlib.Analysis.Calculus.FDeriv.Symmetric
public import Mathlib.Analysis.Complex.Basic

/-!

Wirtinger calculus
```

## ## Notation

- \* The differentiation direction is a *subscript*:  $\partial_v f$  and  $\partial_{\bar{v}} f$  are the anti-holomorphic Wirtinger derivatives of  $f$  in direction  $v$ , splitting the derivative  $d_v f$  (straight  $d$  for the total,  $\partial/\partial\bar{z}$  for the parts). For derivatives (§I) the operators compose,  $\partial_v \partial_{\bar{w}} f$ .
- \* A  $\partial/\partial\bar{z}$  fraction differentiates with respect to a *variable*, not a direct coordinate  $\partial f/\partial x$ , or the argument of a one-variable function in the chain rule  $\partial g/\partial \bar{f}$  (outer  $g : \mathbb{C} \rightarrow \mathbb{C}$ , inner  $f : V \rightarrow \mathbb{C}$ ).
- \*  $\bar{f}$  is the pointwise conjugate  $p \mapsto \text{conj}(f(p))$ .
- \*  $v, w$  are directions in  $V$ .
- \*  $u : V$  is the *fixed* base point a derivative is evaluated at – the implied subscript notation.
- \*  $p : V$  is the *bound* base-point variable when a derivative is repackaged in position: the inner field of an iterated operator (`fun p => dWirtingerAn` or the composite in the chain rule (`fun p => g (f p)`, §D).

Base points ( $u, p$ ) and directions ( $v, w$ ) all live in  $V$ : a vector space tangent space, so a displacement from a point is again an element of  $V$  ( $v$ ).

## ## i. Overview

This module is the *foundation* of physlib's Wirtinger calculus. It defines *directional Wirtinger derivatives* of  $f : V \rightarrow \mathbb{C}$  on a complex vector space in direction  $v : V$  (a complex number when  $V = \mathbb{C}$ , a vector in general):

$$\begin{aligned}\partial_v f &= (1/2)(d_v f - i \cdot d_{i \cdot v} f) && (\text{dWirtingerDir}) \\ \partial_{\bar{v}} f &= (1/2)(d_v f + i \cdot d_{i \cdot v} f) && (\text{dWirtingerAntiDir})\end{aligned}$$

Here  $d_v f = \text{fderiv } \mathbb{R} f u v$  is the real Fréchet derivative along  $v$ , the  $\lim_{t \rightarrow 0} (f(u + t \cdot v) - f(u)) / t$  over real  $t$ ; so "real" names the scalar direction  $v$ , and over all  $v$  these limits form the  $\mathbb{R}$ -linear map  $\text{fderiv } \mathbb{R} f u : V \rightarrow \mathbb{C}$ .

The second direction  $i \cdot v$  is  $v$  turned 90° by the complex structure on  $V$ .  $i \cdot v$  is an orthogonal frame in  $v$ 's own (arbitrary) direction, a rotated and rescaled axes  $(1, i)$ . For  $V = \mathbb{C}$  one may take  $v = 1$ , giving  $d_v f = \partial f/\partial x$  and  $d_{i \cdot v} f = \partial f/\partial y$ ; writing  $z = x + i y$  and  $\bar{z} = x - i y$ , the formulas become classical  $\partial f/\partial z = (1/2)(\partial_x - i \partial_y)f$  and  $\partial f/\partial \bar{z} = (1/2)(\partial_x + i \partial_y)f$ .

The two operators measure the failure of  $\mathbb{C}$ -linearity. The real derivative commutes with real scaling and addition;  $\mathbb{C}$ -linearity asks in addition that it commutes with  $i$ :  $d_{i \cdot v} f = i \cdot d_v f$ . The gap  $d_{i \cdot v} f - i \cdot d_v f$  to that condition is  $-2i \cdot \partial_{\bar{v}} f$ , so  $\partial_{\bar{v}} f$  is the obstruction to  $\mathbb{C}$ -linearity and vanishes precisely when  $f$  is holomorphic. Equivalently, the operators split the real derivative into

anti-holomorphic parts, the directional form of the Dolbeault decomposition  $d = \partial + \bar{\partial}$  which sum back to

$$d_v f = \partial_v f + \bar{\partial}_v f.$$

This is the coordinate-free form of treating  $z$  and  $\bar{z}$  as independent (the  $V = \mathbb{C}$  above). When  $f$  is holomorphic the anti-holomorphic half vanishes and  $\partial_v f$  is the complex derivative (§F). Everything rests on  $f \text{deriv } \mathbb{R}$ , with no lower Wirtinger layer.

On these operators the module builds the **full directional calculus**:

- \* real-linearity, the Leibniz rule, and the finite-sum rule (§B);
- \* the inner-field conjugation lemmas, swapping the two operators (§C);
- \* the two-term Wirtinger chain rule for an outer  $g : \mathbb{C} \rightarrow \mathbb{C}$  (§D);
- \* domain conjugation: precomposing with a conjugate-linear map swaps the two operators (§E);
- \* the holomorphic / anti-holomorphic collapse, keyed on  $\mathbb{C}$ -linearity or conjugate-linearity of the real derivative along  $v$  (§F);
- \* differentiability and locality of the operators on a  $\mathbb{C}^2$  field (§H).

The capstone (§I) is **Schwarz's theorem** in Wirtinger form. On a  $\mathbb{C}^2$  field the holomorphic and anti-holomorphic derivatives in any two directions commute:

$$\partial_v \bar{\partial}_w f = \bar{\partial}_w \partial_v f \quad (\text{dWirtingerDir\_dWirtingerAntiDir\_comm})$$

It is no new analytic fact: it reduces to the symmetry of the second real Fréchet derivative (`ContDiffAt.isSymmSndFDerivAt`), carried out via the `weightedDirDeriv` of §G.

## ## ii. Key results

- `Physlib.Wirtinger.dWirtingerDir` / `dWirtingerAntiDir` : directional Wirtinger derivatives of  $f : V \rightarrow \mathbb{C}$  along  $v$ .
- `Physlib.Wirtinger.dWirtingerDir_add` / `dWirtingerDir_smul` / `dWirtingerDir_mul` / `dWirtingerDir_fun_sum` : real-linearity, the Leibniz rule, and the finite-sum rule (each with an anti-holomorphic dual).
- `Physlib.Wirtinger.dWirtingerDir_star_comp` / `dWirtingerAntiDir_star_comp` : conjugating the inner field swaps the holomorphic and anti-holomorphic operators.
- `Physlib.Wirtinger.dWirtingerDir_comp` / `dWirtingerAntiDir_comp` : the two-term Wirtinger chain rule for an outer  $g : \mathbb{C} \rightarrow \mathbb{C}$ .
- `Physlib.Wirtinger.dWirtingerDir_comp_conjLinear` / `dWirtingerAntiDir_comp_conjLinear` : precomposing with a conjugate-

``C`-linear`  
 map swaps the two operators (with the base point and direction transported through the map).  
 - ``Physlib.Wirtinger.dWirtingerDir_eq_of_clinear` /`  
``dWirtingerAntiDir_eq_zero_of_clinear`` : the holomorphic collapse, key to ``C``-linearity of the real derivative along the direction (each with a conjugate-``C``-linear dual).  
 - ``Physlib.Wirtinger.differentiableAt_dWirtingerDir` /`  
``differentiableAt_dWirtingerAntiDir`` : the directional derivative of a field is itself real-differentiable.  
 - ``Physlib.Wirtinger.dWirtingerDir_congr_of_eventuallyEq` /`  
``dWirtingerAntiDir_congr_of_eventuallyEq`` : the directional derivative only on the field near the point.  
 - ``Physlib.Wirtinger.dWirtingerDir_dWirtingerAntiDir_comm`` : Schwarz's theorem  $\partial_v \partial_w f = \partial_w \partial_v f$  for a ``C2`` ``f``.  
 - ``Physlib.Wirtinger.realLinear_apply_eq_wirtinger`` : the real-linear Wirtinger split  
``L w = a · w + b · star w`` of any ``L : C → L[R] C``, the algebraic input to the  
 - ``Physlib.Wirtinger.fderiv_star_eq`` : the real derivative of a pointwise complex ``f``  
``p ↦ star (f p)`` is ``conjCLE`` composed with ``fderiv ℝ f``, the analytic conjugation lemmas (§C).

### ## iii. Table of contents

- A. The directional Wirtinger operators
- B. Real-linearity and the Leibniz rule
- C. Conjugation
- D. The Wirtinger chain rule
- E. Domain conjugation
- F. The holomorphic collapse
- G. The second-derivative bridge
- H. Differentiability and locality
- I. Schwarz's theorem

### ## iv. References

- Kreutz-Delgado, *\*The Complex Gradient Operator and the CR-Calculus\**,  
arXiv:0906.4835 – directional/multivariable formulation and two-term chain rule (§D); second-order theory behind §G–I.
- Mortini & Rupp, *\*The Clairaut–Schwarz Theorem for Mixed Wirtinger Derivatives\**, Bull. Iranian Math. Soc. 48 (2022), 2643–2647 – the mixed holomorphic/anti-holomorphic symmetry of §I under the same ``C2`` hypothesis with the same reduction to real Schwarz used here.
- Koor, Qiu, Kwek & Rebentrost, *\*A short tutorial on Wirtinger Calculus with*

```

 applications in quantum information*, arXiv:2312.04858 – companion
 exposition of the scalar single/multivariable calculus and sign conventions.
 - *Complex differential form*, Wikipedia (section "The Dolbeault operators") – the
 $d = \partial + \bar{\partial}$ splitting and the $\partial/\bar{\partial}$ notation this module's operators are named a

-/

@[expose] public section

noncomputable section

namespace Physlib.Wirtinger

variable {V : Type*} [NormedAddCommGroup V] [NormedSpace ℝ V] [NormedSpace ℂ V]
 {f : V → ℂ} {u : V}

/-!

A. The directional Wirtinger operators

The two directional operators repackage the real Fréchet derivative of f along
 v and $i \cdot v$ into a holomorphic part $\partial_v f$ and an anti-
holomorphic part $\bar{\partial}_v f$,
the combinations $(1/2)(d_v f \mp i \cdot d_{i \cdot v} f)$. Both share one shape, weightedDirDeriv
the base-point field $p \mapsto (1/2)(d_{b_1} f + c \cdot d_{b_2} f)$, and are its two specializations
at $c = \mp i$, $(b_1, b_2) = (v, i \cdot v)$. Defining the operators through it makes their second
derivative – the engine of Schwarz's theorem (§G, §I) – a single bridge lemma. Both
 $\mathbb{C}$ -valued and depend on the base point u .

-/

/-- The base-point field $p \mapsto (1/2)(d_{b_1} f + c \cdot d_{b_2} f)$, a weighted combination
real Fréchet derivative of f along two directions b_1 , b_2 . The directional Wirtinger
operators are its two specializations: $\text{dWirtingerDir } f \ v$ at $c = -i$,
 $(b_1, b_2) = (v, i \cdot v)$,
and $\text{dWirtingerAntiDir } f \ v$ at $c = i$. Keeping b_1 , b_2 free lets §G differentiate
second time once and reuse the bridge for both operators. -
/
def weightedDirDeriv (f : V → ℂ) (c : ℂ) (b₁ b₂ : V) : V → ℂ :=
 fun p => (1 / 2 : ℂ) * (fderiv ℝ f p b₁ + c * fderiv ℝ f p b₂)

/-- The holomorphic directional Wirtinger derivative $\partial_v f = (1/2)(d_v f - i \cdot d_{i \cdot v} f)$
of $f : V \rightarrow \mathbb{C}$ along the direction vector $v : V$, the weightedDirDeriv at $c = -i$. -/
def dWirtingerDir (f : V → ℂ) (v u : V) : ℂ :=
 weightedDirDeriv f (-Complex.I) v (Complex.I • v) u

```

```

/-- The anti-holomorphic directional Wirtinger derivative
`∂_v f = (1/2)(d_v f + i·d_{i·v} f)` of `f : V → ℂ` along the direction vec
the `weightedDirDeriv` at `c = i`. -/
def dWirtingerAntiDir (f : V → ℂ) (v u : V) : ℂ :=
 weightedDirDeriv f Complex.I v (Complex.I • v) u

/-- Definitional unfolding of `dWirtingerDir` to the explicit Wirtinger com
expand the outer operator of a composition without touching the inner one.
/
lemma dWirtingerDir_apply (g : V → ℂ) (v u : V) :
 dWirtingerDir g v u
 = (1 / 2 : ℂ) * (fderiv ℝ g u v - Complex.I * fderiv ℝ g u (Complex.I • v))
 simp only [dWirtingerDir, weightedDirDeriv]; ring

/-- Definitional unfolding of `dWirtingerAntiDir` to the explicit Wirtinger
/
lemma dWirtingerAntiDir_apply (g : V → ℂ) (v u : V) :
 dWirtingerAntiDir g v u
 = (1 / 2 : ℂ) * (fderiv ℝ g u v + Complex.I * fderiv ℝ g u (Complex.I • v))
 simp only [dWirtingerAntiDir, weightedDirDeriv]

/-!

B. Real-linearity and the Leibniz rule

The directional operators are built from `fderiv ℝ`, so they inherit its
vanishing on constants, additivity, negation, complex-scalar compatibility,
finite-sum rule, and – through the Fréchet product rule – a Wirtinger Leibniz
rule.

-/

/-- Constants have zero holomorphic directional Wirtinger derivative, `∂_v`
/
@[simp] lemma dWirtingerDir_const (c : ℂ) (v u : V) :
 dWirtingerDir (fun _ : V => c) v u = 0 := by
 simp [dWirtingerDir_apply, fderiv_const_apply]

/-- Constants have zero anti-holomorphic directional Wirtinger derivative,
/
@[simp] lemma dWirtingerAntiDir_const (c : ℂ) (v u : V) :
 dWirtingerAntiDir (fun _ : V => c) v u = 0 := by
 simp [dWirtingerAntiDir_apply, fderiv_const_apply]

/-- `dWirtingerDir` of a negated function, `∂_v(-g) = -∂_v g`. Holds with n

```

```

differentiability hypothesis, since `fderiv` of a negation is unconditional. -
/
@[simp] lemma dWirtingerDir_neg (g : V → ℂ) (v u : V) :
 dWirtingerDir (fun p => -(g p)) v u = -(dWirtingerDir g v u) := by
 simp only [dWirtingerDir_apply, fderiv_fun_neg, _root_.neg_apply]; ring

/-- `dWirtingerAntiDir` of a negated function, $\partial_v(-g) = -\partial_v g$. -
/
@[simp] lemma dWirtingerAntiDir_neg (g : V → ℂ) (v u : V) :
 dWirtingerAntiDir (fun p => -(g p)) v u = -(dWirtingerAntiDir g v u) := by
 simp only [dWirtingerAntiDir_apply, fderiv_fun_neg, _root_.neg_apply]; ring

/-- Additivity of `dWirtingerDir`, $\partial_v(g + h) = \partial_v g + \partial_v h$. -
/
lemma dWirtingerDir_add {g h : V → ℂ} (hg : DifferentiableAt ℝ g u)
 (hh : DifferentiableAt ℝ h u) (v : V) :
 dWirtingerDir (g + h) v u = dWirtingerDir g v u + dWirtingerDir h v u := by
 simp only [dWirtingerDir_apply, fderiv_add hg hh, add_apply]; ring

/-- Additivity of `dWirtingerAntiDir`, $\partial_v(g + h) = \partial_v g + \partial_v h$. -
/
lemma dWirtingerAntiDir_add {g h : V → ℂ} (hg : DifferentiableAt ℝ g u)
 (hh : DifferentiableAt ℝ h u) (v : V) :
 dWirtingerAntiDir (g + h) v u = dWirtingerAntiDir g v u + dWirtingerAntiDir h v u := by
 simp only [dWirtingerAntiDir_apply, fderiv_add hg hh, add_apply]; ring

/-- Compatibility of `dWirtingerDir` with complex scalar multiplication,
 $\partial_v(c \cdot g) = c \cdot \partial_v g$. -/
lemma dWirtingerDir_smul (c : ℂ) {g : V → ℂ} (hg : DifferentiableAt ℝ g u) (v : V) :
 dWirtingerDir (c • g) v u = c • dWirtingerDir g v u := by
 simp only [dWirtingerDir_apply, fderiv_const_smul hg c, _root_.smul_apply,
 smul_eq_mul]; ring

/-- Compatibility of `dWirtingerAntiDir` with complex scalar multiplication,
 $\partial_v(c \cdot g) = c \cdot \partial_v g$. -/
lemma dWirtingerAntiDir_smul (c : ℂ) {g : V → ℂ} (hg : DifferentiableAt ℝ g u) (v : V) :
 dWirtingerAntiDir (c • g) v u = c • dWirtingerAntiDir g v u := by
 simp only [dWirtingerAntiDir_apply, fderiv_const_smul hg c, _root_.smul_apply,
 smul_eq_mul]; ring

omit [NormedSpace ℂ V] in
/-- The real Fréchet derivative of a product, evaluated at a tangent `v`. -
/
private lemma fderiv_mul_apply {g h : V → ℂ} (hg : DifferentiableAt ℝ g u)
 (hh : DifferentiableAt ℝ h u) (v : V) :
 fderiv ℝ (g * h) u v = g u * fderiv ℝ h u v + h u * fderiv ℝ g u v := by

```

```

simp using DFunLike.congr_fun (fderiv_mul hg hh) v

/-- The Wirtinger Leibniz rule for `dWirtingerDir`,
`∂_v(g·h) = ∂_v g·h + g·∂_v h`. -/
lemma dWirtingerDir_mul {g h : V → ℂ} (hg : DifferentiableAt ℝ g u)
 (hh : DifferentiableAt ℝ h u) (v : V) :
 dWirtingerDir (g * h) v u = dWirtingerDir g v u * h u + g u * dWirtingerDir h v u := by
 simp only [dWirtingerDir_apply, fderiv_mul_apply hg hh]; ring

/-- The Wirtinger Leibniz rule for `dWirtingerAntiDir`,
`∂_v(g·h) = ∂_v g·h + g·∂_v h`. -/
lemma dWirtingerAntiDir_mul {g h : V → ℂ} (hg : DifferentiableAt ℝ g u)
 (hh : DifferentiableAt ℝ h u) (v : V) :
 dWirtingerAntiDir (g * h) v u =
 dWirtingerAntiDir g v u * h u + g u * dWirtingerAntiDir h v u := by
 simp only [dWirtingerAntiDir_apply, fderiv_mul_apply hg hh]; ring

/-- Finite-sum rule for `dWirtingerDir`, `∂_v(∑_a F_a) = ∑_a ∂_v F_a`. -
/
lemma dWirtingerDir_fun_sum {α : Type*} {s : Finset α} {F : α → V → ℂ}
 (hF : ∀ a ∈ s, DifferentiableAt ℝ (F a) u) (v : V) :
 dWirtingerDir (fun p => ∑ a ∈ s, F a p) v u = ∑ a ∈ s, dWirtingerDir (F a) v u := by
 simp only [dWirtingerDir_apply, fderiv_fun_sum hF, sum_apply]
 rw [Finset.mul_sum, ← Finset.sum_sub_distrib, Finset.mul_sum]

/-- Finite-sum rule for `dWirtingerAntiDir`, `∂_v(∑_a F_a) = ∑_a ∂_v F_a`. -
/
lemma dWirtingerAntiDir_fun_sum {α : Type*} {s : Finset α} {F : α → V → ℂ}
 (hF : ∀ a ∈ s, DifferentiableAt ℝ (F a) u) (v : V) :
 dWirtingerAntiDir (fun p => ∑ a ∈ s, F a p) v u = ∑ a ∈ s, dWirtingerAntiDir (F a) v u := by
 simp only [dWirtingerAntiDir_apply, fderiv_fun_sum hF, sum_apply]
 rw [Finset.mul_sum, ← Finset.sum_add_distrib, Finset.mul_sum]

/-!

C. Conjugation

Conjugating the inner field `f` swaps the two operators, up to an outer conjugate value (`fderiv_star_eq`):

$$\partial_v \bar{f} = \text{conj}(\partial_v f), \quad \partial_v f = \text{conj}(\partial_v \bar{f}).$$

Each operator applied to the conjugate field $\bar{f}$ returns the other operator conjugated – the bar exchanges holomorphic and anti-holomorphic dependence.
`V = ℂ` take the holomorphic `f(z) = z`, with `∂_z z = 1`, `∂_z̄ z = 0`:

```



\*  $\partial_v \bar{f} = \text{conj} (\partial_v f)$  reads  $\partial_z \bar{z} = \text{conj } 0 = 0$  – the conjugate  $\bar{z}$  has no holomorphic part;  
 \* the dual  $\partial_v \bar{f} = \text{conj} (\partial_v f)$  reads  $\partial_z \bar{z} = \text{conj } 1 = 1$  – all of  $\bar{z}$ 's dependence sits in the anti-holomorphic operator.

The chain rule of  $\$D$  builds on these to handle a conjugated inner argument.

-/

/-- Differentiation commutes with conjugation: the real Fréchet derivative of the pointwise conjugate  $p \mapsto \text{star} (f p)$  is  $\text{conjCLE}$  (conjugation on  $\mathbb{C}$ ) composed with  $\text{fderiv } \mathbb{R} f u$ ; in physicists' notation,  $d \bar{f} = \text{conj}(d f)$ . Conjugation is  $\mathbb{R}$ -linear,

so it slides through the real derivative unchanged, whereas it does *not* commute with the holomorphic Wirtinger derivative  $\partial_v$ . The  $\text{star}$  conjugates the *output*  $f p$  so this is not a derivative in a conjugate variable.

This is the analytic core of the conjugation lemmas below ( $\text{dWirtingerDir\_star\_comp}$  and its dual): distributed over the Wirtinger split of  $\text{fderiv } \mathbb{R} f u$  ( $\text{realLinear\_apply\_eq\_wirtinger}$ ,  $\$D$ ), the outer  $\text{conjCLE}$  conjugates coefficients and swaps the holomorphic and anti-holomorphic parts. -

```
lemma fderiv_star_eq {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
 {f : E → ℂ} {u : E} (hf : DifferentiableAt ℝ f u) :
 fderiv ℝ (fun p : E => star (f p)) u =
 Complex.conjCLE.toContinuousLinearMap.comp (fderiv ℝ f u) := by
 rw [show (fun p : E => star (f p)) = Complex.conjCLE.toContinuousLinearMap ∘ f from
 fderiv_comp u Complex.conjCLE.toContinuousLinearMap.differentiableAt hf,
 ContinuousLinearMap.fderiv]
```

/-- Conjugating the function swaps the operators up to an outer conjugation:

$\partial_v \bar{f} = \text{conj} (\partial_v f)$ . -/

```
lemma dWirtingerDir_star_comp (hf : DifferentiableAt ℝ f u) (v : V) :
 dWirtingerDir (fun p => star (f p)) v u = star (dWirtingerAntiDir f v u) := by
 simp only [dWirtingerDir_apply, dWirtingerAntiDir_apply]
 rw [fderiv_star_eq hf]
 simp only [ContinuousLinearMap.comp_apply, ContinuousLinearEquiv.coe_coe,
 Complex.conjCLE_apply, Complex.star_def, map_mul, map_add, map_div, map_one,
 map_ofNat, Complex.conj_I]
 ring
```

/-- Conjugating the function swaps the operators up to an outer conjugation:

$\partial_v \bar{f} = \text{conj} (\partial_v f)$ . Dual of  $\text{dWirtingerDir\_star\_comp}$ . -

/

```
lemma dWirtingerAntiDir_star_comp (hf : DifferentiableAt ℝ f u) (v : V) :
 dWirtingerAntiDir (fun p => star (f p)) v u = star (dWirtingerDir f v u) := by
```

```

simp only [dWirtingerDir_apply, dWirtingerAntiDir_apply]
rw [fderiv_star_eq hf]
simp only [ContinuousLinearMap.comp_apply, ContinuousLinearEquiv.coe_coe,
 Complex.conjCLE_apply, Complex.star_def, map_mul, map_sub, map_div, map_ofNat, Complex.conj_I]
ring

```

```

/-!

```

```

D. The Wirtinger chain rule

```

Composing with an outer  $g : \mathbb{C} \rightarrow \mathbb{C}$  gives a **two-term** chain rule:

$$\partial_v(g \circ f) = (\partial g / \partial f) \cdot \partial_v f + (\partial g / \partial \bar{f}) \cdot \partial_v \bar{f}.$$

A non-holomorphic  $g$  depends on both its argument and its conjugate, so both contribute: the holomorphic  $\partial g / \partial f$  and the anti-holomorphic  $\partial g / \partial \bar{f}$ , each matching inner derivative – two terms where the complex-analytic rule has one. The coefficients come from `realLinear_apply_eq_wirtinger`: every  $\mathbb{R}$ -linear  $L : \mathbb{C} \rightarrow \mathbb{C}$

splits as  $L w = a \cdot w + b \cdot \text{conj } w$ , and on the outer real derivative  $L = \text{fderiv } \mathbb{R} g (f u)$  that gives  $a = \partial g / \partial f$ ,  $b = \partial g / \partial \bar{f}$ . The proof applies this split to the outer derivative, reuses the  $\mathbb{C}$  conjugation lemmas for the  $\partial_v \bar{f}$  term.

```

-/

```

```

/-- Split a real-linear map $\mathbb{C} \rightarrow \mathbb{C}$ into its Wirtinger components. Any
real-linear $L : \mathbb{C} \rightarrow \mathbb{C}$ splits into a holomorphic and an anti-
holomorphic part
with the Wirtinger coefficients $a = \frac{1}{2}(L 1 - i * L i)$, $b = \frac{1}{2}(L 1 + i * L i)$
weights:

```

$$L w = a * w + b * \text{star } w.$$

This is purely algebraic:  $L$  is an arbitrary real-linear map, no derivative involved. Its use is the Wirtinger chain rule (`dWirtingerDir_comp` below), where the weights of the outer differential  $L = \text{fderiv } \mathbb{R} g (f u)$  are the coefficients  $\partial g / \partial f$ . -/

```

lemma realLinear_apply_eq_wirtinger (L : $\mathbb{C} \rightarrow \mathbb{C}$) (w : $\mathbb{C}$) :
 L w =
 ((1 / 2 : $\mathbb{C}$) * (L 1 - Complex.I * L Complex.I)) * w
 + ((1 / 2 : $\mathbb{C}$) * (L 1 + Complex.I * L Complex.I)) * star w := by
calc
 L w = L ((w.re : $\mathbb{R}$) • (1 : $\mathbb{C}$) + (w.im : $\mathbb{R}$) • Complex.I) := by
 congr 1; apply Complex.ext <=> simp
 _ = (w.re : $\mathbb{R}$) • L 1 + (w.im : $\mathbb{R}$) • L Complex.I := by

```

```

 rw [map_add, map_smul, map_smul]
 _ = ((1 / 2 : ℂ) * (L 1 - Complex.I * L Complex.I)) * w
 + ((1 / 2 : ℂ) * (L 1 + Complex.I * L Complex.I)) * star w := by
 apply Complex.ext <=>
 simp [Complex.add_re, Complex.add_im, Complex.sub_re, Complex.sub_im,
 Complex.mul_re, Complex.mul_im, Complex.conj_re, Complex.conj_im,
 Complex.I_re, Complex.I_im] <=>
 ring

/-- The two-term Wirtinger chain rule for `dWirtingerDir`, outer `g : ℂ → ℂ` and inner
`f : V → ℂ`:

$$\partial_v(g \circ f) = (\partial g / \partial f) \cdot \partial_v f + (\partial g / \partial \bar{f}) \cdot \partial_v \bar{f}.$$

`realLinear_apply_eq_wirtinger` splits the chain rule's outer `ℝ`-
linear factor into the
`∂g/∂f`, `∂g/∂f̄` coefficients, each multiplying its inner directional derivative `∂_v f`,
`∂_v f̄`. -/
lemma dWirtingerDir_comp {g : ℂ → ℂ} (hg : DifferentiableAt ℝ g (f u))
 (hf : DifferentiableAt ℝ f u) (v : V) :
 dWirtingerDir (fun p => g (f p)) v u =
 dWirtingerDir g 1 (f u) * dWirtingerDir f v u
 + dWirtingerAntiDir g 1 (f u) * dWirtingerDir (fun p => star (f p)) v u := b
 simp only [dWirtingerDir_apply, dWirtingerAntiDir_apply, smul_eq_mul, mul_one]
 rw [show (fun p => g (f p)) = g ∘ f from rfl, fderiv_comp u hg hf, fderiv_star_eq]
 simp only [ContinuousLinearMap.comp_apply]
 have hA := realLinear_apply_eq_wirtinger (fderiv ℝ g (f u)) (fderiv ℝ f u v)
 have hB := realLinear_apply_eq_wirtinger (fderiv ℝ g (f u)) (fderiv ℝ f u (Complex
 rw [hA, hB]
 simp only [ContinuousLinearEquiv.coe_coe, Complex.conjCLE_apply, Complex.star_def]
 ring

/-- The two-term Wirtinger chain rule for `dWirtingerAntiDir`, the anti-
holomorphic dual of
`dWirtingerDir_comp`:

$$\partial_v(g \circ f) = (\partial g / \partial f) \cdot \partial_v f + (\partial g / \partial \bar{f}) \cdot \partial_v \bar{f}.$$

Same outer `∂g/∂f`, `∂g/∂f̄` coefficients, now each multiplying its anti-
holomorphic inner
derivative `∂_v f`, `∂_v f̄`; same proof as `dWirtingerDir_comp`. -
/
lemma dWirtingerAntiDir_comp {g : ℂ → ℂ} (hg : DifferentiableAt ℝ g (f u))
 (hf : DifferentiableAt ℝ f u) (v : V) :
 dWirtingerAntiDir (fun p => g (f p)) v u =
 dWirtingerDir g 1 (f u) * dWirtingerAntiDir f v u

```

```

 + dWirtingerAntiDir g 1 (f u) * dWirtingerAntiDir (fun p => star (f
simp only [dWirtingerDir_apply, dWirtingerAntiDir_apply, smul_eq_mul, mul
rw [show (fun p => g (f p)) = g ∘ f from rfl, fderiv_comp u hg hf, fderiv
simp only [ContinuousLinearMap.comp_apply]
have hA := realLinear_apply_eq_wirtinger (fderiv ℝ g (f u)) (fderiv ℝ f u)
have hB := realLinear_apply_eq_wirtinger (fderiv ℝ g (f u)) (fderiv ℝ f u)
rw [hA, hB]
simp only [ContinuousLinearEquiv.coe_coe, Complex.conjCLE_apply, Complex.
ring

```

```

/-!

```

### ## E. Domain conjugation

The goal is to differentiate anti-holomorphic functions: a holomorphic  $g$  conjugation of its input (the scalar case is  $g(\bar{z})$ , but the input is a general vector space  $V$ ). This section conjugates a function's *input*: precomposing  $g$  with a domain map  $L : V \rightarrow V$  (forming  $g \circ L$ ) swaps the two operators, whereas  $\mathbb{C}$  conjugation  $\text{conj}$  swaps the two operators. The map  $L$  is **conjugate- $\mathbb{C}$ -linear**: real-linear and continuous, but anti-commuting with  $i$ :

$$L(i \cdot x) = -(i \cdot L x),$$

the abstract form of  $\text{conj}(i \cdot x) = -i \cdot \text{conj } x$ . That sign flip swaps the two operators. The holomorphic combination  $(1/2)(d_v f - i \cdot d_{\{i \cdot v\}} f)$  the  $d_{\{i \cdot v\}} f$  term minus from  $L$ , turning it anti-holomorphic. So precomposition  $g \circ L$  swaps  $g$ 's derivative taken at the mapped point  $L u$  in the mapped direction  $L v$ :

$$\begin{aligned} \partial_v(g \circ L) \text{ at } u &= \partial_{\{L v\}} g \text{ at } L u \\ \partial_{\bar{v}}(g \circ L) \text{ at } u &= \partial_{\{L v\}} g \text{ at } L u \end{aligned}$$

(`dWirtingerDir_comp_conjLinear` and its dual `dWirtingerAntiDir_comp_conjL`)

Concretely on  $\mathbb{C}$ , let  $L : z \mapsto \bar{z}$  and  $g(z) = \log(z)$ , so  $g \circ L$  is  $z \mapsto \log(\bar{z})$ . The theorem computes this composite's derivatives from the known derivative of  $g$  at the operator. Its anti-holomorphic derivative is  $\log$ 's ordinary derivative at the mapped point  $\bar{z}$ :

$$\partial_{\bar{z}} \log(\bar{z}) = 1/\bar{z},$$

while its holomorphic derivative vanishes,  $\partial_z \log(\bar{z}) = 0$ , because  $\log$  is holomorphic. So  $\log(\bar{z})$  is purely anti-holomorphic, with its dependence carried by  $\partial_{\bar{z}}$ .

So precomposing with conjugation turns  $\partial$  into  $\partial^*$  and vice versa: a holomorphic function has zero holomorphic derivative, and its anti-holomorphic derivative is just its ordinary derivative.

complex derivative ( $\$F$ , and the example above). The proof uses only  $\mathbb{L}$ 's anti-commutation

with  $\mathbb{i}$ , so it holds over any complex  $\mathbb{V}$ ,  $\mathbb{V}'$ .

-/

section DomainConjugation

variable {V' : Type\*} [NormedAddCommGroup V'] [NormedSpace ℝ V'] [NormedSpace ℂ V']

omit [NormedSpace ℂ V] [NormedSpace ℂ V'] in

/-- Chain rule for an inner continuous linear map  $\mathbb{L}$ . Because the derivative of a linear map is the map itself, the real Fréchet derivative of  $g \circ \mathbb{L}$  at  $u$ , applied to  $x$  is the derivative of  $g$  at  $\mathbb{L} u$  applied to  $\mathbb{L} x$ . -/

```
private lemma fderiv_comp_clm_apply {g : V' → ℂ} {L : V → L[ℝ] V'} {u : V}
 (hg : DifferentiableAt ℝ g (L u)) (x : V) :
 fderiv ℝ (fun p => g (L p)) u x = fderiv ℝ g (L u) (L x) := by
 rw [show (fun p => g (L p)) = g ∘ (L : V → V') from rfl,
 fderiv_comp u hg L.differentiableAt, ContinuousLinearMap.fderiv,
 ContinuousLinearMap.comp_apply]
```

/-- Domain conjugation swaps the operators: precomposing with a conjugate-linear  $\mathbb{L}$

turns the holomorphic derivative of  $g \circ \mathbb{L}$  at  $u$  into the anti-holomorphic derivative of

$g$  at the mapped point  $\mathbb{L} u$ , in the mapped direction  $\mathbb{L} v$ :

$\partial_v(g \circ \mathbb{L})$  at  $u$  equals  $\partial_{\mathbb{L} v} g$  at  $\mathbb{L} u$ . -/

```
lemma dWirtingerDir_comp_conjLinear {g : V' → ℂ} {L : V → L[ℝ] V'} {u : V}
 (hL : ∀ x : V, L (Complex.I • x) = -(Complex.I • L x))
 (hg : DifferentiableAt ℝ g (L u)) (v : V) :
 dWirtingerDir (fun p => g (L p)) u v = dWirtingerAntiDir g (L v) (L u) := by
 simp only [dWirtingerDir_apply, dWirtingerAntiDir_apply, fderiv_comp_clm_apply hg,
 map_neg]
 ring
```

/-- Dual of `dWirtingerDir_comp_conjLinear`: the anti-holomorphic derivative of  $g \circ \mathbb{L}$  at  $u$  is the holomorphic derivative of  $g$  at the mapped point  $\mathbb{L} u$ , in the mapped direction  $\mathbb{L} v$ :  $\partial_v(g \circ \mathbb{L})$  at  $u$  equals  $\partial_{\mathbb{L} v} g$  at  $\mathbb{L} u$ . -

/

```
lemma dWirtingerAntiDir_comp_conjLinear {g : V' → ℂ} {L : V → L[ℝ] V'} {u : V}
 (hL : ∀ x : V, L (Complex.I • x) = -(Complex.I • L x))
 (hg : DifferentiableAt ℝ g (L u)) (v : V) :
 dWirtingerAntiDir (fun p => g (L p)) u v = dWirtingerDir g (L v) (L u) := by
 simp only [dWirtingerDir_apply, dWirtingerAntiDir_apply, fderiv_comp_clm_apply hg,
 map_neg]
 ring
```

```
end DomainConjugation
```

```
/-!
```

```
F. The holomorphic collapse
```

The two-operator split collapses to one exactly when  $f$  is holomorphic along a direction  $v$ . If the real derivative is  $\mathbb{C}$ -linear there ( $d_{i \cdot v} f = i \cdot d_v f$ ), the Cauchy–Riemann equations hold and  $\partial_v f$  is the full real derivative  $d_v f$  and  $\partial_{\bar{v}} f$  vanishes. Dual conjugate- $\mathbb{C}$ -linear derivative makes  $\partial_v f$  vanish and  $\partial_{\bar{v}} f$  the full derivative.

The lemmas take this  $\mathbb{C}$ -linearity condition directly as hypothesis, not holomorphy. The reason is a clean division of labor.

**\*\*The domain-general collapse.\*\*** Given the identity  $d_{i \cdot v} f = i \cdot d_v f$ , algebra: unfold  $\partial_v f = (1/2)(d_v f - i \cdot d_{i \cdot v} f)$ , substitute the identity, reduces to  $d_v f$  while  $\partial_{\bar{v}} f$  cancels to  $0$ . No property of the domain is needed; a single proof covers every complex  $V$ .

**\*\*The domain-specific bridge.\*\*** Holomorphy is stated through the complex derivative  $fderiv \mathbb{C}$ , but the collapse is about the real derivative  $fderiv \mathbb{R}$ ; the identity  $fderiv \mathbb{C} f u v = i \cdot fderiv \mathbb{R} f u v$  is the bridge between them. Relation between derivatives is domain-specific, so each consumer establishes the bridge in its own domain, then applies the domain-general lemma above.

```
-/
```

```
/-- Holomorphic collapse: along a direction where the real derivative is $\mathbb{C}$ -linear, the holomorphic derivative is the full real derivative, $\partial_v f = d_v f$. -
```

```
lemma dWirtingerDir_eq_of_clinear {v : V}
 (h : fderiv ℝ f u (Complex.I • v) = Complex.I • fderiv ℝ f u v) :
 dWirtingerDir f v u = fderiv ℝ f u v := by
 simp only [dWirtingerDir_apply, h, smul_eq_mul]; rw [← mul_assoc, Complex.I]
```

```
/-- Holomorphic collapse: the anti-holomorphic derivative vanishes along a direction of $\mathbb{C}$ -linearity, $\partial_{\bar{v}} f = 0$. -/
```

```
lemma dWirtingerAntiDir_eq_zero_of_clinear {v : V}
 (h : fderiv ℝ f u (Complex.I • v) = Complex.I • fderiv ℝ f u v) :
 dWirtingerAntiDir f v u = 0 := by
 simp only [dWirtingerAntiDir_apply, h, smul_eq_mul]; rw [← mul_assoc, Complex.I]
```

```
/-- Anti-holomorphic collapse: a direction of conjugate- $\mathbb{C}$ -linearity kills the
```

```

holomorphic derivative, $\partial_v f = 0$. -/
lemma dWirtingerDir_eq_zero_of_antilinear {v : V}
 (h : fderiv ℝ f u (Complex.I • v) = -(Complex.I • fderiv ℝ f u v)) :
 dWirtingerDir f v u = 0 := by
 simp only [dWirtingerDir_apply, h, smul_eq_mul, mul_neg]; rw [← mul_assoc, Complex
/-- Anti-holomorphic collapse: the anti-holomorphic derivative is the full real
derivative along a direction of conjugate- $\mathbb{C}$ -linearity, $\partial_v f = d_v f$. -
/
lemma dWirtingerAntiDir_eq_of_antilinear {v : V}
 (h : fderiv ℝ f u (Complex.I • v) = -(Complex.I • fderiv ℝ f u v)) :
 dWirtingerAntiDir f v u = fderiv ℝ f u v := by
 simp only [dWirtingerAntiDir_apply, h, smul_eq_mul, mul_neg]
 rw [← mul_assoc, Complex.I_mul_I]; ring

/-!

```

## ## G. The second-derivative bridge

Schwarz's theorem (§I) commutes two Wirtinger operators, so it differentiates a directional Wirtinger derivative a *second* time. This section bridges that second derivative to second real Fréchet derivative  $fderiv\ \mathbb{R}\ (fderiv\ \mathbb{R}\ f)\ u$ , where mixed partials are symmetric.

Each directional operator is, definitionally, a combination  $(1/2)(d_{b_1} f + c \cdot d_{b_2} f)$  where the real derivative along two directions ( $c = -i$  holomorphic with  $b_2 = i \cdot b_1$ ,  $c = i$  anti-holomorphic). `weightedDirDeriv` records this as a function of the base point,  $b_1, b_2$  left free: the directions stay fixed while the point  $p$  varies, turning one-point derivative into a field  $V \rightarrow \mathbb{C}$  that can itself be differentiated. Differentials it once more sends each first derivative to  $fderiv\ \mathbb{R}\ (fderiv\ \mathbb{R}\ f)\ u$  on two slots.

The two inner directions  $b_1, b_2$  are intrinsic: they are the pair a Wirtinger derivative already combines ( $v$  and  $i \cdot v$ ). The second differentiation, by contrast, is an ordinary Fréchet derivative along one new direction  $a$ , so the bridge lands directly on the second derivative  $fderiv\ \mathbb{R}\ (fderiv\ \mathbb{R}\ f)\ u$  in the outer slot  $a$  and inner slot  $b_1$  whose slot symmetry (`ContDiffAt.isSymmSndFDerivAt`) drives Schwarz. The *outer* Wirtinger combination is rebuilt afterward by instantiating  $a$  at  $v$  and  $i \cdot v$ .

Because `weightedDirDeriv` and its bridge `fderiv_weightedDirDeriv` are generic in  $b_2$ , one lemma serves every second-order pairing. The four combinations (holomorphic, anti-holomorphic, twice) differ only in their coefficients and all reduce to  $fderiv\ \mathbb{R}\ (fderiv\ \mathbb{R}\ f)\ u$  on the directions  $v, i \cdot v, w, i \cdot w$ . §I discharges the pairing that Kähler geometry needs; the others follow from the same bridge with a different  $(c, b_1, b_2)$ , then `ContDiffAt.isSymmSndFDerivAt` and `ring`.

The operators are already `weightedDirDeriv` by definition (§A); this section differs

that shared field a second time.

**\*\*Structure.\*\***

```
* `hasFDerivAt_fderiv_apply`, `hasFDerivAt_weightedDirDeriv` : the evaluation
 map $p \mapsto d_b f$, and hence `weightedDirDeriv`, is differentiable wherever `fderiv
 * `fderiv_weightedDirDeriv` : the bridge, sending a derivative of `weighted
 direction `a` to the second Fréchet derivative `fderiv ℝ (fderiv ℝ f) u`
 * `dWirtingerDir_eq_weightedDirDeriv`, `dWirtingerAntiDir_eq_weightedDirDeriv`
 `∂_v f`, `∂_v f` are `weightedDirDeriv` at `(c, b_1, b_2) = (-i, v, i·v)` a
 * `fderiv_dWirtingerDir`, `fderiv_dWirtingerAntiDir` : specialize the bridge
 derivative of `∂_v f`, `∂_v f` lands on `fderiv ℝ (fderiv ℝ f) u` in the

- /
```

```
omit [NormedSpace ℂ V] in
/-- The field $p \mapsto d_b f$ is the evaluation map $\cdot b$ composed with `fderiv`
so when `fderiv ℝ f` is differentiable its derivative is `fderiv ℝ (fderiv ℝ f) u`
post-composed with that evaluation. -/
private lemma hasFDerivAt_fderiv_apply (hf' : DifferentiableAt ℝ (fderiv ℝ f) b) :
 HasFDerivAt (fun p => fderiv ℝ f p b)
 ((ContinuousLinearMap.apply ℝ ℂ b).comp (fderiv ℝ (fderiv ℝ f) u)) u
 (ContinuousLinearMap.apply ℝ ℂ b).hasFDerivAt.comp u hf'.hasFDerivAt
```

```
omit [NormedSpace ℂ V] in
/-- The `weightedDirDeriv` is differentiable wherever `fderiv ℝ f` is. -
/
private lemma hasFDerivAt_weightedDirDeriv (hf' : DifferentiableAt ℝ (fderiv ℝ f) b) :
 HasFDerivAt (weightedDirDeriv f c b_1 b_2)
 ((1 / 2 : ℂ) • ((ContinuousLinearMap.apply ℝ ℂ b_1).comp (fderiv ℝ (fderiv ℝ f) u)
 + c • (ContinuousLinearMap.apply ℝ ℂ b_2).comp (fderiv ℝ (fderiv ℝ f) u)
 ((hasFDerivAt_fderiv_apply hf' b_1).add
 ((hasFDerivAt_fderiv_apply hf' b_2).const_mul c)).const_mul (1 / 2))
```

```
omit [NormedSpace ℂ V] in
/-- The bridge: differentiating a `weightedDirDeriv` along a third direction
lands on the second real Fréchet derivative `fderiv ℝ (fderiv ℝ f) u a b` in
two slots. -/
private lemma fderiv_weightedDirDeriv (hf' : DifferentiableAt ℝ (fderiv ℝ f) b) :
 (c : ℂ) (b_1 b_2 a : V) :
 fderiv ℝ (weightedDirDeriv f c b_1 b_2) u a
 = (1 / 2 : ℂ) * (fderiv ℝ (fderiv ℝ f) u a b_1
 + c * fderiv ℝ (fderiv ℝ f) u a b_2) := by
 rw [(hasFDerivAt_weightedDirDeriv hf' c b_1 b_2).fderiv]
```



```

simp only [add_apply, _root_.smul_apply,
 ContinuousLinearMap.coe_comp, Function.comp_apply, ContinuousLinearMap.apply_app,
 smul_eq_mul, mul_add]

/-- A directional derivative is a `weightedDirDeriv`: anti-
holomorphic with `c = i`. -/
private lemma dWirtingerAntiDir_eq_weightedDirDeriv (w : V) :
 (fun p => dWirtingerAntiDir f w p) = weightedDirDeriv f Complex.I w (Complex.I •
 rfl

/-- A directional derivative is a `weightedDirDeriv`: holomorphic with `c = -
i`. -/
private lemma dWirtingerDir_eq_weightedDirDeriv (v : V) :
 (fun p => dWirtingerDir f v p) = weightedDirDeriv f (-
Complex.I) v (Complex.I • v) :=
 rfl

/-- Differentiating the anti-holomorphic directional derivative lands on the second
real Fréchet derivative in the two slots. -/
private lemma fderiv_dWirtingerAntiDir (hf' : DifferentiableAt ℝ (fderiv ℝ f) u)
 (w a : V) :
 fderiv ℝ (fun p => dWirtingerAntiDir f w p) u a
 = (1 / 2 : ℂ) * (fderiv ℝ (fderiv ℝ f) u a w
 + Complex.I * fderiv ℝ (fderiv ℝ f) u a (Complex.I • w)) := by
 rw [dWirtingerAntiDir_eq_weightedDirDeriv, fderiv_weightedDirDeriv hf']

/-- Differentiating the holomorphic directional derivative lands on the second real
Fréchet derivative in the two slots. -/
private lemma fderiv_dWirtingerDir (hf' : DifferentiableAt ℝ (fderiv ℝ f) u)
 (v a : V) :
 fderiv ℝ (fun p => dWirtingerDir f v p) u a
 = (1 / 2 : ℂ) * (fderiv ℝ (fderiv ℝ f) u a v
 - Complex.I * fderiv ℝ (fderiv ℝ f) u a (Complex.I • v)) := by
 rw [dWirtingerDir_eq_weightedDirDeriv, fderiv_weightedDirDeriv hf']; ring

/-!

H. Differentiability and locality

```

Schwarz (§I) and the coordinate layer treat a directional derivative as a field in a point  $p$ , and need two regularity facts about it. Both are public packagings of §G, in `Coordinate.lean`.

**\*\*Differentiability.\*\*** On a  $C^2$  field the directional derivative  $p \mapsto \partial_v f$  is its real-differentiable (`differentiableAt_dWirtingerDir`): by §G it is a `weightedDirDeriv`. `fderiv ℝ f` is differentiable for a  $C^2$   $f$ . Without this a Wirtinger derivative  $c$

be differentiated a second time, as Schwarz (§I) does.

**\*\*Locality.\*\*** The value  $\partial_v f$  at  $u$  depends only on how  $f$  behaves near  $u$  from  $fderiv \mathbb{R} f u$ : the operator is built from it, and a Fréchet derivative on an arbitrarily small neighbourhood. So if  $f_1$  and  $f_2$  coincide on some  $u$  (in Lean  $f_1 =^f[nhds u] f_2$ , where  $=^f$  is equality on a filter-large set and  $[nhds u]$  is the neighbourhood filter of  $u$ ) they have the same directional derivative ( $dWirtingerDir\_congr\_of\_eventuallyEq$ ). This is what makes the operators regular only on a restricted domain (the Kähler potentials, defined on a subset of  $V$ ): the derivative at  $u$  needs only  $f$  near  $u$ , so a consumer can take a locally-equal representative and rely on the local  $C^2$ /holomorphy hypothesis ( $ContDiffAt$ ,  $DifferentiableAt$ ), which likewise depend only on  $f$  near  $u$ . The form asks for regularity only on a neighbourhood of  $u$ , which a restricted domain function has at each point of its domain.

-/

/-- On a  $C^2$  field the holomorphic directional derivative is itself real-differentiable. -/

```
lemma differentiableAt_dWirtingerDir (hf2 : ContDiffAt ℝ 2 f u) (v : V) :
 DifferentiableAt ℝ (fun p => dWirtingerDir f v p) u := by
 rw [dWirtingerDir_eq_weightedDirDeriv]
 have hf' : DifferentiableAt ℝ (fderiv ℝ f) u :=
 (hf2.fderiv_right (m := 1) (by norm_num)).differentiableAt one_ne_zero
 exact (hasFDerivAt_weightedDirDeriv hf' _ _).differentiableAt
```

/-- On a  $C^2$  field the anti-holomorphic directional derivative is itself real-differentiable. -/

```
lemma differentiableAt_dWirtingerAntiDir (hf2 : ContDiffAt ℝ 2 f u) (w : V) :
 DifferentiableAt ℝ (fun p => dWirtingerAntiDir f w p) u := by
 rw [dWirtingerAntiDir_eq_weightedDirDeriv]
 have hf' : DifferentiableAt ℝ (fderiv ℝ f) u :=
 (hf2.fderiv_right (m := 1) (by norm_num)).differentiableAt one_ne_zero
 exact (hasFDerivAt_weightedDirDeriv hf' _ _).differentiableAt
```

/-- The holomorphic directional derivative depends only on the field near  $u$ ; fields agreeing on a neighbourhood have equal derivative. -

/

```
lemma dWirtingerDir_congr_of_eventuallyEq {f1 f2 : V → ℂ} {u : V}
 (h : f1 =^f[nhds u] f2) (v : V) :
 dWirtingerDir f1 v u = dWirtingerDir f2 v u := by
 simp only [dWirtingerDir_apply, h.fderiv_eq]
```

/-- The anti-holomorphic directional derivative depends only on the field near  $u$ .

```

point; dual of `dWirtingerDir_congr_of_eventuallyEq`. -/
lemma dWirtingerAntiDir_congr_of_eventuallyEq {f₁ f₂ : V → ℂ} {u : V}
 (h : f₁ =ᶜ[nhds u] f₂) (v : V) :
 dWirtingerAntiDir f₁ v u = dWirtingerAntiDir f₂ v u := by
 simp only [dWirtingerAntiDir_apply, h.fderiv_eq]

```

```

/-!

```

```

I. Schwarz's theorem

```

```

-/

```

```

/-- **Schwarz's theorem** for the directional Wirtinger operators: on a `C²` field `f`
holomorphic and anti-holomorphic directional derivatives commute in any two directions:
`∂_v ∂_w f = ∂_w ∂_v f`.

```

The commutation adds no analytic input. By the \$G\$ bridge each order expands into a real linear

combination of the second real Fréchet derivative  $\text{fderiv } \mathbb{R} (\text{fderiv } \mathbb{R} f) u$  on the two directions  $v$ ,  $i \cdot v$ ,  $w$ ,  $i \cdot w$ . The two orders give the same combination up to terms in the two slots of that second derivative, and  $\text{ContDiffAt.isSymmSndFderivAt}$ , the symmetric ordinary mixed second partials, equates them. -/

```

theorem dWirtingerDir_dWirtingerAntiDir_comm (hf2 : ContDiffAt ℝ 2 f u) (v w : V) :
 dWirtingerDir (fun p => dWirtingerAntiDir f w p) v u
 = dWirtingerAntiDir (fun p => dWirtingerDir f v p) w u := by
 have hf' : DifferentiableAt ℝ (fderiv ℝ f) u :=
 (hf2.fderiv_right (m := 1) (by norm_num)).differentiableAt one_ne_zero
 have hsymm : IsSymmSndFderivAt ℝ f u := hf2.isSymmSndFderivAt (by simp)
 rw [dWirtingerDir_apply (fun p => dWirtingerAntiDir f w p) v u,
 dWirtingerAntiDir_apply (fun p => dWirtingerDir f v p) w u,
 fderiv_dWirtingerAntiDir hf', fderiv_dWirtingerAntiDir hf',
 fderiv_dWirtingerDir hf', fderiv_dWirtingerDir hf',
 hsymm.eq w v, hsymm.eq w (Complex.I • v),
 hsymm.eq (Complex.I • w) v, hsymm.eq (Complex.I • w) (Complex.I • v)]
 ring

```

```

end Physlib.Wirtinger

```

```

end

```

```

end

```

**1.64 Physlib/Mathematics/Calculus/Wirtinger/Coordinate.lean**

```

/-
Copyright (c) 2026 Andrea Pari. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Andrea Pari
-/
module

public import Physlib.Mathematics.Calculus.Wirtinger.Basic
public import Mathlib.Analysis.Calculus.FDeriv.Pi
public import Mathlib.Analysis.Calculus.FDeriv.RestrictScalars
public import Mathlib.Analysis.Calculus.FDeriv.Star
public import Mathlib.Data.Fintype.Defs

/-!

Partial Wirtinger derivatives ∂_I , $\partial_{\bar{I}}$ in a complex coordinate basis

Notation

The conventions of `Wirtinger.Basic` carry over verbatim, with the direction
replaced by a coordinate index `I` (standing for the direction `Pi.single I 1`

* $\partial_I f$ and $\partial_{\bar{I}} f$ are the holomorphic and anti-holomorphic coordinate W
derivatives of f in the I -th coordinate direction `Pi.single I 1` – the
operator, never on the subscript. They are the directional $\partial_v f / \partial_v$
`Wirtinger.Basic` at $v = \text{Pi.single } I \ 1$. For iterated derivatives the op
 $\partial_I \partial_{\bar{J}} f$.
* z^I is the I -th coordinate $u \mapsto u^I$; $\bar{z}^I = \text{star } z^I$ is its pointwise
 $\bar{f}$ convention of `Wirtinger.Basic`).
* ∂_x , ∂_y are the slot- I real and imaginary Fréchet derivatives, along
`Pi.single I 1` and `Pi.single I i`.
* δ_{IJ} is the Kronecker delta.

i. Overview

Coordinate specialization of `Wirtinger.Basic` to $V = \mathbb{C}^n$ (spelled `ι → ℂ`
`n = |ι|`, `ι` a `Fintype`), fixing the direction to the I -
th basis vector
`Pi.single I 1`:

 $\partial_I f := \partial_v f$ at $v = \text{Pi.single } I \ 1$ (`dWirtingerCoord`)
 $\partial_{\bar{I}} f := \partial_{\bar{v}} f$ at $v = \text{Pi.single } I \ 1$ (`dWirtingerAntiCoord`)

 ∂_I , $\partial_{\bar{I}}$ are the partial Wirtinger derivatives w.r.t. coordinate `I`, t

```

1.64.

PHYSLIB/MATHEMATICS/CALCULUS/WIRTINGER/COORDINATE.LEAN105

other coordinates fixed. Coordinate values:

$$\partial_I z^J = \delta_{IJ}, \quad \partial_I \bar{z}^J = \delta_{IJ}, \quad \partial_I z^J = \partial_I \bar{z}^J = 0.$$

The basis makes the directional calculus computational. The operators ``dWirtingerCoord`` / ``dWirtingerAntiCoord`` (§A, with real-Fréchet unfolding

``(1/2)(\partial_x \mp i \partial_y)``) come with:

- independence: ``\partial_I`` annihilates every ``\bar{z}`` and ``\partial_I`` every ``z`` (the coordinate values above), so ``z`` and ``\bar{z}`` behave as independent variables;
- additivity,  $\mathbb{C}$ -linearity, the Leibniz product rule, and the finite-sum rule (§B-C), so any polynomial in the coordinates and their conjugates differentiates termwise;
- conjugation, swapping the two operators (§B-C): ``\partial_I \bar{f} = conj (\partial_I f)`` and ``\partial_I f = conj (\partial_I \bar{f})``;
- holomorphic collapse (§B-C): for holomorphic ``f``, ``\partial_I f = 0`` and ``\partial_I f`` is the ordinary complex partial ``fderiv \mathbb{C} f u (Pi.single I 1)`` (anti-holomorphic ``f`` dually);
- the per-coordinate chain rule for an outer ``g : \mathbb{C} \rightarrow \mathbb{C}`` (§D), collapsing to a single term ``\partial_I (g \circ f) = deriv g (f u) \cdot \partial_I f`` on holomorphic ``g`` (and ``\partial_I`` likewise);
- the coordinate difference ``z^J - \bar{z}^J`` (§C);
- Schwarz's theorem ``\partial_I \partial_J f = \partial_J \partial_I f`` on  $\mathbb{C}^2$  ``f`` (§E, ``dWirtingerCoord_dWirtingerAntiCoord_comm``).

Indexing by ``I`` casts the calculus in the language of several complex variables. The first derivatives assemble into a gradient, the family of partials ``\partial_I f`` ranging over the coordinates ``I``; the critical points of a holomorphic ``h`` are then where ``\partial_I h = 0`` for every ``I``. The mixed second derivatives assemble into a complex Hessian, the matrix with entries ``\partial_I \partial_J f`` indexed by the pair ``(I, J)``; for a real function ``\Phi`` the entries ``\partial_I \partial_J \Phi`` record its second-order behaviour.

That Hessian is Hermitian, by Schwarz (§E). For a real ``K``, write the Kähler metric ``g_{I\bar{J}} = \partial_I \partial_{\bar{J}} K``; conjugation gives ``star (g_{J\bar{I}}) = \partial_{\bar{J}} \partial_I K``, which Schwarz equates with ``\partial_I \partial_{\bar{J}} K``, so ``g_{I\bar{J}} = star (g_{J\bar{I}})``. That is Kähler-metric hermiticity.

The coordinate maps are Mathlib primitives – ``z^I`` is ``ContinuousLinearMap.proj``, ``\bar{z}`` ``star`` – and the holomorphic collapse reads ``fderiv \mathbb{R} f = fderiv \mathbb{C} f`` off the ``restr`` bridge (§B).

Differentiability convention: hypothesis-bearing rules are pointwise (at ``u``,

``DifferentiableAt``), valid on a proper subdomain (e.g. a slit-domain log Kähler potential); ``funext`` locally for a function-level form. Hypothesis-free constant and coordinate facts are function equalities.

## ## ii. Key results

- ``Physlib.Wirtinger.dWirtingerCoord`` / ``dWirtingerAntiCoord`` : the coordinate Wirtinger operators, definitionally the directional operator along ``Pi.single I 1``; their real-Fréchet form ``dWirtingerCoord_apply`` ``dWirtingerAntiCoord_apply`` is  $\partial_I f = (1/2)(\partial_x \mp i \partial_y) f$ .
- ``Physlib.Wirtinger.dWirtingerCoord_coordProj`` / ``dWirtingerAntiCoord_coordProj`` / ``dWirtingerCoord_conjCoord`` / ``dWirtingerAntiCoord_conjCoord`` : the four Kronecker coordinate values  $\partial_I z^J = \delta_{IJ}$ ,  $\partial_{\bar{I}} \bar{z}^J = \delta_{\bar{I}\bar{J}}$ ,  $\partial_I \bar{z}^J = \partial_{\bar{I}} z^J = 0$ .
- ``Physlib.Wirtinger.dWirtingerCoord_add_apply`` / ``dWirtingerCoord_smul_apply`` / ``dWirtingerCoord_mul_apply`` / ``dWirtingerCoord_fun_sum_apply`` : additivity,  $\mathbb{C}$ -linearity, the Leibniz rule, and the finite-sum rule (with anti-holomorphic duals).
- ``Physlib.Wirtinger.dWirtingerCoord_star_comp_apply`` / ``dWirtingerAntiCoord_star_comp_apply`` : conjugating the inner field swaps two operators,  $\partial_I \bar{f} = \text{conj}(\partial_{\bar{I}} f)$ .
- ``Physlib.Wirtinger.dWirtingerCoord_eq_complex_fderiv_apply`` / ``dWirtingerAntiCoord_eq_zero_of_holomorphic_apply`` : holomorphic collapses the coordinate operators (with anti-holomorphic duals).
- ``Physlib.Wirtinger.dWirtingerCoord_comp_apply`` / ``dWirtingerCoord_comp_holomorphic_apply`` (and their anti-holomorphic duals): the two-term Wirtinger chain rule for an outer  $g : \mathbb{C} \rightarrow \mathbb{C}$ , collapsing to a single-term  $\text{deriv } g(f u) \cdot \partial_I f$  for holomorphic  $g$ .
- ``Physlib.Wirtinger.dWirtingerCoord_coordDiff`` / ``dWirtingerAntiCoord_coordDiff`` : Wirtinger derivatives of the coordinate difference  $z^J - \bar{z}^J$ .
- ``Physlib.Wirtinger.differentiableAt_dWirtingerCoord`` / ``differentiableAt_dWirtingerAntiCoord`` : a first coordinate Wirtinger derivative of a  $\mathbb{C}^2$  function is itself real-differentiable (§E).
- ``Physlib.Wirtinger.dWirtingerCoord_dWirtingerAntiCoord_comm`` : Schwarz's theorem for the coordinate operators,  $\partial_I \partial_{\bar{J}} f = \partial_{\bar{J}} \partial_I f$  on  $\mathbb{C}^2$   $f$ .

## ## iii. Table of contents

- A. The coordinate Wirtinger operators
- B. Properties of ``dWirtingerCoord``
- C. Properties of ``dWirtingerAntiCoord``

1.64.

PHYSLIB/MATHEMATICS/CALCULUS/WIRTINGER/COORDINATE.LEAN107

- D. Wirtinger chain rules for an outer function
- E. Schwarz's theorem for the coordinate operators

-/

@[expose] public section  
noncomputable section

namespace Physlib.Wirtinger

variable {ι : Type\*}

/-!

## A. The coordinate Wirtinger operators

The two coordinate Wirtinger operators are the directional Wirtinger derivatives of `Wirtinger.Basic` along the  $I$ -th coordinate direction `Pi.single I 1`:

$$\begin{aligned} \text{dWirtingerCoord } f \text{ } I &= (1/2) \cdot (\partial_x - i \cdot \partial_y) f \\ \text{dWirtingerAntiCoord } f \text{ } I &= (1/2) \cdot (\partial_x + i \cdot \partial_y) f \end{aligned}$$

where `∂x` and `∂y` are the real Fréchet derivatives of `f` along the slot- $I$

real and imaginary coordinate directions `Pi.single I 1` and `Pi.single I Complex.I` (the latter is `Complex.I • Pi.single I 1`). The sign on the imaginary-direction term is the only difference, making the two operators dual on (anti)holomorphic functions ( $\$B$ ,  $\$C$ ).

Each `∂I` is thus a 1-D directional derivative taken along the standard basis vector `Pi.single I 1` – the whole calculus is the one-variable theory applied direction by direction. The coordinates decouple (`∂I zJ = δIJ`,  $\$B$ ) because the coordinate functions `zJ` are the dual basis to the standard basis: `zJ (Pi.single I 1) = δIJ`.

-/

variable [Fintype ι] [DecidableEq ι]

/-- Holomorphic Wirtinger derivative along the  $I$ -th coordinate of  $\iota \rightarrow \mathbb{C}$ . -  
/

def dWirtingerCoord (f : (ι → ℂ) → ℂ) (I : ι) : (ι → ℂ) → ℂ :=  
 fun u => dWirtingerDir f (Pi.single I 1) u

/-- Anti-holomorphic Wirtinger derivative along the  $I$ -th coordinate of  $\iota \rightarrow \mathbb{C}$ . -  
/

def dWirtingerAntiCoord (f : (ι → ℂ) → ℂ) (I : ι) : (ι → ℂ) → ℂ :=

```

 fun u => dWirtingerAntiDir f (Pi.single I 1) u

/-- Real-Fréchet form of `dWirtingerCoord`:
`dWirtingerCoord f I u = (1/2)(∂x - i · ∂y) f`,
the derivatives along the slot-I real and imaginary coordinate directions.
Unconditional – the directional definition makes it definitional
(`Complex.I · Pi.single I 1 = Pi.single I Complex.I`); no differentiability
hypothesis is needed. -/
lemma dWirtingerCoord_apply {f : (ι → ℂ) → ℂ}
 {u : (ι → ℂ)} (I : ι) :
 dWirtingerCoord f I u = (1 / 2 : ℂ) * (fderiv ℝ f u (Pi.single I 1)
 - Complex.I * fderiv ℝ f u (Pi.single I Complex.I)) := by
 simp only [dWirtingerCoord, dWirtingerDir_apply, ← Pi.single_smul', smul_

/-- Real-Fréchet form of `dWirtingerAntiCoord`:
`∂I f = (1/2)(∂x + i · ∂y) f`,
mirror of `dWirtingerCoord_apply` with the sign flip on the imaginary-
direction term.
Unconditional, as for `dWirtingerCoord_apply`. -/
lemma dWirtingerAntiCoord_apply {f : (ι → ℂ) → ℂ}
 {u : (ι → ℂ)} (I : ι) :
 dWirtingerAntiCoord f I u = (1 / 2 : ℂ) * (fderiv ℝ f u (Pi.single I 1)
 + Complex.I * fderiv ℝ f u (Pi.single I Complex.I)) := by
 simp only [dWirtingerAntiCoord, dWirtingerAntiDir_apply, ← Pi.single_smul

/-!

B. Properties of `dWirtingerCoord`

```

Each rule is the ``d = Pi.single I 1`` specialisation of its ``Wirtinger`` foundation analogue. Rules carrying a differentiability hypothesis are stated **pointwise** (at ``u``, hypothesis ``DifferentiableAt``) – the weakest form, a one to reach for on a function differentiable only on a proper domain; a `co` a function-level equation ``funext``'s locally. The hypothesis-free constant and coordinate facts are stated as function equalities – the constant and holomorphic-coordinate ones (``dWirtingerCoord_const``, ``dWirtingerCoord_coord``).

The holomorphic collapse ``∂I f = fderiv ℂ f`` for holomorphic ``f`` below (with ``∂I f = 0`` in §C) needs the *real* derivative ``fderiv ℝ f u`` to be ``ℂ``-linear. For holomorphic ``f``, ``fderiv ℝ f u`` is the ``ℂ``-linear ``fderiv ℂ f u`` with scalars restricted to ``ℝ``, ``fderiv ℝ f u = (fderiv ℂ f u).restrictScalars ℝ`` (``HasFDerivAt.restrictScalars``). Restricting scalars drops the ``ℂ``-linear *bundling*, not the behaviour: the with ``i``, ``fderiv ℝ f u (i · d) = i · fderiv ℝ f u d`` – the ``ℂ``-linearity the collapse consumes.



1.64.

PHYSLIB/MATHEMATICS/CALCULUS/WIRTINGER/COORDINATE.LEAN109

```
`clinear_of_holomorphic` packages this, via `DifferentiableAt.fderiv_restrictScalars`
`ContinuousLinearMap.coe_restrictScalars`; `DifferentiableAt.restrictScalars` supplies
 $\mathbb{R}$ -differentiability.
```

```
-/
```

```
/-- The real derivative of a holomorphic $f : E \rightarrow \mathbb{C}$ is $\mathbb{C}$ -linear along every direction –
the hypothesis the foundation collapse `dWirtingerDir_eq_of_clinear` consumes. A holomorphic
 f has $fderiv \mathbb{R} f u = (fderiv \mathbb{C} f u).restrictScalars \mathbb{R}$
(`DifferentiableAt.fderiv_restrictScalars`), so its real derivative agrees with the
linear
```

```
complex derivative on every direction. Used at $E = \mathbb{1} \rightarrow \mathbb{C}$ for the coordinate collapse
 $E = \mathbb{C}$ for the outer-`g` chain rule (§D). -/
```

```
private lemma clinear_of_holomorphic {E : Type*} [NormedAddCommGroup E] [NormedSpace
 [NormedSpace $\mathbb{C}$ E] [IsScalarTower $\mathbb{R}$ $\mathbb{C}$ E] {f : E $\rightarrow$ $\mathbb{C}$ } {u : E}
 (hf : DifferentiableAt $\mathbb{C}$ f u) (d : E) :
 fderiv $\mathbb{R}$ f u (Complex.I • d) = Complex.I • fderiv $\mathbb{R}$ f u d := by
 rw [DifferentiableAt.fderiv_restrictScalars $\mathbb{R}$ hf, ContinuousLinearMap.coe_restrictScalars
 map_smul]
```

section

```
variable {f g : ($\mathbb{1} \rightarrow \mathbb{C}$) $\rightarrow$ $\mathbb{C}$ }
```

```
/-- `dWirtingerCoord` is local: functions agreeing on a neighbourhood of u have equal
holomorphic Wirtinger derivative at u ($f_1 =^f[nhds u] f_2 \implies \partial_I f_1 u = \partial_I f_2 u$).
-/
```

```
lemma dWirtingerCoord_congr_of_eventuallyEq_apply {f1 f2 : ($\mathbb{1} \rightarrow \mathbb{C}$) $\rightarrow$ $\mathbb{C}$ }
 {u : ($\mathbb{1} \rightarrow \mathbb{C}$)} (h : f1 =f[nhds u] f2) (I : $\mathbb{1}$) :
 dWirtingerCoord f1 I u = dWirtingerCoord f2 I u :=
 dWirtingerDir_congr_of_eventuallyEq h (Pi.single I 1)
```

```
/-- Constants have zero coordinate derivative: $\partial_I c = 0$. -
-/
```

```
@[simp] lemma dWirtingerCoord_const (c : $\mathbb{C}$) (I : $\mathbb{1}$) :
 dWirtingerCoord (fun _ => c) I = 0 := by
 funext u; exact dWirtingerDir_const c (Pi.single I 1) u
```

```
/-- Pointwise negation rule for the holomorphic coordinate derivative at u :
 $\partial_I (-f) = -\partial_I f$. Used with `dWirtingerCoord_add_apply` to assemble the
coordinate-difference rule `dWirtingerCoord_coordDiff` (§C). -
-/
```

```
lemma dWirtingerCoord_neg_apply {u : ($\mathbb{1} \rightarrow \mathbb{C}$)} (I : $\mathbb{1}$) :
 dWirtingerCoord (fun v => -(f v)) I u = -(dWirtingerCoord f I u) :=
 dWirtingerDir_neg f (Pi.single I 1) u
```

```

omit [Fintype ι] [DecidableEq ι] in
/-- The real Fréchet derivative of the J-th coordinate projection $v \mapsto v J$
Kronecker coordinate-value lemmas `dWirtingerCoord_coordProj` / `dWirtinger
which feed it the slot-I real and imaginary directions. -
/
private lemma fderiv_coordProj (J : ι) (u d : (ι → ℂ)) :
 fderiv ℝ (fun v : (ι → ℂ) => v J) u d = d J := by
 have h : HasFDerivAt (fun v : (ι → ℂ) => v J)
 (ContinuousLinearMap.proj (R := ℝ) (φ := fun _ : ι => ℂ) J) u :=
 (ContinuousLinearMap.proj (R := ℝ) (φ := fun _ : ι => ℂ) J).hasFDerivAt
 rw [h.fderiv, ContinuousLinearMap.proj_apply]

/-- $\partial_I z^J = \delta_{IJ}$. The holomorphic coordinate-independence value; also f
coordinate-difference rule `dWirtingerCoord_coordDiff` and the conjugate-
coordinate value
`dWirtingerAntiCoord_conjCoord` (§C). -/
@[simp] lemma dWirtingerCoord_coordProj (I J : ι) :
 dWirtingerCoord (fun u : (ι → ℂ) => u J) I =
 fun _ => if I = J then 1 else 0 := by
 funext u
 rw [dWirtingerCoord_apply I, fderiv_coordProj, fderiv_coordProj]
 by_cases h : I = J
 · subst h; rw [Pi.single_eq_same, Pi.single_eq_same, if_pos rfl, Complex.
 · rw [Pi.single_eq_of_ne (Ne.symm h), Pi.single_eq_of_ne (Ne.symm h), if_
 mul_zero, sub_zero, mul_zero]

/-- Pointwise additivity of the holomorphic coordinate derivative at `u`:
 $\partial_I (f + g) = \partial_I f + \partial_I g$. Used to assemble the coordinate-
difference rule
`dWirtingerCoord_coordDiff` (§C). -/
lemma dWirtingerCoord_add_apply {u : (ι → ℂ)}
 (hf : DifferentiableAt ℝ f u) (hg : DifferentiableAt ℝ g u) (I : ι) :
 dWirtingerCoord (f + g) I u = dWirtingerCoord f I u + dWirtingerCoord g
 dWirtingerDir_add hf hg (Pi.single I 1)

/-- Pointwise compatibility with complex scalar multiplication at `u`:
 $\partial_I (c \cdot f) = c \cdot \partial_I f$. -/
lemma dWirtingerCoord_smul_apply {u : (ι → ℂ)}
 (c : ℂ) (hf : DifferentiableAt ℝ f u) (I : ι) :
 dWirtingerCoord (c • f) I u = c • dWirtingerCoord f I u :=
 dWirtingerDir_smul c hf (Pi.single I 1)

/-- Pointwise Leibniz rule for the holomorphic coordinate derivative at `u`
 $\partial_I (f \cdot g) = \partial_I f \cdot g + f \cdot \partial_I g$. -/
lemma dWirtingerCoord_mul_apply {u : (ι → ℂ)}

```

1.64.

PHYSLIB/MATHEMATICS/CALCULUS/WIRTINGER/COORDINATE.LEAN 111

```

 (hf : DifferentiableAt ℝ f u) (hg : DifferentiableAt ℝ g u) (I : ι) :
 dWirtingerCoord (f * g) I u =
 dWirtingerCoord f I u * g u + f u * dWirtingerCoord g I u :=
 dWirtingerDir_mul hf hg (Pi.single I 1)

/-- Pointwise finite-sum rule for holomorphic coordinate derivatives at `u`:
`∂_I (∑ a ∈ s, F a) = ∑ a ∈ s, ∂_I (F a)` -/
lemma dWirtingerCoord_fun_sum_apply {α : Type*} {s : Finset α}
 {F : α → (ι → ℂ) → ℂ} {u : (ι → ℂ)}
 (hF : ∀ a ∈ s, DifferentiableAt ℝ (F a) u) (I : ι) :
 dWirtingerCoord (fun v => ∑ a ∈ s, F a v) I u =
 ∑ a ∈ s, dWirtingerCoord (F a) I u :=
 dWirtingerDir_fun_sum hF (Pi.single I 1)

/-- For a holomorphic function, `dWirtingerCoord` is the complex Fréchet derivative
the corresponding coordinate direction: `∂_I f = fderiv ℂ f u (Pi.single I 1)` -
/
lemma dWirtingerCoord_eq_complex_fderiv_apply {u : (ι → ℂ)}
 (hf : DifferentiableAt ℂ f u) (I : ι) :
 dWirtingerCoord f I u = fderiv ℂ f u (Pi.single I 1) := by
 show dWirtingerDir f (Pi.single I 1) u = fderiv ℂ f u (Pi.single I 1)
 rw [dWirtingerDir_eq_of_clinear (clinear_of_holomorphic hf _),
 DifferentiableAt.fderiv_restrictScalars ℝ hf, ContinuousLinearMap.coe_restrictScalars]

/-- Pointwise conjugation bundled as an ℝ-linear CLM (conjugate-
ℂ-linear): the I-th
component is conjugation of the I-th coordinate `Complex.conjCLE ∘ proj I`. Its unde-
function is the `star` of `ι → ℂ` (`conjCLM_apply`). Bundling `star` as a `→L[ℝ]` is
lets the anti-holomorphic lemmas below feed it to the foundation `dWirtingerDir_comp`
which requires its domain map as a continuous linear map. -
/
private def conjCLM : (ι → ℂ) →L[ℝ] (ι → ℂ) :=
 ContinuousLinearMap.pi
 (fun I => Complex.conjCLE.toContinuousLinearMap.comp (ContinuousLinearMap.proj I))

omit [Fintype ι] [DecidableEq ι] in
@[simp] private lemma conjCLM_apply (u : ι → ℂ) : conjCLM u = star u := rfl

omit [Fintype ι] [DecidableEq ι] in
/-- `conjCLM` is conjugate-ℂ-linear: `conj (i·d) = -(i · conj d)`. The `hL` hypotheses
foundation `dWirtingerDir_comp_conjLinear` / `dWirtingerAntiDir_comp_conjLinear` con-
/
private lemma conjCLM_smul_I (d : ι → ℂ) :
 conjCLM (Complex.I • d) = -(Complex.I • conjCLM (ι := ι) d) := by
 funext I
 simp only [conjCLM_apply, Pi.star_apply, Pi.smul_apply, Pi.neg_apply,
```

```

 smul_eq_mul, star_mul', Complex.star_def, Complex.conj_I]
ring

/-- An *anti-holomorphic* function $v \mapsto g(\text{star } v)$ (a holomorphic g pre-
with conjugation) depends on the coordinates only through their conjugates.
holomorphic derivative therefore vanishes, $\partial_I (g \circ \text{star}) = 0$, since ∂_I
differentiates w.r.t. z and treats $\bar{z}$ as constant. Dual to $\partial_I f = 0$ for
holomorphic f . -/
lemma dWirtingerCoord_eq_zero_of_antiHolomorphic_apply {u : (ι → ℂ)}
 (hg : DifferentiableAt ℂ g (star u)) (I : ι) :
 dWirtingerCoord (fun v : (ι → ℂ) => g (star v)) I u = 0 := by
 have hgr : DifferentiableAt ℝ g (conjCLM u) := by
 rw [conjCLM_apply]; exact hg.restrictScalars ℝ
 show dWirtingerDir (fun v : (ι → ℂ) => g (star v))
 (Pi.single I 1) u = 0
 rw [show (fun v : (ι → ℂ) => g (star v))
 = fun v => g (conjCLM v) from by funext v; rw [conjCLM_apply],
 dWirtingerDir_comp_conjLinear conjCLM_smul_I hgr]
 simp only [conjCLM_apply]
 exact dWirtingerAntiDir_eq_zero_of_clinear (clinear_of_holomorphic hg _)

end

/-!

C. Properties of `dWirtingerAntiCoord`

This section is the `dWirtingerAntiCoord` mirror of §B: every rule with z
swapped (locality, constants, negation, additivity, scalar compatibility, L
finite-sum rule), together with the holomorphic collapse $\partial_I f = 0$.

It also collects the two *conjugate-coordinate* values, one per operator:
 $\partial_I \bar{z}^J = 0$ (`dWirtingerCoord_conjCoord`) and $\partial_I \bar{z}^J = \delta_{IJ}$
(`dWirtingerAntiCoord_conjCoord`). Since $\bar{z}^J = \text{star } z^J$, each is the conj
corresponding value on the holomorphic coordinate z^J , read off through the
conjugation lemmas `dWirtingerDir_star_comp` / `dWirtingerAntiDir_star_comp`
recomputed.

-/

section

variable {f g : (ι → ℂ) → ℂ}

/-- `dWirtingerAntiCoord` is local ($f_1 = f_2$ on u) $\Rightarrow \partial_I f_1 u = \partial_I f_2 u$
/
```

1.64.

PHYSLIB/MATHEMATICS/CALCULUS/WIRTINGER/COORDINATE. LEAN 113

```

lemma dWirtingerAntiCoord_congr_of_eventuallyEq_apply {f₁ f₂ : (ι → ℂ) → ℂ}
 {u : (ι → ℂ)} (h : f₁ =f[nhds u] f₂) (I : ι) :
 dWirtingerAntiCoord f₁ I u = dWirtingerAntiCoord f₂ I u :=
 dWirtingerAntiDir_congr_of_eventuallyEq h (Pi.single I 1)

/-- Constants have zero anti-holomorphic coordinate derivative: `∂_I c = 0`. -
/
@[simp] lemma dWirtingerAntiCoord_const (c : ℂ) (I : ι) :
 dWirtingerAntiCoord (fun _ : (ι → ℂ) => c) I = 0 := by
 funext u; exact dWirtingerAntiDir_const c (Pi.single I 1) u

/-- Pointwise negation rule for the anti-holomorphic coordinate derivative at `u`:
`∂_I (-f) = -∂_I f`. Used with `dWirtingerAntiCoord_add_apply` to assemble the
coordinate-difference rule `dWirtingerAntiCoord_coordDiff` (§C). -
/
lemma dWirtingerAntiCoord_neg_apply {u : (ι → ℂ)} (I : ι) :
 dWirtingerAntiCoord (fun v => -(f v)) I u = -(dWirtingerAntiCoord f I u) :=
 dWirtingerAntiDir_neg f (Pi.single I 1) u

/-- `∂_I z^J = 0`. The anti-holomorphic coordinate-independence value; also feeds the
coordinate-difference rule `dWirtingerAntiCoord_coordDiff` and the conjugate-
coordinate
value `dWirtingerCoord_conjCoord` (§C). -/
@[simp] lemma dWirtingerAntiCoord_coordProj (I J : ι) :
 dWirtingerAntiCoord (fun u : (ι → ℂ) => u J) I = 0 := by
 funext u
 simp only [Pi.zero_apply]
 rw [dWirtingerAntiCoord_apply I, fderiv_coordProj, fderiv_coordProj]
 by_cases h : I = J
 · subst h; rw [Pi.single_eq_same, Pi.single_eq_same, Complex.I_mul_I]; ring
 · rw [Pi.single_eq_of_ne (Ne.symm h), Pi.single_eq_of_ne (Ne.symm h),
 mul_zero, add_zero, mul_zero]

/-- `∂_I z̄^J = 0`. The conjugate of `dWirtingerAntiCoord_coordProj` (`z̄^J = star z^J`
read off through `dWirtingerDir_star_comp` rather than recomputed. Used to assemble
coordinate-difference rule `dWirtingerCoord_coordDiff` (§C). -
/
lemma dWirtingerCoord_conjCoord (I J : ι) :
 dWirtingerCoord (fun u : (ι → ℂ) => star (u J)) I = 0 := by
 funext u
 have hd : DifferentiableAt ℝ (fun w : (ι → ℂ) => w J) u :=
 (ContinuousLinearMap.proj (R := ℝ) (φ := fun _ : ι => ℂ) J).differentiableAt
 change dWirtingerDir (fun v => star ((fun w : (ι → ℂ) => w J) v))
 (Pi.single I 1) u = 0
 rw [dWirtingerDir_star_comp hd (Pi.single I 1)]
 show star (dWirtingerAntiCoord (fun w : (ι → ℂ) => w J) I u) = 0

```

```

rw [dWirtingerAntiCoord_coordProj, Pi.zero_apply, star_zero]

/-- $\partial_I \bar{z}^J = \delta_{IJ}$. The conjugate of dWirtingerCoord_coordProj, read of
dWirtingerAntiDir_star_comp rather than recomputed. Used to assemble the
coordinate-difference rule dWirtingerAntiCoord_coordDiff (§C). -
/
lemma dWirtingerAntiCoord_conjCoord (I J : ι) :
 dWirtingerAntiCoord (fun u : (ι → ℂ) => star (u J)) I =
 fun _ => if I = J then 1 else 0 := by
 funext u
 have hd : DifferentiableAt ℝ (fun w : (ι → ℂ) => w J) u :=
 (ContinuousLinearMap.proj (R := ℝ) (φ := fun _ : ι => ℂ) J).differentia
change dWirtingerAntiDir (fun v => star ((fun w : (ι → ℂ) => w J) v))
 (Pi.single I 1) u = if I = J then 1 else 0
rw [dWirtingerAntiDir_star_comp hd (Pi.single I 1)]
show star (dWirtingerCoord (fun w : (ι → ℂ) => w J) I u) = if I = J then
rw [dWirtingerCoord_coordProj]
simp only [apply_ite (star : ℂ → ℂ), star_one, star_zero]

/-- Pointwise additivity of the anti-holomorphic coordinate derivative at `
 $\partial_I (f + g) = \partial_I f + \partial_I g$. Used to assemble the coordinate-
difference rule
dWirtingerAntiCoord_coordDiff (§C). -/
lemma dWirtingerAntiCoord_add_apply {u : (ι → ℂ)}
 (hf : DifferentiableAt ℝ f u) (hg : DifferentiableAt ℝ g u) (I : ι) :
 dWirtingerAntiCoord (f + g) I u = dWirtingerAntiCoord f I u + dWirtinger
dWirtingerAntiDir_add hf hg (Pi.single I 1)

/-- Pointwise compatibility with complex scalar multiplication at `u`:
 $\partial_I (c \cdot f) = c \cdot \partial_I f$. -/
lemma dWirtingerAntiCoord_smul_apply {u : (ι → ℂ)}
 (c : ℂ) (hf : DifferentiableAt ℝ f u) (I : ι) :
 dWirtingerAntiCoord (c • f) I u = c • dWirtingerAntiCoord f I u :=
dWirtingerAntiDir_smul c hf (Pi.single I 1)

/-- Pointwise Leibniz rule for the anti-holomorphic coordinate derivative a
 $\partial_I (f \cdot g) = \partial_I f \cdot g + f \cdot \partial_I g$. -/
lemma dWirtingerAntiCoord_mul_apply {u : (ι → ℂ)}
 (hf : DifferentiableAt ℝ f u) (hg : DifferentiableAt ℝ g u) (I : ι) :
 dWirtingerAntiCoord (f * g) I u =
 dWirtingerAntiCoord f I u * g u + f u * dWirtingerAntiCoord g I u :=
dWirtingerAntiDir_mul hf hg (Pi.single I 1)

/-- Pointwise finite-sum rule for anti-holomorphic coordinate derivatives a
 $\partial_I (\sum a \in s, F a) = \sum a \in s, \partial_I (F a)$. -/
lemma dWirtingerAntiCoord_fun_sum_apply {α : Type*} {s : Finset α}

```

1.64.

PHYSLIB/MATHEMATICS/CALCULUS/WIRTINGER/COORDINATE. LEAN 115

```

 {F : $\alpha \rightarrow (\iota \rightarrow \mathbb{C}) \rightarrow \mathbb{C}$ } {u : ($\iota \rightarrow \mathbb{C}$)}
 (hF : $\forall a \in s, \text{DifferentiableAt } \mathbb{R} (F a) u$) (I : ι) :
 dWirtingerAntiCoord (fun v => $\sum a \in s, F a v$) I u =
 $\sum a \in s, \text{dWirtingerAntiCoord } (F a) I u :=$
 dWirtingerAntiDir_fun_sum hF (Pi.single I 1)

/-- For an anti-holomorphic function the anti-holomorphic Wirtinger derivative equals
the complex Fréchet derivative of `g` at `star u` along the slot-
I real
coordinate direction, `∂_I (g ∘ star) = fderiv ℂ g (star u) (Pi.single I 1)`.
Dual of `dWirtingerCoord_eq_complex_fderiv_apply`. -/
lemma dWirtingerAntiCoord_eq_complex_fderiv_apply {u : ($\iota \rightarrow \mathbb{C}$)}
 (hg : DifferentiableAt $\mathbb{C} g$ (star u)) (I : ι) :
 dWirtingerAntiCoord (fun v : ($\iota \rightarrow \mathbb{C}$) => g (star v)) I u =
 fderiv $\mathbb{C} g$ (star u) (Pi.single I 1) := by
 have hgr : DifferentiableAt $\mathbb{R} g$ (conjCLM u) := by
 rw [conjCLM_apply]; exact hg.restrictScalars $\mathbb{R}$
 show dWirtingerAntiDir (fun v : ($\iota \rightarrow \mathbb{C}$) => g (star v))
 (Pi.single I 1) u = fderiv $\mathbb{C} g$ (star u) (Pi.single I 1)
 rw [show (fun v : ($\iota \rightarrow \mathbb{C}$) => g (star v))
 = fun v => g (conjCLM v) from by funext v; rw [conjCLM_apply],
 dWirtingerAntiDir_comp_conjLinear conjCLM_smul_I hgr]
 simp only [conjCLM_apply, Pi.star_single, star_one]
 rw [dWirtingerDir_eq_of_clinear (cllinear_of_holomorphic hg _),
 DifferentiableAt.fderiv_restrictScalars $\mathbb{R} hg$, ContinuousLinearMap.coe_restrictScalars]

/-- Holomorphic functions have zero anti-holomorphic coordinate derivative: `∂_I f = 0` -/
lemma dWirtingerAntiCoord_eq_zero_of_holomorphic_apply {u : ($\iota \rightarrow \mathbb{C}$)}
 (hf : DifferentiableAt $\mathbb{C} f$ u) (I : ι) :
 dWirtingerAntiCoord f I u = 0 := by
 show dWirtingerAntiDir f (Pi.single I 1) u = 0
 exact dWirtingerAntiDir_eq_zero_of_clinear (cllinear_of_holomorphic hf _)

/-!

Coordinate-difference Wirtinger derivatives

The Wirtinger derivatives of the coordinate difference $z^J - \bar{z}^J = 2i \operatorname{Im}(u^J)$, the
combination any function of the coordinates' imaginary parts differentiates against.
Collected here for reuse: $\partial_I (z^J - \bar{z}^J) = \delta_{IJ}$ and $\partial_I (z^J - \bar{z}^J) = -\delta_{IJ}$.

-/

omit [Fintype ι] [DecidableEq ι] in
/-- The coordinate difference $z^J - \bar{z}^J$ as a sum of the holomorphic coordinate

```

and the negated conjugate coordinate – the form on which the ``dWirtingerCoord`` additivity rules apply. -/

```
private lemma coordDiff_eq_add_neg (J : ι) :
 (fun v : (ι → ℂ) => v J - star (v J))
 = (fun v => v J) + (fun v => -(star (v J))) := by
 funext v; simp [sub_eq_add_neg]
```

```
/-- `∂_I (z^J - z̄^J) = δ_IJ`, from `∂_I z^J = δ_IJ` and
`∂_I z̄^J = 0`. -/
```

```
lemma dWirtingerCoord_coordDiff (I J : ι) :
 dWirtingerCoord (fun v : (ι → ℂ) => v J - star (v J)) I
 = fun _ => if I = J then 1 else 0 := by
 funext u
 rw [coordDiff_eq_add_neg, dWirtingerCoord_add_apply (by fun_prop) (by fun_prop)
 dWirtingerCoord_neg_apply I, dWirtingerCoord_coordProj, dWirtingerCoord
 simp
```

```
/-- `∂_I (z^J - z̄^J) = -δ_IJ`, from `∂_I z^J = 0` and
`∂_I z̄^J = δ_IJ`. -/
```

```
lemma dWirtingerAntiCoord_coordDiff (I J : ι) :
 dWirtingerAntiCoord (fun v : (ι → ℂ) => v J - star (v J)) I
 = fun _ => if I = J then -1 else 0 := by
 funext u
 rw [coordDiff_eq_add_neg, dWirtingerAntiCoord_add_apply (by fun_prop) (by fun_prop)
 dWirtingerAntiCoord_neg_apply I, dWirtingerAntiCoord_coordProj, dWirtingerAntiCoord
 by_cases h : I = J <=> simp [h]
```

end

/-!

## D. Wirtinger chain rules for an outer function

Composing with an outer ``g : ℂ → ℂ`` gives a **two-term** coordinate chain rule. The case ``d = Pi.single I 1`` of the foundation ``dWirtingerDir_comp``:

$$\partial_I (g \circ f) = (\partial g / \partial f) \cdot \partial_I f + (\partial g / \partial \bar{f}) \cdot \partial_I \bar{f}.$$

A non-holomorphic ``g`` depends on both its argument and its conjugate, so both contribute: the holomorphic coefficient ``∂g/∂f`` and the anti-holomorphic ``∂g/∂f̄``, each

times the matching inner coordinate derivative – two terms where the complex analytic rule

has one. The outer ``g`` enters only through these two coefficients, the derivatives ``dWirtingerDir g 1`` and ``dWirtingerAntiDir g 1`` evaluated at ``z`` image of ``u`` under the inner function, where the chain rule reads off ``g``).



1.64.

PHYSLIB/MATHEMATICS/CALCULUS/WIRTINGER/COORDINATE.LEAN 117

holomorphic and anti-holomorphic parts of `g`'s real Fréchet derivative: every  $\mathbb{R}$ -linear

map  $\mathbb{C} \rightarrow \mathbb{C}$  splits uniquely as  $h \mapsto a \cdot h + b \cdot \text{star } h$ , and on  $L = \text{fderiv } \mathbb{R} g \ z$  the weights are  $(a, b) = (\partial g / \partial f, \partial g / \partial \bar{f})$ .

For holomorphic `g` the anti-holomorphic coefficient  $\text{dWirtingerAntiDir } g \ 1$  vanishes.  $\text{dWirtingerDir } g \ 1$  collapses to  $\text{deriv } g \ z$ , leaving the single-term rule

$\partial_I (g \circ f) = \text{deriv } g (f \ u) \cdot \partial_I f$  ( $\text{dWirtingerCoord\_comp\_holomorphic\_apply}$ , with  $\partial_I$  dual). Both collapses are the same  $\text{restrictScalars}$  step as §B – the realder of a holomorphic  $g : \mathbb{C} \rightarrow \mathbb{C}$  is  $\mathbb{C}$ -linear ( $\text{clinear\_of\_holomorphic}$  at  $E = \mathbb{C}$ ).

-/

section

variable {g :  $\mathbb{C} \rightarrow \mathbb{C}$ } {f : ( $\iota \rightarrow \mathbb{C}$ )  $\rightarrow \mathbb{C}$ }

/-- The two-term coordinate chain rule for a real-differentiable outer `g`, pointwise  $\partial_I (g \circ f) = (\partial g / \partial f) \cdot \partial_I f + (\partial g / \partial \bar{f}) \cdot \partial_I \bar{f}$ , with coefficients  $\partial g / \partial f = \text{dWirtingerDir } g \ 1 (f \ u)$  and  $\partial g / \partial \bar{f} = \text{dWirtingerAntiDir } g \ 1 (f \ u)$ . The  $d = \text{Pi.single } I \ 1$  case of the foundation  $\text{dWirtingerDir\_comp}$ . The single-term holomorphic specialization  $\text{dWirtingerCoord\_comp\_holomorphic\_apply}$  is proved from this. /

```
lemma dWirtingerCoord_comp_apply {u : ($\iota \rightarrow \mathbb{C}$)}
 (hg : DifferentiableAt $\mathbb{R} g (f \ u)$) (hf : DifferentiableAt $\mathbb{R} f \ u$) (I : ι) :
 dWirtingerCoord (fun v => g (f v)) I u =
 dWirtingerDir g 1 (f u) * dWirtingerCoord f I u
 + dWirtingerAntiDir g 1 (f u) *
 dWirtingerCoord (fun v : ($\iota \rightarrow \mathbb{C}$) => star (f v)) I u :=
 dWirtingerDir_comp hg hf (Pi.single I 1)
```

/-- The single-term coordinate chain rule for a holomorphic outer `g`, pointwise at  $\partial_I (g \circ f) = \text{deriv } g (f \ u) \cdot \partial_I f$ . From the two-term  $\text{dWirtingerCoord\_comp\_apply}$  holomorphic `g` the anti-holomorphic coefficient  $\text{dWirtingerAntiDir } g \ 1 (f \ u)$  vanishes.  $\text{dWirtingerDir } g \ 1 (f \ u)$  collapses to  $\text{deriv } g (f \ u)$ , both off the  $\mathbb{C}$ -linearity

$\text{clinear\_of\_holomorphic}$  (§B). -/

```
lemma dWirtingerCoord_comp_holomorphic_apply {u : ($\iota \rightarrow \mathbb{C}$)}
 (hg : DifferentiableAt $\mathbb{C} g (f \ u)$) (hf : DifferentiableAt $\mathbb{R} f \ u$) (I : ι) :
 dWirtingerCoord (fun v => g (f v)) I u =
 deriv g (f u) * dWirtingerCoord f I u := by
 rw [dWirtingerCoord_comp_apply (hg.hasFDerivAt.restrictScalars $\mathbb{R}$).differentiableAt
 dWirtingerDir_eq_of_clinear (clinear_of_holomorphic hg 1),
 DifferentiableAt.fderiv_restrictScalars $\mathbb{R} hg$, ContinuousLinearMap.coe_restrictScalars]
```

```

 fderiv_apply_one_eq_deriv,
 dWirtingerAntiDir_eq_zero_of_clinear (clinear_of_holomorphic hg 1), zero
 /- Conjugating the inner field swaps the two operators: $\partial_I \bar{f} = \text{conj} (\partial_I f)$
 `d = Pi.single I 1` case of the foundation `dWirtingerDir_star_comp`. -
 /
lemma dWirtingerCoord_star_comp_apply {u : (ι → ℂ)}
 (hf : DifferentiableAt ℝ f u) (I : ι) :
 dWirtingerCoord (fun v : (ι → ℂ) => star (f v)) I u =
 star (dWirtingerAntiCoord f I u) :=
 dWirtingerDir_star_comp hf (Pi.single I 1)

/- The two-term coordinate chain rule for a real-differentiable outer `g`,
holomorphic
version, pointwise at `u`: $\partial_I (g \circ f) = (\partial g / \partial f) \cdot \partial_I f + (\partial g / \partial \bar{f}) \cdot \partial_I \bar{f}$
coefficients as `dWirtingerCoord_comp_apply`, now multiplying the anti-
holomorphic inner
derivatives. The `d = Pi.single I 1` case of the foundation `dWirtingerAntiCoord_comp_holomorphic`
single-term holomorphic specialization `dWirtingerAntiCoord_comp_holomorphic`
from this. -/
lemma dWirtingerAntiCoord_comp_apply {u : (ι → ℂ)}
 (hg : DifferentiableAt ℝ g (f u)) (hf : DifferentiableAt ℝ f u) (I : ι) :
 dWirtingerAntiCoord (fun v => g (f v)) I u =
 dWirtingerDir g 1 (f u) * dWirtingerAntiCoord f I u
 + dWirtingerAntiDir g 1 (f u) *
 dWirtingerAntiCoord (fun v : (ι → ℂ) => star (f v)) I u :=
 dWirtingerAntiDir_comp hg hf (Pi.single I 1)

/- The single-term coordinate chain rule for a holomorphic outer `g`, anti-
holomorphic
version, pointwise at `u`: $\partial_I (g \circ f) = \text{deriv } g (f u) \cdot \partial_I f$. As in
`dWirtingerCoord_comp_holomorphic_apply`, the $\partial g / \partial \bar{f}$ channel vanishes and
collapses to `deriv g (f u)`. -/
lemma dWirtingerAntiCoord_comp_holomorphic_apply {u : (ι → ℂ)}
 (hg : DifferentiableAt ℂ g (f u)) (hf : DifferentiableAt ℝ f u) (I : ι) :
 dWirtingerAntiCoord (fun v => g (f v)) I u =
 deriv g (f u) * dWirtingerAntiCoord f I u := by
 rw [dWirtingerAntiCoord_comp_apply (hg.hasFderivAt.restrictScalars ℝ).dif
 dWirtingerDir_eq_of_clinear (clinear_of_holomorphic hg 1),
 DifferentiableAt.fderiv_restrictScalars ℝ hg, ContinuousLinearMap.coe_restrictScalars,
 fderiv_apply_one_eq_deriv,
 dWirtingerAntiDir_eq_zero_of_clinear (clinear_of_holomorphic hg 1), zero]

/- Conjugating the inner field swaps the two operators: $\partial_I \bar{f} = \text{conj} (\partial_I f)$
`dWirtingerCoord_star_comp_apply`, the `d = Pi.single I 1` case of the foundation
`dWirtingerAntiDir_star_comp`. -/

```

1.64.

PHYSLIB/MATHEMATICS/CALCULUS/WIRTINGER/COORDINATE.LEAN 119

```
lemma dWirtingerAntiCoord_star_comp_apply {u : (ι → ℂ)}
 (hf : DifferentiableAt ℝ f u) (I : ι) :
 dWirtingerAntiCoord (fun v : (ι → ℂ) => star (f v)) I u =
 star (dWirtingerCoord f I u) :=
 dWirtingerAntiDir_star_comp hf (Pi.single I 1)
```

end

/-!

## E. Schwarz's theorem for the coordinate operators

Specialisations of the multivariable theory along  $\text{Pi.single } I \ 1$ : a first coordinate Wirtinger derivative is again real-differentiable, and **Schwarz's theorem** for the second derivative on a  $C^2$  function,

$$\partial_I \partial_J f = \partial_J \partial_I f \quad (\text{dWirtingerCoord\_dWirtingerAntiCoord\_comm})$$

This commutation is Kähler-metric hermiticity: with  $K$  real,  $g_{I\bar{J}} = \partial_I \partial_{\bar{J}} K$  and  $g_{J\bar{I}} = \partial_J \partial_{\bar{I}} K$ .

-/

section

variable {f : (ι → ℂ) → ℂ} {u : (ι → ℂ)}

/-- On a  $C^2$  function the holomorphic coordinate Wirtinger derivative is itself real-differentiable ( $\text{DifferentiableAt } \mathbb{R} (\partial_I f) u$ ) – the  $d = \text{Pi.single } I \ 1$  case of  $\text{differentiableAt\_dWirtingerDir}$ . -/

```
lemma differentiableAt_dWirtingerCoord (hf2 : ContDiffAt ℝ 2 f u) (I : ι) :
 DifferentiableAt ℝ (fun v => dWirtingerCoord f I v) u :=
 differentiableAt_dWirtingerDir hf2 (Pi.single I 1)
```

/-- On a  $C^2$  function the anti-holomorphic coordinate Wirtinger derivative is itself real-differentiable ( $\text{DifferentiableAt } \mathbb{R} (\partial_{\bar{J}} f) u$ ) – the  $d = \text{Pi.single } J \ 1$  case of  $\text{differentiableAt\_dWirtingerAntiDir}$ . -/

```
lemma differentiableAt_dWirtingerAntiCoord (hf2 : ContDiffAt ℝ 2 f u) (J : ι) :
 DifferentiableAt ℝ (fun v => dWirtingerAntiCoord f J v) u :=
 differentiableAt_dWirtingerAntiDir hf2 (Pi.single J 1)
```

/-- **Schwarz's theorem** for the coordinate Wirtinger operators: on a  $C^2$  function holomorphic and anti-holomorphic derivatives commute,  $\partial_I \partial_{\bar{J}} f = \partial_{\bar{J}} \partial_I f$ . The  $d = \text{Pi.single } I \ 1$ ,  $e = \text{Pi.single } J \ 1$  case of the general  $\text{dWirtingerDir\_dWirtingerAntiDir\_comm}$ . -/

```
theorem dWirtingerCoord_dWirtingerAntiCoord_comm (hf2 : ContDiffAt ℝ 2 f u) (I J : ι)
```

```

 dWirtingerCoord (fun v => dWirtingerAntiCoord f J v) I u
 = dWirtingerAntiCoord (fun v => dWirtingerCoord f I v) J u :=
 dWirtingerDir_dWirtingerAntiDir_comm hf2 (Pi.single I 1) (Pi.single J 1)

end

end Physlib.Wirtinger
end
end

```

### 1.65 Physlib/Mathematics/ConjModule.lean

```

/-
Copyright (c) 2026 Andrea Pari. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Andrea Pari
-/
module

public import Mathlib.Algebra.Module.Equiv.Defs
public import Mathlib.Algebra.Star.Module
public import Mathlib.LinearAlgebra.Basis.Defs
public import Mathlib.Tactic.Ring

/-!

The conjugate module

Over a commutative star-ring `k`, the conjugate module `ConjModule M` of
module `M` is the
same additive group with the scalar action twisted by conjugation: $r \cdot v$:
sesquilinear pairing into a bilinear one: a map conjugate-
linear in a slot `M` is linear in the slot
`ConjModule M`.

`ConjModule M` is a type synonym carrying a fresh `Module k` instance (`Mod
`starRingEnd k`), so the twisted action does not leak onto `M` and vice ver
conjugate-linear identity `conjEquiv : M  $\approx_{s1}$  [starRingEnd k] ConjModule M`,
`ConjModule (ConjModule M)  $\approx_1$  [k] M`, and the transported basis `Basis.conj`

Key results

- `ConjModule` : the conjugate-module type synonym, with its twisted `Modul
- `conjEquiv` : the canonical conjugate-linear equivalence `M $\approx_{s1}$ [starRingE

```

```

- `ConjModule.involution` : the involution $\text{ConjModule } (\text{ConjModule } M) \approx_{\mathbb{1}[k]} M'$.
- `Basis.conj` : a basis of M' transported to a basis of $\text{ConjModule } M'$ (coordinate

-/

@[expose] public section

open Module

variable {k : Type*} [CommRing k] [StarRing k]
variable {M : Type*} [AddCommGroup M] [Module k M]

/-- The conjugate module of M' : the same additive group with the scalar action twisted
conjugation, $r \cdot v = \text{star } r \cdot v'$. A type synonym so the twisted action stays off M'
/
def ConjModule (M : Type*) := M

namespace ConjModule

instance : AddCommGroup (ConjModule M) := inferInstanceAs (AddCommGroup M)

/-- The twisted action $r \cdot v = \text{star } r \cdot v'$, obtained by restricting scalars along the
conjugation ring endomorphism $\text{starRingEnd } k'$. -/
instance instModule : Module k (ConjModule M) :=
 Module.compHom M (starRingEnd k)

end ConjModule

/-- The canonical conjugate-linear equivalence $M \approx_{\text{sl}[\text{starRingEnd } k]} \text{ConjModule } M'$,
the underlying additive group. -/
def conjEquiv : M $\approx_{\text{sl}[\text{starRingEnd } k]}$ ConjModule M where
 toFun v := v
 map_add' _ _ := rfl
 map_smul' r v := by show (r • v : M) = (star (star r) • v : M); rw [star_star]
 invFun v := v
 left_inv _ := rfl
 right_inv _ := rfl

namespace ConjModule

/-- Conjugating twice returns the original module: the k' -
linear isomorphism
 $\text{ConjModule } (\text{ConjModule } M) \approx_{\mathbb{1}[k]} M'$. It is k' -linear, not merely semilinear, because
 $\text{starRingEnd } k'$ composed with itself is the identity. -/
def involution : ConjModule (ConjModule M) $\approx_{\mathbb{1}[k]}$ M :=
 ((conjEquiv (k := k) (M := M)).trans (conjEquiv (k := k) (M := ConjModule M))).sym

```

```

variable {ι : Type*}

/-- Coordinate-wise conjugation on `ι →₀ k`, a conjugate-
linear self-equivalence. -/
noncomputable def starFinsupp : (ι →₀ k) ≈ₛι[starRingEnd k] (ι →₀ k) where
 toFun f := f.mapRange star (star_zero k)
 invFun f := f.mapRange star (star_zero k)
 map_add' f g := by ext i; simp [Finsupp.mapRange_apply, star_add]
 map_smul' r f := by
 ext i
 simp only [Finsupp.mapRange_apply, Finsupp.coe_smul, Pi.smul_apply, smul_assoc,
 starRingEnd_apply, star_mul']
 left_inv f := by ext i; simp [Finsupp.mapRange_apply]
 right_inv f := by ext i; simp [Finsupp.mapRange_apply]

/-- A basis of `M` transported to a basis of `ConjModule M`: the same basis
coordinates conjugated (`(Basis.conj b).repr v = star ∘ b.repr v`). -
/
noncomputable def _root_.Basis.conj (b : Basis ι k M) : Basis ι k (ConjModule M) :=
 Basis.ofRepr
 (((conjEquiv (k := k) (M := M)).symm.trans b.repr).trans starFinsupp)

/-- Coordinates in `Basis.conj b` are the `star` of the coordinates in `b`.
/
@[simp] lemma _root_.Basis.conj_repr_apply (b : Basis ι k M) (v : ConjModule M) :
 (Basis.conj b).repr v i = star (b.repr ((conjEquiv (k := k) (M := M)).symm v) i) := by
 simp

/-- The basis vectors of `Basis.conj b` are those of `b`, viewed through `c`.
/
@[simp] lemma _root_.Basis.conj_apply (b : Basis ι k M) (i : ι) :
 Basis.conj b i = conjEquiv (k := k) (M := M) (b i) := by
 apply (Basis.conj b).repr.injective
 ext j
 rcases eq_or_ne j i with h | h
 · subst h; simp [Basis.conj_repr_apply]
 · simp [Basis.conj_repr_apply, Finsupp.single_eq_of_ne, h]

end ConjModule

end

```

**1.66 Physlib/Mathematics/CrossProductMatrix.lean**

```

/-
Copyright (c) 2026 Giuseppe Sorge. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Giuseppe Sorge
-/
module

public import Mathlib.Data.Matrix.Mul
public import Mathlib.Data.Real.Basic
public import Mathlib.LinearAlgebra.CrossProduct
public import Mathlib.LinearAlgebra.Matrix.Notation
/-!

The hat map on three-dimensional vectors

The hat map sends a vector $\omega : \text{Fin } 3 \rightarrow \mathbb{R}$ to the skew-symmetric matrix $[\omega]_\times$ characterised by $[\omega]_\times * v = \omega \times_3 v$. It realises the correspondence between $\mathbb{R}^3$ and the skew-symmetric 3×3 matrices (the Lie algebra $\mathfrak{so}(3)$), and underlies the angular velocity of a rigid body.

-/

@[expose] public section

namespace Matrix

/-- The hat map $[\omega]_\times$: the skew-symmetric 3×3 matrix acting as the cross product, i.e. $[\omega]_\times * v = \omega \times_3 v$. -/
def crossProductMatrix ($\omega : \text{Fin } 3 \rightarrow \mathbb{R}$) : Matrix (Fin 3) (Fin 3) $\mathbb{R} :=$
 !![0, - ω 2, ω 1; ω 2, 0, - ω 0; - ω 1, ω 0, 0]

/-- The hat matrix acts on a vector as the cross product. -/
/
@[simp]
lemma crossProductMatrix_mulVec ($\omega v : \text{Fin } 3 \rightarrow \mathbb{R}$) :
 crossProductMatrix $\omega * v = \omega \times_3 v :=$ by
 funext i
 fin_cases i <|>
 simp [crossProductMatrix, mulVec, dotProduct, Fin.sum_univ_three, cross_apply] <|>

/-- The hat map produces skew-symmetric matrices. -/
@[simp]
lemma crossProductMatrix_transpose ($\omega : \text{Fin } 3 \rightarrow \mathbb{R}$) :
 (crossProductMatrix ω)T = - crossProductMatrix $\omega :=$ by

```

```

 ext i j
 fin_cases i <;> fin_cases j <;> simp [crossProductMatrix]

/-- The vee map: reads the vector off a `3 × 3` matrix. It is a left inverse
/
def crossProductVee (A : Matrix (Fin 3) (Fin 3) ℝ) : Fin 3 → ℝ := ![A 2 1,

/-- The vee map is a left inverse of the hat map. -/
@[simp]
lemma crossProductVee_crossProductMatrix (ω : Fin 3 → ℝ) :
 crossProductVee (crossProductMatrix ω) = ω := by
 funext i
 fin_cases i <;> simp [crossProductVee, crossProductMatrix]

end Matrix

```

## 1.67 Physlib/Mathematics/DataStructures/FourTree/Basic.lean

```

simp only [card, toMultiset, Multiset.card_bind, Function.comp_apply, Multiset.toMultiset]
rfl
have leaf_iff : ∀ (l : Leaf α4) (y : α4), l.mem y ↔ l.1 = y := by
 rintro ⟨_⟩ _
 rfl
have twig_iff : ∀ (t : Twig α3 α4) (y : α3 × α4),
 t.mem y ↔ t.1 = y.1 ∧ ∃ l ∈ t.2, l.mem y.2 := by
 rintro ⟨_, _⟩ _
 rfl
have branch_iff : ∀ (b : Branch α2 α3 α4) (y : α2 × α3 × α4),
 b.mem y ↔ b.1 = y.1 ∧ ∃ t ∈ b.2, t.mem y.2 := by
 rintro ⟨_, _⟩ _
 rfl
have trunk_iff : ∀ (k : Trunk α1 α2 α3 α4) (y : α1 × α2 × α3 × α4),
 k.mem y ↔ k.1 = y.1 ∧ ∃ b ∈ k.2, b.mem y.2 := by
 rintro ⟨_, _⟩ _
 rfl
obtain ⟨trunks⟩ := T
show (∃ trunk ∈ trunks, trunk.mem x) ↔ x ∈ (root trunks).toMultiset
simp only [toMultiset, Multiset.mem_bind, Multiset.mem_map,
 trunk_iff, branch_iff, twig_iff, leaf_iff, Prod.ext_iff]
tauto
simp only [toMultiset, Multiset.mem_bind, Multiset.mem_map]
exact ⟨trunk, trunk_mem, branch, branch_mem, twig, twig_mem, leaf, leaf_mem⟩

```



1.68.

PHYSLIB/MATHEMATICS/DATASTRUCTURES/FOURTREE/UNIQUEMAP.lean

## 1.68 Physlib/Mathematics/DataStructures/FourTree/UniqueMap.lean

```
apply mem_of_parts (trunk.uniqueMap4 f) (branch.uniqueMap4 f) (twig.uniqueMap4 f)
 (.leaf (f leaf.1))
· exact Multiset.mem_map_of_mem _ htrunk
· exact Multiset.mem_map_of_mem _ hbranch
· exact Multiset.mem_map_of_mem _ htwig
 refine {leaf, hleaf, ?_, rfl}
 intro y hy hn
 exact hnotMem
 (mem_of_parts trunk branch twig y htrunk hbranch htwig hy (by subst heq; simp
· subst heq
 simp [Trunk.uniqueMap4, Branch.uniqueMap4, Twig.uniqueMap4]
simp only [Twig.uniqueMap4, Multiset.mem_map, Multiset.mem_filterMap,
subst heq
refine {trunkT.1, branchT.1, twigT.1, leafT.1, ?_,
 mem_of_parts trunkT branchT twigT leafT trunkT_mem branchT_mem twigT_mem leafT_m
rw [← hPresent]
simp [Trunk.uniqueMap4, Branch.uniqueMap4, Twig.uniqueMap4, hQ10.2]
apply mem_of_parts (trunk.uniqueMap3 f) ((Branch.branch qHu twigs).uniqueMap3 f)
 (.twig (f Q5) (leafs.filter (fun (.leaf Q10) =>
 → Branch.mem (.branch qHu twigs) (qHu, f Q5, Q10)))) leaf
· exact Multiset.mem_map_of_mem _ htrunk
· exact Multiset.mem_map_of_mem _ hbranch
· exact Multiset.mem_map_of_mem _ htwig
· refine Multiset.mem_filter.mpr {hleaf, ?_}
 show → (Branch.branch qHu twigs).mem (qHu, f Q5, leaf.1)
 by_contra hn
 exact {trunk, htrunk, rfl, .branch qHu twigs, hbranch, hn}
subst heq
refine {trunkT.1, branchT.1, twigT.1, leafI.1, ?_,
 mem_of_parts trunkT branchT twigT leafI trunkT_mem branchT_mem twigT_mem leftI_m
simp [Trunk.uniqueMap3, Branch.uniqueMap3]
```

## 1.69 Physlib/Mathematics/DataStructures/Matrix/LieTrace.lean

```
rw [← NormedSpace.exp_sum Finset.univ]
 rw [h_conj, Unitary.coe_star, trace_unitary_conj]
 rw [h_conj, Unitary.coe_star, det_exp_unitary_conj]
erw [Matrix.map_smul, Matrix.map_pow A (algebraMap ℝ ℂ) k]
 rw [Complex.coe_algebraMap, ← Complex.ofReal_exp]
```

### 1.70 Physlib/Mathematics/Distribution/Basic.lean

```
public import Mathlib.Topology.Algebra.Module.Spaces.PointwiseConvergenceCL
- `Distribution.ofFiniteMeasure` is the scalar distribution associated to a
 exact s.le_sup hkn
 add_apply]
 simp only [add_apply, ContinuousLinearEquiv.neg_apply, neg_add_rev,
- Distributions associated to finite measures.
 simp only [zero_apply, neg_eq_zero]
E.2. Distributions associated to finite measures
```

Every finite measure has temperate growth, so integrating Schwartz maps against a scalar distribution.

```
-/
section finiteMeasure

open MeasureTheory

variable [NormedSpace ℝ E] [MeasurableSpace E] [BorelSpace E] [SecondCountableTopology E]

/-- The scalar distribution associated to a finite measure, acting on a Schwartz function. -/
def ofFiniteMeasure (μ : Measure E) [IsFiniteMeasure μ] : E →d[ℝ] ℝ :=
 integralCLM · μ

@[simp]
lemma ofFiniteMeasure_apply (μ : Measure E) [IsFiniteMeasure μ] (η : ℝ(E, ℝ)) :
 ofFiniteMeasure μ η = ∫ x, η x ∂μ :=
 rfl

end finiteMeasure

/-!

E.3. The dirac delta distribution
E.4. The heaviside step function
```

### 1.71 Physlib/Mathematics/Distribution/PowMul.lean

```
public import Mathlib.Analysis.Calculus.ContDiff.Bounds
| 0 => simp
| 1 =>
 rw [norm_iteratedFDeriv_one, RCLike.ofRealCLM.fderiv]
```

```

simp
| (n + 2) =>
 have h : fderiv ℝ ↑(RCLike.ofRealCLM (K := [])) = fun _ => RCLike.ofRealCLM := by
 ext1 y
 exact RCLike.ofRealCLM.fderiv
 rw [← norm_iteratedFDeriv_fderiv, h, iteratedFDeriv_const_of_ne n.succ_ne_zero]
simp

```

## 1.72 Physlib/Mathematics/FDerivCurry.lean

```

simp only [add_apply, ContinuousLinearMap.coe_comp,
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply]
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply]
apply Differentiable.comp <=> fun_prop
apply Differentiable.comp <=> fun_prop
simp only [add_apply, ContinuousLinearMap.coe_comp,
simp only [add_apply, ContinuousLinearMap.coe_comp,

```

## 1.73 Physlib/Mathematics/Fin.lean

```

 ⟨x.val - 1, by omega⟩
simp only [predAboveI, Fin.succAbove, Fin.ext_iff, apply_dite Fin.val, apply_ite F
 Fin.lt_def, Fin.val_castSucc, Fin.val_succ]
split_ifs <=> omega

replace h := Fin.val_ne_of_ne h
simp only [Fin.succAbove, predAboveI, Fin.ext_iff, apply_dite Fin.val, apply_ite F
 Fin.lt_def, Fin.val_castSucc, Fin.val_succ]
split_ifs <=> omega
rw [← (predAboveI_eq_iff (Fin.succAbove_ne i j) (j.predAbove i)).mpr
 (Fin.succAbove_succAbove_predAbove i j),
 Fin.succAbove_succAbove_succAbove_predAbove]
rw [Equiv.apply_eq_iff_eq_symm_apply]
have hσ : σ i ≠ σ (i.succAbove x) := by simp
simp only [Function.comp_apply, finExtractOnPermHom, finExtractOne_symm_inr_apply,
 Equiv.symm_apply_apply, succsAbove_predAboveI hσ, predAboveI_succAbove, id_eq]
have hσ : e i ≠ e (i.succAbove x) := by simp
simp only [Function.comp_apply, finExtractOnePerm, finExtractOnPermHom, Equiv.coe_
 finExtractOne_symm_inr_apply, succsAbove_predAboveI hσ]
simp [finExtractTwo]
left_inv i := by induction i using Fin.cases <=> simp
right_inv i := by induction i using Fin.cases <=> simp
equivCons e 0 = 0 := rfl

```

```

ext x
induction x using Fin.cases <=> rfl
ext x
induction x using Fin.cases <=> rfl
 (Fin.equivCons e).symm (i + 1, hi) = (e.symm (i, Nat.succ_lt_succ_iff.mpr hi)).symm
 (Fin.equivCons e) (i + 1, hi) = (e (i, Nat.succ_lt_succ_iff.mpr hi)).succ

```

### 1.74 Physlib/Mathematics/Fin/Involutions.lean

```

· simp only [succ_eq_add_one, h, ↓reduceDite, Fin.succ_pred, f.2 i.succ,
 dite_eq_ite, ite_eq_right_iff]
exact fun h => False.elim (Fin.succ_ne_zero i h),
· rw [Function.update_apply, if_neg hja]
 have hjf1 : f.1.1 j ≠ a := fun hn => hja (by rw [← f.1.2 j, hn, h])
have hpred (j : Fin n) (hj : f j.succ ≠ 0) :
 (↑(if h : f j.succ = 0 then j else (f j.succ).pred h) + 1 : ℕ) = ↑(f j.succ)
rw [dif_neg hj, Fin.val_pred]
have hv : (f j.succ).val ≠ 0 := fun h => hj (Fin.ext (by simp using omega))
refine hpred j fun hj => Fin.succ_ne_zero j (hf.injective (hj.trans h))
 refine hpred j fun hj => hf' ?_
 rw [← hj, hf]
 rw [if_neg (Fin.succ_ne_zero (f i))]
 rw [if_neg (Fin.succ_ne_zero (f i))]
 rw [Option.eq_some_iff_get_eq]
 simp [hs]
 rintro (_ | i, h) <=> rfl
 rintro (_ | (i, h)) <=> rfl }
by_contra hf0
simp only [Fin.zero_eta] at hf0
Option.isSome_some, Option.get_some, Option.isSome_none, dif_neg hf0] at hf0
exact absurd h (Option.some_ne_none _)
have f_succ_succ_ne_zero_one (i : Fin n) : f.1 i.succ.succ ≠ 0 ∧ f.1 i.succ.succ = 0 := by
 have hf1 : f.1 1 = 0 := by
 simp only [← f.2.2.2]
 rw [f.2.1]
 refine (fun hn => ?_, fun hn => ?_)
 · simp [Fin.ext_iff] using f.2.1.injective (hn.trans hf1.symm)
 · simp [Fin.ext_iff] using f.2.1.injective (hn.trans f.2.2.2.symm)
have hf' (i : Fin n) : f' i ≠ 0 := (f_succ_succ_ne_zero_one i).1
rw [ne_eq, Fin.pred_eq_iff_eq_succ, Fin.succ_zero_eq_one]
exact (f_succ_succ_ne_zero_one i).2
 simp [Fin.ext_iff] at h
 have ht : f.1 (m, by omega) = (x, by omega) := Fin.ext h

```

1.75.

PHYSLIB/MATHEMATICS/GEOMETRY/METRIC/PSEUDORIEMANNIAN/DEFS.lean

```
rw [← ht, f.2.1]
```

### 1.75 Physlib/Mathematics/Geometry/Metric/PseudoRiemannian/Defs.lean

```
simp only [ContinuousLinearMap.map_smul, _root_.smul_apply, smul_smul]
 (fun a v y => by simp only [map_smul, smul_apply]; ring)
 (fun a v y => by simp only [map_smul, smul_apply]; ring),
 add_apply, symm x]
simp only [map_smul, FunLike.coe_smul, Pi.smul_apply, smul_eq_mul, LinearMap.coe
 AddHom.coe_mk, RingHom.id_apply, LinearMap.smul_apply]
 have : T2Space (TangentSpace I x) := inferInstanceAs (T2Space E)
have : T2Space (TangentSpace I x) := inferInstanceAs (T2Space E)
 add_apply,
 root.smul_apply,
 root.smul_apply,
 simp only [cotangentMetricVal, map_smul, smul_apply]; ring)
 simp only [cotangentMetricVal, map_smul, smul_apply]; ring),
 simp only [ContinuousLinearMap.map_add, add_apply]
```

### 1.76 Physlib/Mathematics/Geometry/Metric/Riemannian/Defs.lean

```
simp only [inner_apply, map_add, add_apply]
```

### 1.77 Physlib/Mathematics/InnerProductSpace/Basic.lean

```
 apply le_of_eq_of_le (b := $\sqrt{c} * \|x\|_{Lp}$)
 · simp [WithLp.equiv_apply]
 ring
 · apply h.trans
 rfl
exact fun v => h (WithLp.ofLp v)
· rw [← ofLp_inner_left]
 simp only
 congr
 change _ = (WithLp.linearEquiv 2 $\square$ E) _
 simp only [map_sum, WithLp.linearEquiv_apply, AddEquiv.toEquiv_eq_coe, Equiv.toF
 EquivLike.coe_coe, WithLp.addEquiv_apply]
· rfl
 simp only [smul_eq_mul] at h11 h12 h21 h22
 refine {?, ?}
 · rcases le_total $\|x\| \|y\|$ with h | h
 · rw [max_eq_right h]
```

```

 nlinarith [h21 y, min_le_right c1 c2, sq_nonneg ||y||]
 · rw [max_eq_left h]
 nlinarith [h11 x, min_le_left c1 c2, sq_nonneg ||x||]
 · rcases le_total (re [x,x]) (re [y,y]) with h | h
 · nlinarith [h22 y, le_max_right d1 d2, sq_nonneg ||y||, norm_nonneg
 pow_le_pow_left₀ (norm_nonneg y) (le_max_right ||x|| ||y||) 2]
 · nlinarith [h12 x, le_max_left d1 d2, sq_nonneg ||x||, norm_nonneg x
 pow_le_pow_left₀ (norm_nonneg x) (le_max_left ||x|| ||y||) 2]
 · exact Finset.sum_nonneg fun i _ => sq_nonneg _
 (s := {i}) (fun _ => InnerProductSpace.Core.inner_self_nonneg)
 exact le_of_eq (Real.sq_sqrt InnerProductSpace.Core.inner_self_no
 exact Real.sq_sqrt InnerProductSpace.Core.inner_self_nonneg
 exact (sq_le_sq₀ (norm_nonneg (x j)) (norm_nonneg (x i))).mpr (hj j)
 · rw [not_nonempty_iff] at hnEmpty
 refine {1, 1, zero_lt_one, zero_lt_one, fun x => ?_}
 rw [Subsingleton.elim x 0]
 simp [norm, inner]
 · change |@inner ℝ (WithLp 2 E) _ _ _| ≤ _
 have h1 := norm_inner_le_norm (· := ℝ) (E := WithLp 2 E) (WithLp.toLp
 simp at h1
 apply h1.trans
 congr
 · rw [norm_withLp2_eq_norm2]
 rfl
 · rw [norm_withLp2_eq_norm2]
 rfl
 · have key (z : E) : ||z||2 ≤ √ d * ||z|| := by
 apply le_of_sq_le_sq
 · simp [@mul_pow]
 rw [norm2_sq_eq_re_inner (· := ℝ)]
 simp only [re_to_real]
 apply (h z).2.trans
 apply le_of_eq
 simp only [mul_eq_mul_right_iff, ne_eq, OfNat.ofNat_ne_zero, not_
 pow_eq_zero_iff, norm_eq_zero]
 left
 refine Eq.symm (Real.sq_sqrt ?_)
 linarith
 · positivity
 have h1 := key x
 have h2 := key y
 trans (√ d * ||x||) * (√ d * ||y||)
 · exact mul_le_mul_of_nonneg h1 h2 (by positivity) (by positivity)
 · apply le_of_eq
 ring_nf
 rw [Real.sq_sqrt]

```

```

· ring
· linarith

```

## 1.78 Physlib/Mathematics/KroneckerDelta.lean

```

public import Mathlib.LinearAlgebra.Matrix.Determinant.Basic
· simp
· simp [two_nsmul]
simp [kroneckerDelta]
simp [kroneckerDelta]
/-!

```

# Generalized Kronecker delta

```

-/

```

```

section Generalized
open Matrix

```

```

/-- Integer-valued Kronecker entry via the existing `kroneckerDelta`. -
/
local notation "δℤ" => (fun ρ σ => ((kroneckerDelta ρ σ : ℕ) : ℤ))

```

```

/-- Generalized Kronecker delta:
`δ^{μ₁...μₙ}_{ν₁...νₙ} = det (δ[μᵢ, νⱼ])`.

```

```

This is defined for any finite type `α` with decidable equality. -
/

```

```

def generalizedKroneckerDelta {α ι : Type} [DecidableEq α]
 [DecidableEq ι] [Fintype ι]
 (μ : ι → α) (ν : ι → α) : ℤ :=
 Matrix.det (fun i j => δℤ (μ i) (ν j))

```

```

/-- Swapping two of the upper indices of the generalized Kronecker delta negates it.
This is one row transposition of the underlying determinant. -
/

```

```

lemma generalizedKroneckerDelta_swap {α ι : Type} [DecidableEq α] [DecidableEq ι] [Fintype ι]
 (μ ν : ι → α) {i j : ι} (hij : i ≠ j) :
 generalizedKroneckerDelta (μ ∘ Equiv.swap i j) ν = - generalizedKroneckerDelta μ ν
rw [show generalizedKroneckerDelta (μ ∘ Equiv.swap i j) ν
 = (Matrix.submatrix (fun a b => ((kroneckerDelta (μ a) (ν b) : ℕ) : ℤ))
 (Equiv.swap i j) id).det from rfl,
 Matrix.det_permute, Equiv.Perm.sign_swap hij]
simp [generalizedKroneckerDelta]

```

end Generalized

## 1.79 Physlib/Mathematics/LinearMaps.lean

```
with definitions from Mathlib."
coe_injective f g h := by
coe_injective f g h := by
coe_injective f g h := by
coe_injective f g h := by
```

## 1.80 Physlib/Mathematics/LinearPMap.lean

```
/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
module

public import Mathlib.Analysis.InnerProductSpace.LinearPMap
/-!

LinearPMap

i. Overview

In this module we collect some basic results about `LinearPMap`s.

Most important is the definition of restricted composition.
The composition of two partial linear maps `g : F →ℓ[R] G` and `f : E →ℓ[R] F`
only if the range of `f` is contained in the domain of `g` (c.f. `LinearPMap.compRestricted`
`g.compRestricted f` (`g ∘r f`) is defined to be the composition of `g` with
to exactly those `x : f.domain` for which `f x ∈ g.domain`. This allows one
composition of partial linear maps while having the domain implicitly account

ii. Key results

- `LinearPMap.sum` : The finite sum of partial linear maps.
- `LinearPMap.compRestricted` (`∘r`) : For two partial linear maps
 `g : F →ℓ[R] G` and `f : E →ℓ[R] F`, the composition of `g` with `f`
 with natural domain `{x : f.domain | f x ∈ g.domain}`.
```



- `LinearPMap.instMonoid` : Partial linear maps  $E \rightarrow_{\perp} [R] E$  with `compRestricted` for multiplication and the identity map for `1` comprise a monoid.

## iii. Table of contents

- A. Inequalities
- B. Zero smul
- C. Finite sums
- D. Restricted composition
- E. Monoid
- F. Inverses

## iv. References

-/

@[expose] public section

namespace LinearPMap

open Submodule

variable {R : Type\*} [Ring R]

variable {E : Type\*} [AddCommGroup E] [Module R E]

variable {F : Type\*} [AddCommGroup F] [Module R F]

/-!

## A. Inequalities

-/

section Inequalities

variable (f f<sub>1</sub> f<sub>2</sub> f<sub>3</sub> : E →<sub>⊥</sub>. [R] F) {g g<sub>1</sub> g<sub>2</sub> : E →<sub>⊥</sub>. [R] F}

lemma sub\_le\_zero : f - f ≤ 0 := ⟨le\_top, by simp [sub\_apply]⟩

lemma neg\_add\_le\_zero : -f + f ≤ 0 := ⟨le\_top, by simp [add\_apply]⟩

lemma le\_iff\_neg\_le\_neg : g<sub>1</sub> ≤ g<sub>2</sub> ↔ -g<sub>1</sub> ≤ -g<sub>2</sub> :=  
 ⟨fun ⟨h, h'⟩ ↦ ⟨h, fun \_ \_ h' ↦ by simp [h' h']⟩, fun ⟨h, \_⟩ ↦ ⟨h, fun \_ \_ \_ ↦ b

lemma le\_neg\_iff\_neg\_le : g<sub>1</sub> ≤ -g<sub>2</sub> ↔ -g<sub>1</sub> ≤ g<sub>2</sub> := by rw [le\_iff\_neg\_le\_neg, neg\_neg]

lemma add\_sub\_le\_cancel : f<sub>1</sub> + (f<sub>2</sub> - f<sub>1</sub>) ≤ f<sub>2</sub> :=  
 ⟨by simp [add\_domain, sub\_domain], fun \_ \_ h ↦ by simp [add\_apply, sub\_apply, h]⟩

```

lemma add_sub_le_cancel_left : $f_1 + f_2 - f_1 \leq f_2$:= add_sub_assoc f1 f2 f1

lemma add_sub_le_cancel_right : $f_1 + f_2 - f_2 \leq f_1$:= add_comm f1 f2 ▸ add_s

lemma add_add_sub_le_cancel : $f_1 + f_2 + (f_3 - f_2) \leq f_1 + f_3$:=
 (fun _ _ ↦ by simp_all [add_domain, sub_domain], fun _ _ h ↦ by simp [add

lemma add_sub_sub_le_cancel : $f_1 + f_2 - (f_1 - f_3) \leq f_2 + f_3$:=
 (fun _ _ ↦ by simp_all [add_domain, sub_domain], fun _ _ h ↦ by simp [add

lemma sub_sub_sub_le_cancel_right : $f_1 - f_2 - (f_3 - f_2) \leq f_1 - f_3$:= by
 simp only [sub_eq_add_neg, neg_add]
 exact sub_eq_add_neg (-f3) (-f2) ▸ add_add_sub_le_cancel f1 (-
f2) (-f3)

lemma sub_sub_sub_le_cancel_left : $f_1 - f_2 - (f_1 - f_3) \leq f_3 - f_2$:=
 sub_eq_add_neg f1 f2 ▸ neg_add_eq_sub f2 f3 ▸ add_sub_sub_le_cancel f1 (-
f2) f3

lemma sub_le_of_le_add (h : $g \leq g_1 + g_2$) : $g - g_2 \leq g_1$:= by
 constructor
 · exact (inf_le_of_left_le le_refl).trans (le_inf_iff.mp <| add_domain g1
 · intro ⟨x, hx⟩ ⟨y, hy⟩ rfl
 simp [sub_apply, @h.2 ⟨x, hx.1⟩ ⟨x, ⟨hy, hx.2⟩⟩ rfl, add_apply]

lemma sub_add_le_cancel : $f_1 - f_2 + f_2 \leq f_1$:=
 sub_eq_add_neg f1 f2 ▸ sub_neg_eq_add _ f2 ▸ add_sub_le_cancel_right f1 (
f2)

lemma add_le_of_le_sub (h : $g \leq g_1 - g_2$) : $g + g_2 \leq g_1$:=
 sub_neg_eq_add g g2 ▸ sub_le_of_le_add (sub_eq_add_neg g1 g2 ▸ h)

lemma add_left_le_of_le (h : $g_1 \leq g_2$) : $f + g_1 \leq f + g_2$:= by
 constructor
 · simp only [add_domain, le_inf_iff, inf_le_left, true_and]
 exact (inf_le_of_right_le le_refl).trans h.1
 · intro x y hxy
 simp_rw [add_apply, @h.2 ⟨x, x.2.2⟩ ⟨y, y.2.2⟩ hxy, hxy]

lemma add_right_le_of_le (h : $g_1 \leq g_2$) : $g_1 + f \leq g_2 + f$:=
 add_comm f g1 ▸ add_comm f g2 ▸ add_left_le_of_le f h

lemma sub_right_le_of_le (h : $g_1 \leq g_2$) : $g_1 - f \leq g_2 - f$:=
 sub_eq_add_neg g1 f ▸ sub_eq_add_neg g2 f ▸ add_right_le_of_le (-
f) h

```

```

lemma sub_left_le_of_le (h : $g_1 \leq g_2$) : $f - g_1 \leq f - g_2$:=
 neg_sub g_1 f ▸ neg_sub g_2 f ▸ le_iff_neg_le_neg.mp (sub_right_le_of_le f h)

end Inequalities

/-!
B. Zero smul
-/

section

variable { α : Type*} [Field α] [Module α E] [Module α F]

lemma zero_smul_le (f : $E \rightarrow_{\perp} [\alpha] F$) : $(0 : \alpha) \cdot f \leq 0$:= ⟨le_top, by simp⟩

@[simp]
lemma zero_smul_eq {f : $E \rightarrow_{\perp} [\alpha] F$ } (h : f.domain = $\top$) : $(0 : \alpha) \cdot f = 0$:=
 eq_of_le_of_domain_eq f.zero_smul_le h

end

/-!
C. Finite sums
-/

section Sums

variable { α : Type*} [Fintype α] (f : $\alpha \rightarrow E \rightarrow_{\perp} [R] F$)

/-- A finite sum of partial linear maps.

 `sum f` and ` $\sum a, f a$ ` are equal, but not by definition.
 With `sum f` both `domain` and `toFun` are made explicit. -
/
def sum : $E \rightarrow_{\perp} [R] F$ where
 domain := $\bigcap a, (f a).domain$
 toFun := $\sum a, (f a).toFun \circ_{\perp} inclusion$ (fun _ _ ↦ by simp_all only [mem_iInf])

lemma sum_domain : (sum f).domain = $\bigcap a, (f a).domain$:= rfl

lemma sum_domain_le (a : α) : (sum f).domain $\leq$ (f a).domain := fun _ _ ↦ by simp_all

@[simp]
lemma sum_apply (ψ : (sum f).domain) : sum f ψ = $\sum a, f a \langle \psi, sum_domain_le f a \psi.2 \rangle$
 simp [sum, inclusion_apply]

```

end Sums

```
/-!
D. Restricted composition
-/
```

section Composition

```
variable {G : Type*} [AddCommGroup G] [Module R G]
variable (g g1 g2 : F →1.[R] G) (f f1 f2 : E →1.[R] F)
variable {v : F →1.[R] G} {u : E →1.[R] F}
```

```
/-- `g ∘r f` is the composition of `g` with `f` restricted to a domain consisting of
`x : f.domain` for which `f x ∈ g.domain`. -/
```

```
def compRestricted : E →1.[R] G :=
 g.comp (f.domRestrict <| (g.domain.comap f.toFun).map f.domain.subtype) (
 intro ⟨x, h, _⟩
 simp only [map_coe, subtype_apply, comap_coe, Set.mem_image, Set.mem_preimage,
 toFun_eq_coe, SetLike.mem_coe] at h
 obtain ⟨y, hy, hy'⟩ := h
 rw [domRestrict_apply hy'.symm]
 exact hy)
```

```
@[inherit_doc compRestricted]
infixr:80 " ∘r " => compRestricted
```

```
lemma compRestricted_domain_le : (g ∘r f).domain ≤ f.domain := fun _ h => h.
```

```
lemma compRestricted_domain : (g ∘r f).domain = (g.domain.comap f.toFun).domain
 change (f.domRestrict <| (g.domain.comap f.toFun).map f.domain.subtype).domain
 rw [domRestrict_domain]
 refine inf_of_le_left ?_
 intro x h
 simp only [mem_map, mem_comap, toFun_eq_coe, subtype_apply, Subtype.exists,
 exists_eq_right] at h
 exact h.choose
```

```
lemma mem_compRestricted_domain_iff {x : E} :
 x ∈ (v ∘r u).domain ↔ ∃ h : x ∈ u.domain, u ⟨x, h⟩ ∈ v.domain := by
 simp [compRestricted_domain]
```

```
lemma mem_compRestricted_domain_iff' {x : E} :
 x ∈ (v ∘r u).domain ↔ ∃ y : u.domain, x = y ∧ ∃ y' : v.domain, u y = y'
 simp [mem_compRestricted_domain_iff]
```

```
lemma mem_domain_of_mem_compRestricted_domain (x : (v ∘r u).domain) : u ⟨x,
```

```

(mem_compRestricted_domain_iff.mp x.2).choose_spec

@[simp]
lemma compRestricted_apply (x : (v ∘r u).domain) :
 (v ∘r u) x = v (u ⟨x, x.2.2⟩, mem_domain_of_mem_compRestricted_domain x) := rfl

/-- The zero map is right-absorbing. -/
@[simp]
lemma compRestricted_zero : g ∘r (0 : E →1.[R] F) = 0 := by
 ext
 · simp [mem_compRestricted_domain_iff]
 · exact g.map_zero

lemma compRestricted_assoc {H : Type*} [AddCommGroup H] [Module R H]
 (f1 : G →1.[R] H) (f2 : F →1.[R] G) (f3 : E →1.[R] F) :
 (f1 ∘r f2) ∘r f3 = f1 ∘r f2 ∘r f3 := by
 ext
 · simp only [mem_compRestricted_domain_iff]
 · tauto
 · rfl

/-- `compRestricted` is the same as `comp` when the range of `u` is contained in `v`.
/
lemma compRestricted_eq_comp (h : ∀ x : u.domain, u x ∈ v.domain) :
 v ∘r u = v.comp u h := by
 ext x
 · change _ ↔ x ∈ u.domain
 · simp [mem_compRestricted_domain_iff, h]
 · rfl

/-- `compRestricted` is maximal amongst compositions of `v` with domain restrictions
/
lemma comp_le_compRestricted
 {S : Submodule R E} (h : ∀ x : (u.domRestrict S).domain, u ⟨x, x.2.2⟩ ∈ v.domain)
 v.comp (u.domRestrict S) h ≤ v ∘r u :=
 ⟨fun x hx ↦ mem_compRestricted_domain_iff.mpr ⟨hx.2, h ⟨x, hx⟩⟩, by aesop⟩

lemma compRestricted_mono_left {g g' : F →1.[R] G} (h : g ≤ g') (f : E →1.[R] F) :
 g ∘r f ≤ g' ∘r f := by
 constructor
 · intro x hx
 obtain ⟨hx', hfx⟩ := mem_compRestricted_domain_iff.mp hx
 exact mem_compRestricted_domain_iff.mpr ⟨hx', h.1 hfx⟩
 · intro x y hxy
 exact @h.2 ⟨f ⟨x, x.2.2⟩, mem_domain_of_mem_compRestricted_domain x⟩
 ⟨f ⟨y, y.2.2⟩, mem_domain_of_mem_compRestricted_domain y⟩ (by simp [hxy])

```

```

lemma compRestricted_mono_right (g : F →1.[R] G) {f f' : E →1.[R] F} (h : f
 g ∘r f ≤ g ∘r f' := by
 constructor
 · intro x hx
 obtain ⟨hx', hfx⟩ := mem_compRestricted_domain_iff.mp hx
 exact mem_compRestricted_domain_iff.mpr ⟨h.1 hx', (@h.2 ⟨x, hx'⟩ ⟨x, h.
 · intro x y hxy
 simp only [compRestricted_apply, @h.2 ⟨x, x.2.2⟩ ⟨y, y.2.2⟩ hxy]

@[simp]
lemma neg_compRestricted : (-g) ∘r f = -g ∘r f := rfl

@[simp]
lemma compRestricted_neg : g ∘r (-f) = -g ∘r f := by
 ext x hx hx'
 · simp [mem_compRestricted_domain_iff]
 · obtain ⟨h, h'⟩ := mem_compRestricted_domain_iff.mp (neg_domain (g ∘r f))
 exact g.toFun.map_neg ⟨f ⟨x, h⟩, h'⟩

lemma add_compRestricted : (g1 + g2) ∘r f = g1 ∘r f + g2 ∘r f := by
 ext x hx hx'
 · simp only [mem_compRestricted_domain_iff, add_domain, mem_inf]
 tauto
 · simp [add_apply]

lemma sub_compRestricted : (g1 - g2) ∘r f = g1 ∘r f - g2 ∘r f := by
 simp [sub_eq_add_neg, add_compRestricted]

lemma compRestricted_add_ge : g ∘r f1 + g ∘r f2 ≤ g ∘r (f1 + f2) := by
 constructor
 · intro x hx
 obtain ⟨h1, h1'⟩ := mem_compRestricted_domain_iff.mp hx.1
 obtain ⟨h2, h2'⟩ := mem_compRestricted_domain_iff.mp hx.2
 exact mem_compRestricted_domain_iff.mpr ⟨⟨h1, h2⟩, add_mem h1' h2'⟩
 · intro x y hxy
 obtain ⟨h1, h1'⟩ := mem_compRestricted_domain_iff.mp x.2.1
 obtain ⟨h2, h2'⟩ := mem_compRestricted_domain_iff.mp x.2.2
 simp [← hxy, add_apply, ← g.map_add ⟨f1 ⟨x, h1⟩, h1'⟩ ⟨f2 ⟨x, h2⟩, h2'⟩]

lemma compRestricted_sub_ge : g ∘r f1 - g ∘r f2 ≤ g ∘r (f1 - f2) := by
 simp only [sub_eq_add_neg, ← compRestricted_neg]
 exact compRestricted_add_ge g f1 (-f2)

lemma compRestricted_smul {S : Type*} [DivisionRing S]
 [Module S E] [Module S F] [Module S G] [SMulCommClass S S F] [SMulCommC

```

```

 {c : S} (hc : c ≠ 0) (g : F →ℓ.[S] G) (f : E →ℓ.[S] F) :
 g ∘r (c • f) = c • (g ∘r f) := by
 ext x hx hx'
 · simp [mem_compRestricted_domain_iff, g.domain.smul_mem_iff hc]
 · obtain ⟨h, h'⟩ := mem_compRestricted_domain_iff.mp (smul_domain c (g ∘r f)) hx'
 exact g.toFun.map_smul c ⟨f ⟨x, h⟩, h'⟩

@[simp]
lemma smul_compRestricted {M : Type*} [Monoid M] [DistribMulAction M G] [SMulCommClass M G]
 (c : M) (g : F →ℓ.[R] G) (f : E →ℓ.[R] F) :
 (c • g) ∘r f = c • (g ∘r f) := by
 ext
 · simp [compRestricted_domain]
 · simp

end Composition

/-!
E. Monoid

Partial linear maps `E →ℓ.[R] E` with `compRestricted` for multiplication and
the identity map (domain `⊤`) for `1` comprise a monoid.
-/

section Monoid

instance instMonoid : Monoid (E →ℓ.[R] E) where
 mul := compRestricted
 mul_assoc := compRestricted_assoc
 one := ⟨⊤, topEquiv.toLinearMap⟩
 one_mul f := by
 change ⟨⊤, topEquiv.toLinearMap⟩ ∘r f = f
 ext
 · simp [mem_compRestricted_domain_iff]
 · rfl
 mul_one f := by
 change f ∘r ⟨⊤, topEquiv.toLinearMap⟩ = f
 ext
 · simp [mem_compRestricted_domain_iff]
 · rfl

lemma mul_def (f1 f2 : E →ℓ.[R] E) : f1 * f2 = f1 ∘r f2 := rfl

@[simp]
lemma one_domain : (1 : E →ℓ.[R] E).domain = ⊤ := rfl

```

```

@[simp]
lemma one_toFun : (1 : E →1.[R] E).toFun = topEquiv.toLinearMap := rfl

@[simp]
lemma one_coe : (1 : E →1.[R] E).toFun' = ↑topEquiv.toLinearMap := rfl

end Monoid

/-!
F. Inverses
-/

section Inverses

variable {f : E →1.[R] F} (h_ker : f.toFun.ker = ⊥)
include h_ker

lemma inverse_ker : f.inverse.toFun.ker = ⊥ := by
 refine LinearMap.ker_eq_bot'.mpr fun ⟨y, hy⟩ hy' ↦ ?_
 obtain ⟨x, hx⟩ := inverse_domain (f := f) ▸ hy
 simp_all [inverse_apply_eq (x := x) (y := ⟨y, hy⟩) h_ker hx]

lemma inverse_inverse : f.inverse.inverse = f := by
 ext x hx hx'
 · rw [inverse_domain, inverse_range h_ker]
 · refine inverse_apply_eq (y := ⟨x, hx⟩) (x := ⟨f ⟨x, hx'⟩, by simp [inverse_domain] h_ker)
 · exact inverse_ker h_ker
 · exact inverse_apply_eq (y := ⟨f ⟨x, hx'⟩, by simp [inverse_domain] h_ker)

lemma inverse_compRestricted_eq : f.inverse ∘r f = domRestrict 1 f.domain :
 ext x hx hx'
 · simp [mem_compRestricted_domain_iff, inverse_domain, ← toFun_eq_coe]
 · exact inverse_apply_eq (x := ⟨x, hx.2⟩) h_ker rfl

lemma compRestricted_inverse_eq : f ∘r f.inverse = domRestrict 1 f.inverse :
 nth_rw 1 [← inverse_inverse h_ker]
 exact inverse_compRestricted_eq (inverse_ker h_ker)

end Inverses

end LinearPMap

```



### 1.81 Physlib/Mathematics/List.lean

```

public import Mathlib.Order.Lattice.Nat
 split <=> rfl
 rw [List.eraseIdx_of_length_le ((List.takeWhile_sublist _).length_le.trans (by s
 simpa using h {0, by simp} {1, by simp} (by simp) (by simpa using hPb)
 · simp [hPb]
 split <=> rfl
 exact takeWhile_eraseIdx P (b :: l) n fun i j hij hP => by
 simpa using h i.succ j.succ (by simpa using hij) (by simpa using hP)
 by_cases hPa : P a <=> simp [hPa]
 by_cases hPa : P a <=> simp [hPa, List.eraseIdx_of_length_le]
 simpa using h {0, by simp} {1, by simp} (by simp) (by simpa using hPb)
 · by_cases hPa : P a <=> simp [hPa, hPb]
 simpa using h {0, by simp} {1, by simp} (by simp) (by simpa using hPb)
 rw [dropWhile_eraseIdx P (b :: l) n (fun i j hij hP => by
 simpa using h i.succ j.succ (by simpa using hij) (by simpa using hP))]
 · rw [dropWhile_eraseIdx P (b :: l) n (fun i j hij hP => by
 simpa using h i.succ j.succ (by simpa using hij) (by simpa using hP))]
 simp only [orderedInsertPos, List.takeWhile, decide_not, Fin.zero_eta,
 by_cases h : le1 r0 r1 <=> simp [h]
 split <=> simp_all
 simp only [List.length_nil, Nat.reduceAdd, Fin.val_eq_zero, Fin.zero_eta, Fin.is
 Equiv.symm_apply_apply, OrderIso.coe_symm_toEquiv, Fin.symm_castOrderIso,
 Fin.castOrderIso_apply, Fin.cast_mk, Fin.zero_succAbove, Fin.succ_mk]
 simp only [List.length_cons]
 simp only [orderedInsertPos, decide_not, Nat.succ_eq_add_one,
 simp only [List.length_cons, List.length_nil, Nat.reduceAdd, Fin.val_eq_zero, Fi
 Fin.isValue, Equiv.symm_apply_apply, OrderIso.coe_symm_toEquiv, Fin.symm_castO
 Fin.castOrderIso_apply, Fin.eta, Fin.zero_succAbove]
 simp only [List.length_cons, Fin.eta]
 simp only [orderedInsertPos, decide_not, Nat.succ_eq_add_one,
 split <=> rfl
 have ht (i j k : I) (hij : le1 i j) (hjk : ¬ le1 k j) : ¬ le1 k i :=
 fun hik => hjk (IsTrans.trans (r := le1) k i j hik hij)

```

### 1.82 Physlib/Mathematics/List/InsertionSort.lean

```

simp only [List.length_cons, insertionSortDropMinPos, Nat.succ_eq_add_one]
have h11 : (List.insertionSort r (a :: l)).Pairwise r := List.pairwise_insertionSo
exact h
 have hrb : r b a := (Std.Total.total a b).resolve_left hr
 have hrb : r b a := (Std.Total.total a b).resolve_left hr
 · simp [h, hr]

```

```

have hl : ¬ r a c :=
 fun hn => h (IsTrans.trans _ _ _ ((Std.Total.total a b).resolve_left hr))
simp [List.orderedInsert, (Std.Total.total a b).resolve_left hr]
have hrba : r b a := (Std.Total.total a b).resolve_left hr
· by_cases hac : r a c <=> simp [h, hac, hrba]
· exact absurd (IsTrans.trans _ _ _ hrba hac) h
have hrba : r b a := (Std.Total.total a b).resolve_left hr
have hba : b2.1 ≤ a2.1 := hb ▸ ha1
have val_succAbove {m : ℕ} (p : Fin (m + 1)) (i : Fin m) :
 (p.succAbove i).1 = if i.1 < p.1 then i.1 else i.1 + 1 := by
 split <=> simp_all [Fin.lt_def]
simp only [val_succAbove, ha2, hb]
split_ifs <=> omega
simp only [List.insertionSort, List.length_nil, zero_add, Fin.mk_one, List.nil_append, List.foldr_cons, finCongr_refl, List.length_cons]
rfl
simp only [List.cons_append, List.length_cons,
· rw [orderedInsert_filter_of_pos r a p h _ (List.pairwise_insertionSort r a p h)]
· rw [List.filter_cons_of_pos (by simp using h)]
· rw [orderedInsert_filter_of_neg r a p h]
· rw [List.filter_cons_of_neg (by simp using h)]
conv_lhs => rw [← List.takeWhile_append_dropWhile (p := fun c => decide (c < a2.1)) (l := (l1 ++ l ++ l2).insertionSort r), hlt]

```

### 1.83 Physlib/Mathematics/SchurTriangulation.lean

```

let g : Module.End [] W := Submodule.orthogonalProjectionOnto W ∘1 f.dom
Submodule.inner_orthogonalProjectionOnto_eq_of_mem_left]

```

### 1.84 Physlib/Mathematics/SpecialFunctions/PhysHermite.lean

```

simp [coeff_physHermite_self_succ, Nat.descFactorial_self]
rw [coeff_physHermite_of_lt (by omega), Polynomial.coeff_C_of_ne_zero (
ContinuousLinearMap.coe_comp, Function.comp_apply, ContinuousLinearMap.
one_apply_eq_self, smul_eq_mul, FunLike.coe_smul, Pi.smul_apply, map_of
simp only [Polynomial.deriv_aeval, derivative_physHermite, nsmul_eq_mul,
map_natCast, map_sub, aeval_X, map_ofNat]
apply MeasureTheory.integrable_finsetSum
@[fun_prop]
lemma gaussian_integrable_polynomial {b : ℝ} (hb : 0 < b) (P : Polynomial ℝ) :
MeasureTheory.Integrable fun x : ℝ => (P.aeval x) * Real.exp (-
b * x ^ 2) := by
simp using gaussian_integrable_polynomial_cons (c := 1) hb P

```

```

have prod_eq_smul_aeval_mul_gaussian :
 (deriv^[m] (physHermite p) * deriv^[n] fun x => Real.exp (-
x ^ 2)) =
 (-1 : ℝ) ^ n • fun x => (derivative^[m] (physHermite p) * physHermite n).aeval
 Real.exp (-1 * x ^ 2) := by
 simp only [Pi.mul_apply, deriv_gaussian_eq_physHermite_mul_gaussian, map_mul, Pi
 smul_eq_mul, neg_one_mul]
 ring
rw [prod_eq_smul_aeval_mul_gaussian]
exact MeasureTheory.Integrable.smul _ (gaussian_integrable_polynomial Real.zero_lt
simp only [h1, MeasureTheory.integral_const_mul]
 simp using integral_gaussian 1
 obtain ⟨x, rfl⟩ := natDegree_eq_zero.mp h
 refine Finsupp.mem_span_range_iff_exists_finsupp.mpr ⟨Finsupp.single 0 x, ?_⟩
 exact Submodule.zero_mem _
 exact Submodule.zero_mem _
 simp [coeff_physHermite_self_succ, map_ofNat]
 exact Submodule.smul_mem _ (Submodule.subset_span ⟨n + 1, rfl⟩)
 · rw [coeff_eq_zero_of_natDegree_lt (by omega), coeff_physHermite_of_lt (by omega)
 simp
 have hy : (fun (y : ℝ) => y ^ (2 * l)) = fun y => ((X ^ (2 * l) : Polynomial ℤ))
 simp
 exact hy ▸ polynomial_mem_physHermite_span _

```

## 1.85 Physlib/Mathematics/VariationalCalculus/Basic.lean

```

intro h; apply hx0; exact congrArg (fromL2 ℝ) h
have hcont₂ : ContinuousAt f₂ x₀ := ((toL2 ℝ).continuous.comp hf).continuousAt
-- Cauchy-Schwarz in `WithLp 2 V` bounds the cross term from below
have hcs : -(||f₂ x - x₂|| * ||x₂||) ≤ □f₂ x - x₂, x₂□_ℝ :=
 neg_le_of_abs_le (abs_real_inner_le_norm _ _)
have hxp : 0 < ||x₂|| := norm_pos_iff.mpr hx2
have inner_eq_self_add_diff : □f₂ x, x₂□_ℝ = □x₂, x₂□_ℝ + □f₂ x - x₂, x₂□_ℝ :=
 rw [← inner_add_left]; simp
nlinarith [inner_eq_self_add_diff, hcs, hself, hclose, hxp,
 mul_lt_mul_of_pos_right hclose hxp]
have inner_pos_V : ∀ x ∈ Metric.ball x₀ δ₂, 0 < □f x, f x₀□_ℝ := fun x hx => inn
 let φ₁ : ContDiffBump x₀ :=
 {δ₂ / 4, δ₂ / 2, by positivity, by linarith}
 refine {φ₁.toFun, {φ₁.contDiff, φ₁.hasCompactSupport},
 φ₁.pos_of_mem_ball (Metric.mem_ball_self φ₁.rOut_pos), fun x _ => φ₁.non
 by rw [ContDiffBump.support_eq], fun x hx => φ₁.pos_of_mem_ball ?_}
 -- the closed ball of radius `δ₂/4` lies in the open ball of radius `rOut =

```

```

 rw [Metric.mem_closedBall] at hx
 exact Metric.mem_ball.mpr (lt_of_le_of_lt hx (by show $\delta_2 / 4 < \delta_2$)
-- Show that g is a test function: `φ` is a test function and `f x₀` is s
 have hg_test : IsTestFunction g := IsTestFunction.smul_right hφ_testfun
-- linearity of inner product in the second argument turns the integrand
 simp only [g, inner_smul_right'] at key_integral
 have support_subset : Function.support φ ⊆ Metric.ball x₀ δ₂ :=
 hφ_support_subset.trans (Metric.ball_subset_ball (by linarith))
 exact mul_nonneg (hφ_non_neg x hx) (inner_pos_V x (support_subset h
 simp [Function.notMem_support.mp hx]
-- every nonempty open set has positive measure
 have hμ_ball : 0 < μ (Metric.ball x₀ (δ₂/4)) :=
 Metric.isOpen_ball.measure_pos μ (Metric.nonempty_ball.mpr (by linari
 have hμ : 0 < μ (Metric.closedBall x₀ (δ₂/4)) :=
 lt_of_lt_of_le hμ_ball (measure_mono Metric.ball_subset_closedBall)
 · linarith
 · linarith

-- [4.2.4] putting everything together: positivity contradicts `key_integ
 have integral_pos : 0 < ∫ x, φ x * □ f x, f x₀ □ ℝ ∂μ :=
 (integral_pos_iff_support_of_nonneg h_nonneg integrable_prod).mpr
 (lt_of_lt_of_le hμ (measure_mono closedBall_subset_support))

```

## 1.86 Physlib/Mathematics/VariationalCalculus/HasVarAdjDeriv.lean

```

· rfl
· exact ContDiff.smul (by fun_prop) (by fun_prop [deriv])
· fun_prop [deriv]
· rfl
· exact ContDiff.smul (by fun_prop) hφ.contDiff
· exact hφ.contDiff
 fun_prop [deriv]
 · exact hF.differentiable_linear hφ x 0
 · exact hG.differentiable_linear hφ x 0
 · exact (smooth_R hF hφ x).differentiable (by simp) 0
 · exact (smooth_R hG hφ x).differentiable (by simp) 0
 · exact hF.differentiable_linear hφ x 0
 · exact hG.differentiable_linear hφ x 0
 · exact (smooth_R hF hφ x).differentiable (by simp) 0
 · exact (smooth_R hG hφ x).differentiable (by simp) 0
· exact (hF.smooth_adjoint hφ.contDiff x).differentiable (by simp) 0
· exact (hG.smooth_adjoint hφ.contDiff x).differentiable (by simp) 0
rw [← fderiv_apply_one_eq_deriv, fderiv_fun_comp]
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply, fderiv_eq

```

1.87.

PHYSLIB/MATHEMATICS/VARIATIONALCALCULUS/HASVARADJOINT.lean

```

rw [fderiv_fun_comp _ (by fun_prop)]
rw [← fderiv_apply_one_eq_deriv, fderiv_fun_comp, fderiv_fst]
simp only [ContinuousLinearMap.coe_comp, ContinuousLinearMap.coe_fst', Function.comp_apply]
· exact (hF.smooth_adjoint hφ.contDiff x).differentiable (by simp) 0
rw [← fderiv_apply_one_eq_deriv, fderiv_fun_comp]
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply, fderiv_eq_smul_deriv]
rw [fderiv_fun_comp _ (by fun_prop)]
rw [← fderiv_apply_one_eq_deriv, fderiv_fun_comp, fderiv_snd]
simp only [ContinuousLinearMap.coe_comp, ContinuousLinearMap.coe_snd', Function.comp_apply]
· exact (hF.smooth_adjoint hφ.contDiff x).differentiable (by simp) 0
conv => lhs; rw[fderiv_fun_comp (λ:=ℝ) (g:=(fun u : U => f _ u)) _]
conv => rhs; rw[fderiv_fun_comp (λ:=ℝ) (g:=(fun u : U => f _ u)) _]
 rw[fderiv_fun_comp (λ:=ℝ) (g:=_) (f:=fun s : ℝ => u x + s • φ x) _]
· exact (hF.smooth_R hφ x).differentiable (by simp) 0
· exact (hG.smooth_R hφ x).differentiable (by simp) 0
· exact (hF.smooth_R hφ x).differentiable (by simp) 0
· exact (hG.smooth_R hφ x).differentiable (by simp) 0
have h1 : HasVarAdjDerivAt (fun φ x => c) (fun x => 0) u :=
 HasVarAdjDerivAt.const u (fun _ => c) hF.smooth_at (by fun_prop)
have h1 : HasVarAdjDerivAt (fun φ x => f x) (fun ψ x => 0) u :=
 HasVarAdjDerivAt.const u f hF.smooth_at hf
rw [fderiv_add]
simp only [add_apply]
rw [fderiv_const_smul]
simp only [FunLike.coe_smul, Pi.smul_apply]
rw [Space.grad_add, Space.grad_eq_gradient, Space.grad_eq_gradient]
rw [Space.grad_smul, Space.grad_eq_gradient]
simp only [ContinuousLinearMap.fderiv, ContinuousLinearMap.coe_comp, Function.comp_apply]
· fun_prop
 rw [fderiv_fun_comp]
 · simp only [ContinuousLinearMap.fderiv]
 · fun_prop
 · apply function_differentiableAt_snd
 exact hφ.differentiable (by simp)
 simp only [ContinuousLinearMap.coe_comp, Function.comp_apply]

```

## 1.87 Physlib/Mathematics/VariationalCalculus/HasVarAdjoint.lean

```

-- prepare test function that is one on `K u L`
obtain ⟨y₀, -, hr, hsup⟩ := IsCompact.exists_sSup_image_eq_and_ge (s := {0} u (K u L))
 (IsCompact.union (by simp) (IsCompact.union cK cL)) hnonempty
(f := fun x => ||x||) (by fun_prop)
have hr0 : 0 ≤ r := le_of_le_of_eq (norm_nonneg y₀) hr.symm
have hφ : IsTestFunction (fun x : Y => φ x) :=

```

```

(ContDiffBump.contDiff ϕ , ContDiffBump.hasCompactSupport ϕ)
simp only [ϕ , Metric.mem_closedBall, dist_zero_right]
have hxr : $\|x\| \leq r := \text{le_of_le_of_eq} (\text{hsup } x \text{ (by simp [hx])}) \text{ hr.symm}$
linarith
have h ψ eq : $\forall x \in K \cup L, f \ x = \psi \ x := \text{fun } x \text{ hx} \Rightarrow$ by
 rw [show $\psi \ x = \phi \ x \cdot f \ x$ from rfl, ContDiffBump.one_of_mem_closedBall ϕ]
simp only [hK f ψ (fun z hz $\Rightarrow$ h ψ eq z (by simp [hz])) x rfl,
 hL f ψ (fun z hz $\Rightarrow$ h ψ eq z (by simp [hz])) x rfl,
 unique_on_test_functions hF hG ψ h ψ]
HasVarAdjoint (fun $\phi \ x \Rightarrow \psi \ x \cdot F \ \phi \ x$) (fun $\phi \ x \Rightarrow F' \ (\text{fun } x' \Rightarrow \psi \ x' \cdot$
smul_left hF ψ)
test_fun_preserving ϕ h $\phi :=$ IsTestFunction.family_linearMap_comp h ϕ hf
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply]
test_fun_preserving _ h $\phi :=$
 (by fun_prop, HasCompactSupport.deriv h ϕ .supp)
test_fun_preserving' _ h $\phi :=$
 (by fun_prop, (HasCompactSupport.deriv h ϕ .supp).neg)
· intro $\phi \ \phi' \ h\phi \ x \ hx$
 dsimp[divergence]; congr 4
 have heq : $\phi =^{\text{f}}[\text{nhds } x] \ \phi' :=$ by
 · exact Metric.isOpen_thickening.mem_nhds
 (Metric.self_subset_thickening one_pos K hx)
 · intro y hy
 exact h ϕ y (Metric.thickening_subset_cthickening 1 K hy)
 rw [Filter.EventuallyEq.fderiv_eq heq]
 map_smul' := by simp }
have hf ψ : $\forall i, \text{IsTestFunction fun } y \Rightarrow f' \ i \ (\psi \ y) := \text{fun } i \Rightarrow$
 IsTestFunction.comp_left h ψ (by simp) (by fun_prop)
have hinner : IsTestFunction fun y $\Rightarrow \square dy, \phi \ y \square_{\mathbb{R}} :=$
 IsTestFunction.inner_left (by fun_prop) h ϕ
 rw [(DifferentiableAt.hasAdjFDerivAt
 (h ϕ .differentiable y)).hasAdjoint_fderiv.adjoint_inner_left]
 rw [MeasureTheory.integral_finsetSum]
 simp only [fderiv_fun_const, Pi.zero_apply, _root_.zero_apply,
 · exact hf ψ i
 · exact ((hf ψ i).fderiv_apply (bX i)).smooth
 · exact hinner
 · exact (hf ψ i).smooth
 · exact hinner.fderiv_apply (bX i)
 · exact (hf ψ i).smooth
 · exact hinner
]
 rw [$\leftarrow$ MeasureTheory.integral_finsetSum]
 · exact (((hf ψ i).fderiv_apply (bX i)).neg).smooth
 · exact hinner
 simp only [ContinuousLinearMap.fderiv, ContinuousLinearMap.coe_
convert! h2 using 1

```

1.88.

PHYSLIB/MATHEMATICS/VARIATIONALCALCULUS/ISLOCALIZEDFUNCTIONTRANSFORM. LEAN

rw [Space.grad\_eq\_gradient]

## 1.88 Physlib/Mathematics/VariationalCalculus/IsLocalizedfunctionTransf

```
obtain ⟨L, cL, h⟩ := hF K cK
exact ⟨L, cL, fun ϕ ϕ' hϕ x hx => by dsimp only; rw [h ϕ ϕ' hϕ x hx]⟩
obtain ⟨L, cL, h⟩ := hF K cK
exact ⟨L, cL, fun ϕ ϕ' hϕ x hx => by dsimp only; rw [h ϕ ϕ' hϕ x hx]⟩
obtain ⟨L, cL, h⟩ := hF K cK
exact ⟨L, cL, fun ϕ ϕ' hϕ x hx => by dsimp only; rw [h ϕ ϕ' hϕ x hx]⟩
 · apply Metric.isOpen_thickening.mem_nhds
 exact Metric.self_subset_thickening one_pos K hx
 · intro y hy
 dsimp only
 rw [hϕ y (Metric.thickening_subset_cthickening 1 K hy)]
 · apply Metric.isOpen_thickening.mem_nhds
 exact Metric.self_subset_thickening one_pos K hx
 · intro y hy
 dsimp only
 rw [hϕ y (Metric.thickening_subset_cthickening 1 K hy)]
have h : ϕ =f[nhds x] ϕ' := by
 apply Filter.eventuallyEq_of_mem (s := Metric.thickening 1 K)
 · apply Metric.isOpen_thickening.mem_nhds
 exact Metric.self_subset_thickening one_pos K hx
 · intro y hy
 exact hϕ y (Metric.thickening_subset_cthickening 1 K hy)
exact Filter.EventuallyEq.fderiv_eq h
have h : ϕ =f[nhds x] ϕ' := by
 apply Filter.eventuallyEq_of_mem (s := Metric.thickening 1 K)
 · apply Metric.isOpen_thickening.mem_nhds
 exact Metric.self_subset_thickening one_pos K hx
 · intro y hy
 exact hϕ y (Metric.thickening_subset_cthickening 1 K hy)
exact Filter.EventuallyEq.fderiv_eq h
· intro ϕ ϕ' hϕ x hx
 dsimp
 have h : ϕ =f[nhds x] ϕ' := by
 · apply Metric.isOpen_thickening.mem_nhds
 exact Metric.self_subset_thickening one_pos K hx
 · intro y hy
 exact hϕ y (Metric.thickening_subset_cthickening 1 K hy)
 exact h.deriv_eq
· intro ϕ ϕ' hϕ x hx
 dsimp; congr 1
```

```

have h : $\phi =^f[\text{nhds } x] \phi' := \text{by}
 \cdot \text{apply Metric.isOpen_thickening.mem_nhds}
 \cdot \text{exact Metric.self_subset_thickening one_pos K hx}
 \cdot \text{intro y hy}
 \cdot \text{exact h}\phi y (\text{Metric.thickening_subset_cthickening 1 K hy})
rw [Filter.EventuallyEq.fderiv_eq h]
obtain (L, cL, h) := hF K cK
exact (L, cL, fun $\phi \phi'$ h ϕ x hx => by dsimp only; rw [h $\phi \phi'$ h ϕ x hx])
obtain (L, cL, h) := hF K cK
exact (L, cL, fun $\phi \phi'$ h ϕ x hx => by dsimp only; rw [h $\phi \phi'$ h ϕ x hx])
have h : $\phi =^f[\text{nhds } x] \phi' := \text{by}
 \cdot \text{apply Filter.eventuallyEq_of_mem (s := Metric.thickening 1 K)}$
 \cdot \text{apply Metric.isOpen_thickening.mem_nhds}
 \cdot \text{exact Metric.self_subset_thickening one_pos K hx}
 \cdot \text{intro y hy}
 \cdot \text{exact h}\phi y (\text{Metric.thickening_subset_cthickening 1 K hy})
exact Filter.EventuallyEq.fderiv_eq h$
```

### 1.89 Physlib/Mathematics/VariationalCalculus/IsTestFunction.lean

```

simp only [ContinuousLinearMap.coe_comp, Function.comp_apply]
let reprMap : $X \rightarrow_{\mathbb{R}} \mathbb{R}$:= {
 let f' : $X \rightarrow L[\mathbb{R}] \mathbb{R}$:= reprMap.toContinuousLinearMap
 have h_trace_contDiff : ContDiff $\mathbb{R} \infty f'$:= f'.contDiff
 change IsTestFunction (fun x => f' ((fderiv $\mathbb{R}$ f x) (bX i)))
 (f:=fun x : $X \Rightarrow$ (fderiv $\mathbb{R}$ f x) (bX i)) (g:=f'))
 \cdot simp [f']
 \cdot exact h_trace_contDiff

```

### 1.90 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancel

```

lemma sum_one [AddCommMonoid M] (f : Fin (SMvSpecies 1).numberCharges $\rightarrow$ M)
 $\sum i, f i = f \{0\}$, by simp) := by
 change $\sum (i : \text{Fin } 1), f i = _$
 simp only [Finset.univ_unique, Fin.default_eq_zero, Fin.isValue, Finset.sum rfl]

 simp only [ACCSysCharges.chargesAddCommMonoid_add, toSpecies_apply,
 simp only [HSMul.hSMul, SMul.smul, toSpecies_apply, Fin.isValue,
 simp only [accGrav, toSpecies_apply, Fin.isValue, LinearMap.coe_mk,
 simp only [ACCSysCharges.chargesAddCommMonoid_add, toSpecies_apply,
 simp only [HSMul.hSMul, SMul.smul, toSpecies_apply, Fin.isValue,
 simp only [accSU2, toSpecies_apply, Fin.isValue, LinearMap.coe_mk,

```



1.91.

PHYSLIB/PARTICLES/BEYONDTHESTANDARDMODEL/RHN/ANOMALYCANCELLATION/FAMILYMAPS.LEAN

```
simp only [ACCSysCharges.chargesAddCommMonoid_add, toSpecies_apply,
 simp only [HSMul.hSMul, SMul.smul, toSpecies_apply, Fin.isValue,
simp only [accSU3, toSpecies_apply, Fin.isValue, LinearMap.coe_mk,
 simp only [ACCSysCharges.chargesAddCommMonoid_add, toSpecies_apply,
 simp only [HSMul.hSMul, SMul.smul, toSpecies_apply, Fin.isValue,
simp only [accYY, toSpecies_apply, Fin.isValue, LinearMap.coe_mk,
 simp only [toSpecies_apply, Fin.isValue, neg_mul, one_mul]
simp only [toSpecies_apply, Fin.isValue, neg_mul, one_mul, add_left_inj]
 simp only [toSpecies_apply, Fin.isValue]
 simp only [toSpecies_apply, Fin.isValue]
```

## 1.91 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
simp only [ACCSysCharges.chargesAddCommMonoid_add]
simp only [HSMul.hSMul, ACCSysCharges.chargesModule_smul,
simp only [Function.comp_apply, toSpecies_apply, toSpeciesEquiv_apply,
simp only [toSpecies_apply, toSpeciesEquiv_apply, Fin.zero_eta,
simp only [toSpecies_apply, toSpeciesEquiv_apply, Fin.zero_eta,
 simp only [toSpecies_apply, toSpeciesEquiv_apply, Fin.zero_eta, Fin.isValue, Nat
simp only [Fin.isValue, toSpecies_apply, toSpeciesEquiv, Nat.reduceMul, Equiv.symm
 Equiv.arrowCongr_symm, Equiv.refl_symm, Equiv.symm_symm, sum_one]
simp only [Fin.isValue, toSpecies_apply, sum_one]
simp only [Fin.isValue, toSpecies_apply, sum_one]
simp only [Fin.isValue, toSpecies_apply, sum_one]
simp only [Fin.isValue, toSpecies_apply, add_left_inj, sub_left_inj, sub_right_inj]
simp only [Fin.isValue, toSpecies_apply, sum_one]
simp only [Fin.isValue, toSpecies_apply, add_left_inj]
simp only [Fin.isValue, toSpecies_apply]
simp only [Fin.isValue, toSpecies_apply, sum_one]
```

## 1.92 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
simp only [SMNoGrav_linearACCs] at hS
exact hS {0, by simp}
simp only [SMNoGrav_linearACCs] at hS
exact hS {1, by simp}
| {0, _} => exact hSU2
| {1, _} => exact hSU3
| {0, _} => exact accSU2_invariant
| {1, _} => exact accSU3_invariant
```

**1.93 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancel**

```

simp only [SM_linearACCs] at hS
exact hS {0, by simp}
simp only [SM_linearACCs] at hS
exact hS {1, by simp}
simp only [SM_linearACCs] at hS
exact hS {2, by simp}
| {0, _} => exact hGrav
| {1, _} => exact hSU2
| {2, _} => exact hSU3}
| {0, _} => exact accGrav_invariant
| {1, _} => exact accSU2_invariant
| {2, _} => exact accSU3_invariant

```

**1.94 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancel**

```

Fin.mk_eq_zero, Nat.div_eq_zero_iff, OfNat.ofNat_ne_zero, false_or, imp
not_lt, le_add_iff_nonneg_left, zero_le, IsEmpty.forall_iff, Fin.mk_eq_
Fin.sum_univ_three, Nat.zero_div, Fin.zero_eta, one_mul, Nat.reduceDiv,
mul_neg, Fin.reduceFinMk]
Fin.sum_univ_three, Nat.zero_div, Fin.zero_eta, zero_mul, zero_add, Fin
one_mul, Nat.reduceDiv, Nat.reduceAdd, neg_mul, mul_neg]
Fin.sum_univ_three, Nat.zero_div, Fin.zero_eta, zero_mul, zero_add, Fin
Nat.reduceDiv, one_mul, Nat.reduceAdd, neg_mul, mul_neg]
Fin.sum_univ_three, Nat.zero_div, Fin.zero_eta, zero_mul, zero_add, Fin
Nat.reduceDiv, one_mul, Nat.reduceAdd, neg_mul, mul_neg]
Fin.sum_univ_three, Nat.zero_div, Fin.zero_eta, zero_mul, zero_add, Fin
Nat.reduceDiv, one_mul, Nat.reduceAdd, neg_mul]
Fin.sum_univ_three, Nat.zero_div, Fin.zero_eta, zero_mul, zero_add, Fin
Nat.reduceDiv, one_mul, Nat.reduceAdd, neg_mul]
Fin.sum_univ_three, Nat.zero_div, Fin.zero_eta, zero_mul, zero_add, Fin
Nat.reduceDiv, Nat.reduceAdd, one_mul, neg_mul, mul_neg]
Fin.coe_ofNat_eq_mod, Nat.zero_mod, Nat.zero_div, Fin.zero_eta, Fin.mod
add_zero, Nat.reduceMod, Nat.ofNat_pos, Nat.div_self, Fin.mk_one, Nat.m
Nat.reduceDiv, Fin.reduceFinMk] at h0 h1 h2 h3 h4 h5 h6
| {0, _} => exact h0
| {1, _} => exact h1
| {2, _} => exact h2
| {3, _} => exact h3
| {4, _} => exact h4
| {5, _} => exact h5
| {6, _} => exact h6

```

1.95.

PHYSLIB/PARTICLES/BEYONDTHESTANDARDMODEL/RHN/ANOMALY CANCELLATION/PLUSU1/BMINUSL.LEAN

### 1.95 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
| {0, _} => with_unfolding_all rfl
| {1, _} => with_unfolding_all rfl
| {2, _} => with_unfolding_all rfl
| {3, _} => with_unfolding_all rfl
| {0, _} => with_unfolding_all rfl
simp only [Fin.isValue, BL1_val, toSpecies_apply, one_mul, mul_neg,
simp only [Fin.isValue, BL1_val, mul_one, toSpecies_apply, mul_neg,
```

### 1.96 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
simp only [PlusU1_linearACCs] at hS
exact hS {0, by simp}
simp only [PlusU1_linearACCs] at hS
exact hS {1, by simp}
simp only [PlusU1_linearACCs] at hS
exact hS {2, by simp}
simp only [PlusU1_linearACCs] at hS
exact hS {3, by simp}
simp only [PlusU1_quadraticACCs] at hS
exact hS {0, by simp}
| {0, _} => exact hGrav
| {1, _} => exact hSU2
| {2, _} => exact hSU3
| {3, _} => exact hYY}
| {0, _} => exact hQ}
| {0, _} => exact accGrav_invariant
| {1, _} => exact accSU2_invariant
| {2, _} => exact accSU3_invariant
| {3, _} => exact accYY_invariant
| {0, _} => exact accQuad_invariant
```

### 1.97 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/

```
| {0, _} => with_unfolding_all rfl
| {1, _} => with_unfolding_all rfl
| {2, _} => with_unfolding_all rfl
| {3, _} => with_unfolding_all rfl
| {0, _} => with_unfolding_all rfl
simp only [Fin.isValue, Y1_val, toSpecies_apply, one_mul, mul_neg,
simp only [Fin.isValue, Y1_val, mul_one, toSpecies_apply, mul_neg,
simp only [Fin.isValue, Y1_val, mul_one, toSpecies_apply, mul_neg,
```

**1.98 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancel**

```
OfNat.ofNat_ne_zero, false_or, α_2 , HomogeneousQuadratic, accQuad] at h1
```

**1.99 Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancel**

```
simp_all [α_1 , α_2]
```

**1.100 Physlib/Particles/BeyondTheStandardModel/TwoHDM/API-map.yaml**

```
version: v0.1
```

```
Title: Two Higgs Doublet Model
```

```
Overview: |
```

```
The key data structure is `TwoHiggsDoublet`, the configuration space of
Higgs doublet model (an extension of the Standard Model with a second H
doublet), holding two Higgs vectors ` $\Phi_1 \Phi_2$  : HiggsVec`. The API provide
gauge-group action on this space, its Gram-matrix and Gram-
vector invariants that
characterise the gauge orbits, and the 2HDM potential together with its
theory.
```

```
ParentAPIs:
```

- Higgs Field (Physlib/Particles/StandardModel/HiggsBoson)
- Pauli matrices (Physlib/Relativity/PauliMatrices)

```
References:
```

- <https://arxiv.org/abs/hep-ph/0605184>
- <https://arxiv.org/abs/1605.03237>

```
Requirements:
```

- description: >
 

```
The key data structure `TwoHiggsDoublet`, the configuration space of
 holding two Higgs vectors, is defined.
done: true
location: Physlib/Particles/BeyondTheStandardModel/TwoHDM/Basic.lean (T
```
- description: >

1.100.

PHYSLIB/PARTICLES/BEYONDTHESTANDARDMODEL/TWOHDM/API-

MAP.YAML

The API contains the action of the global gauge group on the two Higgs doublet<sup>153</sup>

done: true  
location: >  
Physlib/Particles/BeyondTheStandardModel/TwoHDM/Basic.lean  
(SMul GaugeGroupI TwoHiggsDoublet, MulAction GaugeGroupI TwoHiggsDoublet)

- description: >  
The API contains an instance of a vector space ( $\mathbb{R}$ -module) on the key data structure `TwoHiggsDoublet`.  
done: false  
location: N/A

- description: >  
The API contains an action of the group  $\mathbb{Z}_2$  swapping the two Higgs vectors.  
done: false  
location: N/A

- description: >  
The API contains the Gram matrix of a two Higgs doublet, together with its self-adjointness and the non-negativity of its determinant and trace.  
done: true  
location: >  
Physlib/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix.lean  
(gramMatrix, gramMatrix\_selfAdjoint, gramMatrix\_det\_nonneg, gramMatrix\_tr\_nonneg)

- description: >  
The API proves that two configurations lie in the same gauge orbit iff they have the same Gram matrix.  
done: true  
location: >  
Physlib/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix.lean  
(mem\_orbit\_gaugeGroupI\_iff\_gramMatrix)

- description: >  
The API proves surjectivity of the Gram matrix: every self-adjoint matrix with non-negative determinant and trace arises as the Gram matrix of some configuration.  
done: true  
location: >  
Physlib/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix.lean  
(gramMatrix\_surjective\_det\_tr)

- description: >  
The API contains the Gram vector (Pauli-basis components of the Gram matrix), with its gauge-orbit characterisation and surjectivity.  
done: true

```

location: >
 Physlib/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix.lean
 (gramVector, mem_orbit_gaugeGroupI_iff_gramVector, gramVector_surject

- description: >
 The API contains the type of 2HDM potential parameters and its gram-
vector
 reparameterization (ξ , η).
done: true
location: >
 Physlib/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean
 (PotentialParameters, PotentialParameters. ξ , PotentialParameters. η)

- description: >
 The API contains the definition of the potential of the two Higgs dou
built from its mass and quartic terms.
done: true
location: >
 Physlib/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean
 (potential, massTerm, quarticTerm)

- description: >
 The API contains a theorem proving that the potential (and its mass a
terms) is invariant under the global gauge group.
done: true
location: >
 Physlib/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean
 (gaugeGroupI_smul_potential, gaugeGroupI_smul_massTerm, gaugeGroupI_s

- description: >
 The API expresses the potential and its terms in terms of the Gram ve
done: true
location: >
 Physlib/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean
 (potential_eq_gramVector, massTerm_eq_gramVector, quarticTerm_eq_gram

- description: >
 The API defines the stability (bounded-below) condition of the potent
gives gram-vector characterisations of it.
done: true
location: >
 Physlib/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean
 (PotentialIsStable, potentialIsStable_iff_massTermReduced_sq_le_quart

- description: >
 The API proves that strong stability (positivity of the reduced quart

```

1.101.

PHYSLIB/PARTICLES/BEYONDTHESTANDARDMODEL/TWOHDM/GRAMMATRIX.LEAN

```

 implies stability of the potential.
done: true
location: >
 Physlib/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean
 (potentialIsStable_of_strong)

- description: >
 The API gives an explicit counterexample invalidating a step in
 arXiv:hep-ph/0605184: an unstable potential whose reduced quartic term is
 non-negative and whose reduced mass term is non-negative wherever the reduced
 quartic term vanishes.
done: true
location: >
 Physlib/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean
 (stabilityCounterExample, stabilityCounterExample_not_potentialIsStable,
 forall_reduced_exists_not_potentialIsStable)

```

### 1.101 Physlib/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix.lean

```

have hinner : $\langle H1.\Phi1, H1.\Phi1 \rangle_{\mathbb{C}} = \langle H2.\Phi1, H2.\Phi1 \rangle_{\mathbb{C}} := \text{congrArg } (\cdot \ 0 \ 0) \ h$
rw [norm_eq_sqrt_re_inner ($\langle \cdot \rangle := \mathbb{C}$) H1. $\Phi1$, norm_eq_sqrt_re_inner ($\langle \cdot \rangle := \mathbb{C}$) H2. $\Phi1$, hinner]
have hinner : $\langle H1.\Phi2, H1.\Phi2 \rangle_{\mathbb{C}} = \langle H2.\Phi2, H2.\Phi2 \rangle_{\mathbb{C}} := \text{congrArg } (\cdot \ 1 \ 1) \ h$
rw [norm_eq_sqrt_re_inner ($\langle \cdot \rangle := \mathbb{C}$) H1. $\Phi2$, norm_eq_sqrt_re_inner ($\langle \cdot \rangle := \mathbb{C}$) H2. $\Phi2$, hinner]
rw [← Complex.conj_mul', inner_conj_symm]
Complex.add_re, ← Complex.ofReal_pow, Complex.ofReal_re]
positivity

have hb : $b = (a * c) ^ 2 := \text{by}$
 rw [mul_pow, hx]
rw [hb, Real.sqrt_sq (mul_nonneg ha hc.le), mul_div_assoc, div_self hc.ne', mul_
 exact eq_snd_norm_of_eq_gramMatrix h.symm
 apply eq_fst_norm_of_eq_gramMatrix h.symm
 exact eq_fst_norm_of_eq_gramMatrix h.symm
 · exact eq_fst_norm_of_eq_gramMatrix h.symm]
rw [isSelfAdjoint_iff] at hKs
have hcomp : $\forall i \ j, (\text{starRingEnd } \mathbb{C}) (K \ j \ i) = K \ i \ j := \text{fun } i \ j \Rightarrow \text{congrFun } (\text{congrF}$
have hK_explicit2 : $K = !![(K \ 0 \ 0).re : \mathbb{C}], K \ 0 \ 1; \text{conj } (K \ 0 \ 1), ((K \ 1 \ 1).re : \mathbb{C})$
 · exact (Complex.conj_eq_iff_re.mp (hcomp 0 0)).symm
 · exact (hcomp 1 0).symm
 · exact (Complex.conj_eq_iff_re.mp (hcomp 1 1)).symm
clear hKs hcomp
· have hD : $(0 : \mathbb{R}) \leq a * b - \|c\| ^ 2 := \text{by linarith}$
 rw [Real.sqrt_sqrt (by positivity), div_pow, div_pow, sq_abs, sq_abs,
 Real.sqrt_sqrt ha_nonneg, Real.sqrt_sqrt hD]

```

```

congr 1 <=> simp [PauliMatrix.pauliBasis, PauliMatrix.pauliSelfAdjoint]
simp [PauliMatrix.pauliMatrix, Fin.sum_univ_three, Complex.real_smul, Mat
 Matrix.cons_val_fin_one, Matrix.cons_val_one, Complex.add_re, Complex.m
 Complex.re_ofNat, Complex.normSq_ofNat, div_self_mul_self', Complex.ofR
 Complex.im_ofNat, neg_zero, zero_div, Complex.add_im, Complex.ofReal_im
 sub_zero, Complex.sub_re, Complex.sub_im, sub_self]
ring
· simp [gramMatrix, ← Complex.ofReal_pow]
· simp [gramMatrix, ← Complex.ofReal_pow]
simp [Complex.mul_re]
have h := gramMatrix_det_nonneg H
rw [gramMatrix_det_eq_gramVector] at h
linarith
have hK_selfAdjoint : IsSelfAdjoint K := by
 rw [isSelfAdjoint_iff]
 simp only [gramVector_eq, h]

```

### 1.102 Physlib/Particles/BeyondTheStandardModel/TwoHDM/Potential

```

rw [show (H.Φ2, H.Φ1) = conj (H.Φ1, H.Φ2) from Eq.symm (conj
 ← Complex.normSq_eq_norm_sq, Complex.normSq_apply, sq, sq)]
simp only [Complex.add_re, Complex.mul_re, Complex.conj_re, Compl
have h1 : H.Φ1 = H.Φ2 := by rw [← sub_eq_zero, h]
TODO "Define a general effective potential for the two Higgs doublet model,
`StandardModel.HiggsField.EffectivePotential`"

have t_pos : 0 < t := arctan_pos.mpr (by positivity)
have t_ne_zero : t ≠ 0 := t_pos.ne'
have sin_t_pos : 0 < sin t :=
 Real.sin_pos_of_pos_of_lt_pi t_pos (by nlinarith [Real.pi_pos])
have cos_t_pos : 0 < cos t := cos_arctan_pos (2 * Real.sqrt (|c| + 1))⁻¹
have hs := Real.mul_le_sin t_pos.le t_le_pi_div_2
rw [div_mul_eq_mul_div, div_le_iff, Real.pi_pos] at hs
nlinarith [mul_le_mul_of_nonneg_left hs t_pos.le, Real.pi_le_four]
rw [sq_sqrt (by positivity)]
rw [← Complex.normSq_eq_norm_sq, Complex.normSq_apply]
simp only [Complex.sub_re, Complex.sub_im, Complex.mul_re, Complex.mu
 Complex.I_im, Complex.ofReal_re, Complex.ofReal_im, Complex.one_re,
 Complex.sin_ofReal_re, Complex.sin_ofReal_im, Complex.cos_ofReal_re]
nlinarith [Real.sin_sq_add_cos_sq t]
have htan : sin t / cos t = (2 * √(|c| + 1))⁻¹ := by
 rw [← tan_eq_sin_div_cos]
 exact tan_arctan _
rw [potential_H_cos_sin, show -cos t ^ 2 / (4 * sin t ^ 2) = -

```



1.103.

PHYSLIB/PARTICLES/FLAVORPHYSICS/CKMMATRIX/PHASEFREEDOM1.lean

```
1 / (4 * (sin t / cos t) ^ 2) by
 field, htan]
```

### 1.103 Physlib/Particles/FlavorPhysics/CKMMatrix/PhaseFreedom.lean

```
conv_lhs => rw [← norm_mul_exp_arg_mul_I [V]ud]
rw [mul_left_comm, ← exp_add]
rw [show (u : ℂ) * I + d * I + arg [V]ud * I = 0 from by
 have hc : (u : ℂ) + d + arg [V]ud = 0 := by
 exact_mod_cast (show u + d + arg [V]ud = (0 : ℝ) by linarith)
 linear_combination hc * I, exp_zero, mul_one]
rw [phaseShiftApply.us]
conv_lhs => rw [← norm_mul_exp_arg_mul_I [V]us]
rw [mul_left_comm, ← exp_add]
rw [show (u : ℂ) * I + s * I + arg [V]us * I = 0 from by
 have hc : (u : ℂ) + s + arg [V]us = 0 := by
 exact_mod_cast (show u + s + arg [V]us = (0 : ℝ) by linarith)
 linear_combination hc * I, exp_zero, mul_one]
conv_lhs => rw [← norm_mul_exp_arg_mul_I [V]ub]
rw [mul_left_comm, ← exp_add]
rw [show (u : ℂ) * I + b * I + arg [V]ub * I = 0 from by
 have hc : (u : ℂ) + b + arg [V]ub = 0 := by
 exact_mod_cast (show u + b + arg [V]ub = (0 : ℝ) by linarith)
 linear_combination hc * I, exp_zero, mul_one]
conv_lhs => rw [← norm_mul_exp_arg_mul_I [V]cs]
rw [mul_left_comm, ← exp_add]
rw [show (c : ℂ) * I + s * I + arg [V]cs * I = 0 from by
 have hc : (c : ℂ) + s + arg [V]cs = 0 := by
 exact_mod_cast (show c + s + arg [V]cs = (0 : ℝ) by linarith)
 linear_combination hc * I, exp_zero, mul_one]
conv_lhs => rw [← norm_mul_exp_arg_mul_I [V]cb]
rw [mul_left_comm, ← exp_add]
rw [show (c : ℂ) * I + b * I + arg [V]cb * I = 0 from by
 have hc : (c : ℂ) + b + arg [V]cb = 0 := by
 exact_mod_cast (show c + b + arg [V]cb = (0 : ℝ) by linarith)
 linear_combination hc * I, exp_zero, mul_one]
conv_lhs => rw [← norm_mul_exp_arg_mul_I [V]tb]
rw [mul_left_comm, ← exp_add]
rw [show (t : ℂ) * I + b * I + arg [V]tb * I = 0 from by
 have hc : (t : ℂ) + b + arg [V]tb = 0 := by
 exact_mod_cast (show t + b + arg [V]tb = (0 : ℝ) by linarith)
 linear_combination hc * I, exp_zero, mul_one]
simp_all only [Fin.isValue, ne_eq, not_or, Decidable.not_not, VubAbs, ofReal_e
exact h1
```

**1.104 Physlib/Particles/FlavorPhysics/CKMMatrix/Relations.lean**

```

simp_all only [SetLike.coe_mem, Unitary.mul_star_self_of_mem, Fin.isValue
 ofReal_one]
exact ht
have := (normSq_Vud_plus_normSq_Vus_ne_zero_ℝ hV)
simp_all only [Fin.isValue, ne_eq, ← Complex.sq_norm, VudAbs, VusAbs]
exact this
have := (normSq_Vud_plus_normSq_Vus_ne_zero_ℝ hV)
simp_all only [Fin.isValue, ne_eq, ← Complex.sq_norm, VudAbs, VusAbs]
exact this
simp_all only [SetLike.coe_mem, Unitary.star_mul_self_of_mem, Fin.isValue
 ofReal_one]
exact ht

```

**1.105 Physlib/Particles/FlavorPhysics/CKMMatrix/Rows.lean**

```

show $\sum k, \text{conj } (V.1 \ 0 \ k) * V.1 \ 0 \ k = 1$
have ht := congrFun (congrFun (mem_unitaryGroup_iff.mp V.prop) 0) 0
simp only [mul_apply, star_apply, RCLike.star_def, Fin.sum_univ_three, on
 rw [Fin.sum_univ_three]
linear_combination ht
show $\sum k, \text{conj } (V.1 \ 1 \ k) * V.1 \ 1 \ k = 1$
have ht := congrFun (congrFun (mem_unitaryGroup_iff.mp V.prop) 1) 1
simp only [mul_apply, star_apply, RCLike.star_def, Fin.sum_univ_three, on
 rw [Fin.sum_univ_three]
linear_combination ht
show $\sum k, \text{conj } (V.1 \ 2 \ k) * V.1 \ 2 \ k = 1$
have ht := congrFun (congrFun (mem_unitaryGroup_iff.mp V.prop) 2) 2
simp only [mul_apply, star_apply, RCLike.star_def, Fin.sum_univ_three, on
 rw [Fin.sum_univ_three]
linear_combination ht
show $\sum k, \text{conj } (V.1 \ 0 \ k) * V.1 \ 1 \ k = 0$
have ht := congrFun (congrFun (mem_unitaryGroup_iff.mp V.prop) 1) 0
simp only [mul_apply, star_apply, RCLike.star_def, Fin.sum_univ_three, on
 not_false_eq_true, Fin.reduceEq] at ht
rw [Fin.sum_univ_three]
linear_combination ht
show $\sum k, \text{conj } (V.1 \ 0 \ k) * V.1 \ 2 \ k = 0$
have ht := congrFun (congrFun (mem_unitaryGroup_iff.mp V.prop) 2) 0
simp only [mul_apply, star_apply, RCLike.star_def, Fin.sum_univ_three, on
 not_false_eq_true, Fin.reduceEq] at ht

```

1.106.

PHYSLIB/PARTICLES/FLAVORPHYSICS/CKMMATRIX/STANDARDPARAMETERIZATION/BASIC.LEAN

```

rw [Fin.sum_univ_three]
linear_combination ht
show $\sum k, \text{conj } (V.1 \ 1 \ k) * V.1 \ 0 \ k = 0$
have ht := congrFun (congrFun (mem_unitaryGroup_iff.mp V.prop) 0) 1
simp only [mul_apply, star_apply, RCLike.star_def, Fin.sum_univ_three, one_apply_n,
 not_false_eq_true, Fin.reduceEq] at ht
rw [Fin.sum_univ_three]
linear_combination ht
show $\sum k, \text{conj } (V.1 \ 1 \ k) * V.1 \ 2 \ k = 0$
have ht := congrFun (congrFun (mem_unitaryGroup_iff.mp V.prop) 2) 1
simp only [mul_apply, star_apply, RCLike.star_def, Fin.sum_univ_three, one_apply_n,
 not_false_eq_true, Fin.reduceEq] at ht
rw [Fin.sum_univ_three]
linear_combination ht
show $\sum k, \text{conj } (V.1 \ 2 \ k) * V.1 \ 0 \ k = 0$
have ht := congrFun (congrFun (mem_unitaryGroup_iff.mp V.prop) 0) 2
simp only [mul_apply, star_apply, RCLike.star_def, Fin.sum_univ_three, one_apply_n,
 not_false_eq_true, Fin.reduceEq] at ht
rw [Fin.sum_univ_three]
linear_combination ht
show $\sum k, \text{conj } (V.1 \ 2 \ k) * V.1 \ 1 \ k = 0$
have ht := congrFun (congrFun (mem_unitaryGroup_iff.mp V.prop) 1) 2
simp only [mul_apply, star_apply, RCLike.star_def, Fin.sum_univ_three, one_apply_n,
 not_false_eq_true, Fin.reduceEq] at ht
rw [Fin.sum_univ_three]
linear_combination ht
fin_cases i <=> assumption
 fin_cases j <=> exact h1
 fin_cases j <=> exact h1
 fin_cases j <=> exact h1

```

## 1.106 Physlib/Particles/FlavorPhysics/CKMMatrix/StandardParameterization

```

fin_cases j <=> fin_cases i <=>
 simp only [standParamAsMatrix, mul_apply, Fin.sum_univ_three, conjTranspose_apply,
 cons_val', cons_val_zero, cons_val_one, cons_val_two, cons_val_fin_one, head_cons,
 head_fin_const, tail_cons, empty_val', RCLike.star_def, map_mul, map_sub, map_neg,
 ← exp_conj, conj_I, conj_ofReal, neg_mul, neg_neg, one_apply_eq, one_apply_ne,
 Fin.isValue, Fin.zero_eta, Fin.mk_one, Fin.reduceFinMk, Fin.reduceEq,
 not_false_eq_true] <=>
 simp only [ofReal_cos, ofReal_sin, exp_neg] <=>
 field_simp <=> ring_nf <=> simp only [Complex.sin_sq] <=> ring
fin_cases i <=>
 simp only [tRow, standParam, standParamAsMatrix, neg_mul, exp_neg, Fin.isValue,

```

```

cons_val_zero, empty_val', cons_val_fin_one, cons_val_two, tail_cons
cons_val_one, head_cons, Fin.zero_eta, Fin.mk_one, Fin.reduceFinMk, c
LinearMap.mk₂_apply, Pi.conj_apply, _root_.map_mul, map_inv₀, ← exp_c
inv_inv, map_sub, map_neg] <=>
simp only [ofReal_sin, ofReal_cos] <=>
field_simp <=>
ring_nf <=>
rw [sin_sq] <=>
have he : cexp (I * ↑δ₁₃) = cexp (I * ↑δ₁₃') := by rw [mul_comm, h, mul_c
simp only [exp_neg, he]
 rw [Complex.norm_real, Real.norm_eq_abs, sq_abs]
 simp [hx]
simp [exp_neg]

```

### 1.107 Physlib/Particles/FlavorPhysics/CKMMatrix/StandardParamet

```

rw [S₁₂]
exact div_nonneg (VAbs_ge_zero 0 1 V) (Real.sqrt_nonneg _)
· rw [S₂₃, if_neg ha]
 exact div_nonneg (VAbs_ge_zero 1 2 V) (Real.sqrt_nonneg _)
rw [S₁₂]
apply div_le_one_of_le₀ _ (Real.sqrt_nonneg _)
calc VusAbs V = √(VusAbs V ^ 2) := (Real.sqrt_sq (VAbs_ge_zero 0 1 V)).sy
_ ≤ √(VudAbs V ^ 2 + VusAbs V ^ 2) :=
 Real.sqrt_le_sqrt (by nlinarith [sq_nonneg (VudAbs V)])
· rw [S₂₃, if_neg ha]
 apply div_le_one_of_le₀ _ (Real.sqrt_nonneg _)
 rw [VudAbs_sq_add_VusAbs_sq, ← VcbAbs_sq_add_VtbAbs_sq]
 calc VcbAbs V = √(VcbAbs V ^ 2) := (Real.sqrt_sq (VAbs_ge_zero 1 2 V)).
_ ≤ √(VcbAbs V ^ 2 + VtbAbs V ^ 2) :=
 Real.sqrt_le_sqrt (by nlinarith [sq_nonneg (VtbAbs V)])
exact Real.norm_of_nonneg (S₁₂_nonneg _)
exact Real.norm_of_nonneg (S₁₃_nonneg _)
exact Real.norm_of_nonneg (S₂₃_nonneg _)
simp [S₁₂, VudAbs_sq_add_VusAbs_sq, ha]
exact Real.norm_of_nonneg (Real.cos_arcsin_nonneg _)
exact Real.norm_of_nonneg (Real.cos_arcsin_nonneg _)
exact Real.norm_of_nonneg (Real.cos_arcsin_nonneg _)
rw [C₁₃, θ₁₃, Real.cos_arcsin, S₁₃, ← VudAbs_sq_add_VusAbs_sq]
· rw [C₁₂_eq_Vud_div_sqrt ha, C₁₃, θ₁₃, Real.cos_arcsin, S₁₃, ← VudAbs_sq
rw [C₁₃, θ₁₃, Real.cos_arcsin, S₁₂, S₁₃, ← VudAbs_sq_add_VusAbs_sq, mul_c
· have h₀ : VudAbs V ^ 2 + VusAbs V ^ 2 = 0 := by
 rw [VudAbs_sq_add_VusAbs_sq, ha]
 ring

```

1.108.

PHYSLIB/PARTICLES/STANDARDMODEL/ANOMALYANCELLATION/BASIC.LEAN

```

 rw [h0]
 · exact (mul_div_cancel0 (VusAbs V) (VAbsub_ne_zero_sqrt_Vud_Vus_ne_zero ha)).symm
 apply CKMMatrix_ext
 simp [standParam, standParamAsMatrix, phaseShift, phaseShiftMatrix, mul_apply,
 Fin.sum_univ_three, ofReal_cos, ofReal_sin, hC12, hS12, hS13, h, exp_neg]
 apply CKMMatrix_ext
 simp [standParam, standParamAsMatrix, phaseShift, phaseShiftMatrix, mul_apply,
 Fin.sum_univ_three, h, exp_neg] <=>
 apply CKMMatrix_ext
 simp [standParam, standParamAsMatrix, phaseShift, phaseShiftMatrix, mul_apply,
 Fin.sum_univ_three, h, exp_neg] <=>
 field_simp
 apply CKMMatrix_ext
 simp [standParam, standParamAsMatrix, phaseShift, phaseShiftMatrix, mul_apply,
 Fin.sum_univ_three, h, exp_neg]
 apply CKMMatrix_ext
 simp [standParam, standParamAsMatrix, phaseShift, phaseShiftMatrix, mul_apply,
 Fin.sum_univ_three, h, exp_neg] <=>
 apply CKMMatrix_ext
 simp [standParam, standParamAsMatrix, phaseShift, phaseShiftMatrix, mul_apply,
 Fin.sum_univ_three, h, exp_neg] <=>
 field_simp
 have hb' : VubAbs [V] ≠ 1 := (ud_us_ne_zero_iff_ub_ne_one V).mp hb
 · simp [uRow, standParam, standParamAsMatrix, hV.1, VudAbs_eq_C12_mul_C13 [V], C13]
 · simp [uRow, standParam, standParamAsMatrix, hV.2.1, VusAbs_eq_S12_mul_C13 [V],
 ← S12_eq_sin_θ12 [V], C13]
 · simp [cRow, standParam, standParamAsMatrix, hV.2.2.1, VcbAbs_eq_S23_mul_C13 [V],
 S23_eq_Csin_θ23 [V], C13]
 · have haU : [U]ud ≠ 0 ∨ [U]us ≠ 0 := by
 rw [ud_us_ne_zero_iff_ub_ne_one] at ha ⊢
 rwa [show norm [U]ub = VubAbs [U] from rfl, hUV]
 · have haU : ¬ ([U]ud ≠ 0 ∨ [U]us ≠ 0) := by
 rw [ud_us_ne_zero_iff_ub_ne_one] at ha ⊢
 rwa [show norm [U]ub = VubAbs [U] from rfl, hUV]

```

## 1.108 Physlib/Particles/StandardModel/AnomalyCancellation/Basic.lean

```

lemma sum_SMSpecies_numberCharges_one {M} [AddCommMonoid M]
 (f : Fin (SMSpecies 1).numberCharges → M) :
 ∑ i, f i = f {0, by simp} := by
 change ∑ (i : Fin 1), f i = _
 simp only [Finset.univ_unique, Fin.default_eq_zero, Fin.isValue, Finset.sum_single]
 rfl

```

```

def toSpeciesEquiv : (SMCharges n).Charges ≈ (Fin 5 → Fin (SMSpecies n).num
 ((Equiv.curry _ _ _).symm.trans (@finProdFinEquiv 5
 (SMSpecies n).numberCharges).arrowCongr (Equiv.refl ℚ))).symm
lemma toSpecies_apply_eq (i : Fin 5) (S : (SMCharges n).Charges) :
 toSpecies i S = fun j => toSpeciesEquiv S i j := by rfl

simp only [ACCSystemCharges.chargesAddCommMonoid_add, toSpecies_apply,
 simp only [HSMul.hSMul, SMul.smul, toSpecies_apply, Fin.isValue,
 simp only [accGrav, toSpecies_apply, Fin.isValue, LinearMap.coe_mk,
 simp only [ACCSystemCharges.chargesAddCommMonoid_add, toSpecies_apply,
 simp only [HSMul.hSMul, SMul.smul, toSpecies_apply, Fin.isValue,
 simp only [accSU2, toSpecies_apply, Fin.isValue, LinearMap.coe_mk,
 simp only [ACCSystemCharges.chargesAddCommMonoid_add, toSpecies_apply,
 simp only [HSMul.hSMul, SMul.smul, toSpecies_apply, Fin.isValue,
 simp only [accSU3, toSpecies_apply, Fin.isValue, LinearMap.coe_mk,
 simp only [ACCSystemCharges.chargesAddCommMonoid_add, toSpecies_apply,
 simp only [HSMul.hSMul, SMul.smul, toSpecies_apply, Fin.isValue,
 simp only [accYY, toSpecies_apply, Fin.isValue, LinearMap.coe_mk,
 simp only [toSpecies_apply, Fin.isValue, neg_mul, one_mul]
 simp only [toSpecies_apply, Fin.isValue]
 simp only [toSpecies_apply, Fin.isValue]

```

### 1.109 Physlib/Particles/StandardModel/AnomalyCancellation/Fam1

```

simp only [ACCSystemCharges.chargesAddCommMonoid_add]
simp only [HSMul.hSMul, ACCSystemCharges.chargesModule_smul, eq_ratCast]

```

### 1.110 Physlib/Particles/StandardModel/AnomalyCancellation/NoGra

```

simp only [SMNoGrav_linearACCs] at hS
exact hS ⟨0, by simp⟩
simp only [SMNoGrav_linearACCs] at hS
exact hS ⟨1, by simp⟩
lemma cubeSol (S : (SMNoGrav n).Sols) : accCube S.val = 0 := S.cubicSol
| ⟨0, _⟩ => exact hSU2
| ⟨1, _⟩ => exact hSU3

```

### 1.111 Physlib/Particles/StandardModel/AnomalyCancellation/NoGra

```

have hE := E_zero_iff_Q_zero.mp hQ
simp_all only [toSpecies_apply_eq, Fin.isValue, sum_SMSpecies_numberCharg
 AddHom.coe_mk]

```

1.112.

PHYSLIB/PARTICLES/STANDARDMODEL/ANOMALYANCELLATION/NOGRAV/ONE/LINEARPARAMETERIZATION

```

erw [hQ, hE]
simp only [accSU2, toSpecies_apply_eq, Fin.isValue, sum_SMSpecies_numberCharges_one,
 LinearMap.coe_mk, AddHom.coe_mk, accSU3] at h1 h2
erw [hQ] at h1 h2

```

## 1.112 Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/L

```

 toSpeciesEquiv S.asCharges= fun i _ => S.asCharges i := by
 funext i j
 match j with
 | (0 : Fin 1) =>
 lemma toSpecies_apply_asCharges (S : linearParameters) (i : Fin 5) :
 toSpecies i S.asCharges = fun _ => S.asCharges i := by
 funext j
 simp only [SMCharges.toSpecies_apply_eq, speciesVal]

 simp only [accSU2, toSpecies_apply_asCharges, Fin.isValue, sum_SMSpecies_numberCharges_one,
 LinearMap.coe_mk, AddHom.coe_mk]
 simp [asCharges])
 simp only [accSU3, SMCharges.toSpecies_apply_eq, sum_SMSpecies_numberCharges_one,
 LinearMap.coe_mk, AddHom.coe_mk, speciesVal, asCharges, neg_add_rev]
 simp only [toSpecies_apply, Fin.isValue, sum_SMSpecies_numberCharges_one, Fin.zero]
 repeat erw [speciesVal]
 repeat erw [asLinear_val]
 simp only [Fin.isValue, toSpecies_apply]
 repeat erw [speciesVal]
 simp only [Fin.isValue, toSpecies_apply_eq]
 match j with
 | (0, _) => rfl
 rw [toSpecies_apply_asCharges]
 simp only [accSU3, toSpecies_apply_eq, Fin.isValue, sum_SMSpecies_numberCharges_one,
 simp only [accSU2,
 Fin.isValue, toSpecies_apply_eq, sum_SMSpecies_numberCharges_one,
 simp only [asCharges, Fin.isValue, toSpecies_apply_eq]
 linear_combination (norm := ring_nf) -(1 * h1)
 simp only [Fin.isValue, Fin.zero_eta, toSpeciesEquiv_apply, Nat.reduceMul]
 ring
 simp only [asCharges, Fin.isValue, neg_add_rev, toSpecies_apply_eq]
 linear_combination (norm := ring_nf) -(1 * h1)
 simp only [Fin.isValue, Fin.zero_eta, toSpeciesEquiv_apply, Nat.reduceMul]
 ring
 simp only [asCharges, Fin.isValue, neg_mul, toSpecies_apply_eq]
 linear_combination (norm := ring_nf) -(1 * h2)
 simp only [Fin.isValue, Fin.zero_eta, toSpeciesEquiv_apply, Nat.reduceMul]

```

```

 ring
 simp only [toSpecies_apply_eq, Fin.isValue, sum_SMSpecies_numberCharges_o
 AddHom.coe_mk]

```

### 1.113 Physlib/Particles/StandardModel/AnomalyCancellation/Permu

```

 erw [toSMSpecies_toSpecies_inv, toSMSpecies_toSpecies_inv, toSMSpecies_

```

### 1.114 Physlib/Particles/StandardModel/Basic.lean

Authors: Nikolai Kashcheev, Joseph Tooby-Smith

```

public import Mathlib.RingTheory.RootsOfUnity.Complex
/-!

```

```

The unquotiented gauge group

```

```

-/

```

```

/-!

```

```

The $\mathbb{Z}_6$ quotient

```

```

-/

```

```

/-- The unitary complex number associated to a sixth root of unity. -
/

```

```

noncomputable def gaugeGroup $\mathbb{Z}_6$ UnitaryOfRoot (α : rootsOfUnity 6 $\mathbb{C}$) : unitar
 ((α : $\mathbb{C}^\times$) : $\mathbb{C}$), by
 have h α : ||(α : $\mathbb{C}^\times$) : $\mathbb{C}$ || = 1 := Complex.norm_eq_one_of_mem_rootsOfUni
 constructor
 · rw [RCLike.star_def, Complex.conj_mul', h α]
 norm_num
 · rw [RCLike.star_def, Complex.mul_conj', h α]
 norm_num

```

```

@[simp]

```

```

lemma gaugeGroup $\mathbb{Z}_6$ UnitaryOfRoot_coe (α : rootsOfUnity 6 $\mathbb{C}$) :
 (gaugeGroup $\mathbb{Z}_6$ UnitaryOfRoot α : $\mathbb{C}$) = ((α : $\mathbb{C}^\times$) : $\mathbb{C}$) := rfl

```

```

/-- The `SU(3)` scalar matrix associated to a sixth root of unity. -
/

```

```

noncomputable def gaugeGroup $\mathbb{Z}_6$ SU3OfRoot (α : rootsOfUnity 6 $\mathbb{C}$) :
 specialUnitaryGroup (Fin 3) $\mathbb{C}$:=

```



```

let z : ℂ := ((α : ℂ×) : ℂ)
(scalar (Fin 3) (z ^ 2), by
 rw [mem_specialUnitaryGroup_iff]
 have hz : ||z|| = 1 := by
 simp [z] using Complex.norm_eq_one_of_mem_rootsOfUnity α.prop
 have hz2 : star (z ^ 2) * z ^ 2 = 1 := by
 rw [RCLike.star_def, Complex.conj_mul', Complex.norm_pow, hz]
 norm_num
 constructor
 · rw [mem_unitaryGroup_iff']
 rw [Matrix.scalar_apply, Matrix.star_eq_conjTranspose, Matrix.diagonal_conjTransp,
 Matrix.diagonal_mul_diagonal, Matrix.diagonal_eq_one]
 funext i
 simp [Pi.star_def] using hz2
 · have hα : z ^ 6 = 1 := by
 simp [z] using (mem_rootsOfUnity' 6 (α : ℂ×)).mp α.prop
 rw [Matrix.scalar_apply, Matrix.det_diagonal, Fin.prod_univ_three]
 calc
 z ^ 2 * z ^ 2 * z ^ 2 = z ^ 6 := by ring
 _ = 1 := hα)

/-- The `SU(2)` scalar matrix associated to a sixth root of unity. -
/
noncomputable def gaugeGroupZ6SU2OfRoot (α : rootsOfUnity 6 ℂ) :
 specialUnitaryGroup (Fin 2) ℂ := by
 let u : unitary ℂ := gaugeGroupZ6UnitaryOfRoot α
 let z : ℂ := ((α : ℂ×) : ℂ)
 let w : ℂ := star ((u ^ 3 : unitary ℂ) : ℂ)
 refine (scalar (Fin 2) w, ?_)
 rw [mem_specialUnitaryGroup_iff]
 have hw : star w * w = 1 := by
 change star (star ((u ^ 3 : unitary ℂ) : ℂ)) *
 star ((u ^ 3 : unitary ℂ) : ℂ) = 1
 rw [star_star]
 exact (u ^ 3 : unitary ℂ).prop.2
 have hα : z ^ 6 = 1 := by
 simp [z] using (mem_rootsOfUnity' 6 (α : ℂ×)).mp α.prop
 have hw2 : w ^ 2 = 1 := by
 calc
 w ^ 2 = star (z ^ 6) := by
 simp [w, u, z, pow_succ]
 ring
 _ = 1 := by simp [hα]
 constructor
 · rw [mem_unitaryGroup_iff']
 rw [Matrix.scalar_apply, Matrix.star_eq_conjTranspose, Matrix.diagonal_conjTransp

```

```

 Matrix.diagonal_mul_diagonal, Matrix.diagonal_eq_one]
 funext i
 simp [Pi.star_def] using hw
 · rw [Matrix.scalar_apply, Matrix.det_diagonal, Fin.prod_univ_two]
 simp [pow_two] using hw2

/-- The element of `GaugeGroupI` associated to a sixth root of unity. -
/
noncomputable def gaugeGroupZ6OfRoot (α : rootsOfUnity 6 ℂ) : GaugeGroupI :
 (gaugeGroupZ6SU3OfRoot α, gaugeGroupZ6SU2OfRoot α, gaugeGroupZ6UnitaryOfRoot α)

@[simp]
lemma gaugeGroupZ6OfRoot_toSU3 (α : rootsOfUnity 6 ℂ) :
 GaugeGroupI.toSU3 (gaugeGroupZ6OfRoot α) = gaugeGroupZ6SU3OfRoot α := r

@[simp]
lemma gaugeGroupZ6OfRoot_toSU2 (α : rootsOfUnity 6 ℂ) :
 GaugeGroupI.toSU2 (gaugeGroupZ6OfRoot α) = gaugeGroupZ6SU2OfRoot α := r

@[simp]
lemma gaugeGroupZ6OfRoot_toU1 (α : rootsOfUnity 6 ℂ) :
 GaugeGroupI.toU1 (gaugeGroupZ6OfRoot α) = gaugeGroupZ6UnitaryOfRoot α := r

lemma gaugeGroupZ6OfRoot_mem_center (α : rootsOfUnity 6 ℂ) :
 gaugeGroupZ6OfRoot α ∈ Subgroup.center GaugeGroupI := by
 rw [Subgroup.mem_center_iff]
 intro g
 apply GaugeGroupI.ext
 · change GaugeGroupI.toSU3 g * gaugeGroupZ6SU3OfRoot α =
 gaugeGroupZ6SU3OfRoot α * GaugeGroupI.toSU3 g
 ext i j
 simp [gaugeGroupZ6SU3OfRoot, Matrix.scalar_apply, mul_comm]
 · change GaugeGroupI.toSU2 g * gaugeGroupZ6SU2OfRoot α =
 gaugeGroupZ6SU2OfRoot α * GaugeGroupI.toSU2 g
 ext i j
 simp [gaugeGroupZ6SU2OfRoot, Matrix.scalar_apply, mul_comm]
 · change GaugeGroupI.toU1 g * gaugeGroupZ6UnitaryOfRoot α =
 gaugeGroupZ6UnitaryOfRoot α * GaugeGroupI.toU1 g
 exact mul_comm _ _

/-- The homomorphism from sixth roots of unity to `GaugeGroupI`. -
/
noncomputable def gaugeGroupZ6Hom : rootsOfUnity 6 ℂ →* GaugeGroupI where
 toFun := gaugeGroupZ6OfRoot
 map_one' := by
 apply GaugeGroupI.ext

```

```

 · change gaugeGroupZ6SU30fRoot 1 = 1
 ext i j
 simp [gaugeGroupZ6SU30fRoot, Matrix.scalar_apply]
 · change gaugeGroupZ6SU20fRoot 1 = 1
 ext i j
 simp [gaugeGroupZ6SU20fRoot, gaugeGroupZ6Unitary0fRoot, Matrix.scalar_apply]
 · change gaugeGroupZ6Unitary0fRoot 1 = 1
 ext
 simp [gaugeGroupZ6Unitary0fRoot]
map_mul' α β := by
 apply GaugeGroupI.ext
 · change gaugeGroupZ6SU30fRoot (α * β) =
 gaugeGroupZ6SU30fRoot α * gaugeGroupZ6SU30fRoot β
 ext i j
 simp [gaugeGroupZ6SU30fRoot, Matrix.scalar_apply, pow_two, mul_left_comm, mul_
 · change gaugeGroupZ6SU20fRoot (α * β) =
 gaugeGroupZ6SU20fRoot α * gaugeGroupZ6SU20fRoot β
 ext i j
 fin_cases i <=> fin_cases j <=>
 simp [gaugeGroupZ6SU20fRoot, gaugeGroupZ6Unitary0fRoot, Matrix.scalar_apply,
 pow_succ] <=>
 ring
 · change gaugeGroupZ6Unitary0fRoot (α * β) =
 gaugeGroupZ6Unitary0fRoot α * gaugeGroupZ6Unitary0fRoot β
 ext
 simp [gaugeGroupZ6Unitary0fRoot]

@[simp]
lemma gaugeGroupZ6Hom_apply (α : rootsOfUnity 6 ℂ) :
 gaugeGroupZ6Hom α = gaugeGroupZ60fRoot α := rfl

@[simp]
lemma gaugeGroupZ6Hom_toSU3 (α : rootsOfUnity 6 ℂ) :
 GaugeGroupI.toSU3 (gaugeGroupZ6Hom α) = gaugeGroupZ6SU30fRoot α := rfl

@[simp]
lemma gaugeGroupZ6Hom_toSU2 (α : rootsOfUnity 6 ℂ) :
 GaugeGroupI.toSU2 (gaugeGroupZ6Hom α) = gaugeGroupZ6SU20fRoot α := rfl

@[simp]
lemma gaugeGroupZ6Hom_toU1 (α : rootsOfUnity 6 ℂ) :
 GaugeGroupI.toU1 (gaugeGroupZ6Hom α) = gaugeGroupZ6Unitary0fRoot α := rfl

noncomputable def gaugeGroupZ6SubGroup : Subgroup GaugeGroupI :=
 gaugeGroupZ6Hom.range

```

```

lemma gaugeGroupZ6OfRoot_mem (α : rootsOfUnity 6 ℂ) :
 gaugeGroupZ6OfRoot α ∈ gaugeGroupZ6SubGroup :=
 {α, rfl}

lemma mem_gaugeGroupZ6SubGroup_iff (g : GaugeGroupI) :
 g ∈ gaugeGroupZ6SubGroup ↔ ∃ α : rootsOfUnity 6 ℂ, gaugeGroupZ6OfRoot α
 simp [gaugeGroupZ6SubGroup]

lemma gaugeGroupZ6SubGroup_le_center :
 gaugeGroupZ6SubGroup ≤ Subgroup.center GaugeGroupI := by
 intro g hg
 rw [mem_gaugeGroupZ6SubGroup_iff] at hg
 rcases hg with {α, rfl}
 exact gaugeGroupZ6OfRoot_mem_center α

instance gaugeGroupZ6SubGroup_normal : gaugeGroupZ6SubGroup.Normal where
 conj_mem n hn g := by
 have hn_center : n ∈ Subgroup.center GaugeGroupI := gaugeGroupZ6SubGroup
 have hcomm : g * n = n * g := (Subgroup.mem_center_iff.mp hn_center) g
 have hconj : g * n * g⁻¹ = n := by
 calc
 g * n * g⁻¹ = n * g * g⁻¹ := by rw [hcomm]
 _ = n := by simp [mul_assoc]
 simp [hconj] using hn
def GaugeGroupZ6 : Type :=
 GaugeGroupI ⊔ gaugeGroupZ6SubGroup

noncomputable instance : Group GaugeGroupZ6 :=
 inferInstanceAs (Group (GaugeGroupI ⊔ gaugeGroupZ6SubGroup))

namespace GaugeGroupZ6

/-- The quotient map from `GaugeGroupI` to `GaugeGroupZ6`. -
/
noncomputable def mk : GaugeGroupI →* GaugeGroupZ6 :=
 QuotientGroup.mk' gaugeGroupZ6SubGroup

@[simp]
lemma mk_gaugeGroupZ6OfRoot (α : rootsOfUnity 6 ℂ) :
 mk (gaugeGroupZ6OfRoot α) = 1 := by
 change ((gaugeGroupZ6OfRoot α : GaugeGroupI) : GaugeGroupI ⊔ gaugeGroupZ6
 rw [QuotientGroup.eq_one_iff]
 exact gaugeGroupZ6OfRoot_mem α

end GaugeGroupZ6

```

```

/-!

The $\mathbb{Z}_2$ quotient

-/

/-- The inclusion of second roots of unity into sixth roots of unity. -
/
noncomputable def gaugeGroup $\mathbb{Z}_2$ RootTo $\mathbb{Z}_6$ Root : rootsOfUnity 2 $\mathbb{C}$ $\rightarrow^*$ rootsOfUnity 6 $\mathbb{C}$:=
 Subgroup.inclusion (rootsOfUnity_le_of_dvd (by norm_num : 2 $\square$ 6))

/-- The element of `GaugeGroupI` associated to a second root of unity. -
/
noncomputable def gaugeGroup $\mathbb{Z}_2$ OfRoot (α : rootsOfUnity 2 $\mathbb{C}$) : GaugeGroupI :=
 gaugeGroup $\mathbb{Z}_6$ OfRoot (gaugeGroup $\mathbb{Z}_2$ RootTo $\mathbb{Z}_6$ Root α)

@[simp]
lemma gaugeGroup $\mathbb{Z}_2$ OfRoot_toSU3 (α : rootsOfUnity 2 $\mathbb{C}$) :
 GaugeGroupI.toSU3 (gaugeGroup $\mathbb{Z}_2$ OfRoot α) =
 gaugeGroup $\mathbb{Z}_6$ SU3OfRoot (gaugeGroup $\mathbb{Z}_2$ RootTo $\mathbb{Z}_6$ Root α) := rfl

@[simp]
lemma gaugeGroup $\mathbb{Z}_2$ OfRoot_toSU2 (α : rootsOfUnity 2 $\mathbb{C}$) :
 GaugeGroupI.toSU2 (gaugeGroup $\mathbb{Z}_2$ OfRoot α) =
 gaugeGroup $\mathbb{Z}_6$ SU2OfRoot (gaugeGroup $\mathbb{Z}_2$ RootTo $\mathbb{Z}_6$ Root α) := rfl

@[simp]
lemma gaugeGroup $\mathbb{Z}_2$ OfRoot_toU1 (α : rootsOfUnity 2 $\mathbb{C}$) :
 GaugeGroupI.toU1 (gaugeGroup $\mathbb{Z}_2$ OfRoot α) =
 gaugeGroup $\mathbb{Z}_6$ UnitaryOfRoot (gaugeGroup $\mathbb{Z}_2$ RootTo $\mathbb{Z}_6$ Root α) := rfl

lemma gaugeGroup $\mathbb{Z}_2$ OfRoot_mem_center (α : rootsOfUnity 2 $\mathbb{C}$) :
 gaugeGroup $\mathbb{Z}_2$ OfRoot α $\in$ Subgroup.center GaugeGroupI :=
 gaugeGroup $\mathbb{Z}_6$ OfRoot_mem_center (gaugeGroup $\mathbb{Z}_2$ RootTo $\mathbb{Z}_6$ Root α)

/-- The homomorphism from second roots of unity to `GaugeGroupI`. -
/
noncomputable def gaugeGroup $\mathbb{Z}_2$ Hom : rootsOfUnity 2 $\mathbb{C}$ $\rightarrow^*$ GaugeGroupI :=
 gaugeGroup $\mathbb{Z}_6$ Hom.comp gaugeGroup $\mathbb{Z}_2$ RootTo $\mathbb{Z}_6$ Root

@[simp]
lemma gaugeGroup $\mathbb{Z}_2$ Hom_apply (α : rootsOfUnity 2 $\mathbb{C}$) :
 gaugeGroup $\mathbb{Z}_2$ Hom α = gaugeGroup $\mathbb{Z}_2$ OfRoot α := rfl

@[simp]
lemma gaugeGroup $\mathbb{Z}_2$ Hom_toSU3 (α : rootsOfUnity 2 $\mathbb{C}$) :

```

```

GaugeGroupI.toSU3 (gaugeGroup $\mathbb{Z}_2$ Hom α) =
 gaugeGroup $\mathbb{Z}_6$ SU3ofRoot (gaugeGroup $\mathbb{Z}_2$ RootTo $\mathbb{Z}_6$ Root α) := rfl

@[simp]
lemma gaugeGroup $\mathbb{Z}_2$ Hom_toSU2 (α : rootsOfUnity 2 $\mathbb{C}$) :
 GaugeGroupI.toSU2 (gaugeGroup $\mathbb{Z}_2$ Hom α) =
 gaugeGroup $\mathbb{Z}_6$ SU2ofRoot (gaugeGroup $\mathbb{Z}_2$ RootTo $\mathbb{Z}_6$ Root α) := rfl

@[simp]
lemma gaugeGroup $\mathbb{Z}_2$ Hom_toU1 (α : rootsOfUnity 2 $\mathbb{C}$) :
 GaugeGroupI.toU1 (gaugeGroup $\mathbb{Z}_2$ Hom α) =
 gaugeGroup $\mathbb{Z}_6$ UnitaryOfRoot (gaugeGroup $\mathbb{Z}_2$ RootTo $\mathbb{Z}_6$ Root α) := rfl
/-- The $\mathbb{Z}_2$ -subgroup of the un-quotiented gauge group which acts trivially on
noncomputable def gaugeGroup $\mathbb{Z}_2$ SubGroup : Subgroup GaugeGroupI :=
 gaugeGroup $\mathbb{Z}_2$ Hom.range

lemma gaugeGroup $\mathbb{Z}_2$ OfRoot_mem (α : rootsOfUnity 2 $\mathbb{C}$) :
 gaugeGroup $\mathbb{Z}_2$ OfRoot α $\in$ gaugeGroup $\mathbb{Z}_2$ SubGroup :=
 $\langle \alpha, \text{rfl} \rangle$

lemma mem_gaugeGroup $\mathbb{Z}_2$ SubGroup_iff (g : GaugeGroupI) :
 g $\in$ gaugeGroup $\mathbb{Z}_2$ SubGroup $\leftrightarrow \exists \alpha$: rootsOfUnity 2 $\mathbb{C}$, gaugeGroup $\mathbb{Z}_2$ OfRoot α
 simp [gaugeGroup $\mathbb{Z}_2$ SubGroup]

lemma gaugeGroup $\mathbb{Z}_2$ SubGroup_le_gaugeGroup $\mathbb{Z}_6$ SubGroup :
 gaugeGroup $\mathbb{Z}_2$ SubGroup $\leq$ gaugeGroup $\mathbb{Z}_6$ SubGroup := by
 intro g hg
 rw [mem_gaugeGroup $\mathbb{Z}_2$ SubGroup_iff] at hg
 rcases hg with $\langle \alpha, \text{rfl} \rangle$
 exact gaugeGroup $\mathbb{Z}_6$ OfRoot_mem (gaugeGroup $\mathbb{Z}_2$ RootTo $\mathbb{Z}_6$ Root α)

lemma gaugeGroup $\mathbb{Z}_2$ SubGroup_le_center :
 gaugeGroup $\mathbb{Z}_2$ SubGroup $\leq$ Subgroup.center GaugeGroupI := by
 intro g hg
 rw [mem_gaugeGroup $\mathbb{Z}_2$ SubGroup_iff] at hg
 rcases hg with $\langle \alpha, \text{rfl} \rangle$
 exact gaugeGroup $\mathbb{Z}_2$ OfRoot_mem_center α

instance gaugeGroup $\mathbb{Z}_2$ SubGroup_normal : gaugeGroup $\mathbb{Z}_2$ SubGroup.Normal where
 conj_mem n hn g := by
 have hn_center : n $\in$ Subgroup.center GaugeGroupI := gaugeGroup $\mathbb{Z}_2$ SubGroup
 have hcomm : g * n = n * g := (Subgroup.mem_center_iff.mp hn_center) g
 have hconj : g * n * g-1 = n := by
 calc
 g * n * g-1 = n * g * g-1 := by rw [hcomm]
 _ = n := by simp [mul_assoc]

```

```

 simp [hconj] using hn
def GaugeGroup $\mathbb{Z}_2$: Type :=
 GaugeGroupI $\sqsubset$ gaugeGroup $\mathbb{Z}_2$ SubGroup

noncomputable instance : Group GaugeGroup $\mathbb{Z}_2$:=
 inferInstanceAs (Group (GaugeGroupI $\sqsubset$ gaugeGroup $\mathbb{Z}_2$ SubGroup))

namespace GaugeGroup $\mathbb{Z}_2$

/-- The quotient map from `GaugeGroupI` to `GaugeGroup $\mathbb{Z}_2``. -
/
noncomputable def mk : GaugeGroupI $\rightarrow^*$ GaugeGroup $\mathbb{Z}_2$:=
 QuotientGroup.mk' gaugeGroup $\mathbb{Z}_2$ SubGroup

@[simp]
lemma mk_gaugeGroup $\mathbb{Z}_2$ OfRoot (α : rootsOfUnity 2 $\mathbb{C}$) :
 mk (gaugeGroup $\mathbb{Z}_2$ OfRoot α) = 1 := by
 change ((gaugeGroup $\mathbb{Z}_2$ OfRoot α : GaugeGroupI) : GaugeGroupI $\sqsubset$ gaugeGroup $\mathbb{Z}_2$ SubGroup)
 rw [QuotientGroup.eq_one_iff]
 exact gaugeGroup $\mathbb{Z}_2$ OfRoot_mem α

end GaugeGroup $\mathbb{Z}_2$

/-!

The $\mathbb{Z}_3$ quotient

-/

/-- The inclusion of third roots of unity into sixth roots of unity. -
/
noncomputable def gaugeGroup $\mathbb{Z}_3$ RootTo $\mathbb{Z}_6$ Root : rootsOfUnity 3 $\mathbb{C}$ $\rightarrow^*$ rootsOfUnity 6 $\mathbb{C}$:=
 Subgroup.inclusion (rootsOfUnity_le_of_dvd (by norm_num : 3 $\sqsubset$ 6))

/-- The element of `GaugeGroupI` associated to a third root of unity. -
/
noncomputable def gaugeGroup $\mathbb{Z}_3$ OfRoot (α : rootsOfUnity 3 $\mathbb{C}$) : GaugeGroupI :=
 gaugeGroup $\mathbb{Z}_6$ OfRoot (gaugeGroup $\mathbb{Z}_3$ RootTo $\mathbb{Z}_6$ Root α)

@[simp]
lemma gaugeGroup $\mathbb{Z}_3$ OfRoot_toSU3 (α : rootsOfUnity 3 $\mathbb{C}$) :
 GaugeGroupI.toSU3 (gaugeGroup $\mathbb{Z}_3$ OfRoot α) =
 gaugeGroup $\mathbb{Z}_6$ SU3OfRoot (gaugeGroup $\mathbb{Z}_3$ RootTo $\mathbb{Z}_6$ Root α) := rfl

@[simp]
lemma gaugeGroup $\mathbb{Z}_3$ OfRoot_toSU2 (α : rootsOfUnity 3 $\mathbb{C}$) :$
```

```

GaugeGroupI.toSU2 (gaugeGroup $\mathbb{Z}_3$ OfRoot α) =
 gaugeGroup $\mathbb{Z}_6$ SU20fRoot (gaugeGroup $\mathbb{Z}_3$ RootTo $\mathbb{Z}_6$ Root α) := rfl

@[simp]
lemma gaugeGroup $\mathbb{Z}_3$ OfRoot_toU1 (α : rootsOfUnity 3 $\mathbb{C}$) :
 GaugeGroupI.toU1 (gaugeGroup $\mathbb{Z}_3$ OfRoot α) =
 gaugeGroup $\mathbb{Z}_6$ UnitaryOfRoot (gaugeGroup $\mathbb{Z}_3$ RootTo $\mathbb{Z}_6$ Root α) := rfl

lemma gaugeGroup $\mathbb{Z}_3$ OfRoot_mem_center (α : rootsOfUnity 3 $\mathbb{C}$) :
 gaugeGroup $\mathbb{Z}_3$ OfRoot α $\in$ Subgroup.center GaugeGroupI :=
 gaugeGroup $\mathbb{Z}_6$ OfRoot_mem_center (gaugeGroup $\mathbb{Z}_3$ RootTo $\mathbb{Z}_6$ Root α)

/-- The homomorphism from third roots of unity to `GaugeGroupI`. -
/
noncomputable def gaugeGroup $\mathbb{Z}_3$ Hom : rootsOfUnity 3 $\mathbb{C}$ $\rightarrow^*$ GaugeGroupI :=
 gaugeGroup $\mathbb{Z}_6$ Hom.comp gaugeGroup $\mathbb{Z}_3$ RootTo $\mathbb{Z}_6$ Root

@[simp]
lemma gaugeGroup $\mathbb{Z}_3$ Hom_apply (α : rootsOfUnity 3 $\mathbb{C}$) :
 gaugeGroup $\mathbb{Z}_3$ Hom α = gaugeGroup $\mathbb{Z}_3$ OfRoot α := rfl

@[simp]
lemma gaugeGroup $\mathbb{Z}_3$ Hom_toSU3 (α : rootsOfUnity 3 $\mathbb{C}$) :
 GaugeGroupI.toSU3 (gaugeGroup $\mathbb{Z}_3$ Hom α) =
 gaugeGroup $\mathbb{Z}_6$ SU30fRoot (gaugeGroup $\mathbb{Z}_3$ RootTo $\mathbb{Z}_6$ Root α) := rfl

@[simp]
lemma gaugeGroup $\mathbb{Z}_3$ Hom_toSU2 (α : rootsOfUnity 3 $\mathbb{C}$) :
 GaugeGroupI.toSU2 (gaugeGroup $\mathbb{Z}_3$ Hom α) =
 gaugeGroup $\mathbb{Z}_6$ SU20fRoot (gaugeGroup $\mathbb{Z}_3$ RootTo $\mathbb{Z}_6$ Root α) := rfl

@[simp]
lemma gaugeGroup $\mathbb{Z}_3$ Hom_toU1 (α : rootsOfUnity 3 $\mathbb{C}$) :
 GaugeGroupI.toU1 (gaugeGroup $\mathbb{Z}_3$ Hom α) =
 gaugeGroup $\mathbb{Z}_6$ UnitaryOfRoot (gaugeGroup $\mathbb{Z}_3$ RootTo $\mathbb{Z}_6$ Root α) := rfl

noncomputable def gaugeGroup $\mathbb{Z}_3$ SubGroup : Subgroup GaugeGroupI :=
 gaugeGroup $\mathbb{Z}_3$ Hom.range

lemma gaugeGroup $\mathbb{Z}_3$ OfRoot_mem (α : rootsOfUnity 3 $\mathbb{C}$) :
 gaugeGroup $\mathbb{Z}_3$ OfRoot α $\in$ gaugeGroup $\mathbb{Z}_3$ SubGroup :=
 $\langle \alpha, \text{rfl} \rangle$

lemma mem_gaugeGroup $\mathbb{Z}_3$ SubGroup_iff (g : GaugeGroupI) :
 $g \in$ gaugeGroup $\mathbb{Z}_3$ SubGroup $\leftrightarrow \exists \alpha$: rootsOfUnity 3 $\mathbb{C}$, gaugeGroup $\mathbb{Z}_3$ OfRoot α
 simp [gaugeGroup $\mathbb{Z}_3$ SubGroup]

```



```

lemma gaugeGroup $\mathbb{Z}_3$ SubGroup_le_gaugeGroup $\mathbb{Z}_6$ SubGroup :
 gaugeGroup $\mathbb{Z}_3$ SubGroup $\leq$ gaugeGroup $\mathbb{Z}_6$ SubGroup := by
 intro g hg
 rw [mem_gaugeGroup $\mathbb{Z}_3$ SubGroup_iff] at hg
 rcases hg with ⟨ α , rfl⟩
 exact gaugeGroup $\mathbb{Z}_6$ OfRoot_mem (gaugeGroup $\mathbb{Z}_3$ RootTo $\mathbb{Z}_6$ Root α)

lemma gaugeGroup $\mathbb{Z}_3$ SubGroup_le_center :
 gaugeGroup $\mathbb{Z}_3$ SubGroup $\leq$ Subgroup.center GaugeGroupI := by
 intro g hg
 rw [mem_gaugeGroup $\mathbb{Z}_3$ SubGroup_iff] at hg
 rcases hg with ⟨ α , rfl⟩
 exact gaugeGroup $\mathbb{Z}_3$ OfRoot_mem_center α

instance gaugeGroup $\mathbb{Z}_3$ SubGroup_normal : gaugeGroup $\mathbb{Z}_3$ SubGroup.Normal where
 conj_mem n hn g := by
 have hn_center : n $\in$ Subgroup.center GaugeGroupI := gaugeGroup $\mathbb{Z}_3$ SubGroup_le_center
 have hcomm : g * n = n * g := (Subgroup.mem_center_iff.mp hn_center) g
 have hconj : g * n * g-1 = n := by
 calc
 g * n * g-1 = n * g * g-1 := by rw [hcomm]
 _ = n := by simp [mul_assoc]
 simp [hconj] using hn
def GaugeGroup $\mathbb{Z}_3$: Type :=
 GaugeGroupI $\sqcap$ gaugeGroup $\mathbb{Z}_3$ SubGroup

noncomputable instance : Group GaugeGroup $\mathbb{Z}_3$:=
 inferInstanceAs (Group (GaugeGroupI $\sqcap$ gaugeGroup $\mathbb{Z}_3$ SubGroup))

namespace GaugeGroup $\mathbb{Z}_3$

/-- The quotient map from `GaugeGroupI` to `GaugeGroup $\mathbb{Z}_3$ `. -
/
noncomputable def mk : GaugeGroupI $\rightarrow^*$ GaugeGroup $\mathbb{Z}_3$:=
 QuotientGroup.mk' gaugeGroup $\mathbb{Z}_3$ SubGroup

@[simp]
lemma mk_gaugeGroup $\mathbb{Z}_3$ OfRoot (α : rootsOfUnity 3 $\mathbb{C}$) :
 mk (gaugeGroup $\mathbb{Z}_3$ OfRoot α) = 1 := by
 change ((gaugeGroup $\mathbb{Z}_3$ OfRoot α : GaugeGroupI) : GaugeGroupI $\sqcap$ gaugeGroup $\mathbb{Z}_3$ SubGroup)
 rw [QuotientGroup.eq_one_iff]
 exact gaugeGroup $\mathbb{Z}_3$ OfRoot_mem α

end GaugeGroup $\mathbb{Z}_3$

/-!

```

```
Gauge groups from quotient choices
```

```

-/
def GaugeGroup : GaugeGroupQuot → Type
| . $\mathbb{Z}_6$ => GaugeGroup $\mathbb{Z}_6$
| . $\mathbb{Z}_2$ => GaugeGroup $\mathbb{Z}_2$
| . $\mathbb{Z}_3$ => GaugeGroup $\mathbb{Z}_3$
| .I => GaugeGroupI

noncomputable instance (q : GaugeGroupQuot) : Group (GaugeGroup q) := by
 cases q <|> dsimp [GaugeGroup] <|> infer_instance

namespace GaugeGroupQuot

/-- The central subgroup of `GaugeGroupI` quotiented by a gauge-
group quotient choice. -/
noncomputable def subgroup : GaugeGroupQuot → Subgroup GaugeGroupI
| . $\mathbb{Z}_6$ => gaugeGroup $\mathbb{Z}_6$ SubGroup
| . $\mathbb{Z}_2$ => gaugeGroup $\mathbb{Z}_2$ SubGroup
| . $\mathbb{Z}_3$ => gaugeGroup $\mathbb{Z}_3$ SubGroup
| .I => $\perp$

/-- The subgroup attached to a gauge-group quotient choice lies in the center
/
lemma subgroup_le_center (q : GaugeGroupQuot) :
 subgroup q ≤ Subgroup.center GaugeGroupI := by
 cases q
 · exact gaugeGroup $\mathbb{Z}_6$ SubGroup_le_center
 · exact gaugeGroup $\mathbb{Z}_2$ SubGroup_le_center
 · exact gaugeGroup $\mathbb{Z}_3$ SubGroup_le_center
 · intro g hg
 change g ∈ ($\perp$: Subgroup GaugeGroupI) at hg
 rw [Subgroup.mem_bot] at hg
 simp [hg]

/-- The subgroup attached to a gauge-group quotient choice is normal in `GaugeGroupI`
/
instance subgroup_normal (q : GaugeGroupQuot) : (subgroup q).Normal := by
 cases q
 · exact gaugeGroup $\mathbb{Z}_6$ SubGroup_normal
 · exact gaugeGroup $\mathbb{Z}_2$ SubGroup_normal
 · exact gaugeGroup $\mathbb{Z}_3$ SubGroup_normal
 · change ($\perp$: Subgroup GaugeGroupI).Normal
 infer_instance

```

```

/-- The quotient map from `GaugeGroupI` to the gauge group selected by a quotient choice.
/
noncomputable def quotientMap (q : GaugeGroupQuot) : GaugeGroupI →* GaugeGroup q :=
 match q with
 | .Z₆ => GaugeGroupZ₆.mk
 | .Z₂ => GaugeGroupZ₂.mk
 | .Z₃ => GaugeGroupZ₃.mk
 | .I => MonoidHom.id GaugeGroupI

@[simp]
lemma quotientMap_I_apply (g : GaugeGroupI) :
 quotientMap .I g = g := rfl

@[simp]
lemma quotientMap_Z₆_gaugeGroupZ₆OfRoot (α : rootsOfUnity 6 ℂ) :
 quotientMap .Z₆ (gaugeGroupZ₆OfRoot α) = 1 :=
 GaugeGroupZ₆.mk_gaugeGroupZ₆OfRoot α

@[simp]
lemma quotientMap_Z₂_gaugeGroupZ₂OfRoot (α : rootsOfUnity 2 ℂ) :
 quotientMap .Z₂ (gaugeGroupZ₂OfRoot α) = 1 :=
 GaugeGroupZ₂.mk_gaugeGroupZ₂OfRoot α

@[simp]
lemma quotientMap_Z₃_gaugeGroupZ₃OfRoot (α : rootsOfUnity 3 ℂ) :
 quotientMap .Z₃ (gaugeGroupZ₃OfRoot α) = 1 :=
 GaugeGroupZ₃.mk_gaugeGroupZ₃OfRoot α

/-- The kernel of the quotient map is the subgroup selected by the quotient choice.
/
lemma mem_subgroup_iff_quotientMap_eq_one (q : GaugeGroupQuot) (g : GaugeGroupI) :
 g ∈ subgroup q ↔ quotientMap q g = 1 := by
 cases q
 · change g ∈ gaugeGroupZ₆SubGroup ↔
 ((g : GaugeGroupI) : GaugeGroupI ⊆ gaugeGroupZ₆SubGroup) = 1
 exact (QuotientGroup.eq_one_iff g).symm
 · change g ∈ gaugeGroupZ₂SubGroup ↔
 ((g : GaugeGroupI) : GaugeGroupI ⊆ gaugeGroupZ₂SubGroup) = 1
 exact (QuotientGroup.eq_one_iff g).symm
 · change g ∈ gaugeGroupZ₃SubGroup ↔
 ((g : GaugeGroupI) : GaugeGroupI ⊆ gaugeGroupZ₃SubGroup) = 1
 exact (QuotientGroup.eq_one_iff g).symm
 · change g ∈ (1 : Subgroup GaugeGroupI) ↔ (MonoidHom.id GaugeGroupI) g = 1
 simp

/-- Two representatives have the same image under the selected quotient map exactly

```

```

quotient lies in the subgroup selected by the quotient choice. -
/
lemma quotientMap_eq_iff (q : GaugeGroupQuot) (g h : GaugeGroupI) :
 quotientMap q g = quotientMap q h $\leftrightarrow$ g / h $\in$ subgroup q := by
 cases q
 · change ((g : GaugeGroupI) : GaugeGroupI $\sqsubset$ gaugeGroup $\mathbb{Z}_6$ SubGroup) =
 ((h : GaugeGroupI) : GaugeGroupI $\sqsubset$ gaugeGroup $\mathbb{Z}_6$ SubGroup) $\leftrightarrow$ g / h $\in$ ga
 exact QuotientGroup.eq_iff_div_mem
 · change ((g : GaugeGroupI) : GaugeGroupI $\sqsubset$ gaugeGroup $\mathbb{Z}_2$ SubGroup) =
 ((h : GaugeGroupI) : GaugeGroupI $\sqsubset$ gaugeGroup $\mathbb{Z}_2$ SubGroup) $\leftrightarrow$ g / h $\in$ ga
 exact QuotientGroup.eq_iff_div_mem
 · change ((g : GaugeGroupI) : GaugeGroupI $\sqsubset$ gaugeGroup $\mathbb{Z}_3$ SubGroup) =
 ((h : GaugeGroupI) : GaugeGroupI $\sqsubset$ gaugeGroup $\mathbb{Z}_3$ SubGroup) $\leftrightarrow$ g / h $\in$ ga
 exact QuotientGroup.eq_iff_div_mem
 · change g = h $\leftrightarrow$ g / h $\in$ ($\perp$: Subgroup GaugeGroupI)
 rw [Subgroup.mem_bot, div_eq_one]

end GaugeGroupQuot
/-!

Gauge bundles and transformations

-/

```

### 1.115 Physlib/Particles/StandardModel/HiggsBoson/Basic.lean

```

public import Mathlib.Geometry.Manifold.VectorBundle.ContMDiffSection
- A.8. Gauge action removing phase from second component
- A.9. To real scalars
simp [ofReal, PiLp.norm_sq_eq_of_L2, Fin.sum_univ_two, Real.sq_sqrt ha]

instance : SMulCommClass $\mathbb{C}$ GaugeGroupI HiggsVec where
 smul_comm r g ϕ := by
 simp [gaugeGroupI_smul_eq, mulVec_smul]
 rw [smul_comm]

instance : SMulCommClass $\mathbb{R}$ GaugeGroupI HiggsVec where
 smul_comm r g ϕ := by
 simp [gaugeGroupI_smul_eq, mulVec_smul]
 rw [smul_comm]

rw [norm_eq_sqrt_re_inner ($\square := \mathbb{C}$), norm_eq_sqrt_re_inner ($\square := \mathbb{C}$), gauge
have h0' : ($\|\phi\|^2 : \mathbb{C}$) = ϕ 0 * (starRingEnd $\mathbb{C}$) (ϕ 0) + ϕ 1 * (starRingE

```

1.115.

PHYSLIB/PARTICLES/STANDARDMODEL/HIGGSBOSON/BASIC.LEAN 177

```

 rw [← ofReal_pow, ← @real_inner_self_eq_norm_sq]
 have h0' : ($\|\varphi\|^2 : \mathbb{C}$) = $\varphi_0 * (\text{starRingEnd } \mathbb{C}) (\varphi_0) + \varphi_1 * (\text{starRingEnd } \mathbb{C}) (\varphi_1)$
 rw [← ofReal_pow, ← @real_inner_self_eq_norm_sq]
 · rintro (g, rfl)
 simp [Unitary.mem_iff, ← Complex.exp_conj, ← Complex.exp_add, Complex.conj_ofNat]
 rw [show vecHead (vecTail φ .ofLp) = φ .ofLp 1 from rfl]
 nth_rewrite 1 [← Complex.norm_mul_exp_arg_mul_I (φ .ofLp 1)]
 rw [← Complex.exp_nat_mul, mul_left_comm, ← Complex.exp_add]
 simp [ofU1Subgroup_smul_eq_smul, vecHead]
 ext i
 fin_cases i <|> simp [ofU1Subgroup_smul_eq_smul]

```

/-!

### A.9 To real scalars

-/

/-- The underlying real values of the Higgs vector. -/

def toRealScalars : HiggsVec →<sub>1</sub>  $\mathbb{R}$  (Fin 4 →  $\mathbb{R}$ ) where

```

 toFun x := fun
 | 0 => (x 0).re
 | 1 => (x 0).im
 | 2 => (x 1).re
 | 3 => (x 1).im
 map_add' x y := by
 ext i
 fin_cases i <|> simp
 map_smul' a x := by
 lemma toRealScalars_smul_real (a : $\mathbb{R}$) (φ : HiggsVec) :
 toRealScalars (a • φ) = a • toRealScalars φ := by
 simp [toRealScalars]

```

```

 lemma ofReal_toRealScalars (a : $\mathbb{R}$) :
 toRealScalars (ofReal a) = !4[Real.sqrt a, 0, 0, 0] := by
 simp [ofReal, toRealScalars]
 funext i
 fin_cases i <|> simp

```

```

 lemma ofReal_toRealScalars_norm (φ : HiggsVec) :
 toRealScalars (ofReal ($\|\varphi\|^2$)) = !4[$\|\varphi\|$, 0, 0, 0] := by
 simp [ofReal, toRealScalars]
 funext i
 fin_cases i <|> simp

```

```

have h1 := ϕ .toHiggsVec_smooth
simp_all only [contMDiff_iff_contDiff]
exact h1
simp [inner_apply, _root_.inner_add_left]
simp [inner_apply, _root_.inner_add_right]

```

### 1.116 Physlib/Particles/StandardModel/HiggsBoson/EffectivePoten

```

/-
Copyright (c) 2026 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Physlib.Particles.StandardModel.HiggsBoson.Basic
public import Mathlib.RingTheory.MvPolynomial.Homogeneous
/-!
The effective potential of the Higgs field

We define a general effective potential for the Higgs field.
For this we define two properties of the potential: invariance under the ga
maximum mass dimension.

Given these, we prove that the potential can be expressed as a polynomial i
Higgs field.

-/

@[expose] public section

noncomputable section

namespace StandardModel

namespace HiggsField

open SpaceTime

/-- A general potential of the Higgs field. -/
abbrev EffectivePotential : Type := HiggsVec → $\mathbb{R}$

namespace EffectivePotential

```

1.116.

PHYSLIB/PARTICLES/STANDARDMODEL/HIGGSBOSON/EFFECTIVEPOTENTIAL.LEAN

/-!

## A. The invariance of the general potential under the gauge group

-/

-- The proposition that the general potential is invariant under  
the global action of the gauge group. -/

def IsInvariant (V : EffectivePotential) : Prop :=  
  $\forall$  (g : GaugeGroupI),  $\forall$  ( $\phi$  : HiggsVec),  $V$  (g •  $\phi$ ) = V  $\phi$

namespace IsInvariant

-- An invariant potential is equal on gauge orbits. -/

lemma eq\_on\_orbits { $\phi_1$   $\phi_2$  : HiggsVec} {V : EffectivePotential} (h : IsInvariant V)  
 (h $\phi$  :  $\phi_1 \in \text{MulAction.orbit GaugeGroupI } \phi_2$ ) :  
 V  $\phi_1$  = V  $\phi_2$  := by  
 obtain (g, hg) := h $\phi$   
 rw [← hg]  
 exact h g  $\phi_2$

-- An invariant potential is equal on Higgs vectors with identical norms. -  
/

lemma eq\_of\_norm\_eq { $\phi_1$   $\phi_2$  : HiggsVec} {V : EffectivePotential} (h : IsInvariant V)  
 (h $\phi$  :  $\|\phi_1\| = \|\phi_2\|$ ) :  
 V  $\phi_1$  = V  $\phi_2$  := h.eq\_on\_orbits <| (HiggsVec.mem\_orbit\_gaugeGroupI\_iff  $\phi_2$   $\phi_1$ ).mpr

lemma factors\_through\_norm {V : EffectivePotential} (h : IsInvariant V) :

$\exists$  (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ), V = f • norm := by  
 use fun a => V !<sub>2</sub>[a, 0]  
 ext  $\phi$   
 simp only [Function.comp\_apply]  
 apply h.eq\_of\_norm\_eq  
 conv\_rhs => rw [PiLp.norm\_eq\_of\_L2]  
 simp

end IsInvariant

/-!

## B. Maximum mass dimension

-/

-- The proposition that the potential `V` has a maximum mass dimension

```

less than or equal to `n` - also implying it is a polynomial. -
/
def HasMaxMassDimLE (V : EffectivePotential) (n : ℕ) : Prop :=
 ∃ p : MvPolynomial (Fin 4) ℝ, (∀ φ : HiggsVec, V φ = p.eval φ.toRealScalars)
 p.totalDegree ≤ n

/-- The polynomial associated to a potential `V` with a maximum mass dimension
less than or equal to `n`. -/
def polynomial (V : EffectivePotential) {n : ℕ} (h : HasMaxMassDimLE V n) :
 MvPolynomial (Fin 4) ℝ := Classical.choose h

lemma polynomial_totalDegree {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n) :
 (polynomial V h).totalDegree ≤ n := (Classical.choose_spec h).2

lemma apply_eq_polynomial {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n) :
 (φ : HiggsVec) : V φ = (polynomial V h).eval φ.toRealScalars := (Classical.choose_spec h).2

/-!

C. Terms of a given mass dimension

-/

/-- The part of a potential at a given mass-dimension. -/
def termOfMassDim (V : EffectivePotential) {n : ℕ} (h : HasMaxMassDimLE V n) :
 HiggsVec → ℝ := fun φ => ((polynomial V h).homogeneousComponent m).eval φ

lemma termOfMassDim_eq_zero_of_max_lt {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n) :
 {m : ℕ} (hm : n < m) (φ : HiggsVec) :
 termOfMassDim V h m φ = 0 := by
 simp only [termOfMassDim]
 rw [MvPolynomial.homogeneousComponent_eq_zero]
 simp only [map_zero]
 have h1 := polynomial_totalDegree h
 grind

lemma termOfMassDim_homogeneity {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n) :
 (φ : HiggsVec) (t : ℝ) : termOfMassDim V h m (t • φ) = t ^ m * termOfMassDim V h m φ := by
 rw [termOfMassDim, termOfMassDim, map_smul, MvPolynomial.eval_eq', MvPolynomial.eval_eq',
 Finset.mul_sum]
 refine Finset.sum_congr rfl fun d hd => ?_
 have hdeg : ∑ i, d i = m := by
 rw [MvPolynomial.support_homogeneousComponent, Finset.mem_filter] at hd
 rw [← Finsupp.degree_eq_sum]
 exact hd.2
 simp only [Pi.smul_apply, smul_eq_mul, mul_pow, Finset.prod_mul_distrib,
```



1.116.

PHYSLIB/PARTICLES/STANDARDMODEL/HIGGSBOSON/EFFECTIVEPOTENTIAL.LEAN

```

 Finset.prod_pow_eq_pow_sum, hdeg]
 ring

lemma apply_eq_sum_termOfMassDim {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDim V) (φ : HiggsVec) :
 V φ = ∑ m ∈ Finset.range (n + 1), termOfMassDim V h m φ := by
 rw [apply_eq_polynomial h, ← MvPolynomial.sum_homogeneousComponent (polynomial V h)]
 simp only [map_sum]
 change ∑ x ∈ Finset.range ((V.polynomial h).totalDegree + 1), termOfMassDim V h x
 symm
 refine Finset.eventually_constant_sum ?_ ?_
 · intro m hm
 simp [termOfMassDim]
 rw [MvPolynomial.homogeneousComponent_eq_zero _ _ (by grind)]
 simp
 · have h1 := polynomial_totalDegree h
 grind

lemma apply_smul_eq_sum_termOfMassDim {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDim V) (φ : HiggsVec) (t : ℝ) :
 V (t • φ) = ∑ m ∈ Finset.range (n + 1), t ^ m * termOfMassDim V h m φ := by
 rw [apply_eq_sum_termOfMassDim h]
 congr
 funext m
 exact termOfMassDim_homogeneity h m φ t

lemma termOfMassDim_isInvariant {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDim V) (m : ℕ) (hV : IsInvariant V) : IsInvariant (termOfMassDim V h m) := by
 intro g φ
 have hV (t : ℝ) := hV g (t • φ)
 have h1 (t : ℝ) : ∑ m ∈ Finset.range (n + 1), t ^ m * (termOfMassDim V h m (g • φ) - termOfMassDim V h m φ) = 0 := by
 simp [mul_sub, ← apply_smul_eq_sum_termOfMassDim]
 rw [smul_comm, hV, sub_eq_zero]
 by_cases hmn : m ≤ n
 · have hp : (∑ k ∈ Finset.range (n + 1), Polynomial.C (termOfMassDim V h k (g • φ) - termOfMassDim V h k φ) * Polynomial.X ^ k) = 0 := by
 apply Polynomial.funext
 intro x
 simp only [Polynomial.eval_finsetSum, Polynomial.eval_mul, Polynomial.eval_C, Polynomial.eval_pow, Polynomial.eval_X, Polynomial.eval_zero]
 rw [← h1 x]
 exact Finset.sum_congr rfl fun k _ => by ring
 have hcoeff := congrArg (fun p => p.coeff m) hp
 simp only [Polynomial.finsetSum_coeff, Polynomial.coeff_C_mul, Polynomial.coeff_0]

```

```

mul_ite, mul_one, mul_zero, Finset.sum_ite_eq, Finset.mem_range, Nat
if_true, Polynomial.coeff_zero] at hcoeff
exact sub_eq_zero.mp hcoeff
· rw [termOfMassDim_eq_zero_of_max_lt h (not_le.mp hmn),
termOfMassDim_eq_zero_of_max_lt h (not_le.mp hmn)]

lemma termOfMassDim_eq_mul_norm {V : EffectivePotential} {n : ℕ}
(h : HasMaxMassDimLE V n) (m : ℕ) (hV : IsInvariant V) (φ : HiggsVec) :
∃ c, termOfMassDim V h m φ = c * ||φ|| ^ m := by
use termOfMassDim V h m !₂[1, 0]
rw [(termOfMassDim_isInvariant h m hV).eq_of_norm_eq (φ₂ := ||φ|| • !₂[1, 0]
(by simp [PiLp.norm_eq_of_L2]), termOfMassDim_homogeneity h m !₂[1, 0]
ring

lemma termOfMassDim_zero_of_odd {V : EffectivePotential} {n : ℕ} (h : HasMa
(hV : IsInvariant V) (φ : HiggsVec) (hodd : Odd m) :
termOfMassDim V h m φ = 0 := by
have h1 : termOfMassDim V h m φ = termOfMassDim V h m ((-
1 : ℝ) • φ) := by
apply (termOfMassDim_isInvariant h m hV).eq_of_norm_eq
simp
rw [termOfMassDim_homogeneity h m φ (-1 : ℝ), hodd.neg_one_pow] at h1
simp only [neg_mul, one_mul] at h1
grind

/-!

D. Potential in terms of the norm of the Higgs field

-/

lemma apply_eq_sum_even_termOfMassDim {V : EffectivePotential} {n : ℕ} (h :
(hV : IsInvariant V) (φ : HiggsVec) :
V φ = ∑ m ∈ Finset.range (n / 2 + 1), termOfMassDim V h (2 * m) φ := by
rw [apply_eq_sum_termOfMassDim h, ← Finset.sum_filter_add_sum_filter_not
(Finset.range (n + 1)) Even]
have hodd : ∑ m ∈ (Finset.range (n + 1)).filter (fun m => ¬ Even m),
termOfMassDim V h m φ = 0 := by
apply Finset.sum_eq_zero
intro m hm
simp only [Finset.mem_filter] at hm
exact termOfMassDim_zero_of_odd h m hV φ (Nat.not_even_iff_odd.mp hm.2)
rw [hodd, add_zero]
have hinj : ∀ x ∈ Finset.range (n / 2 + 1), ∀ y ∈ Finset.range (n / 2 + 1)
2 * x = 2 * y → x = y := fun x _ y _ hxy => by omega
have hset : (Finset.range (n / 2 + 1)).image (fun k => 2 * k)

```

1.117.

**PHYSLIB/PARTICLES/STANDARDMODEL/HIGGSBOSON/POTENTIAL.lean**

```

 = (Finset.range (n + 1)).filter Even := by
 ext a
 simp only [Finset.mem_image, Finset.mem_range, Finset.mem_filter, Nat.even_iff]
 constructor
 · rintro ⟨k, hk, rfl⟩
 exact ⟨by omega, by omega⟩
 · rintro ⟨ha, hae⟩
 exact ⟨a / 2, by omega, by omega⟩
 rw [← hset, Finset.sum_image hinj]

lemma apply_eq_sum_even_termOfMassDim_fin {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V)
 (hV : IsInvariant V) (φ : HiggsVec) :
 V φ = ∑ m : Fin (n/2 + 1), termOfMassDim V h (2 * m) φ := by
 rw [apply_eq_sum_even_termOfMassDim h hV φ, Finset.sum_range]

/-- The potential is equal to the sum of norms to even powers. -
/
lemma apply_eq_sum_norm_pow {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V)
 (hV : IsInvariant V) (φ : HiggsVec) :
 ∃ c : Fin (n/2 + 1) → ℝ, V φ = ∑ m, c m • ||φ|| ^ (2 * m.1) := by
 use fun m' => Classical.choose (termOfMassDim_eq_mul_norm h (2 * m'.1) hV φ)
 rw [apply_eq_sum_even_termOfMassDim_fin h hV φ]
 congr 1
 ext m
 simp using Classical.choose_spec (termOfMassDim_eq_mul_norm h (2 * m.1) hV φ)

end EffectivePotential

end HiggsField

end StandardModel

end

```

## 1.117 Physlib/Particles/StandardModel/HiggsBoson/Potential.lean

```

public import Mathlib.RingTheory.MvPolynomial.Homogeneous
TODO "Define a CoeFun instance for the Higgs Potential (or similar), instead of relying on
`P.toFun`."

exact ((contMDiff_const (I' := (ℝ, ℝ)) (M' := ℝ)).smul φ.normSq_smooth).neg.add
 (((contMDiff_const (I' := (ℝ, ℝ)) (M' := ℝ)).smul φ.normSq_smooth).smul φ.normSq_smooth)
simp only [toFun, neg]
simp only [toFun]
have h2 : ||φ||_H^2 x * (P.□ * ||φ||_H^2 x + - P.μ2) = 0 := by linear_combination h1

```

```

rcases mul_eq_zero.mp h2 with h2 | h2
· exact Or.inl (by simp [normSq] using h2)
· exact Or.inr (by rw [eq_div_iff h]; linear_combination h2)
· simp only [toFun, hD]
 ring
· rw [hV]
 ring
rw [le_div_iff_of_neg (show (4:ℝ) * P.□ < 0 by linarith)]
nlinarith [h1]
refine Or.inl (lt_of_not_ge hμ2, ?_)
simp only [toFun, normSq, neg_mul]
nlinarith [mul_nonneg (sq_nonneg ‖φ x‖) (sq_nonneg ‖φ x‖), sq_nonneg ‖φ x‖
 h, lt_of_not_ge hμ2]

· rw [← hV]
rcases P.neg_□_toFun_neg h□ φ x with hr | hr
· exact Or.inl hr
· exact Or.inr (hr, P.neg_□_quadDiscrim_zero_bound h□ φ x)
 refine Or.inr (?, by linarith)
 rw [sub_nonpos]
 rcases h with h | h
 · exact Real.le_sqrt_of_sq_le (by nlinarith [h.2])
 · exact h.1.trans (Real.sqrt_nonneg _)
 · nlinarith [sq_nonneg P.μ2, h.2]
 · rw [← @neg_le_iff_add_nonneg', ← le_div_iff_of_neg']
 have hdd := (Real.mul_self_sqrt hd).symm
have c_le_of_attainable : ∀ v : ℝ, ((0 < P.μ2 ∧ v ≤ 0) ∨
 (P.μ2 ≤ 0 ∧ v ≤ - P.μ2 ^ 2 / (4 * P.□))) → c ≤ v := by
 intro v hv
 obtain (φ, x, rfl) := (P.neg_□_sol_exists_iff h1 v).mpr hv
 exact hc φ x
 · linarith [c_le_of_attainable (c - 1) (Or.inr (hμ, by linarith))]
 linarith [c_le_of_attainable (- P.μ2 ^ 2 / (4 * P.□) - 1) (Or.inr (hμ, by linarith))]
 · linarith [c_le_of_attainable (c - 1) (Or.inl (hμ, by linarith))]
 · rw [not_le] at hcz
 linarith [c_le_of_attainable 0 (Or.inl (hμ, by linarith))]
have hx' : ‖φ‖_H^2 x = 0 :=
 le_antisymm (h.trans_le (div_nonpos_of_nonpos_of_nonneg hμ2 h□.le)) (no
have attainable_nonneg : ∀ v : ℝ, ((P.μ2 < 0 ∧ 0 ≤ v) ∨
 (0 ≤ P.μ2 ∧ - P.μ2 ^ 2 / (4 * P.□) ≤ v)) → 0 ≤ v := by
 rintro v ((_, hv) | (h0, hv))
 · exact hv
 · simp [le_antisymm hμ2 h0] using hv
· have h1' : P.toFun φ x ≤ 0 := by simp using h 0 0
 have h1'' := attainable_nonneg _ ((h1 (P.toFun φ x)).mp (φ, x, rfl))
 exact fun φ' x' => attainable_nonneg _ ((h1 (P.toFun φ' x')).mp (φ', x'

```

**1.118 Physlib/Particles/StandardModel/Representations.lean**

```
public import Physlib.Meta.TODO.Basic
TODO "Define a structure capturing the fermionic content of the Standard Model, with
 expressed as left-handed Weyl fermions (`Fermion.LeftHandedWeyl`) and including all
 families. The structure should carry a `Module  $\mathbb{C}$ ` instance together with a representation of the
 Lorentz group and a representation of the global gauge group `GaugeGroupI` (built from
 and `fundamentalSU2`)."

```

**1.119 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/B3.lean**

```
simp only [cubeTriLin, TriLinearSymm.mk₃_toFun_apply_apply, cubeTriLinToFun]
erw [Fin.sum_univ_three]
simp only [MSSMACC_linearACCs] at hLin
have h0 := hLin ⟨0, by simp⟩
have h2 := hLin ⟨2, by simp⟩
simp only [accGrav, LinearMap.coe_mk, AddHom.coe_mk, accSU3] at h0 h2
erw [Fin.sum_univ_three] at h0 h2
simp only [Fin.isValue, toSMSpecies_apply, Nat.reduceMul, Hd_apply, Fin.reduceFinMk, Hu_apply] at h0 h2

```

**1.120 Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Basic.lean**

```
lemma sum_MSSMSpecies_numberCharges_eq_expand [AddCommMonoid M]
 (f : Fin MSSMSpecies.numberCharges → M) :
 ∑ i, f i = f ⟨0, by simp⟩ + f ⟨1, by simp⟩ + f ⟨2, by simp⟩ := by
 change ∑ (i : Fin 3), f i = f ⟨0, by simp⟩ + f ⟨1, by simp⟩ + f ⟨2, by simp⟩
 simp only [Fin.sum_univ_three]
 rfl

 simp only [ACCSysCharges.chargesAddCommMonoid_add, toSMSpecies_apply, reduceMul_add, Hd_apply, Fin.reduceFinMk, Hu_apply]
 simp only [HSMul.hSMul, SMul.smul, Fin.isValue, toSMSpecies_apply, reduceMul, Hd_apply, Fin.reduceFinMk, Hu_apply, eq_ratCast, Rat.cast_eq_id, id_eq]
 simp only [accGrav, toSMSpecies_apply, Fin.isValue, LinearMap.coe_mk, AddHom.coe_mk]
 simp only [ACCSysCharges.chargesAddCommMonoid_add, toSMSpecies_apply, reduceMul_add, Hd_apply, Fin.reduceFinMk, Hu_apply]
 simp only [HSMul.hSMul, SMul.smul, Fin.isValue, toSMSpecies_apply, reduceMul, Hd_apply, Fin.reduceFinMk, Hu_apply, eq_ratCast, Rat.cast_eq_id, id_eq]
 simp only [accSU2, toSMSpecies_apply, Fin.isValue, LinearMap.coe_mk, AddHom.coe_mk]

```

```

 simp only [ACCSysCharges.chargesAddCommMonoid_add, toSMSpecies_apply
 mul_add]
 simp only [HSMul.hSMul, SMul.smul, Fin.isValue, toSMSpecies_apply, redu
 Rat.cast_eq_id, id_eq]
 simp only [accSU3, toSMSpecies_apply, Fin.isValue, LinearMap.coe_mk, AddH
 simp only [ACCSysCharges.chargesAddCommMonoid_add, toSMSpecies_apply
 mul_add, Hd_apply, Fin.reduceFinMk, Hu_apply]
 simp only [HSMul.hSMul, SMul.smul, Fin.isValue, toSMSpecies_apply, redu
 Fin.reduceFinMk, Hu_apply, eq_ratCast, Rat.cast_eq_id, id_eq]
 simp only [accYY, toSMSpecies_apply, Fin.isValue, LinearMap.coe_mk, AddHo
 simp only [toSMSpecies_apply, Fin.isValue, neg_mul, one_mul, Hd_apply,
 Hu_apply]
 · simp only [reduceMul, Fin.isValue, sum_MSSMSpecies_numberCharges_eq_e
 simp only [cubeTriLinToFun, toSMSpecies_apply, Fin.isValue, Hd_apply, Fin
 · simp only [sum_MSSMSpecies_numberCharges_eq_expand]
 simp only [cubeTriLinToFun, toSMSpecies_apply, Fin.isValue, Hd_apply, Fin
 · simp only [sum_MSSMSpecies_numberCharges_eq_expand]
 simp only [HomogeneousQuadratic, MSSMACC_quadraticACCs] at hS
 exact hS ⟨0, by simp⟩
 | ⟨0, _⟩ => exact hg
 | ⟨1, _⟩ => exact hsu2
 | ⟨2, _⟩ => exact hsu3
 | ⟨3, _⟩ => exact hyy), by
 | ⟨0, _⟩ => exact hquad), hcube)
 | ⟨0, _⟩ => exact hquad)
 | ⟨0, _⟩ => exact hquad), hcube)
 simp only [Fin.isValue, toSMSpecies_apply, reduceMul, sum_MSSMSpecies_n
 Fin.zero_eta, Fin.mk_one, Hd_apply, Fin.reduceFinMk, smul_eq_mul, Hu_
 simp only [HSMul.hSMul, SMul.smul, Fin.isValue, toSMSpecies_apply]
 simp only [toSMSpecies_apply, Fin.isValue,
 simp only [reduceMul, Fin.isValue, sum_MSSMSpecies_numberCharges_eq_exp
 Fin.mk_one]
 simp only [Fin.isValue, Prod.mk_zero_zero, Prod.mk_one_one]
 simp only [toSMSpecies_apply, Fin.isValue, Hd_apply, Fin.reduceFinMk, H
 simp only [reduceMul, Fin.isValue, sum_MSSMSpecies_numberCharges_eq_exp
 Fin.mk_one]
 simp only [Fin.isValue, Prod.mk_zero_zero, Prod.mk_one_one]

```

### 1.121 Physlib/Particles/SuperSymmetry/MSSMnu/AnomalyCancellation

```

 simp only [cubeTriLin, TriLinearSymm.mk3_toFun_apply_apply, cubeTriLinToF
 erw [Fin.sum_univ_three, B3_val, Y3_val, B3AsCharge, Y3AsCharge]
 simp only [MSSMACC_linearACCs] at hLin
 have h1 := hLin ⟨1, by decide⟩

```

1.122.

PHYSLIB/PARTICLES/SUPERSYMMETRY/MSSMNU/ANOMALY CANCELLATION/ORTHOGY3B3/PLANEWITHY3B3.

```
have h2 := hLin (2, by decide)
have h3 := hLin (3, by decide)
simp only [accSU2, LinearMap.coe_mk, AddHom.coe_mk, accSU3, accYY] at h1 h2 h3
erw [Fin.sum_univ_three] at h1 h2 h3
simp only [Fin.isValue, toSMSpecies_apply, Nat.reduceMul, Hd_apply, Fin.reduceFinM
Hu_apply] at h1 h2 h3
```

### 1.122 Physlib/Particles/SuperSymmetry/MSSMNU/AnomalyCancellation/Ortho

```
simp only [dot.map_add₂, dot.map_smul₂, R.perpY₃, R.perpB₃,
show dot Y₃.val Y₃.val = 216 by with_unfolding_all rfl,
show dot B₃.val B₃.val = 108 by with_unfolding_all rfl,
show dot Y₃.val B₃.val = 108 by with_unfolding_all rfl,
show dot B₃.val Y₃.val = 108 by with_unfolding_all rfl,
mul_zero, add_zero] at h1 h2
have ha : a = a' := by linarith
have hb : b = b' := by linarith
exact ⟨ha, hb, smul_left_injective ℚ hR' (add_left_cancel h)⟩
```

### 1.123 Physlib/Particles/SuperSymmetry/MSSMNU/AnomalyCancellation/Y3.le

```
simp only [cubeTriLin, TriLinearSymm.mk₃_toFun_apply_apply, cubeTriLinToFun]
erw [Fin.sum_univ_three]
simp only [MSSMACC_linearACCs] at hLin
have h3 := hLin (3, by simp)
simp only [accYY, LinearMap.coe_mk, AddHom.coe_mk] at h3
erw [Fin.sum_univ_three] at h3
simp only [Fin.isValue, toSMSpecies_apply, Nat.reduceMul, Hd_apply, Fin.reduceFinM
Hu_apply] at h3
```

### 1.124 Physlib/Particles/SuperSymmetry/N1/Basic.lean

```
/-
Copyright (c) 2026 Andrea Pari. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Andrea Pari
-/
module
```

```
public import Mathlib.Data.Complex.Basic
public import Physlib.Relativity.Tensors.Conjugation.Basic
```

```
/-!
```

```
SUSY N=1 chiral sector: index, configuration, and conjugation data
```

```
i. Overview
```

This file fixes the data that indexes the scalars of the N=1 chiral sector, makes their contractions type-safe, and equips them with conjugation.

A single finite type ``ι`` indexes the chiral scalars (written ``ChiralIndex`` in signatures) – the only index type. Variance (upper versus lower) and hol (a scalar versus its complex conjugate) are not separate index types but the axes of a four-element type ``ChiralColor``, the product of ``chiral``/``anti`` w ``up``/``down``. Each axis is realized as a genuine carrier distinction, not a variance as a module versus its dual (``ι → ℂ`` versus ``Module.Dual ℂ (ι → ℂ)``) holomorphy as a module versus its complex conjugate (``ConjModule``, where ``ι`` as ``-ι``).

The dual-colour involution ``τ`` flips variance and preserves holomorphy. Two indices may contract exactly when their colours are ``τ``-related, so a holom index pairs only with a holomorphic index of the opposite variance, and a conjugate ("barred") index only with a conjugate index of the opposite vari This is the discipline that makes the F-term contraction ``g^{IJ} D_I W \bar{D}_{\bar{J}}` type-check.

The physical field content is the configuration ``ChiralScalarConfiguration ι → ℂ``, carrying ``2 · Fintype.card ι`` real degrees of freedom. The anti-chiral scalars are the complex conjugates of this data, never an independent configuration.

The index data is packaged as a ``ConjTensorSpecies`` over ``ChiralColor``, each distinct carrier from above. Complex conjugation is the conjugate-linear identity

``conjEquiv : M ≈_{s ι} [starRingEnd ℂ] ConjModule M`` on each carrier (anti basis species' holomorphy flip (``conjEquiv``, hence ``conjT``) is built from it. Every colour carries the trivial representation over the trivial group ``Unit``, so no charge. Contracting a colour against its ``τ``-dual is the dot product of vectors, the Kronecker ``δ_{IJ}`` on basis labels: ``contr`` is that pairing ``V`` its cap in ``V (τ c) ⊗ V c``, and ``metric`` the cap ``∑_I b_I ⊗ b_I`` in ``V c ⊗ V c`` equips the chiral sector with the framework's generic tensor API (``Tensor``

Conjugation is intrinsic species data: a ``ConjTensorSpecies`` is a ``TensorSp`` extended with the conjugate-colour involution ``ChiralColor.bar`` and its coh then supplies the map ``conjT`` (conjugate the components and flip each index and its laws. ``bar`` is the holomorphy dual, distinct from and commuting wit



variance dual  $\tau$ ; it is not used in contraction.

Conjugation enters wherever reality does. It is what lets one state that the Kähler metric is Hermitian ( $\text{conjT } g$  equals  $g$  with its two indices swapped), that the anti-chiral sector is the complex conjugate of the chiral one ( $\bar{D}_{\bar{J}} \bar{W} = \text{conjT } (D_I W)$ ), and hence that the F-term  $g^{\{I\bar{J}\}} D_I W \bar{D}_{\bar{J}} \bar{W}$  is real. The species can express none of these alone.

## ## ii. Key results

- `SUSY.N1.ChiralScalarConfiguration` : the scalar configuration space  $\iota \rightarrow \mathbb{C}$ , where  $\iota$  is the finite type indexing the chiral scalars. This is the only field data in the sector.
- `SUSY.N1.ChiralColor` : the four colours `chiral`/`anti`  $\times$  `up`/`down`, with the dual-colour involution `ChiralColor.tau`.
- `SUSY.N1.chiralTensor` : the `ConjTensorSpecies` assembled from the above, whose  $\tau$ -discipline makes the F-term contraction type-safe and whose `bar` carries the chiral-antichiral conjugation in which reality and Hermiticity conditions are ph

## ## iii. Table of contents

- A. The chiral scalar configuration
- B. The chiral colours and the dual involution
- C. Carrier, representation, and basis
- D. The  $\delta$  structure on based finite modules
- E. The chiral-index tensor species
- F. Conjugation

## ## iv. References

-/

@[expose] public section  
open TensorProduct Module ComplexConjugate  
noncomputable section

namespace SUSY.N1

/-!

## ## A. The chiral scalar configuration

-/

-- The chiral scalar configuration: a complex value for each chiral label. This is the sector's only field data. Declared as an `abbrev` so that unification sees through it to `ChiralIndexingType  $\rightarrow$   $\mathbb{C}$`  and applies Mathlib's function-

```

space calculus
lemmas directly. -/
abbrev ChiralScalarConfiguration (ChiralIndexingType : Type*) := ChiralIndexingType

variable (ι : Type) [Fintype ι] [DecidableEq ι]

/-!
B. The chiral colours and the dual involution

-/

/-- The four colours carried by a chiral-sector index: holomorphy (`chiral`
`anti`, a scalar versus its complex conjugate) crossed with variance (`up`
`down`, contravariant versus covariant). Carrying both axes here lets the s
type `ι` label the scalars. -/
inductive ChiralColor | chiralUp | chiralDown | antiUp | antiDown
deriving DecidableEq

namespace ChiralColor

/-- The dual colour: flips variance and preserves holomorphy. Two indices m
exactly when their colours are `τ`-related, so `V^I` pairs only with `V_I`
holomorphy, opposite variance) and never with a conjugate index. -
/
def tau : ChiralColor → ChiralColor
| chiralUp => chiralDown
| chiralDown => chiralUp
| antiUp => antiDown
| antiDown => antiUp

/-- The conjugate colour: flips holomorphy (`chiral`↔`anti`) and preserves
conjugation sends an index to its conjugate carrier, so `bar` swaps `chiral`
Distinct from the variance dual `tau`; the two commute (`bar_tau`). -
/
def bar : ChiralColor → ChiralColor
| chiralUp => antiUp
| antiUp => chiralUp
| chiralDown => antiDown
| antiDown => chiralDown

@[simp] lemma bar_bar (c : ChiralColor) : bar (bar c) = c := by cases c <|>
@[simp] lemma bar_tau (c : ChiralColor) : bar (tau c) = tau (bar c) := by cases c <|>
end ChiralColor

```

```
variable {ι}
```

```
/-!
```

```
C. Carrier, representation, and basis
```

A ``TensorSpecies`` takes, for each colour ``c``, a carrier module, a group representation, and a basis. The carrier depends on *both* axes: variance gives the vector/dual distinction versus ``Module.Dual ℂ (ι → ℂ)`` and holomorphy the conjugate-module distinction (``ConjModule ...``, where ``i`` acts as ``-i``), so all four colours have distinct carriers. The representation is the trivial group ``Unit`` (no charge) for every colour; each basis is indexed by ``piBasis``, its dual ``piBasis.dualBasis``, and the ``Basis.conj`` of each. Variance (``τ``) sends a carrier to its dual; conjugation (``bar``) sends it to its conjugate module.

```
-/
```

`/--` The carrier module of each colour, distinct for all four: the holomorphic vector module ``Module.Dual ℂ (ι → ℂ)`` on the chiral side, and the conjugate module ``ConjModule.Dual ℂ (ι → ℂ)`` on the anti side. Variance is the vector/dual axis, holomorphy the conjugate-module axis; both are genuine carrier data, not labels tracked separately.

```
/
```

```
abbrev chiralModule : ChiralColor → Type
```

```
 | .chiralUp => ι → ℂ
 | .chiralDown => Module.Dual ℂ (ι → ℂ)
 | .antiUp => ConjModule (ι → ℂ)
 | .antiDown => ConjModule (Module.Dual ℂ (ι → ℂ))
```

```
instance instAddCommGroupChiralModule : ∀ c, AddCommGroup (chiralModule (ι := ι) c)
 | .chiralUp | .chiralDown | .antiUp | .antiDown => inferInstance
```

```
noncomputable instance instModuleChiralModule : ∀ c, Module ℂ (chiralModule (ι := ι) c)
 | .chiralUp | .chiralDown | .antiUp | .antiDown => inferInstance
```

`/--` The representation on each colour, taken trivial over the trivial group ``Unit``: scalars carry no charge in this sector. `-/`

```
def chiralRep : (c : ChiralColor) → Representation ℂ Unit (chiralModule (ι := ι) c)
 fun _ => Representation.trivial ℂ Unit _
```

`/--` The standard basis of the vector carrier ``ι → ℂ`` (the indicator functions); the ``chiralUp`` and the reference basis for the  $\delta$  pairing. `-/`

```
def piBasis : Basis ι ℂ (ι → ℂ) := Pi.basisFun ℂ ι
```

`/--` The basis of each colour's carrier, all indexed by ``ι``: ``piBasis`` on the holomorphic side, its dual ``piBasis.dualBasis`` on the holomorphic covectors, and the ``Basis.conj`` of each on the anti-holomorphic side (coordinates ``star``-ed). `-/`

```
noncomputable def chiralBasis : (c : ChiralColor) → Basis ι ℂ (chiralModule (ι := ι) c)
```

```

| .chiralUp => piBasis
| .chiralDown => piBasis.dualBasis
| .antiUp => Basis.conj piBasis
| .antiDown => Basis.conj piBasis.dualBasis

```

```

/-!

```

```

D. The δ structure on based finite modules

```

The contraction, unit, and metric are one  $\delta$  structure in basis coordinates. ``TensorSpecies`` field of that name – the  $\delta$  index-raising tensor  $\delta^{\{IJ\}}$  – Kähler metric  $g_{\{I\bar{J}\}}$ , which is built downstream on top of this sector. A colour with its variance dual  $\tau^c$ , whose carriers are *distinct* (a module conjugates) but share the index  $i$ , so the pairing is the dot product  $\text{across}$   $(M, b)$  and  $(N, b')$ ,  $(x, y) \mapsto \sum_I (b \ x)_I (b' \ y)_{\bar{I}}$ , with  $\text{cap } \sum_I b_I$  single-colour  $\text{cap } \delta\text{Cap}$  ( $b = b'$ ) is what ``metric c`` uses, since its colour; the two-module pairing  $\delta\text{Contr}_2 / \delta\text{Cap}_2$  is what ``contr`` and stays within one holomorphy and needs no conjugation; conjugation is carried ``conjT`` ( $\$F$ ).

```

-/

```

```

variable {M : Type*} [AddCommGroup M] [Module ℂ M]

```

`/--` The  $\delta$   $\text{cap } \sum_I b_I \otimes b_{\bar{I}}$ : the rank-2 tensor in  $M \otimes M$  with two upper components in the basis  $b$  are  $\delta^{ij}$ . It is an element of  $M \otimes M$  (the inverse metric "cap"), not a linear map, and serves as the ``metric`` field of the species (whose two colour). `-/`

```

def deltaCap (b : Basis ι ℂ M) : M ⊗[ℂ] M := ∑ I, b I ⊗[ℂ] b I

```

```

variable {N : Type*} [AddCommGroup N] [Module ℂ N]

```

`/--` The  $\delta$  pairing between two based modules sharing the index  $i$ : the dot product vectors  $(x, y) \mapsto \sum_I (b \ x)_I (b' \ y)_{\bar{I}}$ . Built from Mathlib's ``Basis.toDual``  $M \rightarrow \text{Module.Dual } M$ , sending  $b$  to its dual basis) precomposed on the second transport  $b' \approx b$ . `-/`

```

def deltaBil₂ (b : Basis ι ℂ M) (b' : Basis ι ℂ N) : M →[ι[ℂ]] N →[ι[ℂ]] ℂ :=
 b.toDual.compl₂ (b'.equiv b (Equiv.refl ι)).toLinearMap

```

```

/-/ `deltaBil₂ b b' x y = ∑ I (b x)_I (b' y)_{\bar{I}}. -/

```

```

lemma deltaBil₂_apply (b : Basis ι ℂ M) (b' : Basis ι ℂ N) (x : M) (y : N)
 deltaBil₂ b b' x y = ∑ I, b.equivFun x I * b'.equivFun y I := by
 rw [deltaBil₂, LinearMap.compl₂_apply, LinearEquiv.coe_coe]
 conv_lhs => rw [← b'.sum_equivFun y]
 simp_rw [map_sum, map_smul, Basis.equiv_apply, Equiv.refl_apply, Basis.to
 smul_eq_mul]

```

```

exact Finset.sum_congr rfl fun J _ => mul_comm _ _

/-- The two-module δ contraction $M \otimes N \rightarrow \mathbb{C}$. -/
def deltaContr2 (b : Basis 1 $\mathbb{C}$ M) (b' : Basis 1 $\mathbb{C}$ N) : M $\otimes$ [$\mathbb{C}$] N $\rightarrow$ 1 [$\mathbb{C}$] $\mathbb{C}$:=
 TensorProduct.lift (deltaBil2 b b')

/-- δ Contr2 b b' (x $\otimes$ y) = $\sum_I$ (b x)I (b' y)I. -
/
lemma deltaContr2_tmul (b : Basis 1 $\mathbb{C}$ M) (b' : Basis 1 $\mathbb{C}$ N) (x : M) (y : N) :
 deltaContr2 b b' (x $\otimes$ [$\mathbb{C}$] y) = $\sum I$, b.equivFun x I * b'.equivFun y I := by
 rw [deltaContr2, TensorProduct.lift.tmul, deltaBil2_apply]

/-- δ Contr2 b b' (x $\otimes$ b' J) = xJ: pairing with the second basis reads off a
/
lemma deltaContr2_tmul_basis (b : Basis 1 $\mathbb{C}$ M) (b' : Basis 1 $\mathbb{C}$ N) (x : M) (J : 1) :
 deltaContr2 b b' (x $\otimes$ [$\mathbb{C}$] b' J) = b.equivFun x J := by
 simp [deltaContr2_tmul, Basis.equivFun_self]

/-- δ Contr2 b b' (b I $\otimes$ b' J) = $\delta_{\{IJ\}}$: the two bases are δ -
dual. -/
lemma deltaContr2_basis_basis (b : Basis 1 $\mathbb{C}$ M) (b' : Basis 1 $\mathbb{C}$ N) (I J : 1) :
 deltaContr2 b b' (b I $\otimes$ [$\mathbb{C}$] b' J) = if I = J then 1 else 0 := by
 rw [deltaContr2_tmul_basis, Basis.equivFun_self]

/-- δ Contr2 b b' (x $\otimes$ y) = deltaContr2 b' b (y $\otimes$ x): swapping slots swaps the
/
lemma deltaContr2_comm (b : Basis 1 $\mathbb{C}$ M) (b' : Basis 1 $\mathbb{C}$ N) (x : M) (y : N) :
 deltaContr2 b b' (x $\otimes$ [$\mathbb{C}$] y) = deltaContr2 b' b (y $\otimes$ [$\mathbb{C}$] x) := by
 rw [deltaContr2_tmul, deltaContr2_tmul]
 exact Finset.sum_congr rfl fun I _ => mul_comm _ _

/-- The two-module δ cap $\sum_I b_I \otimes b'_I \in M \otimes N$. -/
def deltaCap2 (b : Basis 1 $\mathbb{C}$ M) (b' : Basis 1 $\mathbb{C}$ N) : M $\otimes$ [$\mathbb{C}$] N := $\sum I$, b I $\otimes$ [$\mathbb{C}$] b' I

omit [DecidableEq 1] in
/-- δ Cap2 b b' = deltaCap2 b' b: swapping the two factors swaps the two
/
lemma deltaCap2_comm (b : Basis 1 $\mathbb{C}$ M) (b' : Basis 1 $\mathbb{C}$ N) :
 TensorProduct.comm $\mathbb{C}$ M N (deltaCap2 b b') = deltaCap2 b' b := by
 rw [deltaCap2, map_sum]
 exact Finset.sum_congr rfl fun I _ => by rw [TensorProduct.comm_tmul]

omit [DecidableEq 1] in
/-- The δ unit2 law (two-module, δ toSpanSingleton form): δ Cap2 b b' is the
 δ Cap2 b b'. -/
lemma deltaUnit2_symm (b : Basis 1 $\mathbb{C}$ M) (b' : Basis 1 $\mathbb{C}$ N) :

```

```

 LinearMap.toSpanSingleton C _ (deltaCap2 b' b) 1 =
 LinearMap.lTensor N (LinearEquiv.refl C M).toLinearMap
 (TensorProduct.comm C M N (LinearMap.toSpanSingleton C _ (deltaCap2
simp only [LinearMap.toSpanSingleton_apply_one]
rw [deltaCap2_comm]
simp only [LinearEquiv.refl_toLinearMap, LinearMap.lTensor_id, LinearMap.

/-- The snake identity (two-module, `contr_unit` law): contracting `x ∈ M`
leg of
`deltaCap2 b' b ∈ N ⊗ M` returns `x`. -/
lemma deltaContr2_unit (b : Basis 1 C M) (b' : Basis 1 C N) (x : M) :
 (TensorProduct.lid C M) ((deltaContr2 b b').rTensor M
 ((TensorProduct.assoc C M N M).symm
 (x ⊗t[C] LinearMap.toSpanSingleton C _ (deltaCap2 b' b) 1))) = x :=
 rw [LinearMap.toSpanSingleton_apply_one, deltaCap2, TensorProduct.tmul_sum,
 map_sum]
conv_rhs => rw [← b.sum_equivFun x]
refine Finset.sum_congr rfl fun I _ => ?_
rw [TensorProduct.assoc_symm_tmul, LinearMap.rTensor_tmul, TensorProduct.
 deltaContr2_tmul_basis]

/-- The `contr_metric` law (two-module): contracting the inner `M`/`N` legs
`deltaCap b ⊗ deltaCap b'` yields `deltaCap2 b' b`. -/
lemma deltaContr2_metric (b : Basis 1 C M) (b' : Basis 1 C N) :
 (TensorProduct.comm C M N ((TensorProduct.lid C N).lTensor M
 (((deltaContr2 b b').rTensor N).lTensor M
 (((TensorProduct.assoc C M N N).symm.toLinearMap.lTensor M)
 ((TensorProduct.assoc C M M (N ⊗t[C] N))
 (LinearMap.toSpanSingleton C _ (deltaCap b) 1 ⊗t[C]
 LinearMap.toSpanSingleton C _ (deltaCap b') 1)))))) =
 LinearMap.toSpanSingleton C _ (deltaCap2 b' b) 1 := by
 rw [LinearMap.toSpanSingleton_apply_one, LinearMap.toSpanSingleton_apply_
 LinearMap.toSpanSingleton_apply_one]
conv_lhs => rw [deltaCap, deltaCap, TensorProduct.sum_tmul]
conv_rhs => rw [deltaCap2]
simp only [TensorProduct.tmul_sum, map_sum]
simp [TensorProduct.assoc_tmul, LinearEquiv.lTensor_tmul, LinearMap.lTens
 TensorProduct.assoc_symm_tmul, LinearMap.rTensor_tmul, TensorProduct.li
 TensorProduct.comm_tmul, deltaContr2_basis_basis, ite_smul]
simp [TensorProduct.ite_tmul, Finset.sum_ite_eq]

/-!
E. The chiral-index tensor species

-/

```

```

/-- The chiral-index tensor species, bundled with its conjugation. Its four colours
`chiral`/`anti` × `up`/`down` carry the four distinct carriers of $\mathcal{S}\mathcal{C}$. `contr c` is the
module δ
pairing of a colour against its variance dual τc ($\forall c \otimes V(\tau c) \rightarrow \mathbb{C}$); `unit c`
across those two carriers; `metric c` is the single-colour δ cap $\sum_I b_I \otimes b_I$. Each
`TensorSpecies` coherence law reduces, by case analysis on the colour, to the corresponding
two-module δ lemma of $\mathcal{S}\mathcal{D}$. The conjugation flips holomorphy (ChiralColor.bar) while
variance; every basis is indexed by i , so barIdx_eq is rfl , and conj_contrComm
 $\text{star } \delta = \delta$. Instantiating `ConjTensorSpecies` this way gives the chiral sector both
the framework's generic tensor API and its conjugation API (conjT and its laws) on
/
def chiralTensor : ConjTensorSpecies $\mathbb{C}$ ChiralColor Unit (chiralModule ($i := i$)) (fun
 (chiralRep ($i := i$)) (chiralBasis ($i := i$)) where
 $\tau := \text{ChiralColor.tau}$
 $\tau_involution c := \text{by cases } c <=> \text{rfl}$
 -- `contr` pairs a colour with its variance dual τc (distinct carriers, e.g. i
 -- against its dual); `unit` is the δ cap across those two carriers; `metric` the
 -- colour with itself.
 contr c := { deltaContr2 (chiralBasis c) (chiralBasis (ChiralColor.tau c)) with
 isIntertwining' g := by ext v; simp [Representation.tprod_apply, chiralRep] }
 unit c := { LinearMap.toSpanSingleton $\mathbb{C}$ _
 (deltaCap2 (chiralBasis (ChiralColor.tau c)) (chiralBasis c)) with
 isIntertwining' g := by ext; simp [Representation.tprod_apply, chiralRep, deltaCap] }
 metric c := { LinearMap.toSpanSingleton $\mathbb{C}$ _ (deltaCap (chiralBasis c)) with
 isIntertwining' g := by ext; simp [Representation.tprod_apply, chiralRep, deltaCap] }
 -- Each coherence law reduces, by case analysis on c , to the matching abstract tensor
module
 -- δ lemma.
 contr_tmul_symm c x y := by cases c <=> exact deltaContr2_comm _ _ _
 unit_symm c := by cases c <=> exact deltaUnit2_symm _ _
 contr_unit c x := by cases c <=> exact deltaContr2_unit _ _ x
 contr_metric c := by cases c <=> exact deltaContr2_metric _ _
 -- Conjugation data: `bar` flips holomorphy, the index set is shared (rfl), `star`
 bar := ChiralColor.bar
 bar_involution := ChiralColor.bar_bar
 bar_tau := ChiralColor.bar_tau
 barIdx_eq _ := rfl
 conj_contrComm := by
 intro d x1 x2
 -- The contraction at every colour is the real δ pairing of two i -
bases, so `star` fixes it.
 -- The `key` lemma evaluates both sides by `deltaContr2_basis_basis` (a syntactic
 -- `if x1 = x2 then 1 else 0`), so the heavy `Basis.conj`/`dualBasis` carriers are
 -- `whnf`'d.
 have key : $\forall \{M_1 M_1' M_2 M_2' : \text{Type}\} [\text{AddCommGroup } M_1] [\text{Module } \mathbb{C} M_1] [\text{AddCommGroup } M_2] [\text{Module } \mathbb{C} M_2] [\text{AddCommGroup } M_1'] [\text{Module } \mathbb{C} M_1'] [\text{AddCommGroup } M_2'] [\text{Module } \mathbb{C} M_2']$

```

```

 (B1 : Basis ι ℂ M1) (B1' : Basis ι ℂ M1') (B2 : Basis ι ℂ M2) (B2' : Basis ι ℂ M2')
 star (deltaContr2 B1 B1' (B1 x1 ⊗t[ℂ] B1' x2))
 = deltaContr2 B2 B2' (B2 x1 ⊗t[ℂ] B2' x2) := by
intro M1 M1' M2 M2' _ _ _ _ _ B1 B1' B2 B2'
rw [deltaContr2_basis_basis, deltaContr2_basis_basis]; split <;> simp
cases d <;> exact key _ _ _ _

```

```

/-!
F. Conjugation

```

Reality is a physical input the bare species cannot express: that the anti-chiral fields are the complex conjugates of the chiral ones, that the Kähler metric is Hermitian, term potential is real. Each is a statement that some quantity equals its own complex conjugate. This section exposes that operationally, the scalar `W` and the holomorphic covector `D` components that it is honest complex conjugation.

Conjugation is bundled into `chiralTensor` itself (§E): as a `ConjTensorSpecies` beside `τ`, and the framework supplies the conjugation map `conjT` and its `conjT\_conjT`, `conjT\_contrT`, `conjT\_eq\_permT\_iff`) once, abstractly, against `ConjTensorSpecies`. The chiral sector's conjugation flips holomorphy (`ChiralTensor` preserving variance, and through `chiralTensor.conjT` the reality and Hermiticity are phrased. The basis index type `ι` is the same for every colour, so the identity is `rfl` and the component reindexing is the identity.

```

-/

```

```

section Conjugation

```

```

open TensorSpecies TensorSpecies.Tensor ConjTensorSpecies ChiralColor

```

```

/-!
The following normalize the output of `(chiralTensor (ι := ι)).conjT` back to canonical colour lists for scalar and anti-holomorphic covector tensors respectively.
-/

```

```

/-- Conjugation of a scalar tensor, normalized back to the scalar colour list.
/
def conjScalar (t : (chiralTensor (ι := ι)).Tensor ![]) :
 (chiralTensor (ι := ι)).Tensor ![] :=
 permT id ⟨Function.bijective_id, fun i => by fin_cases i⟩
 ((chiralTensor (ι := ι)).conjT t)

```



```

/-- Conjugation of a holomorphic covector, normalized to the anti-
holomorphic covector colour
list `[antiDown]`. -/
def conjChiralCovector
 (t : (chiralTensor (ι := ι)).Tensor ![chiralDown]) :
 (chiralTensor (ι := ι)).Tensor ![antiDown] :=
 permT ![0] (by decide, fun i => by fin_cases i; rfl)
 ((chiralTensor (ι := ι)).conjT t)

/-- For scalar tensors, `toField` of the normalized tensor conjugate is the complex
`toField`. -/
lemma toField_conjScalar (t : (chiralTensor (ι := ι)).Tensor ![]) :
 (conjScalar t).toField = star t.toField := by
 rw [conjScalar, toField_permT]
 rw [toField_eq_repr, toField_eq_repr]
 change componentMap (S := (chiralTensor (ι := ι)).toTensorSpecies)
 ((chiralTensor (ι := ι)).bar ∘ ![]) ((chiralTensor (ι := ι)).conjT t) (fun j =>
 star ((basis (S := (chiralTensor (ι := ι)).toTensorSpecies) ![]).repr t (fun j =>
 star (conjTensorSpecies.componentMap_conjT (S := chiralTensor (ι := ι))))
)
 rfl

/-- Component formula for the holomorphic covector conjugate: the `[I]` basis compo
`conjChiralCovector t` is the complex conjugate of the `[I]` component of `t`. -
/
lemma repr_conjChiralCovector
 (t : (chiralTensor (ι := ι)).Tensor ![chiralDown]) (I : ι) :
 (basis (S := (chiralTensor (ι := ι)).toTensorSpecies) ![antiDown]).repr
 (conjChiralCovector t) ![I] =
 star ((basis (S := (chiralTensor (ι := ι)).toTensorSpecies) ![chiralDown]).repr
 t) ![I]
 rw [conjChiralCovector, permT_basis_repr_symm_apply]
 change componentMap (S := (chiralTensor (ι := ι)).toTensorSpecies)
 ((chiralTensor (ι := ι)).bar ∘ ![chiralDown]) ((chiralTensor (ι := ι)).conjT t)
 rw [ConjTensorSpecies.componentMap_conjT (S := chiralTensor (ι := ι))]
 apply congrArg star
 apply congrArg (fun idx => componentMap (S := (chiralTensor (ι := ι)).toTensorSpecies)
 ![chiralDown] t idx)
 funext i
 fin_cases i
 rfl

/-- Conjugation of a holomorphic covector is additive. -/
@[simp]
lemma conjChiralCovector_add
 (t₁ t₂ : (chiralTensor (ι := ι)).Tensor ![chiralDown]) :
 conjChiralCovector (t₁ + t₂) = conjChiralCovector t₁ + conjChiralCovector t₂ :=
 rw [conjChiralCovector, (chiralTensor (ι := ι)).conjT_add]

```

```

simp [conjChiralCovector, map_add]

/-- Conjugation of a holomorphic covector is conjugate-linear: a scalar `r`
`star r`. -/
@[simp]
lemma conjChiralCovector_smul (r : ℂ)
 (t : (chiralTensor (ι := ι)).Tensor ![chiralDown]) :
 conjChiralCovector (r • t) = star r • conjChiralCovector t := by
 rw [conjChiralCovector, (chiralTensor (ι := ι)).conjT_smul]
 simp [conjChiralCovector]

end Conjugation

end SUSY.N1

end

end

```

### 1.125 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Allows

```

case Λ => exact ⟨a, b, by simp⟩
case K1 => exact ⟨b, - a - b, by simp⟩
case topYukawa => exact ⟨b, - a - b, by simp⟩
 refine ⟨b, -b - 2 • a, ?_⟩
 refine ⟨a, b, c, ?_⟩
 refine ⟨a, b, c, ?_⟩
 have hf2 : f4 = - f2 := eq_neg_of_add_eq_zero_left f1_add_f2_eq_zero
def AllowsTermQ5 (x : ChargeSpectrum []) (q5 : []) (T : PotentialTerm) : Prop
instance (x : ChargeSpectrum []) (q5 : []) (T : PotentialTerm) :
 refine .inl ⟨a2, a3, ⟨h2, h3⟩, ?_⟩
 rw [← hsum]
 abel
 · subst h2
 exact .inl ⟨a1, a3, ⟨.inr h1, h3⟩, hsum⟩
 · exact .inr ⟨a1, a2, a3, ⟨h1, h2, h3⟩, hsum⟩
def AllowsTermQ10 (x : ChargeSpectrum []) (q10 : []) (T : PotentialTerm) : Prop
instance (x : ChargeSpectrum []) (q10 : []) (T : PotentialTerm) :

```

### 1.126 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Basic.

```

x = y ↔ x.qHd = y.qHd ∧ x.qHu = y.qHu ∧ x.Q5 = y.Q5 ∧ x.Q10 = y.Q10 :=
⟨fun h => ⟨congrArg qHd h, congrArg qHu h, congrArg Q5 h, congrArg Q10 h⟩

```

1.126.

PHYSLIB/PARTICLES/SUPERSYMMETRY/SU5/CHARGESPECTRUM/BASIC9.lean

```

 fun (h1, h2, h3, h4) => eq_of_parts h1 h2 h3 h4)
 cases x <=> cases y <=> simp [Option.toFinset]
lemma subset_antisymm {x y : ChargeSpectrum []} (hxy : x ≤ y) (hyx : y ≤ x) : x = y :
 eq_of_parts
 (Option.toFinset_inj.mpr (Finset.Subset.antisymm hxy.1 hyx.1))
 (Option.toFinset_inj.mpr (Finset.Subset.antisymm hxy.2.1 hyx.2.1))
 (Finset.Subset.antisymm hxy.2.2.1 hyx.2.2.1)
 (Finset.Subset.antisymm hxy.2.2.2 hyx.2.2.2)
 refine (fun h => subset_antisymm h (empty_subset x), ?_)
 rintro rfl
 simp
 simp only [card]
 simp only [card] at hcard
 have c1 := Finset.card_le_card h.1
 have c2 := Finset.card_le_card h.2.1
 have c3 := Finset.card_le_card h.2.2.1
 have c4 := Finset.card_le_card h.2.2.2
 refine eq_of_parts (Option.toFinset_inj.mpr ?_) (Option.toFinset_inj.mpr ?_) ?_ ?_
 · exact Finset.eq_of_subset_of_card_le h.1 (by omega)
 · exact Finset.eq_of_subset_of_card_le h.2.1 (by omega)
 · exact Finset.eq_of_subset_of_card_le h.2.2.1 (by omega)
 · exact Finset.eq_of_subset_of_card_le h.2.2.2 (by omega)
 y ∈ x.powerset ↔ y.toFinset ≤ x.toFinset := by
cases x <=> cases y <=> simp [Option.powerset]
simp [mem_powerset_iff, subset_def]
 exact mem_powerset_iff_subset.mp (h (self_mem_powerset x))
 · intro h z hz
 exact mem_powerset_iff_subset.mpr (subset_trans (mem_powerset_iff_subset.mp hz)
 ∃ y ∈ S, powerset y n S = {y} := by
intro _
obtain ⟨y, hyS, hy⟩ := S.exists_min_image card (Finset.nonempty_iff_ne_empty.mpr h)
refine ⟨y, hyS, ?_⟩
ext z
simp only [Finset.mem_inter, mem_powerset_iff_subset, Finset.mem_singleton]
constructor
 · rintro ⟨hzy, hzS⟩
 exact eq_of_subset_card hzy (le_antisymm (card_mono hzy) (hy z hzS))
 · rintro rfl
 exact ⟨subset_refl z, hyS⟩
have hoption (a : Option []) (S : Finset []) :
 a ∈ ({none} : Finset (Option [])) ∪ S.map ⟨Option.some, Option.some_injective⟩
 a.toFinset ≤ S := by cases a <=> simp
simp only [ofFinset, Finset.mem_map_equiv, Equiv.symm_symm, toProd, Equiv.coe_fn_m
 Finset.product_eq_sprod, Finset.mem_product, hoption, Finset.mem_powerset]

```

**1.127 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Comple**

```

match x1 with
| none => simp using hy.1
| some a => simp_all
match x2 with
| none => simp using hy.2.1
| some a => simp_all
refine ⟨⟨y1, y2, if x3 = ∅ then {z3} else x3, if x4 = ∅ then {z4} else x4
· rw [mem_completions_iff]
 refine ⟨by simp_all, by simp_all, ?_, ?_⟩
 · split_ifs with h3 <=> simp_all
 · split_ifs with h4 <=> simp_all
· rw [Subset]
 dsimp [hasSubset]
 refine ⟨by simp_all, by simp_all, ?_, ?_⟩
 · split_ifs with h3 <=> simp_all [Finset.singleton_subset_iff]
 · split_ifs with h4 <=> simp_all [Finset.singleton_subset_iff]

```

**1.128 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Map.le**

```

have heq : ∀ {W : Type} [AddCommGroup W] [DecidableEq W] (y : ChargeSpectrum
 (S : PotentialTerm), (ofPotentialTerm y S).toFinset = (ofPotentialTerm
 intro W _ _ y S
 ext i
 simp only [Multiset.mem_toFinset]
rw [heq, heq]
rcases x with ⟨_ | qHd, _ | qHu, Q5, Q10⟩ <=> cases T <=>
 simp only [ofPotentialTerm', map, Option.map_eq_map, Option.map_some, 0
 Multiset.toFinset_map, Finset.val_toFinset, Function.comp_def, map_ad
 Multiset.toFinset_singleton, Finset.image_singleton, Finset.product_e
 ← Finset.prodMap_image_product, Finset.image_image, Prod.map_fst, Pro
simp

```

**1.129 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Minima**

```

simp [mem_ofFinset_iff] at hy
· obtain ⟨z3, hz3, hz3not⟩ :=
 Finset.exists_of_ssubset (ssubset_of_subset_of_ne hsubset.2.2.1 h3)
 refine ⟨⟨x1, x2, insert z3 x3, x4⟩, insert_Q5_mem_minimalSuperSet z3 (h
 rw [Subset]
 dsimp [hasSubset]
 simp_all [Finset.insert_subset_iff]

```

1.130.

PHYSLIB/PARTICLES/SUPERSYMMETRY/SU5/CHARGESPECTRUM/MINIMALLYALLOWSTERM/FINSETTERMS.L

```

· obtain {z4, hz4, hz4not} :=
 Finset.exists_of_ssubset (ssubset_of_subset_of_ne hsubset.2.2.2 h4)
 refine {⟨x1, x2, x3, insert z4 x4⟩, insert_Q10_mem_minimalSuperSet z4 (hy.2.2.2
 rw [Subset]
 dsimp [hasSubset]
 simp_all [Finset.insert_subset_iff]
 (by simp_all) (x2 := (x2, x3, x4)))
 (by simp_all) (x2 := (x3, x4)))

```

### 1.130 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimallyAllow

```

simp [minTopBottom] at h
obtain {qHd, qHu, q5, q10, _, rfl} := h
simp [allowsTerm_iff_subset_allowsTermForm, allowsTermForm, subset_def]
exact {¬qHu, by simp, q10, by simp}
obtain {qHd, qHu, q5, q10, _, rfl} := h
simp [allowsTerm_iff_subset_allowsTermForm, allowsTermForm, subset_def]
 simp [allowsTerm_iff_subset_allowsTermForm, allowsTermForm, subset_def] at hBott
 simp [allowsTerm_iff_subset_allowsTermForm, allowsTermForm, subset_def] at hTop
simp [allowsTerm_iff_subset_allowsTermForm, allowsTermForm, subset_def] at hTop hB
simp [mem_ofFinset_iff] at hx
 exact {h5.1, Finset.insert_subset h5.2 (hn ▶ h10)}
 · simp [allowsTerm_iff_subset_allowsTermForm, allowsTermForm, subset_def]
 exact {¬qHu, by simp, q10, by simp}
 · simp [allowsTerm_iff_subset_allowsTermForm, allowsTermForm, subset_def]

```

### 1.131 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimallyAllow

```

simp [toMultisetsOne, Multiset.card_eq_one]
rintro x rfl
· obtain {a, rfl} := Finset.card_eq_one.mp hacard
· exact {fun a ha => Multiset.mem_of_le h1 (Multiset.mem_toFinset.mp ha), hacard}
· obtain {a, rfl} := Finset.card_eq_one.mp hacard
· have := Multiset.mem_of_mem_erase hb
 simp_all [Finset.insert_subset_iff]
· exact {fun a ha => Multiset.mem_of_le h1 (Multiset.mem_toFinset.mp ha), hacard}
 refine {c, hsub.2, a, (Multiset.mem_erase_of_ne hac).mpr hsub.1, ?_}
 exact congrArg (a ::m ·) (Multiset.pair_comm c a)
 exact {b, hsub.1, a, (Multiset.mem_erase_of_ne hac).mpr hsub.2, rfl}
 refine {a, hsub.1, b, (Multiset.mem_erase_of_ne (Ne.symm hac)).mpr hsub.2,
 rw [← sub_eq_zero, ← hsum]
· obtain rfl | rfl : T = W1 v T = W2 := by tauto
 all_goals

```

```

refine {?_, h}
use {- a - b - c}, {a, b, c}
simp_all [allowsTermForm]
abel
refine {?_, h}
use {a, b, c}
simp_all [allowsTermForm]
abel
x.MinimallyAllowsTerm T ↔ x ∈ minimallyAllowsTermsOfFinset S5 S10 T :=
(fun h => mem_minimallyAllowsTermOfFinset_of_minimallyAllowsTerm x h hx,
 minimallyAllowsTerm_of_mem_minimallyAllowsTermOfFinset)

```

### 1.132 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/OfPote

```

rcases x with ⟨_ | qHd, _ | qHu, Q5, Q10⟩ <=> simp [ofPotentialTerm']
rcases x with ⟨_ | qHd, _ | qHu, Q5, Q10⟩ <=> simp [ofPotentialTerm']
rcases x with ⟨_ | qHd, _ | qHu, Q5, Q10⟩ <=> simp [ofPotentialTerm']
rcases x with ⟨_ | qHd, _ | qHu, Q5, Q10⟩ <=> simp [ofPotentialTerm']
rcases x with ⟨_ | qHd, _ | qHu, Q5, Q10⟩ <=> simp [ofPotentialTerm']
rcases x with ⟨_ | qHd, _ | qHu, Q5, Q10⟩ <=> simp [ofPotentialTerm']
rcases x with ⟨_ | qHd, _ | qHu, Q5, Q10⟩ <=> simp [ofPotentialTerm']
rcases x with ⟨_ | qHd, _ | qHu, Q5, Q10⟩ <=> simp [ofPotentialTerm']

```

### 1.133 Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/PhenoC

```

rcases h q10 hq10 x hx with h' | h' | h'
instance {S5 S10 : Finset []} {charges : Multiset (ChargeSpectrum [])} :

```

### 1.134 Physlib/QFT/AnomalyCancellation/Basic.lean

```

exact congrArg (fun X => X.val) h
 exact h
 exact hv

```

### 1.135 Physlib/QFT/PerturbationTheory/Field0pFreeAlgebra/Basic.1

```

have h1 := congrFun hg x
simp_all only [Function.comp_apply, FreeAlgebra.lift_ι_apply]
exact h1

```

1.136.

PHYSLIB/QFT/PERTURBATIONTHEORY/FIELDOPFREEALGEBRA/GRADING.lean

### 1.136 Physlib/QFT/PerturbationTheory/FieldOpFreeAlgebra/Grading.lean

```
simp only [DirectSum.of_eq_same]
grind
grind
```

### 1.137 Physlib/QFT/PerturbationTheory/FieldSpecification/CrAnSection.lean

```
refine And.intro (Subtype.ext ?_) (Subtype.ext ?_) <|> simp
```

### 1.138 Physlib/QFT/PerturbationTheory/FieldSpecification/NormalOrder.lean

```
simp only [List.orderedInsert.eq_2]
simp only [List.cons_append, List.orderedInsert.eq_2]
simp only [List.orderedInsert.eq_2]
 simp only [List.cons_append, List.orderedInsert.eq_2]
 simp only [List.orderedInsert.eq_2]
```

### 1.139 Physlib/QFT/PerturbationTheory/FieldSpecification/TimeOrder.lean

```
have h2 := orderedInsert_swap_eq_time h2 h1
simp_all
exact (List.append_left_inj _).mpr rfl
```

### 1.140 Physlib/QFT/PerturbationTheory/WickAlgebra/Basic.lean

```
have key {u w v : $\square$.FieldOpFreeAlgebra}
 (hv : v $\in$ TwoSidedIdeal.span $\square$.fieldOpIdealSet) :
 u * v * w $\in$ TwoSidedIdeal.span $\square$.fieldOpIdealSet :=
 TwoSidedIdeal.mul_mem_right _ _ (TwoSidedIdeal.mul_mem_left _ _ hv)
have hBy : $\uparrow$ (bosonicProjF y) $\in$ TwoSidedIdeal.span $\square$.fieldOpIdealSet := by
 rcases bosonicProjF_mem_fieldOpIdealSet_or_zero y hy with h | h
 · exact TwoSidedIdeal.mem_span_iff.mpr fun I hI => hI h
 · simp [h]
have hFy : $\uparrow$ (fermionicProjF y) $\in$ TwoSidedIdeal.span $\square$.fieldOpIdealSet := by
 rcases fermionicProjF_mem_fieldOpIdealSet_or_zero y hy with h | h
 · exact TwoSidedIdeal.mem_span_iff.mpr fun I hI => hI h
 · simp [h]
exact add_mem (add_mem (key hBy) (key hFy)) (add_mem (key hFy) (key hBy))
```

**1.141 Physlib/QFT/PerturbationTheory/WickAlgebra/Grading.lean**

```
simp only [DirectSum.of_eq_same]
grind
grind
grind
```

**1.142 Physlib/QFT/PerturbationTheory/WickAlgebra/SuperCommute.lean**

```
apply superCommute_create_create <=> rfl
apply superCommute_create_create <=> rfl
apply superCommute_create_create <=> rfl
apply superCommute_create_create <=> rfl
apply superCommute_annihilate_annihilate <=> rfl
apply superCommute_annihilate_annihilate <=> rfl
apply superCommute_annihilate_annihilate <=> rfl
apply superCommute_annihilate_annihilate <=> rfl
```

**1.143 Physlib/QFT/PerturbationTheory/WickContraction/ExtractEquation.lean**

```
import all Init.Data.Fin.Fold
```

**1.144 Physlib/QFT/PerturbationTheory/WickContraction/InsertAndCancel.lean**

```
simp_all only [Nat.succ_eq_add_one, erase, Finset.mem_filter, Finset.mem_insert,
exact ha'
```

**1.145 Physlib/QFT/PerturbationTheory/WickContraction/Join.lean**

```
simp_all only [Finset.card_pos]
exact h
```

**1.146 Physlib/QFT/QED/AnomalyCancellation/Basic.lean**

```
simp only [HSMul.hSMul, ACCSystemCharges.chargesModule_smul]
simp only [ACCSystemCharges.chargesAddCommMonoid_add]
simp only [TriLinearSymm.mk3_toFun_apply_apply]
simp only [PureU1_linearACCs] at hS
exact hS ⟨0, by simp⟩
```



**1.147 Physlib/QFT/QED/AnomalyCancellation/BasisLinear.lean**

```

 asCharges k {j, by simp}= 0 := by
simp [asCharges, Fin.ext_iff]
grind
| {0, _}>=>
 simp only [PureU1_linearACCs, accGrav,
 · simp only [asCharges, ↓reduceIte]
toFun S := (Finsupp.linearEquivFunOnFinite ℚ ℚ (Fin n)).symm (S.1 ∘ Fin.castSucc)
map_add' S T := by
 rw [← map_add]
 rfl
map_smul' a S := by
 rw [← map_smul]
 rfl
 simp only [Nat.succ_eq_add_one,
 Finsupp.linearEquivFunOnFinite_symm_apply, Function.comp_apply]
 simp only [HSMul.hSMul, SMul.smul, asLinSols_val]
 · simp only [asCharges, ↓reduceIte, mul_one]
 erw [asCharges_ne_castSucc hkj]
 simp only
 simp only [Nat.succ_eq_add_one, Finsupp.linearEquivFunOnFinite_symm_apply,
 Function.comp_apply]
 simp only [HSMul.hSMul, SMul.smul, asLinSols_val]
 · simp only [asCharges, ↓reduceIte, mul_one]
 erw [asCharges_ne_castSucc hkj]

```

**1.148 Physlib/QFT/QED/AnomalyCancellation/ConstAbs.lean**

```

simp only [ConstAbs, FamilyPermutations, PermGroup, permCharges,
simp only [succ_eq_add_one, accGrav, LinearMap.coe_mk, AddHom.coe_mk, Fin.val_succ]
simp only [succ_eq_add_one, mul_eq, cast_add, cast_mul, cast_ofNat,
erw [pureU1_linear A] at h0
simp only [Fin.val_cast, Fin.val_castAdd, mul_eq, Fin.val_castSucc]
simp only [mul_eq, Fin.val_succ, Fin.val_cast, Fin.val_natAdd]

```

**1.149 Physlib/QFT/QED/AnomalyCancellation/Even/BasisLinear.lean**

```

set_option backward.isDefEq.respectTransparency false in
 simp only [basisAsCharges, succ_eq_add_one, evenFst, evenSnd]
set_option backward.isDefEq.respectTransparency false in
 simp only [basisAsCharges, succ_eq_add_one]
set_option backward.isDefEq.respectTransparency false in

```

```

simp only [basisAsCharges, succ_eq_add_one, evenSnd, evenFst]
| {0, _} => exact basis_linearACC j)
simp only [Function.comp_apply, Pi.neg_apply]
set_option backward.isDefEq.respectTransparency false in
simp only [basis!AsCharges, succ_eq_add_one]
set_option backward.isDefEq.respectTransparency false in
simp only [basis!AsCharges, succ_eq_add_one]
set_option backward.isDefEq.respectTransparency false in
simp only [basis!AsCharges, succ_eq_add_one, evenShiftSnd, evenShiftFst]
set_option backward.isDefEq.respectTransparency false in
simp only [basis!AsCharges, succ_eq_add_one]
set_option backward.isDefEq.respectTransparency false in
simp only [basis!AsCharges, succ_eq_add_one]

| {0, _} => exact basis!_linearACC j)
simp only [succ_eq_add_one, accCubeTriLinSymm,
erw [sum_evenShift, basis!_on_evenShiftZero, basis!_on_evenShiftLast]
simp only [succ_eq_add_one, accCubeTriLinSymm,
erw [sum_even]

```

### 1.150 Physlib/QFT/QED/AnomalyCancellation/LineInPlaneCond.lean

```

simp only [FamilyPermutations_anomalyFreeLinear_apply]
rcases eq_or_ne i j with hij | hij
· rw [hij]
erw [Fin.sum_univ_four] at hcube
rw [sq_eq_sq_iff_eq_or_eq_neg, not_or] at hn
simp only [LineInPlaneProp, hn.1, hn.2, false_or] at l012 l013
-- `l012`, `l013` force `S.val 2 = S.val 3 = -(S.val 0 + S.val 1) / 2`; s
-- cube constraint yields `3 (S.val 0 - S.val 1)^2 (S.val 0 + S.val 1) = 0
have diff_sq_mul_sum_eq_zero : (S.val (0 : Fin 4) - S.val (1 : Fin 4)) ^
(S.val (0 : Fin 4) + S.val (1 : Fin 4)) = 0 := by
linear_combination 4 / 3 * hcube -
(2 * S.val (2 : Fin 4) ^ 2 - S.val (2 : Fin 4) * (S.val (0 : Fin 4) +
(S.val (0 : Fin 4) + S.val (1 : Fin 4)) ^ 2 / 2) / 3 * l012 -
(2 * S.val (3 : Fin 4) ^ 2 - S.val (3 : Fin 4) * (S.val (0 : Fin 4) +
(S.val (0 : Fin 4) + S.val (1 : Fin 4)) ^ 2 / 2) / 3 * l013
rcases mul_eq_zero.mp diff_sq_mul_sum_eq_zero with h | h
· exact hn.1 (sub_eq_zero.mp (sq_eq_zero_iff.mp h))
· exact hn.2 (by linarith)
exact linesInPlane_four S' (lineInPlaneCond_perm hS M)
rcases eq_or_ne i j with hij | hij
· rw [hij]

```

**1.151 Physlib/QFT/QED/AnomalyCancellation/LowDim/One.lean**

```
match i with
| ⟨0, _⟩ => exact hLin
```

**1.152 Physlib/QFT/QED/AnomalyCancellation/Odd/BasisLinear.lean**

```
set_option backward.isDefEq.respectTransparency false in
 simp only [basisAsCharges]
set_option backward.isDefEq.respectTransparency false in
 simp only [basisAsCharges]
set_option backward.isDefEq.respectTransparency false in
 simp only [basisAsCharges, oddSnd, oddFst]
set_option backward.isDefEq.respectTransparency false in
 simp only [basisAsCharges]
 | ⟨0, _⟩ => exact basis_linearACC j)
set_option backward.isDefEq.respectTransparency false in
 simp only [basis!AsCharges]
set_option backward.isDefEq.respectTransparency false in
 simp only [basis!AsCharges]
set_option backward.isDefEq.respectTransparency false in
 simp only [basis!AsCharges, oddShiftSnd, oddShiftFst]
set_option backward.isDefEq.respectTransparency false in
 simp only [basis!AsCharges]
 | ⟨0, _⟩ => exact basis!_linearACC j)
simp only [accCubeTriLinSymm, TriLinearSymm.mk₃_toFun_apply_apply]
erw [sum_oddShift, basis!_on_oddShiftZero]
```

**1.153 Physlib/QFT/QED/AnomalyCancellation/Permutations.lean**

```
| ⟨0, _⟩ => exact accGrav_invariant
```

**1.154 Physlib/QFT/QED/AnomalyCancellation/Sorts.lean**

```
simp only [Sorted, sort, FamilyPermutations, PermGroup, permCharges]
simp only [Equiv.invFun_as_coe, Equiv.apply_symm_apply] at hi
```

**1.155 Physlib/QuantumMechanics/DDimensions/Basic.lean**

```
SchwartzSubmodule Q.d ≤ Q.potentialOperator.domain :=
```

**1.156 Physlib/QuantumMechanics/DDimensions/Hydrogen/LaplaceRung**

```
attribute [local instance 100] LieRing.ofAssociativeRing
```

```

 ← comp_finsetSum, ← finsetSum_comp, sum_smul, smul_add, smul_sub, s
simp [← comp_finsetSum, ← finsetSum_comp, dotProduct, mul_def]
simp only [one_smul, one_div, sub_apply, add_apply, smul_apply, comp_appl
 positionCLM_apply, real_smul, ofReal_mul, ofReal_ofNat, momentumCLM_app
 mul_neg, sub_neg_eq_add]
 position_comp_radiusRegPow_commute, finsetSum_comp, comp_assoc]
simp only [sub_apply, add_apply, smul_apply, smul_eq_mul, smul_add]
rw [positionCompMomentumSqr_comm, positionDotMomentumCompMomentum_comm,
 positionCompMomentumSqr_comm_positionDotMomentumCompMomentum_add,
 positionCompMomentumSqr_comm_momentum_add, positionDotMomentumCompMomen
 positionCompMomentumSqr_comm_radiusRegInvCompPosition_add,
 positionDotMomentumCompMomentum_comm_radiusRegInvCompPosition_add,
 momentum_comm_radiusRegInvCompPosition_add, radiusRegInvCompPosition_co
simp_rw [Finset.sum_sub_distrib, Finset.sum_add_distrib, ← Finset.smul_sum]
 simp_rw [angularMomentum_comp_radiusRegPow_commute, comp_assoc, ← com
 · simp_rw [← comp_finsetSum, ← finsetSum_comp, ← comp_assoc, dotProduct,
 simp_rw [← comp_assoc, ← momentumSqr_comp_angularMomentum_commute, comp_a
 simp_rw [← position_comp_radiusRegPow_commute, ← comp_assoc, ← finsetSum
 simp_rw [← comp_finsetSum (□₀ _ _)]
 simp [Finset.sum_add_distrib, ← comp_finsetSum, ← Finset.smul_sum,
 ∑ i, □₀[d] ∈ (-1) °L □ i °L □ i = □₀ ∈ (-1) °L (□ □_v □) := by rw [← com
simp_rw [← comp_finsetSum, ← comp_assoc, position_comp_radiusRegPow_commu
 ← comp_finsetSum, ← comp_assoc, radiusRegPowCLM_comp_eq, positionSqCLM
 comp_smul, finsetSum_comp, comp_finsetSum, Finset.sum_add_distrib, ← Fi
simp only [add_apply, _root_.smul_apply, _root_.add_apply,
 root.smul_apply, Function.comp_apply, coe_comp, coe_id', smul_eq_mul,
```

**1.157 Physlib/QuantumMechanics/DDimensions/Operators/AngularMomentum**

```

simp only [FunLike.coe_sum, FunLike.coe_smul,
 ContinuousLinearMap.coe_comp, Finset.sum_apply, Pi.smul_apply, Function
```

**1.158 Physlib/QuantumMechanics/DDimensions/Operators/Commutatio**

```

public import Physlib.Relativity.Tensors.RealTensor.Vector.Tensorial
attribute [local instance 100] LieRing.ofAssociativeRing
```

```

simp [dotProduct, mul_def, sum_lie, leibniz_lie]
have hne := Ne.symm (ne_of_lt <| Space.norm_sq_add_unit_sq_pos ε x)
```

1.159.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/MOMENTUM.lean

```
simp [Finset.sum_sub_distrib, ← Finset.smul_sum, ← comp_finsetSum,
simp only [nsmul_eq_mul, smul_apply, sub_apply, add_apply, mul_apply_eq_comp, comp
root.natCast_apply, positionCLM_apply, momentumCLM_apply, neg_mul, mul_neg, sm
sub_neg_eq_add, smul_eq_mul, smul_add]
```

### 1.159 Physlib/QuantumMechanics/DDimensions/Operators/Momentum.lean

```
domain := SchwartzSubmodule d
lemma momentumOperator_apply (ψ : SchwartzSubmodule d) :
lemma momentumOperator_apply_ae (ψ : SchwartzSubmodule d) :
lemma momentumOperator_range (ψ : SchwartzSubmodule d) : [] i ψ ∈ SchwartzSubmodule d
lemma momentumSqOperator_domain_eq : momentumSqOperator.domain = SchwartzSubmodule d
rw [← iInf_const (a := SchwartzSubmodule d) (ι := Fin d)]
```

### 1.160 Physlib/QuantumMechanics/DDimensions/Operators/Multiplication.lean

```
refine (hψ.add hφ).ae_eq ?_
filter_upwards [coeFn_add ψ φ] with x h
zero_mem' := MemHS.zero.ae_eq (by filter_upwards; simp)
refine (hψ.const_smul c).ae_eq ?_
filter_upwards [coeFn_smul c ψ] with x h
filter_upwards [coeFn_add ψ.1 φ.1] with x h
filter_upwards [coeFn_smul c ψ.1] with x h
coeFn_mk ψ.prop
let φ : ℕ → SpaceDHilbertSpace d := fun n ↦
mk ((memHS_coe ψ).indicator (s := s n) (by measurability))
have hφ : ∀ n, φ n = [volume] (s n).indicator ψ := fun n ↦ coeFn_mk _
refine memHS_iff.mpr (by measurability, by measurability, ?_)
refine HasFiniteIntegral.mono (memHS_iff.mp <| memHS_coe (n • φ n)).2.2 ?_
filter_upwards [hfu, coeFn_smul n (φ n), hφ n] with x h₁ h₂ h₃
_ = ||(n • φ n) x|| := by simp [h₂, ← Nat.cast_smul_eq_nsmul ℂ]
filter_upwards [coeFn_sub (φ n) ψ, hφ n] with x h₁ h₂
· have : ∫- x, ||ψ x|| ^ 2 ||_e ≠ τ := (memHS_iff.mp <| memHS_coe ψ).2.2.ne
refine ENNReal.Tendsto.pow (Tendsto.enorm (tendsto_nhds_of_eventually_eq ?_))
refine eventually_atTop.mpr (||u x||+, fun n hn ↦ ?_)
exact (Nat.le_ceil _).trans (by exact_mod_cast hn)
{f : Space d → ℂ} (hf : f.HasTemperateGrowth) : SchwartzSubmodule d ≤ (∑ f).domain
refine (memHS_coe φ).ae_eq ?_
suffices ∀ r : ℝ, r ≤ [r].toNat by simp [s, this]
exact fun r ↦ (Int.le_ceil r).trans (by exact_mod_cast Int.self_le_toNat _)
filter_upwards [coeFn_mk (hψ n)] with _ h
refine lt_of_le_of_lt ?_ (memHS_iff.mp <| memHS_coe ξ).2.2
· rw [← setLIntegral_univ, ← hs_univ,
```

```

 setLIntegral_iUnion_of_directed _ (directed_of_isDirected_le hs_mon
 filter_upwards [coeFn_mk (hw n)] with x h₁
 · exact (memHS_iff.mp <| memHS_coe (φ n)).2.2.ne
 · exact (memHS_iff.mp <| memHS_coe ξ).2.2.ne
suffices ||φ n|| ^ 2 ≤ ||ξ|| * ||φ n|| by
 nlinarith [this, sq_nonneg (||ξ|| - ||φ n||)]
_ = ||φ n, φ n|_C|| := by simp
_ = ||φ, φ f (φ n, hφ n)|_C|| := by
 refine congrArg norm ?_
 filter_upwards [coeFn_mk (hw n), mulOperator_apply_ae (φ n, hφ n)] wi
_ = ||ξ, φ n|_C|| := by
_ ≤ ||ξ|| * ||φ n|| := norm_inner_le_norm ξ (φ n)
rw [isSelfAdjoint_def, mulOperator_adjoint_eq_conj hf, hf']
refine isClosable_of_exists_dense_formalAdjoint (mulOperator_hasDenseDoma
exact (λ (conj ∘ f), mulOperator_hasDenseDomain (by measurability),
 mulOperator_adjoint_eq_conj hf ▶ adjoint_isFormalAdjoint (mulOperator_h
 refine (mem_mulOperator_domain_iff.mp hgψ).ae_eq ?_
refine mem_compRestricted_domain_iff.mpr (h ▶ Submodule.mem_top, ?_)
refine (mem_mulOperator_domain_iff.mp hψ).ae_eq ?_

```

### 1.161 Physlib/QuantumMechanics/DDimensions/Operators/Position.1

```

public import Physlib.SpaceAndTime.Space.Norm.Regularized
simp only [positionCLM]
erw [smulLeftCLM_apply_apply (g := Complex.ofRealCLM ∘ (coordCLM i)) (by
simp [coordCLM_apply, coord_apply]
(hψ : ψ ∈ PolyBddSchwartzMap d a) (h : 0 < d + 2 * (a + s)) :
· have : NeZero d := (hd.ne')
 have lintegral_lt_top_aux : ∀ {S : Set (Space d)} {C p : ℝ},
 IntegrableOn (fun x ↦ ||x|| ^ p) S → ∀ {G : Space d → ENNReal},
 ∫- x in S, G x ≤ ∫- x in S, ||C ^ 2||e * |||x|| ^ p||e →
 ∫- x in S, G x < τ := by
 intro S C p hp G hG
 refine hG.trans_lt ?_
 rw [lintegral_const_mul _ (by fun_prop)]
 exact ENNReal.mul_lt_top enorm_lt_top hp.hasFiniteIntegral
 exact lintegral_lt_top_aux ((integrableOn_norm_rpow_ball_iff Real.z
 (setLIntegral_mono_ae' measurableSet_ball (Eventually.mono hBound
 exact lintegral_lt_top_aux ((integrableOn_norm_rpow_ball_compl_iff
 (setLIntegral_mono' (by measurability) hBound)
 exact (Int.le_ceil s).trans (by exact_mod_cast Int.self_le_toNat [s])
 exact hε (by simp [abs_of_pos, hε_pos])
lemma radiusRegPow_ae_tendsto_radiusPow {d : ℕ} [NeZero d] (s : ℝ) (ψ : λ(S
 exact measure_mono_null h (measure_singleton 0)

```

1.162.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/BASIC.LEAN

```
lemma radiusRegPow_ae_tendsto_iff {d : ℕ} [NeZero d] {s : ℝ} {ψ : ⅈ(Space d, ℂ)}
open Space
lemma radiusRegPowOperator_isSelfAdjoint (ε : ℝ+) (s : ℝ) : IsSelfAdjoint (ⅈ[d] ε s
 mulOperator_isSelfAdjoint_ofReal (by fun_prop) (by ext; simp))
lemma radiusPowOperator_hasDenseDomain (s : ℝ) : (ⅈ[d] s).HasDenseDomain :=
 mulOperator_hasDenseDomain (Measurable.aestronglyMeasurable (by fun_prop))
lemma radiusPowOperator_isSelfAdjoint (s : ℝ) : IsSelfAdjoint (ⅈ[d] s) :=
 mulOperator_isSelfAdjoint_ofReal (Measurable.aestronglyMeasurable (by fun_prop)) (
 (ψ : PolyBddSchwartzSubmodule d | 1 - d / 2 - s|.toNat) :
 PolyBddSchwartzSubmodule d | 1 - d / 2 - s|.toNat ≤ (radiusPowOperator s).domain
 refine MemHS.ae_eq (f := ⅈ s f.1) ?_ ?_
 · exact radiusPowLM_apply_memHS s f.1 _ f.2 (add_floor_toNat_pos_aux d s)
```

## 1.162 Physlib/QuantumMechanics/DDimensions/Operators/SpectralTheory/Ba

/-

Copyright (c) 2026 Gregory J. Loges. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Gregory J. Loges

-/

module

public import Physlib.QuantumMechanics.DDimensions.Operators.Unbounded

/-!

# Spectral theory for closed operators

## i. Overview

In this module we develop the basics for the spectral theory of closed unbounded operators. This forms the basis for the spectral theory of self-adjoint unbounded operators, which are of central importance in quantum mechanics.

Definitions for subsets of  $\mathbb{C}$  associated to an operator  $T : H \rightarrow_{\cdot} [\mathbb{C}] H$  vary by author. Here we adopt those used in [Konrad Schmüdgen, \*Unbounded Self-Adjoint Operators on Hilbert Space\*][Schmudgen2011] summarized in the following table:

| Subset of $\mathbb{C}$ | abbrev.        | $D(T - z)$     | $R(T - z)$    | $(T - z)^{-1}$ |
|------------------------|----------------|----------------|---------------|----------------|
| :                      | :              | :              | :             | :              |
| :                      | :              | :              | :             | :              |
| Regularity domain      | $\rho = \perp$ | continuous     |               |                |
| Resolvent set          | $\rho$         | $\rho = \perp$ | $\rho = \tau$ | continuous     |
| Point spectrum         | $\sigma^p$     | $\neq \perp$   |               |                |

|                     |            |                  |          |            |
|---------------------|------------|------------------|----------|------------|
| Residual spectrum   | $\sigma_r$ | $\neq \emptyset$ | $\neq T$ | continuous |
| Continuous spectrum | $\sigma^c$ |                  |          | not closed |

## ## ii. Key results

Definitions (corresponding to an operator  $T : H \rightarrow_{\mathcal{L}} [C] H$ )

- `LinearPMap.regularityDomain` : The set of regular points. A complex number  $z$  is a regular point if there exists  $c > 0$  such that  $c \cdot \|x\| \leq \|T x - z \cdot x\|$  for all  $x \in H$ .
- `LinearPMap.deficiencySubspace` : Given a complex number  $z$ , the closed subspace of  $H$  which is orthogonal to the range of  $T - z \cdot 1$ .
- `LinearPMap.defectNumber` : Given a complex number  $z$ , the rank of the deficiency subspace as a (possibly infinite) cardinal.
- `LinearPMap.numericalRange` ( $\sigma^n$ ) : The set of complex numbers  $z$  such that  $z = \langle T x, x \rangle$  for some  $x$  in the unit sphere in  $H$ .
- `LinearPMap.resolventSet` ( $\rho$ ) : The set of complex numbers  $z$  for which  $T - z \cdot 1$  has a continuous (equivalently, bounded) inverse with domain all of  $H$ .
- `LinearPMap.spectrum` ( $\sigma$ ) : The complement of the resolvent set.
- `LinearPMap.pointSpectrum` ( $\sigma^p$ ) : The set of complex numbers  $z$  for which  $T - z \cdot 1$  fails to be invertible.
- `LinearPMap.residualSpectrum` ( $\sigma_r$ ) : The set of complex numbers  $z$  for which  $T - z \cdot 1$  has a continuous (equivalently, bounded) inverse with domain not all of  $H$ .
- `LinearPMap.continuousSpectrum` ( $\sigma^c$ ) : The set of complex numbers  $z$  for which the range of  $T - z \cdot 1$  is not dense in  $H$ .

## Main results

- `regularityDomain_isOpen` : The regularity domain is an open subset of  $C$ .
- `closure_range_sub_eq_range_closure_sub` : If  $z$  is a regular point for  $T$ , then the closure of  $(T - z \cdot 1).range$  is  $(T.closure - z \cdot 1).range$ .
- `defectNumber_const` : The defect number is constant on each connected component of the regularity domain.
- `compl_closure_numericalRange_subset_regularityDomain` : The regularity domain is the exterior of the numerical range.
- `numericalRange_convex` : The Toeplitz-Hausdorff theorem – the numerical range is convex.
- `resolventSet_isOpen` and `spectrum_isClosed` : The resolvent set is an open set and its complement, the spectrum, is closed.
- `IsClosed.spectrum_eq` : For a closed operator the spectrum is the union of the point, residual and continuous spectra.

## ## iii. Table of contents

- A. Regularity domain
- B. Deficiency subspace & defect number
- C. Numerical range
  - C.1. The Toeplitz-Hausdorff theorem
- D. Spectrum of a closed operator
  - D.1. Resolvent set



1.162.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/BASIC.LEAN

- D.2. Spectrum
  - D.2.1. Point spectrum
  - D.2.2. Residual spectrum
  - D.2.3. Continuous spectrum
- D.3. Spectrum decomposition
- E. Resolvent identities

## iv. References

- [Konrad Schmüdgen, \*Unbounded Self-Adjoint Operators on Hilbert Space\*][Schmudgen2]

-/

@[expose] public section

namespace LinearPMap

variable {H : Type\*} [NormedAddCommGroup H] [InnerProductSpace ℂ H]

noncomputable section

open Submodule

open Metric

open InnerProductSpace

open Complex

open ComplexConjugate

open Set

open Pointwise

/-- The resolvent,  $(T - z \cdot 1)^{-1}$ . -/

abbrev resolvent (T : H →<sub>ℓ</sub>.[ℂ] H) (z : ℂ) : H →<sub>ℓ</sub>.[ℂ] H := (T - z • 1).inverse

@[inherit\_doc resolvent]

scoped notation "[]" => resolvent

/-!

## A. Regularity domain

-/

/-- `IsLowerBound T z c` is the property that  $c \cdot \|x\| \leq \|T x - z \cdot x\|$  for all  $x$  :  
/

def IsLowerBound (T : H →<sub>ℓ</sub>.[ℂ] H) (z : ℂ) (c : ℝ) : Prop := ∀ x : T.domain, c \* ||x||

lemma isLowerBound\_neg {T : H →<sub>ℓ</sub>.[ℂ] H} {z : ℂ} {c : ℝ} (h : IsLowerBound T z c) :  
  IsLowerBound (-T) (-z) c := by  
  intro x

```

simp only [neg_apply, neg_smul, sub_neg_eq_add, norm_neg_add, h x]

lemma isLowerBound_of_right_le
 {T : H →1.[C] H} {z : C} {c1 c2 : ℝ} (hle : c1 ≤ c2) (h : IsLowerBound
 IsLowerBound T z c1 :=
 fun x ↦ (mul_le_mul_of_nonneg_right hle (norm_nonneg x)).trans (h x)

lemma isLowerBound_of_left_le
 {T1 T2 : H →1.[C] H} (hle : T1 ≤ T2) {z : C} {c : ℝ} (h : IsLowerBound
 IsLowerBound T1 z c :=
 fun x ↦ @hle.2 x ⟨x, hle.1 x.2⟩ rfl ▸ h ⟨x, hle.1 x.2⟩

lemma isLowerBound_closure
 {T : H →1.[C] H} {z : C} {c : ℝ} (h : IsLowerBound T z c) : IsLowerBound
 by_cases hT : T.IsClosable
 · intro x
 obtain ⟨b, hb, hb'⟩ := mem_closure_iff_seq_limit.mp <|
 hT.graph_closure_eq_closure_graph ▸ T.closure.mem_graph x
 rw [nhds_prod_eq] at hb'
 have hb1 := hb'.fst.norm.const_mul c
 have hb2 := (hb'.snd.sub <| hb'.fst.const_smul z).norm
 refine le_of_tendsto_of_tendsto' hb1 hb2 fun n ↦ ?_
 obtain ⟨y, hy1, hy2⟩ := (mem_graph_iff _).mp (hb n)
 exact hy1 ▸ hy2 ▸ h y
 · rwa [closure_def' hT]

/-- The regular points for `T`.

`z : C` is a regular point for `T` iff there exists a constant `c > 0` su
`c * ||x|| ≤ ||(T - z • 1) x||` for all `x ∈ T.domain`. -/
def regularityDomain (T : H →1.[C] H) : Set C := {z : C | ∃ c > 0, IsLowerB

@[simp]
lemma regularityDomain_neg (T : H →1.[C] H) : (-T).regularityDomain = -
T.regularityDomain := by
 ext z
 constructor
 · exact fun ⟨c, hc, h_bound⟩ ↦ ⟨c, hc, neg_neg T ▸ isLowerBound_neg h_bou
 · exact fun ⟨c, hc, h_bound⟩ ↦ ⟨c, hc, neg_neg z ▸ isLowerBound_neg h_bou

@[simp]
lemma regularityDomain_smul (T : H →1.[C] H) {w : C} (hw : w ≠ 0) :
 (w • T).regularityDomain = w • T.regularityDomain := by
 ext z
 constructor
 · intro ⟨c, hc, h_bound⟩

```

1.162.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/BASIC.LEAN

```

 refine (w⁻¹ * z, ?_, ?_)
 · refine (||w||⁻¹ * c, by positivity, fun x ↦ ?_)
 rw [mul_assoc]
 apply (inv_mul_le_iff₀ <| norm_pos_iff.mpr hw).mpr
 rw [← norm_smul, smul_sub, smul_smul, mul_inv_cancel_left₀ hw]
 exact h_bound x
 · simp [hw]
 · intro (u, (c, hc, h_bound), huz)
 refine (||w|| * c, by positivity, fun x ↦ ?_)
 rw [mul_assoc]
 apply (le_inv_mul_iff₀ <| norm_pos_iff.mpr hw).mp
 refine le_of_le_of_eq (h_bound x) ?_
 simp [← norm_inv, ← norm_smul, smul_sub, smul_smul, ← huz, hw]

/-- `T ≤ T` implies `T.regularityDomain ⊆ T.regularityDomain`. -
/
lemma regularityDomain_antitone : Antitone (regularityDomain (H := H)) :=
 fun _ _ hle _ (c, hc, h) ↦ (c, hc, isLowerBound_of_left_le hle h)

/-- `z` is a regular point for `T` iff `T - z • 1` has
 a continuous (equivalently, bounded) inverse. -/
lemma mem_regularityDomain_iff {T : H →ₗ[ℂ] H} {z : ℂ} :
 z ∈ T.regularityDomain ↔ (T - z • 1).toFun.ker = ⊥ ∧ Continuous (□ T z) := by
 constructor
 · intro (c, hc, h_bound)
 have h_ker : (T - z • 1).toFun.ker = ⊥ := by
 ext x
 constructor <=> intro
 · have : c * ||x|| ≤ 0 → ||x|| ≤ 0 := fun h' ↦ nonpos_of_mul_nonpos_right h' hc
 specialize h_bound (x, x.2.1)
 simp_all [sub_apply]
 · simp_all
 use h_ker
 apply LinearMap.continuous_iff_bounded.mpr
 refine (c⁻¹, inv_pos.mpr hc, fun (x, hx) ↦ ?_)
 rw [inverse_domain] at hx
 obtain (y, hy) := hx
 specialize h_bound (y, y.2.1)
 simp_all [le_inv_mul_iff₀, sub_apply, inverse_apply_eq h_ker (y := (x, hx)) hy]
 · intro (h_ker, h_cont)
 obtain (c, hc, h_bound) := LinearMap.continuous_iff_bounded.mp h_cont
 refine (c⁻¹, inv_pos.mpr hc, fun x ↦ ?_)
 apply (inv_mul_le_iff₀ hc).mpr
 have hx : ↑x ∈ (T - z • 1).domain := by simp [sub_domain]
 specialize h_bound ((T - z • 1) (x, hx), by simp [inverse_domain])
 simp only [toFun_eq_coe, inverse_apply_eq h_ker (x := (x, hx)), coe_norm] at h_b

```

```

simp_all [sub_apply]

/-- The regularity domain of `T` contains open balls with radii controlled
/
lemma ball_subset_regularityDomain
 {T : H →1.[C] H} {z : C} {c : ℝ} (h : IsLowerBound T z c) : ball z c ⊆
 intro z' hzc
 refine ⟨c - ‖z - z'‖, by simp_all [dist_eq, norm_sub_rev], fun x ↦ ?_⟩
 calc
 _ = c * ‖x‖ - ‖(z - z') • x‖ := by simp [sub_mul, norm_smul]
 _ ≤ ‖T x - z • x‖ - ‖(z - z') • x‖ := by linarith [h x]
 _ ≤ ‖T x - z • x + (z - z') • x‖ := norm_sub_le_norm_add _ _
 _ = ‖T x - z' • x‖ := by simp [sub_smul]

/-- The regularity domain is an open set. -/
lemma regularityDomain_isOpen (T : H →1.[C] H) : IsOpen T.regularityDomain
 isOpen_iff.mpr fun _ ⟨c, hc, h⟩ ↦ ⟨c, hc, ball_subset_regularityDomain h⟩

/-- `T` and `T.closure` have the same regularity domain. -
/
lemma regularityDomain_closure (T : H →1.[C] H) :
 T.closure.regularityDomain = T.regularityDomain := by
 refine eq_of_le_of_ge (regularityDomain_antitone T.le_closure) ?_
 exact fun _ ⟨c, hc, h⟩ ↦ ⟨c, hc, isLowerBound_closure h⟩

lemma IsClosable.closure_range_sub_eq_range_closure_sub [CompleteSpace H]
 {T : H →1.[C] H} (hT : T.IsClosable) {z : C} (hz : z ∈ T.regularityDomain)
 (T - z • 1).toFun.range.closure = (T.closure - z • 1).toFun.range := by
 ext y
 constructor
 · intro hy
 obtain ⟨b, hb, hby⟩ := mem_closure_iff_seq_limit.mp hy
 let x : ℕ → H := fun n ↦ (hb n).choose
 have hx : ∀ n, x n ∈ T.domain := fun n ↦ (hb n).choose.2.1
 have hx' : ∀ n, T ⟨x n, hx n⟩ - z • x n = b n := fun n ↦ (hb n).choose
 have hCS : CauchySeq x := by
 apply Metric.cauchySeq_iff'.mpr fun ε hε ↦ ?_
 obtain ⟨c, hc, h_bound⟩ := hz
 obtain ⟨N, hN⟩ := cauchySeq_iff'.mp hby.cauchySeq (c * ε) (mul_pos hc
 refine ⟨N, fun n hn ↦ Eq.trans_lt (dist_eq_norm _ _) ((mul_lt_mul_iff
 calc
 _ ≤ ‖T ⟨x n, hx n⟩ - ⟨x N, hx N⟩ - z • (x n - x N)‖ := h_bound _
 _ = ‖b n - b N‖ := by rw [← hx', ← hx', map_sub, smul_sub, sub_sub
 _ = dist (b n) (b N) := (dist_eq_norm _ _).symm
 _ < (c * ε) := hN n hn
 obtain ⟨x₀, hx₀⟩ := CompleteSpace.complete hCS

```

1.162.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/BASIC.LEAN

```

suffices (x₀, y + z • x₀) ∈ T.closure.graph by
 obtain (x₀', rfl, _) := (mem_graph_iff _).mp this
 use (x₀', by simp [sub_domain])
 simp_all [sub_apply]
rw [← hT.graph_closure_eq_closure_graph]
apply mem_closure_iff_seq_limit.mpr
refine (fun n ↦ (x n, b n + z • x n), fun n ↦ ?_, ?_)
· exact (mem_graph_iff _).mpr (⟨(x n, hx n), by simp [← hx' n]⟩)
· exact Filter.Tendsto.prodMk_nhds hx₀ (hby.add <| Filter.Tendsto.const_smul hx₀)
· intro (⟨(x, hx), hxy⟩)
 obtain (b, hb, hb') := mem_closure_iff_seq_limit.mp <|
 hT.graph_closure_eq_closure_graph ▸ T.closure.mem_graph (x, hx.1)
 simp only [coe_toAddSubmonoid, SetLike.mem_coe, mem_graph_iff] at hb
 rw [nhds_prod_eq] at hb'
 apply mem_closure_iff_seq_limit.mpr
 refine (fun n ↦ (b n).2 - z • (b n).1, fun n ↦ ?_, hxy ▸ hb'.snd.sub (hb'.fst.co
 obtain (u, hu₁, hu₂) := hb n
 use (u, by simp [sub_domain])
 simp [sub_apply, ← hu₁, hu₂]

lemma IsClosed.sub_range_isClosed [CompleteSpace H]
 {T : H →ℓ.[ℂ] H} (hT : T.IsClosed) {z : ℂ} (hz : z ∈ T.regularityDomain) :
 root.IsClosed ((T - z • 1).toFun.range : Set H) := by
 have hT' : T.closure = T := hT.isClosable.isClosed_iff.mp hT
 exact (hT' ▸ hT.isClosable.closure_range_sub_eq_range_closure_sub hz) ▸ isClosed_c

/-- `(T.closure - z • 1).range[] = (T† - conj z • 1).ker` -
/
lemma IsUnbounded.orthogonal_closure_sub_range [CompleteSpace H]
 {T : H →ℓ.[ℂ] H} (hT : T.IsUnbounded) (z : ℂ) :
 (T.closure - z • 1).toFun.range[]
 = (T† - conj z • 1).toFun.ker.map (T† - conj z • 1).domain.subtype := by
 let S := T.closure - z • 1
 have hS_domain : S.domain = T.closure.domain := by simp [S, sub_domain]
 have hS_dense : S.HasDenseDomain := hT.hasDenseDomain.mono (by simp [hS_domain, T.
 have hS_adjoint : S† = T† - conj z • 1 := by
 rw [← hT.adjoint_closure_eq_adjoint]
 refine (eq_of_le_of_domain_eq ?_ ?_).symm
 · refine le_of_eq_of_le ?_ (adjoint_sub_le_sub_adjoint T.closure (z • 1) hS_dens
 rcases eq_zero_or_neZero z with rfl | hz₀
 · simp
 · simp [adjoint_smul _ hz₀.ne]
 · ext x
 simp only [sub_domain, smul_domain, one_domain, le_top, inf_of_le_left]
 constructor <=> intro h
 · apply mem_adjoint_domain_of_exists

```

```

 use T.closure† ⟨x, h⟩ - conj z • x
 intro y
 have h_inner : []T.closure† ⟨x, h⟩, y[]_C = []x, T.closure ⟨y, hS_domain
 adjoint_isFormalAdjoint hT.hasDenseDomain.closure ⟨x, h⟩ ⟨y, hS_d
 simp [inner_sub_left, h_inner, S, sub_apply, inner_sub_right, inner
 inner_smul_right]
 · apply mem_adjoint_domain_of_exists
 use S† ⟨x, h⟩ + conj z • x
 intro y
 have h_inner : []S† ⟨x, h⟩, y[]_C = []x, S ⟨y, by simp [hS_domain]][]_C
 adjoint_isFormalAdjoint hS_dense ⟨x, h⟩ ⟨y, by simp [hS_domain]]
 simp [inner_add_left, h_inner, S, sub_apply, inner_sub_right, inner
 inner_smul_right]
 exact hS_adjoint ▸ hS_dense.orthogonal_range

/-- `(T† - conj z • 1).ker[] = (T.closure - z • 1).range` -
/
lemma IsUnbounded.orthogonal_adjoint_sub_ker [CompleteSpace H]
 {T : H →ℂ.[C] H} (hT : T.IsUnbounded) {z : ℂ} (hz : z ∈ T.regularityDom
 ((T† - conj z • 1).toFun.ker.map (T† - conj z • 1).domain.subtype)[]
 = (T.closure - z • 1).toFun.range := by
 have hT' : IsClosable T.closure := hT.isClosable.closureIsClosable
 have hTcl : T.closure.closure = T.closure := hT'.isClosed_iff.mp hT.isClo
 rw [← hTcl, ← hT.orthogonal_closure_sub_range, orthogonal_orthogonal_eq_c
 exact hT'.closure_range_sub_eq_range_closure_sub (T.regularityDomain_clos

/-!
B. Deficiency subspace & defect number
-/

/-- For a partial linear map `T` and any complex number `z`, the closed sub
 is orthogonal to the range of `T - z • 1`.

 `T.defectNumber z` is defined as the rank of this subspace. -
/
def deficiencySubspace (T : H →ℂ.[C] H) (z : ℂ) : ClosedSubmodule ℂ H :=
 ((T - z • 1).toFun.range[], isClosed_orthogonal _)

@[simp]
lemma deficiencySubspace_coe (T : H →ℂ.[C] H) (z : ℂ) :
 T.deficiencySubspace z = (T - z • 1).toFun.range[] := rfl

/-- The rank of `T.deficiencySubspace z = (T - z • 1).range[]`. -
/
def defectNumber (T : H →ℂ.[C] H) (z : ℂ) : Cardinal := Module.rank ℂ (T.de

```

1.162.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/BASIC.LEAN

```
lemma defectNumber_eq (T : H →1.[$\mathbb{C}$] H) (z : $\mathbb{C}$) :
 T.defectNumber z = Module.rank $\mathbb{C}$ (T.deficiencySubspace z) := rfl
```

```
lemma IsClosed.defectNumber_eq_zero_iff [CompleteSpace H]
 {T : H →1.[$\mathbb{C}$] H} (hT : T.IsClosed) {z : $\mathbb{C}$ } (hz : z ∈ T.regularityDomain) :
 T.defectNumber z = 0 ↔ (T - z • 1).toFun.range = ⊥ := by
 haveI := hT.sub_range_isClosed hz -- needed for HasOrthogonalProjection
 rw [← orthogonal_eq_bot_iff, ← rank_eq_zero]
 exact Iff.rfl
```

/-- `T` and `T.closure` have the same defect number at points in their regularity domain /

```
lemma defectNumber_closure [CompleteSpace H]
 {T : H →1.[$\mathbb{C}$] H} {z : $\mathbb{C}$ } (hz : z ∈ T.regularityDomain) :
 T.closure.defectNumber z = T.defectNumber z := by
 by_cases hT : T.IsClosable
 · refine congrArg (fun p : Submodule $\mathbb{C}$ H ↦ Module.rank $\mathbb{C}$ p) ?_
 simp [← hT.closure_range_sub_eq_range_closure_sub hz]
 · rw [closure_def' hT]
```

```
lemma _root_.Submodule.inf_ne_bot_of_rank_lt
 {E F : Submodule $\mathbb{C}$ H} [E.HasOrthogonalProjection] (h_rank : Module.rank $\mathbb{C}$ E < Module.rank $\mathbb{C}$ F)
 E ⊂ F ≠ ⊥ := by
 let Φ : F →L[$\mathbb{C}$] E := E.orthogonalProjectionOnto ⊙L F.subtypeL
 have hΦ : ¬(↑Φ).Injective := fun h' ↦ not_le_of_gt h_rank (Φ.rank_le_of_injective h')
 obtain ⟨x1, x2, h, hx⟩ := Function.not_injective_iff.mp hΦ
 let y : H := x1 - x2
 have hy : y ≠ 0 := fun h' ↦ hx (SetLike.coe_eq_coe.mp <| sub_eq_zero.mp h')
 have hF : y ∈ F := sub_mem (coe_mem x1) (coe_mem x2)
 have hE : y ∈ E := orthogonalProjectionOnto_eq_zero_iff.mp
 (_root_.map_sub Φ _ _ ▸ sub_eq_zero.mpr h)
 exact fun hEF ↦ hy ((mem_bot $\mathbb{C}$).mp <| hEF ▸ ⟨hE, hF⟩)
```

```
lemma IsClosed.exists_inner_eq_zero_of_defectNumber_lt [CompleteSpace H]
 {T : H →1.[$\mathbb{C}$] H} (hT : T.IsClosed)
 {z1 z2 : $\mathbb{C}$ } (hz1 : z1 ∈ T.regularityDomain) (h : T.defectNumber z1 < T.defectNumber z2)
 ∃ x : T.domain, x ≠ 0 ∧ ⊥ ≤ T x - z1 • x, T x - z2 • x ⊥ $\mathbb{C}$ = 0 := by
 obtain ⟨y, h_inf, hy⟩ := (Submodule.ne_bot_iff _).mp (inf_ne_bot_of_rank_lt h)
 obtain ⟨hy1, hy2⟩ := mem_inf.mp h_inf
 haveI := hT.sub_range_isClosed hz1 -- needed for `orthogonal_orthogonal`
 simp only [deficiencySubspace_coe, orthogonal_orthogonal] at hy1 hy2
 obtain ⟨(x, hx), hxy⟩ := hy1
 refine ⟨(x, hx.1), fun h ↦ hy ?_, ?_⟩
 · simp [← hxy, coe_eq_zero.mp, (mk_eq_zero _ _).mp h]
 · apply (mem_orthogonal' _ _).mp at hy2
 simp [← hy2 ((T - z2 • 1) (x, hx.1, by simp)) (by simp), sub_apply, ← hxy]
```

```

lemma IsClosable.defectNumber_eq_of_mem_ball [CompleteSpace H] {T : H →1. [C]
 {z₁ z₂ : C} {c : ℝ} (h : IsLowerBound T z₁ c) (h_ball : z₂ ∈ ball z₁ c)
 T.defectNumber z₁ = T.defectNumber z₂ := by
 by_cases hz₁ : z₁ ∈ T.regularityDomain
 · have hz₂ : z₂ ∈ T.regularityDomain := ball_subset_regularityDomain h h_ball
 rw [← defectNumber_closure hz₁, ← defectNumber_closure hz₂]
 rw [← regularityDomain_closure] at hz₁ hz₂
 by_contra! hne
 let Tc₁ : H →1. [C] H := T.closure
 obtain ⟨x, hx, h'⟩ : ∃ x : Tc₁.domain, x ≠ 0 ∧ [Tc₁ x - z₂ • x, Tc₁ x - z₁ • x]
 rcases lt_or_gt_of_ne hne with hle | hle
 · simp_rw [inner_eq_zero_symm]
 exact hT.closure_isClosed.exists_inner_eq_zero_of_defectNumber_lt hz₁
 · exact hT.closure_isClosed.exists_inner_eq_zero_of_defectNumber_lt hz₂
 refine not_le (a := ||z₁ - z₂|| * ||x||).mpr ?_ le_refl
 refine lt_of_lt_of_le (b := ||Tc₁ x - z₁ • x||) ?_ ?_
 · refine lt_of_lt_of_le ?_ (isLowerBound_closure h x)
 exact (mul_lt_mul_iff_left₀ <| norm_pos_iff.mpr hx).mpr (mem_ball_iff h_ball)
 · rcases eq_or_ne (Tc₁ x - z₁ • x) 0 with heq | hne
 · exact heq ▸ norm_zero (E := H) ▸ mul_nonneg (norm_nonneg _) (norm_nonneg _)
 · apply (mul_le_mul_iff_left₀ (norm_pos_iff.mpr hne)).mpr
 trans [||Tc₁ x - z₂ • x - (z₁ - z₂) • x, Tc₁ x - z₁ • x||]
 · simp [sub_smul, pow_two]
 rw [inner_sub_left, h', zero_sub, inner_smul_left, norm_neg, norm_mul]
 exact mul_le_mul_of_nonneg_left (norm_inner_le_norm _) (norm_nonneg _)
 · false_or_by_contra - - `z₁ ∉ T.regularityDomain` ⇒ `c ≤ 0` ⇒ `z₂ ∈ ∅`
 exact hz₁ ⟨c, lt_of_le_of_lt dist_nonneg h_ball, h⟩

/-- The defect number is constant on each connected component of the regularity domain -/
lemma IsClosable.defectNumber_const [CompleteSpace H]
 {T : H →1. [C] H} (hT : T.IsClosable)
 {z₁ z₂ : C} (hz : z₂ ∈ connectedComponentIn T.regularityDomain z₁) :
 T.defectNumber z₁ = T.defectNumber z₂ := by
 by_cases hz₁ : z₁ ∈ T.regularityDomain
 · have h_joined : JoinedIn T.regularityDomain z₁ z₂ := by
 haveI := T.regularityDomain_isOpen.locPathConnectedSpace
 have hz₂ : z₂ ∈ T.regularityDomain := connectedComponentIn_subset _ _
 apply (joinedIn_iff_joined hz₁ hz₂).mpr
 rw [← mem_pathComponent_iff, pathComponent_eq_connectedComponent]
 exact mem_of_mem_image_val (connectedComponentIn_eq_image hz₁ ▸ hz)
 let path : Path z₁ z₂ := h_joined.somePath
 by_contra! hne
 let a : unitInterval := sSup {r | ∀ r' ≤ r, T.defectNumber (path r') = T.defectNumber z₁}
 have ha : ∀ r < a, T.defectNumber (path r) = T.defectNumber z₁ := by

```



1.162.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/BASIC.LEAN

```

intro r hr
obtain ⟨b, hb, hrb⟩ := lt_sSup_iff.mp hr
exact hb r hrb.le
let c : ℝ := (h_joined.somePath_mem a).choose
have hc_pos : 0 < c := (h_joined.somePath_mem a).choose_spec.1
have hc_bound : IsLowerBound T (path a) c := (h_joined.somePath_mem a).choose_spec.2
obtain ⟨ε, hε, hε_ball⟩ : ∃ ε > 0, ball a ε ⊆ path-1 ball (path a) c := by
 apply Metric.mem_nhds_iff.mp
 refine (IsOpen.mem_nhds_iff ?_).mpr ?_
 · exact path.continuous.isOpen_preimage _ isOpen_ball
 · simp [hc_pos]
obtain ⟨b₁, h₁, h₁'⟩ : ∃ b ∈ ball a ε, T.defectNumber (path b) = T.defectNumber
 rcases le_or_gt ε a with hle | hlt
 · let r : ℝ := a - ε / 2
 have hr : 0 ≤ r := by dsimp [r]; linarith
 have hr' : r < a := sub_lt_self _ (half_pos hε)
 use ⟨r, hr, by linarith [a.2.2]⟩
 exact ⟨by simp [dist, r, abs_div, abs_of_nonneg hε.le, hε], ha _ hr'⟩
 · exact ⟨0, by simp [dist, abs_of_nonneg a.2.1, hlt], by rw [path.source]⟩
obtain ⟨b₂, h₂, h₂'⟩ : ∃ b ∈ ball a ε, T.defectNumber (path b) ≠ T.defectNumber
by_cases! h₀ : a < 1
· by_contra! h'
 let r : unitInterval :=
 ⟨min (a + ε / 2) 1, le_inf_iff.mpr ⟨by linarith [a.2.1], zero_le_one⟩, inf⟩
 refine not_le_of_gt (a := a) (b := r) ?_ ?_
 · apply (Set.inclusion_lt_inclusion <| Set.subset_univ _).mp
 simp [r, hε, h₀]
 · refine le_sSup_iff.mpr fun _ hub ↦ hub fun b hbr ↦ ?_
 rcases lt_or_ge b a with hlt | hle
 · exact ha b hlt
 · refine h' b ?_
 apply mem_ball.mpr
 calc
 _ = (b : ℝ) - a := by simp [dist, hle]
 _ ≤ r - a := by simp [hbr]
 _ = min (ε / 2) (1 - a) := by simp [r, ← min_sub_sub_right]
 _ < ε := by simp [hε]
 · have : a = 1 := eq_of_le_of_ge a.2.2 h₀
 refine ⟨a, mem_ball_self hε, by rw [this, path.target]; exact hne.symm⟩
apply h₁' ▸ h₂'
rw [← defectNumber_eq_of_mem_ball hT hc_bound (hε_ball h₁)]
rw [← defectNumber_eq_of_mem_ball hT hc_bound (hε_ball h₂)]
· false_or_by_contra
 exact (mem_empty_iff_false z₂).mp (connectedComponentIn_eq_empty hz₁ ▸ hz)

```

/-!

```

C. Numerical range
-/

/-- The set $\{\lambda x, T x \mid x \in T.\text{domain} \wedge \|x\| = 1\} \subseteq \mathbb{C}$. -
/
def numericalRange (T : H →ℓ.[$\mathbb{C}$] H) : Set $\mathbb{C}$:= (fun x ↦ $\lambda x, T x$) '' {x

@[inherit_doc numericalRange]
scoped notation "0" => numericalRange

lemma numericalRange_eq (T : H →ℓ.[$\mathbb{C}$] H) : 0 T = (fun x ↦ $\lambda x, T x$) '' {

lemma mem_numericalRange {T : H →ℓ.[$\mathbb{C}$] H} {x : T.domain} (hx : x ≠ 0) :
 ($\|x\|$ ^ 2)-1 * $\lambda x, T x \in 0 T$:= by
 refine ⟨ofReal $\|x\|^{-1} \cdot x$, ?_, ?_⟩
 · simp [norm_smul, inv_mul_cancel₀, hx]
 · simp [map_smul, inner_smul_left, inner_smul_right, ← mul_assoc, pow_two]

lemma numericalRange_nonempty {T : H →ℓ.[$\mathbb{C}$] H} (hT : T.domain ≠ ⊥) : (0 T).
 obtain ⟨x, hx, hx'⟩ := exists_mem_ne_zero_of_ne_bot hT
 use ($\|x\|$ ^ 2)-1 * $\lambda x, T (x, hx)$
 exact mem_numericalRange (x := ⟨x, hx⟩) (Subtype.coe_ne_coe.mp hx')

@[simp]
lemma numericalRange_neg (T : H →ℓ.[$\mathbb{C}$] H) : 0 (-T) = -0 T := by
 ext
 simp [numericalRange_eq, neg_eq_iff_eq_neg]

@[simp]
lemma numericalRange_smul (T : H →ℓ.[$\mathbb{C}$] H) (c : $\mathbb{C}$) : 0 (c • T) = c • 0 T :=
 ext
 simp [numericalRange_eq, inner_smul_right, mem_smul_set]

lemma numericalRange_sub_const (T : H →ℓ.[$\mathbb{C}$] H) (c : $\mathbb{C}$) : 0 (T - c • 1) = 0
 ext z
 constructor
 · intro ⟨x, hx, hxz⟩
 refine ⟨z + c, ⟨⟨x, x.2.1⟩, hx, ?_⟩, by simp⟩
 simp_all [← hxz, sub_apply, inner_sub_right, inner_smul_right]
 · intro ⟨z', ⟨x, hx, hxz⟩, hcz⟩
 simp only [mem_singleton_iff, exists_eq_left] at hcz
 refine ⟨⟨x, by simp [sub_domain]⟩, hx, ?_⟩
 simp_all [← hcz, ← hxz, sub_apply, inner_sub_right, inner_smul_right]

/-- The regularity domain contains the exterior of the numerical range. -
/

```

1.162.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/BASIC.LEAN

```

lemma compl_closure_numericalRange_subset_regularityDomain (T : H →ℂ H) :
 (√root_.closure (0 T))c ⊆ T.regularityDomain := by
 intro z hz
 by_cases hT : T.domain = ⊥
 · refine ⟨1, zero_lt_one, fun ⟨x, hx⟩ ↦ ?_⟩
 rw [hT] at hx
 simp_all
 · use infDist z (0 T)
 constructor
 · exact (infDist_pos_iff_notMem_closure <| numericalRange_nonempty hT).mp hz
 · intro x
 rcases eq_or_ne x 0 with rfl | hx
 · simp
 · let y : T.domain := ofReal ||x||-1 • x
 have hy : ||y|| = 1 := by simp [y, norm_smul, inv_mul_cancel₀, hx]
 have hy' : ||x|| ^ 2 * √y, T y√_ℂ = √x, T x√_ℂ := by
 simp_rw [y, map_smul, SetLike.val_smul, inner_smul_left, inner_smul_right,
 ← mul_assoc, pow_two, ← ofReal_mul]
 field_simp
 simp
 apply (mul_le_mul_iff_left₀ <| norm_pos_iff.mpr hx).mp
 rw [mul_assoc, ← pow_two, mul_comm _ ||x||]
 calc
 _ ≤ ||z - √y, T y√_ℂ|| * ||x|| ^ 2 := mul_le_mul_of_nonneg_right
 (dist_eq z _ ▸ infDist_le_dist_of_mem ⟨y, hy, rfl⟩) (pow_two_nonneg _)
 _ = ||√y, T y√_ℂ|| * ||x|| ^ 2 - z * ||x|| ^ 2 := by simp [norm_sub_rev, ← sub_]
 _ = ||√x, T x√_ℂ - z * ||x|| ^ 2|| := by rw [mul_comm, hy']
 _ = ||√x, T x - z • x√_ℂ|| := by simp [inner_sub_right, inner_smul_right]
 _ ≤ ||x|| * ||T x - z • x|| := norm_inner_le_norm _ _

/-!
C.1. The Toeplitz-Hausdorff theorem
-/

private lemma exists_phase_add_im_eq_zero (z₁ z₂ : ℂ) :
 ∃ θ : ℝ, (exp (I * θ) * z₁ + exp (-I * θ) * z₂).im = 0 := by
 let g : ℝ → ℝ := fun θ ↦ (exp (I * θ) * z₁ + exp (-I * θ) * z₂).im
 have hg : g Real.pi = -g 0 := by simp [g, mul_comm I, exp_neg, add_comm]
 have hmem : (0 : ℝ) ∈ Set.uIcc (g 0) (g Real.pi) := by
 rw [hg]
 rcases le_total (g 0) 0 with h | h
 exacts [Set.mem_uIcc.mpr (.inl ⟨h, by linarith⟩), Set.mem_uIcc.mpr (.inr ⟨by lin
 obtain ⟨θ, -, hθ⟩ := intermediate_value_uIcc (by fun_prop : Continuous g).continuo
 exact ⟨θ, hθ⟩

/-- The Toeplitz-Hausdorff theorem. -/

```

```

theorem numericalRange_convex (T : H →ℂ H) : Convex ℝ (0 T) := by
 intro z₀ hz₀ z₁ hz₁ a b ha hb hab
 rcases eq_or_ne z₁ z₀ with rfl | hz
 · simp [← add_mul, eq_sub_iff_add_eq.mpr hab, hz₁]
 · apply sub_ne_zero.mpr at hz
 obtain ⟨x₀, hx₀, _⟩ := hz₀
 obtain ⟨x₁, hx₁, _⟩ := hz₁
 -- Apply an affine transformation to effectively move the endpoints `z₀`
 let S : H →ℂ H := (z₁ - z₀)-1 • (T - z₀ • 1)
 let y₀ : S.domain := ⟨x₀, by simp [S, sub_domain]⟩
 let y₁ : S.domain := ⟨x₁, by simp [S, sub_domain]⟩
 have hy₀ : ‖y₀‖ = 1 := hx₀
 have hy₁ : ‖y₁‖ = 1 := hx₁
 have h₀ : ⟨↑y₀, S y₀⟩_ℂ = 0 := by simp_all [S, y₀, sub_apply, inner_smul]
 have h₁ : ⟨↑y₁, S y₁⟩_ℂ = 1 := by simp_all [S, y₁, sub_apply, inner_smul]
 suffices ofReal '' unitInterval ⊆ 0 S by
 have hba : a = 1 - b := by linarith
 rw [numericalRange_smul, numericalRange_sub_const] at this
 obtain ⟨c, ⟨d, ⟨x, hx, hxd⟩, hdc⟩, hca⟩ := (image_subset_iff.mp this)
 simp only [mem_singleton_iff, exists_eq_left] at hca hdc hxd
 simp only [real_smul, smul_eq_mul] at *
 use x, hx
 simp_rw [hxd, hba, ofReal_sub, ofReal_one, ← hca, ← hdc]
 field_simp
 simp [mul_sub, mul_comm]
 -- First pick `θ` so that `y₂ = eiθy₁` satisfies `⟨y₀, S y₂⟩ + ⟨y₂, S y₀⟩ = 0`
 obtain ⟨θ, hθ⟩ := exists_phase_add_im_eq_zero ⟨↑y₀, S y₁⟩_ℂ ⟨↑y₁, S y₀⟩_ℂ
 let y₂ : S.domain := exp (I * θ) • y₁
 have hy₂ : ‖y₂‖ = 1 := by simp [y₂, norm_smul, hy₁]
 have hy_im : (⟨↑y₀, S y₂⟩_ℂ).im = -(⟨↑y₂, S y₀⟩_ℂ).im := by
 apply eq_neg_iff_add_eq_zero.mpr
 simp [← hθ, y₂, map_smul, SetLike.val_smul, inner_smul_left, inner_smul_right]
 have h₂ : ⟨↑y₂, S y₂⟩_ℂ = 1 := by
 simp [y₂, map_smul, inner_smul_left, inner_smul_right, h₁, ← exp_conj]
 -- `f` parametrizes the line connecting `y₀` and `y₂` and never vanishes
 -- are linearly independent (since `y₀ = λy₂` implies `0 = ⟨y₀, S y₀⟩ = 0`)
 let f : ℝ → S.domain := fun r ↦ (1 - r : ℂ) • y₀ + (r : ℂ) • y₂
 have hf : ∀ r, f r ≠ 0 := by
 intro r hr
 rcases eq_or_ne r 0 with rfl | hr'
 · exact (hy₁ ▸ zero_ne_one).symm (norm_eq_zero.mpr (by simp_all [f]))
 · apply (pow_ne_zero 2 hr').symm
 apply ofReal_inj.mp
 calc
 _ = (1 - r) ^ 2 * ⟨↑y₀, S y₀⟩_ℂ := by simp [h₀]
 _ = ⟨↑((1 - r : ℂ) • y₀), S ((1 - r : ℂ) • y₀)⟩_ℂ := by

```

1.162.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/BASIC.LEAN

```

 simp [map_smul, inner_smul_left, inner_smul_right, ← mul_assoc, pow_two]
 _ = (↑(-(r • y₂)), S (-(r • y₂)))_C := by simp [eq_neg_iff_add_eq_zero.mpr]
 simp [map_neg, ← Complex.coe_smul, map_smul, inner_smul_left, inner_smul_right]
 -- `g r = (f r)_C / ||f r||²` is real (by def of `θ`) and clearly in `θ`
 -- `g 0 = 0`, `g 1 = 1` and continuity ensure that all of `[0,1]` is also in `θ`
 let g : ℝ → ℝ := fun t ↦ (t ^ 2 + (1 - t) * t * (↑y₀, S y₂)_C + ↑y₂, S y₀)_C).
 have hg₀ : g 0 = 0 := by simp [g]
 have hg₁ : g 1 = 1 := by simp [g, f, coe_norm y₂ ▸ hy₂]
 have hg_cont : Continuous g := Continuous.div₀ (by fun_prop) (by fun_prop) (by s
 intro c (t, ht, htc)
 obtain (r, hr, hrt) := (hg₀ ▸ hg₁ ▸ intermediate_value_Icc zero_le_one hg_cont.c
 rw [← htc, ← hrt]
 refine (||f r||⁻¹ • f r, ?_, ?_)
 · simp only [mem_setOf_eq, norm_smul, norm_inv, norm_norm]
 exact inv_mul_cancel₀ (norm_ne_zero_iff.mpr (hf r))
 · have hf_sq : ofReal (||f r|| ^ 2) ≠ 0 := by simp [hf]
 simp_rw [← Complex.coe_smul, map_smul, SetLike.val_smul, inner_smul_left, inner
 ← mul_assoc, conj_ofReal, ← pow_two, ← ofReal_pow, inv_pow, ofReal_inv]
 apply (inv_mul_eq_iff_eq_mul₀ hf_sq).mpr
 simp_rw [g, ofReal_div, mul_div_cancel₀ _ hf_sq, add_comm (r ^ 2)]
 simp only [f, map_add, map_smul, coe_add, inner_add_left, inner_add_right,
 SetLike.val_smul, inner_smul_left, inner_smul_right, h₀, h₂]
 nth_rw 1 [← re_add_im (↑y₀, S y₂)_C, ← re_add_im (↑y₂, S y₀)_C]
 simp only [hy_im, mul_add, RingHom.map_sub, RingHom.map_one, conj_ofReal, mul
 zero_add, ofReal_neg, neg_mul, mul_neg, mul_one, add_re, ofReal_add, ofReal
 ofReal_sub, ofReal_one, ofReal_pow]
 ring

/-!
D. Spectrum of a closed operator
-/

/-!
D.1. Resolvent set
-/

/-- The resolvent set, `ρ`, of a partial linear map.

 A complex number `z` is in `ρ T` iff the linear map `T - z • 1` from `T.domain` to
 is a bijection with continuous (equivalently, bounded) inverse. -
/
def resolventSet (T : H →ₗ.[C] H) : Set C :=
 {z : C | (T - z • 1).toFun.ker = ⊥ ∧ (T - z • 1).toFun.range = T ∧ Continuous (↑ T

@[inherit_doc resolventSet]
scoped notation "ρ" => resolventSet

```

```

lemma resolventSet_eq (T : H →ℓ.[ℂ] H) :
 ρ T = {z | (T - z • 1).toFun.ker = ⊥ ∧ (T - z • 1).toFun.range = T ∧ Continuous.toFun.rfl}

lemma mem_resolventSet_iff {T : H →ℓ.[ℂ] H} {z : ℂ} :
 z ∈ ρ T ↔ (T - z • 1).toFun.ker = ⊥ ∧ (T - z • 1).toFun.range = T ∧ Continuous.toFun.rfl
 Iff.rfl

/-- If an operator is not closed then its resolvent set is empty. -
/
lemma resolventSet_eq_empty [CompleteSpace H] {T : H →ℓ.[ℂ] H} (h : ¬T.IsClosed) :
 ext z
 simp only [mem_empty_iff_false, iff_false]
 by_contra ⟨h_ker, h_range, h_cont⟩
 suffices (T - z • 1).IsClosed by
 have hTz : T - z • 1 + z • 1 = T :=
 eq_of_le_of_domain_eq (sub_add_le_cancel _) (by simp [add_domain, sub_def])
 exact h <| hTz > this.add_continuous (Continuous.const_smul (by fun_prop) z)
 apply (inverse_closed_iff h_ker).mpr
 apply (isClosed_iff_isClosed_domain_of_continuous h_cont).mpr
 simp [inverse_domain, h_range]

lemma resolventSet_subset_regularityDomain (T : H →ℓ.[ℂ] H) : ρ T ⊆ T.regularityDomain
 fun _ ⟨h_ker, _, h_cont⟩ ↦ mem_regularityDomain_iff.mpr ⟨h_ker, h_cont⟩

/-- For a closed operator the continuity of the resolvent is redundant
in the definition of the resolvent set. -/
lemma IsClosed.resolventSet_eq [CompleteSpace H] {T : H →ℓ.[ℂ] H} (hT : T.IsClosed) :
 ρ T = {z : ℂ | (T - z • 1).toFun.ker = ⊥ ∧ (T - z • 1).toFun.range = T}
 ext z
 rw [mem_resolventSet_iff, mem_setOf_eq, and_congr_right_iff, and_iff_left]
 intro h_ker h_range
 refine continuous_of_isClosed_domain ?_ ?_
 · apply (inverse_closed_iff h_ker).mpr
 exact hT.sub_continuous (Continuous.const_smul (by fun_prop) z) (by simp)
 · simp [inverse_domain, h_range]

/-- For a closed operator the resolvent set consists of those regular points
where the defect number is zero. -/
lemma IsClosed.resolventSet_eq' [CompleteSpace H] {T : H →ℓ.[ℂ] H} (hT : T.IsClosed) :
 ρ T = T.regularityDomain ∩ T.defectNumber⁻¹ {0} := by
 ext z
 constructor
 · intro hp
 have hz : z ∈ T.regularityDomain := T.resolventSet_subset_regularityDomain

```

1.162.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/BASIC.LEAN

```

 exact ⟨hz, (hT.defectNumber_eq_zero_iff hz).mpr hp.2.1⟩
 · intro ⟨h_reg, h_defect⟩
 obtain ⟨h_ker, h_cont⟩ := mem_regularityDomain_iff.mpr h_reg
 exact ⟨h_ker, (hT.defectNumber_eq_zero_iff h_reg).mpr h_defect, h_cont⟩

/-- The resolvent set is an open subset of $\mathbb{C}$. -/
lemma resolventSet_isOpen [CompleteSpace H] (T : H →ℓ.[$\mathbb{C}$] H) : IsOpen (ρ T) := by
 by_cases hT : T.IsClosed
 · rw [hT.resolventSet_eq']
 apply isOpen_iff_forall_mem_open.mpr
 intro z₁ hz₁
 refine ⟨connectedComponentIn T.regularityDomain z₁, fun z₂ hz₂ ↦ ⟨?_, ?_⟩, ?_, ?
 · exact connectedComponentIn_subset _ _ hz₂
 · simp_all [hT.isClosable.defectNumber_const hz₂]
 · exact T.regularityDomain_isOpen.connectedComponentIn
 · exact mem_connectedComponentIn hz₁.1
 · simp [resolventSet_eq_empty hT]

/-!
D.2. Spectrum
-/

/-- The spectrum, σ , of a partial linear map.

 σT is the complement of ρT . A complex number z is in σT iff the linear
from $T.\text{domain}$ to H fails to be bijective or $(T - z \cdot 1)^{-1}$ is not continuous
(equivalently, is not bounded). -/
def spectrum (T : H →ℓ.[$\mathbb{C}$] H) : Set $\mathbb{C}$:= (ρ T)c

@[inherit_doc spectrum]
scoped notation "σ" => spectrum

lemma spectrum_eq (T : H →ℓ.[$\mathbb{C}$] H) : σ T = (ρ T)c := rfl

lemma mem_spectrum_iff {T : H →ℓ.[$\mathbb{C}$] H} {z : $\mathbb{C}$ } :
 z ∈ σ T ↔ (T - z • 1).toFun.ker ≠ ⊥ ∨ (T - z • 1).toFun.range ≠ T ∨ ¬Continuous
 rw [spectrum_eq, mem_compl_iff, mem_resolventSet_iff]
 tauto

/-- If an operator is not closed then its spectrum is all of $\mathbb{C}$. -
/
lemma spectrum_eq_univ [CompleteSpace H] {T : H →ℓ.[$\mathbb{C}$] H} (h : ¬T.IsClosed) : σ T =
 compl_empty ▸ compl_inj_iff.mpr (resolventSet_eq_empty h)

/-- The spectrum is a closed subset of $\mathbb{C}$. -/
lemma spectrum_isClosed [CompleteSpace H] (T : H →ℓ.[$\mathbb{C}$] H) : _root_.IsClosed (σ T) :
```

```

 T.resolventSet_isOpen.isClosed_compl

/-!
D.2.1. Point spectrum
-/

/-- The point spectrum, σ^p , of a partial linear map.

 A complex number z is in $\sigma^p T$ iff $T - z \cdot 1$ is not injective. -
/
def pointSpectrum (T : H $\rightarrow$ $\mathbb{C}$ H) : Set $\mathbb{C}$:= {z : $\mathbb{C}$ | (T - z • 1).toFun.ker

@[inherit_doc pointSpectrum]
scoped notation " σ^p " => pointSpectrum

lemma pointSpectrum_eq (T : H $\rightarrow$ $\mathbb{C}$ H) : $\sigma^p T = \{z \mid (T - z \cdot 1).toFun.ker$
lemma mem_pointSpectrum_iff {T : H $\rightarrow$ $\mathbb{C}$ H} {z : $\mathbb{C}$ } : z $\in \sigma^p T \Leftrightarrow (T - z \cdot$
 Iff.rfl

lemma pointSpectrum_subset_spectrum (T : H $\rightarrow$ $\mathbb{C}$ H) : $\sigma^p T \subseteq \sigma T :=$
 fun _ h $\Rightarrow$ mem_spectrum_iff.mpr (Or.inl h)

/-!
D.2.2. Residual spectrum
-/

/-- The residual spectrum, σ^r , of a partial linear map.

 A complex number z is in $\sigma^r T$ iff $T - z \cdot 1$ is injective but not su
 and $(T - z \cdot 1)^{-1}$ is continuous (equivalently, bounded). -
/
def residualSpectrum (T : H $\rightarrow$ $\mathbb{C}$ H) : Set $\mathbb{C}$:=
 {z : $\mathbb{C}$ | (T - z • 1).toFun.ker = $\perp \wedge (T - z \cdot 1).toFun.range \neq T \wedge$ Contin

@[inherit_doc residualSpectrum]
scoped notation " σ^r " => residualSpectrum

lemma residualSpectrum_eq (T : H $\rightarrow$ $\mathbb{C}$ H) :
 $\sigma^r T = \{z \mid (T - z \cdot 1).toFun.ker = \perp \wedge (T - z \cdot 1).toFun.range \neq T \wedge$
 rfl

lemma mem_residualSpectrum_iff {T : H $\rightarrow$ $\mathbb{C}$ H} {z : $\mathbb{C}$ } :
 z $\in \sigma^r T \Leftrightarrow (T - z \cdot 1).toFun.ker = \perp \wedge (T - z \cdot 1).toFun.range \neq T \wedge$ Co
 Iff.rfl

```



1.162.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/BASIC.LEAN

```
lemma residualSpectrum_subset_spectrum (T : H →ℓ.[ℂ] H) : $\sigma^r T \subseteq \sigma T :=$
 fun _ (⟦_, h, _⟧) ↦ mem_spectrum_iff.mpr (Or.inr <| Or.inl h)
```

```
lemma residualSpectrum_subset_regularityDomain (T : H →ℓ.[ℂ] H) : $\sigma^r T \subseteq T.\text{regularityDomain}$
 fun _ hz ↦ mem_regularityDomain_iff.mpr (hz.1, hz.2.2)
```

/-!

#### D.2.3. Continuous spectrum

-/

/-- The continuous spectrum,  $\sigma^c$ , of a partial linear map.

A complex number  $z$  is in  $\sigma^c T$  iff the range of  $T - z \cdot 1$  is not closed. -  
/

```
def continuousSpectrum (T : H →ℓ.[ℂ] H) : Set ℂ :=
 {z : ℂ | ¬_root_.IsClosed ((T - z • 1).toFun.range : Set H)}
```

```
@[inherit_doc continuousSpectrum]
scoped notation "σc" => continuousSpectrum
```

```
lemma continuousSpectrum_eq (T : H →ℓ.[ℂ] H) :
 $\sigma^c T = \{z \mid \neg_root_.IsClosed ((T - z \cdot 1).toFun.range : Set H)\} := rfl$
```

```
lemma mem_continuousSpectrum_iff {T : H →ℓ.[ℂ] H} {z : ℂ} :
 $z \in \sigma^c T \leftrightarrow \neg_root_.IsClosed ((T - z \cdot 1).toFun.range : Set H) := Iff.rfl$
```

```
lemma continuousSpectrum_subset_spectrum (T : H →ℓ.[ℂ] H) : $\sigma^c T \subseteq \sigma T :=$
 fun _ h (⟦_, h_range, _⟧) ↦ h (by simp [h_range])
```

/-!

### D.3. Spectrum decomposition

-/

```
lemma IsClosed.spectrum_eq [CompleteSpace H] {T : H →ℓ.[ℂ] H} (hT : T.IsClosed) :
 $\sigma T = \sigma^p T \cup \sigma^r T \cup \sigma^c T :=$ by
 refine Subset.antisymm ?_ ?_
 · intro z hσ
 apply mem_spectrum_iff.mp at hσ
 rcases eq_or_ne (T - z • 1).toFun.ker 1 with h_ker | h_ker
 · by_cases h_cont : Continuous (□ T z)
 · left; right; exact (h_ker, (hσ.neg_resolve_left h_ker).neg_resolve_right h_c
 · right
 rw [mem_continuousSpectrum_iff, ← inverse_domain]
 refine fun h ↦ h_cont ?_
 refine continuous_of_isClosed_domain ?_ h
 apply (inverse_closed_iff h_ker).mpr
```

```

 exact hT.sub_continuous (Continuous.const_smul (by fun_prop) _) le_
 · left; left; exact h_ker
 · refine union_subset ?_ T.continuousSpectrum_subset_spectrum
 exact union_subset T.pointSpectrum_subset_spectrum T.residualSpectrum_s

lemma pointSpectrum_inter_residualSpectrum (T : H →ℓ.[$\mathbb{C}$] H) : $\sigma^p T \cap \sigma^r T =$
 ext
 simp only [mem_inter_iff, mem_empty_iff_false, iff_false, not_and]
 exact fun h h' ↦ h h'.1

/-!
E. Resolvent identities
-/

lemma resolvent_sub
 {T1 T2 : H →ℓ.[$\mathbb{C}$] H} (hT : T2.domain ≤ T1.domain) {z : $\mathbb{C}$ } (hz1 : z ∈ ρ
 □ T1 z - □ T2 z = □ T1 z * (T2 - T1) * □ T2 z := by
 symm
 calc
 _ = □ T1 z ∘r ((T2 - z • 1 - (T1 - z • 1)) ∘r □ T2 z) := by
 rw [mul_assoc]
 congr 2
 exact (eq_of_le_of_domain_eq (sub_sub_sub_le_cancel_right _ _ _)) (by
 _ = □ T1 z ∘r ((T2 - z • 1) ∘r □ T2 z - (T1 - z • 1) ∘r □ T2 z) := by
 congr
 exact sub_compRestricted _ _ _
 _ = □ T1 z ∘r (1 - (T1 - z • 1) ∘r □ T2 z) := by
 congr
 rw [compRestricted_inverse_eq hz2.1, inverse_domain, hz2.2.1]
 simp [eq_of_le_of_domain_eq domRestrict_le]
 _ = □ T1 z - □ T1 z ∘r ((T1 - z • 1) ∘r □ T2 z) := by
 nth_rw 2 [← mul_one (□ T1 z)]
 refine (eq_of_le_of_domain_eq (compRestricted_sub_ge _ _ _) ?_).symm
 simp [sub_domain, compRestricted_domain, inverse_domain, hz1.2]
 _ = □ T1 z - (domRestrict 1 T1.domain) ∘r □ T2 z := by
 simp [← compRestricted_assoc, inverse_compRestricted_eq hz1.1, sub_do
 _ = □ T1 z - □ T2 z := by
 ext x
 · suffices □ T2 z ⟨x, by simp [inverse_domain, hz2.2]⟩ ∈ T1.domain by
 simp [sub_domain, mem_compRestricted_domain_iff, inverse_domain,
 have hR2 : (□ T2 z).toFun.range = T2.domain := by simp [inverse_ran
 exact hT (hR2 ▸ mem_range_self _)
 · rfl

lemma resolvent_sub' {T : H →ℓ.[$\mathbb{C}$] H} (z1 z2 : $\mathbb{C}$) (hz1 : z1 ∈ ρ T) (hz2 : z
 □ T z1 - □ T z2 = (z1 - z2) • (□ T z1 * □ T z2) := by

```

1.163.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/SELFADJOINT.LEAN

```

rcases eq_or_ne z1 z2 with rfl | hz
· ext
 · simp [sub_domain, inverse_domain, hz1.2, mul_def, compRestricted_domain]
 · simp [sub_apply]
· let S := T + (z1 - z2) • 1
 have h_domain : S.domain = T.domain := by simp [S, add_domain]
 have hST : S - z1 • 1 = T - z2 • 1 := by
 ext
 · simp [sub_domain, h_domain]
 · simp [S, sub_apply, add_apply, sub_smul, add_sub_assoc, ← sub_eq_add_neg]
 have hR : $\square T z2 = \square S z1$:= by simp [resolvent, hST]
 have hz1' : $z1 \in \rho S := \langle hST \triangleright hz2.1, hST \triangleright hz2.2.1, hR \triangleright hz2.2.2 \rangle$
 suffices (S - T) $\circ_r \square S z1 = (z1 - z2) \cdot \square S z1$ by
 rw [hR, resolvent_sub h_domain.le hz1 hz1']
 simp only [mul_def, compRestricted_assoc, this, compRestricted_smul (sub_ne_zero)]
 calc
 _ = ((z1 - z2) • domRestrict 1 (S - z1 • 1).domain) $\circ_r \square S z1$:= by
 congr
 ext
 · simp [h_domain, sub_domain]
 · simp only [sub_apply, add_apply, add_sub_cancel_left, S]
 rfl
 _ = (z1 - z2) • (domRestrict 1 (S - z1 • 1).domain $\circ_r \square S z1$) := smul_compRest
 _ = (z1 - z2) • $\square S z1$:= by
 congr
 ext
 · simp [mem_compRestricted_domain_iff, ← inverse_range hz1'.1]
 · rfl

end

end LinearPMap

```

## 1.163 Physlib/QuantumMechanics/DDimensions/Operators/SpectralTheory/SelfAdjoint

`/-`

Copyright (c) 2026 Gregory J. Loges. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Gregory J. Loges

`-/`

module

public import Physlib.QuantumMechanics.DDimensions.Operators.SpectralTheory.Symmetri

`/-!`

```

Spectral theory for self-adjoint operators

i. Overview

In this module we develop the spectral theory for self-adjoint operators.

ii. Key results

- `resolventSet_eq_regularityDomain` : The resolvent set and regularity domain
 if $T - z \cdot 1$ has a continuous (equivalently, bounded) inverse then it
- `spectrum_real` : The spectrum of a self-adjoint unbounded operator is real

iii. Table of contents

- A. Resolvent set
- B. Spectrum

iv. References

-/

@[expose] public section

namespace LinearPMap
namespace IsSelfAdjoint

open Complex
open ComplexConjugate
open Set

variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℂ H] [CompleteMetricSpace H]
variable {T : H →ℂ H} (hT : IsSelfAdjoint T)
include hT

/-!
A. Resolvent set
-/

lemma resolventSet_eq_regularityDomain : ρ T = T.regularityDomain := by
 refine Subset.antisymm T.resolventSet_subset_regularityDomain fun z hz =>
 obtain ⟨h_ker, h_cont⟩ := mem_regularityDomain_iff.mp hz
 refine ⟨h_ker, ?_, h_cont⟩
 have h_ker' : (T - conj z • 1).toFun.ker = ⊥ := by
 by_cases hz' : conj z = z
 · exact hz'.symm ▸ h_ker

```

1.163.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/SELFADJOINT.LEAN

```

 · suffices conj $z \notin \sigma^p T$ by simp_all [pointSpectrum_eq]
 refine fun h $\mapsto$ hz' ?_
 obtain (r, hr) := (isSymmetric hT).pointSpectrum_real h
 simp [show z = conj (↑r) from by simp [hr]]
 have h_orthog := (isUnbounded hT).orthogonal_adjoint_sub_ker hz
 rw [isSelfAdjoint_def.mp hT, (isClosed hT).closure_eq, h_ker'] at h_orthog
 simp [← h_orthog]

/-- `(T - z • 1).range = T` is a sufficient condition for ` $z \in \rho T$ `
 (and it is a necessary condition by definition of ` $\rho$ `). -
/
lemma mem_resolventSet_of_range_eq_top {z : ℂ} (h : (T - z • 1).toFun.range = T) : z
 by_cases hz_im : z.im = 0
 · rw [(isClosed hT).resolventSet_eq]
 refine {?, h}
 have h_orthog := (isUnbounded hT).orthogonal_closure_sub_range z
 rw [isSelfAdjoint_def.mp hT, (isClosed hT).closure_eq, conj_eq_iff_im.mpr hz_im,
 Submodule.top_orthogonal_eq_bot, Eq.comm, Submodule.ext_iff] at h_orthog
 ext x
 specialize h_orthog x
 simp_all
 · rw [resolventSet_eq_regularityDomain hT]
 exact (isSymmetric hT).mem_regularityDomain_of_im_ne_zero hz_im

/-!
B. Spectrum
-/

/-- The spectrum of a self-adjoint operator is real. -/
lemma spectrum_real : $\sigma T \subseteq \text{range ofReal}$:= by
 rw [spectrum_eq, resolventSet_eq_regularityDomain hT]
 exact compl_subset_comm.mp (isSymmetric hT).compl_ofReal_subset_regularityDomain

/-- The residual spectrum of a self-adjoint operator is empty. -
/
lemma residualSpectrum_eq_empty : $\sigma^r T = \emptyset$:= by
 apply eq_empty_of_subset_empty
 refine inter_compl_self (ρT) $\triangleright$ subset_inter ?_ ?_
 · exact resolventSet_eq_regularityDomain hT $\triangleright$ T.residualSpectrum_subset_regularity
 · exact T.spectrum_eq $\triangleright$ T.residualSpectrum_subset_spectrum

end IsSelfAdjoint
end LinearPMap

```

**1.164 Physlib/QuantumMechanics/DDimensions/Operators/SpectralTh**

```

/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
module

public import Mathlib.Analysis.InnerProductSpace.Adjoint
public import Mathlib.MeasureTheory.VectorMeasure.Basic
/-!

Spectral measures

i. Overview

A spectral measure μ_S on a measurable space α is a σ -
additive function $\text{Set } \alpha \rightarrow H \rightarrow L[\mathbb{C}] H$
such that each set is mapped to a star projection on H , the empty set and
measurable sets are
mapped to zero, and univ is mapped to the identity.
This is implemented as a structure extending $\text{VectorMeasure } \alpha (H \rightarrow L[\mathbb{C}] H)$
constraining $\mu_S A$ to be a star projection for each set A and $\mu_S \text{univ} =$

For each $x : H$ there is an associated measure μ_x given by $\mu_x A = \|\mu_S A x\|^2$

ii. Key results

- SpectralMeasure : A star projection-valued measure.
- comp_eq_of_inter : For a spectral measure μ_S and measurable sets A
 the composition $\mu_S A \circ \mu_S B = \mu_S (A \cap B)$.

iii. Table of contents

- A. Definition
- B. Composition

iv. References

-/

@[expose] public section

noncomputable section

```

1.164.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/SPECTRALMEASURE.LEAN

```
open ContinuousLinearMap
open MeasureTheory
open Set
```

```
instance (H : Type*) [SeminormedAddCommGroup H] [InnerProductSpace ℂ H] :
 IsAddTorsionFree (H →L[ℂ] H) where
 nsmul_right_injective n hn := by
 refine Function.HasLeftInverse.injective ?_
 use fun f ↦ (n : ℂ)-1 • f
 intro; ext; simp [← Nat.cast_smul_eq_nsmul ℂ, smul_smul, Nat.cast_ne_zero (R :=
```

```
/-!
A. Definition
-/
```

```
/-- A _spectral measure_ on a measurable space α is a σ -
additive function Set α → H →L[ℂ] H
such that each set is mapped to a star projection on H, the empty set and non-
measurable sets
are mapped to zero, and univ is mapped to the identity. -
/
```

```
structure SpectralMeasure
 (α : Type*) [MeasurableSpace α]
 (H : Type*) [NormedAddCommGroup H] [InnerProductSpace ℂ H] [CompleteSpace H]
 extends VectorMeasure α (H →L[ℂ] H) where
 isStarProjection' : ∀ A, IsStarProjection (measureOf' A)
 univ' : measureOf' univ = 1
```

```
namespace SpectralMeasure
```

```
variable {α : Type*} [MeasurableSpace α]
variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℂ H] [CompleteSpace H]
variable (μS : SpectralMeasure α H)
```

```
attribute [coe] toVectorMeasure
```

```
instance instCoeVectorMeasure : Coe (SpectralMeasure α H) (VectorMeasure α (H →L[ℂ] H)) :=
 ⟨toVectorMeasure⟩
```

```
instance instCoeFun : CoeFun (SpectralMeasure α H) fun _ ↦ Set α → H →L[ℂ] H :=
 ⟨fun μS ↦ μS.toVectorMeasure.measureOf'⟩
```

```
lemma isStarProjection (A : Set α) : IsStarProjection (μS A) := μS.isStarProjection'
```

```
@[simp]
lemma univ : μS univ = 1 := μS.univ'
```

```

/-!
B. Composition
-/

@[simp]
lemma comp_self (A : Set α) : μS A $\circ_L$ μS A = μS A := (μS .isStarProjection A)

lemma comp_of_disjoint
 {A B : Set α } (h : Disjoint A B) (hA : MeasurableSet A) (hB : MeasurableSet B) :
 μS A $\circ_L$ μS B = 0 := by
 suffices μS A $\circ_L$ μS (A $\cup$ B) = μS A by simp_all [μS .of_union]
 refine (IsStarProjection.sub_iff_mul_eq_left ?_ ?_).mp ?_
 · exact μS .isStarProjection A
 · exact μS .isStarProjection (A $\cup$ B)
 · rw [μS .of_union h hA hB, add_sub_cancel_left]
 exact μS .isStarProjection B

lemma comp_eq_of_inter {A B : Set α } (hA : MeasurableSet A) (hB : MeasurableSet B) :
 μS A $\circ_L$ μS B = μS (A $\cap$ B) := by
 nth_rw 1 [← inter_union_sdiff B A, ← inter_union_sdiff A B]
 simp only [μS .of_union, hA.inter hB, hB.inter hA, hA.diff hB, hB.diff hA,
 disjoint_sdiff_inter.symm, add_comp, comp_add]
 rw [inter_comm B A, μS .comp_of_disjoint disjoint_sdiff_inter (hA.diff hB)]
 rw [inter_comm A B, μS .comp_of_disjoint disjoint_sdiff_inter.symm (hB.diff hA)]
 simp [μS .comp_of_disjoint disjoint_sdiff_sdiff (hA.diff hB) (hB.diff hA)]

lemma commute (A B : Set α) : Commute (μS A) (μS B) := by
 by_cases hAB : MeasurableSet A $\wedge$ MeasurableSet B
 · simp [commute_iff_eq, mul_def, comp_eq_of_inter, hAB, inter_comm]
 · rcases not_and_or.mp hAB with hA | hB
 · simp [hA]
 · simp [hB]

end SpectralMeasure

end

```

### 1.165 Physlib/QuantumMechanics/DDimensions/Operators/SpectralTheory

```

/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges

```



1.165.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/SYMMETRIC.LEAN

-/  
module

public import Physlib.QuantumMechanics.DDimensions.Operators.SpectralTheory.Basic  
/-!

# Spectral theory for symmetric operators

## i. Overview

In this module we develop the spectral theory for symmetric operators.

The numerical range of an operator,  $\theta T = \{\lambda x, T x \in \mathbb{C} \mid x \in T.\text{domain} \wedge \|x\| = 1\}$ , is the set of all  $\lambda \in \mathbb{C}$  such that  $\lambda x = T x$  for some  $x$  with  $\|x\| = 1$ . For symmetric operators the numerical range consists only of real numbers and it is to discuss its upper/lower bounds. To facilitate this, we define `LinearPMap.realNumericalRange` as the projection of `LinearPMap.numericalRange` onto the real axis. For symmetric operators this simply reinterprets the numerical range as a subset of  $\mathbb{R}$ .

## ii. Key results

- `realNumericalRange` (`θre`) : The projection of the numerical range onto the real axis.
- `compl_ofReal_subset_regularityDomain` : The regularity domain of a symmetric operator is the complement of the numerical range, i.e. all complex numbers with non-zero imaginary part.
- `regularityDomain_isConnected_iff` : The regularity domain of a symmetric operator is connected if and only if it contains a real number.

## iii. Table of contents

- A. Numerical range
- B. Regularity domain

## iv. References

-/

@[expose] public section

namespace LinearPMap  
namespace IsSymmetric

open InnerProductSpace  
open Complex  
open Set

variable {H : Type\*} [NormedAddCommGroup H] [InnerProductSpace  $\mathbb{C}$  H]

```

/-!
A. Numerical range
-/

/-- The projection of the numerical range onto the real axis. -
/
def realNumericalRange (T : H →ℂ H) : Set ℝ := re '' (θ T)

@[inherit_doc realNumericalRange]
local notation "θre" => realNumericalRange

lemma realNumericalRange_eq (T : H →ℂ H) : θre T = re '' θ T := rfl

@[simp]
lemma realNumericalRange_neg (T : H →ℂ H) : θre (-T) = -
θre T := by
 ext
 simp [realNumericalRange_eq, neg_eq_iff_eq_neg]

variable {T : H →ℂ H} (hT : T.IsSymmetric)
include hT

/-- The numerical range of a symmetric operator is contained in the real axis. -
/
lemma im_eq_zero_of_mem_numericalRange {z : ℂ} (hz : z ∈ θ T) : z.im = 0 :=
 obtain {x, hx, hxz} := hz
 simp only [← hT x x] at hxz
 exact conj_eq_iff_im.mp (hxz ▸ isSymmetric_iff_inner_map_self_real.mp hT)

/-- The numerical range of a symmetric operator is contained in the real axis. -
/
lemma numericalRange_subset : θ T ⊆ range ofReal :=
 fun z hz => {z.re, Complex.ext rfl (hT.im_eq_zero_of_mem_numericalRange hz)}

/-- The numerical range of a symmetric operator is equal to its projection. -
/
lemma numericalRange_eq : θ T = ofReal '' θre T := by
 rw [realNumericalRange_eq, ← image_comp]
 exact (image_id _).symm.trans
 (image_congr fun z hz => Complex.ext rfl (hT.im_eq_zero_of_mem_numericalRange hz))

lemma closure_numericalRange_subset : _root_.closure (θ T) ⊆ range ofReal :=
 (closure_mono hT.numericalRange_subset).trans
 Complex.isometry_ofReal.isClosedEmbedding.isClosed_range.closure_subset

/-!

```

1.165.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/SYMMETRIC.LEAN

## B. Regularity domain

-/

/-- The regularity domain of a symmetric operator contains all complex numbers with zero

imaginary part. -/

lemma mem\_regularityDomain\_of\_im\_ne\_zero {z : ℂ} (hz : z.im ≠ 0) : z ∈ T.regularityDomain

apply T.compl\_closure\_numericalRange\_subset\_regularityDomain

refine (mem\_compl\_iff \_).mpr fun h ↦ hz ?\_

obtain {r, \_, rfl} := hT.closure\_numericalRange\_subset h

exact ofReal\_im r

/-- The regularity domain of a symmetric operator contains all complex numbers with zero

imaginary part. -/

lemma compl\_ofReal\_subset\_regularityDomain : (range ofReal)<sup>c</sup> ⊆ T.regularityDomain :=

fun z hz ↦ hT.mem\_regularityDomain\_of\_im\_ne\_zero fun h ↦ hz {z.re, Eq.symm (Comple

/-- If `m` is a lower bound on the numerical range then the regularity domain contains

ofReal '' Iio m ⊆ T.regularityDomain := by

intro z {r, hr, hrz}

refine {m - r, sub\_pos.mpr hr, fun x ↦ ?\_}

rcases eq\_zero\_or\_neZero x with rfl | hx

· simp

· obtain {s, hs, hs'} := hT.numericalRange\_eq ▸ mem\_numericalRange hx.ne

apply h at hs

have hsr : r < s := lt\_of\_lt\_of\_le hr hs

refine le\_of\_mul\_le\_mul\_right ?\_ (norm\_pos\_iff.mpr hx.ne)

calc

\_ = (m - r) \* ||x|| ^ 2 := by rw [mul\_assoc, pow\_two]

\_ ≤ (s - r) \* ||x|| ^ 2 := by nlinarith

\_ = ||s \* ||x|| ^ 2 - r \* ||x|| ^ 2|| := by simp [← sub\_mul, abs\_of\_pos, sub\_pos, hs

\_ = ||(s : ℂ) \* ||x|| ^ 2 - r \* ||x|| ^ 2|| := by simp [← ofReal\_pow, ← ofReal\_mul,

\_ = ||↑x, T x - r \* ||x|| ^ 2|| := by simp [hs', mul\_comm, hx.ne]

\_ = ||↑x, T x - z \* x|| := by simp [inner\_sub\_right, inner\_smul\_right, hrz]

\_ ≤ ||T x - z \* x|| \* ||x|| := mul\_comm \_ ||x|| ▸ norm\_inner\_le\_norm \_

/-- If `m` is an upper bound on the numerical range then the regularity domain contains

ofReal '' Ioi m ⊆ T.regularityDomain := by

intro z {r, hr, hrz}

rw [← neg\_mem\_neg, ← regularityDomain\_neg]

refine hT.neg.Iio\_subset\_regularityDomain (m := -m) ?\_ ?\_

```

 · exact fun _ _ ↦ by simp_all [neg_le.mp, upperBounds]
 · exact ⟨-r, by simp [mem_Ioi.mp hr], by simp [hrz]⟩

/-- The regularity domain of a symmetric operator is connected iff it contains
/
lemma regularityDomain_isConnected_iff :
 IsConnected T.regularityDomain ↔ (ofReal -1 T.regularityDomain).Nonempty
rw [T.regularityDomain_isOpen.isConnected_iff_isPathConnected]
constructor
· intro h
 obtain ⟨f, hf⟩ : JoinedIn T.regularityDomain (-1) I :=
 h.joinedIn _ (hT.mem_regularityDomain_of_im_ne_zero (by simp)) _
 (hT.mem_regularityDomain_of_im_ne_zero (by simp))
 have hIVT := intermediate_value_Icc (f := fun t ↦ (f t).im) zero_le_one
 simp only [Path.source, neg_im, I_im, Path.target] at hIVT
 obtain ⟨t, _, ht⟩ := hIVT (show (0 : ℝ) ∈ Icc (-1) 1 by simp)
 specialize hf t
 rw [← re_add_im (f t), show (f t).im = 0 from ht] at hf
 exact ⟨(f t).re, by simp_all⟩
· intro ⟨r, hr⟩
 apply mem_preimage.mp at hr
 refine isPathConnected_iff.mpr {?, fun z₁ hz₁ z₂ hz₂ ↦ ?_}
 · exact ⟨I, hT.mem_regularityDomain_of_im_ne_zero (by simp)⟩
 · have h : ∀ z ∈ T.regularityDomain, JoinedIn T.regularityDomain z r :=
 intro z hz
 by_cases hz_im : z.im = 0
 · rcases eq_or_ne z r with rfl | hzr
 · exact JoinedIn.refl hr
 · let path : Path z r := {
 toFun t := ((z + r) + (z - r) * cexp (Real.pi * t * I)) / 2
 source' := by simp
 target' := by simp [add_comm z r, add_add_sub_cancel]
 }
 refine ⟨path, fun t ↦ ?_⟩
 by_cases! ht : ∃ n : ℤ, n = (t : ℝ)
 · obtain ⟨n, htn⟩ := ht
 have hexp : cexp (Real.pi * t * I) = (-1) ^ n := by
 rw [← exp_pi_mul_I, ← exp_int_mul, ← htn]
 exact congrArg cexp (by push_cast; ring)
 rcases Int.even_or_odd n with he | ho
 · simp [path, hexp, he.neg_one_zpow, hz]
 · simp [path, hexp, ho.neg_one_zpow, add_comm z r, hr]
 · have hzr' : z.re - r ≠ 0 :=
 fun h ↦ hzr <| Complex.ext (sub_eq_zero.mp h) (by simp [h])
 refine hT.mem_regularityDomain_of_im_ne_zero ?_
 simp [path, hz_im, exp_im, Real.sin_eq_zero_iff, mul_comm, hzr']

```

1.165.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/SPECTRALTHEORY/SYMMETRIC.LEAN

```

 · refine JoinedIn.of_segment_subset fun w ⟨a, b, _, _, _, hw⟩ ↦ ?_
 rcases eq_zero_or_neZero a with rfl | _
 · simp_all
 · exact hT.mem_regularityDomain_of_im_ne_zero fun _ ↦ hz_im (by simp_all [
exact (h z₁ hz₁).trans (h z₂ hz₂).symm

/-- The regularity domain of a lower semibounded symmetric operator is connected. -
/
lemma regularityDomain_isConnected_of_bddBelow (h : BddBelow (0ᵣₑ T)) :
 IsConnected T.regularityDomain := by
 obtain ⟨m, hm⟩ := h
 apply hT.regularityDomain_isConnected_iff.mpr
 exact ⟨m - 1, hT.Iio_subset_regularityDomain hm ⟨m - 1, by simp⟩⟩

/-- The regularity domain of an upper semibounded symmetric operator is connected. -
/
lemma regularityDomain_isConnected_of_bddAbove (h : BddAbove (0ᵣₑ T)) :
 IsConnected T.regularityDomain := by
 obtain ⟨m, hm⟩ := h
 apply hT.regularityDomain_isConnected_iff.mpr
 exact ⟨m + 1, hT.Ioi_subset_regularityDomain hm ⟨m + 1, by simp⟩⟩

/-!
C. Point spectrum
-/

/-- Eigenvalues of a symmetric unbounded operator are real. -
/
lemma pointSpectrum_real : σᵖ T ⊆ range ofReal := by
 intro z hz
 apply hT.numericalRange_subset
 obtain ⟨x, hx, hx₀⟩ := (Submodule.ne_bot_iff _).mp hz
 suffices z = (||x|| ^ 2)⁻¹ * ↑x, T ⟨x, x.2.1⟩ ⊆ ℂ by
 exact this ▸ mem_numericalRange (T := T) (x := ⟨x, x.2.1⟩) (by simp [hx₀])
 rw [ofReal_inv]
 refine (eq_inv_mul_iff_mul_eq₀ (by simp [hx₀])).mpr (Eq.symm ?_)
 calc
 _ = ↑x, (T - z • 1) x + z • x ⊆ ℂ := by simp [sub_apply]
 _ = ↑x, z • x ⊆ ℂ := by simp [← toFun_eq_coe, LinearMap.mem_ker.mp hx]
 _ = ↑(||x|| ^ 2) * z := by simp [inner_smul_right, mul_comm]

end IsSymmetric
end LinearPMap

```

### 1.166 Physlib/QuantumMechanics/DDimensions/Operators/Unbounded.

```
public import Physlib.Mathematics.LinearPMap
```

The appropriate mathematical objects for discussing operators in non-relativistic quantum mechanics

are partially-defined linear map (`LinearPMap`) between complex Hilbert spaces. An import class of operators in NRQM are those which are both densely defined and closed, which we refer to as `_unbounded_`. When `H = H'` operators may also be symmetric or self-adjoint.

essentially self-adjoint (closure is self-adjoint).

In this module we collect results on how the properties ``HasDenseDomain``, ``IsSymmetric``, ``IsSelfAdjoint`` and ``IsEssentiallySelfAdjoint`` interact with operations, closure, adjoints and each other.

Definitions

- ``HasDenseDomain`` : An operator ``U : H  $\rightarrow$   $\mathbb{C}$`  `H'` has dense domain if ``U.d`
- ``IsUnbounded`` : An operator is unbounded if it is both densely defined and not bounded.
- ``IsSymmetric`` : An operator ``T : H  $\rightarrow$   $\mathbb{C}$`  `H'` is symmetric if ``[T x, y]_C = [x, T y]_C` for all ``x y : T.domain``.
- ``IsEssentiallySelfAdjoint`` : An operator ``T : H  $\rightarrow$   $\mathbb{C}$`  `H'` is essentially self-adjoint if its closure is self-adjoint.

Results

- A. Definitions
- B. Basic properties
  - B.1. Dense domain
  - B.2. Closability
  - B.3. Adjoints
  - B.4. Continuity / boundedness
- C. Classes of operators
  - C.1. Symmetric operators
  - C.2. Self-adjoint operators
  - C.3. Essentially self-adjoint operators
  - C.4. Unbounded operators
- [Reed and Simon, \*Methods of Modern Mathematical Physics, Vol. I: Functional Analysis\*]
- [Konrad Schmüdgen, \*Unbounded Self-Adjoint Operators on Hilbert Space\*]

```
A. Definitions
```

```
/-- A LinearPMap `T` is symmetric iff `[T x, y]_C = [x, T y]_C` for all `x`
/
```

```
B. Basic properties
```

```
-/
```

```
/-!
```

```
B.1. Dense domain
```

```
/-- `U.range` = $U^\dagger.ker$
```

1.166.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED3 LEAN

```

 c.f. `LinearMap.orthogonal_range` and `ContinuousLinearMap.orthogonal_range` -
 /
lemma HasDenseDomain.orthogonal_range [CompleteSpace H] (h : U.HasDenseDomain) :
 U.toFun.range[] = U†.toFun.ker.map U†.domain.subtype := by
 ext u
 simp only [mem_orthogonal', Subtype.exists, mem_map, LinearMap.mem_ker, subtype_ap
 exists_and_right, exists_eq_right, toFun_eq_coe]
 constructor
 · intro h'
 exact ⟨mem_adjoint_domain_of_exists u ⟨0, by simp [h']⟩, adjoint_apply_eq h _ (b
 · intro ⟨hu, hu'⟩ v ⟨x, hxv⟩
 simp [← hxv, ← adjoint_isFormalAdjoint h ⟨u, hu⟩, hu']

/-- `U†.ker[] = U.range.closure` -/
lemma HasDenseDomain.orthogonal_adjoint_ker [CompleteSpace H] [CompleteSpace H']
 (h : U.HasDenseDomain) :
 (U†.toFun.ker.map U†.domain.subtype)[] = U.toFun.range.closure :=
 h.orthogonal_range ▸ orthogonal_orthogonal_eq_closure _

B.2. Closability
lemma IsClosed.closure_eq (h : U.IsClosed) : U.closure = U :=
 eq_of_eq_graph (h.isClosable.graph_closure_eq_closure_graph ▸ h.submodule_topologi

lemma IsClosable.isClosed_iff (h : U.IsClosable) : U.IsClosed ↔ U.closure = U :=
 ⟨IsClosed.closure_eq, fun h' ↦ h' ▸ h.closure_isClosed⟩
B.3. Adjoints
@[simp]
lemma adjoint_zero [CompleteSpace H] : (0 : H →ℓ.[$\mathbb{C}$] H')† = 0 := adjoint_of_zero rfl

lemma adjoint_sub_le_sub_adjoint [CompleteSpace H]
 (U1 U2 : H →ℓ.[$\mathbb{C}$] H') (h12 : (U1 - U2).HasDenseDomain) : U1† - U2† ≤ (U1 - U2)†
 simp only [sub_eq_add_neg, ← adjoint_neg]
 exact adjoint_add_le_add_adjoint U1 (-U2) h12

B.4. Continuity / boundedness
-/

/-- `f : E →ℓ[ $\mathbb{C}$ ] F` is continuous iff there exists `M > 0` s.t. `||f x|| ≤ M * ||x||` fo

 This is a (convenient) immediate consequence of
 `IsBoundedLinearMap.isLinearMap_and_continuous_iff_isBoundedLinearMap`. -
 /
lemma _root_.LinearMap.continuous_iff_bounded {E F : Type*} [NontriviallyNormedFie
 [SeminormedAddCommGroup E] [NormedSpace $\mathbb{C}$ E] [SeminormedAddCommGroup F] [NormedS
 {f : E →ℓ[$\mathbb{C}$] F} : Continuous f ↔ ∃ M, 0 < M ∧ ∀ x : E, ||f x|| ≤ M * ||x|| := by

```

```

 refine (and_congr_right_iff (a := IsLinearMap □ f)).mp ?_ f.isLinear
 rw [← isBoundedLinearMap_iff]
 exact IsBoundedLinearMap.isLinearMap_and_continuous_iff_isBoundedLinearMa

/-- Continuous operators are closable. -/
lemma isClosable_of_continuous (h : Continuous U) : U.IsClosable := by
 use U.graph.topologicalClosure.toLinearPMap
 refine (toLinearPMap_graph_eq _ fun x hx hx₁ ↦ ?_).symm
 obtain ⟨b, hb, hb⟩ := mem_closure_iff_seq_limit.mp hx
 rw [nhds_prod_eq] at hb
 apply norm_eq_zero.mp
 apply tendsto_nhds_unique hb.snd.norm
 obtain ⟨M, hM, h_bound⟩ := LinearMap.continuous_iff_bounded.mp h
 refine squeeze_zero (g := fun n ↦ M * ||(b n).1||) (fun _ ↦ norm_nonneg _)
 · obtain ⟨y, hy₁, hy₂⟩ := (mem_graph_iff _).mp (hb n)
 simp only [← hy₁, ← hy₂]
 exact h_bound y
 · exact mul_zero M ▸ (norm_eq_zero.mpr hx₁) ▸ hb.fst.norm.const_mul M

/-- A strengthening of `closure_domain_le_domain_closure` for continuous op
/
lemma closure_domain_eq_domain_closure_of_continuous [CompleteSpace H'] (h :
 U.closure.domain = U.domain.closure := by
 refine eq_of_le_of_ge U.closure_domain_le_domain_closure fun x hx ↦ ?_
 obtain ⟨M, hM, h_bound⟩ := LinearMap.continuous_iff_bounded.mp h
 obtain ⟨b, hb, hb'⟩ := mem_closure_iff_seq_limit.mp hx
 simp only [coe_toAddSubmonoid, SetLike.mem_coe] at hb
 let Ub : ℕ → H' := fun n ↦ U ⟨b n, hb n⟩
 have hCS : CauchySeq Ub := by
 refine Metric.cauchySeq_iff'.mpr fun ε hε ↦ ?_
 obtain ⟨N, hN⟩ := Metric.cauchySeq_iff'.mp hb'.cauchySeq (M⁻¹ * ε) (by
 refine ⟨N, fun n hn ↦ ?_⟩
 refine lt_of_le_of_lt ?_ ((lt_inv_mul_iff₀ hM).mp (hN n hn))
)
 calc
 _ = ||Ub n - Ub N|| := dist_eq_norm _ _
 _ = ||U ⟨b n, hb n⟩ - U ⟨b N, hb N⟩|| := by simp [Ub, map_sub]
 _ ≤ M * ||b n - b N|| := h_bound _
 _ = M * dist (b n) (b N) := by rw [dist_eq_norm]
 obtain ⟨y, hy⟩ := CompleteSpace.complete hCS
 apply mem_domain_iff.mpr
 rw [← (isClosable_of_continuous h).graph_closure_eq_closure_graph]
 use y
 apply mem_closure_iff_seq_limit.mpr
 refine ⟨fun n ↦ (b n, Ub n), fun n ↦ ?_, ?_⟩
 · simp [hb n, Ub]
 · rw [nhds_prod_eq]

```



1.166.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED5 LEAN

```
exact Filter.Tendsto.prodMk hb' hy

/-- A continuous operator is closed iff its domain is closed. -
/
lemma isClosed_iff_isClosed_domain_of_continuous [CompleteSpace H'] (h : Continuous
 U.IsClosed ↔ _root_.IsClosed (U.domain : Set H) := by
 rw [(isClosable_of_continuous h).isClosed_iff]
 have h_domain := closure_domain_eq_domain_closure_of_continuous h
 constructor <=> intro hcl
 · exact hcl ▸ h_domain ▸ isClosed_closure
 · refine (eq_of_le_of_domain_eq U.le_closure ?_).symm
 exact h_domain ▸ hcl.submodule_topologicalClosure_eq.symm

lemma IsClosed.isClosed_toFun_graph (hU : U.IsClosed) :
 root.IsClosed (U.toFun.graph : Set (U.domain × H')) := by
 refine isClosed_of_closure_subset fun {x₁, x₂} hx ↦ ?_
 simp only [SetLike.mem_coe, LinearMap.mem_graph_iff, toFun_eq_coe]
 suffices (↑x₁, x₂) ∈ U.graph.topologicalClosure by
 simp_all [hU.isClosable.graph_closure_eq_closure_graph, hU.closure_eq]
 obtain ⟨b, hb, hbx⟩ := mem_closure_iff_seq_limit.mp hx
 apply mem_closure_iff_seq_limit.mpr
 rw [nhds_prod_eq] at *
 refine ⟨fun n ↦ (↑(b n).1, (b n).2), by simp_all, ?_⟩
 exact Filter.Tendsto.prodMk (tendsto_subtype_rng.mp hbx.fst) hbx.snd

/-- The closed graph theorem for partial linear maps:
 a closed operator with closed domain is continuous.

 This follows immediately from `LinearMap.continuous_of_isClosed_graph`
 and the completeness of `H` and `H'`. -/
lemma IsClosed.continuous_of_isClosed_domain [CompleteSpace H] [CompleteSpace H']
 (hU : U.IsClosed) (h : _root_.IsClosed (U.domain : Set H)) :
 Continuous U := by
 have hCS : CompleteSpace U.domain := instCompleteSpaceSubtypeMemSubmoduleOfIsClosed
 refine @LinearMap.continuous_of_isClosed_graph _ _ _ _ hCS _ _ _ U.toFun ?_
 exact hU.isClosed_toFun_graph

/-- Closability is preserved upon adding a continuous operator. -
/
lemma IsClosable.add_continuous
 (h₁ : U₁.IsClosable) (h₂ : Continuous U₂) (h : U₁.domain ≤ U₂.domain) :
 (U₁ + U₂).IsClosable := by
 use (U₁ + U₂).graph.topologicalClosure.toLinearPMap
 refine (toLinearPMap_graph_eq _ fun {x₁, x₂} hx hx₁ ↦ ?_).symm
 subst hx₁
 refine graph_fst_eq_zero_snd U₁.closure ?_ rfl
```

```

rw [← h1.graph_closure_eq_closure_graph]
apply mem_closure_iff_seq_limit.mpr
obtain ⟨b, hb, hbx⟩ := mem_closure_iff_seq_limit.mp hx
simp only [coe_toAddSubmonoid, SetLike.mem_coe, mem_graph_iff, add_domain
 Subtype.exists, exists_and_left, exists_eq_left, nhds_prod_eq] at *
refine ⟨fun n ↦ ((b n).1, (b n).2 - U2 ⟨(b n).1, h (hb n).choose.1⟩), fun
 · exact ⟨(hb n).choose.1, eq_sub_of_add_eq (hb n).choose_spec)
 · refine Filter.Tendsto.prodMk hbx.fst ?_
 refine sub_zero x2 ▶ hbx.snd.sub ?_
 exact map_zero U2 ▶ (h2.tendsto 0).comp (tendsto_subtype_rng.mpr hbx.fst)

/-- Closability is preserved upon subtracting a continuous operator. -
/
lemma IsClosable.sub_continuous
 (h1 : U1.IsClosable) (h2 : Continuous U2) (h : U1.domain ≤ U2.domain) :
 sub_eq_add_neg U1 U2 ▶ h1.add_continuous h2.neg h

/-- Closedness is preserved upon adding a continuous operator. -
/
lemma IsClosed.add_continuous [CompleteSpace H']
 (h1 : U1.IsClosed) (h2 : Continuous U2) (h : U1.domain ≤ U2.domain) :
 have hcl : (U1 + U2).IsClosable := h1.isClosable.add_continuous h2 h
 apply hcl.isClosed_iff.mpr
 refine eq_of_le_of_ge (le_of_le_graph ?_) (U1 + U2).le_closure
 rw [← hcl.graph_closure_eq_closure_graph]
 intro ⟨x1, x2⟩ hx
 obtain ⟨b, hb, hbx⟩ := mem_closure_iff_seq_limit.mp hx
 simp only [coe_toAddSubmonoid, SetLike.mem_coe, mem_graph_iff, Subtype.exists,
 exists_eq_left, add_domain, inf_of_le_left h] at hb
 rw [nhds_prod_eq] at hbx
 have hb1U2 : ∀ n, (b n).1 ∈ U2.domain := fun n ↦ h (hb n).choose
 have hCS : CauchySeq fun n ↦ U2 ⟨(b n).1, hb1U2 n⟩ := by
 obtain ⟨M, hM, h_bound⟩ := LinearMap.continuous_iff_bounded.mp h2
 refine Metric.cauchySeq_iff'.mpr fun ε hε ↦ ?_
 obtain ⟨N, hN⟩ := Metric.cauchySeq_iff'.mp hbx.fst.cauchySeq (M⁻¹ * ε)
 refine ⟨N, fun n hn ↦ ?_⟩
 calc
 _ = ||U2 ⟨(b n).1, hb1U2 n⟩ - ⟨(b N).1, hb1U2 N⟩|| := by rw [map_sub,
 _ ≤ M * ||(b n).1 - (b N).1|| := h_bound _
 _ < ε := dist_eq_norm (b n).1 (b N).1 ▶ (lt_inv_mul_iff₀ hM).mp (hN n)
 obtain ⟨y, hy⟩ := CompleteSpace.complete hCS
 have hU1 : (x1, x2 - y) ∈ U1.graph := by
 rw [← h1.closure_eq, ← h1.isClosable.graph_closure_eq_closure_graph]
 apply mem_closure_iff_seq_limit.mpr
 refine ⟨fun n ↦ ((b n).1, (b n).2 - U2 ⟨(b n).1, hb1U2 n⟩), fun n ↦ ?_,
 · simp_all [add_apply, eq_sub_iff_add_eq]

```

1.167

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/SPACEDHILBERTSPACE/BASIC.LEAN

```
· rw [nhds_prod_eq]
 exact hbx.fst.prodMk (hbx.snd.sub hy)
have hx1 : x1 ∈ U1.domain := mem_domain_of_mem_graph hU1
have hU2y : U2 ⟨x1, h hx1⟩ = y := by
 refine tendsto_nhds_unique ((h2.tendsto ⟨x1, h hx1⟩).comp ?_) (Filter.tendsto_ma
 exact tendsto_subtype_rng.mpr hbx.fst
simp_all [add_domain, add_apply]

/-- Closedness is preserved upon subtracting a continuous operator. -
/
lemma IsClosed.sub_continuous [CompleteSpace H']
 (h1 : U1.IsClosed) (h2 : Continuous U2) (h : U1.domain ≤ U2.domain) : (U1 - U2).
 sub_eq_add_neg U1 U2 ▸ h1.add_continuous h2.neg h

/-!
C. Classes of operators
-/

/-!
C.1. Symmetric operators
C.2. Self-adjoint operators
 isEssentiallySelfAdjoint_def.mpr (h.isClosed.closure_eq.symm ▸ h)
C.3. Essentially self-adjoint operators
 have h_le' : T.closure ≤ T2 := h_cl.closure_eq ▸ h_cl.isClosable.closure_mono h_le
C.4. Unbounded operators
/-- `U†.range[] = U.closure.ker` -/
lemma IsUnbounded.orthogonal_adjoint_range [CompleteSpace H] [CompleteSpace H']
 (h : U.IsUnbounded) : U†.toFun.range[] = U.closure.toFun.ker.map U.closure.domain
 h.adjoint_adjoint_eq_closure ▸ h.adjoint.hasDenseDomain.orthogonal_range

/-- `U.closure.ker[] = U†.range` -/
lemma IsUnbounded.orthogonal_closure_ker [CompleteSpace H] [CompleteSpace H'] (h : U
 (U.closure.toFun.ker.map U.closure.domain.subtype)[] = U†.toFun.range.closure :=
 h.adjoint_adjoint_eq_closure ▸ h.adjoint.hasDenseDomain.orthogonal_adjoint_ker
```

## 1.167 Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Basic.lean

# Hilbert spaces for quantum mechanics on `Space d`

## i. Overview

## ii. Key results

## iii. Table of contents

- A. Definition
- B. Dual space
- C. Membership
- D. Construction of elements
- E. Coersions
- F. Misc.

## iv. References  
noncomputable section

open Function InnerProductSpace MeasureTheory Measure Set

/-!

## A. Definition

-/

abbrev SpaceDHilbertSpace (d :  $\mathbb{N}$ ) := Lp ( $\alpha$  := Space d)  $\mathbb{C}$  2 volume  
variable {d :  $\mathbb{N}$ } {f g : Space d  $\rightarrow$   $\mathbb{C}$ } ( $\psi$   $\phi$  : SpaceDHilbertSpace d)

variable { $\psi$   $\phi$ } in

lemma ext\_iff :  $\psi = \phi \leftrightarrow \psi =^{\text{[volume]}} \phi := \text{Lp.ext\_iff}$

/-!

## B. Dual space

-/

/-- The anti-linear equivalence between `SpaceDHilbertSpace d` and its dual

This is the map that takes a ket to its corresponding bra and \_vice versa  
/

def toBra : SpaceDHilbertSpace d  $\approx_{\text{sl}}$  [starRingEnd  $\mathbb{C}$ ] StrongDual  $\mathbb{C}$  (SpaceDHilbertSpace d)

lemma toBra\_apply\_apply : toBra  $\psi$   $\phi$  =  $\langle \psi, \phi \rangle_{\mathbb{C}}$  := rfl

@[simp]

lemma toBra\_symm\_apply (f : StrongDual  $\mathbb{C}$  (SpaceDHilbertSpace d)) :  $\langle \text{toBra.s}$

toDual\_symm\_apply

## C. Membership

def MemHS (f : Space d  $\rightarrow$   $\mathbb{C}$ ) : Prop := MemLp f 2 volume

lemma memHS\_coe : MemHS  $\psi$  := Lp.memLp  $\psi$

/-- A function `f` satisfies `MemHS f` if and only if it is a.e. strongly measurable  
and square integrable. -/

lemma memHS\_iff : MemHS f  $\leftrightarrow$  AESTronglyMeasurable f  $\wedge$  Integrable (fun x  $\mapsto$   $\|f\|^2$ )  
and\_congr\_right fun h  $\mapsto$  (and\_iff\_right h).symm.trans (memLp\_two\_iff\_integ

1.167.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/SPACEDHILBERTSPACE4BASIC.LEAN

```
lemma mem_iff {f : Space d →m[volume] ℂ} : f ∈ SpaceDHilbertSpace d ↔ MemHS f := Lp.
lemma MemHS.zero : MemHS (0 : Space d → ℂ) := MemLp.zero
```

```
lemma MemHS.neg (hf : MemHS f) : MemHS (-f) := MemLp.neg hf
lemma MemHS.add (hf : MemHS f) (hg : MemHS g) : MemHS (f + g) := MemLp.add hf hg
lemma MemHS.sub (hf : MemHS f) (hg : MemHS g) : MemHS (f - g) := MemLp.sub hf hg
lemma MemHS.const_smul (c : ℂ) (hf : MemHS f) : MemHS (c • f) := MemLp.const_smul hf
```

```
lemma MemHS.ae_eq (hfg : f ==[volume] g) (hf : MemHS f) : MemHS g := MemLp.ae_eq hfg
```

## D. Construction of elements

section

```
variable (hf : MemHS f) (hg : MemHS g)
 `mk hf` is the element of the Hilbert space defined by `f`. -
/
```

```
def mk : SpaceDHilbertSpace d :=
 (AEEqFun.mk f hf.1, mem_iff.mpr <| hf.ae_eq (AEEqFun.coeFn_mk f hf.1).symm)
```

@[simp]

```
lemma mk_neg : mk hf.neg = -mk hf := rfl
```

@[simp]

```
lemma mk_add : mk (hf.add hg) = mk hf + mk hg := rfl
```

@[simp]

```
lemma mk_sub : mk (hf.sub hg) = mk hf - mk hg := rfl
```

@[simp]

```
lemma mk_const_smul (c : ℂ) : mk (hf.const_smul c) = c • mk hf := rfl
```

```
lemma coeFn_mk : mk hf ==[volume] f := AEEqFun.coeFn_mk f hf.1
```

```
lemma mk_eq_iff : mk hf = mk hg ↔ f ==[volume] g := by simp [mk]
```

```
lemma mk_surjective : ∃ (f : Space d → ℂ) (hf : MemHS f), mk hf = ψ :=
 ⟨ψ, memHS_coe ψ, by simp [mk]⟩
```

```
lemma inner_mk_mk : ⟨mk hf, mk hg⟩_ℂ = ∫ x, starRingEnd ℂ (f x) * g x := by
 filter_upwards [coeFn_mk hf, coeFn_mk hg]
```

```
 simp_all [mul_comm]
```

end

/-!

## E. Coersions

-/

section

```
variable (c : ℂ) (ψ φ : SpaceDHilbertSpace d)
```

```
lemma coeFn_neg : ↑(-ψ) ==[volume] -ψ := Lp.coeFn_neg _
```

```
lemma coeFn_add : ↑(ψ.val + φ.val) ==[volume] ψ + φ := Lp.coeFn_add _ _
```

```

lemma coeFn_sub : ↑(ψ.val - φ.val) ==[volume] ψ - φ := Lp.coeFn_sub _ _
lemma coeFn_smul : ↑(c • ψ) ==[volume] c • ψ := Lp.coeFn_smul _ _
end
F. Misc.
{α : Type*} {l : Filter α} {ψ : α → SpaceDHilbertSpace d} :
Tendsto ψ l (nhds 0) ↔ Tendsto (fun a ↦ ∫- x, ||ψ a x||e ^ 2) l (nhds 0)
end

```

### 1.168 Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/P

```

public import Mathlib.Analysis.Calculus.BumpFunction.InnerProduct
For each `a : ℕ`, `PolyBddSchwartzSubmodule d a` is the submodule correspo
potential operator maps `PolyBddSchwartzSubmodule d τ` to itself. In the sa
- `PolyBddSchwartzSubmodule d (a : ℕ)` : Restriction of `SchwartzSubmodule
def PolyBddSchwartzMap (d : ℕ) (a : ℕ) : Submodule ℂ ⊆ (Space d, ℂ) where
/-- The continuous linear map `schwartzIncl` with domain restricted to `Pol
/
def polyBddSchwartzIncl {d : ℕ} {a : ℕ} : PolyBddSchwartzMap d a → L[ℂ] Spa
{schwartzIncl.domRestrict (PolyBddSchwartzMap d a),
abbrev PolyBddSchwartzSubmodule (d : ℕ) (a : ℕ) : Submodule ℂ (SpaceDHilbe
PolyBddSchwartzMap d a ≈1[ℂ] PolyBddSchwartzSubmodule d a :=
instance {d : ℕ} {a : ℕ} : CoeOut (PolyBddSchwartzMap d a) ⊆ (Space d, ℂ) :
instance {d : ℕ} {a : ℕ} : CoeFun (PolyBddSchwartzMap d a) (fun _ ↦ Space
lemma toFun_eq_coe {d : ℕ} {a : ℕ} (f : PolyBddSchwartzMap d a) (x : Space
{ψ : SchwartzSubmodule d} (hψ : ↑ψ ∈ PolyBddSchwartzSubmodule d a) :
lemma polyBddSchwartzEquiv_coe_ae {d : ℕ} {a : ℕ} (f : PolyBddSchwartzMap
lemma PolyBddSchwartzMap_zero_eq_top (d : ℕ) : PolyBddSchwartzMap d 0 = τ :
simp_all [PolyBddSchwartzMap]
lemma PolyBddSchwartzMap_antitone (d : ℕ) {a b : ℕ} (h : a ≤ b) :
PolyBddSchwartzMap d b ≤ PolyBddSchwartzMap d a := fun _ hx k hk ↦ hx k
lemma of_zero_eq (d : ℕ) : PolyBddSchwartzSubmodule d 0 = SchwartzSubmodule
simp [PolyBddSchwartzSubmodule, polyBddSchwartzIncl, PolyBddSchwartzMap_z
lemma le_SchwartzSubmodule (d : ℕ) (a : ℕ) : PolyBddSchwartzSubmodule d a
PolyBddSchwartzSubmodule d b ≤ PolyBddSchwartzSubmodule d a := by
simp only [PolyBddSchwartzSubmodule, polyBddSchwartzIncl,
ContinuousLinearMap.toLinearMap_domRestrict, LinearMap.range_domRestrict
exact Submodule.map_mono (PolyBddSchwartzMap_antitone d h)
Dense (PolyBddSchwartzSubmodule 0 τ : Set (SpaceDHilbertSpace 0)) := by
suffices PolyBddSchwartzMap 0 τ = τ by
simp [PolyBddSchwartzSubmodule, polyBddSchwartzIncl, this]
rcases k with _ | k <=> simp [add_nonneg]
lemma dense_top (d : ℕ) : Dense (PolyBddSchwartzSubmodule d τ : Set (SpaceD

```

1.169.

PHYSLIB/QUANTUMMECHANICS/DDIMENSIONS/SPACEDHILBERTSPACE/SCHWARTZSUBMODULE.LEAN

```

· -- `d = 0`: Every function `Space 0 ≅ {0} → ℂ` is in `PolyBddSchwartzSubmodule 0`
· -- `d > 0`: Construct a sequence in `PolyBddSchwartzSubmodule d` which tends to 0
-- `ψn = [fn]` is a sequence in `SchwartzSubmodule` which tends to `ξ`
-- `φn = [bnfn]` is a sequence in `SchwartzSubmodule` which tends to `0`
 gcongr
 rw [sub_apply, ← one_mul (f n x), hg, ← sub_mul, norm_mul, norm_mul, norm_mul]
 gcongr
 rw [← ofReal_one, ← ofReal_sub, norm_real, Real.norm_eq_abs]
· exact (continuous_ofReal.comp (b n).continuous).aestronglyMeasurable.mul
 ξ.val.aestronglyMeasurable
· refine lt_of_le_of_lt ?_ (memHS_coe ξ).2
have hσ_ae (n : ℕ) : σ n =[volume] s n := coeFn_mk _
(coeFn_sub (φ n) (σ n)).trans <| (hφ_ae n).sub (hσ_ae n)
(coeFn_sub (ψ n) ξ).trans <| (hψ_ae n).sub EventuallyEq rfl
· refine lt_top_iff_ne_top.mp ?_
 simp [eLpNorm_one_eq_integral_enorm, Real.rpow_ofNat, enorm_pow, enorm_mul]
 using L2.eLpNorm_rpow_two_norm_lt_top ξ
· have : NeZero d := {hd.ne'}
 refine Tendsto.squeeze tendsto_const_nhds hξB (zero_le) (fun n ↦ ?_)
 refine Tendsto.squeeze tendsto_const_nhds hψξ (zero_le) (fun n ↦ ?_)
lemma dense (d : ℕ) (a : ℕ∞) : Dense (PolyBddSchwartzSubmodule d a : Set (SpaceDHilbertSpace d)) :=

```

## 1.169 Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/SchwartzSubmodule

```

- `SchwartzSubmodule d`: Submodule of `SpaceDHilbertSpace d` consisting of the L^2 eq
abbrev SchwartzSubmodule (d : ℕ) := (schwartzIncl (d := d)).range
def schwartzEquiv : [(Space d, ℂ) ≈1[ℂ] SchwartzSubmodule d :=
variable (f g : [(Space d, ℂ)) (ψ : SchwartzSubmodule d)
instance : CoeFun (SchwartzSubmodule d) fun _ ↦ Space d → ℂ := {fun ψ ↦ ψ.val}
lemma zero_eq_top : SchwartzSubmodule 0 = τ := by
lemma dense (d : ℕ) : Dense (SchwartzSubmodule d : Set (SpaceDHilbertSpace d)) :=

```

## 1.170 Physlib/QuantumMechanics/FiniteTarget/HilbertSpace.lean

A finite target quantum mechanical system is one whose states live in a finite dimensional Hilbert space, with the basis states labelled by a finite type `d` (for example the sites of a finite lattice, or the levels of a qudit).

This file contains

- the definition of `FiniteHilbertSpace d`, the Hilbert space of such a system, as a structure wrapping `EuclideanSpace ℂ d`, together with the notation `[d]`;
- its vector space structure (`AddCommGroup` and `Module ℂ`), transferred from `EuclideanSpace ℂ d` along the equivalence `equivEuclidean`;

```
- its Hilbert space structure (`NormedAddCommGroup`, `InnerProductSpace ℂ`
 `FiniteDimensional ℂ` and `CompleteSpace`), induced along `linearEquivEucl`
- the standard orthonormal basis `basisFun`, whose elements are the states
 localized at the points of `d`.
```

```
/-- The Hilbert space of a finite target quantum mechanical system whose target
 is a finite type `d` with decidable equality.
```

It is defined as a structure with a single field `val`, wrapping an element of  
 `EuclideanSpace ℂ d` – the space of functions  $d \rightarrow \mathbb{C}$  carrying the  $L^2$  inner  
 product  $\langle \psi, \phi \rangle = \sum_i \text{conj } (\psi \ i) * \phi \ i$ . Using a structure in preference to  
 `EuclideanSpace ℂ d` itself makes the Hilbert space of states a type of its own  
 with its own API and the notation  $\langle \cdot | \cdot \rangle$ .

Being finite dimensional, it is automatically a complete inner product space,  
 that is, a genuine Hilbert space. -/

```
@[ext]
```

```
structure FiniteHilbertSpace (d : Type*) [Fintype d] [DecidableEq d] where
 /-- The underlying element of `EuclideanSpace ℂ d`. -/
 val : EuclideanSpace ℂ d
```

```
@[inherit_doc FiniteHilbertSpace]
```

```
scoped notation "[|" d "]" => FiniteHilbertSpace d
```

```
namespace FiniteHilbertSpace
```

```
variable {d : Type*} [Fintype d] [DecidableEq d]
```

```
/-- The equivalence between `FiniteHilbertSpace d` and `EuclideanSpace ℂ d`
 given by `val`. -/
```

```
def equivEuclidean : FiniteHilbertSpace d ≈ EuclideanSpace ℂ d where
```

```
 toFun := val
```

```
 invFun := mk
```

```
 left_inv _ := rfl
```

```
 right_inv _ := rfl
```

```
/-!
```

```
The vector space structure on `FiniteHilbertSpace d`
```

The vector space structure is transferred from `EuclideanSpace ℂ d`  
 along the equivalence `equivEuclidean`.

```
-/
```



1.170.

PHYSLIB/QUANTUMMECHANICS/FINITETARGET/HILBERTSPACE.LEAN53

```
noncomputable instance : AddCommGroup (FiniteHilbertSpace d) := equivEuclidean.addCo
```

```
noncomputable instance : Module ℂ (FiniteHilbertSpace d) := equivEuclidean.module ℂ
```

```
@[simp]
```

```
lemma val_add (ψ φ : FiniteHilbertSpace d) : (ψ + φ).val = ψ.val + φ.val := rfl
```

```
@[simp]
```

```
lemma val_smul (c : ℂ) (ψ : FiniteHilbertSpace d) : (c • ψ).val = c • ψ.val := rfl
```

```
@[simp]
```

```
lemma val_zero : (0 : FiniteHilbertSpace d).val = 0 := rfl
```

```
/-- The equivalence between `FiniteHilbertSpace d` and `EuclideanSpace ℂ d`
as a `ℂ`-linear equivalence, upgrading `equivEuclidean`. -
```

```
/
```

```
noncomputable def linearEquivEuclidean : FiniteHilbertSpace d ≈ℂ EuclideanSpace ℂ d
{ equivEuclidean with
 map_add' := fun _ _ => rfl
 map_smul' := fun _ _ => rfl }
```

```
/-!
```

```
The Hilbert space structure on `FiniteHilbertSpace d`
```

```
The norm and inner product are induced from `EuclideanSpace ℂ d` along
`linearEquivEuclidean`, making `FiniteHilbertSpace d` a finite dimensional
(and hence complete) inner product space, that is, a Hilbert space.
```

```
-/
```

```
noncomputable instance : NormedAddCommGroup (FiniteHilbertSpace d) :=
 NormedAddCommGroup.induced _ _ linearEquivEuclidean.toLinearMap linearEquivEuclidean
```

```
@[simp]
```

```
lemma norm_eq_val (ψ : FiniteHilbertSpace d) : ||ψ|| = ||ψ.val|| := rfl
```

```
noncomputable instance : InnerProductSpace ℂ (FiniteHilbertSpace d) :=
 InnerProductSpace.induced linearEquivEuclidean.toLinearMap
```

```
@[simp]
```

```
lemma inner_eq_val (ψ φ : FiniteHilbertSpace d) : inner ℂ ψ φ = inner ℂ ψ.val φ.val
```

```
instance : FiniteDimensional ℂ (FiniteHilbertSpace d) :=
 Module.Finite.equiv linearEquivEuclidean.symm
```

```
instance : CompleteSpace (FiniteHilbertSpace d) := FiniteDimensional.complete ℂ _
```

```

/-- The equivalence between `FiniteHilbertSpace d` and `EuclideanSpace ℂ d`
 as a linear isometry equivalence, upgrading `linearEquivEuclidean`. -
/
noncomputable def isometryEquivEuclidean : FiniteHilbertSpace d ≈ℓi [ℂ] EuclideanSpace ℂ d :=
 toLinearEquiv := linearEquivEuclidean
 norm_map' _ := rfl

/-!

The standard orthonormal basis of `FiniteHilbertSpace d`

-/
/-- The standard orthonormal basis of `FiniteHilbertSpace d`, indexed by `d`
-/
noncomputable def basisFun (d : Type*) [Fintype d] [DecidableEq d] :
 OrthonormalBasis d ℂ (FiniteHilbertSpace d) :=
 (EuclideanSpace.basisFun d ℂ).map isometryEquivEuclidean.symm
lemma basisFun_apply (i : d) : basisFun d i = ⟨EuclideanSpace.single i 1⟩ :=
 rw [basisFun, OrthonormalBasis.map_apply, EuclideanSpace.basisFun_apply];
end FiniteHilbertSpace

```

### 1.171 Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/

```

apply MeasureTheory.integrable_finsetSum
rw [h2, MeasureTheory.integral_finsetSum]
simp_rw [← Finset.sum_mul]
 refine astronglyMeasurable_fun_sum (range n) fun r _ => ?_
 exact (Continuous.astronglyMeasurable (by fun_prop)).mul
 ((aeStronglyMeasurable_of_memHS hf).mul (Continuous.astronglyMeasurable_of_memHS hf))
 rw [Complex.norm_exp, show (-(↑y ^ 2) / (2 * (Q.ξ : ℂ)^2)) =
 ((-y ^ 2 / (2 * Q.ξ^2) : ℝ) : ℂ) by push_cast; ring, Complex.ofReal]
 · have hnorm : ||∑ i ∈ range n, (Complex.I * (↑c * ↑y)) ^ i / (i ! : ℂ)||
 Real.exp ||Complex.I * (↑c * ↑y)|| := by
 refine (norm_sum_le_of_le _ fun i _ => le_of_eq ?_).trans
 (Real.sum_le_exp_of_nonneg (norm_nonneg _) n)
 rw [norm_div, norm_pow, RCLike.norm_natCast]
 refine hnorm.trans_eq ?_
 rw [Complex.norm_mul, Complex.norm_I, one_mul, Complex.norm_mul, Complex.norm_real,
 Real.norm_eq_abs, Real.norm_eq_abs, abs_mul]
 · exact mul_pos ((norm_nonneg (f y)).lt_of_ne' hf) (Real.exp_pos _)
 have key (r : ℕ) : (fun a => (Complex.I * ↑c * ↑a) ^ r / ↑r ! *
 (a ^ r * (f a * ↑(Real.exp (- a ^ 2 / (2 * Q.ξ^2))))) := by
 funext a

```

1.172.

PHYSLIB/QUANTUMMECHANICS/ONEDIMENSION/HARMONICOSCILLATOR/EIGENFUNCTION. LEAN

```

 simp only [Complex.ofReal_exp, Complex.ofReal_div, Complex.ofReal_neg,
 Complex.ofReal_mul, Complex.ofReal_pow, Complex.ofReal_ofNat]
 ring
 funext n
 rw [MeasureTheory.integral_finsetSum]
 · refine Finset.sum_eq_zero fun r _ => ?_
 rw [key r, MeasureTheory.integral_const_mul,
 Q.orthogonal_power_of_mem_orthogonal f hf hOrth r]
 · intro r
 rw [key r]
 exact (Q.mul_power_integrable f hf r).const_mul _
 have hInt : MemLp (fun x => f x * Real.exp (- x^2 / (2 * Q.ξ^2))) 1 volume := by
 rw [hf']
 exact HilbertSpace.mul_gaussian_mem_Lp_one f hf (1/ (2 * Q.ξ^2)) 0 (by simp)
 · exact memLp_one_iff_integrable.mp hInt
 · exact hInt.aestronglyMeasurable

```

### 1.172 Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/Eigenfun

apply And.intro <=> fun\_prop

### 1.173 Physlib/QuantumMechanics/OneDimension/HarmonicOscillator/TISE.lean

```

 rw [show (fun (x : ℝ) => Complex.exp (- x ^ 2 / (2 * Q.ξ ^ 2)))
 = Complex.exp ∘ (fun (x : ℝ) => - x ^ 2 / (2 * Q.ξ ^ 2)) from rfl,
 deriv_comp _ (by fun_prop) (by fun_prop)]
 simp only [pow_two]
 rw [deriv_fun_mul Complex.differentiableAt_ofReal Complex.differentiableAt_ofR]
 simp [eigenfunction_eq, physHermite_zero, physHermite_one, map_ofNat]
 funext x
 have hd : DifferentiableAt ℝ (fun x => physHermite n (x / Q.ξ)) x := by fun_prop
 rw [(hd.hasDerivAt.ofReal_comp).deriv, deriv_physHermite' x (fun x => x / Q.ξ) (by
 simp only [deriv_div_const, deriv_id'])]
 push_cast
 ring
 push_cast
 rw [deriv_physHermite_characteristic_length]
 have hr : (physHermite (n + 1) (x / Q.ξ) : ℝ) =
 2 * (x / Q.ξ) * physHermite n (x / Q.ξ) - 2 * n * physHermite (n - 1) (x /
 rw [physHermite_succ_fun']
 simp [nsmul_eq_mul]
 rw [hr]
 push_cast

```

**1.174 Physlib/QuantumMechanics/OneDimension/HilbertSpace/Basic.**

```
simp only [norm_zero, ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero
 integrable_fun_zero, and_true]
```

**1.175 Physlib/QuantumMechanics/OneDimension/Operators/Momentum.**

```
simp only [Pi.smul_apply, deriv_const_smul _ (hψ x), smul_comm (-
Complex.I * ħ) c]
simp only [Pi.add_apply, deriv_add (hψ1 x) (hψ2 x), smul_eq_mul]
 simp only [map_smul, RingHom.id_apply, smul_comm (-Complex.I * ħ) a]
 · simp [momentumOperatorSchwartz, planewaveFunctional_apply,
 SchwartzMap.fourierTransformCLM_apply]
simp only [SchwartzMap.fourierTransformCLM_apply, smul_eq_mul]
simp only [planewaveFunctional_apply, smul_eq_mul]
simp [Complex.I_sq]
exact Or.inl rfl
have hint : ∀ f g : $\square(\mathbb{R}, \mathbb{C})$,
 MeasureTheory.Integrable (fun x => star (f x) * g x) MeasureTheory.vo
 fun f g => ((ContinuousLinearEquiv.integrable_comp_iff (starL' $\mathbb{R}$)).mpr
 (SchwartzMap.integrable f)).mul_of_top_left (SchwartzMap.memLp_top g)
 rw [momentumOperatorSchwartz_apply, ← fderiv_apply_one_eq_deriv, ← mul_
 mul_comm _ (-Complex.I * $\uparrow\hbar$), mul_assoc]
 rw [MeasureTheory.integral_const_mul, integral_mul_fderiv_eq_neg_fderiv_m
 ← MeasureTheory.integral_neg, ← MeasureTheory.integral_const_mul]
simp only [starRingEnd_apply, fderiv_star]
simp [momentumOperatorSchwartz_apply, mul_assoc]
 · simp only [starRingEnd_apply, fderiv_star]
 exact hint (SchwartzMap.derivCLM $\mathbb{C} \mathbb{C} \psi_1$) ψ_2
 · exact hint ψ_1 (SchwartzMap.derivCLM $\mathbb{C} \mathbb{C} \psi_2$)
 · exact hint $\psi_1 \psi_2$
 · exact fun x _ => (SchwartzMap.differentiable ψ_1).star x
```

**1.176 Physlib/QuantumMechanics/OneDimension/Operators/Parity.le**

```
public import Mathlib.Analysis.Calculus.ContDiff.Operations
 rw [show (fun x : $\mathbb{R}$ => -x) = -(fun x : $\mathbb{R}$ => x) from rfl]
 rw [iteratedFDeriv_neg_apply]
 simp only [Nat.succ_eq_add_one, fderiv_fun_id, Function.comp_apply,
```

**1.177 Physlib/Relativity/Bispinors/Basic.lean**

```
def contrBispinorUp (p : CT[.up]) : CT[.upL, .upR] := permT id (IsReindexing.auto)
def contrBispinorDown (p : CT[.up]) : CT[.downL, .downR] := permT id (IsReindexing.a
def coBispinorUp (p : CT[.down]) : CT[.upL, .upR] := permT id (IsReindexing.auto)
def coBispinorDown (p : CT[.down]) : CT[.downL, .downR] := permT id (IsReindexing.au
```

**1.178 Physlib/Relativity/CliffordAlgebra.lean**

```
SetLike.mk_smul_mk, Finset.sum_apply, AddSubmonoidClass.coe_finsetSum]
```

**1.179 Physlib/Relativity/LorentzAlgebra/Basic.lean**

```
public import Mathlib.Algebra.Lie.SerreConstruction
attribute [local instance 100] LieRing.ofAssociativeRing

simp_all only [neg_mul, lorentzAlgebra, LieAlgebra.Orthogonal.so',
 mem_skewAdjointMatricesLieSubalgebra, mem_skewAdjointMatricesSubmodule, Matrix.I
 IsAdjointPair, mul_neg]
exact h
```

**1.180 Physlib/Relativity/LorentzAlgebra/ExponentialMap.lean**

```
convert! det_exp_real (reindex e e A.1)
```

**1.181 Physlib/Relativity/LorentzGroup/API-map.yaml**

```
version: v0.1
```

```
Title: "Lorentz Group"
```

```
Overview: |
```

```
The Lorentz group is the group of matrices preserving the Minkowski metric,
defined as matrices satisfying $\Lambda * \text{dual } \Lambda = 1$. It is the key symmetry group
of Minkowski spacetime, used throughout the library to define Lorentz tensors,
fermions, and the electromagnetic potential.
```

```
ParentAPIs:
```

```
- Physlib/Relativity/MinkowskiMatrix.lean
```

```
References:
```

- "Lorentz Transformations, Rotations, and Boosts, Jaffe. <https://cdn.ku>.

Requirements:

- description: LorentzGroup is defined as the set of matrices satisfying  
done: true  
location: Physlib/Relativity/LorentzGroup/Basic.lean (LorentzGroup)
- description: Instance as a group  
done: true  
location: Physlib/Relativity/LorentzGroup/Basic.lean (lorentzGroupIsGro
- description: Realization as a closed subgroup of GL (closed embedding t  
done: true  
location: Physlib/Relativity/LorentzGroup/Basic.lean (toGL\_isClosedEmbe
- description: Instance as a Lie group (blocked: needs Cartan's closed su  
done: false  
location: N/A
- description: Inclusion of boosts  
done: true  
location: Physlib/Relativity/LorentzGroup/Boosts/Basic.lean (boost)
- description: Inclusion of rotations  
done: true  
location: Physlib/Relativity/LorentzGroup/Rotations.lean (Rotations)
- description: Different membership conditions  
done: true  
location: Physlib/Relativity/LorentzGroup/Basic.lean (mem\_iff\_self\_mul\_
- description: Definition of proper Lorentz transforms  
done: true  
location: Physlib/Relativity/LorentzGroup/Proper.lean (IsProper)
- description: Definition of orthochronous Lorentz transforms  
done: true  
location: Physlib/Relativity/LorentzGroup/Orthochronous/Basic.lean (IsO
- description: Definition of the restricted Lorentz group  
done: true  
location: Physlib/Relativity/LorentzGroup/Restricted/Basic.lean (restr
- description: Prove topological properties of the Orthochronous Lorentz  
done: false

location: N/A

- description: Prove that every member of the restricted Lorentz group is a combination of a rotation and a boost
- done: false
- location: N/A

## 1.182 Physlib/Relativity/LorentzGroup/Basic.lean

```

public import Mathlib.Topology.Maps.Basic
public import Mathlib.Topology.Algebra.Group.ClosedSubgroup
simp [dual]
/-!

Lorentz group as a closed subset of matrices

-/

lemma continuous_self_mul_dual {d : ℕ} :
 Continuous (fun Λ : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ => Λ * dual Λ) := by
 unfold dual
 fun_prop

/-- The Lorentz group is closed in the ambient matrix space. -
/
lemma isClosed (d : ℕ) :
 IsClosed (LorentzGroup d : Set (Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ)) := by
 rw [LorentzGroup]
 exact isClosed_singleton.preimage continuous_self_mul_dual

/-- The coercion from the Lorentz group to its ambient matrix space is a closed embedding. -
/
lemma isClosedEmbedding_val {d : ℕ} :
 Topology.IsClosedEmbedding
 (Subtype.val : LorentzGroup d → Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ) :=
 (isClosed d).isClosedEmbedding_subtypeVal

/-!

Inverses

-/
Lorentz group as a closed subgroup of the general linear group

Following the plan sketched by PrParadoxy and Joseph Tooby-
```

Smith in physlib issue #833, we realise

``LorentzGroup d`` as a *\*closed\** subgroup of ``GL (Fin 1 ⊕ Fin d) ℝ``. This provides an embedding into the ambient matrix space (``isClosedEmbedding_val``) to the linear group, and is the structural prerequisite for a Lie group structure

The remaining step of that plan, endowing ``LorentzGroup d`` with a Lie group structure as a subgroup of the Lie group ``GL (Fin 1 ⊕ Fin d) ℝ`` and thereby deriving its Lie algebra, is a closed subgroup theorem (Cartan's theorem: a closed subgroup of a Lie group is a Lie group). That theorem is not yet available in Mathlib: there is no ``LieSubgroup`` instance. A ``CharteredSpace`/`IsManifold`` instance is produced from a closed embedding (``IsOpenEmbedding.singletonCharteredSpace``, for open embeddings). The Lie group structure on the Lorentz algebra therefore remain open; see the ``TODO`` at the top of this file

-/

/-- The range of ``toGL``, as a set in ``GL (Fin 1 ⊕ Fin d) ℝ``, is exactly the Lorentz group under the coercion ``GL (Fin 1 ⊕ Fin d) ℝ → Matrix _ _ ℝ``: an element of the linear group lies in the image precisely when its underlying matrix is a Lorentz transformation. -/

```
lemma range_toGL :
 Set.range (@toGL d).toFun =
 Units.val-1 (LorentzGroup d : Set (Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ))
 ext x
 constructor
 · rintro ⟨A, rfl⟩
 exact A.2
 · intro hx
 exact ⟨x.val, hx⟩, Units.ext rfl
```

/-- The homomorphism ``toGL`` is a closed embedding of the Lorentz group into ``GL (Fin 1 ⊕ Fin d) ℝ``. This strengthens ``toGL_embedding``: the image is closed, and the preimage, under the continuous coercion ``GL (Fin 1 ⊕ Fin d) ℝ → Matrix _ _ ℝ``, is ``LorentzGroup d`` (``LorentzGroup.isClosed``). -/

```
lemma toGL_isClosedEmbedding : Topology.IsClosedEmbedding (@toGL d).toFun with
 toIsEmbedding := toGL_embedding
 isClosed_range := by
 rw [range_toGL]
 exact (isClosed d).preimage Units.continuous_val
```

/-- The Lorentz group realised as a subgroup of ``GL (Fin 1 ⊕ Fin d) ℝ``, named ``toGLSubgroup``. -/

```
def toGLSubgroup (d : ℕ) : Subgroup (GL (Fin 1 ⊕ Fin d) ℝ) := (@toGL d).range
```

@[simp]

```
lemma mem_toGLSubgroup {x : GL (Fin 1 ⊕ Fin d) ℝ} :
 x ∈ toGLSubgroup d ↔
```



1.183. *PHYSLIB/RELATIVITY/LORENTZGROUP/BOOSTS/APPLY.LEAN* 261

```

 (x : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ) ∈ LorentzGroup d := by
 constructor
 · rintro ⟨A, rfl⟩
 exact A.2
 · intro hx
 exact ⟨⟨x.val, hx⟩, Units.ext rfl⟩

lemma coe_toGLSubgroup :
 (toGLSubgroup d : Set (GL (Fin 1 ⊕ Fin d) ℝ)) =
 Units.val-1 (LorentzGroup d : Set (Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ))
 range_toGL

/-- The Lorentz group realised as a *closed* subgroup of `GL (Fin 1 ⊕ Fin d) ℝ`. The
 exactly the content of `toGL_isClosedEmbedding`, and is the hypothesis that the (not
 yet-formalised)
 closed subgroup theorem would consume to produce a Lie group structure. -
 /
def toClosedSubgroup (d : ℕ) : ClosedSubgroup (GL (Fin 1 ⊕ Fin d) ℝ) where
 toSubgroup := toGLSubgroup d
 isClosed' := by
 rw [show (toGLSubgroup d).carrier =
 Units.val-1 (LorentzGroup d : Set (Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d)
 coe_toGLSubgroup)]
 exact (isClosed d).preimage Units.continuous_val

/-!

```

### 1.183 Physlib/Relativity/LorentzGroup/Boosts/Apply.lean

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.Tensorial
```

### 1.184 Physlib/Relativity/LorentzGroup/Boosts/Basic.lean

```

have hγ : (γ β) ^ 2 - (γ β) ^ 2 * β ^ 2 = 1 := by
 have hd := γ_det_not_zero β hβ
 rw [show (γ β) ^ 2 - (γ β) ^ 2 * β ^ 2 = (γ β) ^ 2 * (1 - β ^ 2) by ring, γ_sq]
 field_simp
 linear_combination hγ
 linear_combination hγ
 · rw [one_apply]
simp only [Fin.isValue, reduceCtorEq, and_self, ↓reduceIte, Sum.inr.injEq, false
 and_false, and_comm (a := k = i), eq_comm (a := k) (b := j)]

```

```

 and_true, γ_neg]
 split <;> simp
 false_and, and_false, mul_ite, one_mul, mul_one, mul_zero, mul_neg, n
 and_comm (a := k = i), eq_comm (a := k) (b := j)]
 split <;> [skip; split] <;> simp
 simp [boost, hij, Ne.symm hij]
 (boost (0 : Fin d.succ) β hβ).1 (Sum.inl 0) (Sum.inr i.succ) = 0 :=
 boost_inl_0_inr_other hβ (Fin.succ_ne_zero i)
 (boost (0 : Fin d.succ) β hβ).1 (Sum.inr i.succ) (Sum.inl 0) = 0 :=
 boost_inr_other_inl_0 hβ (Fin.succ_ne_zero i)
 (boost (0 : Fin d.succ) β hβ).1 (Sum.inl 0) (Sum.inr (i + 1, h)) = 0 :=
 boost_inl_0_inr_other hβ (Fin.ne_of_val_ne (Nat.succ_ne_zero i))
 (boost (0 : Fin d.succ) β hβ).1 (Sum.inr (i + 1, h)) (Sum.inl 0) = 0 :=
 boost_inr_other_inl_0 hβ (Fin.ne_of_val_ne (Nat.succ_ne_zero i))
 (boost (0 : Fin d.succ) β hβ).1 (Sum.inr 0) (Sum.inr i.succ) = 0 :=
 boost_inr_self_inr_other hβ (Fin.succ_ne_zero i)
 (boost (0 : Fin d.succ) β hβ).1 (Sum.inr i.succ) (Sum.inr 0) = 0 :=
 boost_inr_other_inr_self hβ (Fin.succ_ne_zero i)
 (boost (0 : Fin d.succ) β hβ).1 (Sum.inr 0) (Sum.inr (i + 1, h)) = 0 :=
 boost_inr_self_inr_other hβ (Fin.ne_of_val_ne (Nat.succ_ne_zero i))
 (boost (0 : Fin d.succ) β hβ).1 (Sum.inr (i + 1, h)) (Sum.inr 0) = 0 :=
 boost_inr_other_inr_self hβ (Fin.ne_of_val_ne (Nat.succ_ne_zero i))
 rw [boost_inr_inr_other hβ (Fin.succ_ne_zero i2)]
 simp [Fin.succ_inj, eq_comm]

```

### 1.185 Physlib/Relativity/LorentzGroup/Boosts/Generalized.lean

```

 simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
 simp only [map_add, add_apply, neg_add_rev]
 simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul, RingHom.id_app]
 simp only [genBoostAux₂, LinearMap.coe_mk, AddHom.coe_mk, genBoostAux₁, L
 map_add, Velocity.minkowskiProduct_self_eq_one]
 match_scalars
 ring
 simp only [map_add, add_apply, Velocity.minkowskiProduct_self_eq_one]
 have h2 : (minkowskiProduct ↑v) u.1 = □u.1, v.1□ₘ := by rw [minkowskiPro
 simp only [map_add, _root_.smul_add, _root_.neg_smul, add_apply, _root_.n
 Velocity.minkowskiProduct_self_eq_one, smul_eq_mul, h2]
 field_simp
 ring
 have h1 : □v.1, u.1 + v.1□ₘ = (1 + □u.1, v.1□ₘ) := by
 simp only [smul_apply, map_add, Velocity.minkowskiProduct_self_eq_one, sm
 neg_inj]
 rw [minkowskiProduct_symm]

```

1.186.

PHYSLIB/RELATIVITY/LORENTZGROUP/ORTHOCHRONOUS/BASIC.lean

```
ring
 simp only [map_add, add_apply]
 add_apply, minkowskiProduct_basis_right, basis_apply, mul_ite, mul_one,
 simp only [add_apply]
 simp only [add_apply]
have h := Velocity.one_add_minkowskiProduct_ne_zero u v
apply (smul_right_inj h).mp
match_scalars <=> field_simp
simp only [Velocity.minkowskiProduct_self_eq_one, map_add]
match_scalars <=> ring
simp only [map_add, Velocity.minkowskiProduct_self_eq_one,
 minkowskiProduct_symm v.1 u.1]
match_scalars <=> ring
module
 fun_prop (disch := exact fun x => Velocity.one_add_minkowskiProduct_ne_zero u x)
 fun_prop (disch := exact fun x => Velocity.one_add_minkowskiProduct_ne_zero x u)
have h1 := Velocity.one_add_minkowskiProduct_ne_zero u v
simp only [generalizedBoost_apply_expand, map_add, map_sub, map_smul, add_apply, s
 smul_apply, smul_eq_mul, Velocity.minkowskiProduct_self_eq_one, one_smul,
 minkowskiProduct_symm v.1 u.1]
match_scalars <=> field_simp <=> ring
```

## 1.186 Physlib/Relativity/LorentzGroup/Orthochronous/Basic.lean

```
constructor <=> intro h <=> rcases h1 with h1 | h1 <=> linarith
constructor <=> intro h <=> rcases h1 with h1 | h1 <=> linarith
constructor <=> intro h <=> rcases h1 with h1 | h1 <=> linarith
 simp [ha]
 simp [ha]
rw [stepFunction, if_pos h, if_neg (by linarith)]
change stepFunction (Λ.1 _) = - 1
rw [stepFunction, if_pos h]
rw [if_neg (by norm_num)]
rfl
rw [isOrthochronous_iff_toVector_timeComponet_nonneg, toVector_mul,
 smul_timeComponent_eq_toVector_minkowskiProduct]
exact Lorentz.Velocity.zero_le_minkowskiProduct _ _
· exact isOrthochronous_mul h h'
· have hmn : (Λ * -Λ' : LorentzGroup d) = -(Λ * Λ') := Subtype.ext (by simp)
 rw [← neg_isOrthochronous_iff_not, ← hmn]
 exact isOrthochronous_mul h (neg_isOrthochronous_iff_not.mpr h')
· have hnm : (-Λ * Λ' : LorentzGroup d) = -(Λ * Λ') := Subtype.ext (by simp)
 rw [← neg_isOrthochronous_iff_not, ← hnm]
 exact isOrthochronous_mul (neg_isOrthochronous_iff_not.mpr h) h'
```

```

· have hnn : (-Λ * -Λ' : LorentzGroup d) = Λ * Λ' := Subtype.ext (by simp
 rw [← hnn]
 refine isOrthochronous_mul ?_ ?_ <;> rwa [neg_isOrthochronous_iff_not]
 by_cases h : IsOrthochronous Λ <;> by_cases h' : IsOrthochronous Λ' <;>
 simp_all [orthchroMap_IsOrthochronous, orthchroMap_not_IsOrthochronous,
 isOrthochronous_mul_iff]
 rfl
 rw [map_inv]
 generalize orthchroRep Λ = x
 revert x
 decide

```

### 1.187 Physlib/Relativity/LorentzGroup/Rotations.lean

```

constructor <;> simp
 simp [Matrix.fromBlocks_transpose, Matrix.fromBlocks_multiply] at h

```

### 1.188 Physlib/Relativity/MinkowskiMatrix.lean

```

simp [as_block, fromBlocks_multiply]

```

### 1.189 Physlib/Relativity/PauliMatrices/AsTensor.lean

```

def asTensor : (ContrModule ℂ[ℂ] (LeftHandedWeyl ℂ[ℂ] RightHandedWeyl)) :=
 simp [leftRightToMatrix_symm_expand_tmul, pauliBasis, pauliSelfAdjoint, p
 module
 simp [leftRightToMatrix_symm_expand_tmul, pauliBasis, pauliSelfAdjoint, p
 module
def asConstTensor :
 (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
 (ContrModule.SL2CRep.tprod (leftHandedRep.tprod rightHandedRep)) where

 a' • asTensor
 simp only [add_smul]
 (TensorProduct.map (ContrModule.SL2CRep M)
 (TensorProduct.map (leftHandedRep M) (rightHandedRep M))) (x • as
 calc _ = ∑ x, ((ContrModule.SL2CRep M) (complexContrBasis x) ⊗₊[ℂ]
 AddSubgroup.val_finsetSum, map_add, map_sum]
 simp only [SL2C.toLorentzGroup_apply_coe, map_inv, LorentzGroup
 LinearMap.map_smul_of_tower, LinearEquiv.coe_coe]
 rw [smul_tmul, smul_smul, ← tmul_smul]
 • ((complexContrBasis i) ⊗₊[ℂ] leftRightToMatrix.symm ((pauliB

```

1.190.

PHYSLIB/RELATIVITY/PAULIMATRICES/CLIFFORDALGEBRA.LEAN 265

```

 rw [Finset.sum_comm]
 exact Finset.sum_congr rfl (fun x _ => Finset.sum_comm)
_ = ∑ i, ∑ j, ∑ x, (((SL2C.toLorentzGroup M).1 i x *
 (SL2C.toLorentzGroup M⁻¹).1 x j : ℝ) : ℂ)
 • ((complexContrBasis i)) ⊗ₜ[ℂ] leftRightToMatrix.symm ((pauliBasis j))
_ = ∑ i, ∑ j, (∑ x, (SL2C.toLorentzGroup M).1 i x * (SL2C.toLorentzGroup M⁻¹).1 x j : ℝ)
 • ((complexContrBasis i)) ⊗ₜ[ℂ] leftRightToMatrix.symm ((pauliBasis j))
 refine Finset.sum_congr rfl (fun i _ => (Finset.sum_congr rfl (fun j _ =>
 rw [← Finset.sum_smul]
 simp
_ = ∑ i, ∑ j, (∑ x, (SL2C.toLorentzGroup M).1 i x * (SL2C.toLorentzGroup M⁻¹).1 x j : ℝ)
 congr
 simp
_ = ∑ i, ∑ j, ((1 : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ) i j : ℝ)
 rw [Finset.sum_eq_single i (fun b _ hb => ?_) (fun hb => ?_)]
 · simp only [one_apply_eq, one_smul]
 · simp [one_apply_ne' hb]
 module
 · simp only [Finset.mem_univ, not_true_eq_false] at hb
lemma asConstTensor_apply_one : asConstTensor (1 : ℂ) = asTensor := by

```

## 1.190 Physlib/Relativity/PauliMatrices/CliffordAlgebra.lean

```
Finset.sum_apply, AddSubmonoidClass.coe_finsetSum]
```

## 1.191 Physlib/Relativity/PauliMatrices/Relations.lean

```

Function.comp_apply, Finsupp.coe_smul, Pi.smul_apply, smul_eq_mul]
simp only [Fin.isValue, Fin.cast_eq_self, ofRat_basis_repr_apply]
rw [leftDualLeftUnit_eq_ofRat, contrMetric_eq_ofRat, prodT_ofRat_ofRat, ← map_ns]
rw [dualRightRightUnit_eq_ofRat, contrMetric_eq_ofRat, prodT_ofRat_ofRat, ← map_ns]

```

## 1.192 Physlib/Relativity/PauliMatrices/SelfAdjoint.lean

```

fin_cases i <=>
 simp [pauliSelfAdjoint', pauliSelfAdjoint, pauliBasis, pauliBasis',
 minkowskiMatrix.inr_i_inr_i, Subtype.ext_iff, NegMemClass.coe_neg, neg_neg]

```

## 1.193 Physlib/Relativity/PauliMatrices/ToTensor.lean

```
open Module
```

```

simp only [fromTripleT_apply_basis]
permT id (IsReindexing.auto) {η' | μ ν ⊗ σ^^ | ν α β}^T
permT id (IsReindexing.auto) {σ^^ | μ α β ⊗ εR' | β β' ⊗ εL' | α α' }^T
 permT id (IsReindexing.auto) {σ^^ | μ α β ⊗ εR' | β β' ⊗ εL' | α α' }^T
 dualLeftMetric_eq_ofRat]
 dualRightMetric_eq_ofRat]
 dualLeftMetric_eq_ofRat]
 dualRightMetric_eq_ofRat]
rw [smul_pauliCo, actionT_dualLeftMetric, actionT_dualRightMetric]
rw [toTensor_smul_eq_self, actionT_dualLeftMetric, actionT_dualRightMetric]

```

### 1.194 Physlib/Relativity/SL2C/Basic.lean

```

rw [SpecialLinearGroup.coe_inv, Matrix.inv_def, SpecialLinearGroup.det_coe]
simp
noncomputable def toSelfAdjointMap (M : SL(2, ℂ)) :
 simp [toSelfAdjointMap]
 | 0, 0 | 1, 1 =>
 simp only [Fin.isValue, cons_val', cons_val_zero, cons_val_one, empty_val]
 | 0, 1 | 1, 0 =>
 simp only [Fin.isValue, cons_val', cons_val_zero, cons_val_one, empty_val]
rw [Lorentz.contrContrContractField.same_eq_det_toSelfAdjoint, toMatrix_apply,
 LinearEquiv.apply_symm_apply, toSelfAdjointMap_apply_det,
 Lorentz.contrContrContractField.same_eq_det_toSelfAdjoint]
rw [PauliMatrix.pauliBasis_minkowskiMetric_pauliBasis', _root_.map_smul,
 Finset.smul_sum]
simp only [sub_smul]
exact h1x
exact congrFun (toLorentzGroup_fst_col M) (Sum.inl 0)
rw [LorentzGroup.IsOrthochronous, toLorentzGroup_inl_inl]

```

### 1.195 Physlib/Relativity/SL2C/SelfAdjoint.lean

```

have hxy : x * y = 1 := by rw [show x * y = M.det by rw [he]; simp, detM]
have detA_one : normSq x * normSq y = 1 := by
 rw [← Complex.normSq_mul, hxy, Complex.normSq_one]

```

### 1.196 Physlib/Relativity/Special/TwinParadox/Basic.lean

```

FunLike.coe_sub, Pi.sub_apply, Finset.sum_const_zero, MulZeroClass.zero

```

**1.197 Physlib/Relativity/Tensors/Basic.lean**

```
public import Physlib.Relativity.Tensors.ComponentIdx.Single
```

```
public import Physlib.Relativity.Tensors.Reindexing
```

This file realizes ``Tensor S c`` as a ``PiTensorProduct``, relates pure tensors to their components, constructs the component basis, and supplies the inherited finite-dimensional and module-topology instances. It also defines the natural action of ``G`` on pure tensors and tensors, the associated permutation maps, and the map from rank-zero tensors to the base field.

```
open Module
```

```
variable {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
```

```
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
```

```
 {basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
```

```
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisIdx c) k (
```

```
 (S : TensorSpecies k C G V basisIdx rep b)
```

```
set_option linter.unusedVariables false in
```

```
@[nolint unusedArguments]
```

```
noncomputable abbrev Tensor {n : ℕ} (S : TensorSpecies k C G V basisIdx rep b)
```

```
 (c : Fin n → C) : Type := PiTensorProduct k (fun i => V (c i))
```

```
variable {S : TensorSpecies k C G V basisIdx rep b} {n n' n2 : ℕ} {c : Fin n → C}
```

```
 {c' : Fin n' → C} {c2 : Fin n2 → C}
```

```
/-!
```

```
Components of tensors
```

```
-/
```

```
set_option linter.unusedVariables false in
```

```
@[nolint unusedArguments]
```

```
abbrev Pure (S : TensorSpecies k C G V basisIdx rep b) (c : Fin n → C) : Type :=
```

```
 (i : Fin n) → V (c i)
```

```
 (i j : Fin n) (h : i = j) : LinearEquiv.cast (R := k) (by rw [h]) (p j) = p i :=
```

```
 LinearEquiv.cast (R := k) (by rw [hi] : c i = c') (p i) =
```

```
 LinearEquiv.cast (R := k) (by rw [hj] : c j = c') (p j) := by
```

```
simp only [LinearEquiv.cast_apply, cast_eq]
```

```
 LinearEquiv.cast (R := k) (by rw [hi] : c i = c') (p i) =
```

```
 LinearEquiv.cast (R := k) (by rw [hj] : c1 j = c') (p' j)
```

```
 ↔ LinearEquiv.cast (R := k) (by rw [hi, hj] : c i = c1 j) (p i) =
```

```
simp_all
```

```
a `x` in `V (c i)` then `update p i x` corresponds to `p` where
```

```
(x : V (c i)) : Pure S c := Function.update p i x
```

```
lemma update_eq_function_update {n : ℕ} {c : Fin n → C} [inst : DecidableEq (Fin n)]
```

```
 (p : Pure S c) (i : Fin n)
```

```
 (x : V (c i)) : update p i x = Function.update p i x := rfl
```

```
 (x : V (c i)) : (update p i x) i = x := by
```

```
lemma update_diff {n : ℕ} {c : Fin n → C} [inst : DecidableEq (Fin n)] (p : Pure S c)
```

```
 (x : V (c i)) (hij : i ≠ j) : (update p i x) j = p j := by
```

```

simp_all [update, Function.update]
grind

(p : Pure S c) (i : Fin (n + 1)) (j : Fin n) (x : V (c (i.succAbove j)))
(i : Fin n) (x y : V (c i)) :
(i : Fin n) (r : k) (y : V (c i)) :
(p : Pure S c) (i : Fin (n + 1)) (k : Fin n) (x : V (c (i.succAbove k)))
(p : Pure S c) (i : Fin (n + 1)) (x : V (c i)) :
(φ : ComponentIdx (S := S) c) : k :=
[] i, (b (c i)).repr (p i) (φ i)
lemma component_eq {n : ℕ} {c : Fin n → C} (p : Pure S c) (φ : ComponentIdx
p.component φ = [] i, (b (c i)).repr (p i) (φ i) := by rfl
(φ : ComponentIdx c) :
p.component φ = ((b (c i)).repr (p i) (φ i)) *
((drop p i).component (fun j => φ (i.succAbove j))) := by
(x y : V (c i)) (b : ComponentIdx c) :
(x : k) (y : V (c i)) (b : ComponentIdx c) :
MultilinearMap k (fun i => V (c i)) (ComponentIdx (S := S) c → k) where
noncomputable def basisVector {n : ℕ} (c : Fin n → C) (φ : ComponentIdx (S
fun i => b (c i) (φ i)
simp only [PiTensorProduct.tprodCoeff_eq_smul_tprod]
exact hsmul r _ (h p)
Pure.toTensor (fun i => ((b (c i)).repr (p i)) (x i) • (b (c i)) (x
(fun i => ((b (c i)).repr (p i)) (x i)) fun i => (b (c i)) (x i))
∑ x, ((b (c i)).repr (p i)) x • (b (c i)) x
((b (c i)).repr (p i)) j • (b (c i)) j).symm
exact Basis.sum_equivFun (b (c i)) (p i)
lemma basis_apply {n : ℕ} (c : Fin n → C) (φ : ComponentIdx (S := S) c) :
basis c φ = (Pure.basisVector c φ).toTensor := by
rw [Finset.sum_eq_single φ]
ext φ
change componentMap c p.toTensor φ = _
/-- `componentMap` is the basis representation `(basis c).repr` definitiona
notations wherever a component computation meets a `repr` rewrite. -
/
lemma componentMap_eq_repr {n : ℕ} (c : Fin n → C) (t : S.Tensor c)
(ψ : ComponentIdx (S := S) c) : componentMap c t ψ = (basis c).repr t ψ

/-- Two tensors with the same colour sequence are equal when all their comp
/
lemma componentMap_ext {n : ℕ} {c : Fin n → C} {t₁ t₂ : S.Tensor c}
(h : ∀ b, componentMap c t₁ b = componentMap c t₂ b) : t₁ = t₂ := by
rw [← ofComponents_componentMap c t₁, ← ofComponents_componentMap c t₂]
congr 1; funext b; exact h b

instance {k : Type} [Field k] {C : Type} {G : Type} [Group G]

```



```

 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisIdx c) k (
 (S : TensorSpecies k C G V basisIdx rep b)
noncomputable instance {k : Type} [RCLike k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisIdx c) k (
 (S : TensorSpecies k C G V basisIdx rep b)
instance {k : Type} [RCLike k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisIdx c) k (
 (S : TensorSpecies k C G V basisIdx rep b)
 smul g p := fun i => rep (c i) g (p i)
 change rep (c i) 1 (p i) = p i
 change rep (c i) (g * g') (p i) = rep (c i) g (rep (c i) g' (p i))
lemma actionP_eq {g : G} {p : Pure S c} : g • p = fun i => rep (c i) g (p i) := rfl

lemma rep_cast {g : G} {c c1 : C} (p : V c) (h : c = c1) :
 LinearEquiv.cast (R := k) h (rep c g p) = rep c1 g (LinearEquiv.cast (R := k) h
 subst h
 rfl
lemma actionP_cast {g : G} {p : Pure S c} (h : c = c1) :
 LinearEquiv.cast (R := k) h (g • p) = g • LinearEquiv.cast (R := k) h p := by
 subst h
 rfl

noncomputable instance : SMul G (S.Tensor c) where
 smul g t := PiTensorProduct.map (fun i => rep (c i) g) t

lemma actionT_eq {g : G} {t : S.Tensor c} : g • t =
 PiTensorProduct.map (fun i => rep (c i) g) t := rfl

 simp [actionT_eq]
 simp [actionT_eq]
 change _ = (PiTensorProduct.map (fun i => rep (c i) g) ∘1
 (PiTensorProduct.map (fun i => rep (c i) g')) t
 rw [← PiTensorProduct.map_comp]
 rfl
 rw [actionT_eq, Pure.toTensor, PiTensorProduct.map_tprod]
noncomputable instance : DistribMulAction G (S.Tensor c) where
 smul_zero g := by simp [actionT_zero]
 smul_add g t1 t2 := by simp [actionT_add]
/-- Given a permutation `σ : Fin m → Fin n` of indices satisfying `IsReindexing` th
 (σ : Fin m → Fin n) (h : IsReindexing c c1 σ) (p : Pure S c) : Pure S c1 :=
 fun i => LinearEquiv.cast (R := k) (by simp [h.preserve_color]) (p (σ i))
 (σ : Fin m → Fin n) (h : IsReindexing c c1 σ) (φ : ComponentIdx (S := S) c) :
 Pure.permP σ h (Pure.basisVector c φ) =

```

```

 Pure.basisVector c1 (fun i => basisIdxCongr (by simp [h.preserve_color])
 LinearEquiv.cast (R := k) (by simp [h]) ((b (c1)) x) =
 (b c2) (basisIdxCongr (by simp [h]) x) := by
lemma Pure.permP_equivariant {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) (g : G) (p : Pure S c) :
 Pure.permP σ h (g • p) = g • Pure.permP σ h p := by
 ext i
 simp only [permP, actionP_eq]
 generalize p (σ i) = p
 generalize_proofs h1 h2
 generalize c1 i = c' at *
 generalize c (σ i) = c at *
 subst h2
 simp

/-- Given a permutation `σ : Fin m → Fin n` of indices satisfying `IsReindexing
 (σ : Fin m → Fin n) (h : IsReindexing c c1 σ) : S.Tensor c →1[k] S.Tensor c1
 PiTensorProduct.map (fun i => LinearEquiv.cast (R := k) (M := V)
 (by simp : c (h.toEquiv.symm i) = c1 i)) ∘1
 (PiTensorProduct.reindex k _ h.toEquiv).toLinearMap
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) (p : Pure S c) :
 simp only [permT, Pure.toTensor, LinearMap.coe_comp, LinearEquiv.coe_coe,
 PiTensorProduct.reindex_tprod, PiTensorProduct.map_tprod, LinearEquiv.coe_coe,
 Pure.permP (id : Fin n → Fin n) (by simp : IsReindexing c c id) p = p :
 simp [Pure.permP]
 permT (id : Fin n → Fin n) (by simp : IsReindexing c c id) t = t := by
 let P (t : S.Tensor c) := permT (id : Fin n → Fin n) (by simp : IsReindexing
 (σ : Fin n → Fin n) (hσ : IsReindexing c c σ) (h : σ = id) :
 (σ : Fin n → Fin n) (hσ : IsReindexing c c σ) (h : σ = id) (ht : t = t1)
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) (g : G) (t : S.Tensor c)
 apply induction_on_pure (t := t)
 · intro p
 simp [actionT_pure, permT_pure]
 congr
 simp [Pure.permP_equivariant]
 · intro r t ht
 rw [actionT_smul, map_smul]
 simp [ht]
 · intro t1 t2 h1 h2
 simp [h1, h2]
 {σ σ1 : Fin m → Fin n} {h : IsReindexing c c1 σ} {h1 : IsReindexing c c1 σ1}
 {σ σ1 : Fin m → Fin n} {h : IsReindexing c c1 σ} {h1 : IsReindexing c c1 σ1}
 {σ : Fin m1 → Fin n} {σ2 : Fin m2 → Fin m1} (h : IsReindexing c c1 σ)
 (h2 : IsReindexing c1 c2 σ2) (p : Pure S c) :
 Pure.permP σ2 h2 (Pure.permP σ h p) = Pure.permP (σ ∘ σ2) (h.comp h2) p
 simp [permP, Pure.permP, Function.comp_apply]

```

```

 {σ : Fin m1 → Fin n} {σ2 : Fin m2 → Fin m1} (h : IsReindexing c c1 σ)
 (h2 : IsReindexing c1 c2 σ2) (t : S.Tensor c) :
 permT σ2 h2 (permT σ h t) = permT (σ ∘ σ2) (h.comp h2) t := by
 let P (t : S.Tensor c) := permT σ2 h2 (permT σ h t) = permT (σ ∘ σ2) (h.comp h2) t
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) (t : S.Tensor c)
 (φ : ComponentIdx c1) :
 (basis c1).repr (permT σ h t) φ =
 basisIdxCongr (by simp [IsReindexing.inv_perserve_color]) (φ (h.inv σ i)) :=
 simp [← h'] using ComponentIdx.congr_right _ _ _
 (IsReindexing.inv_apply_apply σ h x).symm
 simp [h'] using (ComponentIdx.congr_right _ _ _
 (IsReindexing.apply_inv_apply σ h x).symm).symm
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ)
 refine Eq.symm (ComponentIdx.congr_right b' i (IsReindexing.inv σ _ (σ i)) ?_)
 simp [IsReindexing.apply_inv_apply]
 simp [IsReindexing.inv_apply_apply]
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) (t : S.Tensor c) :
 trans permT (h.inv σ) (IsReindexing.symm h) ((permT σ h) t)
 simp [IsReindexing.inv_apply_apply]

lemma toField_injective {c : Fin 0 → C} :
 Function.Injective (toField : S.Tensor c → k) :=
 (PiTensorProduct.isEmptyEquiv (Fin 0)).injective
lemma toField_permT {c c1 : Fin 0 → C} (σ : Fin 0 → Fin 0)
 (h : IsReindexing c c1 σ) (t : S.Tensor c) :
lemma toField_basis {c : Fin 0 → C} (b : ComponentIdx (S := S) c) :
 toField (basis c b) = 1 := by
 simp [basis_apply]

 simp [hp1, hp2]

lemma eq_smul_toField {c : Fin 0 → C} (t : Tensor S c) :
 t = toField t • (basis c (@default (ComponentIdx (S := S) c) Unique.instInhabite
 apply toField_injective
 simp

```

## 1.198 Physlib/Relativity/Tensors/ComplexTensor/Basic.lean

```

 /-- The color associated with dual-Left handed fermions. -
 /
 /-- The color associated with dual-Right handed fermions. -
 /
 deriving Fintype
 /-- The modules associated with each of the different types of complex Lorentz vector

```

```

/
abbrev modules : Color → Type
| Color.upL => Fermion.LeftHandedWeyl
| Color.downL => Fermion.DualLeftHandedWeyl
| Color.upR => Fermion.RightHandedWeyl
| Color.downR => Fermion.DualRightHandedWeyl
| Color.up => Lorentz.ContrCModule
| Color.down => Lorentz.CoCModule

instance modulesAddCommGroup : ∀ c, AddCommGroup (modules c)
| Color.upL => inferInstance
| Color.downL => inferInstance
| Color.upR => inferInstance
| Color.downR => inferInstance
| Color.up => inferInstance
| Color.down => inferInstance

noncomputable instance modulesModule : ∀ c, Module ℂ (modules c)
| Color.upL => inferInstance
| Color.downL => inferInstance
| Color.upR => inferInstance
| Color.downR => inferInstance
| Color.up => inferInstance
| Color.down => inferInstance

set_option maxHeartbeats 0 in
 (fun c => match c with
 | Color.upL => Fermion.LeftHandedWeyl
 | Color.downL => Fermion.DualLeftHandedWeyl
 | Color.upR => Fermion.RightHandedWeyl
 | Color.downR => Fermion.DualRightHandedWeyl
 | Color.up => Lorentz.ContrCModule
 | Color.down => Lorentz.CoCModule)
 (fun c => Fin (repDim c))
 (fun c => match c with
 | Color.upL => Fermion.leftHandedRep
 | Color.downL => Fermion.dualLeftHandedRep
 | Color.upR => Fermion.rightHandedRep
 | Color.downR => Fermion.dualRightHandedRep
 | Color.up => Lorentz.ContrCModule.SL2CRep
 | Color.down => Lorentz.CoCModule.SL2CRep)
 (fun c => match c with
 | Color.upL => Fermion.leftBasis
 | Color.downL => Fermion.dualLeftBasis
 | Color.upR => Fermion.rightBasis
 | Color.downR => Fermion.dualRightBasis

```

```

 | Color.up => Lorentz.complexContrBasisFin4
 | Color.down => Lorentz.complexCoBasisFin4) where

contr := fun c =>
 | Color.upL => Fermion.leftDualContraction
 | Color.downL => Fermion.dualLeftContraction
 | Color.upR => Fermion.rightDualContraction
 | Color.downR => Fermion.dualRightContraction
 | Color.up => Lorentz.contrCoContraction
 | Color.down => Lorentz.coContrContraction
metric := fun c =>
 | Color.upL => Fermion.leftMetric
 | Color.downL => Fermion.dualLeftMetric
 | Color.upR => Fermion.rightMetric
 | Color.downR => Fermion.dualRightMetric
 | Color.up => Lorentz.contrMetric
 | Color.down => Lorentz.coMetric
unit := fun c =>
 | Color.upL => Fermion.dualLeftLeftUnit
 | Color.downL => Fermion.leftDualLeftUnit
 | Color.upR => Fermion.dualRightRightUnit
 | Color.downR => Fermion.rightDualRightUnit
 | Color.up => Lorentz.coContrUnit
 | Color.down => Lorentz.contrCoUnit
 | Color.upL => Fermion.leftDualContraction_tmul_symm
 | Color.downL => Fermion.dualLeftContraction_tmul_symm
 | Color.upR => Fermion.rightDualContraction_tmul_symm
 | Color.downR => Fermion.dualRightContraction_tmul_symm
 | Color.upL => Fermion.contr_dualLeftLeftUnit
 | Color.downL => Fermion.contr_leftDualLeftUnit
 | Color.upR => Fermion.contr_dualRightRightUnit
 | Color.downR => Fermion.contr_rightDualRightUnit
 | Color.upL => Fermion.dualLeftLeftUnit_symm
 | Color.downL => Fermion.leftDualLeftUnit_symm
 | Color.upR => Fermion.dualRightRightUnit_symm
 | Color.downR => Fermion.rightDualRightUnit_symm
 simp using Fermion.leftDualContraction_apply_metric
 simp using Fermion.dualLeftContraction_apply_metric
 simp using Fermion.rightDualContraction_apply_metric
 simp using Fermion.dualRightContraction_apply_metric
/-- The basis associated with each of the different types of complex Lorentz vector
/
abbrev basis (c : Color) : Module.Basis (Fin (repDim c)) ℂ (modules c) :=
 match c with
 | Color.upL => Fermion.leftBasis
 | Color.downL => Fermion.dualLeftBasis

```

```

| Color.upR => Fermion.rightBasis
| Color.downR => Fermion.dualRightBasis
| Color.up => Lorentz.complexContrBasisFin4
| Color.down => Lorentz.complexCoBasisFin4

/-- The reps associated with each of the different types of complex Lorentz
/
abbrev rep (c : Color) : Representation $\mathbb{C}$ SL(2, $\mathbb{C}$) (modules c) :=
 match c with
 | Color.upL => Fermion.leftHandedRep
 | Color.downL => Fermion.dualLeftHandedRep
 | Color.upR => Fermion.rightHandedRep
 | Color.downR => Fermion.dualRightHandedRep
 | Color.up => Lorentz.Contr $\mathbb{C}$ Module.SL2CRep
 | Color.down => Lorentz.Co $\mathbb{C}$ Module.SL2CRep

open TensorSpecies Tensor
lemma contrPCoeff_basis {n : $\mathbb{N}$ } {c : Fin n $\rightarrow$ complexLorentzTensor.Color} (i
 (hij : i $\neq$ j $\wedge$ (complexLorentzTensor. τ (c i) = c j))
 (b : ComponentIdx (S := complexLorentzTensor) c) :
 Pure.contrPCoeff i j hij (Pure.basisVector c b) = if b i =
 Fin.cast (by simp [← hij.2, repDim_tau]) (b j)
 then 1 else 0 := by
 simp only [Pure.contrPCoeff, Pure.basisVector]
 generalize_proofs h1 h2
 generalize b i = b1 at *
 generalize b j = b2 at *
 generalize c i = ci at *
 generalize c j = cj at *
 subst h2
 fin_cases ci
 · simp [complexLorentzTensor]
 erw [LinearEquiv.cast_apply]
 simp only [cast_eq]
 erw [Fermion.leftDualContraction_basis]
 grind
 · simp [complexLorentzTensor]
 erw [LinearEquiv.cast_apply]
 simp only [cast_eq]
 erw [Fermion.dualLeftContraction_basis]
 grind
 · simp [complexLorentzTensor]
 erw [LinearEquiv.cast_apply]
 simp only [cast_eq]
 erw [Fermion.dualRightContraction_basis]
 grind

```

### 1.199. *PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/LEMMAS.LEAN*

```

• simp [complexLorentzTensor]
 erw [LinearEquiv.cast_apply]
 simp only [cast_eq]
 erw [Fermion.dualLeftContraction_basis]
 grind
• simp [complexLorentzTensor]
 erw [LinearEquiv.cast_apply]
 simp only [cast_eq]
 erw [Lorentz.coContrContraction_basis]
 grind
• simp [complexLorentzTensor]
 erw [LinearEquiv.cast_apply]
 simp only [cast_eq]
 erw [Lorentz.contrCoContraction_basis]
 grind

```

### 1.199 Physlib/Relativity/Tensors/ComplexTensor/Lemmas.lean

```

 rw [hA, hs, prodT_permT_left, prodT_permT_right, contrT_comm, permT_permT,
 contrT_permT, contrT_permT, permT_permT]
 simp only [LinearMap.neg_apply, map_neg]
 congr 1
 apply permT_congr_eq_id
 decide

```

### 1.200 Physlib/Relativity/Tensors/ComplexTensor/Matrix/Pre.lean

```

def contrContrToMatrix : (ContrModule * [C] ContrModule) ≈1 [C]
 exact congrArg _ (Basis.tensorProduct_apply complexContrBasis complexContrBasis)
def coCoToMatrix : (CoModule * [C] CoModule) ≈1 [C]
 exact congrArg _ (Basis.tensorProduct_apply complexCoBasis complexCoBasis i j)
def contrCoToMatrix : (ContrModule * [C] CoModule) ≈1 [C]
 exact congrArg _ (Basis.tensorProduct_apply complexContrBasis complexCoBasis i j)
def coContrToMatrix : (CoModule * [C] ContrModule) ≈1 [C]
 exact congrArg _ (Basis.tensorProduct_apply complexCoBasis complexContrBasis i j)
lemma contrContrToMatrix_p (v : (ContrModule * [C] ContrModule)) (M : SL(2,C)) :
 contrContrToMatrix (TensorProduct.map (ContrModule.SL2CRep M) (ContrModule.SL2CRep M))
 (TensorProduct.map (ContrModule.SL2CRep M) (ContrModule.SL2CRep M))
 (TensorProduct.map (ContrModule.SL2CRep M) (ContrModule.SL2CRep M)) v)
 ((LinearMap.toMatrix complexContrBasis complexContrBasis) (ContrModule.SL2CRep M))
 ((LinearMap.toMatrix complexContrBasis complexContrBasis)
 (ContrModule.SL2CRep M)) (i, j) k)

```

```

lemma coCoToMatrix_p (v : (CoCModule ⊗[C] CoCModule)) (M : SL(2, C)) :
 coCoToMatrix (TensorProduct.map (CoCModule.SL2CRep M) (CoCModule.SL2CRep M)
 (TensorProduct.map (CoCModule.SL2CRep M) (CoCModule.SL2CRep M))
 (TensorProduct.map (CoCModule.SL2CRep M) (CoCModule.SL2CRep M)) v)
 ((LinearMap.toMatrix complexCoBasis complexCoBasis) (CoCModule.SL2CRep M))
 ((LinearMap.toMatrix complexCoBasis complexCoBasis) (CoCModule.SL2CRep M))
lemma contrCoToMatrix_p (v : (ContrCModule ⊗[C] CoCModule)) (M : SL(2, C)) :
 contrCoToMatrix (TensorProduct.map (ContrCModule.SL2CRep M) (CoCModule.SL2CRep M)
 (TensorProduct.map (ContrCModule.SL2CRep M) (CoCModule.SL2CRep M))
 (TensorProduct.map (ContrCModule.SL2CRep M) (CoCModule.SL2CRep M)) v)
 ((LinearMap.toMatrix complexContrBasis complexContrBasis) (ContrCModule.SL2CRep M))
 ((LinearMap.toMatrix complexCoBasis complexCoBasis) (CoCModule.SL2CRep M))
lemma coContrToMatrix_p (v : (CoCModule ⊗[C] ContrCModule)) (M : SL(2, C)) :
 coContrToMatrix (TensorProduct.map (CoCModule.SL2CRep M) (ContrCModule.SL2CRep M)
 (TensorProduct.map (CoCModule.SL2CRep M) (ContrCModule.SL2CRep M))
 (TensorProduct.map (CoCModule.SL2CRep M) (ContrCModule.SL2CRep M)) v)
 ((LinearMap.toMatrix complexCoBasis complexCoBasis) (CoCModule.SL2CRep M))
 ((LinearMap.toMatrix complexContrBasis complexContrBasis) (ContrCModule.SL2CRep M))
 (i, j) k
 TensorProduct.map (ContrCModule.SL2CRep M) (ContrCModule.SL2CRep M)
 (contrContrToMatrix.symm v) =
rw [← h1, LinearEquiv.symm_apply_apply]
 TensorProduct.map (CoCModule.SL2CRep M) (CoCModule.SL2CRep M) (coCoToMatrix_p v)
rw [← h1, LinearEquiv.symm_apply_apply]
 TensorProduct.map (ContrCModule.SL2CRep M) (CoCModule.SL2CRep M) (contrCoToMatrix_p v)
rw [← h1, LinearEquiv.symm_apply_apply]
 TensorProduct.map (CoCModule.SL2CRep M) (ContrCModule.SL2CRep M) (coContrToMatrix_p v)
rw [← h1, LinearEquiv.symm_apply_apply]

```

### 1.201 Physlib/Relativity/Tensors/ComplexTensor/Metrics/Basic.lean

```

abbrev dualLeftMetric : CT[.downL, .downL] := complexLorentzTensor.metricTensor
abbrev dualRightMetric : CT[.downR, .downR] := complexLorentzTensor.metricTensor
scoped[complexLorentzTensor] notation "εL'" => dualLeftMetric
scoped[complexLorentzTensor] notation "εR'" => dualRightMetric
lemma coMetric_eq_fromConstPair : η' = fromConstPair (S := complexLorentzTensor)
 (c1 := .down) (c2 := .down) Lorentz.coMetric := by
lemma contrMetric_eq_fromConstPair : η = fromConstPair (S := complexLorentzTensor)
 (c1 := .up) (c2 := .up) Lorentz.contrMetric := by
lemma dualLeftMetric_eq_fromConstPair : εL' = fromConstPair Fermion.dualLeftMetric
lemma dualRightMetric_eq_fromConstPair : εR' = fromConstPair Fermion.dualRightMetric
lemma dualLeftMetric_eq_fromPairT : εL' = fromPairT (Fermion.dualLeftMetric)
 rw [dualLeftMetric_eq_fromConstPair, fromConstPair]
 exact Fermion.dualLeftMetric_apply_one

```



1.202.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/METRICS/LEMMA971LEAN

```
lemma dualRightMetric_eq_fromPairT : $\varepsilon R' = \text{fromPairT (Fermion.dualRightMetricVal) :=$
 rw [dualRightMetric_eq_fromConstPair, fromConstPair]
 exact Fermion.dualRightMetric_apply_one
lemma dualLeftMetric_eq_dualLeftBasis : $\varepsilon L' =$
 fromPairT (dualLeftBasis 0 $\otimes_t [\mathbb{C}]$ dualLeftBasis 1)
 - fromPairT (dualLeftBasis 1 $\otimes_t [\mathbb{C}]$ dualLeftBasis 0) := by
 rw [dualLeftMetric_eq_fromPairT, dualLeftMetricVal_expand_tmul]
lemma dualRightMetric_eq_dualRightBasis : $\varepsilon R' =$
 fromPairT (dualRightBasis 0 $\otimes_t [\mathbb{C}]$ dualRightBasis 1)
 - fromPairT (dualRightBasis 1 $\otimes_t [\mathbb{C}]$ dualRightBasis 0) := by
 rw [dualRightMetric_eq_fromPairT, dualRightMetricVal_expand_tmul]
lemma dualLeftMetric_eq_basis : $\varepsilon L' =$
 rw [dualLeftMetric_eq_dualLeftBasis]
lemma dualRightMetric_eq_basis : $\varepsilon R' =$
 rw [dualRightMetric_eq_dualRightBasis]
lemma dualLeftMetric_eq_ofRat : $\varepsilon L' = \text{ofRat fun f =>}$
 rw [dualLeftMetric_eq_basis]
lemma dualRightMetric_eq_ofRat : $\varepsilon R' = \text{ofRat fun f =>}$
 rw [dualRightMetric_eq_basis]
/-- The tensor `dualLeftMetric` is invariant under the action of `SL(2,ℂ)`. -
/
lemma actionT_dualLeftMetric (g : SL(2,ℂ)) : g • $\varepsilon L' = \varepsilon L' :=$ by
/-- The tensor `dualRightMetric` is invariant under the action of `SL(2,ℂ)`. -
/
lemma actionT_dualRightMetric (g : SL(2,ℂ)) : g • $\varepsilon R' = \varepsilon R' :=$ by
```

## 1.202 Physlib/Relativity/Tensors/ComplexTensor/Metrics/Lemmas.lean

```
/-- The dual-left metric is antisymmetric $\{\varepsilon L' \mid \alpha \alpha' = - \varepsilon L' \mid \alpha' \alpha\}^T$. -
/
lemma dualLeftMetric_antisymm : $\{\varepsilon L' \mid \alpha \alpha' = - (\varepsilon L' \mid \alpha' \alpha)\}^T :=$ by
 rw [dualLeftMetric_eq_ofRat, ofRat_basis_repr_apply, ← map_neg, ofRat_basis_repr_a
/-- The dual-right metric is antisymmetric $\{\varepsilon R' \mid \beta \beta' = - \varepsilon R' \mid \beta' \beta\}^T$. -
/
lemma dualRightMetric_antisymm : $\{\varepsilon R' \mid \alpha \alpha' = - (\varepsilon R' \mid \alpha' \alpha)\}^T :=$ by
 rw [dualRightMetric_eq_ofRat, ofRat_basis_repr_apply, ← map_neg, ofRat_basis_repr_a
/-- The contraction of the left metric with the dual-left metric is the unit
lemma leftMetric_contr_dualLeftMetric : $\{\varepsilon L \mid \alpha \beta \otimes \varepsilon L' \mid \beta \gamma = \delta L \mid \alpha \gamma\}^T :=$ by
/-- The contraction of the right metric with the dual-right metric is the unit
lemma rightMetric_contr_dualRightMetric : $\{\varepsilon R \mid \alpha \beta \otimes \varepsilon R' \mid \beta \gamma = \delta R \mid \alpha \gamma\}^T :=$ by
/-- The contraction of the dual-left metric with the left metric is the unit
lemma dualLeftMetric_contr_leftMetric : $\{\varepsilon L' \mid \alpha \beta \otimes \varepsilon L \mid \beta \gamma = \delta L' \mid \alpha \gamma\}^T :=$ by
/-- The contraction of the dual-right metric with the right metric is the unit
lemma dualRightMetric_contr_rightMetric : $\{\varepsilon R' \mid \alpha \beta \otimes \varepsilon R \mid \beta \gamma = \delta R' \mid \alpha \gamma\}^T :=$ by
```

### 1.203 Physlib/Relativity/Tensors/ComplexTensor/Metrics/Pre.lean

```

def contrMetricVal : (ContrModule $\otimes$ [$\mathbb{C}$] ContrModule) :=
 simp only [map_apply, ofRealHom_eq_coe, Fintype.sum_sum_type, Finset.univ,
 not_false_eq_true, minkowskiMatrix.off_diag_zero, Sum.inr.injEq, zero_n,
 one_ne_zero]
 simp only [ofReal_one, Fin.isValue, one_smul, ofReal_zero, zero_smul, add,
 zero_add]
 module
def contrMetric : (Representation.trivial $\mathbb{C}$ SL(2, $\mathbb{C}$) $\mathbb{C}$).IntertwiningMap
 (ContrModule.SL2CRep.tprod ContrModule.SL2CRep) where
 a' • contrMetricVal
 simp only [add_smul]
 (TensorProduct.map (ContrModule.SL2CRep M) (ContrModule.SL2CRep M))
lemma contrMetric_apply_one : contrMetric (1 : $\mathbb{C}$) = contrMetricVal := by
def coMetricVal : (CoModule $\otimes$ [$\mathbb{C}$] CoModule) :=
 simp only [map_apply, ofRealHom_eq_coe, Fintype.sum_sum_type, Finset.univ,
 not_false_eq_true, minkowskiMatrix.off_diag_zero, Sum.inr.injEq, zero_n,
 one_ne_zero]
 simp only [ofReal_one, Fin.isValue, one_smul, ofReal_zero, zero_smul, add,
 zero_add]
 module
def coMetric : (Representation.trivial $\mathbb{C}$ SL(2, $\mathbb{C}$) $\mathbb{C}$).IntertwiningMap
 (CoModule.SL2CRep.tprod CoModule.SL2CRep) where
 toFun := fun a =>
 let a' : $\mathbb{C}$:= a
 a' • coMetricVal
 map_add' := fun x y => by
 simp only [add_smul]
 map_smul' := fun m x => by
 simp only [smul_smul]
 rfl
 isIntertwining' M := by
 refine LinearMap.ext fun x : $\mathbb{C}$ => ?_
 change x • coMetricVal =
 (TensorProduct.map (CoModule.SL2CRep M) (CoModule.SL2CRep M)) (x •
 simp only [map_smul]
 apply congrArg
 simp only [coMetricVal]
 rw [coCoToMatrix_p_symm]
 apply congrArg
 rw [LorentzGroup.toComplex_inv]
 simp only [LorentzGroup.inv_eq_dual, SL2C.toLorentzGroup_apply_coe,

```

#### 1.204. *PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/OF RAT. LEAN*

```

LorentzGroup.toComplex_transpose_mul_minkowskiMatrix_mul_self]

lemma coMetric_apply_one : coMetric (1 : ℂ) = coMetricVal := by
lemma contrCoContraction_apply_metric :
 (TensorProduct.comm ℂ _ _ <|
 (TensorProduct.lid ℂ _).lTensor _ <|
 (contrCoContraction.toLinearMap.rTensor _).lTensor _ <|
 (TensorProduct.assoc ℂ _ _).symm.toLinearMap.lTensor _ <|
 TensorProduct.assoc ℂ _ _ (_ ⊗[ℂ] _) <|
 (coMetric 1) ⊗[ℂ] (coMetric 1)) = coContrUnit (1 : ℝ) := by
 simp [Fin.isValue, tmul_sub, sub_tmul, map_sub]
 simp only [← Representation.IntertwiningMap.toLinearMap_apply]
lemma coContrContraction_apply_metric :
 (TensorProduct.comm ℂ _ _ <|
 (TensorProduct.lid ℂ _).lTensor _ <|
 (coContrContraction.toLinearMap.rTensor _).lTensor _ <|
 (TensorProduct.assoc ℂ _ _).symm.toLinearMap.lTensor _ <|
 TensorProduct.assoc ℂ _ _ (_ ⊗[ℂ] _) <|
 (coMetric 1) ⊗[ℂ] (coMetric 1)) = contrCoUnit (1 : ℝ) := by
 simp [Fin.isValue, tmul_sub, sub_tmul, map_sub]
 simp only [← Representation.IntertwiningMap.toLinearMap_apply]
 simp [Fin.isValue, ↓reduceIte, one_smul, reduceCtorEq, Sum.inr.injEq, one_ne_zero,
```

#### 1.204 Physlib/Relativity/Tensors/ComplexTensor/OfRat.lean

```

open Module
 ((complexLorentzTensor.contr c)
 change Fermion.leftDualContraction (Fermion.leftBasis i ⊗ₜ Fermion.dualLeftBasis
 rw [Fermion.leftDualContraction_basis]
 change Fermion.dualLeftContraction (Fermion.dualLeftBasis i ⊗ₜ Fermion.leftBasis
 rw [Fermion.dualLeftContraction_basis]
 change Fermion.rightDualContraction (Fermion.rightBasis i ⊗ₜ Fermion.dualRightBa
 rw [Fermion.rightDualContraction_basis]
 change Fermion.rightDualContraction (Fermion.rightBasis i ⊗ₜ Fermion.dualRightBa
 rw [Fermion.rightDualContraction_basis]
 change Lorentz.contrCoContraction
 change Lorentz.contrCoContraction
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ)
 ((ofRat (fun b => f (fun i => Fin.cast (by simp [IsReindexing.inv_perserve_color
```

#### 1.205 Physlib/Relativity/Tensors/ComplexTensor/Units/Basic.lean

```

abbrev dualLeftLeftUnit : CT[.downL, .upL] := complexLorentzTensor.unitTensor Color.
```

```

abbrev leftDualLeftUnit : CT[.upL, .downL] := complexLorentzTensor.unitTens
abbrev dualRightRightUnit : CT[.downR, .upR] := complexLorentzTensor.unitTe
abbrev rightDualRightUnit : CT[.upR, .downR] := complexLorentzTensor.unitTe
scoped[complexLorentzTensor] notation "δL'" => dualLeftLeftUnit
scoped[complexLorentzTensor] notation "δL" => leftDualLeftUnit
scoped[complexLorentzTensor] notation "δR'" => dualRightRightUnit
scoped[complexLorentzTensor] notation "δR" => rightDualRightUnit
lemma dualLeftLeftUnit_eq_fromConstPair : δL' = fromConstPair Fermion.dualL
 rw [Fermion.dualLeftLeftUnit]
lemma leftDualLeftUnit_eq_fromConstPair : δL = fromConstPair Fermion.leftDu
 rw [Fermion.leftDualLeftUnit]
lemma dualRightRightUnit_eq_fromConstPair : δR' = fromConstPair Fermion.dua
 rw [Fermion.dualRightRightUnit]
lemma rightDualRightUnit_eq_fromConstPair : δR = fromConstPair Fermion.righ
 rw [Fermion.rightDualRightUnit]
lemma dualLeftLeftUnit_eq_fromPairT : δL' = fromPairT (Fermion.dualLeftLeft
 rw [dualLeftLeftUnit_eq_fromConstPair, fromConstPair]
 exact Fermion.dualLeftLeftUnit_apply_one
lemma leftDualLeftUnit_eq_fromPairT : δL = fromPairT (Fermion.leftDualLeftU
 rw [leftDualLeftUnit_eq_fromConstPair, fromConstPair]
 exact Fermion.leftDualLeftUnit_apply_one
lemma dualRightRightUnit_eq_fromPairT : δR' = fromPairT (Fermion.dualRightR
 rw [dualRightRightUnit_eq_fromConstPair, fromConstPair]
 exact Fermion.dualRightRightUnit_apply_one
lemma rightDualRightUnit_eq_fromPairT : δR = fromPairT (Fermion.rightDualRi
 rw [rightDualRightUnit_eq_fromConstPair, fromConstPair]
 exact Fermion.rightDualRightUnit_apply_one
lemma dualLeftLeftUnit_eq_dualLeftBasis_leftBasis : δL' =
 ∑ i, fromPairT (dualLeftBasis i ⊗+[C] leftBasis i) := by
 rw [dualLeftLeftUnit_eq_fromPairT, dualLeftLeftUnitVal_expand_tmul]
lemma leftDualLeftUnit_eq_leftBasis_dualLeftBasis : δL =
 ∑ i, fromPairT (leftBasis i ⊗+[C] dualLeftBasis i) := by
 rw [leftDualLeftUnit_eq_fromPairT, leftDualLeftUnitVal_expand_tmul]
lemma dualRightRightUnit_eq_dualRightBasis_rightBasis : δR' =
 ∑ i, fromPairT (dualRightBasis i ⊗+[C] rightBasis i) := by
 rw [dualRightRightUnit_eq_fromPairT, dualRightRightUnitVal_expand_tmul]
lemma rightDualRightUnit_eq_rightBasis_dualRightBasis : δR =
 ∑ i, fromPairT (rightBasis i ⊗+[C] dualRightBasis i) := by
 rw [rightDualRightUnit_eq_fromPairT, rightDualRightUnitVal_expand_tmul]
lemma dualLeftLeftUnit_eq_basis : δL' =
 rw [dualLeftLeftUnit_eq_dualLeftBasis_leftBasis]
lemma leftDualLeftUnit_eq_basis : δL =
 rw [leftDualLeftUnit_eq_leftBasis_dualLeftBasis]
lemma dualRightRightUnit_eq_basis : δR' =
 rw [dualRightRightUnit_eq_dualRightBasis_rightBasis]
lemma rightDualRightUnit_eq_basis : δR =

```

1.206.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/UNITS/PRE. LEAN#81

```
rw [rightDualRightUnit_eq_rightBasis_dualRightBasis]
lemma dualLeftLeftUnit_eq_ofRat : $\delta L' = \text{ofRat}$ fun f =>
 rw [dualLeftLeftUnit_eq_basis]
lemma leftDualLeftUnit_eq_ofRat : $\delta L = \text{ofRat}$ fun f =>
 rw [leftDualLeftUnit_eq_basis]
lemma dualRightRightUnit_eq_ofRat : $\delta R' = \text{ofRat}$ fun f =>
 rw [dualRightRightUnit_eq_basis]
lemma rightDualRightUnit_eq_ofRat : $\delta R = \text{ofRat}$ fun f =>
 rw [rightDualRightUnit_eq_basis]
/-- The tensor `dualLeftLeftUnit` is invariant under the action of `SL(2,ℂ)`. -
/
lemma actionT_dualLeftLeftUnit (g : SL(2,ℂ)) : g • $\delta L' = \delta L'$:= by
/- The tensor `leftDualLeftUnit` is invariant under the action of `SL(2,ℂ)`. -
/
lemma actionT_leftDualLeftUnit (g : SL(2,ℂ)) : g • $\delta L = \delta L$:= by
/- The tensor `dualRightRightUnit` is invariant under the action of `SL(2,ℂ)`. -
/
lemma actionT_dualRightRightUnit (g : SL(2,ℂ)) : g • $\delta R' = \delta R'$:= by
/- The tensor `rightDualRightUnit` is invariant under the action of `SL(2,ℂ)`. -
/
lemma actionT_rightDualRightUnit (g : SL(2,ℂ)) : g • $\delta R = \delta R$:= by
```

## 1.206 Physlib/Relativity/Tensors/ComplexTensor/Units/Pre.lean

```
def contrCoUnitVal : ContrModule $\otimes$ [ℂ] CoModule :=
def contrCoUnit : (Representation.trivial $\mathbb{C}$ SL(2,ℂ) $\mathbb{C}$).IntertwiningMap
 (ContrModule.SL2CRep.tprod CoModule.SL2CRep) where
 toFun := fun a =>
 let a' : $\mathbb{C}$:= a
 a' • contrCoUnitVal
 map_add' := fun x y => by
 simp only [add_smul]
 map_smul' := fun m x => by
 simp only [smul_smul]
 rfl
 isIntertwining' M := by
 refine LinearMap.ext fun x : $\mathbb{C}$ => ?_
 change x • contrCoUnitVal =
 (TensorProduct.map (ContrModule.SL2CRep M) (CoModule.SL2CRep M)) (x • contrCoUnitVal)
 simp only [map_smul]
 apply congrArg
 simp only [contrCoUnitVal]
 rw [contrCoToMatrix_ρ_symm]
 apply congrArg
```

```

simp

lemma contrCoUnit_apply_one : contrCoUnit (1 : $\mathbb{C}$) = contrCoUnitVal := by
def coContrUnitVal : CoCModule $\otimes$ [$\mathbb{C}$] ContrCModule :=
def coContrUnit : (Representation.trivial $\mathbb{C}$ SL(2, $\mathbb{C}$) $\mathbb{C}$).IntertwiningMap
 (CoCModule.SL2CRep.tprod ContrCModule.SL2CRep) where
 toFun := fun a =>
 let a' : $\mathbb{C}$:= a
 a' • coContrUnitVal
 map_add' := fun x y => by
 simp only [add_smul]
 map_smul' := fun m x => by
 simp only [smul_smul]
 rfl
 isIntertwining' M := by
 refine LinearMap.ext fun x : $\mathbb{C}$ => ?_
 change x • coContrUnitVal =
 (TensorProduct.map (CoCModule.SL2CRep M) (ContrCModule.SL2CRep M)) (x)
 simp only [map_smul]
 apply congrArg
 simp only [coContrUnitVal]
 rw [coContrToMatrix_ρ_symm]
 apply congrArg
 symm
 refine transpose_eq_one.mp ?h.h.h.a
 simp

lemma coContrUnit_apply_one : coContrUnit (1 : $\mathbb{C}$) = coContrUnitVal := by
lemma contr_contrCoUnit (x : CoCModule) :
 (TensorProduct.lid $\mathbb{C}$ _ <|
 coContrContraction.toLinearMap.rTensor _ <|
 (TensorProduct.assoc $\mathbb{C}$ _ _).symm <|
 x $\otimes$ [$\mathbb{C}$] (contrCoUnit (1 : $\mathbb{C}$))) = x := by
 simp [- Fintype.sum_sum_type, map_sum, tmul_sum, smul_tmul, coContrContra
 contrCoUnit_apply_one, contrCoUnitVal_eq_sum_tmul, sum_tmul]

lemma contr_coContrUnit (x : ContrCModule) :
 (TensorProduct.lid $\mathbb{C}$ _ <|
 contrCoContraction.toLinearMap.rTensor _ <|
 (TensorProduct.assoc $\mathbb{C}$ _ _).symm <|
 x $\otimes$ [$\mathbb{C}$] (coContrUnit (1 : $\mathbb{C}$))) = x := by
 simp [- Fintype.sum_sum_type, map_sum, tmul_sum, smul_tmul, contrCoContra
 coContrUnit_apply_one, coContrUnitVal_eq_sum_tmul, sum_tmul]
 contrCoUnit (1 : $\mathbb{C}$) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLine
 (TensorProduct.comm $\mathbb{C}$ _ _ (coContrUnit (1 : $\mathbb{C}$))) := by
 (coContrUnit (1 : $\mathbb{C}$) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLi

```

1.207.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/UNITS/SYMM.lean

```
(TensorProduct.comm $\mathbb{C}$ _ _ (contrCoUnit (1 : $\mathbb{C}$))) := by
```

## 1.207 Physlib/Relativity/Tensors/ComplexTensor/Units/Symm.lean

```
/-- Swapping indices of `dualLeftLeftUnit` returns
`leftDualLeftUnit`: $\{\delta L' \mid \alpha \alpha' = \delta L \mid \alpha' \alpha\}^T$. -/
lemma dualLeftLeftUnit_symm : $\{\delta L' \mid \alpha \alpha' = \delta L \mid \alpha' \alpha\}^T :=$ by
 rw [dualLeftLeftUnit, unitTensor_eq_permT_dual]
/-- Swapping indices of `leftDualLeftUnit` returns
`dualLeftLeftUnit`: $\{\delta L \mid \alpha \alpha' = \delta L' \mid \alpha' \alpha\}^T$. -/
lemma leftDualLeftUnit_symm : $\{\delta L \mid \alpha \alpha' = \delta L' \mid \alpha' \alpha\}^T :=$ by
 rw [leftDualLeftUnit, unitTensor_eq_permT_dual]
/-- Swapping indices of `dualRightRightUnit` returns `rightDualRightUnit`:
lemma dualRightRightUnit_symm : $\{\delta R' \mid \beta \beta' = \delta R \mid \beta' \beta\}^T :=$ by
 rw [dualRightRightUnit, unitTensor_eq_permT_dual]
/-- Swapping indices of `rightDualRightUnit` returns `dualRightRightUnit`:
lemma rightDualRightUnit_symm : $\{\delta R \mid \beta \beta' = \delta R' \mid \beta' \beta\}^T :=$ by
 rw [rightDualRightUnit, unitTensor_eq_permT_dual]
```

## 1.208 Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Basic.lean

Authors: Nikolai Kashcheev, Joseph Tooby-Smith

```
def complexContrBasis : Basis (Fin 1 $\oplus$ Fin 3) $\mathbb{C}$ ContrModule :=
 (LinearMap.toMatrix complexContrBasis complexContrBasis) (ContrModule.SL2CRep M)
lemma complexContrBasis_p_val (M : SL(2, $\mathbb{C}$)) (v : ContrModule) :
 ((ContrModule.SL2CRep M) v).val =
def complexContrBasisFin4 : Basis (Fin 4) $\mathbb{C}$ ContrModule :=
def complexCoBasis : Basis (Fin 1 $\oplus$ Fin 3) $\mathbb{C}$ CoModule :=
 (LinearMap.toMatrix complexCoBasis complexCoBasis) (CoModule.SL2CRep M) i j =
lemma CoModule.SL2CRep_val (M : SL(2, $\mathbb{C}$)) (v : CoModule) :
 ((CoModule.SL2CRep M) v).val =
 (LorentzGroup.toComplex (SL2C.toLorentzGroup M))-1T *v v.val := by
 rfl

def complexCoBasisFin4 : Basis (Fin 4) $\mathbb{C}$ CoModule :=
def inclCongrRealLorentz : ContrMod 3 $\rightarrow_{SL}$ [Complex.ofRealHom] ContrModule where
 (ContrModule.SL2CRep M) (inclCongrRealLorentz v) =
 inclCongrRealLorentz (ContrMod.rep (SL2C.toLorentzGroup M) v) := by
 (ContrModule.SL2CRep M) (complexContrBasis i) =
/-- The semilinear map including real Lorentz co-vectors into complex covariant
Lorentz vectors. -/
def inclCoRealLorentz : CoMod 3 $\rightarrow_{SL}$ [Complex.ofRealHom] CoModule where
 toFun v := { val := ofReal $\circ$ v.toFin1d $\mathbb{R}$ }
```

```

map_add' x y := by
 ext i
 rw [CoCModule.val_add]
 simp only [Function.comp_apply, Pi.add_apply, map_add]
 simp only [ofReal_add]
map_smul' c x := by
 ext i
 rw [CoCModule.val_smul]
 simp only [Function.comp_apply, ofRealHom_eq_coe, Pi.smul_apply, _root_
 simp only [smul_eq_mul, ofReal_mul]

lemma inclCoRealLorentz_val (v : CoMod 3) :
 (inclCoRealLorentz v).val = ofRealHom ∘ v.toFinldℝ := rfl

lemma complexCoBasis_of_real (i : Fin 1 ⊕ Fin 3) :
 (complexCoBasis i) = inclCoRealLorentz (CoMod.stdBasis i) := by
 apply CoCModule.ext
 simp only [complexCoBasis, Basis.coe_ofEquivFun, inclCoRealLorentz,
 LinearMap.coe_mk, AddHom.coe_mk]
 ext j
 simp only [Function.comp_apply]
 change (Pi.single i 1) j = _
 by_cases h : i = j
 · subst h
 rw [CoMod.toFinldℝ, CoMod.stdBasis_toFinldℝEquiv_apply_same]
 simp
 · rw [CoMod.toFinldℝ, CoMod.stdBasis_toFinldℝEquiv_apply_ne h]
 simp [h]

lemma inclCoRealLorentz_ρ (M : SL(2, ℂ)) (v : CoMod 3) :
 (CoCModule.SL2CRep M) (inclCoRealLorentz v) =
 inclCoRealLorentz (CoMod.rep (SL2C.toLorentzGroup M) v) := by
 ext i
 rw [CoCModule.SL2CRep_val, inclCoRealLorentz_val, inclCoRealLorentz_val]
 change ((LorentzGroup.toComplex (SL2C.toLorentzGroup M))-1T *v
 (ofRealHom ∘ v.toFinldℝ)) i =
 (ofRealHom ∘ ((LorentzGroup.transpose (SL2C.toLorentzGroup M)-1).1 *v
 v.toFinldℝ)) i
 rw [LorentzGroup.toComplex_inv]
 change (LorentzGroup.toComplex (LorentzGroup.transpose (SL2C.toLorentzGroup M))-1
 (ofRealHom ∘ v.toFinldℝ)) i =
 (ofRealHom ∘ ((LorentzGroup.transpose (SL2C.toLorentzGroup M)-1).1 *v
 v.toFinldℝ)) i
 rw [LorentzGroup.toComplex_mulVec_ofReal]

```



1.209.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/VECTOR/PRE/CONTRACTION. LEAN

## 1.209 Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Contraction.lean

```
def contrCoContrBi : ContrModule →1[$\mathbb{C}$] CoModule →1[$\mathbb{C}$] $\mathbb{C}$ where
def contrContrCoBi : CoModule →1[$\mathbb{C}$] ContrModule →1[$\mathbb{C}$] $\mathbb{C}$ where
def contrCoContraction : (ContrModule.SL2CRep.tprod CoModule.SL2CRep).Intertwining
 (Representation.trivial $\mathbb{C}$ SL(2, $\mathbb{C}$) $\mathbb{C}$) where
 toLinearMap := TensorProduct.lift contrCoContrBi
 isIntertwining' M := TensorProduct.ext' fun ψ ϕ => by
 change ((LorentzGroup.toComplex (SL2C.toLorentzGroup M)) *v ψ .toFin13 $\mathbb{C}$) $\square_v$
 ((LorentzGroup.toComplex (SL2C.toLorentzGroup M))-1 $\top$ *v ϕ .toFin13 $\mathbb{C}$) =
 ψ .toFin13 $\mathbb{C}$ $\square_v$ ϕ .toFin13 $\mathbb{C}$
 rw [dotProduct_mulVec, vecMul_transpose, mulVec_mulVec]
 rw [inv_mul_of_invertible (LorentzGroup.toComplex (SL2C.toLorentzGroup M))]
 simp

lemma contrCoContraction_hom_tmul (ψ : ContrModule) (ϕ : CoModule) :
 contrCoContraction ($\psi \otimes_t \phi$) = ψ .toFin13 $\mathbb{C}$ $\square_v$ ϕ .toFin13 $\mathbb{C}$:= by
 contrCoContraction (complexContrBasisFin4 i $\otimes_t$ complexCoBasisFin4 j) =
 contrCoContraction (complexContrBasis i $\otimes_t$ complexCoBasis j) =
def coContrContraction : (CoModule.SL2CRep.tprod ContrModule.SL2CRep).Intertwining
 (Representation.trivial $\mathbb{C}$ SL(2, $\mathbb{C}$) $\mathbb{C}$) where
 toLinearMap := TensorProduct.lift contrContrCoBi
lemma coContrContraction_hom_tmul (ϕ : CoModule) (ψ : ContrModule) :
 coContrContraction ($\phi \otimes_t \psi$) = ϕ .toFin13 $\mathbb{C}$ $\square_v$ ψ .toFin13 $\mathbb{C}$:= by
 coContrContraction (complexCoBasisFin4 i $\otimes_t$ complexContrBasisFin4 j) =
 coContrContraction (complexCoBasis i $\otimes_t$ complexContrBasis j) =
lemma contrCoContraction_tmul_symm (ϕ : ContrModule) (ψ : CoModule) :
 contrCoContraction ($\phi \otimes_t \psi$) = coContrContraction ($\psi \otimes_t \phi$) := by
lemma coContrContraction_tmul_symm (ϕ : CoModule) (ψ : ContrModule) :
 coContrContraction ($\phi \otimes_t \psi$) = contrCoContraction ($\psi \otimes_t \phi$) := by
```

## 1.210 Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Modules.lean

Authors: Nikolai Kashcheev, Joseph Tooby-Smith

@[ext]

```
lemma ext (ψ ψ' : CoModule) (h : ψ .val = ψ' .val) : ψ = ψ' := by
 cases ψ
 cases ψ'
 subst h
 rfl
```

@[simp]

```
lemma val_add (ψ ψ' : CoModule) : (ψ + ψ').val = ψ .val + ψ' .val := rfl
```

```
@[simp]
lemma val_smul (r : ℂ) (ψ : CoCModule) : (r • ψ).val = r • ψ.val := rfl
```

### 1.211 Physlib/Relativity/Tensors/ComplexTensor/Weyl/Basic.lean

```
public import Physlib.Meta.Informal.Basic
public import Physlib.Meta.TODO.Basic
TODO "Rename the Weyl fermion representations `leftHandedRep`, `dualLeftHandedRep`,
`rightHandedRep` and `dualRightHandedRep` to `LeftHandedWeyl.rep`,
`DualLeftHandedWeyl.rep`, `RightHandedWeyl.rep` and `DualRightHandedWeyl.rep`,
respectively, so that each representation lives in the namespace of the module it
acts on, and update all references accordingly."

def leftHandedRep : Representation ℂ SL(2,ℂ) LeftHandedWeyl where
 toFun := fun (ψ : LeftHandedWeyl) =>
 LeftHandedWeyl.toFin2CEquiv.symm (M.1 *v ψ.toFin2C),
 mulVec_mulVec]
def leftBasis : Basis (Fin 2) ℂ LeftHandedWeyl := Basis.ofEquivFun
 (Equiv.linearEquiv ℂ LeftHandedWeyl.toFin2CFun)
 (LinearMap.toMatrix leftBasis leftBasis) (leftHandedRep M) i j = M.1 i j
def dualLeftHandedRep : Representation ℂ SL(2,ℂ) DualLeftHandedWeyl where
 toFun := fun (ψ : DualLeftHandedWeyl) =>
 DualLeftHandedWeyl.toFin2CEquiv.symm ((M.1-1)⊤ *v ψ.toFin2C),
 exact transpose_mul _ _
/-- The standard basis on dual-left-handed Weyl fermions. -
/
def dualLeftBasis : Basis (Fin 2) ℂ DualLeftHandedWeyl := Basis.ofEquivFun
 (Equiv.linearEquiv ℂ DualLeftHandedWeyl.toFin2CFun)
lemma dualLeftBasis_toFin2C (i : Fin 2) : (dualLeftBasis i).toFin2C = Pi.si
 simp only [dualLeftBasis, Basis.coe_ofEquivFun]
lemma dualLeftBasis_ρ_apply (M : SL(2,ℂ)) (i j : Fin 2) :
 (LinearMap.toMatrix dualLeftBasis dualLeftBasis) (dualLeftHandedRep M) i j =
 simp only [dualLeftBasis, Basis.coe_ofEquivFun, Basis.ofEquivFun_repr_apply]
def rightHandedRep : Representation ℂ SL(2,ℂ) RightHandedWeyl where
 toFun := fun (ψ : RightHandedWeyl) =>
 RightHandedWeyl.toFin2CEquiv.symm (M.1.map star *v ψ.toFin2C),
 AddHom.coe_mk, Module.End.mul_apply, LinearEquiv.apply_symm_apply, mul
def rightBasis : Basis (Fin 2) ℂ RightHandedWeyl := Basis.ofEquivFun
 (Equiv.linearEquiv ℂ RightHandedWeyl.toFin2CFun)
 (LinearMap.toMatrix rightBasis rightBasis) (rightHandedRep M) i j = (M.1
def dualRightHandedRep : Representation ℂ SL(2,ℂ) DualRightHandedWeyl where
 toFun := fun (ψ : DualRightHandedWeyl) =>
 DualRightHandedWeyl.toFin2CEquiv.symm ((M.1-1).conjTranspose *v ψ.toF
```

1.211.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/WEYL/BASIC.lean7

```

 exact conjTranspose_mul _ _
/-- The standard basis on dual-right-handed Weyl fermions. -
/
def dualRightBasis : Basis (Fin 2) ℂ DualRightHandedWeyl := Basis.ofEquivFun
 (Equiv.linearEquiv ℂ DualRightHandedWeyl.toFin2ℂFun)
lemma dualRightBasis_toFin2ℂ (i : Fin 2) : (dualRightBasis i).toFin2ℂ = Pi.single i
 simp only [dualRightBasis, Basis.coe_ofEquivFun]
lemma dualRightBasis_ρ_apply (M : SL(2,ℂ)) (i j : Fin 2) :
 (LinearMap.toMatrix dualRightBasis dualRightBasis) (dualRightHandedRep M) i j =
 simp only [dualRightBasis, Basis.coe_ofEquivFun, Basis.ofEquivFun_repr_apply]
 `dualLeftHanded` defined by multiplying an element of
def leftHandedToDual : leftHandedRep.IntertwiningMap dualLeftHandedRep where
 toFun := fun ψ => DualLeftHandedWeyl.toFin2ℂEquiv.symm (![0, 1; -
1, 0] *v ψ.toFin2ℂ)
 change DualLeftHandedWeyl.toFin2ℂEquiv.symm (![0, 1; -
1, 0] *v M.1 *v ψ.val) =
 DualLeftHandedWeyl.toFin2ℂEquiv.symm ((M.1-1)⊤ *v ![0, 1; -
1, 0] *v ψ.val)
lemma leftHandedToDual_hom_apply (ψ : LeftHandedWeyl) :
 leftHandedToDual ψ =
 DualLeftHandedWeyl.toFin2ℂEquiv.symm (![0, 1; -1, 0] *v ψ.toFin2ℂ) := rfl
/-- The morphism from `dualLeftHanded` to
 DualLeftHandedWeyl by the matrix `εa1a2 = ![0, -1; 1, 0]`. -
/
def leftHandedDualTo : dualLeftHandedRep.IntertwiningMap leftHandedRep where
 LeftHandedWeyl.toFin2ℂEquiv.symm (![0, -1; 1, 0] *v ψ.toFin2ℂ)
 change LeftHandedWeyl.toFin2ℂEquiv.symm (![0, -1; 1, 0] *v (M.1-1)⊤ *v ψ.val) =
 LeftHandedWeyl.toFin2ℂEquiv.symm (M.1 *v ![0, -1; 1, 0] *v ψ.val)
lemma leftHandedDualTo_hom_apply (ψ : DualLeftHandedWeyl) :
 leftHandedDualTo ψ =
 LeftHandedWeyl.toFin2ℂEquiv.symm (![0, -1; 1, 0] *v ψ.toFin2ℂ) := rfl
 `dualLeftHanded` defined by multiplying an element of
def leftHandedDualEquiv : leftHandedRep.Equiv dualLeftHandedRep := by
 refine Representation.Equiv.mk' leftHandedToDual leftHandedDualTo ?_ ?_
 · intro x
 simp only [AddHom.toFun_eq_coe, LinearMap.coe_toAddHom,
 Representation.IntertwiningMap.coe_toLinearMap]
 rw [leftHandedDualTo_hom_apply, leftHandedToDual_hom_apply]
 rw [DualLeftHandedWeyl.toFin2ℂ, LinearEquiv.apply_symm_apply, mulVec_mulVec]
 · intro ψ
 simp only [AddHom.toFun_eq_coe, LinearMap.coe_toAddHom,
 Representation.IntertwiningMap.coe_toLinearMap]
 rw [leftHandedDualTo_hom_apply, leftHandedToDual_hom_apply, LeftHandedWeyl.toFin
/-- `leftHandedDualEquiv` acting on an element `ψ : leftHanded` corresponds
lemma leftHandedDualEquiv_hom_hom_apply (ψ : LeftHandedWeyl) :
 leftHandedDualEquiv ψ =

```

```

DualLeftHandedWeyl.toFin2CEquiv.symm (!! [0, 1; -1, 0] *v ψ.toFin2C) :=
/-- The inverse of `leftHandedDualEquiv` acting on an element `ψ : dualLeftH
lemma leftHandedDualEquiv_inv_hom_apply (ψ : DualLeftHandedWeyl) :
 leftHandedDualEquiv.symm ψ =
 LeftHandedWeyl.toFin2CEquiv.symm (!! [0, -1; 1, 0] *v ψ.toFin2C) := rfl
/-- The linear equivalence between `rightHandedWeyl` and `DualRightHandedWe
informal_definition rightHandedWeylDualEquiv where
 deps := [``rightHandedRep, ``dualRightHandedRep]
/-- The linear equivalence `rightHandedWeylDualEquiv` is equivariant with r
`SL(2,C)` on `rightHandedWeyl` and `DualRightHandedWeyl`.
informal_lemma rightHandedWeylDualEquiv_equivariant where
 deps := [``rightHandedWeylDualEquiv]

```

### 1.212 Physlib/Relativity/Tensors/ComplexTensor/Weyl/Contraction

```

dual-left-handed Weyl fermion. -/
def leftDualBi : LeftHandedWeyl →1[C] DualLeftHandedWeyl →1[C] C where
/- The bi-linear map corresponding to contraction of a dual-
left-handed Weyl fermion with a
def dualLeftBi : DualLeftHandedWeyl →1[C] LeftHandedWeyl →1[C] C where
 dual-right-handed Weyl fermion. -/
def rightDualBi : RightHandedWeyl →1[C] DualRightHandedWeyl →1[C] C where
/- The bi-linear map corresponding to contraction of a dual-
right-handed Weyl fermion with a
def dualRightBi : DualRightHandedWeyl →1[C] RightHandedWeyl →1[C] C where
/- The linear map from leftHandedWeyl ⊗ DualLeftHandedWeyl to C given by
summing over components of leftHandedWeyl and DualLeftHandedWeyl in the
Physically, the contraction of a left-handed Weyl fermion with a dual-
left-handed Weyl fermion.
def leftDualContraction : (leftHandedRep.tprod dualLeftHandedRep).Intertwin
 (Representation.trivial C SL(2,C) C) where
 toLinearMap := TensorProduct.lift leftDualBi
 isIntertwining' M := TensorProduct.ext' fun ψ φ => by
 change (M.1 *v ψ.toFin2C) □v (M.1-1T *v φ.toFin2C) = ψ.toFin2C □v φ.toF
 rw [dotProduct_mulVec, vecMul_transpose, mulVec_mulVec]
 simp

lemma leftDualContraction_hom_tmul (ψ : LeftHandedWeyl)
 (φ : DualLeftHandedWeyl) :
 leftDualContraction (ψ ⊗t φ) = ψ.toFin2C □v φ.toFin2C := by
lemma leftDualContraction_basis (i j : Fin 2) :
 leftDualContraction (leftBasis i ⊗t dualLeftBasis j) = if i.1 = j.1 the
 rw [leftDualContraction_hom_tmul]
 simp only [leftBasis_toFin2C, dualLeftBasis_toFin2C, dotProduct_single, m

```

1.212.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/WEYL/CONTRACT1891.LEAN

```

/-- The linear map from DualLeftHandedWeyl $\otimes$ leftHandedWeyl to $\mathbb{C}$ given by
 summing over components of DualLeftHandedWeyl and leftHandedWeyl in the
 Physically, the contraction of a dual-left-handed Weyl fermion with a left-
 handed Weyl fermion.
def dualLeftContraction : (dualLeftHandedRep.tprod leftHandedRep).IntertwiningMap
 (Representation.trivial $\mathbb{C}$ SL(2, $\mathbb{C}$) $\mathbb{C}$) where
 toLinearMap := TensorProduct.lift dualLeftBi
lemma dualLeftContraction_hom_tmul (ϕ : DualLeftHandedWeyl) (ψ : LeftHandedWeyl) :
 dualLeftContraction ($\phi \otimes_t \psi$) = ϕ .toFin2 $\mathbb{C}$ $\sqcap_v$ ψ .toFin2 $\mathbb{C}$:= by
lemma dualLeftContraction_basis (i j : Fin 2) :
 dualLeftContraction (dualLeftBasis i $\otimes_t$ leftBasis j) = if i.1 = j.1 then (1 : $\mathbb{C}$)
 rw [dualLeftContraction_hom_tmul]
 simp only [dualLeftBasis_toFin2 $\mathbb{C}$, leftBasis_toFin2 $\mathbb{C}$, dotProduct_single, mul_one]
The linear map from `rightHandedWeyl  $\otimes$  DualRightHandedWeyl` to ` $\mathbb{C}$ ` given by
 summing over components of `rightHandedWeyl` and `DualRightHandedWeyl` in the
def rightDualContraction : (rightHandedRep.tprod dualRightHandedRep).IntertwiningMap
 (Representation.trivial $\mathbb{C}$ SL(2, $\mathbb{C}$) $\mathbb{C}$) where
 toLinearMap := TensorProduct.lift rightDualBi
 simp only [one_mulVec, vec2_dotProduct, Fin.isValue, RightHandedWeyl.toFin2 $\mathbb{C}$ Equiv_apply]
lemma rightDualContraction_hom_tmul (ψ : RightHandedWeyl)
 (ϕ : DualRightHandedWeyl) :
 rightDualContraction ($\psi \otimes_t \phi$) = ψ .toFin2 $\mathbb{C}$ $\sqcap_v$ ϕ .toFin2 $\mathbb{C}$:= by
lemma rightDualContraction_basis (i j : Fin 2) :
 rightDualContraction (rightBasis i $\otimes_t$ dualRightBasis j) =
 rw [rightDualContraction_hom_tmul]
 simp only [rightBasis_toFin2 $\mathbb{C}$, dualRightBasis_toFin2 $\mathbb{C}$, dotProduct_single, mul_one]
The linear map from DualRightHandedWeyl $\otimes$ rightHandedWeyl to $\mathbb{C}$ given by
 summing over components of DualRightHandedWeyl and rightHandedWeyl in the
def dualRightContraction : (dualRightHandedRep.tprod rightHandedRep).IntertwiningMap
 (Representation.trivial $\mathbb{C}$ SL(2, $\mathbb{C}$) $\mathbb{C}$) where
 toLinearMap := TensorProduct.lift dualRightBi
 simp only [vecMul_one, vec2_dotProduct, Fin.isValue, DualRightHandedWeyl.toFin2 $\mathbb{C}$
 RightHandedWeyl.toFin2 $\mathbb{C}$ Equiv_apply]
lemma dualRightContraction_hom_tmul (ϕ : DualRightHandedWeyl)
 (ψ : RightHandedWeyl) :
 dualRightContraction ($\phi \otimes_t \psi$) = ϕ .toFin2 $\mathbb{C}$ $\sqcap_v$ ψ .toFin2 $\mathbb{C}$:= by
lemma dualRightContraction_basis (i j : Fin 2) :
 dualRightContraction (dualRightBasis i $\otimes_t$ rightBasis j) =
 rw [dualRightContraction_hom_tmul]
 simp only [dualRightBasis_toFin2 $\mathbb{C}$, rightBasis_toFin2 $\mathbb{C}$, dotProduct_single, mul_one]
lemma leftDualContraction_tmul_symm (ψ : LeftHandedWeyl) (ϕ : DualLeftHandedWeyl) :
 leftDualContraction ($\psi \otimes_t [\mathbb{C}] \phi$) = dualLeftContraction ($\phi \otimes_t [\mathbb{C}] \psi$) := by
 rw [leftDualContraction_hom_tmul, dualLeftContraction_hom_tmul, dotProduct_comm]
lemma dualLeftContraction_tmul_symm (ϕ : DualLeftHandedWeyl) (ψ : LeftHandedWeyl) :
 dualLeftContraction ($\phi \otimes_t [\mathbb{C}] \psi$) = leftDualContraction ($\psi \otimes_t [\mathbb{C}] \phi$) := by

```

```

 rw [leftDualContraction_tmul_symm]
lemma rightDualContraction_tmul_symm (ψ : RightHandedWeyl) (φ : DualRightHandedWeyl) :
 rightDualContraction (ψ ⊗ₜ[ℂ] φ) = dualRightContraction (φ ⊗ₜ[ℂ] ψ) :=
 rw [rightDualContraction_hom_tmul, dualRightContraction_hom_tmul, dotProd]
lemma dualRightContraction_tmul_symm (φ : DualRightHandedWeyl) (ψ : RightHandedWeyl) :
 dualRightContraction (φ ⊗ₜ[ℂ] ψ) = rightDualContraction (ψ ⊗ₜ[ℂ] φ) :=
 rw [rightDualContraction_tmul_symm]

```

### 1.213 Physlib/Relativity/Tensors/ComplexTensor/Weyl/Metric.lean

These allow us to go from left-handed to dual-left-handed Weyl fermions and back and from right-handed to dual-right-handed Weyl fermions and back.

```

def leftMetricVal : LeftHandedWeyl ⊗ₜ[ℂ] LeftHandedWeyl :=
 simp only [metricRaw, Matrix.neg_apply, of_apply, cons_val', empty_val',
 Fin.sum_univ_two, Fin.isValue, cons_val_zero, cons_val_one, neg_zero, zero,
 neg_neg, one_smul, add_zero, add_left_inj]
module

lemma leftMetricVal_expand_tmul' : leftMetricVal = leftBasis 1 ⊗ₜ[ℂ] leftBasis 1 -
 leftBasis 0 ⊗ₜ[ℂ] leftBasis 1 := by rw [leftMetricVal_expand_tmul]; a
def leftMetric : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
 (leftHandedRep.tprod leftHandedRep) where
 toFun := fun a =>
 let a' : ℂ := a
 a' • leftMetricVal
 map_add' := fun x y => by
 simp only [add_smul]
 map_smul' := fun m x => by
 simp only [smul_smul]
 rfl
 isIntertwining' M := by
 refine LinearMap.ext fun x : ℂ => ?_
 change x • leftMetricVal =
 (TensorProduct.map (leftHandedRep M) (leftHandedRep M)) (x • leftMetricVal)
 simp only [map_smul]
 apply congrArg
 simp only [leftMetricVal, map_neg, neg_inj]
 rw [leftLeftToMatrix_p_symm]
 apply congrArg
 rw [comm_metricRaw, mul_assoc, ← @transpose_mul]
 simp only [SpecialLinearGroup.det_coe, isUnit_iff_ne_zero, ne_eq, one_ne_zero,
 not_false_eq_true, mul_nonsing_inv, transpose_one, mul_one]

lemma leftMetric_apply_one : leftMetric (1 : ℂ) = leftMetricVal := by

```

1.213.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/WEYL/METRIC.LEAN

```

/-- The metric ε_{aa} as an element of $(\text{dualLeftHanded} \otimes \text{dualLeftHanded}).V$. -
/
def dualLeftMetricVal : (DualLeftHandedWeyl $\otimes$ $\mathbb{C}$) DualLeftHandedWeyl :=
 dualLeftdualLeftToMatrix.symm metricRaw
/-- Expansion of dualLeftMetricVal into the left basis. -
/
lemma dualLeftMetricVal_expand_tmul : dualLeftMetricVal =
 dualLeftBasis 0 $\otimes$ $\mathbb{C}$ dualLeftBasis 1 - dualLeftBasis 1 $\otimes$ $\mathbb{C}$ dualLeftBasis 0 :=
 simp only [dualLeftMetricVal, Fin.isValue]
 rw [dualLeftdualLeftToMatrix_symm_expand_tmul]
 Fin.isValue, cons_val_zero, cons_val_one, zero_smul, one_smul, zero_add, add_zero
module
/-- The metric ε_{aa} as a morphism $\square_{\text{Rep } \mathbb{C} \text{ SL}(2, \mathbb{C})} \square \text{dualLeftHanded} \otimes \text{dualLeftHanded}$
def dualLeftMetric : (Representation.trivial $\mathbb{C} \text{ SL}(2, \mathbb{C}) \mathbb{C}$).IntertwiningMap
 (dualLeftHandedRep.tprod dualLeftHandedRep) where
 a' $\cdot$ dualLeftMetricVal
 change x $\cdot$ dualLeftMetricVal =
 (TensorProduct.map (dualLeftHandedRep M) (dualLeftHandedRep M)) (x $\cdot$ dualLeftMetricVal)
 simp only [dualLeftMetricVal]
 rw [dualLeftdualLeftToMatrix_p_symm]
lemma dualLeftMetric_apply_one : dualLeftMetric (1 : $\mathbb{C}$) = dualLeftMetricVal := by
 change (1 : $\mathbb{C}$) $\cdot$ dualLeftMetricVal = dualLeftMetricVal
def rightMetricVal : (RightHandedWeyl $\otimes$ $\mathbb{C}$) RightHandedWeyl :=
 simp only [metricRaw, Matrix.neg_apply, of_apply, cons_val', empty_val', cons_val',
 Fin.sum_univ_two, Fin.isValue, cons_val_zero, cons_val_one, neg_zero, zero_smul,
 neg_neg, one_smul, add_zero, add_left_inj]
module

lemma rightMetricVal_expand_tmul' : rightMetricVal = rightBasis 1 $\otimes$ $\mathbb{C}$ rightBasis 0
 - rightBasis 0 $\otimes$ $\mathbb{C}$ rightBasis 1 := by rw [rightMetricVal_expand_tmul]; abel
def rightMetric : (Representation.trivial $\mathbb{C} \text{ SL}(2, \mathbb{C}) \mathbb{C}$).IntertwiningMap
 (rightHandedRep.tprod rightHandedRep) where
 toFun := fun a =>
 let a' : $\mathbb{C}$:= a
 a' $\cdot$ rightMetricVal
 map_add' := fun x y => by
 simp only [add_smul]
 map_smul' := fun m x => by
 simp only [smul_smul]
 rfl
 isIntertwining' M := by
 refine LinearMap.ext fun x : $\mathbb{C}$ => ?_
 change x $\cdot$ rightMetricVal =
 (TensorProduct.map (rightHandedRep M) (rightHandedRep M)) (x $\cdot$ rightMetricVal)
 simp only [map_smul]
 apply congrArg

```

```

simp only [rightMetricVal, map_neg, neg_inj]
trans rightRightToMatrix.symm ((M.1).map star * metricRaw * ((M.1).map
· apply congrArg
 rw [star_comm_metricRaw, mul_assoc]
 have h1 : ((M.1)-1H * ((M.1).map star)T) = 1 := by
 trans (M.1)-1H * ((M.1))H
 · rfl
 rw [← @conjTranspose_mul]
 simp only [SpecialLinearGroup.det_coe, isUnit_iff_ne_zero, ne_eq, o
 not_false_eq_true, mul_nonsing_inv, conjTranspose_one]
 rw [h1]
 simp
· rw [← rightRightToMatrix_ρ_symm metricRaw M]

lemma rightMetric_apply_one : rightMetric (1 : ℂ) = rightMetricVal := by
/-- The metric `ε_{dot a}_{dot a}` as an element of `(dualRightHanded ⊗ dual
/
def dualRightMetricVal : DualRightHandedWeyl ⊗[ℂ] DualRightHandedWeyl :=
 dualRightDualRightToMatrix.symm (metricRaw)
lemma dualRightMetricVal_expand_tmul : dualRightMetricVal =
 dualRightBasis 0 ⊗t[ℂ] dualRightBasis 1 - dualRightBasis 1 ⊗t[ℂ] dualRi
 simp only [dualRightMetricVal, Fin.isValue]
 rw [dualRightDualRightToMatrix_symm_expand_tmul]
 Fin.isValue, cons_val_zero, cons_val_one, zero_smul, one_smul, zero_add
module
`[] (Rep ℂ SL(2,ℂ)) [] dualRightHanded ⊗ dualRightHanded`,
def dualRightMetric : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
 (dualRightHandedRep.tprod dualRightHandedRep) where
toFun := fun a =>
 a' • dualRightMetricVal
map_add' := fun x y => by
 simp only [add_smul]
map_smul' := fun m x => by
 simp only [smul_smul]
 rfl
isIntertwining' M := by
 refine LinearMap.ext fun x : ℂ => ?_
 change x • dualRightMetricVal =
 (TensorProduct.map (dualRightHandedRep M) (dualRightHandedRep M)) (x)
 simp only [map_smul]
 apply congrArg
 trans dualRightDualRightToMatrix.symm
 (((M.1)-1).conjTranspose * metricRaw * (((M.1)-1).conjTranspose)T)
 · rw [dualRightMetricVal]
 rw [← metricRaw_comm_star, mul_assoc]
 have h1 : ((M.1).map star * (M.1)-1HT) = 1 := by

```



1.213.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/WEYL/METRIC.lean

```

 refine transpose_eq_one.mp ?_
 rw [@transpose_mul]
 simp only [transpose_transpose, RCLike.star_def]
 change (M.1)-1H * (M.1)H = 1
 rw [← @conjTranspose_mul]
 simp
 rw [h1, mul_one]
 · rw [← dualRightDualRightToMatrix_p_symm metricRaw M]
 rfl

lemma dualRightMetric_apply_one : dualRightMetric (1 : ℂ) = dualRightMetricVal := by
 change (1 : ℂ) • dualRightMetricVal = dualRightMetricVal
lemma leftDualContraction_apply_metric :
 (TensorProduct.comm ℂ _ _ <|
 (TensorProduct.lid ℂ _).lTensor _ <|
 (leftDualContraction.toLinearMap.rTensor _).lTensor _ <|
 (TensorProduct.assoc ℂ _ _).symm.toLinearMap.lTensor _ <|
 TensorProduct.assoc ℂ _ _ (_ ⊗ [ℂ] _) <|
 (leftMetric 1) ⊗+ [ℂ] (dualLeftMetric 1)) = dualLeftLeftUnit (1 : ℂ) := by
 rw [leftMetric_apply_one, dualLeftMetric_apply_one]
 rw [leftMetricVal_expand_tmul', dualLeftMetricVal_expand_tmul]
 simp only [Fin.isValue, tmul_sub, sub_tmul, map_sub, assoc_tmul, LinearMap.lTensor
 LinearEquiv.coe_coe, assoc_symm_tmul, LinearMap.rTensor_tmul,
 Representation.IntertwiningMap.coe_toLinearMap, LinearEquiv.lTensor_tmul, lid_tm
 map_smul, comm_tmul]
 simp only [leftDualContraction_basis]
 rw [dualLeftLeftUnit_apply_one, dualLeftLeftUnitVal_expand_tmul]
lemma dualLeftContraction_apply_metric :
 (TensorProduct.comm ℂ _ _ <|
 (TensorProduct.lid ℂ _).lTensor _ <|
 (dualLeftContraction.toLinearMap.rTensor _).lTensor _ <|
 (TensorProduct.assoc ℂ _ _).symm.toLinearMap.lTensor _ <|
 TensorProduct.assoc ℂ _ _ (_ ⊗ [ℂ] _) <|
 (dualLeftMetric 1) ⊗+ [ℂ] (leftMetric 1)) = leftDualLeftUnit (1 : ℂ) := by
 rw [leftMetric_apply_one, dualLeftMetric_apply_one]
 rw [leftMetricVal_expand_tmul', dualLeftMetricVal_expand_tmul]
 simp only [Fin.isValue, tmul_sub, sub_tmul, map_sub, assoc_tmul, LinearMap.lTensor
 LinearEquiv.coe_coe, assoc_symm_tmul, LinearMap.rTensor_tmul,
 Representation.IntertwiningMap.coe_toLinearMap, LinearEquiv.lTensor_tmul, lid_tm
 map_smul, comm_tmul]
 simp only [dualLeftContraction_basis]
 simp only [Fin.isValue, Fin.coe_ofNat_eq_mod, Nat.mod_succ, ↓reduceIte, one_smul,
 zero_ne_one, zero_smul, sub_zero, one_ne_zero, zero_sub, sub_neg_eq_add]
 rw [leftDualLeftUnit_apply_one, leftDualLeftUnitVal_expand_tmul]
lemma rightDualContraction_apply_metric :
 (TensorProduct.comm ℂ _ _ <|

```

```

(TensorProduct.lid $\mathbb{C}$ _).lTensor _ <|
(rightDualContraction.toLinearMap.rTensor _).lTensor _ <|
(TensorProduct.assoc $\mathbb{C}$ _ _).symm.toLinearMap.lTensor _ <|
TensorProduct.assoc $\mathbb{C}$ _ _ ($\otimes[\mathbb{C}]$ _) <|
(rightMetric 1) $\otimes_t[\mathbb{C}]$ (dualRightMetric 1)) = dualRightRightUnit (1 : $\mathbb{C}$)
rw [rightMetric_apply_one, dualRightMetric_apply_one]
rw [rightMetricVal_expand_tmul', dualRightMetricVal_expand_tmul]
simp only [Fin.isValue, tmul_sub, sub_tmul, map_sub, assoc_tmul, LinearMa
LinearEquiv.coe_coe, assoc_symm_tmul, LinearMap.rTensor_tmul,
Representation.IntertwiningMap.coe_toLinearMap, LinearEquiv.lTensor_tmul
map_smul, comm_tmul]
simp only [rightDualContraction_basis]
rw [dualRightRightUnit_apply_one, dualRightRightUnitVal_expand_tmul]
lemma dualRightContraction_apply_metric :
(TensorProduct.comm $\mathbb{C}$ _ _ <|
(TensorProduct.lid $\mathbb{C}$ _).lTensor _ <|
(dualRightContraction.toLinearMap.rTensor _).lTensor _ <|
(TensorProduct.assoc $\mathbb{C}$ _ _).symm.toLinearMap.lTensor _ <|
TensorProduct.assoc $\mathbb{C}$ _ _ ($\otimes[\mathbb{C}]$ _) <|
(dualRightMetric 1) $\otimes_t[\mathbb{C}]$ (rightMetric 1)) = rightDualRightUnit (1 :
rw [rightMetric_apply_one, dualRightMetric_apply_one]
rw [rightMetricVal_expand_tmul', dualRightMetricVal_expand_tmul]
simp only [Fin.isValue, tmul_sub, sub_tmul, map_sub, assoc_tmul, LinearMa
LinearEquiv.coe_coe, assoc_symm_tmul, LinearMap.rTensor_tmul,
Representation.IntertwiningMap.coe_toLinearMap, LinearEquiv.lTensor_tmul
map_smul, comm_tmul]
simp only [dualRightContraction_basis]
simp only [Fin.isValue, Fin.coe_ofNat_eq_mod, Nat.mod_succ, $\downarrow$ reduceIte, o
zero_ne_one, zero_smul, sub_zero, one_ne_zero, zero_sub, sub_neg_eq_add
rw [rightDualRightUnit_apply_one, rightDualRightUnitVal_expand_tmul]

```

### 1.214 Physlib/Relativity/Tensors/ComplexTensor/Weyl/Modules.lean

```
public import Physlib.Meta.TODO.Basic
```

```
TODO "Make a directory in ./Physlib/Relativity called Fermions for these fi
Make this file (currently ../Modules.lean) the Basic file, and include
definitions for the different types of Weyl fermions."
```

```

structure LeftHandedWeyl where
namespace LeftHandedWeyl
/-- The equivalence between `LeftHandedWeyl` and ` $\text{Fin } 2 \rightarrow \mathbb{C}$ `. -
/
def toFin2CFun : LeftHandedWeyl $\approx$ (Fin 2 $\rightarrow$ $\mathbb{C}$) where

```

1.214.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/WEYL/MODULES.DEDAN

```
-- The instance of `AddCommMonoid` on `LeftHandedWeyl` defined via its equivalence
instance : AddCommMonoid LeftHandedWeyl := Equiv.addCommMonoid toFin2CFun
-- The instance of `AddCommGroup` on `LeftHandedWeyl` defined via its equivalence
instance : AddCommGroup LeftHandedWeyl := Equiv.addCommGroup toFin2CFun
-- The instance of `Module` on `LeftHandedWeyl` defined via its equivalence
instance : Module ℂ LeftHandedWeyl := Equiv.module ℂ toFin2CFun
-- The linear equivalence between `LeftHandedWeyl` and `(Fin 2 → ℂ)`. -
/
def toFin2CEquiv : LeftHandedWeyl ≈1[ℂ] (Fin 2 → ℂ) where
-- The underlying element of `Fin 2 → ℂ` of a element in `LeftHandedWeyl` defined
abbrev toFin2C (ψ : LeftHandedWeyl) := toFin2CEquiv ψ
end LeftHandedWeyl
-- The module in which dual-left handed fermions live. This is equivalent to `Fin 2`
/
structure DualLeftHandedWeyl where
namespace DualLeftHandedWeyl
-- The equivalence between `DualLeftHandedWeyl` and `Fin 2 → ℂ`. -
/
def toFin2CFun : DualLeftHandedWeyl ≈ (Fin 2 → ℂ) where
-- The instance of `AddCommMonoid` on `DualLeftHandedWeyl` defined via its equivalence
instance : AddCommMonoid DualLeftHandedWeyl := Equiv.addCommMonoid toFin2CFun
-- The instance of `AddCommGroup` on `DualLeftHandedWeyl` defined via its equivalence
instance : AddCommGroup DualLeftHandedWeyl := Equiv.addCommGroup toFin2CFun
-- The instance of `Module` on `DualLeftHandedWeyl` defined via its equivalence
instance : Module ℂ DualLeftHandedWeyl := Equiv.module ℂ toFin2CFun
-- The linear equivalence between `DualLeftHandedWeyl` and `(Fin 2 → ℂ)`. -
/
def toFin2CEquiv : DualLeftHandedWeyl ≈1[ℂ] (Fin 2 → ℂ) where
-- The underlying element of `Fin 2 → ℂ` of a element in `DualLeftHandedWeyl` defined
abbrev toFin2C (ψ : DualLeftHandedWeyl) := toFin2CEquiv ψ
end DualLeftHandedWeyl
structure RightHandedWeyl where
namespace RightHandedWeyl
-- The equivalence between `RightHandedWeyl` and `Fin 2 → ℂ`. -
/
def toFin2CFun : RightHandedWeyl ≈ (Fin 2 → ℂ) where
-- The instance of `AddCommMonoid` on `RightHandedWeyl` defined via its equivalence
instance : AddCommMonoid RightHandedWeyl := Equiv.addCommMonoid toFin2CFun
-- The instance of `AddCommGroup` on `RightHandedWeyl` defined via its equivalence
instance : AddCommGroup RightHandedWeyl := Equiv.addCommGroup toFin2CFun
-- The instance of `Module` on `RightHandedWeyl` defined via its equivalence
instance : Module ℂ RightHandedWeyl := Equiv.module ℂ toFin2CFun
-- The linear equivalence between `RightHandedWeyl` and `(Fin 2 → ℂ)`. -
/
def toFin2CEquiv : RightHandedWeyl ≈1[ℂ] (Fin 2 → ℂ) where
-- The underlying element of `Fin 2 → ℂ` of a element in `RightHandedWeyl` defined
```

```

abbrev toFin2C (ψ : RightHandedWeyl) := toFin2CEquiv ψ
end RightHandedWeyl
section DualRightHanded
/-- The module in which dual-right handed fermions live. This is equivalent
/
structure DualRightHandedWeyl where
namespace DualRightHandedWeyl
/-- The equivalence between `DualRightHandedWeyl` and `Fin 2 → C`. -
/
def toFin2CFun : DualRightHandedWeyl ≈1[C] (Fin 2 → C) where
/-- The instance of `AddCommMonoid` on `DualRightHandedWeyl` defined via it
instance : AddCommMonoid DualRightHandedWeyl := Equiv.addCommMonoid toFin2C
/-- The instance of `AddCommGroup` on `DualRightHandedWeyl` defined via its
instance : AddCommGroup DualRightHandedWeyl := Equiv.addCommGroup toFin2CFun
/-- The instance of `Module` on `DualRightHandedWeyl` defined via its equiv
instance : Module C DualRightHandedWeyl := Equiv.module C toFin2CFun
/-- The linear equivalence between `DualRightHandedWeyl` and `(Fin 2 → C)`.
/
def toFin2CEquiv : DualRightHandedWeyl ≈1[C] (Fin 2 → C) where
/-- The underlying element of `Fin 2 → C` of a element in `DualRightHandedWeyl`
abbrev toFin2C (ψ : DualRightHandedWeyl) := toFin2CEquiv ψ
end DualRightHandedWeyl
end DualRightHanded

```

### 1.215 Physlib/Relativity/Tensors/ComplexTensor/Weyl/Two.lean

```

def leftLeftToMatrix : (LeftHandedWeyl ⊗1[C] LeftHandedWeyl) ≈1[C] Matrix (Fin 2) (Fin 2) C :=
/-- Equivalence of `dualLeftHanded ⊗ dualLeftHanded` to `2 x 2` complex matrix
/
def dualLeftdualLeftToMatrix : (DualLeftHandedWeyl ⊗1[C] DualLeftHandedWeyl) ≈1[C] Matrix (Fin 2) (Fin 2) C :=
(Basis.tensorProduct dualLeftBasis dualLeftBasis).repr
/-- Expanding `dualLeftdualLeftToMatrix` in terms of the standard basis. -
/
lemma dualLeftdualLeftToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) C) :
dualLeftdualLeftToMatrix.symm M = ∑ i, ∑ j, M i j •
(dualLeftBasis i ⊗1[C] dualLeftBasis j) := by
simp only [dualLeftdualLeftToMatrix, LinearEquiv.trans_symm, LinearEquiv.exact_congrArg]
exact congrArg _ (Basis.tensorProduct_apply dualLeftBasis dualLeftBasis)
/-- Equivalence of `leftHanded ⊗ dualLeftHanded` to `2 x 2` complex matrix
/
def leftDualLeftToMatrix : (LeftHandedWeyl ⊗1[C] DualLeftHandedWeyl) ≈1[C] Matrix (Fin 2) (Fin 2) C :=
(Basis.tensorProduct leftBasis dualLeftBasis).repr

```

1.215.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/WEYL/TWO.LEAN297

```

/-- Expanding `leftDualLeftToMatrix` in terms of the standard basis. -
/
lemma leftDualLeftToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
 leftDualLeftToMatrix.symm M = ∑ i, ∑ j, M i j • (leftBasis i ⊗t [ℂ] dualLeftBasis j)
 simp only [leftDualLeftToMatrix, LinearEquiv.trans_symm, LinearEquiv.trans_apply,
 exact congrArg _ (Basis.tensorProduct_apply leftBasis dualLeftBasis i j)]
/-- Equivalence of `dualLeftHanded ⊗ leftHanded` to `2 x 2` complex matrices. -
/
def dualLeftLeftToMatrix : (DualLeftHandedWeyl ⊗ [ℂ] LeftHandedWeyl) ≈1 [ℂ]
 Matrix (Fin 2) (Fin 2) ℂ :=
 (Basis.tensorProduct dualLeftBasis leftBasis).repr [] []
/-- Expanding `dualLeftLeftToMatrix` in terms of the standard basis. -
/
lemma dualLeftLeftToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
 dualLeftLeftToMatrix.symm M = ∑ i, ∑ j, M i j • (dualLeftBasis i ⊗t [ℂ] leftBasis j)
 simp only [dualLeftLeftToMatrix, LinearEquiv.trans_symm, LinearEquiv.trans_apply,
 exact congrArg _ (Basis.tensorProduct_apply dualLeftBasis leftBasis i j)]
def rightRightToMatrix : (RightHandedWeyl ⊗ [ℂ] RightHandedWeyl) ≈1 [ℂ]
 Matrix (Fin 2) (Fin 2) ℂ :=
 (Basis.tensorProduct rightBasis dualRightBasis).repr [] []
/-- Equivalence of `dualRightHanded ⊗ dualRightHanded` to `2 x 2` complex matrices. -
/
def dualRightDualRightToMatrix : (DualRightHandedWeyl ⊗ [ℂ] DualRightHandedWeyl) ≈1 [ℂ]
 Matrix (Fin 2) (Fin 2) ℂ :=
 (Basis.tensorProduct dualRightBasis dualRightBasis).repr [] []
/-- Expanding `dualRightDualRightToMatrix` in terms of the standard basis. -
/
lemma dualRightDualRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
 dualRightDualRightToMatrix.symm M =
 ∑ i, ∑ j, M i j • (dualRightBasis i ⊗t [ℂ] dualRightBasis j) := by
 simp only [dualRightDualRightToMatrix, LinearEquiv.trans_symm, LinearEquiv.trans_apply,
 exact congrArg _ (Basis.tensorProduct_apply dualRightBasis dualRightBasis i j)]
/-- Equivalence of `rightHanded ⊗ dualRightHanded` to `2 x 2` complex matrices. -
/
def rightDualRightToMatrix : (RightHandedWeyl ⊗ [ℂ] DualRightHandedWeyl) ≈1 [ℂ]
 Matrix (Fin 2) (Fin 2) ℂ :=
 (Basis.tensorProduct rightBasis dualRightBasis).repr [] []
/-- Expanding `rightDualRightToMatrix` in terms of the standard basis. -
/
lemma rightDualRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
 rightDualRightToMatrix.symm M = ∑ i, ∑ j, M i j • (rightBasis i ⊗t [ℂ] dualRightBasis j)
 simp only [rightDualRightToMatrix, LinearEquiv.trans_symm, LinearEquiv.trans_apply,
 exact congrArg _ (Basis.tensorProduct_apply rightBasis dualRightBasis i j)]
/-- Equivalence of `dualRightHanded ⊗ rightHanded` to `2 x 2` complex matrices. -
/
def dualRightRightToMatrix : (DualRightHandedWeyl ⊗ [ℂ] RightHandedWeyl) ≈1 [ℂ]
 Matrix (Fin 2) (Fin 2) ℂ :=

```

```

 (Basis.tensorProduct dualRightBasis rightBasis).repr []₁
/-- Expanding `dualRightRightToMatrix` in terms of the standard basis. -
/
lemma dualRightRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
 dualRightRightToMatrix.symm M = ∑ i, ∑ j, M i j • (dualRightBasis i ⊗ₜ [
 simp only [dualRightRightToMatrix, LinearEquiv.trans_symm, LinearEquiv.tr
 exact congrArg _ (Basis.tensorProduct_apply dualRightBasis rightBasis i
/-- Equivalence of `dualLeftHanded ⊗ dualRightHanded` to `2 x 2` complex ma
/
def dualLeftDualRightToMatrix : (DualLeftHandedWeyl ⊗[ℂ] DualRightHandedWeyl)
 Matrix (Fin 2) (Fin 2) ℂ :=
 (Basis.tensorProduct dualLeftBasis dualRightBasis).repr []₁
/-- Expanding `dualLeftDualRightToMatrix` in terms of the standard basis. -
/
lemma dualLeftDualRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
 dualLeftDualRightToMatrix.symm M = ∑ i, ∑ j, M i j •
 (dualLeftBasis i ⊗ₜ[ℂ] dualRightBasis j) := by
 simp only [dualLeftDualRightToMatrix, LinearEquiv.trans_symm, LinearEquiv
 exact congrArg _ (Basis.tensorProduct_apply dualLeftBasis dualRightBasi
def leftRightToMatrix : (LeftHandedWeyl ⊗[ℂ] RightHandedWeyl) ≈₁[ℂ] Matrix
/-- The coercion of `Finsupp.linearEquivFunOnFinite` to a function is the u
finitely-supported function, used to bridge it with `Matrix.mulVec`. -
/
private lemma coe_linearEquivFunOnFinite (g : (Fin 2 × Fin 2) →₀ ℂ) :
 Finsupp.linearEquivFunOnFinite ℂ ℂ (Fin 2 × Fin 2) g = ↑g := rfl

lemma leftLeftToMatrix_p (v : (LeftHandedWeyl ⊗[ℂ] LeftHandedWeyl)) (M : SL
 leftLeftToMatrix (TensorProduct.map (leftHandedRep M) (leftHandedRep M))
 (TensorProduct.map (leftHandedRep M) (leftHandedRep M)))
 (leftBasis.tensorProduct leftBasis) (TensorProduct.map (leftHandedRep
 simp only [coe_linearEquivFunOnFinite]
 rw [h1]
 ((LinearMap.toMatrix leftBasis leftBasis) (leftHandedRep M))
 ((LinearMap.toMatrix leftBasis leftBasis) (leftHandedRep M)) (i, j)
 rw [h1]
/-- The group action of `SL(2,ℂ)` on `dualLeftHanded ⊗ dualLeftHanded` is e
lemma dualLeftdualLeftToMatrix_p (v : (DualLeftHandedWeyl ⊗[ℂ] DualLeftHand
 dualLeftdualLeftToMatrix (TensorProduct.map (dualLeftHandedRep M) (dual
 (M.1⁻¹)ᵀ * dualLeftdualLeftToMatrix v * (M.1⁻¹) := by
 nth_rewrite 1 [dualLeftdualLeftToMatrix]
 (dualLeftBasis.tensorProduct dualLeftBasis) (dualLeftBasis.tensorProd
 (TensorProduct.map (dualLeftHandedRep M) (dualLeftHandedRep M)))
 ((dualLeftBasis.tensorProduct dualLeftBasis).repr v)))
 have h1 := (LinearMap.toMatrix_mulVec_repr (dualLeftBasis.tensorProduct
 (dualLeftBasis.tensorProduct dualLeftBasis)
 (TensorProduct.map (dualLeftHandedRep M) (dualLeftHandedRep M)) v)

```

1.215.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/WEYL/TWO.LEAN299

```

simp only [coe_linearEquivFunOnFinite]
rw [h1]
 ((LinearMap.toMatrix dualLeftBasis dualLeftBasis) (dualLeftHandedRep M))
 ((LinearMap.toMatrix dualLeftBasis dualLeftBasis) (dualLeftHandedRep M)) (i,
 * dualLeftdualLeftToMatrix v k.1 k.2) = _
have h1 : $\sum x : \text{Fin } 2, (\sum x1 : \text{Fin } 2, (M.1)^{-1} \bar{x}1 \ i \ * \ \text{dualLeftdualLeftToMatrix } v \ x1 \ x) \ * \ (M.1)^{-1} \ x \ j$
 = $\sum x : \text{Fin } 2, \sum x1 : \text{Fin } 2, ((M.1)^{-1} \ x1 \ i \ * \ \text{dualLeftdualLeftToMatrix } v \ x1 \ x) \ * \ (M.1)^{-1} \ x \ j$:= by
 rw [h1]
simp only [dualLeftBasis_p_apply, transpose_apply]
/-- The group action of `SL(2,ℂ)` on `leftHanded ⊗ dualLeftHanded` is equivalent to
`M.1 * leftDualLeftToMatrix v * (M.1-1)`. -/
lemma leftDualLeftToMatrix_p (v : (LeftHandedWeyl ⊗ [ℂ] DualLeftHandedWeyl)) (M : SL(
 leftDualLeftToMatrix (TensorProduct.map (leftHandedRep M) (dualLeftHandedRep M)
 M.1 * leftDualLeftToMatrix v * (M.1-1) := by
nth_rewrite 1 [leftDualLeftToMatrix]
 (leftBasis.tensorProduct dualLeftBasis) (leftBasis.tensorProduct dualLeftBasis
 (TensorProduct.map (leftHandedRep M) (dualLeftHandedRep M)))
 ((leftBasis.tensorProduct dualLeftBasis).repr (v)))
have h1 := (LinearMap.toMatrix_mulVec_repr (leftBasis.tensorProduct dualLeftBasi
 (leftBasis.tensorProduct dualLeftBasis)
 (TensorProduct.map (leftHandedRep M) (dualLeftHandedRep M)) v)
simp only [coe_linearEquivFunOnFinite]
rw [h1]
 ((LinearMap.toMatrix leftBasis leftBasis) (leftHandedRep M))
 ((LinearMap.toMatrix dualLeftBasis dualLeftBasis) (dualLeftHandedRep M)) (i,
 * leftDualLeftToMatrix v k.1 k.2) = _
have h1 : $\sum x : \text{Fin } 2, (\sum x1 : \text{Fin } 2, M.1 \ i \ x1 \ * \ \text{leftDualLeftToMatrix } v \ x1 \ x) \ * \ (M.1)^{-1}$
 = $\sum x : \text{Fin } 2, \sum x1 : \text{Fin } 2, (M.1 \ i \ x1 \ * \ \text{leftDualLeftToMatrix } v \ x1 \ x) \ * \ (M.1)^{-1}$
 rw [h1]
simp only [leftBasis_p_apply, dualLeftBasis_p_apply, transpose_apply]
/-- The group action of `SL(2,ℂ)` on `dualLeftHanded ⊗ leftHanded` is equivalent to
`(M.1-1)⊤ * leftDualLeftToMatrix v * (M.1)⊤`. -/
lemma dualLeftLeftToMatrix_p (v : (DualLeftHandedWeyl ⊗ [ℂ] LeftHandedWeyl)) (M : SL(
 dualLeftLeftToMatrix (TensorProduct.map (dualLeftHandedRep M) (leftHandedRep M)
 (M.1-1)⊤ * dualLeftLeftToMatrix v * (M.1)⊤ := by
nth_rewrite 1 [dualLeftLeftToMatrix]
 (dualLeftBasis.tensorProduct leftBasis) (dualLeftBasis.tensorProduct leftBasis
 (TensorProduct.map (dualLeftHandedRep M) (leftHandedRep M)))
 ((dualLeftBasis.tensorProduct leftBasis).repr (v)))
have h1 := (LinearMap.toMatrix_mulVec_repr (dualLeftBasis.tensorProduct leftBasi
 (dualLeftBasis.tensorProduct leftBasis)
 (TensorProduct.map (dualLeftHandedRep M) (leftHandedRep M)) v)
simp only [coe_linearEquivFunOnFinite]
rw [h1]

```

```

 ((LinearMap.toMatrix dualLeftBasis dualLeftBasis) (dualLeftHandedRep M))
 ((LinearMap.toMatrix leftBasis leftBasis) (leftHandedRep M)) (i, j)
 * dualLeftLeftToMatrix v k.1 k.2) =
have h1 : $\sum x : \text{Fin } 2, (\sum x1 : \text{Fin } 2, (M.1)^{-1} x1 \ i * \text{dualLeftLeftToMatrix } v \ x1 \ x)$
= $\sum x : \text{Fin } 2, \sum x1 : \text{Fin } 2, ((M.1)^{-1} x1 \ i * \text{dualLeftLeftToMatrix } v \ x1 \ x)$
rw [h1]
simp only [dualLeftBasis_ρ_apply, transpose_apply, leftBasis_ρ_apply]
lemma rightRightToMatrix_ρ (v : (RightHandedWeyl ⊗[C] RightHandedWeyl)) (M : Matrix (Fin 2) (Fin 2) C) :
 rightRightToMatrix (TensorProduct.map (rightHandedRep M) (rightHandedRep M))
 (TensorProduct.map (rightHandedRep M) (rightHandedRep M))
 (TensorProduct.map (rightHandedRep M) (rightHandedRep M)) v)
 =
simp only [coe_linearEquivFunOnFinite]
rw [h1]
 ((LinearMap.toMatrix rightBasis rightBasis) (rightHandedRep M))
 ((LinearMap.toMatrix rightBasis rightBasis) (rightHandedRep M)) (i, j)
 * dualRightDualRightToMatrix v k.1 k.2) =
rw [h1]
/-- The group action of `SL(2,C)` on `dualRightHanded ⊗ dualRightHanded` is
lemma dualRightDualRightToMatrix_ρ (v : (DualRightHandedWeyl ⊗[C] DualRightHandedWeyl)) (M : Matrix (Fin 2) (Fin 2) C) :
 dualRightDualRightToMatrix (TensorProduct.map (dualRightHandedRep M) (dualRightHandedRep M))
 ((M.1-1).conjTranspose) * dualRightDualRightToMatrix v * ((M.1-1).conjTranspose)
 =
nth_rewrite 1 [dualRightDualRightToMatrix]
 (dualRightBasis.tensorProduct dualRightBasis) (dualRightBasis.tensorProduct
 (TensorProduct.map (dualRightHandedRep M) (dualRightHandedRep M))
 ((dualRightBasis.tensorProduct dualRightBasis).repr (v))))
have h1 := (LinearMap.toMatrix_mulVec_repr (dualRightBasis.tensorProduct
 (dualRightBasis.tensorProduct dualRightBasis)
 (TensorProduct.map (dualRightHandedRep M) (dualRightHandedRep M)) v)
simp only [coe_linearEquivFunOnFinite]
rw [h1]
 ((LinearMap.toMatrix dualRightBasis dualRightBasis) (dualRightHandedRep M))
 ((LinearMap.toMatrix dualRightBasis dualRightBasis) (dualRightHandedRep M))
 * dualRightDualRightToMatrix v k.1 k.2) =
have h1 : $\sum x : \text{Fin } 2, (\sum x1 : \text{Fin } 2, (\uparrow M)^{-1H} i \ x1 * \text{dualRightDualRightToMatrix } v \ x1 \ x)$
= $\sum x : \text{Fin } 2, (\sum x1 : \text{Fin } 2, ((\uparrow M)^{-1H} i \ x1 * \text{dualRightDualRightToMatrix } v \ x1 \ x) * (\uparrow M)^{-1H} j \ x) := b$
rw [h1]
simp only [dualRightBasis_ρ_apply]
/-- The group action of `SL(2,C)` on `rightHanded ⊗ dualRightHanded` is equal to
 `(M.1.map star) * rightDualRightToMatrix v * ((M.1-1).conjTranspose)T`.
/
lemma rightDualRightToMatrix_ρ (v : (RightHandedWeyl ⊗[C] DualRightHandedWeyl)) (M : Matrix (Fin 2) (Fin 2) C) :
 rightDualRightToMatrix (TensorProduct.map (rightHandedRep M) (dualRightHandedRep M))
 (M.1.map star) * rightDualRightToMatrix v * ((M.1-1).conjTranspose)T
 =
nth_rewrite 1 [rightDualRightToMatrix]
 (rightBasis.tensorProduct dualRightBasis) (rightBasis.tensorProduct
 (TensorProduct.map (rightHandedRep M) (dualRightHandedRep M))
 (TensorProduct.map (rightHandedRep M) (dualRightHandedRep M)))

```



1.215.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/WEYL/TWO.LEAN 301

```

 ((rightBasis.tensorProduct dualRightBasis).repr (v)))
 have h1 := (LinearMap.toMatrix_mulVec_repr (rightBasis.tensorProduct dualRightBasis)
 (rightBasis.tensorProduct dualRightBasis)
 (TensorProduct.map (rightHandedRep M) (dualRightHandedRep M)) v)
 simp only [coe_linearEquivFunOnFinite]
 rw [h1]
 ((LinearMap.toMatrix rightBasis rightBasis) (rightHandedRep M))
 ((LinearMap.toMatrix dualRightBasis dualRightBasis) (dualRightHandedRep M))
 * rightDualRightToMatrix v k.1 k.2) = _
 have h1 : $\sum x : \text{Fin } 2, (\sum x1 : \text{Fin } 2, (M.1.\text{map star}) i x1 * \text{rightDualRightToMatrix } v x1 x) * ((M.1.\text{map star}) i x1 * \text{rightDualRightToMatrix } v x1 x) * (\uparrow M)^{-1H} j x :=$ by
 rw [h1]
 simp only [rightBasis_ρ_apply, dualRightBasis_ρ_apply]
/- The group action of `SL(2,ℂ)` on `dualRightHanded ⊗ rightHanded` is equivalent to
 `((M.1⁻¹).conjTranspose * rightDualRightToMatrix v * ((M.1.\text{map star}) i x1 * \text{rightDualRightToMatrix } v x1 x) * (\uparrow M)^{-1H} j x :=`
 /
lemma dualRightRightToMatrix_ρ (v : (DualRightHandedWeyl ⊗[ℂ] RightHandedWeyl)) (M :
 dualRightRightToMatrix (TensorProduct.map (dualRightHandedRep M) (rightHandedRep M))
 ((M.1⁻¹).conjTranspose) * dualRightRightToMatrix v * (M.1.\text{map star})ᵀ := by
 nth_rewrite 1 [dualRightRightToMatrix]
 (dualRightBasis.tensorProduct rightBasis) (dualRightBasis.tensorProduct rightBasis)
 (TensorProduct.map (dualRightHandedRep M) (rightHandedRep M)))
 ((dualRightBasis.tensorProduct rightBasis).repr (v)))
 have h1 := (LinearMap.toMatrix_mulVec_repr (dualRightBasis.tensorProduct rightBasis)
 (dualRightBasis.tensorProduct rightBasis)
 (TensorProduct.map (dualRightHandedRep M) (rightHandedRep M)) v)
 simp only [coe_linearEquivFunOnFinite]
 rw [h1]
 ((LinearMap.toMatrix dualRightBasis dualRightBasis) (dualRightHandedRep M))
 ((LinearMap.toMatrix rightBasis rightBasis) (rightHandedRep M)) (i, j) k)
 * dualRightRightToMatrix v k.1 k.2) = _
 ($\uparrow M)^{-1H} i x1 * \text{dualRightRightToMatrix } v x1 x) * (M.1.\text{map star}) j x$
 = $\sum x : \text{Fin } 2, \sum x1 : \text{Fin } 2, ((\uparrow M)^{-1H} i x1 * \text{dualRightRightToMatrix } v x1 x) * (M.1.\text{map star}) j x$
 rw [h1]
 simp only [dualRightBasis_ρ_apply, rightBasis_ρ_apply]
lemma dualLeftDualRightToMatrix_ρ (v : (DualLeftHandedWeyl ⊗[ℂ] DualRightHandedWeyl))
 (M : SL(2,ℂ)) :
 dualLeftDualRightToMatrix (TensorProduct.map (dualLeftHandedRep M) (dualRightHandedRep M))
 (M.1⁻¹)ᵀ * dualLeftDualRightToMatrix v * ((M.1⁻¹).conjTranspose)ᵀ := by
 nth_rewrite 1 [dualLeftDualRightToMatrix]
 (dualLeftBasis.tensorProduct dualRightBasis) (dualLeftBasis.tensorProduct dualRightBasis)
 (TensorProduct.map (dualLeftHandedRep M) (dualRightHandedRep M)))
 ((dualLeftBasis.tensorProduct dualRightBasis).repr (v)))
 have h1 := (LinearMap.toMatrix_mulVec_repr (dualLeftBasis.tensorProduct dualRightBasis)
 (dualLeftBasis.tensorProduct dualRightBasis)
 (TensorProduct.map (dualLeftHandedRep M) (dualRightHandedRep M)) v)

```

```

simp only [coe_linearEquivFunOnFinite]
rw [h1]
 ((LinearMap.toMatrix dualLeftBasis dualLeftBasis) (dualLeftHandedRep M))
 ((LinearMap.toMatrix dualRightBasis dualRightBasis) (dualRightHandedRep M))
 * dualLeftDualRightToMatrix v k.1 k.2) =
have h1 : $\sum x : \text{Fin } 2, (\sum x1 : \text{Fin } 2, (M.1)^{-1} x1 \ i * \text{dualLeftDualRightToMatrix } v \ x1 \ x) * (M.1)^{-1} j \ x :=$ by
 ((M.1)-1 x1 i * dualLeftDualRightToMatrix v x1 x) * (M.1)-1 j x := by
rw [h1]
simp only [dualLeftBasis_ρ_apply, transpose_apply, dualRightBasis_ρ_apply]
lemma leftRightToMatrix_ρ (v : (LeftHandedWeyl ⊗ [C] RightHandedWeyl)) (M :
 leftRightToMatrix (TensorProduct.map (leftHandedRep M) (rightHandedRep M))
 (TensorProduct.map (leftHandedRep M) (rightHandedRep M)))
 (TensorProduct.map (leftHandedRep M) (rightHandedRep M)) v)
simp only [coe_linearEquivFunOnFinite]
rw [h1]
 ((LinearMap.toMatrix leftBasis leftBasis) (leftHandedRep M))
 ((LinearMap.toMatrix rightBasis rightBasis) (rightHandedRep M)) (i,
 j) := by
 rw [h1]
 TensorProduct.map (leftHandedRep M) (leftHandedRep M) (leftLeftToMatrix_ρ_apply v)
 rw [← h1, LinearEquiv.symm_apply_apply]
lemma dualLeftdualLeftToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) C) (M : SL(2, C))
 TensorProduct.map (dualLeftHandedRep M) (dualLeftHandedRep M)
 (dualLeftdualLeftToMatrix.symm v) =
 dualLeftdualLeftToMatrix.symm ((M.1-1)⊤ * v * (M.1-1)) := by
 have h1 := dualLeftdualLeftToMatrix_ρ (dualLeftdualLeftToMatrix.symm v) M
 rw [← h1, LinearEquiv.symm_apply_apply]
lemma leftDualLeftToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) C) (M : SL(2, C))
 TensorProduct.map (leftHandedRep M) (dualLeftHandedRep M) (leftDualLeftToMatrix_ρ_apply v)
 leftDualLeftToMatrix.symm (M.1 * v * (M.1-1)) := by
 have h1 := leftDualLeftToMatrix_ρ (leftDualLeftToMatrix.symm v) M
 rw [← h1, LinearEquiv.symm_apply_apply]
lemma dualLeftLeftToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) C) (M : SL(2, C))
 TensorProduct.map (dualLeftHandedRep M) (leftHandedRep M) (dualLeftLeftToMatrix_ρ_apply v)
 dualLeftLeftToMatrix.symm ((M.1-1)⊤ * v * (M.1)⊤) := by
 have h1 := dualLeftLeftToMatrix_ρ (dualLeftLeftToMatrix.symm v) M
 rw [← h1, LinearEquiv.symm_apply_apply]
 TensorProduct.map (rightHandedRep M) (rightHandedRep M) (rightRightToMatrix_ρ_apply v)
 rw [← h1, LinearEquiv.symm_apply_apply]
lemma dualRightDualRightToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) C) (M : SL(2, C))
 TensorProduct.map (dualRightHandedRep M) (dualRightHandedRep M)
 (dualRightDualRightToMatrix.symm v) =
 dualRightDualRightToMatrix.symm ((M.1-1).conjTranspose * v * (M.1-1)) := by
 have h1 := dualRightDualRightToMatrix_ρ (dualRightDualRightToMatrix.symm v) M
 rw [← h1, LinearEquiv.symm_apply_apply]
lemma rightDualRightToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) C) (M : SL(2, C))
 TensorProduct.map (rightHandedRep M) (dualRightHandedRep M) (rightDualRightToMatrix_ρ_apply v)
 rightDualRightToMatrix.symm (M.1 * v * (M.1-1)) := by
 have h1 := rightDualRightToMatrix_ρ (rightDualRightToMatrix.symm v) M
 rw [← h1, LinearEquiv.symm_apply_apply]

```

1.216.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/WEYL/UNIT.lean

```

 rightDualRightToMatrix.symm ((M.1.map star) * v * (((M.1-1).conjTranspose)⊤)) :=
 have h1 := rightDualRightToMatrix_ρ (rightDualRightToMatrix.symm v) M
 rw [← h1, LinearEquiv.symm_apply_apply]
lemma dualRightRightToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,ℂ)) :
 TensorProduct.map (dualRightHandedRep M) (rightHandedRep M) (dualRightRightToMatrix_ρ
 dualRightRightToMatrix.symm (((M.1-1).conjTranspose) * v * (M.1.map star)⊤) := b
 have h1 := dualRightRightToMatrix_ρ (dualRightRightToMatrix.symm v) M
 rw [← h1, LinearEquiv.symm_apply_apply]
lemma dualLeftDualRightToMatrix_ρ_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,ℂ)) :
 TensorProduct.map (dualLeftHandedRep M) (dualRightHandedRep M)
 (dualLeftDualRightToMatrix.symm v) =
 dualLeftDualRightToMatrix.symm ((M.1-1)⊤ * v * ((M.1-1).conjTranspose)⊤) := by
 have h1 := dualLeftDualRightToMatrix_ρ (dualLeftDualRightToMatrix.symm v) M
 rw [← h1, LinearEquiv.symm_apply_apply]
 TensorProduct.map (leftHandedRep M) (rightHandedRep M) (leftRightToMatrix.symm v)
 rw [← h1, LinearEquiv.symm_apply_apply]
lemma dualLeftDualRightToMatrix_ρ_symm_selfAdjoint (v : Matrix (Fin 2) (Fin 2) ℂ) :
 TensorProduct.map (dualLeftHandedRep M) (dualRightHandedRep M)
 (dualLeftDualRightToMatrix.symm v) =
 dualLeftDualRightToMatrix.symm (SL2C.toSelfAdjointMap (M.transpose-1) ⟨v, hv⟩) :
 rw [dualLeftDualRightToMatrix_ρ_symm]
 TensorProduct.map (leftHandedRep M) (rightHandedRep M) (leftRightToMatrix.symm v)

```

## 1.216 Physlib/Relativity/Tensors/ComplexTensor/Weyl/Unit.lean

```

/-- The left-dual-left unit δ^a_a as an element of $(\text{leftHanded} \otimes \text{dualLeftHanded}).V$ /
def leftDualLeftUnitVal : (LeftHandedWeyl $\otimes$ [ℂ] DualLeftHandedWeyl) :=
 leftDualLeftToMatrix.symm 1

/-- Expansion of `leftDualLeftUnitVal` into the basis. -/
lemma leftDualLeftUnitVal_expand_tmul : leftDualLeftUnitVal =
 leftBasis 0 $\otimes$ [ℂ] dualLeftBasis 0 + leftBasis 1 $\otimes$ [ℂ] dualLeftBasis 1 := by
 simp only [leftDualLeftUnitVal, Fin.isValue]
 erw [leftDualLeftToMatrix_symm_expand_tmul]
/-- The left-dual-left unit δ^a_a as a morphism $\square_{\text{leftHanded} \otimes \text{dualLeftHanded}} (\text{Rep } \mathbb{C} \text{ SL}(2, \mathbb{C})) \square \text{leftHanded} \otimes \text{dualLeftHanded}$ /
def leftDualLeftUnit : (Representation.trivial ℂ SL(2,ℂ) ℂ).IntertwiningMap
 (leftHandedRep.tprod dualLeftHandedRep) where
 toFun := fun a =>
 let a' : ℂ := a
 a' • leftDualLeftUnitVal
 map_add' := fun x y => by
 simp only [add_smul]
 map_smul' := fun m x => by

```

```

 simp only [smul_smul]
 rfl
 isIntertwining' M := by
 refine LinearMap.ext fun x : $\mathbb{C}$ => ?_
 change x • leftDualLeftUnitVal =
 (TensorProduct.map (leftHandedRep M) (dualLeftHandedRep M)) (x • left
 simp only [map_smul]
 apply congrArg
 simp only [leftDualLeftUnitVal]
 rw [leftDualLeftToMatrix_ρ_symm]
 apply congrArg
 simp

lemma leftDualLeftUnit_apply_one : leftDualLeftUnit (1 : $\mathbb{C}$) = leftDualLeftU
 change (1 : $\mathbb{C}$) • leftDualLeftUnitVal = leftDualLeftUnitVal
/-- The dual-left-left unit ` $\delta_a^a$ ` as an element of `(dualLeftHanded $\otimes$ leftH
/
def dualLeftLeftUnitVal : (DualLeftHandedWeyl $\otimes$ [$\mathbb{C}$] LeftHandedWeyl) :=
 dualLeftLeftToMatrix.symm 1
/-- Expansion of `dualLeftLeftUnitVal` into the basis. -/
lemma dualLeftLeftUnitVal_expand_tmul : dualLeftLeftUnitVal =
 dualLeftBasis 0 $\otimes$ [$\mathbb{C}$] leftBasis 0 + dualLeftBasis 1 $\otimes$ [$\mathbb{C}$] leftBasis 1 :
 simp only [dualLeftLeftUnitVal, Fin.isValue]
 rw [dualLeftLeftToMatrix_symm_expand_tmul]
/-- The dual-left-left unit ` $\delta_a^a$ ` as a morphism ` $\square_{_}$ (Rep $\mathbb{C}$ SL(2, $\mathbb{C}$)) $\square$ dualL
def dualLeftLeftUnit :
 (Representation.trivial $\mathbb{C}$ SL(2, $\mathbb{C}$) $\mathbb{C}$).IntertwiningMap
 (dualLeftHandedRep.tprod leftHandedRep) where
 a' • dualLeftLeftUnitVal
 change x • dualLeftLeftUnitVal =
 (TensorProduct.map (dualLeftHandedRep M) (leftHandedRep M)) (x • dual
 simp only [dualLeftLeftUnitVal]
 rw [dualLeftLeftToMatrix_ρ_symm]
/-- Applying the morphism `dualLeftLeftUnit` to `1` returns `dualLeftLeftUn
/
lemma dualLeftLeftUnit_apply_one : dualLeftLeftUnit (1 : $\mathbb{C}$) = dualLeftLeftU
 change (1 : $\mathbb{C}$) • dualLeftLeftUnitVal = dualLeftLeftUnitVal
/-- The right-dual-right unit ` $\delta^{\{dot a\}}_{\{dot a\}}$ ` as an element of
 `(rightHanded  $\otimes$  dualRightHanded).V`. -/
def rightDualRightUnitVal : RightHandedWeyl $\otimes$ [$\mathbb{C}$] DualRightHandedWeyl :=
 rightDualRightToMatrix.symm 1

/-- Expansion of `rightDualRightUnitVal` into the basis. -
/
lemma rightDualRightUnitVal_expand_tmul : rightDualRightUnitVal =
 rightBasis 0 $\otimes$ [$\mathbb{C}$] dualRightBasis 0 + rightBasis 1 $\otimes$ [$\mathbb{C}$] dualRightBasis

```

1.216.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/WEYL/UNIT. LEAN 3.5

```

simp only [rightDualRightUnitVal, Fin.isValue]
rw [rightDualRightToMatrix_symm_expand_tmul]
/-- The right-dual-right unit $\delta^{\{\dot{a}\}}_{\{\dot{a}\}}$ as a morphism
 $\square_{\text{rightDualRightUnitVal}} : (\text{Rep } \mathbb{C} \text{ SL}(2, \mathbb{C})) \square \text{rightHanded} \otimes \text{dualRightHanded}$, manifesting
def rightDualRightUnit : (Representation.trivial $\mathbb{C} \text{ SL}(2, \mathbb{C}) \mathbb{C}$).IntertwiningMap
 (rightHandedRep.tprod dualRightHandedRep) where
toFun := fun a =>
 let a' : $\mathbb{C}$:= a
 a' • rightDualRightUnitVal
map_add' := fun x y => by
 simp only [add_smul]
map_smul' := fun m x => by
 simp only [smul_smul]
rfl
isIntertwining' M := by
 refine LinearMap.ext fun x : $\mathbb{C}$ => ?_
 change x • rightDualRightUnitVal =
 (TensorProduct.map (rightHandedRep M) (dualRightHandedRep M)) (x • rightDualRightUnitVal)
 simp only [map_smul]
 apply congrArg
 simp only [rightDualRightUnitVal]
 rw [rightDualRightToMatrix_rho_symm]
 apply congrArg
 simp only [RCLike.star_def, mul_one]
 symm
 refine transpose_eq_one.mp ?h.h.h.a
 simp only [transpose_mul, transpose_transpose]
 change (M.1)-1H * (M.1)H = 1
 rw [@conjTranspose_nonsing_inv]
 simp

lemma rightDualRightUnit_apply_one : rightDualRightUnit (1 : $\mathbb{C}$) = rightDualRightUnitVal
 change (1 : $\mathbb{C}$) • rightDualRightUnitVal = rightDualRightUnitVal
/-- The dual-right-right unit $\delta_{\{\dot{a}\}}^{\{\dot{a}\}}$ as an element of
 $(\text{rightHanded} \otimes \text{dualRightHanded}).V$. -/
def dualRightRightUnitVal : (DualRightHandedWeyl $\otimes [\mathbb{C}]$ RightHandedWeyl) :=
 dualRightRightToMatrix.symm 1

/-- Expansion of `dualRightRightUnitVal` into the basis. -
/
lemma dualRightRightUnitVal_expand_tmul : dualRightRightUnitVal =
 dualRightBasis 0 $\otimes_{[\mathbb{C}]}$ rightBasis 0 + dualRightBasis 1 $\otimes_{[\mathbb{C}]}$ rightBasis 1 := by
 simp only [dualRightRightUnitVal, Fin.isValue]
 rw [dualRightRightToMatrix_symm_expand_tmul]
/-- The dual-right-right unit $\delta_{\{\dot{a}\}}^{\{\dot{a}\}}$ as a morphism
 $\square_{\text{dualRightRightUnitVal}} : (\text{Rep } \mathbb{C} \text{ SL}(2, \mathbb{C})) \square \text{dualRightHanded} \otimes \text{rightHanded}$, manifesting

```

```

def dualRightRightUnit : (Representation.trivial $\mathbb{C}$ SL(2, $\mathbb{C}$) $\mathbb{C}$).IntertwiningM
 (dualRightHandedRep.tprod rightHandedRep) where
 toFun := fun a =>
 let a' : $\mathbb{C}$:= a
 a' • dualRightRightUnitVal
 map_add' := fun x y => by
 simp only [add_smul]
 map_smul' := fun m x => by
 simp only [smul_smul]
 rfl
 isIntertwining' M := by
 refine LinearMap.ext fun x : $\mathbb{C}$ => ?_
 change x • dualRightRightUnitVal =
 (TensorProduct.map (dualRightHandedRep M) (rightHandedRep M)) (x • du
 simp only [map_smul]
 apply congrArg
 simp only [dualRightRightUnitVal]
 rw [dualRightRightToMatrix_ρ_symm]
 apply congrArg
 simp only [mul_one, RCLike.star_def]
 symm
 change (M.1)-1H * (M.1)H = 1
 rw [@conjTranspose_nonsing_inv]
 simp

lemma dualRightRightUnit_apply_one : dualRightRightUnit (1 : $\mathbb{C}$) = dualRight
 change (1 : $\mathbb{C}$) • dualRightRightUnitVal = dualRightRightUnitVal
/-- Contraction on the right with `dualLeftLeftUnit` does nothing. -
/
lemma contr_dualLeftLeftUnit (x : LeftHandedWeyl) :
 (TensorProduct.lid $\mathbb{C}$ _ <|
 leftDualContraction.toLinearMap.rTensor _ <|
 (TensorProduct.assoc $\mathbb{C}$ _ _).symm <|
 x ⊗t [$\mathbb{C}$] (dualLeftLeftUnit (1 : $\mathbb{C}$))) = x := by
 simp [- Fintype.sum_sum_type, smul_tmul, leftDualContraction_basis,
 dualLeftLeftUnit_apply_one, dualLeftLeftUnitVal_expand_tmul, add_tmul,

/-- Contraction on the right with `leftDualLeftUnit` does nothing. -
/
lemma contr_leftDualLeftUnit (x : DualLeftHandedWeyl) :
 (TensorProduct.lid $\mathbb{C}$ _ <|
 dualLeftContraction.toLinearMap.rTensor _ <|
 (TensorProduct.assoc $\mathbb{C}$ _ _).symm <|
 x ⊗t [$\mathbb{C}$] (leftDualLeftUnit (1 : $\mathbb{C}$))) = x := by
 obtain ⟨c, hc⟩ := (Submodule.mem_span_range_iff_exists_fun $\mathbb{C}$).mp (Basis.m
 simp [- Fintype.sum_sum_type, smul_tmul, dualLeftContraction_basis,

```

1.216.

PHYSLIB/RELATIVITY/TENSORS/COMPLEXTENSOR/WEYL/UNIT.LEAN07

```

 leftDualLeftUnit_apply_one, leftDualLeftUnitVal_expand_tmul, add_tmul, tmul_add]

/-- Contraction on the right with `dualRightRightUnit` does nothing. -
/
lemma contr_dualRightRightUnit (x : RightHandedWeyl) :
 (TensorProduct.lid $\mathbb{C}$ _ <|
 rightDualContraction.toLinearMap.rTensor _ <|
 (TensorProduct.assoc $\mathbb{C}$ _ _).symm <|
 x $\otimes_t$ [$\mathbb{C}$] (dualRightRightUnit (1 : $\mathbb{C}$))) = x := by
 simp [- Fintype.sum_sum_type, smul_tmul, rightDualContraction_basis,
 dualRightRightUnit_apply_one, dualRightRightUnitVal_expand_tmul, add_tmul, tmul_

/-- Contraction on the right with `rightDualRightUnit` does nothing. -
/
lemma contr_rightDualRightUnit (x : DualRightHandedWeyl) :
 (TensorProduct.lid $\mathbb{C}$ _ <|
 dualRightContraction.toLinearMap.rTensor _ <|
 (TensorProduct.assoc $\mathbb{C}$ _ _).symm <|
 x $\otimes_t$ [$\mathbb{C}$] (rightDualRightUnit (1 : $\mathbb{C}$))) = x := by
 obtain {c, hc} := (Submodule.mem_span_range_iff_exists_fun $\mathbb{C}$).mp (Basis.mem_span d
 simp [- Fintype.sum_sum_type, smul_tmul, dualRightContraction_basis,
 rightDualRightUnit_apply_one, rightDualRightUnitVal_expand_tmul, add_tmul, tmul_
lemma dualLeftLeftUnit_symm :
 dualLeftLeftUnit (1 : $\mathbb{C}$) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinearMa
 (TensorProduct.comm $\mathbb{C}$ _ _ (dualLeftLeftUnit (1 : $\mathbb{C}$))) := by
 rw [dualLeftLeftUnit_apply_one, dualLeftLeftUnitVal_expand_tmul]
 rw [leftDualLeftUnit_apply_one, leftDualLeftUnitVal_expand_tmul]
lemma leftDualLeftUnit_symm :
 leftDualLeftUnit (1 : $\mathbb{C}$) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinearMa
 (TensorProduct.comm $\mathbb{C}$ _ _ (leftDualLeftUnit (1 : $\mathbb{C}$))) := by
 rw [dualLeftLeftUnit_apply_one, dualLeftLeftUnitVal_expand_tmul]
 rw [leftDualLeftUnit_apply_one, leftDualLeftUnitVal_expand_tmul]
lemma dualRightRightUnit_symm :
 dualRightRightUnit (1 : $\mathbb{C}$) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinear
 (TensorProduct.comm $\mathbb{C}$ _ _ (dualRightRightUnit (1 : $\mathbb{C}$))) := by
 rw [dualRightRightUnit_apply_one, dualRightRightUnitVal_expand_tmul]
 rw [rightDualRightUnit_apply_one, rightDualRightUnitVal_expand_tmul]
lemma rightDualRightUnit_symm :
 rightDualRightUnit (1 : $\mathbb{C}$) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinear
 (TensorProduct.comm $\mathbb{C}$ _ _ (rightDualRightUnit (1 : $\mathbb{C}$))) := by
 rw [dualRightRightUnit_apply_one, dualRightRightUnitVal_expand_tmul]
 rw [rightDualRightUnit_apply_one, rightDualRightUnitVal_expand_tmul]

```

**1.217 Physlib/Relativity/Tensors/ComponentIdx/Basic.lean**

```

/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Physlib.Relativity.Tensors.TensorSpecies.Basic
/-!

Component indices

i. Overview

For a tensor species `S` and a list of colors `c : Fin n → C`, `ComponentIdx`
is the type of choices of a basis index for every color in `c`. For example
tensor with colors `[.up, .down]`, an element of `ComponentIdx [.up, .down]`
records one contravariant component index and one covariant component index.

This file contains the basic definition and congruence/cast API. Operations
component indices induced by tensor products and contractions live in sibli.

ii. Key results

- `TensorSpecies.Tensor.ComponentIdx` is the type of component indices for
 with a given list of colors.
- `TensorSpecies.Tensor.ComponentIdx.congr_right` relates entries whose pos
- `TensorSpecies.Tensor.ComponentIdx.cast` casts component indices along eq

iii. Table of contents

- A. Basic definitions

iv. References

There are no known references for the material in this module.

-/

@[expose] public section

namespace TensorSpecies

variable {k : Type} [CommRing k] {C G : Type} [Group G]

```



1.218.

PHYSLIB/RELATIVITY/TENSORS/COMPONENTIDX/CONTRACTION.lean

```
{V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
{basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
{rep : (c : C) → Representation k G (V c)}
{b : (c : C) → Module.Basis (basisIdx c) k (V c)}
{S : TensorSpecies k C G V basisIdx rep b}

namespace Tensor

/-!

A. Basic definitions

-/

set_option linter.unusedVariables false in
/-- Given a list of indices `c : Fin n → C`, the type `ComponentIdx c` is the type of
component indices of a tensor with those colors. For instance, if `c = ![.up, .down]`
then an element of `ComponentIdx c` specifies one basis index for `.up` and one for
`.down`. -/
@[nolint unusedArguments]
abbrev ComponentIdx {n : ℕ} {S : TensorSpecies k C G V basisIdx rep b}
 (c : Fin n → C) : Type :=
 Π j, basisIdx (c j)

lemma ComponentIdx.congr_right {n : ℕ} {c : Fin n → C} (b : ComponentIdx (S := S) c)
 (i j : Fin n) (h : i = j) : b i = basisIdxCongr (by simp [h]) (b j) := by
 subst h
 simp [basisIdxCongr]

/-- Casting of a `ComponentIdx` through equivalent color maps. -
/
def ComponentIdx.cast {n m : ℕ} {c : Fin n → C} {cm : Fin m → C}
 (h : n = m) (hc : c = cm ∘ Fin.cast h) (b : ComponentIdx (S := S) c) :
 ComponentIdx (S := S) cm := fun j =>
 basisIdxCongr (by simp [hc]) (b (Fin.cast h.symm j))

end Tensor

end TensorSpecies
```

## 1.218 Physlib/Relativity/Tensors/ComponentIdx/Contraction.lean

```
/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
```

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Joseph Tooby-Smith

-/

module

public import Physlib.Relativity.Tensors.ComponentIdx.Basic

public import Physlib.Relativity.Tensors.Contraction.SuccSuccAbove

/-!

# Contractions of component indices

## i. Overview

This file contains the component-index API induced by dropping a pair of component indices from a tensor.

The constructions here describe how component indices restrict along ``Fin.succSuccAbove``, and how the fiber of this restriction is equivalent to component choices at the contracted positions.

## ii. Key results

- ``TensorSpecies.Tensor.ComponentIdx.dropPair`` restricts a component index to two positions.
- ``TensorSpecies.Tensor.ComponentIdx.DropPairSection`` is the finite set of indices mapping to a fixed restricted component index.
- ``TensorSpecies.Tensor.ComponentIdx.DropPairSection.ofFinEquiv`` identifies ``DropPairSection`` with the two basis indices at the dropped positions.

## iii. Table of contents

- A. Dropping a pair
- B. Sections of the drop-pair map

## iv. References

There are no known references for the material in this module.

-/

@[expose] public section

namespace TensorSpecies

variable {k : Type} [CommRing k] {C G : Type} [Group G]

{V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]

1.218.

PHYSLIB/RELATIVITY/TENSORS/COMPONENTIDX/CONTRACTION. LEAN1

```
{basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
{rep : (c : C) → Representation k G (V c)}
{b : (c : C) → Module.Basis (basisIdx c) k (V c)}
{S : TensorSpecies k C G V basisIdx rep b}
```

namespace Tensor

namespace ComponentIdx

/-!

## A. Dropping a pair

-/

/-- The `ComponentIdx` obtained by dropping two components. -

/

```
def dropPair {n : ℕ} {c : Fin (n + 1 + 1) → C}
 (i j : Fin (n + 1 + 1)) (b : ComponentIdx (S := S) c) :
 ComponentIdx (S := S) (c ∘ Fin.succSuccAbove i j) :=
 fun m => b (Fin.succSuccAbove i j m)
```

/-!

## B. Sections of the drop-pair map

-/

/-- Given a coordinate parameter

`b :  $\prod k, \text{Fin } (S.\text{repDim } ((c \circ i.\text{succAbove} \circ j.\text{succAbove}) k))$ `, the  
coordinate parameter ` $\prod k, \text{Fin } (S.\text{repDim } (c k))$ ` which map down to `b`. -

/

```
def DropPairSection {n : ℕ} {c : Fin (n + 1 + 1) → C}
 {i j : Fin (n + 1 + 1)} {j : Fin (n + 1 + 1)}
 (b : ComponentIdx (S := S) (c ∘ Fin.succSuccAbove i j)) :
 Finset (ComponentIdx (S := S) c) :=
 {b' : ComponentIdx c | dropPair i j b' = b}
```

namespace DropPairSection

```
lemma mem_iff_apply_succSuccAbove_eq {n : ℕ} {c : Fin (n + 1 + 1) → C}
 {i j : Fin (n + 1 + 1)}
 (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
 (b' : ComponentIdx c) :
 b' ∈ DropPairSection (S := S) b ↔
 ∀ m, b' (Fin.succSuccAbove i j m) = b m := by
```

```

simp only [DropPairSection, Finset.mem_filter, Finset.mem_univ, true_and]
rw [funext_iff]
simp [dropPair]

@[simp]
lemma mem_self_of_dropPair {n : ℕ} {c : Fin (n + 1 + 1) → C}
 {i j : Fin (n + 1 + 1)}
 (b : ComponentIdx (c)) :
 b ∈ DropPairSection (S := S) (b.dropPair i j) := by
 simp [DropPairSection]

/-- Given a `b` in `ComponentIdx (c ∘ Fin.succSuccAbove i j)` and
 an `x` in `Fin (S.repDim (c i)) × Fin (S.repDim (c j))`, the corresponding
 coordinate parameter in `ComponentIdx c`. -/
def ofFin {n : ℕ} {c : Fin (n + 1 + 1) → C}
 {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (S := S) (c ∘ Fin.succSuccAbove i j))
 (x : basisIdx (c i) × basisIdx (c j)) :
 ComponentIdx (S := S) c := fun m =>
 if hij : m = i then basisIdxCongr (by subst hij; rfl) x.1
 else if hij : m = j then basisIdxCongr (by subst hij; rfl) x.2
 else
 basisIdxCongr (by simp)
 (b (Fin.predPredAbove i j hij m (by omega)))

@[simp]
lemma ofFin_apply_fst {n : ℕ} {c : Fin (n + 1 + 1) → C}
 {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
 (x : basisIdx (c i) × basisIdx (c j)) :
 ofFin (S := S) hij b x i = x.1 := by
 simp [ofFin]

@[simp]
lemma ofFin_apply_snd {n : ℕ} {c : Fin (n + 1 + 1) → C}
 {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
 (x : basisIdx (c i) × basisIdx (c j)) :
 ofFin (S := S) hij b x j = x.2 := by
 simp [ofFin]
 intro h
 omega

lemma ofFin_mem_succSuccAboveSection {n : ℕ} {c : Fin (n + 1 + 1) → C}
 {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
 (x : basisIdx (c i) × basisIdx (c j)) :
 ofFin (S := S) hij b x ∈ DropPairSection b := by
 simp only [DropPairSection, Finset.mem_filter, Finset.mem_univ, true_and]
 ext m

```

1.218.

PHYSLIB/RELATIVITY/TENSORS/COMPONENTIDX/CONTRACTION. LEAN3

```
simp only [ofFin, dropPair, Fin.succSuccAbove_ne_fst, ↓reduceDite, Fin.succSuccAbove_ne_fst]
Function.comp_apply]
symm
apply ComponentIdx.congr_right
simp

/-- The equivalence between `ContrSection b` and
`basisIdx (c i) × basisIdx (c j)` -/
def ofFinEquiv {n : ℕ} {c : Fin n.succ.succ → C}
 {i j : Fin (n + 1 + 1)} (hij : i ≠ j)
 (b : ComponentIdx (c ∘ Fin.succSuccAbove i j)) :
 basisIdx (c i) × basisIdx (c j) ≈ DropPairSection (S := S) b where
invFun b' := ⟨b'.1 i, b'.1 j⟩
toFun x := ⟨ofFin hij b x, ofFin_mem_succSuccAboveSection hij b x⟩
right_inv b' := by
 ext m
 simp
 rcases Fin.eq_or_exists_succSuccAbove i j hij m with rfl | rfl | ⟨m, rfl⟩
 · simp
 · simp
 · simp [ofFin]
 obtain ⟨b', hb'⟩ := b'
 simp only [mem_iff_apply_succSuccAbove_eq] at hb'
 simp [hb']
 symm
 apply ComponentIdx.congr_right
 simp
left_inv x := by
 simp

@[simp]
lemma ofFinEquiv_apply_fst {n : ℕ} {c : Fin (n + 1 + 1) → C}
 {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
 (x : basisIdx (c i) × basisIdx (c j)) :
 (ofFinEquiv (S := S) hij b x).1 i = x.1 := by
 simp [ofFinEquiv]

@[simp]
lemma ofFinEquiv_apply_snd {n : ℕ} {c : Fin (n + 1 + 1) → C}
 {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
 (x : basisIdx (c i) × basisIdx (c j)) :
 (ofFinEquiv (S := S) hij b x).1 j = x.2 := by
 simp [ofFinEquiv]

end DropPairSection
```

```

end ComponentIdx
end Tensor
end TensorSpecies

```

### 1.219 Physlib/Relativity/Tensors/ComponentIdx/Product.lean

```

/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Physlib.Relativity.Tensors.ComponentIdx.Basic
/-!

Products of component indices

i. Overview

This file contains the component-index API induced by appending two lists of
component indices.

The main construction identifies component indices for appended color lists
of component indices for each side of the append.

ii. Key results

- `TensorSpecies.Tensor.ComponentIdx.prod` is the equivalence between
 `ComponentIdx (Fin.append c c1)` and `ComponentIdx c × ComponentIdx c1`.

iii. Table of contents

- A. Product equivalence

iv. References

There are no known references for the material in this module.

-/

@[expose] public section

```

```
namespace TensorSpecies
```

```
variable {k C G : Type} [CommRing k] [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
 {rep : (c : C) → Representation k G (V c)}
 {b : (c : C) → Module.Basis (basisIdx c) k (V c)}
 {S : TensorSpecies k C G V basisIdx rep b}
```

```
namespace Tensor
```

```
/-!
```

```
A. Product equivalence
```

```
-/
```

```
/-- The equivalence between `ComponentIdx (Fin.append c c1)` and
 `ComponentIdx c × ComponentIdx c1` formed by products. -
```

```
/
```

```
def ComponentIdx.prod {n1 n2 : ℕ} {c : Fin n1 → C} {c1 : Fin n2 → C} :
 ComponentIdx (S := S) (Fin.append c c1) ≈
 ComponentIdx (S := S) c × ComponentIdx (S := S) c1 where
 toFun p := (fun i => basisIdxCongr (by simp) (p (Fin.castAdd n2 i))),
 fun i => basisIdxCongr (by simp) (p (Fin.natAdd n1 i)))
 invFun p := Fin.addCases (fun i => basisIdxCongr (by simp) (p.1 i))
 (fun i => basisIdxCongr (by simp) (p.2 i))
 left_inv p := by
 ext1 i
 revert i
 simp [Fin.forall_fin_add]
 right_inv p := by simp
```

```
@[simp]
```

```
lemma ComponentIdx.prod_symm_natAdd {n1 n2 : ℕ} {c : Fin n1 → C} {c1 : Fin n2 → C}
 (p : ComponentIdx (S := S) c) (q : ComponentIdx (S := S) c1) (i : Fin n2) :
 ComponentIdx.prod.symm (p, q) (Fin.natAdd n1 i) =
 basisIdxCongr (by simp) (q i) := by simp [ComponentIdx.prod]
```

```
@[simp]
```

```
lemma ComponentIdx.prod_symm_castAdd {n1 n2 : ℕ} {c : Fin n1 → C} {c1 : Fin n2 → C}
 (p : ComponentIdx (S := S) c) (q : ComponentIdx (S := S) c1) (i : Fin n1) :
 ComponentIdx.prod.symm (p, q) (Fin.castAdd n2 i) =
 basisIdxCongr (by simp) (p i) := by simp [ComponentIdx.prod]
```

```
end Tensor
```

end TensorSpecies

## 1.220 Physlib/Relativity/Tensors/ComponentIdx/Single.lean

```

/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Physlib.Relativity.Tensors.ComponentIdx.Basic
/-!

Component indices for one-index tensors

i. Overview

This file defines the canonical equivalence between component indices for a
color and the basis indices of that color.

ii. Key results

- `TensorSpecies.Tensor.ComponentIdx.single` is the equivalence between
 `ComponentIdx ![c]` and `basisIdx c`.
- `TensorSpecies.Tensor.ComponentIdx.single_apply` and
 `TensorSpecies.Tensor.ComponentIdx.single_symm_apply` are simp lemmas for
 directions of this equivalence.

iii. Table of contents

- A. Single-index equivalence

iv. References

There are no known references for the material in this module.

-/

@[expose] public section

namespace TensorSpecies

variable {k : Type} [CommRing k] {C G : Type} [Group G]
```



```

{V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
{basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
{rep : (c : C) → Representation k G (V c)}
{b : (c : C) → Module.Basis (basisIdx c) k (V c)}
{S : TensorSpecies k C G V basisIdx rep b}

namespace Tensor

/-!

A. Single-index equivalence

-/

/-- The equivalence between component indices for a single color and the basis indices
of that color. -/
def ComponentIdx.single {c : C} :
 ComponentIdx (S := S) ![c] ≈ basisIdx c where
 toFun b := basisIdxCongr (by simp) (b 0)
 invFun b := fun _ => basisIdxCongr (by simp) b
 left_inv b := by
 ext i
 cases Fin.fin_one_eq_zero i
 simp [basisIdxCongr]
 rfl
 right_inv b := by
 simp [basisIdxCongr]
 rfl

@[simp]
lemma ComponentIdx.single_apply {c : C} (b : ComponentIdx (S := S) ![c]) :
 ComponentIdx.single (S := S) b = basisIdxCongr (by simp) (b 0) := rfl

@[simp]
lemma ComponentIdx.single_symm_apply {c : C} (b : basisIdx c) (i : Fin 1) :
 (ComponentIdx.single (S := S) (c := c)).symm b i = basisIdxCongr (by simp) b :=

end Tensor

end TensorSpecies

```

## 1.221 Physlib/Relativity/Tensors/Conjugation/Basic.lean

```

/-

```

Copyright (c) 2026 Andrea Pari. All rights reserved.  
 Released under Apache 2.0 license as described in the file LICENSE.  
 Authors: Andrea Pari  
 -/  
 module

```
public import Physlib.Relativity.Tensors.Contraction.Basic
public import Physlib.Relativity.Tensors.Contraction.Basis
public import Physlib.Mathematics.ConjModule
public import Mathlib.Algebra.Star.Basic
public import Mathlib.LinearAlgebra.Finsupp.LSum
```

```
/-!
```

```
Conjugation structure on a tensor species
```

```
i. Overview
```

Each index of a tensor carries a colour naming the representation it transforms under. If an index already has the variance dual  $\tau c$ , the colour an index must meet to contract with is  $\bar{c}$ . Under second involution, the conjugate colour  $\bar{c}$ : the colour an index transforms under is  $\bar{c}$ . Conjugation. A spinor index is the textbook example: complex conjugation sends left-handed Weyl

spinor to a right-handed one,  $(\psi_\alpha)^* = \bar{\psi}_{\dot{\alpha}}$ , so  $\bar{\phantom{x}}$  swaps the left- and right-handed colours

while fixing a real vector colour. Variance is untouched, so  $\bar{\phantom{x}}$  commutes with raising and lowering indices. In the chiral sector  $\bar{\phantom{x}}$  swaps chiral and anti-chiral indices.

$\text{conjT}$  conjugates a tensor at the basis level:  $\text{star}$  each coordinate in the tensor and place it at the conjugate colour. Reality and Hermiticity are stated through the metric when conjugation fixes it, and a metric is Hermitian when conjugating it swaps the indices.

Conjugation is intrinsic species data, not a detachable add-on:

a  $\text{ConjTensorSpecies}$  is a  $\text{TensorSpecies}$  bundled ( $\text{extends}$ ) with the conjugate-colour involution  $\bar{\phantom{x}}$  and the recipe needs.  $\bar{c}$  shares basis labels with  $c$  ( $\text{barIdx\_eq}$ ) and the coefficients survive  $\text{star}$  ( $\text{conj\_contrComm}$ ), which together make conjugation a contraction.  $\text{conjT}$  is then  $\text{conj}$ -semilinear, involutive, and commutes with raising and lowering indices. Last makes reality and Hermiticity compatible with raising and lowering indices.

At the single-index level,  $\text{conjEquiv} : V c \approx_{s1} V (\bar{c})$  realises this conjugation on a vector's coordinates, conjugates them with  $\text{ConjModule.starFinsupp}$ , and re-seats them at the

conjugate colour. It rests on the conjugate module  $\text{ConjModule}$  ( $\text{Physlib.Maths}$ ). The same vectors with the scalar action twisted by conjugation ( $\text{i}$  acts as  $\bar{\phantom{x}}$ ) on conjugate colours with such conjugate-module carriers is what makes a metric

genuinely bilinear and ``IsHermitian`` an honest conjugate-transpose.

## ## ii. Key results

- ``ConjTensorSpecies`` is a tensor species bundled with its conjugation.
- ``ConjTensorSpecies.conjT`` is the conjugation of a tensor.
- ``ConjTensorSpecies.conjT_conjT`` proves that conjugation is an involution.
- ``ConjTensorSpecies.conjT_contrT`` proves that conjugation commutes with contraction
- ``ConjTensorSpecies.conjT_eq_permT_iff`` is the componentwise criterion for ``conjT t = permT σ h t``, the workhorse for proving reality and Hermiticity conditions
- ``ConjTensorSpecies.conjEquiv`` is the single-slot conjugate-linear isomorphism ``V c ≈sl V (bar c)``.
- ``ConjTensorSpecies.IsHermitian`` is the structural conjugate-transpose condition on a metric slot.

## ## iii. Table of contents

- A. The conjugation structure
- B. Conjugation of tensors
- C. The involution law
- D. Commutation with contraction
- E. The slot conjugation
- F. Hermitian pairings

-/

@[expose] public section  
noncomputable section

open Module TensorSpecies TensorSpecies.Tensor

/-!

## ## A. The conjugation structure

A ``ConjTensorSpecies`` bundles a ``TensorSpecies`` (``extends``) with the conjugation data. Conjugation is defined at the basis level, where it is "``star`` the components" (§B). new fields are bookkeeping on colours and index sets, trivial to supply per instance: ``bar`` (the colour involution), ``bar_involution``, ``bar_tau`` (it commutes with ``τ``), and ``barIdx_eq`` (a conjugate share basis labels). The one substantive field is ``conj_contrComm``, that tensor coefficients are unchanged by ``star``; this is what makes conjugation commute with contraction and for a real (δ) contraction it is ``star δ = δ``.

```

-/

/-- A tensor species bundled with a conjugation. Beyond the `TensorSpecies`
the conjugate-colour involution `bar`, the index-set identification `barIdx`
contraction-coherence `conj_contrComm`. Also carries `bar_involution` and `
involutive and commutes with the variance dual). -/
structure ConjTensorSpecies (k : Type) [CommRing k] [StarRing k] (C : Type)
 (V : C → Type) [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 (basisIdx : C → Type) [V c, Fintype (basisIdx c)] [V c, DecidableEq (ba
 (rep : (c : C) → Representation k G (V c)) (b : (c : C) → Basis (basisI
 extends TensorSpecies k C G V basisIdx rep b where
/-- The conjugate colour: the colour an index transforms in under complex
variance (commutes with `τ`). -/
bar : C → C
/-- `bar` is an involution. -/
bar_involution : Function.Involutive bar
/-- `bar` commutes with the variance dual `τ`. -/
bar_tau : V c, bar (toTensorSpecies.τ c) = toTensorSpecies.τ (bar c)
/-- Conjugation fixes the index set: the conjugate colour reuses the same
Conjugation conjugates components in an adapted basis; it never permutes
sets of `c` and `bar c` coincide. -/
barIdx_eq : V c, basisIdx (bar c) = basisIdx c
/-- Conjugation is compatible with contraction at the basis level: `star`
coefficient at colour `d` equals the coefficient at the conjugate colour
labels carried over by `barIdx_eq`. For a real (δ) contraction this is `s
/
conj_contrComm : V (d : C) (x1 : basisIdx d) (x2 : basisIdx (toTensorSpec
 star (toTensorSpecies.contr d (b d x1 ⊗+[k] b (toTensorSpecies.τ d) x
 = toTensorSpecies.contr (bar d) (b (bar d) ((Equiv.cast (barIdx_eq
 b (toTensorSpecies.τ (bar d)) (basisIdxCongr (bar_tau d)
 ((Equiv.cast (barIdx_eq (toTensorSpecies.τ d))).symm x2)))

namespace ConjTensorSpecies

open Tensor

variable {k : Type} [CommRing k] [StarRing k] {C : Type} {G : Type} [Group
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (ba
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisI
 (S : ConjTensorSpecies k C G V basisIdx rep b)

/-!

B. Conjugation of tensors

```

We define the conjugation map ``conjT`` through its action on components, record that components in place (``componentMap_conjT``), and show it is ``conj``-semilinear and additive.

```

-/

/-- Reindex component labels of `bar ∘ c` back to `c`, slotwise via the cast `barIdx`
/
def componentReindex {n : ℕ} (c : Fin n → C) :
 ComponentIdx (S := S.toTensorSpecies) (S.bar ∘ c) ≈ ComponentIdx (S := S.toTensorSpecies) (S.bar)
 Equiv.piCongrRight fun i => Equiv.cast (S.barIdx_eq (c i))

/-- Conjugation of a tensor: conjugate the components and reindex the basis to the c
colours. `conj`-semilinear by construction (see `conjT_smul`). -
/
def conjT {n : ℕ} {c : Fin n → C} (t : S.Tensor c) : S.Tensor (S.bar ∘ c) :=
 ofComponents (S.bar ∘ c)
 (fun b => star (componentMap c t (S.componentReindex c b)))

/-- Components of a conjugated tensor: the `star` of the original components, reindex
/
@[simp] lemma componentMap_conjT {n : ℕ} {c : Fin n → C} (t : S.Tensor c)
 (b : ComponentIdx (S := S.toTensorSpecies) (S.bar ∘ c)) :
 componentMap (S.bar ∘ c) (S.conjT t) b
 = star (componentMap c t (S.componentReindex c b)) := by
 simp only [conjT, componentMap_ofComponents]

/-- Conjugation is semilinear: scalar multiplication pulls out as `star r`. -
/
@[simp] lemma conjT_smul {n : ℕ} {c : Fin n → C} (r : k) (t : S.Tensor c) :
 S.conjT (r • t) = star r • S.conjT t := by
 apply componentMap_ext
 intro b
 simp only [componentMap_conjT, map_smul, Pi.smul_apply, smul_eq_mul, star_mul']

/-- Conjugation is additive. -/
@[simp] lemma conjT_add {n : ℕ} {c : Fin n → C} (t₁ t₂ : S.Tensor c) :
 S.conjT (t₁ + t₂) = S.conjT t₁ + S.conjT t₂ := by
 apply componentMap_ext
 intro b
 simp only [componentMap_conjT, map_add, Pi.add_apply, star_add]

/-- Componentwise criterion for `conjT t = permT σ h t`. The conjugate of `t` equal
recolouring `permT σ h t` exactly when, at every component, the `star`-
conjugated reindexed
component of `t` matches the corresponding component of `permT σ h t`. This package

```

`componentMap\_conjT` expansion and the `repr`/`componentMap` bridge that the proofs downstream would otherwise repeat by hand; the caller is left only with specific

```
permutation bookkeeping on the right-hand side. -/
lemma conjT_eq_permT_iff {n m : ℕ} {c : Fin n → C} {c' : Fin m → C}
 {σ : Fin n → Fin m} (h : IsReindexing c' (S.bar ∘ c) σ)
 (t : S.Tensor c) (t' : S.Tensor c') :
 S.conjT t = permT σ h t' ↔
 ∀ φ : ComponentIdx (S := S.toTensorSpecies) (S.bar ∘ c),
 star (componentMap c t (S.componentReindex c φ))
 = (Tensor.basis (S.bar ∘ c)).repr (permT σ h t') φ := by
 constructor
 · intro H φ
 rw [← H, ← componentMap_eq_repr, componentMap_conjT]
 · intro H
 apply componentMap_ext
 intro φ
 rw [componentMap_conjT]
 exact H φ
```

/-!

## C. The involution law

We prove `conjT\_conjT`: conjugating a tensor twice returns it, up to the identity  $\text{bar} \circ \text{bar} \circ c = c$ . The supporting lemmas reconcile the iterated basis-label casts.

-/

/-- Undoing the two cast reindexings (at `bar c` then at `c`) on a label of conjugated

colour returns the `bar\_involution` cast. Free from `barIdx\_eq` by `cast\_cast` irrelevance, with no coherence field needed. -/

```
private lemma barIdx_involutive_symm (c : C) (y : basisIdx (S.bar (S.bar c)))
 Equiv.cast (S.barIdx_eq c) (Equiv.cast (S.barIdx_eq (S.bar c)) y)
 = basisIdxCongr (S.bar_involution c) y := by
 simp only [basisIdxCongr, Equiv.cast_apply, cast_cast]
```

/-- The identity permutation satisfies `IsReindexing` from `c` to `bar ∘ bar ∘ c` involution. -/

```
lemma isReindexing_bar_bar {n : ℕ} (c : Fin n → C) :
 IsReindexing c (S.bar ∘ S.bar ∘ c) (id : Fin n → Fin n) :=
 (Function.bijective_id, fun i => (S.bar_involution (c i)).symm)
```

/-- Conjugation is an involution: conjugating twice returns the original tensor

```

`bar_involution` recolouring (the identity permutation `permT`). -
/
lemma conjT_conjT {n : ℕ} {c : Fin n → C} (t : S.Tensor c) :
 S.conjT (S.conjT t)
 = permT id (S.isReindexing_bar_bar c) t := by
 apply componentMap_ext
 intro φ
 rw [componentMap_conjT, componentMap_conjT, star_star]
 rw [componentMap_eq_repr (S.bar ∘ S.bar ∘ c), permT_basis_repr_symm_apply,
 ← componentMap_eq_repr c]
 refine congrArg (fun ψ => (componentMap c) t ψ) ?_
 funext i
 have hinv : (IsReindexing.inv id (S.isReindexing_bar_bar c)) i = i :=
 IsReindexing.inv_apply_apply id _ i
 show Equiv.cast (S.barIdx_eq (c i)) (Equiv.cast (S.barIdx_eq (S.bar (c i))) (φ i))
 rw [S.barIdx_involutive_symm]
 exact basisIdxCongr_heq_arg _ _ (by rw [hinv]; exact HEq.rfl)

/-!

D. Commutation with contraction

We prove `conjT_contrT`: conjugation commutes with contracting two dual-
coloured slots. The
contraction expands as a sum over the contracted index pair, and `conj_contrComm` ma
conjugated coefficients to those on the `bar`-images.

-/

/-- Slots with dual colour in `c` have dual colour in `bar ∘ c`, as `bar` commutes w
/
lemma contrCond_bar {n : ℕ} {c : Fin (n + 1 + 1) → C} {i j : Fin (n + 1 + 1)}
 (h : i ≠ j ∧ S.τ (c i) = c j) :
 i ≠ j ∧ S.τ ((S.bar ∘ c) i) = (S.bar ∘ c) j :=
 ⟨h.1, by rw [Function.comp_apply, Function.comp_apply, ← S.bar_tau, h.2]⟩

/-- Conjugation commutes with contraction: conjugating a contracted tensor equals co
conjugate on the `bar`-images of the same two slots. -/
lemma conjT_contrT {n : ℕ} {c : Fin (n + 1 + 1) → C} (i j : Fin (n + 1 + 1))
 (h : i ≠ j ∧ S.τ (c i) = c j) (t : S.Tensor c) :
 S.conjT (contrT n i j h t)
 = contrT n i j (S.contrCond_bar h) (S.conjT t) := by
 apply componentMap_ext
 intro φ
 rw [componentMap_conjT, componentMap_eq_repr, Tensor.contrT_basis_repr_apply]
 have hrhs := Tensor.contrT_basis_repr_apply (S := S.toTensorSpecies) (c := S.bar ∘

```

```

 (i := i) (j := j) (S.contrCond_bar h) (S.conjT t) $\varnothing$
 rw [componentMap_eq_repr]
 refine Eq.trans ?_ hrhs.symm
 rw [star_sum]
 -- `componentReindex` carries the `bar`-side contraction section onto the
side one: it
 -- commutes with `dropPair` definitionally, so the section bijection is j
 have hmem : $\forall a : \text{ComponentIdx } (S := S.\text{toTensorSpecies}) (S.\text{bar} \circ c),$
 a $\in$ ComponentIdx.DropPairSection $\varnothing \leftrightarrow$
 S.componentReindex c a $\in$
 ComponentIdx.DropPairSection (S.componentReindex (c $\circ$ i.succSuccAbove j))
 intro a
 simp only [ComponentIdx.DropPairSection, Finset.mem_filter, Finset.mem_exact]
 exact (Equiv.apply_eq_iff_eq (S.componentReindex (c $\circ$ i.succSuccAbove j)))
 refine (Finset.sum_equiv (Equiv.subtypeEquiv (S.componentReindex c) hmem))
 (fun _ => by simp) ?_.symm
 intro b''
 simp only [Equiv.subtypeEquiv_apply]
 rw [← componentMap_eq_repr (S.bar $\circ$ c), componentMap_conjT, componentMap_star_mul']
 congr 1
 rw [S.conj_contrComm (c i) ((S.componentReindex c b''.1) i)
 (basisIdxCongr (by rw [h.2]) ((S.componentReindex c b''.1) j)),
 show (S.componentReindex c b''.1) i = Equiv.cast (S.barIdx_eq (c i)) (b i),
 show (S.componentReindex c b''.1) j = Equiv.cast (S.barIdx_eq (c j)) (b j),
 Equiv.symm_apply_apply]
 congr 2
 exact congrArg (b (S. τ (S.bar (c i)))) (basisIdxCongr_heq_arg _ _
 (HEq.symm ((cast_heq _ _).trans ((cast_heq _ _).trans (cast_heq _ _)))))
/-!

E. The slot conjugation

`conjEquiv` is the conjugate-linear isomorphism $\forall c \approx_{sl} [\text{starRingEnd } k] V$ (
coordinates of a vector in the species basis, conjugate them (`star`), and
seat them as the
coordinates at the conjugate colour (the index sets agree by `barIdx_eq`).
slot
shadow of `conjT`, packaged as a bundled equivalence so the Hermitian-
metric layer can apply it to a
metric slot. Semilinearity is built in via the coordinate `star`; invertibi
involutivity together with `bar`'s.

-/

```



```

/-- The conjugate-linear slot isomorphism $V\ c \approx_{sl} [\text{starRingEnd } k] V (\text{bar } c)$: conjugate
coordinates in the species basis and relabel to the conjugate colour via barIdx_eq
/
def conjEquiv {c : C} : V c $\approx_{sl}$ [starRingEnd k] V (S.bar c) :=
 ((b c).repr.trans (ConjModule.starFinsupp (k := k))).trans
 ((Finsupp.domLCongr (Equiv.cast (S.barIdx_eq c).symm)).trans (b (S.bar c)).repr.

/-- conjEquiv on a basis vector: $b\ c\ i \mapsto b\ (\text{bar } c)\ i$ (relabelled through barIdx_eq
basis coordinates of $b\ c\ i$ are the indicator at i and star fixes 0 and 1 .
/
@[simp] lemma conjEquiv_basis {c : C} (i : basisIdx c) :
 S.conjEquiv (b c i) = b (S.bar c) (Equiv.cast (S.barIdx_eq c).symm i) := by
 simp [conjEquiv, ConjModule.starFinsupp, Finsupp.domLCongr_apply, Finsupp.mapRange

/-!

F. Hermitian pairings

A metric slot pairs a colour c with its conjugate $\text{bar } c$. IsHermitian is the st
 $g_{\{I\bar{J}\}} = \text{conj } g_{\{J\bar{I}\}}$: conjugating and swapping the two slots through conjEquiv
 star . Because $V\ c$ and $V\ (\text{bar } c)$ are genuinely different modules this is the ho
conjugate-transpose, not a bare $g = g.\text{flip}$. The condition is fixed here as IsHerm
downstream metric layer instantiates it for a concrete pairing.

-/

open scoped TensorProduct in
/-- A pairing $g : V\ c \otimes V\ (\text{bar } c) \rightarrow k$ is Hermitian when conjugating and swapping i
via conjEquiv returns its star : $g\ (x \otimes y) = \text{star}\ (g\ (\text{conjEquiv}.\text{symm } y \otimes \text{conjEquiv}.\text{symm } x))$
/
def IsHermitian {c : C} (g : V c $\otimes$ [k] V (S.bar c) $\rightarrow$ [k] k) : Prop :=
 $\forall$ (x : V c) (y : V (S.bar c)),
 g (x $\otimes$ [k] y) = star (g (S.conjEquiv.symm y $\otimes$ [k] S.conjEquiv x))

end ConjTensorSpecies
end
end

```

## 1.222 Physlib/Relativity/Tensors/Constructors.lean

```

open Module
variable {k : Type} [RCLike k] {C : Type} {G : Type} [Group G]
 {V : C $\rightarrow$ Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {basisIdx : C $\rightarrow$ Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]

```

```

 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisI
 {S : TensorSpecies k C G V basisIdx rep b}
attribute [-simp] LinearEquiv.cast_apply
/-- The equivalence between `V c` and `Pure S ![c]`. -/
noncomputable def Pure.fromSingleP {c : C} : V c →1[k] Pure S ![c] where
/-- The equivalence between `V c` and `S.Tensor ![c]`. -/
noncomputable def fromSingleT {c : C} : V c →1[k] S.Tensor ![c] :=
 (PiTensorProduct.subsingletonEquiv (s := fun i => V (![c] i)) 0).symm
 fromSingleT.symm p.toTensor = Pure.fromSingleP.symm p :=
 PiTensorProduct.subsingletonEquiv_apply_tprod 0 p

lemma fromSingleT_eq_pureT {c : C} (x : V c) :
 fromSingleT (S := S) x = Pure.toTensor (fun 0 => x : Pure S ![c]) := by
 obtain ⟨p, rfl⟩ := Pure.fromSingleP (S := S).symm.surjective x
lemma actionT_fromSingleT {c : C} (x : V c) (g : G) :
 g • fromSingleT (S := S) x = fromSingleT (rep c g x) := by
 (h : c = c1) (x : V c) :
 fromSingleT (LinearEquiv.cast (R := k) h x) =
 permT id (by simp [h]) (fromSingleT (S := S) x) := by
lemma contrT_fromSingleT_fromSingleT {c : C} (x : V c)
 (y : V (S.τ c)) :
 contrT (S := S) 0 0 1 (by simp; rfl) (prodT (fromSingleT x) (fromSingleT
 (S.contr c) (x ⊗t[k] y) • (Pure.toTensor default) := by
 rw [fromSingleT_eq_pureT, fromSingleT_eq_pureT, prodT_pure, contrT_pure,
 Pure.contrPCoeff]
 simp only [Nat.reduceAdd, Nat.succ_eq_add_one, Fin.isValue]
 congr
 ext i
 fin_cases i

`V c1 ⊗t[k] V c2 ` defined
 V c1 ⊗t[k] V c2 →1[k] S.Tensor ![c1, c2] where
 map_add' x y := by simp
 map_smul' r x := by simp
lemma fromPairT_tmul {c1 c2 : C} (x : V c1)
 (y : V c2) : fromPairT (x ⊗t[k] y) =
 (prodT (fromSingleT (S := S) x) (fromSingleT y)) := by
lemma fromPairT_eq_pure {c1 c2 : C} (x : V c1) (y : V c2) :
 fromPairT (S := S) (x ⊗t[k] y) = Pure.toTensor (fun | 0 => x | 1 => y)
 rw [fromPairT_tmul, fromSingleT_eq_pureT, fromSingleT_eq_pureT, prodT_pur
 congr
 funext i
 fin_cases i
 · rfl
 · rfl

```

```

 (x : V c1 ⊗[k] V c2)
 g • fromPairT (S := S) x = fromPairT (TensorProduct.map (rep c1 g)
 (rep c2 g) x) := by
let P (x : V c1 ⊗[k] V c2) : Prop :=
 g • fromPairT (S := S) x = fromPairT (TensorProduct.map (rep c1 g)
 (rep c2 g) x)
· simp [P]
 simp [P, hx, hy]
 (x : V c1 ⊗[k] V c2) :
 (LinearEquiv.cast (R := k) (M := V) h) x : _ ⊗[k] V c2' =
 permT id (by simp [h]) (fromPairT (S := S) x) := by
induction' x using TensorProduct.induction_on with x y x1 x2 h1 h2
· simp
· simp only [Nat.succ_eq_add_one, Nat.reduceAdd, map_tmul, LinearMap.id_coe, id_eq,
 LinearEquiv.coe_coe]
 rw [fromPairT_tmul, fromPairT_tmul]
 simp only [Nat.succ_eq_add_one, Nat.reduceAdd, permT_permT, CompTriple.comp_eq]
 rw [fromSingleT_map, prodT_permT_right, permT_permT]
 simp only [Nat.succ_eq_add_one, Nat.reduceAdd, CompTriple.comp_eq]
 congr
 exact List.ofFn_inj.mp rfl
· simp [h1, h2]
 (x : V c1 ⊗[k] V c2) :
 (fromPairT (S := S) x) := by
let P (x : V c1 ⊗[k] V c2) : Prop :=
 (fromPairT (S := S) x)
 (x : V c) (y : V (S.τ c) ⊗[k] V c2) : S.Tensor ![c2] :=
let T1 : V c ⊗[k] (V (S.τ c) ⊗[k] (V c2)) := x ⊗t[k] y
let T3 : (V c ⊗[k] V (S.τ c)) ⊗[k] V c2 := (TensorProduct.assoc _ _ _).symm T1
let T4 : k ⊗[k] V c2 := (S.contr c).toLinearMap.rTensor (V c2) T3
let T5 : V c2 := TensorProduct.lid _ _ T4
 (x : V c) (y1 : V (S.τ c)) (y2 : V c2) :
 S.contr c (x ⊗t[k] y1) • fromSingleT y2 := by
 change (S.contr c) (x ⊗t[k] y1) ⊗t[k] y2
 change (S.contr c) (x ⊗t[k] y1) • y2
 (x : V c) (y1 : V (S.τ c)) (y2 : V c2) :
 simp only [Nat.reduceAdd, Nat.succ_eq_add_one, Fin.isValue, map_smul]
 (x : V c)
 (y : V (S.τ c) ⊗[k] V c2) :
let P (y : V (S.τ c) ⊗[k] V c2) : Prop :=
noncomputable def fromPairTContr {c c1 c2 : C}(x : V c1 ⊗[k] V c) (y : V (S.τ c) ⊗[k]
 let V1 := (V c1)
 let V2 := (V c)
 let V2' := (V (S.τ c))
 let V3 := V c2
 let T1 : (V c1 ⊗[k] V c) ⊗[k] (V (S.τ c) ⊗[k] V c2) := x ⊗t[k] y

```

```

let T2 : V1 ⊗[k] (V2 ⊗[k] (V2' ⊗[k] V3)) := TensorProduct.assoc _ _ _ T
let T3 : V1 ⊗[k] ((V2 ⊗[k] V2') ⊗[k] V3) := (TensorProduct.assoc _ _ _ T)
let T4 : V1 ⊗[k] (k ⊗[k] V3) := ((S.contr c).toLinearMap.rTensor _).lTensor T4
let T5 : V1 ⊗[k] V3 := (TensorProduct.lid _).lTensor _ T4
lemma fromPairTContr_tmul_tmul {c c1 c2 : C} (x1 : V c1) (x2 : V c) (y1 : V
 (y2 : V c2) : fromPairTContr (x1 ⊗t[k] x2) (y1 ⊗t[k] y2) =
 (S.contr c) (x2 ⊗t[k] y1) • fromPairT (x1 ⊗t[k] y2) := by
 change x1 ⊗t[k] ((S.contr c) (x2 ⊗t[k] y1) ⊗t[k] y2)
 change x1 ⊗t[k] (((S.contr c) (x2 ⊗t[k] y1) : k) • y2)
 (x1 : V c1) (x2 : V c) (y1 : V (S.τ c))
 (y2 : V c2) :
 contrT 2 1 2 (by simp; rfl) (prodT (fromPairT (x1 ⊗t[k] x2)) (fromPairT
 simp only [map_smul, fromPairTContr_tmul_tmul, fromPairT_eq_pure, permT_p
 Pure.contrP, contrT_pure, prodT_pure]
 congr
 funext i
 fin_cases i
 · rfl
 · rfl

 (x : V c1 ⊗[k] V c)
 (y : V (S.τ c) ⊗[k] V c2) :
 let P (x : V c1 ⊗[k] V c)
 (y : V (S.τ c) ⊗[k] V c2) : Prop :=
 let P1 (x : V c1 ⊗[k] V c) := P x y
 let P2 (y : V (S.τ c) ⊗[k] V c2) : Prop :=
 (x : V c ⊗[k] V c1)
 (φ : ComponentIdx ![c, c1]) :
 (basis ![c, c1]).repr (fromPairT (S := S) x) φ =
 (Basis.tensorProduct (b c) (b c1)).repr x (φ 0, φ 1) := by
 let P (x : (V c ⊗[k] V c1)) :=
 (fromPairT (S := S) x) φ =
 (Basis.tensorProduct (b c) (b c1)).repr x (φ 0, φ 1)
 rfl
 fromPairT (S := S) (b c b0 ⊗t[k] b c1 b1) =
 (v : (Representation.trivial k G k).IntertwiningMap ((rep c1).tprod (
 S.Tensor ![c1, c2] := fromPairT (v (1 : k))
 (v : (Representation.trivial k G k).IntertwiningMap ((rep c1).tprod (re
 (g : G) : g • fromConstPair (S := S) v = fromConstPair v := by
 exact congrArg _ (LinearMap.congr_fun (v.isIntertwining' g) 1).symm
 `V c1 ⊗[k] V c2` defined
 V c1 ⊗[k] (V c2 ⊗[k] V c3) →1[k] S.Tensor ![c1, c2, c3] where
 lemma fromTripleT_tmul {c1 c2 c3 : C} (x : V c1)
 (y : V c2) (z : V c3) :
 (prodT (fromSingleT (S := S) x) (prodT (fromSingleT y) (fromSingleT z
 (x : V c1 ⊗[k] (V c2 ⊗[k] V c3)) (g : G) :

```

```

g • fromTripleT (S := S) x = fromTripleT (TensorProduct.map (rep c1 g)
 (TensorProduct.map (rep c2 g) (rep c3 g)) x) := by
let P (x : V c1 ⊗[k] (V c2 ⊗[k] V c3)) : Prop :=
 g • fromTripleT (S := S) x = fromTripleT (TensorProduct.map (rep c1 g)
 (TensorProduct.map (rep c2 g) (rep c3 g)) x)
· simp [P]
let P1 (y : V c2 ⊗[k] V c3) : Prop :=
· simp [P1, P]
 rw [← permT_equivariant, ← prodT_equivariant, ← prodT_equivariant]
 simp [← actionT_fromSingleT]
 simp [P1, P, hx, hy, tmul_add]
simp [P, hx, hy]
(x : V c ⊗[k] (V c1 ⊗[k] V c2))
(φ : ComponentIdx ![c, c1, c2]) :
(basis ![c, c1, c2]).repr (fromTripleT (S := S) x) φ =
(Basis.tensorProduct (b c) (Basis.tensorProduct (b c1) (b c2))).repr x
(φ 0, φ 1, φ 2) := by
let P (x : V c ⊗[k] (V c1 ⊗[k] V c2)) := (basis ![c, c1, c2]).repr (fromTripleT x)
 (Basis.tensorProduct (b c) (Basis.tensorProduct (b c1) (b c2))).repr x
 (φ 0, φ 1, φ 2)
let P1 (y : V c1 ⊗[k] V c2) : Prop :=
 rfl
fromTripleT (S := S) (b c b0 ⊗[k] (b c1 b1 ⊗[k] b c2 b2)) =
(v : (Representation.trivial k G k).IntertwiningMap
 ((rep c1).tprod ((rep c2).tprod (rep c3)))) :
S.Tensor ![c1, c2, c3] := fromTripleT (v (1 : k))
(v : (Representation.trivial k G k).IntertwiningMap ((rep c1).tprod ((rep c2).tp
 (g : G) : g • fromConstTriple (S := S) v = fromConstTriple v := by
exact congrArg _ (LinearMap.congr_fun (v.isIntertwining' g) 1).symm

```

## 1.223 Physlib/Relativity/Tensors/Contraction/Basic.lean

## i. Overview

This file defines tensor contraction for an arbitrary `TensorSpecies`.

The colors of a tensor record the type of each index. A pair of positions can be contracted when their colors are dual, expressed by a proof

`S.τ (c i) = c j`, where `S.τ` is the color involution in the tensor species. For a tensor with colors `c : Fin (n + 1 + 1) → C`, contraction at positions `i` and `j` produces a tensor with two fewer indices and colors `c ∘ Fin.succSuccAbove i j`.

The construction is first made on pure tensors in

``Physlib.Relativity.Tensors.Contraction.Pure``. There, ``Pure.dropPair`` removes two contracted factors from the remaining pure tensor, ``Pure.contrPCoeff`` computes the scalar contraction of the two removed factors using ``S.contr``, ``Pure.contrP`` multiplies that scalar by the dropped tensor. This file extends the operation linearly to arbitrary tensors via ``contrT``.

## ## ii. Key results

- ``TensorSpecies.Tensor.contrT`` is the linear contraction map on tensors.
- ``TensorSpecies.Tensor.contrT_pure`` identifies ``contrT`` on pure tensors with ``Pure.contrP``.
- ``TensorSpecies.Tensor.contrT_equivariant`` says contraction commutes with symmetry-group action.
- ``TensorSpecies.Tensor.contrT_permT``, ``TensorSpecies.Tensor.contrT_symm``, ``TensorSpecies.Tensor.contrT_comm`` describe the interaction of contraction with reindexing, swapping the contracted pair, and performing two contractions.

## ## iii. Related files

- ``Physlib.Relativity.Tensors.Contraction.SuccSuccAbove`` contains the finite index maps used to drop two positions.
- ``Physlib.Relativity.Tensors.Contraction.Pure`` defines ``Pure.dropPair``, ``Pure.contrPCoeff``, ``Pure.contrP``, and the multilinear map used to define ``contrT``.
- ``Physlib.Relativity.Tensors.Contraction.Products`` proves compatibility between contractions and tensor products.

## ## iv. Table of contents

- A. Tensor contraction

## ## v. References

There are no known references for the material in this module.

open Module

```
variable {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisIdx c) (V c)}
 {S : TensorSpecies k C G V basisIdx rep b}
```

## ## A. Tensor contraction

```
simp [P, hp, hr]
```

```
(hσ : IsReindexing c c1 σ) (t : Tensor S c) :
```

```
contrT n1 i j hij (permT σ hσ t) = permT _ (hσ.succSuccAbove i j hij.1)
```

```

permT _ (hσ.succSuccAbove i j hij.1)
permT id (IsReindexing.succSuccAbove_comm i1 j1 i2 j2 hij1.left hij2.left)
permT id (IsReindexing.succSuccAbove_comm i1 j1 i2 j2 hij1.left hij2.left)

```

## 1.224 Physlib/Relativity/Tensors/Contraction/Basis.lean

```

public import Physlib.Relativity.Tensors.ComponentIdx.Contraction
open Module
variable {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
{V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
{basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
{rep : (c : C) → Representation k G (V c)} {b : (c : C) → Module.Basis (basisIdx c) (V c)}
{S : TensorSpecies k C G V basisIdx rep b}
attribute [-simp] LinearEquiv.cast_apply
(φ : ComponentIdx (c ◦ Fin.succSuccAbove i j)) :
(basis (c ◦ Fin.succSuccAbove i j)).repr (contrT n i j h t) φ =
∑ (b' : DropPairSection φ), (basis c).repr t b'.1 *
(S.contr (c i) (b (c i) (b'.1 i) ⊗t[k] b (S.τ (c i))
(basisIdxCongr (by rw [h.2]) (b'.1 j)))) := by
change if b'.dropPair i j = φ then _ else 0
split_ifs with hd
· subst hd
· simp [Pure.contrPCoeff, Pure.basisVector]
congr 2
generalize_proofs h1 h2
generalize b' j = bj
generalize c j = cj at *
subst h2
rfl
rw [Basis.repr_self, Finsupp.single_eq_of_ne fun h => hb (Subtype.ext h), zero_mul]
intro b''
rw [Basis.repr_self, Finsupp.single_eq_of_ne ?_, zero_mul]
exact fun h => hd (h ▸ (Finset.mem_filter.mp b''.2).2)
simp only [map_smul, Finsupp.coe_smul, Pi.smul_apply, smul_eq_mul, h1, Finset.mul_assoc]
simp only [map_add, Finsupp.coe_add, Pi.add_apply, h1, h2, add_mul, ← Finset.sum]
(φ : ComponentIdx (c ◦ Fin.succSuccAbove i j)) :
(basis (c ◦ Fin.succSuccAbove i j)).repr (contrT n i j h t) φ =
(basis c).repr t (DropPairSection.ofFinEquiv h.1 φ (x1, x2)).1 *
((S.contr (c i)
(b (c i) x1 ⊗t[k] b (S.τ (c i)) (basisIdxCongr (by rw [h.2]) x2)))) := by
rw [contrT_basis_repr_apply h t φ, ← (DropPairSection.ofFinEquiv h.1 φ).sum_comp,
(h : i ≠ j ∧ S.τ (c i) = c j) (b : ComponentIdx (S := S) c) :

```

**1.225 Physlib/Relativity/Tensors/Contraction/Products.lean**

```

variable {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (ba
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Module.Basis
 {S : TensorSpecies k C G V basisIdx rep b}
attribute [-simp] LinearEquiv.cast_apply
 (by simp_all [Fin.ext_iff]) (p1.prodP p) = contrPCoeff i j hij p := by
 simp only [contrPCoeff, prodP_apply_natAdd]
 generalize_proofs ha hb hc hd he
 generalize p i = pi at *
 generalize p j = pj at *
 generalize c i = ci at *
 generalize c j = cj at *
 subst he
 subst hb
 simp [LinearEquiv.cast_apply]
 (by simp_all [Fin.ext_iff]) (p.prodP p1) = contrPCoeff i j hij p := by
 simp only [contrPCoeff, prodP_apply_castAdd]
 generalize_proofs ha hb hc hd he
 generalize p i = pi at *
 generalize p j = pj at *
 generalize c i = ci at *
 generalize c j = cj at *
 subst he
 subst hb
 simp [LinearEquiv.cast_apply]
 p1.prodP (dropPair i j hij.1 p) = permP _ (IsReindexing.append_succSucc
 simp [LinearEquiv.cast_apply]
 simp [LinearEquiv.cast_apply]
 ((permT id (IsReindexing.append_succSuccAbove_natAdd i j)) <|
 contrP (Fin.natAdd n1 i) (Fin.natAdd n1 j) (by simp using hij) <|
 simp only [contrP, map_smul, Nat.add_eq]
 rw [prodT_pure]
 generalize_proofs ha hb hc hd
 erw [map_smul]
 congr
 erw [permT_pure]
 ((permT id (IsReindexing.append_succSuccAbove_natAdd i j)) <|
 contrT _ (Fin.natAdd n1 i) (Fin.natAdd n1 j)
 (by simp using hij) <| prodT t1 t) := by
 ((permT id (IsReindexing.append_succSuccAbove_natAdd i j)) <|
 apply induction_on_pure
 · intro p
 let P2 (t1 : Tensor S c1) := P p.toTensor t1

```



```

change P2 t1
apply induction_on_pure
· intro p1
 simp only [Nat.add_eq, finSumFinEquiv_apply_right, contrT_pure, P2, P]
 rw [Pure.prodP_contrP_snd, prodT_pure, contrT_pure]
 rfl
· intro r t h1
 simp_all only [map_smul, LinearMap.smul_apply, P2, P]
· intro t1 t2 h1 h2
 simp_all only [map_add, LinearMap.add_apply, P2, P]
· intro r t h1
 simp_all only [map_smul, P1, P]
· intro t1 t2 h1 h2
 simp_all only [map_add, P1, P]
{c1 : Fin n1 → C} (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ S.τ (c i) = c j)
(contrT _ (i.natAdd n1) (j.natAdd n1) (by simp using hij) <| prodT t1 t) =
((permT id (IsReindexing.on_id_symm (IsReindexing.append_succSuccAbove_natAdd i

```

## 1.226 Physlib/Relativity/Tensors/Contraction/Pure.lean

```

open Module
variable {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
{V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
{basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
{rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisIdx c) k (V c)}
{S : TensorSpecies k C G V basisIdx rep b}
permP id (IsReindexing.succSuccAbove_comm i1 j1 i2 j2 hij1 hij2)
(x : V (c i)) :
(x : V (c j)) :
(x : V (c (succSuccAbove i j m))) :
(σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)) (hσ : IsReindexing c c1 σ) (p : Pure S
S.contr (c i) (p i ⊗t (LinearEquiv.cast (R := k) (by simp [hij.2]) (p j))))

attribute [-simp] LinearEquiv.cast_apply
(σ : Fin n1 → Fin n) (hσ : IsReindexing c c1 σ) (p : Pure S c) :
simp only [contrPCoeff, permP]
generalize_proofs h1 h2 h3 h4 h5
generalize p (σ j) = pj at *
generalize p (σ i) = pi at *
generalize c (σ j) = cj at *
generalize c (σ i) = ci at *
subst h2
subst h4
rfl

```

```

 (p : Pure S c) (x : V (c (succSuccAbove i j m))) :
open TensorProduct
 (p : Pure S c) (x y : V (c i)) :
 simp only [contrPCoeff, update_same, add_tmul, map_add]
 repeat rw [Pure.update_diff]
 all_goals grind
 (p : Pure S c) (x y : V (c j)) :
 simp only [contrPCoeff, update_same, tmul_add, map_add]
 repeat rw [Pure.update_diff]
 all_goals grind
 (p : Pure S c) (r : k) (x : V (c i)) :
 simp only [contrPCoeff, update_same, smul_tmul]
 repeat rw [Pure.update_diff]
 simp only [tmul_smul, map_smul, smul_eq_mul]
 all_goals grind
 (p : Pure S c) (r : k) (x : V (c j)) :
 simp only [contrPCoeff, update_same, tmul_smul, map_smul]
 repeat rw [Pure.update_diff]
 simp only [smul_eq_mul]
 all_goals grind
 rw [contrPCoeff, contrPCoeff, S.contr_tmul_symm]
 generalize_proofs h1 h2 h3 h4
 generalize p j = pj at *
 generalize p i = pi at *
 generalize c j = cj at *
 generalize c i = ci at *
 subst h2
 rfl
 simp only [contrPCoeff, actionP_eq, Pure.rep_cast]
 generalize_proofs h1 h2
 generalize p j = pj at *
 generalize p i = pi at *
 generalize c i = ci at *
 generalize (LinearEquiv.cast (R := k) h2) pj = pj' at *
 trans (S.contr ci) (((rep ci).tprod (rep (S.τ ci))) g (pi ⊗+[k] pj'))
 · simp
 rw [(S.contr _).isIntertwining]
 simp
 (p : Pure S c) (x y : V (c m)) :
 (p : Pure S c) (r : k) (x : V (c m)) :
 MultilinearMap k (fun i => V (c i))

```

1.227.

PHYSLIB/RELATIVITY/TENSORS/CONTRACTION/SUCCSUCCABOVE.lean

### 1.227 Physlib/Relativity/Tensors/Contraction/SuccSuccAbove.lean

```
rcases Fin.eq_self_or_eq_succAbove i a with rfl | ⟨a, rfl⟩
 apply ((succSuccAbove_strictMono i j).range_inj (OrderEmbedding.strictMono _)).mp
set_option warning.simp.varHead false in
lemma apply_succSuccAbove_symm {c : Fin (n + 1 + 1) → C} (i j : Fin (n + 1 + 1))
```

### 1.228 Physlib/Relativity/Tensors/Dual.lean

```
variable {k : Type} [RCLike k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Module.Basis (basisIdx c) (V c)}
 {S : TensorSpecies k C G V basisIdx rep b}
attribute [-simp] LinearEquiv.cast_apply
rw [toDualMap_apply, fromDualMap_apply, prodT_permT_right, prodT_contrT_snd,
 contrT_permT, contrT_permT, contrT_comm, permT_permT, permT_permT]
rw [prodT_assoc', prodT_swap, contrT_permT, contrT_permT]
rw [contrT_congr (n := 1 + 2) (Fin.natAdd (m := 4) 1 1) (Fin.natAdd (m := 4) 1
 (by rfl) (by rfl))]
rw (transparency := .instances) [contrT_prodT_snd 1 2 (by change _ ^ S.τ (S.τ c))]
rw [contrT_dual_metricTensor_metricTensor, prodT_permT_right, prodT_swap]
simp only [CompTriple.comp_eq, permT_permT, Fin.reduceNatAdd, Fin.cast_eq_self]
rw [permT_permT]
rw (transparency := .instances) [permT_permT, contrT_permT]
τ_τ_apply, IsReindexing.on_id, Matrix.cons_val_fin_one, implies_true]
```

### 1.229 Physlib/Relativity/Tensors/Elab.lean

```
- Suppose `T` and `T'` are tensors with color `[c1, c2]`.
- Suppose `T2` is a tensor with color `[c3]`.
- If `T3` is a tensor with color `[S.τ c1, S.τ c2]`, then
- If `T4` is a tensor with color `[c2, c1]` then
/-- For syntax of the form `T` where `T` is `Tensor S c` this returns
/-- For a term of the form `T` where `T` is `Tensor S c`,
/-- Whether `T1` and `T2` elaborate to tensors of definitionally equal colour (equal
 Used to decide whether an identity reindexing between them may be dropped: the bare
 un-`permT`'d term is well typed exactly when the two colours agree by `rfl`. The
 elaborates speculatively and restores the elaborator state afterwards. -
/
def colorsDefEq (T1 T2 : Term) : TermElabM Bool := do
 let s ← saveState
 try
```

```

 let ty1 ← inferType (← elabTerm T1 none)
 let ty2 ← inferType (← elabTerm T2 none)
 let result ← Meta.isDefEq ty1 ty2
 s.restore
 return result
 catch _ =>
 s.restore
 return false

/-- Wraps `T` in the reindexing `permT ![lPerm] IsReindexing.auto`. -
/
def permWrap (lPerm : List ℕ) (T : Term) : TermElabM Term := do
 let permString := "![" ++ String.intercalate ", " (lPerm.map toString) ++
 let P ← stringToTerm permString
 return Syntax.mkApp (mkIdent ``Tensor.permT) #[P, mkIdent ``IsReindexing.

/-- The syntax for the addition of two tensor trees. The right-
hand side is reindexed by the
 permutation `lPerm` relating the two index lists; the reindexing is dropped
 permutation is the identity *and* the two colours already agree definition
 sum is well typed, i.e. the identity reindexing is provably trivial by `r`
/
def addTermMap (lPerm : List ℕ) (T1 T2 : Term) : TermElabM Term := do
 if lPerm = List.range lPerm.length then
 if ← colorsDefEq T1 T2 then
 return Syntax.mkApp (mkIdent ``HAdd.hAdd) #[T1, T2]
 return Syntax.mkApp (mkIdent ``HAdd.hAdd) #[T1, ← permWrap lPerm T2]

/-- The syntax for an equality of two tensor trees. The right-
hand side is reindexed by the
 permutation `lPerm` relating the two index lists; the reindexing is dropped
 permutation is the identity *and* the two colours already agree definition
 equality is well typed, i.e. the identity reindexing is provably trivial
/
def equalTermMap (lPerm : List ℕ) (T1 T2 : Term) : TermElabM Term := do
 if lPerm = List.range lPerm.length then
 if ← colorsDefEq T1 T2 then
 return Syntax.mkApp (mkIdent ``Eq) #[T1, T2]
 return Syntax.mkApp (mkIdent ``Eq) #[T1, ← permWrap lPerm T2]
 let lPerm ← getPermutation indicesLeft indicesRight
 let addSyntax ← addTermMap lPerm (← syntaxFull a) (← syntaxFull b)
 let lPerm ← getPermutation indicesLeft indicesRight
 let equalSyntax ← equalTermMap lPerm (← syntaxFull a) (← syntaxFull b)
 /-- Removes redundant `Tensorial.toTensor` coercions from an elaborated term
 every subterm `Tensorial.toTensor t` whose `Tensorial` instance is the canonical
 on `S.Tensor c` (`Tensorial.self`) is replaced by `t` itself, since there

```

the identity. Coercions coming from genuine tensorial instances (e.g. on Lorentz v) carry real information and are left untouched. -/

```
def stripToTensorSelf (e : Expr) : Expr :=
 e.replace fun s => do
 guard (s.isAppOf ``DFunLike.coe)
 guard (s.getAppNumArgs ≥ 2)
 let f := s.appFn!.appArg!
 guard (f.isAppOf ``Tensorial.toTensor)
 guard (f.appArg!.isAppOf ``Tensorial.self)
 return s.appArg!
```

expression corresponding to a tensor tree. The redundant ``Tensorial.toTensor`` coercion inserted for bare ``S.Tensor`` terms are removed via ``stripToTensorSelf``. -

```
/
 return stripToTensorSelf (← instantiateMVars tensorExpr)
```

We pin the elaboration of the ``{ ... }T`` notation for each construct it supports: bare nodes, a genuine ``Tensorial`` instance, negation, evaluation, scalar multiplication, action, products, contractions, addition and equality. Together these check that the inserts neither a redundant ``Tensorial.toTensor`` wrapper on a bare ``S.Tensor`` term nor identity ``permT`` when the two sides already align, while keeping the ``toTensor`` coercion on a genuine ``Tensorial`` instance and a genuine ``permT`` for a real reindexing.

#### section Tests

```
variable {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Module.Basis (basisIdx c) (V c)}
 {S : TensorSpecies k C G V basisIdx rep b}
 {c : Fin 2 → C} {t t' : S.Tensor c} (a : k) (g : G) (y : basisIdx (c 0))
 {c1 c2 c3 : C} {u : S.Tensor ![c1, c2]} {u' : S.Tensor ![c2, c1]}
 {w : S.Tensor ![c3]} {td : S.Tensor ![S.τ c1, S.τ c2]}
 {M : Type} [AddCommMonoid M] [Module k M] [Tensorial S c M] (m : M)

-- A bare `S.Tensor` carries no redundant `Tensorial.toTensor` wrapper.
/-- info: t : S.Tensor c -/
#guard_msgs in
#check {t | α β}T

-- A genuine `Tensorial` instance (here the abstract `m : M`) keeps its `toTensor` coercion.
/-- info: Tensorial.toTensor m : S.Tensor c -/
#guard_msgs in
#check {m | α β}T

-- Negation of a tensor expression.
/-- info: -t : S.Tensor c -/
```

```

#guard_msgs in
#check {-(t | $\alpha \beta$)}T

-- Evaluation of an index at a basis value `[y]`.
/-- info: (evalT 0 y) t : S.Tensor (c ◦ Fin.succAbove 0) -
/
#guard_msgs in
#check {t | [y] β }T

-- Scalar multiplication ` $\cdot_t``.
/-- info: a • t : S.Tensor c -/
#guard_msgs in
#check {a •t t | $\alpha \beta$ }T

-- Group action ` $\cdot_a``.
/-- info: g • t : S.Tensor c -/
#guard_msgs in
#check {g •a t | $\alpha \beta$ }T

-- Tensor product of two tensors with no shared indices.
/-- info: (prodT u) w : S.Tensor (Fin.append ![c1, c2] ![c3]) -
/
#guard_msgs in
#check {u | $\alpha \beta \otimes w$ | γ }T

-- Contraction of both index pairs of a product.
/--
info: (contrT 0 0 1 ...) ((contrT 2 1 3 ...) ((prodT u) td)) :
 S.Tensor ((Fin.append ![c1, c2] ![S.τ c1, S.τ c2] ◦ Fin.succSuccAbove 1 3
 Fin.succSuccAbove 0 1)
-/
#guard_msgs (whitespace := lax) in
#check {u | $\alpha \beta \otimes td$ | $\alpha \beta$ }T
-- Addition with aligned indices: no redundant identity `permT` (nor `toTen
/-- info: t + t' : S.Tensor c -/
#guard_msgs in
#check ({t | $\alpha \beta + t'$ | $\alpha \beta$ }T)
-- Equality with aligned indices: no redundant identity `permT`.
/-- info: t = t' : Prop -/
#guard_msgs in
#check ({t | $\alpha \beta = t'$ | $\alpha \beta$ }T : Prop)
-- Equality with reordered indices: a genuine `permT` reindexing is inserte
/-- info: u = (permT ![1, 0] ...) u' : Prop -/
#guard_msgs in
#check ({u | $\alpha \beta = u'$ | $\beta \alpha$ }T : Prop)
end Tests$$
```

**1.230 Physlib/Relativity/Tensors/Evaluation.lean**

```

variable {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Module.Basis (basisIdx c) (V c)}
 {S : TensorSpecies k C G V basisIdx rep b}
noncomputable def evalPCoeff (i : Fin (n + 1)) (φ : basisIdx (c i)) (p : Pure S c) :
 (b (c i)).repr (p i) φ
 (φ : basisIdx (c i)) (p : Pure S c)
 (x : V (c i)) :
 evalPCoeff i φ (p.update i x) = (b (c i)).repr x φ := by
 (φ : basisIdx (c i)) (p : Pure S c)
 (x : V (c (i.succAbove j))) :
 evalPCoeff i φ (p.update (i.succAbove j) x) = evalPCoeff i φ p := by
lemma evalPCoeff_basisVector (i : Fin (n + 1)) (φ : basisIdx (c i)) (b' : ComponentI
 evalPCoeff i φ (Pure.basisVector c b') = if b' i = φ then (1 : k) else 0 := by
/-- Given a `i : Fin (n + 1)`, a `φ : basisIdx (c i)` and a pure tensor
 `p : Pure S c`, `evalP i φ p` is the tensor formed by evaluating the `i`th index
 of `p` at `φ`. -/
noncomputable def evalP (i : Fin (n + 1)) (φ : basisIdx (c i)) (p : Pure S c) :
 Tensor S (c ∘ i.succAbove) := evalPCoeff i φ p • (drop p i).toTensor
 (φ : basisIdx (c i)) (p : Pure S c)
 (x y : V (c j)) :
 evalP i φ (p.update j (x + y)) =
 evalP i φ (p.update j x) + evalP i φ (p.update j y) := by
 (φ : basisIdx (c i)) (p : Pure S c)
 (x : V (c j)) :
 evalP i φ (p.update j (r • x)) =
 r • evalP i φ (p.update j x) := by
 (i : Fin (n + 1)) (φ : basisIdx (c i)) :
 MultilinearMap k (fun i => V (c i))
toFun p := evalP i φ p
 change (update p m (x + y)).evalP i φ = _
 change (update p k (r • y)).evalP i φ = _
/-- Given a `i : Fin (n + 1)`, a `φ : Fin (S.repDim (c i))` and a tensor
 `t : Tensor S c`, `evalT i φ t` is the tensor formed by evaluating the `i`th index
 of `t` at `φ`. -/
 (φ : basisIdx (c i)) :
 PiTensorProduct.lift (Pure.evalPMultilinear i φ)
 (φ : basisIdx (c i)) (p : Pure S c) :
 evalT i φ p.toTensor = Pure.evalP i φ p := by
 change _ = Pure.evalPMultilinear i φ p

```

```

attribute [-simp] Matrix.cons_val_zero Matrix.cons_val Fin.succAbove_zero
/-- Evaluating the single-index basis tensor `basis ![c] (single.symm b)` a
yields the field element `1` if `b = x` (transported across `![c] 0 = c`)
evaluation of a one-index basis tensor is the Kronecker delta. -
/
lemma evalT_basis_single {c : C} (b : basisIdx c) (x : basisIdx (![c] 0)) :
 (evalT 0 x (basis (S := S) ![c] (ComponentIdx.single.symm b))).toField
 if basisIdxCongr (by simp) b = x then 1 else 0 := by
 rw [evalT_basis]
 simp only [ComponentIdx.single_symm_apply]
 split_ifs
 · exact toField_basis _
 · simp

/-- Basis expansion of a one-index tensor: every `t : Tensor S ![c]` is the
indices `i` of its evaluation coefficient `toField (evalT 0 i t)` times t
basis tensor. -/
lemma eq_sum_evalT_of_single_tensor_basis {c : C} (t : Tensor S ![c]) :
 t = $\sum i$, toField (evalT 0 i t) • basis ![c] (ComponentIdx.single.symm
 (basisIdxCongr (by simp) i)) := by
 induction' t using Tensor.induction_on_basis with b a t h t1 t2 h1 h2
 · obtain {i, rfl} := ComponentIdx.single.symm.surjective b
 conv_rhs => enter [2, i]; rw [evalT_basis_single]
 simp
 · simp
 · conv_lhs => rw [h]
 simp [Finset.smul_sum, smul_smul]
 · simp [add_smul, Finset.sum_add_distrib]
 grind

/-- Reconstruction of a tensor from the evaluations of its last index: ever
is the sum over basis indices `i` of the evaluation `evalT (Fin.last n) i`
the basis covector `basis ![c (Fin.last n)] (single.symm i)`, with the ap
permuted back into the last slot. -/
lemma eq_sum_evalT {n : ℕ} {c : Fin (n + 1) → C} (t : Tensor S c) :
 t = $\sum i$, permT id (IsReindexing.append_succ_last c) (prodT (evalT (Fin.
 (basis ![c (Fin.last n)] (ComponentIdx.single.symm i)))) := by
 induction' t using Tensor.induction_on_basis with b a t h t1 t2 h1 h2
 · conv_rhs => enter [2, i]; rw [evalT_basis]
 generalize_proofs h1 h2 h3
 rw [Finset.sum_eq_single (b (Fin.last n))]
 · simp only [Nat.succ_eq_add_one, Nat.reduceAdd, $\downarrow$ reduceIte]
 rw [prodT_basis, basis_apply, permT_pure]
 congr
 funext i

```



```

simp only [Pure.basisVector, Pure.permP_basisVector]
congr
refine Fin.addCases (fun j => ?_) (fun j => ?_) i
· simp only [id_eq, ComponentIdx.prod_symm_castAdd, Function.comp_apply]
 erw [basisIdxCongr_apply_apply]
 exact ComponentIdx.congr_right b _ _ (by rw [Fin.succAbove_last]; rfl)
· simp only [id_eq, ComponentIdx.prod_symm_natAdd, ComponentIdx.single_symm_ap
 basisIdxCongr_apply_apply]
 erw [basisIdxCongr_apply_apply]
 exact ComponentIdx.congr_right _ _ _ (by fin_cases j; rfl)
· intro j h1 h1
 rw [if_neg (by grind)]
 simp
· simp
· simp
· conv_lhs => rw [h]
 simp [Finset.smul_sum]
· simp [Finset.sum_add_distrib]
 grind

/-- Reconstruction of a tensor from the evaluations of its first index: every `t : T
is the sum over basis indices `i` of the basis covector `basis ![c 0] (single.symm
with the evaluation `evalT 0 i t`, with the prepended index permuted back into the
This is the first-index analogue of `eq_sum_evalT`. -/
lemma eq_sum_evalT_zero {n : ℕ} {c : Fin (n + 1) → C} (t : Tensor S c) :
 t = ∑ i, permT _ (IsReindexing.append_of_first c)
 (prodT (basis ![c 0] (ComponentIdx.single_symm i)) (evalT 0 i t)) := by
 induction' t using Tensor.induction_on_basis with b a t h t1 t2 h1 h2
· conv_rhs => enter [2, i]; rw [evalT_basis]
 generalize_proofs h1 h2 h3
 rw [Finset.sum_eq_single (b 0)]
· simp only [↓reduceIte]
 rw [prodT_basis, basis_apply, permT_pure]
 congr
 funext i
 simp only [Pure.basisVector, Pure.permP_basisVector]
 congr
 refine Fin.cases ?_ ?_ i
· simp only [ComponentIdx.prod, Equiv.coe_fn_symm_mk, Fin.cast_zero, Fin.addCa
 ComponentIdx.single_symm_apply, basisIdxCongr_apply_apply]
 exact ComponentIdx.congr_right b 0 0 rfl
· intro j
 simp only [ComponentIdx.prod, Equiv.coe_fn_symm_mk, Fin.addCases]
 rw [dif_neg (by simp)]
 simp only [eqRec_eq_cast, basisIdxCongr, Equiv.cast_apply, cast_cast]
 symm

```

```

 rw [cast_eq_iff_heq]
 congr 1
 · intro j h1 h1
 rw [if_neg (by grind)]
 simp
 · simp
 · simp
 · conv_lhs => rw [h]
 simp [Finset.smul_sum]
 · simp [Finset.sum_add_distrib]
 grind
end Tensor

```

### 1.231 Physlib/Relativity/Tensors/LeviCivita/Basic.lean

```

/-
Copyright (c) 2026 Robert Sneiderman. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Robert Sneiderman
-/
module

public import Physlib.Relativity.Tensors.RealTensor.Basic
public import Physlib.Relativity.Tensors.OfInt
public import Physlib.Mathematics.KroneckerDelta
/-!

The Levi-Civita tensor as a real Lorentz tensor

This file defines the rank-four Levi-Civita tensor $\epsilon^{uv\rho\sigma}$ as a real Lorentz
tensor of spatial dimensions, with $\epsilon^{0123} = 1$, and proves its antisymmetry under
adjacent transposition of indices.

The component on a multi-index f is the generalized Kronecker delta of order $|f|$
identity, i.e. the sign of f when f is a permutation and 0 otherwise.
components are carried by TensorSpecies.Tensor.TensorInt.toTensor.

-/

@[expose] public section

open Matrix
open MatrixGroups

```

```
open TensorProduct
noncomputable section
```

```
namespace realLorentzTensor
open TensorSpecies
open Tensor
open KroneckerDelta
```

```
/-- The Levi-Civita tensor $\epsilon^{u^v p}$ as a real Lorentz tensor in $d = 3$, with ϵ^{0123}
The component on a multi-index f is the generalized Kronecker delta of f against
identity, i.e. the sign of f when f is a permutation and 0 otherwise. -
/
```

```
noncomputable def leviCivita : $\mathbb{R}T[3, .up, .up, .up, .up]$:=
 TensorInt.toTensor (S := realLorentzTensor 3)
 (c := ![Color.up, Color.up, Color.up, Color.up]) fun f =>
 generalizedKroneckerDelta (fun i => finSumFinEquiv (f i)) (id : Fin 4 $\rightarrow$ Fin 4)
```

```
/-- The Levi-Civita tensor $\epsilon^{u^v p}$ as a real Lorentz tensor. -
/
scoped[realLorentzTensor] notation "ε4" => leviCivita
```

```
/-- The $\text{TensorInt.toTensor}$ form of the Levi-Civita tensor. -
/
```

```
lemma leviCivita_eq_ofInt : $\epsilon_4 =$
 TensorInt.toTensor (S := realLorentzTensor 3)
 (c := ![Color.up, Color.up, Color.up, Color.up]) fun f =>
 generalizedKroneckerDelta (fun i => finSumFinEquiv (f i)) (id : Fin 4 $\rightarrow$ Fin 4) :
 rfl
```

```
/-- The components of the Levi-Civita tensor in the standard basis are the generalized
Kronecker delta of the multi-index against the identity. -
/
```

```
lemma leviCivita_basis_repr_apply
 (b : ComponentIdx (S := realLorentzTensor 3) ![Color.up, Color.up, Color.up, Col
 (Tensor.basis _).repr ϵ_4 b
 = (generalizedKroneckerDelta (fun i => finSumFinEquiv (b i)) (id : Fin 4 $\rightarrow$ Fin
 rw [leviCivita_eq_ofInt, TensorInt.basis_repr_apply]
```

```
/-- The Levi-Civita tensor vanishes on any multi-index with a repeated value: if two
index positions $i \neq j$ carry the same basis index, the component is zero. -
/
```

```
lemma leviCivita_basis_repr_eq_zero_of_eq
 {b : ComponentIdx (S := realLorentzTensor 3) ![Color.up, Color.up, Color.up, Col
 {i j : Fin 4} (hij : $i \neq j$) (h : b i = b j) :
 (Tensor.basis _).repr ϵ_4 b = 0 := by
```

```

rw [leviCivita_basis_repr_apply]
have hdet : generalizedKroneckerDelta (fun i => finSumFinEquiv (b i))
 (id : Fin 4 → Fin 4) = 0 := by
 rw [show generalizedKroneckerDelta (fun i => finSumFinEquiv (b i)) (id)
 = Matrix.det (fun a c => ((kroneckerDelta (finSumFinEquiv (b a))
 from rfl)
 refine Matrix.det_zero_of_row_eq hij (funext fun c => ?_)
 rw [congrArg (↑finSumFinEquiv) h]
 rw [hdet, Int.cast_zero]

/-- The Levi-Civita tensor is antisymmetric in its first two indices
 $\{\epsilon_4 \mid \mu \nu \rho \sigma = - \epsilon_4 \mid \nu \mu \rho \sigma\}^T$. -/
lemma leviCivita_antisymm : $\{\epsilon_4 \mid \mu \nu \rho \sigma = - (\epsilon_4 \mid \nu \mu \rho \sigma)\}^T :=$ by
 apply (Tensor.basis _).repr.injective
 ext b
 rw [permT_basis_repr_symm_apply, leviCivita_eq_ofInt, TensorInt.basis_repr
 map_neg, Finsupp.neg_apply, TensorInt.basis_repr_apply, ← Int.cast_neg]
 congr 1
 rw [← generalizedKroneckerDelta_swap _ _ (Fin.zero_ne_one (n := 2))]
 congr 1
 funext i
 fin_cases i <=> rfl

/-- The Levi-Civita tensor is antisymmetric in its middle two indices
 $\{\epsilon_4 \mid \mu \nu \rho \sigma = - \epsilon_4 \mid \mu \rho \nu \sigma\}^T$. -/
lemma leviCivita_antisymm_mid : $\{\epsilon_4 \mid \mu \nu \rho \sigma = - (\epsilon_4 \mid \mu \rho \nu \sigma)\}^T :=$ by
 apply (Tensor.basis _).repr.injective
 ext b
 rw [permT_basis_repr_symm_apply, leviCivita_eq_ofInt, TensorInt.basis_repr
 map_neg, Finsupp.neg_apply, TensorInt.basis_repr_apply, ← Int.cast_neg]
 congr 1
 rw [← generalizedKroneckerDelta_swap _ _ (show (1 : Fin 4) ≠ 2 by decide)]
 congr 1
 funext i
 fin_cases i <=> rfl

/-- The Levi-Civita tensor is antisymmetric in its last two indices
 $\{\epsilon_4 \mid \mu \nu \rho \sigma = - \epsilon_4 \mid \mu \nu \sigma \rho\}^T$. -/
lemma leviCivita_antisymm_last : $\{\epsilon_4 \mid \mu \nu \rho \sigma = - (\epsilon_4 \mid \mu \nu \sigma \rho)\}^T :=$ by
 apply (Tensor.basis _).repr.injective
 ext b
 rw [permT_basis_repr_symm_apply, leviCivita_eq_ofInt, TensorInt.basis_repr
 map_neg, Finsupp.neg_apply, TensorInt.basis_repr_apply, ← Int.cast_neg]
 congr 1
 rw [← generalizedKroneckerDelta_swap _ _ (show (2 : Fin 4) ≠ 3 by decide)]
 congr 1

```

```

 funext i
 fin_cases i <;> rfl

end realLorentzTensor

```

### 1.232 Physlib/Relativity/Tensors/MetricTensor.lean

```

variable {k : Type} [RCLike k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [∀ c, AddCommGroup (V c)] [∀ c, Module k (V c)]
 {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Module.Basis (basisIdx c) k (V c)}
 {S : TensorSpecies k C G V basisIdx rep b}
attribute [-simp] LinearEquiv.cast_apply
 fromConstPair (S.metric c)
 (fromPairTContr ((S.metric c) (1 : k))
 ((S.metric ((S.τ c))) (1 : k))) = (unitTensor c) := by
change _ = fromPairT ((S.unit c) (1 : k))
 fromPairTContr ((S.metric c) (1 : k))
 ((S.metric (S.τ c)) (1 : k)) =

```

### 1.233 Physlib/Relativity/Tensors/OfInt.lean

```

open Module
variable {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [∀ c, AddCommGroup (V c)] [∀ c, Module k (V c)]
 {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisIdx c) k (V c)}
 {S : TensorSpecies k C G V basisIdx rep b}
variable {S : TensorSpecies k C G V basisIdx rep b}

```

### 1.234 Physlib/Relativity/Tensors/Product.lean

```

public import Physlib.Relativity.Tensors.ComponentIdx.Product
open Module
variable {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [∀ c, AddCommGroup (V c)] [∀ c, Module k (V c)]
 {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisIdx c) k (V c)}
 {S : TensorSpecies k C G V basisIdx rep b} {n n' n2 : ℕ} {c : Fin n → C} {c' : Fin n' → C}
 {c2 : Fin n2 → C}
Fin.addCases (fun i => LinearEquiv.cast (R := k) (by simp) (p1 i))
 (fun i => LinearEquiv.cast (R := k) (by simp) (p2 i))

```

```

@[simp]
lemma Pure.prodP_apply_castAdd {n1 n2} {c : Fin n1 → C} {c1 : Fin n2 → C}
 (p1 : Pure S c) (p2 : Pure S c1) (i : Fin n1) :
 Pure.prodP p1 p2 (Fin.castAdd n2 i) =
 LinearEquiv.cast (R := k) (by simp) (p1 i) := by
 simp [Pure.prodP]

@[simp]
lemma Pure.prodP_apply_natAdd {n1 n2} {c : Fin n1 → C} {c1 : Fin n2 → C}
 (p1 : Pure S c) (p2 : Pure S c1) (i : Fin n2) :
 Pure.prodP p1 p2 (Fin.natAdd n1 i) =
 LinearEquiv.cast (R := k) (by simp) (p2 i) := by
 simp [Pure.prodP]

 | Sum.inl i => LinearEquiv.cast (R := k) (by simp) (p1 i)
 | Sum.inr i => LinearEquiv.cast (R := k) (by simp) (p2 i) := by
 simp only [finSumFinEquiv_apply_left, LinearEquiv.cast_apply, Fin.addCa
 simp only [finSumFinEquiv_apply_right, LinearEquiv.cast_apply, Fin.addC
 (φ : ComponentIdx c) (φ1 : ComponentIdx (S := S) c1) :
 Pure.prodP (Pure.basisVector c φ) (Pure.basisVector c1 φ1) =
 Pure.basisVector _ (ComponentIdx.prod.symm (φ, φ1)) := by
 revert i
 rw [Fin.forall_fin_add]
 simp only [prodP_apply_castAdd, prodP_apply_natAdd]
 constructor
 · intro i
 symm
 simp only [basisVector, ComponentIdx.prod, Equiv.coe_fn_symm_mk, Fin.ad
 LinearEquiv.cast_apply]
 rw [basis_congr (c1 := (c i)) (by simp)]
 simp
 · intro i
 symm
 simp only [basisVector, ComponentIdx.prod, Equiv.coe_fn_symm_mk, Fin.ad
 LinearEquiv.cast_apply]
 rw [basis_congr (c1 := (c1 i)) (by simp)]
 simp
 (φ : ComponentIdx (Fin.append c c1)) :
 (p.prodP p1).component φ = p.component (ComponentIdx.prod φ).1 *
 p1.component (ComponentIdx.prod φ).2 := by
 simp [component]
 simp only [prodP_apply_castAdd, LinearEquiv.cast_apply, ComponentIdx.pr
 generalize_proofs h1 h2 h3
 generalize p x = p'
 generalize c x = c' at *
 subst h3

```

```

 rfl
 · funext x
 simp only [prodP_apply_natAdd, LinearEquiv.cast_apply, ComponentIdx.prod, Equiv.
 generalize_proofs h1 h2 h3
 generalize p1 x = p1'
 generalize c1 x = c1' at *
 subst h3
 rfl
 revert i
 rw [Fin.forall_fin_add]
 simp only [actionP_eq, prodP_apply_castAdd, prodP_apply_natAdd]
 constructor
 · intro j
 generalize_proofs h1 h2
 generalize p j = p'
 generalize c j = c' at *
 subst h2
 rfl
 · intro j
 generalize_proofs h1 h2
 generalize p1 j = p1'
 generalize c1 j = c1' at *
 subst h2
 rfl
 prodP p p0 = permP id IsReindexing.append_zero_right p := by
lemma Pure.prodP_swap {n n1} {c : Fin n → C} {c1 : Fin n1 → C} (p : Pure S c) (p1 :
 Pure.prodP p p1 = permP _ IsReindexing.append_swap (Pure.prodP p1 p) := by
 refine Fin.addCases (fun i => ?_) (fun i => ?_) i
 · have h0 : (i.castAdd n1).append (Fin.natAdd n1) (Fin.castAdd n) = i.natAdd n1 :=
 simp [permP, ← congr_right _ _ _ h0]
 · have h0 : (i.natAdd n).append (Fin.natAdd n1) (Fin.castAdd n) = i.castAdd n := b
 simp [permP, ← congr_right _ _ _ h0]
 (σ : Fin n' → Fin n) (h : IsReindexing c c' σ) (p : Pure S c) (p2 : Pure S c2) :
 Pure.prodP (permP σ h p) p2 = permP _ (h.append_congr_left c2) (Pure.prodP p p2)
 ext i
 refine Fin.addCases (fun i => ?_) (fun i => ?_) i
 · have h0 : (i.castAdd n2).append (Fin.castAdd n2 ∘ σ) (Fin.natAdd n) = (σ i).cast
 simp
 simp [permP, ← congr_right _ _ _ h0]
 · have h0 : (i.natAdd n').append (Fin.castAdd n2 ∘ σ) (Fin.natAdd n) = i.natAdd n
 simp [permP, ← congr_right _ _ _ h0]
 (σ : Fin n' → Fin n) (h : IsReindexing c c' σ) (p : Pure S c) (p2 : Pure S c2) :
 prodP p2 (permP σ h p) = permP _ (h.append_congr_right c2) (Pure.prodP p2 p) :=
 ext i
 refine Fin.addCases (fun i => ?_) (fun i => ?_) i
 · have h0 : (i.castAdd n').append (Fin.castAdd n) (Fin.natAdd n2 ∘ σ) = i.castAdd

```

```

 simp [permP, ← congr_right _ _ _ h0]
 · have h0 : (i.natAdd n2).append (Fin.castAdd n) (Fin.natAdd n2 ∘ σ) = (σ
 simp [permP, ← congr_right _ _ _ h0]
 prodP (prodP p p1) p2 =
 permP _ IsReindexing.append_assoc_right (prodP p (prodP p1 p2)) := by
 refine Fin.addCases (fun i => Fin.addCases (fun i => ?_) (fun i => ?_) i)
 · have h0 : (i.natAdd (n + n1)).cast (by grind) = (i.natAdd n1).natAdd n
 simp [permP, ← congr_right _ _ _ h0]
 · have h0 : ((i.castAdd n1).castAdd n2).cast (by grind) = i.castAdd (n1 +
 simp [permP, ← congr_right _ _ _ h0]
 · have h0 : ((i.natAdd n).castAdd n2).cast (by grind) = (i.castAdd n2).na
 simp [permP, ← congr_right _ _ _ h0]
 prodP p (prodP p1 p2) = permP _ IsReindexing.append_assoc_left (prodP (
 refine Fin.addCases (fun i => ?_) (fun i => Fin.addCases (fun i => ?_) (f
 · have h0 : (i.castAdd (n1 + n2)).cast (by grind) = (i.castAdd n1).castAd
 simp [permP, ← congr_right _ _ _ h0]
 · have h0 : ((i.castAdd n2).natAdd n).cast (by grind) = (i.natAdd n).cast
 simp [permP, ← congr_right _ _ _ h0]
 · have h0 : ((i.natAdd n1).natAdd n).cast (by grind) = i.natAdd (n + n1)
 simp [permP, ← congr_right _ _ _ h0]
B.13. Linearity of the product
attribute [-simp] LinearEquiv.cast_apply
@[simp]
lemma Pure.prodP_update_left {n1 n2} {c : Fin n1 → C} {c1 : Fin n2 → C}
 (p1 : Pure S c) (p2 : Pure S c1) (i : Fin n1) (x : V (c i)) :
 Pure.prodP (Pure.update p1 i x) p2
 = Pure.update (Pure.prodP p1 p2) (Fin.castAdd n2 i)
 (LinearEquiv.cast (R := k) (by simp) x) := by
 ext i
 revert i
 rw [Fin.forall_fin_add]
 simp only [prodP_apply_castAdd, prodP_apply_natAdd]
 constructor
 · intro j
 generalize_proofs h1 h2 h3
 by_cases h : j = i
 · subst h
 simp
 · rw [update_diff _ _ _ _ (by grind), update_diff _ _ _ _ (by simp; gri
 simp
 · intro j
 generalize_proofs h1 h2 h3
 rw [update_diff]
 · simp
 · simp [Fin.ext_iff]
 grind

```



```

@[simp]
lemma Pure.prodP_update_right {n1 n2} {c : Fin n1 → C} {c1 : Fin n2 → C}
 (p1 : Pure S c) (p2 : Pure S c1) (i : Fin n2) (x : V (c1 i)) :
 Pure.prodP p1 (Pure.update p2 i x)
 = Pure.update (Pure.prodP p1 p2) (Fin.natAdd n1 i)
 (LinearEquiv.cast (R := k) (by simp) x) := by
 ext i
 revert i
 rw [Fin.forall_fin_add]
 simp only [prodP_apply_castAdd, prodP_apply_natAdd]
 constructor
 · intro j
 generalize_proofs h1 h2 h3
 rw [update_diff]
 · simp
 · simp [Fin.ext_iff]
 grind
 · intro j
 generalize_proofs h1 h2 h3
 by_cases h : j = i
 · subst h
 simp
 · rw [update_diff _ _ _ _ (by grind), update_diff _ _ _ _ (by simp; grind)]
 simp
lemma Pure.prodP_update_add_toTensor_left {n1 n2} {c : Fin n1 → C} {c1 : Fin n2 → C}
 (p1 : Pure S c) (p2 : Pure S c1) (i : Fin n2) (x y : V (c1 i)) :
 (Pure.prodP p1 (Pure.update p2 i (x + y))).toTensor
 = (Pure.prodP p1 (Pure.update p2 i x)).toTensor
 + (Pure.prodP p1 (Pure.update p2 i y)).toTensor := by simp
/-!

C. Products of tensors

-/
open TensorProduct
 refine PiTensorProduct.lift (MultilinearMap.mk' (fun p1 => PiTensorProduct.lift
 (MultilinearMap.mk' (fun p2 => (Pure.prodP p1 p2).toTensor) ?_ ?_)) ?_ ?_)
 · intro p2 i x y
 repeat rw [← Pure.update_eq_function_update (S := S)]
 simp
 · intro p2 i r p2'
 repeat rw [← Pure.update_eq_function_update (S := S)]
 simp
 · intro p1 i x y
 ext p2
 repeat rw [← Pure.update_eq_function_update (S := S)]

```

```

 simp
 · intro p1 i r p1'
 ext p2
 repeat rw [← Pure.update_eq_function_update (S := S)]
 simp
 simp [prodT, Pure.toTensor]
lemma prodT_basis' {n1 n2} {c : Fin n1 → C} {c1 : Fin n2 → C}
 (b : ComponentIdx c) (b1 : ComponentIdx (S := S) c1) :
 (basis c b).prodT (basis c1 b1) =
 basis (Fin.append c c1) (ComponentIdx.prod.symm (b, b1)) := by
 rw [prodT_basis]
 simp [basis_apply]
set_option synthInstance.maxHeartbeats 0 in
 simp only [Basis.constr_apply_fintype, Basis.equivFun_apply, AddHom.toF
 LinearMap.coe_toAddHom, map_sum, map_smul, lift.tmul]
 conv_rhs => rw [← (basis (Fin.append c c1)).sum_repr x]
 congr
 funext φ
 congr 1
 simp only [prodT_basis, Prod.mk.eta, Equiv.symm_apply_apply]
 exact (basis_apply (Fin.append c c1) φ).symm
 permT id (IsReindexing.append_zero_right) t := by
 = permT id (IsReindexing.append_zero_right) t
lemma prodT_zero_right {n} {c : Fin n → C}
 {c1 : Fin 0 → C} (t : S.Tensor c) (t1 : S.Tensor c1) :
 prodT t t1 = (toField t1) • permT id (IsReindexing.append_zero_right) t
conv_lhs => rw [Tensor.eq_smul_toField t1]
rw [map_smul]
congr 1
convert prodT_default_right _
simp only [basis_apply]
congr
exact Unique.eq_default (Pure.basisVector c1 default)

lemma prodT_swap {n n1} {c : Fin n → C} {c1 : Fin n1 → C} (t : S.Tensor c)
 prodT t t1 = permT _ IsReindexing.append_swap (prodT t1 t) := by
 induction' t using induction_on_pure with p r t ht t1 t2 ht1 ht2
 · induction' t1 using induction_on_pure with q r t ht t1 t2 ht1 ht2
 · simp [prodT_pure, permT_pure, permT_pure, prodT_pure, Pure.prodP_swap]
 · simp [map_smul, ht]
 · simp [map_add, ht1, ht2]
 · simp [ht]
 · simp [ht1, ht2]
 (σ : Fin n' → Fin n) (h : IsReindexing c c' σ) (t : S.Tensor c) (t2 : S
 prodT (permT σ h t) t2 = permT _ (h.append_congr_left c2) (prodT t t2)
 induction' t using induction_on_pure with p r t ht t1 t2 ht1 ht2

```

```

· induction' t2 using induction_on_pure with q r t ht t1 t2 ht1 ht2
· simp only [prodT_pure, permT_pure, permT_pure, prodT_pure]
· simp only [map_smul, ht]
· simp only [map_add, ht1, ht2]
· simp [ht]
· simp [ht1, ht2]
 (σ : Fin n' → Fin n) (h : IsReindexing c c' σ) (t : S.Tensor c) (t2 : S.Tensor c)
 prodT t2 (permT σ h t) = permT _ (h.append_congr_right c2) (prodT t2 t) := by
induction' t using induction_on_pure with p r t ht t1 t2 ht1 ht2
· induction' t2 using induction_on_pure with q r t ht t1 t2 ht1 ht2
· simp only [prodT_pure, permT_pure, permT_pure, prodT_pure]
 congr
 simp
· simp [map_smul, ht]
· simp [map_add, ht1, ht2]
· simp [ht]
· simp [ht1, ht2]
 {c1 : Fin n1 → C} {c2 : Fin n2 → C} (t : S.Tensor c) (t1 : S.Tensor c1) (t2 : S.Tensor c2)
 prodT (prodT t t1) t2 = permT _ IsReindexing.append_assoc_right (prodT t (prodT t1 t2))
induction' t using induction_on_pure with p r t ht t1 t2 ht1 ht2
· induction' t1 using induction_on_pure with q r t ht t1 t2 ht1 ht2
· induction' t2 using induction_on_pure with q r t ht t1 t2 ht1 ht2
 · simp [prodT_pure, permT_pure, permT_pure, prodT_pure, Pure.prodP_assoc]
 · simp [ht]
 · simp [ht1, ht2]
· simp [ht]
· simp [ht1, ht2]
· simp [ht]
· simp [ht1, ht2]
 {c1 : Fin n1 → C} {c2 : Fin n2 → C} (t : S.Tensor c) (t1 : S.Tensor c1) (t2 : S.Tensor c2)
 prodT t (prodT t1 t2) = permT _ IsReindexing.append_assoc_left (prodT (prodT t t1) t2)
induction' t using induction_on_pure with p r t ht t1 t2 ht1 ht2
· induction' t1 using induction_on_pure with q r t ht t1 t2 ht1 ht2
· induction' t2 using induction_on_pure with q r t ht t1 t2 ht1 ht2
 · simp [prodT_pure, permT_pure, permT_pure, prodT_pure, Pure.prodP_assoc']
 · simp [ht]
 · simp [ht1, ht2]
· simp [ht]
· simp [ht1, ht2]
· simp [ht]
· simp [ht1, ht2]

```

**1.235 Physlib/Relativity/Tensors/RealTensor/Basic.lean**

```

deriving Fintype
/-- The modules associated with each of the different types of real Lorentz
/
abbrev modules (d : ℕ) : Color → Type
| Color.up => Lorentz.ContrMod d
| Color.down => Lorentz.CoMod d

instance modulesAddCommGroup (d : ℕ) : ∀ c, AddCommGroup (modules d c)
| Color.up => inferInstance
| Color.down => inferInstance

instance modulesModule (d : ℕ) : ∀ c, Module ℝ (modules d c)
| Color.up => inferInstance
| Color.down => inferInstance

TODO "Replace Lorentz.ContrMod and Lorentz.CoMod in the definition of realL
directly with Lorentz.Vector and Lorentz.Covector, and
representations defined on them."

def realLorentzTensor (d : ℕ := 3) : TensorSpecies
 ℝ realLorentzTensor.Color (LorentzGroup d)
 (fun | Color.up => Lorentz.ContrMod d | Color.down => Lorentz.CoMod d)
 (fun _ => Fin 1 ⊗ Fin d)
 (fun | Color.up => Lorentz.ContrMod.rep | Color.down => Lorentz.CoMod.r
 (fun | Color.up => Lorentz.contrBasis d | Color.down => Lorentz.coBasis
 contr := fun c =>
 | Color.up => Lorentz.contrCoContract
 | Color.down => Lorentz.coContrContract
 metric := fun c =>
 | Color.up => Lorentz.preContrMetric d
 | Color.down => Lorentz.preCoMetric d
 unit := fun c =>
 | Color.up => Lorentz.preCoContrUnit d
 | Color.down => Lorentz.preContrCoUnit d
open TensorSpecies Tensor
simp only [Pure.contrPCoeff, Pure.basisVector]
generalize_proofs h1 h2
generalize c i = ci at *
generalize c j = cj at *
subst h2
fin_cases ci
· simp [realLorentzTensor]
 erw [LinearEquiv.cast_apply]
 simp only [cast_eq]

```

1.236.

*PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/COVECTOR/BASIC.lean*

```

 erw [Lorentz.contrCoContract_basis]
 · simp [realLorentzTensor]
 erw [LinearEquiv.cast_apply]
 simp only [cast_eq]
 erw [Lorentz.coContrContract_basis]
 contrT n i j h t = $\sum (\mu : \text{Fin } 1 \oplus \text{Fin } d), \text{permT id (by$
 (evalT ((Fin.predAbove 0 i).predAbove j) μ (evalT i μ t)) := by
 simp only [contrPCoeff_basis, ite_smul, one_smul, zero_smul]
 rw [contrT_eq_sum_evalT, map_sum]
 open ComponentIdx in
 lemma contrT_basis_repr_apply_eq_fin {n d : $\mathbb{N}$ } {c : Fin (n + 1 + 1) $\rightarrow$ realLorentzTens
 {i j : Fin (n + 1 + 1)}
 {h : i $\neq$ j $\wedge$ (realLorentzTensor d). τ (c i) = c j}
 (t : $\mathbb{R}T(d, c)$) (b : ComponentIdx (c $\circ$ Fin.succSuccAbove i j)) :
 (basis (c $\circ$ Fin.succSuccAbove i j)).repr (contrT n i j h t) b =
 $\sum (x : \text{Fin } 1 \oplus \text{Fin } d), ((\text{basis } c).repr t$
 (DropPairSection.ofFinEquiv h.1 b (x, x))) := by
 rw [contrT_basis_repr_apply_eq_sum_fin]
 generalize_proofs h h2 h3
 generalize c j = cj at *
 generalize c i = ci at *
 subst h3
 fin_cases ci
 · simp [realLorentzTensor]
 congr
 funext x
 conv_lhs =>
 enter [2, x];
 erw [Lorentz.contrCoContract_basis]
 simp
 · simp [realLorentzTensor]
 congr
 funext x
 conv_lhs =>
 enter [2, x];
 erw [Lorentz.coContrContract_basis]
 simp

```

## 1.236 Physlib/Relativity/Tensors/RealTensor/CoVector/Basic.lean

```

public import Mathlib.Analysis.InnerProductSpace.PiL2
public import Mathlib.Geometry.Manifold.IsManifold.Basic
open Module

```

```

@[ext]
lemma eq_of_apply_eq {d : ℕ} {v w : CoVector d} (h : ∀ i, v i = w i) : v =
 apply (equivEuclid d).injective
 ext i
 exact h i

 rw [← neg_add_eq_zero, h]
@[simp]
lemma apply_sum {d : ℕ} {ι : Type} [Fintype ι] (f : ι → CoVector d) (i : Fin d)
 (Σ j, f j) i = Σ j, f j i := Finset.sum_apply i Finset.univ f

```

### 1.237 Physlib/Relativity/Tensors/RealTensor/CoVector/Representation

```

/-
Copyright (c) 2026 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Mathlib.RepresentationTheory.Basic
public import Physlib.Relativity.LorentzGroup.Basic
public import Physlib.Relativity.Tensors.RealTensor.CoVector.Basic
/-!

Representation of the Lorentz group on Lorentz vectors

In this module we define the representation of the Lorentz group on Lorentz
This does not define the MulAction on `Lorentz.CoVector`, which is induced
by its tensor structure.

-/

@[expose] public section

open Module Matrix MatrixGroups Complex TensorProduct

noncomputable section

namespace Lorentz

```

1.237.

PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/COVECTOR/REPRESENTATION. LEAN

namespace CoVector

attribute [-simp] Fintype.sum\_sum\_type

/-- The representation of the Lorentz group on Lorentz vectors. -  
/

def rep {d : ℕ} : Representation ℝ (LorentzGroup d) (CoVector d) where  
 toFun  $\Lambda$  := Matrix.toLinAlgEquiv basis (LorentzGroup.transpose  $\Lambda^{-1}$ )  
 map\_one' := by  
 simp only [inv\_one, LorentzGroup.transpose\_one, lorentzGroupIsGroup\_one\_coe, \_root\_.mul\_one]  
 map\_mul' x y := by  
 simp only [\_root\_.mul\_inv\_rev, LorentzGroup.inv\_eq\_dual, LorentzGroup.transpose\_mul\_coe, \_root\_.map\_mul]

/-!

## Properties of the representation.

-/

lemma rep\_apply\_eq\_mulVec (d : ℕ) ( $\Lambda$  : LorentzGroup d) (v : CoVector d) :  
 rep  $\Lambda$  v = (LorentzGroup.transpose  $\Lambda^{-1}$ ) \*<sub>v</sub> v := by rfl

lemma rep\_apply\_eq\_sum (d : ℕ) ( $\Lambda$  : LorentzGroup d) (v : CoVector d) (k : Fin 1  $\oplus$  Fin d) :  
 rep  $\Lambda$  v k =  $\sum j$ , ( $\Lambda^{-1}$ ).1 j k • v j := rfl

lemma rep\_apply\_eq\_sum\_coe (d : ℕ) ( $\Lambda$  : LorentzGroup d) (v : CoVector d) (k : Fin 1  $\oplus$  Fin d) :  
 rep  $\Lambda$  v k =  $\sum j$ ,  $\Lambda.1^{-1}$  j k • v j := by  
 rw [rep\_apply\_eq\_sum, LorentzGroup.coe\_inv]

lemma rep\_apply\_basis {d} ( $\mu$  : Fin 1  $\oplus$  Fin d) ( $\Lambda$  : LorentzGroup d) :  
 rep  $\Lambda$  (basis  $\mu$ ) =  $\sum j$ ,  $\Lambda.1^{-1}$   $\mu$  j • basis j := by  
 ext k  
 simp [rep\_apply\_eq\_sum\_coe, apply\_sum]

lemma rep\_toMatrix (d : ℕ) ( $\Lambda$  : LorentzGroup d) :  
 LinearMap.toMatrix basis basis (rep  $\Lambda$ ) =  $\Lambda.1^{-1\top}$  := by  
 simp only [rep, MonoidHom.coe\_mk, OneHom.coe\_mk]  
 rw [← LorentzGroup.coe\_inv]  
 exact (LinearEquiv.eq\_symm\_apply (LinearMap.toMatrix basis basis)).mp rfl

lemma rep\_injective (d : ℕ) ( $\Lambda$  : LorentzGroup d) : Function.Injective (rep  $\Lambda$ ) := by  
 intro v1 v2 h  
 rw [rep\_apply\_eq\_mulVec, rep\_apply\_eq\_mulVec] at h  
 exact Matrix.mulVec\_injective\_of\_isUnit (isUnit\_of\_invertible \_) h

lemma rep\_surjective (d : ℕ) ( $\Lambda$  : LorentzGroup d) : Function.Surjective (rep  $\Lambda$ ) := by

```

 intro v
 use (LorentzGroup.transpose Λ) *v v
 rw [rep_apply_eq_mulVec]
 simp [← LorentzGroup.transpose_inv, LorentzGroup.coe_inv]

lemma rep_bijective (d : $\mathbb{N}$) (Λ : LorentzGroup d) : Function.Bijective (rep
 (rep_injective d Λ , rep_surjective d Λ)

end CoVector

end Lorentz

```

### 1.238 Physlib/Relativity/Tensors/RealTensor/CoVector/Tensorial.

```

/-
Copyright (c) 2026 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Matteo Cipollina, Joseph Tooby-Smith
-/
module

public import Physlib.Relativity.Tensors.RealTensor.CoVector.Basic
public import Physlib.Relativity.Tensors.RealTensor.Basic
public import Mathlib.Geometry.Manifold.ChartedSpace
/-!

Tensorial nature of Lorentz covectors

We define the tensorial instance on `Lorentz.coVector`, and show
prove properties related to the Lorentz group action and the basis.

-/

@[expose] public section

open Module
open Matrix
open MatrixGroups
open Complex
open TensorProduct

noncomputable section

namespace Lorentz

```



1.238.

PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/COVECTOR/TENSORIAL.lean

```
open realLorentzTensor
```

```
namespace CoVector
```

```
open TensorSpecies
```

```
open Tensor
```

```
/-!
```

```
Tensorial
```

```
-/
```

```
/-- The equivalence between the type of indices of a Lorentz vector and
```

```
 `Fin 1  $\oplus$  Fin d`. -/
```

```
def indexEquiv {d : $\mathbb{N}$ } :
```

```
 ComponentIdx (S := (realLorentzTensor d)) ![Color.down] $\approx$ Fin 1 $\oplus$ Fin d :=
```

```
 ComponentIdx.single (S := realLorentzTensor d) (c := Color.down)
```

```
instance tensorial {d : $\mathbb{N}$ } : Tensorial (realLorentzTensor d) ![:down] (CoVector d) w
```

```
 toTensor := LinearEquiv.symm <|
```

```
 Equiv.toLinearEquiv
```

```
 ((Tensor.basis (S := (realLorentzTensor d)) ![:down])).repr.toEquiv.trans <|
```

```
 Finsupp.equivFunOnFinite.trans <|
```

```
 (Equiv.piCongrLeft' _ indexEquiv))
```

```
 { map_add := fun x y => by
```

```
 simp [map_add]
```

```
 rfl
```

```
 map_smul := fun c x => by
```

```
 simp [_root_.map_smul]
```

```
 rfl}
```

```
open Tensorial
```

```
lemma toTensor_symm_apply {d : $\mathbb{N}$ } (p : $\mathbb{R}T[d, .down]$) :
```

```
 (toTensor (self := tensorial)).symm p =
```

```
 (Equiv.piCongrLeft' _ indexEquiv <|
```

```
 Finsupp.equivFunOnFinite <|
```

```
 (Tensor.basis (S := (realLorentzTensor d)) _).repr p) := rfl
```

```
lemma toTensor_symm_pure {d : $\mathbb{N}$ } (p : Pure (realLorentzTensor d) ![:down]) (i : Fin
```

```
 (toTensor (self := tensorial)).symm p.toTensor i =
```

```
 ((Lorentz.coBasis d).repr (p 0)) (indexEquiv.symm i 0) := by
```

```
 rw [toTensor_symm_apply]
```

```
 simp only [Equiv.piCongrLeft'_apply, Finsupp.equivFunOnFinite_apply, Tensor.basis_
```

```
 Pure.component, Finset.univ_unique, Fin.default_eq_zero, Fin.isValue, Finset.pro
```

```

 cons_val_zero]
 rfl

/-!

Basis

-/

set_option backward.isDefEq.respectTransparency false in
lemma toTensor_symm_basis {d : ℕ} (μ : Fin 1 ⊕ Fin d) :
 (toTensor (self := tensorial)).symm (Tensor.basis ![Color.down] (indexEquiv
 basis μ := by
 funext i
 simp [Tensor.basis_apply, toTensor_symm_pure, Pure.basisVector, Finsupp.s
 indexEquiv]

lemma toTensor_basis_eq_tensor_basis {d : ℕ} (μ : Fin 1 ⊕ Fin d) :
 toTensor (basis μ) = Tensor.basis ![Color.down] (indexEquiv.symm μ) :=
 rw [← toTensor_symm_basis]
 simp

lemma basis_eq_map_tensor_basis {d} : basis =
 ((Tensor.basis (S := realLorentzTensor d) ![Color.down])).map
 toTensor.symm).reindex indexEquiv := by
 ext μ
 rw [← toTensor_symm_basis]
 simp

lemma tensor_basis_map_eq_basis_reindex {d} :
 (Tensor.basis (S := realLorentzTensor d) ![Color.down])).map toTensor.symm
 basis.reindex indexEquiv.symm := by
 rw [basis_eq_map_tensor_basis]
 ext μ
 simp

lemma tensor_basis_repr_toTensor_apply {d : ℕ} (p : CoVector d) (μ : Component d) :
 (Tensor.basis ![Color.down]).repr (toTensor p) μ =
 p (indexEquiv μ) := by
 obtain ⟨p, rfl⟩ := toTensor.symm.surjective p
 simp only [LinearEquiv.apply_symm_apply]
 apply induction_on_pure (t := p)
 · intro p
 simp only [Tensor.basis_repr_pure, Pure.component, Finset.univ_unique,
 Fin.isValue, Finset.prod_singleton, cons_val_zero, toTensor_symm_pure,
 Equiv.symm_apply_apply]

```

1.238.

PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/COVECTOR/TENSORIAL.lean

```

 rfl
 · intro r t h
 simp [h]
 · intro t1 t2 h1 h2
 simp [h1, h2]

/-!

The action of the Lorentz group

-/

set_option backward.isDefEq.respectTransparency false in
lemma smul_eq_sum {d : ℕ} (i : Fin 1 ⊕ Fin d) (Λ : LorentzGroup d) (p : CoVector d)
 (Λ • p) i = ∑ j, Λ⁻¹.1 j i * p j := by
 obtain {p, rfl} := toTensor.symm.surjective p
 rw [smul_toTensor_symm]
 apply induction_on_pure (t := p)
 · intro p
 rw [actionT_pure, toTensor_symm_pure]
 conv_lhs =>
 enter [1, 2]
 change (LorentzGroup.transpose Λ⁻¹) *v (p 0)
 rw [coBasis_repr_apply]
 conv_lhs => simp [indexEquiv]
 rw [mulVec_eq_sum]
 simp only [Finset.sum_apply, Pi.smul_apply, transpose_apply, MulOpposite.smul_eq,
 MulOpposite.unop_op, toTensor_symm_pure, coBasis_repr_apply]
 rfl
 · intro r t h
 simp only [actionT_smul, _root_.map_smul, apply_smul, h, Finset.mul_sum, mul_left_distrib]
 · intro t1 t2 h1 h2
 simp only [actionT_add, map_add, h1, h2, apply_add, mul_add, Finset.sum_add_distrib]

lemma smul_eq_mulVec {d} (Λ : LorentzGroup d) (p : CoVector d) :
 Λ • p = (LorentzGroup.transpose Λ⁻¹).1 *v p := by
 funext i
 rw [smul_eq_sum, mulVec_eq_sum]
 simp only [op_smul_eq_smul, Finset.sum_apply, Pi.smul_apply, transpose_apply, smul_eq, mul_comm]
 rfl

lemma smul_add {d : ℕ} (Λ : LorentzGroup d) (p q : CoVector d) :
 Λ • (p + q) = Λ • p + Λ • q := by simp

set_option backward.isDefEq.respectTransparency false in
```

```

@[simp]
lemma smul_sub {d : ℕ} (Λ : LorentzGroup d) (p q : CoVector d) :
 Λ • (p - q) = Λ • p - Λ • q := by
 rw [smul_eq_mulVec, smul_eq_mulVec, smul_eq_mulVec, Matrix.mulVec_sub]

set_option backward.isDefEq.respectTransparency false in
lemma smul_zero {d : ℕ} (Λ : LorentzGroup d) :
 Λ • (0 : CoVector d) = 0 := by
 rw [smul_eq_mulVec, Matrix.mulVec_zero]

set_option backward.isDefEq.respectTransparency false in
lemma smul_neg {d : ℕ} (Λ : LorentzGroup d) (p : CoVector d) :
 Λ • (-p) = - (Λ • p) := by
 rw [smul_eq_mulVec, smul_eq_mulVec, Matrix.mulVec_neg]

/-- The Lorentz action on vectors as a continuous linear map. -
/
def actionCLM {d : ℕ} (Λ : LorentzGroup d) :
 CoVector d →L[ℝ] CoVector d :=
 LinearMap.toContinuousLinearMap
 { toFun := fun v => Λ • v
 map_add' := smul_add Λ
 map_smul' := fun c v => by
 funext i
 simp only [RingHom.id_apply, apply_smul, smul_eq_sum, Finset.mul_su

lemma actionCLM_apply {d : ℕ} (Λ : LorentzGroup d) (p : CoVector d) :
 actionCLM Λ p = Λ • p := rfl

set_option backward.isDefEq.respectTransparency false in
lemma smul_basis {d : ℕ} (Λ : LorentzGroup d) (μ : Fin 1 ⊕ Fin d) :
 Λ • basis μ = ∑ v, Λ⁻¹.1 μ v • basis v := by
 funext i
 rw [smul_eq_sum, Fintype.sum_apply]
 simp only [apply_smul, basis_apply, mul_ite, mul_one, mul_zero,
 Finset.sum_ite_eq, Finset.sum_ite_eq', Finset.mem_univ, ↓reduceIte]

end CoVector

end Lorentz

```

**1.239 Physlib/Relativity/Tensors/RealTensor/Matrix/Pre.lean**

```

/-- Equivalence of `ContrMod ⊗ ContrMod` to `(1 + d) x (1 + d)` real matrices. -
/
def contrContrToMatrixRe {d : ℕ} : (ContrMod d ⊗[ℝ] ContrMod d) ≈1[ℝ]
/-- Equivalence of `CoMod ⊗ CoMod` to `(1 + d) x (1 + d)` real matrices. -
/
def coCoToMatrixRe {d : ℕ} : (CoMod d ⊗[ℝ] CoMod d) ≈1[ℝ]
/-- Equivalence of `ContrMod d ⊗ CoMod d` to `(1 + d) x (1 + d)` real matrices. -
/
def contrCoToMatrixRe {d : ℕ} : (ContrMod d ⊗[ℝ] CoMod d) ≈1[ℝ]
/-- Equivalence of `CoMod d ⊗ ContrMod d` to `(1 + d) x (1 + d)` real matrices. -
/
def coContrToMatrixRe : (CoMod d ⊗[ℝ] ContrMod d) ≈1[ℝ]
lemma contrContrToMatrixRe_ρ {d : ℕ} (v : (ContrMod d ⊗[ℝ] ContrMod d)) (M : LorentzGroup d) :
 contrContrToMatrixRe (TensorProduct.map (ContrMod.rep M) (ContrMod.rep M) v) =
 (TensorProduct.map (ContrMod.rep M) (ContrMod.rep M))
 (TensorProduct.map (ContrMod.rep M) (ContrMod.rep M)) v
 ((LinearMap.toMatrix (contrBasis d) (contrBasis d)) (ContrMod.rep M))
 ((LinearMap.toMatrix (contrBasis d) (contrBasis d)) (ContrMod.rep M)) (i, j) k
lemma coCoToMatrixRe_ρ {d : ℕ} (v : (CoMod d ⊗[ℝ] CoMod d)) (M : LorentzGroup d) :
 coCoToMatrixRe (TensorProduct.map (CoMod.rep M) (CoMod.rep M) v) =
 (TensorProduct.map (CoMod.rep M) (CoMod.rep M))
 (TensorProduct.map (CoMod.rep M) (CoMod.rep M)) v
 ((LinearMap.toMatrix (coBasis d) (coBasis d)) (CoMod.rep M))
 ((LinearMap.toMatrix (coBasis d) (coBasis d)) (CoMod.rep M)) (i, j) k
lemma contrCoToMatrixRe_ρ {d : ℕ} (v : (ContrMod d ⊗[ℝ] CoMod d)) (M : LorentzGroup d) :
 contrCoToMatrixRe (TensorProduct.map (ContrMod.rep M) (CoMod.rep M) v) =
 (TensorProduct.map (ContrMod.rep M) (CoMod.rep M))
 (TensorProduct.map (ContrMod.rep M) (CoMod.rep M)) v
 ((LinearMap.toMatrix (contrBasis d) (contrBasis d)) (ContrMod.rep M))
 ((LinearMap.toMatrix (coBasis d) (coBasis d)) (CoMod.rep M)) (i, j) k
lemma coContrToMatrixRe_ρ {d : ℕ} (v : (CoMod d ⊗[ℝ] ContrMod d)) (M : LorentzGroup d) :
 coContrToMatrixRe (TensorProduct.map (CoMod.rep M) (ContrMod.rep M) v) =
 (TensorProduct.map (CoMod.rep M) (ContrMod.rep M))
 (TensorProduct.map (CoMod.rep M) (ContrMod.rep M)) v
 ((LinearMap.toMatrix (coBasis d) (coBasis d)) (CoMod.rep M))
 ((LinearMap.toMatrix (contrBasis d) (contrBasis d)) (ContrMod.rep M)) (i, j) k
 TensorProduct.map (ContrMod.rep M) (ContrMod.rep M) (contrContrToMatrixRe.symm v)
refine contrContrToMatrixRe.injective ?_
simp [contrContrToMatrixRe_ρ]
 TensorProduct.map (CoMod.rep M) (CoMod.rep M) (coCoToMatrixRe.symm v) =
 TensorProduct.map (ContrMod.rep M) (CoMod.rep M) (contrCoToMatrixRe.symm v) =
rw [← h1, LinearEquiv.symm_apply_apply]
 TensorProduct.map (CoMod.rep M) (ContrMod.rep M) (coContrToMatrixRe.symm v) =
rw [← h1, LinearEquiv.symm_apply_apply]

```

**1.240 Physlib/Relativity/Tensors/RealTensor/Metrics/Basic.lean**

```

open Module
 η' d = fromConstPair (S := realLorentzTensor d) (c1 := .down) (c2 := .d
 (Lorentz.preCoMetric d) := by
 η d = fromConstPair (S := realLorentzTensor d)
 (c1 := .up) (c2 := .up) (Lorentz.preContrMetric d) := by
 η' d = fromPairT (S := realLorentzTensor d) (c1 := .down) (c2 := .down)
 (Lorentz.preCoMetricVal d) := by
 rw [coMetric_eq_fromConstPair, fromConstPair, Lorentz.preCoMetric_apply_o
 η d = fromPairT (S := realLorentzTensor d) (c1 := .up) (c2 := .up)
 (Lorentz.preContrMetricVal d) := by
 rw [contrMetric_eq_fromConstPair, fromConstPair, Lorentz.preContrMetric_a
 rw [coMetric_eq_fromPairT, fromPairT_basis_repr,
 Lorentz.preCoMetricVal_expand_tmul_minkowskiMatrix]
 simp only [map_sum, Finsupp.coe_finsetSum, Finset.sum_apply, map_smul, Fi
 Pi.smul_apply, Basis.tensorProduct_repr_tmul_apply, Basis.repr_self, Fi
 smul_eq_mul]
 rw [Finset.sum_eq_single (b 0)] <=>
 simp +contextual [minkowskiMatrix.as_diagonal, Matrix.diagonal_apply]

 rw [contrMetric_eq_fromPairT, fromPairT_basis_repr,
 Lorentz.preContrMetricVal_expand_tmul_minkowskiMatrix]
 simp only [map_sum, Finsupp.coe_finsetSum, Finset.sum_apply, map_smul, Fi
 Pi.smul_apply, Basis.tensorProduct_repr_tmul_apply, Basis.repr_self, Fi
 smul_eq_mul]
 rw [Finset.sum_eq_single (b 0)] <=>
 simp +contextual [minkowskiMatrix.as_diagonal, Matrix.diagonal_apply]

```

**1.241 Physlib/Relativity/Tensors/RealTensor/Metrics/Pre.lean**

```

open scoped TensorProduct
/-- The metric η^{aa} as an element of $(\text{ContrMod } d \otimes [\mathbb{R}] \text{ ContrMod } d)$. -
/
def preContrMetricVal (d : ℕ := 3) : ContrMod d ⊗ [ℝ] ContrMod d :=
lemma preContrMetricVal_expand_tmul_minkowskiMatrix {d : ℕ} : preContrMetri
 ∑ i, (minkowskiMatrix i i) • (contrBasis d i ⊗+ [ℝ] contrBasis d i) := b
 rw [preContrMetricVal, contrContrToMatrixRe_symm_expand_tmul]
 exact Finset.sum_congr rfl fun i _ => Finset.sum_eq_single_of_mem i (Fins
 fun j _ hj => smul_eq_zero_of_left (minkowskiMatrix.off_diag_zero hj.sy

 rw [preContrMetricVal_expand_tmul_minkowskiMatrix]

```

1.241.

PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/METRICS/PRE.LEAN363

```

simp [Fintype.sum_sum_type, minkowskiMatrix.inl_0_inl_0, minkowskiMatrix.inr_i_inr_i,
sub_eq_add_neg]
/-- The metric η^{aa} as a morphism $\square_{\mathbb{R}} (\text{Rep } \mathbb{R} (\text{LorentzGroup } d)) \square \text{ContrMod.rep} \otimes \text{ContrMod.rep}$ -/
def preContrMetric (d : $\mathbb{N}$:= 3) :
 (Representation.trivial $\mathbb{R}$ (LorentzGroup d) $\mathbb{R}$).IntertwiningMap
 ((ContrMod.rep).tprod (ContrMod.rep)) where
toFun := fun a => a • (preContrMetricVal d)
map_add' := fun x y => add_smul x y _
map_smul' := fun m x => mul_smul m x _
isIntertwining' M := by
 refine LinearMap.ext fun x : $\mathbb{R}$ => ?_
 simp only [LinearMap.coe_comp, Function.comp_apply]
 change x • (preContrMetricVal d) =
 (TensorProduct.map (ContrMod.rep M) (ContrMod.rep M)) (x • (preContrMetricVal d))
 simp only [map_smul]
 apply congrArg
 simp only [preContrMetricVal]
 conv_rhs =>
 rw [contrContrToMatrixRe_p_symm]
 apply congrArg
 simp

lemma preContrMetric_apply_one {d : $\mathbb{N}$ } : (preContrMetric d) (1 : $\mathbb{R}$) = preContrMetricVal d
one_smul $\mathbb{R}$ _

/-- The metric η_{ii} as an element of $(\text{CoMod } d \otimes [\mathbb{R}] \text{CoMod } d)$. -/
/
def preCoMetricVal (d : $\mathbb{N}$:= 3) : CoMod d $\otimes$ [$\mathbb{R}$] CoMod d :=
lemma preCoMetricVal_expand_tmul_minkowskiMatrix {d : $\mathbb{N}$ } : preCoMetricVal d =
 $\sum i$, (minkowskiMatrix i i) • (coBasis d i $\otimes_{+} [\mathbb{R}]$ coBasis d i) := by
 rw [preCoMetricVal, coCoToMatrixRe_symm_expand_tmul]
 exact Finset.sum_congr rfl fun i _ => Finset.sum_eq_single_of_mem i (Finset.mem_univ d)
 fun j _ hj => smul_eq_zero_of_left (minkowskiMatrix.off_diag_zero hj.symm) _

 rw [preCoMetricVal_expand_tmul_minkowskiMatrix]
 simp [Fintype.sum_sum_type, minkowskiMatrix.inl_0_inl_0, minkowskiMatrix.inr_i_inr_i,
sub_eq_add_neg]
/-- The metric η_{ii} as a morphism $\square_{\mathbb{C}} (\text{Rep } \mathbb{C} (\text{LorentzGroup } d)) \square \text{CoMod.rep} \otimes \text{CoMod.rep}$ -/
def preCoMetric (d : $\mathbb{N}$:= 3) : (Representation.trivial $\mathbb{R}$ (LorentzGroup d) $\mathbb{R}$).IntertwiningMap
 ((CoMod.rep).tprod (CoMod.rep)) where
toFun := fun a => a • preCoMetricVal d
map_add' := fun x y => add_smul x y _
map_smul' := fun m x => mul_smul m x _
isIntertwining' M := by
 refine LinearMap.ext fun x : $\mathbb{R}$ => ?_
 simp only [LinearMap.coe_comp, Function.comp_apply]

```

```

change x • preCoMetricVal d =
 (TensorProduct.map (CoMod.rep M) (CoMod.rep M)) (x • preCoMetricVal d)
simp only [_root_.map_smul]
apply congrArg
simp only [preCoMetricVal]
rw [coCoToMatrixRe_p_symm]
apply congrArg
rw [← LorentzGroup.coe_inv, LorentzGroup.transpose_mul_minkowskiMatrix_

lemma preCoMetric_apply_one {d : ℕ} : (preCoMetric d) (1 : ℝ) = preCoMetric
one_smul ℝ _
 (TensorProduct.comm ℝ _ _ <|
 (TensorProduct.lid ℝ _).lTensor _ <|
 (contrCoContract.toLinearMap.rTensor (CoMod d)).lTensor (ContrMod d)
 (TensorProduct.assoc ℝ (ContrMod d) (CoMod d) (CoMod d)).symm.toLinea
 (ContrMod d) <|
 TensorProduct.assoc ℝ (ContrMod d) (ContrMod d) ((CoMod d) ⊗[ℝ] (CoMo
 (preContrMetric d 1) ⊗t[ℝ] (preCoMetric d 1)) = preCoContrUnit d (1 :
 _ = (TensorProduct.comm ℝ _ _ <|
 (TensorProduct.lid ℝ _).lTensor _ <|
 (contrCoContract.toLinearMap.rTensor (CoMod d)).lTensor (ContrMod d)
 (TensorProduct.assoc ℝ (ContrMod d) (CoMod d) (CoMod d)).symm.toLinea
 (ContrMod d) <|
 TensorProduct.assoc ℝ (ContrMod d) (ContrMod d) ((CoMod d) ⊗[ℝ] (CoMo
 preContrMetricVal_expand_tmul_minkowskiMatrix,
 preCoMetricVal_expand_tmul_minkowskiMatrix, sum_tmul]
 simp_rw [tmul_sum, ← smul_tmul', tmul_smul, smul_smul]
 _ = (TensorProduct.comm ℝ _ _ <| (TensorProduct.lid ℝ _).lTensor _ <|
 simp [map_sum, map_smul]
 _ = (TensorProduct.comm ℝ _ _ <| (TensorProduct.lid ℝ _).lTensor _ <|
 rw [Finset.sum_eq_single_of_mem x (Finset.mem_univ x)]
 · simp [minkowskiMatrix.η_apply_mul_η_apply_diag, contrCoContract_b
 simp [contrCoContract_basis, if_neg (Ne.symm hb)]
simp [map_sum]
 (TensorProduct.comm ℝ _ _ <|
 (TensorProduct.lid ℝ _).lTensor _ <|
 (coContrContract.toLinearMap.rTensor (ContrMod d)).lTensor (CoMod d) <|
 (TensorProduct.assoc ℝ (CoMod d) (ContrMod d) (ContrMod d)).symm.toLine
 (CoMod d) <|
 TensorProduct.assoc ℝ (CoMod d) (CoMod d) ((ContrMod d) ⊗[ℝ] (ContrMo
 (preCoMetric d 1) ⊗t[ℝ] (preContrMetric d 1)) = preContrCoUnit d (1 : ℝ
 _ = (TensorProduct.comm ℝ _ _ <| (TensorProduct.lid ℝ _).lTensor _ <|
 (coContrContract.toLinearMap.rTensor (ContrMod d)).lTensor (CoMod d)
 (TensorProduct.assoc ℝ (CoMod d) (ContrMod d) (ContrMod d)).symm.toLi
 (CoMod d) <|
 TensorProduct.assoc ℝ (CoMod d) (CoMod d) ((ContrMod d) ⊗[ℝ] (ContrMo

```



1.242.

PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/TOCOMPLEX.LEAN 365

```
 preCoMetricVal_expand_tmul_minkowskiMatrix,
 preContrMetricVal_expand_tmul_minkowskiMatrix, sum_tmul]
 simp_rw [tmul_sum, ← smul_tmul', tmul_smul, smul_smul]
 _ = (TensorProduct.comm ℝ _ _ <| (TensorProduct.lid ℝ _).lTensor _ <|
 simp [map_sum, map_smul])
 _ = (TensorProduct.comm ℝ _ _ <| (TensorProduct.lid ℝ _).lTensor _ <|
 rw [Finset.sum_eq_single_of_mem x (Finset.mem_univ x)]
 · simp [minkowskiMatrix.η_apply_mul_η_apply_diag, coContrContract_basis]
 simp [coContrContract_basis, if_neg (Ne.symm hb)]
simp [map_sum]
```

## 1.242 Physlib/Relativity/Tensors/RealTensor/ToComplex.lean

Authors: Nikolai Kashcheev, Joseph Tooby-Smith

```
 simp only [map_smul, Finsupp.coe_smul, Pi.smul_apply, smul_eq_mul, Complex.ofReal]
 rw [← smul_smul]
 rfl
 simp only [Finsupp.coe_finsetSum, Finset.sum_apply]
/-!
pure
-/
/-- For a given color, the map turning a real Lorentz vector into a complex one. -
/
noncomputable def toComplexVector (c : realLorentzTensor.Color) :
 realLorentzTensor.modules 3 c →SL [Complex.ofRealHom] complexLorentzTensor.modules
 (colorToComplex c) where
toFun v := match c with
| Color.up => ∑ i, ((Lorentz.contrBasis 3).repr v i) •
 Lorentz.complexContrBasisFin4 (finSumFinEquiv i)
| Color.down => ∑ i, ((Lorentz.coBasis 3).repr v i) •
 Lorentz.complexCoBasisFin4 (finSumFinEquiv i)
map_add' v1 v2 := by
 match c with
| Color.up =>
 simp only [map_add, Finsupp.coe_add, Pi.add_apply, Nat.reduceAdd, ← Finset.sum]
 congr
 funext x
 rw [add_smul]
 rfl
| Color.down =>
 simp only [map_add, Finsupp.coe_add, Pi.add_apply, Nat.reduceAdd, ← Finset.sum]
 congr
 funext x
 rw [add_smul]
```

```

 rfl
map_smul' r v := by
 match c with
 | Color.up =>
 simp only [map_smul, Finsupp.coe_smul, Pi.smul_apply, smul_eq_mul, Na
 Complex.ofRealHom_eq_coe, Complex.coe_smul]
 rw [Finset.smul_sum]
 congr
 funext x
 rw [← smul_smul]
 rfl
 | Color.down =>
 simp only [map_smul, Finsupp.coe_smul, Pi.smul_apply, smul_eq_mul, Na
 Complex.ofRealHom_eq_coe, Complex.coe_smul]
 rw [Finset.smul_sum]
 congr
 funext x
 rw [← smul_smul]
 rfl

lemma toComplexVector_up_eq_inclCongrRealLorentz (v : Lorentz.ContrMod 3) :
 toComplexVector Color.up v = Lorentz.inclCongrRealLorentz v := by
 calc
 toComplexVector Color.up v
 = $\sum i, v.toFinld\mathbb{R} i \cdot \text{Lorentz.complexContrBasis } i$:= by
 simp [toComplexVector, Lorentz.contrBasis_repr_apply,
 Lorentz.complexContrBasisFin4, Lorentz.ContrMod.toFinld $\mathbb{R}$ _eq_val]
 rfl
 _ = $\sum i, v.toFinld\mathbb{R} i \cdot \text{Lorentz.inclCongrRealLorentz}$
 (Lorentz.ContrMod.stdBasis i) := by
 simp only [Lorentz.complexContrBasis_of_real]
 _ = Lorentz.inclCongrRealLorentz
 ($\sum i, v.toFinld\mathbb{R} i \cdot \text{Lorentz.ContrMod.stdBasis } i$) := by
 rw [map_sum]
 simp only [LinearMap.map_smulsl, Complex.ofRealHom_eq_coe, Comple
 _ = Lorentz.inclCongrRealLorentz v := by
 rw [← Lorentz.ContrMod.stdBasis_decomp v]

lemma toComplexVector_down_eq_inclCoRealLorentz (v : Lorentz.CoMod 3) :
 toComplexVector Color.down v = Lorentz.inclCoRealLorentz v := by
 calc
 toComplexVector Color.down v
 = $\sum i, v.toFinld\mathbb{R} i \cdot \text{Lorentz.complexCoBasis } i$:= by
 simp [toComplexVector, Lorentz.coBasis_repr_apply, Lorentz.comple
 rfl
 _ = $\sum i, v.toFinld\mathbb{R} i \cdot \text{Lorentz.inclCoRealLorentz}$

```

1.242.

PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/TOCOMPLEX.LEAN 367

```

 (Lorentz.CoMod.stdBasis i) := by
 simp only [Lorentz.complexCoBasis_of_real]
 _ = Lorentz.inclCoRealLorentz
 (∑ i, v.toFinIdℝ i • Lorentz.CoMod.stdBasis i) := by
 rw [map_sum]
 simp only [LinearMap.map_smulₛ₁, Complex.ofRealHom_eq_coe, Complex.coe_smul]
 _ = Lorentz.inclCoRealLorentz v := by
 rw [← Lorentz.CoMod.stdBasis_decomp v]

/-- The function which turns a real pure tensor into a complex one. -
/
noncomputable def toComplexPure {c : Fin n → Color} (p : Pure realLorentzTensor c) :
 Pure complexLorentzTensor (colorToComplex c) := fun i =>
 toComplexVector (c i) (p i)

lemma toComplexPure_component {c : Fin n → Color} (p : Pure realLorentzTensor c)
 (φ : ComponentIdx c) : (toComplexPure p).component (ComponentIdx.complexify φ) =
 p.component φ := by
 simp [Pure.component, toComplexPure]
 congr
 funext x
 generalize φ x = φx at *
 generalize p x = px at *
 clear φ p
 generalize_proofs h1 h2 h3
 let b (c : Color) : Basis (Fin (complexLorentzTensor.repDim (colorToComplex c))) ℂ :=
 (complexLorentzTensor.modules (colorToComplex c)) :=
 match colorToComplex c with
 | Color.upL => Fermion.leftBasis
 | Color.downL => Fermion.dualLeftBasis
 | Color.upR => Fermion.rightBasis
 | Color.downR => Fermion.dualRightBasis
 | complexLorentzTensor.Color.up => Lorentz.complexContrBasisFin4
 | complexLorentzTensor.Color.down => Lorentz.complexCoBasisFin4
 let b' (c : Color) : Basis (Fin 1 ⊕ Fin 3) ℝ (realLorentzTensor.modules 3 c) :=
 (match c with
 | Color.up => Lorentz.contrBasis
 | Color.down => Lorentz.coBasis : Basis (Fin 1 ⊕ Fin 3) ℝ (realLorentzTensor.m
 let P (c : Color) (px : realLorentzTensor.modules 3 c) (φx : Fin 1 ⊕ Fin 3)
 (h2 : 4 = repDim (colorToComplex c)) : Prop :=
 ((b c).repr
 (toComplexVector c px))
 (Fin.cast h2 (finSumFinEquiv φx)) =
 ↑(((b' c).repr
 px)
 φx)

```

```

suffices h : P (c x) px ϕ x h2 by exact h
generalize c x = c at *
fin_cases c
· simp only [colorToComplex, toComplexVector, Nat.reduceAdd, Fin.cast_eq_
 trans ($\sum$ x, (Lorentz.complexContrBasisFin4.repr ((Lorentz.contrBasis.re
 Lorentz.complexContrBasisFin4 (finSumFinEquiv x))))
 (finSumFinEquiv ϕ x))
· simp only [Fintype.sum_sum_type, Finset.univ_unique, Fin.default_eq_z
 Finset.sum_singleton, Nat.reduceAdd, Finsupp.coe_add, Finsupp.coe_fin
 Finset.sum_apply]
 rfl
simp [- Fintype.sum_sum_type, Lorentz.complexContrBasisFin4]
trans $\sum$ x, (((Lorentz.contrBasis.repr px) x • Lorentz.complexContrBasis
 (Lorentz.complexContrBasis x) ϕ x))
· simp [Basis.repr_self, Complex.real_smul]
 rfl
simp [- Fintype.sum_sum_type, Finsupp.single_apply]
· simp only [colorToComplex, toComplexVector, Nat.reduceAdd, Fin.cast_eq_
 trans ($\sum$ x, (Lorentz.complexCoBasisFin4.repr ((Lorentz.coBasis.repr px)
 Lorentz.complexCoBasisFin4 (finSumFinEquiv x))))
 (finSumFinEquiv ϕ x))
· simp only [Fintype.sum_sum_type, Finset.univ_unique, Fin.default_eq_z
 Finset.sum_singleton, Nat.reduceAdd, Finsupp.coe_add, Finsupp.coe_fin
 Finset.sum_apply]
 rfl
simp [- Fintype.sum_sum_type, Lorentz.complexCoBasisFin4]
trans $\sum$ x, (((Lorentz.coBasis.repr px) x •
 Lorentz.complexCoBasis.repr (Lorentz.complexCoBasis x) ϕ x))
· simp [Basis.repr_self, Complex.real_smul]
 rfl
simp [- Fintype.sum_sum_type, Finsupp.single_apply]

lemma actionP_toComplexPure {n : $\mathbb{N}$ } (c : Fin n $\rightarrow$ Color) (p : Pure realLoren
 (Λ : SL(2, $\mathbb{C}$)) :
 Λ • toComplexPure p = toComplexPure (toLorentzGroup Λ • p) := by
ext i
simp [Pure.actionP_eq, toComplexPure]
let b (c : Color) : Representation $\mathbb{C}$ _ (complexLorentzTensor.modules (col
 match colorToComplex c with
 | Color.upL => Fermion.leftHandedRep
 | Color.downL => Fermion.dualLeftHandedRep
 | Color.upR => Fermion.rightHandedRep
 | Color.downR => Fermion.dualRightHandedRep
 | complexLorentzTensor.Color.up => Lorentz.ContrCModule.SL2CRep
 | complexLorentzTensor.Color.down => Lorentz.CoCModule.SL2CRep
let b' (c : Color) : Representation $\mathbb{R}$ _ (realLorentzTensor.modules 3 c) :

```

```

 (match c with
 | Color.up => Lorentz.ContrMod.rep
 | Color.down => Lorentz.CoMod.rep)
 let P (c : Color) (px : realLorentzTensor.modules 3 c) : Prop :=
 b c $\wedge$ (toComplexVector c px) = toComplexVector c (b' c (toLorentzGroup $\wedge$) px)
 change P (c i) (p i)
 generalize p i = p at *
 generalize c i = c at *
 fin_cases c
 · simp_all [P, b, b', colorToComplex]
 calc
 (Lorentz.ContrModule.SL2CRep $\wedge$) ((toComplexVector Color.up) p)
 = (Lorentz.ContrModule.SL2CRep $\wedge$) (Lorentz.inclCongrRealLorentz p) := by
 exact congrArg (Lorentz.ContrModule.SL2CRep $\wedge$)
 (toComplexVector_up_eq_inclCongrRealLorentz p)
 _ = Lorentz.inclCongrRealLorentz (Lorentz.ContrMod.rep (toLorentzGroup $\wedge$) p) :
 rw [Lorentz.inclCongrRealLorentz_p]
 _ = (toComplexVector Color.up) ((Lorentz.ContrMod.rep (toLorentzGroup $\wedge$)) p) :
 exact (toComplexVector_up_eq_inclCongrRealLorentz
 ((Lorentz.ContrMod.rep (toLorentzGroup $\wedge$)) p)).symm
 · simp_all [P, b, b', colorToComplex]
 calc
 (Lorentz.CoModule.SL2CRep $\wedge$) ((toComplexVector Color.down) p)
 = (Lorentz.CoModule.SL2CRep $\wedge$) (Lorentz.inclCoRealLorentz p) := by
 exact congrArg (Lorentz.CoModule.SL2CRep $\wedge$)
 (toComplexVector_down_eq_inclCoRealLorentz p)
 _ = Lorentz.inclCoRealLorentz (Lorentz.CoMod.rep (toLorentzGroup $\wedge$) p) := by
 rw [Lorentz.inclCoRealLorentz_p]
 _ = (toComplexVector Color.down) ((Lorentz.CoMod.rep (toLorentzGroup $\wedge$)) p) :=
 exact (toComplexVector_down_eq_inclCoRealLorentz
 ((Lorentz.CoMod.rep (toLorentzGroup $\wedge$)) p)).symm

lemma toComplex_pure {n : $\mathbb{N}$ } (c : Fin n $\rightarrow$ Color) (p : Pure realLorentzTensor c) :
 toComplex p.toTensor = (toComplexPure p).toTensor := by
 apply (Tensor.basis _).repr.injective
 ext ϕ
 obtain { ϕ , rfl} := TensorSpecies.Tensor.ComponentIdx.complexify.surjective ϕ
 rw [basis_repr_pure, toComplex_repr, toComplexPure_component]
 simp
 induction' v using induction_on_pure with p r t h t1 t2
 · rw [actionT_pure, toComplex_pure, actionT_pure, actionP_toComplexPure, toComplexPure]
 · simp
 rw [← h]
 change $\wedge$ • (r : $\mathbb{C}$) • toComplex t = _
 rw [actionT_smul]
 rfl

```

```

 · simp_all
slots. On colours this is encoded in the `IsReindexing` predicate, and on t
/-- The `IsReindexing` condition is preserved under `colorToComplex`. -
/
@[simp] lemma isReindexing_colorToComplex {n m : ℕ}
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) :
 IsReindexing (colorToComplex ∘ c) (colorToComplex ∘ c1) σ := by
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ)
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ)
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) (t : ℝT(3, c)) :
 permT (S := complexLorentzTensor) σ (isReindexing_colorToComplex (c :=
let h' : IsReindexing (colorToComplex ∘ c) (colorToComplex ∘ c1) σ :=
 isReindexing_colorToComplex (c := c) (c1 := c1) h
lemma isReindexing_prodTColorToComplex {n m : ℕ}
 IsReindexing (Fin.append (colorToComplex ∘ c) (colorToComplex ∘ c1))
 -- For `σ = id`, `IsReindexing.on_id` reduces the goal to pointwise color
 apply (IsReindexing.on_id
 (isReindexing_prodTColorToComplex (c := c) (c1 := c1))
 -- Double induction on the tensor basis: first over `t`, then over `t1`.
 -- additive cases follow from linearity of `prodT`, `toComplex` and `prod
 induction t using induction_on_basis with
 | h b =>
 induction t1 using induction_on_basis with
 | h b1 =>
 simp (config := { failIfUnchanged := false })
 [prodTColorToComplex, prodT_pure, permT_pure, Pure.prodP_basisVecto
 Pure.permP_basisVector, Tensor.basis_apply, toComplex_pure_basisV
 colorToComplex_append, basisIdxCongr_eq_cast]
 | hzero => simp [prodTColorToComplex]
 | hsmul r ta hta => simp [map_smul, hta, prodTColorToComplex]
 | hadd ta tb hta htb => simp [map_add, hta, htb, prodTColorToComplex]
 | hzero => simp [prodTColorToComplex]
 | hsmul r ta hta => simp [map_smul, hta, prodTColorToComplex]
 | hadd ta tb hta htb => simp [map_add, hta, htb, prodTColorToComplex]
 rw [contrPCoeff_basis, complexLorentzTensor.contrPCoeff_basis]
 simp only [Function.comp_apply, ComponentIdx.complexify_apply, Nat.redu
 Fin.cast_inj, EmbeddingLike.apply_eq_iff_eq]
 simp_all only [ne_eq]
 obtain ⟨left, right⟩ := h
 split
 next h => simp_all only [Complex.ofReal_one]
 next h => simp_all only [Complex.ofReal_zero]
 rw [Pure.dropPair_basisVector]
 rw [← Tensor.basis_apply]

```

**1.243 Physlib/Relativity/Tensors/RealTensor/Units/Pre.lean**

```

def preContrCoUnitVal (d : ℕ := 3) : ContrMod d ⊗[ℝ] CoMod d :=
 Finset.sum_const_zero, add_zero, one_apply_eq, one_smul, zero_add]
def preContrCoUnit (d : ℕ := 3) :
 (Representation.trivial ℝ (LorentzGroup d) ℝ).IntertwiningMap
 ((ContrMod.rep).tprod (CoMod.rep)) where
 toFun := fun a => a • preContrCoUnitVal d
 map_add' := fun x y => by
 simp only [add_smul]
 map_smul' := fun m x => by
 simp only [smul_smul]
 rfl
 isIntertwining' M := by
 refine LinearMap.ext fun x : ℝ => ?_
 simp only [LinearMap.coe_comp, Function.comp_apply]
 change x • preContrCoUnitVal d =
 (TensorProduct.map (ContrMod.rep M) (CoMod.rep M)) (x • preContrCoUnitVal d)
 simp only [map_smul]
 apply congrArg
 simp only [preContrCoUnitVal]
 rw [contrCoToMatrixRe_p_symm]
 apply congrArg
 simp

lemma preContrCoUnit_apply_one {d : ℕ} : (preContrCoUnit d) (1 : ℝ) = preContrCoUnit
def preCoContrUnitVal (d : ℕ := 3) : CoMod d ⊗[ℝ] ContrMod d :=
 Finset.sum_const_zero, add_zero, one_apply_eq, one_smul, zero_add]
 `□_ (Rep ℝ (LorentzGroup d)) □ CoMod.rep ⊗ ContrMod.rep`, manifesting the invarian
def preCoContrUnit (d : ℕ) : (Representation.trivial ℝ (LorentzGroup d) ℝ).Intertwin
 ((CoMod.rep).tprod (ContrMod.rep)) where
 toFun := fun a => a • preCoContrUnitVal d
 map_add' := fun x y => by
 simp only [add_smul]
 map_smul' := fun m x => by
 simp only [smul_smul]
 rfl
 isIntertwining' M := by
 refine LinearMap.ext fun x : ℝ => ?_
 simp only [LinearMap.coe_comp, Function.comp_apply]
 change x • preCoContrUnitVal d =
 (TensorProduct.map (CoMod.rep M) (ContrMod.rep M)) (x • preCoContrUnitVal d)
 simp only [map_smul]
 apply congrArg
 simp only [preCoContrUnitVal]
 rw [coContrToMatrixRe_p_symm]

```

```

 apply congrArg
 symm
 refine transpose_eq_one.mp ?h.h.h.a
 simp

lemma preCoContrUnit_apply_one {d : ℕ} : (preCoContrUnit d) (1 : ℝ) = preCo
lemma contr_preContrCoUnit {d : ℕ} (x : CoMod d) :
 (TensorProduct.lid ℝ _ <|
 coContrContract.toLinearMap.rTensor _ <|
 (TensorProduct.assoc ℝ (CoMod d) (ContrMod d) (CoMod d)).symm <|
 x ⊗ₜ[ℝ] (preContrCoUnit d (1 : ℝ))) = x := by
 have h1 : (TensorProduct.assoc ℝ (CoMod d) (ContrMod d) (CoMod d)).symm
 (x ⊗ₜ[ℝ] (preContrCoUnit d) (1 : ℝ))
 simp only [Fintype.sum_sum_type, Finset.univ_unique, Fin.default_eq_zero,
 Finset.sum_singleton, map_add, map_sum]
 have h2 : coContrContract.toLinearMap.rTensor _ (∑ i, (x ⊗ₜ[ℝ] contrBasis d i))
 = ∑ i, ((coContrContract) (x ⊗ₜ[ℝ] contrBasis d i)) ⊗ₜ[ℝ] coBasis d i
 erw [h2]
 have h3 (i : Fin 1 ⊕ Fin d) : (coContrContract)
lemma contr_preCoContrUnit {d : ℕ} (x : ContrMod d) :
 (TensorProduct.lid ℝ _ <|
 contrCoContract.toLinearMap.rTensor _ <|
 (TensorProduct.assoc ℝ (ContrMod d) (CoMod d) (ContrMod d)).symm <|
 x ⊗ₜ[ℝ] (preCoContrUnit d (1 : ℝ))) = x := by
 have h1 : ((TensorProduct.assoc ℝ (ContrMod d) (CoMod d) (ContrMod d)).sy
 (x ⊗ₜ[ℝ] (preCoContrUnit d) (1 : ℝ)))
 simp only [Fintype.sum_sum_type, Finset.univ_unique, Fin.default_eq_zero,
 Finset.sum_singleton, map_add, map_sum]
 have h2 : contrCoContract.toLinearMap.rTensor _ (∑ i, (x ⊗ₜ[ℝ] coBasis d i))
 = ∑ i, ((contrCoContract) (x ⊗ₜ[ℝ] coBasis d i)) ⊗ₜ[ℝ] contrBasis d i
 erw [h2]
 have h3 (i : Fin 1 ⊕ Fin d) : (contrCoContract)
 simp only [sum_tmul, smul_tmul, tmul_smul, map_sum, map_smul, smul_eq
 (preContrCoUnit d) (1 : ℝ) = LinearMap.lTensor _ (LinearEquiv.cast (by
 (TensorProduct.comm ℝ _ _ ((preCoContrUnit d) (1 : ℝ))) := by
 (preCoContrUnit d) (1 : ℝ) = LinearMap.lTensor _ (LinearEquiv.cast (by
 (TensorProduct.comm ℝ _ _ ((preContrCoUnit d) (1 : ℝ))) := by

```

### 1.244 Physlib/Relativity/Tensors/RealTensor/Vector/Basic.lean

```

public import Mathlib.Analysis.InnerProductSpace.PiL2
open Module

@[ext]

```



1.245.

*PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/VECTOR/CAUSALITY/LIGHTLIKE.LEAN*

```
lemma eq_of_apply_eq {d : ℕ} {v w : Vector d} (h : ∀ i, v i = w i) : v = w := by
 apply (equivEuclid d).injective
 ext i
 simp using h i

 rw [← neg_add_eq_zero, h]
 simp only [ContinuousLinearMap.fderiv, ContinuousLinearMap.coe_comp, Function.comp]
 /- The continuous linear map corresponding to the creation of a
 Lorentz Vector with only non-zero spatial components. -
 /
def ofSpatialComponent {d : ℕ} : EuclideanSpace ℝ (Fin d) → L[ℝ] Vector d where
 toFun xs := ∑ i, xs i • basis (Sum.inr i)
 map_add' xs ys := by
 simp [add_smul, Finset.sum_add_distrib]
 map_smul' c xs := by
 simp [smul_smul, Finset.smul_sum]
```

## 1.245 Physlib/Relativity/Tensors/RealTensor/Vector/Causality/LightLike

```
split_ifs with h h2 <=> simp_all
simp [causallyPrecedes, pastLightConeBoundary]
rw [show v (Sum.inl 0) = w (Sum.inl 0) from h_time] at hv
linarith
set_option linter.unusedVariables false in
 refine {r, ?_}
 rw [hr]
 simp [abs_mul]
```

## 1.246 Physlib/Relativity/Tensors/RealTensor/Vector/MinkowskiProduct.lean

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.Tensorial
open Module
 rw [toField_eq_repr, contrT_basis_repr_apply_eq_fin]
 rw [prodT_basis_repr_apply, contrT_basis_repr_apply_eq_fin]
 simp only [tensor_basis_repr_toTensor_apply]
 change ∑ x, (∑ y, minkowskiMatrix y x * p y) * q x = _
 enter [2, x, 1]
 rw [Finset.sum_eq_single x
 (fun y _ h => by rw [minkowskiMatrix.off_diag_zero h, zero_mul]) (by simp)]
 Finset.sum_singleton, minkowskiMatrix.inl_0_inl_0, one_mul, minkowskiMatrix.inr_
 neg_mul, Finset.sum_neg_distrib]
ring
```

```

simp only [minkowskiProductMap_toCoord, mul_comm]
simp only [minkowskiProductMap_toCoord, apply_add, add_mul, Finset.sum_ap
simp only [minkowskiProductMap_toCoord, apply_smul, mul_sub, Finset.mul_s
simp [PiLp.inner_apply, conj_trivial, mul_comm]
 rw [← minkowskiMatrix.mul_η_diag_eq_iff (μ := μ), ← minkowskiProduct_ba
 ext1 p
simp only [map_apply_eq_basis_mulVec, adjoint, LinearMap.toMatrix_symm,
 LinearMap.toMatrix_toLin, mulVec_eq_sum, op_smul_eq_smul, Finset.sum_ap
 transpose_apply, smul_eq_mul, minkowskiMatrix.dual_apply, Finset.mul_su
refine Finset.sum_congr rfl fun μ _ => Finset.sum_congr rfl fun v _ =>?
ring_nf
rw [minkowskiMatrix.η_apply_sq_eq_one]
have hp : p = ∑ μ, p μ • basis μ := (Basis.sum_repr basis p).symm
have hq : q = ∑ v, q v • basis v := (Basis.sum_repr basis q).symm

```

### 1.247 Physlib/Relativity/Tensors/RealTensor/Vector/Pre/Basic.le

```

def contrBasis (d : ℕ := 3) : Basis (Fin 1 ⊕ Fin d) ℝ (ContrMod d) :=
 (LinearMap.toMatrix (contrBasis d) (contrBasis d)) (ContrMod.rep M) i j
lemma contrBasis_repr_apply {d : ℕ} (p : ContrMod d) (i : Fin 1 ⊕ Fin d) :
def contrBasisFin (d : ℕ := 3) : Basis (Fin (1 + d)) ℝ (ContrMod d) :=
lemma contrBasisFin_repr_apply {d : ℕ} (p : ContrMod d) (i : Fin (1 + d)) :
lemma continuous_contr {T : Type} [TopologicalSpace T] (f : T → ContrMod d)
lemma contr_continuous {T : Type} [TopologicalSpace T] (f : ContrMod d → T)
def coBasis (d : ℕ := 3) : Basis (Fin 1 ⊕ Fin d) ℝ (CoMod d) :=
 (LinearMap.toMatrix (coBasis d) (coBasis d)) (CoMod.rep M) i j =
lemma coBasis_repr_apply {d : ℕ} (p : CoMod d) (i : Fin 1 ⊕ Fin d) :
def coBasisFin (d : ℕ := 3) : Basis (Fin (1 + d)) ℝ (CoMod d) :=
lemma coBasisFin_repr_apply {d : ℕ} (p : CoMod d) (i : Fin (1 + d)) :
open Representation
/-- The morphism of representations from `ContrMod.rep` to `CoMod.rep` defini
def Contr.toCo (d : ℕ) : IntertwiningMap (ContrMod.rep (d := d)) (CoMod.rep
 toFun := fun ψ => CoMod.toFinldREquiv.symm (η *v ψ.toFinldℝ)
 map_add' := by
 intro ψ ψ'
 simp only [map_add, mulVec_add]
 map_smul' := by
 intro r ψ
 simp only [_root_.map_smul, mulVec_smul, RingHom.id_apply]
 isIntertwining' g := by
 ext1 ψ
 conv_lhs =>
 change CoMod.toFinldREquiv.symm (η *v (g.1 *v ψ.toFinldℝ))
 rw [mulVec_mulVec, LorentzGroup.minkowskiMatrix_comm, ← mulVec_mulVec

```

1.248.

PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/VECTOR/PRE/CONTRACTION. LEAN

rfl

/-- The morphism of representations from `CoMod.rep` to `ContrMod.rep` defined by multiplication with the metric. -/

```
def Co.toContr (d : ℕ) : IntertwiningMap (CoMod.rep (d := d)) (ContrMod.rep (d := d))
 toFun := fun ψ => ContrMod.toFinldREquiv.symm (η *v ψ.toFinldR)
```

```
 change ContrMod.toFinldREquiv.symm (η *v ((LorentzGroup.transpose g-1).1 *v
```

```
 rw [mulVec_mulVec, ← LorentzGroup.comm_minkowskiMatrix, ← mulVec_mulVec]
```

/-- The isomorphism between `ContrMod.rep` and `CoMod.rep` induced by multiplication

```
def contrIsoCo (d : ℕ) : Representation.Equiv (ContrMod.rep (d := d)) (CoMod.rep (d := d)) :=
 refine Representation.Equiv.mk' (Contr.toCo d) (Co.toContr d) ?_ ?_
```

```
· intro x
```

```
 simp [Contr.toCo, Co.toContr]
```

```
· intro x
```

```
 simp [Contr.toCo, Co.toContr]
```

```
 ContrMod.rep ∧ (ContrMod.stdBasis μ) = ∑ j, ∧.1 j μ • ContrMod.stdBasis j := by
```

## 1.248 Physlib/Relativity/Tensors/RealTensor/Vector/Pre/Contraction.lean

/-- The linear map from ContrMod d ⊗ CoMod d to ℝ given by

```
def contrCoContract : ((ContrMod.rep).tprod (CoMod.rep)).IntertwiningMap
```

```
 (Representation.trivial ℝ (LorentzGroup d) ℝ) where
```

```
 toLinearMap := TensorProduct.lift (contrModCoModBi d)
```

```
 isIntertwining' ∧ := by
```

```
 ext ψ φ
```

```
 simp only [Representation.tprod_apply, AlgebraTensorModule.curry_apply,
```

```
 LinearMap.restrictScalars_self, curry_apply, LinearMap.coe_comp, Function.comp
```

```
 map_tmul, lift_tmul, Representation.isTrivial_def, LinearMap.id_comp]
```

```
 change (∧.1 *v ψ.toFinldR) □v ((LorentzGroup.transpose ∧-1).1 *v φ.toFinldR) =
```

```
 rw [dotProduct_mulVec, LorentzGroup.transpose_val,
```

```
 vecMul_transpose, mulVec_mulVec, LorentzGroup.coe_inv, inv_mul_of_invertible ∧
```

```
 simp only [one_mulVec]
```

```
 rfl
```

```
local notation "□" ψ "," φ "□m" => contrCoContract (ψ ⊗t φ)
```

```
lemma contrCoContract_hom_tmul (ψ : ContrMod d) (φ : CoMod d) :
```

```
 □ψ, φ□m = ψ.toFinldR □v φ.toFinldR := by
```

/-- The linear map from CoMod d ⊗ ContrMod d to ℝ given by

```
def coContrContract : ((CoMod.rep (d := d)).tprod (ContrMod.rep (d := d))).IntertwiningMap
```

```
 (Representation.trivial ℝ (LorentzGroup d) ℝ) where
```

```
 toLinearMap := TensorProduct.lift (coModContrModBi d)
```

```
 isIntertwining' ∧ := by
```

```
 ext ψ φ
```

```
 change ((LorentzGroup.transpose ∧-1).1 *v ψ.toFinldR) □v (∧.1 *v φ.toFinldR) =
```

```
 rw [dotProduct_mulVec, LorentzGroup.transpose_val, mulVec_transpose, vecMul_vecM
```

```

 LorentzGroup.coe_inv, inv_mul_of_invertible Λ .1]
 simp only [vecMul_one]
 rfl
local notation " φ ", " ψ " $\square_m$ => coContrContract ($\varphi \otimes_t \psi$)
lemma coContrContract_hom_tmul (φ : CoMod d) (ψ : ContrMod d) :
 $\square\varphi, \psi\square_m = \varphi.toFinldR \square_v \psi.toFinldR := by$
lemma contrCoContract_tmul_symm (φ : ContrMod d) (ψ : CoMod d) : $\square\varphi, \psi\square_m =$
lemma coContrContract_tmul_symm (φ : CoMod d) (ψ : ContrMod d) : $\square\varphi, \psi\square_m =$
/-- The linear map from ContrMod d $\otimes$ ContrMod d to $\mathbb{R}$ induced by the homomorphism
def contrContrContract : ((ContrMod.rep (d := d)).tprod (ContrMod.rep (d := d))
 (Representation.trivial $\mathbb{R}$ (LorentzGroup d) $\mathbb{R}$) := contrCoContract.comp
 ((Contr.toCo d).lTensor (ContrMod.rep (d := d))))
/-- The linear map from ContrMod d $\otimes$ ContrMod d to $\mathbb{R}$ induced by the homomorphism
def contrContrContractField : ContrMod d $\otimes[\mathbb{R}]$ ContrMod d $\rightarrow[\mathbb{R}]$ $\mathbb{R} :=$
 contrContrContract.toLinearMap
lemma contrContrContract_hom_tmul (φ : ContrMod d) (ψ : ContrMod d) :
/- The linear map from CoMod d $\otimes$ CoMod d to $\mathbb{R}$ induced by the homomorphism
def coCoContract : ((CoMod.rep (d := d)).tprod (CoMod.rep (d := d))).Intertwining
 (Representation.trivial $\mathbb{R}$ (LorentzGroup d) $\mathbb{R}$) := coContrContract.comp
 ((Co.toContr d).lTensor (CoMod.rep (d := d)))
local notation " ψ ", " φ " $\square_m$ => coCoContract ($\psi \otimes_t \varphi$)
lemma coCoContract_hom_tmul (φ : CoMod d) (ψ : CoMod d) :
 $\square\varphi, \psi\square_m = \varphi.toFinldR \square_v \eta * \psi.toFinldR := by$ rfl
variable (x y : ContrMod d)
lemma action_tmul (g : LorentzGroup d) : $\square$ ContrMod.rep g x, ContrMod.rep g
 LinearMap.congr_fun (contrContrContract.isIntertwining' g) (x $\otimes_t[\mathbb{R}]$ y)
lemma right_parity : $\square$ x, ContrMod.rep LorentzGroup.parity y $\square_m = \sum i, x.val$
 trans x.val (Sum.inl 0) * ((ContrMod.rep LorentzGroup.parity) y).val (Sum
 $\sum i : \text{Fin } d, - (x.val (\text{Sum.inr } i) * ((ContrMod.rep LorentzGroup.parity)$
lemma self_parity_eq_zero_iff : $\square$ y, ContrMod.rep LorentzGroup.parity y $\square_m =$
 have h1 := congrFun hn i
 simp only [Pi.zero_apply, mul_eq_zero, or_self] at h1
 simp only [h1]
 rfl
lemma nondegenerate : ($\forall (x : \text{ContrMod } d), \square x, y\square_m = 0) \leftrightarrow y = 0 := by$
lemma matrix_eq_iff_eq_forall' : ($\forall (v : \text{ContrMod } d), (\Lambda *_{\square} v) = \Lambda' *_{\square} v) \leftrightarrow$
 $\forall (w v : \text{ContrMod } d), \square v, \Lambda *_{\square} w\square_m = \square v, \Lambda' *_{\square} w\square_m := by$
lemma matrix_eq_iff_eq_forall : $\Lambda = \Lambda' \leftrightarrow \forall (w v : \text{ContrMod } d), \square v, \Lambda *_{\square} w\square_m =$
lemma matrix_eq_id_iff : $\Lambda = 1 \leftrightarrow \forall (w v : \text{ContrMod } d), \square v, \Lambda *_{\square} w\square_m = \square v, w\square_m$
 $\forall (w v : \text{ContrMod } d), \square \Lambda *_{\square} v, \Lambda *_{\square} w\square_m = \square v, w\square_m := by$
 $\forall (w : \text{ContrMod } d), \square \Lambda *_{\square} w, \Lambda *_{\square} w\square_m = \square w, w\square_m := by$
lemma inl_sq_eq (v : ContrMod d) : v.val (Sum.inl 0) $^2 =$
lemma le_inl_sq (v : ContrMod d) : $\square v, v\square_m \leq v.val (\text{Sum.inl } 0) ^ 2 := by$
lemma ge_abs_inner_product (v w : ContrMod d) : v.val (Sum.inl 0) * w.val (Sum.inl 0)
lemma ge_sub_norm (v w : ContrMod d) : v.val (Sum.inl 0) * w.val (Sum.inl 0)
lemma basis_left {v : ContrMod d} ($\mu : \text{Fin } 1 \otimes \text{Fin } d$) :
```

1.249.

PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/VECTOR/REPRESENTATION. LEAN

```
simp only [Fin.isValue, PiLp.inner_apply, Fin.sum_univ_three, ofReal_sub, ofReal_m
smul_cons, smul_zero, real_smul, mul_one, smul_empty, smul_neg, Matrix.sub_apply
Matrix.smul_apply, one_apply_eq, of_apply, cons_val', cons_val_zero, cons_val_fi
cons_val_one, sub_neg_eq_add, ne_eq, zero_ne_one, not_false_eq_true, one_apply_n
one_ne_zero]
simp only [Fin.isValue, Function.comp_apply, inner_self_eq_norm_sq_to_K, Real.norm
RCLike.ofReal_real_eq_id, id_eq, sq_abs, ofReal_add, ofReal_pow, I_sq, mul_neg,
contrCoContract (contrBasis d i $\otimes_t$ coBasis d j) = if i = j then (1 : $\mathbb{R}$) else 0 :
coContrContract (coBasis d i $\otimes_t$ [$\mathbb{R}$] contrBasis d j) = if i = j then (1 : $\mathbb{R}$) else
```

## 1.249 Physlib/Relativity/Tensors/RealTensor/Vector/Representation.lean

/-

Copyright (c) 2026 Joseph Tooby-Smith. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Joseph Tooby-Smith

-/

module

public import Mathlib.RepresentationTheory.Basic

public import Physlib.Relativity.LorentzGroup.Basic

public import Physlib.Relativity.Tensors.RealTensor.Vector.Basic

/-!

# Representation of the Lorentz group on Lorentz vectors

In this module we define the representation of the Lorentz group on Lorentz vectors.  
This does not define the MulAction on `Lorentz.Vector`, which is induced  
by its tensor structure.

-/

@[expose] public section

open Module Matrix MatrixGroups Complex TensorProduct

noncomputable section

namespace Lorentz

namespace Vector

attribute [-simp] Fintype.sum\_sum\_type

```

/-- The representation of the Lorentz group on Lorentz vectors. -
/
def rep {d : ℕ} : Representation ℝ (LorentzGroup d) (Vector d) where
 toFun Λ := Matrix.toLinAlgEquiv basis Λ
 map_one' := EmbeddingLike.map_eq_one_iff.mpr rfl
 map_mul' x y := by simp only [lorentzGroupIsGroup_mul_coe, _root_.map_mul]

/-!

Properties of the representation.

-/

lemma rep_apply_eq_mulVec (d : ℕ) (Λ : LorentzGroup d) (v : Vector d) :
 rep Λ v = Λ *v v := by rfl

lemma rep_apply_eq_sum (d : ℕ) (Λ : LorentzGroup d) (v : Vector d) (k : Fin d) :
 rep Λ v k = ∑ j, Λ.1 k j • v j := rfl

lemma rep_apply_basis {d} (μ : Fin 1 ⊕ Fin d) (Λ : LorentzGroup d) :
 rep Λ (basis μ) = ∑ j, Λ.1 j μ • basis j := by
 ext k
 simp [rep_apply_eq_sum, apply_sum]

lemma rep_toMatrix (d : ℕ) (Λ : LorentzGroup d) :
 LinearMap.toMatrix basis basis (rep Λ) = Λ.1 := by
 simp only [rep, MonoidHom.coe_mk, OneHom.coe_mk]
 exact (LinearEquiv.eq_symm_apply (LinearMap.toMatrix basis basis)).mp rfl

lemma rep_injective (d : ℕ) (Λ : LorentzGroup d) : Function.Injective (rep
 intro v1 v2 h
 rw [rep_apply_eq_mulVec, rep_apply_eq_mulVec] at h
 exact Matrix.mulVec_injective_of_isUnit (isUnit_of_invertible Λ.1) h

lemma rep_surjective (d : ℕ) (Λ : LorentzGroup d) : Function.Surjective (rep
 intro v
 use Λ-1 *v v
 rw [rep_apply_eq_mulVec]
 simp

lemma rep_bijective (d : ℕ) (Λ : LorentzGroup d) : Function.Bijective (rep
 ⟨rep_injective d Λ, rep_surjective d Λ⟩

@[fun_prop]
lemma rep_contDiff (d : ℕ) {n} (Λ : LorentzGroup d) : ContDiff ℝ n (rep Λ)
 refine (contDiff_apply ↑(rep Λ)).mp ?_

```

1.250.

*PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/VECTOR/TENSORIAL* 319 ~~LEAN~~

```
 intro μ
 simp only [rep_apply_eq_sum, smul_eq_mul]
 fun_prop

lemma rep_left_injective (d : ℕ) : Function.Injective (rep (d := d)) := by
 intro Λ Λ' h
 apply LorentzGroup.eq_of_mulVec_eq
 intro v
 rw [← rep_apply_eq_mulVec, ← rep_apply_eq_mulVec]
 exact LinearMap.congr_fun h v

end Vector

end Lorentz
```

## 1.250 Physlib/Relativity/Tensors/RealTensor/Vector/Tensorial.lean

```
/-
Copyright (c) 2026 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Matteo Cipollina, Joseph Tooby-Smith
-/
module

public import Physlib.Relativity.Tensors.RealTensor.Basic
public import Physlib.Relativity.Tensors.RealTensor.Vector.Basic
/-!

Tensorial nature of Lorentz vectors

We define the tensorial instance on `Lorentz.Vector`, and show
prove properties related to the Lorentz group action and the basis.

-/

@[expose] public section

open Module
open Matrix
open MatrixGroups
open Complex
open TensorProduct
```

noncomputable section

namespace Lorentz  
open realLorentzTensor

namespace Vector

open TensorSpecies  
open Tensor

/-!

## Tensorial

-/

/-- The equivalence between the type of indices of a Lorentz vector and  
`Fin 1  $\oplus$  Fin d`. -/

```
def indexEquiv {d : ℕ} :
 ComponentIdx (S := (realLorentzTensor d)) ![Color.up] $\approx$ Fin 1 $\oplus$ Fin d :
 ComponentIdx.single (S := realLorentzTensor d) (c := Color.up)
```

```
instance tensorial {d : ℕ} : Tensorial (realLorentzTensor d) ![:up] (Vector
 toTensor := LinearEquiv.symm <|
 Equiv.toLinearEquiv
 ((Tensor.basis (S := (realLorentzTensor d)) ![:up])).repr.toEquiv.trans
 Finsupp.equivFunOnFinite.trans <|
 (Equiv.piCongrLeft' _ indexEquiv))
 { map_add := fun x y => by
 simp [map_add]
 rfl
 map_smul := fun c x => by
 simp [_root_.map_smul]
 rfl }
```

open Tensorial

```
lemma toTensor_symm_apply {d : ℕ} (p : RT[d, .up]) :
 (toTensor (self := tensorial)).symm p =
 (Equiv.piCongrLeft' _ indexEquiv <|
 Finsupp.equivFunOnFinite <|
 (Tensor.basis (S := (realLorentzTensor d)) _).repr p) := rfl
```

```
lemma toTensor_symm_pure {d : ℕ} (p : Pure (realLorentzTensor d) ![:up]) (i :
 (toTensor (self := tensorial)).symm p.toTensor i =
 ((Lorentz.contrBasis d).repr (p 0)) (indexEquiv.symm i 0) := by
```



1.250.

PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/VECTOR/TENSORIAL 3&EAN

```
rw [toTensor_symm_apply]
simp only [Equiv.piCongrLeft'_apply, Finsupp.equivFunOnFinite_apply, Tensor.basis_
 Pure.component, Finset.univ_unique, Fin.default_eq_zero, Finset.prod_singleton,
 rfl]

/-!

Basis

-/

set_option backward.isDefEq.respectTransparency false in
lemma toTensor_symm_basis {d : ℕ} (μ : Fin 1 ⊗ Fin d) :
 (toTensor (self := tensorial)).symm (Tensor.basis ![Color.up] (indexEquiv.symm μ)
 basis μ := by
 funext i
 simp [Tensor.basis_apply, toTensor_symm_pure, Pure.basisVector, Finsupp.single_app
 indexEquiv]

lemma toTensor_basis_eq_tensor_basis {d : ℕ} (μ : Fin 1 ⊗ Fin d) :
 toTensor (basis μ) = Tensor.basis ![Color.up] (indexEquiv.symm μ) := by
 rw [← toTensor_symm_basis]
 simp

lemma basis_eq_map_tensor_basis {d} : basis =
 ((Tensor.basis
 (S := realLorentzTensor d) ![Color.up])).map toTensor.symm).reindex indexEquiv :=
 ext μ
 rw [← toTensor_symm_basis]
 simp

lemma tensor_basis_map_eq_basis_reindex {d} :
 (Tensor.basis (S := realLorentzTensor d) ![Color.up])).map toTensor.symm =
 basis.reindex indexEquiv.symm := by
 rw [basis_eq_map_tensor_basis]
 ext μ
 simp

lemma tensor_basis_repr_toTensor_apply {d : ℕ} (p : Vector d) (μ : ComponentIdx ![Co
 (Tensor.basis ![Color.up])).repr (toTensor p) μ =
 p (indexEquiv μ) := by
 obtain ⟨p, rfl⟩ := toTensor.symm.surjective p
 simp only [Nat.succ_eq_add_one, Nat.reduceAdd, LinearEquiv.apply_symm_apply]
 apply induction_on_pure (t := p)
 · intro p
 simp [Tensor.basis_repr_pure, toTensor_symm_pure, Pure.component]
```

```

 rfl
 · intro r t h
 simp [h]
 · intro t1 t2 h1 h2
 simp [h1, h2]

/-!

The action of the Lorentz group

-/

set_option backward.isDefEq.respectTransparency false in
lemma smul_eq_sum {d : ℕ} (i : Fin 1 ⊕ Fin d) (Λ : LorentzGroup d) (p : Vector d) :
 (Λ • p) i = ∑ j, Λ.1 i j * p j := by
 obtain ⟨p, rfl⟩ := toTensor.symm.surjective p
 rw [smul_toTensor_symm]
 apply induction_on_pure (t := p)
 · intro p
 have toTensor_symm_pure_val : ∀ (q : Pure (realLorentzTensor d) ![:up])
 (toTensor (self := tensorial)).symm q.toTensor j = (q 0).val j := fun
 rw [toTensor_symm_pure, contrBasis_repr_apply]
 rfl
 rw [actionT_pure]
 simp only [toTensor_symm_pure_val]
 show (Λ.1 *v (p 0)).val i = ∑ j, Λ.1 i j * (p 0).val j
 rw [ContrMod.mulVec_val, mulVec_eq_sum]
 simp only [Finset.sum_apply, Pi.smul_apply, transpose_apply,
 MulOpposite.smul_eq_mul_unop, MulOpposite.unop_op]
 · intro r t h
 simp only [actionT_smul, _root_.map_smul, apply_smul, h, Finset.mul_sum]
 · intro t1 t2 h1 h2
 simp only [actionT_add, map_add, apply_add, h1, h2, mul_add, Finset.sum]

lemma smul_eq_mulVec {d} (Λ : LorentzGroup d) (p : Vector d) :
 Λ • p = Λ.1 *v p := by
 funext i
 rw [smul_eq_sum, mulVec_eq_sum]
 simp only [op_smul_eq_smul, Finset.sum_apply, Pi.smul_apply, transpose_apply,
 mul_comm]

lemma smul_add {d : ℕ} (Λ : LorentzGroup d) (p q : Vector d) :
 Λ • (p + q) = Λ • p + Λ • q := by simp

set_option backward.isDefEq.respectTransparency false in
@[simp]

```

1.250.

PHYSLIB/RELATIVITY/TENSORS/REALTENSOR/VECTOR/TENSORIAL3.BAN

```
lemma smul_sub {d : ℕ} (Λ : LorentzGroup d) (p q : Vector d) :
 Λ • (p - q) = Λ • p - Λ • q := by
 rw [smul_eq_mulVec, smul_eq_mulVec, smul_eq_mulVec, Matrix.mulVec_sub]

set_option backward.isDefEq.respectTransparency false in
lemma smul_zero {d : ℕ} (Λ : LorentzGroup d) :
 Λ • (0 : Vector d) = 0 := by
 rw [smul_eq_mulVec, Matrix.mulVec_zero]

set_option backward.isDefEq.respectTransparency false in
lemma smul_neg {d : ℕ} (Λ : LorentzGroup d) (p : Vector d) :
 Λ • (-p) = - (Λ • p) := by
 rw [smul_eq_mulVec, smul_eq_mulVec, Matrix.mulVec_neg]

lemma neg_smul {d} (Λ : LorentzGroup d) (p : Vector d) :
 (-Λ) • p = - (Λ • p) := by
 funext i
 simp [smul_eq_sum, neg_apply]

lemma _root_.LorentzGroup.eq_of_action_vector_eq {d : ℕ}
 {Λ Λ' : LorentzGroup d} (h : ∀ p : Vector d, Λ • p = Λ' • p) :
 Λ = Λ' := by
 apply LorentzGroup.eq_of_mulVec_eq
 simp_all [smul_eq_mulVec]

/-!

B. The continuous action of the Lorentz group

-/

/-- The Lorentz action on vectors as a continuous linear map. -
/
def actionCLM {d : ℕ} (Λ : LorentzGroup d) :
 Vector d → L[ℝ] Vector d :=
 LinearMap.toContinuousLinearMap
 { toFun := fun v => Λ • v
 map_add' := smul_add Λ
 map_smul' := fun c v => by
 simp only [RingHom.id_apply]
 funext i
 simp only [smul_eq_sum, apply_smul, Finset.mul_sum, mul_left_comm]}

lemma actionCLM_apply {d : ℕ} (Λ : LorentzGroup d) (p : Vector d) :
 actionCLM Λ p = Λ • p := rfl
```

```

lemma actionCLM_injective {d : ℕ} (Λ : LorentzGroup d) :
 Function.Injective (actionCLM Λ) := by
 intro x1 x2
 simp [actionCLM_apply]

lemma actionCLM_surjective {d : ℕ} (Λ : LorentzGroup d) :
 Function.Surjective (actionCLM Λ) := by
 intro x1
 use (actionCLM Λ⁻¹) x1
 simp [actionCLM_apply]

set_option backward.isDefEq.respectTransparency false in
lemma smul_basis {d : ℕ} (Λ : LorentzGroup d) (μ : Fin 1 ⊕ Fin d) :
 Λ • basis μ = ∑ v, Λ.1 v μ • basis v := by
 funext i
 rw [smul_eq_sum, Fintype.sum_apply]
 simp [basis_apply, Finset.sum_ite_eq, Finset.sum_ite_eq']

end Vector

end Lorentz

```

### 1.251 Physlib/Relativity/Tensors/RealTensor/Velocity/Basic.lean

```

 simp only [zero, Fin.isValue, map_add, map_smul, _root_.add_app
 FunLike.coe_smul, Pi.smul_apply, minkowskiProduct_basis_right]
 ext1

```

### 1.252 Physlib/Relativity/Tensors/Reindexing.lean

```

/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module

public import Physlib.Relativity.Tensors.ComponentIdx.Single
public import Physlib.Relativity.Tensors.Contraction.SuccSuccAbove
public import Mathlib.Topology.Algebra.Module.ModuleTopology
public import Mathlib.Analysis.RCLike.Basic
public import Mathlib.Tactic.Cases
/-!

```

# Reindexing of tensor components

In this file we give results related to the reindexing of tensors.

If a tensor has indices specified by a list of colors  $c : \text{Fin } n \rightarrow C$ , then reindexing a tensor corresponds to a bijection  $\sigma : \text{Fin } m \rightarrow \text{Fin } n$  such that  $c \circ \sigma = c1$  for some list of colors  $c1 : \text{Fin } m \rightarrow C$ . A reindexing might take a tensor  $\psi^i_j$  to a tensor  $\psi^j_i$  by reordering the indices, or it might take a tensor  $\psi^{i,j,k}$  to a tensor  $\psi^{i,k,j}$ .

We are interested in the interaction of reindexing with the following operations on

- `Fin.append` corresponds to the product of tensors.
- `Fin.succAbove` corresponds to the evaluation of a tensor at a given index.
- `Fin.succSuccAbove` corresponds to the contraction of a tensor at two given indices.

-/

@[expose] public section

open Module

namespace TensorSpecies

```
variable {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisIdx c) k (V c)}
 (S : TensorSpecies k C G V basisIdx rep b)
```

namespace Tensor

```
/-- Given two lists of indices `c : Fin n → C` and `c1 : Fin m → C` a map
 `σ : Fin m → Fin n` satisfies the condition `IsReindexing c c1 σ` if it is:
- A bijection
- Forms a commutative triangle with `c` and `c1`.
-/
```

```
def IsReindexing {n m : ℕ} (c : Fin n → C) (c1 : Fin m → C)
 (σ : Fin m → Fin n) : Prop :=
 Function.Bijective σ ∧ ∀ i, c (σ i) = c1 i
```

namespace IsReindexing

/-!

## Properties of the underlying function

-/

```

lemma injective {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) : Function.Injective σ :=

lemma surjective {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) : Function.Surjective σ :

lemma auto {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) := by {simp [IsReindexing]
 IsReindexing c c1 σ := h

@[simp]
lemma on_id {n : ℕ} {c c1 : Fin n → C} :
 IsReindexing c c1 (id : Fin n → Fin n) ↔ ∀ i, c i = c1 i := by
 simp [IsReindexing]

lemma on_id_symm {n : ℕ} {c c1 : Fin n → C} (h : IsReindexing c1 c id) :
 IsReindexing c c1 (id : Fin n → Fin n) := by
 simp at h ⊢
 exact fun i => (h i).symm

/-- For a map `σ` satisfying `IsReindexing c c1 σ`, the inverse of that map
/
def inv {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
 (σ : Fin m → Fin n) (h : IsReindexing c c1 σ) : Fin n → Fin m :=
 Fintype.bijInv h.1

/-- For a map `σ : Fin m → Fin n` satisfying `IsReindexing c c1 σ`,
that map lifted to an equivalence between
`Fin n` and `Fin m`. -/
def toEquiv {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) :
 Fin n ≈ Fin m where
 toFun := inv σ h
 invFun := σ
 left_inv := Fintype.rightInverse_bijInv h.1
 right_inv := Fintype.leftInverse_bijInv h.1

lemma apply_inv_apply {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
 (σ : Fin m → Fin n) (h : IsReindexing c c1 σ) (x : Fin m) :
 h.inv σ (σ x) = x := by
 change h.toEquiv (h.toEquiv.symm x) = x
 simp

```

```
lemma inv_apply_apply {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
 (σ : Fin m → Fin n) (h : IsReindexing c c1 σ) (x : Fin n) :
 σ (h.inv σ x) = x := by
 change h.toEquiv.symm (h.toEquiv x) = x
 simp
```

```
lemma preserve_color {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) :
 ∀ (x : Fin m), c1 x = (c ∘ σ) x := by
 intro x
 obtain ⟨y, rfl⟩ := h.toEquiv.surjective x
 simp only [Function.comp_apply]
 rw [h.2]
```

```
set_option warning.simp.varHead false in
@[simp, nolint simpVarHead]
lemma inv_perserve_color {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) (x : Fin n) :
 c1 (h.inv σ x) = c x := by
 obtain ⟨x, rfl⟩ := h.toEquiv.symm.surjective x
 change c1 (h.toEquiv _) = _
 simp only [Equiv.apply_symm_apply]
 rw [h.preserve_color]
 rfl
```

```
set_option warning.simp.varHead false in
@[simp, nolint simpVarHead]
lemma toEquiv_symm_perserve_color {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) (x : Fin m) :
 c (h.toEquiv.symm x) = c1 x := by
 obtain ⟨x, rfl⟩ := h.toEquiv.surjective x
 rw [h.preserve_color]
 rfl
```

```
/-!
```

```
Constructors
```

```
-/
open Fin
```

```
/-- The inverse of a map satisfying `IsReindexing c c1 σ` is a reindexing of `c1` by
/
```

```
lemma symm {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
 {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) : IsReindexing c1 c (h.inv σ) :=
 ⟨h.toEquiv.bijective, h.inv_perserve_color⟩
```

```

lemma symm_of_id {n : ℕ} {c c1 : Fin n → C} (h : IsReindexing c c1 id) :
 IsReindexing c1 c id := by
 simp at h ⊢
 exact fun i => (h i).symm

/-- The composition of two maps satisfying `IsReindexing` also satisfies th
/
lemma comp {n n1 n2 : ℕ} {c : Fin n → C} {c1 : Fin n1 → C}
 {c2 : Fin n2 → C} {σ : Fin n1 → Fin n} {σ2 : Fin n2 → Fin n1}
 (h : IsReindexing c c1 σ) (h2 : IsReindexing c1 c2 σ2) : IsReindexing c
 (h.1.comp h2.1, fun x => (h.2 (σ2 x)).trans (h2.2 x))

lemma append_congr_left {n n' n2 : ℕ} {c : Fin n → C} {c' : Fin n' → C}
 {σ : Fin n' → Fin n} (c2 : Fin n2 → C) (h : IsReindexing c c' σ) :
 IsReindexing (Fin.append c c2) (Fin.append c' c2)
 (Fin.append (Fin.castAdd n2 ∘ σ) (Fin.natAdd n)) := by
 refine {?_, fun i => ?_}
 · have heq : (Fin.append (Fin.castAdd n2 ∘ σ) (Fin.natAdd n)) : Fin (n' +
 ↑(finSumFinEquiv.symm.trans
 (((Equiv.ofBijective σ h.1).sumCongr (Equiv.refl (Fin n2))))).trans
 ext i
 refine Fin.addCases (fun a => ?_) (fun a => ?_) i <=>
 simp [Fin.append_left, Fin.append_right]
 rw [heq]
 exact Equiv.bijective _
 · refine Fin.addCases (fun a => ?_) (fun a => ?_) i <=>
 simp [Fin.append_left, Fin.append_right, h.2]

lemma append_congr_right {n n' n2 : ℕ} {c : Fin n → C} {c' : Fin n' → C}
 {σ : Fin n' → Fin n} (c2 : Fin n2 → C) (h : IsReindexing c c' σ) :
 IsReindexing (Fin.append c2 c) (Fin.append c2 c')
 (Fin.append (Fin.castAdd n) (Fin.natAdd n2 ∘ σ)) := by
 refine {?_, fun i => ?_}
 · have heq : (Fin.append (Fin.castAdd n) (Fin.natAdd n2 ∘ σ)) : Fin (n2 +
 ↑(finSumFinEquiv.symm.trans
 (((Equiv.refl (Fin n2)).sumCongr (Equiv.ofBijective σ h.1))))).trans
 ext i
 refine Fin.addCases (fun a => ?_) (fun a => ?_) i <=>
 simp [Fin.append_left, Fin.append_right]
 rw [heq]
 exact Equiv.bijective _
 · refine Fin.addCases (fun a => ?_) (fun a => ?_) i <=>
 simp [Fin.append_left, Fin.append_right, h.2]

lemma append_zero_right {n} {c : Fin n → C}

```



```

 {c1 : Fin 0 → C} : IsReindexing c (Fin.append c c1) id := by
 simp only [Nat.add_zero, IsReindexing.on_id]
 have P : ∀ (i : Fin (n + 0)), c i = Fin.append c c1 i := by
 rw [Fin.forall_fin_add]
 simp only [Fin.append_left, Fin.append_right, IsEmpty.forall_iff, and_true]
 simp only [Fin.castAdd_zero, Fin.cast_eq_self, implies_true]
 exact P

lemma append_swap {n n2 : ℕ} {c : Fin n → C} {c2 : Fin n2 → C} :
 IsReindexing (Fin.append c c2) (Fin.append c2 c)
 (Fin.append (Fin.natAdd n) (Fin.castAdd n2)) := by
 refine {?, fun i => ?_}
 · have heq : (Fin.append (Fin.natAdd n) (Fin.castAdd n2) : Fin (n2 + n) → Fin (n +
 ↑(finSumFinEquiv.symm.trans
 ((Equiv.sumComm (Fin n2) (Fin n)).trans finSumFinEquiv))) := by
 ext i
 refine Fin.addCases (fun a => ?_) (fun a => ?_) i <;>
 simp [Fin.append_left, Fin.append_right]
 rw [heq]
 exact Equiv.bijective _
 · refine Fin.addCases (fun a => ?_) (fun a => ?_) i <;>
 simp [Fin.append_left, Fin.append_right]

lemma append_assoc_right {n1 n2 n3 : ℕ} {c : Fin n1 → C} {c2 : Fin n2 → C} {c3 : F
 IsReindexing (Fin.append c (Fin.append c2 c3)) (Fin.append (Fin.append c c2) c3)
 (Fin.cast (by grind)) :=
 ((finCongr (by grind)).bijective, fun i => (congrFun (Fin.append_assoc c c2 c3) i))

lemma append_assoc_left {n1 n2 n3 : ℕ} {c : Fin n1 → C} {c2 : Fin n2 → C} {c3 : Fin
 IsReindexing (Fin.append (Fin.append c c2) c3) (Fin.append c (Fin.append c2 c3))
 (Fin.cast (by grind)) :=
 ((finCongr (by grind)).bijective, fun i => congrFun (Fin.append_assoc c c2 c3) _)

lemma append_succAbove_natAdd {n n1 : ℕ} {c : Fin n → C} {c1 : Fin (n1 + 1) → C}
 (i : Fin (n1 + 1)) :
 IsReindexing (Fin.append c c1 ∘ (Fin.natAdd n i).succAbove)
 (Fin.append c (c1 ∘ i.succAbove)) id := by
 refine {Function.bijective_id, fun x => ?_}
 simp only [Function.comp_apply, id_eq]
 refine Fin.addCases (fun a => ?_) (fun a => ?_) x
 · have hidx : (Fin.natAdd n i).succAbove (Fin.castAdd n1 a) = Fin.castAdd (n1 + 1)
 rw [Fin.succAbove_of_castSucc_lt]
 · ext
 simp
 · simp only [Fin.lt_def, Fin.val_castSucc, Fin.val_castAdd, Fin.val_natAdd]
 omega

```

```

 simp [hidx, Fin.append_left]
 · have hidx : (Fin.natAdd n i).succAbove (Fin.natAdd n a) = Fin.natAdd n
 have hcond : ((Fin.natAdd n a).castSucc < Fin.natAdd n i) ↔ (a.castSu
 simp only [Fin.lt_def, Fin.val_castSucc, Fin.val_natAdd]
 omega
 simp only [Fin.succAbove, hcond]
 split_ifs <;> ext <;> simp [Nat.add_assoc]
 simp [hidx, Fin.append_right]

lemma append_succAbove_castAdd {n n1 : ℕ} {c : Fin (n + 1) → C} {c1 : Fin (
 (i : Fin (n + 1)) :
 IsReindexing (Fin.append c c1 ◦ (Fin.castAdd (n1 + 1) i).succAbove)
 (Fin.append (c ◦ i.succAbove) c1) (Fin.cast (by grind))) := by
 refine ((finCongr (by grind)).bijective, fun y => ?_)
 simp only [Function.comp_apply]
 refine Fin.addCases (fun a => ?_) (fun a => ?_) y
 · have hidx : (Fin.castAdd (n1 + 1) i).succAbove (Fin.cast (by grind) (Fi
 = Fin.castAdd (n1 + 1) (i.succAbove a) := by
 have hcond : ((Fin.cast (by grind) (Fin.castAdd (n1 + 1) a)).castSucc
 Fin.castAdd (n1 + 1) i) ↔ (a.castSucc < i) := by
 simp only [Fin.lt_def, Fin.val_castSucc, Fin.val_cast, Fin.val_natAdd]
 simp only [Fin.succAbove, hcond]
 split_ifs <;> ext <;> simp
 simp [hidx, Fin.append_left]
 · have hidx : (Fin.castAdd (n1 + 1) i).succAbove (Fin.cast (by grind) (Fi
 = Fin.natAdd (n + 1) a := by
 rw [Fin.succAbove_of_le_castSucc]
 · ext
 simp [Nat.add_right_comm]
 · simp only [Fin.le_def, Fin.val_castSucc, Fin.val_cast, Fin.val_natAdd]
 omega
 simp [hidx, Fin.append_right]

/-- Removing two entries from the right component of `Fin.append c1 c` comm
 append: removing the `i`-th and `j`-th entries of `c` and then appending
 removing the corresponding entries of `Fin.append c1 c`, via the identity
 This is used for the commutation of taking a *product* of tensors
 with *contraction* of indices. -/
lemma append_succSuccAbove_natAdd {n n1 : ℕ} {c : Fin (n + 1 + 1) → C}
 {c1 : Fin n1 → C} (i j : Fin (n + 1 + 1)) :
 IsReindexing (Fin.append c1 c ◦ (Fin.natAdd n1 i).succSuccAbove (Fin.na
 (Fin.append c1 (c ◦ i.succSuccAbove j)) id := by
 apply And.intro (Function.bijective_id)
 simp [forall_fin_add, succSuccAbove_comm_natAdd i j, succSuccAbove_natAdd

/-- Given a reindexing of `c` by `c1` via `σ` for which the index `i` is se

```

```

removing the `i`-th entry of `c1` and the first entry of `c` yields a reindexing of
`c` by `c1` via the map sending `j` to the predecessor of
`σ (i.succAbove j)`. -/
lemma succAbove_of_eq_zero {n n1 : ℕ} {c : Fin (n + 1) → C} {c1 : Fin (n1 + 1) → C}
 {σ : Fin (n1 + 1) → Fin (n + 1)} (i : Fin (n1 + 1))
 (h : IsReindexing c c1 σ) (hi : σ i = 0) :
 IsReindexing (c ∘ Fin.succ) (c1 ∘ i.succAbove)
 (fun j => (σ (i.succAbove j)).pred (by simp [← hi, h.injective.eq_iff])) := by
 refine (⟨?, ?_⟩, ?_)
 · intro x1 x2 h1
 simp [h.injective.eq_iff] using h1
 · intro k
 suffices ha : ∃ a, σ (i.succAbove a) = k.succ by
 obtain ⟨a, ha⟩ := ha
 use a
 simp [ha]
 obtain ⟨j, hj⟩ := h.surjective k.succ
 simp only [← hj, h.injective.eq_iff, Fin.exists_succAbove_eq_iff, ne_eq]
 grind
 · intro x
 simp [h.preserve_color]

/-- Given a reindexing of `c` by `c1` via `σ` for which the index `i` is not sent to
removing the `i`-th entry of `c1` and the `(σ i)`-th entry of `c` yields a reindex
`c ∘ (σ i).succAbove` by `c1 ∘ i.succAbove` via the map
`(σ i).pred.predAbove ∘ σ ∘ i.succAbove`. -/
lemma succAbove_of_neq_zero {n n1 : ℕ} {c : Fin (n + 1) → C} {c1 : Fin (n1 + 1) → C}
 {σ : Fin (n1 + 1) → Fin (n + 1)} (i : Fin (n1 + 1))
 (h : IsReindexing c c1 σ) (hi : σ i ≠ 0) :
 IsReindexing (c ∘ (σ i).succAbove) (c1 ∘ i.succAbove)
 ((Fin.pred (σ i) hi).predAbove ∘ σ ∘ i.succAbove) := by
 have hpr : σ i = ((σ i).pred hi).succ := (Fin.succ_pred _ _).symm
 have hne : ∀ x, σ (i.succAbove x) ≠ σ i := fun x heq =>
 Fin.succAbove_ne i x (h.injective heq)
 refine (⟨?, ?_⟩, ?_)
 · intro x1 x2 h2
 simp only [Function.comp_apply] at h2
 apply i.succAbove_right_injective (h.injective ?_)
 suffices h' :
 ((σ i).pred hi).succ.succAbove ((σ i).pred hi).predAbove (σ (i.succAbove x1))
 ((σ i).pred hi).succ.succAbove ((σ i).pred hi).predAbove (σ (i.succAbove x2))
 rwa [Fin.succ_succAbove_predAbove (hpr ▸ hne x1),
 Fin.succ_succAbove_predAbove (hpr ▸ hne x2)] at h'
 simp using h2
 · intro k
 simp only [Function.comp_apply]

```

```

suffices h' : ∃ a, σ (i.succAbove a) = (σ i).succAbove k by
 conv => enter [1, a]; rw [← ((σ i).pred hi).succ.succAbove_right_inj]
 obtain ⟨a, h'⟩ := h'
 exact ⟨a, by rw [Fin.succ_succAbove_predAbove (hpr ▸ hne a), ← hpr]⟩
obtain ⟨j, hj⟩ := h.surjective ((σ i).succAbove k)
simp only [← hj, h.injective.eq_iff, Fin.exists_succAbove_eq_iff, ne_eq]
rintro rfl
simp at hj
· intro x
 simp only [h.preserve_color, Function.comp_apply]
 congr 1
 conv_lhs => enter[1]; rw [hpr]
 exact Fin.succ_succAbove_predAbove (hpr ▸ hne x)

/-- Given a reindexing of `c` by `c1` via `σ`, removing the `i`-th
entry of `c1` and the
 `(σ i)`-th entry of `c` yields a reindexing of `c ∘ (σ i).succAbove` by `c1`
This unifies `succAbove_of_eq_zero` and `succAbove_of_neq_zero` via a case
whether `σ i = 0`. This is used for the commutation of *permutation*
of indices with *evaluation* of indices. -/
lemma succAbove {n n1 : ℕ} {c : Fin (n + 1) → C} {c1 : Fin (n1 + 1) → C}
 {σ : Fin (n1 + 1) → Fin (n + 1)} (i : Fin (n1 + 1))
 (h : IsReindexing c c1 σ) :
 IsReindexing (c ∘ (σ i).succAbove) (c1 ∘ i.succAbove)
 (if hi : σ i = 0 then fun j => (σ (i.succAbove j)).pred (by simp [← h])
 else (Fin.pred (σ i) hi).predAbove ∘ σ ∘ i.succAbove) := by
 by_cases hi : σ i = 0
 · simp [hi] using IsReindexing.succAbove_of_eq_zero i h hi
 · simp [hi] using IsReindexing.succAbove_of_neq_zero i h hi

/-- Given a reindexing of `c` by `c1` via `σ` and two distinct indices `i` and `j`
 `i`-th and `j`-th entries of `c1` and the `(σ i)`-th and `(σ j)`-th
entries of `c` yields a
 reindexing of `c ∘ (σ i).succAbove (σ j)` by `c1 ∘ i.succAbove j`
This is used for the commutation of *permutation* of indices with
contraction of indices. -/
lemma succSuccAbove {n n1 : ℕ} {c : Fin (n + 1 + 1) → C}
 {c1 : Fin (n1 + 1 + 1) → C}
 (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j)
 {σ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)} (hσ : IsReindexing c c1 σ) :
 IsReindexing (c ∘ (σ i).succAbove (σ j))
 (c1 ∘ i.succAbove j) (i.funPredPredAbove j hij σ hσ.1) := by
 apply And.intro
 · exact Fin.funPredPredAbove_bijective i j hij σ hσ.left
 · intro m
 simp [Fin.funPredPredAbove, hσ.2]

```

```

/-- Removing two pairs of entries from `c` in either order gives the same colour list
 removing the `i1`-th and `j1`-th entries and then the (shifted) `i2`-
th and `j2`-th entries
 matches removing the `i2`-th and `j2`-th entries first and then the (shifted) `i1`-
th and
 `j1`-th entries, via the identity permutation.
 This is used for the commutation of two *contractions*. -
/
lemma succSuccAbove_comm {n : ℕ} {c : Fin (n + 1 + 1 + 1 + 1) → C}
 (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
 (hij1 : i1 ≠ j1) (hij2 : i2 ≠ j2) :
 let i2' := (i1.succSuccAbove j1 i2);
 let j2' := (i1.succSuccAbove j1 j2);
 have hij2j2' : i2' ≠ j2' := by simp [i2', j2', hij2];
 let i1' := (predPredAbove i2' j2' hij2j2' i1 (by simp [i2', j2']));
 let j1' := (predPredAbove i2' j2' hij2j2' j1 (by simp [i2', j2']));
 IsReindexing ((c ∘ i2'.succSuccAbove j2') ∘ i1'.succSuccAbove j1')
 ((c ∘ i1.succSuccAbove j1) ∘ i2.succSuccAbove j2) id := by
 apply And.intro (Function.bijective_id)
 simp only [id_eq, Function.comp_apply]
 intro i
 rw [succSuccAbove_comm_apply]
 · simp [hij1]
 · simp [hij2]

open Fin in
/-- Removing one entry and then a pair of entries from `c` in either order gives the
 colour list: removing the `k`-th entry and then the (shifted) `i`-
th and `j`-th entries
 matches removing the `i`-th and `j`-th entries first and then the (shifted) `k`-
th entry,
 via the identity permutation.
 This is used for the commutation of a *contraction* with an *evaluation*. -
/
lemma succAbove_succSuccAbove_comm {n : ℕ} {c : Fin (n + 1 + 1 + 1) → C}
 (k : Fin (n + 1 + 1 + 1)) (i j : Fin (n + 1 + 1)) (hij : i ≠ j) :
 let i' := k.succAbove i;
 let j' := k.succAbove j;
 have hij' : i' ≠ j' := by simp [i', j', hij];
 let k' := predPredAbove i' j' hij' k (by simp [i', j', Ne.symm]);
 IsReindexing ((c ∘ i'.succSuccAbove j') ∘ k'.succAbove)
 ((c ∘ k.succAbove) ∘ i.succSuccAbove j) id := by
 intro i' j' hij' k'
 refine (Function.bijective_id, fun m => ?_)
 simp only [id_eq, Function.comp_apply]

```

```

congr 1
show i'.succSuccAbove j' (k'.succAbove m) = k.succAbove (i.succSuccAbove m)
apply Fin.val_injective
simp only [i', j', k', Fin.succSuccAbove, Fin.succAbove, Fin.predPredAbove,
 val_succ, apply_ite Fin.val, apply_dite Fin.val]
grind (splits := 60)

/-- Removing two single entries from `c` in either order gives the same colour
removing the `k1`-th entry and then the (shifted) `k2`-th
entry matches removing the
`k2`-th entry first and then the (shifted) `k1`-th entry, via the identity
This is used for the commutation of two *evaluations* of indices. -
/
lemma succAbove_succAbove_comm {n : ℕ} {c : Fin (n + 1 + 1) → C}
 (k1 : Fin (n + 1 + 1)) (k2 : Fin (n + 1)) :
 let k2' := k1.succAbove k2;
 let k1' := k2.predAbove k1;
 IsReindexing ((c ∘ k2'.succAbove) ∘ k1'.succAbove)
 ((c ∘ k1.succAbove) ∘ k2.succAbove) id := by
 intro k2' k1'
 refine ⟨Function.bijective_id, fun m => ?_⟩
 simp only [id_eq, Function.comp_apply]
 congr 1
 exact Fin.succAbove_succAbove_succAbove_predAbove k1 k2 m

/-- Splitting a list of colours `c : Fin (n + 1) → C` into its first `n` entries
last entry recovers `c` : the identity permutation matches
`Fin.append (c ∘ (Fin.last n).succAbove) ![c (Fin.last n)]` with `c`. -
/
lemma append_succ_last {n : ℕ} (c : Fin (n + 1) → C) :
 IsReindexing (Fin.append (c ∘ (Fin.last n).succAbove) ![c (Fin.last n)])
 rw [Fin.succAbove_last, on_id]
 refine Fin.addCases (fun i => ?_) (fun i => ?_)
 · simp only [Fin.append_left, Function.comp_apply]; rfl
 · fin_cases i; simp only [Fin.append_right, Matrix.cons_val_fin_one]; rfl

/-- Splitting a list of colours `c : Fin (n + 1) → C` into its first entry
`n` entries recovers `c` : the canonical reindexing `Fin (1 + n) ≃ Fin (n + 1)`
`Fin.append ![c 0] (c ∘ Fin.succAbove 0)` with `c`. -/
lemma append_of_first {n : ℕ} (c : Fin (n + 1) → C) :
 IsReindexing (Fin.append ![c 0] (c ∘ Fin.succAbove 0)) c (Fin.cast (by
 refine ⟨(finCongr (by grind)).bijective, fun i => ?_⟩
 rcases Fin.eq_zero_or_eq_succ i with rfl | ⟨i, rfl⟩
 · rfl
 · simp using congrArg (Fin.append ![c 0] (c ∘ Fin.succ)) (a1 := Fin.cast

```

1.253. *PHYSLIB/RELATIVITY/TENSORS/TENSORSPECIES/BASIC.LEAN* ~~415~~

```

 (a₂ := Fin.natAdd 1 i) (by ext; grind)

/-- Casting the domain along an equality `n1 = n` of lengths is a reindexing of `c`
`c ∘ Fin.cast h`. -/
lemma fin_cast_isReindexing (n n1 : ℕ) {c : Fin n → C} (h : n1 = n) :
 IsReindexing c (c ∘ Fin.cast h) (Fin.cast h) := by
 apply And.intro
 · exact Equiv.bijective (finCongr h)
 · intro i
 rfl

end IsReindexing

end Tensor

end TensorSpecies

```

### 1.253 Physlib/Relativity/Tensors/TensorSpecies/Basic.lean

```

public import Mathlib.RepresentationTheory.Rep.Basic
public import Physlib.Mathematics.PiTensorProduct
public import Mathlib.Algebra.Lie.OfAssociative
open Module
open scoped TensorProduct

TODO "Include in the condition of `TensorSpecies` a relationship between
the metrics and the basis vectors."
(V : C → Type) [V c, AddCommGroup (V c)] [V c, Module k (V c)]
(basisIdx : C → Type) [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
(rep : (c : C) → Representation k G (V c)) (basis : (c : C) → Basis (basisIdx c))
/-- The contraction of vectors with dual colors. -/
contr : (c : C) → ((rep c).tprod (rep (τ c))).IntertwiningMap (Representation.trivial k G k)
/-- The invariant unit tensor for a given color. -/
unit : (c : C) → ((Representation.trivial k G k)).IntertwiningMap ((rep (τ c)).tprod (rep c))
/-- The invariant metric tensor for a given color. -/
metric : (c : C) → ((Representation.trivial k G k)).IntertwiningMap ((rep c).tprod (rep (τ c)))
contr_tmul_symm : ∀ c (x : V c) (y : V (τ c)),
 contr c (x ⊗ₜ[k] y) = contr (τ c) (y ⊗ₜ Equiv.cast (congrArg V (τ_involution c)) x)
/-- The unit is symmetric. -/
unit_symm : ∀ c, unit c (1 : k) =
 LinearMap.lTensor _ (LinearEquiv.cast (τ_involution c)).toLinearMap
 (TensorProduct.comm k _ _ (unit (τ c) (1 : k)))
/-- Contraction with unit leaves invariant. -/
contr_unit : ∀ c (x : V c),

```

```

 (TensorProduct.lid k _ <|
 (contr c).toLinearMap.rTensor _ <|
 (TensorProduct.assoc k (V c) (V (τ c)) (V c)).symm <|
 x ⊗τ[k] (unit c (1 : k))) = x
/-- On contracting metrics we get the unit. -/
contr_metric : ∀ c,
 (TensorProduct.comm k _ _ <|
 (TensorProduct.lid k _ _ <|
 ((contr c).toLinearMap.rTensor (V (τ c))).lTensor (V c) <|
 (TensorProduct.assoc k (V c) (V (τ c)) (V (τ c))).symm.toLinearMap.lTensor
 TensorProduct.assoc k (V c) (V c) (V (τ c) ⊗τ[k] V (τ c)) <|
 (metric c 1) ⊗τ[k] (metric (τ c) 1)) = unit c (1 : k)
Properties of the basis
/-- `basisIdxCongr` only depends on its endpoints up to `HEq` of the arguments
colours and heterogeneously equal labels give equal casts. -
/
lemma basisIdxCongr_heq_arg {c1 c2 d : C} (h1 : c1 = d) (h2 : c2 = d)
 {x : basisIdx c1} {y : basisIdx c2} (hxy : HEq x y) :
 basisIdxCongr h1 x = basisIdxCongr h2 y := by
 subst h1; subst h2; cases hxy; rfl

variable {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
 {rep : (c : C) → Representation k G (V c)} {basis : (c : C) → Basis (basisIdx c) (V c)}
 (S : TensorSpecies k C G V basisIdx rep basis)
omit [(c : C) → Fintype (basisIdx c)] [(c : C) → DecidableEq (basisIdx c)]
basis c i = LinearEquiv.cast (R := k) h.symm (basis c1 (basisIdxCongr h1 i))
omit [(c : C) → Fintype (basisIdx c)] [(c : C) → DecidableEq (basisIdx c)]
LinearEquiv.cast (R := k) h (basis c i) = basis c1 (basisIdxCongr h1 i)
def numIndices {S : TensorSpecies k C G V basisIdx rep basis} {n : ℕ} {c : C}
 (_ : PiTensorProduct k (fun i => V (c i))) : ℕ := n

```

### 1.254 Physlib/Relativity/Tensors/Tensorial.lean

```

public import Physlib.Relativity.Tensors.Evaluation
variable {k : Type} [CommRing k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (basisIdx c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Module.Basis (basisIdx c) (V c)}
 {S : TensorSpecies k C G V basisIdx rep b}
attribute [-simp] LinearEquiv.cast_apply
class Tensorial {n : outParam ℕ}

```



```

 {k : outParam Type} [CommRing k] {C : outParam Type} {G : outParam Type} [Group
 {V : outParam (C → Type)} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {rep : outParam ((c : C) → Representation k G (V c))}
 {b : outParam ((c : C) → Module.Basis (basisIdx c) k (V c))}
 (S : outParam (TensorSpecies k C G V basisIdx rep b))
noncomputable instance self {n : ℕ} (S : TensorSpecies k C G V basisIdx rep b) (c :
lemma self_toTensor_apply {n : ℕ} (S : TensorSpecies k C G V basisIdx rep b)
 TensorSpecies.numIndices (S := S) (toTensor t)
 simp [toTensor_smul, map_add]
 [Tensorial S c2 M2] {ι ι2 : Type} {b : Module.Basis ι k M} {b2 : Module.Basis ι2
 (h2 : b2 = ((Tensor.basis (S := S) c2).map toTensor.symm).reindex e2) :
 [Tensorial S c2 M2] {ι ι2 : Type} {b : Module.Basis ι k M} {b2 : Module.Basis ι2
 (h2 : b2 = ((Tensor.basis (S := S) c2).map toTensor.symm).reindex e2) :
attribute [-simp] Matrix.cons_val_zero Matrix.cons_val Fin.succAbove_zero

open Tensor in
/-- Double basis expansion of an element of a tensor product $M \otimes[k] M_2$ of two
one-index spaces. Given bases b , b_2 of M , M_2 coming from the single-
index tensor bases,
every $x : M \otimes[k] M_2$ is the double sum over i, j of the iterated evaluation coe
toField (evalT 0 j (evalT 0 i (toTensor x))) times $b \ i \otimes_t b_2 \ j$. -
/
lemma prod_eq_sum_eval {c c2 : C} {M2 : Type}
 [Tensorial S ![c] M] [AddCommMonoid M2] [Module k M2]
 [Tensorial S ![c2] M2] {b : Module.Basis (basisIdx c) k M}
 {b2 : Module.Basis (basisIdx c2) k M2}
 (h : b = ((Tensor.basis (S := S) ![c]).map toTensor.symm).reindex ComponentIdx.s
 (h2 : b2 = ((Tensor.basis (S := S) ![c2]).map toTensor.symm).reindex ComponentId
 (x : M $\otimes[k]$ M2) : x = $\sum i : (basisIdx c), \sum j : (basisIdx c2),$
 toField (evalT 0 (basisIdxCongr (by rfl) j)
 (evalT 0 (basisIdxCongr (by rfl) i) (toTensor x))) • b i $\otimes_t[k]$ b2 j := by
apply toTensor.injective
conv_lhs => rw [eq_sum_evalT_zero (toTensor x)]
simp only [map_sum, map_smul]
refine Finset.sum_congr rfl fun i _ => ?_
rw [eq_sum_evalT_zero ((evalT 0 i) (toTensor x))]
simp only [map_sum]
refine Finset.sum_congr rfl fun j _ => ?_
simp only [prodT_zero_right, map_smul, permT_basis, prodT_basis']
congr 1
subst h h2
simp only [toTensor_tprod, Function.comp_apply, Module.Basis.coe_reindex, Module.B
 LinearEquiv.apply_symm_apply, prodT_basis']
congr 1
funext i
fin_cases i <=> rfl

```

```

variable {k : Type} [RCLike k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Module.Basis
 (S : TensorSpecies k C G V basisIdx rep b) {c : Fin n → C} {M : Type}
 [AddCommGroup M] [Module k M]

```

### 1.255 Physlib/Relativity/Tensors/UnitTensor.lean

```

variable {k : Type} [RCLike k] {C : Type} {G : Type} [Group G]
 {V : C → Type} [V c, AddCommGroup (V c)] [V c, Module k (V c)]
 {basisIdx : C → Type} [V c, Fintype (basisIdx c)] [V c, DecidableEq (ba
 {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Module.Basis
 {S : TensorSpecies k C G V basisIdx rep b}
attribute [-simp] LinearEquiv.cast_apply
 fromConstPair (S.unit c)
 rw [unitTensor, fromConstPair, S.unit_symm]
 rw [unitTensor, fromConstPair]
 simp [fromPairT]
 generalize (S.unit (S.τ c)) 1 = u at *
 induction' u using TensorProduct.induction_on with x y
 · simp
 · simp [fromSingleT_map]
 generalize (fromSingleT (S := S) y) = y at *
 generalize (fromSingleT (S := S) x) = x at *
 induction' y using induction_on_pure with p
 · induction' x using induction_on_pure with x r t
 · simp [permT_pure]
 repeat rw [prodT_pure, permT_pure]
 congr 1
 ext i
 fin_cases i
 · rfl
 · rfl
 · simp_all
 · simp_all
 · simp_all
 · simp_all
 · simp_all
lemma unit_fromSingleTContrFromPairT_eq_fromSingleT {c : C} (x : V c) :
 fromSingleTContrFromPairT x ((S.unit c) (1 : k)) =
 conv_rhs => rw [← S.contr_unit c x]
 rfl

```

**1.256 Physlib/SpaceAndTime/GalileanGroup/Basic.lean**

```

/-
Copyright (c) 2026 Rob Sneiderman. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Rob Sneiderman
-/
module

public import Physlib.SpaceAndTime.Space.EuclideanGroup.Basic
public import Physlib.SpaceAndTime.Space.Origin
public import Physlib.SpaceAndTime.Time.Basic

/-!
The Galilean group

This file defines Galilean transformations in `d` spatial dimensions, together with
law and their action on `Time × Space d`.

An element consists of a spatial orthogonal transformation `R`, a boost velocity `v`
translation `a`, and a time translation `b`. We use the active convention
`(t, x) ↦ (t + b, R x + v t + a)`.
-/

@[expose] public section

variable {d : ℕ}

/-- A Galilean transformation in `d` spatial dimensions. The fields are, in order, t
orthogonal part, boost velocity, spatial translation, and time translation. -
/
@[ext]
structure GalileanGroup (d : ℕ := 3) where
 /-- The spatial orthogonal part. -/
 rotation : Matrix.orthogonalGroup (Fin d) ℝ
 /-- The boost velocity. -/
 velocity : EuclideanSpace ℝ (Fin d)
 /-- The spatial translation. -/
 spaceTranslation : EuclideanSpace ℝ (Fin d)
 /-- The time translation. -/
 timeTranslation : Time

namespace GalileanGroup

/-! ## A. Basic support lemmas -/

```

```

lemma orthogonal_smul_smul (R : Matrix.orthogonalGroup (Fin d) ℝ) (c : ℝ)
 (v : EuclideanSpace ℝ (Fin d)) :
 R • (c • v) = c • (R • v) := by
 change (DistribMulAction.toLinearEquiv ℝ (EuclideanSpace ℝ (Fin d)) R) (c •
 c • ((DistribMulAction.toLinearEquiv ℝ (EuclideanSpace ℝ (Fin d)) R) v))
 rw [map_smul]

/-! ## B. Group operations -/

/-- The identity Galilean transformation. -/
instance : One (GalileanGroup d) where
 one := ⟨1, 0, 0, 0⟩

@[simp]
lemma one_rotation : (1 : GalileanGroup d).rotation = 1 := rfl

@[simp]
lemma one_velocity : (1 : GalileanGroup d).velocity = 0 := rfl

@[simp]
lemma one_spaceTranslation : (1 : GalileanGroup d).spaceTranslation = 0 :=

@[simp]
lemma one_timeTranslation : (1 : GalileanGroup d).timeTranslation = 0 := rfl

/-- The product whose action is composition: `(g * h) • tx = g • h • tx`. -
/
instance : Mul (GalileanGroup d) where
 mul g h :=
 ⟨g.rotation * h.rotation,
 g.rotation • h.velocity + g.velocity,
 g.spaceTranslation + g.rotation • h.spaceTranslation + h.timeTranslat
 g.timeTranslation + h.timeTranslation⟩

@[simp]
lemma mul_rotation (g h : GalileanGroup d) :
 (g * h).rotation = g.rotation * h.rotation := rfl

@[simp]
lemma mul_velocity (g h : GalileanGroup d) :
 (g * h).velocity = g.rotation • h.velocity + g.velocity := rfl

@[simp]
lemma mul_spaceTranslation (g h : GalileanGroup d) :
 (g * h).spaceTranslation =
 g.spaceTranslation + g.rotation • h.spaceTranslation + h.timeTranslat

```

```

 rfl

@[simp]
lemma mul_timeTranslation (g h : GalileanGroup d) :
 (g * h).timeTranslation = g.timeTranslation + h.timeTranslation := rfl

/-- The inverse Galilean transformation. -/
instance : Inv (GalileanGroup d) where
 inv g :=
 (g.rotation-1,
 -(g.rotation-1 • g.velocity),
 -(g.rotation-1 • g.spaceTranslation) + g.timeTranslation.val • (g.rotation-1 •
 -g.timeTranslation)

@[simp]
lemma inv_rotation (g : GalileanGroup d) :
 g-1.rotation = g.rotation-1 := rfl

/-- The inverse boost velocity formula. -/
@[simp]
lemma inv_velocity (g : GalileanGroup d) :
 g-1.velocity = -(g.rotation-1 • g.velocity) := rfl

@[simp]
lemma inv_spaceTranslation (g : GalileanGroup d) :
 g-1.spaceTranslation =
 -(g.rotation-1 • g.spaceTranslation) +
 g.timeTranslation.val • (g.rotation-1 • g.velocity) := rfl

@[simp]
lemma inv_timeTranslation (g : GalileanGroup d) :
 g-1.timeTranslation = -g.timeTranslation := rfl

/-- The Galilean transformations form a group under composition. -
/
instance : Group (GalileanGroup d) where
 mul_assoc g h k := by
 refine GalileanGroup.ext ?_ ?_ ?_ ?_
 · simpa using (mul_assoc g.rotation h.rotation k.rotation)
 · simp [mul_smul, smul_add, add_assoc]
 · simp only [mul_spaceTranslation, mul_rotation, mul_velocity, mul_timeTranslation,
 Time.add_val]
 rw [mul_smul, smul_add, smul_add, add_smul, smul_add, orthogonal_smul_smul]
 abel
 · ext
 simp [Time.add_val, add_assoc]

```

```

one_mul g := by
 refine GalileanGroup.ext ?_ ?_ ?_ ?_
 · simp
 · simp
 · simp
 · ext
 simp
mul_one g := by
 refine GalileanGroup.ext ?_ ?_ ?_ ?_
 · simp
 · simp
 · simp
 · ext
 simp
inv_mul_cancel g := by
 refine GalileanGroup.ext ?_ ?_ ?_ ?_
 · simp
 · simp
 · simp [smul_neg, add_comm, add_left_comm]
 · ext
 simp

instance : Inhabited (GalileanGroup d) where
 default := 1

/-! ## C. Action on time and space -/

/-- The Galilean action on space at a fixed time. -/
noncomputable def actSpace (g : GalileanGroup d) (t : Time) (x : Space d) :
 Space.vectorToSpace
 (g.rotation • (x -v (0 : Space d)) + t.val • g.velocity + g.spaceTransl

/-- The Galilean action on `Time × Space d`. -/
noncomputable def act (g : GalileanGroup d) (tx : Time × Space d) : Time ×
 (tx.1 + g.timeTranslation, g.actSpace tx.1 tx.2)

noncomputable instance : MulAction (GalileanGroup d) (Time × Space d) where
 smul := act
 one_smul tx := by
 rcases tx with ⟨t, x⟩
 change act (1 : GalileanGroup d) (t, x) = (t, x)
 ext i <=> simp [act, actSpace]
 mul_smul g h tx := by
 rcases tx with ⟨t, x⟩
 change act (g * h) (t, x) = act g (act h (t, x))
 ext i

```

```

 · simp [act, Time.add_val, add_comm, add_left_comm]
 · simp only [act, actSpace, Space.vectorToSpace_apply, Space.vectorToSpace_vsub,
 mul_rotation, mul_velocity, mul_spaceTranslation, mul_timeTranslation, Time.
 rw [mul_smul, smul_add, smul_add, smul_add, orthogonal_smul_smul]
 simp only [PiLp.add_apply, PiLp.smul_apply]
 ring_nf

@[simp]
lemma smul_fst (g : GalileanGroup d) (tx : Time × Space d) :
 (g • tx).1 = tx.1 + g.timeTranslation := rfl

@[simp]
lemma smul_snd (g : GalileanGroup d) (tx : Time × Space d) :
 (g • tx).2 = g.actSpace tx.1 tx.2 := rfl

@[simp]
lemma smul_mk (g : GalileanGroup d) (t : Time) (x : Space d) :
 g • ((t, x) : Time × Space d) = (t + g.timeTranslation, g.actSpace t x) := rfl

@[simp]
lemma actSpace_apply (g : GalileanGroup d) (t : Time) (x : Space d) (i : Fin d) :
 g.actSpace t x i =
 (g.rotation • (x - v (0 : Space d))) i + t.val * g.velocity i + g.spaceTranslation i
 simp [actSpace, Space.vectorToSpace, add_assoc]

/-! ## D. Subgroup inclusions -/

/-- A Euclidean spatial transformation as a Galilean transformation with zero boost
translation. -/
def ofEuclidean (g : EuclideanGroup d) : GalileanGroup d :=
 (g.linear, 0, g.translation, 0)

@[simp]
lemma ofEuclidean_rotation (g : EuclideanGroup d) :
 (ofEuclidean g).rotation = g.linear := rfl

@[simp]
lemma ofEuclidean_velocity (g : EuclideanGroup d) :
 (ofEuclidean g).velocity = 0 := rfl

@[simp]
lemma ofEuclidean_spaceTranslation (g : EuclideanGroup d) :
 (ofEuclidean g).spaceTranslation = g.translation := rfl

@[simp]
lemma ofEuclidean_timeTranslation (g : EuclideanGroup d) :

```

```

 (ofEuclidean g).timeTranslation = 0 := rfl

/-- Inclusion of the Euclidean group into the Galilean group. -
/
def euclidean.incl : EuclideanGroup d →* GalileanGroup d where
 toFun := ofEuclidean
 map_one' := rfl
 map_mul' g h := by
 ext i <;> simp [ofEuclidean]

/-- A pure orthogonal spatial transformation as a Galilean transformation.
/
def ofOrthogonal (R : Matrix.orthogonalGroup (Fin d) ℝ) : GalileanGroup d :
 (R, 0, 0, 0)

@[simp]
lemma ofOrthogonal_rotation (R : Matrix.orthogonalGroup (Fin d) ℝ) :
 (ofOrthogonal R).rotation = R := rfl

@[simp]
lemma ofOrthogonal_velocity (R : Matrix.orthogonalGroup (Fin d) ℝ) :
 (ofOrthogonal R).velocity = 0 := rfl

@[simp]
lemma ofOrthogonal_spaceTranslation (R : Matrix.orthogonalGroup (Fin d) ℝ) :
 (ofOrthogonal R).spaceTranslation = 0 := rfl

@[simp]
lemma ofOrthogonal_timeTranslation (R : Matrix.orthogonalGroup (Fin d) ℝ) :
 (ofOrthogonal R).timeTranslation = 0 := rfl

/-- Inclusion of the orthogonal group into the Galilean group. -
/
def orthogonal.incl : Matrix.orthogonalGroup (Fin d) ℝ →* GalileanGroup d where
 toFun := ofOrthogonal
 map_one' := rfl
 map_mul' R S := by
 ext i <;> simp [ofOrthogonal]

/-- Inclusion of the existing spatial rotation subgroup into the Galilean group.
/
noncomputable def rotation.incl : EuclideanGroup.RotationGroup d →* GalileanGroup d where
 toFun r := ofOrthogonal (r : EuclideanGroup d).linear
 map_one' := rfl
 map_mul' r s := by
 ext i <;> simp [ofOrthogonal]

```



```

/-- A pure spatial translation as a Galilean transformation. -
/
def ofSpaceTranslation (a : EuclideanSpace $\mathbb{R}$ (Fin d)) : GalileanGroup d :=
 ⟨1, 0, a, 0⟩

@[simp]
lemma ofSpaceTranslation_rotation (a : EuclideanSpace $\mathbb{R}$ (Fin d)) :
 (ofSpaceTranslation a).rotation = 1 := rfl

@[simp]
lemma ofSpaceTranslation_velocity (a : EuclideanSpace $\mathbb{R}$ (Fin d)) :
 (ofSpaceTranslation a).velocity = 0 := rfl

@[simp]
lemma ofSpaceTranslation_spaceTranslation (a : EuclideanSpace $\mathbb{R}$ (Fin d)) :
 (ofSpaceTranslation a).spaceTranslation = a := rfl

@[simp]
lemma ofSpaceTranslation_timeTranslation (a : EuclideanSpace $\mathbb{R}$ (Fin d)) :
 (ofSpaceTranslation a).timeTranslation = 0 := rfl

/-- Inclusion of spatial translations into the Galilean group. -
/
def spaceTranslation.incl :
 Multiplicative (EuclideanSpace $\mathbb{R}$ (Fin d)) →* GalileanGroup d where
 toFun a := ofSpaceTranslation a.toAdd
 map_one' := rfl
 map_mul' a b := by
 ext i <=> simp [ofSpaceTranslation]

/-- Inclusion of the existing spatial translation subgroup into the Galilean group.
/
noncomputable def spaceTranslationGroup.incl :
 EuclideanGroup.TranslationGroup d →* GalileanGroup d :=
 euclidean.incl.comp (EuclideanGroup.TranslationGroup.incl d)

/-- A pure time translation as a Galilean transformation. -
/
def ofTimeTranslation (b : Time) : GalileanGroup d :=
 ⟨1, 0, 0, b⟩

@[simp]
lemma ofTimeTranslation_rotation (b : Time) :
 (ofTimeTranslation (d := d) b).rotation = 1 := rfl

```

```

@[simp]
lemma ofTimeTranslation_velocity (b : Time) :
 (ofTimeTranslation (d := d) b).velocity = 0 := rfl

@[simp]
lemma ofTimeTranslation_spaceTranslation (b : Time) :
 (ofTimeTranslation (d := d) b).spaceTranslation = 0 := rfl

@[simp]
lemma ofTimeTranslation_timeTranslation (b : Time) :
 (ofTimeTranslation (d := d) b).timeTranslation = b := rfl

/-- Inclusion of time translations into the Galilean group. -
/
def timeTranslation.incl : Multiplicative Time →* GalileanGroup d where
 toFun b := ofTimeTranslation b.toAdd
 map_one' := rfl
 map_mul' a b := by
 ext i <|> simp [ofTimeTranslation]

end GalileanGroup

```

### 1.257 Physlib/SpaceAndTime/Space/API-map.yaml

version: v0.1

Title: Space

Overview: |

The key data structure is `Space d`, a structure with a single field `v` modelling `d`-dimensional flat Euclidean space (default `d = 3`) with a fixed choice of length unit and origin. The API equips it with metric/tensor, vector-space, norm, inner-product and smooth-manifold structures, and theorems, definitions and lemmas (basis, coordinates, length units, cross product, derivatives of functions and distributions, and the actions of translations).

ParentAPIs:

- "Distribution (Physlib/Mathematics/Distribution)"
- "Time (Physlib/SpaceAndTime/Time)"
- "Lorentz vectors (Physlib/Relativity/Tensors/RealTensor/Vector)"

References:

- "Lean Zulip, #Physlib > Space vs EuclideanSpace: <https://leanprover.zulipchat.com/#narrow/stream/113498-PhysLean/topic/Space.20vs.20EuclideanSpace/with/575938877>"

## Requirements:

- description: "The key data structure ``Space d`` (a structure with field ``val : Fin d``)  
done: true  
location: "Physlib/SpaceAndTime/Space/Basic.lean (Space)"
- description: "The API contains the structure of an ``ℝ``-vector space (module) on ``Space d``."  
done: true  
location: "Physlib/SpaceAndTime/Space/Module.lean (AddCommGroup (Space d), Module (Space d) ℝ)"
- description: "The API contains a norm, an inner product and a metric on ``Space d`` dimensional real inner-product space and metric space."  
done: true  
location: "Physlib/SpaceAndTime/Space/Module.lean (Norm, InnerProductSpace ℝ (Space d))"
- description: "The API contains a choice of origin (the vector-space zero) and the standard affine-isometry chart to ``EuclideanSpace``."  
done: true  
location: "Physlib/SpaceAndTime/Space/Origin.lean (Zero (Space d), chartEuclideanSpace d)"
- description: "The API contains a smooth-manifold structure on ``Space d`` (charted manifold)."  
done: true  
location: "Physlib/SpaceAndTime/Space/Basic.lean (homEuclideanSpaceSpace, CharteManifold (Space d))"
- description: "The API contains basis vectors for ``Space d``."  
done: true  
location: "Physlib/SpaceAndTime/Space/Module.lean (basis : OrthonormalBasis (Fin d) ℝ)"
- description: "The API contains a ``VSub`` instance to ``EuclideanSpace``."  
done: true  
location: "Physlib/SpaceAndTime/Space/Basic.lean (VSub (EuclideanSpace ℝ (Fin d)))"
- description: "The API contains a ``NormedAddTorsor`` instance from ``EuclideanSpace`` to ``Space d``."  
done: true  
location: "Physlib/SpaceAndTime/Space/Basic.lean (AddTorsor and NormedAddTorsor (Space d) ℝ)"
- description: "The API contains translations of points and of distributions on ``Space d``."  
done: true  
location: "Physlib/SpaceAndTime/Space/Basic.lean (AddAction (EuclideanSpace ℝ (Fin d)))"
- description: "The API contains the action of the group of rotations on ``Space d``."  
done: true  
location: "Physlib/SpaceAndTime/Space/EuclideanGroup/Basic.lean (RotationGroup);"

- description: "The API contains the relation between Spaces of different dimensions."  
done: true  
location: "Physlib/SpaceAndTime/Space/Slice.lean (slice : Space d.succ d)"
- description: "The API contains the type `LengthUnit` of choices of length units."  
done: true  
location: "Physlib/SpaceAndTime/Space/LengthUnit.lean (LengthUnit, scalar)"
- description: "The API contains the cross product on `EuclideanSpace  $\mathbb{R}$  d`."  
done: true  
location: "Physlib/SpaceAndTime/Space/CrossProduct.lean ( $\times_{\mathbb{R}^3}$ , time\_derivative)"
- description: "The API contains the derivatives of functions from `Space d` to  $\mathbb{R}$ ."  
done: true  
location: "Physlib/SpaceAndTime/Space/Derivatives/Basic.lean (Space.deriv)"
- description: "The API contains the derivatives of functions from `Space d` to  $\mathbb{R}^n$ ."  
done: false  
location: N/A
- description: "The API contains a means of constructing distributions on `Space d` and slice distributions."  
done: true  
location: "Physlib/SpaceAndTime/Space/IsDistBounded.lean (IsDistBounded)"
- description: "The API contains the derivatives of distributions from `Space d` to  $\mathbb{R}$ ."  
done: true  
location: "Physlib/SpaceAndTime/Space/Derivatives/Basic.lean (distDeriv)"
- description: "The API contains the derivatives of distributions from `Space d` to  $\mathbb{R}^n$ ."  
done: false  
location: N/A

### 1.258 Physlib/SpaceAndTime/Space/Basic.lean

```
public import Mathlib.Geometry.Manifold.Instances.Real
instance instNontrivial {d : ℕ} [NeZero d] : Nontrivial (Space d) where
 obtain ⟨v1, hv⟩ := exists_ne (0 : EuclideanSpace ℝ (Fin d))
 simp [ne_eq, eq_vadd_iff_vsub_eq, vsub_self] using hv.symm
open Manifold Real
```

```
/--
```

```
`Space d` is homeomorphic to d-dimensional euclidean space. This is used to
instance on `Space d`.
```

```

-/
noncomputable def homEuclideanSpaceSpace (d : ℕ) : EuclideanSpace ℝ (Fin d) ≈t Space ℝ
 toFun v := ⟨EuclideanSpace.equiv (Fin d) ℝ v⟩
 invFun s := EuclideanSpace.equiv (Fin d) ℝ|>.symm s.val
 rw [Metric.continuous_iff]
 intro b ε hε
 use ε
 simp_all [dist, Real.sqrt_eq_rpow]
 rw [Metric.continuous_iff]
 intro b ε hε
 use ε
 simp_all [dist, Real.sqrt_eq_rpow]
noncomputable instance (d : ℕ) : ChartedSpace (EuclideanSpace ℝ (Fin d)) (Space d) :=
 (homEuclideanSpaceSpace d).symm.toOpenPartialHomeomorph.singletonChartedSpace (b)
instance (d : ℕ) : IsManifold (□ d) τ (Space d) :=
 (homEuclideanSpaceSpace d).symm.toOpenPartialHomeomorph.isManifold_singleton (by s)
lemma chartAt_eq (d : ℕ) (p : Space d) :
 chartAt (EuclideanSpace ℝ (Fin d)) p =
 (homEuclideanSpaceSpace d).symm.toOpenPartialHomeomorph :=
 rfl

```

### 1.259 Physlib/SpaceAndTime/Space/ConstantSliceDist.lean

```

public import Mathlib.Analysis.Calculus.ParametricIntegral
have heq : fderiv ℝ (fun x => (slice i).symm (r, x)) x =
 (slice i).symm.toContinuousLinearMap.comp
 (ContinuousLinearMap.id ℝ (Space d))) := by
 rw [fderiv_fun_comp, DifferentiableAt.fderiv_prodMk (by fun_prop) (by fun_prop)
 · simp only [fderiv_slice_symm, fderiv_fun_const, Pi.zero_apply, fderiv_fun_id]
 · fun_prop
 · fun_prop]
 rw [norm_iteratedFDeriv_one, fderiv_fun_comp _ _ (by fun_prop), heq]
 · exact ContinuousLinearMap.opNorm_comp_le _ _
 · exact (η.smooth'.differentiable (by simp)).differentiableAt
 simp only [Nat.succ_eq_add_one, ContinuousLinearMap.coe_comp, Function.comp_apply]
 dsimp only [F, F']
 exact ((η.smooth'.differentiable (by simp)).comp (by fun_prop)).differentiableAt
 · exact (η.smooth'.of_le (by simp)).comp ((slice i).symm.contDiff)
have hb : ||fderiv ℝ (fun x => η ((slice i).symm (r, x))) x|| ≤
 ||iteratedFDeriv ℝ 1 ↑η ((slice i).symm (r, x))|| *
 ||(slice i).symm.toContinuousLinearMap.comp
 (ContinuousLinearMap.id ℝ (Space d))|| := by
 have heq : fderiv ℝ (fun x => (slice i).symm (r, x)) x =
 (slice i).symm.toContinuousLinearMap.comp

```

```

 (ContinuousLinearMap.prod (0 : Space d → L[ℝ] ℝ)
 (ContinuousLinearMap.id ℝ (Space d))) := by
 rw [fderiv_fun_comp, DifferentiableAt.fderiv_prodMk (by fun_prop) (
 · simp only [fderiv_slice_symm, fderiv_fun_const, Pi.zero_apply, fd
 · fun_prop
 · fun_prop
 rw [norm_iteratedFDeriv_one, fderiv_fun_comp _ _ (by fun_prop), heq]
 · exact ContinuousLinearMap.opNorm_comp_le _ _
 · exact (η.smooth'.differentiable (by simp)).differentiableAt
 refine le_trans hb ?_
 rw [mul_right_comm]
 gcongr
 simp using hbound x r
 rw [fderiv_fun_comp]
 simp only [ContinuousLinearMap.coe_comp, Function.comp_apply,
 · exact (η.smooth'.differentiable (by simp)).differentiableAt
 have hdiff : Differentiable ℝ (iteratedFDeriv ℝ n (↑η : Space d.succ →
 apply ContDiff.differentiable_iteratedFDeriv (n := (n + 1 : ℕ))
 · exact Nat.cast_lt.mpr (by omega)
 · exact η.smooth'.of_le ENat.LEInfty.out
 exact ((schwartzMap_slice_integral_contDiff (n + 1) η i).differenti
 (Nat.cast_lt.mpr (by omega))).differentiableAt
 rw [← fderiv_continuousMultilinear_apply_const_apply, ← fderiv_fun_slic
 · exact (hdiff.continuousMultilinear_apply_const
 (Fin.tail fun j => (slice i).symm (0, y j))).differentiableAt
 · exact hdiff.differentiableAt
 rw [schwartzMap_slice_integral_iteratedFDeriv_apply, ← ContinuousMultilin
 · rfl
 · exact schwartzMap_iteratedFDeriv_slice_symm_integrable η x i
 · rfl
 · simp
 simp only [Nat.succ_eq_add_one, _root_.add_apply]
 simp only [Nat.succ_eq_add_one, _root_.smul_apply, smul_eq_mul, RingHom
 obtain ⟨hrt1, hbound⟩ := hrt η x
 exact η.differentiable.differentiableAt
 · exact η.differentiable.differentiableAt

```

### 1.260 Physlib/SpaceAndTime/Space/CrossProduct.lean

```

· simp only [FunLike.coe_sub, FunLike.coe_smul, Pi.sub_apply,
 Pi.smul_apply, smul_eq_mul]
rw [Time.fderiv_euclid, Time.fderiv_euclid]
· intro i
 repeat fun_prop

```

```

 · fun_prop
 · fun_prop
 · fun_prop
 · fun_prop
 · simp [Fin.zero_eta, Fin.isValue]
 fun_prop
 · simp [Fin.isValue]
 fun_prop
 · simp [Fin.isValue]

```

### 1.261 Physlib/SpaceAndTime/Space/Derivatives/Basic.lean

```

public import Mathlib.Analysis.InnerProductSpace.Calculus
 MDifferentiableAt (α d) (α(ℝ, M) f x ↔ DifferentiableAt ℝ f x := by
 apply h.comp (I' := α d)
 deriv μ f x = mfderiv (α d) (α(ℝ, M) f x (EuclideanSpace.single μ 1) := by
 change _ = mfderiv (α d) (α(ℝ, M)
 ContinuousLinearMap.coe_comp]
A.3. Derivative distributes over addition and subtraction
@[to_fun]
 simp only [_root_.add_apply, Pi.add_apply]
 repeat fun_prop

/-- Derivatives on space distribute over subtraction. -/
@[to_fun]
lemma deriv_sub [NormedAddCommGroup M] [NormedSpace ℝ M]
 (f1 f2 : Space d → M) (hf1 : Differentiable ℝ f1) (hf2 : Differentiable ℝ f2) :
 ∂[u] (f1 - f2) = ∂[u] f1 - ∂[u] f2 := by
 rw [deriv_eq_fderiv_fun]
 ext x
 rw [fderiv_sub]
 rfl
 rw [fderiv_const_smul, FunLike.coe_smul, Pi.smul_apply, Pi.smul_apply]
 simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
 rw [fderiv_fun_pow]
 FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
 simp only [FunLike.coe_sum, Finset.sum_apply]
 apply DifferentiableAt.inner <=> fun_prop
 apply Differentiable.inner <=> fun_prop
 apply ContDiff.inner <=> fun_prop
 apply ContDiff.inner <=> fun_prop
 simp only [fderiv_fun_const, Pi.zero_apply, _root_.zero_apply, inner_zero_right,
 fderiv_fun_id, ContinuousLinearMap.coe_id', id_eq, basis_inner, zero_add]

```

```

simp only [fderiv_fun_id, ContinuousLinearMap.coe_id', id_eq, inner_basis
 Pi.ofNat_apply, _root_.zero_apply, inner_zero_left, add_zero]
 simp only [_root_.add_apply]
@[inherit_doc distDeriv]
macro "∂d[" i:term "]" : term => `(distDeriv $i)

(∂d[μ] f) ε = fderivD ℝ f ε (basis μ) := by
(∂d[ν] (∂d[μ] f)) = (∂d[μ] (∂d[ν] f)) := by

```

### 1.262 Physlib/SpaceAndTime/Space/Derivatives/Curl.lean

```

WithLp.toLp 2 fun i => ∂[(i+1)] (fi (i+2)) x - ∂[(i+2)] (fi (i+1)) x
simp only [Fin.isValue, Pi.zero_apply, PiLp.zero_apply, fderiv_fun_const,
 root.zero_apply, sub_self]
simp only [Fin.isValue, fderiv_fun_const, Pi.zero_apply, _root_.zero_appl
 · simp only [Fin.zero_eta, Fin.isValue, Fin.reduceAdd, zero_add, Pi.add_a
simp only [Fin.isValue, Pi.smul_apply, PiLp.smul_apply, smul_eq_mul]
rw [deriv_coord_smul, deriv_coord_smul, mul_sub]
repeat fun_prop
simp only [_root_.add_apply, Pi.add_apply]
simp only [FunLike.coe_sub, Pi.sub_apply]
simp only [Fin.isValue, Fin.univ_val_map, List.ofFn_succ, zero_add, Fin.s
 Fin.reduceAdd, Fin.succ_one_eq_two, List.ofFn_zero, Multiset.sum_coe, L
 List.sum_nil, add_zero, Pi.ofNat_apply]
simp only [Fin.isValue, Pi.zero_apply, PiLp.zero_apply]
rw [deriv_commute]
simp only [Fin.isValue, sub_self]
· exact hf
 ∇ × (∇ × f) = ∇ (∇ □ f) - Δv f := by
· simp only [Fin.reduceFinMk, Fin.isValue, Fin.reduceAdd, zero_add, Pi.su
 simp only [Fin.isValue, Pi.sub_apply]
 repeat fun_prop
 simp only [ContinuousLinearMap.fderiv, ContinuousLinearMap.coe_comp, Fu
 t * (x (i+1) * t * fderiv ℝ f (t • x) y (i+2) + f (t • x) (i+2) * y (
 (x (i+2) * t * fderiv ℝ f (t • x) y (i+1) + f (t • x) (i+1) * y (i+2)
 simp only [Function.comp_apply, ContinuousLinearMap.fderiv, fderiv_fun_
 Pi.zero_apply, ContinuousLinearMap.zero_smulRight, add_zero, Continuo
 FunLike.coe_smul, ContinuousLinearMap.coe_id', Pi.smul_apply, id_eq,
 simp [fderiv_f]
 exact h1
 fderiv_fun_id, ContinuousLinearMap.coe_comp, _root_.add_apply,
 FunLike.coe_smul, Pi.smul_apply, _root_.zero_apply, smul_zero,
change f = -curl fun x => ∫ (t : ℝ) in 0..1, homotopyOperatorIntegrand f
· linear_combination (norm := {simp [Fin.sum_univ_three]; ring}) - t ^ 2

```



```

· linear_combination (norm := {simp [Fin.sum_univ_three]; ring}) - t ^ 2 * x 2 * h
 fderiv_fun_id, ContinuousLinearMap.coe_comp, _root_.add_apply,
 FunLike.coe_smul, Pi.smul_apply, _root_.zero_apply, smul_zero,
rw [fderiv_comp _ (by exact
simp only [ContinuousLinearMap.coe_comp, ContinuousLinearEquiv.coe_coe,
rw [fderiv_fun_comp _ (by fun_prop) (by fun_prop)]
 toFun dfdx := WithLp.toLp 2 fun i => dfdx (basis (i+2)) (i+1) - dfdx (basis (i
 simp only [Fin.isValue, _root_.add_apply, PiLp.add_apply]
 ring
 simp only [Fin.isValue, FunLike.coe_smul, Pi.smul_apply, PiLp.smul_apply,
 smul_eq_mul, RingHom.id_apply]
 ring
 fun_prop
map_add' f1 f2 := by simp
map_smul' a f := by simp

@[inherit_doc distCurl]
macro (name := distCurlNotation) "∇^d" "x" f:term:100 : term => `(distCurl $f)
lemma distCurl_coord_apply {i : Fin 3}
 (f : Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η : [](Space, ℝ)) :
 (∇^d × f) η i = - f (SchwartzMap.evalCLM ℝ Space ℝ (basis (i+2)) (fderivCLM ℝ Spa
 + f (SchwartzMap.evalCLM ℝ Space ℝ (basis (i+1)) (fderivCLM ℝ Space ℝ η)) (i+2))
simp [distCurl, fderivD_apply]
 (∇^d × f) η 1 = - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 0) (fderivCLM ℝ Space ℝ
simp using distCurl_coord_apply (i := 1) f η
 (∇^d × f) η 2 = - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 1) (fderivCLM ℝ Space ℝ
simp using distCurl_coord_apply (i := 2) f η
 (∇^d × f) η = WithLp.toLp 2 fun i =>
 - f (SchwartzMap.evalCLM ℝ Space ℝ (basis (i+2)) (fderivCLM ℝ Space ℝ η)) (i+1)
 + f (SchwartzMap.evalCLM ℝ Space ℝ (basis (i+1)) (fderivCLM ℝ Space ℝ η)) (i+2)
simp [distCurl_coord_apply]
 ∇^d × (∇^d f) = 0 := by
simp only [distCurl_coord_apply, Fin.isValue, _root_.zero_apply, PiLp.zero_apply]
rw [distGrad_eq_sum_basis, distGrad_eq_sum_basis]
simp only [Fin.isValue, neg_smul, Finset.sum_neg_distrib, PiLp.neg_apply, WithLp.o
 WithLp.ofLp_smul, PiLp.ofLp_single, Finset.sum_apply, Pi.smul_apply, Pi.single_a
 smul_eq_mul, mul_ite, mul_one, mul_zero, Finset.sum_ite_eq, Finset.mem_univ, ↗re
 neg_neg]
rw [← map_neg, ← map_add, ← ContinuousLinearMap.map_zero f]
congr
ext x
simp only [Fin.isValue, _root_.add_apply, _root_.neg_apply, _root_.zero_apply]
rw [schwartzMap_fderiv_comm]
simp

```

**1.263 Physlib/SpaceAndTime/Space/Derivatives/Div.lean**

```

-- $\partial f_i / \partial x_i$
 $\sum i, \partial[i] (fun x => (f x) i) x$
simp only [Pi.ofNat_apply, fderiv_fun_const, _root_.zero_apply, Multiset.
simp only [fderiv_fun_const, Pi.ofNat_apply, _root_.zero_apply, Multiset.
simp only [_root_.add_apply]
simp only [FunLike.coe_smul, Pi.smul_apply, smul_eq_mul]
 simp only [_root_.add_apply, EuclideanSpace.inner_single_right, PiL
 simp only [FunLike.coe_smul, Pi.smul_apply, EuclideanSpace.inner_si
@[inherit_doc distDiv]
macro (name := distDivNotation) "∇d" "□" f:term:100 : term => `(distDiv $f)

(∇d □ f) η = $\sum i, fderivD \mathbb{R} f \eta (basis i) i$:= by
(∇d □ f) η = $\sum i, \partial^d[i] f \eta i$:= by
lemma distDiv_ofFunction {d : ℕ} {f : Space d → EuclideanSpace ℝ (Fin d)}
{hf : IsDistBounded f} (η : □(Space d, ℝ)) :
(∇d □ (distOfFunction f hf)) η =
- $\int x : Space d, \square f x, \nabla \eta x \square_{\mathbb{R}}$:= by
have integrable_lemma (i j : Fin d) :
(((SchwartzMap.evalCLM ℝ (Space d) ℝ (basis i))
(fderivCLM ℝ (Space d) ℝ) η)) x • f x) j) volume := by
rw [MeasureTheory.integral_finsetSum]
simp only [inner_smul_right, EuclideanSpace.inner_single_right, conj_tr
simp only [evalCLM_apply_apply, fderivCLM_apply, PiLp.smul_apply, smul_

```

**1.264 Physlib/SpaceAndTime/Space/Derivatives/Grad.lean**

```

- B.6. Gradient of constant distributions
- B.7. The gradient of a Schwartz map
simp only [fderiv_zero, Pi.zero_apply, _root_.zero_apply]
simp only [fderiv_fun_const, Pi.ofNat_apply, _root_.zero_apply]
 $\nabla f x = \sum i, \partial[i] f x \cdot EuclideanSpace.single i 1$:= by
(∇ f x) i = $\partial[i] f x$:= by
 $\square \nabla f x, EuclideanSpace.single i 1 \square_{\mathbb{R}} = \partial[i] f x$:= by
lemma grad_eq_gradient {d} (f : Space d → ℝ) :
rw [grad_eq_gradient f]
rw [grad_eq_gradient f]
gradient f x = $\sum i, \partial[i] f x \cdot basis i$:= by
 $\square \nabla f (t \cdot x), basis.repr x \square_{\mathbb{R}} = _root_.deriv (fun r => f (r \cdot x)) t$:=
_ = $\|x\| * \square \nabla f (t \cdot x), \|t \cdot x\|^{-1} \cdot basis.repr (t \cdot x) \square_{\mathbb{R}}$:= by
_ = $\|x\| * _root_.deriv ((fun r => f (r \cdot x)) \circ (fun r => \|x\|^{-1} * r)) (t$
_ = $(f (1 \cdot x) - f 0) + f 0$:= by
simp only [fderiv_norm_sq_apply, FunLike.coe_smul, coe_innerSL_apply, Pi.

```

```

lemma integrable_isDistBounded_inner_grad_schwartzMap {d : ℕ}
 {f : Space d → EuclideanSpace ℝ (Fin d)}
 (hf : IsDistBounded f) (η : ⌈(Space d, ℝ)⌋) :
 apply MeasureTheory.integrable_finsetSum
 have integrable_lemma (i j : Fin d) :
 Integrable (fun x => (((SchwartzMap.evalCLM ℝ (Space d) ℝ (basis i))
 ((fderivCLM ℝ (Space d) ℝ) η)) x • f x) j) volume := by
 convert! integrable_lemma i i using 2
 simp only [EuclideanSpace.inner_single_right, conj_trivial, one_mul, evalCLM_apply,
 fderivCLM_apply, PiLp.smul_apply, smul_eq_mul, mul_eq_mul_right_iff]
lemma integrable_isDistBounded_inner_grad_schwartzMap_spherical {d : ℕ}
 {f : Space d → EuclideanSpace ℝ (Fin d)}
 (hf : IsDistBounded f) (η : ⌈(Space d, ℝ)⌋) :
 • (homeomorphUnitSphereProd (Space d)).symm)
 ((volume (α := Space d)).toSphere.prod
 (Measure.volumeIoiPow (Module.finrank ℝ (Space d) - 1))) := by
 (volume (α := Space d)))
 simp only [Function.comp_apply, Homeomorph.symm_apply_apply]
 exact Homeomorph.measurableEmbedding (homeomorphUnitSphereProd (Space d))
@[inherit_doc distGrad]
scoped[Space] notation "∇d" => distGrad

 (y : EuclideanSpace ℝ (Fin d)) : ⌈∇d f η, y⌋ℝ = fderivD ℝ f η (basis.repr.symm
 ContinuousLinearMap.coe_comp, LinearMap.coe_toContinuousLinearMap', LinearEquiv.
 ∇d f = g := by
 ∇d f η =
 have hx (y : EuclideanSpace ℝ (Fin d)) : ⌈∇d f η, y⌋ℝ =
 have h1 : ∀ y, ⌈∇d f η -
 have h2 := h1 (∇d f η -
 (∇d f).toFun = fun ε => WithLp.toLp 2 fun i => ∂d[i] f ε := by
 (∇d f) ε = fun i => ∂d[i] f ε := by
 change (∇d f).toFun ε = fun i => ∂d[i] f ε
B.6. Gradient of constant distributions

-/

@[simp]
lemma distGrad_const {d} (c : ℝ) :
 ∇d (Distribution.const ℝ (Space d) c) = 0 := by
 ext ε i
 simp only [distGrad_apply, distDeriv_apply]
 simp [Distribution.fderivD_const]

/-!

B.7. The gradient of a Schwartz map

```

gradSchwartz  $\eta$   $x = \nabla \eta$   $x :=$  by

### 1.265 Physlib/SpaceAndTime/Space/Derivatives/Iterated.lean

```

| nil => simp using hf
| cons i L ih => exact (contDiff_apply $\mathbb{R}$ $\mathbb{R}$ i).comp (Space.deriv_contDiff
| nil => rfl
 simp only [List.foldr, ih hf hg]
 exact Space.deriv_add _ _ ((iteratedDerivList_contDiff L hf).different
 ((iteratedDerivList_contDiff L hg).differentiable (by simp))
| nil => rfl
 simp only [List.foldr, ih hf]
 exact Space.deriv_const_smul c ((iteratedDerivList_contDiff L hf).dif
lemma iteratedDeriv_increment_zero [NeZero d] [AddCommGroup M] [Module $\mathbb{R}$ M]
(I : MultiIndex d) (f : Space d $\rightarrow$ M) :
obtain (n, rfl) := Nat.exists_eq_succ_of_ne_zero (NeZero.ne d)
| nil => rfl
 simp only [List.foldr, ih hf]
 exact Space.deriv_commute _
 ((iteratedDerivList_contDiff L hf).of_le (WithTop.coe_le_coe.mpr le
simp [iteratedDeriv] using (iteratedDerivList_commute_deriv I.toList i h
| nil => simp
| cons i L ih => simp [List.foldr] using (tsupport_deriv_subset i).trans
 $\partial^{\mathbf{I}}$ f x = 0 :=
image_eq_zero_of_notMem_tsupport fun h => hx (tsupport_iteratedDeriv_subs

```

### 1.266 Physlib/SpaceAndTime/Space/Derivatives/Laplacian.lean

```

- `distLaplacian` : The Laplacian operator on distributions on `Space d`.
- C. Laplacian of distributions
 - C.1. Laplacian of constant distributions
noncomputable def laplacian {d} (f : Space d $\rightarrow$ $\mathbb{R}$) : Space d $\rightarrow$ $\mathbb{R}$:=
 $\nabla \square \nabla$ f
lemma laplacian_eq_sum_snd_deriv {d} (f : Space d $\rightarrow$ $\mathbb{R}$) :
 Δ f = fun x => $\sum i, \partial[i] (\partial[i] f) x :=$ by
 unfold laplacian div grad
 simp
 Δ (fun x => f x i) x
scoped[Space] notation " Δ_v " => laplacianVec

open Physlib Distribution

/-!

```

```

C. Laplacian of distributions

-/

/-- The distributional `distLaplacian` operator. -/
noncomputable def distLaplacian {d} :
 ((Space d) →d[$\mathbb{R}$] $\mathbb{R}$) →1[$\mathbb{R}$] (Space d) →d[$\mathbb{R}$] $\mathbb{R}$:=
 distDiv ∘1 distGrad

@[inherit_doc distLaplacian]
scoped[Space] notation " Δ^d " => distLaplacian

/-!

C.1. Laplacian of constant distributions

-/

@[simp]
lemma distLaplacian_const {d : $\mathbb{N}$ } (c : $\mathbb{R}$) :
 Δ^d (Distribution.const $\mathbb{R}$ (Space d) c) = 0 := by
 simp [distLaplacian, distGrad_const]

```

## 1.267 Physlib/SpaceAndTime/Space/DistOfFunction.lean

```

public import Physlib.SpaceAndTime.Space.Derivatives.Basic
- D. Derivatives
- E. Components
 simp only [_root_.add_apply]
 simp only [_root_.smul_apply, smul_eq_mul, RingHom.id_apply]
/-!

```

```

D. Derivatives

```

```

-/

```

```

/-- The distributional spatial derivative of the distribution associated to a differentiable function is the distribution associated to the corresponding pointwise spatial derivative. -/

```

```

lemma distDeriv_distOfFunction {d : $\mathbb{N}$ } (μ : Fin d) (f : Space d → F)
 (hf : IsDistBounded f) (hf_deriv : IsDistBounded ($\partial[\mu]$ f))
 (hf_diff : Differentiable $\mathbb{R}$ f) :
 $\partial^d[\mu]$ (distOfFunction f hf) = distOfFunction ($\partial[\mu]$ f) hf_deriv := by

```

```

ext η
rw [distDeriv_apply, Physlib.Distribution.fderivD_apply, distOfFunction_a
 distOfFunction_apply]
change -∫ (x : Space d), fderiv ℝ η x (basis μ) • f x =
 ∫ (x : Space d), η x • fderiv ℝ f x (basis μ)
rw [integral_smul_fderiv_eq_neg_fderiv_smul_of_integrable]
· exact hf.integrable_space_fderiv η (basis μ)
· exact hf_deriv.integrable_space η
· exact hf.integrable_space η
· fun_prop
· intro x _
 exact hf_diff.differentiableAt
E. Components

```

### 1.268 Physlib/SpaceAndTime/Space/EuclideanGroup/Action.lean

```

/-
Copyright (c) 2026 Shaopeng Zhu. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Shaopeng Zhu
-/
module

public import Physlib.SpaceAndTime.Space.Origin
public import Physlib.SpaceAndTime.Space.EuclideanGroup.AffineGroup

/-!
The action of the Euclidean group on `Space`

The Euclidean group `EuclideanGroup d = ℝd ⋈ O(d)` (`Space/EuclideanGroup/B
of rigid motions of `d`-dimensional space. This file makes that geometric m
endows the affine space of points `Space d` (`Space/Basic.lean`) with a `Mu
`EuclideanGroup d` by isometries, and specialises it to the rotations.

Main results

* `EuclideanGroup.smul_vsub_smul` – displacements transform by the linear p
* `EuclideanGroup.dist_smul` – the action preserves distance.
* `EuclideanGroup.rotation_smul_origin` / `rotation_smul_vsub_origin` – rot
and act about it by their orthogonal part.
* `EuclideanGroup.chartEuclidean_smul` – agreement with the affine-
isometry model
(`AffineGroup.lean`); the rest of the file does not depend on it.

```

1.268.

PHYSLIB/SPACEANDTIME/SPACE/EUCLIDEANGROUP/ACTION.LEAN 419

## Implementation notes

`Space d` is an affine space (`NormedAddTorsor`) with no canonical origin, so the associated vector-space zero `(0 : Space d)` as the basepoint:

$g \cdot p = (g.linear \cdot (p -_v (0 : Space d)) + g.translation) +_v (0 : Space d)$ .

The `Zero` instance and the chart `Space.chartEuclidean` are defined in `Space/Origin`. -/

@[expose] public section

variable {d : ℕ}

namespace EuclideanGroup

/-! ## Part 1: the action of the Euclidean group on `Space`

The motion  $g = \langle t, Q \rangle$  acts by  $g \cdot p = (Q \cdot (p -_v origin) + t) +_v origin$ . The `Mul` reduce to the semidirect-product group law of `EuclideanGroup`. - /

/-- The action of the Euclidean group on the affine space of points `Space d`:  $g = \langle t, Q \rangle$  acts by  $g \cdot p = (Q \cdot (p -_v origin) + t) +_v origin$ . The `Mul` reduce to the semidirect-product group law of `EuclideanGroup`. - /

noncomputable instance : MulAction (EuclideanGroup d) (Space d) where  
 smul g p := (g.linear • (p -\_v (0 : Space d)) + g.translation) +\_v (0 : Space d)  
 one\_smul p := by  
 show ((1 : Matrix.orthogonalGroup (Fin d) ℝ) • (p -\_v (0 : Space d)) + 0) +\_v (0 : Space d) = p  
 simp  
 mul\_smul g h p := by  
 show (((g \* h).linear • (p -\_v (0 : Space d)) + (g \* h).translation) +\_v (0 : Space d)) = (((g.linear • ((h.linear • (p -\_v (0 : Space d)) + h.translation) +\_v (0 : Space d)) -\_v (0 : Space d))) + g.translation) +\_v (0 : Space d)  
 simp [vadd\_vsub, add\_comm, add\_assoc, mul\_smul]

/-- Coordinate formula for the action:  $(g \cdot p) i = (Q \cdot (p -_v origin)) i + t i$ . -/

@[simp] lemma smul\_apply (g : EuclideanGroup d) (p : Space d) (i : Fin d) :  
 (g • p) i = (g.linear • (p -\_v (0 : Space d))) i + g.translation i := by  
 show ((g.linear • (p -\_v (0 : Space d)) + g.translation) +\_v (0 : Space d)) i = (g.linear • (p -\_v (0 : Space d))) i + g.translation i  
 simp [Space.vadd\_apply]

/-- The displacement between two points transforms by the \*\*orthogonal part alone\*\*:

```

translation cancels. This is the key lemma behind `dist_smul`. -
/
@[simp] lemma smul_vsub_smul (g : EuclideanGroup d) (p q : Space d) :
 (g • p) -_v (g • q) = g.linear • (p -_v q) := by
 show ((g.linear • (p -_v (0 : Space d)) + g.translation) +_v (0 : Space d))
 -_v ((g.linear • (q -_v (0 : Space d)) + g.translation) +_v (0 : Space d))
 = g.linear • (p -_v q)
 rw [vadd_vsub_vadd_cancel_right, add_sub_add_right_eq_sub, ← smul_sub,
 vsub_sub_vsub_cancel_right]

/-- The Euclidean group acts on `Space d` by isometries: every rigid mo
-/
lemma dist_smul (g : EuclideanGroup d) (p q : Space d) :
 dist (g • p) (g • q) = dist p q := by
 rw [dist_eq_norm_vsub (EuclideanSpace ℝ (Fin d)) (g • p) (g • q),
 dist_eq_norm_vsub (EuclideanSpace ℝ (Fin d)) p q, smul_vsub_smul]
 exact (orthogonalToLinearIsometryEquiv g.linear).norm_map _

/-! ## Part 2: specialisation to rotations

The `RotationGroup d` action is the restriction of the Euclidean action alo
`RotationGroup d ≤ EuclideanGroup d` (`RotationGroup` elements have zero tr
below record that rotations fix the origin, act by their orthogonal part ab
preserve distance. -/

/-- The rotation-group action is the restriction of the Euclidean action: `
(definitional). -/
@[simp] lemma rotation_smul_eq (r : RotationGroup d) (p : Space d) :
 r • p = (r : EuclideanGroup d) • p := rfl

/-- A rotation fixes the origin: its translation part vanishes
(`RotationGroup ≤ OriginStabilizer`), so `↑r • 0 = 0`. Stated in the `↑r` f
form of `r • _`, via `rotation_smul_eq`) so it is a well-
formed `simp` lemma. -/
@[simp] lemma rotation_smul_origin (r : RotationGroup d) :
 (r : EuclideanGroup d) • (0 : Space d) = (0 : Space d) := by
 have h_trans : (r : EuclideanGroup d).translation = 0 := by
 apply r.property.right
 have h_rot : (r : EuclideanGroup d) • ((0 : Space d)) =
 ((r : EuclideanGroup d).linear • (0 : EuclideanSpace ℝ (Fin d)) + 0)
 show ((r : EuclideanGroup d).linear • ((0 : Space d) -
v (0 : Space d))
 + (r : EuclideanGroup d).translation) +_v (0 : Space d) = _
 rw [vsub_self, h_trans]
 simp [h_rot]

```



1.269.

**PHYSLIB/SPACEANDTIME/SPACE/EUCLIDEANGROUP/AFFINEGROUP.lean**

```
/-- A rotation acts on the displacement from the origin by its orthogonal part, for
` (r • p) -v origin = Q • (p -v origin) `. -/
lemma rotation_smul_vsub_origin (r : RotationGroup d) (p : Space d) :
 (r • p) -v (0 : Space d) = (r : EuclideanGroup d).linear • (p -
v (0 : Space d)) := by
 rw [rotation_smul_eq]
 nth_rewrite 1 [← rotation_smul_origin r]
 rw [smul_vsub_smul]

/-- The rotation group acts on `Space d` by isometries (inherited from `dist_smul`
/
lemma rotation_dist_smul (r : RotationGroup d) (p q : Space d) :
 dist (r • p) (r • q) = dist p q :=
 dist_smul (r : EuclideanGroup d) p q

/-! ## Part 3: relation to the affine isometry action (optional bridge)

`chartEuclidean_smul` records that, under the chart `Space.chartEuclidean` (`Space/0
`p ↦ p -v origin`, the Part 1 action is the transport of `toAffineIsometryMulEquiv`
(`AffineGroup.lean`) from `EuclideanSpace` to `Space`. Nothing in Parts 1–2 depends
/

/-- Under the standard chart, the Euclidean action on `Space d` is the
transport of `toAffineIsometryMulEquiv` acting on `EuclideanSpace`:
`chart (g • p) = (toAffineIsometryMulEquiv g) (chart p)`. -
/
lemma chartEuclidean_smul (g : EuclideanGroup d) (p : Space d) :
 Space.chartEuclidean d (g • p) = toAffineIsometryMulEquiv g (Space.chartEuclidean d p)
 rw [Space.chartEuclidean_apply]
 rw [toAffineIsometryMulEquiv_apply, toAffineIsometryHom_apply]
 have h_left : g • p -v (0 : Space d) = g.linear • (p -v (0 : Space d)) + g.translation
 exact
 (eq_vadd_iff_vsub_eq (g • p) (g.linear • (p -v (0 : Space d)) + g.translation)
 ((0 : Space d))).mp
 rfl
 simp [h_left]
 exact add_comm' (g.linear • (p -v (0 : Space d))) g.translation
end EuclideanGroup
```

## 1.269 Physlib/SpaceAndTime/Space/EuclideanGroup/AffineGroup.lean

/-

Copyright (c) 2026 Shaopeng Zhu. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Shaopeng Zhu

-/

module

public import Physlib.SpaceAndTime.Space.EuclideanGroup.Basic

/-!

# The inclusion of the Euclidean group into the affine automorphism group

This file has two parts.

**\*\*Part 1: the inclusion.\*\*** The abstract Euclidean group ``EuclideanGroup n`` is included into Mathlib's affine automorphism group as the composite of two maps

``EuclideanGroup n →* AffineIsometryEquiv ℝ (EuclideanSpace ℝ (Fin n)) →*`

\* ``EuclideanGroup.orthogonalToLinearIsometryEquiv`` : an orthogonal matrix and a linear equivalence of ``EuclideanSpace ℝ (Fin n)``; the linear ingredient of the first leg

\* ``EuclideanGroup.toAffineIsometryHom`` : the first leg,  $\langle t, Q \rangle \mapsto (x \mapsto Q x + t)$

\* ``AffineIsometryEquiv.toAffineEquivHom`` : the second leg, ``AffineIsometryEquiv`` as a monoid homomorphism.

\* ``EuclideanGroup.toAffineEquiv`` : the composite of the two legs; the result is recorded elsewhere.

**\*\*Part 2: strengthening the first leg to an isomorphism.\*\*** Every affine isomorphism ``EuclideanSpace ℝ (Fin n)`` is  $x \mapsto Q x + t$  for a unique orthogonal ``Q`` and vector ``t``; the first leg is in fact a group isomorphism. We record it as ``EuclideanGroup.toAffineIsometryMulEquiv``, a ``MulEquiv`` whose fields are

\* ``toFun``, ``map_mul`` : reused from ``EuclideanGroup.toAffineIsometryHom`` in

\* ``invFun`` : built from ``EuclideanGroup.linearIsometryEquivToOrthogonal``, the linear bridge ``orthogonalToLinearIsometryEquiv``;

\* ``left_inv`` : from the round trip ``orthogonalToLinearIsometryEquiv_left_inv`` and the projection lemma ``linearIsometryEquiv_constVAdd_mul``;

\* ``right_inv`` : from the round trip ``orthogonalToLinearIsometryEquiv_right_inv``.

Part 2 is self-contained; nothing in part 1 depends on it.

-/

@[expose] public section

variable {n : ℕ}

namespace EuclideanGroup

/-! ## Part 1: the inclusion

1.269.

PHYSLIB/SPACEANDTIME/SPACE/EUCLIDEANGROUP/AFFINEGROUP. ~~LEBAN~~

The chain is: `orthogonalToLinearIsometryEquiv` (linear ingredient)  $\rightarrow$  `toAffineIsome` (first leg)  $\rightarrow$  `AffineIsometryEquiv.toAffineEquivHom` (second leg)  $\rightarrow$  `toAffineEquiv` (the composite). -/

```
open scoped Matrix in
/-- An orthogonal matrix viewed as a linear isometry equivalence of `EuclideanSpace
the linear ingredient of the first leg `toAffineIsometryHom`. -
/
noncomputable def orthogonalToLinearIsometryEquiv
 (Q : Matrix.orthogonalGroup (Fin n) ℝ) :
 EuclideanSpace ℝ (Fin n) $\approx_{\text{li}}$ [ℝ] EuclideanSpace ℝ (Fin n) :=
 (DistribMulAction.toLinearEquiv ℝ (EuclideanSpace ℝ (Fin n)) Q).isometryOfInner fun
 simp only [DistribMulAction.toLinearEquiv_apply,
 EuclideanSpace.inner_eq_star_dotProduct, star_trivial]
 show (Q.val *v y.ofLp) $\sqsubseteq_v$ (Q.val *v x.ofLp) = y.ofLp $\sqsubseteq_v$ x.ofLp
 have hQ : (Q.val)T * Q.val = 1 := (Matrix.mem_orthogonalGroup_iff' (Fin n) ℝ).mp
 rw [Matrix.dotProduct_mulVec, Matrix.vecMul_mulVec, hQ, Matrix.vecMul_one]
```

```
@[simp] lemma orthogonalToLinearIsometryEquiv_apply
 (Q : Matrix.orthogonalGroup (Fin n) ℝ) (x : EuclideanSpace ℝ (Fin n)) :
 orthogonalToLinearIsometryEquiv Q x = Q • x := rfl
```

/-- The first leg of the inclusion:  $\langle t, Q \rangle \mapsto (x \mapsto Q x + t)$ , bundled as a monoid hom from the Euclidean group into the affine isometry group of `EuclideanSpace ℝ (Fin n)` -/

```
noncomputable def toAffineIsometryHom :
 EuclideanGroup n $\rightarrow^*$
 AffineIsometryEquiv ℝ (EuclideanSpace ℝ (Fin n)) (EuclideanSpace ℝ (Fin n)) where
toFun A := AffineIsometryEquiv.constVAdd ℝ (EuclideanSpace ℝ (Fin n)) A.translation
 (orthogonalToLinearIsometryEquiv A.linear).toAffineIsometryEquiv
map_one' := by
 apply AffineIsometryEquiv.ext
 intro x; simp
map_mul' A B := by
 apply AffineIsometryEquiv.ext
 intro x
 simp [mul_smul, add_assoc]
```

/-- Unfolds `toAffineIsometryHom` into its translation and linear factors. -/

```
@[simp] lemma toAffineIsometryHom_apply (A : EuclideanGroup n) :
 toAffineIsometryHom A =
 AffineIsometryEquiv.constVAdd ℝ (EuclideanSpace ℝ (Fin n)) A.translation *
 (orthogonalToLinearIsometryEquiv A.linear).toAffineIsometryEquiv := rfl
```

```

/-- The second leg of the inclusion: `AffineIsometryEquiv.toAffineEquiv` but
homomorphism into the affine automorphism group. -/
noncomputable def _root_.AffineIsometryEquiv.toAffineEquivHom :
 AffineIsometryEquiv ℝ (EuclideanSpace ℝ (Fin n)) (EuclideanSpace ℝ (Fin n))
 AffineEquiv ℝ (EuclideanSpace ℝ (Fin n)) (EuclideanSpace ℝ (Fin n)) w
toFun e := e.toAffineEquiv
map_one' := by
 apply AffineEquiv.ext; intro x; trivial
map_mul' e e' := by
 apply AffineEquiv.ext; intro x; simp

/-- The inclusion of the Euclidean group into Mathlib's affine automorphism
of the two legs `toAffineIsometryHom` and `AffineIsometryEquiv.toAffineEquivHom`
/
noncomputable def toAffineEquiv :
 EuclideanGroup n →* AffineEquiv ℝ (EuclideanSpace ℝ (Fin n)) (EuclideanSpace ℝ (Fin n))
 AffineIsometryEquiv.toAffineEquivHom.comp toAffineIsometryHom

/-! ## Part 2: strengthening the first leg to an isomorphism

The first leg `toAffineIsometryHom` is in fact a group isomorphism: every a
`EuclideanSpace ℝ (Fin n)` is $x \mapsto Qx + t$ for a unique orthogonal `Q` and
declarations below supply the remaining fields of the `MulEquiv` `toAffineIsometryHom`
field order: the `invFun` ingredient, the two lemmas proving `left_inv`, and
`right_inv`. Nothing in part 1 depends on this section. -
/

/-- The `invFun` ingredient of `toAffineIsometryMulEquiv`: a linear isometry
as an orthogonal matrix, inverse to the linear bridge `orthogonalToLinearIsometry`
round-trip (see lemmas below). -/
noncomputable def linearIsometryEquivToOrthogonal
 (L : EuclideanSpace ℝ (Fin n) ≃ℝ EuclideanSpace ℝ (Fin n)) :
 Matrix.orthogonalGroup (Fin n) ℝ :=
 let b := EuclideanSpace.basisFun (Fin n) ℝ
 ⟨LinearMap.toMatrix b.toBasis b.toBasis L.toLinearEquiv, by
 have hb : LinearMap.toMatrix b.toBasis b.toBasis L.toLinearEquiv
 = b.toBasis.toMatrix (b.map L) := by
 ext i j
 simp [LinearMap.toMatrix_apply, Module.Basis.toMatrix_apply,
 OrthonormalBasis.map_apply, OrthonormalBasis.coe_toBasis_repr_apply]
 rw [hb]
 exact b.toMatrix_orthonormalBasis_mem_orthogonal (b.map L)⟩

/-- `linearIsometryEquivToOrthogonal` is a left inverse of `orthogonalToLinearIsometry`
Together with `linearIsometryEquiv_constVAdd_mul`, this proves `left_inv` of
`toAffineIsometryMulEquiv`. -/

```

1.269.

PHYSLIB/SPACEANDTIME/SPACE/EUCLIDEANGROUP/AFFINEGROUP.4231N

```
@[simp] lemma orthogonalToLinearIsometryEquiv_left_inv
 (Q : Matrix.orthogonalGroup (Fin n) ℝ) :
 linearIsometryEquivToOrthogonal (orthogonalToLinearIsometryEquiv Q) = Q := by
 apply Subtype.ext
 have hlin :
 ((orthogonalToLinearIsometryEquiv Q).toLinearEquiv :
 EuclideanSpace ℝ (Fin n) →1[ℝ] EuclideanSpace ℝ (Fin n))
 = Matrix.toLin (EuclideanSpace.basisFun (Fin n) ℝ).toBasis
 (EuclideanSpace.basisFun (Fin n) ℝ).toBasis Q.val := by
 ext x; rfl
 show LinearMap.toMatrix _ _
 ((orthogonalToLinearIsometryEquiv Q).toLinearEquiv :
 EuclideanSpace ℝ (Fin n) →1[ℝ] EuclideanSpace ℝ (Fin n)) = Q.val
 rw [hlin, LinearMap.toMatrix_toLin]
```

/-- The affine map  $x \mapsto t + v + L x$ , projected to its linear isometry component, is  $L$ . Together with `orthogonalToLinearIsometryEquiv_left_inv`, this proves `left_inv` of `toAffineIsometryMulEquiv`. -/

```
@[simp] lemma linearIsometryEquiv_constVAdd_mul
 (t : EuclideanSpace ℝ (Fin n))
 (L : EuclideanSpace ℝ (Fin n) →1[ℝ] EuclideanSpace ℝ (Fin n)) :
 ((AffineIsometryEquiv.constVAdd ℝ (EuclideanSpace ℝ (Fin n)) t *
 L.toAffineIsometryEquiv).linearIsometryEquiv) = L := by
 apply LinearIsometryEquiv.ext; intro x
 have h := (AffineIsometryEquiv.constVAdd ℝ (EuclideanSpace ℝ (Fin n)) t *
 L.toAffineIsometryEquiv).map_vsub x 0
 rw [vsub_eq_sub, sub_zero] at h
 rw [h]
 simp
```

/-- `linearIsometryEquivToOrthogonal` is a right inverse of `orthogonalToLinearIsometryEquiv`. this proves `right_inv` of `toAffineIsometryMulEquiv`. -/

```
@[simp] lemma orthogonalToLinearIsometryEquiv_right_inv
 (L : EuclideanSpace ℝ (Fin n) →1[ℝ] EuclideanSpace ℝ (Fin n)) :
 orthogonalToLinearIsometryEquiv (linearIsometryEquivToOrthogonal L) = L := by
 apply LinearIsometryEquiv.ext; intro x
 rw [orthogonalToLinearIsometryEquiv_apply]
 show Matrix.toEuclideanLin (linearIsometryEquivToOrthogonal L).val x = L x
 rw [Matrix.toEuclideanLin_eq_toLin_orthonormal]
 show Matrix.toLin _ _
 (LinearMap.toMatrix _ _
 (L.toLinearEquiv : EuclideanSpace ℝ (Fin n) →1[ℝ] EuclideanSpace ℝ (Fin n)))
 rw [Matrix.toLin_toMatrix]
 rfl
```

/-- `EuclideanGroup n ≈* AffineIsometryEquiv ℝ (EuclideanSpace ℝ (Fin n))`: the Euclidean group is isomorphic to the affine isometry group.

is the full group of affine isometries of Euclidean space. This upgrades ``t` a group isomorphism with the same underlying map. `-/`

```

noncomputable def toAffineIsometryMulEquiv :
 EuclideanGroup n ≈*
 AffineIsometryEquiv ℝ (EuclideanSpace ℝ (Fin n)) (EuclideanSpace ℝ (F
toFun := toAffineIsometryHom
invFun e := ⟨e 0, linearIsometryEquivToOrthogonal e.linearIsometryEquiv⟩
left_inv A := by
 refine EuclideanGroup.ext ?_ ?_
 · simp
 · simp [linearIsometryEquiv_constVAdd_mul, orthogonalToLinearIsometryEq
right_inv e := by
 apply AffineIsometryEquiv.ext; intro x
 simp only [toAffineIsometryHom_apply,
 orthogonalToLinearIsometryEquiv_right_inv,
 AffineIsometryEquiv.coe_mul, Function.comp_apply,
 LinearIsometryEquiv.coe_toAffineIsometryEquiv,
 AffineIsometryEquiv.coe_constVAdd, vadd_eq_add]
 have h := e.map_vadd 0 x
 simp [vadd_eq_add, add_zero] at h
 rw [h, add_comm]
map_mul' := toAffineIsometryHom.map_mul'

/-- `toAffineIsometryMulEquiv` agrees with `toAffineIsometryHom`. -
/
@[simp] lemma toAffineIsometryMulEquiv_apply (A : EuclideanGroup n) :
 toAffineIsometryMulEquiv A = toAffineIsometryHom A := rfl

end EuclideanGroup

```

### 1.270 Physlib/SpaceAndTime/Space/EuclideanGroup/Basic.lean

```

/-
Copyright (c) 2026 Shaopeng Zhu. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Shaopeng Zhu
-/
module

public import Mathlib.Analysis.InnerProductSpace.PiL2

/-!
Euclidean group

```

1.270.

PHYSLIB/SPACEANDTIME/SPACE/EUCLIDEANGROUP/BASIC.LEAN 427

This file defines the Euclidean group as translations composed with orthogonal maps, with the special Euclidean group, translation subgroup, and rotation subgroups.

The affine group, together with the inclusion of the Euclidean group into it, is defined as ``Physlib.SpaceAndTime.Space.EuclideanGroup.AffineGroup``.

-/

@[expose] public section

/-- An n-dimensional `Euclidean group` is a group of rotations, reflections, and translations.

-/

@[ext]

structure EuclideanGroup (n : ℕ) where

/-- The translation part of a Euclidean transformation. -

/

translation : EuclideanSpace ℝ (Fin n)

/-- The orthogonal linear part of a Euclidean transformation. -

/

linear : Matrix.orthogonalGroup (Fin n) ℝ

namespace EuclideanGroup

/-- Group structure on  $E(n) = \mathbb{R}^n \ltimes O(n)$ . The orthogonal component acts on translations via the inherited matrix-vector action on `EuclideanSpace ℝ (Fin n)`, transported through the coordinate action `Q.val *v v.ofLp`. -/

noncomputable instance : Group (EuclideanGroup n) where

mul A B := ⟨A.translation + A.linear • B.translation, A.linear \* B.linear⟩

mul\_assoc A B C := by

refine EuclideanGroup.ext ?\_ ?\_

· show A.translation + A.linear • B.translation + (A.linear \* B.linear) • C.translation = A.translation + A.linear • (B.translation + B.linear • C.translation)

rw [mul\_smul, smul\_add, add\_assoc]

· exact mul\_assoc A.linear B.linear C.linear

one := ⟨0, 1⟩

one\_mul A := by

refine EuclideanGroup.ext ?\_ ?\_

· show 0 + (1 : Matrix.orthogonalGroup (Fin n) ℝ) • A.translation = A.translation

rw [zero\_add, one\_smul]

· exact one\_mul A.linear

mul\_one A := by

refine EuclideanGroup.ext ?\_ ?\_

· show A.translation + A.linear • 0 = A.translation

rw [smul\_zero, add\_zero]

· exact mul\_one A.linear

inv A := ⟨A.linear<sup>-1</sup> • (-A.translation), A.linear<sup>-1</sup>⟩

```

inv_mul_cancel A := by
 refine EuclideanGroup.ext ?_ ?_
 · show A.linear-1 • (-A.translation) + A.linear-1 • A.translation = 0
 rw [← smul_add, neg_add_cancel, smul_zero]
 · exact inv_mul_cancel A.linear

/-- ### `One`/`Mul` projection lemmas

These expose the semidirect-product formulas behind the `Group` instance so
reduce the translation/linear components of `1` and `A * B`. -
/

@[simp] lemma one_translation : (1 : EuclideanGroup n).translation = 0 := rfl

@[simp] lemma one_linear : (1 : EuclideanGroup n).linear = 1 := rfl

@[simp] lemma mul_translation (A B : EuclideanGroup n) :
 (A * B).translation = A.translation + A.linear • B.translation := rfl

@[simp] lemma mul_linear (A B : EuclideanGroup n) :
 (A * B).linear = A.linear * B.linear := rfl

private lemma det_coe_inv {n : ℕ} (Q : Matrix.orthogonalGroup (Fin n) ℝ) :
 (Q-1).val.det = (Q.val.det)-1 := by
 apply eq_inv_of_mul_eq_one_right
 rw [← Matrix.det_mul, ← Submonoid.coe_mul, mul_inv_cancel,
 OneMemClass.coe_one, Matrix.det_one]

private lemma coe_inv {n : ℕ} (Q : Matrix.orthogonalGroup (Fin n) ℝ) :
 (Q-1).val = (Q.val)-1 := by
 symm
 apply Matrix.inv_eq_right_inv
 rw [← Submonoid.coe_mul, mul_inv_cancel, OneMemClass.coe_one]

/-- Special Euclidean Group is the subgroup with det(Q) = 1 where Q ∈ O(n)
/
def SpecialEuclideanGroup (n : ℕ) : Subgroup (EuclideanGroup n) where
 carrier := {g | g.linear.val.det = 1}
 mul_mem' {a b} ha hb := by
 show (a.linear * b.linear).val.det = 1
 rw [Submonoid.coe_mul, Matrix.det_mul, ha, hb, one_mul]
 one_mem' := by
 show (1 : 1(Matrix.orthogonalGroup (Fin n) ℝ)).val.det = 1
 rw [OneMemClass.coe_one, Matrix.det_one]
 inv_mem' {a} ha := by
 show (a.linear-1).val.det = 1

```



```

 rw [det_coe_inv, ha, inv_one]

/-- The inclusion of the special Euclidean group into the Euclidean group. -
/
noncomputable def SpecialEuclideanGroup.incl (n : ℕ) :
 SpecialEuclideanGroup n →* EuclideanGroup n := (SpecialEuclideanGroup n).subtype

/-- Translation is the subgroup with Q = 1. -/
def TranslationGroup (n : ℕ) : Subgroup (EuclideanGroup n) where
 carrier := {g | g.linear.val = 1}
 mul_mem' {a b} ha hb := by
 show (a.linear * b.linear).val = 1
 rw [Submonoid.coe_mul, ha, hb, one_mul]
 one_mem' := by
 show (1 : 1(Matrix.orthogonalGroup (Fin n) ℝ)).val = 1
 rw [OneMemClass.coe_one]
 inv_mem' {a} ha := by
 show (a.linear⁻¹).val = 1
 rw [coe_inv, ha, inv_one]

/-- The inclusion of the translation subgroup into the Euclidean group. -
/
noncomputable def TranslationGroup.incl (n : ℕ) :
 TranslationGroup n →* EuclideanGroup n := (TranslationGroup n).subtype

/-- MonoidHom including a translation vector into the Euclidean Group. -
/
def translationVector.incl (n : ℕ) :
 Multiplicative (EuclideanSpace ℝ (Fin n)) →* EuclideanGroup n where
 toFun v := ⟨v.toAdd, 1⟩
 map_one' := by rfl
 map_mul' x y := by
 refine EuclideanGroup.ext ?_ ?_
 · show Multiplicative.toAdd (x * y) =
 Multiplicative.toAdd x +
 (1 : Matrix.orthogonalGroup (Fin n) ℝ) • Multiplicative.toAdd y
 simp
 · show 1 = 1 * 1
 simp [mul_one]

/-- An API feature: the translation vector inclusion image is the `TranslationGroup`
/
lemma translationVector.incl_range :
 Set.range (@translationVector.incl n) = (TranslationGroup n : Set (EuclideanGroup n))
 ext g
 constructor

```

```

· rintro ⟨v, hv⟩
 show g.linear.val = 1
 rw [← hv]
 rfl
· intro h
 rw [Set.mem_range]
 refine ⟨g.translation, ?_⟩
 show ((g.translation, 1) : EuclideanGroup n) = g
 refine EuclideanGroup.ext rfl ?_
 apply Subtype.ext
 simp [h.symm]

/-- The translation by the zero vector is the identity of the Euclidean group. -
/
lemma translation_zero : translationVector.incl n
 (Multiplicative.ofAdd (0 : EuclideanSpace ℝ (Fin n))) = 1 := by
 simp

/-- The subgroup of `EuclideanGroup n` whose elements fix the origin
(translation = 0). This is the copy of `O(n)` sitting inside `E(n)`. -
/
def OriginStabilizer (n : ℕ) : Subgroup (EuclideanGroup n) where
 carrier := {g | g.translation = 0}
 mul_mem' {a b} ha hb := by
 show a.translation + a.linear • b.translation = 0
 rw [ha, hb, smul_zero, zero_add]
 one_mem' := rfl
 inv_mem' {a} ha := by
 show a.linear-1 • (-a.translation) = 0
 rw [ha, neg_zero, smul_zero]

/-- Rotation Group is the subgroup of E(n) consisting of rotations about the
elements with `det = 1` (orientation-preserving) and `translation = 0` (origin
fixing). -/
noncomputable def RotationGroup (n : ℕ) : Subgroup (EuclideanGroup n) :=
 SpecialEuclideanGroup n ⊓ OriginStabilizer n

/-- The inclusion of the rotation subgroup into the Euclidean group. -
/
noncomputable def RotationGroup.incl (n : ℕ) :
 RotationGroup n →* EuclideanGroup n := (RotationGroup n).subtype

variable {n} (p : EuclideanSpace ℝ (Fin n))
/-- The subgroup of rotation about a spatial point `p : EuclideanSpace ℝ (Fin n)`
elements of the form $T(p) * r * T(-p)$ with $T(\cdot) : \text{translationVector.incl } n$
and $r : \text{RotationGroup}$ where r is viewed as a rotation about the origin. Not

```

$p) = T(p)^{-1}$ .

-/

```
noncomputable def RotationsAbout : Subgroup (EuclideanGroup n) where
 carrier := {g | ∃ r : RotationGroup n, g = translationVector.incl n (Multiplicative.ofAdd (-
 * (r : EuclideanGroup n) * translationVector.incl n (Multiplicative.ofAdd (-
p)))}
```

```
 mul_mem' {a b} ha hb := by
```

```
 obtain ⟨r1, hr1⟩ := ha
```

```
 obtain ⟨r2, hr2⟩ := hb
```

```
 use r1 * r2
```

```
 rw [hr1, hr2]
```

```
 simp only [ofAdd_neg, map_inv, conj_mul, Subgroup.coe_mul]
```

```
 one_mem' := by
```

```
 simp; use 1
```

```
 constructor <=> simp
```

```
 inv_mem' {a} ha := by
```

```
 obtain ⟨ra, hra⟩ := ha
```

```
 use ra-1
```

```
 rw [hra]
```

```
 simp [mul_inv_rev, mul_assoc]
```

```
/-- The inclusion of rotations about `p` into the Euclidean group. -
```

```
/
```

```
noncomputable def RotationsAbout.incl : RotationsAbout p →* EuclideanGroup n :=
 (RotationsAbout p).subtype
```

```
/-- Conjugate a rotation about `p` back to a rotation about the origin. -
```

```
/
```

```
noncomputable def RotationsAbout.toOrigin :
```

```
 RotationsAbout p →* RotationGroup n where
```

```
 toFun g := ⟨translationVector.incl n (Multiplicative.ofAdd (-
p))
```

```
 * (g : EuclideanGroup n) * translationVector.incl n (Multiplicative.ofAdd p), by
```

```
 obtain ⟨g, hg⟩ := g
```

```
 obtain ⟨r, hr⟩ := hg
```

```
 simp; rw [hr]; simp [mul_assoc]
```

```
 map_one' := by simp
```

```
 map_mul' := by
```

```
 intro x y
```

```
 obtain ⟨a, ha⟩ := x
```

```
 obtain ⟨b, hb⟩ := y
```

```
 obtain ⟨r1, hr1⟩ := ha
```

```
 obtain ⟨r2, hr2⟩ := hb
```

```
 apply Subtype.ext
```

```
 simp only [Subgroup.coe_mul]
```

```
 rw [hr1, hr2]
```

```

 simp [mul_assoc]

/-- Conjugate a rotation about the origin to a rotation about `p`. -
/
noncomputable def RotationsAbout.fromOrigin :
 RotationGroup n →* RotationsAbout p where
 toFun g := (translationVector.incl n (Multiplicative.ofAdd p)
 * (g : EuclideanGroup n) * translationVector.incl n (Multiplicative.ofAdd
p)), by use g)
 map_one' := by simp
 map_mul' := by
 intro x y
 obtain ⟨a, ha⟩ := x
 obtain ⟨b, hb⟩ := y
 obtain ⟨r1, hr1⟩ := ha
 obtain ⟨r2, hr2⟩ := hb
 simp

/-- `RotationsAbout.toOrigin p` followed by `RotationsAbout.fromOrigin p` is
forward leg of the isomorphism `RotationsAboutEquiv`. -/
lemma RotationsAbout.fromOrigin_comp_toOrigin :
 (RotationsAbout.fromOrigin p).comp (RotationsAbout.toOrigin p) =
 MonoidHom.id (RotationsAbout p) := by
 apply MonoidHom.ext
 intro x
 apply Subtype.ext
 simp only [MonoidHom.coe_comp, Function.comp_apply, MonoidHom.id_apply, S
 unfold RotationsAbout.toOrigin
 unfold RotationsAbout.fromOrigin
 simp [mul_assoc]

/-- `RotationsAbout.fromOrigin p` followed by `RotationsAbout.toOrigin p` is
backward leg of the isomorphism `RotationsAboutEquiv`. -/
lemma RotationsAbout.toOrigin_comp_fromOrigin :
 (RotationsAbout.toOrigin p).comp (RotationsAbout.fromOrigin p) =
 MonoidHom.id (RotationGroup n) := by
 apply MonoidHom.ext
 intro x
 apply Subtype.ext
 simp only [MonoidHom.coe_comp, Function.comp_apply, MonoidHom.id_apply, S
 unfold RotationsAbout.toOrigin
 unfold RotationsAbout.fromOrigin
 simp [mul_assoc]

/-- API feature: conjugation by the translation `T(p)` exhibits the rotation
isomorphic to the rotations about the origin `RotationGroup n`. -

```

1.270.

PHYSLIB/SPACEANDTIME/SPACE/EUCLIDEANGROUP/BASIC.LEAN 433

```
/
noncomputable def RotationsAboutEquiv : RotationsAbout p $\approx$ * RotationGroup n :=
 MonoidHom.toMulEquiv (RotationsAbout.toOrigin p) (RotationsAbout.fromOrigin p)
 (RotationsAbout.fromOrigin_comp_toOrigin p) (RotationsAbout.toOrigin_comp_fromOrigin p)

/-- API feature: the degenerate identity that `RotationsAbout 0 = RotationGroup n` -
/
lemma RotationsAbout_zero : RotationsAbout (0 : EuclideanSpace $\mathbb{R}$ (Fin n)) = RotationGroup n
 apply Subgroup.ext
 intro g
 constructor
 · intro hg
 obtain ⟨g1, hg1⟩ := hg
 simp at hg1
 rw [hg1]
 simp
 · intro hg
 use ⟨g, hg⟩
 simp
/-- Rotations are members of special orthogonal groups and can be viewed as members
orthogonal groups. -/
def specialOrthogonal.incl (n : $\mathbb{N}$) :
 Matrix.specialOrthogonalGroup (Fin n) $\mathbb{R}$ $\rightarrow$ * Matrix.orthogonalGroup (Fin n) $\mathbb{R}$:=
 Submonoid.inclusion Matrix.specialUnitaryGroup_le_unitaryGroup

/-- The Euclidean group element given by a rotation about the origin (zero translation) -
/
def ofRotation (Q : Matrix.specialOrthogonalGroup (Fin n) $\mathbb{R}$) :
 EuclideanGroup n := {0, specialOrthogonal.incl n Q}

/-- Specialization to a group element from a rotation and a translation. -
/
def ofRotationTranslation (Q : Matrix.specialOrthogonalGroup (Fin n) $\mathbb{R}$)
 (t : EuclideanSpace $\mathbb{R}$ (Fin n)) : EuclideanGroup n :=
 {t, specialOrthogonal.incl n Q}

/-- The specialization projects back to the translation component. -
/
@[simp]
lemma ofRotationTranslation_translation (Q : Matrix.specialOrthogonalGroup (Fin n) $\mathbb{R}$)
 (t : EuclideanSpace $\mathbb{R}$ (Fin n)) :
 (ofRotationTranslation Q t).translation = t := rfl

/-- The specialization projects back to the linear (rotation) component. -
/
@[simp]
```

```

lemma ofRotationTranslation_linear (Q : Matrix.specialOrthogonalGroup (Fin
 (t : EuclideanSpace ℝ (Fin n))) :
 (ofRotationTranslation Q t).linear = specialOrthogonal.incl n Q := rfl

/-- API feature: the inclusion image decomposes as group product. -
/
@[simp]
lemma ofRotationTranslation_decompose (Q : Matrix.specialOrthogonalGroup (F
 (t : EuclideanSpace ℝ (Fin n))) :
 (ofRotationTranslation Q t) =
 (translationVector.incl n (Multiplicative.ofAdd t)) * (ofRotation (Q))
refine EuclideanGroup.ext ?_ ?_
· show t = t + (1 : Matrix.orthogonalGroup (Fin n) ℝ) • 0
 rw [smul_zero, add_zero]
· show specialOrthogonal.incl n Q = 1 * specialOrthogonal.incl n Q
 rw [one_mul]

/-- The isomorphism's forward map: a special orthogonal matrix as a rotatio
/
noncomputable def specialOrthogonal.toRotation (n : ℕ) :
 Matrix.specialOrthogonalGroup (Fin n) ℝ →* RotationGroup n where
toFun g := ⟨ofRotation g, by
 refine {?_, ?_}
 · show (ofRotation g).linear.val.det = 1
 exact (Matrix.mem_specialOrthogonalGroup_iff.mp g.property).right
 · show (ofRotation g).translation = 0
 rfl)
map_one' := rfl
map_mul' x y := by
 apply Subtype.ext
 refine EuclideanGroup.ext ?_ ?_
 · show (0 : EuclideanSpace ℝ (Fin n)) =
 0 + (specialOrthogonal.incl n x : Matrix.orthogonalGroup (Fin n) ℝ)
 rw [smul_zero, add_zero]
 · show specialOrthogonal.incl n (x * y)
 = specialOrthogonal.incl n x * specialOrthogonal.incl n y
 rw [map_mul]

/-- The isomorphism's inverse map: the linear part of a rotation about the
orthogonal matrix. -/
noncomputable def specialOrthogonal.fromRotation (n : ℕ) :
 RotationGroup n →* Matrix.specialOrthogonalGroup (Fin n) ℝ where
toFun g := ⟨g.val.linear, ⟨g.val.linear.property, (g.property).left⟩⟩
map_one' := rfl
map_mul' _ _ := rfl

```

```

/-- `specialOrthogonal.toRotation n` followed by `specialOrthogonal.fromRotation n`
the forward leg of the isomorphism `specialOrthogonalEquiv`. -
/
lemma specialOrthogonal.fromRotation_comp_toRotation :
 (specialOrthogonal.fromRotation n).comp (specialOrthogonal.toRotation n) =
 MonoidHom.id (Matrix.specialOrthogonalGroup (Fin n) ℝ) := by
 apply MonoidHom.ext
 intro x
 rfl

/-- `specialOrthogonal.fromRotation n` followed by `specialOrthogonal.toRotation n`
the backward leg of the isomorphism `specialOrthogonalEquiv`. -
/
lemma specialOrthogonal.toRotation_comp_fromRotation :
 (specialOrthogonal.toRotation n).comp (specialOrthogonal.fromRotation n) =
 MonoidHom.id (RotationGroup n) := by
 apply MonoidHom.ext
 intro x
 apply Subtype.ext
 refine EuclideanGroup.ext ?_ ?_
 · show (0 : EuclideanSpace ℝ (Fin n)) = x.val.translation
 have h : x.val.translation = 0 := x.property.right
 rw [h]
 · rfl

/-- API feature: $SO(n) \approx^* \text{RotationGroup } n$ -/
noncomputable def specialOrthogonalEquiv :
 Matrix.specialOrthogonalGroup (Fin n) ℝ $\approx^*$ RotationGroup n :=
 MonoidHom.toMulEquiv (specialOrthogonal.toRotation n) (specialOrthogonal.fromRotation n)
 specialOrthogonal.fromRotation_comp_toRotation specialOrthogonal.toRotation_comp_fromRotation

end EuclideanGroup

```

### 1.271 Physlib/SpaceAndTime/Space/Integrals/Basic.lean

```

- `integral_volume_eq_spherical` : The integral of a function over `Space d` with
- `lintegral_volume_eq_spherical` : The lower Lebesgue integral of a function over `
lemma integral_volume_eq_spherical (d : ℕ) [NeZero d] (f : Space d → F)
 ∫ x, f x ∂volume = ∫ x, f (x.2.1 • x.1.1) ∂(volume (α := Space d).toSphere.prod
 (Measure.volumeIoiPow (Module.finiteRank ℝ (Space d) - 1))) := by
 (volume (α := Space d)))
 (Homeomorph.measurableEmbedding (homeomorphUnitSphereProd (Space d))))]
simp only [homeomorphUnitSphereProd_apply_snd_coe, homeomorphUnitSphereProd_apply_snd_coe]
let f' : (x : (Space d)) → F := fun x => f (‖x‖ • ‖x‖-1 • x)

```

```

have h1 : $\forall^m x \in \partial(\text{volume } (\alpha := \text{Space } d)), x \neq 0 := \text{by}$
lemma integrable_spherical_of_integrable {d : $\mathbb{N}$ } {F : Type*}
 [NormedAddCommGroup F] [NormedSpace $\mathbb{R}$ F] {f : Space d $\rightarrow$ F}
 (hf : Integrable f volume) :
 Integrable
 (fun x : $\uparrow(\text{Metric.sphere } (0 : \text{Space } d) 1) \times \text{Set.Ioi } (0 : \mathbb{R}) \Rightarrow$
 f (x.2.1 • x.1.1))
 (volume ($\alpha := \text{Space } d$).toSphere.prod
 (Measure.volumeIoiPow (Module.finiteRank $\mathbb{R}$ (Space d) - 1))) := by
let s : Set (Space d) := {0}c
have h1 : Integrable (fun x : s $\Rightarrow$ f x.1)
 (.comap (Subtype.val (p := fun x $\Rightarrow$ x $\in$ s)) volume) := by
change Integrable (f • Subtype.val)
 (.comap (Subtype.val (p := fun x $\Rightarrow$ x $\in$ s)) volume)
rw [← MeasureTheory.integrableOn_iff_comap_subtypeVal]
· exact hf.integrableOn
· simp [s]
have he := MeasureTheory.Measure.measurePreserving_homeomorphUnitSphereProd
 (volume ($\alpha := \text{Space } d$))
have hcomp :
 Integrable
 ((fun x : s $\Rightarrow$ f x.1) • (homeomorphUnitSphereProd (Space d)).symm)
 (volume ($\alpha := \text{Space } d$).toSphere.prod
 (Measure.volumeIoiPow (Module.finiteRank $\mathbb{R}$ (Space d) - 1))) := by
rw [← he.integrable_comp_emb]
· convert h1
 simp only [Function.comp_apply, Homeomorph.symm_apply_apply]
· exact Homeomorph.measurableEmbedding (homeomorphUnitSphereProd (Space d))
exact hcomp

lemma integral_volume_eq_spherical_iterated {d : $\mathbb{N}$ } [NeZero d] {F : Type*}
 [NormedAddCommGroup F] [NormedSpace $\mathbb{R}$ F] (f : Space d $\rightarrow$ F)
 (hf : Integrable f volume) :
 $\int x : \text{Space } d, f x =$
 $\int n : \uparrow(\text{Metric.sphere } (0 : \text{Space } d) 1),$
 $\int r : \text{Set.Ioi } (0 : \mathbb{R}), f (r.1 \bullet n.1)$
 $\partial(\text{Measure.volumeIoiPow } (\text{Module.finiteRank } \mathbb{R} (\text{Space } d) - 1))$
 $\partial(\text{volume } (\alpha := \text{Space } d).toSphere) := \text{by}$
rw [integral_volume_eq_spherical]
rw [MeasureTheory.integral_prod]
exact integrable_spherical_of_integrable hf

lemma integral_volume_eq_spherical_integral {d : $\mathbb{N}$ } [NeZero d] {F : Type*}
 [NormedAddCommGroup F] [NormedSpace $\mathbb{R}$ F] (f : Space d $\rightarrow$ F)
 (hf : Integrable f volume) :
 $\int x : \text{Space } d, f x =$

```



1.272. *PHYSLIB/SPACEANDTIME/SPACE/INTEGRALS/NORMPOW.LEAN* 437

```

 ∫ n : ↑(Metric.sphere (0 : Space d) 1),
 ∫ r : Set.Ioi (0 : ℝ), (r.1 ^ (d - 1)) • f (r.1 • n.1)
 ∂(.comap Subtype.val volume)
 ∂(volume (α := Space d).toSphere) := by
 rw [integral_volume_eq_spherical_iterated f hf]
 congr
 funext n
 simp [Measure.volumeIoiPow]
 erw [integral_withDensity_eq_integral_smul (by fun_prop)]
 congr
 funext r
 have hr : 0 ≤ (r : ℝ) := le_of_lt r.2
 rw [NNReal.smul_def, Real.coe_toNNReal _ (pow_nonneg hr (d - 1))]

lemma lintegral_volume_eq_spherical (d : ℕ) [NeZero d]
 (f : Space d → ENNReal) (hf : Measurable f) :
 ∫- x, f x ∂volume = ∫- x, f (x.2.1 • x.1.1) ∂(volume (α := Space d).toSphere.prod
 (Measure.volumeIoiPow (Module.finrank ℝ (Space d) - 1))) := by
 (volume (α := Space d)))
 simp only [homeomorphUnitSphereProd_apply_snd_coe, homeomorphUnitSphereProd_apply_
 let f' : (x : (Space d)) → ENNReal := fun x => f (||↑x|| • ||↑x||-1 • ↑x)
lemma lintegral_volume_eq_spherical_mul (d : ℕ) [NeZero d]
 (f : Space d → ENNReal) (hf : Measurable f) :
 ∫- x, f x ∂volume = ∫- x, f (x.2.1 • x.1.1) * .ofReal (x.2.1 ^ (d - 1))
 ∂(volume (α := Space d).toSphere.prod (Measure.volumeIoiPow 0)) := by
 simp only [finrank_eq_dim, pow_zero, ENNReal.ofReal_one]

```

## 1.272 Physlib/SpaceAndTime/Space/Integrals/NormPow.lean

```

public import Physlib.SpaceAndTime.Space.IsDistBounded
- `radial_norm_power_spherical_integral_eq_space_integral`: The spherical-
coordinate integral
 identity for integer powers of the norm against a Schwartz map.
open MeasureTheory SchwartzMap
lemma radial_jacobian_zpow_mul_self
 {d p : ℕ} [NeZero d] {q : ℤ} (hp_int : (p : ℤ) = q + (d : ℤ))
 {r : ℝ} (hr : 0 < r) :
 r ^ (d - 1) * (r ^ q * r) = r ^ p := by
 have hz : r ≠ 0 := ne_of_gt hr
 calc
 r ^ (d - 1) * (r ^ q * r)
 = r ^ (d - 1 : ℤ) * (r ^ q * r ^ (1 : ℤ)) := by
 rw [← zpow_natCast, zpow_one, ← Nat.cast_pred (Nat.pos_of_neZero d)]
 _ = r ^ ((d - 1 : ℤ) + (q + 1)) := by

```

```

 rw [← zpow_add0 hz q 1, ← zpow_add0 hz (d - 1 : ℤ) (q + 1)]
_ = r ^ (p : ℤ) := by
 congr 1; omega
_ = r ^ p := by
 rw [zpow_natCast]

private lemma radial_jacobian_zpow
 {d p : ℕ} [NeZero d] {q : ℤ} (hp_int : (p : ℤ) = q + (d : ℤ))
 (hp_pos : 0 < p) {r : ℝ} (hr : 0 < r) :
 r ^ (d - 1) * r ^ q = r ^ (p - 1) := by
 have hz : r ≠ 0 := ne_of_gt hr
 calc
 r ^ (d - 1) * r ^ q
 = r ^ ((d - 1 : ℤ) + q) := by
 rw [← Nat.cast_pred (Nat.pos_of_neZero d), ← zpow_natCast, ← zpow_add]
_ = r ^ ((p - 1 : ℕ) : ℤ) := by
 congr 1; omega
_ = r ^ (p - 1) := by
 rw [zpow_natCast]

lemma radial_norm_power_spherical_integral_eq_space_integral
 {d p : ℕ} [NeZero d] {q : ℤ} (hp_int : (p : ℤ) = q + (d : ℤ))
 (hp_pos : 0 < p) (η : ℝ (Space d, ℝ)) :
 ∫ x : Space d, η x * ||x|| ^ q =
 ∫ (n : ↑(Metric.sphere (0 : Space d) 1)),
 ∫ (r : Set.Ioi (0 : ℝ)),
 r.1 ^ (p - 1) * η (r.1 • n.1)
 ∂(.comap Subtype.val volume)
 ∂(volume (α := Space d).toSphere) := by
 have hf : Integrable (fun x : Space d => η x * ||x|| ^ q) volume :=
 IsDistBounded.integrable_space_mul (IsDistBounded.pow q (by omega)) η
 rw [integral_volume_eq_spherical_integral
 (d := d) (fun x : Space d => η x * ||x|| ^ q) hf]
 congr
 funext n
 congr
 funext r
 have hr : 0 < (r : ℝ) := r.2
 have hnorm := norm_smul_sphere n (le_of_lt hr)
 simp only [smul_eq_mul]
 rw [hnorm, ← radial_jacobian_zpow hp_int hp_pos hr]
 ring

lemma integrable0n_norm_rpow_ball_iff {d : ℕ} [NeZero d] {b : ℝ} (hb : 0 <
 root.MeasureTheory.integrable0n_indicator_iff measurableSet_Ioo, hI
 Nat.cast_pred (Nat.pos_of_neZero d)]

```

1.273.

*PHYSLIB/SPACEANDTIME/SPACE/INTEGRALS/RADIALANGULARMEASURE.lean*

```
lemma integrable0n_norm_rpow_ball_compl_iff {d : ℕ} [NeZero d] {a : ℝ} (ha : 0 < a)
 root.MeasureTheory.integrable0n_indicator_iff measurableSet_Ici, hInter,
 Nat.cast_pred (Nat.pos_of_neZero d)]
lemma integrable0n_norm_rpow_iff_of_isBounded_nhds {d : ℕ} [NeZero d] {s : Set (Space d)}
 · exact (integrable0n_norm_rpow_ball_iff ha p).mp (Integrable0n.mono_set h ha')
 · exact Integrable0n.mono_set ((integrable0n_norm_rpow_ball_iff hb p).mpr h) hb'
lemma integrable0n_norm_rpow_of_compl_nhds {d : ℕ} [NeZero d] {s : Set (Space d)}
 exact Integrable0n.mono_set ((integrable0n_norm_rpow_ball_compl_iff ha p).mpr h) h
```

### 1.273 Physlib/SpaceAndTime/Space/Integrals/RadialAngularMeasure.lean

```
rw [NNReal.smul_def, Real.coe_toNNReal _ (by positivity)]
all_goals fun_prop
lemma lintegral_radialMeasure_eq_spherical_mul (d : ℕ) [NeZero d]
 (f : Space d → ENNReal) (hf : Measurable f) :
 ∂(volume (α := Space d).toSphere.prod (Measure.volumeIoiPow 0)) := by
 have hpos : (0 : ℝ) < x.2 := x.2.2
 have hnorm : ||(x.2 : ℝ) • (x.1 : Space d)|| = x.2 := by
 rw [norm_smul, mem_sphere_zero_iff_norm.mp x.1.2, mul_one, Real.norm_eq_abs,
 abs_of_nonneg hpos.le]
 rw [hnorm, mul_right_comm, ← ENNReal.ofReal_mul (by positivity), one_div,
 inv_mul_cancel₀ (by positivity), ENNReal.ofReal_one, one_mul]
all_goals fun_prop
lemma integrable0n_radialAngularMeasure_of_spherical {d : ℕ} [NeZero d] (f : Space d → ℝ)
 (volume (α := Space d).toSphere.prod (Measure.volumeIoiPow 0)) :
 have hpre : (fun x : ℝ => (1 : ℝ) + x)⁻¹' Set.Ioi 1 = Set.Ioi 0 := by
 ext x
 simp
 have integrable0n_rpow_neg :
 Integrable0n (fun x : ℝ => ((1 : ℝ) + x) ^ (- (n + 2) : ℝ)) (Set.Ioi 0) := by
 rw [← hpre]
 exact ((measurePreserving_add_left volume (1 : ℝ)).integrable0n_comp_preimage
 (measurableEmbedding_addLeft 1)
 (f := fun y : ℝ => y ^ (- (n + 2) : ℝ)) (s := Set.Ioi 1)).mpr
 (integrable0n_Ioi_rpow_of_lt (by linarith [Nat.cast_nonneg (α := ℝ) n]) one_pos))
 refine integrable0n_rpow_neg.congr_fun (fun x hx => ?_) measurableSet_Ioi
 rw [Set.mem_Ioi] at hx
 rw [abs_of_nonneg (Real.rpow_nonneg (by linarith) _)]
 rw [MeasureTheory.integrable_prod_iff (AEMeasurable.astronglyMeasurable (by fun_p
 refine ((MeasureTheory.integrable0n_iff_comap_subtypeVal measurableSet_Ioi).mp
 (integrable_neg_pow_on_ioi dml)).congr ?_
 filter_upwards with y
 have hy : (0 : ℝ) < y.1 := y.2
 simp only [Function.comp_apply]
```

```

rw [abs_of_pos hy, abs_of_nonneg (Real.rpow_nonneg (by linarith) _)]
congr 1
ring

```

### 1.274 Physlib/SpaceAndTime/Space/IsDistBounded.lean

```

/-- The boundedness condition on a function `Space d → F`
· apply MeasureTheory.integrable_finsetSum
· apply MeasureTheory.integrable_finsetSum
 simp_all only [add_sub_cancel_right, Prod.snd_add]
 exact h1
| 0 =>
 simp_all only [zero_tsub, CharP.cast_eq_zero, neg_zero, Real.norm_eq_
 radialAngularMeasure_zero_eq_volume]
 exact h1
convert! h1 using 1
rw [mul_comm, ← zpow_natCast, abs_of_nonneg (by positivity)]
· apply MeasureTheory.integrable_finsetSum
· apply le_of_eq
 ring
convert! h0 using 1
(hf : IsDistBounded f) (c : ℝ) : IsDistBounded (fun x => c • f x) := hf
(hf : IsDistBounded f) (c : ℝ) : IsDistBounded (fun x => c * f x) := hf
IsDistBounded (fun x => f (x - c)) := hf.comp_add_right (- c)
lemma inv_shift {d : ℕ} (g : Space d) (hd : 2 ≤ d := by omega) :
 IsDistBounded (d := d) (fun x => ||x - g||-1) := by
 convert IsDistBounded.pow_shift (d := d) (-1) g (by omega) using 1
lemma inv {d : ℕ} (hd : 2 ≤ d := by omega) :
 IsDistBounded (d := d) (fun x => ||x||-1) := by
 convert IsDistBounded.pow (d := d) (-1) (by omega) using 1
lemma log_norm {d : ℕ} (hd : 2 ≤ d := by omega) :
 IsDistBounded (d := d) (fun x => Real.log ||x||) := by
 simp only [Real.norm_eq_abs]
 IsDistBounded (fun x => ||x|| * f x) := hf.norm_smul_isDistBounded
 IsDistBounded (fun x => x i * f x) := hf.component_smul_isDistBounded i
 (hf : IsDistBounded f) (y : Space d) : IsDistBounded (fun x => □y, x□_ℝ
 hf.isDistBounded_smul_inner y
 convert! hf.isDistBounded_smul_inner y using 2
 IsDistBounded (fun x => □y, x□_ℝ * f x) := hf.isDistBounded_smul_inner
lemma mul_inner_pow_neg_two {d : ℕ} (y : Space d) (hd : 2 ≤ d := by omega)
· apply IsDistBounded.pow (d := d) (-1) (by omega)

```

**1.275 Physlib/SpaceAndTime/Space/LengthUnit.lean**

```

simp [toReal]
· exact zero_le
rfl
simp only [one_div, NNReal.coe_inv]
rw [toReal, inv_div]
 (x / y : ℝ) * (y / z : ℝ) = x / z := by
simp [div_eq_val, toReal]

```

```

rw [toReal]

```

The references for the numerical definitions used below are:

- \* the BIPM SI Brochure for the meter, the speed of light, and SI prefixes:  
<https://www.bipm.org/documents/d/guest/si-brochure-9-en-pdf>
- \* NIST Handbook 44, Appendix C, for the international foot-based units and  
the international nautical mile:  
<https://doi.org/10.6028/NIST.HB.44-2023>
- \* IAU 2012 Resolution B2 for the astronomical unit:  
[https://iauarchive.eso.org/static/resolutions/IAU2012\\_English.pdf](https://iauarchive.eso.org/static/resolutions/IAU2012_English.pdf)
- \* the IAU Style Manual recommendations for the Julian year convention used in  
the light-year:  
[https://iauarchive.eso.org/publications/proceedings\\_rules/units/](https://iauarchive.eso.org/publications/proceedings_rules/units/)
- \* IAU 2015 Resolution B2 for the exact parsec convention:  
[https://iauarchive.eso.org/static/resolutions/IAU2015\\_English.pdf](https://iauarchive.eso.org/static/resolutions/IAU2015_English.pdf)

```

NNReal.eq <| by simp [miles, yards]; rw [toReal]; norm_num
simp [furlongs, yards]; rw [toReal]; norm_num

```

**1.276 Physlib/SpaceAndTime/Space/Module.lean**

```

public import Physlib.SpaceAndTime.Space.Origin
public import Mathlib.Analysis.Calculus.ContDiff.WithLp
public import Mathlib.Analysis.Calculus.FDeriv.WithLp
Instances in Lean

```

In Lean, an ``instance`` supplies a typeclass automatically. When a definition or theorem needs a structure such as ``AddCommGroup (Space d)``, ``Module ℝ (Space d)``, ``NormedAddCommGroup (Space d)``, ``InnerProductSpace ℝ (Space d)``, or ``MeasurableSpace (Space d)``, typeclass inference searches for the corresponding instance and inserts it without the user passing it explicitly.

These instances make ``Space d`` usable with standard mathematical notation and

with the Mathlib API. For example, they allow expressions such as ``p + q``, ``c • p``, ``||p||``, ``inner ℝ p q``, and measurable-set arguments involving the Borel structure. They also make general theorems about modules, normed groups, in product spaces, and measurable spaces apply directly to ``Space d``.

For ``Space d``, these instances are intentional choices rather than inheritance facts: the type was defined as a structure instead of an abbreviation for Euclidean space. In particular, the additive and module structures choose an origin, while the norm, inner product, and Borel structure choose the standard Euclidean coordinate geometry.

```
lemma norm_smul_sphere {d : ℕ} (n : ↑(Metric.sphere (0 : Space d) 1))
 {r : ℝ} (hr : 0 ≤ r) :
 ||(r • (n : Space d))|| = r := by
 simp [norm_smul, mem_sphere_zero_iff_norm.mp n.2, abs_of_nonneg hr]
```

A basis in Lean is typically represented by ``Module.Basis ι R M``: an indexed family of vectors in an ``R``-module ``M`` such that every element of ``M`` has a unique finite linear expansion in those vectors. The index type ``ι`` names the basis vectors, and the map ``basis.repr`` gives the coordinate representation of a vector with respect to that basis.

For inner product spaces, Lean also has ``OrthonormalBasis ι R M``. This is a basis whose vectors are orthonormal, packaged together with a linear isometry equivalence between ``M`` and its coordinate space. It can be coerced to the underlying ``Module.Basis`` using ``basis.toBasis`` when only the linear-algebraic basis structure is needed.

The standard basis below is indexed by ``Fin d``, so the basis vector ``basis i`` is the unit vector in the ``i``th coordinate direction of ``Space d``.

```
-/
 ContinuousLinearMap.coe_comp, Function.comp_apply]
 that equivalent to `[] d` and that equivalent to `[](ℝ, Space d)` -
/
noncomputable def modelDiffeo {d} : Diffeomorph ([] d) [](ℝ, Space d) (Space
 simp [← Function.id_def, homEuclideanSpaceSpace] using by fun_prop
 apply contMDiff_iff.mpr {by simp using by fun_prop, fun x y => ?_}
 simp [homEuclideanSpaceSpace] using by fun_prop
lemma modelDiffeo_apply {d : ℕ} (p : Space d) :
 basis μ = mfdderiv ([] d) [](ℝ, Space d) (modelDiffeo (d := d)) x
 simp only [modelDiffeo_apply, mfdderiv, writtenInExtChartAt, extChartAt,
 OpenPartialHomeomorph.extend, OpenPartialHomeomorph.refl_partialEquiv,
 OpenPartialHomeomorph.singletonChartedSpace_chartAt_eq, modelWithCorner,
 PartialEquiv.trans_refl, PartialEquiv.refl_coe, Homeomorph.symm_toOpenP,
 OpenPartialHomeomorph.symm_toPartialEquiv, PartialEquiv.symm_symm,
```

```

 OpenPartialHomeomorph.toFun_eq_coe, Homeomorph.toOpenPartialHomeomorph_apply,
 CompTriple.comp_eq, modelWithCornersSelf_coe, Set.range_id,
 OpenPartialHomeomorph.coe_toPartialEquiv_symm, Homeomorph.toOpenPartialHomeomorph
 fderivWithin_univ]
have h := fderiv_space_components i ((↑modelDiffeo ∘ ↑(homEuclideanSpaceSpace d)))
 (by simpa [Function.comp_def, homEuclideanSpaceSpace] using by fun_prop)
 (((homEuclideanSpaceSpace d).symm x)) ((EuclideanSpace.single μ 1))
convert! h.symm
simp only [basis_apply, homEuclideanSpaceSpace, PiLp.continuousLinearEquiv_symm_ap
 Homeomorph.homeomorph_mk_coe, Equiv.coe_fn_mk, Function.comp_apply, modelDiffeo
 PiLp.continuousLinearEquiv_apply, Homeomorph.homeomorph_mk_coe_symm, Equiv.symm_
change _ = fderiv ℝ (EuclideanSpace.proj i) _ (EuclideanSpace.single μ 1)
simp only [ContinuousLinearMap.fderiv, PiLp.proj_apply, PiLp.single_apply]

```

### 1.277 Physlib/SpaceAndTime/Space/Norm/Basic.lean

```

public import Physlib.SpaceAndTime.Space.Derivatives.Laplacian
public import Physlib.SpaceAndTime.Space.Integrals.NormPow
public import Physlib.Mathematics.Distribution.PowMul
- `distDiv_norm_zpow_smul_repr_self_eq_smul` : The divergence of the distribution de
 `x ↦ ||x|| ^ q • x`.
- `distLaplacian_distOfFunction_norm_zpow` : The Laplacian of the distribution defin
 of the norm.
- `distDiv_inv_pow_eq_dim` : The divergence of `x ↦ ||x|| ^ (-
d) • x` equals `d * volume (ball 0 1)`
 times the Dirac delta at the origin.
- `distLaplacian_fundamentalSolution_norm_zpow` : The Laplacian of the power-
form fundamental
 solution `||x|| ^ (2 - d)`, in every dimension (trivial at `d = 0, 2`).
- `distLaplacian_fundamentalSolution_log_norm` : The Laplacian of the two-
dimensional logarithmic
 fundamental solution `Real.log ||x||`.
- B.3. Divergence of radial norm-power distributions
- B.4. The Laplacian of distributions based on powers
- B.5. Divergence equal dirac delta
- B.6. The Laplacian of the fundamental solution
simp only [one_div, mul_inv_rev, fderiv_add_const, FunLike.coe_smul, Pi.smul_apply]
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply, fderiv_eq_smul_deriv]
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply, fderiv_eq_smul_deriv]
 ∇d (distOfFunction (fun x : Space d => (normPowerSeries n x) ^ m) (by fun_prop))
lemma gradient_dist_normPowerSeries_zpow_tendsTo_distGrad_norm {d : ℕ} [NeZero d] (m
 (hm : - (d - 1 : ℕ) ≤ m) (η : □(Space d, ℝ))
 (y : EuclideanSpace ℝ (Fin d)) :
 □(∇d (distOfFunction

```

```

 (fun x : Space d => (normPowerSeries n x) ^ m) (by fun_prop))) η, y[ℝ]
 (□ (□ ∇d (distOfFunction (fun x : Space d => ||x|| ^ m)
change Filter.Tendsto (fun n => - ∫ (x : Space d),
 Filter.atTop (□ (- ∫ (x : Space d), fderiv ℝ η x (basis.repr.symm y) *
 simp only [norm_mul, Real.norm_eq_abs, mul_eq_mul_left_iff, abs_eq_zero]
lemma gradient_dist_normPowerSeries_zpow_tendsTo {d : ℕ} [NeZero d] (m : ℤ)
 (hm : - (d - 1 : ℕ) + 1 ≤ m)
 (η : □(Space d, ℝ)) (y : EuclideanSpace ℝ (Fin d)) :
 □(∇d (distOfFunction (fun x : Space d => (normPowerSeries n x) ^ m)
 (□ (□ distOfFunction (fun x : Space d => (m * ||x|| ^ (m - 2)) • basis.repr
 omega) η, y[ℝ])) := by
 have h1 (n : ℕ) (x : Space d) :
 omega
 ∇d (distOfFunction (fun x : Space d => Real.log (normPowerSeries n x))
lemma gradient_dist_normPowerSeries_log_tendsTo_distGrad_norm {d : ℕ} (hd :
 (η : □(Space d, ℝ)) (y : EuclideanSpace ℝ (Fin d)) :
 □(∇d (distOfFunction
 (fun x : Space d => Real.log (normPowerSeries n x)) (by fun_prop))) η,
 (□ (□ ∇d (distOfFunction (fun x : Space d => Real.log ||x||)
 haveI : NeZero d := ⟨by omega⟩
 ∫ (x : Space d), fderiv ℝ η x (basis.repr.symm y) * Real.log (normPower
 Filter.atTop (□ (- ∫ (x : Space d), fderiv ℝ η x (basis.repr.symm y) *
 simp only [norm_mul, Real.norm_eq_abs, mul_eq_mul_left_iff, abs_eq_zero]
lemma gradient_dist_normPowerSeries_log_tendsTo {d : ℕ} (hd : 2 ≤ d)
 (η : □(Space d, ℝ)) (y : EuclideanSpace ℝ (Fin d)) :
 □(∇d (distOfFunction (fun x : Space d => Real.log (normPowerSeries n x)
 (□ (□ distOfFunction (fun x : Space d => (||x|| ^ (- 2 : ℤ)) • basis.repr
 omega) η, y[ℝ])) := by
 haveI : NeZero d := ⟨by omega⟩
 simp only [distOfFunction_inner]
 have h1 (n : ℕ) (x : Space d) :
 enter [1, n, 2, x]
 simp only [basis_repr_inner_eq]
lemma distGrad_distOfFunction_norm_zpow {d : ℕ} [NeZero d]
 (m : ℤ) (hm : - (d - 1 : ℕ) + 1 ≤ m) :
 ∇d (distOfFunction (fun x : Space d => ||x|| ^ m)
 (IsDistBounded.pow m (by omega)))
 = distOfFunction (fun x : Space d => (m * ||x|| ^ (m - 2)) • basis.repr x)
 (gradient_dist_normPowerSeries_zpow_tendsTo_distGrad_norm m (by omega)
lemma distGrad_distOfFunction_log_norm {d : ℕ} (hd : 2 ≤ d := by omega) :
 ∇d (distOfFunction (fun x : Space d => Real.log ||x||)
 = distOfFunction (fun x : Space d => (||x|| ^ (- 2 : ℤ)) • basis.repr x)
 omega) := by
 (gradient_dist_normPowerSeries_log_tendsTo_distGrad_norm hd η y)
 (gradient_dist_normPowerSeries_log_tendsTo hd η y)
B.3. Divergence of radial norm-power distributions

```



```

-/
open Distribution

private lemma integrable_real_pow_mul_schwartz
 (ψ : ℝ → ℝ) (k : ℕ) :
 Integrable (fun x : ℝ => x ^ k * ψ x) volume := by
 refine (ψ.integrable_pow_mul volume k).mono' (by fun_prop) ?_
 filter_upwards with x
 simp [norm_mul, norm_pow]

private lemma radial_power_deriv_integral_by_parts
 {d : ℕ} (η : ℝ → ℝ)
 (n : ↑(Metric.sphere (0 : Space d) 1))
 (p : ℕ) (hp : 0 < p) :
 - ∫ (r : Set.Ioi (0 : ℝ)),
 r.1 ^ p * (deriv (fun a => η (a • n.1)) r.1)
 ∂(.comap Subtype.val volume)
 =
 (p : ℝ) * ∫ (r : Set.Ioi (0 : ℝ)),
 r.1 ^ (p - 1) * η (r.1 • n.1)
 ∂(.comap Subtype.val volume) := by
 let η' : ℝ → ℝ := SchwartzMap.compCLM (g := fun a => a • n.1) ℝ (by
 apply And.intro
 · fun_prop
 · intro n'
 match n' with
 | 0 =>
 use 1, 1
 simp [norm_smul]
 | 1 =>
 use 0, 1
 intro x
 simp [fderiv_smul_const, iteratedFDeriv_succ_eq_comp_right]
 rw [(continuousMultilinearCurryRightEquiv' ℝ 0 ℝ (Space d)).symm.norm_map]
 simp [ContinuousLinearMap.norm_smulRight_apply]
 | n' + 1 + 1 =>
 use 0, 0
 intro x
 simp only [Real.norm_eq_abs, pow_zero, mul_one, norm_le_zero_iff]
 rw [iteratedFDeriv_succ_eq_comp_right]
 conv_lhs =>
 enter [2, 3, y]
 simp [fderiv_smul_const]
 rw [iteratedFDeriv_succ_const]
 rfl) (by use 1, 1; simp [norm_smul]) η

```

```

have hη'_apply (x : ℝ) : η' x = η (x • n.1) := by
 simp [η']
have hmul_iter_apply :
 ∀ k x, ((Physlib.Distribution.powOneMul ℝ)^[k] η') x = x ^ k * η' x :
 intro k
 induction k with
 | zero =>
 intro x
 simp
 | succ k ih =>
 intro x
 rw [Function.iterate_succ_apply']
 rw [Physlib.Distribution.powOneMul_apply, ih]
 rw [pow_succ]
 change x * (x ^ k * η' x) = x ^ k * x * η' x
 ring
have hleft_subtype :
 ∫ (r : Set.Ioi (0 : ℝ)),
 r.1 ^ p * _root_.deriv (fun a => η (a • n.1)) r.1
 ∂(.comap Subtype.val volume)
 =
 ∫ (x : ℝ) in Set.Ioi (0 : ℝ),
 x ^ p * _root_.deriv (fun a => η (a • n.1)) x := by
 exact MeasureTheory.integral_subtype_comap (μ := volume)
 (s := Set.Ioi (0 : ℝ)) measurableSet_Ioi
 (fun x : ℝ => x ^ p * _root_.deriv (fun a => η (a • n.1)) x)
have hright_subtype :
 ∫ (r : Set.Ioi (0 : ℝ)),
 r.1 ^ (p - 1) * η (r.1 • n.1)
 ∂(.comap Subtype.val volume)
 =
 ∫ (x : ℝ) in Set.Ioi (0 : ℝ),
 x ^ (p - 1) * η (x • n.1) := by
 exact MeasureTheory.integral_subtype_comap (μ := volume)
 (s := Set.Ioi (0 : ℝ)) measurableSet_Ioi
 (fun x : ℝ => x ^ (p - 1) * η (x • n.1))
rw [hleft_subtype, hright_subtype]
have hIBP :
 ∫ (x : ℝ) in Set.Ioi (0 : ℝ),
 x ^ p * _root_.deriv (fun a => η (a • n.1)) x
 =
 (0 : ℝ) - (0 : ℝ) -
 ∫ (x : ℝ) in Set.Ioi (0 : ℝ),
 ((p : ℝ) * x ^ (p - 1)) * η (x • n.1) := by
 refine MeasureTheory.integral_Ioi_mul_deriv_eq_deriv_mul
 (a := (0 : ℝ))

```

```

(u := fun x : ℝ => x ^ p)
(u' := fun x : ℝ => (p : ℝ) * x ^ (p - 1))
(v := fun x : ℝ => η (x • n.1))
(v' := fun x : ℝ => _root_.deriv (fun a => η (a • n.1)) x)
(a' := (0 : ℝ)) (b' := (0 : ℝ)) ?_ ?_ ?_ ?_ ?_ ?_
· intro x hx
 simp using (hasDerivAt_pow p x)
· intro x hx
 exact DifferentiableAt.hasDerivAt (by fun_prop :
 DifferentiableAt ℝ (fun x : ℝ => η (x • n.1)) x)
· have hderiv_int :
 Integrable (fun x : ℝ =>
 x ^ p * ((SchwartzMap.derivCLM ℝ ℝ) η') x) volume :=
 integrable_real_pow_mul_schwartz ((SchwartzMap.derivCLM ℝ ℝ) η') p
exact hderiv_int.integrableOn.congr_fun (by
 intro x hx
 have hderiv_eq :
 root.deriv η' x = _root_.deriv (fun a => η (a • n.1)) x := by
 congr 1
 simp [SchwartzMap.derivCLM_apply, hderiv_eq])
 measurableSet_Ioi
· have hbase :
 Integrable (fun x : ℝ => x ^ (p - 1) * η' x) volume :=
 integrable_real_pow_mul_schwartz η' (p - 1)
have hconst :
 Integrable (fun x : ℝ => (p : ℝ) * (x ^ (p - 1) * η' x)) volume :=
 hbase.const_mul (p : ℝ)
exact hconst.integrableOn.congr_fun (by
 intro x hx
 ring_nf
 simp [hη'_apply, mul_assoc])
 measurableSet_Ioi
· have hcont :
 ContinuousAt (fun x : ℝ => x ^ p * η (x • n.1)) (0 : ℝ) := by
 fun_prop
have hlim := tendsto_nhdsWithin_of_tendsto_nhds
 (s := Set.Ioi (0 : ℝ)) hcont.tendsto
simp only [ne_eq, hp.ne', not_false_eq_true, zero_pow, zero_smul, zero_mul] at
exact hlim
· have hzero :
 Filter.Tendsto (fun x : ℝ => x ^ p * η' x) Filter.atTop (□ (0 : ℝ)) := by
 have hsch :
 Filter.Tendsto (fun x : ℝ => ((Physlib.Distribution.powOneMul ℝ)^[p] η')
 Filter.atTop (□ (0 : ℝ)) :=
 Filter.Tendsto.mono_left
 (((Physlib.Distribution.powOneMul ℝ)^[p] η').toZeroAtInfty.zero_at_infty

```

```

 atTop_le_cocompact
 exact hsch.congr' (Filter.Eventually.of_forall (fun x => by
 rw [hmul_iter_apply p x]))
 exact hzero
calc
- ∫ (x : ℝ) in Set.Ioi (0 : ℝ),
 x ^ p * _root_.deriv (fun a => η (a • n.1)) x
= ∫ (x : ℝ) in Set.Ioi (0 : ℝ),
 ((p : ℝ) * x ^ (p - 1)) * η (x • n.1) := by
 rw [hIBP]
 ring
_ = (p : ℝ) * ∫ (x : ℝ) in Set.Ioi (0 : ℝ),
 x ^ (p - 1) * η (x • n.1) := by
 rw [← integral_const_mul]
 congr
 funext x
 ring

private lemma distDiv_norm_zpow_smul_repr_self_apply_eq_radial_deriv
 {d p : ℕ} [NeZero d] (q : ℤ) (hq : 0 < q + (d : ℤ))
 (hp_int : (p : ℤ) = q + (d : ℤ))
 (η : □(Space d, ℝ)) :
 (∇d □ (distOfFunction (fun x : Space d => ||x|| ^ q • basis.repr x)
 (IsDistBounded.zpow_smul_repr_self q (by omega)))) η =
 - ∫ n, (∫ (r : Set.Ioi (0 : ℝ)),
 r.1 ^ p * (_root_.deriv (fun a => η (a • n.1)) r.1)
 ∂(.comap Subtype.val volume))
 ∂(volume (α := Space d).toSphere) := by
let F : Space d → ℝ := fun x =>
 inner ℝ (||x|| ^ q • basis.repr x) (grad η x)
calc
 (∇d □ (distOfFunction (fun x : Space d => ||x|| ^ q • basis.repr x)
 (IsDistBounded.zpow_smul_repr_self q (by omega)))) η
 = - ∫ x, F x := by
 rw [distDiv_ofFunction]
_ = - ∫ r, F (r.2.1 • r.1.1)
 ∂(volume (α := Space d).toSphere.prod
 (Measure.volumeIoiPow (Module.finrank ℝ (Space d) - 1))) := by
 rw [integral_volume_eq_spherical]
_ = - ∫ n, (∫ r, F (r.1 • n.1)
 ∂(Measure.volumeIoiPow (Module.finrank ℝ (Space d) - 1)))
 ∂(volume (α := Space d).toSphere) := by
 rw [MeasureTheory.integral_prod]
 exact integrable_isDistBounded_inner_grad_schwartzMap_spherical
 (IsDistBounded.zpow_smul_repr_self q (by omega)) η
_ = - ∫ n, (∫ (r : Set.Ioi (0 : ℝ)),

```

```

r.1 ^ p * (_root_.deriv (fun a => η (a • n.1)) r.1)
∂(.comap Subtype.val volume))
∂(volume (α := Space d).toSphere) := by
 congr
 funext n
 simp [F, Measure.volumeIoiPow]
 erw [integral_withDensity_eq_integral_smul (by fun_prop)]
 · congr
 funext r
 have hr : 0 < (r : ℝ) := r.2
 have hnorm := norm_smul_sphere n (le_of_lt hr)
 rw [NNReal.smul_def]
 rw [Real.coe_toNNReal _ (pow_nonneg (le_of_lt hr) (d - 1))]
 · simp only [smul_eq_mul]
 rw [hnorm]
 rw [← grad_smul_inner_space (n : Space d) (↑η)
 (SchwartzMap.differentiable η) (r : ℝ) hr]
 rw [real_inner_comm]
 simp only [inner_smul_right]
 rw [← radial_jacobian_zpow_mul_self hp_int hr]
 ring

lemma distDiv_norm_zpow_smul_repr_self_eq_smul
 {d : ℕ} [NeZero d] (q : ℤ) (hq : 0 < q + (d : ℤ)) :
 ∇d □ (distOfFunction (fun x : Space d => ||x|| ^ q • basis.repr x)
 (IsDistBounded.zpow_smul_repr_self q (by omega))) =
 (((q + d : ℤ) : ℝ) •
 distOfFunction (fun x : Space d => ||x|| ^ q)
 (IsDistBounded.pow q (by omega))) := by
 ext η
 let p : ℕ := Int.toNat (q + (d : ℤ))
 have hp_int : (p : ℤ) = q + (d : ℤ) := by
 dsimp [p]
 exact Int.toNat_of_nonneg (le_of_lt hq)
 have hp_pos : 0 < p := by
 have : (0 : ℤ) < (p : ℤ) := by
 rw [hp_int]
 exact hq
 exact_mod_cast this
 have hcoef : (((q + d : ℤ) : ℝ)) = (p : ℝ) := by
 exact_mod_cast hp_int.symm
 calc
 (∇d □ (distOfFunction (fun x : Space d => ||x|| ^ q • basis.repr x)
 (IsDistBounded.zpow_smul_repr_self q (by omega)))) η
 = - ∫ n, (∫ (r : Set.Ioi (0 : ℝ)),
 r.1 ^ p * (_root_.deriv (fun a => η (a • n.1)) r.1)

```

```

 ∂(.comap Subtype.val volume))
 ∂(volume (α := Space d).toSphere) := by
 exact distDiv_norm_zpow_smul_repr_self_apply_eq_radial_deriv q hq
_ = ∫ n, (p : ℝ) * ∫ (r : Set.Ioi (0 : ℝ)),
 r.1 ^ (p - 1) * η (r.1 • n.1)
 ∂(.comap Subtype.val volume)
 ∂(volume (α := Space d).toSphere) := by
 rw [← integral_neg]
 congr
 funext n
 exact radial_power_deriv_integral_by_parts η n p hp_pos
_ = (p : ℝ) * ∫ n : ↑(Metric.sphere (0 : Space d) 1),
 ∫ (r : Set.Ioi (0 : ℝ)),
 r.1 ^ (p - 1) * η (r.1 • n.1)
 ∂(.comap Subtype.val volume)
 ∂(volume (α := Space d).toSphere) := by
 rw [integral_const_mul]
_ = (p : ℝ) * ∫ x : Space d, η x * ||x|| ^ q := by
 rw [← radial_norm_power_spherical_integral_eq_space_integral hp_i]
_ = (((q + (d : ℤ) : ℤ) : ℝ) •
 distOfFunction (fun x : Space d => ||x|| ^ q)
 (IsDistBounded.pow q (by omega))) η := by
 simp [distOfFunction_apply, mul_comm]
 left
 rw [← hcoef]
 norm_num

/-!

B.4. The Laplacian of distributions based on powers

-/

lemma distLaplacian_distOfFunction_norm_zpow {d : ℕ} [NeZero d] (m : ℤ)
 (hdiv : 0 < m - 2 + (d : ℤ)) :
 Δd (distOfFunction (fun x : Space d => ||x|| ^ m)
 (IsDistBounded.pow m (by omega))) =
 (((m : ℝ) * ((m - 2 + d : ℤ) : ℝ)) •
 distOfFunction (fun x : Space d => ||x|| ^ (m - 2))
 (IsDistBounded.pow (m - 2) (by omega))) := by
 rw [distLaplacian]
 change ∇d □ (∇d (distOfFunction (fun x : Space d => ||x|| ^ m)
 (IsDistBounded.pow m (by omega)))) = _
 rw [distGrad_distOfFunction_norm_zpow m (by omega)]
 have hdist :
 distOfFunction (fun x : Space d => (m * ||x|| ^ (m - 2)) • basis.repr x

```

```

 (by
 simp [← smul_smul]
 refine IsDistBounded.const_fun_smul ?_ ↑m
 apply IsDistBounded.zpow_smul_repr_self
 omega) =
 (m : ℝ) • distOfFunction
 (fun x : Space d => ||x|| ^ (m - 2) • basis.repr x)
 (IsDistBounded.zpow_smul_repr_self (m - 2) (by omega)) := by
 convert distOfFunction_smul_fun
 (fun x : Space d => ||x|| ^ (m - 2) • basis.repr x)
 (IsDistBounded.zpow_smul_repr_self (m - 2) (by omega)) (m : ℝ) using 1
 ext x
 simp [smul_smul]
 rw [hdist]
 rw [map_smul]
 rw [distDiv_norm_zpow_smul_repr_self_eq_smul (m - 2) hdiv]
 rw [smul_smul]

/-!

B.5. Divergence equal dirac delta
/-- The distributional divergence of the radial field `x ↦ ||x|| ^ (-
d) • x` (i.e. `x / ||x|| ^ d`)
equals `d * volume (Metric.ball 0 1)` – the surface area of the unit sphere `S^{d-1}` – times the
Dirac delta at the origin. This is the Gauss-law identity underlying the fundamental
the Laplacian: away from `0` the field is divergence-free, and all of its flux conce
origin. -/
lemma distDiv_inv_pow_eq_dim {d : ℕ} [NeZero d] :
 ∀ d (fun x : Space d => ||x|| ^ (- d : ℤ) • basis.repr x)
 (IsDistBounded.zpow_smul_repr_self (- d : ℤ) (by omega)) =
 (d * (volume (α := Space d)).real (Metric.ball 0 1)) • diracDelta ℝ 0 := by
 _ = - ∫ x, ||x||-1 ^ d • basis.repr x, Space.grad η x|_ℝ := by
 simp only [zpow_neg, zpow_natCast, distDiv_ofFunction, inv_pow]
 _ = - ∫ x, ||x||-1 ^ (d - 1) * ||x||-1 • basis.repr x, Space.grad η x|_ℝ := by
 simp only [← pow_sub_one_mul (NeZero.ne d), inv_pow, inner_smul_left, conj
 map_inv₀, neg_inj]
 ring_nf
 _ = - ∫ x, ||x||-1 ^ (d - 1) * (_root_.deriv (fun a => η (a • ||x||-1 • x)) ||x||) :
 simp only [real_inner_comm,
 ← grad_inner_space_unit_vector _ (SchwartzMap.differentiable η)]
 _ = - ∫ r, ||r.2.1||-1 ^ (d - 1) * (_root_.deriv (fun a => η (a • r.1)) ||r.2.1||)
 ∂(volume (α := Space d).toSphere.prod
 (Measure.volumeIoiPow (Module.finiteRank ℝ (Space d) - 1))) := by
 rw [← MeasureTheory.MeasurePreserving.integral_comp (f := homeomorphUnitSp
 (MeasureTheory.Measure.measurePreserving_homeomorphUnitSphereProd

```

```

(volume (α := Space d)))
(Homeomorph.measurableEmbedding (homeomorphUnitSphereProd (Space d)
congr 1
simp only [inv_pow, homeomorphUnitSphereProd_apply_snd_coe, norm_
homeomorphUnitSphereProd_apply_fst_coe]
let f (x : Space d) : ℝ :=
 (||↑x|| ^ (d - 1))-1 * _root_.deriv (fun a => η (a • ||↑x||-1 • ↑x))
conv_rhs =>
 enter [2, x]
 change f x.1
rw [MeasureTheory.integral_subtype_comap (by simp), ← setIntegral
change ∫ x in Set.univ, f x = ∫ (x : Space d) in _, f x
exact setIntegral_congr_set (MeasureTheory.ae_eq_univ.mpr (by simp
_ = - ∫ n, (∫ r, ||r.1||-1 ^ (d - 1) *
- ∂((Measure.volumeIoiPow (Module.finrank ℝ (Space d) - 1))))
∂(volume (α := Space d).toSphere) := by
 rw [MeasureTheory.integral_prod]
 /- Integrable condition. -/
 convert integrable_isDistBounded_inner_grad_schwartzMap_spherical
 (IsDistBounded.inv_pow_smul_repr_self (d) (by omega)) η
 rename_i r
 simp only [Real.norm_eq_abs, inv_pow, Function.comp_apply,
 homeomorphUnitSphereProd_symm_apply_coe, map_smul]
 let x : Space d := r.2.1 • r.1.1
 have hr := r.2.2
 simp [-Subtype.coe_prop] at hr
 trans (r.2.1 ^ (d - 1))-1 * _root_.deriv (fun a => η (a • ||↑x||-1
 · simp [x, norm_smul]
 left
 congr
 funext a
 congr
 simp [smul_smul]
 rw [abs_of_nonneg (le_of_lt hr)]
 field_simp
 simp only [one_smul]
 rw [abs_of_nonneg (le_of_lt hr)]
 rw [← grad_inner_space_unit_vector, real_inner_comm, ← pow_sub_on
 simp only [norm_smul, Real.norm_eq_abs, abs_of_nonneg (le_of_lt h
 norm_eq_of_mem_sphere, mul_one, map_smul, inner_smul_left, map_
 mul_inv_rev, x]
 exact SchwartzMap.differentiable η
∂(volume (α := Space d).toSphere) := by
 congr
 funext n
 simp [Measure.volumeIoiPow]

```



```

 erw [integral_withDensity_eq_integral_smul]
 congr
 funext r
 have hr := r.2
 simp [-Subtype.coe_prop] at hr
 rw [abs_of_nonneg hr.le, NNReal.smul_def, Real.coe_toNNReal _ (by positivity),
 smul_eq_mul, ← mul_assoc, mul_inv_cancel₀ (pow_ne_zero (d - 1) hr.ne'),
 _ = - ∫ n, (-η 0) ∂(volume (α := Space d).toSphere) := by
 congr
 funext n
 let η' (n : ↑(Metric.sphere 0 1)) : ℝ → ℝ := compCLM (g := fun a => a •
 apply And.intro
 · fun_prop
 · intro n'
 match n' with
 | 0 =>
 use 1, 1
 simp [norm_smul]
 | 1 =>
 use 0, 1
 intro x
 simp [fderiv_smul_const]
 | n' + 1 + 1 =>
 use 0, 0
 intro x
 simp only [Real.norm_eq_abs, pow_zero, mul_one, norm_le_zero_iff]
 rw [iteratedFDeriv_succ_eq_comp_right]
 conv_lhs =>
 enter [2, 3, y]
 simp [fderiv_smul_const]
 rw [iteratedFDeriv_succ_const]
 rfl) (by use 1, 1; simp [norm_smul]) η
 rw [MeasureTheory.integral_subtype_comap (by simp),
 MeasureTheory.integral_Ioi_of_hasDerivAt_of_tendsto (f := fun a => η (a
 · simp
 · refine ContinuousAt.continuousWithinAt ?_
 fun_prop
 · intro x hx
 refine DifferentiableAt.hasDerivAt ?_
 have := η.differentiable
 fun_prop
 · exact (integrable ((derivCLM ℝ ℝ) (η' n))).integrableOn
 · exact Filter.Tendsto.mono_left (η' n).toZeroAtInfty.zero_at_infty' atTop
 _ = η 0 * (d * (volume (α := Space d)).real (Metric.ball 0 1)) := by
 simp only [integral_const, Measure.toSphere_real_apply_univ, finrank_eq_dim,
 mul_neg, neg_neg]

```

```

 ring
 simp only [_root_.smul_apply, diracDelta_apply, smul_eq_mul]
 /-!

B.6. The Laplacian of the fundamental solution

-/

/-- The distributional Laplacian of $\|x\|^{(2-d)}$ is $(2-d) * d * \text{volume}$
times the Dirac delta at the origin. For $d \geq 3$ this $\|x\|^{(2-d)}$ is the
fundamental solution of the Laplacian, and for $d = 1$ it is $\|x\|$. When d
vanishes, so the identity collapses to the trivial $\Delta^d 1 = 0$; the genuine
dimensional
fundamental solution is the logarithm, proved in distLaplacian_fundamental.
The statement also holds vacuously for $d = 0$, where the space is trivial.
/
lemma distLaplacian_fundamentalSolution_norm_zpow {d : ℕ} :
 Δ^d (distOfFunction (fun x : Space d => $\|x\|^{-(d : \mathbb{Z}) - 2}$))
 (IsDistBounded.pow _ (by omega))) =
 ((- ((d : ℝ) - 2)) * d *
 (volume (α := Space d)).real (Metric.ball 0 1)) • diracDelta ℝ 0 :=
by_cases h : d = 0
· subst h
 have hzero :
 distOfFunction (fun x : Space 0 => $\|x\|^2$)
 (IsDistBounded.pow _ (by omega)) = 0 := by
 ext η
 rw [distOfFunction_apply]
 refine integral_eq_zero_of_ae (ae_of_all _ fun x => ?_)
 rw [Subsingleton.elim x 0]
 simp only [norm_zero, smul_eq_mul, Pi.zero_apply, mul_eq_zero]
 right
 linarith
 simp [hzero]
· haveI : NeZero d := ⟨by omega⟩
 rw [distLaplacian]
 change $\nabla^d \square (\nabla^d (\text{distOfFunction}$
 (fun x : Space d => $\|x\|^{-(d : \mathbb{Z}) - 2}$))
 (IsDistBounded.pow (-(d : ℤ) - 2) (by omega)))) = _
 rw [distGrad_distOfFunction_norm_zpow (-(d : ℤ) - 2) (by omega)]
 simp only [neg_sub, Int.cast_sub, Int.cast_ofNat, Int.cast_natCast, sub]
 have hdist :
 distOfFunction
 (fun x : Space d =>
 ((2 - (d : ℝ)) * $\|x\|^{-(d : \mathbb{Z})}$) • basis.repr x)
 (by

```

```

 simp [smul_smul] using
 (IsDistBounded.const_fun_smul
 (F := EuclideanSpace ℝ (Fin d))
 (IsDistBounded.zpow_smul_repr_self (- (d : ℤ)) (by omega))
 (2 - (d : ℝ)))) =
 (2 - (d : ℝ)) • distOfFunction
 (fun x : Space d =>
 ||x|| ^ (- (d : ℤ)) • basis.repr x)
 (IsDistBounded.zpow_smul_repr_self (- (d : ℤ)) (by omega)) := by
 convert distOfFunction_smul_fun
 (fun x : Space d =>
 ||x|| ^ (- (d : ℤ)) • basis.repr x)
 (IsDistBounded.zpow_smul_repr_self (- (d : ℤ)) (by omega))
 (2 - (d : ℝ)) using 1
 ext x
 simp [smul_smul]
 rw [hdist]
 rw [map_smul]
 have hdiv :
 $\nabla^d \square$ (distOfFunction
 (fun x : Space d =>
 ||x|| ^ (- (d : ℤ)) • basis.repr x)
 (IsDistBounded.zpow_smul_repr_self (- (d : ℤ)) (by omega))) =
 (d * (volume (α := Space d)).real
 (Metric.ball 0 1)) • diracDelta ℝ 0 := by
 exact distDiv_inv_pow_eq_dim (d := d)
 rw [hdiv]
 rw [smul_smul]
 ring_nf

```

*-- In dimension two the fundamental solution of the Laplacian is the logarithm: the distributional Laplacian of `Real.log ||x||` is `2 * volume (Metric.ball 0 1)` times `delta` at the origin. -/*

```

lemma distLaplacian_fundamentalSolution_log_norm :
 Δ^d (distOfFunction (fun x : Space 2 => Real.log ||x||) IsDistBounded.log_norm) =
 (2 * (volume (α := Space 2)).real (Metric.ball 0 1)) • diracDelta ℝ 0 := by
 rw [distLaplacian]
 change $\nabla^d \square$ (∇^d (distOfFunction (fun x : Space 2 => Real.log ||x||)
 IsDistBounded.log_norm)) = _
 rw [distGrad_distOfFunction_log_norm (by norm_num)]
 simp only [Nat.cast_ofNat] using distDiv_inv_pow_eq_dim (d := 2)

```

**1.278 Physlib/SpaceAndTime/Space/Norm/IteratedLaplacian.lean**

```

/-
Copyright (c) 2026 Lazar Milikic. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lazar Milikic
-/
module

public import Physlib.SpaceAndTime.Space.Norm.Basic
/-!

Iterated Laplacians of norm distributions

i. Overview

This file proves the distributional identity corresponding to the classical
dimensional
formula that, in dimension $2 * m + 1$, applying the Laplacian $m + 1$ time
gives a nonzero constant multiple of the Dirac delta at the origin.

ii. Key results

- `iterated_distLaplacian_norm_zpow_odd_eq_smul_diracDelta` : The $(m + 1)$
fold
 Laplacian of the norm in dimension $2 * m + 1$ is a nonzero multiple of t

iii. Table of contents

- A. The odd-dimensional iterated Laplacian of the norm

iv. References

-/

@[expose] public section

open SchwartzMap NNReal Physlib
noncomputable section

namespace Space

open MeasureTheory
open Distribution

/-!

```

1.278.

PHYSLIB/SPACEANDTIME/SPACE/NORM/ITERATEDLAPLACIAN.LEAN457

## A. The odd-dimensional iterated Laplacian of the norm

-/

/-- The scalar factor in the odd-dimensional iterated Laplacian of the norm. -  
/

```
noncomputable def oddNormIteratedLaplacianCoeff (m : ℕ) : ℝ :=
 (∏ k ∈ Finset.range m,
 (((1 : ℤ) - 2 * (k : ℤ) : ℤ) : ℝ) *
 (((1 : ℤ) - 2 * (k : ℤ) - 2 + (2 * m + 1 : ℤ) : ℤ) : ℝ))) *
 (((1 : ℤ) - 2 * (m : ℤ) : ℝ) * (2 * m + 1 : ℝ) *
 (volume (α := Space (2 * m + 1))).real (Metric.ball 0 1))
```

```
private lemma oddNormIteratedLaplacianCoeff_factor_ne_zero {m k : ℕ} (hk : k < m) :
 (((1 : ℤ) - 2 * (k : ℤ) : ℤ) : ℝ) *
 (((1 : ℤ) - 2 * (k : ℤ) - 2 + (2 * m + 1 : ℤ) : ℤ) : ℝ)) ≠ 0 := by
 apply mul_ne_zero
 · have h : (1 : ℤ) - 2 * (k : ℤ) ≠ 0 := by omega
 exact_mod_cast h
 · have h : (1 : ℤ) - 2 * (k : ℤ) - 2 + (2 * m + 1 : ℤ) ≠ 0 := by omega
 exact_mod_cast h
```

/-- The scalar factor in the odd-dimensional iterated Laplacian of the norm is nonzero  
/

```
lemma oddNormIteratedLaplacianCoeff_ne_zero (m : ℕ) :
 oddNormIteratedLaplacianCoeff m ≠ 0 := by
 unfold oddNormIteratedLaplacianCoeff
 apply mul_ne_zero
 · rw [Finset.prod_ne_zero_iff]
 intro k hk
 exact oddNormIteratedLaplacianCoeff_factor_ne_zero (by simp using hk)
 · repeat' apply mul_ne_zero
 · have h : (1 : ℤ) - 2 * (m : ℤ) ≠ 0 := by omega
 exact_mod_cast h
 · exact_mod_cast (by omega : (2 * m + 1 : ℕ) ≠ 0)
 · exact ne_of_gt <| ENNReal.toReal_pos
 (Metric.measure_ball_pos volume 0 one_pos).ne'
 measure_ball_lt_top.ne
```

```
private lemma distLaplacian_norm_zpow_odd_boundary (m : ℕ) :
 Δd (distOfFunction (fun x : Space (2 * m + 1) =>
 ||x|| ^ ((1 : ℤ) - 2 * (m : ℤ)))
 (IsDistBounded.pow _ (by omega))) =
 (((1 : ℤ) - 2 * (m : ℤ) : ℝ) * (2 * m + 1 : ℝ) *
 (volume (α := Space (2 * m + 1))).real (Metric.ball 0 1)) •
```

```

 diracDelta ℝ 0 := by
rcases m with _ | m
· rw [distLaplacian]
 change $\nabla^d \square (\nabla^d (\text{distOfFunction } (\text{fun } x : \text{Space } 1 \Rightarrow \|x\| ^ (1 : \mathbb{Z}))$
 (IsDistBounded.pow 1 (by omega)))) = _
 rw [distGrad_distOfFunction_norm_zpow 1 (by omega)]
 simp only [Int.cast_one, one_mul]
 convert! distDiv_inv_pow_eq_dim (d := 1) using 1
· ext x
 ring_nf
· convert! distLaplacian_fundamentalSolution_norm_zpow
 (d := 2 * m.succ + 1) using 4
· simp; ring_nf
· simp; ring
· simp

private lemma iterated_distLaplacian_norm_zpow_odd_until_boundary
 (m k : ℕ) (hk : k ≤ m) :
 ((distLaplacian (d := 2 * m + 1))^[k])
 (distOfFunction (fun x : Space (2 * m + 1) => \|x\| ^ (1 : ℤ))
 (IsDistBounded.pow 1 (by omega))) =
 (∏ j ∈ Finset.range k,
 (((1 : ℤ) - 2 * (j : ℤ) : ℤ) : ℝ) *
 (((1 : ℤ) - 2 * (j : ℤ) - 2 + (2 * m + 1 : ℤ) : ℤ) : ℝ))) •
 distOfFunction (fun x : Space (2 * m + 1) => \|x\| ^ ((1 : ℤ) - 2 * (
 (IsDistBounded.pow _ (by omega))) := by
induction k with
| zero =>
 simp
| succ k ih =>
 have hk_le : k ≤ m := Nat.le_of_succ_le hk
 rw [Function.iterate_succ_apply']
 rw [ih hk_le]
 rw [map_smul]
 rw [distLaplacian_distOfFunction_norm_zpow
 (d := 2 * m + 1) ((1 : ℤ) - 2 * (k : ℤ)) (by omega)]
 rw [smul_smul]
 have hdist :
 distOfFunction
 (fun x : Space (2 * m + 1) => \|x\| ^ ((1 : ℤ) - 2 * (k : ℤ) -
 (IsDistBounded.pow _ (by omega))) =
 distOfFunction
 (fun x : Space (2 * m + 1) => \|x\| ^ ((1 : ℤ) - 2 * ((k + 1 :
 (IsDistBounded.pow _ (by omega))) := by
 have hexp :
 ((1 : ℤ) - 2 * (k : ℤ) - 2) =

```

```

 ((1 : ℤ) - 2 * ((k + 1 : ℕ) : ℤ)) := by
 norm_num
 ring
 ext η
 simp [distOfFunction_apply, hexp]
 rw [hdist]
 rw [Finset.prod_range_succ]
 congr 1

/-- In dimension `2 * m + 1`, the `(m + 1)`-fold distributional Laplacian of the
distribution induced by the norm is a nonzero multiple of the Dirac delta at the ori
/
lemma iterated_distLaplacian_norm_zpow_odd_eq_smul_diracDelta (m : ℕ) :
 ((distLaplacian (d := 2 * m + 1))^[m + 1])
 (distOfFunction (fun x : Space (2 * m + 1) => ||x|| ^ (1 : ℤ))
 (IsDistBounded.pow 1 (by omega))) =
 oddNormIteratedLaplacianCoeff m • diracDelta ℝ 0 := by
 rw [Function.iterate_succ_apply']
 rw [iterated_distLaplacian_norm_zpow_odd_until_boundary m m le_refl]
 rw [map_smul]
 rw [distLaplacian_norm_zpow_odd_boundary m]
 rw [smul_smul]
 unfold oddNormIteratedLaplacianCoeff
 rfl

end Space

```

## 1.279 Physlib/SpaceAndTime/Space/Norm/Regularized.lean

```

/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
module

public import Physlib.SpaceAndTime.Space.Derivatives.Basic
public import Physlib.SpaceAndTime.Space.Integrals.NormPow
/-!

Regularized powers of the norm on space

i. Overview

```

This file contains basic API for regularized powers of the norm on `Space d`  
 $x \mapsto (\|x\|^2 + \varepsilon^2)^{(s/2)}$ .

## ii. Key results

- `normRegularizedPow` : The regularized norm power  $x \mapsto (\|x\|^2 + \varepsilon^2)^{(s/2)}$
- `normRegularizedPow_pos` : Positivity for nonzero regularization parameter
- `normRegularizedPow_hasTemperateGrowth` : Temperate growth of regularized norm power
- `normRegularizedPow_measurable` : Measurability of regularized norm power

## iii. Table of contents

- A. Regularized powers of the norm

## iv. References

-/

@[expose] public section

noncomputable section

namespace Space

open MeasureTheory Function

/-!

## A. Regularized powers of the norm

-/

/-- Power of regularized norm,  $(\|x\|^2 + \varepsilon^2)^{(s/2)}$ . -/  
 def normRegularizedPow (d : ℕ) (ε s : ℝ) : Space d → ℝ :=  
 fun x ↦ ( $\|x\|^2 + \varepsilon^2$ ) ^ (s / 2)

lemma normRegularizedPow\_eq (d : ℕ) (ε s : ℝ) :  
 normRegularizedPow d ε s = fun x ↦ ( $\|x\|^2 + \varepsilon^2$ ) ^ (s / 2) := rfl

/-- For a nonzero regularization parameter,  $\|x\|^2 + \varepsilon^2$  is positive. -/  
 /

lemma norm\_sq\_add\_unit\_sq\_pos {d : ℕ} (ε : ℝ<sup>+</sup>) (x : Space d) : 0 <  $\|x\|^2 + \varepsilon^2$   
 Left.add\_pos\_of\_nonneg\_of\_pos (sq\_nonneg  $\|x\|$ ) (sq\_pos\_iff.mpr <| Units.

/-- The regularized norm power is positive for nonzero regularization parameter -/  
 /



```

lemma normRegularizedPow_pos (d : ℕ) (ε : ℝ+) (s : ℝ) (x : Space d) :
 0 < normRegularizedPow d ε s x :=
 Real.rpow_pos_of_pos (norm_sq_add_unit_sq_pos ε x) (s / 2)

/-- The regularized norm power has temperate growth. -/
lemma normRegularizedPow_hasTemperateGrowth (d : ℕ) (ε : ℝ+) (s : ℝ) :
 HasTemperateGrowth (normRegularizedPow d ε s) := by
 let f1 := fun (x : ℝ) ↦ (ε ^ 2) ^ (s / 2) * x
 let f2 := fun (x : Space d) ↦ (1 + ||x|| ^ 2) ^ (s / 2)
 let f3 := fun (x : Space d) ↦ ε.1-1 • x
 have h123 : normRegularizedPow d ε s = f1 ∘ f2 ∘ f3 := by
 ext
 simp only [normRegularizedPow, f1, f2, f3, comp_apply, norm_smul, norm_inv, Real
 rw [← Real.mul_rpow (sq_nonneg ↑ε) (add_nonneg (zero_le_one' _) (sq_nonneg _))]
 simp [mul_add, mul_pow, add_comm]
 rw [h123]
 fun_prop

@[fun_prop]
lemma normRegularizedPow_measurable (d : ℕ) (ε s : ℝ) :
 Measurable (normRegularizedPow d ε s) := by
 rw [normRegularizedPow_eq]
 fun_prop

end Space

```

## 1.280 Physlib/SpaceAndTime/Space/Origin.lean

```

/-
Copyright (c) 2026 Shaopeng Zhu. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Shaopeng Zhu, Joseph Tooby-Smith
-/
module

public import Physlib.SpaceAndTime.Space.Basic
public import Mathlib.Analysis.Normed.Affine.Isometry

/-!
The origin of `Space` and the Euclidean chart

The choice of origin for `Space d` is its vector-space zero `(0 : Space d)`. This fi
that `Zero` instance and the standard chart, isolated so they can be shared by the f
structure (`Space/Module.lean`) and the Euclidean action (`Space/EuclideanGroup/Acti

```

without those depending on each other.

```
* `(0 : Space d)` – the coordinate origin, the point all of whose coordinates are 0
* `Space.chartEuclidean` – the standard affine isometry $\text{Space } d \approx^{\text{ai}} \mathbb{R} \text{ EuclideanSpace } \mathbb{R} (\text{Fin } d)$
 `p ↦ p -v 0`, identifying a point with its coordinate vector relative to the origin
-/
```

```
@[expose] public section
```

```
namespace Space
```

```
instance {d} : Zero (Space d) where
 zero := (fun _ => 0)
```

```
@[simp]
lemma zero_val {d : ℕ} : (0 : Space d).val = fun _ => 0 := rfl
```

```
@[simp]
lemma zero_apply {d : ℕ} (i : Fin d) :
 (0 : Space d) i = 0 := by
 simp [zero_val]
```

```
/-- A Euclidean vector, based at the chosen origin, viewed as a point of `Space d`
/
noncomputable def vectorToSpace {d : ℕ} (v : EuclideanSpace ℝ (Fin d)) : Space d :=
 v +v (0 : Space d)
```

```
@[simp]
lemma vectorToSpace_apply {d : ℕ} (v : EuclideanSpace ℝ (Fin d)) (i : Fin d) :
 vectorToSpace v i = v i := by
 simp [vectorToSpace]
```

```
@[simp]
lemma vectorToSpace_vsub_zero {d : ℕ} (v : EuclideanSpace ℝ (Fin d)) :
 vectorToSpace v -v (0 : Space d) = v := by
 ext i
 simp [vectorToSpace]
```

```
/-- The standard chart $\text{Space } d \approx^{\text{ai}} \mathbb{R} \text{ EuclideanSpace } \mathbb{R} (\text{Fin } d)$, `p ↦ p -v 0`,
 identifying a point with its coordinate vector relative to the origin (the vector-space
 zero `(0 : Space d)`). -/
noncomputable def chartEuclidean (d : ℕ) :
 Space d ≈ai ℝ EuclideanSpace ℝ (Fin d) :=
 (AffineIsometryEquiv.vaddConst ℝ (0 : Space d)).symm
```

```
@[simp] lemma chartEuclidean_apply (d : ℕ) (p : Space d) :
 chartEuclidean d p = p -v (0 : Space d) := rfl

end Space
```

### 1.281 Physlib/SpaceAndTime/Space/Slice.lean

```
rw [fderiv_fun_comp, DifferentiableAt.fderiv_prodMk (by fun_prop)]
simp only [Nat.succ_eq_add_one, fderiv_slice_symm, fderiv_fun_id, fderiv_fun_const,
 ContinuousLinearMap.coe_comp, ContinuousLinearEquiv.coe_coe, Function.comp_apply,
 root.zero_apply]
rw [fderiv_fun_comp, DifferentiableAt.fderiv_prodMk (by fun_prop)]
simp only [Nat.succ_eq_add_one, fderiv_slice_symm, fderiv_fun_const, Pi.zero_apply,
 ContinuousLinearMap.coe_comp, ContinuousLinearEquiv.coe_coe, Function.comp_apply,
 ContinuousLinearMap.prod_apply, _root_.zero_apply, ContinuousLinearMap.coe_id',
 rw [fderiv_fun_comp _ _ (by fun_prop)]
simp only [Nat.succ_eq_add_one, ContinuousLinearMap.coe_comp, Function.comp_apply,
 rw [fderiv_fun_comp _ _ (by fun_prop)]
simp only [Nat.succ_eq_add_one, ContinuousLinearMap.coe_comp, Function.comp_apply,
```

### 1.282 Physlib/SpaceAndTime/Space/Translations.lean

```
simp only [fderiv_fun_id, fderiv_fun_const, Pi.zero_apply, sub_zero]
lemma distTranslate_ofFunction {d : ℕ} (a : EuclideanSpace ℝ (Fin d))
 (f : Space d → X) (hf : IsDistBounded f) :
 trans ∫ (x : Space d), η ((x - basis.repr.symm a) + basis.repr.symm a) •
 let f' := fun x : Space d => η (x + basis.repr.symm a) • f (x)
 change _ = ∫ (x : Space d), f' (x - basis.repr.symm a)
```

### 1.283 Physlib/SpaceAndTime/SpaceTime/Basic.lean

```
public import Physlib.Relativity.Tensors.RealTensor.Vector.Tensorial
public import Physlib.Meta.Informal.Basic
ext1 μ
```

### 1.284 Physlib/SpaceAndTime/SpaceTime/Boosts.lean

```
fin_cases i <=>
 simp [Lorentz.Vector.smul_eq_sum, Fin.sum_univ_three, boost_inl_0_inr_other,
 boost_inr_other_inr, boost_inr_inr_other, boost_inr_other_inl_0] <=>
```

```

Fin.isValue, Finset.sum_singleton, Fin.sum_univ_succ, toTimeAndSpace_sy
toTimeAndSpace_symm_apply_inr]
simp [γ_neg]
simp only [toTimeAndSpace_symm_apply_inr]
simp [γ_neg]
rw [Finset.sum_eq_single ⟨n, by omega⟩] <=>
 simp +contextual [boost_inr_inr_other, Fin.ext_iff]

```

### 1.285 Physlib/SpaceAndTime/SpaceTime/Derivatives.lean

```

public import Physlib.Relativity.Tensors.RealTensor.CoVector.Tensorial
- `manifoldDeriv` : The derivative of a function from `SpaceTime d` to a manifold
 the `μ` coordinate.
- `contDiff_deriv` : If `f` is C^{n+1} then $\partial_\mu f$ is C^n .
- `differentiable_deriv` : If `f` is C^2 then $\partial_\mu f$ is differentiable.
- `deriv_commute` : Derivatives on `SpaceTime d` commute (Clairaut's theorem)
 - A.4. Smoothness and differentiability of the derivative
 - A.5. Derivatives commute
 - A.6. The derivative of a function composed with a Lorentz transformation
 - A.7. Spacetime derivatives in terms of time and space derivatives
lemma deriv_eq {M : Type} [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
 {d : ℕ} (μ : Fin 1 ⊕ Fin d) (f : SpaceTime d → M) (y : SpaceTime d) :
 ∂_μ f y =
 fderiv ℝ f y (Lorentz.Vector.basis μ) := by
 rfl

/-- The derivative of a function from `SpaceTime d` to a manifold along the
coordinate, as a tangent vector at the value of the function. -
/
noncomputable def manifoldDeriv {E H N : Type} [NormedAddCommGroup E] [NormedSpace ℝ E]
 [TopologicalSpace H] (I : ModelWithCorners ℝ E H) [TopologicalSpace N]
 [ChartedSpace H N] {d : ℕ} (μ : Fin 1 ⊕ Fin d) (f : SpaceTime d → N) :
 (y : SpaceTime d) → TangentSpace I (f y) :=
 fun y => mfderiv [](ℝ, SpaceTime d) I f y
 ((Lorentz.Vector.basis μ : SpaceTime d) : TangentSpace [](ℝ, SpaceTime d))

lemma manifoldDeriv_eq {E H N : Type} [NormedAddCommGroup E] [NormedSpace ℝ E]
 [TopologicalSpace H] (I : ModelWithCorners ℝ E H) [TopologicalSpace N]
 [ChartedSpace H N] {d : ℕ} (μ : Fin 1 ⊕ Fin d) (f : SpaceTime d → N)
 (y : SpaceTime d) :
 manifoldDeriv I μ f y =
 mfderiv [](ℝ, SpaceTime d) I f y
 ((Lorentz.Vector.basis μ : SpaceTime d) : TangentSpace [](ℝ, SpaceTime d))

```

```

/-- The spacetime derivative is the manifold derivative for functions into normed space
/
lemma deriv_eq_mfderiv {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
 {d : ℕ} (μ : Fin 1 ⊕ Fin d) (f : SpaceTime d → M) (y : SpaceTime d) :
 deriv μ f y =
 mfderiv (ℝ, SpaceTime d) (ℝ, M) f y
 ((Lorentz.Vector.basis μ : SpaceTime d) : TangentSpace (ℝ, SpaceTime d) y)
rw [deriv_eq, ← mfderiv_eq_fderiv]
rfl

lemma deriv_eq_manifoldDeriv {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
 {d : ℕ} (μ : Fin 1 ⊕ Fin d) (f : SpaceTime d → M) (y : SpaceTime d) :
 deriv μ f y = manifoldDeriv (ℝ, M) μ f y := by
 rw [deriv_eq_mfderiv, manifoldDeriv_eq]

@[simp]
lemma manifoldDeriv_const {E H N : Type} [NormedAddCommGroup E] [NormedSpace ℝ E]
 [TopologicalSpace H] (I : ModelWithCorners ℝ E H) [TopologicalSpace N]
 [CharteredSpace H N] {d : ℕ} (μ : Fin 1 ⊕ Fin d) (n : N) (y : SpaceTime d) :
 manifoldDeriv I μ (fun _ : SpaceTime d => n) y = 0 := by
 simp [manifoldDeriv]

 simp only [ContinuousLinearMap.fderiv, ContinuousLinearMap.coe_comp, Function.comp]
lemma deriv_apply_eq {d : ℕ} (μ v : Fin 1 ⊕ Fin d) (f : SpaceTime d → Lorentz.Vector ℝ)
 (hf : Differentiable ℝ f)
 (y : SpaceTime d) :
 ∂_μ f y v = fderiv ℝ (fun x => f x v) y (Lorentz.Vector.basis μ) := by
 rw [deriv_eq]
 exact fderiv_vector f hf y _ v

A.4. Smoothness and differentiability of the derivative

-/

/-- If `f` is `C^{n+1}` then `∂_μ f` is `C^n`. -/
@[fun_prop]
lemma contDiff_deriv {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ}
 {n : WithTop ℕ} (μ : Fin 1 ⊕ Fin d) (f : SpaceTime d → M) (hf : ContDiff ℝ (n + 1) f) :
 ContDiff ℝ n (∂_μ f) := by
 -- `∂_μ f = fun x => fderiv ℝ f x (Lorentz.Vector.basis μ)`; use
 -- `ContDiff.clm_apply` with `ContDiff.fderiv_right`.
 show ContDiff ℝ n (fun x => fderiv ℝ f x (Lorentz.Vector.basis μ))
 exact (ContDiff.fderiv_right (m := n) hf (by rfl)).clm_apply contDiff_const

/-- If `f` is `C^2` then `∂_μ f` is differentiable. -/
@[fun_prop]

```

```

lemma differentiable_deriv {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
 (μ : Fin 1 ⊕ Fin d) (f : SpaceTime d → M) (hf : ContDiff ℝ 2 f) :
 Differentiable ℝ (∂_ μ f) :=
 (contDiff_deriv μ f (n := 1) (by norm_cast)).differentiable one_ne_zero

/-!

A.5. Derivatives commute

-/

/-- Derivatives on spacetime commute with one another (Clairaut's theorem).
/
lemma deriv_commute {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d
 (μ v : Fin 1 ⊕ Fin d) (f : SpaceTime d → M) (hf : ContDiff ℝ 2 f) :
 ∂_ μ (∂_ v f) = ∂_ v (∂_ μ f) := by
 ext x
 show fderiv ℝ (fun y => fderiv ℝ f y (Lorentz.Vector.basis v)) x (Lorentz
 fderiv ℝ (fun y => fderiv ℝ f y (Lorentz.Vector.basis μ)) x (Lorentz.Ve
 rw [fderiv_clm_apply, fderiv_clm_apply]
 simp only [fderiv_fun_const, Pi.ofNat_apply, ContinuousLinearMap.comp_zero,
 ContinuousLinearMap.flip_apply]
 rw [IsSymmSndFderivAt.eq]
 · apply ContDiffAt.isSymmSndFderivAt
 exact hf.contDiffAt
 simp only [minSmoothness_of_isRCLikeNormedField, le_refl]
 · have h1 := hf.differentiable (by norm_cast); fun_prop
 · fun_prop
 · have h1 := hf.differentiable (by norm_cast); fun_prop
 · fun_prop

/-!

A.6. The derivative of a function composed with a Lorentz transformation
ContinuousLinearMap.coe_comp, Function.comp_apply]
A.7. Spacetime derivatives in terms of time and space derivatives
conv_rhs => rw [fderiv_fun_comp _ (by fun_prop) (by fun_prop)]
simp only [Prod.mk.eta, ContinuousLinearEquiv.symm_apply_apply, Continuous
 rw [fderiv_fun_comp]
simp only [Prod.mk.eta, toTimeAndSpace_symm_fderiv, ContinuousLinearMap.c
simp only [fderiv_fun_const, Pi.zero_apply, fderiv_fun_id, ContinuousLine
 root.zero_apply, ContinuousLinearMap.coe_id', id_eq]
conv_rhs => rw [fderiv_fun_comp _ (by fun_prop) (by fun_prop)]
ContinuousLinearMap.coe_comp, Function.comp_apply]
rw [fderiv_fun_comp]
simp only [Fin.isValue, Prod.mk.eta, toTimeAndSpace_symm_fderiv, Continuo

```

1.286. *PHYSLIB/SPACEANDTIME/SPACETIME/LORENTZACTION.LEAN* 467

```

simp only [Fin.isValue, fderiv_fun_id, fderiv_fun_const, Pi.zero_apply,
 root.zero_apply]
 simp only [_root_.add_apply]
exact congrFun (deriv_commute v μ ↑κ (smooth κ 2)) x
rw [_root_.sum_apply]
simp only [FunLike.coe_smul, Pi.smul_apply]
rw [_root_.sum_apply]
simp only [inv_mul_cancel, map_one, one_apply_eq_self]
 rw [fderiv_fun_comp _ (by fun_prop) (by fun_prop)]
 exact absurd (CoVector.indexEquiv.symm_apply_eq.mp hx) hb
 refine continuous_finsetSum Finset.univ (fun μ _ => ?_)
 simp only [FunLike.coe_sum, FunLike.coe_smul, Finset.sum_apply,
 simp only [Nat.succ_eq_add_one, Nat.reduceAdd, ContinuousLinearMap.coe_comp,
 exact absurd (CoVector.indexEquiv.symm_apply_eq.mp hx) hb

```

### 1.286 Physlib/SpaceAndTime/SpaceTime/LorentzAction.lean

```

set_option synthInstance.maxHeartbeats 40000
 simp only [_root_.add_apply, smul_add, lorentzGroup_smul_dist_apply]
 simp [lorentzGroup_smul_dist_apply, _root_.add_apply, smul_add]

```

### 1.287 Physlib/SpaceAndTime/SpaceTime/TimeSlice.lean

```

 simp only [ContinuousLinearMap.coe_comp, Function.comp_apply]
 simp only [ContinuousLinearMap.coe_comp, Function.comp_apply]
simp only [Fin.isValue, one_div, FunLike.coe_smul, Pi.smul_apply]
simp only [toTimeAndSpace_fderiv, Fin.isValue, ContinuousLinearMap.coe_comp,
simp only [toTimeAndSpace_fderiv, ContinuousLinearMap.coe_comp, ContinuousLinearEq

```

### 1.288 Physlib/SpaceAndTime/Time/Derivatives.lean

```

Authors: Nikolai Kashcheev, Joseph Tooby-Smith
public import Mathlib.Analysis.Calculus.Deriv.Inv
public import Mathlib.Analysis.InnerProductSpace.Calculus
- `manifoldDeriv` : The derivative of a function from `Time` to a manifold.
 - A.1. Derivatives of functions into vector spaces
 - A.2. Derivatives of functions into manifolds

/-!

A.1. Derivatives of functions into vector spaces

```

-/

### A.2. Derivatives of functions into manifolds

-/

```
open Manifold in
/-- The time derivative of a function from `Time` to a manifold, as a tangent vector, is the value of the function. -/
noncomputable def manifoldDeriv {E H N : Type} [NormedAddCommGroup E] [NormedSpace ℝ E] [TopologicalSpace H] (I : ModelWithCorners ℝ E H) [TopologicalSpace N] [ChartedSpace H N] (f : Time → N) : (t : Time) → TangentSpace I (f t) : TangentSpace I (f t) :=
 fun t => mfderiv (λ (t : Time) : TangentSpace (ℝ, Time) t => f t) t
```

```
open Manifold in
lemma manifoldDeriv_eq {E H N : Type} [NormedAddCommGroup E] [NormedSpace ℝ E] [TopologicalSpace H] (I : ModelWithCorners ℝ E H) [TopologicalSpace N] [ChartedSpace H N] (f : Time → N) (t : Time) :
 manifoldDeriv I f t = mfderiv (λ (t : Time) : TangentSpace (ℝ, Time) t => f t) t :=
```

```
open Manifold in
/-- The time derivative is the manifold derivative for functions into normed spaces. -/
lemma deriv_eq_mfderiv [NormedAddCommGroup M] [NormedSpace ℝ M] (f : Time → M) (t : Time) :
 deriv f t = mfderiv (λ (t : Time) : TangentSpace (ℝ, Time) t => f t) t := by
 rw [deriv_eq, ← mfderiv_eq_fderiv]
 rfl
```

```
open Manifold in
lemma deriv_eq_manifoldDeriv [NormedAddCommGroup M] [NormedSpace ℝ M] (f : Time → M) (t : Time) :
 deriv f t = manifoldDeriv (λ (t : Time) : TangentSpace (ℝ, Time) t => f t) t := by
 rw [deriv_eq_mfderiv, manifoldDeriv_eq]
```

```
open Manifold in
@[simp]
lemma manifoldDeriv_const {E H N : Type} [NormedAddCommGroup E] [NormedSpace ℝ E] [TopologicalSpace H] (I : ModelWithCorners ℝ E H) [TopologicalSpace N] [ChartedSpace H N] (n : N) :
 manifoldDeriv I (fun _ : Time => n) t = 0 := by
 simp [manifoldDeriv]
```



```

/-!

 fderiv_fun_comp t (differentiableAt_inv hg) hg]
field_simp
ring
apply ContDiff.comp <=> fun_prop
· simp only [ContinuousLinearMap.fderiv, ContinuousLinearMap.coe_comp, Function.co

```

### 1.289 Physlib/SpaceAndTime/Time/TimeMan.lean

```

exact contMDiffOn_chart (x := (⟨0⟩ : TimeMan))
exact contMDiffOn_chart_symm (x := (⟨0⟩ : TimeMan))

```

### 1.290 Physlib/SpaceAndTime/Time/TimeTransMan.lean

```

exact contMDiffOn_chart (x := (⟨0⟩ : TimeTransMan))
exact contMDiffOn_chart_symm (x := (⟨0⟩ : TimeTransMan))

```

### 1.291 Physlib/SpaceAndTime/Time/TimeUnit.lean

```

simp [toReal]
· exact zero_le
rfl
simp only [one_div, NNReal.coe_inv]
rw [toReal, inv_div]
(x / y : ℝ) * (y / z : ℝ) = x / z := by
simp [div_eq_val, toReal]
rw [toReal]
lemma minutes_div_seconds : minutes / seconds = (60 : ℝ≥0) := NNReal.eq <| by
simp [minutes]; rw [toReal]
simp [hours]; rw [toReal]; norm_num
simp [days]; rw [toReal]; norm_num
simp [weeks]; rw [toReal]; norm_num
simp [days, minutes]; rw [toReal]; norm_num
simp [weeks, minutes]; rw [toReal]; norm_num
lemma days_div_hours : days / hours = (24 : ℝ≥0) := NNReal.eq <| by
simp [hours, days]; rw [toReal]; norm_num
simp [weeks, hours]; rw [toReal]; norm_num

```

**1.292 Physlib/SpaceAndTime/TimeAndSpace/Basic.lean**

Authors: Zhi Kai Pong, Joseph Tooby-Smith, Rob Sneiderman

In this module we define ``TimeAndSpace d`` as the product of ``Time`` and ``Space d``. We prove basic lemmas about derivatives of functions and distributions on both. The derivative and distribution results are in the namespace ``Space`` by convention.

- ``TimeAndSpace d`` : Euclidean spacetime as the product of time and space.
- ``TimeAndSpace.time`` : The projection from ``TimeAndSpace d`` to ``Time``.
- ``TimeAndSpace.space`` : The projection from ``TimeAndSpace d`` to ``Space d``.
- A. The coordinate product and derivatives involving time and space
  - A.1. The ``TimeAndSpace`` coordinate product
  - A.2. Space and time derivatives in terms of curried functions
  - A.3. Commuting time and space derivatives
  - A.4. Differentiability conditions
  - A.5. Time derivative commute with curl
  - A.6. Constant of time derivative and space derivatives zero
  - A.7. Equal up to a constant of time and space derivatives equal

`/-!`

`## A. The coordinate product and derivatives involving time and space`

`-/`

`### A.1. The `TimeAndSpace` coordinate product`

`/-- Euclidean spacetime as the product of time and d`-dimensional space. -`

`/`

`abbrev TimeAndSpace (d : ℕ := 3) := Time × Space d`

`namespace TimeAndSpace`

`variable {d : ℕ}`

`/-- The time-coordinate projection from `TimeAndSpace d`. -`

`/`

`noncomputable def time {d : ℕ} : TimeAndSpace d → L[ℝ] Time :=  
ContinuousLinearMap.fst ℝ Time (Space d)`

`/-- The spatial-coordinate projection from `TimeAndSpace d`. -`

`/`

`noncomputable def space {d : ℕ} : TimeAndSpace d → L[ℝ] Space d :=  
ContinuousLinearMap.snd ℝ Time (Space d)`

`@[simp]`

`lemma time_apply (tx : TimeAndSpace d) :  
time tx = tx.1 := rfl`

`@[simp]`

```

lemma space_apply (tx : TimeAndSpace d) :
 space tx = tx.2 := rfl

/-- The time projection is nonexpanding for the product metric. -
/
lemma dist_time_le (tx ty : TimeAndSpace d) :
 dist (time tx) (time ty) ≤ dist tx ty := by
 change dist tx.1 ty.1 ≤ dist tx ty
 rw [Prod.dist_eq]
 exact le_max_left (dist tx.1 ty.1) (dist tx.2 ty.2)

/-- The spatial projection is nonexpanding for the product metric. -
/
lemma dist_space_le (tx ty : TimeAndSpace d) :
 dist (space tx) (space ty) ≤ dist tx ty := by
 change dist tx.2 ty.2 ≤ dist tx ty
 rw [Prod.dist_eq]
 exact le_max_right (dist tx.1 ty.1) (dist tx.2 ty.2)

end TimeAndSpace

namespace Space

A.2. Space and time derivatives in terms of curried functions
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply]
simp only [fderiv_fun_const, Pi.zero_apply, fderiv_fun_id, ContinuousLinearMap.pro
_root.zero_apply, ContinuousLinearMap.coe_id', id_eq]
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply]
simp only [fderiv_fun_id, fderiv_fun_const, Pi.zero_apply, ContinuousLinearMap.pro
ContinuousLinearMap.coe_id', id_eq, _root.zero_apply]
A.3. Commuting time and space derivatives
exact fderiv_space_eq_fderiv_curry f t' x dx (hf.differentiable (by simp))
· exact fderiv_time_eq_fderiv_curry (fun t x => fderiv ℝ 1f (t, x) (0, dx)) t dt x
exact fderiv_time_eq_fderiv_curry f t dt x' (hf.differentiable (by simp))
· exact fderiv_space_eq_fderiv_curry (fun t x => fderiv ℝ 1f (t, x) (dt, 0)) t x d
· simp only [fderiv_fun_const, Pi.zero_apply, ContinuousLinearMap.comp_zero, zero_
ContinuousLinearMap.flip_apply]
exact (hf.contDiffAt.isSymmSndFderivAt (by simp)).eq (0, dx) (dt, 0)
all_goals fun_prop
A.4. Differentiability conditions
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply, ContinuousLinear
simp only [ContinuousLinearMap.coe_comp, Function.comp_apply, ContinuousLinear
A.5. Time derivative commute with curl
rw [Time.deriv_eq, fderiv_fun_sub]
congr <=>
 (funext x'

```

```

 rw [Time.deriv_euclid]
 have h1 := hf.differentiable (by simp)
 fun_prop)
 all_goals
 apply Differentiable.differentiableAt
A.6. Constant of time derivative and space derivatives zero
 simp only [_root_.zero_apply]
 rw [← Time.deriv_eq, h t x, smul_zero]
 apply ContinuousLinearMap.coe_injective
 apply (Space.basis (d := d)).toBasis.ext
 intro i
 simp only [ContinuousLinearMap.toLinearMap_zero, OrthonormalBasis.coe_toBasis
 ContinuousLinearMap.coe_coe, LinearMap.zero_apply]
 rw [← h t x i, Space.deriv_eq_fderiv_basis]
 have h1 : ∀ t x, g x = k t := fun t x => by rw [← hg t x, hk t x]
 rw [hk, ← h1 t 0]
A.7. Equal up to a constant of time and space derivatives equal
 rw [Time.deriv_eq, fderiv_fun_sub]
 all_goals fun_prop
 rw [Space.deriv_eq_fderiv_basis, fderiv_fun_sub]
 all_goals fun_prop
 simp only [_root_.add_apply]
 simp only [_root_.add_apply]
 have h1 := smooth k 2
 · simp only [fderiv_fun_const, Pi.zero_apply, ContinuousLinearMap.comp_zero,
 ContinuousLinearMap.flip_apply]
 exact (h1.contDiffAt.isSymmSndFDerivAt (by simp)).eq (0, basis j) (0, basis i)
 all_goals fun_prop
 have h1 := smooth k 2
 · simp only [fderiv_fun_const, Pi.zero_apply, ContinuousLinearMap.comp_zero,
 ContinuousLinearMap.flip_apply]
 exact (h1.contDiffAt.isSymmSndFDerivAt (by simp)).eq (0, basis i) (1, 0)
 all_goals fun_prop
 simp only [Fin.isValue, map_add, _root_.add_apply, PiLp.add_apply, Fin.isValue]
 simp only [Fin.isValue, map_smul, FunLike.coe_smul, Pi.smul_apply,

```

### 1.293 Physlib/SpaceAndTime/TimeAndSpace/ConstantTimeDist.lean

```

 convert! h0' using 1
 convert! h0' using 1
 rw [fderiv_fun_comp, DifferentiableAt.fderiv_prodMk]
 simp only [fderiv_fun_const, Pi.zero_apply, fderiv_fun_id]
 · convert! h1 x t
lemma time_integral_differentiable {d : ℕ} (η : □(Time × Space d, ℝ)) :

```

1.293.

PHYSLIB/SPACEANDTIME/TIMEANDSPACE/CONSTANTTIMEDIST. LEAN73

```

convert! h0' using 1
trans k * (||1 + ||t|| ^ rt)^-1 * ||ContinuousLinearMap.prod (0 : Space d → L[ℝ] Time
 (ContinuousLinearMap.id ℝ (Space d)))||
swap
· apply le_of_eq
 ring
refine mul_le_mul_of_nonneg ?_ ?_ (by positivity) (by positivity)
· convert! h1 x t
 simp
· rfl
rw [fderiv_fun_comp, DifferentiableAt.fderiv_prodMk]
simp only [fderiv_fun_const, Pi.zero_apply, fderiv_fun_id]
 rw [fderiv_fun_comp, DifferentiableAt.fderiv_prodMk]
 simp only [fderiv_fun_const, Pi.zero_apply, fderiv_fun_id, ContinuousLinearM
 Function.comp_apply, ContinuousLinearMap.prod_apply, _root_.zero_apply,
have hη_diff : ∀ m : ℕ, Differentiable ℝ (iteratedFDeriv ℝ m ↑η) := by
 intro m
 refine ContDiff.differentiable_iteratedFDeriv (n := (m + 1 : ℕ)) ?_ ?_
 · exact Nat.cast_lt.mpr (by omega)
 · exact η.smooth'.of_le (by exact ENat.LEInfty.out)
have hη_diff' : ∀ (m : ℕ) (t : Time),
 Differentiable ℝ (iteratedFDeriv ℝ m (fun x => η (t, x))) := by
 intro m t
 refine ContDiff.differentiable_iteratedFDeriv (n := (m + 1 : ℕ)) ?_ ?_
 · exact Nat.cast_lt.mpr (by omega)
 · exact (η.smooth'.of_le (by exact ENat.LEInfty.out)).comp (by fun_prop)
 simp only [Function.comp_apply,
 exact (hη_diff' n t).differentiableAt
 simp only [fderiv_fun_const, Pi.zero_apply, fderiv_fun_id,
 ContinuousLinearMap.coe_comp, Function.comp_apply, ContinuousLinearMap.prod
 root.zero_apply, ContinuousLinearMap.coe_id', id_eq]
 · exact (hη_diff n).differentiableAt
 · exact ((hη_diff n).fun_comp (by fun_prop)).differentiableAt
 · exact (hη_diff n).differentiableAt
 · exact ((hη_diff n).fun_comp (by fun_prop)).differentiableAt
 convert! hbound t using 1
 simp only [_root_.add_apply]
 simp only [_root_.smul_apply, smul_eq_mul, RingHom.id_apply]
simp only [fderiv_fun_const, Pi.zero_apply, fderiv_fun_id, ContinuousLinearMap.coe
 Function.comp_apply, ContinuousLinearMap.prod_apply, _root_.zero_apply,
 simp only [fderiv_fun_id, fderiv_fun_const, Pi.zero_apply,
 ContinuousLinearMap.coe_comp, Function.comp_apply, ContinuousLinearMap.prod
 ContinuousLinearMap.coe_id', id_eq, _root_.zero_apply]
 simp only [one_mul]
 simp only [fderiv_fun_id, fderiv_fun_const, Pi.ofNat_apply,
 ContinuousLinearMap.comp_apply, ContinuousLinearMap.prod_apply,

```

```
ContinuousLinearMap.id_apply, _root_.zero_apply]
```

### 1.294 Physlib/SpaceAndTime/TimeAndSpace/EuclideanGroup/Action.1

```

/-
Copyright (c) 2026 Rob Sneiderman. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Rob Sneiderman
-/
module

public import Physlib.SpaceAndTime.Space.EuclideanGroup.Action
public import Physlib.SpaceAndTime.TimeAndSpace.Basic
/-!

The action of the Euclidean group on `TimeAndSpace`

i. Overview

In this file we define the action of the Euclidean group on `TimeAndSpace d`
the time coordinate and acts on the space coordinate by the usual Euclidean
group action on
`Space d`.

ii. Key results

- `TimeAndSpace.instMulActionEuclideanGroup` : The action of the Euclidean
 `TimeAndSpace d`.
- `TimeAndSpace.dist_smul` : The Euclidean group action on `TimeAndSpace d`
- `TimeAndSpace.smul_hasTemperateGrowth` : The Euclidean group action has t
- `TimeAndSpace.antilipschitz_smul` : The Euclidean group action is antilip

iii. Table of contents

- A. The Euclidean group action
- B. Temperate growth of the coordinate action

iv. References

-/

@[expose] public section

namespace TimeAndSpace

```

1.294.

PHYSLIB/SPACEANDTIME/TIMEANDSPACE/EUCLIDEANGROUP/ACTION71.EAN

```
variable {d : ℕ}
```

```
/-!
```

```
A. The Euclidean group action
```

```
-/
```

```
/-- The Euclidean group acts on `TimeAndSpace d` by fixing time and acting on space. /
```

```
noncomputable instance instMulActionEuclideanGroup :
 MulAction (EuclideanGroup d) (TimeAndSpace d) where
 smul g tx := (tx.1, g • tx.2)
 one_smul tx := by
 rcases tx with ⟨t, x⟩
 show ((t, (1 : EuclideanGroup d) • x) : TimeAndSpace d) = (t, x)
 simp
 mul_smul g h tx := by
 rcases tx with ⟨t, x⟩
 show ((t, (g * h) • x) : TimeAndSpace d) = (t, g • h • x)
 simp [mul_smul]
```

```
/-- Coordinate formula for the Euclidean-group action on `TimeAndSpace d`. - /
```

```
@[simp]
lemma smul_mk (g : EuclideanGroup d) (t : Time) (x : Space d) :
 g • ((t, x) : TimeAndSpace d) = (t, g • x) := by
 show ((t, g • x) : TimeAndSpace d) = (t, g • x)
 rfl
```

```
/-- The Euclidean-group action fixes the first coordinate. - /
```

```
@[simp]
lemma fst_smul (g : EuclideanGroup d) (tx : TimeAndSpace d) :
 (g • tx).1 = tx.1 := by
 rcases tx with ⟨t, x⟩
 rw [smul_mk]
```

```
/-- The Euclidean-group action applies to the second coordinate. - /
```

```
@[simp]
lemma snd_smul (g : EuclideanGroup d) (tx : TimeAndSpace d) :
 (g • tx).2 = g • tx.2 := by
 rcases tx with ⟨t, x⟩
 rw [smul_mk]
```

```

/-- The Euclidean-group action fixes the time projection. -
/
lemma time_smul (g : EuclideanGroup d) (tx : TimeAndSpace d) :
 time (g • tx) = time tx := by
 simp [time]

/-- The Euclidean-group action applies to the space projection. -
/
lemma space_smul (g : EuclideanGroup d) (tx : TimeAndSpace d) :
 space (g • tx) = g • space tx := by
 simp [space]

/-- The Euclidean-group action on `TimeAndSpace d` preserves product distance. -
/
lemma dist_smul (g : EuclideanGroup d) (tx ty : TimeAndSpace d) :
 dist (g • tx) (g • ty) = dist tx ty := by
 rcases tx with ⟨t, x⟩
 rcases ty with ⟨t', x'⟩
 show dist ((t, g • x) : TimeAndSpace d) (t', g • x') = dist (t, x) (t', x')
 rw [Prod.dist_eq, Prod.dist_eq, EuclideanGroup.dist_smul]

/-!

B. Temperate growth of the coordinate action

-/

/-- The linear part of the Euclidean-group action on `TimeAndSpace d`. -
/
noncomputable def actionLinearMap (g : EuclideanGroup d) :
 TimeAndSpace d → L[ℝ] TimeAndSpace d :=
 ContinuousLinearMap.prod (ContinuousLinearMap.fst ℝ Time (Space d))
 (Space.basis.repr.symm.toContinuousLinearMap.comp
 ((EuclideanGroup.orthogonalToLinearIsometryEquiv g.linear).toContinuousLinearMap
 (Space.basis.repr.toContinuousLinearMap.comp
 (ContinuousLinearMap.snd ℝ Time (Space d)))))

/-- The translation part of the Euclidean-group action on `TimeAndSpace d`. -
/
noncomputable def actionTranslation (g : EuclideanGroup d) : TimeAndSpace d → TimeAndSpace d :=
 (0, Space.basis.repr.symm g.translation)

lemma smul_eq_actionLinearMap_add (g : EuclideanGroup d) (tx : TimeAndSpace d) :
 g • tx = actionLinearMap g tx + actionTranslation g := by
 refine Prod.ext ?_ ?_

```



1.295.

PHYSLIB/SPACEANDTIME/TIMEANDSPACE/EUCLIDEANGROUP/SCHWARTZACTION.LEAN

```
· simp [actionLinearMap, actionTranslation]
· ext i
 have hx : tx.2 -v (0 : Space d) = Space.basis.repr tx.2 := by
 ext j
 simp [Space.vsub_apply, Space.zero_apply, Space.basis_repr_apply]
 rw [TimeAndSpace.snd_smul, EuclideanGroup.smul_apply]
 simp [actionLinearMap, actionTranslation, hx, Space.add_apply, Space.basis_repr_

/-- The Euclidean-group action on `TimeAndSpace d` has temperate growth. -
/
lemma smul_hasTemperateGrowth (g : EuclideanGroup d) :
 Function.HasTemperateGrowth (fun tx : TimeAndSpace d => g • tx) := by
 have hfun : (fun tx : TimeAndSpace d => g • tx) =
 fun tx => actionLinearMap g tx + actionTranslation g := by
 funext tx
 exact smul_eq_actionLinearMap_add g tx
 rw [hfun]
 exact Function.HasTemperateGrowth.add (actionLinearMap g).hasTemperateGrowth
 (Function.HasTemperateGrowth.const (actionTranslation g))

/-- The Euclidean-group action on `TimeAndSpace d` is an isometry. -
/
lemma isometry_smul (g : EuclideanGroup d) :
 Isometry (fun tx : TimeAndSpace d => g • tx) :=
 Isometry.of_dist_eq (TimeAndSpace.dist_smul g)

/-- The Euclidean-group action on `TimeAndSpace d` is antilipschitz. -
/
lemma antilipschitz_smul (g : EuclideanGroup d) :
 AntilipschitzWith 1 (fun tx : TimeAndSpace d => g • tx) :=
 (isometry_smul g).antilipschitz

end TimeAndSpace
```

## 1.295 Physlib/SpaceAndTime/TimeAndSpace/EuclideanGroup/SchwartzAction.

```
/-
Copyright (c) 2026 Rob Sneiderman. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Rob Sneiderman
-/
module

public import Mathlib.RepresentationTheory.Continuous.Basic
```

```

public import Physlib.SpaceAndTime.TimeAndSpace.EuclideanGroup.Action
/-!

The Euclidean group action on Schwartz maps over `TimeAndSpace`

i. Overview

In this file we define the pullback action of the Euclidean group on Schwartz
`TimeAndSpace d`. The action is
`g • η = fun tx => η (g-1 • tx)`.

ii. Key results

- `TimeAndSpace.schwartzEuclideanAction` : The Euclidean group action on Schwartz
 continuous representation (`ContRepresentation`).
- `TimeAndSpace.instMulActionSchwartzMap` : The induced `MulAction` instance.
- `TimeAndSpace.smul_schwartzMap_apply` : Pointwise formula for the action.

iii. Table of contents

- A. The pullback action on Schwartz maps

iv. References

-/

@[expose] public section
noncomputable section

open SchwartzMap

namespace TimeAndSpace

variable {d : ℕ}

/-!

A. The pullback action on Schwartz maps

-/

variable {F : Type} [NormedAddCommGroup F] [NormedSpace ℝ F]

/-- The Euclidean-group pullback action on Schwartz maps over `TimeAndSpace d`
representation `ContRepresentation ℝ (EuclideanGroup d) → (TimeAndSpace d, F)`

```

1.295.

*PHYSLIB/SPACEANDTIME/TIMEANDSPACE/EUCLIDEANGROUP/SWARTZACTION.LEAN*

`ContRepresentation R G V` unfolds to the monoid homomorphism  $G \rightarrow^* V \rightarrow L[R] V$  into  
\*continuous\*-linear maps: the builder `SchwartzMap.compCLM0fAntilipschitz` already  
continuous-linear map for free, and the  $\rightarrow L$  codomain lets the action compose under  
plain mathlib `Representation` ( $G \rightarrow^* V \rightarrow L[R] V$ ) is recovered via  
`ContRepresentation.toRepresentation`. -/

```
noncomputable def schwartzEuclideanAction {d : ℕ} :
 ContRepresentation ℝ (EuclideanGroup d) (TimeAndSpace d, F) where
 toFun g := SchwartzMap.compCLM0fAntilipschitz (λ := ℝ)
 (g := fun tx : TimeAndSpace d => g⁻¹ • tx)
 (TimeAndSpace.smul_hasTemperateGrowth g⁻¹)
 (TimeAndSpace.antilipschitz_smul g⁻¹)
 map_one' := by
 ext η tx
 simp
 map_mul' g h := by
 ext η tx
 simp only [_root_.mul_inv_rev, SchwartzMap.compCLM0fAntilipschitz_apply,
 Function.comp_apply]
 rw [mul_smul]
 rfl
```

/-- Pointwise formula for the monoid-homomorphism form of the Schwartz-  
map pullback action. -/

@[simp]

```
lemma schwartzEuclideanAction_apply {d : ℕ} (g : EuclideanGroup d) (η : TimeAndSpace d → F)
 (tx : TimeAndSpace d) :
 (schwartzEuclideanAction g η) tx = η (g⁻¹ • tx) := rfl
```

/-- The Euclidean group acts on Schwartz maps over `TimeAndSpace d` by pullback. -  
/

```
noncomputable instance instMulActionSchwartzMap {d : ℕ} :
 MulAction (EuclideanGroup d) (TimeAndSpace d, F) where
 smul g η := schwartzEuclideanAction g η
 one_smul η := by
 ext tx
 change (schwartzEuclideanAction (1 : EuclideanGroup d) η) tx = η tx
 rw [schwartzEuclideanAction_apply]
 simp
 mul_smul g h η := by
 ext tx
 change (schwartzEuclideanAction (g * h) η) tx =
 (schwartzEuclideanAction g (schwartzEuclideanAction h η)) tx
 simp only [schwartzEuclideanAction_apply, _root_.mul_inv_rev]
 rw [mul_smul]
```

/-- Pointwise formula for the `MulAction` instance on Schwartz maps. -

```

/
@[simp]
lemma smul_schwartzMap_apply {d : ℕ} (g : EuclideanGroup d) (η : □(TimeAndSpace d)) (tx : TimeAndSpace d) :
 (g • η) tx = η (g-1 • tx) := rfl

/-- Applying `g` and then `h` to a Schwartz map is the pullback action of `h` to `g`. -/
lemma schwartzEuclideanAction_mul_apply {d : ℕ} (g h : EuclideanGroup d) (η : □(TimeAndSpace d, F)) :
 schwartzEuclideanAction h (schwartzEuclideanAction g η) =
 schwartzEuclideanAction (h * g) η := by
 ext tx
 simp only [schwartzEuclideanAction_apply, _root_.mul_inv_rev]
 rw [mul_smul]

/-- Each Euclidean-group pullback action on Schwartz maps is injective. -/
lemma schwartzEuclideanAction_injective {d : ℕ} (g : EuclideanGroup d) :
 Function.Injective (schwartzEuclideanAction (F := F) g) := by
 intro η1 η2 hη
 ext tx
 have htx := congrArg (fun η : □(TimeAndSpace d, F) => η (g • tx)) hη
 simpa using htx

/-- Each Euclidean-group pullback action on Schwartz maps is surjective. -/
lemma schwartzEuclideanAction_surjective {d : ℕ} (g : EuclideanGroup d) :
 Function.Surjective (schwartzEuclideanAction (F := F) g) := by
 intro η
 use schwartzEuclideanAction g-1 η
 ext tx
 simp

end TimeAndSpace

end

```

### 1.296 Physlib/StatisticalMechanics/CanonicalEnsemble/Basic.lean

```

 apply Integrable.mul_prod (f := (fun (i : ι) => 1)) (g := □1.energy) <;
 · have h1 := integrable_energy_add □ □1 T h1 h2
 simp_all only [μProd_add]
 exact h1

```

**1.297 Physlib/StatisticalMechanics/CanonicalEnsemble/Finite.lean**

```

have h_cases : ∀ s : Set (Fin 0 → 1), s = ∅ ∨ s = Set.univ := fun s =>
 s.eq_empty_or_nonempty.imp_right fun ⟨y, hy⟩ =>
 Set.eq_univ_of_forall fun x => by rwa [Subsingleton.elim x y]
rw [mathematicalPartitionFunction_eq_integral, MeasureTheory.integral_fintype]
rw [inv_mul_eq_div]
have hZpos : 0 < ∫.mathematicalPartitionFunction T := by
 rw [mathematicalPartitionFunction_of_fintype]
 exact Finset.sum_pos (fun j _ => Real.exp_pos _) Finset.univ_nonempty
rw [probability, div_le_one hZpos, mathematicalPartitionFunction_of_fintype]
simp [neg_mul] using Finset.single_le_sum
(f := fun j : 1 => Real.exp (- β T * ∫.energy j)) (fun j _ => Real.exp_nonneg _)
(Finset.mem_univ i)
have hZne : ∫.mathematicalPartitionFunction T ≠ 0 :=
 (mathematicalPartitionFunction_pos_finite ∫ T).ne'
simp_rw [probability, ← Finset.sum_div, ← mathematicalPartitionFunction_of_fintype]
exact div_self hZne
have hsum : ∑ i, ∫.probability T i * log (∫.probability T i) ≤ 0 :=
 Fintype.sum_nonpos fun i =>
 mul_nonpos_iff.2 <| 0r.inl ⟨probability_nonneg_finite ∫ T i,
 Real.log_nonpos (probability_nonneg_finite ∫ T i) (probability_le_one ∫ T i)⟩
rw [shannonEntropy, neg_mul, neg_nonneg]
exact mul_nonpos_iff.2 <| 0r.inl ⟨kB_pos.le, hsum⟩
have hd : ∀ c : ℝ, HasDerivAt (fun x => Real.exp (-x * c)) (-
c * Real.exp (-b * c)) b := by
 intro c
 have h : HasDerivAt (fun x => -x * c) (-c) b := by
 simp using (hasDerivAt_id b).neg.mul_const c
 simp [mul_comm] using h.exp
have h_sum : HasDerivAt (fun x => ∑ i, Real.exp (-x * ∫.energy i))
 (∑ i, -∫.energy i * Real.exp (-b * ∫.energy i)) b :=
 HasDerivAt.fun_sum fun i _ => hd (∫.energy i)
simp [Finset.sum_neg_distrib] using h_sum.deriv
have hd : ∀ c : ℝ, HasDerivAt (fun x => Real.exp (-x * c)) (-
c * Real.exp (-b * c)) b := by
 intro c
 have h : HasDerivAt (fun x => -x * c) (-c) b := by
 simp using (hasDerivAt_id b).neg.mul_const c
 simp [mul_comm] using h.exp
have h_sum : HasDerivAt (fun x => ∑ i, ∫.energy i * Real.exp (-
x * ∫.energy i))
 (∑ i, -(∫.energy i)^2 * Real.exp (-b * ∫.energy i)) b := by

```

```

 refine HasDerivAt.fun_sum fun i _ => ?_
 simp [sq, mul_comm, mul_left_comm, mul_assoc] using
 (hd (⊠.energy i)).const_mul (⊠.energy i)
 simp [Finset.sum_neg_distrib, sq, mul_comm, mul_left_comm, mul_assoc] us
 have hZ_diff := (differentiable_mathematicalPartitionFunctionBetaReal ⊠)
 have hZ_ne : ⊠.mathematicalPartitionFunctionBetaReal b ≠ 0 :=
 (mathematicalPartitionFunctionBetaReal_pos ⊠ b).ne'
 have hU : ⊠.meanEnergyBetaReal
 = ⊠.meanEnergyNumerator / ⊠.mathematicalPartitionFunctionBetaReal :=
 simp [meanEnergyBetaReal, probabilityBetaReal, meanEnergyNumerator,
 mathematicalPartitionFunctionBetaReal, Pi.div_apply, Finset.sum_d
 have hprob : $\sum i, (\text{⊠.energy } i)^2 * \text{⊠.probabilityBetaReal } b \ i$
 = ($\sum i, (\text{⊠.energy } i)^2 * \text{Real.exp } (-b * \text{⊠.energy } i)$)
 / ⊠.mathematicalPartitionFunctionBetaReal := by
 simp [probabilityBetaReal, Finset.sum_div, mul_div_assoc]
 rw [hprob, hU, deriv_div hN_diff hZ_diff hZ_ne, deriv_meanEnergyNumerator
 deriv_mathematicalPartitionFunctionBetaReal, Pi.div_apply]
 set S2 := $\sum i, (\text{⊠.energy } i) ^ 2 * \text{Real.exp } (-b * \text{⊠.energy } i)$
 field_simp
 ring

```

### 1.298 Physlib/StatisticalMechanics/CanonicalEnsemble/Lemmas.lean

```

 rw [NNReal.toReal]
 field_simp
 simp [toReal, hT0]
 have hTne : (T.val : ℝ) ≠ 0 := by exact_mod_cast hT.ne'
 have hkβT : T.val * (kB * (T.β : ℝ)) = 1 := by
 rw [kB_mul_beta T hT, mul_one_div, div_self hTne]
 rw [helmholtzFreeEnergy_def, partitionFunction_def, Real.log_div hZpos.ne
 Real.log_pow, ⊠.thermodynamicEntropy_eq_differentialEntropy_sub_correct
 ⊠.differentialEntropy_eq_kB_beta_meanEnergy_add_kB_log_mathZ (T := T) h
 linear_combination ⊠.meanEnergy T * hkβT

 exact (mathematicalPartitionFunction_eq_integral (⊠:=⊠) (T:=T))

```

### 1.299 Physlib/StatisticalMechanics/MicroCanonicalEnsemble/Basic

```

/-- The number of degrees of freedom is finite. -/
/-- The dim_fin to the instance cache is added so that things like the meas
/

```

1.300.

PHYSLIB/STATISTICALMECHANICS/MICROCANONICALENSEMBLE/IDEALGAS.LEAN

### 1.300 Physlib/StatisticalMechanics/MicroCanonicalEnsemble/IdealGas.lean

```

· simp_rw [Set.setOf_forall]
 exact MeasurableSet.iInter fun i => MeasurableSet.iInter fun ax =>
 measurableSet_le (by fun_prop) measurable_const
· simp_rw [← WithTop.coe_sum]
 exact WithTop.isOpenEmbedding_coe.measurableEmbedding.measurable.comp (by fun_
simp_rw [show ∀ (x y i p_i), eq_pm (x, y) (i, Sum.inl p_i) = x (i, p_i) from fun _
show ∀ (x y i m_i), eq_pm (x, y) (i, Sum.inr m_i) = y (i, m_i) from fun _ _ _
 refine Measurable.exists fun i => Measurable.exists fun j => ?_
 exact measurable_const.lt (by fun_prop)
 refine Measurable.ite (measurableSet_setOf.mpr ?_) (by fun_prop) (by fun_prop)
 exact h_measurable_box.comp measurable_fst
· exact Filter.Eventually.of_forall fun _ => by positivity
· fun_prop
· exact Filter.Eventually.of_forall fun _ => by positivity
· exact Measurable.astronglyMeasurable (by fun_prop)
· exact Filter.Eventually.of_forall fun _ => by positivity
· refine (Measurable.ite ?_ measurable_const measurable_const).astronglyMeasurable
simp_rw [Set.setOf_forall]
 exact MeasurableSet.iInter fun i => MeasurableSet.iInter fun j =>
 measurableSet_le (by fun_prop) measurable_const
· refine (Measurable.ite ?_ measurable_const measurable_const).ennreal_ofReal
simp_rw [Set.setOf_forall]
 exact MeasurableSet.iInter fun i => MeasurableSet.iInter fun j =>
 measurableSet_le (by fun_prop) measurable_const
· have h_integrand_prod : ∀ (a : Fin n × Fin 3 → ℝ),
 (if ∀ (x : Fin n) (x_1 : Fin 3), |a (x, x_1)| ≤ V ^ (3⁻¹ : ℝ) / 2 then 1 else 0)
simp_rw [h_integrand_prod]
 convert! ← MeasureTheory.integral_fintype_prod_eq_prod (ι := Fin n × Fin 3) (□ :
 (f := fun _ r => if |r| ≤ V ^ (3⁻¹ : ℝ) / 2 then 1 else 0)
 swap
 have h_integral_ld :
 (∫ (x : ℝ), if |x| ≤ V ^ (3⁻¹ : ℝ) / 2 then 1 else 0) = V ^ (3⁻¹ : ℝ) := by
 rw [Finset.prod_const, Finset.card_univ, Fintype.card_prod, Fintype.card_fin, Fi
 h_integral_ld, ← Real.rpow_mul_natCast hV.le]
· have h_gaussian :=
/-- The ideal gas law: PV = nRT. In our unitsless system, R = 1. -
/

```

### 1.301 Physlib/StatisticalMechanics/MicroCanonicalEnsemble/ThermoQuantities.lean

```

public import QuantumInfo.ForMathlib.ComplexLaplaceTransform
* Ideally this would be an NNReal, but ∫ (NNReal) doesn't work right now, so it wo

```

```

separate proof anyway
if h : E = T then 0 else Real.exp (-β * (E.untop h))) ∧ (H.partitionZ d

```

### 1.302 Physlib/StringTheory/FTheory/SU5/Fluxes/Basic.lean

```

simp only [numChiralD, numAntiChiralD, not_le, eq_sub_iff_add_eq] using
 Multiset.sum_filter_add_sum_filter_not (s := chiralIndicesOfD F) (fun x
simp only [numChiralL, numAntiChiralL, not_le, eq_sub_iff_add_eq] using
 Multiset.sum_filter_add_sum_filter_not (s := chiralIndicesOfL F) (fun x
simp only [numChiralQ, numAntiChiralQ, not_le, eq_sub_iff_add_eq] using
 Multiset.sum_filter_add_sum_filter_not (s := chiralIndicesOfQ F) (fun x
simp only [numChiralU, numAntiChiralU, not_le, eq_sub_iff_add_eq] using
 Multiset.sum_filter_add_sum_filter_not (s := chiralIndicesOfU F) (fun x
simp only [numChiralE, numAntiChiralE, not_le, eq_sub_iff_add_eq] using
 Multiset.sum_filter_add_sum_filter_not (s := chiralIndicesOfE F) (fun x

```

### 1.303 Physlib/StringTheory/FTheory/SU5/Fluxes/NoExotics/ChiralI

```

by_contra hn
simp only [not_le] at hn
have h1 := Multiset.sum_le_card_nsmul
 (Multiset.filter (fun x => x < 0) F.chiralIndicesOfD) (-
1)
 (fun x hx => by
 have := (Multiset.mem_filter.mp hx).2
 omega)
rw [nsmul_eq_mul] at h1
have h2 : 0 < Multiset.card (Multiset.filter (fun x => x < 0) F.chiralInd
 Multiset.card_pos_iff_exists_mem.mpr ⟨ci, Multiset.mem_filter.mpr ⟨hci,
by_contra hn
simp only [not_le] at hn
have h1 := Multiset.sum_le_card_nsmul
 (Multiset.filter (fun x => x < 0) F.chiralIndicesOfL) (-
1)
 (fun x hx => by
 have := (Multiset.mem_filter.mp hx).2
 omega)
rw [nsmul_eq_mul] at h1
have h2 : 0 < Multiset.card (Multiset.filter (fun x => x < 0) F.chiralInd
 Multiset.card_pos_iff_exists_mem.mpr ⟨ci, Multiset.mem_filter.mpr ⟨hci,
by_contra hn
simp only [not_le] at hn
have h1 := Multiset.sum_le_card_nsmul

```



1.303.

PHYSLIB/STRINGTHEORY/FTHEORY/SU5/FLUXES/NOEXOTICS/CHIRALINDICES.LEAN

```
(Multiset.filter (fun x => x < 0) F.chiralIndicesOfQ) (-
1)
 (fun x hx => by
 have := (Multiset.mem_filter.mp hx).2
 omega)
rw [nsmul_eq_mul] at h1
have h2 : 0 < Multiset.card (Multiset.filter (fun x => x < 0) F.chiralIndicesOfQ)
 Multiset.card_pos_iff_exists_mem.mpr {ci, Multiset.mem_filter.mpr {hci, hn}}
by_contra hn
simp only [not_le] at hn
have h1 := Multiset.sum_le_card_nsmul
 (Multiset.filter (fun x => x < 0) F.chiralIndicesOfU) (-
1)
 (fun x hx => by
 have := (Multiset.mem_filter.mp hx).2
 omega)
rw [nsmul_eq_mul] at h1
have h2 : 0 < Multiset.card (Multiset.filter (fun x => x < 0) F.chiralIndicesOfU)
 Multiset.card_pos_iff_exists_mem.mpr {ci, Multiset.mem_filter.mpr {hci, hn}}
by_contra hn
simp only [not_le] at hn
have h1 := Multiset.sum_le_card_nsmul
 (Multiset.filter (fun x => x < 0) F.chiralIndicesOfE) (-
1)
 (fun x hx => by
 have := (Multiset.mem_filter.mp hx).2
 omega)
rw [nsmul_eq_mul] at h1
have h2 : 0 < Multiset.card (Multiset.filter (fun x => x < 0) F.chiralIndicesOfE)
 Multiset.card_pos_iff_exists_mem.mpr {ci, Multiset.mem_filter.mpr {hci, hn}}
have h := F.numChiralD_eq_sum_sub_numAntiChiralD
rw [hF.2.2.2, hF.2.2.1] at h
omega
have h := F.numChiralL_eq_sum_sub_numAntiChiralL
rw [hF.2.1, hF.1] at h
omega
have h := F.numChiralQ_eq_sum_sub_numAntiChiralQ
rw [hF.2.1, hF.1] at h
omega
have h := F.numChiralU_eq_sum_sub_numAntiChiralU
rw [hF.2.2.2.1, hF.2.2.1] at h
omega
have h := F.numChiralE_eq_sum_sub_numAntiChiralE
rw [hF.2.2.2.2.1, hF.2.2.2.2.2] at h
omega
have hle := Multiset.single_le_sum
```

```

 (fun x hx => chiralIndicesOfD_nonneg_of_noExotics F hF x hx) ci hci
 rwa [F.chiralIndicesOfD_sum_eq_three_of_noExotics hF] at hle
 have hle := Multiset.single_le_sum
 (fun x hx => chiralIndicesOfL_nonneg_of_noExotics F hF x hx) ci hci
 rwa [F.chiralIndicesOfL_sum_eq_three_of_noExotics hF] at hle
 have hle := Multiset.single_le_sum
 (fun x hx => chiralIndicesOfQ_nonneg_of_noExotics F hF x hx) ci hci
 rwa [F.chiralIndicesOfQ_sum_eq_three_of_noExotics hF] at hle
 have hle := Multiset.single_le_sum
 (fun x hx => chiralIndicesOfU_nonneg_of_noExotics F hF x hx) ci hci
 rwa [F.chiralIndicesOfU_sum_eq_three_of_noExotics hF] at hle
 have hle := Multiset.single_le_sum
 (fun x hx => chiralIndicesOfE_nonneg_of_noExotics F hF x hx) ci hci
 rwa [F.chiralIndicesOfE_sum_eq_three_of_noExotics hF] at hle
 have hs : S.map (fun x => x.1) ≤ F.chiralIndicesOfD := Multiset.map_le_ma
 have hpos : 0 ≤ (F.chiralIndicesOfD - S.map (fun x => x.1)).sum :=
 Multiset.sum_nonneg fun x hx =>
 chiralIndicesOfD_nonneg_of_noExotics F hF x (Multiset.mem_of_le tsub_l
 have sum_add_compl_eq_three : (S.map (fun x => x.1)).sum +
 (F.chiralIndicesOfD - S.map (fun x => x.1)).sum = 3 := by
 rw [← Multiset.sum_add, add_tsub_cancel_of_le hs,
 F.chiralIndicesOfD_sum_eq_three_of_noExotics hF]
 have hs : S.map (fun x => (x.1 + x.2)) ≤ F.chiralIndicesOfL := Multiset.m
 have hpos : 0 ≤ (F.chiralIndicesOfL - S.map (fun x => (x.1 + x.2))).sum :=
 Multiset.sum_nonneg fun x hx =>
 chiralIndicesOfL_nonneg_of_noExotics F hF x (Multiset.mem_of_le tsub_l
 have sum_add_compl_eq_three : (S.map (fun x => (x.1 + x.2))).sum +
 (F.chiralIndicesOfL - S.map (fun x => (x.1 + x.2))).sum = 3 := by
 rw [← Multiset.sum_add, add_tsub_cancel_of_le hs,
 F.chiralIndicesOfL_sum_eq_three_of_noExotics hF]
 have hs : S.map (fun x => x.1) ≤ F.chiralIndicesOfQ := Multiset.map_le_ma
 have hpos : 0 ≤ (F.chiralIndicesOfQ - S.map (fun x => x.1)).sum :=
 Multiset.sum_nonneg fun x hx =>
 chiralIndicesOfQ_nonneg_of_noExotics F hF x (Multiset.mem_of_le tsub_l
 have sum_add_compl_eq_three : (S.map (fun x => x.1)).sum +
 (F.chiralIndicesOfQ - S.map (fun x => x.1)).sum = 3 := by
 rw [← Multiset.sum_add, add_tsub_cancel_of_le hs,
 F.chiralIndicesOfQ_sum_eq_three_of_noExotics hF]
 have hs : S.map (fun x => (x.1 - x.2)) ≤ F.chiralIndicesOfU := Multiset.m
 have hpos : 0 ≤ (F.chiralIndicesOfU - S.map (fun x => (x.1 - x.2))).sum :=
 Multiset.sum_nonneg fun x hx =>
 chiralIndicesOfU_nonneg_of_noExotics F hF x (Multiset.mem_of_le tsub_l
 have sum_add_compl_eq_three : (S.map (fun x => (x.1 - x.2))).sum +
 (F.chiralIndicesOfU - S.map (fun x => (x.1 - x.2))).sum = 3 := by
 rw [← Multiset.sum_add, add_tsub_cancel_of_le hs,
 F.chiralIndicesOfU_sum_eq_three_of_noExotics hF]

```

1.304.

*PHYSLIB/STRINGTHEORY/FTHEORY/SU5/QUANTA/FIVEQUANTA.LEAN*87

```
have hs : S.map (fun x => (x.1 + x.2)) ≤ F.chiralIndicesOfE := Multiset.map_le_map
have hpos : 0 ≤ (F.chiralIndicesOfE - S.map (fun x => (x.1 + x.2))).sum :=
 Multiset.sum_nonneg fun x hx =>
 chiralIndicesOfE_nonneg_of_noExotics F hF x (Multiset.mem_of_le tsub_le_self hx)
have sum_add_compl_eq_three : (S.map (fun x => (x.1 + x.2))).sum +
 (F.chiralIndicesOfE - S.map (fun x => (x.1 + x.2))).sum = 3 := by
 rw [← Multiset.sum_add, add_tsub_cancel_of_le hs,
 F.chiralIndicesOfE_sum_eq_three_of_noExotics hF]
```

### 1.304 Physlib/StringTheory/FTheory/SU5/Quanta/FiveQuanta.lean

```
have hl : Multiset.filter (fun p => p.1 = z) x = 0 := by
 rw [Multiset.filter_eq_nil]
 rintro (a, b) hab rfl
 exact h (Multiset.mem_map_of_mem Prod.fst hab)
simp [toChargeMap, hl]
rw [reduce]
refine Multiset.Nodup.map (fun q1 q2 h => ?_) (Multiset.nodup_dedup x.toCharges)
exact congrArg Prod.fst h
 simp [Multiset.filter_eq', h]
rw [reduce_filter x p.1 hp]
simp
have hE : F.toFluxesFive.NoExotics := ((FluxesFive.noExotics_iff_mem_elemsNoExotics)
have hnn : ∀ a ∈ F.reduce.toFluxesFive.map (fun f => f.M + f.N), 0 ≤ a := by
 obtain (f, hf, rfl) := Multiset.mem_map.mp ha
 clear ha hE
rw [FluxesFive.numChiralL, FluxesFive.chiralIndicesOfL, Multiset.filter_eq_self.mp
 ← FluxesFive.chiralIndicesOfL_sum_eq_three_of_noExotics _ hE, FluxesFive.chiralI
 toFluxesFive, toFluxesFive, Multiset.map_map, Multiset.map_map]
exact reduce_sum_eq_sum_toCharges F (fun _ =>
 ((fun f => f.M + f.N, by simp), fun x y => by simp [add_add_add_comm]))
rw [FluxesFive.numAntiChiralL, FluxesFive.chiralIndicesOfL,
 Multiset.filter_eq_nil.mpr ?_, Multiset.sum_zero]
intro a ha
obtain (f, hf, rfl) := Multiset.mem_map.mp ha
replace hf := mem_powerset_sum_of_mem_reduce_toFluxesFive hf
clear ha
generalize F.toFluxesFive = G at *
revert f
revert G
decide
have hE : F.toFluxesFive.NoExotics := ((FluxesFive.noExotics_iff_mem_elemsNoExotics)
have hnn : ∀ a ∈ F.reduce.toFluxesFive.map (fun f => f.M), 0 ≤ a := by
 obtain (f, hf, rfl) := Multiset.mem_map.mp ha
```

```

clear ha hE
rw [FluxesFive.numChiralD, FluxesFive.chiralIndicesOfD, Multiset.filter_eq
 ← FluxesFive.chiralIndicesOfD_sum_eq_three_of_noExotics _ hE, FluxesFiv
 toFluxesFive, toFluxesFive, Multiset.map_map, Multiset.map_map]
exact reduce_sum_eq_sum_toCharges F (fun _ => ⟨(fun f => f.M, by simp), f
rw [FluxesFive.numAntiChiralD, FluxesFive.chiralIndicesOfD,
 Multiset.filter_eq_nil.mpr ?_, Multiset.sum_zero]
intro a ha
obtain ⟨f, hf, rfl⟩ := Multiset.mem_map.mp ha
replace hf := mem_powerset_sum_of_mem_reduce_toFluxesFive hf
clear ha
generalize F.toFluxesFive = G at *
revert f
revert G
decide
have hcard : ∀ v : Fluxes, (x.filter (fun y => y.2 = v)).card =
 (x.toFluxesFive.filter (fun y => y = v)).card := by
 intro v
 rw [toFluxesFive, Multiset.filter_map]
 simp
have hmap : ∀ v : Fluxes, Multiset.map (fun y => (y.1, v)) (x.filter (fun
 x.filter (fun y => y.2 = v) := by
 intro v
 trans Multiset.map (fun y => y) (x.filter (fun y => y.2 = v))
 · refine Multiset.map_congr rfl fun y hy => ?_
 simp only [Multiset.mem_filter] at hy
 rw [← hy.2]
 · simp
 rw [hcard ⟨1, -1⟩, h2]
 rw [hcard ⟨0, 1⟩, h2]
 rw [hmap ⟨1, -1⟩, hmap ⟨0, 1⟩, hx]
have reduce_sum_N_mul_eq : ∀ h : $\square \rightarrow \square$, (F.reduce.map fun x => (x.2.N : $\square$
 (F.map fun x => (x.2.N : $\square$) * h x.1).sum := by
 intro h
 let f : $\square \rightarrow \text{Fluxes} \rightarrow \square := \text{fun } q5 =>
 \{ \text{toFun} := \text{fun } x => x.N \cdot h \text{ } q5
 \text{map_zero}' := \text{by simp}
 \text{map_add}' := \text{by}
 \text{intros } x \text{ } y
 \text{simp [add_mul]} \}
 simp [anomalyCoefficient]
exact ⟨reduce_sum_N_mul_eq fun q => q, reduce_sum_N_mul_eq fun q => q * q$
```

1.305.

PHYSLIB/STRINGTHEORY/FTHEORY/SU5/QUANTA/TENQUANTA.LEAN489

### 1.305 Physlib/StringTheory/FTheory/SU5/Quanta/TenQuanta.lean

```
have h1 : x.filter (fun p => p.1 = z) = 0 := by
 rw [Multiset.filter_eq_nil]
 rintro (a, b) hp rfl
 exact h (Multiset.mem_map_of_mem Prod.fst hp)
simp [toChargeMap, h1]
rw [reduce]
refine Multiset.Nodup.map ?_ (Multiset.nodup_dedup _)
intro a b h
exact congrArg Prod.fst h
have hE : F.toFluxesTen.NoExotics := ((FluxesTen.noExotics_iff_mem_elemsNoExotics
have numChiralU_eq_sum := F.toFluxesTen.numChiralU_eq_sum_sub_numAntiChiralU
rw [hE.2.2.1, hE.2.2.2.1, sub_zero, FluxesTen.chiralIndicesOfU] at numChiralU_eq_s
have hmap : (F.reduce.toFluxesTen.map (fun f => f.M - f.N)).sum =
 (F.toFluxesTen.map (fun f => f.M - f.N)).sum := by
 have h := reduce_sum_eq_sum_toCharges F
 (fun _ => ((fun f => f.M - f.N, by simp), fun a b => by simp; ring) : Fluxes
 simp [toFluxesTen, Multiset.map_map, Function.comp] using h
rw [FluxesTen.numChiralU, FluxesTen.chiralIndicesOfU, Multiset.filter_eq_self.mpr,
· exact numChiralU_eq_sum.symm
intro a ha
obtain (f, hf, rfl) := Multiset.mem_map.mp ha
replace hf := mem_powerset_sum_of_mem_reduce_toFluxesTen hf
clear ha hmap numChiralU_eq_sum hE
generalize F.toFluxesTen = G at *
revert f
revert G
decide
rw [FluxesTen.numAntiChiralU, FluxesTen.chiralIndicesOfU, Multiset.filter_eq_nil.m
 Multiset.sum_zero]
intro a ha
obtain (f, hf, rfl) := Multiset.mem_map.mp ha
replace hf := mem_powerset_sum_of_mem_reduce_toFluxesTen hf
clear ha
generalize F.toFluxesTen = G at *
revert f
revert G
decide
have hE : F.toFluxesTen.NoExotics := ((FluxesTen.noExotics_iff_mem_elemsNoExotics
have numChiralQ_eq_sum := F.toFluxesTen.numChiralQ_eq_sum_sub_numAntiChiralQ
rw [hE.1, hE.2.1, sub_zero, FluxesTen.chiralIndicesOfQ] at numChiralQ_eq_sum
have hmap : (F.reduce.toFluxesTen.map (fun f => f.M)).sum =
 (F.toFluxesTen.map (fun f => f.M)).sum := by
 have h := reduce_sum_eq_sum_toCharges F
 (fun _ => ((fun f => f.M, by simp), fun a b => by simp) : Fluxes →+ ℤ))
```

```

 simp [toFluxesTen, Multiset.map_map, Function.comp] using h
 rw [FluxesTen.numChiralQ, FluxesTen.chiralIndicesOfQ, Multiset.filter_eq
 · exact numChiralQ_eq_sum.symm
 intro a ha
 obtain ⟨f, hf, rfl⟩ := Multiset.mem_map.mp ha
 replace hf := mem_powerset_sum_of_mem_reduce_toFluxesTen hf
 clear ha hmap numChiralQ_eq_sum hE
 generalize F.toFluxesTen = G at *
 revert f
 revert G
 decide
 rw [FluxesTen.numAntiChiralQ, FluxesTen.chiralIndicesOfQ, Multiset.filter
 Multiset.sum_zero]
 intro a ha
 obtain ⟨f, hf, rfl⟩ := Multiset.mem_map.mp ha
 replace hf := mem_powerset_sum_of_mem_reduce_toFluxesTen hf
 clear ha
 generalize F.toFluxesTen = G at *
 revert f
 revert G
 decide
 have hE : F.toFluxesTen.NoExotics := ((FluxesTen.noExotics_iff_mem_elemsN
 have numChiralE_eq_sum := F.toFluxesTen.numChiralE_eq_sum_sub_numAntiChir
 rw [hE.2.2.2.2.1, hE.2.2.2.2.2, sub_zero, FluxesTen.chiralIndicesOfE] at
 have hmap : (F.reduce.toFluxesTen.map (fun f => f.M + f.N)).sum =
 (F.toFluxesTen.map (fun f => f.M + f.N)).sum := by
 have h := reduce_sum_eq_sum_toCharges F
 (fun _ => (({fun f => f.M + f.N, by simp}, fun a b => by simp; ring)
 simp [toFluxesTen, Multiset.map_map, Function.comp] using h
 rw [FluxesTen.numChiralE, FluxesTen.chiralIndicesOfE, Multiset.filter_eq
 · exact numChiralE_eq_sum.symm
 intro a ha
 obtain ⟨f, hf, rfl⟩ := Multiset.mem_map.mp ha
 replace hf := mem_powerset_sum_of_mem_reduce_toFluxesTen hf
 clear ha hmap numChiralE_eq_sum hE
 generalize F.toFluxesTen = G at *
 revert f
 revert G
 decide
 rw [FluxesTen.numAntiChiralE, FluxesTen.chiralIndicesOfE, Multiset.filter
 Multiset.sum_zero]
 intro a ha
 obtain ⟨f, hf, rfl⟩ := Multiset.mem_map.mp ha
 replace hf := mem_powerset_sum_of_mem_reduce_toFluxesTen hf
 clear ha
 generalize F.toFluxesTen = G at *

```

```

revert f
revert G
decide
have weighted_N_sum_eq : $\forall w : \square \rightarrow \square$,
 (Multiset.map (fun x => (x.2.N : $\square$) * w x.1) F.reduce).sum =
 (Multiset.map (fun x => (x.2.N : $\square$) * w x.1) F).sum := fun w =>
 reduce_sum_eq_sum_toCharges F fun q5 =>
 { toFun := fun x => (x.N : $\square$) * w q5
 map_zero' := by simp
 map_add' := fun x y => by simp [add_mul] }
exact (weighted_N_sum_eq fun q => q, congrArg (3 * ·) (weighted_N_sum_eq fun q =>

```

### 1.306 Physlib/Thermodynamics/IdealGas/Basic.lean

```

-- Unfold the entropy and expand every `log (x / y)` into `log x - log y`,
-- so both `hS` and the goal become linear in the individual logarithms.
unfold entropy at hS
rw [Real.log_div hUa.ne' hU0.ne', Real.log_div hUb.ne' hU0.ne',
 Real.log_div hVa.ne' hV0.ne', Real.log_div hVb.ne' hV0.ne'] at hS
rw [Real.log_div hUa.ne' hUb.ne', Real.log_div hVa.ne' hVb.ne']
-- The difference of the two entropies is `N * R` times the goal, so the
-- goal is exactly the second factor of a vanishing product.
have key : $N * R * (c * (\log Ua - \log Ub) + (\log Va - \log Vb)) = 0$:= by
 linear_combination hS
exact (mul_eq_zero.mp key).resolve_left (mul_ne_zero hN.ne' hR.ne')
 have hlog := adiabatic_relation_log hUa hUb hVa hVb hN hU0 hV0 hR hS
 -- The product is `exp` of the left-hand side of `hlog`, i.e. `exp 0 = 1`.
 show (Ua / Ub) ^ c * (Va / Vb) = 1
 rw [Real.rpow_def_of_pos (div_pos hUa hUb), ← Real.exp_log (div_pos hVa hVb),
 ← Real.exp_add, mul_comm (log (Ua / Ub)) c, hlog, Real.exp_zero]

```

### 1.307 Physlib/Thermodynamics/Temperature/Basic.lean

```

(1 / (kB * (T : $\mathbb{R}$))), div_nonneg zero_le_one (mul_nonneg kB_nonneg T.val.2))

lemma β _toReal (T : Temperature) : (β T : $\mathbb{R}$) = 1 / (kB * (T : $\mathbb{R}$)) := rfl
 ((1 / (kB * β), div_nonneg zero_le_one (mul_nonneg kB_nonneg β .2)))
 apply NNReal.coe_injective
 show 1 / (kB * (1 / (kB * $\uparrow\beta$))) = $\uparrow\beta$ '
 rw [mul_one_div, one_div_div, mul_div_cancel_left, _ kB_ne_zero]

lemma of β _toReal (β : $\mathbb{R}_{\geq 0}$) : (of β β).toReal = 1 / (kB * (β : $\mathbb{R}$)) := rfl
 apply Temperature.ext

```

```

apply NNReal.coe_injective
show 1 / (kB * (1 / (kB * (T : ℝ)))) = (T : ℝ)
rw [mul_one_div, one_div_div, mul_div_cancel_left, _ kB_ne_zero]
rw [β_toReal]
exact one_div_pos.mpr (mul_pos kB_pos (by exact_mod_cast hT_pos))
have hg : ContinuousOn (fun b : ℝ ≥ 0 => (1 : ℝ) / (kB * (b : ℝ))) (Set.Ioi 0)
 apply ContinuousOn.div continuousOn_const (by fun_prop)
 intro b hb
 exact ne_of_gt (mul_pos kB_pos (by exact_mod_cast hb))
rw [hind.continuousOn_iff]
refine (continuous_real_toNNReal.comp_continuousOn hg).congr ?_
intro b hb
exact (Real.toNNReal_of_nonneg (div_nonneg zero_le_one (mul_nonneg kB_pos (by exact_mod_cast hb))
 exact mul_ne_zero kB_ne_zero (ne_of_gt hb))
 rw [show ((ofβ (Real.toNNReal x)).val : ℝ) = (ofβ (Real.toNNReal x)).toReal,
 ofβ_toReal, Real.coe_toNNReal x hx.le]
filter_upwards [eventually_gt_atTop 0] with b hb
have : 0 < (1 : ℝ) / (kB * (b : ℝ)) := one_div_pos.mpr (mul_pos kB_pos (b : ℝ))
simp [ofβ_toReal] using this
have h : Tendsto (fun b : ℝ ≥ 0 => a * (b : ℝ)) atTop atTop :=
 (NNReal.tendsto_coe_atTop.2 tendsto_id).const_mul_atTop ha
simp only [one_div]
exact h.inv_tendsto_atTop
 atTop (λ (0 : ℝ)) :=
tendsto_const_inv_mul_atTop kB kB_pos
refine tendsto_nhdsWithin_iff.2 (tendsto_toReal_ofβ_atTop, ?_)
simp using eventually_pos_ofβ
simp only [betaFromReal, β_toReal, Temperature.toReal, ofNNReal_val, Real.coe_val]
have hg : HasDerivAt (fun t : ℝ => kB * t) kB (T.val : ℝ) := by
 simp using (hasDerivAt_id (T.val : ℝ)).const_mul kB
have h_deriv : HasDerivAt (fun t : ℝ => 1 / (kB * t))
 (-1 / (kB * (T.val : ℝ) ^ 2)) (T.val : ℝ) := by
 have h := (hasDerivAt_const (T.val : ℝ) (1 : ℝ)).div hg (mul_ne_zero kB_pos)
 have hval : (-1 : ℝ) / (kB * (T.val : ℝ) ^ 2)
 = (0 * (kB * (T.val : ℝ)) - 1 * kB) / (kB * (T.val : ℝ) ^ 2) := by
 rw [mul_pow]
 field_simp
 rw [hval]
exact h
exact (h_deriv.hasDerivWithinAt).congr beta_fun_T_eq_on_Ioi (beta_fun_T_eq_on_Ioi ht)
show 0 < betaFromReal t
rw [beta_fun_T_eq_on_Ioi ht]
exact one_div_pos.mpr (mul_pos kB_pos ht)
rw [beta_fun_T_eq_on_Ioi (show (T.val : ℝ) ∈ Set.Ioi 0 from hT_pos), β_toReal]
rfl
rw [h_beta_at_T]

```



1.308.

*PHYSLIB/THERMODYNAMICS/TEMPERATURE/TEMPERATUREUNITS.LEAN*

```
exact hF_deriv
exact hF_deriv'.comp (T.val : ℝ) (deriv_beta_wrt_T (T := T) hT_pos) h_map
```

### 1.308 Physlib/Thermodynamics/Temperature/TemperatureUnits.lean

```
simp [toReal]
· exact zero_le
rfl
simp only [one_div, NNReal.coe_inv]
rw [toReal, inv_div]
simp [div_eq_val, toReal]
rw [toReal]
```

### 1.309 Physlib/Units/Basic.lean

```
public import Physlib.Meta.TODO.Basic
TemperatureUnit.div_eq_val, NNReal.coe_mul, coe_rpow]
rw [toReal]
```

### 1.310 Physlib/Units/Dimension.lean

```
simp [div_eq_mul_inv, sub_eq_add_neg]
simp [div_eq_mul_inv, sub_eq_add_neg]
simp [div_eq_mul_inv, sub_eq_add_neg]
simp [div_eq_mul_inv, sub_eq_add_neg]
simp [div_eq_mul_inv, sub_eq_add_neg]
| succ n ih => rw [pow_succ, length_mul, ih, succ_nsmul]
| succ n ih => rw [pow_succ, time_mul, ih, succ_nsmul]
| succ n ih => rw [pow_succ, mass_mul, ih, succ_nsmul]
| succ n ih => rw [pow_succ, charge_mul, ih, succ_nsmul]
| succ n ih => rw [pow_succ, temperature_mul, ih, succ_nsmul]
```

### 1.311 Physlib/Units/Examples.lean

```
show (1609.344 : ℝ)-1 * 400 = _
norm_num [WithDim.scaleUnit_val, M, NNReal.smul_def]
unfold EnergyMass
simp only [eq_iff_iff, UnitDependent.scaleUnit_apply_fun_left,
UnitDependent.scaleUnit_apply_fun, Dimensionful.of_scaleUnit, dim_apply,
WithDim.scaleUnit_val, WithDim.smul_val, map_mul, mul_pow, NNReal.smul_def, smul
```

```

 NNReal.coe_mul]
ring_nf
simp [mul_assoc, mul_eq_mul_left_iff, dimScale_ne_zero]
simp [EnergyMass, speedOfLight, toDimensionful_apply_apply, dimScale, SI]
rw [← energyMass_isDimensionallyCorrect SI u]

```

### 1.312 Physlib/Units/UnitDependent.lean

```

public import Mathlib.Topology.Algebra.Module.Spaces.ContinuousLinearMap
 · simp [LengthUnit.div_eq_val, toReal]
 · simp [TimeUnit.div_eq_val, toReal]
 · simp [MassUnit.div_eq_val, toReal]
 · simp [ChargeUnit.div_eq_val, toReal]
 · simp [TemperatureUnit.div_eq_val, toReal]
 · simp [LengthUnit.scale, LengthUnit.div_eq_val, toReal]
 · simp [TimeUnit.scale, TimeUnit.div_eq_val, toReal]
 · simp [MassUnit.scale, MassUnit.div_eq_val, toReal]
 · simp [ChargeUnit.scale, ChargeUnit.div_eq_val, toReal]
 · simp [TemperatureUnit.scale, TemperatureUnit.div_eq_val, toReal]
 simp [LengthUnit.div_eq_val, toReal]
 simp [TimeUnit.div_eq_val, toReal]
 simp [MassUnit.div_eq_val, toReal]
 simp [ChargeUnit.div_eq_val, toReal]
 simp [TemperatureUnit.div_eq_val, toReal]
simp only [IsDimensionallyCorrect, funext_iff]
rfl
simp only [IsDimensionallyCorrect, funext_iff]
rfl
simp only [IsDimensionallyCorrect, funext_iff]
rfl

```

### 1.313 Physlib/Units/WithDim/Area.lean

```

simp [squareFoot, dimScale, LengthUnit.feet, toDimensionful_apply_apply,]
simp [NNReal.coe_ofScientific]
rw [toReal]
simp [NNReal.coe_ofScientific]
rw [toReal]
simp [NNReal.coe_ofScientific]
rw [toReal]

```

**1.314 Physlib/Units/WithDim/Basic.lean**

```

/-!
Inherited instances
-/

instance (d : Dimension) (M : Type) [Inhabited M] : Inhabited (WithDim d M) where
 default := ⟨default⟩

instance (d : Dimension) (M : Type) [Zero M] : Zero (WithDim d M) where
 zero := ⟨0⟩

@[simp]
lemma val_zero {d : Dimension} {M : Type} [Zero M] :
 (0 : WithDim d M).val = 0 := rfl

instance (d : Dimension) (M : Type) [Add M] : Add (WithDim d M) where
 add m1 m2 := ⟨m1.val + m2.val⟩

@[simp]
lemma val_add {d : Dimension} {M : Type} [Add M] (m1 m2 : WithDim d M) :
 (m1 + m2).val = m1.val + m2.val := rfl

instance (d : Dimension) (M : Type) [Neg M] : Neg (WithDim d M) where
 neg m := ⟨-m.val⟩

@[simp]
lemma val_neg {d : Dimension} {M : Type} [Neg M] (m : WithDim d M) :
 (-m).val = -m.val := rfl

instance (d : Dimension) (M : Type) [Sub M] : Sub (WithDim d M) where
 sub m1 m2 := ⟨m1.val - m2.val⟩

@[simp]
lemma val_sub {d : Dimension} {M : Type} [Sub M] (m1 m2 : WithDim d M) :
 (m1 - m2).val = m1.val - m2.val := rfl

instance (d : Dimension) (M : Type) [AddSemigroup M] :
 AddSemigroup (WithDim d M) where
 add_assoc m1 m2 m3 := by
 ext
 simp [add_assoc]

instance (d : Dimension) (M : Type) [AddCommSemigroup M] :
 AddCommSemigroup (WithDim d M) where
 add_comm m1 m2 := by

```

```

 ext
 simp [add_comm]

instance (d : Dimension) (M : Type) [AddMonoid M] :
 AddMonoid (WithDim d M) where
 zero_add m := by
 ext
 simp [zero_add]
 add_zero m := by
 ext
 simp [add_zero]
 nsmul := nsmulRec

instance (d : Dimension) (M : Type) [AddCommMonoid M] :
 AddCommMonoid (WithDim d M) where
 add_comm m1 m2 := by
 ext
 simp [add_comm]

instance (d : Dimension) (M : Type) [AddGroup M] :
 AddGroup (WithDim d M) where
 sub_eq_add_neg m1 m2 := by
 ext
 simp [sub_eq_add_neg]
 neg_add_cancel m := by
 ext
 simp [neg_add_cancel]
 zsmul := zsmulRec

instance (d : Dimension) (M : Type) [AddCommGroup M] :
 AddCommGroup (WithDim d M) where
 add_comm m1 m2 := by
 ext
 simp [add_comm]

instance (d : Dimension) (M : Type) [LE M] : LE (WithDim d M) where
 le m1 m2 := m1.val ≤ m2.val

@[simp]
lemma le_def {d : Dimension} {M : Type} [LE M] (m1 m2 : WithDim d M) :
 m1 ≤ m2 ↔ m1.val ≤ m2.val := Iff.rfl

instance (d : Dimension) (M : Type) [LT M] : LT (WithDim d M) where
 lt m1 m2 := m1.val < m2.val

@[simp]

```

```
lemma lt_def {d : Dimension} {M : Type} [LT M] (m1 m2 : WithDim d M) :
 m1 < m2 ↔ m1.val < m2.val := Iff.rfl
```

```
instance (d : Dimension) (M : Type) [Preorder M] :
 Preorder (WithDim d M) where
 le_refl m := by
 exact le_refl m.val
 le_trans m1 m2 m3 h12 h23 := by
 change m1.val ≤ m3.val
 exact le_trans h12 h23
 lt_iff_le_not_ge m1 m2 := by
 change m1.val < m2.val ↔ m1.val ≤ m2.val ∧ ¬ m2.val ≤ m1.val
 exact lt_iff_le_not_ge
```

```
instance (d : Dimension) (M : Type) [PartialOrder M] :
 PartialOrder (WithDim d M) where
 le_antisymm m1 m2 h12 h21 := by
 ext
 exact le_antisymm h12 h21
```

TODO "Induce further non-additive algebraic, additional order, and topological instances on `WithDim d M` from instances on `M`."

### 1.315 Physlib/Units/WithDim/Speed.lean

```
simp [NNReal.coe_ofScientific]
rw [toReal]
simp only [WithDim.smul_val, smul_eq_mul, mul_one, NNReal.coe_mul, coe_rpow, NNReal.coe_ofNat]
rw [toReal]
simp only [WithDim.smul_val, smul_eq_mul, mul_one, NNReal.coe_mul, coe_rpow, NNReal.coe_ofNat]
rw [toReal]
```

### 1.316 PhyslibAlpha.lean

module

```
public import PhyslibAlpha.Basic
public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation
public import PhyslibAlpha.ClassicalFieldTheory.Local.Action
public import PhyslibAlpha.ClassicalFieldTheory.Local.EulerLagrange
public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstOrder
```

```

public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Basic
public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Criter
public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Densit
public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Integr
public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Regula
public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Support
public import PhyslibAlpha.ClassicalFieldTheory.Local.JetPoint
public import PhyslibAlpha.ClassicalFieldTheory.Local.JetPointFiber
public import PhyslibAlpha.ClassicalFieldTheory.Local.JetPointRegularity
public import PhyslibAlpha.ClassicalFieldTheory.Local.Lagrangian
public import PhyslibAlpha.ClassicalFieldTheory.Local.TotalDerivative
public import PhyslibAlpha.ClassicalFieldTheory.Local.TotalDivergence
public import PhyslibAlpha.ClassicalFieldTheory.Local.TotalDivergenceEquiv
public import PhyslibAlpha.ClassicalFieldTheory.Local.Variation
public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.HalfPlane
public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.Line
public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.Ring
public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.SphericalCylinder
public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.SolidCylinder
public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.SolidSphere
public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.SphericalShell
public import PhyslibAlpha.QuantumMechanics.QuantumHarmonicOscillator
public import PhyslibAlpha.QuantumMechanics.StinespringDilation
public import PhyslibAlpha.Mathematics.PartialDerivativeTest
public import PhyslibAlpha.ClassicalMechanics.CoupledSpringPotential

```

### 1.317 PhyslibAlpha/Basic.lean

```

/-
Copyright (c) 2026 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module
public import Physlib.Meta.TODO.Basic
/-!

```

#### ## Overview

PhyslibAlpha is an extension of Physlib with a lighter review process. We expect the file structure to match where possible that of Physlib.

The idea is that it sits between the high review standards of Physlib and just allowing anything in the project.

1.318. *PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/ACTION.LEAN*<sup>49</sup>

The review standard is similar to that of the arXiv, with a "one-look" policy.

We will look at the code once, and if it looks good, we will merge it, assuming it passes CI checks.

We will not promise to maintain files in PhyslibAlpha, if they break, we will simply record when they broke.

We will make an effort to move files which are used frequently from PhyslibAlpha to undertaking the more rigorous review process.

-/

@[expose] public section

### **1.318 PhyslibAlpha/ClassicalFieldTheory/Local/Action.lean**

/-

Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Juan Jose Fernandez Morales

-/

module

public import PhyslibAlpha.ClassicalFieldTheory.Local.Lagrangian

public import PhyslibAlpha.ClassicalFieldTheory.Local.Variation

/-!

# Local action functionals

## i. Overview

This module defines the local action functional associated with a local Lagrangian, the first notions needed to talk about variational criticality.

For the first implementation pass, the action is defined directly as the integral of Lagrangian evaluated along jets of a field. The integrability conditions needed for its variations are kept explicit in the API.

## ii. Key results

- `ClassicalFieldTheory.Local.actionDensity` : the density associated with a field.
- `ClassicalFieldTheory.Local.action` : the action of a field.
- `ClassicalFieldTheory.Local.HasFiniteAction` : finiteness of the action integral.

```

- `ClassicalFieldTheory.Local.IsAdmissibleForAction` : symmetric admissibil
 field pair for the action functional.
- `ClassicalFieldTheory.Local.actionVariation` : the action under an admiss
- `ClassicalFieldTheory.Local.IsCritical` : vanishing first derivative of t

```

```

iii. Table of contents

```

```

- A. Action densities and action
- B. Action under variation
- C. Critical fields

```

```

iv. References

```

```

- J. Cortés and A. Haupt, *Lecture Notes on Mathematical Methods of Classic
 Chapter 5.

```

```

-/

```

```

@[expose] public section

```

```

open MeasureTheory
open Physlib

```

```

namespace ClassicalFieldTheory
namespace Local

```

```

open scoped ContDiff Topology

```

```

/-!

```

```

A. Action densities and action

```

```

-/

```

```

/-- The action density determined by a local Lagrangian and a field. -
/

```

```

noncomputable def actionDensity (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ :=
 L.alongField f

```

```

/-- The integrability condition for the action density of a field. -
/

```

```

def HasFiniteAction (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ
 Integrable (actionDensity L f)

```

```

/-- Symmetric admissibility predicate for the action functional: the pair `
 when the field is smooth and the corresponding action density is integrable

```



```

/
def IsAdmissibleForAction (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop :=
 ContDiff ℝ ∞ f ∧ HasFiniteAction L f

/-- The action associated with a local Lagrangian. -/
noncomputable def action (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : ℝ :=
 ∫ x, actionDensity L f x

@[simp]
lemma actionDensity_apply (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
 actionDensity L f x = L (jetAt k f x) := rfl

@[simp]
lemma action_eq_integral (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin
 action L f = ∫ x, actionDensity L f x := rfl

/-!
B. Action under variation

-/

/-- The field obtained by varying `f` in the direction `η` with parameter `s`. -
/
noncomputable def variedField (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 (s : ℝ) :
 Space d → EuclideanSpace ℝ (Fin m) :=
 fun x => f x + s • η x

/-- Smoothness of the varied field for smooth `f` and admissible `η`. -
/
lemma variedField_contDiff (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 (s : ℝ)
 (hf : ContDiff ℝ ∞ f) :
 ContDiff ℝ ∞ (variedField f η s) := by
 exact hf.add (η.isTestFunction.contDiff.const_smul s)

/-- The action of a field under an admissible variation. -
/
noncomputable def actionVariation (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) : ℝ → ℝ :=

```

```

 fun s => action L (variedField f η s)

/-- Explicit integrability condition for the family of varied action densities
/
def HasFiniteActionVariation (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) : Prop :=
 ∀ s : ℝ, Integrable (actionDensity L (variedField f η s))

/-- Locality package for the action density under compactly supported variation
 variation parameter `s`, the change in the action density is a continuous compactly
 function. Combined with finiteness of the base action, this implies finiteness of the
 action. -/
def HasCompactlySupportedActionVariationDifference (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) : Prop :=
 ∀ s : ℝ,
 Continuous (fun x => actionDensity L (variedField f η s) x - actionDensity L f x)
 HasCompactSupport (fun x => actionDensity L (variedField f η s) x - actionDensity L f x)

/-- If the base action is finite and every varied action density differs from the base
 a continuous compactly supported function, then the whole varied family has finite
 action.
/
lemma hasFiniteActionVariation_of_hasFiniteAction_of_compactlySupportedDifference
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 (hbase : HasFiniteAction L f)
 (hdiff : HasCompactlySupportedActionVariationDifference L f η) :
 HasFiniteActionVariation L f η := by
 intro s
 rcases hdiff s with ⟨hcont, hsupp⟩
 have hsub :
 Integrable (fun x => actionDensity L (variedField f η s) x - actionDensity L f x)
 exact hcont.integrable_of_hasCompactSupport hsupp
 have hadd :
 Integrable
 (fun x =>
 (actionDensity L (variedField f η s) x - actionDensity L f x)
 + actionDensity L f x) := by
 exact hsub.add hbase
 simp_all [sub_eq_add_neg, add_assoc]
 exact hadd

@[simp]
lemma variedField_apply (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (s : ℝ)
 (x : Space d) :

```

```
variedField f η s x = f x + s • η x := rfl
```

```
@[simp]
```

```
lemma actionVariation_apply (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (s : ℝ) :
 actionVariation L f η s = action L (variedField f η s) := rfl
```

```
lemma jetAt_variedField_eq_of_notMem_tsupport
 (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (s : ℝ)
 {x : Space d}
 (hf : ContDiff ℝ ∞ f) (hx : x ∉ tsupport η.toFun) :
 jetAt k (variedField f η s) x = jetAt k f x := by
 have hcoords :
 jetCoordinatesAt k (variedField f η s) x = jetCoordinatesAt k f x := by
 have h1 := jetCoordinatesAt_add_smul k f η.toFun x s hf η.isTestFunction.contDiff
 simp_all [jetCoordinatesAt_eq_zero_of_notMem_tsupport k η.toFun hx]
 exact h1
 rw [jetAt_eq_ofBaseCoordinates, jetAt_eq_ofBaseCoordinates]
 exact congrArg (JetPoint.ofBaseCoordinates x) hcoords
```

```
lemma actionDensity_variedField_eq_of_notMem_tsupport
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 (s : ℝ)
 {x : Space d} (hf : ContDiff ℝ ∞ f) (hx : x ∉ tsupport η.toFun) :
 actionDensity L (variedField f η s) x = actionDensity L f x := by
 simp [actionDensity_apply, jetAt_variedField_eq_of_notMem_tsupport k f η s hf hx]
```

```
lemma hasCompactlySupportedActionVariationDifference_of_continuousInCoordinates
 (L : Lagrangian d m k) (hcontL : Lagrangian.ContinuousInCoordinates L)
 (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) :
 HasCompactlySupportedActionVariationDifference L f η := by
 intro s
 have hbaseCont : Continuous (actionDensity L f) := by
 exact Lagrangian.continuousAlongField_of_inCoordinates L hcontL f hf
 have hvarCont : Continuous (actionDensity L (variedField f η s)) := by
 exact Lagrangian.continuousAlongField_of_inCoordinates L hcontL (variedField f η s hf)
 refine (hvarCont.sub hbaseCont, ?_)
 refine η.hasCompactSupport.of_isClosed_subset (isClosed_tsupport _) ?_
 have hsupp :
 Function.support (fun x => actionDensity L (variedField f η s) x - actionDensity L f x)
 ⊆ tsupport η.toFun := by
 intro x hx
```

```

 by_contra hxt
 rw [Function.mem_support] at hx
 have heq := actionDensity_variedField_eq_of_notMem_tsupport L f η s hf
 rw [heq] at hx
 exact hx (sub_self _)
 simp [tsupport] using closure_minimal hsupp (isClosed_tsupport _)

/-!
C. Critical fields

-/

/-- A field is critical for a local action if every admissible compactly su
vanishing first derivative at `s = 0`, under the explicit finite-
action hypothesis for the varied
family. -/
def IsCritical (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin
 ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)), HasFiniteActionVa
 HasDerivAt (actionVariation L f η) 0 0

end Local
end ClassicalFieldTheory

```

### 1.319 PhyslibAlpha/ClassicalFieldTheory/Local/EulerLagrange.lean

```

/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/
module

public import PhyslibAlpha.ClassicalFieldTheory.Local.Action
/-!
Local Euler-Lagrange operators

i. Overview

This module defines the local Euler-Lagrange operator associated with a loc

In the first implementation pass, the derivatives of the Lagrangian with re
coordinates are packaged explicitly as part of the local Lagrangian data. T
close to the one in the book while avoiding a premature smooth structure on

```

1.319.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/EULERLAGRANGE.lean

## ii. Key results

- `ClassicalFieldTheory.Local.eulerLagrangeTerm` : a single summand in the local Euler-Lagrange formula.
- `ClassicalFieldTheory.Local.eulerLagrangeComponent` : one component `E\_a(L)` of the
- `ClassicalFieldTheory.Local.eulerLagrangeOp` : the full local Euler-Lagrange operator.

## iii. Table of contents

- A. Euler-Lagrange summands
- B. Components of the Euler-Lagrange operator
- C. The full operator

## iv. References

- J. Cortés and A. Haupt, \*Lecture Notes on Mathematical Methods of Classical Physics Chapter 5.

-/

@[expose] public section

open scoped BigOperators  
open Physlib

namespace ClassicalFieldTheory  
namespace Local

/-!

## A. Euler-Lagrange summands

-/

/-- The summand  $(-1)^{|I|} D_I (\partial L / \partial u^{a_I})$  in the local Euler-Lagrange formula. -/

noncomputable def eulerLagrangeTerm (L : Lagrangian d m k) (I : DerivativeIndex d k)  
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ :=  
 fun x =>  
 (-1 : ℝ) ^ I.1.order \* iteratedTotalDerivative k I.1 (L.coordDeriv I a) f x

@[simp]

lemma eulerLagrangeTerm\_apply (L : Lagrangian d m k) (I : DerivativeIndex d k) (a :  
 (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :  
 eulerLagrangeTerm L I a f x =  
 (-1 : ℝ) ^ I.1.order \* iteratedTotalDerivative k I.1 (L.coordDeriv I a) f x :=

```

/-!
B. Components of the Euler-Lagrange operator

-/

/-- The `a`-th component of the local Euler-Lagrange operator. -
/
noncomputable def eulerLagrangeComponent (L : Lagrangian d m k) (a : Fin m)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ :=
 fun x => ∑ I : DerivativeIndex d k, eulerLagrangeTerm L I a f x

@[simp]
lemma eulerLagrangeComponent_apply (L : Lagrangian d m k) (a : Fin m)
 (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
 eulerLagrangeComponent L a f x = ∑ I : DerivativeIndex d k, eulerLagran

/-!
C. The full operator

-/

/-- The local Euler-Lagrange operator attached to a local Lagrangian. -
/
noncomputable def eulerLagrangeOp (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) :
 Space d → EuclideanSpace ℝ (Fin m) :=
 fun x => WithLp.toLp 2 fun a => eulerLagrangeComponent L a f x

lemma eulerLagrangeOp_apply (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (x : Space d) (a : Fin m) :
 eulerLagrangeOp L f x a = eulerLagrangeComponent L a f x := by
 simp [eulerLagrangeOp]

@[simp]
lemma eulerLagrangeOp_eq (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) :
 eulerLagrangeOp L f = fun x => WithLp.toLp 2 fun a => eulerLagrangeComponent L a f x := by
 rfl

end Local
end ClassicalFieldTheory

```

### 1.320 PhyslibAlpha/ClassicalFieldTheory/Local/FirstOrder.lean

```

/-

```

1.320.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTORDER.LEAN07

Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Juan Jose Fernandez Morales

-/  
module

public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation

/-!

# First-order local field theory

## i. Overview

This module provides a thin usability layer for first-order local field theory, i.e. specialization of the local CFT stack to  $k = 1$ .

The definitions here do not introduce a new theory. They are aliases and projections existing coordinate-readout stack:

- first-order jet points are ``JetPoint d m 1``,
- first-order lagrangians are ``Lagrangian d m 1``,
- field values are still the zero-order jet coordinates,
- first derivatives are the coordinates indexed by ``MultiIndex.increment 0 i``.

This keeps the general finite-order API as the source of truth while making the common order

case easier to state in examples and later mechanics bridges.

## ii. Key results

- ``ClassicalFieldTheory.Local.FirstOrderJetPoint``
- ``ClassicalFieldTheory.Local.FirstOrderLagrangian``
- ``ClassicalFieldTheory.Local.firstDerivativeIndex``
- ``ClassicalFieldTheory.Local.JetPoint.firstDerivCoord``
- ``ClassicalFieldTheory.Local.firstOrderJetAt``

## iii. Table of contents

- A. First-order aliases
- B. First derivative indices
- C. First-order jet projections
- D. First-order evaluation and variational API

## iv. References

- J. Cortés and A. Haupt, \*Lecture Notes on Mathematical Methods of Classical Physics arXiv:1612.03100v2, Chapter 5.

```

-/

@[expose] public section

open Physlib

namespace ClassicalFieldTheory
namespace Local

/-!
A. First-order aliases

-/

/-- Coordinate data for first-order local jet points. -/
abbrev FirstOrderJetCoordinates (d m : ℕ) := JetCoordinates d m 1

/-- Coordinate-level first-order jet points. -/
abbrev FirstOrderJetPoint (d m : ℕ) := JetPoint d m 1

/-- Fiber-direction data for first-order jet points. -/
abbrev FirstOrderJetFiberData (d m : ℕ) := JetFiberData d m 1

/-- First-order local lagrangians. -/
abbrev FirstOrderLagrangian (d m : ℕ) := Lagrangian d m 1

/-!
B. First derivative indices

-/

/-- The first-derivative coordinate index in the source direction `i`. -
/
def firstDerivativeIndex (d : ℕ) (i : Fin d) : DerivativeIndex d 1 :=
 ⟨Physlib.MultiIndex.increment 0 i, by simp⟩

@[simp]
lemma firstDerivativeIndex_coe (d : ℕ) (i : Fin d) :
 ((firstDerivativeIndex d i : DerivativeIndex d 1) : MultiIndex d) =
 Physlib.MultiIndex.increment 0 i := rfl

lemma firstDerivativeIndex_order (d : ℕ) (i : Fin d) :
 ((firstDerivativeIndex d i : DerivativeIndex d 1) : MultiIndex d).order =
 simp [firstDerivativeIndex]

```



1.320.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTORDER.LEAN09

/-!

## C. First-order jet projections

-/

namespace JetPoint

variable {d m :  $\mathbb{N}$ }

/-- The first derivative coordinate  $u^a_i$  of a first-order jet point. -

/

def firstDerivCoord (J : FirstOrderJetPoint d m) (i : Fin d) (a : Fin m) :  $\mathbb{R}$  :=  
 J.coord (firstDerivativeIndex d i) a

/-- The bundled target vector of first derivative coordinates in the source direction

/

def firstDerivVector (J : FirstOrderJetPoint d m) (i : Fin d) : EuclideanSpace  $\mathbb{R}$  (Fin m)  
 WithLp.toLp 2 fun a => J.firstDerivCoord i a

@[simp]

lemma firstDerivVector\_apply (J : FirstOrderJetPoint d m) (i : Fin d) (a : Fin m) :  
 J.firstDerivVector i a = J.firstDerivCoord i a := by  
 simp [firstDerivVector]

@[simp]

lemma firstDerivCoord\_ofBaseCoordinates (x : Space d) (u : FirstOrderJetCoordinates d m)  
 (i : Fin d) (a : Fin m) :  
 (JetPoint.ofBaseCoordinates x u).firstDerivCoord i a = u (firstDerivativeIndex d i) a

end JetPoint

/-!

## D. First-order evaluation and variational API

-/

/-- The first-order coordinate-level jet point of a field at a point. -

/

noncomputable abbrev firstOrderJetAt (f : Space d  $\rightarrow$  EuclideanSpace  $\mathbb{R}$  (Fin m)) (x : Space d) :  
 FirstOrderJetPoint d m :=  
 jetAt 1 f x

@[simp]

lemma firstOrderJetAt\_base (f : Space d  $\rightarrow$  EuclideanSpace  $\mathbb{R}$  (Fin m)) (x : Space d) :  
 (firstOrderJetAt f x).base = x := rfl

```

lemma firstOrderJetAt_value (f : Space d → EuclideanSpace ℝ (Fin m)) (x : S
 (firstOrderJetAt f x).value = f x := by
 simp [firstOrderJetAt]

@[simp]
lemma firstOrderJetAt_firstDerivCoord (f : Space d → EuclideanSpace ℝ (Fin
 (x : Space d) (i : Fin d) (a : Fin m) :
 (firstOrderJetAt f x).firstDerivCoord i a =
 ∂^[Physlib.MultiIndex.increment 0 i] (fun y => (f y) a) x := rfl

/-- Evaluate a first-order local lagrangian along the first jets of a field
/
noncomputable abbrev firstOrderActionDensity (L : FirstOrderLagrangian d m)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ :=
 actionDensity L f

/-- The action of a first-order local lagrangian. -/
noncomputable abbrev firstOrderAction (L : FirstOrderLagrangian d m)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : ℝ :=
 action L f

/-- The Euler-Lagrange operator of a first-order local lagrangian. -
/
noncomputable abbrev firstOrderEulerLagrangeOp (L : FirstOrderLagrangian d
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → EuclideanSpace ℝ (
 eulerLagrangeOp L f

theorem firstOrder_isCritical_iff_eulerLagrange_zero
 (L : FirstOrderLagrangian d m) (f : Space d → EuclideanSpace ℝ (Fin m))
 (hLf : IsAdmissibleForAction L f)
 (hL : Lagrangian.SmoothInCoordinates L) :
 IsCritical L f ↔ firstOrderEulerLagrangeOp L f = 0 := by
 exact isCritical_iff_eulerLagrange_zero_of_admissibleForAction_and_smooth

end Local
end ClassicalFieldTheory

```

### 1.321 PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation.lean

```

/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/

```

1.321.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATION/SLAEAN

module

public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Criterion

/-!

# First variation and the Euler-Lagrange criterion

## i. Overview

This module is the public entry point for the local first-variation theory. The core linearized density objects live in `FirstVariation.Basic`, the analytic derivation tools and hypotheses live in `FirstVariation.Density`, `FirstVariation.IntegrationByParts`, `FirstVariation.Regularity`, and `FirstVariation.Criterion`, and this file exposes the surface-level statements of the local Euler-Lagrange criterion.

## ii. Key results

- `ClassicalFieldTheory.Local.hasSmoothEulerLagrangeRegularity\_of\_smoothInCoordinates`
- `ClassicalFieldTheory.Local.isCritical\_iff\_eulerLagrange\_zero\_of\_contDiff\_and\_smooth`
- `ClassicalFieldTheory.Local.isCritical\_iff\_eulerLagrange\_zero\_of\_admissibleForAction\_and\_smoothInCoordinates`

## iii. Table of contents

- A. Smooth-coordinate regularity
- B. Final Euler-Lagrange criteria

## iv. References

- J. Cortés and A. Haupt, \*Lecture Notes on Mathematical Methods of Classical Physics\*, Chapter 5, Theorem 5.2.

-/

@[expose] public section

open scoped ContDiff

namespace ClassicalFieldTheory

namespace Local

/-!

## A. Smooth-coordinate regularity

-/

```

theorem hasSmoothEulerLagrangeRegularity_of_smoothInCoordinates
 (L : Lagrangian d m k) (hL : Lagrangian.SmoothInCoordinates L) :
 HasSmoothEulerLagrangeRegularity L := by
 exact hasSmoothEulerLagrangeRegularity_of_contDiffCoordDerivInCoordinates

/-!
B. Final Euler-Lagrange criteria

-/

theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_smoothInCoordinates
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : C
 (hL : Lagrangian.SmoothInCoordinates L)
 (hbase : HasFiniteAction L f) :
 IsCritical L f ↔ eulerLagrangeOp L f = 0 := by
 exact
 isCritical_iff_eulerLagrange_zero_of_hasFiniteAction_and_continuousInCo
 L f hf hL.2 hL.1 hbase

theorem isCritical_iff_eulerLagrange_zero_of_admissibleForAction_and_smooth
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (hLf : IsAdmissibleForAction L f)
 (hL : Lagrangian.SmoothInCoordinates L) :
 IsCritical L f ↔ eulerLagrangeOp L f = 0 := by
 rcases hLf with ⟨hf, hbase⟩
 exact isCritical_iff_eulerLagrange_zero_of_contDiff_and_smoothInCoordinates

end Local
end ClassicalFieldTheory

```

### 1.322 PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Basis

```

/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/
module

public import PhyslibAlpha.ClassicalFieldTheory.Local.EulerLagrange
public import Physlib.Mathematics.VariationalCalculus.Basic
/-!
First variation core objects

```

1.322.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATION/BASIC.LEAN

## i. Overview

This module contains the basic objects used throughout the local first-variation theory:  
the linearized density before integration by parts and its Euler-Lagrange pairing.

## ii. Key results

- `ClassicalFieldTheory.Local.firstVariationDensityTerm`
- `ClassicalFieldTheory.Local.firstVariationDensity`
- `ClassicalFieldTheory.Local.firstVariationValue`

## iii. Table of contents

- A. First-variation values

## iv. References

- J. Cortés and A. Haupt, \*Lecture Notes on Mathematical Methods of Classical Physics\* Chapter 5, Theorem 5.2.

-/

@[expose] public section

open MeasureTheory  
open InnerProductSpace  
open Physlib  
open scoped BigOperators ContDiff

namespace ClassicalFieldTheory  
namespace Local

/-!

## A. First-variation values

-/

/-- A single term in the linearized first-variation density before integration by parts  
/

noncomputable def firstVariationDensityTerm (L : Lagrangian d m k)  
 (f : Space d → EuclideanSpace ℝ (Fin m))  
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (I : DerivativeIndex d k)  
 (a : Fin m) :  
 Space d → ℝ :=

```

 fun x => L.coordDeriv I a (jetAt k f x) * $\partial^{[I.1]}$ (fun y => (η y) a) x

/-- The pointwise linearized first-variation density before integration by
/
noncomputable def firstVariationDensity (L : Lagrangian d m k)
 (f : Space d $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace $\mathbb{R}$ (Fin m))) : Space d $\rightarrow$ $\mathbb{R}$:=
 fun x => $\sum$ I : DerivativeIndex d k, $\sum$ a : Fin m, firstVariationDensityTerm L f η I a x

/-- The value predicted by the first-variation formula for an admissible variation
/
noncomputable def firstVariationValue (L : Lagrangian d m k)
 (f : Space d $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace $\mathbb{R}$ (Fin m))) : $\mathbb{R}$:=
 $\int$ x, $\square$ eulerLagrangeOp L f x, η x $\square$ $\mathbb{R}$

@[simp]
lemma firstVariationDensityTerm_apply (L : Lagrangian d m k)
 (f : Space d $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace $\mathbb{R}$ (Fin m))) (I : DerivativeIndex d k)
 (a : Fin m) (x : Space d) :
 firstVariationDensityTerm L f η I a x =
 L.coordDeriv I a (jetAt k f x) * $\partial^{[I.1]}$ (fun y => (η y) a) x := rfl

@[simp]
lemma firstVariationDensity_apply (L : Lagrangian d m k)
 (f : Space d $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace $\mathbb{R}$ (Fin m))) (x : Space d) :
 firstVariationDensity L f η x =
 $\sum$ I : DerivativeIndex d k, $\sum$ a : Fin m, firstVariationDensityTerm L f η I a x

@[simp]
lemma firstVariationValue_eq_integral (L : Lagrangian d m k)
 (f : Space d $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace $\mathbb{R}$ (Fin m))) :
 firstVariationValue L f η = $\int$ x, $\square$ eulerLagrangeOp L f x, η x $\square$ $\mathbb{R}$:= rfl

lemma firstVariationValue_eq_zero_of_eulerLagrange_zero
 (L : Lagrangian d m k) (f : Space d $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace $\mathbb{R}$ (Fin m))) (hEuler : eulerLagrangeOp L f η = 0) :
 firstVariationValue L f η = 0 := by
 unfold firstVariationValue
 rw [hEuler]
 simp
end Local

```

1.323.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATION/CRITERION.LEAN

end ClassicalFieldTheory

### 1.323 PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Criterion

/-

Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Juan Jose Fernandez Morales

-/

module

public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Density

public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.IntegrationByParts

public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Regularity

/-!

# First variation criteria

## i. Overview

This module assembles the analytic ingredients of the local first-variation proof into the

packaged first-variation formula and the internal Euler-Lagrange criteria used by the facade.

## ii. Key results

- `ClassicalFieldTheory.Local.

isCritical\_iff\_eulerLagrange\_zero\_of\_hasFiniteAction\_and\_continuousInCoordinates`

## iii. Table of contents

- A. First-variation assembly

- B. Final Euler-Lagrange criterion

## iv. References

- J. Cortés and A. Haupt, \*Lecture Notes on Mathematical Methods of Classical Physics\*, Chapter 5, Theorem 5.2.

-/

@[expose] public section

open MeasureTheory

```

open InnerProductSpace
open Physlib
open scoped BigOperators ContDiff

namespace ClassicalFieldTheory
namespace Local

/-!
A. First-variation assembly

-/

/-- Explicit global finiteness hypothesis for all admissible variations. -
/
def AllVariationsHaveFiniteAction (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop :=
 ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)), HasFiniteActionVa

/-- The first-variation formula for the local action, packaged as a reusable
/
def HasFirstVariationFormula (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop :=
 ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)), HasFiniteActionVa
 HasDerivAt (actionVariation L f η) (firstVariationValue L f η) 0

lemma isCritical_of_eulerLagrange_zero (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m))
 (hfirst : HasFirstVariationFormula L f) (hEuler : eulerLagrangeOp L f = 0) :
 IsCritical L f := by
 intro η hη
 have hderiv := hfirst η hη
 have hzero : firstVariationValue L f η = 0 :=
 firstVariationValue_eq_zero_of_eulerLagrange_zero L f η hEuler
 rw [hzero] at hderiv
 exact hderiv

lemma eulerLagrange_zero_of_isCritical (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m))
 (hfirst : HasFirstVariationFormula L f)
 (hfin : AllVariationsHaveFiniteAction L f)
 (hcont : Continuous (eulerLagrangeOp L f))
 (hcrit : IsCritical L f) :
 eulerLagrangeOp L f = 0 := by
 apply fundamental_theorem_of_variational_calculus' (@volume (Space d) _)
 · exact hcont
 · intro g hg

```



1.323.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATIONANDCRITERION.LEAN

```

let η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)) := ⟨g, hg⟩
have hηfin : HasFiniteActionVariation L f η := hfin η
have hfirstη : HasDerivAt (actionVariation L f η) (firstVariationValue L f η) 0
 hfirst η hηfin
have hcritη : HasDerivAt (actionVariation L f η) 0 0 := hcrit η hηfin
have hzero : firstVariationValue L f η = 0 := HasDerivAt.unique hfirstη hcritη
simp [firstVariationValue, η] using hzero

theorem isCritical_iff_eulerLagrange_zero (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m))
 (hfirst : HasFirstVariationFormula L f)
 (hfin : AllVariationsHaveFiniteAction L f)
 (hcont : Continuous (eulerLagrangeOp L f)) :
 IsCritical L f ↔ eulerLagrangeOp L f = 0 := by
 constructor
 · exact eulerLagrange_zero_of_isCritical L f hfirst hfin hcont
 · exact isCritical_of_eulerLagrange_zero L f hfirst

private lemma hasFirstVariationFormula_of_underIntegral_linearized_and_parts
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (hint : HasActionVariationDerivativeUnderIntegral L f)
 (hpoint : HasPointwiseLinearizedDensityFormula L f)
 (hibp : HasIntegratedByPartsFormula L f) :
 HasFirstVariationFormula L f := by
 intro η hη
 have hderiv := hint η hη
 have hlinearized :
 (∫ x, deriv (fun s : ℝ => actionDensity L (variedField f η s) x) 0)
 = ∫ x, firstVariationDensity L f η x := by
 apply integral_congr_ae
 filter_upwards with x
 simp using hpoint η x
 have hvalue :
 (∫ x, deriv (fun s : ℝ => actionDensity L (variedField f η s) x) 0)
 = firstVariationValue L f η := by
 rw [hlinearized, hibp η]
 rw [hvalue] at hderiv
 exact hderiv

private lemma hasFirstVariationFormula_of_contDiff_underIntegral_and_parts
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ
 (hint : HasActionVariationDerivativeUnderIntegral L f)
 (hibp : HasIntegratedByPartsFormula L f) :
 HasFirstVariationFormula L f := by
 exact hasFirstVariationFormula_of_underIntegral_linearized_and_parts L f hint
 (hasPointwiseLinearizedDensityFormula_of_contDiff L f hf) hibp

```

```

private lemma hasFirstVariationFormula_of_underIntegral_linearized_and_termwise
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (hint : HasActionVariationDerivativeUnderIntegral L f)
 (hpoint : HasPointwiseLinearizedDensityFormula L f)
 (hterm : HasTermwiseIntegratedByPartsFormula L f) :
 HasFirstVariationFormula L f := by
 exact hasFirstVariationFormula_of_underIntegral_linearized_and_parts L f
 (hasIntegratedByPartsFormula_of_termwise L f hterm)

private lemma hasFirstVariationFormula_of_contDiff_underIntegral_and_termwise
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : C
 (hint : HasActionVariationDerivativeUnderIntegral L f)
 (hterm : HasTermwiseIntegratedByPartsFormula L f) :
 HasFirstVariationFormula L f := by
 exact hasFirstVariationFormula_of_contDiff_underIntegral_and_parts L f hf
 (hasIntegratedByPartsFormula_of_termwise L f hterm)

/-!
B. Intermediate Euler-Lagrange criteria

-/

/-- The local Euler-Lagrange criterion obtained from the packaged analytic
first-variation formula. This is the current formalized form of Theorem 5.2
/
private theorem isCritical_iff_eulerLagrange_zero_of_underIntegral_lineariz
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (hint : HasActionVariationDerivativeUnderIntegral L f)
 (hpoint : HasPointwiseLinearizedDensityFormula L f)
 (hibp : HasIntegratedByPartsFormula L f)
 (hfin : AllVariationsHaveFiniteAction L f)
 (hcont : Continuous (eulerLagrange0p L f)) :
 IsCritical L f ↔ eulerLagrange0p L f = 0 := by
 exact isCritical_iff_eulerLagrange_zero L f
 (hasFirstVariationFormula_of_underIntegral_linearized_and_parts L f hin
 hfin hcont)

/-- Variant of the local Euler-Lagrange criterion where the integration-
by-parts input is reduced
to a termwise hypothesis. -/
private theorem isCritical_iff_eulerLagrange_zero_of_underIntegral_lineariz
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (hint : HasActionVariationDerivativeUnderIntegral L f)
 (hpoint : HasPointwiseLinearizedDensityFormula L f)
 (hterm : HasTermwiseIntegratedByPartsFormula L f)

```

1.323.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATIONANDCRITERION.LEAN

```
(hfin : AllVariationsHaveFiniteAction L f)
(hcont : Continuous (eulerLagrangeOp L f)) :
IsCritical L f $\leftrightarrow$ eulerLagrangeOp L f = 0 := by
exact isCritical_iff_eulerLagrange_zero_of_underIntegral_linearized_and_parts L f
hint hpoint (hasIntegratedByPartsFormula_of_termwise L f hterm) hfin hcont

private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_underIntegral_and_term
(L : Lagrangian d m k) (f : Space d $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin m)) (hf : ContDiff $\mathbb{R}$
(hint : HasActionVariationDerivativeUnderIntegral L f)
(hterm : HasTermwiseIntegratedByPartsFormula L f)
(hfin : AllVariationsHaveFiniteAction L f)
(hcont : Continuous (eulerLagrangeOp L f)) :
IsCritical L f $\leftrightarrow$ eulerLagrangeOp L f = 0 := by
exact isCritical_iff_eulerLagrange_zero L f
(hasFirstVariationFormula_of_contDiff_underIntegral_and_termwise L f hf hint hte
hfin hcont

private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_continuous_coordDeriv
(L : Lagrangian d m k) (f : Space d $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin m)) (hf : ContDiff $\mathbb{R}$
(hcontVar : $\forall \eta$: AdmissibleVariation d (EuclideanSpace $\mathbb{R}$ (Fin m)),
 $\forall I$: DerivativeIndex d k, $\forall a$: Fin m,
Continuous (fun p : $\mathbb{R} \times$ Space d $\Rightarrow$
L.coordDeriv I a (jetAt k (variedField f η p.1) p.2)))
(hterm : HasTermwiseIntegratedByPartsFormula L f)
(hfin : AllVariationsHaveFiniteAction L f)
(hcont : Continuous (eulerLagrangeOp L f)) :
IsCritical L f $\leftrightarrow$ eulerLagrangeOp L f = 0 := by
exact isCritical_iff_eulerLagrange_zero_of_contDiff_underIntegral_and_termwise L f
(hasActionVariationDerivativeUnderIntegral_of_contDiff_of_continuous_coordDeriv
hcontVar)
hterm hfin hcont

private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_regular
(L : Lagrangian d m k) (f : Space d $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin m)) (hf : ContDiff $\mathbb{R}$
(hcontVar : $\forall \eta$: AdmissibleVariation d (EuclideanSpace $\mathbb{R}$ (Fin m)),
Lagrangian.ContinuousCoordDerivAlongFamily L (fun s : $\mathbb{R} \Rightarrow$ variedField f η s))
(hcoeff : Lagrangian.ContDiffCoordDerivAlongField L f)
(hfin : AllVariationsHaveFiniteAction L f)
(hcont : Continuous (eulerLagrangeOp L f)) :
IsCritical L f $\leftrightarrow$ eulerLagrangeOp L f = 0 := by
exact isCritical_iff_eulerLagrange_zero_of_contDiff_underIntegral_and_termwise L f
(hasActionVariationDerivativeUnderIntegral_of_contDiff_of_regular L f hf hcontVa
(hasTermwiseIntegratedByPartsFormula_of_regular L f hcoeff)
hfin hcont

private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_regularityAt
```

```

(L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : C
(hreg : HasEulerLagrangeRegularityAt L f)
(hfin : AllVariationsHaveFiniteAction L f)
(hcont : Continuous (eulerLagrangeOp L f)) :
IsCritical L f ↔ eulerLagrangeOp L f = 0 := by
rcases hreg with ⟨hcoeff, hcontVar⟩
exact isCritical_iff_eulerLagrange_zero_of_contDiff_and_regular L f hf hco
hfin hcont

private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_smoothReg
(L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : C
(hreg : HasSmoothEulerLagrangeRegularity L)
(hfin : AllVariationsHaveFiniteAction L f)
(hcont : Continuous (eulerLagrangeOp L f)) :
IsCritical L f ↔ eulerLagrangeOp L f = 0 := by
exact isCritical_iff_eulerLagrange_zero_of_contDiff_and_regularityAt L f
hfin hcont

private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_coordinat
(L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : C
(hcoord : Lagrangian.ContDiffCoordDerivInCoordinates L)
(hfin : AllVariationsHaveFiniteAction L f)
(hcont : Continuous (eulerLagrangeOp L f)) :
IsCritical L f ↔ eulerLagrangeOp L f = 0 := by
exact isCritical_iff_eulerLagrange_zero_of_contDiff_and_smoothRegularity
(hasSmoothEulerLagrangeRegularity_of_contDiffCoordDerivInCoordinates L

private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_regularity
(L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : C
(hreg : HasEulerLagrangeRegularityAt L f)
(hfin : AllVariationsHaveFiniteAction L f) :
IsCritical L f ↔ eulerLagrangeOp L f = 0 := by
rcases hreg with ⟨hcoeff, hcontVar⟩
exact isCritical_iff_eulerLagrange_zero_of_contDiff_and_regular L f hf hco
(continuous_eulerLagrangeOp_of_regular L f hcoeff)

private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_smoothReg
(L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : C
(hreg : HasSmoothEulerLagrangeRegularity L)
(hfin : AllVariationsHaveFiniteAction L f) :
IsCritical L f ↔ eulerLagrangeOp L f = 0 := by
exact isCritical_iff_eulerLagrange_zero_of_contDiff_and_regularityAt' L f

private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_coordinat
(L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : C
(hcoord : Lagrangian.ContDiffCoordDerivInCoordinates L)

```

1.324.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATION/DENSITY.LEAN

```
(hfin : AllVariationsHaveFiniteAction L f) :
 IsCritical L f $\leftrightarrow$ eulerLagrangeOp L f = 0 := by
exact isCritical_iff_eulerLagrange_zero_of_contDiff_and_smoothRegularity' L f hf
 (hasSmoothEulerLagrangeRegularity_of_contDiffCoordDerivInCoordinates L hcoord) h

private theorem
 isCritical_iff_eulerLagrange_zero_of_contDiff_and_coordinateRegularity_of_hasFiniteAction
 (L : Lagrangian d m k) (f : Space d $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin m)) (hf : ContDiff $\mathbb{R}$
 (hcoord : Lagrangian.ContDiffCoordDerivInCoordinates L)
 (hbase : HasFiniteAction L f)
 (hlocal :
 $\forall$ η : AdmissibleVariation d (EuclideanSpace $\mathbb{R}$ (Fin m)),
 HasCompactlySupportedActionVariationDifference L f η) :
 IsCritical L f $\leftrightarrow$ eulerLagrangeOp L f = 0 := by
have hfin : AllVariationsHaveFiniteAction L f := by
 intro η
 exact hasFiniteActionVariation_of_hasFiniteAction_of_compactlySupportedDifference
 L f η hbase (hlocal η)
exact isCritical_iff_eulerLagrange_zero_of_contDiff_and_coordinateRegularity' L f hf hcoord hbase

theorem isCritical_iff_eulerLagrange_zero_of_hasFiniteAction_and_continuousInCoordinates
 (L : Lagrangian d m k) (f : Space d $\rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin m)) (hf : ContDiff $\mathbb{R}$
 (hcoord : Lagrangian.ContDiffCoordDerivInCoordinates L)
 (hcontL : Lagrangian.ContinuousInCoordinates L)
 (hbase : HasFiniteAction L f) :
 IsCritical L f $\leftrightarrow$ eulerLagrangeOp L f = 0 := by
exact isCritical_iff_eulerLagrange_zero_of_contDiff_and_coordinateRegularity_of_hasFiniteAction
 L f hf hcoord hbase
 (fun η =>
 hasCompactlySupportedActionVariationDifference_of_continuousInCoordinates
 L hcontL f hf η)

end Local
end ClassicalFieldTheory
```

## 1.324 PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Density.lean

/-

Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Juan Jose Fernandez Morales

-/

module

```

public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Support
public import Mathlib.Analysis.Calculus.ParametricIntegral
/-!
First variation density formulas

i. Overview

This module contains the pointwise and integral first-variation formulas be
parts: differentiation of the varied action density, dominated differentiat
and the corresponding packaged hypotheses.

ii. Key results

- `ClassicalFieldTheory.Local.hasPointwiseLinearizedDensityFormula_of_contD
- `ClassicalFieldTheory.Local.hasActionVariationDerivativeUnderIntegral_of_

iii. Table of contents

- A. Differentiation under the integral sign
- B. Pointwise linearization

iv. References

- J. Cortés and A. Haupt, *Lecture Notes on Mathematical Methods of Classic
 Chapter 5, Theorem 5.2.

-/

@[expose] public section

open MeasureTheory
open InnerProductSpace
open Physlib
open scoped BigOperators ContDiff

namespace ClassicalFieldTheory
namespace Local

/-!
A. Differentiation under the integral sign

-/

/-- Differentiation of the varied action under the integral sign, packaged
/
def HasActionVariationDerivativeUnderIntegral (L : Lagrangian d m k)

```

1.324.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATION/DENSITY.LEAN

```

 (f : Space d → EuclideanSpace ℝ (Fin m)) :
 Prop :=
 ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)), HasFiniteActionVariation L
 HasDerivAt (actionVariation L f η)
 (∫ x, deriv (fun s : ℝ => actionDensity L (variedField f η s) x) 0) 0

/-- Pointwise linearization of the action density before integration by parts. -
/
def HasPointwiseLinearizedDensityFormula (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop :=
 ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)), ∀ x : Space d,
 deriv (fun s : ℝ => actionDensity L (variedField f η s) x) 0 =
 firstVariationDensity L f η x

/-- A direct bridge from Mathlib's dominated differentiation-
under-the-integral theorem to the
local action variation. This isolates the measure-theoretic input needed to prove
`HasActionVariationDerivativeUnderIntegral`. -/
lemma actionVariation_hasDerivAt_of_dominated_loc_of_deriv_le
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 {F' : ℝ → Space d → ℝ} {bound : Space d → ℝ} {ε : ℝ} (hε : 0 < ε)
 (hmeas :
 ∀f s in nhds (0 : ℝ),
 AESTronglyMeasurable (actionDensity L (variedField f η s)) volume)
 (hfinite0 : Integrable (actionDensity L (variedField f η 0)))
 (hF'_meas : AESTronglyMeasurable (F' 0) volume)
 (hbound :
 ∀m x ∂volume, ∀ s ∈ Metric.ball (0 : ℝ) ε, ||F' s x|| ≤ bound x)
 (hbound_int : Integrable bound volume)
 (hderiv :
 ∀m x ∂volume, ∀ s ∈ Metric.ball (0 : ℝ) ε,
 HasDerivAt (fun r : ℝ => actionDensity L (variedField f η r) x) (F' s x) s)
 HasDerivAt (actionVariation L f η) (∫ x, F' 0 x) 0 := by
 have hs : Metric.ball (0 : ℝ) ε ∈ nhds (0 : ℝ) := Metric.ball_mem_nhds _ hε
 exact
 (hasDerivAt_integral_of_dominated_loc_of_deriv_le
 (μ := volume) (F := fun s x => actionDensity L (variedField f η s) x)
 (x₀ := (0 : ℝ)) (s := Metric.ball (0 : ℝ) ε) hs
 hmeas hfinite0 hF'_meas hbound hbound_int hderiv).2

/-- A dominated bound for the pointwise first variation along the varied-
field family near
`s = 0`. This packages the analytic domination needed to differentiate the action un
integral sign. -/
def HasDominatedVariationDerivativeNear (L : Lagrangian d m k)

```

```

(f : Space d → EuclideanSpace ℝ (Fin m))
(η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) : Prop :=
 ∃ ε > 0, ∃ bound : Space d → ℝ,
 AESTronglyMeasurable (firstVariationDensity L f η) volume ∧
 Integrable bound volume ∧
 ∀m x ∂volume, ∀ s ∈ Metric.ball (0 : ℝ) ε,
 ||firstVariationDensity L (variedField f η s) η x|| ≤ bound x

private lemma continuous_coordDeriv_at_zero
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 (hcont : ∀ I : DerivativeIndex d k, ∀ a : Fin m,
 Continuous (fun p : ℝ × Space d =>
 L.coordDeriv I a (jetAt k (variedField f η p.1) p.2)))
 (I : DerivativeIndex d k) (a : Fin m) :
 Continuous (fun x : Space d => L.coordDeriv I a (jetAt k f x)) := by
 have hpair : Continuous (fun x : Space d => ((0 : ℝ), x)) := by
 fun_prop
 have hcoeff0 :
 Continuous (fun x : Space d => L.coordDeriv I a (jetAt k (variedField
 exact (hcont I a).comp hpair
 simp [variedField_zero] using hcoeff0

private lemma firstVariationDensityTerm_norm_le_of_coeff_bound
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 (ψ : DerivativeIndex d k → Fin m → Space d → ℝ)
 (hψ : ∀ I : DerivativeIndex d k, ∀ a : Fin m, ∀ x : Space d,
 ψ I a x = ∂^[I.1] (fun y => (η y) a) x)
 (C : DerivativeIndex d k → Fin m → ℝ)
 (K : DerivativeIndex d k → Fin m → Set (Space d))
 (hKzero : ∀ I : DerivativeIndex d k, ∀ a : Fin m, ∀ x, x ∉ K I a → ψ I
 (hC : ∀ I : DerivativeIndex d k, ∀ a : Fin m,
 ∀ p ∈ Metric.closedBall (0 : ℝ) 1 ×s K I a,
 ||L.coordDeriv I a (jetAt k (variedField f η p.1) p.2)|| ≤ C I a)
 {x : Space d} {s : ℝ} (hs : s ∈ Metric.ball (0 : ℝ) 1)
 (I : DerivativeIndex d k) (a : Fin m) :
 ||firstVariationDensityTerm L (variedField f η s) η I a x|| ≤ C I a * ||ψ
by_cases hx : x ∈ K I a
· have hs' : s ∈ Metric.closedBall (0 : ℝ) 1 := Metric.ball_subset_closed
 have hcoeff := hC I a (s, x) ⟨hs', hx⟩
 calc
 ||firstVariationDensityTerm L (variedField f η s) η I a x||
 = ||L.coordDeriv I a (jetAt k (variedField f η s) x) * ψ I a x|| := b
 rw [hψ I a x]
 simp [firstVariationDensityTerm]

```



1.324.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATIONDENSITY.LEAN

```

 _ = ||L.coordDeriv I a (jetAt k (variedField f η s) x)|| * ||ψ I a x|| := by
 rw [norm_mul]
 _ ≤ C I a * ||ψ I a x|| := by
 exact mul_le_mul_of_nonneg_right hcoeff (norm_nonneg _)
 · have hψzero : ψ I a x = 0 := hKzero I a x hx
 rw [hψ I a x] at hψzero
 rw [hψ I a x]
 simp [firstVariationDensityTerm, hψzero]

```

```

private lemma norm_firstVariationDensity_le_bound
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 (ψ : DerivativeIndex d k → Fin m → Space d → ℝ)
 (hψ : ∀ I : DerivativeIndex d k, ∀ a : Fin m, ∀ x : Space d,
 ψ I a x = ∂^[I.1] (fun y => (η y) a) x)
 (C : DerivativeIndex d k → Fin m → ℝ)
 (K : DerivativeIndex d k → Fin m → Set (Space d))
 (hKzero : ∀ I : DerivativeIndex d k, ∀ a : Fin m, ∀ x, x ∉ K I a → ψ I a x = 0)
 (hC : ∀ I : DerivativeIndex d k, ∀ a : Fin m,
 ∀ p ∈ Metric.closedBall (0 : ℝ) 1 xs K I a,
 ||L.coordDeriv I a (jetAt k (variedField f η p.1) p.2)|| ≤ C I a)
 {x : Space d} {s : ℝ} (hs : s ∈ Metric.ball (0 : ℝ) 1) :
 ||firstVariationDensity L (variedField f η s) η x||
 ≤ ∑ I : DerivativeIndex d k, ∑ a : Fin m, C I a * ||ψ I a x|| := by
 have houter :
 ||∑ I : DerivativeIndex d k, ∑ a : Fin m,
 firstVariationDensityTerm L (variedField f η s) η I a x||
 ≤ ∑ I : DerivativeIndex d k, ||∑ a : Fin m,
 firstVariationDensityTerm L (variedField f η s) η I a x|| := by
 exact norm_sum_le _ _
 calc
 ||firstVariationDensity L (variedField f η s) η x||
 = ||∑ I : DerivativeIndex d k, ∑ a : Fin m,
 firstVariationDensityTerm L (variedField f η s) η I a x|| := by
 simp [firstVariationDensity]
 _ ≤ ∑ I : DerivativeIndex d k, ||∑ a : Fin m,
 firstVariationDensityTerm L (variedField f η s) η I a x|| := houter
 _ ≤ ∑ I : DerivativeIndex d k, ∑ a : Fin m,
 ||firstVariationDensityTerm L (variedField f η s) η I a x|| := by
 refine Finset.sum_le_sum ?_
 intro I _
 exact norm_sum_le _ _
 _ ≤ ∑ I : DerivativeIndex d k, ∑ a : Fin m, C I a * ||ψ I a x|| := by
 refine Finset.sum_le_sum ?_
 intro I _
 refine Finset.sum_le_sum ?_

```

```

 intro a _
 exact firstVariationDensityTerm_norm_le_of_coeff_bound
 L f η ψ h ψ C K hKzero hC hs I a

/-- A concrete dominated bound for the varied first-variation density, obtained
the jet-coordinate coefficient family in `(s, x)`. The compact support of the
of the admissible variation supplies the integrable majorant. -
/
lemma hasDominatedVariationDerivativeNear_of_continuous_coordDeriv
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace $\mathbb{R}$ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace $\mathbb{R}$ (Fin m)))
 (hcont : $\forall$ I : DerivativeIndex d k, $\forall$ a : Fin m,
 Continuous (fun p : $\mathbb{R} \times$ Space d =>
 L.coordDeriv I a (jetAt k (variedField f η p.1) p.2))) :
 HasDominatedVariationDerivativeNear L f η := by
 classical
 let ψ : DerivativeIndex d k → Fin m → Space d → $\mathbb{R}$:=
 fun I a x => $\partial^{[I.1]}$ (fun y => (η y) a) x
 have h ψ : $\forall$ I : DerivativeIndex d k, $\forall$ a : Fin m, IsTestFunction (ψ I a)
 intro I a
 simp [C] using iteratedDeriv_coord_isTestFunction η I a
 have h ψ _eval : $\forall$ I : DerivativeIndex d k, $\forall$ a : Fin m, $\forall$ x : Space d,
 ψ I a x = $\partial^{[I.1]}$ (fun y => (η y) a) x := by
 intro I a x
 rfl
 choose K hKcompact hKzero using fun I : DerivativeIndex d k => fun a : Fin m =>
 exists_compact_iff_hasCompactSupport.mpr (h ψ I a).supp
 let C : DerivativeIndex d k → Fin m → $\mathbb{R}$:= fun I a =>
 Classical.choose <|
 ((isCompact_closedBall (0 : $\mathbb{R}$) 1).prod (hKcompact I a)).exists_bound
 ((hcont I a).continuousOn)
 have hC : $\forall$ I : DerivativeIndex d k, $\forall$ a : Fin m,
 $\forall$ p ∈ Metric.closedBall (0 : $\mathbb{R}$) 1 $\times^s$ K I a,
 ||L.coordDeriv I a (jetAt k (variedField f η p.1) p.2)|| ≤ C I a := by
 intro I a
 simp [C] using Classical.choose_spec
 ((isCompact_closedBall (0 : $\mathbb{R}$) 1).prod (hKcompact I a)).exists_bound
 ((hcont I a).continuousOn)
 let bound : Space d → $\mathbb{R}$:=
 fun x => $\sum$ I : DerivativeIndex d k, $\sum$ a : Fin m, C I a * || ψ I a x||
 have hbound_int : Integrable bound volume := by
 refine integrable_finsetSum Finset.univ ?_
 intro I _
 refine integrable_finsetSum Finset.univ ?_
 intro a _
 exact ((h ψ I a).integrable (μ := volume)).norm.const_mul (C I a)

```

1.324.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATIONDENSITY.LEAN

```

have hfirst_int :
 Integrable (firstVariationDensity L f η) volume := by
 refine integrable_finsetSum Finset.univ ?_
 intro I _
 refine integrable_finsetSum Finset.univ ?_
 intro a _
 have hcoeff0 :
 Continuous (fun x : Space d => L.coordDeriv I a (jetAt k f x)) :=
 continuous_coordDeriv_at_zero L f η hcont I a
 exact firstVariationDensityTerm_integrable_of_continuous_coordDeriv L f η I a hcoeff0
have hbound :
 ∀ x : Space d, ∀ s ∈ Metric.ball (0 : ℝ) 1,
 ||firstVariationDensity L (variedField f η s) η x|| ≤ bound x := by
 intro x s hs
 simpa [bound] using norm_firstVariationDensity_le_bound
 L f η ψ hψ_eval C K hKzero hC hs
 refine {1, zero_lt_one, bound, hfirst_int.aestronglyMeasurable, hbound_int, ?_}
 exact Filter.Eventually.of_forall hbound

/-!
B. Pointwise linearization

-/

lemma hasDerivAt_actionDensity_variedField_zero_of_contDiff
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ
 ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)), ∀ x : Space d,
 HasDerivAt (fun s : ℝ => actionDensity L (variedField f η s) x)
 (firstVariationDensity L f η x) 0 := by
 intro η x
 have hjet :
 ∀ s : ℝ, jetAt k (variedField f η s) x =
 (jetAt k f x).lineMap (jetDirectionAt k η x) s := by
 intro s
 exact jetAt_add_smul k f η x s hf η.isTestFunction.contDiff
 have hfun :
 (fun s : ℝ => actionDensity L (variedField f η s) x) =
 fun s : ℝ => L ((jetAt k f x).lineMap (jetDirectionAt k η x) s) := by
 funext s
 rw [actionDensity_apply]
 simp [hjet s]
 rw [hfun]
 simpa [Lagrangian.fiberDerivative, firstVariationDensity, firstVariationDensityTerm,
 jetDirectionAt_coord] using
 (L.hasDerivAt_lineMap (jetAt k f x) (jetDirectionAt k η x))

```

```

lemma hasDerivAt_actionDensity_variedField_of_contDiff
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : C
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (x : Space d) (s
 HasDerivAt (fun r : ℝ => actionDensity L (variedField f η r) x)
 (firstVariationDensity L (variedField f η s) η x) s := by
 have hzero :
 HasDerivAt (fun t : ℝ => actionDensity L (variedField (variedField f
 (firstVariationDensity L (variedField f η s) η x) 0 := by
 exact hasDerivAt_actionDensity_variedField_zero_of_contDiff L (variedFi
 (variedField_contDiff f η s hf) η x
 have hshift :
 HasDerivAt (fun t : ℝ => actionDensity L (variedField f η (t + s)) x)
 (firstVariationDensity L (variedField f η s) η x) 0 := by
 simp [variedField_variedField, add_comm] using hzero
 let k : ℝ → ℝ := fun t => actionDensity L (variedField f η (t + s)) x
 have hk : HasDerivAt k (firstVariationDensity L (variedField f η s) η x)
 simp [k] using hshift
 simp [k, sub_eq_add_neg, add_assoc] using
 (HasDerivAt.comp_sub_const (f := k) (x := s) (a := s) hk)

lemma hasPointwiseLinearizedDensityFormula_of_contDiff
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : C
 HasPointwiseLinearizedDensityFormula L f := by
 intro η x
 exact (hasDerivAt_actionDensity_variedField_zero_of_contDiff L f hf η x).

lemma hasActionVariationDerivativeUnderIntegral_of_contDiff_of_dominatedVar
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : C
 (hdom : ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
 HasFiniteActionVariation L f η →
 HasDominatedVariationDerivativeNear L f η) :
 HasActionVariationDerivativeUnderIntegral L f := by
 intro η hη
 rcases hdom η hη with ⟨ε, hε, bound, hF'_meas, hbound_int, hbound⟩
 let F' : ℝ → Space d → ℝ := fun s => firstVariationDensity L (variedField
 have hmeas :
 ∀f s in nhds (0 : ℝ),
 AESTronglyMeasurable (actionDensity L (variedField f η s)) volume :
 exact Filter.Eventually.of_forall fun s => (hη s).aestronglyMeasurable
 have hderiv :
 ∀m x ∂volume, ∀ s ∈ Metric.ball (0 : ℝ) ε,
 HasDerivAt (fun r : ℝ => actionDensity L (variedField f η r) x) (F'
 filter_upwards with x
 intro s hs
 simp [F'] using hasDerivAt_actionDensity_variedField_of_contDiff L f h
 have hfinite0 : Integrable (actionDensity L (variedField f η 0)) := hη 0

```

1.325.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATION/INTEGRATIONBYPARTS.LEAN

```

have hF0_meas : AESTronglyMeasurable (F' 0) volume := by
 simp [F', firstVariationDensity, firstVariationDensityTerm, variedField] using
have hresult :=
 actionVariation_hasDerivAt_of_dominated_loc_of_deriv_le L f η hε hmeas hfinite0
 hF0_meas hbound hbound_int hderiv
have hvalue :
 (∫ x, F' 0 x)
 = ∫ x, deriv (fun s : ℝ => actionDensity L (variedField f η s) x) 0 := by
 apply integral_congr_ae
 filter_upwards with x
 simp [F', firstVariationDensity, firstVariationDensityTerm, variedField] using
 (hasPointwiseLinearizedDensityFormula_of_contDiff L f hf η x).symm
rw [hvalue] at hresult
exact hresult

lemma hasActionVariationDerivativeUnderIntegral_of_contDiff_of_continuous_coordDeriv
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ
 (hcontVar : ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
 ∀ I : DerivativeIndex d k, ∀ a : Fin m,
 Continuous (fun p : ℝ × Space d =>
 L.coordDeriv I a (jetAt k (variedField f η p.1) p.2))) :
 HasActionVariationDerivativeUnderIntegral L f := by
 refine hasActionVariationDerivativeUnderIntegral_of_contDiff_of_dominatedVariation
 intro η _hη
 exact hasDominatedVariationDerivativeNear_of_continuous_coordDeriv L f η (hcontVar

lemma hasActionVariationDerivativeUnderIntegral_of_contDiff_of_regular
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ
 (hcontVar : ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
 Lagrangian.ContinuousCoordDerivAlongFamily L (fun s : ℝ => variedField f η s))
 HasActionVariationDerivativeUnderIntegral L f := by
 apply hasActionVariationDerivativeUnderIntegral_of_contDiff_of_continuous_coordDeriv
 intro η I a
 exact hcontVar η I a

end Local
end ClassicalFieldTheory

```

## 1.325 PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Integrati

/-

Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.  
 Released under Apache 2.0 license as described in the file LICENSE.  
 Authors: Juan Jose Fernandez Morales

```

-/
module

public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Support
/-!
First variation integration by parts

i. Overview

This module contains the repeated integration-by-parts step needed for the
variation
formula, together with the termwise and summed packaged versions used later
Euler-Lagrange criterion.

ii. Key results

- `ClassicalFieldTheory.Local.integral_mul_iteratedDeriv_eq_sign`
- `ClassicalFieldTheory.Local.hasTermwiseIntegratedByPartsFormula_of_regula
- `ClassicalFieldTheory.Local.hasIntegratedByPartsFormula_of_termwise`

iii. Table of contents

- A. Repeated integration by parts
- B. Termwise formulas

iv. References

- J. Cortés and A. Haupt, *Lecture Notes on Mathematical Methods of Classic
 Chapter 5, Theorem 5.2.

-/

@[expose] public section

open MeasureTheory
open InnerProductSpace
open Physlib
open scoped BigOperators ContDiff

namespace ClassicalFieldTheory
namespace Local

/-!
A. Repeated integration by parts

-/

```

1.325.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATIONANDINTEGRATIONBYPARTS.LEAN

*-- The integration-by-parts step sending the linearized density to the Euler-Lagrange pairing. -/*

```
def HasIntegratedByPartsFormula (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop :=
 ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
 ∫ x, firstVariationDensity L f η x = firstVariationValue L f η
```

*-- Termwise integration-by-parts data for the linearized first-variation density. -/*

```
def HasTermwiseIntegratedByPartsFormula (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop :=
 ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
 ∀ I : DerivativeIndex d k, ∀ a : Fin m,
 Integrable (firstVariationDensityTerm L f η I a) ∧
 Integrable (fun x => eulerLagrangeTerm L I a f x * (η x) a) ∧
 (∫ x, firstVariationDensityTerm L f η I a x)
 = ∫ x, eulerLagrangeTerm L I a f x * (η x) a
```

```
private lemma integral_mul_space_deriv_eq_neg_deriv_mul (i : Fin d) {g h : Space d →
 (hg : ContDiff ℝ ∞ g) (hh : IsTestFunction h) :
 ∫ x, g x * ∂[i] h x = ∫ x, (-∂[i] g x) * h x := by
 rw [Space.deriv_eq_fderiv_fun, Space.deriv_eq_fderiv_fun]
 calc
 ∫ x, g x * fderiv ℝ h x (Space.basis i)
 = -∫ x, fderiv ℝ g x (Space.basis i) * h x := by
 rw [integral_mul_fderiv_eq_neg_fderiv_mul_of_integrable]
 · simp [Space.deriv_eq_fderiv_basis] using
 (IsTestFunction.integrable (μ := volume) <|
 IsTestFunction.mul_left (contDiff_space_deriv hg i) hh)
 · simp [Space.deriv_eq_fderiv_basis] using
 (IsTestFunction.integrable (μ := volume) <|
 IsTestFunction.mul_left hg (isTestFunction_space_deriv hh i))
 · exact IsTestFunction.integrable (μ := volume) <| IsTestFunction.mul_left
 · intro x hx
 exact Differentiable.differentiableAt (hg.differentiable (by simp))
 · intro x hx
 exact Differentiable.differentiableAt hh.differentiable
 _ = ∫ x, (-fderiv ℝ g x (Space.basis i)) * h x := by
 rw [← integral_neg]
 congr with x
 ring
```

```
private lemma integral_mul_iteratedDerivList_eq_sign (L : List (Fin d)) {g h : Space
 (hg : ContDiff ℝ ∞ g) (hh : IsTestFunction h) :
 ∫ x, g x * L.foldr (fun i f => ∂[i] f) h x =
```

```

 ∫ x, (((-1 : ℝ) ^ L.length) * L.foldr (fun i f => ∂[i] f) g x) * h x
induction L generalizing g h with
| nil =>
 simp
| cons i L ih =>
 have hdg : ContDiff ℝ ∞ (∂[i] g) := by
 exact contDiff_space_deriv hg i
 have hhL : IsTestFunction (L.foldr (fun j f => ∂[j] f) h) := by
 exact iteratedDerivList_isTestFunction L hh
 calc
 ∫ x, g x * ∂[i] (L.foldr (fun j f => ∂[j] f) h) x
 = ∫ x, (-∂[i] g x) * L.foldr (fun j f => ∂[j] f) h x := by
 exact integral_mul_space_deriv_eq_neg_deriv_mul i hg hhL
 _ = -∫ x, ∂[i] g x * L.foldr (fun j f => ∂[j] f) h x := by
 rw [← integral_neg]
 congr with x
 ring
 _ = -∫ x, (((-1 : ℝ) ^ L.length) *
 L.foldr (fun j f => ∂[j] f) (∂[i] g) x) * h x := by
 rw [ih hdg hh]
 _ = ∫ x, (((-1 : ℝ) ^ (L.length + 1)) *
 ∂[i] (L.foldr (fun j f => ∂[j] f) g) x) * h x := by
 rw [← integral_neg]
 congr with x
 rw [iteratedDerivList_commute_deriv L i hg, pow_succ, mul_assoc]
 ring

/-- Repeated integration by parts for iterated coordinate derivatives again
/
lemma integral_mul_iteratedDeriv_eq_sign {g h : Space d → ℝ} (I : MultiIndex)
 (hg : ContDiff ℝ ∞ g) (hh : IsTestFunction h) :
 ∫ x, g x * ∂^[I] h x =
 ∫ x, (((-1 : ℝ) ^ I.order) * ∂^[I] g x) * h x := by
 simp [Space.iteratedDeriv, Physlib.MultiIndex.length_toList] using
 integral_mul_iteratedDerivList_eq_sign I.toList hg hh

/-!
B. Termwise formulas

-/

/-- Concrete termwise integration by parts for the first-
variation density, assuming the jet
coordinate coefficient functions are smooth along the field. -
/
lemma hasTermwiseIntegratedByPartsFormula_of_contDiff_coordDeriv

```



1.325.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATIONINTEGRATIONBYPARTS.LEAN

```

(L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
(hcoeff : ∀ I : DerivativeIndex d k, ∀ a : Fin m,
 ContDiff ℝ ∞ (fun x => L.coordDeriv I a (jetAt k f x))) :
 HasTermwiseIntegratedByPartsFormula L f := by
 intro η I a
 let g : Space d → ℝ := fun x => L.coordDeriv I a (jetAt k f x)
 let h : Space d → ℝ := fun x => (η.toFun x) a
 have hg : ContDiff ℝ ∞ g := hcoeff I a
 have hh : IsTestFunction h := by
 simpa [h] using η.coord_euclidean a
 constructor
 · simpa [g, h, firstVariationDensityTerm, Space.iteratedDeriv] using
 firstVariationDensityTerm_integrable_of_continuous_coordDeriv L f η I a hg.con
 constructor
 · have htd : ContDiff ℝ ∞ (∂^[I.1] g) :=
 Space.iteratedDeriv_contDiff I.1 hg
 have htdSign : ContDiff ℝ ∞
 (fun x => ((-1 : ℝ) ^ I.1.order) * ∂^[I.1] g x) := by
 apply ContDiff.mul
 · fun_prop
 · exact htd
 exact
 (IsTestFunction.integrable (μ := volume) <|
 IsTestFunction.mul_left htdSign hh)
 · exact
 integral_mul_iteratedDeriv_eq_sign I.1 hg hh

lemma hasTermwiseIntegratedByPartsFormula_of_regular
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (hcoeff : Lagrangian.ContDiffCoordDerivAlongField L f) :
 HasTermwiseIntegratedByPartsFormula L f := by
 exact hasTermwiseIntegratedByPartsFormula_of_contDiff_coordDeriv L f hcoeff

private lemma integral_firstVariationDensity_eq_termwise_sum
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 (hterm : HasTermwiseIntegratedByPartsFormula L f) :
 ∫ x, firstVariationDensity L f η x =
 ∑ I : DerivativeIndex d k, ∑ a : Fin m,
 ∫ x, firstVariationDensityTerm L f η I a x := by
 calc
 ∫ x, firstVariationDensity L f η x
 = ∫ x, ∑ I : DerivativeIndex d k, ∑ a : Fin m,
 firstVariationDensityTerm L f η I a x := by
 rfl
 _ = ∑ I : DerivativeIndex d k, ∑ a : Fin m,

```

```

 ∫ x, firstVariationDensityTerm L f η I a x := by
 rw [MeasureTheory.integral_finsetSum Finset.univ
 (fun I _ => integrable_finsetSum Finset.univ (fun a _ => (hterm
 apply Finset.sum_congr rfl
 intro I _
 rw [MeasureTheory.integral_finsetSum Finset.univ (fun a _ => (hterm
private lemma integral_eulerLagrange_termwise_sum_eq_pairing
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 (hterm : HasTermwiseIntegratedByPartsFormula L f) :
 ∑ I : DerivativeIndex d k, ∑ a : Fin m, ∫ x, eulerLagrangeTerm L I a f x
 = firstVariationValue L f η := by
 calc
 ∑ I : DerivativeIndex d k, ∑ a : Fin m, ∫ x, eulerLagrangeTerm L I a f x
 = ∫ x, ∑ I : DerivativeIndex d k, ∑ a : Fin m,
 eulerLagrangeTerm L I a f x * (η x) a := by
 calc
 ∑ I : DerivativeIndex d k, ∑ a : Fin m,
 ∫ x, eulerLagrangeTerm L I a f x * (η x) a
 =
 ∑ I : DerivativeIndex d k,
 ∫ x, ∑ a : Fin m, eulerLagrangeTerm L I a f x * (η x) a :
 apply Finset.sum_congr rfl
 intro I _
 rw [← MeasureTheory.integral_finsetSum Finset.univ
 (fun a _ => (hterm η I a).2.1)]
 - =
 ∫ x, ∑ I : DerivativeIndex d k,
 ∑ a : Fin m, eulerLagrangeTerm L I a f x * (η x) a := by
 rw [← MeasureTheory.integral_finsetSum Finset.univ
 (fun I _ => integrable_finsetSum Finset.univ (fun a _ =>
_ = ∫ x, [eulerLagrangeOp L f x, η x]_ℝ := by
 apply integral_congr_ae
 filter_upwards with x
 rw [PiLp.inner_apply]
 simp_rw [eulerLagrangeOp_apply, eulerLagrangeComponent_apply]
 rw [Finset.sum_comm]
 apply Finset.sum_congr rfl
 intro a _
 rw [show [∑ I : DerivativeIndex d k, eulerLagrangeTerm L I a f x,
 (η.toFun x).ofLp a]_ℝ =
 (∑ I : DerivativeIndex d k, eulerLagrangeTerm L I a f x) *
 (η.toFun x).ofLp a by
 norm_num [inner, Inner.inner]
 ring]

```

1.326.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATION/REGULARITY.LEAN

```

 rw [← Finset.sum_mul]
 _ = firstVariationValue L f η := by
 rfl

lemma hasIntegratedByPartsFormula_of_termwise (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace $\mathbb{R}$ (Fin m))
 (hterm : HasTermwiseIntegratedByPartsFormula L f) :
 HasIntegratedByPartsFormula L f := by
 intro η
 calc
 $\int x, \text{firstVariationDensity } L f \eta x$
 = $\sum I : \text{DerivativeIndex } d k, \sum a : \text{Fin } m,$
 $\int x, \text{firstVariationDensityTerm } L f \eta I a x := by$
 exact integral_firstVariationDensity_eq_termwise_sum L f η hterm
 _ = $\sum I : \text{DerivativeIndex } d k, \sum a : \text{Fin } m,$
 $\int x, \text{eulerLagrangeTerm } L I a f x * (\eta x) a := by$
 apply Finset.sum_congr rfl
 intro I _
 apply Finset.sum_congr rfl
 intro a _
 exact (hterm η I a).2.2
 _ = firstVariationValue L f η := by
 exact integral_eulerLagrange_termwise_sum_eq_pairing L f η hterm

end Local
end ClassicalFieldTheory
```

## 1.326 PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Regularity

/-

Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Juan Jose Fernandez Morales

-/

module

public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Support

/-!

# First variation regularity

## i. Overview

This module contains regularity consequences used in the local Euler-Lagrange criterion: continuity

of the Euler-Lagrange operator from coefficient regularity, and the bridge regularity of the local Lagrangian to the packaged smooth-regularity statement.

## ii. Key results

```
- `ClassicalFieldTheory.Local.continuous_eulerLagrangeOp_of_regular`
- `ClassicalFieldTheory.Local.hasSmoothEulerLagrangeRegularity_of_contDiffC
```

## iii. Table of contents

```
- A. Continuity of the Euler-Lagrange operator
- B. Smooth regularity in coordinates
```

## iv. References

```
- J. Cortés and A. Haupt, *Lecture Notes on Mathematical Methods of Classic
 Chapter 5, Theorem 5.2.
```

```
-/
```

```
@[expose] public section
```

```
open MeasureTheory
open InnerProductSpace
open Physlib
open scoped BigOperators ContDiff
```

```
namespace ClassicalFieldTheory
namespace Local
```

```
/-!
```

```
A. Continuity of the Euler-Lagrange operator
```

```
-/
```

```
/-- The combined regularity package currently needed to derive the local Eu
Lagrange
```

```
criterion for a field `f`: smooth coefficient functions along `f`, and cont
corresponding coefficient families along every admissible varied-
field family based at `f`. -/
```

```
def HasEulerLagrangeRegularityAt (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop :=
 Lagrangian.ContDiffCoordDerivAlongField L f ∧
 ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
 Lagrangian.ContinuousCoordDerivAlongFamily L (fun s : ℝ => variedFiel
```

1.326.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATION/REGULARITY.LEAN

*-- A local Lagrangian has smooth Euler-Lagrange regularity if every smooth field satisfies the regularity package currently needed by the local first-variation proof. -*  
*/*

def HasSmoothEulerLagrangeRegularity (L : Lagrangian d m k) : Prop :=  
 ∀ f : Space d → EuclideanSpace ℝ (Fin m),  
 ContDiff ℝ ∞ f → HasEulerLagrangeRegularityAt L f

lemma continuous\_eulerLagrangeTerm\_of\_regular  
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))  
 (hcoeff : Lagrangian.ContDiffCoordDerivAlongField L f)  
 (I : DerivativeIndex d k) (a : Fin m) :  
 Continuous (eulerLagrangeTerm L I a f) := by  
 have htd : ContDiff ℝ ∞  
 (iteratedTotalDerivative k I.1 (L.coordDeriv I a) f) := by  
 exact  
 Space.iteratedDeriv\_contDiff I.1 (hcoeff I a)  
 have hsign : ContDiff ℝ ∞  
 (fun \_ : Space d => (-1 : ℝ) ^ I.1.order) := by  
 fun\_prop  
 exact (hsign.mul htd).continuous

lemma continuous\_eulerLagrangeComponent\_of\_regular  
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))  
 (hcoeff : Lagrangian.ContDiffCoordDerivAlongField L f)  
 (a : Fin m) :  
 Continuous (eulerLagrangeComponent L a f) := by  
 refine continuous\_finsetSum Finset.univ ?\_  
 intro I \_  
 exact continuous\_eulerLagrangeTerm\_of\_regular L f hcoeff I a

lemma continuous\_eulerLagrangeOp\_of\_regular  
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))  
 (hcoeff : Lagrangian.ContDiffCoordDerivAlongField L f) :  
 Continuous (eulerLagrangeOp L f) := by  
 let g : Space d → Fin m → ℝ := fun x a => eulerLagrangeComponent L a f x  
 have hg : Continuous g := continuous\_pi fun a =>  
 continuous\_eulerLagrangeComponent\_of\_regular L f hcoeff a  
 change Continuous (fun x : Space d => (WithLp.toLp 2 (g x) : EuclideanSpace ℝ (Fin m)))  
 fun\_prop

*/-!*  
*## B. Smooth regularity in coordinates*

*-/*

```

theorem hasSmoothEulerLagrangeRegularity_of_contDiffCoordDerivInCoordinates
 (L : Lagrangian d m k) (hcoord : Lagrangian.ContDiffCoordDerivInCoordinates L)
 HasSmoothEulerLagrangeRegularity L := by
 intro f hf
 refine ⟨Lagrangian.contDiffCoordDerivAlongField_of_inCoordinates L hcoord
 intro η
 exact Lagrangian.continuousCoordDerivAlongFamily_of_inCoordinates L hcoord
 (fun s : ℝ => variedField f η s)
 (continuous_jetBaseCoordinates_variedField k f η hf)

end Local
end ClassicalFieldTheory

```

### 1.327 PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Su

```

/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/
module

public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Basic
/-!
First variation support lemmas

i. Overview

This module collects the reusable support lemmas used by the analytic part
first-variation proof: basic identities for varied fields, iterated derivative
test functions, and continuity of the varied local-jet coordinate map.

ii. Key results

- `ClassicalFieldTheory.Local.variedField_zero`
- `ClassicalFieldTheory.Local.variedField_variedField`

iii. Table of contents

- A. Varied fields
- B. Iterated derivative support lemmas
- C. Varied local-jet coordinates

iv. References

```

1.327.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATION/SUPPORT.LEAN

- J. Cortés and A. Haupt, \*Lecture Notes on Mathematical Methods of Classical Physics  
Chapter 5, Theorem 5.2.

-/

@[expose] public section

open MeasureTheory  
open InnerProductSpace  
open Physlib  
open scoped BigOperators ContDiff

namespace ClassicalFieldTheory  
namespace Local

/-!

## A. Varied fields

-/

@[simp]

lemma variedField\_zero (f : Space d → EuclideanSpace ℝ (Fin m))  
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) :  
 variedField f η 0 = f := by  
 funext x  
 simp [variedField]

@[simp]

lemma variedField\_variedField (f : Space d → EuclideanSpace ℝ (Fin m))  
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))  
 (s t : ℝ) :  
 variedField (variedField f η s) η t = variedField f η (s + t) := by  
 funext x  
 simp [variedField, add\_assoc, add\_smul]

/-!

## B. Iterated derivative support lemmas

-/

lemma contDiff\_space\_deriv {g : Space d → ℝ} (hg : ContDiff ℝ ∞ g)  
 (i : Fin d) : ContDiff ℝ ∞ (∂[i] g) := by  
 have hfamilly : ContDiff ℝ ∞ (fun x : Space d => fun j : Fin d => ∂[j] g x) := by  
 simpa using (Space.deriv\_contDiff (n := ∞) hg)  
 exact (contDiff\_apply ℝ ℝ i).comp hfamilly

```

lemma isTestFunction_space_deriv {g : Space d → ℝ} (hg : IsTestFunction g)
 IsTestFunction (∂[i] g) := by
 simpa [Space.deriv_eq_fderiv_fun] using IsTestFunction.fderiv_apply hg (S

lemma iteratedDerivList_contDiff (L : List (Fin d)) {g : Space d → ℝ}
 (hg : ContDiff ℝ ∞ g) :
 ContDiff ℝ ∞ (L.foldr (fun i h => ∂[i] h) g) := by
 induction L generalizing g with
 | nil =>
 simpa using hg
 | cons i L ih =>
 have htail : ContDiff ℝ ∞ (L.foldr (fun j h => ∂[j] h) g) := ih hg
 exact contDiff_space_deriv htail i

lemma iteratedDerivList_isTestFunction (L : List (Fin d)) {g : Space d → ℝ}
 (hg : IsTestFunction g) :
 IsTestFunction (L.foldr (fun i h => ∂[i] h) g) := by
 induction L generalizing g with
 | nil =>
 simpa using hg
 | cons i L ih =>
 simp only [List.foldr]
 exact isTestFunction_space_deriv (ih hg) i

lemma iteratedDerivList_commute_deriv (L : List (Fin d)) (i : Fin d)
 {g : Space d → ℝ} (hg : ContDiff ℝ ∞ g) :
 L.foldr (fun j h => ∂[j] h) (∂[i] g) = ∂[i] (L.foldr (fun j h => ∂[j] h) g)
 induction L generalizing g with
 | nil =>
 rfl
 | cons j L ih =>
 simp only [List.foldr]
 rw [ih hg]
 rw [Space.deriv_commute (u := j) (v := i)]
 have h2 : ContDiff ℝ (2 : ℕ∞) (L.foldr (fun j h => ∂[j] h) g) := by
 exact (iteratedDerivList_contDiff L hg).of_le (by
 exact WithTop.coe_le_coe.mpr le_top)
 exact h2

lemma iteratedDeriv_coord_isTestFunction (η : AdmissibleVariation d (Euclid
 (I : DerivativeIndex d k) (a : Fin m) :
 IsTestFunction (fun x => ∂^[I.1] (fun y => (η y) a) x) := by
 simpa [Space.iteratedDeriv, Space.coord] using
 iteratedDerivList_isTestFunction I.1.toList (η.coord_euclidean a)

```



1.327.

*PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/FIRSTVARIATIONANDSUPPORT.LEAN*

```
lemma firstVariationDensityTerm_integrable_of_continuous_coordDeriv
 (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 (I : DerivativeIndex d k) (a : Fin m)
 (hcont : Continuous (fun x : Space d =>
 L.coordDeriv I a (jetAt k f x))) :
 Integrable (firstVariationDensityTerm L f η I a) := by
 let ψ : Space d → ℝ := fun x => ∂^[I.1] (fun y => (η y) a) x
 have hψ : IsTestFunction ψ := iteratedDeriv_coord_isTestFunction η I a
 have hψcont : Continuous ψ := hψ.contDiff.continuous
 have htermCont : Continuous (fun x => L.coordDeriv I a (jetAt k f x) * ψ x) :=
 hcont.mul hψcont
 have hsupp : HasCompactSupport (fun x => L.coordDeriv I a (jetAt k f x) * ψ x) :=
 HasCompactSupport.mul_left hψ.supp
 exact
 htermCont.integrable_of_hasCompactSupport hsupp

/-!
C. Varied local-jet coordinates

-/
```

```
lemma continuous_jetBaseCoordinates_variedField
 (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m))
 (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 (hf : ContDiff ℝ ∞ f) :
 Continuous
 (fun p : ℝ × Space d => (p.2, jetCoordinatesAt k (variedField f η p.1) p.2)) :
 have hfjet : Continuous (jetCoordinatesAt k f) :=
 (jetCoordinatesAt_contDiff k f hf).continuous
 have hηjet : Continuous (jetCoordinatesAt k η) :=
 (jetCoordinatesAt_contDiff k η η.isTestFunction.contDiff).continuous
 have hcoord :
 Continuous
 (fun p : ℝ × Space d =>
 jetCoordinatesAt k f p.2 + p.1 • jetCoordinatesAt k η p.2) := by
 exact (hfjet.comp continuous_snd).add (continuous_fst.smul (hηjet.comp continuous_snd))
 have hpair :
 Continuous
 (fun p : ℝ × Space d =>
 (p.2, jetCoordinatesAt k f p.2 + p.1 • jetCoordinatesAt k η p.2)) := by
 exact Continuous.prodMk continuous_snd hcoord
 have heq :
 (fun p : ℝ × Space d => (p.2, jetCoordinatesAt k (variedField f η p.1) p.2)) =
 (fun p : ℝ × Space d =>
 (p.2, jetCoordinatesAt k f p.2 + p.1 • jetCoordinatesAt k η p.2)) := by
```

```

 funext p
 congr 1
 exact
 (jetCoordinatesAt_add_smul k f η p.2 p.1 hf η.isTestFunction.contDiff
 rw [heq]
 exact hpair

end Local
end ClassicalFieldTheory

```

### 1.328 PhyslibAlpha/ClassicalFieldTheory/Local/JetPoint.lean

```

/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/
module

public import Physlib.SpaceAndTime.Space.Derivatives.DerivativeIndex
public import Physlib.SpaceAndTime.Space.Derivatives.Iterated
/-!
Coordinate-level jet points

i. Overview

This module introduces the coordinate-level point of the locally trivialized
jet bundle for
fields on `Space d` with values in `EuclideanSpace ℝ (Fin m)`.

This module only formalizes the local coordinate model $\text{Jet}^k(\Omega, \mathbb{R}^m) \approx \Omega \times$
attempt to define global jet bundles.

At this core stage, a jet point is represented by:

- its base point in `Space d`,
- and its jet coordinates indexed by derivative indices of order at most `k`

In particular, the zero-th order field value is not stored separately: it is
the coordinate.

ii. Key results

- `ClassicalFieldTheory.Local.JetCoordinates`

```

1.328.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/JETPOINT.LEAN 543

```
- `ClassicalFieldTheory.Local.JetPoint`
- `ClassicalFieldTheory.Local.jetCoordinatesAt`
- `ClassicalFieldTheory.Local.jetAt`
```

## iii. Table of contents

```
- A. Jet coordinates
- B. Jet points
- C. Jet points of fields
```

## iv. References

```
- J. Cortés and A. Haupt, *Lecture Notes on Mathematical Methods of Classical Physics
 arXiv:1612.03100v2, Chapter 5, Section 5.1.
```

-/

@[expose] public section

open Physlib

```
namespace ClassicalFieldTheory
namespace Local
```

/-!

## A. Jet coordinates

-/

/-- Coordinate data  $u^a_I$  for  $k$ -th order jet points.

For each derivative index  $I$ , these are the scalar components of the corresponding  $\text{EuclideanSpace } \mathbb{R} (\text{Fin } m)$  fiber coordinate. We keep this componentwise form because local Euler-Lagrange formulas are indexed by both  $I$  and  $a$ . -

/

```
abbrev JetCoordinates (d m k : $\mathbb{N}$) := DerivativeIndex d k $\rightarrow$ Fin m $\rightarrow$ $\mathbb{R}$
```

/-!

## B. Jet points

-/

/-- The coordinate-level points of the locally trivialized  $k$ -jet bundle for fields

$\text{Space } d \rightarrow \text{EuclideanSpace } \mathbb{R} (\text{Fin } m)$ . -/

```
structure JetPoint (d m k : $\mathbb{N}$) where
```

```

/-- The base point of the jet point. -/
base : Space d
/-- The jet coordinates indexed by all derivative indices of order at most k
/
fiber : JetCoordinates d m k

namespace JetPoint

variable {d m k : ℕ}

/-- The coordinate u^a_I of a jet point. -/
def coord (J : JetPoint d m k) (I : DerivativeIndex d k) (a : Fin m) : ℝ :=

/-- The zero-th order value encoded by a jet point. -/
def value (J : JetPoint d m k) : EuclideanSpace ℝ (Fin m) :=
 WithLp.toLp 2 fun a => J.coord 0 a

@[simp]
lemma value_apply (J : JetPoint d m k) (a : Fin m) :
 J.value a = J.coord 0 a := by
 simp [value]

@[ext]
lemma ext (J K : JetPoint d m k) (hbase : J.base = K.base)
 (hcoord : ∀ I a, J.coord I a = K.coord I a) :
 J = K := by
 cases J with
 | mk x u =>
 cases K with
 | mk y v =>
 dsimp [coord] at hbase hcoord
 subst y
 congr
 funext I a
 exact hcoord I a

/-- Build a jet point from a base point and its jet coordinates. -
/
def ofBaseCoordinates (x : Space d) (u : JetCoordinates d m k) : JetPoint d m k :=
 { base := x
 fiber := u }

/-- The base point and jet coordinates of a jet point. -/
def toBaseCoordinates (J : JetPoint d m k) : Space d × JetCoordinates d m k :=
 (J.base, J.fiber)

@[simp]

```

1.328.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/JETPOINT.LEAN 545

```
lemma ofBaseCoordinates_base (x : Space d) (u : JetCoordinates d m k) :
 (ofBaseCoordinates x u).base = x := rfl
```

```
@[simp]
lemma ofBaseCoordinates_fiber (x : Space d) (u : JetCoordinates d m k) :
 (ofBaseCoordinates x u).fiber = u := rfl
```

```
@[simp]
lemma ofBaseCoordinates_coord (x : Space d) (u : JetCoordinates d m k)
 (I : DerivativeIndex d k) (a : Fin m) :
 (ofBaseCoordinates x u).coord I a = u I a := rfl
```

```
@[simp]
lemma ofBaseCoordinates_base_fiber (J : JetPoint d m k) :
 ofBaseCoordinates J.base J.fiber = J := by
 apply JetPoint.ext
 · rfl
 · intro I a
 rfl
```

```
@[simp]
lemma toBaseCoordinates_ofBaseCoordinates (x : Space d) (u : JetCoordinates d m k) :
 (ofBaseCoordinates x u).toBaseCoordinates = (x, u) := rfl
```

end JetPoint

/-!

## C. Jet points of fields

-/

/-- The jet-coordinate function determined by a field `g`. -

/

```
noncomputable def jetCoordinatesAt (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m))
 (x : Space d) :
 JetCoordinates d m k :=
 fun I a => ∂^[I.1] (fun y => (g y) a) x
```

/-- The coordinate-level `k`-jet point of a field `f` at the point `x`. -

/

```
noncomputable def jetAt (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space
 JetPoint d m k where
 base := x
 fiber := jetCoordinatesAt k f x
```

```
@[simp]
```

```

lemma jetAt_base (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
 (jetAt k f x).base = x := rfl

@[simp]
lemma jetAt_value (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
 (jetAt k f x).value = f x := by
 ext a
 simp [JetPoint.value, JetPoint.coord, jetAt, jetCoordinatesAt, Space.iter]

@[simp]
lemma jetAt_coord (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
 (I : DerivativeIndex d k) (a : Fin m) :
 (jetAt k f x).coord I a = ∂^[I.1] (fun y => (f y) a) x := rfl

lemma jetAt_coord_zero (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
 (a : Fin m) :
 (jetAt k f x).coord 0 a = (f x) a := by
 simp [jetAt_coord]

@[simp]
lemma jetCoordinatesAt_eq (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m)) :
 (I : DerivativeIndex d k) (a : Fin m) :
 jetCoordinatesAt k g x I a = ∂^[I.1] (fun y => (g y) a) x := rfl

lemma jetCoordinatesAt_zero (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m)) :
 (x : Space d) (a : Fin m) :
 jetCoordinatesAt k g x 0 a = (g x) a := by
 simp [jetCoordinatesAt]

lemma jetAt_eq_ofBaseCoordinates (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m)) :
 (x : Space d) :
 jetAt k f x = JetPoint.ofBaseCoordinates x (jetCoordinatesAt k f x) :=
 apply JetPoint.ext
 · rfl
 · intro I a
 rfl

end Local
end ClassicalFieldTheory

```

### 1.329 PhyslibAlpha/ClassicalFieldTheory/Local/JetPointFiber.lean

/-

Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.

1.329.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/JETPOINTFIBER.557AN

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Juan Jose Fernandez Morales

-/

module

public import PhyslibAlpha.ClassicalFieldTheory.Local.JetPoint

/-!

# Fiber directions on jet points

## i. Overview

This module adds the affine fiber-direction structure on coordinate-level jet points.

At this stage, it introduces:

- fiber-coordinate data on jet points,
- affine translation and line maps in the jet fiber,
- and the jet-fiber direction determined by a field.

## ii. Key results

- `ClassicalFieldTheory.Local.JetFiberData`
- `ClassicalFieldTheory.Local.JetPoint.addFiber`
- `ClassicalFieldTheory.Local.JetPoint.lineMap`
- `ClassicalFieldTheory.Local.jetDirectionAt`

## iii. Table of contents

- A. Fiber-coordinate data
- B. Affine fiber structure on jet points
- C. Fiber directions determined by fields

## iv. References

-/

@[expose] public section

open Physlib

open scoped ContDiff

namespace ClassicalFieldTheory

namespace Local

/-!

```
A. Fiber-coordinate data
```

```
-/
```

```
/-- Fiber-coordinate data for jet points of order `k`. This records only the
not the base point in `Space d`. -/
```

```
structure JetFiberData (d m k : ℕ) where
```

```
 /-- The fiber coordinates indexed by all derivative indices of order at most k -/
 /
```

```
 coord : JetCoordinates d m k
```

```
namespace JetFiberData
```

```
variable {d m k : ℕ}
```

```
instance : CoeFun (JetFiberData d m k) (fun _ => DerivativeIndex d k → Fin m)
 coe V := V.coord
```

```
@[ext]
```

```
lemma ext (V W : JetFiberData d m k) (hcoord : ∀ I a, V.coord I a = W.coord I a) :
```

```
 cases V with
```

```
 | mk coordV =>
```

```
 cases W with
```

```
 | mk coordW =>
```

```
 have h : coordV = coordW := by
```

```
 funext I
```

```
 funext a
```

```
 exact hcoord I a
```

```
 cases h
```

```
 rfl
```

```
/-- The zero-th order component of a jet-fiber direction, corresponding to the base point -/
```

```
def value (V : JetFiberData d m k) : EuclideanSpace ℝ (Fin m) :=
 WithLp.toLp 2 fun a => V.coord 0 a
```

```
@[simp]
```

```
lemma value_apply (V : JetFiberData d m k) (a : Fin m) :
```

```
 V.value a = V.coord 0 a := by
```

```
 simp [value]
```

```
instance : Zero (JetFiberData d m k) where
```

```
 zero := { coord := fun _ _ => 0 }
```

```
instance : Add (JetFiberData d m k) where
```

```
 add V W := { coord := fun I a => V.coord I a + W.coord I a }
```



1.329.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/JETPOINTFIBER.lean

```
instance : SMul $\mathbb{R}$ (JetFiberData d m k) where
 smul c V := { coord := fun I a => c * V.coord I a }

@[simp]
lemma zero_coord (I : DerivativeIndex d k) (a : Fin m) :
 (0 : JetFiberData d m k).coord I a = 0 := rfl

@[simp]
lemma add_coord (V W : JetFiberData d m k) (I : DerivativeIndex d k) (a : Fin m) :
 (V + W).coord I a = V.coord I a + W.coord I a := rfl

@[simp]
lemma smul_coord (c : $\mathbb{R}$) (V : JetFiberData d m k) (I : DerivativeIndex d k) (a : Fin m) :
 (c • V).coord I a = c * V.coord I a := rfl

end JetFiberData

/-!
B. Affine fiber structure on jet points

-/

namespace JetPoint

variable {d m k : $\mathbb{N}$ }

/-- Translate a jet point by a fiber-direction increment. -
/
def addFiber (J : JetPoint d m k) (V : JetFiberData d m k) : JetPoint d m k where
 base := J.base
 fiber := fun I a => J.fiber I a + V.coord I a

/-- The affine line in jet space through `J` in the fiber direction `V`. -
/
def lineMap (J : JetPoint d m k) (V : JetFiberData d m k) (s : $\mathbb{R}$) : JetPoint d m k :
 J.addFiber (s • V)

@[simp]
lemma addFiber_base (J : JetPoint d m k) (V : JetFiberData d m k) :
 (J.addFiber V).base = J.base := rfl

@[simp]
lemma addFiber_value (J : JetPoint d m k) (V : JetFiberData d m k) :
 (J.addFiber V).value = J.value + V.value := by
 ext a
```

```

 rfl

@[simp]
lemma addFiber_coord (J : JetPoint d m k) (V : JetFiberData d m k)
 (I : DerivativeIndex d k) (a : Fin m) :
 (J.addFiber V).coord I a = J.coord I a + V.coord I a := rfl

@[simp]
lemma lineMap_base (J : JetPoint d m k) (V : JetFiberData d m k) (s : ℝ) :
 (J.lineMap V s).base = J.base := rfl

@[simp]
lemma lineMap_coord (J : JetPoint d m k) (V : JetFiberData d m k) (s : ℝ)
 (I : DerivativeIndex d k) (a : Fin m) :
 (J.lineMap V s).coord I a = J.coord I a + s * V.coord I a := rfl

end JetPoint

/-!
C. Fiber directions determined by fields

-/

/-- The jet-fiber direction determined by a field `g` at a point `x`. -
/
noncomputable def jetDirectionAt (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m))
 (x : Space d) :
 JetFiberData d m k where
 coord := jetCoordinatesAt k g x

@[simp]
lemma jetDirectionAt_coord (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m))
 (I : DerivativeIndex d k) (a : Fin m) :
 (jetDirectionAt k g x).coord I a = ∂^[I.1] (fun y => (g y) a) x := rfl

lemma jetDirectionAt_coord_zero (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m))
 (x : Space d) (a : Fin m) :
 (jetDirectionAt k g x).coord 0 a = (g x) a := by
 simp [jetDirectionAt_coord]

lemma jetCoordinatesAt_add_smul (k : ℕ) (f g : Space d → EuclideanSpace ℝ (Fin m))
 (x : Space d) (s : ℝ)
 (hf : ContDiff ℝ ∞ f) (hg : ContDiff ℝ ∞ g) :
 jetCoordinatesAt k (fun y => f y + s • g y) x =
 jetCoordinatesAt k f x + s • jetCoordinatesAt k g x := by
 funext I

```

1.329.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/JETPOINTFIBER.1551AN

```

funext a
have hfa : ContDiff $\mathbb{R} \infty$ (fun y => (f y) a) := by
 exact (contDiff_piLp_apply (□ := $\mathbb{R}$) (n := ∞) (p := 2)
 (E := fun _ : Fin m => $\mathbb{R}$) (i := a)).comp hf
have hga : ContDiff $\mathbb{R} \infty$ (fun y => (g y) a) := by
 exact (contDiff_piLp_apply (□ := $\mathbb{R}$) (n := ∞) (p := 2)
 (E := fun _ : Fin m => $\mathbb{R}$) (i := a)).comp hg
have hadd :
 (fun y => (f y + s • g y) a) = (fun y => (f y) a) + s • fun y => (g y) a := by
 funext y
 simp [smul_eq_mul]
have hsg : ContDiff $\mathbb{R} \infty$ (s • fun y => (g y) a) := by
 exact hga.const_smul s
simp [jetCoordinatesAt]
have hsum := congrFun (Space.iteratedDeriv_add I.1 hfa hsg) x
have hsmul := congrFun (Space.iteratedDeriv_const_smul I.1 s hga) x
calc
 $\partial^{[I.1]} ((fun y => (f y) a) + s • fun y => (g y) a) x$
 = $(\partial^{[I.1]} (fun y => (f y) a) + \partial^{[I.1]} (s • fun y => (g y) a)) x := by$
 simp using hsum
 _ = $\partial^{[I.1]} (fun y => (f y) a) x + s * \partial^{[I.1]} (fun y => (g y) a) x := by$
 simp [Pi.add_apply, Pi.smul_apply, smul_eq_mul, hsmul]

lemma jetAt_add_smul (k : $\mathbb{N}$) (f g : Space d → EuclideanSpace $\mathbb{R}$ (Fin m))
 (x : Space d) (s : $\mathbb{R}$)
 (hf : ContDiff $\mathbb{R} \infty$ f) (hg : ContDiff $\mathbb{R} \infty$ g) :
 jetAt k (fun y => f y + s • g y) x = (jetAt k f x).lineMap (jetDirectionAt k g x)
 apply JetPoint.ext
 · simp [JetPoint.lineMap]
 · intro I a
 have hfa : ContDiff $\mathbb{R} \infty$ (fun y => (f y) a) := by
 exact (contDiff_piLp_apply (□ := $\mathbb{R}$) (n := ∞) (p := 2)
 (E := fun _ : Fin m => $\mathbb{R}$) (i := a)).comp hf
 have hga : ContDiff $\mathbb{R} \infty$ (fun y => (g y) a) := by
 exact (contDiff_piLp_apply (□ := $\mathbb{R}$) (n := ∞) (p := 2)
 (E := fun _ : Fin m => $\mathbb{R}$) (i := a)).comp hg
 have hadd :
 (fun y => (f y + s • g y) a) = (fun y => (f y) a) + s • fun y => (g y) a :=
 funext y
 simp [smul_eq_mul]
 change $\partial^{[I.1]} (fun y => (f y + s • g y) a) x =$
 ((jetAt k f x).lineMap (jetDirectionAt k g x) s).coord I a
 rw [JetPoint.lineMap_coord, jetAt_coord, jetDirectionAt_coord]
 rw [hadd]
 have hsg : ContDiff $\mathbb{R} \infty$ (s • fun y => (g y) a) := by
 exact hga.const_smul s

```

```

have hsum := congrFun (Space.iteratedDeriv_add I.1 hfa hsg) x
have hsmul := congrFun (Space.iteratedDeriv_const_smul I.1 s hga) x
calc
 ∂^[I.1] ((fun y => (f y) a) + s • fun y => (g y) a) x
 = (∂^[I.1] (fun y => (f y) a) + ∂^[I.1] (s • fun y => (g y) a)) x :
 _ = ∂^[I.1] (fun y => (f y) a) x + s * ∂^[I.1] (fun y => (g y) a) x :
 simp [Pi.add_apply, Pi.smul_apply, smul_eq_mul, hsmul]

end Local
end ClassicalFieldTheory

```

### 1.330 PhyslibAlpha/ClassicalFieldTheory/Local/JetPointRegularity

```

/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/
module

public import PhyslibAlpha.ClassicalFieldTheory.Local.JetPointFiber
/-!
Regularity and support for jet-coordinate maps

i. Overview

This module collects the analytic facts about coordinate-
level jet maps that are needed later in
the local variational argument.

At this stage, it provides:

- smoothness of jet-coordinate maps of smooth fields,
- smoothness of the base-plus-coordinate jet map,
- and vanishing of jet coordinates outside topological support.

ii. Key results

- `ClassicalFieldTheory.Local.jetCoordinatesAt_contDiff`
- `ClassicalFieldTheory.Local.jetBaseCoordinates_contDiff`
- `ClassicalFieldTheory.Local.jetCoordinatesAt_eq_zero_of_notMem_tsupport`
- `ClassicalFieldTheory.Local.jetDirectionAt_eq_zero_of_notMem_tsupport`

iii. Table of contents

```

1.330.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/JETPOINTREGULARITY. LEAN

- A. Smoothness of jet-coordinate maps
- B. Support and vanishing lemmas

## iv. References

-/

@[expose] public section

open Physlib  
open scoped ContDiff Topology

namespace ClassicalFieldTheory  
namespace Local

/-!

## A. Smoothness of jet-coordinate maps

-/

/-- Smoothness of the jet-coordinate map of a smooth field. -

/

lemma jetCoordinatesAt\_contDiff (k :  $\mathbb{N}$ ) (g : Space d  $\rightarrow$  EuclideanSpace  $\mathbb{R}$  (Fin m))  
 (hg : ContDiff  $\mathbb{R} \infty$  g) :  
 ContDiff  $\mathbb{R} \infty$  (jetCoordinatesAt k g) := by  
 refine contDiff\_pi.2 ?\_  
 intro I  
 refine contDiff\_pi.2 ?\_  
 intro a  
 have hga : ContDiff  $\mathbb{R} \infty$  (fun y => (g y) a) := by  
 exact (contDiff\_piLp\_apply ( $\square := \mathbb{R}$ ) (n :=  $\infty$ ) (p := 2)  
 (E := fun \_ : Fin m =>  $\mathbb{R}$ ) (i := a)).comp hg  
 simpa [jetCoordinatesAt] using Space.iteratedDeriv\_contDiff I.1 hga

/-- Smoothness of the base-plus-coordinate jet map of a smooth field. -

/

lemma jetBaseCoordinates\_contDiff (k :  $\mathbb{N}$ ) (g : Space d  $\rightarrow$  EuclideanSpace  $\mathbb{R}$  (Fin m))  
 (hg : ContDiff  $\mathbb{R} \infty$  g) :  
 ContDiff  $\mathbb{R} \infty$  (fun x : Space d => (x, jetCoordinatesAt k g x)) := by  
 exact contDiff\_id.prodMk (jetCoordinatesAt\_contDiff k g hg)

/-!

## B. Support and vanishing lemmas

-/

```

private lemma notMem_tsupport_coord {g : Space d → EuclideanSpace ℝ (Fin m)}
 (hx : x ∉ tsupport g) (a : Fin m) :
 x ∉ tsupport (fun y => (g y) a) := by
 rw [notMem_tsupport_iff_eventuallyEq] at hx ⊢
 filter_upwards [hx] with y hy
 simp [hy]

/-- Outside the topological support of a field, all of its jet coordinates
/
lemma jetCoordinatesAt_eq_zero_of_notMem_tsupport (k : ℕ)
 (g : Space d → EuclideanSpace ℝ (Fin m)) {x : Space d}
 (hx : x ∉ tsupport g) :
 jetCoordinatesAt k g x = 0 := by
 funext I
 funext a
 exact Space.iteratedDeriv_eq_zero_of_notMem_tsupport I.1 (notMem_tsupport

/-- Outside the topological support of a field, the corresponding jet-
fiber direction vanishes. -/
lemma jetDirectionAt_eq_zero_of_notMem_tsupport (k : ℕ)
 (g : Space d → EuclideanSpace ℝ (Fin m)) {x : Space d}
 (hx : x ∉ tsupport g) :
 jetDirectionAt k g x = 0 := by
 ext I a
 exact Space.iteratedDeriv_eq_zero_of_notMem_tsupport I.1 (notMem_tsupport

end Local
end ClassicalFieldTheory

```

### 1.331 PhyslibAlpha/ClassicalFieldTheory/Local/Lagrangian.lean

```

/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/
module

public import PhyslibAlpha.ClassicalFieldTheory.Local.JetPointRegularity
public import PhyslibAlpha.ClassicalFieldTheory.Local.TotalDerivative
/-!
Local Lagrangians

```

1.331.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/LAGRANGIAN.LEAN55

## ## i. Overview

This module defines local Lagrangians of finite order for fields on  $\text{Space } d$  with values in  $\text{EuclideanSpace } \mathbb{R} \text{ (Fin } m)$ .

In the first local stage of the Classical Field Theory development, a local  $k$ -th order

Lagrangian is treated as a function on  $\text{JetPoint } d \ m \ k$ . This matches the local book level picture

$L : \text{Jet}^k(\Omega, \mathbb{R}^m) \rightarrow \mathbb{R}$  while postponing any stronger smoothness packaging until the structure on local jet-point data has been made explicit enough to support it naturally.

## ## ii. Key results

- `ClassicalFieldTheory.Local.Lagrangian` : local  $k$ -th order Lagrangians.
- `ClassicalFieldTheory.Local.Lagrangian.coordDeriv` : coordinate derivatives with respect to jet coordinates  $u^a_I$ .
- `ClassicalFieldTheory.Local.Lagrangian.SmoothInCoordinates` : the combined public package for smooth local lagrangians in explicit jet coordinates.
- `ClassicalFieldTheory.Local.Lagrangian.alongField` : evaluate a local Lagrangian along a field.

## ## iii. Table of contents

- A. Local Lagrangians
- B. Regularity packages
- C. Evaluation along a field

## ## iv. References

- J. Cortés and A. Haupt, \*Lecture Notes on Mathematical Methods of Classical Physics Chapter 5.

-/

@[expose] public section

open Physlib

open scoped BigOperators ContDiff

namespace ClassicalFieldTheory

namespace Local

/-!

## ## A. Local Lagrangians

```

-/

/-- A local `k`-th order Lagrangian for fields `Space d → EuclideanSpace ℝ`
/
structure Lagrangian (d m k : ℕ) where
 /-- The underlying jet-dependent function. -/
 toFun : JetPoint d m k → ℝ
 /-- The derivatives of the Lagrangian with respect to the jet coordinates
 /
 coordDeriv : DerivativeIndex d k → Fin m → JetPoint d m k → ℝ
 /-- The first derivative of the Lagrangian along affine lines in the jet-
 fiber coordinates is
 given by the pairing with the coordinate derivatives. -
 /
 hasDerivAlongFiber :
 ∀ J : JetPoint d m k, ∀ V : JetFiberData d m k,
 HasDerivAt (fun s : ℝ => toFun (J.lineMap V s))
 (∑ I : DerivativeIndex d k, ∑ a : Fin m, coordDeriv I a J * V.coord I a)

namespace Lagrangian

variable {d m k : ℕ}

instance : CoeFun (Lagrangian d m k) (fun _ => JetPoint d m k → ℝ) where
 coe L := L.toFun

/-- The derivative of `L` along the fiber-jet direction `V` based at `J`. -
/
noncomputable def fiberDerivative (L : Lagrangian d m k) (J : JetPoint d m k)
 (V : JetFiberData d m k) : ℝ :=
 ∑ I : DerivativeIndex d k, ∑ a : Fin m, L.coordDeriv I a J * V.coord I a

lemma hasDerivAt_lineMap (L : Lagrangian d m k) (J : JetPoint d m k) (V : JetFiberData d m k) :
 HasDerivAt (fun s : ℝ => L (J.lineMap V s)) (L.fiberDerivative J V) 0 :
 simp [fiberDerivative] using L.hasDerivAlongFiber J V

/-!
B. Regularity packages

-/

/-- Continuity of a local lagrangian in the explicit local jet coordinates
/
def ContinuousInCoordinates (L : Lagrangian d m k) : Prop :=
 Continuous (fun p : Space d × JetCoordinates d m k =>
 L (JetPoint.ofBaseCoordinates p.1 p.2))

```



1.331.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/LAGRANGIAN. LEAN57

```

/-- Intrinsic smoothness of the jet-coordinate derivatives of a local Lagrangian in
local jet coordinates `(x, u^a_I)` . -/
def ContDiffCoordDerivInCoordinates (L : Lagrangian d m k) : Prop :=
 ∀ I : DerivativeIndex d k, ∀ a : Fin m,
 ContDiff ℝ ∞
 (fun p : Space d × JetCoordinates d m k =>
 L.coordDeriv I a (JetPoint.ofBaseCoordinates p.1 p.2))

/-- Public regularity package for a smooth local lagrangian in explicit local jet co-
continuity of the lagrangian itself together with smoothness of all jet-
coordinate derivatives. -/
def SmoothInCoordinates (L : Lagrangian d m k) : Prop :=
 L.ContinuousInCoordinates ∧ L.ContDiffCoordDerivInCoordinates

/-- Smoothness of all coefficient functions `x ↦ ∂L/∂u_I^a (j^k f(x))` along a field
/
def ContDiffCoordDerivAlongField (L : Lagrangian d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop :=
 ∀ I : DerivativeIndex d k, ∀ a : Fin m,
 ContDiff ℝ ∞ (fun x => L.coordDeriv I a (jetAt k f x))

/-- Continuity of the coefficient family `(s, x) ↦ ∂L/∂u_I^a (j^k(F s)(x))` along a
one-parameter family of fields `F : ℝ → Space d → EuclideanSpace ℝ (Fin m)` . -
/
def ContinuousCoordDerivAlongFamily (L : Lagrangian d m k)
 (F : ℝ → Space d → EuclideanSpace ℝ (Fin m)) : Prop :=
 ∀ I : DerivativeIndex d k, ∀ a : Fin m,
 Continuous (fun p : ℝ × Space d => L.coordDeriv I a (jetAt k (F p.1) p.2))

lemma continuousAlongField_of_inCoordinates (L : Lagrangian d m k)
 (hcont : ContinuousInCoordinates L) (f : Space d → EuclideanSpace ℝ (Fin m))
 (hf : ContDiff ℝ ∞ f) :
 Continuous (fun x : Space d => L (jetAt k f x)) := by
 exact
 hcont.comp (jetBaseCoordinates_contDiff k f hf).continuous

lemma contDiffCoordDerivAlongField_of_inCoordinates (L : Lagrangian d m k)
 (hcoord : ContDiffCoordDerivInCoordinates L) (f : Space d → EuclideanSpace ℝ (Fin m))
 (hf : ContDiff ℝ ∞ f) :
 ContDiffCoordDerivAlongField L f := by
 intro I a
 have hjet : ContDiff ℝ ∞ (fun x : Space d => (x, jetCoordinatesAt k f x)) :=
 jetBaseCoordinates_contDiff k f hf
 have hcomp :=
 (hcoord I a).comp hjet

```

```

exact hcomp

lemma continuousCoordDerivAlongFamily_of_inCoordinates (L : Lagrangian d m
 (hcoord : ContDiffCoordDerivInCoordinates L)
 (F : $\mathbb{R} \rightarrow \text{Space } d \rightarrow \text{EuclideanSpace } \mathbb{R} \text{ (Fin } m)$)
 (hjet : Continuous (fun p : $\mathbb{R} \times \text{Space } d \Rightarrow$ (p.2, jetCoordinatesAt k (F
 ContinuousCoordDerivAlongFamily L F := by
intro I a
have hbase :
 Continuous
 (fun p : $\mathbb{R} \times \text{Space } d \Rightarrow$
 L.coordDeriv I a (JetPoint.ofBaseCoordinates p.2 (jetCoordinatesA
 exact (hcoord I a).continuous.comp hjet
 simp only [jetAt_eq_ofBaseCoordinates] using hbase

/-!
C. Evaluation along a field

-/

/-- Evaluate a local Lagrangian along the `k`-jets of a field. -
/
noncomputable def alongField (L : Lagrangian d m k)
 (f : $\text{Space } d \rightarrow \text{EuclideanSpace } \mathbb{R} \text{ (Fin } m)$) (x : $\text{Space } d \rightarrow \mathbb{R}$:=
 evalOnJet k L f

@[simp]
lemma alongField_apply (L : Lagrangian d m k)
 (f : $\text{Space } d \rightarrow \text{EuclideanSpace } \mathbb{R} \text{ (Fin } m)$) (x : $\text{Space } d$) :
 L.alongField f x = L (jetAt k f x) := rfl

end Lagrangian
end Local
end ClassicalFieldTheory

```

### 1.332 PhyslibAlpha/ClassicalFieldTheory/Local/TotalDerivative.1

```

/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/
module

```

1.332.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/TOTALDERIVATIVE551LEAN

```
public import PhyslibAlpha.ClassicalFieldTheory.Local.JetPoint
```

```
/-!
```

```
Total derivatives on local jet-dependent functions
```

```
i. Overview
```

This module defines total derivatives of local jet-dependent functions by differentiation along jets of fields.

For the first local stage, this keeps the operator close to the use made in the Euler-Lagrange

formula, without yet introducing a separate coordinate-level derivative calculus on

```
ii. Key results
```

- `ClassicalFieldTheory.Local.evalOnJet` : evaluate a jet-dependent function along a field.

- `ClassicalFieldTheory.Local.totalDerivative` : total derivative in one coordinate

- `ClassicalFieldTheory.Local.iteratedTotalDerivative` : iterated total derivative in multi-index.

```
iii. Table of contents
```

- A. Evaluation on jets
- B. Total derivatives

```
iv. References
```

```
-/
```

```
@[expose] public section
```

```
open Physlib
```

```
namespace ClassicalFieldTheory
```

```
namespace Local
```

```
/-!
```

```
A. Evaluation on jets
```

```
-/
```

```
/-- Evaluate a local jet-dependent function along the `k`-jet of a field. -/
```

```
noncomputable def evalOnJet (k : ℕ) (F : JetPoint d m k → ℝ)
 (f : Space d → EuclideanSpace ℝ (Fin m)) :
```

```

 Space d → ℝ :=
 fun x => F (jetAt k f x)

/-!
B. Total derivatives

-/

/-- The total derivative of a jet-dependent function in the coordinate direction i. -
/
noncomputable def totalDerivative (k : ℕ) (i : Fin d) (F : JetPoint d m k → ℝ)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ :=
 ∂[i] (evalOnJet k F f)

/-- The iterated total derivative indexed by a multi-index. -
/
noncomputable def iteratedTotalDerivative (k : ℕ) (I : MultiIndex d) (F : JetPoint d m k → ℝ)
 (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ :=
 ∂^[I] (evalOnJet k F f)

@[simp]
lemma evalOnJet_apply (k : ℕ) (F : JetPoint d m k → ℝ)
 (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
 evalOnJet k F f x = F (jetAt k f x) := rfl

@[simp]
lemma totalDerivative_eq (k : ℕ) (i : Fin d) (F : JetPoint d m k → ℝ)
 (f : Space d → EuclideanSpace ℝ (Fin m)) :
 totalDerivative k i F f = ∂[i] (fun x => F (jetAt k f x)) := rfl

@[simp]
lemma iteratedTotalDerivative_eq (k : ℕ) (I : MultiIndex d) (F : JetPoint d m k → ℝ)
 (f : Space d → EuclideanSpace ℝ (Fin m)) :
 iteratedTotalDerivative k I F f = ∂^[I] (fun x => F (jetAt k f x)) := rfl

lemma iteratedTotalDerivative_zero (k : ℕ) (F : JetPoint d m k → ℝ)
 (f : Space d → EuclideanSpace ℝ (Fin m)) :
 iteratedTotalDerivative k (0 : MultiIndex d) F f = evalOnJet k F f := by
 simp [iteratedTotalDerivative]

lemma iteratedTotalDerivative_single (k : ℕ) (i : Fin d) (F : JetPoint d m k → ℝ)
 (f : Space d → EuclideanSpace ℝ (Fin m)) :
 iteratedTotalDerivative k (Physlib.MultiIndex.increment 0 i) F f =
 totalDerivative k i F f := by
 simp [iteratedTotalDerivative, totalDerivative]

```

1.333.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/TOTALDIVERGENCE.lean

```
end Local
end ClassicalFieldTheory
```

### 1.333 PhyslibAlpha/ClassicalFieldTheory/Local/TotalDivergence.lean

```
/-
```

Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Juan Jose Fernandez Morales

```
-/
```

```
module
```

```
public import PhyslibAlpha.ClassicalFieldTheory.Local.FirstOrder
```

```
/-!
```

```
Total divergences in local classical field theory
```

```
i. Overview
```

This module introduces the local coordinate API for total-divergence lagrangians.

If  $B_i$  is a jet-dependent current of order  $k$ , its total divergence

$\sum_i D_i B_i$

is naturally a lagrangian density of order  $k + 1$ . In the current Alpha stack, local coordinates carry their jet-coordinate derivatives as explicit data, so this module does not try to compute that lagrangian automatically from the current. Instead, it packages the data needed to prove equivalence results:

- a local lagrangian of order  $k + 1$ ,
- a current whose total divergence agrees with the lagrangian density along every field,
- and the Euler-Lagrange-triviality property of that lagrangian.

This keeps the API faithful to the standard total-divergence principle while avoiding the need for symbolic calculus for coordinate derivatives of total derivatives.

```
ii. Key results
```

- `ClassicalFieldTheory.Local.HasTotalDivergenceDensity`
- `ClassicalFieldTheory.Local.IsEulerLagrangeTrivial`
- `ClassicalFieldTheory.Local.TotalDivergence`

```
iii. Table of contents
```

- A. Total-divergence predicates
- B. Packaged total-divergence lagrangians

## iv. References

- J. Cortés and A. Haupt, \*Lecture Notes on Mathematical Methods of Classical Mechanics\*, arXiv:1612.03100v2, Chapter 5.

-/

@[expose] public section

open Physlib  
open scoped BigOperators

namespace ClassicalFieldTheory  
namespace Local

/-!

## A. Total-divergence predicates

-/

/-- A lagrangian density has total-divergence form if, along every field, it is the total derivatives of a jet-dependent current. A current of order `k` gives rise to a current of order `k + 1`. -/

```
def HasTotalDivergenceDensity (L : Lagrangian d m (k + 1))
 (current : Fin d → JetPoint d m k → ℝ) : Prop :=
 ∀ f : Space d → EuclideanSpace ℝ (Fin m),
 actionDensity L f = fun x => ∑ i : Fin d, totalDerivative k i (current
```

/-- A local lagrangian is Euler-Lagrange-trivial if its Euler-Lagrange operator vanishes along every field. -/

```
def IsEulerLagrangeTrivial (L : Lagrangian d m k) : Prop :=
 ∀ f : Space d → EuclideanSpace ℝ (Fin m), eulerLagrangeOp L f = 0
```

/-!

## B. Packaged total-divergence lagrangians

-/

/-- A packaged local total divergence.

The current has order `k`, while the associated lagrangian has order `k + 1`.

1.333.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/TOTALDIVERGENCE561.EAN

coordinate expression  $\text{'L} = \sum_i D_i B_i \text{'}$ . The Euler-Lagrange-triviality field records the variationally trivial direction needed by later equivalence results. -  
/

```
structure TotalDivergence (d m k : ℕ) where
 /-- The lagrangian represented by the total divergence of the current. -
 /
 lagrangian : Lagrangian d m (k + 1)
 /-- The current whose total divergence gives the lagrangian density. -
 /
 current : Fin d → JetPoint d m k → ℝ
 /-- The density agrees with the total divergence of the current along every field.
 /
 hasTotalDivergenceDensity : HasTotalDivergenceDensity lagrangian current
 /-- The associated lagrangian is recorded as Euler-Lagrange-trivial. -/
 isEulerLagrangeTrivial : IsEulerLagrangeTrivial lagrangian
```

namespace TotalDivergence

variable {d m k : ℕ}

```
instance : CoeFun (TotalDivergence d m k) (fun _ => Lagrangian d m (k + 1)) where
 coe T := T.lagrangian
```

@[simp]

```
lemma actionDensity_eq (T : TotalDivergence d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) :
 actionDensity T.lagrangian f =
 fun x => ∑ i : Fin d, totalDerivative k i (T.current i) f x :=
 T.hasTotalDivergenceDensity f
```

```
lemma eulerLagrangeOp_eq_zero (T : TotalDivergence d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) :
 eulerLagrangeOp T.lagrangian f = 0 :=
 T.isEulerLagrangeTrivial f
```

```
lemma isCritical (T : TotalDivergence d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m))
 (hTf : IsAdmissibleForAction T.lagrangian f)
 (hT : Lagrangian.SmoothInCoordinates T.lagrangian) :
 IsCritical T.lagrangian f := by
 exact
 (isCritical_iff_eulerLagrange_zero_of_admissibleForAction_and_smoothInCoordinates
 T.lagrangian f hTf hT).2 (T.eulerLagrangeOp_eq_zero f)
```

```

end TotalDivergence

end Local
end ClassicalFieldTheory

```

### 1.334 PhyslibAlpha/ClassicalFieldTheory/Local/TotalDivergenceEq

```

/-
Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Juan Jose Fernandez Morales
-/
module

public import PhyslibAlpha.ClassicalFieldTheory.Local.TotalDivergence
/-!
Lagrangian equivalence up to total divergences

i. Overview

This module adds the local coordinate API for lagrangians that differ by a

In the current Alpha stack, local lagrangians carry their jet-
coordinate derivatives explicitly.
Consequently, this module does not construct the modified lagrangian by adding
divergence syntactically. Instead, it packages the data needed by later equ

- two local lagrangians of the same order,
- a total-divergence lagrangian witnessing the density difference,
- and equality of the corresponding Euler-Lagrange operators.

This is the field-theory analogue of the classical fact that adding a total
divergence does not change the variational equations, while keeping the cur
which facts are data and which facts are proved.

ii. Key results

- `ClassicalFieldTheory.Local.HasTotalDivergenceDifference`
- `ClassicalFieldTheory.Local.IsEulerLagrangeEquivalent`
- `ClassicalFieldTheory.Local.IsEulerLagrangeEquivalent.eulerLagrangeOp_eq`
- `ClassicalFieldTheory.Local.IsEulerLagrangeEquivalent.isCritical_iff`
- `ClassicalFieldTheory.Local.TotalDivergenceEquivalence`
- `ClassicalFieldTheory.Local.TotalDivergenceEquivalence.isCritical_iff`

```



1.334.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/TOTALDIVERGENCEEQUIVALENCE.LEAN

## iii. Table of contents

- A. Equivalence predicates
- B. Euler-Lagrange equivalence API
- C. Packaged total-divergence equivalences

## iv. References

- J. Cortés and A. Haupt, \*Lecture Notes on Mathematical Methods of Classical Physics  
arXiv:1612.03100v2, Chapter 5.

-/

@[expose] public section

namespace ClassicalFieldTheory  
namespace Local

/-!

## A. Equivalence predicates

-/

/-- A lagrangian `target` differs from `source` by the density of a packaged total d

The total divergence has current order `k`, hence its associated lagrangian and both  
lagrangians have order `k + 1`. -/

```
def HasTotalDivergenceDifference (source target : Lagrangian d m (k + 1))
 (T : TotalDivergence d m k) : Prop :=
 ∀ f : Space d → EuclideanSpace ℝ (Fin m),
 actionDensity target f =
 fun x => actionDensity source f x + actionDensity T.lagrangian f x
```

/-- Two local lagrangians are Euler-Lagrange equivalent when they determine the same  
Euler-Lagrange operator along every field. -/

```
def IsEulerLagrangeEquivalent (source target : Lagrangian d m k) : Prop :=
 ∀ f : Space d → EuclideanSpace ℝ (Fin m),
 eulerLagrangeOp target f = eulerLagrangeOp source f
```

/-!

## B. Euler-Lagrange equivalence API

-/

namespace IsEulerLagrangeEquivalent

```

variable {d m k : ℕ}

lemma refl (L : Lagrangian d m k) : IsEulerLagrangeEquivalent L L := by
 intro f
 rfl

lemma symm {L1 L2 : Lagrangian d m k} (h : IsEulerLagrangeEquivalent L1 L2) :
 IsEulerLagrangeEquivalent L2 L1 := by
 intro f
 exact (h f).symm

lemma trans {L1 L2 L3 : Lagrangian d m k}
 (h12 : IsEulerLagrangeEquivalent L1 L2) (h23 : IsEulerLagrangeEquivalent L2 L3) :
 IsEulerLagrangeEquivalent L1 L3 := by
 intro f
 exact (h23 f).trans (h12 f)

lemma eulerLagrangeOp_eq {source target : Lagrangian d m k}
 (h : IsEulerLagrangeEquivalent source target)
 (f : Space d → EuclideanSpace ℝ (Fin m)) :
 eulerLagrangeOp target f = eulerLagrangeOp source f :=
 h f

lemma eulerLagrangeOp_eq_zero_iff {source target : Lagrangian d m k}
 (h : IsEulerLagrangeEquivalent source target)
 (f : Space d → EuclideanSpace ℝ (Fin m)) :
 eulerLagrangeOp target f = 0 ↔ eulerLagrangeOp source f = 0 := by
 rw [h.eulerLagrangeOp_eq f]

lemma isCritical_iff {source target : Lagrangian d m k}
 (h : IsEulerLagrangeEquivalent source target)
 (f : Space d → EuclideanSpace ℝ (Fin m))
 (hsourcef : IsAdmissibleForAction source f)
 (htargetf : IsAdmissibleForAction target f)
 (hsource : Lagrangian.SmoothInCoordinates source)
 (htarget : Lagrangian.SmoothInCoordinates target) :
 IsCritical target f ↔ IsCritical source f := by
 rw [isCritical_iff_eulerLagrange_zero_of_admissibleForAction_and_smoothInCoordinates,
 target f htargetf htarget,
 isCritical_iff_eulerLagrange_zero_of_admissibleForAction_and_smoothInCoordinates,
 source f hsourcef hsource,
 h f]

lemma isCritical_target_of_source {source target : Lagrangian d m k}
 (h : IsEulerLagrangeEquivalent source target)
 (f : Space d → EuclideanSpace ℝ (Fin m))

```

1.334.

PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/TOTALDIVERGENCEEQUIVALENCE.LEAN

```
(hsourcef : IsAdmissibleForAction source f)
(htargetf : IsAdmissibleForAction target f)
(hsource : Lagrangian.SmoothInCoordinates source)
(htarget : Lagrangian.SmoothInCoordinates target)
(hcrit : IsCritical source f) :
 IsCritical target f :=
(h.isCritical_iff f hsourcef htargetf hsource htarget).2 hcrit

lemma isCritical_source_of_target {source target : Lagrangian d m k}
 (h : IsEulerLagrangeEquivalent source target)
 (f : Space d → EuclideanSpace ℝ (Fin m))
 (hsourcef : IsAdmissibleForAction source f)
 (htargetf : IsAdmissibleForAction target f)
 (hsource : Lagrangian.SmoothInCoordinates source)
 (htarget : Lagrangian.SmoothInCoordinates target)
 (hcrit : IsCritical target f) :
 IsCritical source f :=
(h.isCritical_iff f hsourcef htargetf hsource htarget).1 hcrit

end IsEulerLagrangeEquivalent

/-!
C. Packaged total-divergence equivalences

-/

/-- A packaged equivalence between two local lagrangians whose densities differ by a
divergence.

The field `sameEulerLagrangeOp` records the variational consequence in the current A
will become a theorem once the stack has enough symbolic calculus for coordinate der
total divergences. -/
structure TotalDivergenceEquivalence (d m k : ℕ) where
 /-- The original local lagrangian. -/
 source : Lagrangian d m (k + 1)
 /-- The modified local lagrangian. -/
 target : Lagrangian d m (k + 1)
 /-- The total divergence giving the density difference. -
 /
 divergence : TotalDivergence d m k
 /-- The target density is the source density plus the total-
divergence density. -/
 hasTotalDivergenceDifference :
 HasTotalDivergenceDifference source target divergence
 /-- The two lagrangians have the same Euler-Lagrange operator. -
 /
```

```

 sameEulerLagrangeOp : IsEulerLagrangeEquivalent source target

namespace TotalDivergenceEquivalence

variable {d m k : ℕ}

@[simp]
lemma actionDensity_eq (E : TotalDivergenceEquivalence d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) :
 actionDensity E.target f =
 fun x => actionDensity E.source f x + actionDensity E.divergence.lagrange
 E.hasTotalDivergenceDifference f

lemma eulerLagrangeOp_eq (E : TotalDivergenceEquivalence d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) :
 eulerLagrangeOp E.target f = eulerLagrangeOp E.source f :=
 E.sameEulerLagrangeOp f

lemma eulerLagrangeOp_eq_zero_iff (E : TotalDivergenceEquivalence d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m)) :
 eulerLagrangeOp E.target f = 0 ↔ eulerLagrangeOp E.source f = 0 :=
 E.sameEulerLagrangeOp.eulerLagrangeOp_eq_zero_iff f

lemma isCritical_iff (E : TotalDivergenceEquivalence d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m))
 (hsourcef : IsAdmissibleForAction E.source f)
 (htargetf : IsAdmissibleForAction E.target f)
 (hsource : Lagrangian.SmoothInCoordinates E.source)
 (htarget : Lagrangian.SmoothInCoordinates E.target) :
 IsCritical E.target f ↔ IsCritical E.source f := by
 exact E.sameEulerLagrangeOp.isCritical_iff f hsourcef htargetf hsource ht

lemma isCritical_target_of_source (E : TotalDivergenceEquivalence d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m))
 (hsourcef : IsAdmissibleForAction E.source f)
 (htargetf : IsAdmissibleForAction E.target f)
 (hsource : Lagrangian.SmoothInCoordinates E.source)
 (htarget : Lagrangian.SmoothInCoordinates E.target)
 (hcrit : IsCritical E.source f) :
 IsCritical E.target f :=
 E.sameEulerLagrangeOp.isCritical_target_of_source f hsourcef htargetf hsource

lemma isCritical_source_of_target (E : TotalDivergenceEquivalence d m k)
 (f : Space d → EuclideanSpace ℝ (Fin m))
 (hsourcef : IsAdmissibleForAction E.source f)
 (htargetf : IsAdmissibleForAction E.target f)

```

1.335.

*PHYSLIBALPHA/CLASSICALFIELDTHEORY/LOCAL/VARIATION.LEAN*569

```
(hsource : Lagrangian.SmoothInCoordinates E.source)
(htarget : Lagrangian.SmoothInCoordinates E.target)
(hcrit : IsCritical E.target f) :
 IsCritical E.source f :=
 E.sameEulerLagrangeOp.isCritical_source_of_target f hsourcef htargetf hsource htargetf
end TotalDivergenceEquivalence

end Local
end ClassicalFieldTheory
```

### 1.335 PhyslibAlpha/ClassicalFieldTheory/Local/Variation.lean

*/-*

Copyright (c) 2026 Juan Jose Fernandez Morales. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Juan Jose Fernandez Morales

*-/*

module

public import Physlib.ClassicalFieldTheory.Local.Variation

*/-!*

# Alpha extensions for admissible local variations

## i. Overview

This module adds the Euclidean component API needed by the coordinate-readout CFT stack in  
`PhyslibAlpha`.

The underlying `AdmissibleVariation` structure remains the maintained one from `Physlib`  
only adds helper lemmas used by the Alpha development.

## ii. Key results

- `ClassicalFieldTheory.Local.AdmissibleVariation.coord\_euclidean`

*-/*

@[expose] public section

open Physlib

namespace ClassicalFieldTheory

```

namespace Local

namespace AdmissibleVariation

variable {d m : ℕ}

/-- A Euclidean component of an admissible variation is again a test function
/
@[fun_prop]
lemma coord_euclidean (η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
 IsTestFunction (fun x => (η.toFun x) a) := by
 exact η.isTestFunction.comp_left (g := fun v : EuclideanSpace ℝ (Fin m) =>
 (by simp) (by fun_prop))

end AdmissibleVariation

end Local
end ClassicalFieldTheory

```

### 1.336 PhyslibAlpha/ClassicalMechanics/CoupledSpringPotential.lean

```

/-
Copyright (c) 2026 Bjørn Kjos-Hanssen. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Bjørn Kjos-Hanssen
-/
module

public import Mathlib.Analysis.Calculus.Gradient.Basic
public import Mathlib.Analysis.Calculus.ContDiff.FTaylorSeries
public import Mathlib.LinearAlgebra.QuadraticForm.Basic
public import Mathlib.Analysis.Calculus.FDeriv.Analytic
public import Mathlib.Analysis.Analytic.IteratedFDeriv
public import Mathlib.Analysis.Calculus.FDeriv.Symmetric
public import Mathlib.Analysis.InnerProductSpace.PiL2
public import PhyslibAlpha.Mathematics.PartialDerivativeTest
/-!
Coupled spring potential

As a proof of concept, we use the second derivative test in
`PhyslibAlpha.Mathematics.PartialDerivativeTest`
to prove that the coupled spring potential
`U := fun x : EuclideanSpace ℝ (Fin 2) => (x 0)^2 + x 0 * x 1 + (x 1)^2`
has a local minimum at zero.

```

1.336.

*PHYSLIBALPHA/CLASSICALMECHANICS/COUPLEDSPRINGPOTENTIAL 52/EAN*

-/

@[expose] public section

/-- The potential energy of a pair of coupled springs. -/

noncomputable def couplingPotential (x : EuclideanSpace  $\mathbb{R}$  (Fin 2)) :  $\mathbb{R}$  :=  
 (x 0) ^ 2 + x 0 \* x 1 + (x 1) ^ 2

/-

The coupling potential is analytic everywhere (it is a polynomial).

-/

lemma couplingPotential\_analyticAt (z : EuclideanSpace  $\mathbb{R}$  (Fin 2)) :  
 AnalyticAt  $\mathbb{R}$  couplingPotential z := by  
 have h0 : AnalyticAt  $\mathbb{R}$  (fun x : EuclideanSpace  $\mathbb{R}$  (Fin 2) => x 0) z :=  
 (EuclideanSpace.proj (□ :=  $\mathbb{R}$ ) 0).analyticAt z  
 have h1 : AnalyticAt  $\mathbb{R}$  (fun x : EuclideanSpace  $\mathbb{R}$  (Fin 2) => x 1) z :=  
 (EuclideanSpace.proj (□ :=  $\mathbb{R}$ ) 1).analyticAt z  
 unfold couplingPotential  
 exact ((h0.pow 2).add (h0.mul h1)).add (h1.pow 2)

/-

The derivative of the coupling potential at any point, as an explicit linear functional.

-/

lemma couplingPotential\_hasFDerivAt (x : EuclideanSpace  $\mathbb{R}$  (Fin 2)) :  
 HasFDerivAt couplingPotential  
 ((2 \* x 0 + x 1) • (EuclideanSpace.proj (□ :=  $\mathbb{R}$ ) 0)  
 + (x 0 + 2 \* x 1) • (EuclideanSpace.proj (□ :=  $\mathbb{R}$ ) 1)) x := by  
 have h0 : HasFDerivAt (fun x : EuclideanSpace  $\mathbb{R}$  (Fin 2) => x 0)  
 (EuclideanSpace.proj (□ :=  $\mathbb{R}$ ) 0) x := by  
 exact ContinuousLinearMap.hasFDerivAt (EuclideanSpace.proj (□ :=  $\mathbb{R}$ ) 0)  
 have h1 : HasFDerivAt (fun x : EuclideanSpace  $\mathbb{R}$  (Fin 2) => x 1)  
 (EuclideanSpace.proj (□ :=  $\mathbb{R}$ ) 1) x := by  
 norm\_num [hasFDerivAt\_iff\_isLittleO\_nhds\_zero];  
 convert! HasFDerivAt.add (HasFDerivAt.add (h0.pow 2) (h0.mul h1)) (h1.pow 2) using  
 ext; norm\_num; ring

/-

The gradient of the coupling potential vanishes at the origin.

-/

lemma couplingPotential\_gradient\_zero :  
 gradient couplingPotential (!₂[0, 0] : EuclideanSpace  $\mathbb{R}$  (Fin 2)) = 0 := by  
 convert! (InnerProductSpace.toDual  $\mathbb{R}$  (EuclideanSpace  $\mathbb{R}$  (Fin 2))).symm\_apply\_eq.mpr  
 convert! HasFDerivAt.fderiv (couplingPotential\_hasFDerivAt \_) using 1  
 norm\_num [fderiv\_apply\_one\_eq\_deriv]

```

/-
The value of the second derivative quadratic map of the coupling potential.
-/
lemma couplingPotential_iteratedFDeriv_two (z : EuclideanSpace ℝ (Fin 2))
 (a b : EuclideanSpace ℝ (Fin 2)) :
 iteratedFDeriv ℝ 2 couplingPotential z ![a, b]
 = 2 * a 0 * b 0 + a 0 * b 1 + a 1 * b 0 + 2 * a 1 * b 1 := by
 have h_second_deriv : fderiv ℝ (fderiv ℝ couplingPotential) z
 = (2 • (EuclideanSpace.proj (□ := ℝ) 0).smulRight
 (EuclideanSpace.proj (□ := ℝ) 0) + (EuclideanSpace.proj (□ := ℝ) 0).smul
 (EuclideanSpace.proj (□ := ℝ) 1) + (EuclideanSpace.proj (□ := ℝ) 1).smul
 (EuclideanSpace.proj (□ := ℝ) 0) + 2 • (EuclideanSpace.proj (□ := ℝ) 1)
 (EuclideanSpace.proj (□ := ℝ) 1)) := by
 refine' HasFDerivAt.fderiv _;
 rw [show fderiv ℝ couplingPotential = _ from funext fun x =>
 HasFDerivAt.fderiv (couplingPotential_hasFDerivAt x)];
 rw [hasFDerivAt_iff_isLittle0_nhds_zero];
 norm_num [Asymptotics.isLittle0_iff];
 intro ε hε; filter_upwards [Metric.ball_mem_nhds _ hε] with x hx; ring
 erw [show ((z.ofLp 0 * 2 + x.ofLp 0 * 2 + z.ofLp 1 + x.ofLp 1) • Euclid
 + (z.ofLp 0 + x.ofLp 0 + z.ofLp 1 * 2 + x.ofLp 1 * 2) • EuclideanSpac
 - ((z.ofLp 0 * 2 + z.ofLp 1) • EuclideanSpace.proj 0 + (z.ofLp 0 + z.
 • EuclideanSpace.proj 1) - (2 • x.ofLp 0 • EuclideanSpace.proj 0 + x.
 • EuclideanSpace.proj 1 + x.ofLp 1 • EuclideanSpace.proj 0 + 2 • x.of
 • EuclideanSpace.proj 1) : EuclideanSpace ℝ (Fin 2) →L[ℝ] ℝ) = 0 from
 ext; norm_num; ring]; norm_num [hε.le];
 positivity;
 rw [iteratedFDeriv_succ_apply_right]; simp +decide [h_second_deriv]; ring

/-
The second derivative quadratic map of the coupling potential is positive d
at the origin.
-/
lemma couplingPotential_posDef :
 (iteratedFDerivQuadraticMap couplingPotential
 (!₂[0, 0] : EuclideanSpace ℝ (Fin 2))).PosDef := by
 intros y hy;
 convert! (show 0 < 2 * y 0 ^ 2 + 2 * y 0 * y 1 + 2 * y 1 ^ 2 by
 exact not_le.mp fun h => hy <| by
 ext i
 fin_cases i <=> norm_num <=> nlinarith! [sq_nonneg (y.ofLp
generalize_proofs at *;
 convert! couplingPotential_iteratedFDeriv_two !₂[0, 0] y y using 1; ring!

lemma coupled_spring_potential :

```



### 1.337. *PHYSLIBALPHA/MATHEMATICS/PARTIALDERIVATIVETEST.lean* 173

```

let U := fun x : EuclideanSpace ℝ (Fin 2) => (x 0)^2 + x 0 * x 1 + (x 1)^2
IsLocalMin U {
 ofLp := ![0,0]
} := by
show IsLocalMin couplingPotential (![0, 0] : EuclideanSpace ℝ (Fin 2))
exact second_derivative_test_analyticAt couplingPotential_gradient_zero
(couplingPotential_analyticAt _) couplingPotential_posDef

```

### 1.337 PhyslibAlpha/Mathematics/PartialDerivativeTest.lean

*/-*

Copyright (c) 2026 Bjørn Kjos-Hanssen. All rights reserved.  
Released under Apache 2.0 license as described in the file LICENSE.  
Authors: Bjørn Kjos-Hanssen

*-/*

module

```

public import Mathlib.Analysis.Calculus.Gradient.Basic
public import Mathlib.Analysis.Calculus.ContDiff.FTaylorSeries
public import Mathlib.LinearAlgebra.QuadraticForm.Basic
public import Mathlib.Analysis.Calculus.FDeriv.Analytic
public import Mathlib.Analysis.Analytic.IteratedFDeriv
public import Mathlib.Analysis.Calculus.FDeriv.Symmetric
public import Mathlib.Analysis.InnerProductSpace.PiL2

```

*/-!*

# The Second Partial Derivatives Test

We prove a version of the second partial derivative test from calculus for analytic functions  $f : V \rightarrow \mathbb{R}$ , where  $V$  is a finite-dimensional vector space.

## Main results

\* `second_derivative_test`:

Suppose  $f$  is a real-valued function on a finite-dimensional inner product space that

has vanishing gradient at  $x_0$ , and has a power series on a ball of positive radius around  $x_0$ . If the second Frechét derivative is positive definite at  $x_0$  then  $f$  has local minimum at  $x_0$ .

## Tags

partial derivative test, calculus

*-/*

```

@[expose] public section

/-- Update a vector of length 2 in coordinate 0. -/
@[simp]
lemma Function.update₀ {α : Type*} {a b c : α} : Function.update ![a,b] 0 c
 ext i; fin_cases i <=> simp

/-- Update a vector of length 2 in coordinate 1. -/
@[simp]
lemma Function.update₁ {α : Type*} {a b c : α} : Function.update ![a,b] 1 c
 ext i; fin_cases i <=> simp

open Nat ContinuousMultilinearMap Finset Function

/-- The Hessian companion as a bilinear map. -/
noncomputable def hessianBilinearCompanion {V : Type*} [NormedAddCommGroup
 [NormedSpace ℝ V] (f : V → ℝ) (x₀ : V) : V →₁[ℝ] V →₁[ℝ] ℝ :=
 LinearMap.mk₂ ℝ (fun a b => iteratedFDeriv ℝ 2 f x₀ ![a,b] + iteratedFDeriv
 (fun _ _ _ => by
 simp_rw [Matrix.vecCons, succ_eq_add_one, reduceAdd, ← curryLeft_apply,
 map_add]
 simp only [curryLeft_apply, succ_eq_add_one, reduceAdd,
 ContinuousMultilinearMap.add_apply]
 abel) (by
 simp_rw [Matrix.vecCons, ← curryLeft_apply]
 simp only [map_smul, curryLeft_apply, succ_eq_add_one, reduceAdd,
 ContinuousMultilinearMap.smul_apply, smul_eq_mul]
 ring_nf
 simp) (fun _ _ _ => by
 simp_rw [Matrix.vecCons, succ_eq_add_one, reduceAdd, ← curryLeft_apply,
 map_add]
 simp only [ContinuousMultilinearMap.add_apply, curryLeft_apply, succ_eq_add_one,
 Matrix.Fin.cons_vecEmpty, Matrix.Fin.cons_vecCons]
 abel) (by
 simp_rw [Matrix.vecCons, ← curryLeft_apply]
 simp only [map_smul, ContinuousMultilinearMap.smul_apply, curryLeft_apply,
 reduceAdd, smul_eq_mul]
 simp_rw [← mul_add]
 simp)
)

/-- The second iterated Frechét derivative as a quadratic map. -
/
noncomputable def iteratedFDerivQuadraticMap {V : Type*} [NormedAddCommGroup
 [NormedSpace ℝ V] (f : V → ℝ) (x₀ : V) : QuadraticMap ℝ V ℝ := {

```

```

toFun := fun y => iteratedFDeriv ℝ 2 f x₀ ![y,y]
exists_companion' := (hessianBilinearCompanion f x₀, fun x y => by
 have ha (u v b) := (iteratedFDeriv ℝ 2 f x₀).map_update_add' ![u,v] b x y
 have ha₀ := ha x (x + y) 0
 have ha₁ (u) := ha u x 1
 simp only [update₀, MultilinearMap.toFun_eq_coe, coe_coe, update₁, hessianBiline
 at ha₀ ha₁ ha ⊢
 rw [ha₀, ha₁, add_assoc, add_assoc]
 apply add_left_cancel_iff.mpr
 rw [← add_assoc, add_comm]
 simp)
toFun_smul := fun u v => by
 have hsm (b c) := (iteratedFDeriv ℝ 2 f x₀).map_update_smul' ![b,v] c u v
 have hsm₀ := hsm v 0
 have hsm₁ := hsm (u • v) 1
 simp only [update₀, update₁, MultilinearMap.toFun_eq_coe, coe_coe, smul_eq_mul]
 at hsm₀ hsm₁ hsm
 rw [smul_eq_mul, mul_assoc, ← hsm₀, hsm₁]}

/-- A continuous multilinear map is bilinear. -/
noncomputable def continuousBilinearMapOfContinuousMultilinearMap
 {α : Type*} [NontriviallyNormedField α] [CompleteSpace α]
 {V : Type*}
 [NormedAddCommGroup V] [NormedSpace α V] [FiniteDimensional α V]
 (g : ContinuousMultilinearMap α (fun _ : Fin 2 => V) α) : V →L[α] V →L[α] α := {
toFun := fun x => {
 toFun := fun y => g.toFun ![x,y]
 map_add' := fun a b => by simpa [update₁] using g.map_update_add ![x,b] 1 a b
 map_smul' := fun m a => by simpa [update₁] using g.map_update_smul ![x,a] 1 m a
 cont := g.cont.comp' <| continuous_const.matrixVecCons
 <| continuous_id'.matrixVecCons continuous_const}
map_add' := fun a b => by ext c; simpa [update₀] using g.map_update_add ![a,c] 0 a
map_smul' := fun c x => by ext y; simpa [update₀] using g.map_update_smul ![x,y] 0
cont := continuous_clm_apply.mpr fun x => g.cont.comp'
 <| continuous_id'.matrixVecCons continuous_const}

/-- . -/
def QuadraticMap.toMultilinearMap {V : Type*} [AddCommGroup V] [Module ℝ V]
 (Q : QuadraticMap ℝ V ℝ) : MultilinearMap ℝ (fun _ : Fin 2 => V) ℝ := {
 toFun := fun v => Q.polarBilin (v 0) (v 1)
 map_update_add' := by simp
 map_update_smul' := by simp}

/-- . -/

```

```

noncomputable def QuadraticMap.toMultilinearMapHalfPolarBilin
 {V : Type*} [AddCommGroup V] [Module ℝ V]
 (Q : QuadraticMap ℝ V ℝ) :
 MultilinearMap ℝ (fun _ : Fin 2 => V) ℝ :=
 let B := Q.polarBilin
 { toFun := fun v => (1/2) * B (v 0) (v 1)
 map_update_add' := by
 simp +zetaDelta only [one_div, Fin.isValue, polarBilin_apply_apply, F
 update_self, ne_eq, one_ne_zero, not_false_eq_true, update_of_ne, p
 zero_ne_one, polar_add_right, forall_const] at *
 intro m
 apply And.intro
 · intro x y
 exact Distrib.left_distrib 2-1 _ _
 · intro x y
 exact Distrib.left_distrib 2-1 _ _
 map_update_smul' := by
 simp +decide only [one_div, Fin.isValue, update_apply, smul_eq_mul, F
 ↓reduceIte, map_smul, one_ne_zero, LinearMap.smul_apply, zero_ne_on
 intros
 constructor
 · intros
 ring_nf
 · intros
 ring_nf
 }

/--
The multilinear map associated to a quadratic map is continuous.
-/
theorem QuadraticMap.toMultilinearMap_continuous {V : Type*}
 [NormedAddCommGroup V] [NormedSpace ℝ V]
 [FiniteDimensional ℝ V] (Q : QuadraticMap ℝ V ℝ) : Continuous Q.toMulti
have h_bilinear : ∃ B : V →1[ℝ] V →L[ℝ] ℝ, ∀ x y, B x y = Q.toMultilinear
have h_bilinear : ∃ B : V →1[ℝ] V →L[ℝ] ℝ, ∀ x y, B x y = Q.polarBilin
have h_bilinear : ∀ x : V, ∃ Bx : V →L[ℝ] ℝ, ∀ y : V, Bx y = Q.polarB
 intro x
 use (by
 apply ContinuousLinearMap.mk
 swap
 exact Q.polarBilin x
 simp only [AddHom.toFun_eq_coe, LinearMap.coe_toAddHom]
 exact LinearMap.continuous_of_finiteDimensional (Q.polarBilin x)
)
 intro y; rfl
 -- exact fun x => (ContinuousLinearMap.mk (Q.polarBilin x), fun

```

```

 choose B hB using h_bilinear;
 refine ({ toFun := B, map_add' := ?_, map_smul' := ?_ }, hB) <=> aesop;
 aesop;
-- Since B is a linear map between finite-dimensional spaces, it is continuous.
obtain (B, hB) := h_bilinear;
have hB_cont : Continuous B := by
 exact B.continuous_of_finiteDimensional;
convert hB_cont.comp (show Continuous fun v : Fin 2 → V => v 0 from continuous_apply 0
Continuous.clm_apply <|
 show Continuous fun v : Fin 2 → V => v 1 from continuous_apply 1 using 1; aesop

/-- . -/
theorem QuadraticMap.toMultilinearMapHalfPolarBilin_continuous {V : Type*}
 [NormedAddCommGroup V] [NormedSpace ℝ V]
 [FiniteDimensional ℝ V] (Q : QuadraticMap ℝ V ℝ) :
 Continuous Q.toMultilinearMapHalfPolarBilin := by
 have h_bilinear :
 ∃ B : V →ℝ V →ℝ ℝ, ∀ x y, B x y = Q.toMultilinearMapHalfPolarBilin ![x, y]
 have h_bilinear : ∃ B : V →ℝ V →ℝ ℝ, ∀ x y, B x y = (1/2) * Q.polarBilin x y
 have h_bilinear : ∀ x : V, ∃ Bx : V →ℝ ℝ, ∀ y : V, Bx y = (1/2) * Q.polarBilin x y
 intro x
 use (by
 apply ContinuousLinearMap.mk (
 toLinearMap := by
 let q := fun y : V => (1/2) * Q.polarBilin x y
 refine IsLinearMap.mk' q ?_
 refine { map_add := ?_, map_smul := ?_ }
 · intro a b
 unfold q
 have : Q.polarBilin x (a+b) =
 Q.polarBilin x a +
 Q.polarBilin x b := by exact LinearMap.map_add (Q.polarBilin x)
 rw [this]
 linarith
 · intro c a
 unfold q
 simp only [one_div, map_smul, polarBilin_apply_apply, smul_eq_mul]
 linarith
)
 (by simp; apply LinearMap.continuous_of_finiteDimensional)
)
 intro y; simp
 -- refine (ContinuousLinearMap.mk (by
 -- let q := fun y : V => (1/2) * Q.polarBilin x y

```

```

-- refine IsLinearMap.mk' q ?_
-- refine { map_add := ?_, map_smul := ?_ }
-- · intro a b
-- unfold q
-- have : Q.polarBilin x (a+b) =
-- Q.polarBilin x a +
-- Q.polarBilin x b := by exact LinearMap.map_add (Q.pola
-- rw [this]
-- linarith
-- · intro c a
-- unfold q
-- simp
-- linarith), fun y => by simp)
choose B hB using h_bilinear;
refine ({ toFun := B, map_add' := ?_, map_smul' := ?_ }, hB)
· intro x y
 simp_all only [one_div, polarBilin_apply_apply]
 ext i
 rw [hB (x+y) i]
 simp only [polar_add_left, _root_.add_apply]
 rw [hB x i, hB y i]
 linarith
intro m x
ext y
rw [hB (m • x), _root_.smul_apply, hB x y]
simp only [one_div, polarBilin_apply_apply, polar_smul_left, smul_eq_
ring
simp_all only [one_div, polarBilin_apply_apply]
obtain ⟨w, h⟩ := h_bilinear
unfold QuadraticMap.toMultilinearMapHalfPolarBilin
simp only [one_div, Fin.isValue, polarBilin_apply_apply, MultilinearMap
Matrix.cons_val_zero, Matrix.cons_val_one, Matrix.cons_val_fin_one]
use w
-- Since B is a linear map between finite-dimensional spaces, it is con
obtain ⟨B, hB⟩ := h_bilinear;
have hB_cont : Continuous B := by
 exact B.continuous_of_finiteDimensional;
convert! hB_cont.comp (show Continuous fun v : Fin 2 → V => v 0 from con
Continuous.clm_apply <|
 show Continuous fun v : Fin 2 → V => v 1 from continuous_apply 1 usin

/--
Construct a continuous multilinear map from a quadratic map on a finite dim
-/
def QuadraticMap.toContinuousMultilinearMap {V : Type*} [NormedAddCommGroup
[FiniteDimensional ℝ V] (Q : QuadraticMap ℝ V ℝ) :
```

```

ContinuousMultilinearMap ℝ (fun _ : Fin 2 → V) ℝ :=
{ Q.toMultilinearMap with
 cont := Q.toMultilinearMap_continuous }

/-- . -/
noncomputable def QuadraticMap.toContinuousMultilinearMapHalfPolarBilin {V : Type*}
 [NormedAddCommGroup V] [NormedSpace ℝ V] [FiniteDimensional ℝ V] (Q : QuadraticMap ℝ V ℝ) :
 ContinuousMultilinearMap ℝ (fun _ : Fin 2 → V) ℝ :=
{ Q.toMultilinearMapHalfPolarBilin with
 cont := Q.toMultilinearMapHalfPolarBilin_continuous }

/--
The constructed continuous multilinear map agrees with the polar bilinear form.
-/
theorem QuadraticMap.toContinuousMultilinearMap_apply {V : Type*} [NormedAddCommGroup V]
 [NormedSpace ℝ V] [FiniteDimensional ℝ V] (Q : QuadraticMap ℝ V ℝ) (x y : V) :
 Q.toContinuousMultilinearMap ![x, y] = Q.polarBilin x y := by
 rfl

/-- . -/
theorem QuadraticMap.toContinuousMultilinearMap_applyHalf {V : Type*} [NormedAddCommGroup V]
 [NormedSpace ℝ V] [FiniteDimensional ℝ V] (Q : QuadraticMap ℝ V ℝ) (x y : V) :
 Q.toContinuousMultilinearMapHalfPolarBilin ![x, y] = (1/2) * Q.polarBilin x y := by
 rfl

/-- . -/
lemma coercive_of_posdefHalf {V : Type*} [NormedAddCommGroup V] [NormedSpace ℝ V]
 [FiniteDimensional ℝ V] {F : QuadraticMap ℝ V ℝ}
 (hf' : F.PosDef) :
 IsCoercive (continuousBilinearMapOfContinuousMultilinearMap
 F.toContinuousMultilinearMapHalfPolarBilin) := by
 obtain hsub | hnt := subsingleton_or_nontrivial V
 · exact ⟨1, one_pos, fun u => by rw [Subsingleton.elim u 0]; simp⟩
 haveI := hnt
 have h₀ : ∃ x : ↑(Metric.sphere 0 1), ∀ (y : ↑(Metric.sphere 0 1)),
 (fun y => F.toContinuousMultilinearMapHalfPolarBilin ![y, y]) x.1 ≤
 (fun y => F.toContinuousMultilinearMapHalfPolarBilin ![y, y])
 y.1 := by
 obtain ⟨x, hx⟩ := IsCompact.exists_isMinOn
 (f := (fun y => F.toContinuousMultilinearMapHalfPolarBilin ![y, y]))
 (isCompact_sphere (0:V) 1) (NormedSpace.sphere_nonempty.mpr (by simp))
 (Continuous.continuousOn <| by
 refine Continuous.comp ?_
 (continuous_id.matrixVecCons (continuous_id.matrixVecCons continuous_const
 exact map_continuous _))

```

```

use {x,hx.1}
intro y
simp only [mem_sphere_iff_norm, sub_zero, IsMinOn, IsMinFilter,
 Filter.eventually_principal] at hx
apply hx.2
simp
simp only [Subtype.forall, mem_sphere_iff_norm, sub_zero, Subtype.exists,
 obtain {m,hm} := h₀]
use F.toContinuousMultilinearMapHalfPolarBilin ![m, m]
rw [continuousBilinearMapOfContinuousMultilinearMap]
constructor
· unfold QuadraticMap.toContinuousMultilinearMapHalfPolarBilin
 QuadraticMap.toMultilinearMapHalfPolarBilin
 change 0 < (fun v ↦ (1/2) * (F (v 0 + v 1) - F (v 0) - F (v 1))) ![m,
 have (x y : V) : F (x + y) = F x + F y + F.polarBilin x y := QuadraticMap
 simp only [succ_eq_add_one, reduceAdd, Fin.isValue, Matrix.cons_val_z
 Matrix.cons_val_fin_one, gt_iff_lt]
 rw [this]
 ring_nf
 suffices 0 < (F.polarBilin m) m by linarith
 simp only [QuadraticMap.polarBilin, LinearMap.mk₂_apply, QuadraticMap
 nsmul_eq_mul, cast_ofNat, ofNat_pos, mul_pos_iff_of_pos_left]
 apply hf'
 intro hc
 subst hc
 simp at hm
· intro u
 by_cases hu : u = 0
 · subst hu
 unfold QuadraticMap.toContinuousMultilinearMapHalfPolarBilin
 QuadraticMap.toMultilinearMapHalfPolarBilin
 simp [QuadraticMap.polar]
 · have h₁ : ||u|| * ||u||⁻¹ = 1 := CommGroupWithZero.mul_inv_cancel _ <| no
 repeat (
 refine le_of_mul_le_mul_right ?_ <| Right.inv_pos.mpr <| norm_pos_if
 rw [mul_assoc, h₁]
 simp only [mul_one, MultilinearMap.toFun_eq_coe, coe_coe,
 ContinuousLinearMap.coe_mk', LinearMap.coe_mk, AddHom.coe_mk])
 have h₂ := update₁ ▶ update₁ ▶
 F.toContinuousMultilinearMapHalfPolarBilin.map_update_smul' ![(||u||⁻¹
 simp only [MultilinearMap.toFun_eq_coe, coe_coe, smul_eq_mul] at h₂
 have : F.toContinuousMultilinearMapHalfPolarBilin ![u, u] * ||u||⁻¹
 = F.toContinuousMultilinearMapHalfPolarBilin ![(||u||⁻¹ • u, u] :=
 simp_rw [Matrix.vecCons, ← curryLeft_apply, mul_comm]
 simp
 rw [this, mul_comm, ← h₂]

```



```

 exact hm.2 (||u||-1 • u) (by
 rw [← h1, norm_smul, mul_comm]
 congr
 refine Real.norm_of_nonneg ?_
 simp)

/--
The polar bilinear form of the second-derivative quadratic map is the symmetrized
Hessian.
-/
lemma polarBilin_iteratedFDerivQuadraticMap {V : Type*} [NormedAddCommGroup V] [NormedSpace ℝ V]
 (f : V → ℝ) (x₀ : V) (x y : V) :
 (iteratedFDerivQuadraticMap f x₀).polarBilin x y
 = iteratedFDeriv ℝ 2 f x₀ ![x, y] + iteratedFDeriv ℝ 2 f x₀ ![y, x] := by
 unfold iteratedFDerivQuadraticMap;
 simp +decide [QuadraticMap.polarBilin, QuadraticMap.polar];
 simp +decide [iteratedFDeriv_succ_apply_right, sub_eq_iff_eq_add];
 simp +decide [Fin.init];
 ring

/--
Symmetry of the second iterated Fréchet derivative (Schwarz's theorem).
-/
lemma iteratedFDeriv_two_swap {V : Type*} [NormedAddCommGroup V] [NormedSpace ℝ V]
 {f : V → ℝ} {x₀ : V} (hf : ContDiffAt ℝ 2 f x₀) (x y : V) :
 iteratedFDeriv ℝ 2 f x₀ ![y, x] = iteratedFDeriv ℝ 2 f x₀ ![x, y] := by
 convert hf.isSymmSndFDerivAt le_top using 1;
 rw [isSymmSndFDerivAt_iff_iteratedFDeriv];
 constructor <=> intro h;
 · convert hf.isSymmSndFDerivAt le_top using 1;
 rw [isSymmSndFDerivAt_iff_iteratedFDeriv];
 · convert congr_arg (fun g => g ![y, x]) h.symm using 1;
 simp only [domDomCongr_apply, Fin.revPerm_apply]
 exact congr_arg _ (by ext i; fin_cases i <=> rfl)

/-- Positive definiteness implies coercivity.
The proof uses the general fact `coercive_of_posdefHalf`
but requires a differentiability assumption on `f`.
-/
lemma coercive_of_posdef_of_contdiff {V : Type*}
 [NormedAddCommGroup V] [NormedSpace ℝ V]
 [FiniteDimensional ℝ V] {f : V → ℝ} {x₀ : V}
 (hf : ContDiffAt ℝ 2 f x₀)
 (hf' : (iteratedFDerivQuadraticMap f x₀).PosDef) :

```

```

 IsCoercive (continuousBilinearMapOfContinuousMultilinearMap
 (iteratedFDeriv $\mathbb{R}$ 2 f x0)) := by
 have key : $\forall x y : V$,
 (iteratedFDerivQuadraticMap f x0).toContinuousMultilinearMapHalfPolar
 = iteratedFDeriv $\mathbb{R}$ 2 f x0 ![x, y] := by
 intro x y
 rw [QuadraticMap.toContinuousMultilinearMap_applyHalf,
 polarBilin_iteratedFDerivQuadraticMap, iteratedFDeriv_two_swap hf]
 ring
 have heq : continuousBilinearMapOfContinuousMultilinearMap
 (iteratedFDerivQuadraticMap f x0).toContinuousMultilinearMapHalfPol
 = continuousBilinearMapOfContinuousMultilinearMap (iteratedFDeriv $\mathbb{R}$ 2
 ext x y
 simp [continuousBilinearMapOfContinuousMultilinearMap] using key x y
 rw [← heq]
 exact coercive_of_posdefHalf hf'

/-- Positive definiteness implies coercivity. -/
lemma coercive_of_posdef {V : Type*} [NormedAddCommGroup V] [NormedSpace $\mathbb{R}$
 [FiniteDimensional $\mathbb{R}$ V] {f : V → $\mathbb{R}$ } {x0 : V}
 (hf' : (iteratedFDerivQuadraticMap f x0).PosDef) :
 IsCoercive (continuousBilinearMapOfContinuousMultilinearMap
 (iteratedFDeriv $\mathbb{R}$ 2 f x0)) := by
 obtain hsub | hnt := subsingleton_or_nontrivial V
 · exact ⟨1, one_pos, fun u => by rw [Subsingleton.elim u 0]; simp⟩
 haveI := hnt
 have h0 : $\exists x : \uparrow(\text{Metric.sphere } 0 \ 1), \forall (y : \uparrow(\text{Metric.sphere } 0 \ 1)),$
 (fun y ↦ (iteratedFDeriv $\mathbb{R}$ 2 f x0) ![y, y]) x.1 ≤ (fun y ↦ (iteratedFDe
 y.1 := by
 obtain ⟨x, hx⟩ := IsCompact.exists_isMinOn (f := (fun y => (iteratedFDe
 (isCompact_sphere (0:V) 1) (NormedSpace.sphere_nonempty.mpr (by simp))
 (Continuous.continuousOn <| by
 refine Continuous.comp ?_
 (continuous_id.matrixVecCons (continuous_id.matrixVecCons continu
 exact map_continuous _)
 use ⟨x, hx.1⟩
 intro y
 simp only [mem_sphere_iff_norm, sub_zero, IsMinOn, IsMinFilter,
 Filter.eventually_principal] at hx
 apply hx.2
 simp
 simp only [Subtype.forall, mem_sphere_iff_norm, sub_zero, Subtype.exists,
 obtain ⟨m, hm⟩ := h0
 use iteratedFDeriv $\mathbb{R}$ 2 f x0 ![m, m]
 rw [continuousBilinearMapOfContinuousMultilinearMap]
 constructor

```

```

· exact hf' m (by intro hc; simp [hc] at hm)
· intro u
 by_cases hu : u = 0
 · simp [hu, iteratedFDeriv_succ_apply_left]
 · have h1 : ||u|| * ||u||-1 = 1 := CommGroupWithZero.mul_inv_cancel _ <| norm_ne_zero
 repeat (
 refine le_of_mul_le_mul_right ?_ <| Right.inv_pos.mpr <| norm_pos_iff.mpr hu
 rw [mul_assoc, h1]
 simp only [mul_one, MultilinearMap.toFun_eq_coe, coe_coe,
 ContinuousLinearMap.coe_mk', LinearMap.coe_mk, AddHom.coe_mk])
 have h2 := update1 > update1 >
 (iteratedFDeriv ℝ 2 f x₀).map_update_smul' ![||u||-1 • u, u] 1 ||u||-1 u
 simp only [MultilinearMap.toFun_eq_coe, coe_coe, smul_eq_mul] at h2
 have : (iteratedFDeriv ℝ 2 f x₀) ![u, u] * ||u||-1
 = (iteratedFDeriv ℝ 2 f x₀) ![||u||-1 • u, u] := by
 simp_rw [Matrix.vecCons, ← curryLeft_apply, mul_comm]
 simp
 rw [this, mul_comm, ← h2]
 exact hm.2 (||u||-1 • u) (by
 rw [← h1, norm_smul, mul_comm]
 congr
 refine Real.norm_of_nonneg ?_
 simp)

/-- . -/
theorem le_of_little0 {V : Type*}
 [NormedAddCommGroup V] [InnerProductSpace ℝ V]
 {f : V → ℝ} {x₀ x : V} {C : ℝ}
 (hx : C * (||x - x₀|| ^ 2) ≤ iteratedFDeriv ℝ 2 f x₀ fun _ ↦ x - x₀)
 (hx₀ : fderiv ℝ f x₀ x = fderiv ℝ f x₀ x₀)
 (h1 : ||f x - ∑ i ∈ range 3, 1 / ↑i ! * iteratedFDeriv ℝ i f x₀ fun _ ↦ x - x₀||
 ≤ C / 2 * ||x - x₀|| ^ 2) :
 f x₀ ≤ f x := by
 have rev_ineq {a b c d : ℝ} (h : a + b ≤ c + d) (h' : d ≤ b) : a ≤ c := by
 linarith
 refine rev_ineq ?_ <| mul_le_mul_of_nonneg_right (by convert hx using 2) (show 0 ≤
 simp only [range_add_one, range_zero, insert_empty_eq, one_div, mem_insert, OfNat.
 mem_singleton, OfNat.ofNat_ne_zero, or_self, not_false_eq_true, sum_insert, fact
 cast_ofNat, one_ne_zero, factorial_one, cast_one, inv_one, iteratedFDeriv_one_ap
 one_mul, sum_singleton, factorial_zero, iteratedFDeriv_zero_apply, Real.norm_eq
 have := le_of_max_le_right (hx₀ > h1)
 linarith

/-- Second partial derivative test, "little oh" form. -/
theorem isLocalMin_of_posDef_of_little0 {V : Type*} [NormedAddCommGroup V]

```

```

[InnerProductSpace ℝ V] [FiniteDimensional ℝ V] {f : V → ℝ} {x₀ : V}
(h : (fun x => f x - ∑ i ∈ range 3, 1 / (i)! * iteratedFDeriv ℝ i f x₀
 =o[nhds x₀] fun x => ||x - x₀|| ^ 2) (h₀ : gradient f x₀ = 0) (hcd : Co
(hf : (iteratedFDerivQuadraticMap f x₀).PosDef) : IsLocalMin f x₀ := by
have ⟨C,hC⟩ := coercive_of_posdef hf
have ⟨C,hC⟩ := coercive_of_posdef_of_contdiff hcd hf
simp only [Asymptotics.IsLittle0, Asymptotics.IsBig0With] at h
apply (h (half_pos hC.1)).mono
intro x
have hx := hC.2 (x - x₀)
simp only [continuousBilinearMapOfContinuousMultilinearMap, MultilinearMa
 coe_coe, ContinuousLinearMap.coe_mk', LinearMap.coe_mk,
 AddHom.coe_mk] at hx
rw [mul_assoc, show ![x - x₀, x - x₀] = fun _ => x - x₀ by
 ext i; fin_cases i <=> simp] at hx
have hx₀ : inner ℝ (gradient f x₀) (x - x₀) = fderiv ℝ f x₀ (x - x₀) := b
simp only [h₀, inner_zero_left, map_sub] at hx₀
simp only [norm_pow, norm_norm]
rw [← pow_two] at hx
exact le_of_little0 hx <| sub_eq_zero.mp hx₀.symm

/-- Having a power series implies quadratic approximation. -
/
lemma little0_of_powerseries {V : Type*} [NormedAddCommGroup V] [NormedSpace
 {f : V → ℝ} {x₀ : V} {p : FormalMultilinearSeries ℝ V ℝ}
 {r : NNReal} (h : HasFPowerSeriesOnBall f p x₀ r) :
 (fun x => f x - p.partialSum 3 (x - x₀)) =o[nhds x₀] fun x => ||x - x₀||
 have hbig : (fun y => f (x₀ + y) - p.partialSum 3 y) =o[nhds 0] fun y
 HasFPowerSeriesAt.isBig0_sub_partialSum_pow h.hasFPowerSeriesAt _
 convert hbig.comp_tendsto (show Filter.Tendsto (fun x : V => x - x₀)
 tendsto_sub_nhds_zero_iff.mpr Filter.tendsto_id) |> fun h => h.trans
 · aesop
 · rw [Asymptotics.isLittle0_iff_tendsto'] <=> norm_num
 simp [pow_succ, mul_div_assoc] using tendsto_norm_sub_self x₀

@[nontriviality]
lemma isLocalMin.of_subsingleton {V : Type*} [TopologicalSpace V]
 [Subsingleton V] {f : V → ℝ} {x₀ : V} : IsLocalMin f x₀ := by
 simp [IsLocalMin, IsMinFilter]

/-- The second partial derivative test. -/
theorem second_derivative_test {V : Type*} [NormedAddCommGroup V] [InnerPro
 [FiniteDimensional ℝ V] {f : V → ℝ} {x₀ : V} {p : FormalMultilinearSeries
 (h₀ : gradient f x₀ = 0) {r : NNReal}
 (h₁ : HasFPowerSeriesOnBall f p x₀ r)

```

1.338.

**PHYSLIBALPHA/QUANTUMMECHANICS/QUANTUMHARMONICOSCILLATOR.lean**

```

 (hf : (iteratedFDerivQuadraticMap f x₀).PosDef) : IsLocalMin f x₀ := by
 nontriviality V
 have h₂ (x : V) (i : ℕ) : p i (fun _ => x - x₀)
 = 1 / (i)! * iteratedFDeriv ℝ i f x₀ fun _ => x - x₀ := by
 rw [← HasFPowerSeriesOnBall.factorial_smul h₁ (x - x₀) i]
 ring_nf
 field_simp
 have h₃ (x : V) : ∑ i ∈ range 3, p i (fun _ => x - x₀)
 = ∑ i ∈ range 3, 1 / (i)! * iteratedFDeriv ℝ i f x₀ fun _ => x - x₀
 congr
 ext
 rw [h₂]
 have h₄ (x : V) := congrArg (HSub.hSub (f x)) (h₃ x)
 exact isLocalMin_of_posDef_of_littleo (funext_iff.mpr h₄ ▸ littleO_of_powerseries
 h₁.analyticAt.contDiffAt hf)

/--
A more convenient form of the second partial derivative test: it suffices that `f`
is analytic at `x₀` (so that an explicit power series need not be produced by hand).
-/
theorem second_derivative_test_analyticAt {V : Type*} [NormedAddCommGroup V]
 [InnerProductSpace ℝ V] [FiniteDimensional ℝ V] {f : V → ℝ} {x₀ : V}
 (h₀ : gradient f x₀ = 0) (han : AnalyticAt ℝ f x₀)
 (hf : (iteratedFDerivQuadraticMap f x₀).PosDef) : IsLocalMin f x₀ := by
 obtain ⟨p, r, hr⟩ := han
 obtain ⟨r', hr'0, hr'lt⟩ := ENNReal.lt_iff_exists_nnreal_btwn.mp hr.r_pos
 exact second_derivative_test h₀ (hr.mono (by exact_mod_cast hr'0) hr'lt.le) hf

```

### 1.338 PhyslibAlpha/QuantumMechanics/QuantumHarmonicOscillator.lean

```

/-
Copyright (c) 2026 Bjørn Kjos-Hanssen. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Bjørn Kjos-Hanssen
-/
module
public import Mathlib.Data.Matrix.PEquiv
public import Mathlib.Probability.Distributions.Poisson.Basic
public import Mathlib.Analysis.Normed.Lp.lpSpace
/-!
Quantum harmonic oscillator
-/

noncomputable section

```

```

/-- Annihilation operator. -/
def a (x : $\mathbb{N} \rightarrow \mathbb{C}$) : $\mathbb{N} \rightarrow \mathbb{C}$:= fun n => $\sqrt{n+1}$ * x (n + 1)

def aLin : ($\mathbb{N} \rightarrow \mathbb{C}$) $\rightarrow_{\mathbb{C}}$ ($\mathbb{N} \rightarrow \mathbb{C}$) := {
 toFun := a
 map_add' := by
 unfold a
 intro x y
 ext n
 simp only [Pi.add_apply]
 ring_nf
 map_smul' := by
 intro m x
 unfold a
 ext n
 simp only [Pi.smul_apply, smul_eq_mul, RingHom.id_apply]
 ring_nf
}

/-- Creation operator. -/
def a_dag (x : $\mathbb{N} \rightarrow \mathbb{C}$) : $\mathbb{N} \rightarrow \mathbb{C}$:= fun n => ite (n = 0) 0 ($\sqrt{n}$ * x (n - 1))

def a_dagLin : ($\mathbb{N} \rightarrow \mathbb{C}$) $\rightarrow_{\mathbb{C}}$ ($\mathbb{N} \rightarrow \mathbb{C}$) := {
 toFun := a_dag
 map_add' := by
 unfold a_dag
 intro x y
 ext n
 simp only [Pi.add_apply]
 ring_nf
 split_ifs <=> simp
 map_smul' := by
 intro m x
 unfold a_dag
 ext n
 simp only [Pi.smul_apply, smul_eq_mul, RingHom.id_apply, mul_ite, mul_zero]
 ring_nf
}

def ϵ (n : $\mathbb{N}$) (c : $\mathbb{C}$) : $\mathbb{N} \rightarrow \mathbb{C}$:= fun i => ite (i = n) c 0

/-- Verify that a_dag really is the transpose of a. -/
lemma a_dag_eq (i j : $\mathbb{N}$) : a_dag (ϵ i 1) j = star (a (ϵ j 1) i) := by
 unfold a a_dag ϵ
 simp only [mul_ite, mul_one, mul_zero, RCLike.star_def]

```

1.338.

PHYSLIBALPHA/QUANTUMMECHANICS/QUANTUMHARMONICOSCILLATOR.lean

```

split_ifs with _ _ _ g₃
all_goals try omega
all_goals try exact Eq.symm ((fun {z} ↦ Complex.conj_eq_iff_re.mpr) rfl)
rw [← g₃]; simp

/-- Verify that $a |n + 1\rangle = \sqrt{n + 1} |n\rangle$ -/
lemma verify_a (n : ℕ) :
 a (ε (n + 1) 1) =
 ε n (√(n + 1)) := by
 unfold a ε
 ext i
 simp only [Nat.add_right_cancel_iff, mul_ite, mul_one, mul_zero]
 split_ifs with g₀
 · rw [g₀]
 · rfl

/-- Verify that $a^\dagger |n\rangle = \sqrt{n+1} |n+1\rangle$. -/
lemma verify_a_dag (n : ℕ) :
 a_dag (ε n 1) = ε (n + 1) (√(n + 1)) := by
 unfold a_dag ε
 ext i
 split_ifs with _ _ _ h
 all_goals try omega
 · rfl
 · rw [h]; simp
 · simp

lemma verify_a_dag_a (n : ℕ) (x : ℕ → ℂ) :
 a_dag (a x) n = n * x n := by
 unfold a_dag a
 split_ifs with g
 · rw [g]; simp
 · have h : n - 1 + 1 = n := by omega
 repeat rw [h]
 norm_cast
 rw [h]
 rw [← mul_assoc]
 congr
 norm_cast
 refine Real.mul_self_sqrt ?_
 simp

lemma verify_a_a_dag (n : ℕ) (x : ℕ → ℂ) :
 a (a_dag x) n = (n + 1) * x n := by
 unfold a_dag a
 simp only [Nat.add_eq_zero_iff, one_ne_zero, and_false, ↓reduceIte, Nat.cast_add,
```

```

 add_tsub_cancel_right]
 rw [← mul_assoc]
 congr
 norm_cast
 refine Real.mul_self_sqrt ?_
 norm_cast
 omega

lemma commutation_relation :
 a ∘ a_dag - a_dag ∘ a = id := by
 unfold a a_dag
 ext x i
 simp only [Pi.sub_apply, Function.comp_apply, Nat.add_eq_zero_iff, one_ne_zero,
 ↓reduceIte, Nat.cast_add, Nat.cast_one, add_tsub_cancel_right, id_eq]
 split_ifs with g₀
 · rw [g₀]; simp
 · norm_cast
 have : i - 1 + 1 = i := by omega
 rw [this]
 repeat rw [← mul_assoc]
 have : (i + 1) * x i - i * x i = x i := by
 ring_nf
 rw [← this]
 congr
 · norm_cast
 refine Real.mul_self_sqrt ?_
 linarith
 · norm_cast
 refine Real.mul_self_sqrt ?_
 simp

def coherentState (α : ℂ) : ℕ → ℂ :=
 fun n : ℕ => Real.exp (-||α||^2 / 2) * α ^ n / √(n.factorial)

def probabilityOf (n : ℕ) (α : ℂ) : NNReal :=
 ⟨||coherentState α n||^2, by simp⟩

/-- Coherent state has a Poisson distribution. -/
lemma probabilityOf_eq_poisson_C (n : ℕ) (α : ℂ) :
 let Λ := ⟨||α|| ^ 2, sq_nonneg ||α||⟩
 probabilityOf n α =
 ProbabilityTheory.poissonMeasure Λ {n} := by
 unfold probabilityOf ProbabilityTheory.poissonMeasure coherentState
 simp only [Complex.ofReal_exp, Complex.ofReal_div, Complex.ofReal_neg, Complex.ofReal_ofNat,
 Complex.norm_div, Complex.norm_mul, norm_pow, Complex.norm_eq_abs, MeasurableSpace.measurableSet_top, MeasureTheory.Meas

```



1.338.

PHYSLIBALPHA/QUANTUMMECHANICS/QUANTUMHARMONICOSCILLATOR81.lean

```

 MeasureTheory.Measure.smul_apply, MeasureTheory.Measure.dirac_apply', Set.indicator
 Pi.one_apply, smul_eq_mul]
have : |√(n.factorial : ℝ)| = √(n.factorial : ℝ) := by simp
simp_rw [this]
simp only [Pi.single, Function.update, eq_rec_constant, Pi.zero_apply, dite_eq_ite]
mul_one, mul_zero, tsum_ite_eq]
simp_rw [← mul_div]
field_simp
have : √(n.factorial ^ 2) = (n.factorial : ℝ) := by
 refine Real.sq_sqrt ?_
 simp
simp_rw [this]
congr
rw [max_eq_left]
· have (a b : ℝ) (h : a = b) : a / n.factorial = b / n.factorial := by
 rw [h]
 apply this
norm_cast
have (a b c d : ℝ) (hab : a = b) (hcd : c = d) :
 a * c = b * d := by rw [hab, hcd]
apply this
· norm_cast
 have : ||α||^2 ≥ 0 := by simp
 have (r : ℝ) :
 Complex.exp (-(r/2)) = Real.exp (-(r/2)) := by simp
 rw [this]
 have (r : ℝ) : ||Complex.ofReal (Real.exp r)|| = Real.exp r := by simp
 rw [this]
 have h₀ (r : ℝ) : (Real.exp r) ^ (2:ℝ) = Real.exp (r * 2) :=
 Eq.symm (Real.exp_mul r 2)
 have h₁ (r : ℝ) : (Real.exp r) ^ (2:ℝ) = Real.exp r ^ 2 :=
 Real.rpow_two (Real.exp r)
 simp_rw [h₁] at h₀
 rw [h₀]
 congr
 simp only [neg_mul, isUnit_iff_ne_zero, ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true]
 IsUnit.div_mul_cancel, neg_inj]
 norm_cast
· rw [pow_right_comm]
 congr
· apply div_nonneg
 · apply mul_nonneg
 · apply Real.exp_nonneg
 · simp
 · simp

```

```

/-- The only eigenvectors of `a` are the coherent states. -
/
lemma coherentState_only_eigenvector ($\alpha : \mathbb{C}$) ($v : \mathbb{N} \rightarrow \mathbb{C}$) :
 a v = $\alpha \cdot v \leftrightarrow$
 v = (Complex.exp ($\uparrow \| \alpha \|$ ^ 2 / 2) * v 0) • coherentState α := by
 constructor
 intro hv
 ext n
 induction n with
 | zero =>
 unfold coherentState
 simp only [Complex.ofReal_exp, Complex.ofReal_div, Complex.ofReal_neg,
 Complex.ofReal_ofNat, Pi.smul_apply, pow_zero, mul_one, Nat.factorial,
 Real.sqrt_one, Complex.ofReal_one, div_one, smul_eq_mul]
 field_simp
 rw [mul_assoc]
 rw [← Complex.exp_add]
 field_simp
 ring_nf
 simp
 | succ n hn =>
 unfold a at hv
 have h₀ := congrFun hv n
 simp only [Pi.smul_apply, smul_eq_mul] at h₀
 have : v (n + 1) = $\alpha \cdot v\ n / \sqrt{n + 1}$:= by
 field_simp
 rw [← h₀, mul_comm]
 rw [this, hn]
 unfold coherentState
 field_simp
 ring_nf
 simp only [one_div, Complex.ofReal_exp, Complex.ofReal_mul, Complex.ofR
 Complex.ofReal_div, Complex.ofReal_neg, Complex.ofReal_one, Complex.o
 Pi.smul_apply, smul_eq_mul]
 rw [pow_add]
 field_simp
 ring_nf
 repeat rw [mul_assoc]
 congr
 field_simp
 ring_nf
 rw [add_comm]
 nth_rw 2 [add_comm]
 have : (n+1).factorial = (n+1) * n.factorial := rfl
 rw [this]

```

1.338.

PHYSLIBALPHA/QUANTUMMECHANICS/QUANTUMHARMONICOSCILLATOR.lean

```

 norm_num
 field_simp
intro h
rw [h]
unfold a
ext n
have : (Complex.exp (↑||α|| ^ 2 / 2)) ≠ 0 := by simp
field_simp
unfold coherentState
simp only [Complex.ofReal_exp, Complex.ofReal_div, Complex.ofReal_neg, Complex.ofR
 Complex.ofReal_ofNat, Pi.smul_apply, smul_eq_mul]
field_simp
ring_nf
field_simp
rw [mul_assoc, add_comm 1 n, add_comm 1 (n:ℝ)]
have : (n+1).factorial = (n+1) * n.factorial := rfl
rw [this]
norm_num
field_simp

lemma eigenvector_coherentState (α : ℂ) :
 a (coherentState α) = α • coherentState α := by
 rw [coherentState_only_eigenvector α (coherentState α)]
 have : Complex.exp (↑||α|| ^ 2 / 2) * coherentState α 0 = 1 := by
 unfold coherentState
 simp only [Complex.ofReal_exp, Complex.ofReal_div, Complex.ofReal_neg, Complex.o
 Complex.ofReal_ofNat, pow_zero, mul_one, Nat.factorial_zero, Nat.cast_one, Rea
 Complex.ofReal_one, div_one]
 rw [← Complex.exp_add]
 field_simp
 ring_nf
 simp
 rw [this]
 simp

/-- None of the `coherentState` eigenvectors are proportional. -
/
lemma distinct_eigenvectors_a (α β c : ℂ)
 (hc : coherentState α = c • coherentState β) : α = β := by
 have h₀ := congrFun hc 0
 have h₁ := congrFun hc 1
 simp [coherentState] at h₀ h₁
 by_cases hc₀ : c = 0
 · subst c
 simp at h₀
 generalize Complex.exp (-↑||α|| ^ 2 / 2) = A at *
```

```

 subst A
 field_simp at h₁
 exact h₁

/-- Formal eigenvectors for `a_dag` (not in $\square^2(\mathbb{C})$)
(fails at n=0) . -/
lemma a_dagalmost_eigenvector {α : ℂ} (hα : α ≠ 0) {n : ℕ} (hn : n ≠ 0) :
 a_dag (fun n => √(n.factorial) / α^n) n =
 α • (fun n => √(n.factorial) / α^n) n := by
 unfold a_dag
 split_ifs with g₀
 · tauto
 · simp only [smul_eq_mul]
 rw [← Nat.mul_factorial_pred hn]
 norm_num
 have : Complex.ofReal √((n-1).factorial) ≠ 0 := by
 simp only [ne_eq, Complex.ofReal_eq_zero, Nat.cast_nonneg, Real.sqrt_
 Nat.cast_eq_zero]
 exact Nat.factorial_ne_zero (n - 1)
 field_simp
 have : α ^ n = α * α ^ (n - 1) := by
 exact Eq.symm (mul_pow_sub_one hn α)
 rw [this]
 ring_nf

lemma a_dag_nullspace
 {v : ℕ → ℂ} (hv : a_dag v = 0) : v = 0 := by
 unfold a_dag at hv
 ext n
 induction n with
 | zero =>
 have := congrFun hv 1
 simp at this ⊢
 tauto
 | succ n hn =>
 have := congrFun hv (n + 2)
 simp only [Nat.add_eq_zero_iff, OfNat.ofNat_ne_zero, and_false, ↓reduce
 Nat.cast_ofNat, Nat.add_one_sub_one, Pi.zero_apply, mul_eq_zero,
 Complex.ofReal_eq_zero] at this hn hv ⊢
 cases this with
 | inl h =>
 exfalso
 revert h
 refine Real.sqrt_ne_zero'.mpr ?_
 linarith
 | inr h =>

```

1.338.

PHYSLIBALPHA/QUANTUMMECHANICS/QUANTUMHARMONICOSCILLATOR91LEAN

tauto

```
lemma no_a_dag_eigenvector ($\alpha : \mathbb{C}$) (v : $\mathbb{N} \rightarrow \mathbb{C}$) :
 a_dag v = $\alpha \cdot v \leftrightarrow v = 0$:= by
 constructor
 · intro hv
 by_cases h α : $\alpha = 0$
 · subst α ; apply a_dag_nullspace; rw [hv]; simp
 have := congrFun hv 0
 unfold a_dag at this
 ext n
 induction n with
 | zero => simp at this; tauto
 | succ n hn =>
 unfold a_dag at hv
 have := congrFun hv (n+1)
 simp_all
 intro
 subst v
 unfold a_dag
 simp
 rfl
```

```
lemma commutationRelation : {aLin, a_dagLin} = 1 := by
 simp only [Bracket.bracket]
 unfold aLin a_dagLin
 ext x i
 change _ = id x i
 rw [← commutation_relation]
 simp
```

*-- The `coherentState` with parameter `0` is just the first basis vector. -*  
*/*

```
example : coherentState 0 = fun n => ite (n = 0) 1 0 := by
 unfold coherentState
 ext n
 split_ifs with g0
 · subst n; simp
 · rw [zero_pow g0]
 simp
```

*-- The coherent state belongs to  $\ell^2(\mathbb{C})$ . -*  
 def coherentState\_ℓ2 ( $\alpha : \mathbb{C}$ ) : lp (fun \_ :  $\mathbb{N} \Rightarrow \mathbb{C}$ ) 2 := {

```

val := coherentState α
property := by
 simp only [lp, MemEq, OfNat.ofNat_ne_zero, $\downarrow$ reduceIte, ENNReal.ofNat_ne
 ENNReal.toReal_ofNat, Real.rpow_ofNat, AddSubgroup.mem_mk, AddSubmono
 AddSubsemigroup.mem_mk, Set.mem_setOf_eq, coherentState, Complex.ofReal
 Complex.ofReal_div, Complex.ofReal_neg, Complex.ofReal_pow, Complex.ofReal
 Complex.norm_div, Complex.norm_mul, norm_pow, Complex.norm_real, Real
 use (||Complex.exp ($-\uparrow$ || α) ^ 2 / 2)^2 * Complex.exp (|| α ||^2)||)
 suffices HasSum (fun i : $\mathbb{N}$ $\mapsto$ (|| α || ^ i
 / | $\sqrt{i}$.factorial|) ^ 2)
 ||Complex.exp ($\uparrow$ || α) ^ 2)|| by
 simp_rw [div_pow] at *
 simp_rw [mul_pow, $\leftarrow$ mul_div]
 simp only [sq_abs, Nat.cast_nonneg, Real.sq_sqrt, Complex.norm_mul, n
 exact HasSum.const_smul ($\gamma := \mathbb{R}$) _ this
 have (r : $\mathbb{R}$) : | $\sqrt{r}$ | = $\sqrt{r}$:= by
 rw [abs_eq_self]
 simp
 simp_rw [this, div_pow]
 have (i : $\mathbb{N}$) : (|| α || ^ i) ^ 2 = (|| α || ^ 2) ^ i := pow_right_comm || α || i 2
 simp_rw [this]
 rw [$\leftarrow$ Complex.ofReal_pow]
 generalize || α ||^2 = A
 simp only [Nat.cast_nonneg, Real.sq_sqrt, Complex.norm_exp_ofReal]
 convert! NormedSpace.expSeries_hasSum_exp ($\gamma := \mathbb{R}$) A
 · simp [NormedSpace.expSeries]
 field_simp
 · exact Real.exp_eq_exp_ $\mathbb{R}$
}

```

### 1.339 PhyslibAlpha/QuantumMechanics/StinespringDilation.lean

```

/-
Copyright (c) 2026 Bjørn Kjos-Hanssen. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Bjørn Kjos-Hanssen
-/
module
public import Mathlib.Data.Matrix.PEquiv
public import Mathlib.Probability.Distributions.Poisson.Basic
public import Mathlib.Analysis.Normed.Lp.lpSpace
public import Physlib.Meta.TODO.Basic
public import Mathlib
/-!

```

1.339.

PHYSLIBALPHA/QUANTUMMECHANICS/STINESPRINGDILATION.LEAN595

# Stinespring dilation

-/

@[expose] public section

noncomputable section

TODO "There is a different version of the Stienspring dilation in  
`QuantumInfo.Channels.CPTP`. We should unify the the version here with that one.  
Some of the definitions here are more general then the ones in `QuantumInfo` as th  
do not restrict to  $\mathbb{C}$ . This is something we should modify in `QuantumInfo`."

open Matrix MatrixOrder ComplexOrder RCLike TensorProduct Kronecker

/-- Completely positive map given by a (not necessarily minimal) Kraus family. -  
/

def krausApply {R : Type\*} [Mul R] [Star R] [AddCommMonoid R]  
 {q r : Type\*} [Fintype q] [Fintype r]  
 (K : r → Matrix q q R) (ρ : Matrix q q R) : Matrix q q R :=  
 ∑ i, K i \* ρ \* (K i)<sup>H</sup>

/-- Kraus operator preserves PSD property. -/

lemma krausApply.posSemidef {R : Type\*} [Ring R] [PartialOrder R] [StarRing R]  
 [AddLeftMono R]  
 {q r : Type\*} [Fintype q] [Fintype r]  
 (K : r → Matrix q q R)  
 {ρ : Matrix q q R} (hp : ρ.PosSemidef) :  
 (krausApply K ρ).PosSemidef :=  
 posSemidef\_sum \_ fun \_ \_ => hp.mul\_mul\_conjTranspose\_same \_

/-- Quantum channel. -/

def QuantumChannel {R : Type\*} [Mul R] [One R] [Star R] [AddCommMonoid R]  
 {q r : Type\*} [Fintype q] [Fintype r] [DecidableEq q]  
 (K : r → Matrix q q R) :=  
 ∑ i, (K i)<sup>H</sup> \* K i = 1

/-- Quantum operation. -/

def QuantumOperation {R : Type\*} [RCLike R]  
 {q r : Type\*} [Fintype q] [Fintype r] [DecidableEq q]  
 (K : r → Matrix q q R) := ∑ i, (K i)<sup>H</sup> \* K i ≤ 1

/-- Density matrix. -/

def densityMatrix {R : Type\*} [Ring R] [PartialOrder R] [StarRing R] (d : Type\*) [Fi  
 {ρ : Matrix d d R // ρ.PosSemidef ∧ ρ.trace = 1}

/-- Density matrices are closed under real convex combinations. -

/

```

def densityMatrix.convex_comb {R : Type*} [RCLike R]
 {d : ℕ} (ρ₀ ρ₁ : densityMatrix (Fin d) (R := R)) {t : R}
 (hp₀ : 0 ≤ t) (hp₁ : 0 ≤ 1 - t) : densityMatrix (Fin d) (R := R) :=
 (t • ρ₀.1 + (1 - t) • ρ₁.1, by
 constructor
 · exact (ρ₀.2.1.smul hp₀).add (ρ₁.2.1.smul hp₁)
 · rw [trace_add, trace_smul, smul_eq_mul, trace_smul, ρ₀.2.2, ρ₁.2.2]
 simp)

/-- Also known as `partialTraceRight`. -/
def tr₂ {R : Type*} [Ring R] {m n m' : Type*} [Fintype n]
 (ρ : Matrix (m × n) (m' × n) R) : Matrix m m' R :=
 fun i j => ∑ k, ρ (i, k) (j, k)

/-- `stinespringOp` is often written as `V`. -/
def stinespringOp {R : Type*} [Ring R]
 {m r : Type*} [Fintype r] [DecidableEq r]
 (K : r → Matrix m m R) : Matrix (m × r) m R :=
 let V₀ : Matrix (m × r) (m × Fin 1) R :=
 ∑ i, K i ⊗ₖ single i (0 : Fin 1) (1 : R)
 fun x y => V₀ x (y, 0)

/-- The Stinespring dilation. -/
def stinespringDilation {R : Type*} [Ring R] [StarRing R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
 (K : r → Matrix m m R)
 (ρ : Matrix m m R) :=
 let V := stinespringOp K;
 V * ρ * VH

/-- The partial trace of the Stinespring dilation. -/
def stinespringForm {R : Type*} [Ring R] [StarRing R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
 (K : r → Matrix m m R) :=
 fun ρ => tr₂ (stinespringDilation K ρ)

/-- A useful identity for Stinespring dilations. -/
lemma stinespringOp_adjoint_mul_self {R : Type*} [Ring R] [StarRing R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
 (K : r → Matrix m m R) :
 ∑ i, star K i * K i = (stinespringOp K)H * stinespringOp K := by
 ext i j
 unfold stinespringOp
 rw [Matrix.mul_apply]
 rw [Matrix.sum_apply]
 simp only [Pi.star_apply, Matrix.mul_apply, star_apply, single, Fin.isVal

```



1.339.

PHYSLIBALPHA/QUANTUMMECHANICS/STINESPRINGDILATION.LEAN597

```

 kroneckerMap_apply, of_apply, and_true, mul_ite, mul_one, mul_zero, Finset.sum_i
 Finset.mem_univ, ↓reduceIte, conjTranspose_apply];
 erw [Finset.sum_product, Finset.sum_comm]

/-- A useful identity for completely positive, trace non-
increasing maps. -/
lemma stinespringForm_CPTNI {R : Type*} [RCLike R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m] [DecidableEq m]
 (K : r → Matrix m m R)
 (hK : $\sum i, (K i)^H * K i \leq 1$) :
 (stinespringOp K)^H * (stinespringOp K) ≤ 1 := by
 convert hK
 rw [← stinespringOp_adjoint_mul_self]
 rfl

/-- The Stinespring operator of a completely positive, trace-
preserving maps
is an isometry. (Note that `stinespringOp K` is not a square matrix in general.) -
/
lemma stinespringForm_CPTP_isometry {R : Type*} [Ring R] [StarRing R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m] [DecidableEq m]
 {K : r → Matrix m m R}
 (hK : $\sum i, (K i)^H * K i = 1$) :
 (stinespringOp K)^H * (stinespringOp K) = 1 := by
 rw [← hK]
 rw [← stinespringOp_adjoint_mul_self]
 rfl

/--
Proving the columns of `V = stinespringOp K` are independent is a step
on the way to constructing the unitary dilation. -/
lemma stinespringOrtho {R : Type*} [RCLike R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m] [DecidableEq m]
 {K : r → Matrix m m R}
 (hK : $\sum i, (K i)^H * K i = 1$) :
 Orthonormal (□ := R)
 fun j : m => WithLp.toLp 2 fun i : m × r => stinespringOp K i j := by
 refine orthonormal_iff_ite.mpr ?_
 intro i j
 have h₁ : (((stinespringOp K)^H * stinespringOp K) i j)
 = ((1 : Matrix m m R) i j) := by
 rw [stinespringForm_CPTP_isometry hK]
 rw [mul_apply] at h₁
 by_cases g₀ : i = j
 · subst i
 simp only [conjTranspose_apply, star_def, one_apply_eq] at h₁

```

```

 rw [← h₁]
 simp only [inner_self_eq_norm_sq_to_K]
 generalize stinespringOp K = α
 simp only [↓reduceIte]
 simp_rw [RCLike.conj_mul]
 norm_cast
 exact EuclideanSpace.norm_sq_eq (WithLp.toLp 2 fun i ↦ α i j)
 · rw [if_neg g₀]
 have : (1 : Matrix m m R) i j = 0 := by
 exact one_apply_ne' fun a ↦ g₀ (id (Eq.symm a))
 rw [this] at h₁
 rw [← h₁]
 simp only [inner, conjTranspose_apply, star_def]
 congr
 ext x
 ring_nf

/-- `m` will of course be finite and bounded by `n` here,
but no need to assume or prove that.
-/
lemma basisCard {R : Type*} [RCLike R] {n m : Type*} [Fintype n] {s : Matrix m m R}
(ho : Orthonormal R fun j ↦ WithLp.toLp 2 fun i ↦ s i j) :
 Fintype.card n =
 ho.toSubtypeRange.exists_orthonormalBasis_extension.choose.card :=
 Fintype.card_coe _ ▸ (Nat.cast_inj.mp <|
 (rank_eq_card_basis <| PiLp.basisFun _ _ _).symm.trans <|
 rank_eq_card_basis
 ho.toSubtypeRange.exists_orthonormalBasis_extension.choose_spec.choose.

/-- Calculating the cardinality of the orthonormal basis obtained by
extending the Stinespring orthonormal set, the columns of `stinespringOp K`
/
lemma stinespringCard {R : Type*} [RCLike R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m] [DecidableEq m]
 {K : r → Matrix m m R}
 (hK : ∑ i, (K i)H * K i = 1) :
 Fintype.card (m × r) = (stinespringOrtho
 hK).toSubtypeRange.exists_orthonormalBasis_extension.choose.card :=
 basisCard <| stinespringOrtho hK

open Finset in
/-- We need the 1 matrix, which we don't seem to have for an arbitrary
`[Fintype m]`.
Since we are comparing `Fin r` and `Fin (r-1)` we also cannot too

```

1.339.

PHYSLIBALPHA/QUANTUMMECHANICS/STINESPRINGDILATION.LEAN599

easily use an arbitrary `[Fintype r] [Zero r]`.

-/

```

theorem complCard {R : Type*} [RCLike R] {m r : ℕ}
 {K : Fin r → Matrix (Fin m) (Fin m) R}
 (hK : $\sum i, (K i)^H * K i = 1$) (z : Fin r) :
 let $\square$:= fun j ↦ WithLp.toLp 2 fun i ↦ stinespringOp K i j;
 let theRange := Submodule.span R (Set.range $\square$);
 let u' := (exists_orthonormalBasis R theRange $\square$).choose;
 Fintype.card (Fin m × Fin (r-1)) = #u' := by
 obtain (s,hs) : $\exists s, r = s + 1$:= by
 refine Nat.exists_eq_succ_of_ne_zero ?_
 have := z.2
 omega
 subst hs
 intro $\square$ theRange u'
 let u : Finset theRange :=
 (Set.range fun i => ($\square$ i, Submodule.subset_span (i, rfl))))).toFinset
 have hind := (stinespringOrtho hK).linearIndependent
 have hinj := hind.injective
 have h0 : #u = m := by
 simp only [u, Set.toFinset_range]
 have : m = #(Finset.univ : Finset (Fin m)) := by simp
 simp_rw [this]
 apply card_image_of_injective
 intro i j h
 apply hinj
 simp only [Subtype.mk.injEq, WithLp.toLp.injEq, $\square$] at h ⊢
 exact h
 have h1 : m * (s + 1) = #u + #u' := by
 have := Submodule.finrank_add_finrank_orthogonal theRange
 simp only [finrank_euclideanSpace, Fintype.card_prod,
 Fintype.card_fin] at this
 rw [← this]
 congr
 all_goals apply Module.finrank_eq_card_finset_basis
 · simp only [theRange, u, Set.toFinset_range, mem_image, mem_univ, true_and]
 apply (Module.Basis.span hind).reindex
 apply Equiv.ofBijective
 fun i => ($\square$ i, Submodule.mem_span_of_mem <| by simp [$\square$]), by use i)
 constructor
 · intro i j h
 exact hinj (by aesop)
 · intro x
 have (a,ha) := x.2
 use a
 simp_rw [ha]

```

```

 · exact (exists_orthonormalBasis R _).choose_spec.choose.toBasis
simp only [add_tsub_cancel_right, Fintype.card_prod, Fintype.card_fin]
linarith

/--
See discussion at https://leanprover.zulipchat.com/#narrow/channel/217875-Is-there-code-for-X.3F/topic/succAbove.20and.20predAbove.20lemmas/with/5842
-/
def Fin.predAbove_of_ne {n : ℕ} {k i : Fin n}
 (h : i ≠ k) : Fin (n - 1) := by
 by_cases H : i.1 > k.1
 · exact ⟨i.1 - 1, by omega⟩
 · exact ⟨i.1, by omega⟩

/-- A "missing lemma" for `Fin` types. -/
lemma Fin.predAbove_of_ne_injective (n : ℕ) (k x y : Fin n)
 (hx : x ≠ k) (hy : y ≠ k)
 (heq : Fin.predAbove_of_ne hx = Fin.predAbove_of_ne hy) : x = y := by
 unfold predAbove_of_ne at heq
 split_ifs at heq
 all_goals
 · simp only [mk.injEq] at heq
 omega

/-- The way this is written, `Fin r` and `Fin (r-1)` both occur
so it is tricky to go to a general `Fintype`.
-/
def onbPart {R : Type*} [RCLike R]
 {m r : ℕ} {K : Fin r → Matrix (Fin m) (Fin m) R}
 (hK : $\sum i, (K i)^H * K i = 1$) (x : Fin m × Fin r) {z : Fin r} (hx : $\neg x.2 = 0$)
 -- if we make it `r+2` then the ` $x.2 \neq 0$ ` becomes unused.
 Fin m × Fin r → R := by
 let theRange := Submodule.span R <| Set.range
 fun j => WithLp.toLp 2 fun i => stinespringOp K i j
 have (w : Fin m × Fin (r-1)) :=
 ((exists_orthonormalBasis R theRange []).choose_spec.choose
 (Finset.equivOfCardEq (complCard hK z) ⟨w, Finset.mem_univ _⟩)).1.1
 apply this
 exact (x.1, Fin.predAbove_of_ne hx)

/- The custom in quantum information theory is to use
 $|e_1\rangle\langle e_1|$ as ancillary; we allow an arbitrary (standard) basis vector.
-/
lemma onbPart_inner {R : Type*} [RCLike R] {m r : ℕ} {K : Fin r → Matrix (F
 (hK : $\sum i, (K i)^H * K i = 1$) {z : Fin r}

```

1.339.

PHYSLIBALPHA/QUANTUMMECHANICS/STINESPRINGDILATION. LEAN 601

```

{y : Fin m × Fin r} (hy : ¬y.2 = z)
{x : Fin m × Fin r} (hx : ¬x.2 = z)
(h : y ≠ x) :
inner R (WithLp.toLp 2 <| onbPart hK y hy)
 (WithLp.toLp 2 <| onbPart hK x hx) = 0 := by
let theRange := Submodule.span R <| Set.range
 fun j => WithLp.toLp 2 fun i => stinespringOp K i j
let α := (exists_orthonormalBasis R theRange[]).choose_spec
have := α.choose_orthonormal.2
simp only [Pairwise, ne_eq, Submodule.coe_inner, Subtype.forall,
 Subtype.mk.injEq] at this
have h₁ := this (WithLp.toLp 2 <| onbPart hK y hy)
 (by simp [onbPart]) (by
 simp only [onbPart, WithLp.toLp_ofLp, Subtype.coe_eta]
 rw [α.choose_spec]
 simp)
 (WithLp.toLp 2 <| onbPart hK x hx)
 (by simp [onbPart]) (by
 simp only [onbPart, WithLp.toLp_ofLp, Subtype.coe_eta]
 rw [α.choose_spec]
 simp) (by
 simp only [onbPart, WithLp.toLp_ofLp, SetLike.coe_eq_coe]
 rw [α.choose_spec]
 simp only [SetLike.coe_eq_coe, EmbeddingLike.apply_eq_iff_eq,
 Subtype.mk.injEq, Prod.mk.injEq, not_and]
 intro hyz
 contrapose! h
 have : y.2.1 ≠ z := Fin.val_ne_of_ne hy
 have : x.2.1 ≠ z := Fin.val_ne_of_ne hx
 have : y.2.1 = x.2.1 := by
 suffices y.2 = x.2 by rw [this]
 apply Fin.predAbove_of_ne_injective
 omega
 have : y.2 = x.2 := by omega
 exact Prod.ext hyz this)
rw [← h₁]
simp_rw [α.choose_spec]

/-- The vectors in the Stinespring orthonormal basis have norm 1. -
/
lemma onbPart_norm {R : Type*} [RCLike R] {m r : ℕ} {K : Fin r → Matrix (Fin m) (Fin
 (hK : ∑ i, (K i)ᐃ * K i = 1) (x : Fin m × Fin r)
 {z : Fin r} (hx : ¬x.2 = z) :
 ||WithLp.toLp 2 <| onbPart hK x hx|| = 1 :=
let theRange := Submodule.span R <| Set.range
 fun j => WithLp.toLp 2 fun i => stinespringOp K i j

```

```

(exists_orthonormalBasis R theRange[]).choose_spec.choose.orthonormal.1 _

/-- Also known as `unitaryDilation`. Respects x,y order. -
/
def Ud {R : Type*} [RCLike R] {m r : ℕ}
 {K : Fin r → Matrix (Fin m) (Fin m) R}
 (hK : $\sum i, (K i)^H * K i = 1$) (z : Fin r) :
 Matrix (Fin m × Fin r) (Fin m × Fin r) R := by
 intro x y
 by_cases hy : y.2 = z
 · exact stinespringOp K x y.1
 · exact onbPart hK y hy x

/-- This generalization of Stinespring dilation has the right
"shape" but otherwise nothing specific to it. -/
def general_dilation {R : Type*}
 {m r : Type*} [DecidableEq r]
 (z : r)
 (S : Matrix (m × r) m R)
 (M : Matrix (m × r) (m × r) R) :
 Matrix (m × r) (m × r) R := fun x y =>
 ite (y.2 = z) (S x y.1) (M x y)

/-- A general, not necessarily unitary, dilation. -/
def dilation {R : Type*} [Ring R]
 {m r : Type*} [Fintype r] [DecidableEq r]
 (K : r → Matrix m m R) (z : r) (M : Matrix (m × r) (m × r) R) :
 Matrix (m × r) (m × r) R := general_dilation z (stinespringOp K) (M)

/-- One version of orthonormality of `stinespringOp`. -/
theorem Ud_orthonormal₁ {R : Type*} [RCLike R] {m r : ℕ} {K : Fin r → Matrix
 (hK : $\sum i, (K i)^H * K i = 1$) (z : Fin r) :
 Orthonormal R fun y ↦ if hy : y.2 = z then WithLp.toLp 2 fun i ↦ stines
 else WithLp.toLp 2 fun i ↦ onbPart hK y hy i := by
 constructor
 · intro i
 simp only
 split_ifs with g₀
 · apply (stinespringOrtho hK).1
 · apply onbPart_norm

```

1.339.

PHYSLIBALPHA/QUANTUMMECHANICS/STINESPRINGDILATION.LEAN603

```

· intro i j h
 simp only
 let theRange := Submodule.span R <| Set.range
 fun j => WithLp.toLp 2 fun i ↦ stinespringOp K i j
 split_ifs with g₀ g₁ g₂
· apply (stinespringOrtho hK).2
 contrapose! h
 refine Prod.ext_iff.mpr ?_
 constructor
 · tauto
 · rw [g₀, g₁]
· -- use that they came from `theRange`, `theRange[]` respectively.
 have h₀ : (WithLp.toLp 2 fun i₁ ↦ stinespringOp K i₁ i.1) ∈ theRange := by
 unfold theRange
 generalize stinespringOp K = α
 apply Submodule.mem_span_of_mem
 simp
 have h₁ : (WithLp.toLp 2 fun i ↦ onbPart hK j g₁ i) ∈ theRange[] := by
 unfold theRange
 simp [onbPart]
 exact h₁ _ h₀
· have h₀' : (WithLp.toLp 2 fun i₁ ↦ stinespringOp K i₁ j.1) ∈ theRange := by
 unfold theRange
 generalize stinespringOp K = α
 apply Submodule.mem_span_of_mem
 simp
 have h₁ : (WithLp.toLp 2 fun t ↦ onbPart hK i g₀ t) ∈ theRange[] := by
 unfold theRange
 simp [onbPart]
 have := h₁ _ h₀'
 generalize (WithLp.toLp 2 fun i₁ ↦ onbPart hK i g₀ i₁) = α at *
 generalize (WithLp.toLp 2 fun i ↦ stinespringOp K i j.1) = β at *
 exact inner_eq_zero_symm.mp (h₁ β h₀')
· exact onbPart_inner hK g₀ g₂ h

/-- The Stinespring dilation columns form an orthonormal basis. -
/
theorem Ud_orthonormal₂ {R : Type*} [RCLike R]
 {m r : ℕ} {K : Fin r → Matrix (Fin m) (Fin m) R}
 (hK : ∑ i, (K i)ᵀ * K i = 1) (z : Fin r) :
 Orthonormal R fun y ↦
 WithLp.toLp 2 fun i ↦ if hy : y.2 = z then stinespringOp K i y.1 else onbPart
 have h₀ := Ud_orthonormal₁ hK z
 constructor
 · intro i

```

```

 have := h₀.1 i
 rw [← this]
 congr
 ext y
 simp only
 split_ifs with g₀ <=> simp
 · intro _ _ hij
 have := h₀.2 hij
 simp only at this ⊢
 rw [← this]
 split_ifs at * with g₀ g₁ <=> rfl

/-- If `β` has length 1 then the dot product of `β` with itself is 1. -
/
lemma smul_self_one_of_norm_one {R : Type*} [RCLike R]
 {t : Type*} [Fintype t] {β : t → R} (hj : ||WithLp.toLp 2 β|| = 1) :
 ∑ x, (starRingEnd R) (β x) * β x = 1 := by
 refine Eq.symm ((fun {z w} ↦ RCLike.ext_iff.mpr) ?_)
 constructor
 · simp only [one_re, map_sum, mul_re, conj_re, conj_im, neg_mul, sub_
 rw [← one_pow 2]
 rw [← hj]
 simp_rw [← RCLike.norm_sq_eq_def]
 exact EuclideanSpace.norm_sq_eq (WithLp.toLp 2 β)
 · simp only [one_im, map_sum, mul_im, conj_re, conj_im, neg_mul]
 symm
 apply Fintype.sum_eq_zero
 ring_nf
 simp

/-- A matrix whose columns are orthonormal is unitary. -/
theorem unitary_of_orthonormal {R : Type*} [RCLike R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m] [DecidableEq m]
 (α : Matrix (m × r) (m × r) R)
 (h₀ : Orthonormal R fun i ↦ WithLp.toLp 2 (α i)) : α * star α = 1 := by
 ext i j
 rw [mul_apply]
 apply star_injective
 simp only [star_apply, star_def, star_sum, star_mul', RingHomCompTriple
 RingHom.id_apply]
 by_cases H : i = j
 · subst i
 simp only [one_apply_eq, map_one]
 exact smul_self_one_of_norm_one <| h₀.1 _
 · rw [one_apply_ne' <| H ° Eq.symm, map_zero]
 convert h₀.2 H

```



1.339.

PHYSLIBALPHA/QUANTUMMECHANICS/STINESPRINGDILATION.LEAN605

```

 simp only [inner]
 congr
 ext l
 nth_rw 1 [mul_comm]
 rfl

/-- The transpose of the unitary dilation is unitary. -/
lemma Ud_unitaryT {R : Type*} [RCLike R]
 {m r : ℕ} {K : Fin r → Matrix (Fin m) (Fin m) R}
 (hK : ∑ i, (K i)H * K i = 1) (z : Fin r) :
 (Ud hK z)T ∈ unitary _ := by
 have H₀ := unitary_of_orthonormal (Ud hK z)T
 <| Ud_orthonormal₂ hK z
 constructor
 · exact (mul_eq_one_comm_of_card_eq _ _ _ rfl).mp H₀
 · exact H₀

/-- The unitary dilation `Ud` is in fact unitary. -/
lemma Ud_unitary {R : Type*} [RCLike R]
 {m r : ℕ} {K : Fin r → Matrix (Fin m) (Fin m) R}
 (hK : ∑ i, (K i)H * K i = 1) (z : Fin r) :
 (Ud hK z) ∈ unitary _ := by
 have := Ud_unitaryT hK z
 generalize Ud hK z = U at *
 have : star U * U = 1 := by
 have := this.2
 have : (UT * star UT)T = 1T := transpose_inj.mpr this
 simp only [transpose_mul, transpose_transpose, transpose_one] at this
 have : (star UT)T = star U := by
 exact Eq.symm (Matrix.ext fun i ↦ congrFun rfl)
 rw [← this]
 tauto
 constructor
 · exact this
 · exact (mul_eq_one_comm_of_card_eq _ _ _ rfl).mp this

open Kronecker TensorProduct

/-- Taking the partial trace of a tensor product with a matrix of trace 1 is the
identity map. -/
lemma tr₂_e₀Xe₀ {R : Type*} [RCLike R]
 {m w : Type*} [Fintype w] [Zero w]
 (e : Matrix w w R) (htr : e.trace = 1)
 (ρ : Matrix m m R) :
 tr₂ (ρ ⊗ₖ e) = ρ := by

```

```

 unfold tr₂ kroneckerMap
 simp only [of_apply]
 ext i j
 have : $\sum x, \rho_{ij} * e_{xx}$
 = $\rho_{ij} * \sum x, e_{xx}$:= by rw [Finset.mul_sum]
 rw [this]
 unfold trace at htr
 simp only [diag_apply] at htr
 rw [htr]
 simp

/-- The Stinespring unitary form. -/
def stinespringUnitaryForm {R : Type*} [RCLike R] {m r : ℕ}
 {K : Fin r → Matrix (Fin m) (Fin m) R}
 (hK : $\sum i, (K i)^H * K i = 1$) (z : Fin r)
 (ρ : Matrix (Fin m) (Fin m) R) :
 (Matrix (Fin m) (Fin m) R) :=
 let U := Ud hK z
 tr₂ (U * (ρ ⊗ₖ (single z z 1)) * UH)

/-- The Stinespring unitary form, general version. -/
def stinespringUnitaryForm_e {R : Type*} [RCLike R] {m r : ℕ}
 {K : Fin r → Matrix (Fin m) (Fin m) R}
 (hK : $\sum i, (K i)^H * K i = 1$) (z : Fin r) (e : Matrix (Fin r) (Fin r) R)
 (ρ : Matrix (Fin m) (Fin m) R) :
 (Matrix (Fin m) (Fin m) R) :=
 let U := Ud hK z
 tr₂ (U * (ρ ⊗ₖ e) * UH)

/-- Trace-free version of the Stinespring Dilation Theorem. -
/
theorem tracefree_version {R : Type*} [RCLike R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Zero r] [Fintype m] [DecidableEq m]
 {K : r → Matrix m m R}
 (ρ : Matrix m m R) :
 let K' := fun i x y => star <| K i y x; let U := (stinespringOp K');
 UH * (ρ ⊗ₖ (1 : Matrix r r R)) * U = stinespringForm K ρ := by
 -- Since my proof broke in 4.27 -> 4.31, here's Aristotle's proof.
 simp only [stinespringOp, star_def, Fin.isValue, stinespringForm, stinespringForm]
 ext x y
 simp only [Fin.isValue, Matrix.mul_apply, conjTranspose_apply, star_def,
 Matrix.one_apply, mul_ite, mul_one, mul_zero, tr₂]
 ring_nf;
 simp only [Fin.isValue, Matrix.sum_apply, kroneckerMap_apply, map_sum, map_sum,
 RingHomCompTriple.comp_apply, RingHom.id_apply, Fintype.sum_prod_type,
 Finset.mem_univ, ↓reduceIte];

```

1.339.

PHYSLIBALPHA/QUANTUMMECHANICS/STINESPRINGDILATION.LEAN607

```
simp only [single, Fin.isValue, of_apply, and_true, MonoidWithZeroHom.map_ite_one_,
mul_one, mul_zero, Finset.sum_ite_eq', Finset.mem_univ, ↓reduceIte];
exact Finset.sum_comm

/-- A Heisberg picture / Schrödinger picture view of the Stinespring dilation. -
/
theorem heisenberg_schrödinger {R : Type*} [RCLike R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Zero r] [Fintype m] [DecidableEq m]
 {K : r → Matrix m m R}
 (ρ : Matrix m m R) :
let K' := fun i x y => star <| K i y x
let U := (stinespringOp K'); let V := stinespringOp K
let schrödinger := tr₂ (V * ρ * VH); -- evolve the state forward: V = V(t), ρ = ρ(t)
let heisenberg := UH * (ρ ⊗k (1 : Matrix r r R)) * U;
-- ρ ⊗k 1 is an "observable"; evolve it backward
schrödinger = heisenberg := by
intro K' U
rw [tracefree_version]
rfl

/-- A further generalization of `stinespringGeneralForm`. -
/
def generalForm {R : Type*} [RCLike R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
 (z : r)
 (S : Matrix (m × r) m R)
 (M : Matrix (m × r) (m × r) R) :=
let U := general_dilation z S M
fun ρ => tr₂ (U * (ρ ⊗k (single z z 1)) * UH)

/-- General form of the Stinespring dilation. -/
def stinespringGeneralForm {R : Type*} [RCLike R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
 (K : r → Matrix m m R) (z : r)
 (M : Matrix (m × r) (m × r) R) :=
let U := dilation K z M
fun ρ => tr₂ (U * (ρ ⊗k (single z z 1)) * UH)

/-- Even more general form of the Stinespring dilation. -
/
def stinespringGeneralForm_e {R : Type*} [RCLike R]
 {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
 (K : r → Matrix m m R) (z : r) (e : Matrix r r R)
 (M : Matrix (m × r) (m × r) R) :=
let U := dilation K z M
fun ρ => tr₂ (U * (ρ ⊗k e) * UH)
```

```

/-- When we plug in `M = Ud hK`
into the general `stinespringGeneralForm`,
then we do get
`stinespringUnitaryForm hK`.
-/
theorem unitaryForm_of_general {R : Type*} [RCLike R] {m r : ℕ}
 {K : Fin r → Matrix (Fin m) (Fin m) R}
 (hK : $\sum i, (K i)^H * K i = 1$) (z : Fin r) :
 stinespringGeneralForm K z (Ud hK z) =
 stinespringUnitaryForm hK z := by
 unfold
 stinespringUnitaryForm tr₂ Ud
 stinespringGeneralForm dilation general_dilation tr₂
 ext a b
 congr
 ext c
 repeat rw [mul_apply]
 repeat rw [Fintype.sum_prod_type]
 congr
 ext d
 congr
 ext e
 repeat rw [mul_apply]
 simp only [kroneckerMap_apply, ite_mul, dite_mul,
 conjTranspose_apply, star_def]
 repeat rw [Fintype.sum_prod_type]
 congr
 · ext f
 congr
 ext g
 simp only [ite_eq_right_iff, left_eq_dite_iff, mul_eq_mul_right_iff, mul_eq_mul_left_iff]
 intro hg
 subst g
 intro h
 simp at h ⊢
 · split_ifs with g₀ <=> rfl

/-- The Stinespring unitary form as a general form applied to the unitary d
/
theorem unitaryForm_of_general_e {R : Type*} [RCLike R] {m r : ℕ}
 {K : Fin r → Matrix (Fin m) (Fin m) R}
 (hK : $\sum i, (K i)^H * K i = 1$) (z : Fin r) (e : Matrix (Fin r) (Fin r) R)
 stinespringGeneralForm_e K z e (Ud hK z) =
 stinespringUnitaryForm_e hK z e := by

```

1.339.

PHYSLIBALPHA/QUANTUMMECHANICS/STINESPRINGDILATION.LEAN609

```
unfold
 stinespringUnitaryForm_e tr₂ Ud
 stinespringGeneralForm_e dilation general_dilation tr₂
ext a b
congr
ext c
repeat rw [mul_apply]
repeat rw [Fintype.sum_prod_type]
congr
ext d
congr
ext e
repeat rw [mul_apply]
simp only [kroneckerMap_apply, ite_mul, dite_mul,
 conjTranspose_apply, star_def]
repeat rw [Fintype.sum_prod_type]
congr
· ext f
 congr
 ext g
 simp only [ite_eq_right_iff, left_eq_dite_iff, mul_eq_mul_right_iff, mul_eq_zero]
 intro hg
 subst g
 intro h
 simp at h ⊢
· split_ifs with g₀ <=> rfl
```

/--

Note we don't need any special properties of M,  
and we don't need K to be CPTP.

Uses `Fin` types because of the use of  
`Fin.sum\_univ\_succAbove` in the proof.

-/

```
lemma stinespringGeneralForm_works {R : Type*} [RCLike R] {m r : ℕ}
 (K : Fin r → Matrix (Fin m) (Fin m) R) (z : Fin r)
 (M : Matrix (Fin m × Fin r) (Fin m × Fin r) R) :
 stinespringGeneralForm K z M = krausApply K := by
 -- my 4.27 proof failed in 4.31 so this is Aristotle:
 unfold stinespringGeneralForm krausApply dilation general_dilation stinespring
 ext p i j;
 simp only [Fin.isValue, Matrix.sum_apply, kroneckerMap_apply, Matrix.mul_apply,
 conjTranspose_apply, star_def];
 simp only [single, Fin.isValue, of_apply, and_true, mul_ite, mul_one, mul_zero,
 Finset.sum_ite_eq', Finset.mem_univ, ↓reduceIte, Finset.sum_ite, not_and,
```

```

 Finset.sum_const_zero, add_zero];
 refine Finset.sum_congr rfl fun x _ => ?_
 rw [← Finset.sum_subset
 (Finset.subset_univ (Finset.image (fun y : Fin m => (y, z)) Fins
 · rw [Finset.sum_image]
 · simp only [and_true, Finset.sum_filter, ite_not, ↓reduceIte];
 refine Finset.sum_congr rfl fun y _ => ?_
 erw [Finset.sum_product, Finset.sum_product]
 simp [Finset.sum_ite, Finset.filter_eq', Finset.filter_ne'];
 · simp only [Finset.coe_univ, Set.injOn_univ];
 exact fun a b h => by injection h;
 · aesop

/--
Notice that unitarity is a side property, it is not why
the Stinespring form works.

Here `z` is the coordinate used for the ancilla.
-/
lemma stinespringUnitaryForm_works {R : Type*} [RCLike R] {m r : ℕ}
 {K : Fin r → Matrix (Fin m) (Fin m) R}
 (hK : $\sum i, (K i)^H * K i = 1$) (z : Fin r) :
 stinespringUnitaryForm hK z = krausApply K := by
 rw [← stinespringGeneralForm_works K z (Ud hK z)]
 rw [unitaryForm_of_general]

/-- The "orthogonal" CPTP completion of a CPTNI map.
`Vtilde` is an alternative name for `krausCompletion`.
-/
def krausCompletion {R : Type*} [RCLike R] {m r : ℕ}
 (K : Fin r → Matrix (Fin m) (Fin m) R) :
 Matrix (Fin m × Fin (r+1)) (Fin m) R := fun x => dite (x.2 < r)
 (fun H => stinespringOp K ⟨x.1, ⟨x.2, H⟩⟩)
 fun _ => (CFC.sqrt (1 - (stinespringOp K)^H * (stinespringOp K)) : Matrix

/-- Entrywise formula for the Stinespring isometry: its `((x₁, x₂), y)` ent
/
theorem stinespringOp_apply {R : Type*} [Ring R] {m r : Type*} [Fintype r]
 [Fintype m] [DecidableEq m] (K : r → Matrix m m R) (x : m × r) (y : m)
 stinespringOp K x y = K x.2 x.1 y := by
 unfold stinespringOp
 simp [Matrix.sum_apply, Matrix.kroneckerMap_apply, Matrix.single_apply]

/-- The Gram matrix of the Stinespring isometry is $\sum i, (K i)^H * K i$. -
/

```

1.339.

PHYSLIBALPHA/QUANTUMMECHANICS/STINESPRINGDILATION. LEAN 611

```
theorem stinespringOp_gram {R : Type*} [RCLike R] {m r : ℕ}
 (K : Fin r → Matrix (Fin m) (Fin m) R) :
 (stinespringOp K)H * stinespringOp K = ∑ i, star (K i) * K i := by
 ext a b
 rw [Matrix.mul_apply, Matrix.sum_apply]
 simp only [Matrix.conjTranspose_apply, stinespringOp_apply]
 rw [← Finset.univ_product_univ, Finset.sum_product, Finset.sum_comm]
 apply Finset.sum_congr rfl
 intro i _
 rw [Matrix.mul_apply]
 apply Finset.sum_congr rfl
 intro x1 _
 simp [Matrix.star_apply]
```

/-- Mar 14, 2026 by Bjørn for 4.27

June 13, 2026 by Aristotle for 4.31 including

`stinespringOp\_gram` and `stinespringOp\_apply`. -/

```
lemma krausCompletion_isometry_of_TNI {R : Type*} [RCLike R] {m r : ℕ}
```

```
 {K : Fin r → Matrix (Fin m) (Fin m) R}
```

```
 (hK : ∑ i, star K i * K i ≤ 1) :
```

```
 (krausCompletion K)H * krausCompletion K = 1 := by
```

```
 set S := stinespringOp K with hS
```

```
 -- `SH * S = ∑ i, (K i)H * K i`, hence `1 - SH * S` is positive semidefinite.
```

```
 have hgram : SH * S = ∑ i, star (K i) * K i := stinespringOp_gram K
```

```
 have h0 : 0 ≤ 1 - SH * S := by rw [hgram]; exact sub_nonneg.mpr hK
```

```
 -- `W` is the (selfadjoint) square root of `1 - SH * S`, completing the isometry.
```

```
 set W := CFC.sqrt (1 - SH * S) with hW
```

```
 have hWsa : WH = W := IsSelfAdjoint.of_nonneg (CFC.sqrt_nonneg _)
```

```
 have hWW : WH * W = 1 - SH * S := by
```

```
 rw [hWsa]; exact CFC.sqrt_mul_sqrt_self _ h0
```

```
 -- The first `r` blocks of the completion are the blocks of `S`.
```

```
 have hcast : ∀ (x : Fin m) (i : Fin r) (c : Fin m),
```

```
 krausCompletion K (x, i.castSucc) c = S (x, i) c := by
```

```
 intro x i c
```

```
 rw [hS]
```

```
 unfold krausCompletion
```

```
 rw [dif_pos (by exact i.isLt)]
```

```
 congr 1
```

```
 -- The last block of the completion is `W`.
```

```
 have hlast : ∀ (x : Fin m) (c : Fin m),
```

```
 krausCompletion K (x, Fin.last r) c = W x c := by
```

```
 intro x c
```

```
 rw [hW, hS]
```

```
 unfold krausCompletion
```

```
 rw [dif_neg (by simp)]
```

```
 -- `CH * C = SH * S + WH * W` by splitting the row sum into the first `r` blocks a
```

```

have key : (krausCompletion K)H * krausCompletion K = SH * S + WH * W :=
 ext a b
 rw [Matrix.mul_apply, Matrix.add_apply, Matrix.mul_apply, Matrix.mul_ap
 simp only [Matrix.conjTranspose_apply]
 have hSsum : (∑ p : Fin m × Fin r, star (S p a) * S p b)
 = ∑ x, ∑ i, star (S (x, i) a) * S (x, i) b := by
 rw [← Finset.univ_product_univ, Finset.sum_product]
 rw [← Finset.univ_product_univ, Finset.sum_product]
 simp only [Fin.sum_univ_castSucc]
 rw [Finset.sum_add_distrib, hSsum]
 congr 1
 · apply Finset.sum_congr rfl; intro x _
 apply Finset.sum_congr rfl; intro i _
 rw [hcast x i a, hcast x i b]
 · apply Finset.sum_congr rfl; intro x _
 rw [hlast x a, hlast x b]
-- Finally `SH * S + WH * W = SH * S + (1 - SH * S) = 1`.
rw [key, hWW]
abel

/-- A unital operator. -/
def unital {R : Type*} [RCLike R] {m r : ℕ}
 (K : Fin r → Matrix (Fin m) (Fin m) R) := ∑ i, K i * star (K i) = 1

/-- A subunital operator. -/
def subunital {R : Type*} [RCLike R] {m r : ℕ}
 (K : Fin r → Matrix (Fin m) (Fin m) R) := ∑ i, K i * star (K i) ≤ 1

/-- The identity `Tr_B (A ⊗ B) = Tr(B) · A` -/
lemma partialTrace_tensor {R : Type*} [RCLike R] {m n : ℕ}
 (A : Matrix (Fin m) (Fin m) R) (B : Matrix (Fin n) (Fin n) R) :
 tr₂ (A ⊗k B) = (trace B) · A := by
 unfold tr₂ trace kroneckerMap
 simp only [of_apply, diag_apply]
 ext i j
 simp only [Matrix.smul_apply, smul_eq_mul]
 have := @Finset.sum_mul (a := A i j) (ι := Fin n)
 (s := Finset.univ) (f := fun k => B k k) _ _
 rw [this]
 simp_rw [mul_comm]

/-- A unitary dilation view of the application of a Kraus operator. -
/
lemma krausApply_of_tensor {R : Type*} [RCLike R] {m r : ℕ}

```



1.339.

PHYSLIBALPHA/QUANTUMMECHANICS/STINESPRINGDILATION. LEAN 613

```

 {K : Fin r → Matrix (Fin m) (Fin m) R}
 (hK : $\sum i, (K\ i)^H * K\ i = 1$) (z : Fin r)
 (p α : Matrix (Fin m) (Fin m) R) (β : Matrix (Fin r) (Fin r) R)
 (hβ : β.trace = 1) (h : (Ud hK z) * (p ⊗k (single z z 1)) * (Ud hK z)H = α ⊗k β)
 krausApply K p = α := by
 rw [← stinespringUnitaryForm_works hK]
 · unfold stinespringUnitaryForm
 simp only
 rw [h]
 rw [partialTrace_tensor]
 rw [hβ]
 simp

/-- Trace of partial trace equals trace. -/
lemma trace_tr2 {R : Type*} [RCLike R] {m n : ℕ}
 (p : Matrix (Fin m × Fin n) (Fin m × Fin n) R) :
 trace p = trace (tr2 p) := Fintype.sum_prod_type fun x ↦ p x x

/-- The Kraus completion as a map from
operations to channels. -/
def krausCompletionChannelMap {R : Type*} [RCLike R] {q r : ℕ}
 {K : Fin r → Matrix (Fin q) (Fin q) R} (hK : QuantumOperation K) :
 {K : Fin (r+1) → Matrix (Fin q) (Fin q) R | QuantumChannel K} := by
 constructor
 swap
 · exact fun i x => krausCompletion K (x, i)
 · unfold QuantumChannel
 rw [← krausCompletion_isometry_of_TNI hK]
 ext x y
 rw [mul_apply, Fintype.sum_prod_type, Finset.sum_comm, Matrix.sum_apply]
 congr

/-- The "not orthogonal" CPTP completion of a CPTNI map. -
/
lemma CPTP_of_CPTNI {R : Type*} [RCLike R]
 {q r : ℕ}
 {K : Fin r → Matrix (Fin q) (Fin q) R}
 (hq : QuantumOperation K) :
 ∃ K' : Fin (r+1) → Matrix (Fin q) (Fin q) R,
 QuantumChannel K' ∧
 ∀ i, ∀ H : i ≠ Fin.last r, K' i = K {i.1, Fin.val_lt_last H} := by
 use (fun i x => krausCompletion K (x, i))
 constructor
 · exact (krausCompletionChannelMap hq).2

```

```

· unfold krausCompletion stinespringOp
 simp only [ne_eq, Fin.isValue]
 intro i H
 split_ifs with g₀
 · unfold kroneckerMap single
 simp only [Fin.isValue, of_apply, mul_ite, mul_one, mul_zero]
 have (j : Fin q × Fin l) : (0 = j.2) = True := by
 have := j.2.2
 simp only [Fin.isValue, eq_iff_iff, iff_true]
 omega
 simp_rw [this]
 ext a b
 erw [Finset.sum_apply]
 erw [Finset.sum_fn]
 simp
 · exact False.elim <| H <| Fin.eq_last_of_not_lt g₀

/-- Partial trace on the left of a tensor product. -/
def partialTraceLeft {R : Type*} [RCLike R]
 {m n : Type*} [Fintype m]
 (ρ : Matrix (m × n)
 (m × n) R) : Matrix (n) (n) R :=
fun i j => ∑ k : m, ρ (k, i) (k, j)

/-- A version of the Stinespring Dilation Theorem. -/
theorem stinespringForm_eq {R : Type*} [RCLike R] {m r : ℕ}
 (K : Fin r → Matrix (Fin m) (Fin m) R)
 (ρ : Matrix (Fin m) (Fin m) R) :
 tr₂ (stinespringDilation K ρ) = krausApply K ρ := by
 unfold tr₂ stinespringDilation krausApply
 ext i j
 simp only [stinespringOp, Fin.isValue, Matrix.mul_apply, conjTranspose_ap
 simp [Matrix.mul_apply, Finset.sum_mul, Matrix.sum_apply, kroneckerMap_ap
 Matrix.single]

```

### 1.340 PhyslibAlpha/SpaceAndTime/Space/Surfaces/HalfPlane.lean

```

/-
Copyright (c) 2026 Robert Sneiderman. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Robert Sneiderman
-/
module

```

1.340.

PHYSLIBALPHA/SPACEANDTIME/SPACE/SURFACES/HALFPLANE.LEAN15

```
public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.Line
/-!
```

```
Half-plane surface in `Space 3`
```

The half-plane is the coordinate plane in `Space 3` with nonnegative second coordinate.

```
-/
```

```
@[expose] public section
open SchwartzMap NNReal
noncomputable section
open Physlib Distribution
variable (α : Type) {E F F' : Type} [RCLike α] [NormedAddCommGroup E] [NormedAddCommGroup F]
 [NormedAddCommGroup F'] [NormedSpace ℝ E] [NormedSpace ℝ F]
```

```
namespace Space
```

```
open MeasureTheory Real
```

```
/-!
```

```
A. The definition of the half-plane surface
```

```
-/
```

```
/-- The domain of the half-plane inside `Space 2`. -/
def halfPlaneDomain : Set (Space 2) := {x | 0 ≤ x (1 : Fin 2)}
```

```
/-- The coordinate plane embedding used for the half-plane surface in `Space 3`. -
/
def halfPlane : Space 2 → Space 3 := fun x => (slice (2 : Fin 3)).symm (0, x)
```

```
lemma halfPlane_eq : halfPlane = (slice (2 : Fin 3)).symm ∘ (fun x : Space 2 => (0,
```

```
lemma halfPlane_injective : Function.Injective halfPlane := by
 intro x y h
 have h' := congrArg (slice (2 : Fin 3)) h
 simp [halfPlane] at h'
 exact h'
```

```
@[fun_prop]
lemma halfPlane_continuous : Continuous halfPlane := by
 rw [halfPlane_eq]
 fun_prop
```

```

lemma halfPlane_measurableEmbedding : MeasurableEmbedding halfPlane :=
 Continuous.measurableEmbedding halfPlane_continuous halfPlane_injective

@[simp]
lemma norm_halfPlane (x : Space 2) : ||halfPlane x|| = ||x|| := by
 rw [halfPlane, norm_slice_symm_eq]
 simp

/-!

B. The measure associated with the half-plane

-/

/-- The measure on `Space 3` corresponding to integration over a half-
plane. -/
def halfPlaneMeasure : Measure (Space 3) :=
 MeasureTheory.Measure.map halfPlane (volume.restrict halfPlaneDomain)

instance halfPlaneMeasure_hasTemperateGrowth :
 halfPlaneMeasure.HasTemperateGrowth := by
 rw [halfPlaneMeasure]
 refine { exists_integrable := ?_ }
 obtain ⟨n, hn⟩ := MeasureTheory.Measure.HasTemperateGrowth.exists_integra
 (μ := volume (α := Space 2))
 use n
 rw [MeasurableEmbedding.integrable_map_iff halfPlane_measurableEmbedding]
 convert! hn.restrict using 1
 ext x
 simp [norm_halfPlane]

instance halfPlaneMeasure_sFinite : SFinite halfPlaneMeasure := by
 rw [halfPlaneMeasure]
 exact Measure.instSFiniteMap (volume.restrict halfPlaneDomain) halfPlane

/-!

C. The distribution associated with the half-plane

-/

/-- The distribution on `Space 3` corresponding to integration over a half-
plane. -/
def halfPlaneDist : (Space 3) →d[ℝ] ℝ :=
 SchwartzMap.integralCLM ℝ halfPlaneMeasure

```

1.340.

PHYSLIBALPHA/SPACEANDTIME/SPACE/SURFACES/HALFPLANE.LEAN17

```

lemma halfPlaneDist_apply_eq_integral_halfPlaneMeasure (f : $\square$ (Space 3, $\mathbb{R}$)) :
 halfPlaneDist f = $\int$ x, f x ∂ halfPlaneMeasure := by
 rw [halfPlaneDist, SchwartzMap.integralCLM_apply]

lemma halfPlaneDist_apply_eq_integral_volume (f : $\square$ (Space 3, $\mathbb{R}$)) :
 halfPlaneDist f = $\int$ x, f (halfPlane x) ∂ (volume.restrict halfPlaneDomain) := by
 rw [halfPlaneDist_apply_eq_integral_halfPlaneMeasure, halfPlaneMeasure,
 MeasurableEmbedding.integral_map halfPlane_measurableEmbedding]

/-!

D. The half-plane has ambient volume zero

-/

/-- The coordinate plane containing the half-plane in `Space 3`. -
/
def halfPlaneSubmodule : Submodule $\mathbb{R}$ (Space 3) where
 carrier := {x | x (2 : Fin 3) = 0}
 zero_mem' := by simp
 add_mem' hx hy := by
 rw [Set.mem_setOf_eq] at hx hy $\vdash$
 rw [Space.add_apply, hx, hy, add_zero]
 smul_mem' c x hx := by
 rw [Set.mem_setOf_eq] at hx $\vdash$
 rw [Space.smul_apply, hx, mul_zero]

lemma halfPlane_mem_halfPlaneSubmodule (x : Space 2) : halfPlane x $\in$ halfPlaneSubmodule
 simp [halfPlaneSubmodule, halfPlane]

lemma halfPlane_image_domain_subset_halfPlaneSubmodule :
 halfPlane '' halfPlaneDomain $\subseteq$ (halfPlaneSubmodule : Set (Space 3)) := by
 rintro x ⟨y, _, rfl⟩
 exact halfPlane_mem_halfPlaneSubmodule y

lemma halfPlaneSubmodule_ne_top : halfPlaneSubmodule $\neq$ $\top$:= by
 intro htop
 have hbasis : basis (2 : Fin 3) $\in$ halfPlaneSubmodule := by
 rw [htop]
 exact Submodule.mem_top
 simp [halfPlaneSubmodule] at hbasis

lemma volume_halfPlane_image_domain :
 volume (halfPlane '' halfPlaneDomain : Set (Space 3)) = 0 := by
 refine measure_mono_null halfPlane_image_domain_subset_halfPlaneSubmodule ?_

```

```

rw [volume_eq_addHaar]
exact MeasureTheory.Measure.addHaar_submodule
 (Space.basis.toBasis.addHaar : Measure (Space 3))
 halfPlaneSubmodule halfPlaneSubmodule_ne_top

end Space

```

### 1.341 PhyslibAlpha/SpaceAndTime/Space/Surfaces/Line.lean

```

/-
Copyright (c) 2026 Robert Sneiderman. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Robert Sneiderman
-/
module
public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.SphericalShell
public import Physlib.SpaceAndTime.Space.Integrals.Basic
public import Mathlib.MeasureTheory.Measure.Lebesgue.EqHaar
/-!

Line surfaces in `Space d`

-/
@[expose] public section
open SchwartzMap NNReal
noncomputable section
open Physlib Distribution
variable (α : Type) {E F F' : Type} [RCLike α] [NormedAddCommGroup E] [NormedAddCommGroup F'] [NormedSpace ℝ E] [NormedSpace ℝ F]

namespace Space

open MeasureTheory Real

/-!

A. The definition of the line surface

-/

/-- The coordinate line embedded in `Space d`. -/
def line (d : ℕ) [NeZero d] : ℝ → Space d := fun r =>
 r • basis (0 : Fin d)

```

1.341. *PHYSLIBALPHA/SPACEANDTIME/SPACE/SURFACES/LINE.LEAN* 619

```
lemma line_eq_smul_basis (d : ℕ) [NeZero d] :
 line d = fun r => r • basis (0 : Fin d) := rfl

lemma line_injective (d : ℕ) [NeZero d] : Function.Injective (line d) := by
 intro x y h
 have h0 := congrArg (fun p : Space d => p (0 : Fin d)) h
 simpa [line] using h0

@[fun_prop]
lemma line_continuous (d : ℕ) [NeZero d] : Continuous (line d) := by
 rw [line_eq_smul_basis]
 fun_prop

lemma line_measurableEmbedding (d : ℕ) [NeZero d] : MeasurableEmbedding (line d) :=
 Continuous.measurableEmbedding (line_continuous d) (line_injective d)

@[simp]
lemma norm_line (d : ℕ) [NeZero d] (r : ℝ) : ||line d r|| = ||r|| := by
 rw [line, norm_smul]
 simp

/-!

B. The measure associated with the line

-/

/-- The measure on `Space d` corresponding to integration along a coordinate line. -
/
def lineMeasure (d : ℕ) [NeZero d] : Measure (Space d) :=
 MeasureTheory.Measure.map (line d) volume

instance lineMeasure_hasTemperateGrowth (d : ℕ) [NeZero d] :
 (lineMeasure d).HasTemperateGrowth := by
 rw [lineMeasure]
 refine { exists_integrable := ?_ }
 obtain ⟨r, hr⟩ := Measure.HasTemperateGrowth.exists_integrable (μ := volume (α :=
 use r
 rw [MeasurableEmbedding.integrable_map_iff]
 · convert hr using 1
 ext x
 simp [norm_line]
 · exact line_measurableEmbedding d

/-!
```

## C. The distribution associated with the line

-/

/-- The distribution on `Space d` corresponding to integration along a coordinate line.  
One can roughly think of this distribution as taking a test function `f` as a mass, charge or current density concentrated on a line. -

/

```
def lineDist (d : ℕ) [NeZero d] : (Space d) → d[ℝ] ℝ :=
 SchwartzMap.integralCLM ℝ (lineMeasure d)
```

```
lemma lineDist_apply_eq_integral_lineMeasure (d : ℕ) [NeZero d] (f : ℝ → ℝ) :
 lineDist d f = ∫ x, f x ∂lineMeasure d := by
 rw [lineDist, SchwartzMap.integralCLM_apply]
```

```
lemma lineDist_apply_eq_integral_volume (d : ℕ) [NeZero d] (f : ℝ → ℝ) :
 lineDist d f = ∫ r : ℝ, f (line d r) := by
 rw [lineDist_apply_eq_integral_lineMeasure, lineMeasure,
 MeasurableEmbedding.integral_map (line_measurableEmbedding d)]
```

/-!

## D. The line has ambient volume zero

-/

/-- The linear subspace spanned by the coordinate line in `Space d`. -

/

```
def lineSubmodule (d : ℕ) [NeZero d] : Submodule ℝ (Space d) :=
 ℝ • basis (0 : Fin d)
```

```
lemma line_mem_lineSubmodule (d : ℕ) [NeZero d] (r : ℝ) : line d r ∈ lineSubmodule d
 rw [line_eq_smul_basis]
 exact Submodule.smul_mem _ r (Submodule.mem_span_singleton_self (basis (0 : Fin d)))
```

```
lemma range_line_subset_lineSubmodule (d : ℕ) [NeZero d] :
 Set.range (line d) ⊆ (lineSubmodule d : Set (Space d)) := by
 rintro x ⟨r, rfl⟩
 exact line_mem_lineSubmodule d r
```

```
lemma lineSubmodule_ne_top (d : ℕ) [NeZero d] (hd : 2 ≤ d) : lineSubmodule d ≠ top
 intro htop
 have hbasis : basis (1 : Fin d) ∈ lineSubmodule d := by
 rw [htop]
 exact Submodule.mem_top
 obtain ⟨c, hc⟩ := (Submodule.mem_span_singleton.mp hbasis)
```



1.342. *PHYSLIBALPHA/SPACEANDTIME/SPACE/SURFACES/RING.lean* 21

```

have hcoord := congrArg (fun p : Space d => p (1 : Fin d)) hc
have hd1 : d ≠ 1 := by omega
simp [basis_apply, hd1] at hcoord

lemma volume_line_range (d : ℕ) [NeZero d] (hd : 2 ≤ d) :
 volume (Set.range (line d) : Set (Space d)) = 0 := by
 refine measure_mono_null (range_line_subset_lineSubmodule d) ?_
 rw [volume_eq_addHaar]
 exact MeasureTheory.Measure.addHaar_submodule
 (Space.basis.toBasis.addHaar : Measure (Space d))
 (lineSubmodule d) (lineSubmodule_ne_top d hd)

end Space

```

### 1.342 PhyslibAlpha/SpaceAndTime/Space/Surfaces/Ring.lean

```

/-
Copyright (c) 2026 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module
public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.SphericalShell
public import Physlib.SpaceAndTime.Space.Translations
public import Mathlib.MeasureTheory.Integral.BoundedContinuousFunction
/-!

Ring surface in `Space 3`

-/
@[expose] public section
open SchwartzMap NNReal
noncomputable section
open Physlib Distribution
variable (α : Type) {E F F' : Type} [RCLike α] [NormedAddCommGroup E] [NormedAddCommGroup F]
 [NormedAddCommGroup F'] [NormedSpace ℝ E] [NormedSpace ℝ F]

namespace Space

open MeasureTheory Real

/-!

A. The definition of the ring surface

```

```

-/

/-- The embedding of the unit ring (a circle `S¹` in `Space 2`) into `Space 3`
/
def ring : Metric.sphere (0 : Space 2) 1 → Space 3 := fun x =>
 (slice 2).symm (0, Space.sphericalShell 2 x)

lemma ring_eq : ring = (slice 2).symm ∘ (fun x => (0, sphericalShell 2 x))

lemma ring_injective : Function.Injective ring := by
 intro x y h
 simp [ring] at h
 exact sphericalShell_injective _ h

@[fun_prop]
lemma ring_continuous : Continuous ring := by
 apply Continuous.comp
 · fun_prop
 · fun_prop

lemma ring_measurableEmbedding : MeasurableEmbedding ring :=
 Continuous.measurableEmbedding ring_continuous ring_injective

/-!

B. The measure associated with the ring

-/

/-- The measure on `Space 3` corresponding to integration around a ring. -
/
def ringMeasure : Measure (Space 3) :=
 MeasureTheory.Measure.map ring (MeasureTheory.Measure.toSphere volume)

instance ringMeasure_hasTemperateGrowth :
 ringMeasure.HasTemperateGrowth := by
 rw [ringMeasure]
 refine { exists_integrable := ?_ }
 use 0
 simp

instance ringMeasure_prod_volume_hasTemperateGrowth :
 (ringMeasure.prod (volume (α := Space))).HasTemperateGrowth := by
 exact IsDistBounded.instHasTemperateGrowthProdProdOfOpensMeasurableSpace

```

```

instance ringMeasure_sFinite: SFinite ringMeasure := by
 rw [ringMeasure]
 exact Measure.instSFiniteMap volume.toSphere ring

instance ringMeasure_finite: IsFiniteMeasure ringMeasure := by
 rw [ringMeasure]
 exact Measure.isFiniteMeasure_map volume.toSphere ring

lemma integrable_ringMeasure_of_continuous (f : Space → ℝ) (hf : Continuous (f ∘ ring)) :
 Integrable f ringMeasure := by
 rw [ringMeasure]
 rw [MeasurableEmbedding.integrable_map_iff]
 · let f' : BoundedContinuousFunction (Metric.sphere (0 : Space 2) 1) ℝ :=
 BoundedContinuousFunction.mkOfCompact {f ∘ ring, hf}
 exact BoundedContinuousFunction.integrable _ f'
 · exact ring_measurableEmbedding

lemma integrable_ringMeasure_of_continuous_euclid (f : Space → EuclideanSpace ℝ (Fin 2))
 (hf : Continuous (f ∘ ring)) :
 Integrable f ringMeasure := by
 rw [ringMeasure]
 rw [MeasurableEmbedding.integrable_map_iff]
 · exact BoundedContinuousFunction.integrable _
 (BoundedContinuousFunction.mkOfCompact {f ∘ ring, hf})
 · exact ring_measurableEmbedding

lemma ringMeasure_prod_volume_map :
 (ringMeasure.prod (volume (α := Space))).map (fun x : Space × Space => (x.1, x.2)
 = (ringMeasure.prod (volume (α := Space))) := by
 refine (MeasureTheory.MeasurePreserving.skew_product (f := id) (g := fun x => fun
 ?_ ?_ ?_).map_eq
 · exact MeasurePreserving.id ringMeasure
 · fun_prop
 · filter_upwards with x
 exact Measure.IsAddRightInvariant.map_add_right_eq_self (x)

@[simp]
lemma ringMeasure_univ : ringMeasure Set.univ = ENNReal.ofReal ((2 : ℝ) * π) := by
 rw [ringMeasure, Measure.map_apply]
 simp only [Set.preimage_univ, Measure.toSphere_apply_univ, finrank_eq_dim, Nat.cas
 volume_metricBall_two, Nat.ofNat_nonneg, ENNReal.ofReal_mul, ENNReal.ofReal_ofNa
 · fun_prop
 · exact MeasurableSet.univ

```

```

/-!

C. The distribution associated with the ring

-/

/-- The distribution on `Space 3` corresponding to integration around a ring
/
def ringDist : (Space 3) →d[ℝ] ℝ :=
 SchwartzMap.integralCLM ℝ ringMeasure

lemma ringDist_apply_eq_integral_ringMeasure (f : ℝ(Space 3, ℝ)) :
 ringDist f = ∫ x, f x ∂ringMeasure := by
 rw [ringDist, SchwartzMap.integralCLM_apply]

lemma ringDist_eq_integral_delta (f : ℝ(Space 3, ℝ)) :
 ringDist f = ∫ z, diracDelta ℝ z f ∂ringMeasure := by
 rw [ringDist_apply_eq_integral_ringMeasure]
 simp

end Space

```

### 1.343 PhyslibAlpha/SpaceAndTime/Space/Surfaces/SolidCylinder.lean

```

/-
Copyright (c) 2026 Robert Sneiderman. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Robert Sneiderman
-/
module

public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.SolidSphere
public import Physlib.SpaceAndTime.Space.Integrals.Basic
public import Mathlib.MeasureTheory.Integral.Prod
/-!

Solid cylinder surface in `Space 3`

The solid cylinder is the closed unit disk in `Space 2` extruded along the
It is the solid analogue of the spherical cylinder, in the same way that the
solid analogue of the spherical shell. Like the solid sphere it is a region
volume, so the measure associated with it is built from the ambient volume
sectional
disk (the solid-sphere measure in `Space 2`) extruded along the axis, rather

```

1.343.

*PHYSLIBALPHA/SPACEANDTIME/SPACE/SURFACES/SOLIDCYLINDER615AN*

a lower-dimensional surface measure. The measure-zero requirement is therefore not a  
and is replaced by a statement that the solid cylinder has positive ambient volume.

-/

```
@[expose] public section
open SchwartzMap NNReal
noncomputable section
open Physlib Distribution
variable (α : Type) {E F F' : Type} [RCLike α] [NormedAddCommGroup E] [NormedAddCommGroup F]
 [NormedAddCommGroup F'] [NormedSpace ℝ E] [NormedSpace ℝ F]
```

namespace Space

open MeasureTheory Real

/-!

## A. The definition of the solid cylinder surface

-/

/-- The map embedding a cross-sectional disk extruded along the axis into `Space 3`.  
cut out by the solid-cylinder measure, which is supported on the closed unit disk.  
/

```
def solidCylinder : Space 2 × ℝ → Space 3 := fun x =>
 (slice 2).symm (x.2, x.1)
```

```
lemma solidCylinder_eq :
 solidCylinder = (slice 2).symm ∘ (fun x : Space 2 × ℝ => (x.2, x.1)) := rfl
```

```
lemma solidCylinder_injective : Function.Injective solidCylinder := by
 intro x y h
 have h' := congrArg (slice 2) h
 simp only [solidCylinder, ContinuousLinearEquiv.apply_symm_apply, Prod.mk.injEq] at h'
 exact Prod.ext h'.2 h'.1
```

@[fun\_prop]

```
lemma solidCylinder_continuous : Continuous solidCylinder := by
 rw [solidCylinder_eq]
 fun_prop
```

```
lemma solidCylinder_measurableEmbedding : MeasurableEmbedding solidCylinder :=
 Continuous.measurableEmbedding solidCylinder_continuous solidCylinder_injective
```

@[simp]

```

lemma norm_solidCylinder (x : Space 2 × ℝ) :
 ||solidCylinder x|| = √(||x.2|| ^ 2 + ||x.1|| ^ 2) := by
 rw [solidCylinder, norm_slice_symm_eq]

/-!

B. The measure associated with the solid cylinder

-/

/-- The measure on `Space 3` corresponding to integration over a solid cylinder
 pushforward of the product of the solid-sphere (closed unit disk) measure
 line measure along the axis. -/
def solidCylinderMeasure : Measure (Space 3) :=
 MeasureTheory.Measure.map solidCylinder ((solidSphereMeasure 2).prod (volume (α := ℝ)))

instance solidCylinderMeasure_hasTemperateGrowth :
 solidCylinderMeasure.HasTemperateGrowth := by
 rw [solidCylinderMeasure]
 refine { exists_integrable := ?_ }
 obtain ⟨n, hn⟩ := MeasureTheory.Measure.HasTemperateGrowth.exists_integrable
 (μ := volume (α := ℝ))
 use n
 rw [MeasurableEmbedding.integrable_map_iff solidCylinder_measurableEmbedding]
 change Integrable
 (fun x : Space 2 × ℝ => (1 + ||solidCylinder x||) ^ (-(n : ℝ)))
 ((solidSphereMeasure 2).prod (volume (α := ℝ)))
 apply Integrable.mono' (hn.comp_snd (solidSphereMeasure 2))
 · apply AEMeasurable.aestronglyMeasurable
 exact ((continuous_const.add solidCylinder_continuous.norm).rpow_const
 (fun x => 0r.inl (by positivity : (1 : ℝ) + ||solidCylinder x|| ≠ 0))).
 · filter_upwards with x
 rw [Real.norm_eq_abs, abs_of_nonneg (Real.rpow_nonneg (by positivity) _)]
 apply Real.rpow_le_rpow_of_nonpos
 · positivity
 · rw [norm_solidCylinder]
 gcongr
 refine Real.le_sqrt_of_sq_le ?_
 nlinarith [sq_nonneg ||x.1||, norm_nonneg x.2]
 · simp

instance solidCylinderMeasure_sFinite : SFinite solidCylinderMeasure := by
 rw [solidCylinderMeasure]
 exact Measure.instSFiniteMap ((solidSphereMeasure 2).prod (volume (α := ℝ)))

/-!

```

1.344.

*PHYSLIBALPHA/SPACEANDTIME/SPACE/SURFACES/SOLIDSPHERE.lean*

## C. The distribution associated with the solid cylinder

-/

/-- The distribution on `Space 3` corresponding to integration over a solid cylinder.  
One can roughly think of this distribution as taking a test function `f` to its integral over  
a mass, charge or current density spread over a solid cylinder. -

/

def solidCylinderDist : (Space 3) → d[ℝ] ℝ :=  
 SchwartzMap.integralCLM ℝ solidCylinderMeasure

lemma solidCylinderDist\_apply\_eq\_integral\_solidCylinderMeasure (f : ℝ → ℝ) :  
 solidCylinderDist f = ∫ x, f x ∂solidCylinderMeasure := by  
 rw [solidCylinderDist, SchwartzMap.integralCLM\_apply]

lemma solidCylinderDist\_apply\_eq\_integral\_disk\_volume (f : ℝ → ℝ) :  
 solidCylinderDist f =  
 ∫ x, f (solidCylinder x) ∂((solidSphereMeasure 2).prod (volume (α := ℝ))) := by  
 rw [solidCylinderDist\_apply\_eq\_integral\_solidCylinderMeasure, solidCylinderMeasure  
 MeasurableEmbedding.integral\_map solidCylinder\_measurableEmbedding]

/-!

## D. The solid cylinder has positive ambient volume

-/

lemma solidCylinderMeasure\_univ\_pos : 0 < solidCylinderMeasure Set.univ := by  
 rw [solidCylinderMeasure, Measure.map\_apply solidCylinder\_measurableEmbedding.meas  
 MeasurableSet.univ, Set.preimage\_univ, ← Set.univ\_prod\_univ, Measure.prod\_prod]  
 refine ENNReal.mul\_pos (solidSphereMeasure\_univ\_pos 2).ne' ?\_  
 simp

end Space

### 1.344 PhyslibAlpha/SpaceAndTime/Space/Surfaces/SolidSphere.lean

/-

Copyright (c) 2026 Robert Sneiderman. All rights reserved.  
Released under Apache 2.0 license as described in the file LICENSE.

Authors: Robert Sneiderman

-/

module

```

public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.SphericalShell
public import Physlib.SpaceAndTime.Space.Integrals.Basic
public import Mathlib.MeasureTheory.Measure.Lebesgue.EqHaar
/-!

```

```

Solid sphere surfaces in `Space d`

```

The solid sphere is the closed unit ball in `Space d`. Unlike the line or the shell, it is a region of positive ambient volume, so the measure associated with ambient volume restricted to the ball rather than a pushforward of a lower-dimensional measure.

The requirement that the surface has ambient measure zero is therefore not needed and is replaced by a statement that the solid sphere has positive ambient volume.

```

-/
@[expose] public section
open SchwartzMap NNReal
noncomputable section
open Physlib Distribution
variable (α : Type) {E F F' : Type} [RCLike α] [NormedAddCommGroup E] [NormedAddCommGroup F'] [NormedSpace ℝ E] [NormedSpace ℝ F]

```

```

namespace Space

```

```

open MeasureTheory Real

```

```

/-!

```

```

A. The definition of the solid sphere surface

```

```

-/

```

```

/-- The inclusion into `Space d` of its closed unit ball `B^d` (the subtype of `Space d` consisting of points with norm at most 1) -/

```

```

def solidSphere (d : ℕ) : Metric.closedBall (0 : Space d) 1 → Space d := fun x => x

```

```

lemma solidSphere_injective (d : ℕ) : Function.Injective (solidSphere d) :=
 intro x y h
 simp [solidSphere] at h
 grind

```

```

lemma solidSphere_continuous (d : ℕ) : Continuous (solidSphere d) := continuous_id

```

```

lemma solidSphere_measurableEmbedding (d : ℕ) : MeasurableEmbedding (solidSphere d) :=
 apply Continuous.measurableEmbedding
 · exact solidSphere_continuous d

```



1.344.

*PHYSLIBALPHA/SPACEANDTIME/SPACE/SURFACES/SOLIDSPHERE.L*~~629~~

```
· exact solidSphere_injective d

@[simp]
lemma norm_solidSphere_le (d : ℕ) (x : Metric.closedBall (0 : Space d) 1) :
 ||solidSphere d x|| ≤ 1 := by
 have hx := x.2
 rw [Metric.mem_closedBall, dist_eq_norm, sub_zero] at hx
 exact hx

/-!

B. The measure associated with the solid sphere

-/

/-- The measure on `Space d` corresponding to integration over a solid sphere, i.e.
 ambient volume measure restricted to the closed unit ball. -
/
def solidSphereMeasure (d : ℕ) : Measure (Space d) :=
 volume.restrict (Metric.closedBall (0 : Space d) 1)

instance solidSphereMeasure_isFiniteMeasure (d : ℕ) :
 IsFiniteMeasure (solidSphereMeasure d) := by
 rw [solidSphereMeasure, isFiniteMeasure_restrict]
 exact (Metric.isBounded_closedBall).measure_lt_top.ne

instance solidSphereMeasure_hasTemperateGrowth (d : ℕ) :
 (solidSphereMeasure d).HasTemperateGrowth :=
 inferInstance

/-!

C. The distribution associated with the solid sphere

-/

/-- The distribution on `Space d` corresponding to integration over a solid sphere.
 One can roughly think of this distribution as taking a test function `f` to its in
 a mass, charge or current density spread over a solid ball. -
/
def solidSphereDist (d : ℕ) : (Space d) →d[ℝ] ℝ :=
 SchwartzMap.integralCLM ℝ (solidSphereMeasure d)

lemma solidSphereDist_apply_eq_integral_solidSphereMeasure (d : ℕ) (f : $\square$ (Space d, ℝ)
 solidSphereDist d f = $\int x, f x \partial$ solidSphereMeasure d := by
 rw [solidSphereDist, SchwartzMap.integralCLM_apply]
```

```

lemma solidSphereDist_apply_eq_integral_closedBall (d : ℕ) (f : ℝ(Space d) → ℝ) :
 solidSphereDist d f = ∫ x in Metric.closedBall (0 : Space d) 1, f x :=
 rw [solidSphereDist_apply_eq_integral_solidSphereMeasure, solidSphereMeasure_apply_eq_integral]
/-!

D. The solid sphere has positive ambient volume

-/

lemma solidSphere_volume_pos (d : ℕ) :
 0 < volume (Metric.closedBall (0 : Space d) 1) := by
 apply lt_of_lt_of_le (b := volume (Metric.ball (0 : Space d) 1))
 · exact (Metric.isOpen_ball).measure_pos volume (Metric.nonempty_ball.mpr)
 · exact measure_mono Metric.ball_subset_closedBall

lemma solidSphereMeasure_univ_pos (d : ℕ) : 0 < solidSphereMeasure d Set.univ := by
 rw [solidSphereMeasure, Measure.restrict_apply_univ]
 exact solidSphere_volume_pos d

end Space

```

### 1.345 PhyslibAlpha/SpaceAndTime/Space/Surfaces/SphericalCylinder

```

/-
Copyright (c) 2026 Robert Sneiderman. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Robert Sneiderman
-/
module

public import PhyslibAlpha.SpaceAndTime.Space.Surfaces.Ring
public import Physlib.SpaceAndTime.Space.Integrals.Basic
public import Mathlib.MeasureTheory.Integral.Prod
/-!

Spherical cylinder surface in `Space 3`

The spherical cylinder is the unit circular shell in `Space 2` extruded along the
third coordinate.

-/

```

1.345.

PHYSLIBALPHA/SPACEANDTIME/SPACE/SURFACES/SPHERICALCYLINDER. LEAN

```
@[expose] public section
open SchwartzMap NNReal
noncomputable section
open Physlib Distribution
variable (α : Type) {E F F' : Type} [RCLike α] [NormedAddCommGroup E] [NormedAddCommGroup F]
 [NormedAddCommGroup F'] [NormedSpace ℝ E] [NormedSpace ℝ F]

namespace Space

open MeasureTheory Real

/-!

A. The definition of the spherical cylinder surface

-/

/-- The map embedding the unit circular shell extruded along the axis into `Space 3`
/
def sphericalCylinder : Metric.sphere (0 : Space 2) 1 × ℝ → Space 3 := fun x =>
 (slice 2).symm (x.2, sphericalShell 2 x.1)

lemma sphericalCylinder_eq :
 sphericalCylinder = (slice 2).symm ∘ (fun x => (x.2, sphericalShell 2 x.1)) := r

lemma sphericalCylinder_injective : Function.Injective sphericalCylinder := by
 intro x y h
 have h' := congrArg (slice 2) h
 simp [sphericalCylinder] at h'
 exact Prod.ext (sphericalShell_injective 2 h'.2) h'.1

@[fun_prop]
lemma sphericalCylinder_continuous : Continuous sphericalCylinder := by
 apply Continuous.comp
 · fun_prop
 · fun_prop

lemma sphericalCylinder_measurableEmbedding : MeasurableEmbedding sphericalCylinder
 Continuous.measurableEmbedding sphericalCylinder_continuous sphericalCylinder_injective

@[simp]
lemma norm_sphericalCylinder (x : Metric.sphere (0 : Space 2) 1 × ℝ) :
 ||sphericalCylinder x|| = √(||x.2|| ^ 2 + 1) := by
 rw [sphericalCylinder, norm_slice_symm_eq, norm_sphericalShell]
 norm_num
```

```

/-!

B. The measure associated with the spherical cylinder

-/

/-- The measure on `Space 3` corresponding to integration over a spherical
/
def sphericalCylinderMeasure : Measure (Space 3) :=
 MeasureTheory.Measure.map sphericalCylinder
 ((MeasureTheory.Measure.toSphere volume).prod (volume (α := ℝ)))

instance sphericalCylinderMeasure_hasTemperateGrowth :
 sphericalCylinderMeasure.HasTemperateGrowth := by
 rw [sphericalCylinderMeasure]
 refine { exists_integrable := ?_ }
 obtain (n, hn) := MeasureTheory.Measure.HasTemperateGrowth.exists_integrable
 (μ := volume (α := ℝ))
 use n
 rw [MeasurableEmbedding.integrable_map_iff sphericalCylinder_measurableEmbedding]
 change Integrable
 (fun x : Metric.sphere (0 : Space 2) 1 × ℝ =>
 (1 + ||sphericalCylinder x||) ^ (-(n : ℝ)))
 ((MeasureTheory.Measure.toSphere volume).prod (volume (α := ℝ)))
 apply Integrable.mono' (hn.comp_snd (MeasureTheory.Measure.toSphere volume))
 · apply AEMeasurable.aestronglyMeasurable
 exact ((continuous_const.add sphericalCylinder_continuous.norm).rpow_const
 (fun x => 0r.inl (by positivity : (1 : ℝ) + ||sphericalCylinder x|| ≠ 0)))
 · filter_upwards with x
 rw [Real.norm_eq_abs, abs_of_nonneg (Real.rpow_nonneg (by positivity) _)]
 apply Real.rpow_le_rpow_of_nonpos
 · positivity
 · simp only [norm_sphericalCylinder]
 exact add_le_add_right (Real.le_sqrt_of_sq_le (le_add_of_nonneg_right _))
 · simp

instance sphericalCylinderMeasure_sFinite : SFinite sphericalCylinderMeasure :=
 rw [sphericalCylinderMeasure]
 exact Measure.instSFiniteMap ((MeasureTheory.Measure.toSphere volume).prod
 sphericalCylinder)

/-!

C. The distribution associated with the spherical cylinder

-/

```

1.346.

PHYSLIBALPHA/SPACEANDTIME/SPACE/SURFACES/SPHERICALSHELL.lean

```

/-- The distribution on `Space 3` corresponding to integration over a spherical cylinder
/
def sphericalCylinderDist : (Space 3) →d[ℝ] ℝ :=
 SchwartzMap.integralCLM ℝ sphericalCylinderMeasure

lemma sphericalCylinderDist_apply_eq_integral_sphericalCylinderMeasure (f : ℝ → ℝ) (Space
 sphericalCylinderDist f = ∫ x, f x ∂sphericalCylinderMeasure := by
 rw [sphericalCylinderDist, SchwartzMap.integralCLM_apply]

lemma sphericalCylinderDist_apply_eq_integral_sphere_volume (f : ℝ → ℝ) (Space 3, ℝ) :
 sphericalCylinderDist f =
 ∫ x, f (sphericalCylinder x)
 ∂((MeasureTheory.Measure.toSphere volume).prod (volume (α := ℝ))) := by
 rw [sphericalCylinderDist_apply_eq_integral_sphericalCylinderMeasure, sphericalCylinder
 MeasurableEmbedding.integral_map sphericalCylinder_measurableEmbedding]

end Space

```

### 1.346 PhyslibAlpha/SpaceAndTime/Space/Surfaces/SphericalShell.lean

```

/-
Copyright (c) 2026 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
module
public import Physlib.SpaceAndTime.Space.ConstantSliceDist
public import Physlib.SpaceAndTime.Space.Norm.Basic
public import Mathlib.MeasureTheory.Integral.IntervalIntegral.Basic
/-!

Spherical surfaces on Space.

-/
@[expose] public section

open SchwartzMap NNReal
noncomputable section
open Physlib Distribution
variable (α : Type) {E F F' : Type} [RCLike α] [NormedAddCommGroup E] [NormedAddCommGroup F]
[NormedAddCommGroup F'] [NormedSpace ℝ E] [NormedSpace ℝ F]

```

```

namespace Space

open MeasureTheory Real

/-!

A. The definition of the spherical shell surface

-/

/-- The inclusion into `Space d` of its unit sphere `S^{d-1}`
 (the subtype coercion `x ↦ x.1`). -/
def sphericalShell (d : ℕ) : Metric.sphere (0 : Space d) 1 → Space d := fun

lemma sphericalShell_injective (d : ℕ) : Function.Injective (sphericalShell
 intro x y h
 simp [sphericalShell] at h
 grind

lemma sphericalShell_continuous (d : ℕ) : Continuous (sphericalShell d) :=

lemma sphericalShell_measurableEmbedding (d : ℕ) : MeasurableEmbedding (sph
 apply Continuous.measurableEmbedding
 · exact sphericalShell_continuous d
 · exact sphericalShell_injective d

@[simp]
lemma norm_sphericalShell (d : ℕ) (x : Metric.sphere (0 : Space d) 1) :
 ||sphericalShell d x|| = 1 := by
 simp [sphericalShell, Metric.sphere]

/-!

B. The measure associated with the spherical shell

-/

/-- The measure on `Space d` corresponding to integration around a spherica
 /
def sphericalShellMeasure (d : ℕ) : Measure (Space d) :=
 MeasureTheory.Measure.map (sphericalShell d) (MeasureTheory.Measure.toSph

instance sphericalShellMeasure_hasTemperateGrowth (d : ℕ) :
 (sphericalShellMeasure d).HasTemperateGrowth := by
 rw [sphericalShellMeasure]
 refine { exists_integrable := ?_ }
```

```

 use 0
 simp

/-!

C. The distribution associated with the spherical shell

-/

/-- The distribution associated with a spherical shell.
 One can roughly think of this distribution as the distribution which
 takes test functions `f (r)` to ` $\int d^3r f(r) \rho(r)$ ` where ` $\rho(r)$ ` is the
 mass, charge or current etc. distribution. -/
def sphericalShellDist (d : ℕ) : (Space d) → d[ℝ] ℝ :=
 SchwartzMap.integralCLM ℝ (sphericalShellMeasure d)

lemma sphericalShellDist_apply_eq_integral_sphericalShellMeasure (d : ℕ) (f : $\square$ (Space
 sphericalShellDist d f = $\int x, f x \partial \text{sphericalShellMeasure } d$:= by
 rw [sphericalShellDist, SchwartzMap.integralCLM_apply]

lemma sphericalShellDist_apply_eq_integral_sphere_volume (d : ℕ) (f : $\square$ (Space d, ℝ))
 sphericalShellDist d f =
 $\int x, f (\text{sphericalShell } d \ x) \partial (\text{MeasureTheory.Measure.toSphere volume})$:= by
 rw [sphericalShellDist_apply_eq_integral_sphericalShellMeasure, sphericalShellMeas
 MeasurableEmbedding.integral_map (sphericalShell_measurableEmbedding d)]

end Space

```

### 1.347 QuantumInfo/Channels/Bundled.lean

```

coe_injective x y h := funext_hermitian fun M ↦
coe_injective := DFunLike.coe_injective.comp injective_toHMap
 have h1 := f.pos h
 simp_all only [HermitianMat.val_eq_coe, map_sub, ge_iff_le]
 exact h1
coe_injective := DFunLike.coe_injective.comp injective_toPMap
 have := f.pos h
 simp_all only [HermitianMat.val_eq_coe, map_sub, ge_iff_le]
 exact this
coe_injective x y h := injective_toPMap <| PMap.injective_toHMap <|
coe_injective := DFunLike.coe_injective.comp injective_toPTPMap
def of_kraus_CPTPMap {κ : Type*} [Fintype κ]
 coe_injective := (DFunLike.coe_injective (F := PMap dIn dOut ℂ)).comp injective_to

```

```

have := f.pos h
simp_all only [HermitianMat.val_eq_coe, map_sub, ge_iff_le]
exact this
coe_injective := (DFunLike.coe_injective (F := PMap dIn dOut ℂ)).comp inj
have := f.pos h
simp_all only [HermitianMat.val_eq_coe, map_sub, ge_iff_le]
exact this

```

### 1.348 QuantumInfo/Channels/CPTP.lean

```

convert! Λ_2 .map.IsTracePreserving_iff_trace_choi.1 Λ_2 .TP;
exact
convert! rfl
convert! MatrixMap.piProd_comp _ _;
simp_all only [mul_comm, Matrix.mul_apply]
exact hv
have := b.sum_inner_mul_inner (EuclideanSpace.single i 1) (EuclideanSpace.single i 1)
simp_all [inner]
exact this
convert! congr_arg (fun m : Matrix (dIn × dOut × dOut) (dIn × dOut × dOut) => (U.val * m * U.val.conjTranspose)) <_> using 1
convert! congr_arg (fun f => (U.val * f * U.val.conjTranspose)) <_> using 1

```

### 1.349 QuantumInfo/Channels/Dual.lean

```

convert! LinearEquiv.apply_symm_apply b.toDualEquiv f;
convert! congr_arg (fun g => g x) h_inv using 1;
omit [Fintype dOut] in

```

### 1.350 QuantumInfo/Channels/Pinching.lean

```

convert! (HermitianMat.mat_cfc_mul p.M _ _).symm
convert! congr($((p.M.cfc_const 0).symm).mat)
· simp
convert! congr($(pinching_sum σ).mat)
exact Submodule.sum_mem _ fun k _ => by
have h1 := hv k;
simp only [HermitianMat.ker_conj p.nonneg, HermitianMat.conjTranspose_mem Submodule.mem_comap] at h1
exact h1

```



**1.351 QuantumInfo/Channels/Unbundled.lean**

```

simp only [add_kron, LinearMap.add_apply]
exact Matrix.PosSemidef.add (h₁ n h) (h₂ n h)
have h1 :=
simp_all only [inv_one, mul_one, ge_iff_le]
exact h1
 (hM (by exact hXle :
convert! Matrix.PosSemidef.submatrix

```

**1.352 QuantumInfo/ClassicalInfo/Distribution.lean**

```

coe_injective _ _ h :=

```

**1.353 QuantumInfo/ClassicalInfo/Prob.lean**

```

isBot_zero a := a.2.1
norm_cast
· rw [ENNReal.coe_le_coe, toReal_le, toReal, neg_le_neg_iff]
simp [negLog]; rfl
· simp; rfl
 exact lt_neg_of_lt_neg h
 conv_rhs =>
 enter [1, 1]
 rw [h, Real.log_exp, neg_neg]
 coe_toReal, toReal, toReal_ofReal hx]
· exact _root_.zero_le
have h1 := ne_of_gt hp
simp_all only [unitInterval.coe_pos, ne_eq, Set.Icc.coe_eq_zero, negLog_eq_top_iff]
exact h1
· simp [h₂]; rfl
 simp only [ENNReal.coe_lt_coe, ← NNReal.coe_lt_coe]
 show x < -Real.log a

```

**1.354 QuantumInfo/Entropy/DPI.lean**

```

next h => simp_all only; norm_cast
 convert! congr_arg (fun f => f i j) h_support using 1)
convert! h_jensen using 1
convert! congr_fun (congr_fun (h_conj_apply (twirlingU σ f)) p) q using 1
 convert! Or.inl rfl
 · simp_all only [Finset.sum_empty, HermitianMat.zero_apply]

```

```

 convert! congr_arg2 (· + ·) rfl <_> using 1
 · convert! prod_traceRight_uniform_entry p a1 a2 b1 b2 using 1
congr 3
convert! congr_arg Real.log (sandwichedTraceFunctional_conj_unitary_MSt
 exact congr($$.purify_trace ξ)

```

### 1.355 QuantumInfo/Entropy/Relative.lean

```

 convert! congr_arg Complex.re h_trace using 1;
convert! congr_arg (fun x : HermitianMat d ℂ => x.val) h_cfc_eq using 1
 have h_supportProj_mul_A : (A.supportProj.val.toEuclideanLin v) = (Su
 simp only [val_eq_coe, Submodule.coe_orthogonalProjectionOnto_apply
 simp
 simp_all only [← hw, ← Matrix.mulVec_mulVec]
 exact h_supportProj_mul_B
positivity
positivity
 · convert! HasDerivAt.rpow (hasDerivAt_const _ _) (hasDerivAt_id 1)
convert! HasDerivAt.sum fun i _ => h_deriv i using 1
 convert! ((hasDerivAt_id 0).mul (hasDerivAt_const _ _)).exp.const_m
 convert! hasDerivAt_const (0 : ℝ) (0 : ℝ) using 1
 · have h1 := HasDerivAt.div (hasDerivAt_id (1 : ℝ) |> HasDerivAt.con
 (hasDerivAt_id (1 : ℝ)) (by norm_num)
 simp_all only [id_eq, mul_one, sub_self, sub_zero, one_pow, div_one]
 exact h1
have := (hb.rpow (hasDerivAt_id 1) hc_pos).sub hb
simp_all only [hasDerivAt_deriv_iff, id_eq, mul_one, sub_self, Real.rpow_
 one_mul, add_sub_cancel_left]
exact this
 norm_num [hc]
 filter_upwards [h_bound, Metric.ball_mem_nhds 1 (show (0 : ℝ) <
 convert! Filter.Tendsto.comp (show Filter.Tendsto (fun M : { M : Matr
 · convert! conj_supportProj_eq_of_ker_le A pM hker |> Eq.symm;
convert! Finset.abs_sum_le_sum_abs _ Finset.univ |> le_trans <| Finset.su
 exact (δ3, δ3_pos, fun x hx hx' hx'' h hh hh' => by rw [abs_div, di
 convert! mul_le_mul_of_nonneg_left (h_exp_ineq (h * Real.log x)
 convert! mul_le_mul_of_nonneg_left h_mean_value (show 0 ≤ x by linar
 convert! h_cfc_cont.tendsto.comp _ using 2;
convert! h_deriv.add (hasDerivAt_trace_rpow_sub_trace p.M p.nonneg) usi
convert! HasDerivAt.add (HasDerivAt.sub h_cross_term.1 h_cross_term.2)
convert! h_deriv using 2
 convert! congr(WithLp.ofLp $(h_top (WithLp.toLp 2 (Pi.single j 1
 convert! congr_fun (h_kronecker σ1.1.mat σ2.1.mat u v) (i, j) using
 exact 0r.inl (by exact congr(WithLp.ofLp $hu i));

```

```

 exact hz''.trans (by exact congr(WithLp.ofLp $hz' (a, b)));
 exact hz''.resolve_left (by exact hv)
 convert! congr(WithLp.ofLp $(hp_top (EuclideanSpace.single j 1)) i) usi
 convert! hz_mul using 1;
 exact 0r.inr (by exact congr(WithLp.ofLp $hv b));
 exact hz''.symm.trans (by exact congr(WithLp.ofLp $hz' (i, j)));
· convert! ker_kron_le_of_le _ _ _ h.1 h.2 using 1
split_ifs <=> simp_all [HermitianMat.conj_submatrix] <=>
 exact (HermitianMat.ker_reindex_le_iff σ.M ρ.M e.symm).mp <_>
simp only [SandwichedRelRentropy, hα, ↓reduceDite, Std.le_refl, sub_self, inner_ze
 ENNReal.coe_eq_zero, NNReal.eq_iff, NNReal.coe_zero]
simp [NNReal.toReal]
· rw [continuousOn_congr (f := fun α ↦ ENNReal.ofReal
 have hα₀ : 0 < α := by linarith
 have hα₁ : α ≠ 1 := by linarith
 simp only [dif_pos hα₀, if_neg hα₁, ENNReal.ofReal]
 rw [Real.toNNReal_of_nonneg]
 rfl
· rw [continuousOn_congr (f := fun α ↦ ENNReal.ofReal
 have hα₀ : 0 < α := hα.1
 have hα₁ : α ≠ 1 := ne_of_lt hα.2
 split_ifs
 · norm_cast
 · rw [ENNReal.ofReal, Real.toNNReal_of_nonneg]
 rfl
norm_cast
intro
rfl
apply tendsto_finsetSum
 simp_rw [← Finset.mul_sum, neg_mul]
 exact Filter.tendsto_neg_atTop_atBot.comp
 (tendsto_natCast_atTop_atTop.atTop_mul_const h_split_sum)
· apply tendsto_finsetSum
 · simp_all only [gt_iff_lt, ← EReal.coe_ennreal_lt_coe_ennreal_iff, EReal.coe_en
 ↓reduceIte]
 exact hx'
set K : HermitianMat (dA × dB) ℂ := ρ.traceRight.M.kerProj ⊗ₖ (1 : HermitianMat dB)
have hKx : x ∈ K.ker := by
 refine (HermitianMat.inner_zero_iff ρ.nonneg (HermitianMat.kronecker_nonneg
 (by simp [HermitianMat.kerProj] using
 (HermitianMat.projector_nonneg (S := ρ.traceRight.M.ker))))
 (by rw [HermitianMat.zero_le_iff]; exact Matrix.PosSemidef.one))).1 ?_ hx
 rw [HermitianMat.inner_comm, inner_kron_one_eq_inner_traceRight, HermitianMat.in
 simp [hKdef, MState.exp_val] using
 (ρ.traceRight.exp_val_eq_zero_iff (by simp [HermitianMat.kerProj] using
 (HermitianMat.projector_nonneg (S := ρ.traceRight.M.ker)))).2 (by simp)

```

```

have hsum : K + p.traceRight.M.supportProj ⊗K (1 : HermitianMat dB ℂ) = 1
 rw [hKdef, ← HermitianMat.add_kronecker, p.traceRight.M.kerProj_add_sup
 HermitianMat.kronecker_one_one]
have key := congrArg (fun T : HermitianMat (dA × dB) ℂ => T.lin x) hsum
simp [HermitianMat.lin, HermitianMat.mat_add, map_add, LinearMap.add_app
 Matrix.toLpLin_apply, (K.mem_ker_iff_mulVec_zero x).1 hKx] using key
have hKx : x ∈ K.ker := by
 refine (HermitianMat.inner_zero_iff p.nonneg hK_nonneg).1 ?_ hx
 rw [HermitianMat.inner_comm, inner_one_kron_eq_inner_traceLeft, Hermiti
 simp [K, MState.exp_val] using
 (p.traceLeft.exp_val_eq_zero_iff (by simp [HermitianMat.kerProj] usi
 (HermitianMat.projector_nonneg (S := p.traceLeft.M.ker))))).2 (by si
have key := congrArg (fun T : HermitianMat (dA × dB) ℂ => T.lin x) hsum
simp [P, HermitianMat.lin, HermitianMat.mat_add, map_add, LinearMap.add
 Matrix.toLpLin_apply, (K.mem_ker_iff_mulVec_zero x).1 hKx] using key
refine tendsto_finsetSum _ fun i _ => ?_

```

### 1.356 QuantumInfo/Entropy/SSA.lean

```

noncomputable def Matrix.opNorm (A : Matrix m n ℝ) : ℝ :=
 rw [V_sigma, Matrix.reindex_apply, Matrix.conjTranspose_submatrix, Matrix
 V_rho_isometry (σBC.reindex (Equiv.prodComm dB dC))
 (by rw [HermitianMat.mat_reindex]; exact hσ.reindex _)]
 simp
have h_comp : Matrix.toEuclideanLin (A * B) = Matrix.toEuclideanLin A ∘1
 ext; simp [toEuclideanLin]
simp only [Matrix.opNorm, h_comp]
exact ContinuousLinearMap.opNorm_comp_le (LinearMap.toContinuousLinearMap
 (LinearMap.toContinuousLinearMap (Matrix.toEuclideanLin B))
convert! hA.reindex e
 convert! h_inv_pos.inv using 1;
 convert! h_inv_pos.posSemidef using 1;
 convert! h_comm_inv h_comm using 1;
rw [T1_mat, Matrix.reindex_apply, Matrix.conjTranspose_submatrix, Matrix.
 h_kron, Matrix.submatrix_one_equiv]
 convert! W_mat_sq_eq_conj pAB σBC hp hσ |> Eq.symm using 1;
· convert! h_sorted using 1;
 convert! S_mat_conj_rhs_eq_one pAB σBC hp hσ using 1;
 convert! Matrix.nonsing_inv_nonsing_inv _ _;
 convert! h_inv_symm.inv using 1;
 convert! Matrix.nonsing_inv_mul _ _;
convert! HermitianMat.reindex_le_reindex_iff (Equiv.prodAssoc dA dB dC)
 refine' Finset.sum_bij (fun x _ => (x.snd.1, x.fst.1, x.fst.2)) _
 aesop (add simp [Finset.mem_filter, Sigma.ext_iff, Prod.ext_iff]);

```

```

convert! inner_kron_one_eq_inner_traceRight B (M.reindex (Equiv.prodComm d1 d2)
convert! congr_arg (fun f => A.cfc f) (show Real.log ◦ (fun x => x-1) = -
Real.log from funext fun x => ?_) using 1
convert! hp.reindex (Equiv.prodAssoc d1 d2 d3).symm
convert! congr_arg2 (· + ·) _ _ using 1;
convert! inner_add_left _ _ _ using 1;
convert! inner_kron_one_eq_inner_traceRight _ _ using 1;
convert! h_pos_def _ _ _ (_ , _) <=> norm_num [*];
exact continuous_pi fun _ => continuous_pi fun _ => continuous_finsetSum _ fun
convert! h_reindex_cont.comp _ using 2;
· convert! Svn_continuous.continuousAt.tendsto.comp (MState.traceRight_continuo
have := Svn_wm p.SWAP.assoc.SWAP.assoc
simp_all only [MState.traceRight_assoc, MState.traceRight_SWAP, MState.traceLeft_r
MState.traceLeft_left_assoc, MState.traceLeft_SWAP, MState.assoc'_assoc, ge_iff_
exact this

```

### 1.357 QuantumInfo/Entropy/VonNeumann.lean

```

convert! traceLeft_eq_transpose_mul_conj ψ using 1
convert! Matrix.IsHermitian.charpoly_eq hA using 1;

```

### 1.358 QuantumInfo/ForMathlib/ComplexLaplaceTransform.lean

```

public import Mathlib.Algebra.Order.Star.Real
def ComplexLaplaceTransform {α : Type*} [MeasureTheory.MeasureSpace α]
def ComplexLaplaceConvergenceDomain {α : Type*} [MeasureTheory.MeasureSpace α]
{α : Type*} [MeasureTheory.MeasureSpace α] {E : α → WithTop ℝ} {z : ℂ}
{α : Type*} [MeasureTheory.MeasureSpace α] {E : α → WithTop ℝ} {z : ℂ}
exact (hint (by
exact
{α : Type*} [MeasureTheory.MeasureSpace α] {E : α → WithTop ℝ} :

```

### 1.359 QuantumInfo/ForMathlib/ContinuousLinearMap.lean

```

/-!
Continuous linear maps

This file collects auxiliary results about `ContinuousLinearMap`s that are not (yet)
Mathlib.

Main results

```

```

* `ContinuousLinearMap.ker_mk`: the kernel of the continuous linear map built
 semilinear map `f` agrees with the kernel of `f`.
* `ContinuousLinearMap.support_eq_sup_eigenspace_nonzero`: the range of a s
 linear map on a finite-dimensional Euclidean space is the supremum of its
 nonzero eigenvalues.
-/

```

### 1.360 QuantumInfo/ForMathlib/ContinuousSup.lean

```
public import Mathlib.Algebra.Order.Star.Real
```

### 1.361 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/BlockD

```

/-!
`2 × 2` block operators on a Hilbert sum

This file develops the operator-matrix ("block operator") calculus on the t
fold Hilbert
sum $\mathbb{H} \oplus \mathbb{H}$, the main tool in the block-operator proof of the Jensen operat

Main definitions

* `HSum  $\mathbb{H}$ `: the $\mathbb{H}^2$ direct sum $\mathbb{H} \oplus \mathbb{H}$ of two copies of a Hilbert space $\mathbb{H}$
 $\text{PiLp } 2 \text{ (fun } _ : \text{Fin } 2 \Rightarrow \mathbb{H})$.
* `hsumProj`, `hsumIncl`: the coordinate projections $\text{HSum } \mathbb{H} \rightarrow L[\mathbb{C}] \mathbb{H}$ and i
 $\mathbb{H} \rightarrow L[\mathbb{C}] \text{HSum } \mathbb{H}$ exhibiting $\text{HSum } \mathbb{H}$ as a direct sum; they are mutually a
* `blockDiagonal A B`: the block-diagonal operator $\text{diag}(A, B)$ on $\text{HSum } \mathbb{H}$
* `blockOp A00 A01 A10 A11`: a general 2×2 block operator on $\text{HSum } \mathbb{H}$.
* `blockDiagonalHom`: the $\mathbb{H}^*$ -algebra homomorphism $(A, B) \mapsto \text{diag}(A, B)$.

Here $L \mathbb{H} = \mathbb{H} \rightarrow L[\mathbb{C}] \mathbb{H}$ is the algebra of bounded operators (quantum observab
representing a pair of observables as a block-diagonal operator on $\mathbb{H} \oplus \mathbb{H}$
trick underlying the operator inequalities used throughout quantum informat
-/
/-- A two-fold Hilbert sum used for the block-operator proof of Jensen's in
The inner-product instance on $\mathbb{H}$ is unused by the underlying type synonym
document that $\text{HSum } \mathbb{H}$ is the $\mathbb{H}^2$ direct sum of two copies of a Hilbert sp
/
@[nolint unusedArguments]
/-- The continuous linear equivalence between the Hilbert sum $\text{HSum } \mathbb{H}$ and

```

1.362.

QUANTUMINFO/FORMATHLIB/HAYATAGROUP/TRACEINEQUALITY/GENERALIZEDPERSPECTIVEFUNCTION.LE

```
function space `Fin 2 →  $\mathbb{C}$ `, forgetting the ` $\mathbb{C}$ ` inner-product structure on the index
/
/-- The `i`-th coordinate projection `HSum  $\mathbb{C}$  → L[ $\mathbb{C}$ ]  $\mathbb{C}$ ` of the Hilbert sum onto its
`i`-th summand. -/
/-- The `i`-th coordinate inclusion ` $\mathbb{C}$  → L[ $\mathbb{C}$ ] HSum  $\mathbb{C}$ `, embedding ` $\mathbb{C}$ ` as the `i`-
th summand
of the Hilbert sum and placing `0` in the other summand. -
/
 · exact h0 z
 · exact h1 z
/-- The ` $\mathbb{R}$ `-`*`-algebra homomorphism sending a pair of operators `(A, B)` to the
block-diagonal operator `diag(A, B)` on the Hilbert sum. -
/
 simp [blockDiagonal, hsumProj, hsumIncl, hsumEquiv, add_apply]
 erw [inner_add_left, hz0, hz1]
 erw [inner_add_left, hz0, hz1]
```

### 1.362 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/GeneralizedPe

```
exact (hfconc (K := K))
exact (hfconc (K := K))
```

### 1.363 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/HilbertSchmid

```
exact
omit [CompleteSpace $\mathbb{C}$] in
omit [CompleteSpace $\mathbb{C}$] in
 exact
 exact
 exact hmap
 exact
```

### 1.364 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperato

```
omit [CompleteSpace $\mathbb{C}$] in
```

### 1.365 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperato

```
simp only [blockOp, add_apply, ContinuousLinearMap.comp_apply,
omit [CompleteSpace $\mathbb{C}$] in
```

```
omit [CompleteSpace α] in
omit [CompleteSpace α] in
omit [CompleteSpace α] in
omit [CompleteSpace α] in
```

### 1.366 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/Jensen

```
omit [CompleteSpace α] in
omit [Nontrivial α] [CompleteSpace α] in
```

### 1.367 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LiebAn

```
omit [Nontrivial α] [CompleteSpace α] in
 exact congrArg (fun X => X * (1 : L α)) hcomm.symm
 have h1 := (mul_add (toOp T) B (-A)).symm
 simp_all only [sub_eq_add_neg, mul_neg, add_apply, rightMulHS_apply, ne
 exact h1
 exact
 exact
```

### 1.368 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/Lowner

```
 exact hp.1
 exact hp.2
 exact (CFC.nnrpow_eq_rpow (A := α) (a := A) (x := q) hq0)
 exact convex0n_id (α := $\mathbb{R}$) (s := Set.Ici (0 : α)) hs
 simp_all [sub_eq_add_neg]
 exact hid.add hconst
 exact_mod_cast sub_pos.mpr hp.1
 exact this
 norm_cast
```

### 1.369 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/Lowner

```
exact (LownerHeinzCore.one_div_operatorAntitone0n_Ioi (α := L α))
exact (LownerHeinzCore.one_div_operatorConvex0n_Ioi (α := L α))
exact (LownerHeinzCore.one_div_add_t_operatorAntitone0n_Ici (α := L α) t
exact (LownerHeinzCore.one_div_add_t_operatorConvex0n_Ici (α := L α) t ht
exact (LownerHeinzCore.ratio_add_t_operatorMonotone0n_Ici (α := L α) t ht
exact (LownerHeinzCore.ratio_add_t_operatorConcave0n_Ici (α := L α) t ht)
```



```

exact (LowerHeinzCore.power_Icc_zero_one_operatorMonotoneOn_Ici (□ := L □) p hp)
exact (LowerHeinzCore.power_Icc_zero_one_operatorConcaveOn_Ici (□ := L □) p hp)
exact (LowerHeinzCore.power_Icc_one_two_operatorConvexOn_Ici (□ := L □) p hp)
exact (LowerHeinzCore.power_Icc_neg_one_zero_neg_operatorMonotoneOn_Ioi (□ := L □) p hp)
exact (LowerHeinzCore.power_Icc_neg_one_zero_neg_operatorConcaveOn_Ioi (□ := L □) p hp)

```

### 1.370 QuantumInfo/ForMathlib/HermitianMat/Basic.lean

```

coe_injective _ _ h := HermitianMat.ext h
apply AddSubgroup.val_finsetSum
Matrix.isSymmetric_toEuclideanLin_iff.symm.mp A.H
convert! ContinuousLinearMap.orthogonal_range A.lin

```

### 1.371 QuantumInfo/ForMathlib/HermitianMat/CFC.lean

```

apply continuous_finsetSum _
-- Let `M = ||A₀|| + 1`. For `B` with `||B - A₀|| < 1` we get `||B|| ≤ M`, so `σ(B) ⊆ cl`
obtain ⟨M, hM⟩ : ∃ M : ℝ, ∀ B : HermitianMat d ℂ, ||B - A₀|| < 1 →
 spectrum ℝ B.mat ⊆ Metric.closedBall 0 M := by
 have := norm_sub_norm_le B A₀; linarith
 exact (spectrum_subset_closedBall B).trans (Metric.closedBall_subset_closedBall M)
-- `K = closedBall 0 M \ U` is compact and disjoint from `σ(A₀)`.
have hK_compact : IsCompact K := IsCompact.diff (ProperSpace.isCompact_closedBall M)
have hK_disjoint : K ∩ spectrum ℝ A₀.mat = ∅ :=
 Set.eq_empty_of_forall_notMem fun x hx => hx.1.2 <| hAU hx.2
-- For each `t ∈ K` there are `δ_t, ε_t > 0` such that `||B - A₀|| < δ_t` and `|s - t| < ε_t`
-- make `B.mat - s` a unit: units are open and `A₀.mat - t` is one.
have h_unitary : ∀ t ∈ K, ∃ δ_t > 0, ∃ ε_t > 0, ∀ B : HermitianMat d ℂ, ||B - A₀|| < δ_t →
 ∀ s : ℝ, |s - t| < ε_t → IsUnit (B.mat - algebraMap ℝ (Matrix d d ℂ) s) := by
 simp_all [Set.ext_iff, spectrum.mem_iff]
have h_unitary_open : IsOpen {B : Matrix d d ℂ | IsUnit B} := Units.isOpen
have h_unitary_cont : Continuous (fun p : HermitianMat d ℂ × ℝ =>
 p.1.mat - algebraMap ℝ (Matrix d d ℂ) p.2) := by
 obtain ⟨ε, ε_pos, hε⟩ :=
 Metric.isOpen_iff.mp (h_unitary_open.preimage h_unitary_cont) (A₀, t) h_unitary
 exact ⟨ε, ε_pos, ε, ε_pos, fun B hB s hs => hε (show (B, s) ∈ Metric.ball (A₀, t) ε)
 simp only [Metric.mem_ball, Prod.dist_eq]
 · simp [dist_eq_norm] using hs)
-- Compactness gives a finite subcover of `ε`-balls; set `δ = min 1 (min_j δ_j)`.
obtain ⟨δ, hδ_pos, hδ⟩ : ∃ δ > 0, ∀ t ∈ K, ∃ ε_t > 0, ∀ B : HermitianMat d ℂ, ||B - A₀|| < δ_t →
 ∀ s : ℝ, |s - t| < ε_t → IsUnit (B.mat - algebraMap ℝ (Matrix d d ℂ) s) := by
 have := hK_compact.elim_nhds_subcover (fun t => Metric.ball t (ε t))
 fun t ht => Metric.ball_mem_nhds t (hε t ht)

```

```

simp_all [Set.subset_def]
obtain ⟨t, ht₁, ht₂⟩ := this
 by_cases ht : t.Nonempty <|> simp_all [Finset.Nonempty]
 · exact ⟨Finset.min' (t.image δ) (·, Finset.mem_image_of_mem δ ht.cho
 by have := Finset.min'_mem (t.image δ) (·, Finset.mem_image_of_me
 fun i hi => Finset.min'_le _ _ (Finset.mem_image_of_mem δ hi))
 · exact ⟨1, zero_lt_one⟩
 refine' ⟨Min.min δ_min 1, lt_min hδ_min_pos zero_lt_one, fun x hx => _⟩
obtain ⟨i, hi, hi'⟩ := ht₂ x hx
exact ⟨ε i - |x - i|, sub_pos.mpr (by simp_all [abs_sub_comm]; exact hi
 fun B hB s hs => h i (ht₁ i hi) B (lt_of_lt_of_le hB (min_le_of_left
 (by rw [abs_lt] at *; constructor <|> linarith [abs_le.mp (show |x
-- For `B` with `||B - A₀|| < δ`, any `t ∈ σ(B)` lies outside `K`.
 have h_unit : IsUnit (B.mat - algebraMap ℝ (Matrix d d ℂ) t) :=
 hε_t B hB t (by simp using hε_t_pos)
 exact ht (by have h1 := h_unit.neg; simp_all; exact h1)
filter_upwards [Metric.ball_mem_nhds A₀ (show 0 < Min.min δ 1 by positivity
 fun t ht => Classical.not_not.1 fun h => h_not_in_K B
 (lt_of_lt_of_le (by simp [dist_eq_norm] using hB) (min_le_left _ _))
 (hM B (lt_of_lt_of_le (by simp [dist_eq_norm] using hB) (min_le_right
 convert! this.comp (show ContinuousWithinAt (fun p => (x
 exact ContinuousWithinAt.prodMk (continuousWithinAt_const
exact integrable_finsetSum _ fun i _ => Integrable.smul_const (hg i) _
· rw [intervalIntegral.integral_congr fun t ht =>
 HermitianMat.cfc_toMat_eq_sum_smul_proj A (f t), intervalIntegral
 exact ⟨ f x, by simp using HermitianMat.spectrum_cfc_eq_image A f > Se
 by cases lt_or_gt_of_ne hx₂ <|>
 linarith [hf x (lt_of_le_of_ne (h_pos x hx₁) (Ne.symm hx₂))
rintro _ { x, hx, rfl } ; exact if hx0 : x = 0 then by
 simp [hx0] using hf₂ else hf x (lt_of_le_of_ne (h_pos x hx) (
 le_of_lt;
 convert! h1 using 1
convert! congr_arg (fun y => ∑ j, x.ofLp j * y j) h_cfc_def using 1

```

### 1.372 QuantumInfo/ForMathlib/HermitianMat/Inner.lean

```

convert! (Matrix.trace_conjTranspose (A.mat * B.mat)).symm using 1
-- The kernel inclusion `range ≤ ker` says `(toEuclideanLin B) ∘ (toEucli
-- `B * A = 0`, which by Hermitianness of `A` and `B` is `A * B = 0`.
rw [LinearMap.range_le_ker_iff,
 show Matrix.toEuclideanLin B.val ∘₁ Matrix.toEuclideanLin A.val
 = Matrix.toEuclideanLin (B.val * A.val) from by
 ext v; simp [Matrix.toLpLin_apply, Matrix.mulVec_mulVec, Matrix.toEuc
LinearEquiv.map_eq_zero_iff,

```

```

show B.val * A.val = (A.val * B.val).conjTranspose from by simp [Matrix.conjTranspose_eq_zero]
-- Factoring `A = C^H C` and `B = D^H D`, a cyclic trace identity gives `tr (A * B)`
-- where `E = D C^H`, so `re (tr (A * B)) = 0` forces `E = 0` and hence `A * B = 0`
obtain ⟨C, hC⟩ : ∃ C : Matrix n n ℂ, A.val = C.conjTranspose * C :=
 CStarAlgebra.nonneg_iff_eq_star_mul_self.mp (Matrix.nonneg_iff_posSemidef.mpr hA)
open MatrixOrder in
obtain ⟨D, hD⟩ : ∃ D : Matrix n n ℂ, B.val = D.conjTranspose * D :=
 CStarAlgebra.nonneg_iff_eq_star_mul_self.mp (Matrix.nonneg_iff_posSemidef.mpr hB)
have htr : (A.val * B.val).trace
 = ((D * C.conjTranspose).conjTranspose * (D * C.conjTranspose)).trace := by
 rw [hC, hD, Matrix.conjTranspose_mul, Matrix.conjTranspose_conjTranspose,
 show C.conjTranspose * C * (D.conjTranspose * D)
 = C.conjTranspose * (C * D.conjTranspose * D) from by simp [Matrix.mul_assoc,
 Matrix.trace_mul_comm,
 show C * D.conjTranspose * D * C.conjTranspose
 = C * D.conjTranspose * (D * C.conjTranspose) from by simp [Matrix.mul_assoc]
 refine ⟨fun h => ?_, fun h => by rw [h]; simp⟩
have hE0 : D * C.conjTranspose = 0 :=
 Matrix.trace_conjTranspose_mul_self_eq_zero_iff.mp <| RCLike.ext
 (by rw [← htr]; simpa using h)
 (by simpa using (RCLike.nonneg_iff.mp
 (Matrix.posSemidef_conjTranspose_mul_self (D * C.conjTranspose)).trace_nonneg))
have hE0' : C * D.conjTranspose = 0 := by
 simpa [Matrix.conjTranspose_mul] using congrArg Matrix.conjTranspose hE0
rw [hC, hD, show C.conjTranspose * C * (D.conjTranspose * D)
 = C.conjTranspose * (C * D.conjTranspose) * D from by simp [Matrix.mul_assoc],
simp
convert! ← Matrix.frobenius_norm_def A.mat
convert! isClosed_singleton.preimage (f := fun (x : Matrix n n ℂ) ↦ (x.conjTranspose
intro C
set U : Matrix d d ℂ := A.H.eigenvectorUnitary.val
set V : Matrix d d ℂ := B.H.eigenvectorUnitary.val
-- `U`, `V` are unitary and diagonalize `A`, `B`; `C = U^H V`.
have hUU : star U * U = 1 := A.H.eigenvectorUnitary.2.1
have hUU' : U * star U = 1 := A.H.eigenvectorUnitary.2.2
have hAeq : A.mat = U * Matrix.diagonal (RCLike.ofReal ∘ A.H.eigenvalues) * star U
 simpa [Unitary.conjStarAlgAut_apply] using A.H.spectral_theorem
have hBeq : B.mat = V * Matrix.diagonal (RCLike.ofReal ∘ B.H.eigenvalues) * star V
 simpa [Unitary.conjStarAlgAut_apply] using B.H.spectral_theorem
have hC : C = star U * V := rfl
have hCH : C.conjTranspose = star V * U := by
 simp [hC, Matrix.conjTranspose_mul, Matrix.star_eq_conjTranspose]
-- Conjugating `A * B` by `U` turns it into a product of two diagonals and `C`, `C`
have key : A.mat * B.mat =
 U * (Matrix.diagonal (RCLike.ofReal ∘ A.H.eigenvalues) * C *

```

```

 Matrix.diagonal (RCLike.ofReal ◦ B.H.eigenvalues) * C.conjTranspose
conv_lhs => rw [hAeq, hBeq]
rw [hCH, hC]
simp only [Matrix.mul_assoc, hUU', mul_one]
have hC_norm : ∀ i j, (||C i j|| ^ 2 : ℂ) = C i j * star (C i j) := fun i j
 simp [Complex.mul_conj, Complex.normSq_eq_norm_sq]
-- The trace of the diagonal product expands entrywise to the doubly-
stochastic sum.
have hC_trace : (Matrix.diagonal (RCLike.ofReal ◦ A.H.eigenvalues) * C *
 Matrix.diagonal (RCLike.ofReal ◦ B.H.eigenvalues) * C.conjTranspose).
 ((∑ i, ∑ j, A.H.eigenvalues i * B.H.eigenvalues j * ||C i j|| ^ 2 : ℝ)
push_cast
simp only [Matrix.trace, Matrix.diag_apply, Matrix.mul_apply, hC_norm]
simp only [Matrix.diagonal_apply, ite_mul, zero_mul, mul_ite, mul_zero,
 Finset.sum_ite_eq, Finset.sum_ite_eq', Finset.mem_univ, if_true, Func
 Matrix.conjTranspose_apply]
exact Finset.sum_congr rfl fun _ => Finset.sum_congr rfl fun _ => b
 simp only [RCLike.ofReal_eq_complex_ofReal]; ring
rw [inner_eq_re_trace, key, Matrix.trace_mul_cycle, hUU, one_mul, hC_trac
exact Complex.ofReal_re _

```

### 1.373 QuantumInfo/ForMathlib/HermitianMat/LiebConcavity.lean

```

public import QuantumInfo.ForMathlib.HermitianMat.Schatten
 constructor
 · intro h x
 apply ext_inner_left ℂ
 intro y
 rw [ContinuousLinearMap.adjoint_inner_right, h]
 · intro h x y
 rw [← ContinuousLinearMap.adjoint_inner_left, h]
set_option synthInstance.maxHeartbeats 80000 in
set_option synthInstance.maxHeartbeats 80000 in
omit [Fintype d] in
omit [Fintype d] in
 convert! h_joint_concave (Φ_mem_pdSet σ₁ hσ₁) (Φ_mem_pdSet σ₂ hσ₂)

```

### 1.374 QuantumInfo/ForMathlib/HermitianMat/LogExp.lean

```

convert! A.H.log_smul_of_ne_zero hx
 simp_all only [sub_nonneg]
 exact h
 convert! logApprox_eq_cfc_scalar x hx T hT using 1;

```

```

 refine' tendsto_finsetSum _ fun i _ => _;
 exact (1, fun T hT => by exact logApprox_mono hA hB hAB T (zero_lt_one.trans_le h
 convert! hA.add_posSemidef _ using 1
 convert! h_integral_mono using 1;

```

### 1.375 QuantumInfo/ForMathlib/HermitianMat/Order.lean

```

 simp [Matrix.smul_mulVec, hv]
 exact

```

### 1.376 QuantumInfo/ForMathlib/HermitianMat/Peierls.lean

```

 convert! hg.map_sum_le _ _ _ <=> simp_all [mul_comm]
 convert! Finset.sum_le_sum fun i _ => h_jensen i using 1
 convert! hg.map_sum_le _ _ _ <=> simp_all [mul_comm]
 convert! Finset.sum_le_sum fun i _ => h_jensen i using 1
 have h1 := h_sum.trans
 simp_all only
 exact h1
 have h1 := h_sum.trans
 simp_all only
 exact h1

```

### 1.377 QuantumInfo/ForMathlib/HermitianMat/Proj.lean

```

let P := S.subtypeL.comp S.orthogonalProjectionOnto
 have h1 := S.inner_starProjection_left_eq_right (EuclideanSpace.single i 1) (EuclideanSpace.single j 1)
 simp_all [EuclideanSpace.inner_single_right, EuclideanSpace.inner_single_left]
 exact h1)
let P := S.subtypeL.comp S.orthogonalProjectionOnto
 (S.subtypeL.comp S.orthogonalProjectionOnto) v = 0 ↔ v ∈ S[]
suffices h_trace : ((S.subtypeL.comp S.orthogonalProjectionOnto).toMatrix (EuclideanSpace.single i 1) (EuclideanSpace.single j 1)) = 0
suffices h_trace : ((S.subtypeL.comp S.orthogonalProjectionOnto).toMatrix (EuclideanSpace.single i 1) (EuclideanSpace.single j 1)) = 0
simp [Submodule.orthogonalProjectionOnto_mem_subspace_eq_self]
have h_proj : ∀ j : n, S.orthogonalProjectionOnto (EuclideanSpace.single j 1) = ∑ k, (inner (EuclideanSpace.single j 1) (EuclideanSpace.single k 1)) • (EuclideanSpace.single k 1)
 have h_proj : S.orthogonalProjectionOnto (EuclideanSpace.single j 1) = ∑ k, (inner (EuclideanSpace.single j 1) (EuclideanSpace.single k 1)) • (EuclideanSpace.single k 1)
 convert b.sum_repr (S.orthogonalProjectionOnto (EuclideanSpace.single j 1))
 convert! h_proj using 3
convert! congr_arg (fun x : EuclideanSpace (_) n => x i) (h_proj j) using 1
convert! projector_eq_sum_rankOne _ b using 1

```

**1.378 QuantumInfo/ForMathlib/HermitianMat/Reindex.lean**

```
ext1; simp
```

**1.379 QuantumInfo/ForMathlib/HermitianMat/Rpow.lean**

```
public import QuantumInfo.ForMathlib.HermitianMat.LogExp
convert! cfc_pos_of_pos hA _ _
convert! congr_fun (congr_fun h_inv_def i) j using 1
convert! Matrix.nonsing_inv_mul _ _;
refine tendsto_finsetSum _ fun i _ => ?_
simp [Real.finsetProd_rpow _ _ (fun i _ => hA _)]
have h1 :=
simp_all only [Std.le_refl, map_zero, mat_zero, Matrix.conjTranspose_
conj_apply_mat, conjTranspose_mat, ne_eq, OfNat.ofNat_ne_zero, not_
root.zero_pow, zero_smul, Real.sqrt_zero, ge_iff_le]
exact h1
have h1 := Matrix.isHermitian_mul_conjTranspose_self M.conjTranspose
simp_all only [Matrix.conjTranspose_conjTranspose]
exact h1)
exact prod_singularValuesSorted_eq_compoundSV N k hk
have hrpow := (Real.finsetProd_rpow (s := Finset.univ)
exact hprod.trans hrpow.symm
rw [intervalIntegral.integral_finsetSum]
(tendsto_finsetSum Finset.univ fun i _ =>
```

**1.380 QuantumInfo/ForMathlib/HermitianMat/Schatten.lean**

```
convert! congr_arg (· ^ (1 / p)) _ using 1
convert! congr_arg _ (A.cfc_sq_rpow_eq_cfc_rpow hA p hp.le) using 1
convert! rfl
convert! (A ^ 2).mat_cfc (· ^ (p / 2))
convert! Finset.sum_nonneg fun i _ => h_nonneg i using 1;
convert! Finset.sum_le_sum fun i _ => Finset.sum_le_sum fun j _ => mul_le
```

**1.381 QuantumInfo/ForMathlib/HermitianMat/Trace.lean**

```
set_option linter.overlappingInstances false in
set_option linter.overlappingInstances false in
set_option linter.overlappingInstances false in
convert! congrArg RCLike.re A.H.sum_eigenvalues_eq_trace
```

**1.382 QuantumInfo/ForMathlib/IsMaximalSelfAdjoint.lean**

```

/-!
Maximal self-adjoint subrings

This file introduces the `IsMaximalSelfAdjoint R α` typeclass, which records that the
`TrivialStar` ring `R` carries the self-adjoint part of a star ring `α`. It bundles
additive, `R`-linear map `selfadjMap : α →+ R` that inverts `algebraMap R α` on self-
adjoint
elements. The guiding example is `R = ℝ`, `α = ℂ`: it lets a quantity such as the tr
Hermitian matrix be valued in `ℝ` instead of `ℂ`, reflecting that physical observabl
self-adjoint and take real expectation values.
-/

/-- `IsMaximalSelfAdjoint R α` witnesses that `R` is the maximal `TrivialStar` subri
star ring `α`, via an additive map `selfadjMap : α →+ R` collecting the self-
adjoint part of
each element. This lets `HermitianMat.trace` return `[]` when `[]` already has a trivi
and the "clean" underlying type otherwise, e.g. `ℝ` when the input field is `ℂ`. -
/
/-- The additive map sending an element of `α` to its self-
adjoint part in `R`. -/
/-- `selfadjMap` pulls scalar multiplication by `R` out of its argument. -
/
/-- On self-adjoint elements, `selfadjMap` is a section of `algebraMap R α`. -
/
instance instTrivialStar_IsMaximalSelfAdjoint {R} [Star R] [TrivialStar R] [CommSemi
 IsMaximalSelfAdjoint R R where
theorem trivial_selfadjMap {R} [Star R] [TrivialStar R] [CommSemiring R] :
 (selfadjMap : R →+ R) = .id R := by

```

**1.383 QuantumInfo/ForMathlib/Isometry.lean**

```

 convert! h_perm_ortho using 2;
 have h1 := cfc_conj_unitary (A := A) f u⁻¹
 simp_all only [inv_inv]
 exact h1
 rw [inner_sum, Finset.sum_eq_add_sum_sdiff_singleton _ _ (by simp)]
 exact {Σ i ∈ f.support, .of _ i {f i, hf₁ i}, by simp_all; exact hf₂}
 exact fun h => by simp [h] at this
 exact fun h => by simp [h] at this
LinearMap.sharedEigenbasis (isSymmetric_toEuclideanLin_iff.symm.mp hA)
 (isSymmetric_toEuclideanLin_iff.symm.mp hB) (commute_euclideanLin hAB)
 (isSymmetric_toEuclideanLin_iff.symm.mp hA)

```

```

 (isSymmetric_toEuclideanLin_iff.symm.mp hB)
 (isSymmetric_toEuclideanLin_iff.symm.mp hA)
 (isSymmetric_toEuclideanLin_iff.symm.mp hB)
 rw [isSymmetric_toEuclideanLin_iff.symm] at hA hB
 rw [isSymmetric_toEuclideanLin_iff.symm] at hA hB
 simp only [inner, ← starRingEnd_apply] at this
 · convert! congr_fun (congr_fun this j) i using 1;
 simp [Matrix.mulVec, dotProduct];
 · convert! Matrix.SharedEigenbasis.star_shared_mul_A_mul_IsDiag hA hB hAB
 · convert! Matrix.SharedEigenbasis.star_shared_mul_B_mul_IsDiag hA hB hAB

```

### 1.384 QuantumInfo/ForMathlib/LimSupInf.lean

```

public import Mathlib.Algebra.Order.Star.Real
 simp only [Filter.eventually_atTop, sSup_le_iff, Set.mem_setOf_eq, forall_exists_index]
 · exact { 0, { 0, fun _ => zero_le } };
 Nat.cast_pos, Nat.findGreatest_pos, zero_lt_one, or_true, implies_true
 · exact { 0, { 0, fun _ => zero_le } };
 convert! add_le_add_left (ENNReal.ofReal_le_ofReal hk.le) (y : ℝ)
 simp_all only [gt_iff_lt, ge_iff_le, not_lt_zero, one_div, Filter.eventually_atTop]
 simp_all only [Filter.eventually_atTop]
 Filter.eventually_atTop, sSup_le_iff, Set.mem_setOf_eq, forall_exists_index
 simp_all only [one_div, gt_iff_lt, Filter.eventually_atTop]
 Filter.eventually_map, Filter.eventually_atTop, Set.mem_setOf_eq]
 · simp_all only [gt_iff_lt, Filter.eventually_atTop]
 simp only [Filter.limsup_eq, Filter.eventually_atTop, le_sInf_iff, Set.mem_setOf_eq]
 forall_exists_index]

```

### 1.385 QuantumInfo/ForMathlib/LinearEquiv.lean

```

/-!
Relabelling linear equivalences

```

## i. Overview

This module provides linear equivalences obtained by relabelling the index function space  $d \rightarrow R$  or of  $\text{EuclideanSpace } d$  along an index equivalence together with lemmas relating them to `Matrix.reindex`.

## ii. Key results

- `LinearEquiv.of_relabel` : the  $R$ -linear equivalence  $(d_2 \rightarrow R) \approx_1 [R]$  ( $d$  index equivalence  $e : d \approx d_2$ ).



- `LinearEquiv.euclidean\_of\_relabel` : the `EuclideanSpace` analogue of `of\_relabel`  
 - `Matrix.reindex\_toLin` and `Matrix.reindex\_toEuclideanLin` : reindexing a matrix  
 its associated linear map by these relabelling equivalences.

## iii. Table of contents

This can be filled in later.

## iv. References

-/

/-- The `R`-linear equivalence `(d₂ → R) ≈₁[R] (d → R)` that relabels the coordinate  
 function along an index equivalence `e : d ≈ d₂`. This is the linear-  
 equivalence packaging of

`Equiv.piCongrLeft`. -/

/-- The `[]`-linear equivalence `EuclideanSpace [] d₂ ≈₁[[]] EuclideanSpace [] d` that r  
 coordinates of a vector along an index equivalence `e : d ≈ d₂`. This is the `Euclid  
 analogue of `LinearEquiv.of\_relabel`, obtained by transporting it across the `WithLp  
 identifications. -/

### 1.386 QuantumInfo/ForMathlib/Majorization.lean

```
exact h_det_zero
show UH * U = 1 by exact Matrix.mem_unitaryGroup_iff'.mp hU]
 exact fun n i => rfl
 intro n hn; rcases n with (_, _ | n) <| trivial
convert! eigenvalue_le_singularValuesSorted_sq A h i using 1
norm_num [Real.finsetProd_rpow _ _ fun i _ => hx_nn _,
 Real.finsetProd_rpow _ _ fun i _ => hy_nn _]
```

### 1.387 QuantumInfo/ForMathlib/Matrix.lean

```
convert! congrFun (h₂ (Pi.single j 1)) i using 1
rw [isSymmetric_toEuclideanLin_iff.symm] at hA hB
convert! (Fintype.sum_prod_type_right _).symm
convert! (Fintype.sum_prod_type _).symm
 · simp [Matrix.det_apply', Polynomial.eval_finsetSum];
 · simp [Matrix.det_apply', Polynomial.eval_finsetSum];
 rw [show U * UH = 1 from by exact hU.2];
 · simp [Polynomial.eval_finsetSum, Polynomial.eval_mul, Polynomial.eval_prod,
 · simp [Polynomial.eval_finsetSum, Polynomial.eval_mul, Polynomial.eval_prod,
intro x y i j
```

```

split_ifs with h
· subst h; simp
· rfl
intro x y i j
split_ifs with h
· simp
· simp
obtain ⟨a, ha, b, hb, hx_eq⟩ := spectrum_prod_complex hA hB x (by exact
convert! Finset.sum_nonneg fun i _ => h_sum_nonneg i;
convert! h

```

### 1.388 QuantumInfo/ForMathlib/MatrixNorm/TraceNorm.lean

```

rw [conjTranspose_smul, RCLike.star_def, Matrix.mul_smul, smul_mul, smu
have h1 := b.sum_inner_mul_inner (EuclideanSpace.single i 1) (Euclidean
simp_all [inner]
exact h1)

```

### 1.389 QuantumInfo/ForMathlib/Minimax.lean

```

public import Mathlib.Topology.Algebra.Module.Spaces.ContinuousLinearMap
public import QuantumInfo.ForMathlib.SionMinimax
refine continuous_finsetSum _ fun i _ ↦ .mul (by fun_prop) ?_

```

### 1.390 QuantumInfo/ForMathlib/Misc.lean

```

simp_all [eq_comm]
exact a
Equiv.sigmaFiberEquiv_apply]
exact ⟨ e'.trans σ, by simp_all [Function.comp]; grind)

```

### 1.391 QuantumInfo/ForMathlib/SionMinimax.lean

```

simp only [Subtype.forall, Set.mem_sdiff, Set.mem_inter_iff, and_imp]
simp_all [Set.range]
exact hf
convert! lowerSemicontinuousOn_ciSup (s := S) (f := fun (i : Bool) x' ↦ i
exact le_trans (ciInf_le (by simp_all [Set.range]; exact hf) { w_1,
have h1 := (isCompact_iff_isCompact_univ.mp hS1).isSeqCompact (x := x
simp_all only [Subtype.coe_prop, implies_true, Set.mem_univ, true_and

```

```

exact h1
 convert! (hfc₂ yₙ hyₙ).bddBelow hS₁
 convert! (hfc₂ yₙ hyₙ).bddBelow hS₁
 convert! (hfc₂ y₀' y₀'.2).bddBelow hS₁

```

### 1.392 QuantumInfo/ForMathlib/ULift.lean

```

/-!
Typeclass instances for `ULift`

This file equips `ULift` with algebraic and analytic typeclass instances lifted from
underlying type, including `Star`, `StarMul`, `StarRing`, `NormedField`, `DenselyNormed`
and `RCLike` structures, together with the corresponding `simp` lemmas relating operations
`ULift.{v, u} α` to operations on the underlying `α`.
-/

instance ULift.instInvolutiveStar {α : Type u} [InvolutiveStar α] :
 InvolutiveStar (ULift.{v,u} α) where

```

### 1.393 QuantumInfo/Measurements/P0VM.lean

```

convert! Λ.measurementMap_apply_matrix m.mat

```

### 1.394 QuantumInfo/Operators/Unitary.lean

```

This file is intended for lemmas about unitary matrices (`Matrix.unitaryGroup`) and
apply to `Bra`s, `Ket`s, and `MState` mixed states.
/-- The No-cloning theorem, saying that if states `ψ` and `φ` can both be perfectly
using a unitary `U` and a fiducial state `f`, and they aren't identical (their inner
less than 1), then the two states must be orthogonal to begin with. In short: only one
states can be simultaneously cloned. -/
(hψ : U < pure (ψ ⊗ f) = pure (ψ ⊗ ψ))
(hφ : U < pure (φ ⊗ f) = pure (φ ⊗ φ))
(H : (pure ψ, pure φ) ⊗ Prob < (1 : ℝ)) :
have h2 : (pure (ψ ⊗ ψ), pure (φ ⊗ φ)) ⊗ Prob : ℝ) =
 (pure (ψ ⊗ f), U < pure (φ ⊗ f)) ⊗ Prob := by
 convert! ← h2

```

### 1.395 QuantumInfo/ResourceTheory/FreeState.lean

```

convert! h₂

```

```

convert! ← hm₁
· simp only [ht₁, relabel_cast_isFree]
 exact free_prod hσ₂f hσ₃f
 exact qRelEntropy_heq_congr ht (statePow_add ρ m n) (by rw [MState.rela

```

### 1.396 QuantumInfo/ResourceTheory/HypothesisTesting.lean

```

 convert! h with T
push Not at h
simp at h
rw [h] at hT₄
have h_sup_zero := hT₄ ▸ le_iSup₂ σ hσ₁
have h_eq := le_antisymm h_sup_zero (by simp)
rw [Subtype.ext_iff, Set.Icc.coe_zero] at h_eq
simp only [MState.exp_val] at h_eq
rw [HermitianMat.inner_zero_iff σ.nonneg hT₂] at h_eq
· grw [← h_eq]
 rwa [HermitianMat.ker, HermitianMat.ker,
 ContinuousLinearMap.ker_le_ker_iff_range_le_range] at hσ₂ <;> simp
· simp using _root_.bot_eq_zero
have h := hA m
simp at h
apply h
· have := congrArg (·.ofLp 0) hm
 simp [ProbDistribution.coin, HermitianMat.diagonal, HermitianMat.lin] a
 exact this
· have := congrArg (·.ofLp 1) hm
 simp [ProbDistribution.coin, HermitianMat.diagonal, HermitianMat.lin] a
 exact this
convert! le_iInf_comp _ []dualSubtype
simp
simp only [← coe_le_coe, NNReal.coe_add, NNReal.coe_div, NNReal.coe_mul]
simp [toReal]
simp only [sub_add_cancel]

```

### 1.397 QuantumInfo/ResourceTheory/SteinsLemma.lean

```

public import QuantumInfo.ForMathlib.LimSupInf
public import QuantumInfo.ResourceTheory.FreeState
public import QuantumInfo.ResourceTheory.HypothesisTesting
 refine sandwichedRelEntropy_heq_congr h_Hn_eq ?_ ?_
 · rw [MState.relabel_cast]
 refine (cast_heq _ _).trans ?_

```

```

 congr 1
 · exact pow_mul i m (n / m)
 · exact statePow_mul ρ m (n / m)
 · rw [MState.relabel_cast]
 exact cast_heq _ _
 simp
 intro h
 have := congrArg Subtype.val h
 simp at this
 · rfl
simp_all only [one_div, Real.log_inv, Filter.eventually_atTop, Nat.cast_max, Nat.c
convert! Real.exp_monotone (log_le_f i n x)
convert! Real.exp_strictMono (f_le_log i n x) using 1
gcongr with k
gcongr with k
convert! le_max_right _ _
 rw [toReal, neg_add', Real.exp_sub, Real.exp_log (by positivity)]
 rw [toReal, neg_add', Real.exp_sub, Real.exp_log (by positivity)]
convert! ← hliminfP1 using 1
 exact (ENNReal.ofReal_eq_coe_nnreal _).symm
 · convert! hlimsupP2
convert! ← hlimsupP2' using 1
 convert! hliminf_g2
 convert! hliminf_g1 using 3 with n
 rw [Subtype.ext_iff, val_eq_coe, val_eq_coe, coe_zero]
 simp [toReal]
 linarith
 ENNReal.lt_add_right hR2 (by simp [← NNReal.coe_eq_zero, toReal, hε₀.ne'])
 · norm_cast
 exact hdpos n
exact zero_le
 simp [Prob.toNNReal]
 change (1 - ε' : ℝ) < 1
 simp [hε'.1]
 convert! h 0; simp

```

### 1.398 QuantumInfo/States/Ensemble.lean

```

abbrev MEnsemble.states : MEnsemble d α → (α → MState d) := ProbDistribution.RandVar
abbrev PEnsemble.states : PEnsemble d α → (α → Ket d) := ProbDistribution.RandVar.va
 apply AddSubgroup.val_finsetSum -- *laughs in defeq*
 simp_all only [ne_eq, Finset.mem_univ, smul_eq_zero, Set.Icc.coe_eq_zero, not_or
 forall_const]
 exact hne0

```

```
convert! (spectral_decomposition_sum p.Hermitian) using 1
```

### 1.399 QuantumInfo/States/Entanglement.lean

```
· simp only [pure_average_NNReal, Fin.isValue, ← NNReal.coe_le_coe]
 conv_lhs =>
 enter [1, 1]
 rw [trivial_pEnsemble_average ψ _ 0]
 simp only [pure_average_NNReal, ← NNReal.coe_inj]
 simp [mix_pEnsemble_pure_average (NNReal.toReal ∘ g ∘ KetUpToPhase.mk)]
· simp only [average_NNReal, Fin.isValue, ← NNReal.coe_le_coe]
 conv_lhs =>
 enter [1, 1]
 rw [trivial_mEnsemble_average _ (pure ψ) 0]
 simp only [average_NNReal, ← NNReal.coe_inj]
 conv_rhs =>
 enter [1, 1]
 rw [mix_mEnsemble_pure_average (toReal ∘ f) hmix, Function.comp_apply]
 rfl
convert! h_partial_trace i j
simp [traceRight_pure_MES]
```

### 1.400 QuantumInfo/States/Mixed/MState.lean

```
/-- An MState is a witness that d is nonempty. -/
@[implicit_reducible]
def nonempty : Nonempty d := by
 convert! Finset.single_le_sum (fun y _ => p.psd.eigenvalues_nonneg y) (Fin
 exact dotProduct_comm (φ : d → ℂ) (star (ψ : d → ℂ)))
· -- A state is pure iff its spectrum is constant.
 apply (pure_iff_constant_spectrum p).mpr
 convert! congr_arg Complex.re h_eigenvalues using 1;
 -- Each eigenvalue satisfies $\lambda_i (1 - \lambda_i) \geq 0$, and these terms sum to
 -- $\sum \lambda_i - \sum \lambda_i^2 = 1 - 1 = 0$, so every λ_i is 0 or 1 .
 have hbin : ∀ i, (p.spectrum i).val = 0 ∨ (p.spectrum i).val = 1 := by
 have hnn : ∀ i ∈ Finset.univ, 0 ≤ (p.spectrum i).val * (1 - (p.spectrum i).val)
 fun i _ => mul_nonneg (p.spectrum i).2.1 (by linarith [(p.spectrum i).val * (1 - (p.spectrum i).val) = 0])
 have hz : ∑ i, (p.spectrum i).val * (1 - (p.spectrum i).val) = 0 := by
 have e : ∑ i, (p.spectrum i).val * (1 - (p.spectrum i).val)
 = (∑ i, (p.spectrum i : ℝ)) - ∑ i, (p.spectrum i).val ^ 2 := by
 rw [← Finset.sum_sub_distrib]
 exact Finset.sum_congr rfl fun i _ => by ring
 rw [e, h_eigenvalues, p.spectrum.normalized]; norm_num
```

```

 exact fun i => (mul_eq_zero.mp ((Finset.sum_eq_zero_iff_of_nonneg hnn).mp hz i
 (Finset.mem_univ i))).imp id fun h => by linarith
 -- Since $\sum \lambda_i = 1$, not all eigenvalues vanish, so one of them equals `1`.
 by_contra hcon
 simp only [not_exists] at hcon
 simp [fun i => (hbin i).resolve_right (hcon i)] using p.spectrum.normalized
 exact ⟨i, ProbDistribution.constant_of_exists_one (Subtype.ext hi)⟩
 simp only [Function.comp_apply, mat_M, mat_apply, HermitianMat.smul_apply, Complex
 rw [AddSubgroup.val_finsetSum]
 exact MState.ext_m (by ext i j; exact h_simp i j)
 convert! congr_fun hz₂ (i, j) using 1;
 exact ξ'.normalized'
 rw [Finset.sum_eq_add_sum_sdiff_singleton i _ (by simp)] at h_diag;
 · convert! <Module.finrank $\mathbb{C}$ (LinearMap.range (Matrix.mulVecLin (Matrix.of fu
 convert! this using 1;
 have h1 := h_eigenvectorUnitary i j
 convert! h1 using 1;
 simp [mul_assoc]
 exact Finset.sum_comm
 convert! p.SWAP.assoc.SWAP.traceRight_assoc
theorem traceNorm_eq_one (p : MState d) : p.m.traceNorm = 1 :=

```

#### 1.401 QuantumInfo/States/Mixed/TraceDistance.lean

```

p.traceNorm_eq_one, σ.traceNorm_eq_one] at htri ⊢

```

#### 1.402 QuantumInfo/States/Pure/Braket.lean

```

coe_injective _ _ h := by rwa [Ket.mk.injEq]
coe_injective _ _ h := by rwa [Bra.mk.injEq]
⟨conj ψ, by simp_all; exact ψ.2⟩
⟨conj ψ, by simp_all; exact ψ.2⟩
coe_injective x y h := by
 rw [EuclideanSpace.norm_eq]; convert! Real.sqrt_one
 simp only [ketToEuclidean]; convert! ψ.normalized'

```

#### 1.403 QuantumInfo/States/Pure/Qubit.lean

```

convert! this

```

**1.404 README.md**

```
[![]](https://img.shields.io/badge/Lean-v4.31.0-blue)](https://github.com/leanprover/lean4)
PhyslibAlpha
```

PhyslibAlpha sits downstream of `./Physlib` (within the same repository). The standards then Physlib making it easier to contribute large PRs, AI-generated content, or formalizations which are not-quite-perfect. PRs for PhyslibAlpha must pass basic linter checks, described [here](https://github.com/leanprover/lean4/blob/master/scripts/PhyslibAlpha/README.md). PRs must also look like they have been reviewed. 1) Whether the content is main-stream physics, 2) Whether the content is well-written and clear.

Because of the lower-review bar for PhyslibAlpha we cannot promise to maintain a high level of review for PhyslibAlpha.

**1.405 docs/references.bib**

```
@Book{Reed1972,
 author = {Reed, Michael and Simon, Barry},
 title = {Functional Analysis},
 series = {Methods of Modern Mathematical Physics},
 volume = {1},
 year = {1972},
 publisher = {Academic Press},
 isbn = {978-0-12-585001-8},
 doi = {10.1016/B978-0-12-585001-8.X5001-6}
}

@Book{Schmudgen2012,
 author = {Konrad Schmudgen},
 title = {Unbounded Self-Adjoint Operators on Hilbert Space},
 series = {Graduate Texts in Mathematics},
 volume = {265},
 year = {2012},
 publisher = {Springer},
 address = {Dordrecht},
 isbn = {978-94-007-4753-1},
 doi = {10.1007/978-94-007-4753-1}
}
```



**1.406 lake-manifest.json**

```
{
 "version": "1.2.0",
 "rev": "fabf563a7c95a166b8d7b6efca11c8b4dc9d911f",
 "inputRev": "v4.31.0",
 "rev": "0bc516c1b9db83658d6475c40d9b1ed71219b921",
 "inputRev": "v4.31.0",
 "rev": "63045536fe95024e6c18fc7b48e03f506701c5bc",
 "rev": "5c7542ed018c78194f1e2b903eaf6a792b74c03d",
 "rev": "24b0d9dc081c5423f8eec7e866c441e5184f29d9",
 "inputRev": "main",
 "rev": "e3cb2f741431ce31bf73549fb52316a57368b06f",
 "rev": "f46324995fca5f0483b742e4eb4daec7f4ee50d2",
 "rev": "fa08db58b30eb033edcdab331bba000827f9f785",
 {
 "url": "https://github.com/leanprover/leansqlite",
 "scope": "",
 "rev": "0be4df908d1a8e75b58961041e2b4973692623df",
 "name": "leansqlite",
 "inputRev": "main",
 "configFile": "lakefile.lean"
 },
 {
 "url": "https://github.com/leanprover/lean4-cli",
 "scope": "leanprover",
 "rev": "92564e5770e4d09f2d86dfbf8adale9c715b384c",
 "name": "Cli",
 "inputRev": "v4.31.0",
 "configFile": "lakefile.toml"
 },
 "rev": "a2e430a4c9d3ad24078b8581fe0162fc5b0c9a6c",
 "lakeDir": ".lake",
 "fixedToolchain": false
}
```

**1.407 lakefile.toml**

```
name = "Physlib"
defaultTargets = ["Physlib", "QuantumInfo"]

[[require]]
name = "«doc-gen4»"
git = "https://github.com/leanprover/doc-gen4"
rev = "v4.31.0"

[[require]]
name = "mathlib"
git = "https://github.com/leanprover-community/mathlib4.git"
rev = "v4.31.0"
```

```
[[lean_lib]]
name = "Physlib"
moreLeanArgs = ["-Dwarn.sorry=false", "-Dweak.says.verify=true"]

[[lean_lib]]
name = "PhyslibAlpha"
moreLeanArgs = ["-Dwarn.sorry=false", "-Dweak.says.verify=true"]

[[lean_lib]]
name = "QuantumInfo"
moreLeanArgs = ["-Dwarn.sorry=false", "-Dweak.says.verify=true"]

[[lean_exe]]
name = "check_file_imports"
srcDir = "scripts"

[[lean_exe]]
name = "style_lint"
srcDir = "scripts/MetaPrograms"

[[lean_exe]]
name = "type_former_lint"
srcDir = "scripts"

[[lean_exe]]
name = "lint_all"
srcDir = "scripts"
supportInterpreter = true

[[lean_exe]]
name = "check_dup_tags"
srcDir = "scripts/MetaPrograms"
supportInterpreter = true

[[lean_exe]]
name = "sorry_lint"
srcDir = "scripts/MetaPrograms"
supportInterpreter = true

[[lean_exe]]
name = "runPhyslibLinters"
srcDir = "scripts/MetaPrograms"
supportInterpreter = true

[[lean_exe]]
```

```
name = "runPhyslibAlphaLinters"
srcDir = "scripts/PhyslibAlpha"
supportInterpreter = true
```

```
[[lean_exe]]
name = "free_simps"
srcDir = "scripts/MetaPrograms"
```

```
[[lean_exe]]
name = "spelling"
srcDir = "scripts/MetaPrograms"
supportInterpreter = true
```

```
[[lean_exe]]
name = "check_rfl"
srcDir = "scripts/MetaPrograms"
```

```
[[lean_exe]]
name = "redundant_imports"
srcDir = "scripts/MetaPrograms"
supportInterpreter = true
```

```
[[lean_exe]]
name = "module_doc_lint"
srcDir = "scripts/MetaPrograms"
```

```
[[lean_exe]]
name = "stats"
srcDir = "scripts"
supportInterpreter = true
```

```
[[lean_exe]]
name = "local_stats"
srcDir = "scripts/MetaPrograms"
supportInterpreter = true
```

```
[[lean_exe]]
name = "make_tag"
srcDir = "scripts/MetaPrograms"
supportInterpreter = true
```

```
[[lean_exe]]
name = "TODO_to_yaml"
srcDir = "scripts/MetaPrograms"
supportInterpreter = true
```

```

[[lean_exe]]
name = "informal"
srcDir = "scripts/MetaPrograms"
supportInterpreter = true

[[lean_exe]]
name = "find_TODOs"
srcDir = "scripts"

[[lean_exe]]
name = "notes"
srcDir = "scripts/MetaPrograms"
supportInterpreter = true

[[lean_exe]]
name = "mathlib_textLint_on_physlib"
srcDir = "scripts"

[[lean_exe]]
name = "no_docs"
srcDir = "scripts/MetaPrograms"

```

#### 1.408 lean-toolchain

leanprover/lean4:v4.31.0

#### 1.409 scripts/LinterExemption.txt

```

QuantumInfo/Capacity/Capacity.lean
QuantumInfo/Capacity/Capacity_doc.lean
QuantumInfo/Channels/Bundled.lean
QuantumInfo/Channels/CPTP.lean
QuantumInfo/Channels/DegradableOrder.lean
QuantumInfo/Channels/Dual.lean
QuantumInfo/Channels/MatrixMap.lean
QuantumInfo/Channels/Pinching.lean
QuantumInfo/Channels/Unbundled.lean
QuantumInfo/ClassicalInfo/Capacity.lean
QuantumInfo/ClassicalInfo/Channel.lean
QuantumInfo/ClassicalInfo/Distribution.lean
QuantumInfo/ClassicalInfo/Entropy.lean
QuantumInfo/ClassicalInfo/ForMathlib/Analysis/SpecialFunctions/Log/NegMulLo
QuantumInfo/ClassicalInfo/Prob.lean

```

QuantumInfo/Entropy/Axiomatized/Defs.lean  
 QuantumInfo/Entropy/Axiomatized/Renyi.lean  
 QuantumInfo/Entropy/DPI.lean  
 QuantumInfo/Entropy/Relative.lean  
 QuantumInfo/Entropy/SSA.lean  
 QuantumInfo/Entropy/VonNeumann.lean  
 QuantumInfo/ForMathlib/ComplexLaplaceTransform.lean  
 QuantumInfo/ForMathlib/ContinuousSup.lean  
 QuantumInfo/ForMathlib/Filter.lean  
 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/GeneralizedPerspectiveFunction.lean  
 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/HilbertSchmidtOperatorSpace.lean  
 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperatorInequality.lean  
 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperatorInequalityIImpIV.lean  
 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/JensenOperatorInequalityIVtoV.lean  
 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LiebAndoTrace.lean  
 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LownerHeinzCore.lean  
 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/LownerHeinzTheorem.lean  
 QuantumInfo/ForMathlib/HayataGroup/TraceInequality/OperatorGeometricMean.lean  
 QuantumInfo/ForMathlib/HermitianMat.lean  
 QuantumInfo/ForMathlib/HermitianMat/Basic.lean  
 QuantumInfo/ForMathlib/HermitianMat/CFC.lean  
 QuantumInfo/ForMathlib/HermitianMat/Inner.lean  
 QuantumInfo/ForMathlib/HermitianMat/Jordan.lean  
 QuantumInfo/ForMathlib/HermitianMat/LiebConcavity.lean  
 QuantumInfo/ForMathlib/HermitianMat/LogExp.lean  
 QuantumInfo/ForMathlib/HermitianMat/NonSingular.lean  
 QuantumInfo/ForMathlib/HermitianMat/Order.lean  
 QuantumInfo/ForMathlib/HermitianMat/Peierls.lean  
 QuantumInfo/ForMathlib/HermitianMat/Proj.lean  
 QuantumInfo/ForMathlib/HermitianMat/Reindex.lean  
 QuantumInfo/ForMathlib/HermitianMat/Rpow.lean  
 QuantumInfo/ForMathlib/HermitianMat/Schatten.lean  
 QuantumInfo/ForMathlib/HermitianMat/Sqrt.lean  
 QuantumInfo/ForMathlib/HermitianMat/Trace.lean  
 QuantumInfo/ForMathlib/HermitianMat/Unitary.lean  
 QuantumInfo/ForMathlib/Isometry.lean  
 QuantumInfo/ForMathlib/LimSupInf.lean  
 QuantumInfo/ForMathlib/Majorization.lean  
 QuantumInfo/ForMathlib/Matrix.lean  
 QuantumInfo/ForMathlib/MatrixNorm/TraceNorm.lean  
 QuantumInfo/ForMathlib/Minimax.lean  
 QuantumInfo/ForMathlib/Misc.lean  
 QuantumInfo/ForMathlib/SionMinimax.lean  
 QuantumInfo/ForMathlib/Superadditive.lean  
 QuantumInfo/ForMathlib/Tactic/Commutes.lean  
 QuantumInfo/ForMathlib/Tactic/Commutes/Attribute.lean

```

QuantumInfo/ForMathlib/Unitary.lean
QuantumInfo/Measurements/POVM.lean
QuantumInfo/Regularized.lean
QuantumInfo/ResourceTheory/FreeState.lean
QuantumInfo/ResourceTheory/HypothesisTesting.lean
QuantumInfo/ResourceTheory/ResourceTheory.lean
QuantumInfo/ResourceTheory/SteinsLemma.lean
QuantumInfo/States/Ensemble.lean
QuantumInfo/States/Entanglement.lean
QuantumInfo/States/Mixed/Fidelity.lean
QuantumInfo/States/Mixed/MState.lean
QuantumInfo/States/Mixed/TraceDistance.lean
QuantumInfo/States/Pure/BargmannInvariant.lean
QuantumInfo/States/Pure/BlochSphere.lean
QuantumInfo/States/Pure/Bracket.lean
QuantumInfo/States/Pure/Qubit.lean

```

#### 1.410 scripts/MetaPrograms/TODO\_to\_yaml.lean

```

| PhyslibAlpha
| PhyslibAlpha => "Physlib Alpha"
| PhyslibAlpha => ""
 PhyslibCategory.PhyslibAlpha,
let s : String := n.toString
if s.startsWith "Physlib.ClassicalMechanics" then
else if s.startsWith "Physlib.CondensedMatter" then
else if s.startsWith "Physlib.Cosmology" then
else if s.startsWith "Physlib.Electromagnetism" then
else if s.startsWith "Physlib.Mathematics" then
else if s.startsWith "Physlib.Meta" then
else if s.startsWith "Physlib.Optics" then
else if s.startsWith "Physlib.Particles" then
else if s.startsWith "Physlib.QFT" then
else if s.startsWith "Physlib.QuantumMechanics" then
else if s.startsWith "Physlib.Relativity" then
else if s.startsWith "Physlib.StatisticalMechanics" then
else if s.startsWith "Physlib.Thermodynamics" then
else if s.startsWith "Physlib.QuantumInfo" then
else if s.startsWith "PhyslibAlpha" then
 PhyslibCategory.PhyslibAlpha
Lean.enableInitializersExecution
let env ← importModules (loadExts := true) #['Physlib, `QuantumInfo, `Phy

```

**1.411 scripts/MetaPrograms/check\_dup\_tags.lean**

```
Lean.enableInitializersExecution
let env ← importModules (loadExts := true) #['Physlib, `QuantumInfo] {} 0
```

**1.412 scripts/MetaPrograms/informal.lean**

```
Lean.enableInitializersExecution
```

**1.413 scripts/MetaPrograms/local\_stats.lean**

```
Lean.enableInitializersExecution
```

**1.414 scripts/MetaPrograms/module\_doc\_lint.lean**

```
/- The array of modules exempt from all linters, read from `scripts/LinterExemption
This is used to lint `QuantumInfo` file-by-file. -/
def linterExemptions : IO (Array FilePath) := do
 let path := (mkFilePath ["scripts", "LinterExemption"]).addExtension "txt"
 unless (← path.pathExists) do return #[]
 let lines ← IO.FS.lines path
 return lines.filterMap (fun l ↦ if l.trim == "" then none else some (mkFilePath [l

/- The file paths of the modules imported into the module `mods` (e.g. `Physlib`).
-/
def importedFilePaths (mods : Name) : IO (Array FilePath) := do
 let (modData, _) ← readModuleData mFile
 return modData.imports.filterMap (fun imp ↦

def main (_ : List String) : IO UInt32 := do
 initSearchPath (← findSysroot)
 let filePaths := (← importedFilePaths `Physlib) ++ (← importedFilePaths `QuantumIn
 let exemptions ← linterExemptions
 let modulesToCheck := filePaths.filter (fun p ↦ !noLint.contains p ∧ !exemptions.c
```

**1.415 scripts/MetaPrograms/notes.lean**

```
Lean.enableInitializersExecution
```

**1.416 scripts/MetaPrograms/redundant\_imports.lean**

```
Lean.enableInitializersExecution
```

**1.417 scripts/MetaPrograms/runPhyslibLinters.lean**

```

/-- The file paths of modules exempt from all linters, read from `scripts/L
This is used to lint `QuantumInfo` file-by-file. -/
def linterExemptions : IO (Array String) := do
 let path : FilePath := (System.mkFilePath ["scripts", "LinterExemption"])
 unless (← path.pathExists) do return #[]
 let lines ← IO.FS.lines path
 return lines.filterMap (fun l ↦ if l.trimAscii.copy == "" then none else

/-- The file path (as a string e.g. `QuantumInfo/Foo/Bar.lean`) of the modu
/
def moduleToFilePathString (mod : Name) : String :=
 ((System.mkFilePath (mod.toString.splitToList (· == '.'))).addExtension "

/-- Run the Batteries linter on a given module, skipping declarations in ex
Returns `true` if linting failed. -/
unsafe def runLinterOnModule (module : Name) (exemptions : Array String) :
 Lean.enableInitializersExecution
 let env ← getEnv
 -- Skip declarations whose source module is listed in `scripts/LinterEx
 let decls := decls.filter fun n =>
 match env.getModuleIdxFor? n with
 | some idx => !exemptions.contains (moduleToFilePathString env.header
 | none => true
 -- The `defsWithUnderscore` linter flags any `def`/`instance`/structure
projection whose name
 -- contains an underscore. It produces many false positives in Physlib
 -- `_physlib` instance suffix, mirroring mathlib's exempted `_mathlib`,
 -- which elaborate to `def`s but follow the snake_case lemma convention
 let linters := linters.filter (·.name != `defsWithUnderscore)
 return true
 return false
let exemptions ← linterExemptions
let modulesToLint := #[`Physlib, `QuantumInfo]
let failures ← modulesToLint.mapM (runLinterOnModule · exemptions)
if failures.any id then
 IO.Process.exit 1
else
 IO.Process.exit 0

```



**1.418 scripts/MetaPrograms/sorry\_lint.lean**

```
Lean.enableInitializersExecution
```

**1.419 scripts/MetaPrograms/spelling.lean**

```
Lean.enableInitializersExecution
```

**1.420 scripts/MetaPrograms/spellingWords.txt**

```
DualLeftHandedWeyl
DualRightHandedWeyl
comoving
conformal
cosmological
critical
distances
dust
friedmann
horizons
lookback
luminosity
milne
notions
observational
radiation
redshift
sitter
universe
accelerated
cosmic
dynamical
eternal
fluid
qualitative
singularity
```

**1.421 scripts/MetaPrograms/style\_lint.lean**

```
if String.contains l.trimAsciiStart.copy " " then
```

```

 if l.length > 100 ∧ ¬ String.contains l "http" then
 if String.contains l s then
/- The file paths of the modules imported into the module `mods` (e.g. `Ph
 found by reading the module's `.olean` file. -/
def importedFilePaths (mods : Name) : IO (Array FilePath) := do
 let (modData, _) ← readModuleData mFile
 return modData.imports.filterMap (fun imp ↦

/- The file paths of modules which should be skipped by the linters, read
 `scripts/LinterExemption.txt`. This is used to lint `QuantumInfo` file-
 by-file. -/
def linterExemptions : IO (Array String) := do
 let path : FilePath := mkFilePath ["scripts", "LinterExemption.txt"]
 unless (← path.pathExists) do return #[]
 let lines ← IO.FS.lines path
 return lines.filterMap (fun l ↦ if l.trimAscii.copy == "" then none else

def main (_ : List String) : IO UInt32 := do
 initSearchPath (← findSysroot)
 let filePaths := (← importedFilePaths `Physlib) ++ (← importedFilePaths `
 let exemptions ← linterExemptions
 let filePaths := filePaths.filter (fun p ↦ !exemptions.contains p.toStrin

```

### 1.422 scripts/PhyslibAlpha/README.md

# Meta programs for PhyslibAlpha

As well as passing a one-look review, code in PhyslibAlpha has to pass a nu  
which we describe in this file.

## Lean-based linters

### runPhyslibAlphaLinters

```

lake exe runPhyslibAlphaLinters

```

This picks up things like lack of doc-strings on definitions, or incompatib

## Python-based linters

### alphaFileImports.py

```

./scripts/PhyslibAlpha/alphaFileImports.py

```
```
```

This checks that all PhyslibAlpha files are included in the file `PhyslibAlpha.lean` even if commented out based out with info of the commit where they broke.

```
noAlphaImports.py
```

```
```
```

```
./scripts/PhyslibAlpha/noAlphaImports.py
```

```
```
```

This checks that no file in `./Physlib` or `./QuantumInfo` imports a file from `./Ph

```
alphaPythonLinters.sh
```

```
```
```

```
./scripts/PhyslibAlpha/alphaPythonLinters.sh
```

```
```
```

Checks things like line length, `simp`s which are not `simp only` or final tactics.

### 1.423 scripts/PhyslibAlpha/alphaFileImports.py

```
#!/usr/bin/env python3
```

```
import os
```

```
import re
```

```
from pathlib import Path
```

```
def main():
```

```
 # Get the current working directory
```

```
 script_dir = Path.cwd()
```

```
 physlib_alpha_dir = script_dir / "PhyslibAlpha"
```

```
 physlib_alpha_lean = script_dir / "PhyslibAlpha.lean"
```

```
 # Check if PhyslibAlpha directory exists
```

```
 if not physlib_alpha_dir.exists():
```

```
 print(f"Error: {physlib_alpha_dir} directory not found")
```

```
 return False
```

```
 # Check if PhyslibAlpha.lean file exists
```

```
 if not physlib_alpha_lean.exists():
```

```
 print(f"Error: {physlib_alpha_lean} file not found")
```

```
 return False
```

```
 # Get all .lean files in PhyslibAlpha directory (recursively, including nested s
```

```
 # and convert each to its module name, e.g. PhyslibAlpha/Sub/File.lean -> Physli
```

```

lean_modules = set()
for f in physlib_alpha_dir.rglob("*.lean"):
 rel = f.relative_to(physlib_alpha_dir).with_suffix("")
 module_name = ".".join(("PhyslibAlpha",) + rel.parts)
 lean_modules.add(module_name)

if not lean_modules:
 print(f"No .lean files found in {physlib_alpha_dir}")
 return True

Read PhyslibAlpha.lean and extract imports
with open(physlib_alpha_lean, 'r') as f:
 content = f.read()

Extract import statements (looking for "import PhyslibAlpha.<module>"
import_pattern = r'import\s+(PhyslibAlpha(?:\.\w+)+)'
imported_modules = set(re.findall(import_pattern, content))

Check for missing imports
missing = lean_modules - imported_modules

if missing:
 print(f"Error: The following .lean files are not imported in {physlib_alpha_dir}")
 for module_name in sorted(missing):
 print(f" - public import {module_name}")
 return False
else:
 print(f"✓ All {len(lean_modules)} .lean files in {physlib_alpha_dir} imported")
 return True

if __name__ == "__main__":
 success = main()
 exit(0 if success else 1)

```

#### 1.424 scripts/PhyslibAlpha/alphaPythonLinters.sh

```
#!/usr/bin/env bash
```

```
set -exo pipefail
```

```
This is adapted from the linter for mathlib4.
```

```
#####
```

```

1. Call the Lean file linter, implemented in Python

touch scripts/style-exceptions.txt

git ls-files 'PhyslibAlpha/*.lean' | xargs ./scripts/lint-
style.py "$@"

2. Global checks on the PhyslibAlpha repository

2.1 Check for executable bit on Lean files

executable_files="$(find . -name '*.lean' -type f \(-perm -
u=x -o -perm -g=x -o -perm -o=x \))"

if [[-n "$executable_files"]]
then
 echo "ERROR: The following Lean files have the executable bit set."
 echo "$executable_files"
 exit 1
fi

2.2 Check that there are no filenames with the same lower-
case reduction

`uniq -D`: print all duplicate lines
ignore_case_clashes="$(git ls-files | sort --ignore-case | uniq -
D --ignore-case)"

if [-n "${ignore_case_clashes}"]; then
 printf '$The following files have the same lower-case form:\n\n%s\n
Please, make sure to avoid such clashes!\n' "${ignore_case_clashes}"
 exit 1
fi

```

### 1.425 scripts/PhyslibAlpha/noAlphaImports.py

```

#!/usr/bin/env python3
"""

```

This module validates that no files in the Physlib and QuantumInfo directories contain import statements that reference PhyslibAlpha. It walks through all .lean files in these directories and reports any violations found, returning an exit code indicating success or failure of the validation check.

It checks each line in these files.

```

"""
import os
import sys

def check_no_alpha_imports():
 """Check that no files in ./Physlib or ./QuantumInfo contain lines with
 directories = ['./Physlib', './QuantumInfo']
 violations = []

 for directory in directories:
 if not os.path.exists(directory):
 print(f"Warning: Directory {directory} does not exist")
 continue

 for root, dirs, files in os.walk(directory):
 for file in files:
 if file.endswith('.lean'):
 filepath = os.path.join(root, file)
 try:
 with open(filepath, 'r', encoding='utf-
8') as f:
 for line_num, line in enumerate(f, 1):
 if 'import' in line and 'PhyslibAlpha' in line:
 violations.append((filepath, line_num,
 except Exception as e:
 print(f"Error reading {filepath}: {e}")

 if violations:
 print("Found violations:")
 for filepath, line_num, line in violations:
 print(f" {filepath}:{line_num}: {line}")
 return False
 else:
 print("No violations found. All files passed the check.")
 return True

if __name__ == '__main__':
 success = check_no_alpha_imports()
 sys.exit(0 if success else 1)

```

#### 1.426 scripts/PhyslibAlpha/runPhyslibAlphaLinters.lean

```

import Batteries.Tactic.Lint
import Batteries.Data.Array.Basic

```

```
import Lake.CLI.Main
/-!
```

A minimized version of the Batteries script runLinter dedicated to Physlib.

Made as an attempt to overcome issues outline here:

<https://leanprover.zulipchat.com/#narrow/channel/113488-general/topic/runLinter.20in>

```
-/
open Lean Core Elab Command Batteries.Tactic.Lint
open System (FilePath)

open Lake

/-- Run the Batteries linter on a given module and update the linter if `update` is
/
unsafe def runLinterOnModule (module : Name): IO Unit := do
 initSearchPath (← findSysroot)
 Lean.enableInitializersExecution
 -- If the linter is being run on a target that doesn't import `Batteries.Tactic.Li
 -- the linters are ineffective. So we import it here.
 let lintModule := `Batteries.Tactic.Lint
 let lintFile ← find0Lean lintModule
 unless (← lintFile.pathExists) do
 -- run `lake build +Batteries.Tactic.Lint` (and ignore result) if the file hasn'
 let child ← IO.Process.spawn {
 cmd := (← IO.getEnv "LAKE").getD "lake"
 args := #["build", s!"+{lintModule}"]
 stdin := .null
 }
 _ ← child.wait
 let nolints := #[]
 let env ← importModules #[module, lintModule] {} (trustLevel := 1024) (loadExts :=
 let ctx := { fileName := "", fileMap := default }
 let state := { env }
 Prod.fst <$> (CoreM.toIO · ctx state) do
 let decls ← getDeclsInPackage module.getRoot
 let linters ← getChecks (slow := true) (runAlways := none) (runOnly := none)
 -- The `defsWithUnderscore` linter flags any `def`/`instance`/structure-
 projection whose name
 -- contains an underscore. It produces many false positives in Physlib (e.g. the
 -- `_physlib` instance suffix, mirroring mathlib's exempted `_mathlib`, and `inf
 -- which elaborate to `def`s but follow the snake_case lemma convention). Disabl
 let linters := linters.filter (·.name != `defsWithUnderscore)
 println! "Results been linted with the following linters:"
 println! linters.map (·.name)
```

```

println! "Starting parallel running on linters on all declarations. Results
 shown below."
let results ← lintCore decls linters
let results := results.map fun (linter, decls) =>
 .mk linter <| nolints.foldl (init := decls) fun decls (linter', decl') =>
 if linter.name == linter' then decls.erase decl' else decls
let failed := results.any (!_.2.isEmpty)
if failed then
 let fmtResults ←
 formatLinterResults results decls (groupByFilename := true) (useErrors := true)
 s!"in {module}" (runSlowLinters := true) .medium linters.size
 IO.print (← fmtResults.toString)
 IO.Process.exit 1
else
 IO.println s!"-- Linting passed for {module}."
 IO.Process.exit 0

unsafe def main (_ : List String) : IO Unit := do
 let modulesToLint := #[`PhyslibAlpha]
 modulesToLint.forM <| runLinterOnModule

```

### 1.427 scripts/README.md

- step 3: Checks all files are imported to `Physlib.lean`.

### 1.428 scripts/check\_file\_imports.lean

```

-- Only `.lean` files correspond to modules; skip any other files (e.g.
-- `API-map.yaml` sitting at the top of an API directory).
if entry.path.extension == some "lean" then
 let .some mod := FilePath.fileStem entry.fileName
 | continue
 r := r.push (root.mkStr mod)
else if top.path.extension == some "lean" then
 needed := needed.push root

```

### 1.429 scripts/dag\_traversal.py

```

#!/usr/bin/env python3
"""
Reusable parallel DAG traversal for Lean import graphs.

```



Parses the import DAG from ``lean`` source files and parallelizes an action over a forward or backward traversal. Each module is submitted to the thread pool the moment its last in-DAG dependency finishes, giving true maximum parallelism.

CLI usage:

```
dag_traversal.py --forward 'lake build {}'
dag_traversal.py --forward -j4 'echo {}'
```

By default, `{}` is replaced with the file path (e.g. `Mathlib/Foo/Bar.lean`). Use `--module` to substitute the module name instead (e.g. `Mathlib.Foo.Bar`):

```
dag_traversal.py --backward --module 'my_script {}'
```

Library usage:

```
from dag_traversal import DAG, DAGTraverser

dag = DAG.from_directories(Path("."))
traverser = DAGTraverser()
results = traverser.traverse(dag, my_action, direction="backward")
"""
```

```
import argparse
import atexit
import heapq
import json
import os
import shlex
import signal
import subprocess
import sys
import time
from concurrent.futures import Future, ThreadPoolExecutor
from dataclasses import dataclass, field
from pathlib import Path
from threading import Event, Lock
from typing import Callable
```

```
ANSI escape codes
CLEAR_LINE = "\033[2K"
HIDE_CURSOR = "\033[?25l"
SHOW_CURSOR = "\033[?25h"
```

```
class ShutdownError(Exception):
 """Raised when a shutdown has been requested (e.g. Ctrl-
```

C)."""

```
class DAGTraverser:
 """Per-session state for parallel DAG traversal.

 Encapsulates shutdown coordination, in-flight file tracking, and
 active build management so that independent traversals don't share
 locks or state.
 """

 def __init__(self):
 #: Set on KeyboardInterrupt. Action callbacks can check this
 #: to bail out early instead of spawning new subprocesses.
 self.shutdown_event = Event()

 # Thread-safe registry of in-flight file modifications.
 # On abnormal exit (e.g. Ctrl-C), files get reverted.
 self._inflight_lock = Lock()
 self._inflight: dict[Path, str] = {} # filepath -> original content

 # Active lake build subprocesses, tracked so force_exit can kill them
 self._active_builds_lock = Lock()
 self._active_builds: set[subprocess.Popen] = set()

 atexit.register(self._revert_inflight)

 def inflight_register(self, path: Path, content: str):
 """Register original file content so it can be reverted on abnormal
 exit.
 """
 with self._inflight_lock:
 self._inflight[path] = content

 def inflight_unregister(self, path: Path):
 """Unregister a file after processing completes normally."""
 with self._inflight_lock:
 self._inflight.pop(path, None)

 def _revert_inflight(self):
 with self._inflight_lock:
 for path, content in self._inflight.items():
 try:
 path.write_text(content)
 except Exception:
 pass
 self._inflight.clear()
```

```

def _kill_builds(self):
 """Kill all active lake build process groups."""
 with self._active_builds_lock:
 for proc in self._active_builds:
 try:
 os.killpg(proc.pid, signal.SIGKILL)
 except (ProcessLookupError, PermissionError):
 try:
 proc.kill()
 except ProcessLookupError:
 pass

def force_exit(self, code: int = 1):
 """Kill active builds, revert in-flight files, and force-
 exit.

 Uses os._exit to bypass ThreadPoolExecutor's atexit handler, which
 would block joining worker threads that may still be running builds.
 """
 self._kill_builds()
 self._revert_inflight()
 sys.stdout.flush()
 sys.stderr.flush()
 os._exit(code)

def lake_build(
 self, module_name: str, project_dir: Path, timeout: int = 600,
) -> tuple[bool, str]:
 """Run lake build for a module. Returns (success, output).

 Checks shutdown_event before spawning and polls it every 0.5s during
 the build, killing the subprocess and raising ShutdownError if set.
 Each build runs in its own process group (start_new_session) so the
 entire tree (lake + lean children) can be killed cleanly.
 """
 if self.shutdown_event.is_set():
 raise ShutdownError("shutdown requested")
 proc = subprocess.Popen(
 ["lake", "build", module_name],
 stdout=subprocess.PIPE,
 stderr=subprocess.STDOUT,
 text=True,
 cwd=project_dir,
 start_new_session=True,
)
 with self._active_builds_lock:

```

```

 self._active_builds.add(proc)
 try:
 deadline = time.monotonic() + timeout
 # We poll communicate() with a short timeout instead of using
 # subprocess.run(timeout=...) so we can check shutdown_event
 # between iterations – this lets Ctrl-C abort promptly even
 # during long builds.
 while True:
 remaining = deadline - time.monotonic()
 if remaining <= 0:
 return False, "Build timed out"
 try:
 stdout, _ = proc.communicate(timeout=min(remaining, 0.5))
 return proc.returncode == 0, stdout
 except subprocess.TimeoutExpired:
 if self.shutdown_event.is_set():
 raise ShutdownError("shutdown requested")
 finally:
 with self._active_builds_lock:
 self._active_builds.discard(proc)
 if proc.poll() is None:
 try:
 os.killpg(proc.pid, signal.SIGKILL)
 except (ProcessLookupError, PermissionError):
 proc.kill()
 proc.wait()

def traverse(
 self,
 dag: "DAG",
 action: Callable[[str, Path], object],
 direction: str = "backward",
 max_workers: int | None = None,
 module_callback: Callable[[str, object, Exception | None], None] | None = None,
 progress_callback: Callable[[int, int, int], None] | None = None,
 stop_on_failure: bool = False,
 skip: set[str] | None = None,
 weights: dict[str, int] | None = None,
) -> list["TraversalResult"]:
 """Process modules in DAG order with maximum parallelism.

```

When more modules are ready than workers available, modules with longer successor chains are scheduled first (critical-path priority), ensuring that work which unblocks the most future parallelism runs earliest.

## Args:

dag: The DAG to traverse.  
 action: Callable(module\_name, filepath) -> result. Raise to signal failure.  
 direction: "backward" (downstream first) or "forward" (upstream first).  
 max\_workers: Thread pool size (default: cpu\_count).  
 module\_callback: Called after each module as (module\_name, result, error).  
 progress\_callback: Called after each module as (completed, total, inflight).  
 stop\_on\_failure: If True, when a module fails (action raises), its successors in the DAG are skipped.  
 skip: Set of module names to mark as completed without running the action. Useful after an initial build (e.g. a full ``lake build`` identifies which modules are already clean).  
 weights: Optional per-module weights for critical-path priority.  
 Modules not in the dict default to weight 0 (e.g. skip-set modules). When None, uniform weight 1 is used.

## Returns:

List of TraversalResult for all processed modules.

"""

```
all_names = set(dag.modules.keys())
```

```
if direction == "backward":
```

```
 # Process modules that nothing depends on first.
```

```
 # A module's "dependencies" are its importers (things that import it);
```

```
 # once all importers are done, this module can run.
```

```
 deps_of = {
```

```
 name: [m for m in info.importers if m in all_names]
```

```
 for name, info in dag.modules.items()
```

```
 }
```

```
 successors_of = {
```

```
 name: [m for m in info.imports if m in all_names]
```

```
 for name, info in dag.modules.items()
```

```
 }
```

```
elif direction == "forward":
```

```
 # Process modules with no imports first.
```

```
 deps_of = {
```

```
 name: [m for m in info.imports if m in all_names]
```

```
 for name, info in dag.modules.items()
```

```
 }
```

```
 successors_of = {
```

```
 name: [m for m in info.importers if m in all_names]
```

```
 for name, info in dag.modules.items()
```

```

 }
else:
 raise ValueError(f"Unknown direction: {direction!r}")

if max_workers is None:
 max_workers = os.cpu_count() or 4

total = len(dag.modules)
if total == 0:
 return []

Pre-compute critical-path depths so we can prioritise modules
that unblock the costliest chains of future work.
depths = _compute_depths(successors_of, weights)

lock = Lock()
done_event = Event()
remaining_deps: dict[str, int] = {
 name: len(deps_of[name]) for name in dag.modules
}
all_results: list[TraversalResult] = []
completed_count = 0
inflight = 0
skipped: set[str] = set()

Priority queue of ready modules: (-depth, name). Negative depth
gives us a max-heap so deeper chains are scheduled first.
ready_queue: list[tuple[int, str]] = []

def resolve(rel: Path) -> Path:
 if dag.project_root:
 return dag.project_root / rel
 return rel

def mark_skipped(name: str):
 """Iteratively mark a module and all its successors as skipped.
 stack = [name]
 while stack:
 n = stack.pop()
 if n in skipped:
 continue
 skipped.add(n)
 for succ in successors_of.get(n, []):
 if succ in remaining_deps and succ not in skipped:
 stack.append(succ)

```

```

def _drain_ready() -> list[str]:
 """Pop modules from the priority queue up to available workers.

 Must be called with *lock* held. Returns the names to submit
 (submission itself happens outside the lock).
 """
 nonlocal inflight
 to_submit: list[str] = []
 while ready_queue and inflight < max_workers:
 _, name = heapq.heappop(ready_queue)
 inflight += 1
 to_submit.append(name)
 return to_submit

def _do_submit(name: str):
 nonlocal inflight
 info = dag.modules[name]
 try:
 fut = executor.submit(action, name, resolve(info.filepath))
 except RuntimeError:
 # Executor shut down (e.g. KeyboardInterrupt race) – give back
 # the inflight slot so the scheduler doesn't deadlock.
 with lock:
 inflight -= 1
 return
 fut.add_done_callback(lambda f, n=name: on_done(f, n))

def on_done(future: Future, module_name: str):
 nonlocal completed_count, inflight

 filepath = resolve(dag.modules[module_name].filepath)
 try:
 result = future.result()
 tr = TraversalResult(module_name, filepath, result, None)
 except Exception as e:
 tr = TraversalResult(module_name, filepath, None, e)

 failed = tr.error is not None

 with lock:
 all_results.append(tr)
 completed_count += 1
 inflight -= 1

 if stop_on_failure and failed:
 # Don't unlock successors – mark them as skipped

```

```

 for succ in successors_of.get(module_name, []):
 if succ in remaining_deps:
 mark_skipped(succ)
 else:
 # Decrement dep counts and collect newly ready modules.
 # Skip-set modules that become ready are completed inli
 # (no action) and their successors are cascaded.
 cascade = [module_name]
 while cascade:
 current = cascade.pop()
 for succ in successors_of.get(current, []):
 if succ in remaining_deps and succ not in skip
 remaining_deps[succ] -= 1
 if remaining_deps[succ] == 0:
 if succ in skip_set:
 fp = resolve(dag.modules[succ].file
 all_results.append(TraversalResult(
 completed_count += 1
 cascade.append(succ)
 else:
 heapq.heappush(
 ready_queue,
 (-depths.get(succ, 0), succ),
)

 cc = completed_count
 sc = len(skipped)
 to_submit = _drain_ready()
 ic = inflight

 if cc + sc >= total:
 done_event.set()

 if module_callback:
 module_callback(module_name, tr.result, tr.error)
 if progress_callback:
 progress_callback(cc, total - sc, ic)

 for name in to_submit:
 _do_submit(name)

Process skip modules before starting the executor. We iterate in
topological order (BFS from zero-dep seeds) so that each skipped
module's successors have their dep counts correctly decremented.
skip_set = skip or set()

```



```

if skip_set:
 from collections import deque

 skip_queue: deque[str] = deque(
 name for name, count in remaining_deps.items()
 if count == 0 and name in skip_set
)
 while skip_queue:
 name = skip_queue.popleft()
 filepath = resolve(dag.modules[name].filepath)
 all_results.append(TraversalResult(name, filepath))
 completed_count += 1
 for succ in successors_of.get(name, []):
 if succ in remaining_deps:
 remaining_deps[succ] -= 1
 if remaining_deps[succ] == 0 and succ in skip_set:
 skip_queue.append(succ)

 # Update display so it doesn't sit at [0/N] during skip processing.
 if progress_callback:
 progress_callback(completed_count, total, 0)

executor = ThreadPoolExecutor(max_workers=max_workers)
try:
 # Seed the priority queue with all zero-dep modules.
 seeds = [
 name for name, count in remaining_deps.items()
 if count == 0 and name not in skip_set
]
 if not seeds and completed_count < total:
 raise ValueError(
 f"No modules with zero dependencies -- possible cycle in DAG "
 f"({total} modules)"
)
 if not seeds:
 done_event.set()

 with lock:
 for name in seeds:
 heapq.heappush(ready_queue, (-depths.get(name, 0), name))
 to_submit = _drain_ready()

 for name in to_submit:
 _do_submit(name)

 # Update display to show inflight count now that seeds are submitted.

```

```

 if progress_callback and to_submit:
 with lock:
 progress_callback(completed_count, total - len(skipped))

 # Wait until all modules are processed or skipped.
 # We cannot use executor.shutdown(wait=True) here because on_do
 # callbacks submit new futures – shutdown would reject them.
 # Use a timeout loop so the main thread can receive KeyboardInterrupt
 # (Event.wait() without timeout blocks signal delivery on Linux)
 while not done_event.wait(timeout=0.5):
 pass
 executor.shutdown(wait=True)
 except KeyboardInterrupt:
 self.shutdown_event.set()
 executor.shutdown(wait=False, cancel_futures=True)
 raise

 unprocessed = total - completed_count - len(skipped)
 if unprocessed > 0:
 unreached = [
 name
 for name, c in remaining_deps.items()
 if c > 0 and name not in skipped
]
 raise ValueError(
 f"Traversal incomplete: {completed_count}/{total} modules p
 f"{len(skipped)} skipped, {unprocessed} unreached (possible
 f"Examples: {unreached[:5]}"
)

 return all_results

```

```

@dataclass
class ModuleInfo:
 """Information about a single module in the DAG."""

 name: str
 filepath: Path
 imports: list[str] = field(default_factory=list)
 importers: list[str] = field(default_factory=list)

```

```

@dataclass
class TraversalResult:
 """Result of processing a single module."""

```

```

module_name: str
filepath: Path
result: object = None
error: Exception | None = None

def _parse_all_imports(
 filepaths: list[Path], project_root: Path,
) -> dict[Path, list[str]]:
 """Parse imports for all files using ``lean --deps-json``.

 Returns a dict mapping each filepath to its list of imported module names.
 Files that fail to parse are recorded with empty imports and a warning
 is printed to stderr, so a single broken file does not abort the entire
 DAG build.

 The output entries are matched to input files by position (``lean`` emits
 one entry per input file in the same order).
 """
 cmd = ["lake", "env", "lean", "--deps-json", "--stdin"]
 stdin_data = "\n".join(str(f) for f in filepaths)
 proc = subprocess.run(
 cmd, input=stdin_data, capture_output=True, text=True, cwd=project_root,
)
 if proc.returncode != 0:
 raise RuntimeError(
 f"lean --deps-json failed (exit {proc.returncode}): \n{proc.stderr}"
)

 data = json.loads(proc.stdout)
 entries = data["imports"]

 if len(entries) != len(filepaths):
 raise RuntimeError(
 f"lean --deps-json returned {len(entries)} entries "
 f"for {len(filepaths)} files"
)

 result: dict[Path, list[str]] = {}
 for filepath, entry in zip(filepaths, entries):
 if entry.get("errors"):
 errors = "; ".join(entry["errors"])
 print(
 f"warning: lean --deps-json: {filepath}: {errors}",
 file=sys.stderr,
)

```

```

)
 imports: list[str] = []
 if entry.get("result"):
 for imp in entry["result"]["imports"]:
 imports.append(imp["module"])
 result[filepath] = imports

 return result

def _filepath_to_module(filepath: str) -> str:
 """Convert a file path like Mathlib/Foo/Bar.lean to Mathlib.Foo.Bar."""
 return ".".join(Path(filepath).with_suffix("").parts)

class DAG:
 """The import DAG for a set of Lean modules."""

 def __init__(
 self,
 modules: dict[str, ModuleInfo] | None = None,
 project_root: Path | None = None,
):
 self.modules: dict[str, ModuleInfo] = modules or {}
 self.project_root: Path | None = project_root

 @staticmethod
 def from_directories(
 project_root: Path,
 directories: list[str] | None = None,
) -> "DAG":
 """Build DAG by parsing imports from .lean files in directories.

 Uses ``lean --deps-json`` to parse imports correctly.
 """
 if directories is None:
 directories = ["."]

 # Collect all .lean file paths (relative to project_root).
 # Skip the Lake build directory (`.lake/`); it contains vendored
 # dependencies that we don't want to modify or build directly.
 rel_paths: list[Path] = []
 for directory in directories:
 dir_path = project_root / directory
 if not dir_path.exists():
 continue

```

```

 for root, dirs, files in os.walk(dir_path):
 dirs[:] = [d for d in dirs if d != ".lake"]
 for fname in files:
 if not fname.endswith(".lean"):
 continue
 full_path = Path(root) / fname
 rel_paths.append(full_path.relative_to(project_root))

 if not rel_paths:
 import sys as _sys
 _sys.stderr.write(
 f"warning: no .lean files found under {project_root} "
 f"(directories={list(directories)}).\n"
 " hint: pass --directories <your-source-
dir> if your files\n"
 " aren't directly under the project root.\n"
)
 return DAG({}, project_root.resolve())

Batch-parse imports using lean --deps-json.
all_imports = _parse_all_imports(rel_paths, project_root)

modules: dict[str, ModuleInfo] = {}
for rel_path in rel_paths:
 module_name = _filepath_to_module(str(rel_path))
 imports = all_imports.get(rel_path, [])
 modules[module_name] = ModuleInfo(
 name=module_name,
 filepath=rel_path,
 imports=[i for i in imports if i != module_name],
)

Build reverse edges (importers)
for name, info in modules.items():
 for imp in info.imports:
 if imp in modules:
 modules[imp].importers.append(name)

return DAG(modules, project_root.resolve())

def subset(self, module_names: set[str]) -> "DAG":
 """Return a sub-DAG containing only the specified modules."""
 new_modules: dict[str, ModuleInfo] = {}
 for name in module_names:
 if name not in self.modules:
 continue

```

```

 info = self.modules[name]
 new_modules[name] = ModuleInfo(
 name=name,
 filepath=info.filepath,
 imports=[i for i in info.imports if i in module_names],
 importers=[i for i in info.importers if i in module_names],
)
 return DAG(new_modules, self.project_root)

def forward_cone(self, seeds: set[str]) -> set[str]:
 """Return *seeds* plus all modules that transitively import any seed
 from collections import deque

 result: set[str] = set()
 queue: deque[str] = deque(s for s in seeds if s in self.modules)
 while queue:
 name = queue.popleft()
 if name in result:
 continue
 result.add(name)
 for imp in self.modules[name].importers:
 if imp in self.modules and imp not in result:
 queue.append(imp)
 return result

def levels_backward(self) -> list[list[str]]:
 """Compute levels for backward (downstream-first) traversal.

 Useful for display/reporting. DAGTraverser.traverse does not use
 this -- it schedules dynamically for maximum parallelism.

 Level 0 = modules not imported by any other module in the DAG.
 Level n+1 = modules whose importers are all at level <= n.
 """
 remaining_importers = {
 name: len(info.importers) for name, info in self.modules.items()
 }
 remaining = set(self.modules.keys())
 levels: list[list[str]] = []

 while remaining:
 level = sorted(m for m in remaining if remaining_importers[m] == 0)
 if not level:
 print(
 f"warning: cycle in import DAG, dumping {len(remaining)}",
 f"modules into final level",
)

```

```

 file=sys.stderr,
)
 levels.append(sorted(remaining))
 break
 levels.append(level)
 for m in level:
 remaining.discard(m)
 for imp in self.modules[m].imports:
 if imp in remaining_importers:
 remaining_importers[imp] -= 1

 return levels

def _compute_depths(
 successors_of: dict[str, list[str]],
 weights: dict[str, int] | None = None,
) -> dict[str, int]:
 """Compute the longest successor-chain length for each module.

 Without *weights*:
 depth[m] = 0 if m has no successors, else
 1 + max(depth[s] for s in successors_of[m]).

 With *weights*:
 w(m) = weights.get(m, 0)
 depth[m] = w(m) if m has no successors, else
 w(m) + max(depth[s] for s in successors_of[m]).

 Modules absent from *weights* (e.g. skip-set modules that are
 processed instantly) default to weight 0, so they don't inflate
 the critical-path estimate.

 Used to prioritise modules whose completion unblocks the costliest
 chains of future work, improving overall parallelism.

 Nodes involved in cycles (which shouldn't exist in a well-
 formed
 import graph) are assigned depth 0.
 """
 depths: dict[str, int] = {}
 visiting: set[str] = set() # cycle detection

 for start in successors_of:
 if start in depths:
 continue

```

```

Iterative post-order DFS
stack: list[tuple[str, bool]] = [(start, False)]
while stack:
 node, processed = stack.pop()
 if processed:
 visiting.discard(node)
 succs = [s for s in successors_of.get(node, []) if s in dep
 if weights is not None:
 w = weights.get(node, 0)
 depths[node] = (w + max(depths[s] for s in succs)) if s
 else:
 depths[node] = (1 + max(depths[s] for s in succs)) if s
 continue
 if node in depths:
 continue
 if node in visiting:
 # Cycle – assign depth 0 and don't recurse further.
 depths[node] = 0
 continue
 visiting.add(node)
 stack.append((node, True))
 for s in successors_of.get(node, []):
 if s not in depths and s in successors_of:
 stack.append((s, False))

return depths

def traverse_dag(
 dag: DAG,
 action: Callable[[str, Path], object],
 direction: str = "backward",
 **kwargs,
) -> list[TraversalResult]:
 """Convenience wrapper: create a one-off DAGTraverser and traverse."""
 return DAGTraverser().traverse(dag, action, direction, **kwargs)

CLI

class Display:
 """ANSI progress display for DAG traversal.
```



Subclass and override ``\_status\_line`` (and add an ``on\_module`` callback) to customise for a specific script.

```

"""
def __init__(self):
 self.lock = Lock()
 self.completed = 0
 self.total = 0
 self.inflight = 0
 self.messages: list[str] = []
 self.displayed_lines = 0
 self.started = False

def start(self, total: int):
 self.total = total
 self.started = True
 print(HIDE_CURSOR, end="", flush=True)
 self._redraw()

def stop(self):
 if self.started:
 print(SHOW_CURSOR, end="", flush=True)
 self.started = False

def _status_line(self) -> str:
 return f"[{self.completed}/{self.total}] Working on: {self.inflight}"

def _redraw(self):
 if not self.started:
 return
 if self.displayed_lines > 0:
 print(f"\033[{self.displayed_lines}A", end="")
 for msg in self.messages:
 print(CLEAR_LINE + msg)
 self.messages.clear()

 print(CLEAR_LINE + self._status_line(), flush=True)
 self.displayed_lines = 1

def on_progress(self, completed: int, total: int, inflight: int):
 with self.lock:
 self.completed = completed
 self.total = total
 self.inflight = inflight
 self._redraw()

```

```

class _CliDisplay(Display):
 """Display subclass for the CLI that clears lines on stop."""

 def __init__(self):
 super().__init__()
 self.ok = 0
 self.fail = 0

 def stop(self):
 if self.started:
 if self.displayed_lines > 0:
 print(f"\033[{self.displayed_lines}A", end="")
 for _ in range(self.displayed_lines):
 print(CLEAR_LINE)
 print(f"\033[{self.displayed_lines}A", end="", flush=True)
 print(SHOW_CURSOR, end="", flush=True)
 self.started = False

 def _status_line(self) -> str:
 return (
 f"[{self.completed}/{self.total}] "
 f"Working on: {self.inflight} ok: {self.ok} fail: {self.fail}"
)

 def on_module(self, module_name: str, result, error: Exception | None):
 with self.lock:
 if error:
 self.fail += 1
 self.messages.append(f" \u2717 {module_name}: {error}")
 else:
 self.ok += 1
 self.messages.append(f" \u2713 {module_name}")
 self._redraw()

class _CommandFailed(Exception):
 """Raised when a shell command exits with non-zero status."""

 def __init__(self, exit_code: int, output: str = ""):
 self.exit_code = exit_code
 self.output = output
 super().__init__(f"exit {exit_code}")

def _cli_main():

```

```

 parser = argparse.ArgumentParser(
 description="Run a command on each Lean module in import-
DAG order.",
 epilog="Use {} in the command as a placeholder for the file path "
 "(or module name with --module).",
)

 direction = parser.add_mutually_exclusive_group(required=True)
 direction.add_argument(
 "--forward",
 action="store_const",
 const="forward",
 dest="direction",
 help="Process upstream (roots) first",
)
 direction.add_argument(
 "--backward",
 action="store_const",
 const="backward",
 dest="direction",
 help="Process downstream (leaves) first",
)

 parser.add_argument(
 "command",
 help="Command template; {} is replaced with filepath or module name",
)
 parser.add_argument(
 "--module",
 action="store_true",
 help="Replace {} with module name (Mathlib.Foo.Bar) instead of file path",
)
 parser.add_argument(
 "-j",
 "--max-workers",
 type=int,
 default=None,
 help="Max parallel workers (default: cpu count)",
)
 parser.add_argument(
 "--dir",
 type=Path,
 default=Path("."),
 help="Project root directory (default: cwd)",
)
 parser.add_argument(

```

```

 "--directories",
 nargs="+",
 default=None,
 help="Directories to scan (default: '.', relative to project root s
dir)",
)

 args = parser.parse_args()

 print("Building import DAG...", end="", flush=True)
 dag = DAG.from_directories(args.dir, args.directories)
 print(f" {len(dag.modules)} modules")

 use_module = args.module
 cmd_template = args.command

 def action(module_name: str, filepath: Path) -> int:
 replacement = module_name if use_module else str(filepath)
 cmd = cmd_template.replace("{", shlex.quote(replacement))
 result = subprocess.run(
 cmd, shell=True, cwd=dag.project_root,
 capture_output=True, text=True,
)
 if result.returncode != 0:
 raise _CommandFailed(result.returncode, result.stdout + result.
 return result.returncode

 display = _CliDisplay()
 display.start(len(dag.modules))

 try:
 results = traverse_dag(
 dag,
 action,
 direction=args.direction,
 max_workers=args.max_workers,
 module_callback=display.on_module,
 progress_callback=display.on_progress,
 stop_on_failure=True,
)
 except KeyboardInterrupt:
 display.stop()
 print("Interrupted.")
 sys.exit(1)
 finally:
 display.stop()

```

```

Summary
ok = sum(1 for r in results if r.error is None)
failed = sum(1 for r in results if r.error is not None)
skipped = len(dag.modules) - len(results)

print(f"Done: {ok} ok, {failed} failed, {skipped} skipped")

if failed:
 print(f"\nFailed modules:")
 for r in results:
 if r.error:
 print(f" {r.module_name}: {r.error}")
 if isinstance(r.error, _CommandFailed) and r.error.output.strip():
 for line in r.error.output.strip().splitlines():
 print(f" {line}")

if skipped:
 processed = {r.module_name for r in results}
 skipped_names = sorted(set(dag.modules.keys()) - processed)
 print(f"\nSkipped modules ({skipped}):")
 for name in skipped_names[:20]:
 print(f" {name}")
 if len(skipped_names) > 20:
 print(f" ... and {len(skipped_names) - 20} more")

sys.exit(1 if failed else 0)

if __name__ == "__main__":
 _cli_main()

```

### 1.430 scripts/lint-style.sh

```
touch scripts/LinterExemption.txt
```

```

Lint all Lean files in `Physlib` and `QuantumInfo`, except those listed (one path
in `scripts/LinterExemption.txt`. The exemption list lets us lint `QuantumInfo`
file-by-file: remove a file from the list once it passes the linters.
git ls-files 'Physlib/*.lean' 'QuantumInfo/*.lean' \
 | grep -vxF -f scripts/LinterExemption.txt \
 | xargs ./scripts/lint-style.py "$@"

```

**1.431 scripts/openAI\_doc\_check.lean**

```
unsafe def main (args : List String) : IO UInt32 := do
 Lean.enableInitializersExecution
```

**1.432 scripts/rm\_set\_option.py**

```
#!/usr/bin/env python3
"""
Remove unnecessary `set_option ... false in` from a Lean project.

Tries removing each occurrence (that isn't followed by a comment), testing
whether the file still builds. Processes files in reverse import-
DAG order
(downstream first) to minimize unnecessary rebuilds.
"""

import argparse
import hashlib
import json
import os
import threading
import time
from dataclasses import dataclass
from pathlib import Path
from threading import Lock
from typing import Callable

try:
 from dag_traversal import (
 DAG,
 DAGTraverser,
 Display,
 ShutdownError,
)
 from set_option_utils import (
 DEFAULT_OPTIONS,
 PROJECT_DIR,
 commented_pattern,
 lake_build_with_progress,
 lakefile_pattern,
 removable_pattern,
)
except ImportError as _e:
```

```

 raise SystemExit(
 f"error: {_e}\n\n"
 f" This script depends on sibling Python files in {Path(__file__).parent}:\n"
 " - dag_traversal.py\n"
 " - set_option_utils.py\n"
 " These are mathlib scripts, not pip packages. Copy them into your\n"
 " `scripts/` directory alongside this script."
)

PROGRESS_FILE = PROJECT_DIR / "scripts" / ".rm_set_option_progress.jsonl"

_progress_lock = Lock()

def _current_toolchain() -> str:
 return (PROJECT_DIR / "lean-toolchain").read_text().strip()

def file_sha256(path: Path) -> str:
 return hashlib.sha256(path.read_bytes()).hexdigest()

def load_progress() -> dict[str, str] | None:
 """Load progress file. Returns {module: sha256} or None if invalid/missing."""
 if not PROGRESS_FILE.exists():
 return None
 progress: dict[str, str] = {}
 toolchain = _current_toolchain()
 with open(PROGRESS_FILE) as f:
 for i, line in enumerate(f):
 line = line.strip()
 if not line:
 continue
 try:
 record = json.loads(line)
 except json.JSONDecodeError:
 continue
 if i == 0:
 if record.get("toolchain") != toolchain:
 return None
 continue
 if "module" in record and "sha256" in record:
 progress[record["module"]] = record["sha256"]
 return progress

```

```

def init_progress():
 """Write toolchain header to a fresh progress file."""
 with open(PROGRESS_FILE, "w") as f:
 json.dump({"toolchain": _current_toolchain()}, f)
 f.write("\n")

def save_progress(module: str, sha256: str):
 """Append a completion record (thread-safe)."""
 with _progress_lock:
 with open(PROGRESS_FILE, "a") as f:
 json.dump({"module": module, "sha256": sha256}, f)
 f.write("\n")

@dataclass
class FileResult:
 """Result of processing one file."""

 removable: int = 0 # lines we attempted to remove
 removed: int = 0 # successfully removed
 kept: int = 0 # had to keep (build failed)
 skipped: int = 0 # with trailing comments

@dataclass
class Summary:
 """Aggregated results."""

 total_files: int = 0
 files_fully_cleaned: int = 0
 files_partially_cleaned: int = 0
 files_unchanged: int = 0
 files_errored: int = 0
 files_timed_out: int = 0
 total_removed: int = 0
 total_kept: int = 0
 total_skipped: int = 0
 duration: float = 0.0

class _RemoveDisplay(Display):
 """Progress display for the remove script."""

 def __init__(self):

```



```

 super().__init__()
 self.total_removed = 0
 self.total_kept = 0

 def _status_line(self) -> str:
 return (
 f"[{self.completed}/{self.total}] "
 f"Working on: {self.inflight} "
 f"Removed: {self.total_removed} Kept: {self.total_kept}"
)

 def on_module(self, module_name: str, result: FileResult | None, error: Exception | None):
 with self.lock:
 if result:
 self.total_removed += result.removed
 self.total_kept += result.kept
 if result.removed > 0 and result.kept == 0:
 sym = "\u2713"
 elif result.removed > 0:
 sym = "~"
 else:
 sym = "\u00b7"
 parts = []
 if result.removed:
 parts.append(f"-{result.removed}")
 if result.kept:
 parts.append(f"kept {result.kept}")
 if result.skipped:
 parts.append(f"skipped {result.skipped}")
 detail = " ".join(parts)
 self.messages.append(f" {sym} {module_name} {detail}")
 elif error:
 self.messages.append(f" ! {module_name}: {error}")
 self._redraw()

 def handle_lakefile(options: list[str], value: str = "false") -> bool:
 """Check and remove options from lakefile.lean. Returns True if changed."""
 lakefile = PROJECT_DIR / "lakefile.lean"
 content = lakefile.read_text()
 changed = False
 for opt in options:
 if opt not in content:
 continue
 pat = lakefile_pattern(opt, value)
 new_content = pat.sub("", content)

```

```

 if new_content != content:
 content = new_content
 changed = True
 print(f"Removed {opt} from lakefile.lean")
 if changed:
 lakefile.write_text(content)
 return changed

def scan_files(dag: DAG, options: list[str], value: str = "false") -> dict[str, list[int]]:
 """Find files with removable set_option lines.

 Returns dict of module_name -> list of 0-indexed line numbers.
 """
 removable_pats = [removable_pattern(opt, value) for opt in options]
 commented_pats = [commented_pattern(opt, value) for opt in options]
 results: dict[str, list[int]] = {}
 for name, info in dag.modules.items():
 filepath = dag.project_root / info.filepath
 if not filepath.exists():
 continue
 lines = filepath.read_text().splitlines(keepends=True)
 removable = []
 for i, line in enumerate(lines):
 if any(p.match(line) for p in commented_pats):
 continue
 if any(p.match(line) for p in removable_pats):
 removable.append(i)
 if removable:
 results[name] = removable
 return results

def count_skipped(filepath: Path, options: list[str], value: str = "false"):
 """Count set_option lines with trailing comments."""
 commented_pats = [commented_pattern(opt, value) for opt in options]
 count = 0
 for line in filepath.read_text().splitlines():
 if any(p.match(line) for p in commented_pats):
 count += 1
 return count

def initial_build():
 """Run a full lake build so all .oleans are fresh.
```



```

Phase 2: revert, then one-at-a-time bottom-
to-top
abs_path.write_text(original_text)
current = list(original)
removed = 0
kept = 0

for line_idx in reversed(removable_lines):
 test = current[:line_idx] + current[line_idx + 1 :]
 abs_path.write_text("".join(test))

 ok, _ = traverser.lake_build(module_name, PROJECT_DIR, time
 if ok:
 current = test
 removed += 1
 else:
 abs_path.write_text("".join(current))
 kept += 1

 save_progress(module_name, file_sha256(abs_path))
 return FileResult(
 removable=len(removable_lines),
 removed=removed,
 kept=kept,
 skipped=skipped,
)
except BaseException:
 abs_path.write_text(original_text)
 raise
finally:
 traverser.inflight_unregister(abs_path)

return process_file

def print_summary(summary: Summary):
 print()
 print("=" * 60)
 print("SUMMARY")
 print("=" * 60)
 print(f" Files processed: {summary.total_files}")
 print(f" Fully cleaned: {summary.files_fully_cleaned}")
 print(f" Partially cleaned: {summary.files_partially_cleaned}")
 print(f" Unchanged: {summary.files_unchanged}")
 print(f" Errors: {summary.files_errored}")

```

```

 if summary.files_timed_out:
 print(f" Not yet processed: {summary.files_timed_out} (global timeout)
 print(f" Lines removed: {summary.total_removed}")
 print(f" Lines kept: {summary.total_kept}")
 print(f" Lines skipped (comment): {summary.total_skipped}")
 print(f" Duration: {summary.duration:.0f}s")

def write_github_outputs(summary: Summary):
 """Write key stats to $GITHUB_OUTPUT when running in GitHub Actions."""
 output_file = os.environ.get("GITHUB_OUTPUT")
 if not output_file:
 return
 files_modified = summary.files_fully_cleaned + summary.files_partially_cleaned
 with open(output_file, "a") as f:
 f.write(f"lines_removed={summary.total_removed}\n")
 f.write(f"files_modified={files_modified}\n")
 f.write(f"files_timed_out={summary.files_timed_out}\n")

def main():
 parser = argparse.ArgumentParser(
 description="Remove unnecessary `set_option ... false` in Lean pr
 "To use outside mathlib, copy `rm_set_option.py`, `dag_traversal
 "`set_option_utils.py` to a subdirectory of your project named `
 "and then run from the project root with `scripts/rm_set_option.
 formatter_class=argparse.RawDescriptionHelpFormatter,
)
 parser.add_argument(
 "--option",
 help="Only scan/remove this specific option (default: all known options)",
)
 parser.add_argument(
 "--value",
 default="false",
 help="Value of the option to scan/remove (default: false)",
)
 parser.add_argument(
 "--dry-run",
 action="store_true",
 help="Scan and report without modifying files or building",
)
 parser.add_argument(
 "--max-workers",
 type=int,
 default=None,

```

```

 help="Max parallel workers (default: cpu_count)",
)
 parser.add_argument(
 "--timeout",
 type=int,
 default=600,
 help="Build timeout per module in seconds (default: 600)",
)
 parser.add_argument(
 "--files",
 nargs="*",
 help="Only process these files (paths relative to project root)",
)
 parser.add_argument(
 "--directories",
 nargs="+",
 default=None,
 help="Directories to scan when building the import DAG (default: '.').",
)
 parser.add_argument(
 "--no-initial",
 action="store_true",
 help="Skip the initial lake build (assumes .oleans are already fresh)",
)
 parser.add_argument(
 "--resume",
 action="store_true",
 help="Resume a previous interrupted run, skipping already-processed modules",
)
 parser.add_argument(
 "--global-timeout",
 type=int,
 default=None,
 metavar="SECONDS",
 help="Stop processing new modules after this many seconds and exit gracefully, preserving the progress file for the next run",
)
 args = parser.parse_args()

 options = [args.option] if args.option else DEFAULT_OPTIONS

 start_time = time.time()

 # Step 1: lakefile.lean
 if not args.dry_run and (PROJECT_DIR / "lakefile.lean").exists():

```

```

 handle_lakefile(options, args.value)
 elif (PROJECT_DIR / "lakefile.toml").exists():
 opts_str = ", ".join(options)
 print(
 f"Note: lakefile.toml detected. If {opts_str} is set in `leanOptions`\n"
 " there, you'll need to remove the entry manually – this script only\n"
 " edits lakefile.lean."
)

Step 2: build DAG
print("Building import DAG...", flush=True)
full_dag = DAG.from_directories(PROJECT_DIR, args.directories)
print(f" {len(full_dag.modules)} modules parsed")

Step 3: scan for removable lines
print("Scanning for removable set_option lines...", flush=True)
removable_map = scan_files(full_dag, options, args.value)

if args.files:
 # Filter to requested files
 requested = set()
 for f in args.files:
 mod = f.replace("/", ".").removesuffix(".lean")
 requested.add(mod)
 removable_map = {k: v for k, v in removable_map.items() if k in requested}

total_removable = sum(len(v) for v in removable_map.values())
print(f" {len(removable_map)} files with {total_removable} removable lines")

Step 3b: filter out modules completed in a previous run
resumed = 0
if args.resume:
 progress = load_progress()
 if progress:
 to_skip = []
 for name in list(removable_map):
 if name in progress:
 fp = full_dag.project_root / full_dag.modules[name].filepath
 if fp.exists() and file_sha256(fp) == progress[name]:
 to_skip.append(name)
 for name in to_skip:
 del removable_map[name]
 resumed = len(to_skip)
 if resumed:
 total_removable = sum(len(v) for v in removable_map.values())
 print(f" Resuming: skipped {resumed} already-
```

```

processed modules"
 f" ({len(removable_map)} remaining, {total_removable})

 if not removable_map:
 print("Nothing to do.")
 if PROGRESS_FILE.exists():
 PROGRESS_FILE.unlink()
 return

 # Step 4: initial build to ensure all .oleans are fresh
 if not args.dry_run and not args.no_initial:
 initial_build()

 # Step 5: traverse full DAG, skipping modules without removable lines.
 # We use the full DAG (not a subset) to ensure the backward traversal
 # respects all import edges. Without this, two modules in the target
 # set that are connected through intermediate non-target modules could
 # be processed concurrently, causing `lake build` to race on rebuilding
 # shared upstream .oleans.
 target_modules = set(removable_map.keys())
 skip_modules = set(full_dag.modules.keys()) - target_modules

 # Weight = LOC × number of removable lines, so expensive files get
 # critical-path priority in the DAG traversal scheduler.
 weights = {}
 for name, removable_lines in removable_map.items():
 fp = full_dag.project_root / full_dag.modules[name].filepath
 loc = len(fp.read_text().splitlines())
 weights[name] = loc * len(removable_lines)

 if args.dry_run:
 sub_dag = full_dag.subset(target_modules)
 levels = sub_dag.levels_backward()
 print(f"\nBackward traversal: {len(levels)} levels")
 for i, level in enumerate(levels):
 count = sum(len(removable_map.get(m, [])) for m in level)
 print(f" Level {i}: {len(level)} files, {count} removable line")
 print(f"\nTotal: {len(removable_map)} files, {total_removable} line")
 print("(dry run – no changes made)")
 return

 # Initialize progress file (preserves existing entries on resume)
 if not PROGRESS_FILE.exists() or resumed == 0:
 init_progress()

 traverser = DAGTraverser()

```



```

_timer = None
if args.global_timeout:
 def _global_timeout_handler():
 print(
 f"\nGlobal timeout ({args.global_timeout}s) reached - "
 "stopping gracefully and preserving progress for next run...",
 flush=True,
)
 traverser.shutdown_event.set()

 _timer = threading.Timer(args.global_timeout, _global_timeout_handler)
 _timer.daemon = True
 _timer.start()

display = _RemoveDisplay()
action = make_process_file(removable_map, options, args.timeout, traverser, args)

display.start(len(full_dag.modules))
try:
 results = traverser.traverse(
 full_dag,
 action,
 direction="backward",
 max_workers=args.max_workers,
 progress_callback=display.on_progress,
 module_callback=display.on_module,
 skip=skip_modules,
 weights=weights,
)
except KeyboardInterrupt:
 display.stop()
 print("\nInterrupted. Press Ctrl-C again if the process doesn't exit.", flush=True)
 traverser.force_exit(1)
finally:
 display.stop()
 if _timer is not None:
 _timer.cancel()

Step 6: summarize (only count target modules, not skipped ones)
target_results = [tr for tr in results if tr.module_name in target_modules]
summary = Summary(total_files=len(target_results), duration=time.time() - start_

for tr in target_results:
 r: FileResult | None = tr.result
 if tr.error:

```

```

 if isinstance(tr.error, ShutdownError):
 summary.files_timed_out += 1
 else:
 summary.files_errored += 1
 continue
 if r is None:
 continue
 summary.total_removed += r.removed
 summary.total_kept += r.kept
 summary.total_skipped += r.skipped
 if r.removed > 0 and r.kept == 0:
 summary.files_fully_cleaned += 1
 elif r.removed > 0:
 summary.files_partially_cleaned += 1
 else:
 summary.files_unchanged += 1

print_summary(summary)
write_github_outputs(summary)

Clean up progress file only when the run fully completed without error
If a global timeout fired or there were errors, keep it for the next run
if summary.files_errored == 0 and summary.files_timed_out == 0 and PROGRESS_FILE.exists():
 PROGRESS_FILE.unlink()
 print(" (progress file cleaned up)")
elif summary.files_timed_out > 0:
 print(" (progress file kept – resume on next run)")

if __name__ == "__main__":
 main()

```

### 1.433 scripts/set\_option\_utils.py

```

#!/usr/bin/env python3
"""Shared configuration and helpers for set_option scripts."""

import re
import subprocess
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent.parent

DEFAULT_OPTIONS = [

```

```

 "backward.isDefEq.respectTransparency",
 "backward.whnf.reducibleClassField",
 "backward.inferInstanceAs.wrap",
]

def set_option_line(option: str, value: str = "false") -> str:
 """Return the set_option line text for an option."""
 return f"set_option {option} {value} in\n"

def removable_pattern(option: str, value: str = "false") -> re.Pattern:
 """Match bare `set_option X <value> in` lines (no trailing comment)."""
 escaped = re.escape(option)
 escaped_val = re.escape(value)
 return re.compile(rf"^\s*set_option {escaped} {escaped_val} in\s*$")

def commented_pattern(option: str, value: str = "false") -> re.Pattern:
 """Match `set_option X <value> in -- ...` lines."""
 escaped = re.escape(option)
 escaped_val = re.escape(value)
 return re.compile(rf"^\s*set_option {escaped} {escaped_val} in\s+--")

def lakefile_pattern(option: str, value: str = "false") -> re.Pattern:
 """Match lakefile entries for an option."""
 escaped = re.escape(option)
 escaped_val = re.escape(value)
 return re.compile(
 rf"^\s*({escaped},\s*{escaped_val}),?\s*\n",
 re.MULTILINE,
)

PROGRESS_RE = re.compile(r"\[(\d+)/(\d+)\]")

def lake_build_with_progress(project_dir: Path) -> tuple[int, str]:
 """Run ``lake build`` with a progress display.

 Returns (returncode, captured_output).
 """
 proc = subprocess.Popen(
 ["lake", "build"],
)

```

```
 stdout=subprocess.PIPE,
 stderr=subprocess.STDOUT,
 text=True,
 cwd=project_dir,
)
 output_lines: list[str] = []
 for line in proc.stdout:
 output_lines.append(line)
 m = PROGRESS_RE.search(line)
 if m:
 print(f"\r [{m.group(1)}/{m.group(2)}]", end="", flush=True)
 proc.wait()
 print() # newline after progress
 return proc.returncode, "".join(output_lines)
```

#### 1.434 scripts/stats.lean

Lean.enableInitializersExecution